Fabric fundamentals documentation

Get started with Fabric, a unified platform for your organization's data and analytics needs.

About Fabric

OVERVIEW

What is Fabric?

Fabric terminology

What's new in Fabric

Migrate to Fabric

GET STARTED

Start a Fabric trial

Fabric home navigation

End-to-end tutorials

Context sensitive Help pane

Get started with Fabric items

CONCEPT

Get started with a task flow

Find items in OneLake catalog

Promote and certify items

HOW-TO GUIDE

Manage multiple task flow canvases

Apply sensitivity labels

Copilot in Fabric

CONCEPT

Overview of Copilot in Fabric

Copilot in Fabric FAQ

Privacy, security, and responsible use for Copilot in Fabric

HOW-TO GUIDE

Enable Copilot in Fabric

OpenAI as a subprocessor

Workspaces

CONCEPT

Fabric workspace

Workspace roles

GET STARTED

Create a workspace

HOW-TO GUIDE

Workspace access control

Get Help

HOW-TO GUIDE

Use the integrated Help pane

Check for known issues

Contact Support

What is Microsoft Fabric?

Microsoft Fabric is an analytics platform that supports end-to-end data workflows, including data ingestion, transformation, real-time stream processing, analytics, and reporting. It provides integrated experiences such as Data Engineering, Data Factory, Data Science, Real-Time Intelligence, Data Warehouse, and Databases, which operate over a shared compute and storage model.

Fabric is delivered as a software-as-a-service (SaaS) platform and uses OneLake as a centralized, logical data lake for storing and accessing data across all workloads. In tandem with OneLake, the OneLake Catalog provides a centralized experience for discovering, exploring, and governing data and analytics artifacts across the tenant. AI capabilities are built into the platform to assist with data preparation, analysis, and development tasks, reducing the need for manual service integration and enabling efficient analysis of large-scale data.

> **Note**
>
> - The Fabric Analyst in a Day (FAIAD) workshop is free, hands-on training for analysts working with Power BI and Fabric. Get hands-on experience analyzing data and building reports using Fabric. The workshop covers key concepts like working with lakehouses, creating reports, and analyzing data in the Fabric environment.
> - Join the new Fabric user panel to share feedback and help shape Fabric and Power BI. Participate in surveys and 1:1 sessions with the product team. Learn more and sign up at Fabric user panel.

Capabilities of Fabric

Microsoft Fabric provides several integrated capabilities:

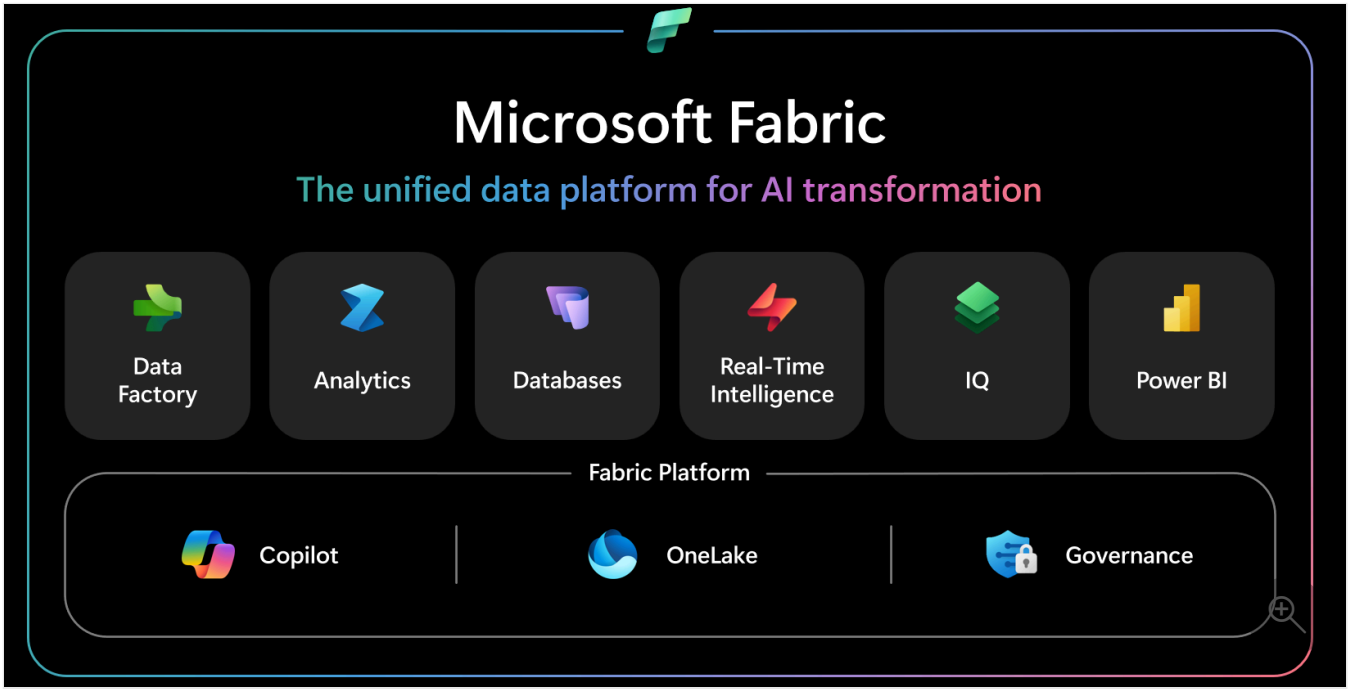
- **Role-specific workloads:** Fabric offers distinct workloads for data engineers, data scientists, business analysts, and database administrators. Each workload provides tools, APIs, and user experiences optimized for common tasks such as data ingestion, transformation, modeling, querying, and reporting. they can also be combined within a single solution to support end-to-end scenarios.

- **OneLake (storage):** All Fabric workloads operate over OneLake, a unified logical data lake built on Azure Data Lake Storage. OneLake enables shared access to data across workloads without requiring data movement or duplication.

- **Copilot support:** Fabric includes Copilot features that assist with tasks such as authoring queries, pipelines and code, generating summaries and insights, and accelerating common development and analysis workflows.
- **Microsoft 365 integration:** Fabric integrates with Microsoft 365 applications, enabling data to be analyzed and consumed in tools such as Excel and shared through collaboration surfaces like Microsoft Teams.
- **Microsoft Foundry integration:** Fabric integrates with Microsoft Foundry to enable the use of prebuilt models and tooling for machine learning and AI scenarios, including model development, deployment, and inference.
- **Unified data management and governance:** Fabric provides centralized data discovery, access control, and governance capabilities, helping organizations manage data access, sharing, and compliance consistently across workloads.

Microsoft Fabric architecture

The following diagram illustrates how Microsoft Fabric is built on a software-as-a-service (SaaS) platform that unifies multiple analytics experiences within a single environment.



At the top of the diagram are the core Fabric workloads, such as Data Factory, Analytics, Databases, Real-Time Intelligence, IQ, and Power BI. Each workload provides specialized capabilities tailored to different analytics tasks, but all workloads operate within the same Fabric environment and can share data and artifacts without duplication.

Beneath these workloads is the Fabric platform layer, which provides shared services used consistently across the experiences:

- **OneLake** is the centralized, logical data lake for Fabric. All workloads store and access data through OneLake, enabling zero-copy access patterns and allowing data to remain in its original location while being reused across experiences.
- **Copilot** provides AI assistance embedded directly within Fabric workloads to help with authoring, exploration, and routine development tasks, while respecting tenant, data, and permission boundaries.
- **Governance** represents centralized administration and data governance, including permissions, sensitivity labels, and auditing. These controls are applied automatically and inherited across Fabric items. Governance is powered by Purview, which is built into Fabric. Purview-backed governance extends to data shared across tenants through OneLake data sharing, so sensitivity labels, access policies, and compliance controls remain enforced regardless of where shared data is consumed. Microsoft Fabric centralizes governance and discovery capabilities within the OneLake Catalog, which serves as a unified hub to find, explore, secure and use the Fabric items you need, and govern the data you own. You can assess governance state, receive recommended actions, and improve data trust and compliance across workspaces and domains. To learn more, see What is the OneLake Catalog?.

This SaaS foundation enables end-to-end analytics scenarios such as ingesting data with Data Factory, processing it with engineering or real-time workloads, and visualizing it in Power BI without manually integrating separate services or managing underlying infrastructure. Fabric centralizes data discovery, administration, and governance, with Microsoft Purview built in to enforce consistent security and compliance across the platform.

Components of Microsoft Fabric

Microsoft Fabric offers the following workloads, each customized for a specific role and task:

- **Power BI** - Power BI lets you connect to data sources, create interactive charts and dashboards, and share insights across your organization. This allows business owners access to all data in Fabric quickly and effectively, enabling better data-focused decisions. For more information, see What is Power BI?
- **Databases** - Databases in Fabric are a developer-friendly transactional database such as Azure SQL Database, which allows you to easily create your operational database in Fabric. Using the mirroring capability, you can bring data from various systems together into OneLake. You can continuously replicate your existing data estate directly into Fabric's OneLake, including data from Azure SQL Database, Azure Cosmos DB, Azure Databricks, Snowflake, and Fabric SQL database. For more information, see SQL database in Microsoft Fabric and What is Mirroring in Fabric?

- **Data Factory** - Data Factory provides a modern data integration experience to ingest, prepare, and transform data from a rich set of data sources. It incorporates the simplicity of Power Query, and you can use more than 200 native connectors to connect to data sources on-premises and in the cloud. For more information, see What is Data Factory in Microsoft Fabric?

- **Industry Solutions** - Fabric provides industry-specific data solutions that address unique industry needs and challenges, and include data management, analytics, and decision-making. For more information, see Industry Solutions in Microsoft Fabric.

- **Real-Time Intelligence** - Real-Time Intelligence analyzes data as it arrives, such as IoT sensor readings, application logs, or website clickstreams. It enables the extraction of insights, visualization, and action on data in motion by handling data ingestion, transformation, storage, modeling, analytics, visualization, tracking, AI, and real-time actions. The Real-Time hub in Real-Time Intelligence provides a wide variety of no-code connectors, converging into a catalog of organizational data that is protected, governed, and integrated across Fabric. For more information, see What is Real-Time Intelligence in Fabric?.

- **Data Engineering** - Fabric Data Engineering provides Apache Spark for processing large datasets, with notebooks and tools for writing and scheduling data transformation jobs. It enables you to create, manage, and optimize infrastructures for collecting, storing, processing, and analyzing vast data volumes. Fabric Spark's integration with Data Factory allows you to schedule and orchestrate notebooks and Spark jobs. For more information, see What is Data engineering in Microsoft Fabric?

- **Fabric Data Science** - Fabric Data Science enables you to build, deploy, and operationalize machine learning models from Fabric. It integrates with Azure Machine Learning to provide built-in experiment tracking and model registry. Data scientists can enrich organizational data with predictions and business analysts can integrate those predictions into their BI reports, allowing a shift from descriptive to predictive insights. For more information, see What is Data science in Microsoft Fabric?

- **Fabric Data Warehouse** - Fabric Data Warehouse provides industry leading SQL performance and scale. It separates compute from storage, enabling independent scaling of both components. Additionally, it natively stores data in the open Delta Lake format. For more information, see What is data warehousing in Microsoft Fabric?

- **IQ (preview)** - IQ (preview) is a new workload for unifying business semantics across data, models, and systems. It includes the items of ontology, plan, Fabric Graph, data agent, operations agent, and semantic models. IQ powers consistent decisions, reusable metrics, and context-aware automation across the Fabric platform. For more information, see What is IQ (preview)?

Fabric helps organizations and individuals analyze their data and create reports, dashboards, and machine learning models. It implements a data mesh architecture. For more information, see What is a data mesh?

OneLake: The unification of lakehouses

The Microsoft Fabric platform unifies the OneLake and lakehouse architecture across an enterprise.

OneLake

A data lake is the foundation for all Fabric workloads. In Fabric, this lake is called OneLake. OneLake is built into the platform and serves as a single store for all organizational data.

OneLake is built on ADLS (Azure Data Lake Storage) Gen2. It provides a single SaaS experience and a tenant-wide store for data that serves both professional and citizen developers. It simplifies the user experience by removing the need to understand complex infrastructure details like resource groups, RBAC, Azure Resource Manager, redundancy, or regions. You don't need an Azure account to use Fabric.

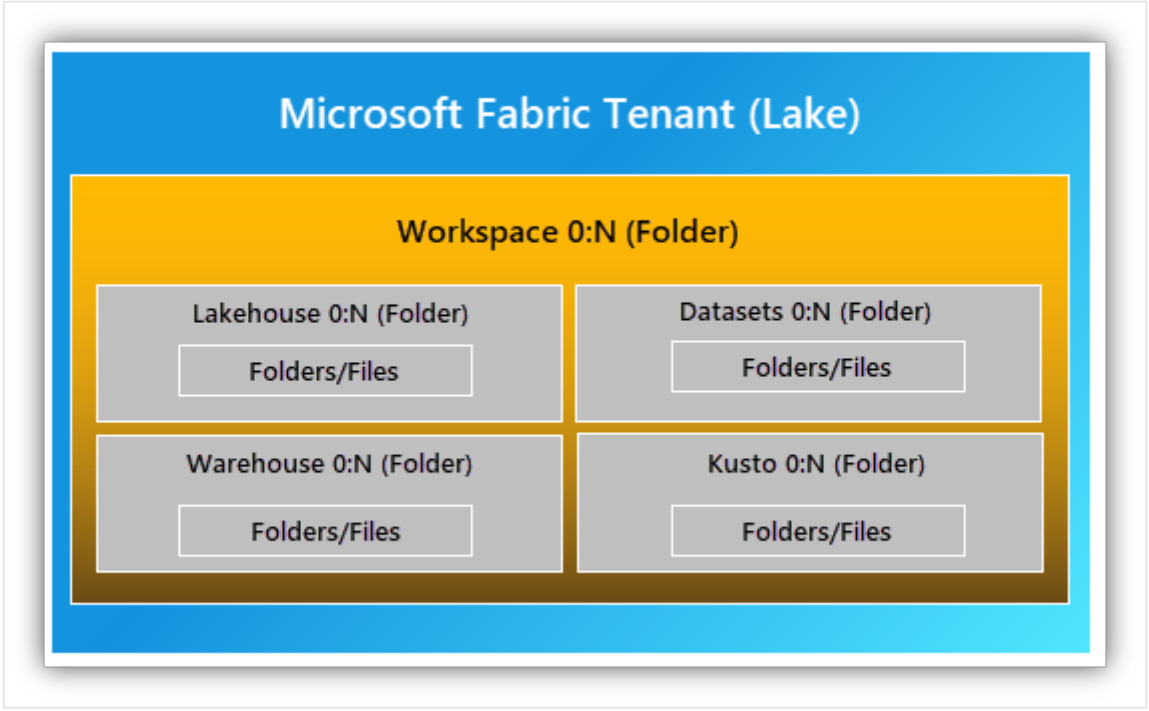
OneLake prevents data silos by offering a unified storage system that makes data discovery, sharing, and consistent policy enforcement easy. For more information, see What is OneLake?

OneLake also supports cross-tenant data sharing, which lets you share live, governed datasets with external organizations across Microsoft Entra tenant boundaries without copying data. Recipients access shared data in place, and governance policies remain enforced at the source. For more information, see external data sharing.

OneLake and lakehouse data hierarchy

OneLake's hierarchical design simplifies organization-wide management. Fabric includes OneLake by default, so no upfront provisioning is needed. Each tenant gets one unified OneLake with single file-system namespace that spans users, regions, and clouds. OneLake organizes data into containers for easy handling. The tenant maps to the root of OneLake and is at the top level of the hierarchy. You can create multiple workspaces (which are like folders) within a tenant.

The following image shows how Fabric stores data in OneLake. You can have several workspaces per tenant and multiple lakehouses within each workspace. A lakehouse is a collection of files, folders, and tables that acts as a database over a data lake. To learn more, see What is a lakehouse?.



Every developer and business unit in the tenant creates their own workspaces in OneLake. They ingest data into lakehouses and start processing, analyzing, and collaborating on that data—like using OneDrive in Microsoft Office.

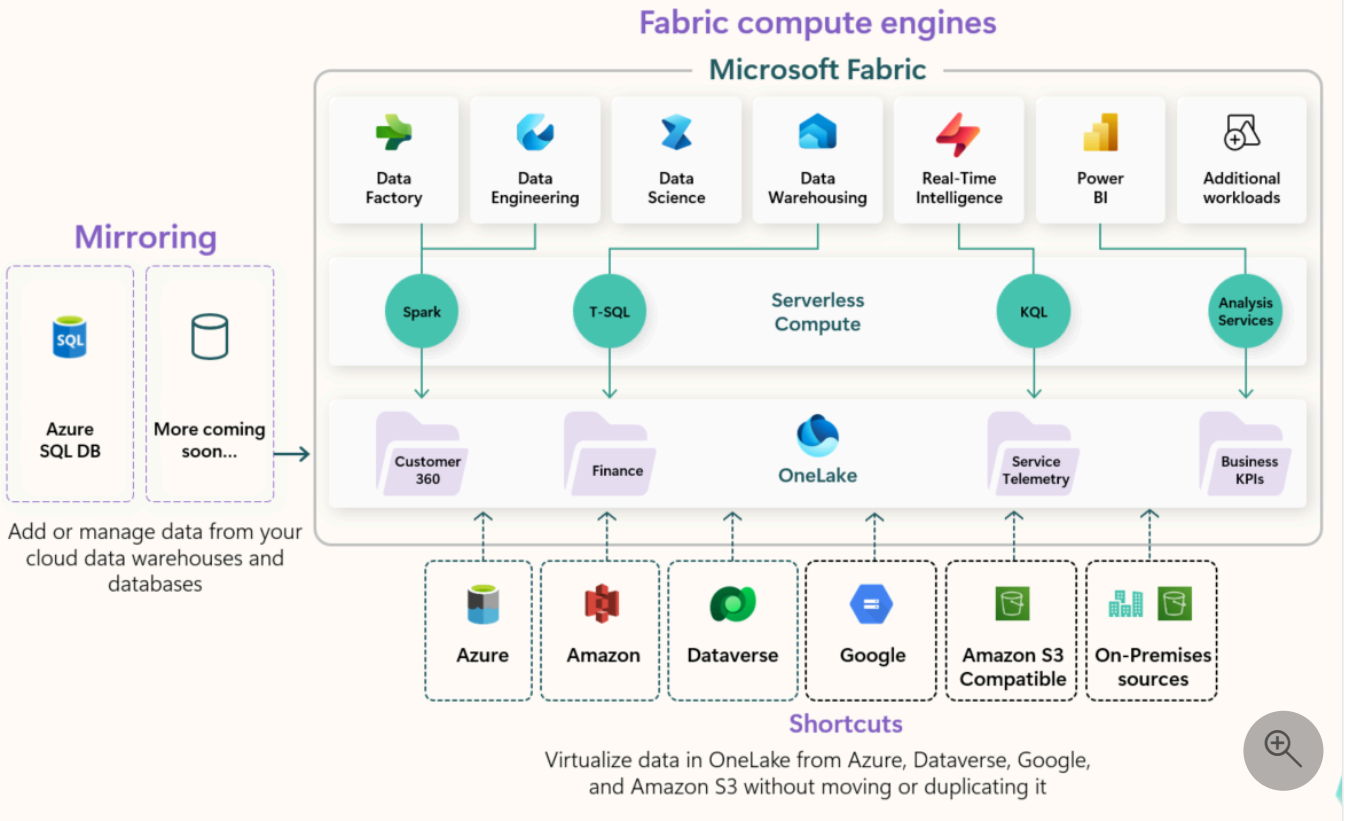
Real-Time hub: the unification of data streams

The Real-Time hub is a foundational location for data in motion. It provides a unified SaaS experience and tenant-wide logical place for streaming data. It lists data from every source, allowing users to discover, ingest, manage, and react to it. It contains both streams and KQL Database tables. Streams include **Data streams**, **Microsoft sources** (such as Azure Event Hubs, Azure IoT Hub, Azure SQL Database (DB) Change Data Capture (CDC), Azure Cosmos DB CDC, Azure Data Explorer, and PostgreSQL DB CDC), **Fabric events** (workspace item events, OneLake events, and Job events), and **Azure events**, including Azure Blob Storage events and external events from Microsoft 365 or other clouds services.

The Real-Time hub makes it easy to discover, ingest, manage, and consume data-in-motion from a wide variety of sources to collaborate and develop streaming applications in one place. For more information, see What is the Real-Time hub?

Fabric compute engines

All Microsoft Fabric compute experiences come preconfigured with OneLake, like Office apps automatically use organizational OneDrive. Experiences like Data Engineering, Data Warehouse, Data Factory, Power BI, and Real-Time Intelligence use OneLake as their native store without extra setup.



OneLake lets you instantly mount your existing PaaS storage accounts by using the Shortcut feature. Shortcuts provide zero-copy access to external data sources, such as Azure Data Lake Storage, Amazon S3, and Google Cloud Storage, without ETL or data migration. You can also create shortcuts to other storage systems, allowing you to analyze cross-cloud data with intelligent caching that reduces egress costs and brings data closer to compute.

Fabric solutions for ISVs

If you're an Independent Software Vendors (ISVs) looking to integrate your solutions with Microsoft Fabric, you can use one of the following paths based on your desired level of integration:

- **Interop** - Integrate your solution with the OneLake Foundation and establish basic connections and interoperability with Fabric.
- **Develop on Fabric** - Build your solution on top of the Fabric platform or seamlessly embed Fabric's functionalities into your existing applications. You can easily use Fabric capabilities with this option.
- **Build a Fabric workload** - Create customized workloads and experiences in Fabric, tailoring your offerings to maximize their effect within the Fabric ecosystem.

ISVs can also use OneLake shortcuts and governed cross-tenant data sharing to securely access and integrate customer data across tenant boundaries without duplicating it.

For more information, see the Fabric ISV partner ecosystem.

Related content

- Microsoft Fabric terminology
- Create a workspace
- Navigate to your items from Microsoft Fabric Home page
- End-to-end tutorials in Microsoft Fabric

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 03/31/2026

What's new in Microsoft Fabric?

This article is continuously updated with preview/general availability status of Microsoft Fabric.

- To stay up to date with the latest news and features, see the Microsoft Fabric Updates Blog.
- For Microsoft Fabric release plans, see the Microsoft Fabric Roadmap.
- For community, marketing, case studies, and industry news, see the Microsoft Fabric Blog.
- For the latest in Power BI, see What's new in Power BI?

New to Microsoft Fabric?

- Learning Paths for Fabric
- Get started with Microsoft Fabric
- End-to-end tutorials in Microsoft Fabric
- Fabric terminology

Features currently in preview

The following table lists the features of Microsoft Fabric that are currently in preview. Preview features are sorted alphabetically.

> **Note**
>
> Features currently in preview are available under supplemental terms of use. Review the legal terms that apply to Azure features that are in beta, preview, or otherwise not yet released into general availability. Microsoft Fabric provides previews to give you a chance to evaluate and share feedback with the product group on preview features before they become generally available (GA).

Expand table

Feature	Learn more
Activator as business events publisher (Preview)	Activator provides a no-code way to publish business events in Microsoft Fabric. When Activator detects that a condition is met in your data, it can emit a structured business event into Real-Time hub, making that signal discoverable, routable, and consumable by your entire organization. For

Feature	Learn more
	more information, see What is Fabric Activator? and Business events overview.
Activator rule management (Preview)	Activator rule management (Preview) provides a central Real-Time Hub view for rules created across Fabric experiences.
Activator rules from catalog and workspace (Preview)	Activator rules from catalog and workspace views (Preview) let you create and manage rules from the context of the data they monitor. For more information, see Create and activate an Activator rule.
Advanced DAX generation for semantic models (Preview)	Advanced DAX generation for Fabric data agents (Preview) lets a data agent plan its approach, inspect semantic model values and intermediate results, refine its reasoning, and execute a final DAX query before answering. For more information, see Fabric data agent runtime and Semantic model best practices for data agent.
Advanced edit for data destination queries in Dataflow Gen2 (Preview)	You can now use advanced edit for data destination queries in Dataflow Gen2 to modify destination-side query logic directly in authoring.
AI-assisted Synapse to Fabric migration skills (Preview)	AI-assisted Synapse to Fabric migration skills (Preview) move Synapse Spark pools, lake databases, notebooks, Spark job definitions, and Data Factory pipelines to Microsoft Fabric from the command line by using REST APIs, with lift-and-shift and migrate-and-modernize paths and a read-only assessment step. For more information, see Skills for Microsoft Fabric.
AI Functions multimodal support (Preview)	AI Functions apply one-line, LLM-powered transformations to large pandas or PySpark DataFrames with high concurrency by default. For more information, see Use multimodal input with AI Functions (Preview).
ALTER COLUMN for Fabric Data Warehouse (Preview)	ALTER TABLE ... ALTER COLUMN (Preview) lets you make supported schema changes directly on warehouse tables with familiar T-SQL syntax. Supported scenarios complete as metadata-only updates without rewriting Parquet files. For more information, see ALTER TABLE (Transact-SQL) syntax for warehouse in Fabric.
ALTER DATABASE SET options in SQL database	`ALTER DATABASE SET` options in SQL database are now available as a preview feature. For more information, see ALTER DATABASE SET options (Transact-SQL) for SQL database in Fabric. Collation support and full-text indexing are also now available in SQL database as preview features.
Anomaly detection (Preview)	With a no-code interface, automatic model selection, and flexible alerts, tracking changes and unexpected events is easy with Anomaly detection in Real-Time Intelligence (Preview). For more information, see AI-Powered Real-Time Intelligence with Anomaly Detection (Preview). Billing for Anomaly Detector starts in December.
Anonymous data access for Fabric Apps (Preview)	Anonymous data access for Fabric Apps (Preview) lets an app expose selected entity operations to unauthenticated visitors while tenant settings

Feature	Learn more
	and data-model roles limit access. For more information, see Anonymous data access.
Approval activity in Data Factory pipelines (Preview)	The new Approval activity (Preview) in Fabric Data Factory pipelines pauses execution until a reviewer approves or rejects through Outlook or Teams, then routes pipeline flow dynamically based on the outcome (approved, rejected, or timed out). For more information, see Approval activity documentation.
Associate an identity for items (Preview)	Associated identities for items (Preview) let you associate a user, service principal, or managed identity with Fabric Lakehouses and Eventstreams via REST API, removing the dependency on the item owner. For more information, see Manage identities associated with Fabric items.
Automatic index compaction in SQL database	Automatic index compaction helps you reduce the consumption of storage space, disk I/O, memory, and improve workload performance without investing time and effort into index maintenance jobs.
Autoscaling for Virtual Network Data Gateway (Preview)	Autoscaling for Virtual Network Data Gateway (Preview) automatically adjusts the number of active gateway nodes within admin-defined minimum and maximum limits based on workload demand, so the gateway absorbs spikes without manual intervention and scales down to reduce cost during idle periods.
AWS Glue mirroring (Preview)	A mirrored AWS Glue catalog brings AWS Glue Iceberg tables into OneLake for Fabric analysis without duplicating data. For more information, see Mirrored AWS Glue catalog.
Azure Event Hubs source workspace identity authentication (Preview)	Azure Event Hubs source workspace identity authentication lets an Eventstream Azure Event Hubs source use workspace identity instead of shared access keys for Microsoft Entra ID-based access. For more information, see Add an Azure Event Hubs source to an eventstream.
Azure Monitor mirroring (Preview)	Azure Monitor Logs mirroring (Preview) brings Log Analytics workspace tables into OneLake for Fabric analysis without duplicating data. For more information, see Mirror Azure Monitor in Microsoft Fabric.
Azure Monitor to Fabric Eventhouse (Preview)	Azure Monitor to Fabric Eventhouse (Preview) lets you route VM telemetry via Azure Monitor Agent and Data Collection Rules into Eventhouse for schema-managed ingestion, ad hoc queries, time-series analytics, and activation. For more information, see Send virtual machine client data to Fabric and Azure Data Explorer (Preview).
Back up Dataflow query results (Preview)	Dataflow query-result backup (Preview) preserves prior query results in a lakehouse so you can compare or recover earlier refresh output.
Branch workspace admin profile (Preview)	Branch workspace admin profiles (Preview) let administrators define governed identities and settings that developers use to create branch

Feature	Learn more
	workspaces. For more information, see Development process using branch workspaces.
Build Agent with AI (Preview)	The Build agent with AI (Preview) feature is an AI-assisted creation experience that generates and refines Fabric Data Agent configurations through an interactive workflow, replacing manual setup with schema- and conversation-aware recommendations for agent instructions, data source guidance, and example queries; the preview focuses on SQL and Eventhouse scenarios. For more information, see Fabric data agent concepts.
Bulk Copy API for faster ingestion (Preview)	Bulk Copy API (Preview) writes data directly to Fabric Data Warehouse tables from application code without first staging files, replacing row-by-row `INSERT` statements to improve ingestion throughput. You can use it through SqlBulkCopy for C#, SQLServerBulkCopy for Java, the bcp utility, and tools such as SSIS, Azure Data Factory, and Informatica. For more information, see Ingest data into your warehouse by using BCP API (Preview).
Business Actions activity (Preview)	Business Actions activity (Preview) lets pipelines call supported business systems and workflows. For more information, see Business Actions.
Capacity Operation Events in Real-Time Hub (Preview)	Capacity Operation Events in Real-Time Hub (Preview) provide operation-level telemetry for capacity consumption, duration, throttling, workspaces, and items. For more information, see Explore Fabric capacity operation events.
Centralized data governance in the OneLake catalog (Preview)	A new centralized data governance experience in the OneLake catalog is in preview. Data owners can view aggregated insights on the items they created, consider improving their governance by taking recommended actions, and access more information along with all available tools in Fabric.
Change event streaming in SQL database in Fabric (Preview)	Change event streaming (CES) captures row-level DML changes (inserts, updates, and deletes) in SQL database in Fabric and publishes each change to Fabric Eventstream or Azure Event Hubs in near real time as a CloudEvent, enabling downstream analytics and event-driven architectures with no ETL. For more information, see Change event streaming overview.
Change Git branch with Contributor role (Preview)	Change Git branch with Contributor role lets workspace Contributors and Members switch the connected Git branch without workspace Admin permissions. For more information, see Allow Contributors and Members to switch branches.
CI/CD Support for SQL analytics endpoint (Preview)	CI/CD support for SQL analytics endpoint (Preview) lets you manage SQL analytics endpoint definitions as DacFx database projects in Git alongside other Fabric items and deploy incremental schema changes through Fabric Deployment Pipelines. For more information, see Source control (preview).
Conditional activity retries in Data Factory pipelines (Preview)	Conditional activity retries (Preview) let you define exactly when a pipeline activity retries based on the error code, error message, or both with AND/OR logic, so long-running pipelines automatically recover from transient failures

Feature	Learn more
	while fast-failing on errors that don't self-resolve. For more information, see Activity overview.
Configurable Data Retention in Fabric Data Warehouse (Preview)	Configurable data retention (Preview) in Fabric Data Warehouse lets you set the data history retention window for a warehouse between 1 and 120 days with a single T-SQL command. This retention period governs time travel, point-in-time clones, restore points, and warehouse snapshots. For more information, see Data retention in Fabric Data Warehouse (Preview).
Continuous ingestion from Azure Storage to Eventhouse (Preview)	You can now use continuous ingestion from Azure Storage to Eventhouse (preview) to automatically and efficiently ingest data from Azure Storage into Eventhouse. For more information, see Get data from Azure Storage.
Copilot in Fabric is available worldwide	Copilot in Fabric is now available to all customers. For more information on release status, see Release status of AI and Copilot experiences in Fabric.
Copy and share warehouse SQL queries with context (Preview)	You can now copy and share warehouse SQL queries with context (Preview). You can paste a query with its results or generate a link that opens the query in another tool.
Copy job and Eventstream integration (Preview)	Copy job and Eventstream integration (Preview) uses Eventstream as a Copy job source or destination. For more information, see What is Copy job?.
Create AI Auto-Description for Semantic Models (Preview)	Auto-Description for Semantic Models (Preview) uses Copilot to generate clear descriptions for semantic models from the model's metadata and structure, which model owners and contributors can review, edit, and regenerate directly from the semantic model details page in the OneLake catalog. For more information, see OneLake catalog overview.
Custom authorization for API for GraphQL (Preview)	Authorizer User Data Functions for API for GraphQL (Preview) let you attach a User Data Function that runs before each GraphQL operation and evaluates request context such as tenant ID, role, or token claims to allow, restrict, or deny access based on your own policy logic. For more information, see What is Microsoft Fabric API for GraphQL? and Create a Fabric User data functions item.
Custom sort in Dataflow Gen2 (Preview)	Custom sort in Dataflow Gen2 (Preview) defines multi-column sort precedence in one dialog. For more information, see Sort columns.
Data agent feedback in Microsoft 365 Copilot (Preview)	Data agent feedback in Microsoft 365 Copilot (Preview) helps creators review consumer feedback and improve agent responses. For more information, see Data agent consumer and creator feedback.
Data agent runtime model upgrade (Preview)	Data agent runtime model upgrade (Preview) uses a newer model for planning, source routing, semantic querying, and natural language to SQL. For more information, see Fabric data agent runtimes.
Data agents return up to 1,000 rows (Preview)	Fabric data agents return up to 1,000 rows (Preview), expanding the results available for downstream analysis. For more information, see Data agent file

Feature	Learn more
	support .
Data Agent support for Eventhouse functions, materialized views, and shortcuts (Preview)	Data Agent now discovers and queries Eventhouse user-defined functions (UDFs), materialized views, and shortcut tables, so it calls validated KQL logic, uses pre-aggregated views for faster answers, and reaches data federated from sources outside the eventhouse. For more information, see Fabric data agent concepts.
Data agent topics (Preview)	Topics in Fabric data agents (Preview) provide large, subject-specific guidance for supported data sources. For more information, see Configure Fabric data agents.
Database agent (Preview)	Database agent (Preview) analyzes SQL and PostgreSQL performance, availability, workload health, and cost signals in Database Hub.
Database Hub (Preview)	Database Hub (Preview) provides a unified operational experience for managing database estates across Azure, Microsoft Fabric, on-premises environments, and supported multicloud environments.
Data Factory Adaptive performance tuning (Preview)	Adaptive Performance Tuning is designed to intelligently optimize data movement performance based on your configuration and runtime context. This preview feature makes tuning Data Factory simpler, safer, and more effective without requiring deep manual expertise or trial-and-error adjustments. To enable, see the adaptivePerformanceTuning property when using copy activity.
Data Factory in Fabric connection recency (Preview)	Connection recency (Preview) adds last-linked-to-items and last-credentials-used properties to connections for audit and lifecycle management. For more information, see Data source management.
Data Factory MCP (Preview)	Data Factory MCP (Preview) enables AI assistants to create, test, and deploy Dataflow Gen2 through natural language without manual configuration. For more information, see Data Factory MCP on GitHub .
Data Factory On-premises data gateway manual update option (Preview)	The Gateway December 2025 release (version 3000.298) provides a new manual update option (in preview) via the Fabric portal or API.The November release serves as the baseline version for this feature, and you can start performing manual updates beginning in December. For more information, see Update an on-premises data gateway.
Data Factory SharePoint data source site picker (Preview)	The SharePoint Site Picker (Preview) replaces manual URL entry with a browsable, searchable dropdown for selecting SharePoint sites across Dataflow Gen2, Pipelines, Copy job, and Lakehouse shortcuts. For more information, see SharePoint Folder connector.
Dataflow data visuals (Preview)	Data visuals in Dataflow Gen2 (Preview) profile and visualize transformed data while you author a query. For more information, see Create data visuals in Dataflow Gen2 .

Feature	Learn more
Dataflow destination expressions (Preview)	Dynamic destination expressions in Dataflow Gen2 (Preview) derive destination names from parameters, refresh dates, and workspace context. For more information, see Use the dynamic expression editor.
Dataflow dynamic warehouse schema (Preview)	Dynamic warehouse destination schemas (Preview) evolve destination tables when Dataflow Gen2 query output changes. For more information, see Data destination schema options.
Dataflow offset transformation (Preview)	Offset transformation in Dataflow Gen2 (Preview) shifts date, datetime, or time values forward or backward without custom expressions.
Dataflow refresh details in Monitoring hub (Preview)	Dataflow refresh details in Monitoring hub (Preview) expose query-level refresh details without leaving the hub. For more information, see Monitor Dataflow Gen2.
Dataflow shared queries (Preview)	Shared queries in Dataflow Gen2 (Preview) let colleagues publish, discover, and reuse governed Power Query logic. For more information, see My Queries and Shared Queries.
Data protection: DSPM for AI for Fabric Copilots and data agents (Preview)	Data Security Posture Management (DSPM) for AI for Fabric (Preview) monitors Copilot and data agent interactions to detect sensitive information in AI prompts and responses, investigate risky AI behavior, and apply governance through Purview Audit and eDiscovery. For more information, see DSPM for AI in Fabric.
Data replication from Lakehouse with Delta Change Feed (Preview)	The Fabric Lakehouse Table connector provides changed data from a Fabric Lakehouse via Delta Change Data Feed (CDF), to supported destinations. For more information, see Data Replication from Fabric Lakehouse with Delta Change Data Feed (Preview).
Data Warehouse Migration Assistant for Teradata (Preview)	Get started planning your Teradata migration to Fabric Data Warehouse (preview) with the Fabric Migration Assistant for Data Warehouse.
Delegated Custom Live Pool management (Preview)	Delegated Custom Live Pool management (Preview) lets workspace Members configure pools through Fabric environments when an administrator enables delegation. For more information, see Configure Custom Live Pools.
Database Hub (Preview)	Database Hub (Preview) provides a unified operational experience for managing your database estate across Azure, Microsoft Fabric, on-premises environments, and supported multicloud environments.
dbt Job in Fabric Data Factory (Preview)	A dbt Job in Fabric Data Factory lets you author, schedule, and monitor dbt projects natively with serverless execution, integrated testing and documentation, and governance via Entra ID and SQL security policies. For more information, see dbt job in Microsoft Fabric (preview).

Feature	Learn more
Delegated OneLake Shortcuts (Preview)	Delegated OneLake Shortcuts (Preview) let you attach an organizational account, service principal, or workspace identity to a OneLake shortcut so downstream users access the target through the delegated identity. The same model works across Microsoft Fabric tenants for parent-subsidiary and test-to-production sharing without individual cross-tenant permissions. For more information, see OneLake shortcuts and Create a OneLake shortcut.
Delta column mapping in the SQL analytics endpoint	SQL analytics endpoint now supports Delta tables with column mapping enabled. For more information, see Delta column mapping. This feature is currently in preview.
DeltaFlow transformation in Fabric Real-Time Intelligence's Eventstream (preview)	DeltaFlow is a capability in Fabric Eventstream that transforms raw Change Data Capture (CDC) events into a flattened, analytics-ready format. Instead of working with deeply nested Debezium JSON payloads, DeltaFlow produces tabular rows that closely mirror the structure of the source database tables, enriched with metadata columns that describe each change. Eventstream supports DeltaFlow transformations, which are currently in preview. For more information, see DeltaFlow output transformation (preview) and Business events overview.
Deployment plans (Preview)	Deployment plans (Preview) define item deployment order and pre/post actions on a visual canvas. For more information, see Plan CI/CD for Fabric solutions.
Digital twin builder (Preview)	Digital twin builder (preview) is an item within the Real-Time Intelligence workload. Digital twins create data-driven, real-time representations of entities. It's a data modeling item that creates digital representations of real-world environments, to optimize physical operations using data. For more information, see What is digital twin builder (preview)?
Efficient Scaledown for Spark jobs (Preview)	Efficient Scaledown (Preview) decouples Spark shuffle data from executor lifetime by routing shuffle output to Azure Blob Storage, so clusters scale down faster and tolerate executor loss with no notebook or query changes. It requires the Native Execution Engine on Fabric Spark Runtime 3.5 or later. For more information, see Efficient scaledown and Native execution engine for Fabric Data Engineering.
Evaluate Power Query programmatically (Preview)	Evaluate Power Query Programmatically in Microsoft Fabric (Preview) introduces a public REST API to execute Power Query M scripts programmatically, enabling automation, integration with Spark and pipelines, and access to 100+ data sources with 90-second timeout for transformations. For more information, see Execute Query API Reference.
Eventhouse workspace-monitoring logs (Preview)	Eventhouse workspace-monitoring logs (Preview) expose throttling, suboptimal-size, and scale-out events. For more information, see Monitor Eventhouse.

Feature	Learn more
Eventstream Business Events publisher (Preview)	Eventstream now publishes Business Events as a built-in destination, so you filter, aggregate, threshold, and emit governed, discoverable business signals from the Eventstream canvas with no code.
Eventstream custom connector (Preview)	Eventstream custom connector (Preview) adds proprietary or unsupported streaming sources through an extensible connector framework. For more information, see Custom stream connector.
Eventstream processing logs (Preview)	Eventstream processing logs (Preview) send Azure Stream Analytics warnings, failures, and recovery events to workspace monitoring. For more information, see Eventstream workspace monitoring.
Eventstream workspace monitoring (Preview)	Workspace monitoring for Eventstreams (Preview) lets you choose which eventstreams emit performance, error, and node-health data to three KQL tables in the workspace monitoring Eventhouse. For more information, see Eventstream monitoring in Workspace Monitoring and Workspace Monitoring overview.
Eventstream workspace Private Link (Preview)	Eventstream workspace Private Link (Preview) secures supported streaming workloads at workspace scope.
Excel to Delta tables with shortcut transformations (Preview)	Excel to Delta tables with shortcut transformations (Preview) extends the zero-code shortcut transformations ingestion model to multi-sheet `.xlsx` and `.xls` workbooks, with automatic sheet discovery, wildcard sheet matching, combined-table or table-per-sheet output, schema drift handling, continuous sync, and sanitization of sheet names into valid Delta table names. For more information, see Shortcut transformations (file).
Extended Auto partition support in Copy job for Oracle, Fabric Lakehouse, and SAP HANA (Preview)	Extended Auto partition support in Copy job (Preview) automatically scales out reads and writes for Oracle, Fabric Lakehouse, and SAP HANA without predefining partition columns or custom sharding logic. For more information, see What is Copy job?
Fabric Actions activity (Preview)	Fabric Actions activity (Preview) gives pipelines authenticated access to supported Fabric REST operations. For more information, see Fabric Actions.
Fabric Activator integration with warehouse SQL queries (Preview)	Fabric Activator integration with warehouse SQL queries (Preview) lets you define conditions on query results and trigger follow-up actions. For more information, see Set alerts on warehouse queries.
Fabric Admin APIs	Fabric Admin APIs are designed to streamline administrative tasks. The initial set of Fabric Admin APIs is tailored to simplify the discovery of workspaces, Fabric items, and user access details.
Fabric Apps backend with the Rayfin SDK (Preview)	Rayfin (Preview) is an open-source SDK and CLI that lets developers and coding agents define a complete application backend in code—data models, APIs, identity, access policies, and business logic—and deploy it to Microsoft Fabric, where the resulting apps run as first-class items with built-in

Feature	Learn more
	governance and direct access to OneLake. For more information, see Fabric Apps.
Fabric Apps functions and connectors (Preview)	Fabric Apps functions and connectors (Preview) add Fabric data connectors, PostgreSQL support, server-side functions, secret storage, application storage, and built-in metrics.
Fabric-CICD tool v1.2.0 – new bulk publish mode (Preview)	fabric-cicd bulk publish mode lets fabric-cicd publish multiple items in one Fabric bulk import API call and fall back to standard publishing for unsupported scenarios.
Fabric CLI in Azure DevOps (Preview)	The Fabric CLI in Azure DevOps (Preview) provides the Fabric CLI (`fab`) as a built-in Azure DevOps pipeline task for automating workspace and item management without manual tool installation. Get started with the Fabric CLI (generally available) at https://aka.ms/FabCLI.
Fabric Connection inside Notebook (Preview)	Fabric Connection inside Notebook (Preview) enables you to create and manage cloud data source connections directly within notebooks. For more information, see What is Fabric Connection in notebooks?
Fabric Data Agent integration with Azure AI Agent Service (Preview)	We're excited to launch the integration of data agents in Fabric with Azure AI Agent Service from Microsoft Foundry. To get started, see Consume a Fabric Data Agent in Microsoft Copilot Studio (preview). The Fabric data agent SDK is also available in preview.
Fabric data agents evaluation with the Python SDK (Preview)	You can now use the Python SDK to programmatically evaluate Fabric data agents. For more information, see Consume a Fabric Data Agent in Microsoft Copilot Studio (preview).
Fabric data engineering agent (Project Osmos) (Preview)	Fabric data engineering agent (Project Osmos) runs persistent, Fabric-hosted tasks that autonomously plan, execute, and validate data engineering work against authorized lakehouse resources. Start tasks from a supported command-line client, review operational settings, and monitor progress and results in Fabric.
Fabric Data Factory Upsert table action in the Lakehouse Connector (Preview)	The Upsert (Preview) table action in the Lakehouse Connector is in preview.
Fabric Data Warehouse GPU query acceleration (Preview)	GPU-accelerated Fabric Data Warehouse (Early Access Preview) uses NVIDIA accelerated computing to run eligible T-SQL queries on GPUs with no query rewrites, delivering up to 7x faster performance at high concurrency across all SQL analytics endpoints and warehouses in a workspace once you enable it from workspace settings. For more information, see Query acceleration in Fabric Data Warehouse (preview).
Fabric Data Warehouse MCP server (Preview)	Agentic warehouse development with MCP and Agent Skills (Preview) introduces a remote Data Warehouse MCP server that lets AI coding agents work with Fabric warehouses through authenticated, governed tools. For

Feature	Learn more
	more information, see Data Warehouse MCP server and Skills for Microsoft Fabric.
Fabric IQ data-agent and ontology answers (Preview)	Fabric IQ data-agent and ontology answers (Preview) extend business-question answering beyond reports and semantic models.
Fabric IQ (Preview) workload	Fabric IQ (preview) is a new workload for unifying business semantics across data, models, and systems to power intelligent agents and decisions grounded in a live, holistic view of the business.
Fabric item relations API (Preview)	Fabric item relations API (Preview) returns upstream and downstream item dependencies, typed relation edges, and related workspaces for impact analysis, catalogs, CI/CD checks, and AI agents. For more information, see Fabric REST API reference.
Fabric Lakehouse support in dbt job (Preview)	Fabric Lakehouse support in dbt job (Preview) lets you run dbt projects directly against a new or existing lakehouse and apply dbt models and tests where your data is stored. For more information, see dbt job in Microsoft Fabric.
Fabric Maps in Real-Time Dashboards (Preview)	Fabric Maps in Real-Time Dashboards (Preview) lets you add a published Fabric Map as a dashboard tile while keeping the map's layers, filters, and styling in the map item. For more information, see Add a Fabric Maps tile.
Fabric Migration Assessment for Data Factory (Preview)	The new Fabric Migration Assessment for Data Factory (Preview) provides a built-in assessment and migration experience that evaluates ADF pipeline readiness and migrates supported pipelines into a Fabric workspace with connection mapping. For more information, see Migration planning for Azure Data Factory to Fabric Data Factory.
Fabric Observability Insights (Preview)	Fabric Observability Insights (Preview) uses an operations agent and Fabric-native context to investigate pipeline failures. For more information, see Workspace monitoring overview.
Fabric SKU estimator (Preview)	The Microsoft Fabric SKU estimator, now available in preview, is an enhanced version of the previously introduced Microsoft Fabric Capacity Calculator. For more information, see Introducing the Microsoft Fabric SKU estimator (preview) and Mastering SKU Estimations with the Microsoft Fabric SKU Estimator.
File-level Git commit (Preview)	File-level Git commit (Preview) commits selected files from a supported Fabric item while unfinished files remain in the workspace. For more information, see Compare and commit Fabric items.
Folder REST API (Preview)	You can now create and manage workspace folders in automation scenarios and integrate with other systems and tools. The Folder Rest API is now in preview. To get started, see Fabric REST API Folders.

Feature	Learn more
Git Integration Workspace Relation API (Preview)	The Workspace Relations API lets automation create, query, update, and delete relationships between parent and branched workspaces. For more information, see Development process using the Branch out experience.
Google Lakehouse Runtime Catalog mirroring (Preview)	Google Lakehouse Runtime Catalog mirroring (Preview) brings Google-managed Iceberg catalog metadata into OneLake without copying data, so you can use the tables across Fabric workloads. For more information, see Google Lakehouse Runtime Catalog mirroring.
Improved NL2SQL Engine for Fabric Data Agent (Preview)	Improved NL2SQL Engine (Preview) for Fabric Data Agent improves accuracy, transparency, and resilience when translating natural language to SQL across lakehouses, warehouses, and mirrored databases. It uses example queries to follow patterns, asks clarifying questions when intent is ambiguous, and surfaces structured diagnostics, available when you enable the Preview Fabric Data Agent Runtime. For more information, see Fabric data agent concepts.
Investigator insights in Operations Agent (Preview)	Investigator insights in Operations Agent analyze related data after an operations agent alert and surface scope, observations, patterns, and recommended next steps in Teams. For more information, see Operations Agent actions.
Invoke SSIS Package activity in Data Factory pipelines (Preview)	The Invoke SSIS Package activity (preview) enables package execution from pipeline orchestration in Fabric. For more information, read Blog: Invoke SSIS Package Activity in Microsoft Fabric (preview).
Item History in Fabric Capacity Metrics App (Preview)	The Item History page in the Fabric Capacity Metrics App (Preview) provides a 30-day view of compute capacity consumption with interactive visuals and slicers for workspace and item-level analysis. For more information, see Understand the metrics app item history page (preview).
Item soft-delete and recovery events (Preview)	Item soft-delete and recovery events (Preview) expose lifecycle events in Real-Time Hub for automation and audit scenarios. For more information, see Fabric workspace item events.
JobInsight Diagnostics Library (Preview)	JobInsight is a diagnostics library to analyze completed Spark applications via APIs for queries, jobs, stages, tasks, executors, and event logs. For more information, see Job Insight Library (Preview).
Lakehouse auto-binding in Git (Preview)	Lakehouse auto-binding in Git (Preview) resolves lakehouse references automatically as notebooks move across Git-connected workspaces, making notebooks portable across development, test, and production environments. For more information, see Notebook source control and deployment.
Lakehouse support for git integration and deployment pipelines (Preview)	The Lakehouse now integrates with the lifecycle management capabilities in Microsoft Fabric, providing a standardized collaboration between all development team members throughout the product's life. Lakehouse git integration and deployment pipelines facilitates an effective product

Feature	Learn more
	versioning and release process by continuously delivering features and bug fixes into multiple environments.
Large Eventstream and Eventhouse messages (Preview)	Large message support (Preview) ingests richer event payloads through Eventstream and Eventhouse. For more information, see Eventstream overview.
Load Fabric OneLake Data in Excel	Easily load Fabric OneLake data into Excel with integrated OneLake catalog and modern Get Data experience (preview). For more information and steps to get started, see OneLake catalog and Get Data are integrated into Excel for Windows.
Manage failure notifications from the monitoring hub in Fabric (Preview)	The Schedule failures page in the Fabric monitoring hub (Preview) lists every scheduled item with failure notifications configured and lets you add, edit, or remove recipients for any item from one place, without opening each item individually. For more information, see Schedule failures (preview) and Receive notifications for failed scheduled jobs.
Mapping Data Flow Transforms in Dataflow Gen2 (Preview)	Mapping data flow transforms in Dataflow Gen2 (Preview) introduces Spark-based transformation capabilities directly in the Dataflow Gen2 creation experience, enabling large-scale data transformations with Spark compute and migration of mapping data flows to Fabric. For pricing details, see Dataflow Gen2 pricing.
Maps external feature services (Preview)	External feature services in Fabric Maps (Preview) connect hosted GIS and standards-based geospatial services. For more information, see Create layers from external feature services.
Materialized Lake View refresh features (Preview)	The Refresh Materialized Lake View activity in Fabric Data Factory pipelines in Fabric Data Factory pipelines lets you refresh materialized lake views on demand or on a schedule within your pipeline orchestration. Event-driven refresh for materialized lake views triggers dependent materialized lake view refreshes when OneLake file events or pipeline and notebook job events occur. For more information, see Schedule a materialized lake view refresh.
Maven support in Fabric Environment (Preview)	Maven support in Fabric Environment (Preview) lets Scala and Java developers add libraries from Maven repositories to a Fabric Environment by uploading and managing `pom.xml` files directly, so transitive dependencies are resolved without manually downloading and uploading JARs. For more information, see Manage libraries in Fabric environments.
MAX_ROLLOVER_FILES in extended event sessions using Azure Storage blobs (preview)	For new extended event sessions in SQL database in Fabric, you can specify the `MAX_ROLLOVER_FILES` option to retain only the specified number of the most recent Azure Storage blobs. For more information, see event_file target.
Max vCores limit option in SQL databases (Preview)	The Max vCores limit option (Preview) lets you cap SQL database compute at 4 or 32 vCores to control costs and protect shared capacities from noisy-neighbor scenarios. For more information, see Control compute usage.

Feature	Learn more
MCP remote servers for Activator and Eventhouse in Real-Time Intelligence (preview)	Fabric Real-Time Intelligence provides hosted Model Context Protocol (MCP) remote servers for Activator and Eventhouse, enabling AI assistants and agents to interact with RTI components through natural language. By using the Activator MCP server, AI assistants can create monitoring rules, manage alerts, and trigger actions in Fabric Activator. The Eventhouse remote MCP server allows AI agents to query, reason over, and act on real-time data stored in an RTI eventhouse by configuring a URL that points to the Eventhouse. For more information, see Get started with the remote MCP server for eventhouses (preview) and Get started with the remote MCP server for Activator (preview). This feature is currently in preview.
Metropolis VoiceLake (Preview)	Metropolis VoiceLake (Preview) brings supported communications data into a lakehouse with a semantic model and Power BI reports.
Microsoft ADO.NET Driver for Fabric Data Engineering (Preview)	The Microsoft ADO.NET Driver for Fabric Data Engineering (Preview) lets .NET applications connect to Spark SQL in Fabric using standard ADO.NET patterns with Microsoft Entra ID authentication, connection pooling, and async prefetch via Livy APIs. For more information, see Microsoft ADO.NET driver for Microsoft Fabric Data Engineering (preview).
Microsoft Fabric Embed (Preview)	Microsoft Fabric Embed (Preview) lets you embed Real-Time Dashboards in JavaScript or TypeScript apps with delegated Microsoft Entra access. For more information, see Power BI embedded analytics and What is Microsoft Fabric Embed in Microsoft Fabric (Preview)?.
Microsoft Fabric policies (Preview)	Microsoft Fabric policies let administrators centrally define and enforce condition-based governance rules at tenant or capacity scope. Policies support centralized management in the OneLake catalog, audit logging, REST APIs, Git integration, and CI/CD automation.
Microsoft ODBC Driver (Preview)	The Microsoft ODBC Driver for Fabric Data Engineering (Preview) enables .NET, Python, and other ODBC-compatible applications and BI tools to connect to Spark SQL in Fabric through Livy APIs with Microsoft Entra ID authentication, session reuse, and async prefetch. For more information, see Microsoft ODBC driver for Microsoft Fabric Data Engineering (Preview).
Mirrored Dremio catalog in OneLake (Preview)	Fabric can mirror Dremio Iceberg REST Catalog metadata to enable zero-copy access to Dremio-managed Iceberg tables in OneLake via shortcuts. Tables appear in OneLake within seconds after selection, with optional auto-include of newly added tables and SQL analytics endpoint querying across Fabric workloads. For more information, see Bring your Dremio data into OneLake (Preview) and Mirrored Dremio catalog.
Mirroring for Azure Database for MySQL (Preview)	Mirroring in Fabric provides an easy experience to avoid complex ETL (Extract, Transform, Load) processes and seamlessly integrate your existing Azure Database for MySQL data with the rest of your data in Fabric. For more information, see Mirroring Azure Database for MySQL.

Feature	Learn more
ML endpoint runtime and tensor support (Preview)	ML endpoint runtime and tensor support (Preview) add Spark 4.1 and tensor-based model support for real-time serving scenarios.
Modern evaluation engine for VNet data gateways (Preview)	Modern evaluation engine for VNet data gateways (Preview) provides an opt-in execution path for supported mashup evaluations so you can test semantic model refresh, Dataflow Gen2 refresh, and DirectQuery performance without changing your private network architecture. For more information, see Modern evaluation engine for VNet data gateways.
Modern Get Data Azure browsing (Preview)	Azure resource browsing in Modern Get Data (Preview) selects supported Azure resources without copying endpoint values from the Azure portal. For more information, see Get data in Dataflow Gen2.
Monitor for Fabric Data Warehouse (Preview)	Monitor for Fabric Data Warehouse (Preview) (previously Query Activity) is unified monitoring for Fabric Data Warehouse that shows active and historical queries side by side, supports cross-execution performance analysis powered by Query Insights, and lets you cancel long-running queries with one click. For more information, see Monitor T-SQL queries in Fabric Data Warehouse (Preview).
MSSQL extension for VS Code Fabric integration (Preview)	MSSQL extension for VS Code Fabric integration (Preview) adds support for connecting, running queries, and managing objects in SQL databases in Fabric directly from Visual Studio Code. Download the extension at MSSQL extension at marketplace.visualstudio.com.
Multiple-Schema Inferencing in Eventstream (Preview)	Multiple schema inferencing (preview) lets you work with multiple data sources that emit varying schemas by inferring and managing multiple schemas simultaneously. For more information, see Enhancing Data Transformation Flexibility with Multiple-Schema Inferencing in Eventstream (Preview).
Native Execution Engine JSON acceleration (Preview)	Native Execution Engine JSON acceleration (Preview) reads and parses JSON through the vectorized native path so supported Spark transformations can stay columnar without code changes. For more information, see Native execution engine for Fabric Data Engineering and Apache Spark runtime in Fabric.
Native Planning Engine (Preview)	Native Planning Engine (Preview) scales planning models to tens of millions of cells and recalculates dependent values as inputs change. For more information, see Planning in Fabric.
Notebook Copilot inline code completion (Preview)	Now in preview, Copilot Inline Code Completion (Preview) is an AI feature that assists data scientists and engineers in writing Python code more quickly and easily. For more information, see Notebook Copilot inline code completion (Preview).
Notebook debug within vscode.dev (Preview)	You can now place breakpoints and debug your Notebook code with the Synapse VS Code - Remote extension in vscode.dev. This update first starts with the Fabric Runtime 1.3 (EOSA).

Feature	Learn more
Notebook toolkit for agentic development (Preview)	Fabric Notebook Toolkit (Preview) gives coding agents notebook-aware tools for authoring and validation.
Observability for Fabric Data Agent in Microsoft Foundry (Preview)	Observability for Fabric Data Agent in Microsoft Foundry (Preview) surfaces telemetry for every call to a Fabric Data Agent used as a tool inside a Foundry agent, including latency, status, and error details, so agent builders can debug and monitor production agents. For more information, see Fabric data agent concepts.
OneLake Catalog search API, MCP, and CLI tools (Preview)	OneLake Catalog search API and MCP tool (Preview) brings cross-workspace discovery to code with a single search request to locate matching items across your accessible estate based on catalog metadata and user permissions, with the capability also included as a built-in tool in the Fabric Core MCP server for AI agents. The Fabric CLI fab find command extends the search to the terminal so you can locate items across workspaces, filter or exclude by item type, and pipe results through JMESPath. For more information, see OneLake catalog overview.
OneLake Catalog table discovery (Preview)	OneLake Catalog table discovery (Preview) lets you find tables from semantic models, lakehouses, and mirrored databases by table name, description, or exact column name in global search and the Search API. For more information, see OneLake Catalog Search API and Fabric Core remote MCP server.
OneLake table read API (Preview)	Use the table read API to read rows from a Delta Lake or Apache Iceberg table in OneLake. For more information, see Overview of OneLake table APIs.
On-premises data gateway manual update (Preview)	Manual update for the on-premises data gateway is now in preview, allowing administrators to trigger gateway updates through the Fabric portal, API, or PowerShell scripts on their own schedule. For more information, see Update an on-premises data gateway.
On-premises data gateway soft delete (Preview)	On-premises data gateway soft delete (Preview) retains supported standard mode gateways for 30 days by default so tenant and gateway admins can recover accidental deletions. For more information, see Gateway soft delete.
Ontology as data-agent context (Preview)	Ontology as data-agent context (Preview) grounds query generation in enterprise entities, relationships, definitions, and source bindings. For more information, see Ontology as context.
Ontology as Real-Time Dashboard source (Preview)	Ontology as a Real-Time Dashboard source (Preview) brings business entities and relationships into live visualizations.
Ontology (preview) item	*Ontology (preview)* in Fabric IQ lets you define entity types, relationships, properties, and other constraints to organize data according to your business vocabulary. For more information, see What is ontology (preview)?

Feature	Learn more
OpenAI plugins for Eventhouse (Preview)	You can now use two powerful AI plugins for Eventhouse: AI Embed Text Plugin and AI Chat Completion Prompt Plugin. Connect Eventhouse data to OpenAI-powered applications for advanced analytics and AI scenarios. For more information, see ai_embed_text (preview) and ai_chat_completion (preview).
Operations Agent for Pipelines (Preview)	Operations Agent for Pipelines is an AI-powered monitoring agent in Data Factory that continuously watches your pipeline runs and proactively notifies you through Microsoft Teams when failures or anomalies occur. You can configure the agent to monitor specific scenarios such as all pipeline failures or runs exceeding expected durations, without writing custom alerting logic. For more information, see Operations Agent for Pipelines (preview).
Operations Agent performance view (Preview)	Operations Agent performance view (Preview) shows monitoring, reasoning, and model-usage telemetry for an operations agent.
Optimal refresh for materialized lake view updates and deletes (Preview)	Optimal refresh for updates and deletes lets materialized lake views process changed and deleted source rows incrementally. Refresh hints identify unique rows so Fabric can avoid a full refresh when supported source data changes.
Oracle Database CDC source connector for Fabric Eventstream (Preview)	The Oracle Database CDC source connector for Eventstream (Preview) streams changes from an Oracle database into Fabric as a structured change event with before and after values, using Oracle LogMiner for on-premises or cloud databases running Oracle 12c and later. For more information, see Add Oracle Database CDC source to an eventstream (preview).
Pagination support for Eventstream HTTP connector (Preview)	You can add an HTTP source to an eventstream (preview) to ingest paginated REST API responses through page-based or cursor-based pagination, eliminating custom orchestration when bringing activity logs, audit records, or operational events into Eventstream.
Partitioned compute for Dataflow Gen2 (Preview)	Partitioned compute is a capability of the Dataflow Gen2 engine that allows parts of your dataflow logic to run in parallel, reducing the time to complete its evaluations. For more information, see partitioned compute in Dataflow Gen2.
Pass Parameter Values to Fabric Items (Preview)	Activator enables you to automatically activate Fabric items like pipeline and notebook whenever certain data conditions are met. You can not only activate and execute Fabric items but also pass values to the parameters defined in your Fabric items.
Pipeline-level dependencies (Preview)	Pipeline-level dependencies (Preview) run downstream pipelines only when configured upstream conditions are met. For more information, see Pipeline run conditions.
Prebuilt Foundry Tools in Fabric preview	The preview of prebuilt Foundry Tools in Fabric is an integration with Foundry Tools, formerly known as Azure AI services. Foundry Tools

Feature	Learn more
	(preview) allow for easy enhancement of data with prebuilt AI models without any prerequisites. Currently, prebuilt Foundry Tools are in preview and include support for the Microsoft Azure OpenAI Service, Azure Language in Foundry Tools, and Azure Translator in Foundry Tools.
Public API capabilities for Dataflow Gen2 dataflows in Fabric Data Factory (Preview)	This preview of the Dataflows Gen 2 public APIs enables users to create, update, and monitor their data workflows programmatically. The APIs support a wide range of operations including dataflows CRUD (Create, Read, Update, and Delete), scheduling, and monitoring, making it easier for users to manage their data integration processes.
Public parameter values to refresh a Dataflow Gen2 (Preview)	Learn more about the new public parameters capability for Dataflow Gen2 with CI/CD support, and the support for this new mode within the Dataflow refresh activity in Data Pipelines.
Real-Time Dashboard AI custom visual builder (Preview)	AI custom visual builder (Preview) generates purpose-built HTML visuals for Real-Time Dashboards from natural-language descriptions. For more information, see AI custom visual in Real-Time Dashboards.
Real-Time Dashboards Time Series Visualization (Preview)	Real-Time Dashboard-specific visuals in Real-Time Dashboards adds dedicated capabilities for navigating, comparing, and customizing time-based data, including a legend search bar, pin-and-overlay for series comparison, multiple aligned-time panels, flexible Y-axis scaling, color assignment, and zoom with a time slider.
Recent data in Dataflow Gen2 (Preview)	Recent data in Dataflow Gen2 (Preview) provides quick access to your most frequently used data items in the Power Query ribbon and Modern Get Data, letting you jump directly to recently used tables, files, folders, and databases without navigation. For more information, see Get data experience and What is Dataflow Gen2?
Reference data enrichment in Eventstream (Preview)	Reference data enrichment in Eventstream joins streaming events with Delta reference tables from a lakehouse by using no-code or SQL-based enrichment. For more information, see Reference data join in Eventstream using Lakehouse.
Refreshed Monitor hub (Preview)	Refreshed Monitor hub (Preview) adds at-scale health views across workloads, jobs, apps, agents, alerts, and capacities. For more information, see Monitor hub.
Refresh SQL analytics endpoint activity (Preview)	The Refresh SQL analytics endpoint activity (Preview) refreshes a lakehouse SQL analytics endpoint on demand or on a schedule within a Fabric pipeline.
Resource Profiles in Fabric Data Engineering (Preview)	Resource Profiles in Fabric Data Engineering (Preview) are preconfigured Spark compute profiles—write-heavy, read-heavy for Spark, and read-heavy for Power BI—that you select per environment so Fabric applies workload-aware settings automatically without manual tuning of dozens of Spark properties. For more information, see Configure Resource Profile Configurations.

Feature	Learn more
REST APIs for connections and gateways (Preview)	REST APIs for connections and gateways are now in preview. These new APIs allow developers to programmatically manage and interact with connections and gateways within Fabric.
Scalar user-defined functions (UDFs)	Scalar user-defined functions (UDFs) are now supported as a preview feature. For more information, see CREATE FUNCTION for Fabric Data Warehouse.
Scheduled User Data Functions (Preview)	Scheduled User Data Functions (Preview) run parameterized business logic on a recurring cadence through the Fabric job scheduler, with execution history available in the monitoring hub. For more information, see User Data Functions overview and Schedule a User Data Function.
Send email from Dataflow Gen2 (Preview)	Send email from Dataflow Gen2 (Preview) sends data-driven email directly from a dataflow without a separate orchestration step.
Service principal support for Fabric data agents (Preview)	Fabric data agents now support service principal (SPN) authentication, so custom apps and Microsoft Foundry agents call the data agent API with an application identity instead of a delegated user token. The data agent inherits workspace permissions.
Snowflake security-role mirroring (Preview)	Snowflake security-role mirroring (Preview) brings Snowflake role definitions into Fabric. For more information, see Extended capabilities in Mirroring.
SQL analytics endpoint metadata sync option (preview)	You can now opt in to the new SQL analytics endpoint metadata sync option, a preview feature. The new option, which is currently opt-in only and currently applies only to new SQL analytics endpoints, is faster, provides a new manual metadata sync option, and provides time travel querying.
SQL database data virtualization (Preview)	Data virtualization (preview) in SQL database enables querying external data stored in OneLake using T-SQL. With data virtualization syntax, you can execute Transact-SQL (T-SQL) queries on files that store data in common data formats in OneLake. You can combine this data with locally stored relational data by using joins.
SQL database skills (Preview)	SQL database skills (Preview) give AI coding agents workload-specific guidance for authoring schemas, querying data, and operating SQL databases as part of broader Fabric solutions. For more information, see Work with SQL database skills in Microsoft Fabric.
SQL database support for private links (Preview)	You can use tenant and workspace-level private links to provide secure access for data traffic in Microsoft Fabric, including SQL database. For SQL database, workspace-level private links are in preview. For more information, see Set up and use private links.
Stream Mirrored Database change feeds into Eventstreams (Preview)	Mirrored Database change feed connector for Eventstreams (Preview) streams Delta Change Data Feed (CDF) row-level inserts, updates, and deletes from a mirrored database directly into a Fabric Eventstream for low-latency,

Feature	Learn more
	event-driven processing, with full source schema and change metadata fidelity and support for Azure SQL, Cosmos DB, Oracle, PostgreSQL, Snowflake, and Open Mirroring partners. For more information, see Extended capabilities in mirroring for Fabric.
Time Travel using SQL analytics endpoint (Preview)	Time Travel using SQL analytics endpoint (Preview) extends the Fabric Data Warehouse time travel experience to the SQL analytics endpoint so you can query Lakehouse data exactly as it looked at any prior point in time by adding `OPTION (FOR TIMESTAMP AS OF '...')` once at the T-SQL statement level.
Update policies on accelerated shortcuts (Preview)	Update policies on accelerated shortcuts (Preview) apply ingestion-time transformations to query-accelerated OneLake shortcuts. For more information, see Update policies on accelerated shortcuts.
Upgrade Dataflow Gen1 to Dataflow Gen2 with the Upgrade Wizard (Preview)	The Dataflows Upgrade Wizard upgrades one or more Dataflow Gen1 items in place while retaining their IDs, names, schedules, and connections. For more information, see Upgrade Dataflow Gen1 to Dataflow Gen2 using the Upgrade Wizard.
Upsert to delta table with Lakehouse connector (Preview)	We've added upsert support to the Lakehouse connector, allowing direct writes to Delta tables, in both Copy job and Copy activity within Pipeline. For more information, see Configure Lakehouse in a copy activity.
Warehouse AI functions (preview)	Fabric Data Warehouse now brings AI directly into T-SQL, enabling you to categorize and classify text, analyze sentiment, extract structured information, translate text across languages, and even correct grammar - all using built-in AI functions. To get started, see Use AI functions in Fabric Data Warehouse (preview).
Warehouse analytical functions (Preview)	Warehouse analytical functions (Preview) add built-in statistical calculations to Fabric Data Warehouse and the SQL analytics endpoint. For more information, see Aggregate functions.
Warehouse CI/CD 2.0 (Preview)	Warehouse item definition version 2.0 uses DacFx-based incremental extraction for Git integration and more accurate comparisons and deployment plans in deployment pipelines. For more information, see Source control with Warehouse and Upgrade Fabric Data Warehouse system files.
Warehouse custom SQL pools (Preview)	Using custom SQL pools to balance performance and cost explains how workload routing and pool allocation affect performance, concurrency, and capacity consumption. For more information, see Configure custom SQL pools.
Warehouse data clustering (Preview)	Data clustering is a technique used to organize and store data based on similarity. Data clustering improves query performance and reduces compute and storage access costs for queries by grouping similar records together. For more information and to get started, see Data clustering in Fabric Data Warehouse and Use data clustering in Fabric Data Warehouse (Preview).

Feature	Learn more
Warehouse pre/post deployment support (Preview)	Warehouse pre/post deployment support (Preview) runs validation and setup scripts around automated warehouse deployments.
Warehouse source control (Preview)	Using Source control with Warehouse (preview), you can manage development and deployment of versioned warehouse objects. You can use SQL Database Projects extension available inside of Visual Studio Code. For more information on warehouse source control, see CI/CD with Warehouses in Microsoft Fabric.
Warehouse T-SQL enhancements (Preview)	Warehouse T-SQL enhancements (Preview) add constructs such as `QUALIFY` and expanded grouping syntax. For more information, see QUALIFY.
Warehouse User Data Function integration (Preview)	Warehouse User Data Function integration (Preview) invokes custom Python logic from T-SQL external functions. For more information, see Fabric User Data Functions in Warehouse.
Workspace-level surge protection (Preview)	Workspace-level surge protection lets capacity admins cap a workspace's compute consumption, automatically or manually block excessive consumers, exempt mission-critical workspaces, and restore blocked workspaces. For more information, see Manage surge protection for Fabric capacities.
Workspace monitoring (Preview)	Workspace monitoring uses a monitoring item to collect logs and metrics from supported Fabric items. Centralize telemetry from multiple workspaces, query it with KQL, build dashboards and reports, create alerts, and use Operations Agent to investigate issues.
Workspace outbound access protection for Azure and Fabric events (Preview)	Workspace outbound access protection (OAP) for Azure and Fabric events (Preview) extends outbound network controls to cross-workspace event consumption. For more information, see Tenant private links for Azure and Fabric events.
Workspace outbound access protection for Fabric Maps (Preview)	Workspace outbound access protection for Fabric Maps (Preview) allows only approved map data sources and external geospatial services through workspace data connection rules. For more information, see Workspace outbound access protection for Fabric Maps.
Workspace outbound access protection for Operations Agent (Preview)	Workspace outbound access protection for Operations Agent applies workspace policies to outbound actions such as Teams notifications, workflow triggers, and cross-workspace operations. For more information, see Workspace outbound access protection for Operations Agent.
Workspace outbound access protection for Real-Time Intelligence (Preview)	Workspace outbound access protection (OAP) for Real-Time Intelligence (Preview) extends workspace-level outbound network controls to Eventstream, Eventhouse, KQL QuerySet, Activator, Real-Time Dashboards, and Azure and Fabric events, so admins block unwanted outbound connections by default and allow only approved destinations through data connection rules. For more information, see Workspace outbound access protection.

Feature	Learn more
Zero-copy access to OneLake data in Azure Databricks (Preview)	OneLake catalog federation in Azure Databricks (Preview) enables Unity Catalog to query Fabric data stored in OneLake without copying, keeping OneLake as the source of truth while allowing Databricks compute to analyze tables through synchronized metadata and zero-copy data access. For more information, see Enable OneLake catalog federation.

Generally available features

The following table lists the features of Microsoft Fabric that recently transitioned to general availability (GA).

Expand table

Month	Feature	Learn more
September 2026	**Fabric IQ in Microsoft 365 Copilot (Generally Available)**	Fabric IQ in Microsoft 365 Copilot Chat and Copilot Cowork answers business questions with governed Power BI reports and semantic models. For more information, see Fabric IQ in Microsoft 365 Copilot.
September 2026	**SQL CDC custom capture names (Generally Available)**	SQL CDC custom capture names let Copy job use existing CDC tables that don't follow the default naming convention. For more information, see Change data capture in Copy job.
September 2026	**Copy job workspace monitoring (Generally Available)**	Copy job workspace monitoring centralizes execution status, throughput, and health trends. For more information, see Monitor a Copy job.
September 2026	**Custom Copy job staging (Generally Available)**	Custom Copy job staging selects the staging database and schema for CDC and SCD Type 2 scenarios. For more information, see Change data capture in Copy job.
September 2026	**Snowflake read-only CDC (Generally Available)**	Snowflake read-only CDC lets Copy job ingest changes from read-only Snowflake databases. For more information, see Snowflake CDC in Copy job.
September 2026	**Oracle CDC initial-load improvements (Generally Available)**	Oracle CDC initial-load improvements improve onboarding performance and scale across multiple source tables.
September 2026	**PostgreSQL connector for Copy job (Generally Available)**	Azure Database for PostgreSQL connector supports full and watermark-based incremental copy. For more information, see Copy job connectors.
September 2026	**Multi-folder Copy job (Generally Available)**	Multi-folder Copy job loads multiple source folders into separate destination tables. For more information, see Copy job connectors.

Month	Feature	Learn more
September 2026	**SharePoint folders in Copy job (Generally Available)**	SharePoint folders in Copy job use SharePoint folders as sources or destinations. For more information, see Copy job connectors.
September 2026	**Optimized Dataflow copy to lakehouse (Generally Available)**	Optimized Dataflow copy to lakehouse reduces serialization overhead when Dataflow Gen2 loads staged data into lakehouse tables. For more information, see Staged data options.
September 2026	**My Queries in Dataflow Gen2 (Generally Available)**	My Queries saves personal Power Query logic for reuse across dataflows. For more information, see My Queries and Shared Queries.
September 2026	**New Dataflow destinations (Generally Available)**	New Dataflow Gen2 destinations write prepared data to Excel workbooks and Snowflake. For more information, see Excel workbook destinations.
September 2026	**Dataflow diagnostics download (Generally Available)**	Dataflow diagnostics download provides structured run logs for troubleshooting and support. For more information, see Monitor Dataflow Gen2.
September 2026	**Connection authentication allowlist (Generally Available)**	Connection authentication allowlist centrally restricts allowed authentication methods. For more information, see Authentication allowlist.
September 2026	**Connection tenant allowlist (Generally Available)**	Connection tenant allowlist limits cloud connections to trusted Microsoft Entra tenants. For more information, see Tenant allowlist.
September 2026	**Cloud connection Admin APIs (Generally Available)**	Cloud connection Admin APIs inventory tenant connections, identify orphaned connections, and support governance automation. For more information, see Connection Admin APIs.
September 2026	**Atlan connector for Fabric (Generally Available)**	Atlan connector for Fabric adds business context and deeper lineage across Fabric items.
September 2026	**ArcGIS Maps for Fabric (Generally Available)**	ArcGIS Maps for Microsoft Fabric adds spatial analysis, stream layers, and OneLake mapping to Fabric.
September 2026	**Telmai Data Reliability (Generally Available)**	Telmai Data Reliability for Fabric monitors OneLake tables for volume, schema, freshness, and completeness changes.
September 2026	**Celonis Process Intelligence (Generally Available)**	Celonis Process Intelligence brings process flows and root-cause analysis into Fabric.
September 2026	**Spectral Core Fabric Migration (Generally Available)**	Spectral Core Fabric Migration guides database movement, SQL translation, schema conversion, and validation.

Month	Feature	Learn more
September 2026	**Financial Fabric DataHub (Generally Available)**	Financial Fabric DataHub brings capital-markets datasets and conversational exploration into Fabric.
September 2026	**IntuigenceAI for Fabric (Generally Available)**	IntuigenceAI for Fabric turns industrial documents and logs into a queryable knowledge graph in OneLake.
September 2026	**Capacity Metrics app enhancements (Generally Available)**	Capacity Metrics app enhancements add health-state duration, capacity-units-per-second views, heatmaps, contributor drilldowns, and chargeback integration. For more information, see Capacity Metrics app.
September 2026	**OneLake catalog Govern experience (Generally Available)**	OneLake catalog Govern experience centralizes governance for tenants, domains, capacities, workspaces, tags, and policies. For more information, see Govern your data estate.
September 2026	**Materialized lake views (Generally Available)**	Materialized lake views provide reusable, managed lakehouse transformations for analytics.
September 2026	**Capacity alert templates (Generally Available)**	Capacity alert templates create common capacity-health alerts from ready-made patterns.
September 2026	**Event Schema Registry (Generally Available)**	Event Schema Registry defines and validates contracts for reliable event producers and consumers. For more information, see Schema Registry overview.
September 2026	**New Eventstream connectors (Generally Available)**	New Eventstream connectors add Cribl, Solace PubSub+, SAP Datasphere, MongoDB CDC, and Mirrored Database Change Feed sources. For more information, see Add a Cribl source and Add a Solace PubSub+ source.
September 2026	**Activator query visibility (Generally Available)**	KQL and SQL query visibility in Activator shows the queries behind supported alerts and actions. For more information, see Create Activator alerts from a KQL Queryset.
September 2026	**Activator Fabric-item actions (Generally Available)**	Activator Fabric-item actions start Copy jobs, Dataflows, and User Data Functions from Activator rules. For more information, see Trigger Fabric items.
September 2026	**Activator stateful conditions (Generally Available)**	Activator stateful conditions evaluate changes over time instead of treating each event independently. For more information, see Activator rules overview.
September 2026	**Granular shortcut-database sharing (Generally Available)**	Granular shortcut-database sharing shares selected Eventhouse data without exposing an entire database. For more information, see OneLake shortcuts.

Month	Feature	Learn more
September 2026	**Eventhouse entity diagram (Generally Available)**	Eventhouse entity diagrams visualize KQL database entities and relationships. For more information, see View an entity diagram.
September 2026	**Eventhouse outbound access protection (Generally Available)**	Eventhouse outbound access protection blocks outbound traffic by default and permits approved destinations through data connection rules. For more information, see Workspace outbound access protection.
September 2026	**Eventhouse workspace Private Link (Generally Available)**	Eventhouse workspace Private Link secures supported ingestion, querying, monitoring, and reuse within private-link-enabled workspaces. For more information, see Tenant and workspace Private Links.
September 2026	**Eventhouse customer-managed keys (Generally Available)**	Eventhouse customer-managed keys encrypt dedicated physical clusters with organization-controlled Azure Key Vault keys. For more information, see Eventhouse encryption.
September 2026	**Fabric IQ MCP Server (Generally Available)**	Fabric IQ MCP Server brings governed Fabric business context to compatible AI clients. For more information, see Fabric IQ MCP Server.
September 2026	**Fabric data agent visualizations (Generally Available)**	Fabric data agent visualizations return interactive charts with text and tables. For more information, see Get visual responses from a data agent.
September 2026	**Data-agent MCP tasks (Generally Available)**	Data-agent MCP tasks support long-running Model Context Protocol operations through a published data-agent endpoint. For more information, see Data agent as an MCP server.
September 2026	**Lakehouse maintenance activity (Generally Available)**	Lakehouse maintenance activity runs `VACUUM` and `OPTIMIZE` operations as orchestrated pipeline steps. For more information, see Lakehouse maintenance activity.
September 2026	**Pipeline retry backoff (Generally Available)**	Pipeline retry backoff progressively increases the interval between activity retry attempts. For more information, see Activity retries.
September 2026	**Extended mirroring capabilities (Generally Available)**	Extended mirroring capabilities add change feeds and source-view mirroring for advanced replication scenarios. For more information, see Extended capabilities in Mirroring.
September 2026	**Copy job CDC (Generally Available)**	Change data capture in Copy job incrementally processes inserted, updated, and deleted source records. For more information, see Change data capture in Copy job.
September 2026	**Copy job SCD Type 2 (Generally Available)**	SCD Type 2 in Copy job preserves historical record versions for supported CDC scenarios. For more information, see Change data capture in Copy job.

Month	Feature	Learn more
September 2026	**Copy job audit columns (Generally Available)**	Copy job audit columns add operational metadata to destination rows, including Snowflake destinations. For more information, see Audit columns in Copy job.
September 2026	**DLP restrict access (Generally Available)**	DLP restrict access (Generally Available) automatically limits access when Microsoft Purview policies detect sensitive information. For more information, see Restrict access action for DLP policies.
September 2026	**Fabric Core MCP Server (Generally Available)**	Fabric Core MCP Server gives compatible AI agents authenticated, audited access to Fabric workspaces, items, permissions, search, and capacity information. For more information, see Get started with Fabric Core MCP Server.
September 2026	**Branch workspaces and selective branching (Generally Available)**	Branch workspaces and selective branching create governed, isolated workspaces with only the items developers need. For more information, see Development process using branch workspaces.
September 2026	**Compare and commit Git changes (Generally Available)**	Compare and commit Git changes provides scoped diffs, conflict resolution, and item commits. For more information, see Compare and commit Fabric items.
September 2026	**Bulk item definition APIs (Generally Available)**	Bulk item definition APIs export or import multiple Fabric item definitions in one operation. For more information, see Items API reference.
September 2026	**OneLake outbound access protection (Generally Available)**	OneLake outbound access protection blocks external shortcuts and copy operations by default and allows approved connections. For more information, see Manage outbound access.
September 2026	**OneLake security Members and Data UX (Generally Available)**	OneLake security Members and Data experiences show role assignments and the data each role can access. For more information, see Create and manage OneLake security roles.
September 2026	**Custom Live Pools (Generally Available)**	Custom Live Pools keep scheduled Spark clusters ready with custom libraries for predictable notebook startup. For more information, see Custom Live Pools overview.
September 2026	**Spark connector for SQL databases (Generally Available)**	Spark connector for SQL databases supports production reads and bulk writes across supported SQL platforms. For more information, see Spark connector for SQL databases.
September 2026	**Fabric Livy API high concurrency (Generally Available)**	Fabric Livy API high concurrency runs multiple isolated Spark workloads in shared sessions. For more information, see High concurrency in the Fabric Livy API.
September 2026	**ADO.NET driver high concurrency (Generally Available)**	ADO.NET driver high concurrency runs concurrent Spark SQL queries through shared sessions. For more information, see ADO.NET driver high concurrency.

Month	Feature	Learn more
September 2026	**JDBC driver high concurrency (Generally Available)**	JDBC driver high concurrency runs concurrent Spark SQL workloads from Java applications. For more information, see JDBC driver for Fabric Data Engineering.
September 2026	**ODBC driver high concurrency (Generally Available)**	ODBC driver high concurrency runs concurrent Spark SQL workloads on Windows and Linux. For more information, see ODBC driver for Fabric Data Engineering.
September 2026	**Lakehouse Explorer resizable columns (Generally Available)**	Resizable Lakehouse Explorer columns adjust table-preview widths for long names and values. For more information, see Navigate the Lakehouse explorer.
September 2026	**SQL pool statement classifier (Generally Available)**	SQL pool statement classification automatically separates data-loading and query statements into workload pools. For more information, see Custom SQL pools.
September 2026	**Distributed bitmap filters (Generally Available)**	Distributed bitmap filters reduce data movement for supported fact-to-dimension joins in Fabric Data Warehouse.
September 2026	**Business Events (Generally Available)**	Business Events publish governed business signals for independent operational and analytical consumers, and a default eventhouse automatically retains every published event. For more information, see Business events overview.
September 2026	**Richer Real-Time Dashboard visuals (Generally Available)**	Richer Real-Time Dashboard visuals add responsive KPI modes, reference lines, titles, text, and dividers. For more information, see Customize Real-Time Dashboard visuals.
September 2026	**Guided streaming-data monitoring (Generally Available)**	Guided streaming-data monitoring walks you through building no-code monitoring in Real-Time Hub. For more information, see Get started with Real-Time Hub.
September 2026	**Warehouse operations skill sqldw-cli (Generally Available)**	The Warehouse operations skill sqldw-cli runs bounded, read-only diagnostics for warehouse and SQL analytics endpoint issues and returns evidence, recommended actions, and validation steps. For more information, see Diagnose warehouse workloads with Warehouse operations skill sqldw-cli.
September 2026	**Capacity overage (Generally Available)**	Capacity overage is now generally available for Fabric F SKUs and is enabled by default when you create a capacity. It pays for eligible excess capacity usage to help prevent throttling, up to a configurable rolling 24-hour threshold. For more information, see Capacity overage in Microsoft Fabric and Enable capacity overage in Microsoft Fabric.
September 2026	**Vector indexes and vector search in SQL**	Vector indexes and vector search in SQL database in Fabric are now generally available. For more information, see CREATE VECTOR INDEX and Vector indexes and vector search general availability.

Month	Feature	Learn more
	database in Fabric (Generally Available)	
September 2026	**SharePoint List mirroring (Generally available)**	SharePoint List mirroring is now generally available. You can continuously replicate SharePoint list data into OneLake and use it across Fabric workloads. For more information, see SharePoint List mirroring.
September 2026	**dbt job in Fabric Data Factory (Generally Available)**	dbt job in Fabric Data Factory lets you author, schedule, and monitor dbt projects natively with serverless execution, integrated testing and documentation, and governance through Microsoft Entra ID and SQL security policies. For more information, see dbt job in Microsoft Fabric.
September 2026	**Result set caching**	Result set caching works by persisting the final result sets for applicable `SELECT` T-SQL queries, bypassing complex compilation and data processing of the original query to return queries faster. To learn more, review the preview announcement at Result Set Caching for Microsoft Fabric Data Warehouse. Result set caching is enabled by default.
September 2026	**SQL query editor enhancements (Generally Available)**	SQL query editor enhancements add a faster results grid, scalable object explorer and IntelliSense, autosave controls, bulk query management, `.sql` file import and export, and integrations with OneLake analytics and semantic models. For more information, see SQL query editor.
September 2026	**Fabric networking communication policies Admin API (Generally Available)**	The List Networking Communication Policies Admin API provides a tenant-wide, paginated view of workspace inbound and outbound networking policies. For more information, see List Networking Communication Policies.
September 2026	**ODBC to ADBC driver transition (Generally Available)**	ADBC drivers for supported Power BI and Fabric connectors are available for validation before they become the default and embedded ODBC drivers are disabled. For more information, see Transition from ODBC to ADBC drivers.
August 2026	**Activator rules in Eventstream (Generally Available)**	Activator rules in Eventstream let you create, edit, delete, and open alert rules without leaving Eventstream. For more information, see Add a Fabric Activator destination to an eventstream.
August 2026	**COPY INTO with workspace identity (Generally Available)**	COPY INTO with workspace identity lets Fabric Data Warehouse load approved data from OneLake or Azure Data Lake Storage Gen2 by using the workspace identity. For more information, see Ingest data into your warehouse using the COPY statement.
August 2026	**Data-driven styling and map improvements (Generally Available)**	Data-driven styling and map improvements add value-based colors, data legends, marker rotation, traffic flow overlays, map

Month	Feature	Learn more
		view options, and improved layer settings to Fabric Maps. For more information, see Configure layer settings in Fabric Maps.
August 2026	**Eventstream MQTT connector (Generally Available)**	The Eventstream MQTT connector connects MQTT brokers to Eventstream with TLS, mutual TLS, custom certificate authorities, and private network connectivity. For more information, see Add an MQTT source to an eventstream.
August 2026	**Fabric Data Agent integration with Microsoft Copilot Studio (Generally Available)**	Fabric Data Agent integration with Microsoft Copilot Studio adds a Fabric data agent to a Copilot Studio agent as a Fabric IQ Data MCP tool while retaining source permissions.
August 2026	**Metadata Sync support for Delta Checkpoint V2 (Generally Available)**	Metadata Sync support for Delta Checkpoint V2 lets both legacy and new Metadata Sync discover and synchronize Delta tables that use the Checkpoint V2 format. For more information, see SQL analytics endpoint metadata synchronization.
August 2026	**Set alerts from Anomaly Detector (Generally Available)**	Set alerts directly from Anomaly Detector lets you create an alert from a published anomaly detector configuration without switching to Real-Time Hub. For more information, see Anomaly detection in Real-Time Intelligence.
August 2026	**Mirroring for Google BigQuery (Generally Available)**	Mirroring in Fabric offers a simple, low-latency way to continuously replicate your Google BigQuery data into OneLake with no ETL pipelines, no data movement, and no proprietary formats. Once in Fabric, mirrored BigQuery data is queryable in near real time across Power BI Direct Lake, the SQL analytics endpoint, Data Engineering, Data Science, Real-Time Intelligence, and Copilot. For more information, see Mirrored databases from Google BigQuery and Tutorial: Set up mirroring for Google BigQuery.
August 2026	**Migration Assistant to SQL database (Generally Available)**	Migration Assistant to SQL database is generally available and lets you validate a source DACPAC for compatibility before provisioning a SQL database in Fabric. For more information, see Fabric Migration Assistant for SQL database.
August 2026	**Warehouse IDENTITY columns (Generally Available)**	IDENTITY columns are now generally available in Fabric Data Warehouse. You can use `IDENTITY_INSERT` and reseed operations for migration and controlled data loads. For more information, see IDENTITY columns and Use IDENTITY columns to create surrogate keys.
August 2026	**Real-Time Dashboard KPI visualization (Generally Available)**	The KPI visualization in Real-Time Dashboards is now generally available, providing at-a-glance metrics with goals, status indicators, trends, and conditional formatting. For more information, see Customize Real-Time Dashboard visuals and Apply conditional formatting in Real-Time Dashboards.

Month	Feature	Learn more
August 2026	**PowerTable billing personas (Generally Available)**	Billing personas for PowerTable in Fabric Planning are generally available, with role-based billing for Planners, Stakeholders, and Viewers. For more information, see Billing and usage for planning in Fabric.
August 2026	**Workspace customer-managed key APIs (Generally Available)**	Workspace customer-managed key APIs are generally available, so administrators can automate workspace encryption-key assignment and removal. For more information, see Customer-managed keys for Fabric workspaces and Assign workspace encryption.
August 2026	**Data source routing in Fabric data agents (Generally Available)**	Data source routing in Fabric data agents selects the most relevant lakehouse, warehouse, semantic model, or KQL database for each question by using schema metadata, source descriptions, example queries, and routing rules. For more information, see Configure a Fabric data agent and Provide example queries.
August 2026	**Capacity Overview Events in Real-Time Hub (Generally Available)**	Capacity Overview Events in Real-Time Hub (Generally Available) stream capacity summary and state signals so you can monitor utilization and health, detect throttling or lifecycle changes, and trigger alerts or automated actions. For more information, see Introduction to Azure and Fabric events and Explore Fabric capacity overview events.
August 2026	**Fabric Runtime 2.0 (Generally Available)**	Fabric Runtime 2.0 (Generally Available) includes Apache Spark 4.1, Delta Lake 4.2, Python 3.13, Java 21, Scala 2.13, and Azure Linux 3.0, and remains opt-in until its planned default rollout. For more information, see Runtime 2.0 in Fabric.
July 2026	**Custom CA and mTLS in Eventstream connectors (Generally Available)**	Custom CA and mTLS support in Eventstream streaming connectors lets you use custom CA and client certificates from Azure Key Vault with the MQTT, Apache Kafka, AWS MSK, and Confluent Cloud for Apache Kafka source connectors. For more information, see Add and manage eventstream sources.
July 2026	**Native execution engine UDFs and complex types (Generally Available)**	In Fabric Data Engineering, native execution engine support for Python UDFs, Scala UDFs, and complex data types accelerates Spark processing with no changes to existing notebooks or jobs. For more information, see Native execution engine UDF and complex type support.
July 2026	**Materialized lake views analytics and insights (Generally Available)**	Materialized lake views analytics and insights show refresh trends, failure patterns, and prioritized recommendations so you can improve materialized lake view performance and cost. For more information, see View insights for materialized lake view runs.

Month	Feature	Learn more
July 2026	**Usage-based resource estimations (Generally Available)**	Usage-based resource estimations help the Fabric Data Warehouse query optimizer use prior execution cardinalities to correct under- and over-estimates for more efficient resource requests and concurrency.
July 2026	**Folders in Eventhouse tree (Generally Available)**	Folders in Eventhouse tree let you create, rename, delete, and move folders in the Eventhouse UI to organize tables, shortcuts, materialized views, functions, and data streams. For more information, see Manage and monitor a KQL database table.
July 2026	**Eventhouse update policies with accelerated shortcuts (Generally Available)**	Eventhouse update policies with accelerated shortcuts let update policy queries join to accelerated shortcut external tables for governed ingestion-time enrichment.
July 2026	**Fabric Maps tilesets (Generally Available)**	Fabric Maps tilesets create PMTiles from OneLake geospatial data so large static spatial datasets render as fast, interactive maps in Fabric. For more information, see Create tilesets.
July 2026	**Real-Time Dashboard (Generally Available)**	Real-Time Dashboard (Generally Available) lets you monitor Eventhouse data with live refresh, Copilot-created tiles, drill-down exploration, Activator alerts, and team sharing. For more information, see What is Real-Time Dashboard? and Create a Real-Time Dashboard.
July 2026	**Spark job monitoring (Generally Available)**	Job concurrency and queue monitoring (Generally Available) adds a capacity-level view so you can compare Spark resource use across workspaces, identify contention, and tune limits. For more information, see Job concurrency and queue monitoring and Concurrency limits and queueing in Apache Spark for Fabric.
July 2026	**Resource Instance Rules for OneLake Network Security for Azure Services (Generally Available)**	Resource Instance Rules for OneLake let you allow specific Azure resource instances to access OneLake over the public endpoint while Fabric still evaluates data permissions. For more information, see Resource Instance rules for Inbound access.
July 2026	**Lakehouse Query Explorer (Generally Available)**	Lakehouse Query Explorer lets you run Spark SQL in the Lakehouse explorer to inspect data, test joins, create views, and iterate before you move logic to notebooks or SQL analytics endpoints. For more information, see Query data with the Lakehouse query explorer.
July 2026	**OneLake item-size reporting (Generally Available)**	OneLake item-size reporting gives you a workspace storage report that breaks down item data, system data, soft-deleted data, and billing status in the Fabric portal. For more information, see Get the size of OneLake items.

Month	Feature	Learn more
July 2026	**OneLake storage tiers and lifecycle management (Generally Available)**	OneLake storage tiers and lifecycle management let you keep active data in the hot tier and move less-used data to cool or cold tiers with lifecycle policies. For more information, see Get the size of OneLake items and OneLake diagnostics.
July 2026	**Plan (Generally Available)**	Plan in Fabric IQ (Generally Available) is a unified no-code platform for collaborative planning, reporting, analytics, data integration, and management. For more information, see What is plan?
July 2026	**Dropped warehouse recovery (Generally Available)**	Dropped warehouse recovery lets Workspace Administrators restore accidentally dropped warehouses from the Workspace Recycle Bin within a configurable retention window (7 to 90 days), recovering table schemas, data, snapshots, permissions, views, and stored procedures. For more information, see Manage workspaces.

Community

This section summarizes new Microsoft Fabric community opportunities for prospective and current influencers and MVPs.

- Sign up for the Fabric Community Newsletter: visit Fabric Community News and select **Subscribe** from the **Options** menu.
- Ask and answer questions in the Microsoft Fabric community.
- Join a local Fabric User Group or join a local event.
- The Microsoft Fabric Career Hub has everything you need on your certification journey.
- Vote for your favorite new product feature ideas at Microsoft Fabric Ideas.
- To learn about the Microsoft MVP Award and to find MVPs, see mvp.microsoft.com.
- Are you a student? Learn more about the Microsoft Learn Student Ambassadors program.
- Watch and subscribe to Microsoft Fabric videos on YouTube.
- Join the Microsoft Fabric user panel to share real-world experiences and feedback with the Fabric and Power BI product teams through surveys and 1:1 meetings.
- Spread your Fabric knowledge, insights, and best practices with others. To learn more, see the Super User Program.

Power BI

> **Important**

To access Power BI, use a web browser that's Chrome 94 or newer, Microsoft Edge 94 or newer, Safari 16.4 or newer, Firefox 93 or newer, or an equivalent version. If you use an older version, upgrade your web browser to a newer version by August 31, 2024. If you use an outdated browser version after this date, you might be prevented from accessing features in Power BI.

For updates to Power BI Desktop and the Power BI service, see What's new in Power BI?

Stay up to date

To stay up to date with the latest news and features, see the Microsoft Fabric Updates Blog.

For Microsoft Fabric release plans, see the Microsoft Fabric Roadmap.

Related content

- Modernization Best Practices and Reusable Assets Blog
- Microsoft Fabric migration overview
- Fabric Known Issues
- Microsoft Fabric Blog
- Fabric terminology
- What's new in Power BI?
- Microsoft Fabric videos on YouTube
- Microsoft Fabric community

ⓘ**Note:** The author created this article with assistance from AI. Learn more

Last updated on 10/02/2026

Fabric terminology

Learn the definitions of terms used in Fabric, including terms specific to each Fabric workload.

General terms

The following terms apply across all Fabric workloads.

Capacity

A *capacity* is a dedicated set of resources that you can use at a given time. Capacity defines the ability of a resource to perform an activity or to produce output. Different items consume different capacity at a certain time. Fabric offers capacity through the Fabric SKU and Trials. For more information, see What is capacity?

Capacity units (CUs)

Capacity units are the unit of compute measurement in Fabric. All workloads and operations consume capacity units from your available Fabric capacity. Consumption is classified as interactive or background operations. For more information, see Fabric operations.

Delta Lake

A Delta Lake is the standard table format across all Fabric workloads. When you ingest data into Fabric, it's stored as Delta tables by default. For more information, see Delta Lake table format interoperability.

Direct Lake

Direct Lake is a storage mode in Fabric that lets semantic models read Delta tables directly from OneLake without importing data or using DirectQuery. Direct Lake combines the performance of import mode with the data freshness of DirectQuery. For more information, see Direct Lake overview.

Domain

A domain lets your organization group workspaces into logical business areas, such as "Finance" or "Marketing." Domains help Fabric administrators delegate management and apply governance policies above the workspace level. For more information, see Domains.

Item

An *item* is an object in Fabric, such as a lakehouse, notebook, warehouse, or eventhouse. Each Fabric workload includes different item types. For example, the Data Engineering workload includes the lakehouse, notebook, and Spark job definition items.

Semantic model

A semantic model is a metadata layer in Fabric that defines tables, relationships, measures, and data connections. Semantic models are the data source for Power BI reports, dashboards, and other analytics experiences in Fabric. For more information, see Create and manage semantic models.

Tenant

A *tenant* is a single instance of Fabric for an organization, aligned with a Microsoft Entra tenant.

Workload

A *workload* is a collection of capabilities targeted to a specific functionality. The Fabric workloads include Data Engineering, Data Factory, Data Science, Data Warehouse, Databases, Industry Solutions, Real-Time Intelligence, Fabric IQ, and Power BI.

Workspace

A *workspace* is a collection of items that brings together different functionality in a single environment designed for collaboration. It acts as a container that uses capacity for the work that is executed, and provides controls for who can access the items in it. For example, in a workspace, users create reports, notebooks, and semantic models. For more information, see Workspaces.

Data Engineering

The Data Engineering workload provides tools for large-scale data processing and transformation by using Apache Spark.

Apache Spark job

An *Apache Spark job* is part of a Spark application that runs in parallel with other jobs in the application. A job consists of multiple tasks. For more information, see Spark job monitoring.

Apache Spark job definition

An *Apache Spark job definition* is a set of parameters that indicates how a Spark application should be run. It allows you to submit batch or streaming jobs to the Spark cluster. For more information, see What is an Apache Spark job definition?

Lakehouse

A *lakehouse* is a database built over a data lake, containing files, folders, and tables. A lakehouse is used by the Apache Spark engine and SQL engine for big data processing. Lakehouses support ACID transactions when using the open-source Delta formatted tables. The lakehouse item is hosted within a unique workspace folder in Microsoft OneLake. It contains files in various formats (structured and unstructured) organized in folders and subfolders. For more information, see What is a lakehouse?

Notebook

A *notebook* is a multi-language interactive programming tool that supports authoring code and markdown, running and monitoring Spark jobs, viewing results, and collaborating with team members. You can use notebooks to explore and process data and to build machine learning experiments. A notebook can be transformed into a pipeline activity for orchestration.

Spark application

A *Spark application* is a program written by a user using one of Spark's API languages (Scala, Python, Spark SQL, or Java) or Microsoft-added languages (.NET with C# or F#). When an application runs, it's divided into one or more Spark jobs that run in parallel to process the data faster. For more information, see Spark application monitoring.

V-order

V-order is a write optimization to the parquet file format that enables fast reads. All Fabric engines write v-ordered parquet files by default.

Data Factory

The Data Factory workload provides data integration and orchestration capabilities, including pipelines, dataflows, and connectors for moving and transforming data.

Connector

A *connector* is a component in Data Factory that you use to connect to different types of data stores. After you connect, you can transform the data. For more information, see Connectors.

Dataflow Gen2

A *Dataflow Gen2* is a low-code interface for ingesting data from hundreds of data sources and transforming your data. Dataflow Gen1 exists in Power BI. Dataflow Gen2 offers extra capabilities compared to Dataflows in Azure Data Factory or Power BI. You can't upgrade from Gen1 to Gen2. For more information, see Dataflows in the Data Factory overview.

Pipeline

A *pipeline* is an item in Data Factory used for orchestrating data movement and transformation. These pipelines are different from the deployment pipelines in Fabric. For more information, see Pipelines.

Trigger

A *trigger* is an automation capability in Data Factory that initiates pipelines based on specific conditions, such as schedules or data availability.

Data Science

The Data Science workload provides tools for building, training, and deploying machine learning models within the Fabric platform.

Data Wrangler

Data Wrangler is a notebook-based tool for exploratory data analysis. It combines a grid-like data display with dynamic summary statistics and a set of common data-cleansing operations. Each operation generates code that you can save back to the notebook as a reusable script.

Experiment

An *experiment* is the primary unit of organization and control for all related machine learning runs. For more information, see Machine learning experiments in Fabric.

Model

A *model* is a machine learning file trained to recognize certain types of patterns. You train a model over a set of data, and you provide it with an algorithm that it uses to reason over and learn from that data set. For more information, see Machine learning model.

Run

A *run* corresponds to a single execution of model code. In MLflow, tracking is based on experiments and runs.

Data Warehouse

The Data Warehouse workload provides enterprise data warehousing with full T-SQL capabilities for structured data storage and analysis.

SQL analytics endpoint

A *SQL analytics endpoint* is a feature that you use to run T-SQL queries against your data for analysis and insights. SQL analytics endpoints are available for lakehouses, mirrored databases, and SQL databases in Fabric. For more information, see SQL analytics endpoint.

Warehouse

A *warehouse* is an item that functions as a traditional data warehouse and supports the full transactional T-SQL capabilities you expect from an enterprise data warehouse. For more information, see Data Warehouse.

Databases

The Databases workload provides transactional database capabilities within Fabric, including SQL database in Fabric and Cosmos DB in Fabric.

Cosmos DB in Fabric

Cosmos DB in Fabric is a NoSQL database in Fabric based on Azure Cosmos DB for building applications that require low-latency data access. For more information, see Cosmos DB in Fabric.

SQL database in Fabric

A *SQL database in Fabric* is a transactional database based on Azure SQL Database that you use to create your operational database in Fabric. For more information, see SQL database in Fabric.

Fabric IQ

Fabric IQ is part of Microsoft IQ, a set of capabilities that form the enterprise intelligence layer of the Microsoft stack. In Microsoft IQ, Fabric IQ works alongside Work IQ and Foundry IQ to provide context for a complete view of your organization. The Fabric IQ piece provides context on business entities and data.

As workload in Fabric, IQ (preview) groups items related to unifying business semantics across data, models, and systems to power intelligent agents and decisions.

For more information about Fabric IQ, see Fabric IQ overview.

Ontology

An *ontology* is an item where you can define entity types, relationships, properties, and other constraints to organize data according to your business vocabulary. For more information, see What is ontology (preview)?

Plan

A *plan* is a unified no-code platform for collaborative planning, reporting, analytics, data integration, and management. For more information, see What is plan (preview)?

OneLake

OneLake is Fabric's unified, multicloud data lake that serves as the single storage layer for all Fabric workloads. For more information, see What is OneLake?

Data sharing (cross-tenant)

Data sharing is a OneLake feature that you can use to share live, governed datasets across Microsoft Entra tenants without copying data.

Mirroring

Mirroring is the process of copying data from an external source into Fabric to create a mirrored database or catalog. For more information, see What is mirroring?

OneLake security

OneLake security is the security model that OneLake uses to manage access and permissions for data stored in Fabric. For more information, see Get started with OneLake security. External recipients access shared data in place, and all governance policies remain enforced at the source. For more information, see External data sharing.

Shortcut

A *shortcut* is an embedded reference within OneLake that points to another file store location. Shortcuts enable access to external operational data sources without copying data or building ETL pipelines. You can combine shortcuts with OneLake data sharing for cross-tenant governed access to shared datasets. For more information, see OneLake shortcuts.

Power BI

Power BI provides business intelligence capabilities for creating interactive reports, dashboards, and data visualizations.

For specific Power BI terminology, see Glossary of the Power BI service.

Real-Time hub

Real-Time hub is the centralized location in Fabric for discovering and managing real-time data streams across your organization. Every Fabric tenant automatically includes the hub. For more information, see Real-Time hub overview.

Data stream

A *data stream* is a continuously flowing data source that appears in Real-Time hub. Data streams include stream outputs from eventstreams and tables from KQL databases. They appear automatically in the hub for any user with access.

Fabric event

A *Fabric event* is an event generated by a Fabric or Azure resource, such as a workspace item change, a job completion, or an Azure Blob Storage update. You can subscribe to Fabric events to trigger downstream actions like pipeline runs or notifications. For more information, see Introduction to Azure and Fabric events.

Real-Time Intelligence

The Real-Time Intelligence workload provides tools for ingesting, processing, and analyzing real-time data streams.

Activator

An *activator* is a no-code, low-code item that you use to create alerts, triggers, and actions on your data streams. For more information, see Activator.

Anomaly detector

An *anomaly detector* is an item that detects anomalies in eventhouse tables and sets alerts. For more information, see Anomaly detection (preview).

Digital twin builder

A *digital twin builder* is an item that creates digital representations of real-world environments to optimize physical operations using data. For more information, see What is digital twin builder (preview)?

Event schema set

An *event schema set* is an item that organizes one or more related schemas into schema sets, enabling logical grouping and centralized access control. You can manage who can view, edit, or modify schemas at the group level, making it easier to govern schema usage across teams or projects. For more information, see Schema registry overview (preview).

Eventhouse

An *eventhouse* is an item for storing and analyzing large volumes of data, particularly in scenarios that require real-time analytics. Eventhouses support real-time data streams, so you can ingest, process, and analyze data with low latency. A single workspace can hold multiple eventhouses, an

eventhouse can hold multiple KQL databases, and each database can hold multiple tables. For more information, see Eventhouse overview.

Eventstream

An *eventstream* is an item that provides a centralized place in the Fabric platform to capture, transform, and route real-time events to destinations with a no-code experience. An eventstream consists of various streaming data sources, ingestion destinations, and an event processor when the transformation is needed. For more information, see Fabric eventstreams.

KQL database

A *KQL database* is an item that holds data in a format that you can execute KQL queries against. KQL databases are items under an eventhouse. For more information, see KQL database.

KQL queryset

A *KQL queryset* is the item used to run queries, view results, and manipulate query results on data from your Data Explorer database. The queryset includes the databases and tables, the queries, and the results. The KQL queryset allows you to save queries for future use, or export and share queries with others. For more information, see Query data in the KQL Queryset.

Map

A *map* is an item that visualizes real-time and historical location data in Fabric, helping you monitor live events, analyze spatial patterns, and understand geographic context alongside time-based insights. For more information, see About Fabric maps.

Operations agent

An *operations agent* is an item that monitors real-time data, tracks key metrics, and recommends actions based on defined business rules. Each operations agent is a dedicated Fabric item designed for a specific business process. For more information, see Operations agent.

Real-time dashboard

A *real-time dashboard* is an item for monitoring and visualizing streaming data. You can use a real-time dashboard to ingest, query, and display data with low latency. For more information, see Real-time dashboards overview.

Workload Development Kit

The Workload Development Kit (WDK) enables partners and developers to build, validate, and publish custom workloads that extend the Fabric platform.

Extensibility Toolkit

The *Extensibility Toolkit* is the modern evolution of the Workload Development Kit. It provides an SDK and starter kit for building custom workloads that integrate into Fabric. For more information, see Fabric Extensibility Toolkit.

Workload Hub

The *Workload Hub* is the central place in Fabric where users discover, add, and manage custom workloads at the tenant, capacity, or workspace level. For more information, see How to consume workloads.

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 07/08/2026

Microsoft Fabric migration overview

Microsoft Fabric provides a unified analytics platform. By migrating your data and workloads to Fabric, you can take advantage of its integrated capabilities. This article summarizes all migration-related articles in the documentation, grouped by migration source.

Migrate from legacy, on-premises, or PaaS platforms

Expand table

Article	Description
Microsoft Fabric adoption roadmap	Learn the strategic and tactical considerations and action items that lead to the successful adoption of Microsoft Fabric, and help build a data culture in your organization.
Migrate to Power BI	Learn how to plan and conduct a migration from a third-party BI tool to Power BI.

Migrate from Azure Analysis Services

Expand table

Article	Description
Migrate Azure Analysis Services to Power BI	Migrate Azure Analysis Services to Power BI by using the Power BI Premium migration feature in Power BI.

Migrate from Azure Data Factory (ADF)

Expand table

Article	Description
Planning your upgrade from Azure Data Factory to Data Factory in Fabric	Plan your upgrade from Azure Data Factory to Fabric Data Factory.
How to Use Azure Data Factory item (Mount) in Fabric	The Azure Data Factory item in Microsoft Fabric allows you to bring in your Azure Data Factory artifacts to Fabric instantly.
Migrate from Azure Workflow Orchestration Manager to Microsoft Fabric Apache Airflow	Learn to migrate from Azure workflow orchestration manager to Apache Airflow Job in Microsoft Fabric.

Article	Description
job	

Migrate from Azure SQL Database

Expand table

Article	Description
SqlPackage and Copy job	Migrate from Azure SQL Database or on-premises SQL Server to SQL database in Fabric with minimal downtime by using a `.bacpac` file and SqlPackage, then a Copy job.

Migrate from Azure Data Explorer

Expand table

Article	Description
Migrate from Azure Data Explorer to Fabric Real-Time intelligence (Eventhouse)	Step-by-step guidance for migrating your Azure Data Explorer workloads to Fabric Eventhouse.

Migrate from Azure Synapse Analytics Data Explorer

Expand table

Article	Description
Migrate from Azure Synapse Data Explorer to Fabric Eventhouse (preview)	Step-by-step guidance for migrating your Azure Synapse Data Explorer (Kusto) databases to Fabric Eventhouse.

Migrate from Azure Synapse Analytics dedicated SQL pools

Expand table

Article	Description
Migration Assistant for Fabric Data Warehouse	Learn how to use the Migration Assistant to move data and objects from Azure Synapse Analytics SQL Data Warehouse to Fabric Data Warehouse, including supported scenarios and limitations.

Article	Description
Migration methods: Azure Synapse Analytics dedicated SQL pools to Fabric Data Warehouse	Learn the methods of migration for data warehousing in Azure Synapse dedicated SQL pools to Fabric.
Migration planning: Azure Synapse Analytics dedicated SQL pools to Fabric Data Warehouse	Plan for migration of data warehousing in Azure Synapse dedicated SQL pools to Fabric.

Migrate from Azure Synapse Analytics Spark

Expand table

Article	Description
Migrating from Azure Synapse Spark to Fabric	Learn about migrating from Azure Synapse Spark to Fabric, including key considerations and different migration scenarios.
Migrate data and pipelines from Azure Synapse Analytics to Fabric	Learn about your different options for migrating data and pipelines from Azure Synapse Analytics to Fabric.
Migrate Hive Metastore metadata from Azure Synapse Analytics to Fabric	Learn about your different options for migrating Hive Metastore metadata from Azure Synapse Spark to Fabric.
Migrate notebooks from Azure Synapse Analytics to Fabric	Learn about your different options for migrating your Azure Synapse Spark notebooks to Fabric.
Migrate Spark configurations from Azure Synapse Analytics to Fabric	Learn about migrating Spark configurations from Azure Synapse Spark to Fabric, including prerequisites and migration options.
Migrate Spark job definition from Azure Synapse to Fabric	Learn about migrating Spark job definitions from Azure Synapse Spark to Fabric, including migration prerequisites and options.
Migrate Spark libraries from Azure Synapse to Fabric	Learn about migrating Spark libraries from Azure Synapse Spark to Fabric, including migration prerequisites and options.
Migrate Spark pools from Azure Synapse Analytics to Fabric	Learn about migrating Apache Spark pools from Azure Synapse Spark to Fabric, including migration prerequisites and options.

Migrate from Power BI dataflows (Gen1)

Expand table

Article	Description
Migrate from Dataflow Gen1 to Dataflow Gen2	Learn how to migrate from Power BI dataflows, now known as *Dataflow Gen1*, to Dataflow Gen2 in Data Factory for Microsoft Fabric.

Migrate from Spark

Expand table

Article	Description
Migrate existing workspace libraries and Spark properties to a Microsoft Fabric environment	Learn how to migrate your existing workspace libraries and Apache Spark properties to a default Fabric environment.

Migrate from SQL Server instances

Expand table

Article	Description
Migrate with Migration Assistant	Migrate from an on-premises SQL Server database to a SQL database in Fabric by using the Fabric Migration Assistant.
SqlPackage and Copy job	Migrate from Azure SQL Database or on-premises SQL Server to SQL database in Fabric with minimal downtime by using a `.bacpac` file and SqlPackage, then a Copy job.

Migration resources

- Azure Architectures for Microsoft Fabric
- Migration to Fabric - IFS customer case story
- Enable Microsoft Fabric for your organization
- Microsoft Fabric security fundamentals

Related content

- What's new in Microsoft Fabric?
- Migrating from Power BI capacity to Microsoft Fabric capacity

ⓘ**Note:** The author created this article with assistance from AI. Learn more

Last updated on 09/29/2026

Try Microsoft Fabric for free

The Microsoft Fabric trial capacity gives you **free access for 60 days** to explore almost everything Fabric has to offer—across data engineering, data science, real-time analytics, business intelligence, and more. Whether you're new to Fabric or coming from the product to learn more, this trial is your gateway to understanding how Fabric brings all your data and analytics tools together in one place.

Start a Fabric trial for free

> **Quick start**: Learn how to set up your Fabric trial capacity | Learn how to start using your Fabric trial capacity

> **Note**
>
> **Share your feedback as a new Fabric user**
>
> Your honest take on what works and what doesn't will directly shape product improvements. Sign up to participate and help drive Fabric's future.

What's included—and what's not

Your trial is configured as either an F4 capacity (4 capacity units) or an F64 capacity (64 capacity units) and is complemented with a Power BI Individual Trial if you don't already have a Power BI Premium Per User license.

> **Tip**
>
> If your trial currently has 4 capacity units, you might be eligible to increase the capacity to 64 capacity units. For more information, see Increase trial capacity.

With your trial capacity, you can:

- Use all Fabric workloads, including **Data Factory**, **Data Science**, **Data Engineering**, **Real-Time Analytics**, and **Power BI**.
- Create **pipelines**, **notebooks**, and more.

- Collaborate in **workspaces** and monitor usage with the **Microsoft Fabric Capacity Metrics app**.
- Store up to **1 TB of data** in OneLake.

However, some features aren't available:

- Copilot and Trusted Workspace Access aren't supported.
- AI Experiences such as Data agent, AI functions and AI services aren't supported.
- Private Link is disabled.

Learn more about which Microsoft Fabric features are supported in a trial capacity.

> **Note**
>
> The Fabric Analyst in a Day (FAIAD) workshop is a free, hands-on training designed for analysts working with Power BI and Microsoft Fabric. You can get hands-on experience on how to analyze data and build reports, using Fabric. It covers key concepts like working with lakehouses, creating reports, and analyzing data in the Fabric environment.

When your Fabric trial ends

Your Microsoft Fabric trial capacity lasts for 60 days, unless you cancel it earlier. As the trial nears its end, you'll see notifications in the Fabric portal and in **OneLake catalog** > **Govern** > **Capacities**, helping you track how much time remains.

Once the trial expires:

- Access to the trial capacity is revoked.
- Any workspaces assigned to the trial capacity will be reassigned to Pro and non Power BI Fabric items will no longer be accessible unless the workspace is reassigned to a paid F or P capacity.
- Non Power BI Fabric items (like notebooks and pipelines) remain in your tenant, but they become inactive until reassigned to a valid capacity.
- All content remains stored in OneLake for 7 days and can be reactivated by assigning the workspace to a paid Fabric F or Power BI Premium P capacity.

For more information, see ending your trial. If Copilot becomes unavailable after your trial ends, see Troubleshoot Copilot unavailability after a trial expires.

> **Tip**
>
> - To check your trial status or trial expiration date, go to **OneLake catalog** > **Govern** > **Capacities** > **Trial**.
> - If you need to request a trial extension, contact your Microsoft sales representative or contact the **Azure sales team** for assistance.
> - If you're ready to continue using Fabric, visit the **Purchase Fabric page** to explore your options.

To learn more about limitations and get answers to frequently asked questions about the Fabric trial, see the FAQ.

Set up your Fabric trial capacity

You can start a trial capacity several different ways. The first two methods make you the Capacity administrator of the trial capacity.

- **Method 1**: Start a new trial capacity from the Account manager.
- **Method 2**: Trigger a new trial capacity by trying to use a Fabric feature.
- **Method 3**: Join an existing trial capacity.

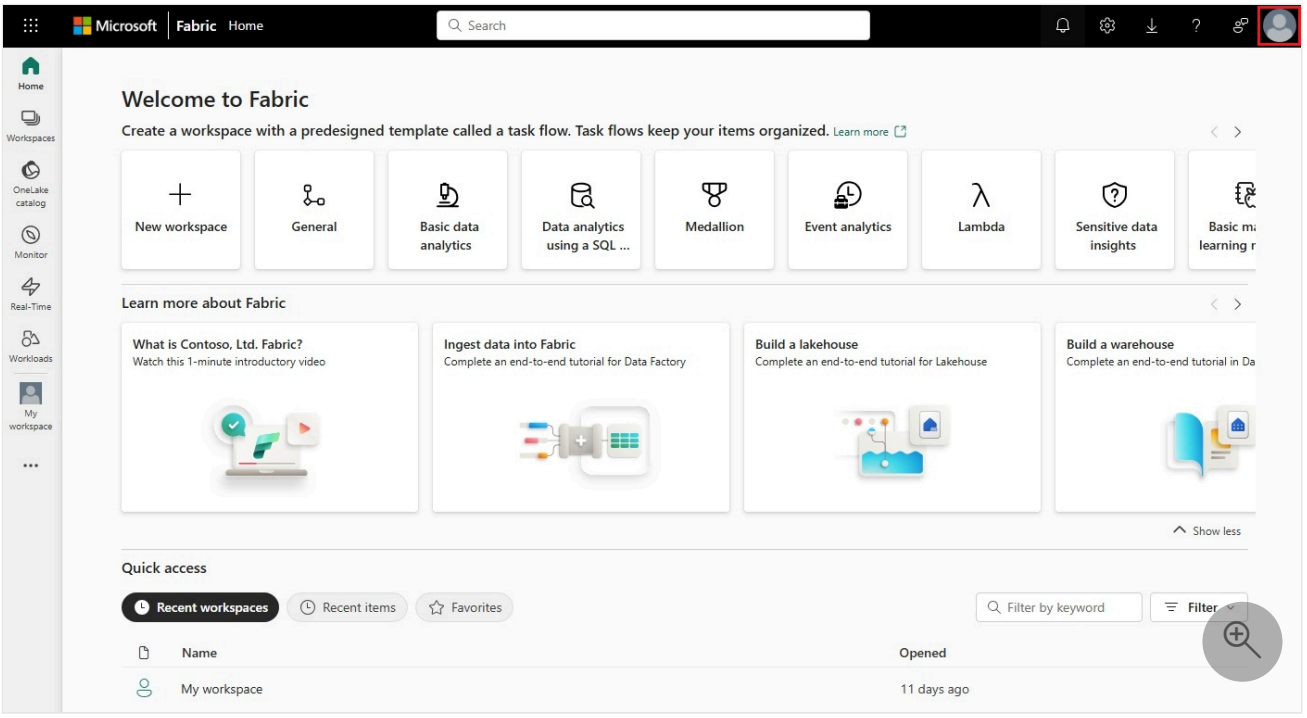
Method 1: Start a new trial capacity from the Account manager

You can sign up for a trial capacity. As the Capacity administrator, you manage who else can use your trial by giving coworkers permission to create workspaces in your trial capacity. Every user in the tenant has contributor permissions by default unless you explicitly manage who has access. Assigning workspaces to the trial capacity automatically adds coworkers (with roles in those workspaces) to the trial capacity. To start your Fabric trial capacity, select this button:

Start a Fabric trial for free

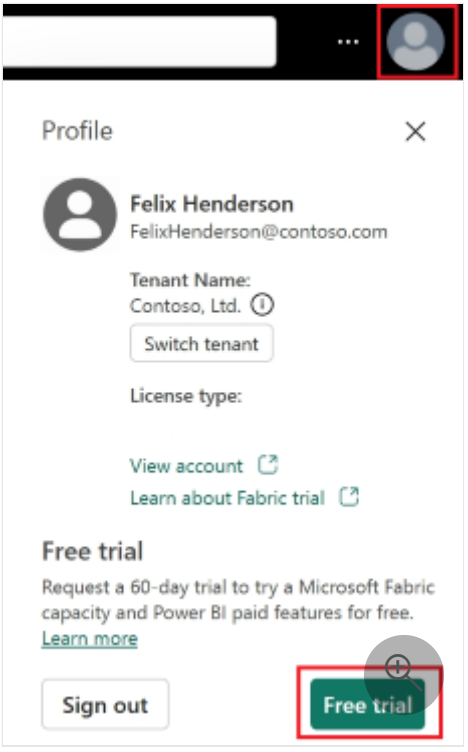
To become the Capacity administrator of that trial, follow these steps:

1. To open your Account manager, select the tiny photo from the upper-right corner of the Fabric homepage.



2. In the Account manager, select **Start trial**. If you don't see **Start trial**, **Free trial**, or a **Trial status**, trials might be disabled for your tenant.

> **Note**
>
> If the Account manager already displays **Trial status**, you might already have a **Power BI trial** or a **Fabric (Free) trial** in progress. To test this out, attempt to use a Fabric feature. For more information, see Start using Fabric.



3. If prompted, agree to the terms and then select **Start trial**.

4. Within the **Activate your 60-day free Fabric trial** capacity prompt, please review the drop down selection of available **Trial capacity regions**. You can either accept the default home region or update it to a new region location that best suits your needs. Once you make your selection, agree to the terms and conditions, then select **Activate**.

   > **ⓘ Important**
   >
   > When planning your region deployment for Fabric capacities, it's important to make the appropriate selection from the outset. If you decide to move a workspace containing Fabric items to a different region after your trial is created, you need to delete all existing Fabric items before proceeding. For detailed instructions, see Moving your data around.
   >
   > To ensure the best performance and resource management, we recommend deploying your capacity in a region where your existing resources are already located. This approach minimizes network latency, reduces ingress and egress charges, and enhances overall reliability.

5. Once your trial capacity is ready, you receive a confirmation message. Select **Fabric Home Page** to begin a guided walkthrough of Fabric. You're now the Capacity administrator for that trial capacity. To learn how to share your trial capacity using workspaces, see Share trial capacities.

Congratulations. You now have a Fabric trial capacity and a complementary Power BI individual trial (if you didn't already have a Power BI *paid* license). To share your capacity, see Share trial capacities..

Method 2: Trigger a Fabric trial by trying to use a Fabric feature

If your organization enabled Fabric and Power BI trials, attempting to use a Fabric feature launches a Fabric trial capacity. If your Fabric administrator enabled Microsoft Fabric for the tenant but you still don't have access to a Fabric-enabled capacity, follow these steps:

1. Try to create a Fabric item in a workspace that you own (such as **My Workspace**) that doesn't currently support Fabric items (that is, not a Fabric, Premium, or Trial workspace).
2. Follow the prompt to start a Fabric trial capacity.

Once you agree, your trial starts and the workspace is reassigned to a trial capacity. You're the Capacity administrator and can add additional workspaces to the trial capacity by changing their type to Trial in workspace settings.

Method 3: Join an existing trial

You can join a trial started by a coworker by adding your workspace to their existing trial capacity. This action only is possible if the trial capacity administrator gives you or the entire organization **Contributor permissions** to the trial capacity.

For more information, see Sharing trial capacities.

Start using your trial

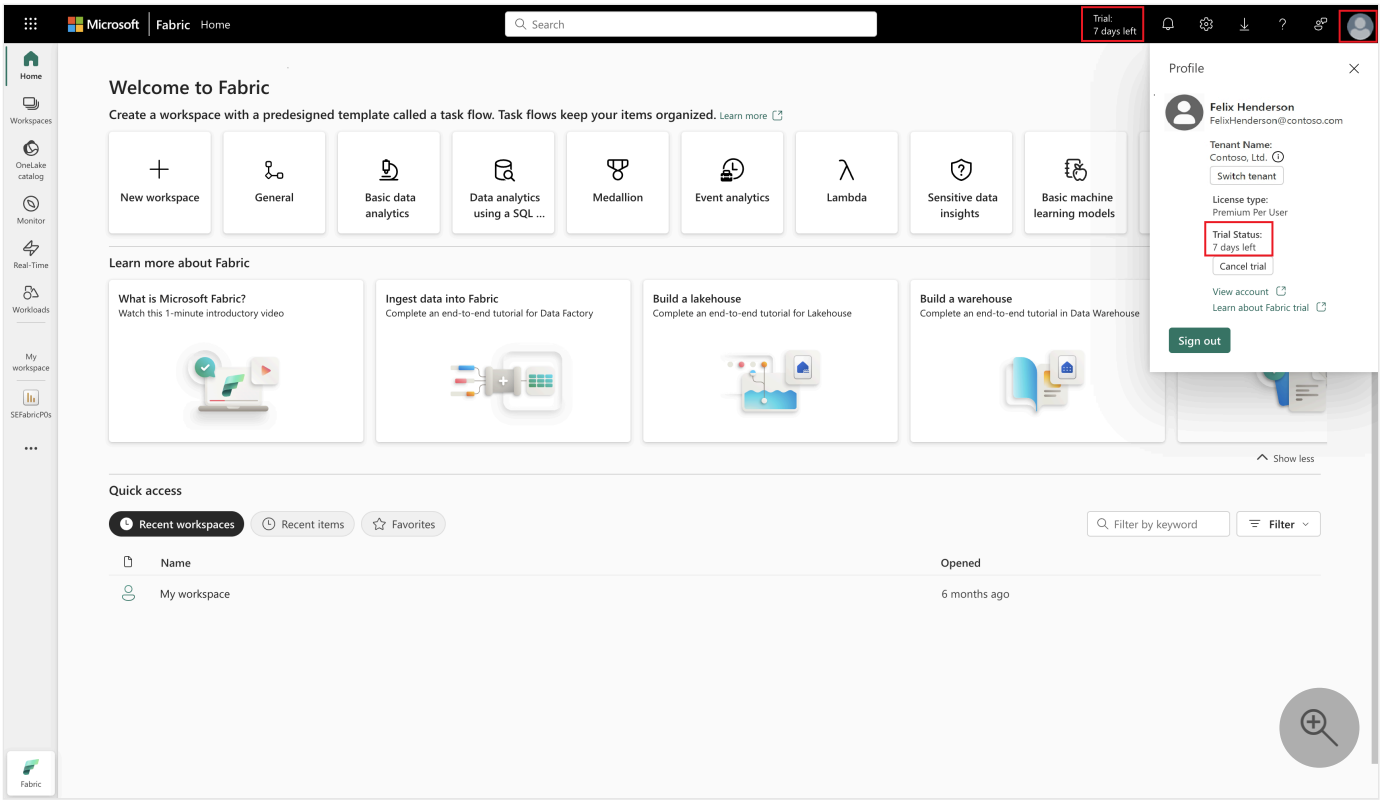
To start using your Fabric capacity trial, create a new workspace. Assign that workspace to your trial capacity using the *Trial* workspace type, and then all the items in that workspace are saved and executed in that trial capacity. Invite colleagues to those workspaces so they can share the trial experience with you. If you, as the Capacity administrator, enable **Contributor permissions**, then others can also assign their workspaces to your trial capacity. For more information about sharing, see Share trial capacities.

- **Existing Power BI users**: If you're an existing Power BI user, jump to Start the Fabric trial.
- **New Power BI users**: The Fabric trial requires a per-user license. Open Fabric to sign up for a Fabric (Free) license. Once you have the free license, you can begin participating in the Fabric capacity trial.
    - You might already have a license and not realize it. For example, some versions of Microsoft 365 include a Power BI Pro license. Open Fabric and select your Account manager to see if you already have a license, and which license it is. Read on to see how to open your Account manager.

Start a Fabric trial for free

Get the status of your trial

To see the status of your trial, open your Account manager again and look for the **Trial status**. Your Account manager keeps track of the number of days remaining in your trial. You can also see the countdown in the Fabric menu bar or go to **OneLake catalog** > **Govern** > **Capacities** > **Trial**.



Share trial capacities

The person who starts the Fabric trial capacity becomes the Capacity administrator for that trial capacity. Other users on the same tenant can also start a Fabric trial capacity and become the Capacity administrator for their own trial capacity. Hundreds of users can use each trial capacity. But, Microsoft sets a limit on the number of trial capacities that can be created on a single tenant. To help others in your organization try out Fabric, share your trial capacity. There are several ways to share.

Share by enabling contributor permissions

To share and collaborate during a trial, you need to make sure the right tenant settings are enabled. Trial enablement is managed through **Tenant settings** and **Help and support settings**, with optional capacity assignment.

1. Go to **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**.
2. Expand **Users can create Fabric items**, and turn it on. Optionally, scope this setting to specific security groups.
3. Under **Help and support settings**, enable **Users can try Microsoft Fabric paid features**. This allows users to start a trial.
4. (Optional) Assign trial capacity in **Capacity settings** if you want to control where trial workspaces are created.

> **Note**
>
> The previous option **Enabled for the entire organization** in the **Trial** section is no longer available. Trial enablement is now managed through tenant settings.

Share by assigning workspaces

If you're the Capacity administrator, assign the trial capacity to multiple workspaces. Anyone with access to one of those workspaces can now use the Fabric capacity trial.

1. Open **Workspaces** and select the name of a workspace.
2. Select **Workspace settings**, then **Workspace type**.
3. Select the **Fabric Trial** as the workspace type and select **Apply**.

For more information, see Use Workspace settings.

Increase trial capacity

If you're the Capacity administrator or Tenant administrator, you might be eligible to increase the trial capacity by following these steps:

1. Go to **OneLake catalog** > **Govern** > **Capacities**, and then choose the **Trial** tab.
2. Your current trial size is displayed here. Select the **Change size** button to increase from 4 capacity units to 64 capacity units.
3. Select **Apply**. If you're eligible to change the size, you see a success message and receive a notification.

You can also downgrade from 64 capacity units to 4 capacity units using these same steps.

> **Note**
>
> Eligibility to increase trial capacity may vary. If you do not see the option, please contact your Microsoft sales representative or contact Azure sales team for assistance. Changing the trial capacity doesn't change the length of the trial. The number of remaining days doesn't reset or become extended.

End a Fabric trial capacity

End a Fabric capacity trial by canceling, letting it expire, or purchasing the full Fabric experience.

For more information, see Canceling, expiring, and closing.

Cancel the Fabric trial

Capacity admins and Fabric admins can cancel a trial capacity. The user who starts a trial automatically becomes the Capacity administrator for that trial capacity. The Fabric administrator has full access to all Fabric management tasks. All Fabric items (non-Power BI items) in those workspaces become unusable and may be deleted after 7 days. Only the Capacity or Fabric administrator can cancel the Fabric trial capacity.

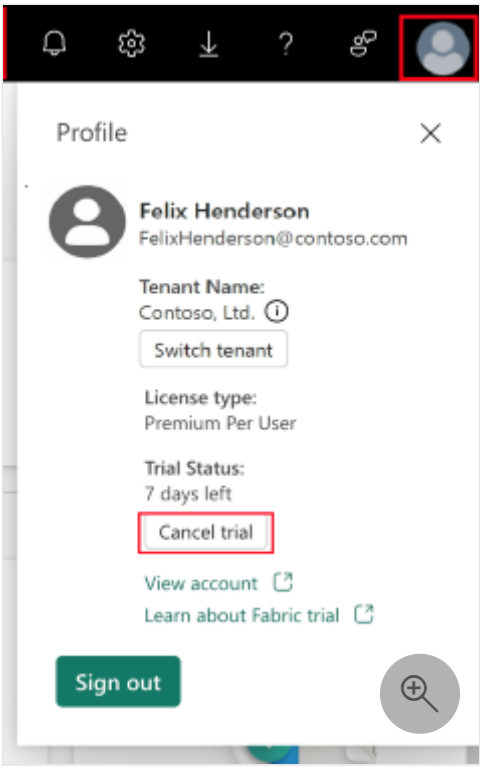
One reason to cancel a trial capacity is when the Capacity administrator of a trial capacity leaves the company. Because Microsoft limits the number of trial capacities available per tenant, you might want to remove the unmanaged trial capacity to make room for other users to sign up for a new trial capacity.

If you don't move the workspaces and their contents to a new capacity that supports Fabric:

- The workspace type of any workspaces assigned to that trial capacity changes to Power BI Pro.
- All non Power BI Fabric items in the workspaces become unusable and may be deleted after seven days. Your Power BI items are unaffected and still available when the workspace type returns to Power BI Pro.
- You can't create workspaces that support Fabric capabilities.
- You can't share Fabric items, such as machine learning models, warehouses, and notebooks, and collaborate on them with other Fabric users.
- You can't create any other analytics solutions using these Fabric items.

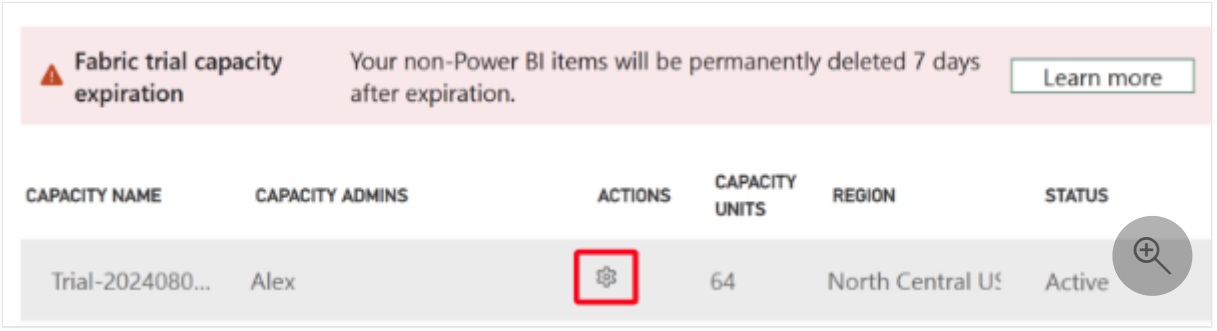
> **Note**
>
> Only capacity and Fabric admins can cancel a Fabric trial capacity.

- **Cancel a trial using your Account manager**: As a trial capacity admin, you can cancel your free Fabric trial capacity from your Account manager. Canceling the trial this way ends the trial capacity for yourself and anyone else you invited to the trial.
  - Open your Account manager and select **Cancel trial**.



- **Cancel the Fabric trial capacity in Govern**: As a Capacity or Fabric administrator, you can use **OneLake catalog** > **Govern** > **Capacities** to cancel a Fabric trial capacity:

    1. Go to **OneLake catalog** > **Govern** > **Capacities**.
    2. Then choose the **Trials** tab.
    3. Select the cog icon for the trial capacity that you want to delete.



Let the trial expire

A standard Fabric trial capacity lasts 60 days. If you don't reassign workspaces to a paid Fabric capacity before the end of the trial period, non-Power BI Fabric items are removed according to the retention policy upon removal. You have seven days after the expiration date to save your non-Power BI Fabric items by assigning the workspaces to an F or P capacity that supports Fabric.

To retain your Fabric items, before your trial ends, purchase Fabric.

Purchase the full Fabric experience

If you want to retain your data and continue to use Microsoft Fabric, purchase a capacity and migrate your workspaces to that capacity. You can also migrate your workspaces to a capacity that you already own that supports Fabric items.

Frequently asked questions

Here are some things to consider and answers to frequently asked questions about the Fabric trial.

Can I use a Fabric trial capacity for production workloads?

A Fabric trial capacity is intended for evaluation and testing only. Because trial capacities are time-bound, don't rely on them for ongoing production use, business-critical workloads, or long-term operational processes. For ongoing reliability and support for production scenarios, use a paid Fabric capacity. For more information, see Purchase the full Fabric experience.

Is every user eligible to start a Fabric trial?

Trial eligibility depends on factors such as your tenant settings, license state, and the number of trial capacities available in your tenant. Microsoft determines trial availability by region and tenant, and availability can change over time. For troubleshooting guidance, see Why can't I start a trial? If you're still unable to start a trial, you may contact Microsoft Support.

Can I start another Fabric trial after my current trial ends?

Eligibility for another trial isn't guaranteed, and Microsoft might restrict repeated or successive trial usage. To continue using Fabric after your trial ends, you can purchase a paid Fabric capacity or ask your Fabric administrator whether an existing capacity is available for your workspace.

What if I don't already have an assigned Power BI PPU?

If you don't already have an assigned Power BI Premium Per User (PPU) license, you receive a Power BI Individual Trial when initiating a Fabric trial capacity. This individual trial enables you to perform the actions and use the features that a PPU license enables. Your Account manager still displays the nontrial licenses assigned to you. But in order to make full use of Fabric, your Fabric trial capacity comes with a complementary Power BI Individual trial.

Why can't I start a trial?

If you don't see the **Start trial** button in your Account manager:

- Your Fabric administrator might have disabled access, and you can't start a Fabric trial capacity. To request access, contact your Fabric administrator.
- You're an existing Power BI individual trial user, and you don't see **Start trial** in your Account manager. You can start a Fabric trial by attempting to create a Fabric item. When you attempt to create a Fabric item, you receive a prompt to start a Fabric trial. If you don't see this prompt, it's possible your Fabric administrator deactivated this action.

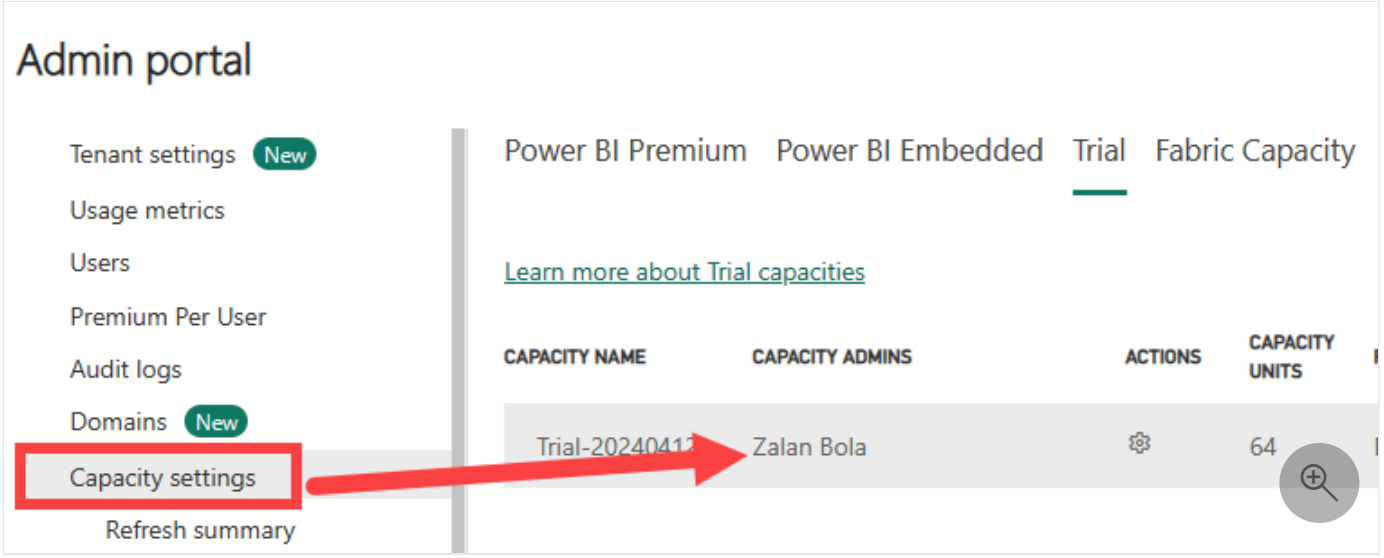
If you do see the **Start trial** button in your Account manager but can't start a trial:

- You might not be able to start a trial capacity if your tenant exhausted its limit of trial capacities. If that's the case, you have the following options:
  - Request another trial capacity user to share their trial capacity workspace with you. Give users access to workspaces.
  - Purchase a Fabric capacity from Azure by performing a search for *Microsoft Fabric*.

How do I look up the trial Capacity administrator?

To request access to a trial capacity or to check whether your organization has the Fabric tenant setting enabled, you can contact your Capacity administrator. Ask your Fabric administrator to use **OneLake catalog** > **Govern** > **Capacities** to look up your Capacity administrator. You can find your Fabric administrator as the service administrator in the Workspace Settings of your workspace.

If you're the Capacity or Fabric administrator, go to **OneLake catalog** > **Govern** > **Capacities**, and then choose the **Trial** tab.



How do I know if my tenant has reached its trial capacity limit?

Microsoft limits the number of trial capacities available per tenant. If your tenant has exhausted its trial capacity limit, you'll encounter one of these scenarios:

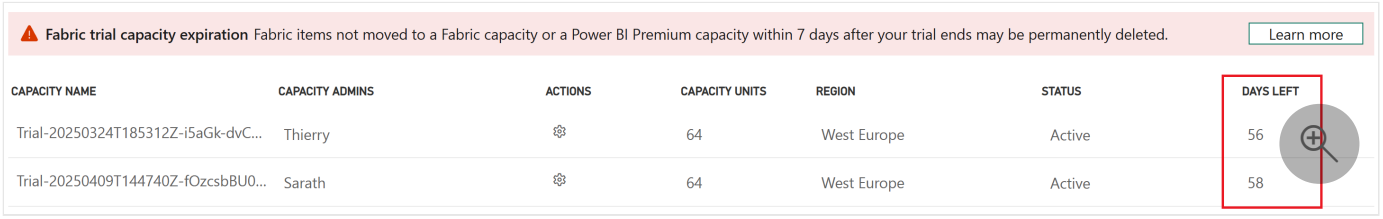
- When attempting to start a new trial from the Account manager, the **Start trial** option might not be available or you may receive an error message indicating the trial limit has been reached.
- If you try to trigger a trial by creating a Fabric item, the trial activation prompt won't appear.

To resolve this issue:

- Contact your Fabric administrator to check existing trial capacities in **OneLake catalog** > **Govern** > **Capacities** > **Trial**. If there are inactive or unmanaged trial capacities (for example, from users who left the organization), consider canceling them to free up capacity for new trials.
- Request access to an existing trial capacity from a Capacity administrator who can share their trial by assigning workspaces or enabling contributor permissions.
- Consider purchasing a Fabric capacity if trial limits prevent your organization from testing Fabric features.

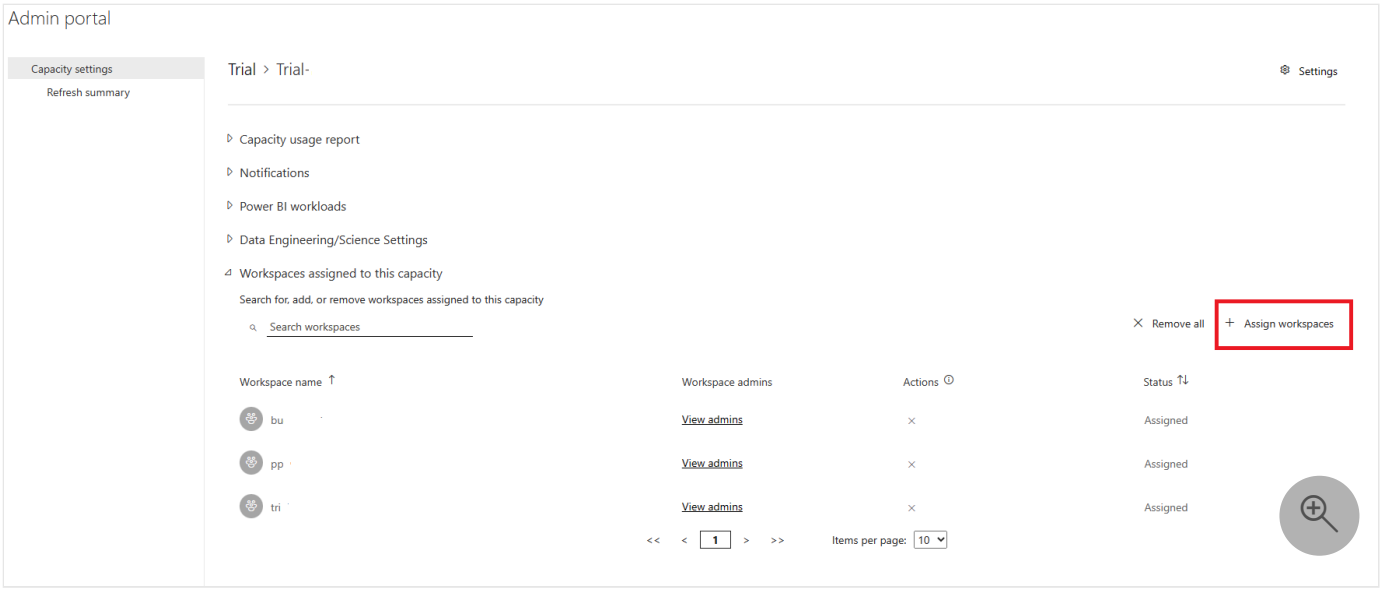
How do I look up the number of days remaining in my trial?

Go to **OneLake catalog** > **Govern** > **Capacities**, and then choose the **Trial** tab. The **Days left** column tracks the remaining days for each trial.



Why can't I assign a workspace to the trial capacity in my workspace settings?

This can occur when the Fabric administrator turns off trials after you start a trial. To add your workspace to the trial capacity, go to **OneLake catalog** > **Govern** > **Capacities** > **Trial**, and choose the name of the capacity. If you don't see your workspace assigned, add it here.



What is the region for my Fabric trial capacity?

If you start the trial using the Account manager and didn't change the default region selection, your trial capacity is usually located in the home region for your tenant. See Find your Fabric home region for information about how to find your home region, where your data is stored.

> **Note**
>
> In most cases, the default trial Capacity region matches your home region. However, in some cases, Fabric trials are created by default in a similar but different region (for example, East US instead of Central US).

How do I select a different region for my Fabric trial capacity?

The easiest way to select a non-default region for your Fabric trial capacity is at the point of initiating the trial from the Account Manager. The list of available regions for selection will be available during this step.

What effect does region have on my Fabric trial capacity?

Not all regions are available for the Fabric trial capacity. Start by looking up your home region and then check to see if your region is supported for the Fabric trial. If your home region doesn't have Fabric enabled, don't use the Account manager to start a trial. To start a trial in a region that isn't your home region, follow the steps in Other ways to start a Fabric trial. If you already started a trial from Account manager, cancel that trial and follow the steps in Other ways to start a Fabric trial instead.

Can I move my tenant to another region?

You can't move your organization's tenant between regions by yourself. If you need to change your organization's default data location from the current region to another region, you must contact support to manage the migration for you. For more information, see Move between regions.

How can I see the Fabric trial capacity availability by Azure region?

To learn more about regional availability for Fabric trials, see Fabric trial capacities are available in all regions.

How is the Fabric trial capacity different from a Power BI individual trial?

A Power BI individual trial provides use rights equivalent to a Power BI Premium Per User license. A Fabric trial capacity is for storing Fabric items and running Fabric workloads. All rules guiding Power BI licenses and what you can do in the Power BI workload remain the same with a Fabric trial capacity, which is why having a complementary Power BI individual trial is useful. The key difference is that a Fabric capacity is required to use non-Power BI workloads and items.

Does the Fabric trial capacity support autoscale?

The Fabric trial capacity doesn't support autoscale. If you need more compute capacity, you can purchase a Fabric capacity in Azure.

How can trial Capacity administrators migrate existing workspaces into a trial capacity?

Trial Capacity administrators can migrate existing workspaces into a trial capacity using workspace settings and choosing **Fabric Trial** as the workspace type. To learn how to migrate workspaces, see create workspaces.

How many SQL databases can I create in a Fabric trial capacity?

Currently, you can create up to three SQL databases in a Fabric trial capacity.

Next steps

Start a Fabric trial for free

Related content

- Learn about licenses
- Review Fabric terminology

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/29/2026

Start a Microsoft Fabric free trial with your personal email

If you don't have access to a work account, you can still start a Microsoft Fabric free trial by using a personal email address.

This article walks you through creating or using an Azure account with your personal email, setting up a user in Microsoft Entra ID, signing in to Fabric, and activating the free trial.

> **Note**
>
> **Share your feedback as a new Fabric user**
>
> Your honest take on what works and what doesn't will directly shape product improvements. Sign up to participate and help drive Fabric's future.

Prerequisites

Before you begin, make sure you have the required information and tools ready.

- A personal email address (for example, Outlook, Gmail, or Yahoo).
- A phone number and payment method for Azure account verification, if prompted.
- The Microsoft Authenticator app for two-factor authentication. This is optional but recommended.

Create an Azure trial account

An Azure account is required to access Microsoft Fabric. Create or sign in to an Azure account so you can access the services required to start a Fabric trial.

1. Go to the Azure free account page.
2. Select **Try Azure for free**.
3. If you have an existing Azure account, sign in with it. Otherwise, select **Sign up** and use your personal email address to register.
4. Follow the prompts to complete the setup. You may be asked to verify your identity with a phone number and provide a payment method. Microsoft uses this information for

verification purposes, and you won't be charged unless you explicitly upgrade to a paid subscription.

5. After completing the registration, you should have access to the Azure portal.

Create a user in Microsoft Entra ID

Create a user identity that you'll use to sign in to Microsoft Fabric and start the trial.

1. In the Azure portal, search for and open **Microsoft Entra ID**.
2. From the left menu, expand **Manage** and select **Users**.
3. Select **New user** and then **Create new user**. Use the steps in Create a user in Microsoft Entra ID article to create a user with your personal email address. Set a password and complete the user creation process. You don't have to set role assignments or other properties.
4. After the user is created, open the **Users** pane and copy the user principal name (UPN).

Sign in to Microsoft Fabric

After you create the user account, sign in to Fabric by using that account's credentials.

1. Go to the Fabric portal.
2. Sign in by using the UPN and password for the user you created.
3. For security purposes, enable two-factor authentication using the Microsoft Authenticator app when prompted.
4. Complete any required authentication prompts.

Activate the Fabric free trial

Once you're signed in, activate the free trial from your Fabric profile menu.

1. From the Fabric portal, select your profile icon in the upper-right corner.
2. Select **Free trial**, choose a capacity region, and select **Activate**.

After activation finishes, you can start creating and exploring Fabric items.

Related content

After activation, use these resources to continue learning and building in Fabric.

- Get started with Microsoft Fabric

- Learn about licenses
- Learn about Fabric security and governance

ⓘ **Note:** The author created this article with assistance from AI. Learn more

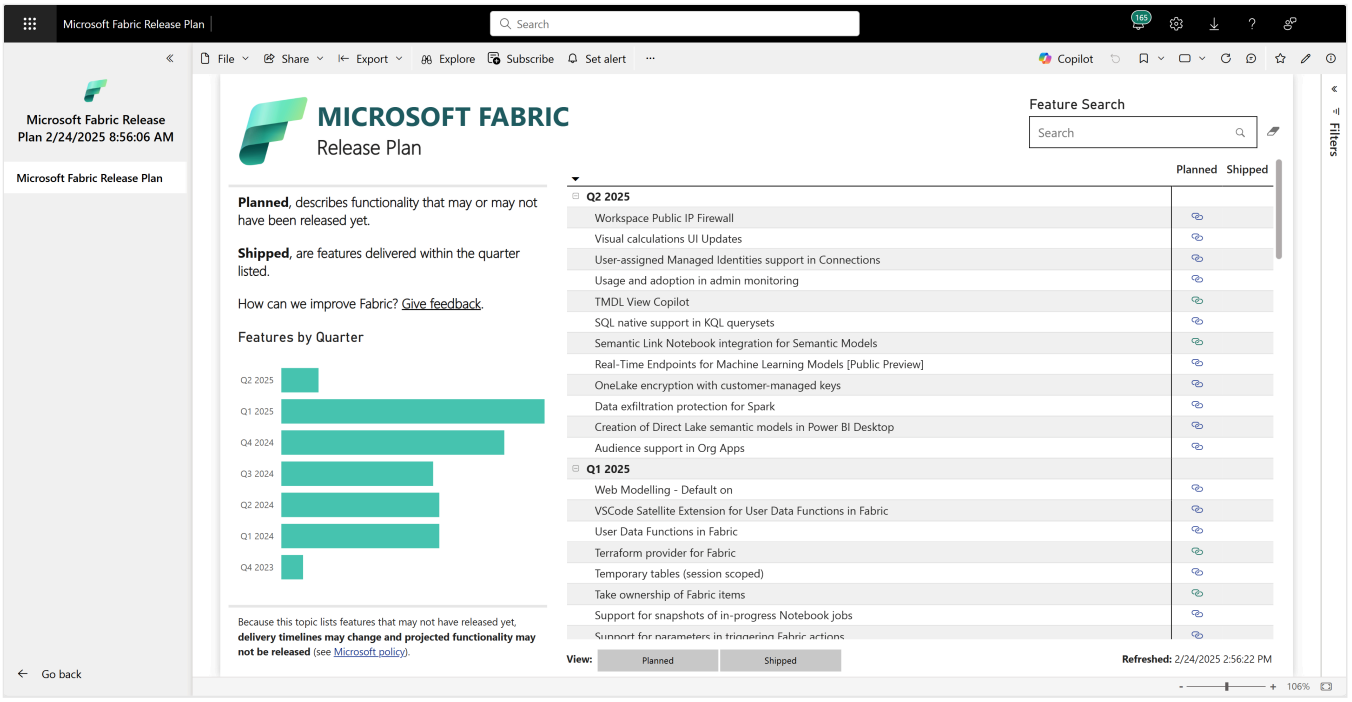
---

Last updated on 04/08/2026

Connect to the Microsoft Fabric Release Plan report

Article • 05/22/2025

This article tells you how to install the template app for the Microsoft Fabric Release Plan report, and how to connect to the data source.

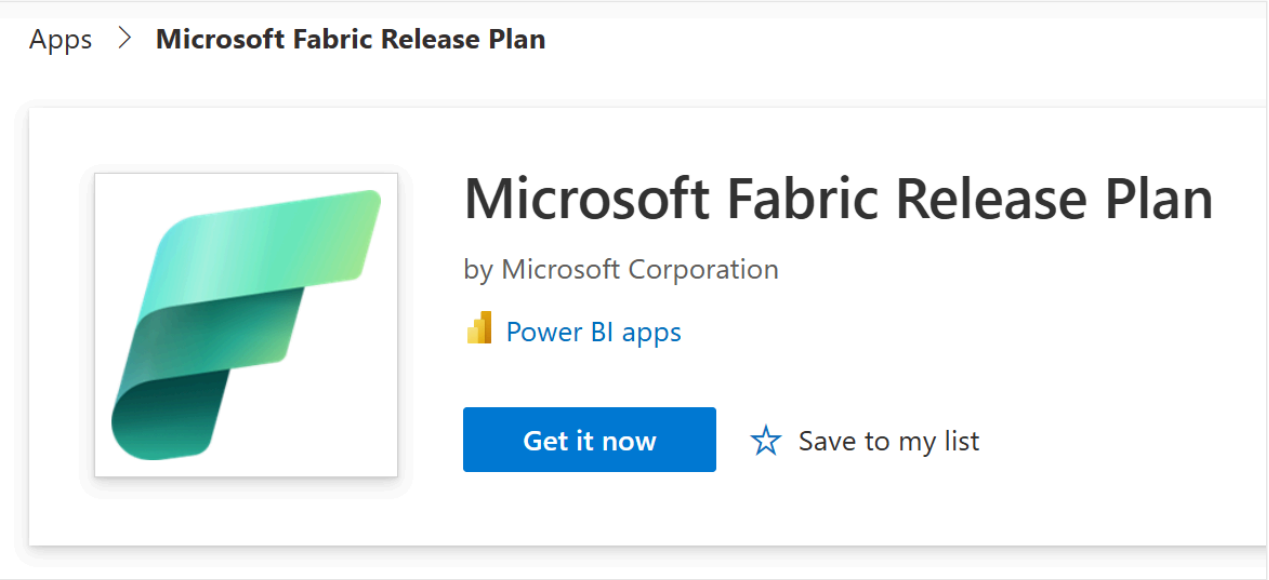


After you install the template app and connected to the data sources, you can customize the report as per your needs. You can then distribute it as an app to colleagues in your organization.

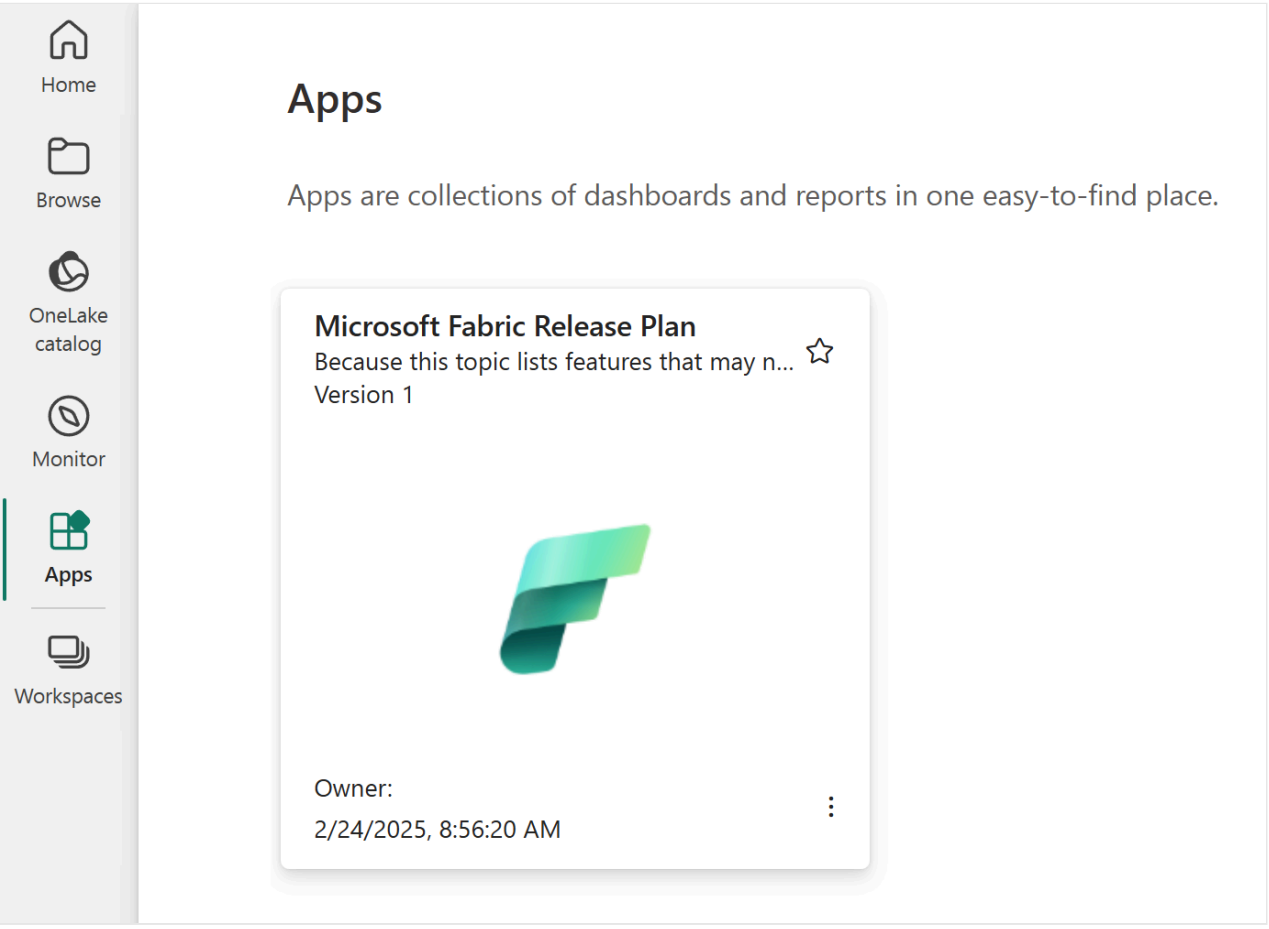
Install the app

To get the Microsoft Fabric Release Plan app, you need to install it from AppSource. The app is available for free.

1. Visit Microsoft Fabric Release Plan in AppSource.
2. Select **Get it now**.



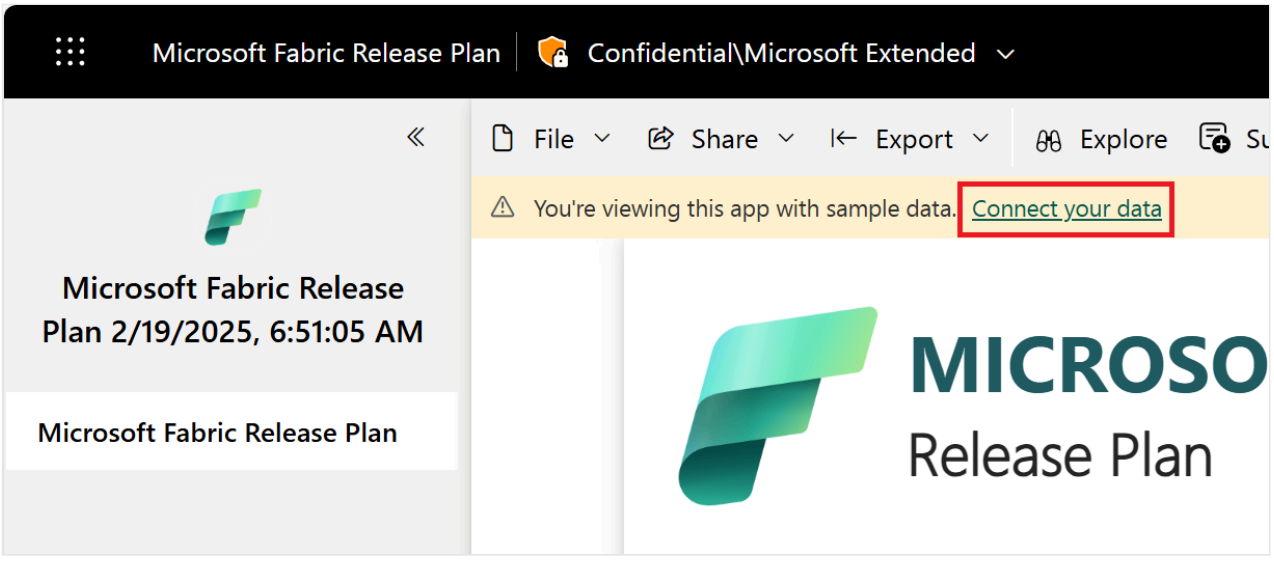
3. When prompted, select **Install**. Once the app is installed, it shows on your **Apps** page.



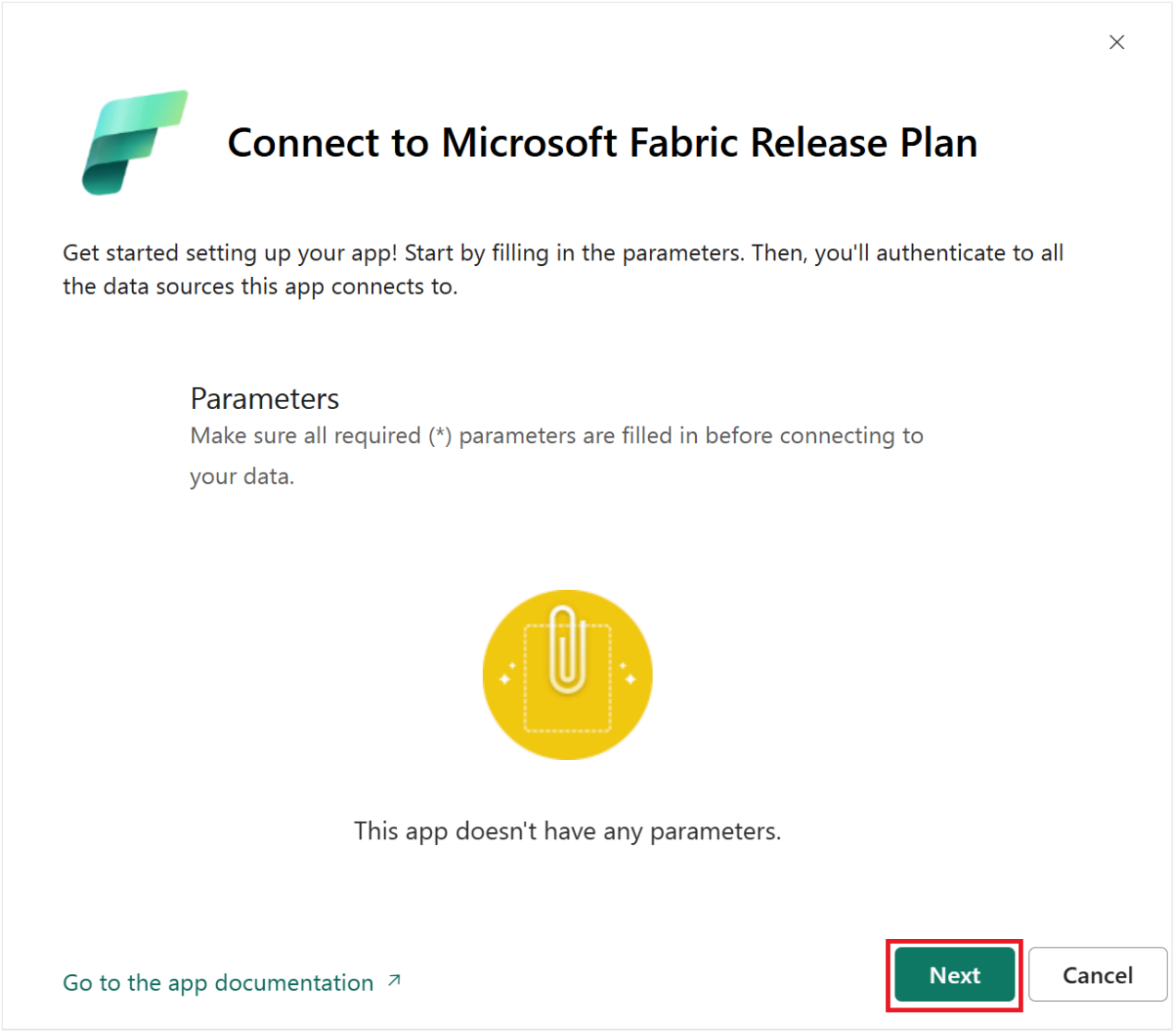
Connect to data sources

1. Select the icon on your Apps page to open the app. The app opens, showing sample data.

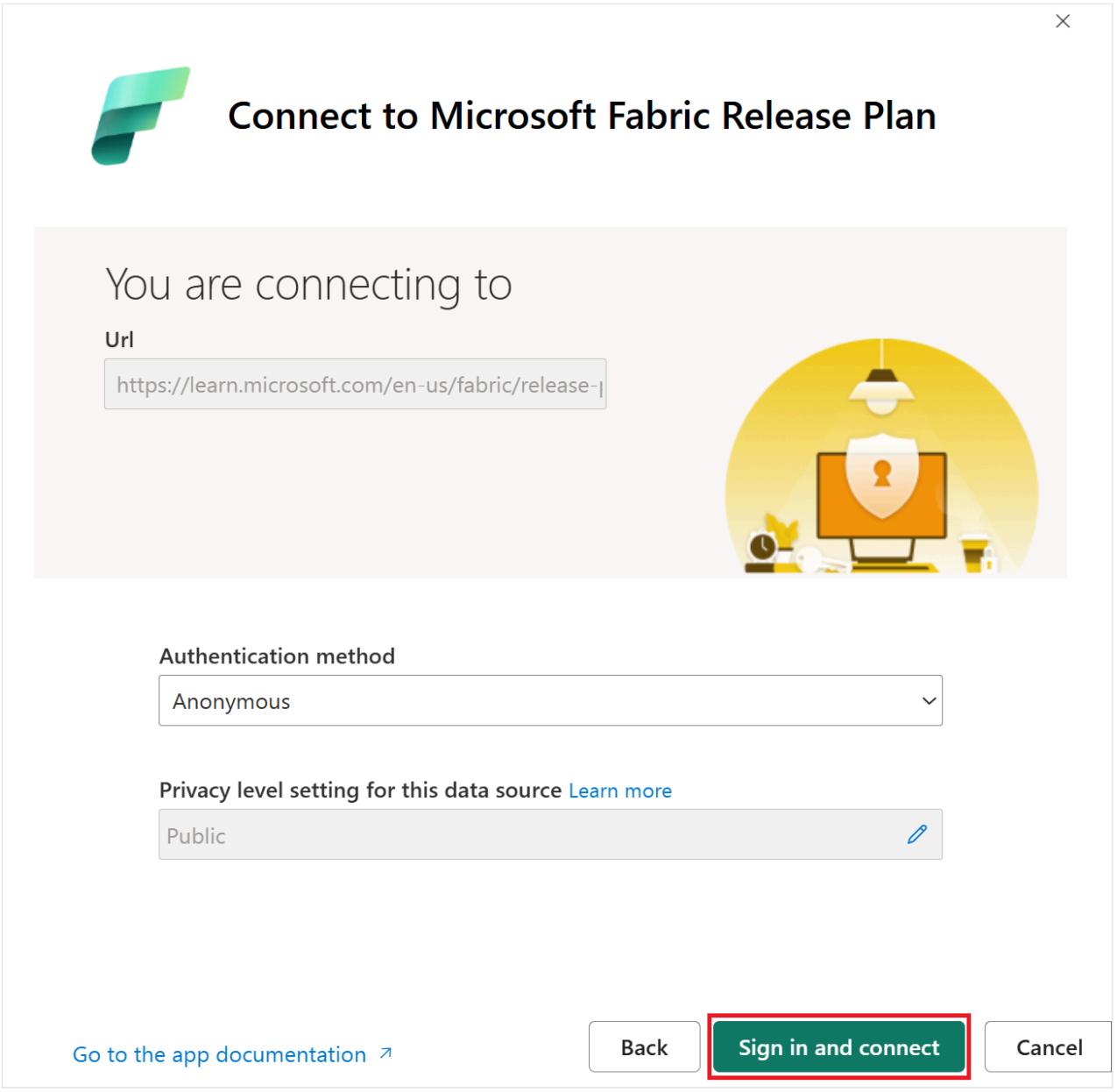
2. Select the **Connect your data** link on the banner at the top of the page.



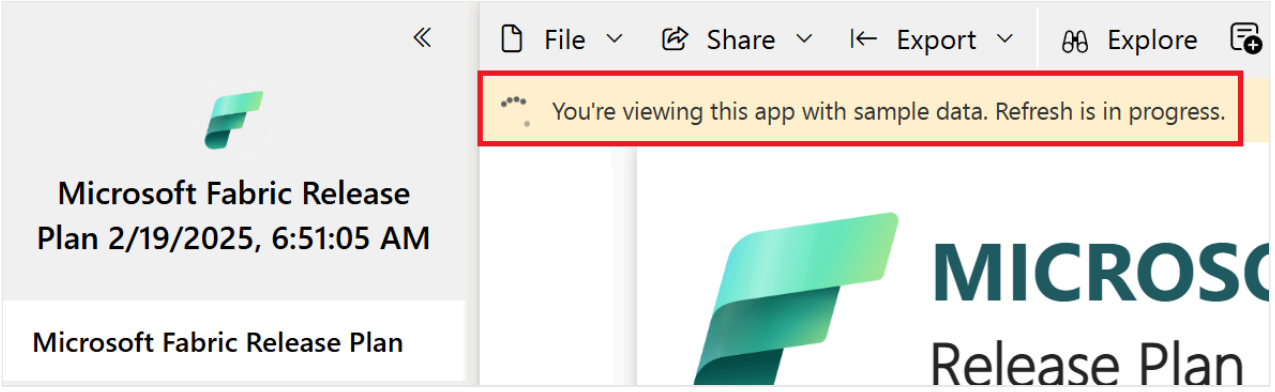
3. The parameters dialog appears. There are no required parameters, so you can select **Next**.



4. The authentication method dialog appears. Recommended values are prepopulated as an Anonymous authentication method. select **Sign in and connect**.



5. The report connects to the data sources and is populated with up-to-date data. You see sample data while the refresh is in progress.



When the data refresh is complete, you're in the workspace associated with the app.

Set up a report refresh schedule

To keep the report data up to date, you can set up a refresh schedule.

Customize and share

For details, see Customize and share the app.

Resources

- Help improve Microsoft Fabric by submitting an idea.
- Questions? Try asking the Microsoft Fabric Community.

Related content

- What are Microsoft Fabric template apps?
- Install and distribute template apps in your organization

Microsoft Fabric preview information

Article • 01/26/2025

This article describes the meaning of *preview* in Microsoft Fabric, and explains how preview experiences and features can be used.

Preview experiences and features are released with limited capabilities, but are made available on a *preview* basis so customers can get early access and provide feedback.

Preview experiences and features:

- Are subject to separate supplemental preview terms.
- Aren't meant for production use.
- Are not subject to SLAs and support is provided as best effort in certain cases. However, Microsoft Support is eager to get your feedback on the preview functionality, and might provide best effort support in certain cases.
- May have limited or restricted functionality.
- May be available only in selected geographic areas.

Who can enable a preview experiences and features

To enable a preview experience or feature, you need to have a *Fabric administrator* admin role.

> **Note**
>
> When a preview feature is delegated, it can be enabled by a capacity admin for that capacity.

How do I enable a preview experience or feature

To enable a preview experience or feature, follow these steps:

1. Navigate to the admin portal.

2. Select tenant settings tab.

3. Select the preview experience or experience you want to enable.

4. Enable experience using the tenant setting.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

End-to-end data lifecycle in Microsoft Fabric

Organizations typically rely on multiple disconnected services to ingest, store, transform, analyze, and visualize data. This fragmentation creates data silos, increases integration overhead, and slows time to insight. Microsoft Fabric addresses these challenges by unifying every stage of the data lifecycle into a single platform built on a shared foundation.

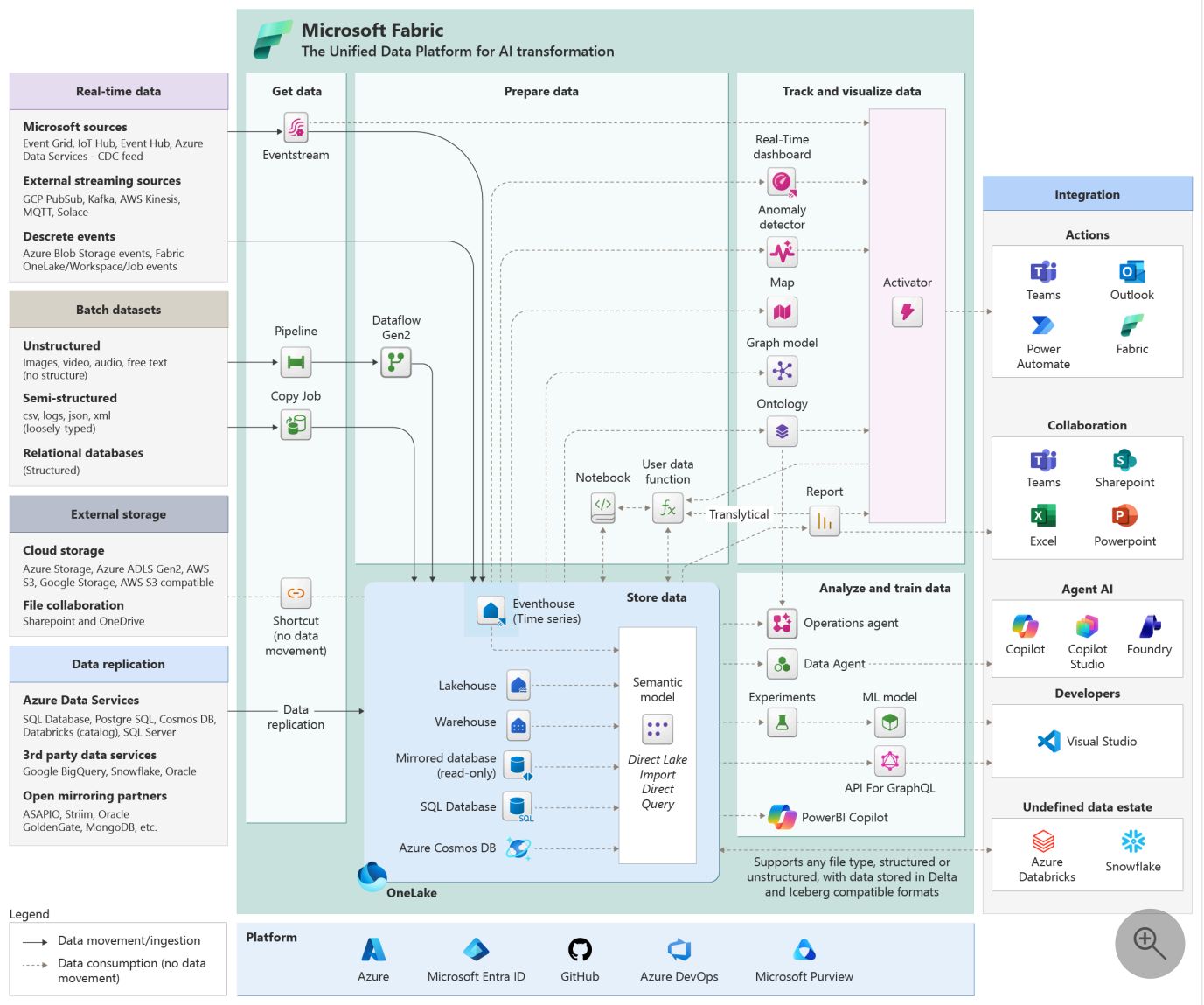
At the center of this architecture is OneLake, a single organizational data lake that stores all data in open Delta Parquet format. OneLake is provisioned automatically with every Fabric tenant. Because every Fabric workload reads from and writes to OneLake, data doesn't move between engines. A dataset ingested through a pipeline, refined in a notebook, and visualized in a Power BI report stays in one place throughout its journey.

The data lifecycle consists of six stages, and Fabric provides purpose-built tools for each:

- **Get data**: Bring data into OneLake from hundreds of sources in real time, on a schedule, through continuous database replication, or by referencing external storage in place.
- **Store data**: Persist data in storage formats optimized for your workload, whether that's flexible big data analytics, structured SQL queries, real-time event analysis, transactional processing, or governed business reporting.
- **Prepare and transform**: Clean, reshape, and enrich data using low-code visual transformations or code-first notebooks and reusable functions, without moving data out of OneLake.
- **Analyze and train**: Build and operationalize machine learning models, run advanced analytics, query data programmatically, and explore insights through natural-language AI agents.
- **Track and visualize**: Surface insights through interactive reports, monitor live data streams on real-time dashboards, and trigger automated actions when conditions are met.
- **External integration**: Securely connect to external services for automation, collaboration, developer tooling, and CI/CD.

Governance spans every stage of the data lifecycle, helping organizations discover, manage, secure, and monitor their Fabric data estate.

The following diagram shows how these stages connect and which Fabric items participate at each stage. Each stage is covered in depth in a dedicated article. Use the links in each section to explore the capabilities and tools available at that stage.



Get data

Different types of datasets come from a wide variety of data sources across different data scenarios, including data replication, external storage references, batch datasets, and real-time data streams. You ingest and transform these datasets through Fabric's integration tools. The data lands in OneLake, the centralized data storage for all of Fabric. Key ingestion methods include:

- **Eventstreams** for real-time event ingestion and routing.
- **Data pipelines** for batch and scheduled data movement with more than 200 connectors.
- **Mirroring** for continuous replication from operational databases without building ETL pipelines.

- **Shortcuts** for no-copy data virtualization from external storage like Azure Data Lake, Amazon S3, or Google Cloud Storage. Shortcuts can also reference data shared from other Fabric workspaces or tenants.
- **OneLake data sharing** for cross-tenant access to live, governed datasets without copying data across organizational boundaries.

For more information, see Get data into Microsoft Fabric.

Store data

Once ingested, all data lands in OneLake in open Delta Parquet format. OneLake provides a single data lake for your entire organization with no separate provisioning needed. Because OneLake maintains a single copy of the data, you can share governed datasets across tenants by using OneLake data sharing without duplicating storage. Fabric offers several storage items optimized for different workloads:

- **Lakehouse** for flexible big data storage that combines files and managed Delta tables with an automatic SQL analytics endpoint.
- **Warehouse** for structured, relational analytics with full T-SQL support, stored procedures, and ACID transactions.
- **Eventhouse** for real-time analytics on streaming and telemetry data by using Kusto Query Language (KQL).
- **SQL Database** for transactional workloads and operational analytics.
- **Semantic models** for curated business logic, measures, and hierarchies that power reports and AI.

For more information, see Store data in Microsoft Fabric.

Prepare and transform data

Once in OneLake, you can further transform the data by using either code-first engines or low-code tools, all within Fabric with no data movement between engines:

- **Dataflow Gen2** provides a low-code Power Query interface for data cleansing, transformation, and enrichment.
- **Notebooks** offer a Jupyter-like environment for Python, T-SQL, and Scala-based data engineering.
- **User Data Functions** allow you to embed reusable custom Python logic that can be invoked from pipelines, notebooks, and Activator rules.

For more information, see Prepare and transform data.

Analyze data and train models

Use the prepared data to train ML models and perform advanced analytics. Fabric's Data Science workload provides an environment for building, training, and operationalizing ML models:

- **MLflow experiments** track model training runs with automatic logging of hyperparameters, metrics, and items.
- **ML models** are registered in an MLflow-powered registry for versioning, metadata tracking, and reproducibility.
- **Data agents** and **operations agents** let you interact with data by using natural language and act on conditions and patterns found.
- **GraphQL APIs** provide a flexible data-access layer for developers to query multiple Fabric data sources through a single endpoint.
- **Copilot for Power BI** uses generative AI for ad-hoc analysis, DAX generation, and natural-language data exploration.

For more information, see Analyze and train data in Microsoft Fabric.

Track and visualize data

Use the prepared and modeled data to create reports, dashboards, and real-time alerts:

- **Power BI reports** provide interactive data visualization built on semantic models, with distribution across Microsoft 365 apps like Teams, SharePoint, PowerPoint, and Excel.
- **Translytical taskflows** enable users to take action directly from Power BI reports by calling user data functions.
- **Real-Time Intelligence dashboards** monitor streaming data with sub-second latency by using KQL queries and visual authoring.
- **Activator** detects conditions in streaming data and triggers automated actions such as Teams alerts, emails, or Power Automate flows.
- **Fabric IQ** maps enterprise data to a shared business ontology and enables AI agents to reason over your data with full business context.

For more information, see Track and visualize data.

Govern data

The OneLake catalog is the central place for discovery, governance, and security across the Fabric data estate. Its Govern section provides insights into the data estate and recommended actions to improve its governance posture. Microsoft Purview provides information protection, data loss prevention, audit, and risk and compliance capabilities for supported Fabric data.

External integration

Fabric integrates with external systems for both data ingestion and insights delivery:

- **Power Automate** and **Data Activator** enable real-time workflow automation based on data conditions.
- **Microsoft 365** integration surfaces insights in Teams, SharePoint, PowerPoint, and Excel.
- **REST APIs** and **client libraries** provide programmatic access to Fabric resources.
- **Microsoft Entra ID** handles authentication, conditional access, and service principal support.
- **Git integration** with Azure DevOps and GitHub enables version control and CI/CD for Fabric items.

For more information, see External integration and platform connectivity.

Natural language and AI support

Natural language support comes in the form of Power BI Copilot, Data Agents, and Operations Agents, which can reason over enterprise data in OneLake and produce answers based on the data items that users can access. You can integrate Data Agents into Microsoft 365 Copilot, Microsoft Foundry, and Copilot Studio so users can get insights from OneLake within their existing workflows across different applications.

Related content

- Get data into Microsoft Fabric
- Store data in Microsoft Fabric
- Prepare and transform data
- Analyze and train data in Microsoft Fabric
- Track and visualize data
- Govern in the OneLake catalog
- External integration and platform connectivity

ⓘ**Note:** The author created this article with assistance from AI. Learn more

Last updated on 09/29/2026

Get data into Microsoft Fabric

Microsoft Fabric provides multiple ways to bring data into your analytics environment. Whether you need to process streaming events in real time, replicate operational databases, orchestrate batch pipelines, or access data without copying it, Fabric offers built-in capabilities to support each scenario. Fabric also supports governed data sharing patterns through OneLake, enabling cross-tenant and cross-workspace access to live datasets without duplication.

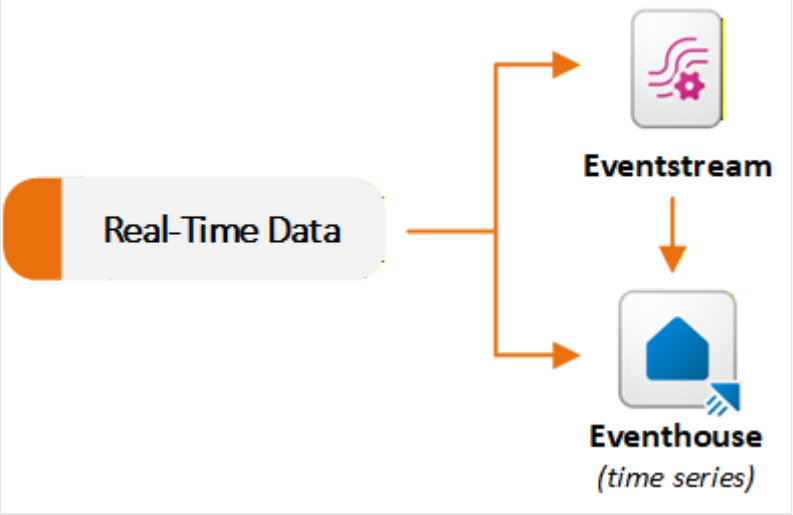
This article describes the primary data ingestion and data movement options in Fabric. It covers:

- **Real-time ingestion** with Eventstreams and Eventhouse
- **Batch orchestration** with Data Factory pipelines and Copy job
- **Near real-time replication** with Mirroring
- **Data virtualization** with OneLake shortcuts

Use this overview to understand how each approach works and choose the strategy that best fits your workload requirements for latency, transformation, and operational complexity.

Real-time data ingestion

Eventstreams and Eventhouse items in the Real-Time Intelligence workload support streaming data scenarios. Eventstreams ingest and process real-time events, and Eventhouses store and query those events at scale. You typically use an Eventstream to capture and route data to an Eventhouse. You can also use each capability independently based on your requirements. The following diagram shows how real-time datasets flow to Eventstream and Eventhouse in Fabric:



Ingest and route events with Eventstream

Eventstream provides a no-code experience to ingest events into Fabric, apply in-stream transformations, and route data to multiple destinations. An Eventstream acts as a real-time ingestion pipeline. You create an Eventstream and add one or more source connectors. Fabric supports many streaming sources, including internal Fabric events such as Fabric workspace events, OneLake file events, and pipeline job events.

After events start to flow, you can apply optional real-time transformations through a drag-and-drop editor. For example, you can filter events, compute time-window aggregates, join multiple streams, or reshape fields without writing code.

You can send the processed stream to one or more supported destinations. Eventstreams can expose Apache Kafka endpoints through custom endpoint sources and destinations. This capability enables Kafka producers to stream events into Fabric and Kafka consumers to consume events from Fabric.

Eventstreams do not store data permanently. They stream events through memory and forward them to configured destinations. This design makes Eventstreams suitable for real-time extract, transform, load (ETL) scenarios and for distributing streaming data to multiple targets. For example, you can ingest telemetry from Internet of Things (IoT) sensors, filter and aggregate data in real time, send the refined stream to an Eventhouse for analytics, and route anomaly events to Activator for alerting.

Ingest data directly into Eventhouse

Eventhouses can ingest data directly from multiple sources. Fabric includes an integrated **Get data** experience within Eventhouse. The wizard connects to sources such as local files, Azure Storage, Amazon S3, Azure Event Hubs, and OneLake. You can load data into a Kusto Query Language (KQL) database table in real time or batch mode by using the Eventhouse user interface.

You can also select an existing Eventstream in Fabric as a source. For example, if you use an Eventstream that ingests data from IoT Hub or Kafka, you can route its output directly to a KQL database table without additional configuration.

Batch data ingestion

Data Factory provides the primary experience for traditional extract, transform, load (ETL) and extract, load, transform (ELT) pipelines. It includes a large connector library. Fabric Data Factory provides a list of native connectors for on-premises and cloud data stores, including databases, software as a service (SaaS) applications, and file-based systems. These connectors help you connect to almost any source system.

Orchestrate data movement with pipelines

You can build pipelines that use these connectors to copy or move data into OneLake or analytical stores. This approach supports:

- Unstructured datasets such as images, video, and audio
- Semi-structured datasets such as JSON, CSV, and XML
- Structured datasets from supported relational database systems

In a pipeline, you combine multiple orchestration components, including:

- Data movement activities, such as Copy data and Copy job
- Data transformation activities, such as Dataflow Gen2, Delete data, Fabric Notebook, and SQL script
- Control flow activities, such as ForEach, Lookup, Set Variable, and Webhook

You can run a pipeline on demand, on a schedule, or in response to events. For example, you can schedule a pipeline to run every two hours during weekdays or trigger it when a new file is created in OneLake.

Simplify data movement with Copy job

Copy job supports multiple data delivery patterns, including bulk copy, incremental copy, and change data capture (CDC) replication. You can use Copy job to move data from a source to OneLake without creating a pipeline, while still accessing advanced configuration options. Copy job supports many sources and destinations. It offers more control than Mirroring and less operational complexity than managing pipelines that use the Copy activity.

Replicate data with Mirroring

Mirroring replicates data from external systems into Fabric in near real time with automated setup. You connect to an external system, such as Azure SQL Database, SQL Server, Oracle, SAP, or Snowflake. Fabric continuously replicates data or metadata into OneLake. Mirroring supports three types:

- **Database mirroring** replicates entire databases and tables.
- **Metadata mirroring** synchronizes metadata such as catalog names, schemas, and tables instead of physically moving data. This approach uses shortcuts so that data remains in its source system while still being accessible in Fabric.
- **Open mirroring** uses the open Delta Lake table format. Developers can write application changes directly to a mirrored database item in OneLake by using public APIs.

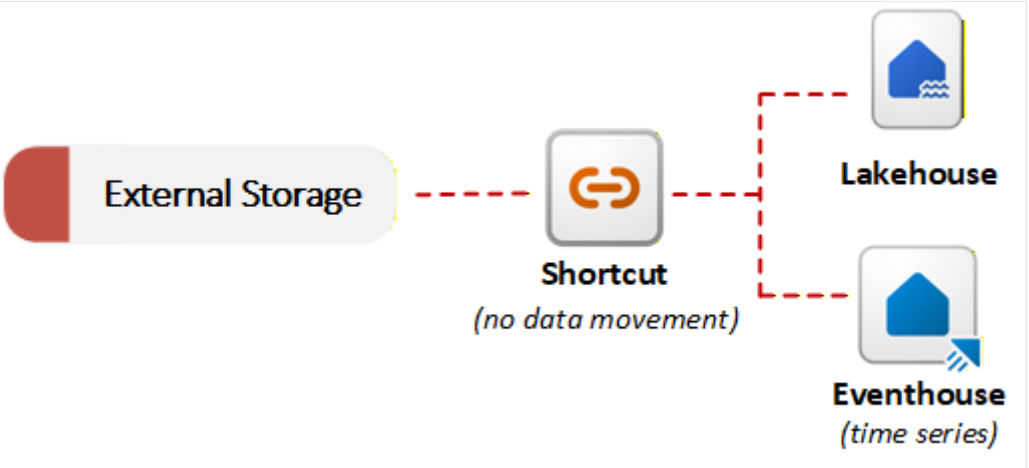
Fabric listens for source-system changes (through change data capture or similar methods) and applies those changes in near real time to the mirrored copy. The result is a live, queryable dataset that stays in sync with low latency, without a complex ETL pipelines.

Mirroring currently supports various sources, including Azure SQL Database, SQL Managed Instance, Azure Cosmos DB, Azure Database for PostgreSQL, Google BigQuery, Oracle, SAP, Snowflake, and SQL Server. It also supports data sources from partner solutions that have implemented the Open Mirroring API. Mirrored data is stored in OneLake as up-to-date Delta tables. Fabric maintains these tables automatically so that you can use them for real-time analytics or combine them with other Fabric data. This capability supports hybrid transactional and analytical processing scenarios, where operational data continuously flows into your analytics platform.

Mirroring removes the need to build incremental load pipelines manually. From a Mirroring cost perspective, compute operations that keep mirrored databases in sync don't use Capacity Units (CUs) from your Fabric capacity. Mirrored data storage in OneLake is also free up to the terabyte limit in your Fabric SKU (for example, F64 includes 64 TB of free mirrored database storage).

Access external data with shortcuts

Fabric provides shortcuts to enable data virtualization. A shortcut in OneLake references data stored in an external system, such as Azure Data Lake Storage Gen2, Amazon S3, or SharePoint. Shortcuts can also reference data within OneLake itself, including data from other workspaces and data shared across tenants through OneLake data sharing. Instead of copying data, shortcuts allow OneLake to reference external and internal files as part of the unified data lake. You can query or join external data with local data without performing an initial migration. This no-copy ingestion approach is useful when data residency requirements or duplication concerns prevent moving data. The following diagram shows how shortcuts connect external storage systems to Fabric items without copying data:



OneLake can detect the data type referenced by a shortcut and apply either file transformations or AI transformations without requiring a pipeline or custom code. These transformations work on any shortcut target, including data shared from other tenants through OneLake data sharing. OneLake maintains the resulting Delta table in sync with the source automatically. For example, you can convert `.csv` files to Delta tables or apply AI-based sentiment analysis to `.txt` files in a folder.

Combined with Mirroring, shortcuts give you flexible data access patterns. You can keep data in place by using shortcuts, or you can replicate data by using mirroring. In both cases, data is ready for Fabric analytics tools without complex ETL.

Decision guide: Choose a data movement strategy

Microsoft Fabric provides several options for bringing data into Fabric, including Eventstreams for real-time processing, Mirroring, pipelines with Copy activities, Copy job, and shortcuts. Each option offers a different balance of control, automation, and operational complexity. When you need live, governed access to data that's already in OneLake (whether in the same tenant or shared from another organization), consider combining OneLake data sharing with shortcuts instead of replicating data.

For guidance on selecting the appropriate approach for your scenario, see Microsoft Fabric decision guide: Choose a data movement strategy.

Related content

- End-to-end data lifecycle in Microsoft Fabric
- Store data in Microsoft Fabric
- Prepare and transform data
- Analyze and train data in Microsoft Fabric
- Track and visualize data
- External integration and platform connectivity

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 03/31/2026

Store data in Microsoft Fabric

Microsoft Fabric provides multiple storage options designed to support analytics, real-time processing, and operational reporting within a unified platform. Choosing the right storage experience helps you optimize performance, manage costs, and align your data architecture with workload requirements. Regardless of its source or preparation method, all data lands in a unified storage foundation called OneLake.

This article explains how data is stored in Fabric and describes the core storage experiences available. The following sections cover:

- **OneLake** – The unified, logical data lake that underpins all Fabric workloads.
- **Lakehouse** – Store and analyze structured and unstructured data using Delta tables.
- **Warehouse** – Store relational data optimized for high-performance SQL analytics.
- **Eventhouse** – Store and query high-volume, real-time event data.
- **Databases and other storage experiences** – Understand additional storage capabilities available within Fabric.

Use this overview to understand how each storage option works and choose the best fit for your analytical and operational scenarios.

Lakehouse for flexible data storage

A Lakehouse is a core storage item in Fabric that uses OneLake to store data in both file and table formats. A Lakehouse represents a curated folder structure in OneLake and includes a SQL interface. A Lakehouse stores data as Delta Parquet files. You can organize raw files such as CSV files or images in folders, and you can create managed Delta tables for structured data. This model supports both structured and unstructured data in the same environment.

Fabric automatically provisions a SQL analytics endpoint for each Lakehouse. You and tools such as Power BI can query Delta tables by using Transact-SQL, as if querying a relational database. The Lakehouse combines the scalability and flexibility of a data lake with core warehouse capabilities, including direct table querying and schema management.

Warehouse for structured analytics

A Warehouse in Fabric provides a traditional SQL data warehouse experience (with tables, SQL views, stored procedures, and more) on Fabric's unified storage. When you create a Warehouse, it stores data in OneLake in Delta format as an organized set of Delta tables with an ANSI SQL interface on top. The Warehouse provides dedicated compute and fine-tuned

performance for complex SQL queries and BI-style workloads. It supports features like indexing, stored procedures, and robust ACID transactions on tables.

The Warehouse and Lakehouse share the same underlying OneLake storage. You can integrate them by using shortcuts or other interoperability features when needed. However, you typically keep them separate for different use cases. The Warehouse is ideal for structured, relational star-schema data that you need to slice and dice with SQL. You can use Fabric pipelines to load data into the Warehouse. Power BI can connect by using Direct Lake or DirectQuery to retrieve data without import.

Decision guide: Lakehouse vs. Warehouse

Warehouses and Lakehouses serve distinct but complementary roles.

- Warehouses are optimized for structured, enterprise-scale data warehousing with full T-SQL support, ACID transactions, and strong schema enforcement—ideal for BI and reporting. Choose a Warehouse for governed, high-performance SQL workloads and a Lakehouse for big data processing, exploratory analytics, and scenarios that involve varied data formats or external lake integration.
- Lakehouses offer flexible, scalable storage for both structured and unstructured data, supporting Spark-based data engineering and read-only SQL analytics through automatic endpoints.

Many organizations benefit from using both together: Lakehouses for ingestion and transformation, and Warehouses for refined analytics and reporting. To learn more, see the decision guide.

Mirrored databases for near real-time replication

A mirrored database in Fabric is a continuously replicated copy of an external operational database, such as Azure SQL Database, SQL Server, Azure Cosmos DB, or Snowflake. Fabric stores mirrored data in OneLake in Delta Lake format.

Mirroring synchronizes source changes into Fabric in near real time without requiring traditional extract, transform, load pipelines. After replication, the data becomes immediately queryable through SQL endpoints and is available across Fabric workloads, including Power BI, Spark notebooks, and pipelines.

This architecture supports hybrid transactional and analytical processing (HTAP) scenarios, where you analyze operational data while maintaining source system integrity. If the source data is already stored in a location accessible through OneLake shortcuts (such as Azure Data

Lake Storage or another Fabric workspace), consider using shortcuts for zero-copy access instead of mirroring. Mirroring is best suited for operational databases that require continuous change data capture, while shortcuts are ideal when you need live, read-only access without replication.

OneLake shortcuts for zero-copy data access

OneLake shortcuts are logical links that reference data in external storage systems or in other Fabric workspaces without copying it. Shortcuts make referenced data appear as part of the local OneLake namespace, so all Fabric compute engines (Spark, SQL, Power BI) can query shortcut targets alongside native data. This approach maintains a single version of truth and avoids storage duplication.

You can also use OneLake data sharing to extend shortcut access across Microsoft Entra tenant boundaries. Data owners grant OneLake permissions to external identities, and recipients create shortcuts to the shared data in their own workspaces. Governance policies remain enforced at the source. For more information, see OneLake shortcuts and external data sharing.

Eventhouse for real-time event analytics

An Eventhouse provides a scalable real-time analytics environment designed to ingest, store, and analyze high volumes of event data. It is the foundational engine for Real-Time Intelligence workloads.

An Eventhouse hosts one or more Kusto Query Language databases based on the Kusto engine. These databases automatically index and partition data by ingestion time. You query data by using Kusto Query Language.

Eventhouse is well suited for telemetry, security logs, compliance records, and financial transactions where low-latency analytics and high-scale ingestion are required.

SQL database for transactional workloads

SQL databases in Fabric support transactional and operational analytics workloads. They provide a fully managed relational database experience with support for T-SQL, including data definition (DDL), manipulation (DML), and querying (DQL) capabilities. You can use stored procedures, views, and functions to build transactional and analytical solutions.

SQL databases use an automatic mirroring service to replicate transactional tables into OneLake for analytics. When you create a SQL database, Fabric starts a replication engine that captures insert, update, and delete operations through the SQL engine change feed and writes

those changes into OneLake as Delta Parquet files. Replication occurs in near real time and starts automatically. All supported tables are mirrored by default. This behavior ensures that the OneLake copy remains synchronized with the operational database.

SQL databases integrate with other Fabric experiences such as Power BI, notebooks, user data functions, pipelines, and external tools through the TDS protocol. This integration enables you to build end-to-end solutions, from data ingestion and transformation to visualization and reporting, without leaving the Fabric environment. The platform automatically handles indexing and performance optimization, so you don't need to manually tune or manage infrastructure.

Cosmos DB for distributed NoSQL workloads

Cosmos DB in Microsoft Fabric is a fully managed, distributed NoSQL database designed for high-throughput and globally distributed applications. It supports flexible schema models and semi-structured JSON data.

Cosmos DB is automatically mirrored into OneLake in Delta format to support analytics without affecting operational performance. Replication is continuous and near real time and requires no manual configuration.

After replication, data becomes accessible through a SQL analytics endpoint. You can query data by using Transact-SQL, create views, and integrate with Power BI, notebooks, and pipelines.

The SQL analytics endpoint provides a read-only interface to the mirrored data, ensuring that analytical queries don't interfere with transactional operations. This architecture supports hybrid transactional and analytical processing (HTAP), so you can unify operational and analytical workloads within a single platform.

Semantic model for business logic and reporting

Semantic models provide the structured, curated layer that defines business logic, measures, hierarchies, relationships, and metadata on top of raw data in Microsoft Fabric. They make data interpretable and reusable across the platform for analytics experiences.

Semantic models in Fabric are tightly integrated with the platform's capacity model and workspace structure. Semantic models support three query modes: Import, DirectQuery, and Direct Lake. Each mode offers different trade-offs between performance, freshness, and scalability:

- **Import mode** copies data from the source into the semantic model during scheduled or manual refreshes. This mode offers the fastest query performance because Power BI

operates on in-memory data, but it introduces latency between source updates and report visibility. Import mode is ideal for high-performance dashboards where real-time data isn't critical.

- **DirectQuery mode** sends queries directly to the source system at runtime without storing data in the semantic model. This approach ensures up-to-date results but can lead to slower performance depending on the source system's responsiveness. DirectQuery is suitable for scenarios where data freshness is more important than speed, such as operational reporting.

- **Direct Lake mode** allows Power BI to query Delta tables stored in OneLake directly. It combines the performance characteristics of Import with the freshness of DirectQuery. It avoids data duplication and uses the lake-native architecture for scalable, near real-time analytics. Direct Lake is recommended for large-scale analytics on Fabric-managed data.

Semantic models also enable conversational AI, semantic search, enterprise reporting, and cross-domain reasoning by bringing together advanced features like Fabric Data Agents, Power BI Copilot, Ontologies, and Power BI reports. Business users can also access semantic models through Excel, where they can explore data and insights in a PivotTable interface that uses live data from the semantic model.

Decision guide: Choose the right data store

Microsoft Fabric provides multiple data store options, each optimized for specific workloads:

- **Lakehouse** for large-scale data engineering and open-format storage like Delta and Iceberg, with support for Spark and SQL engines.
- **Warehouse** for structured, relational analytics with high-performance SQL capabilities and enterprise reporting.
- **Eventhouse** for real-time telemetry and log analytics by using Kusto Query Language.
- **SQL database** for transactional workloads and operational analytics.
- **Cosmos DB** for globally distributed NoSQL applications, multi-model applications with low-latency access.
- **OneLake shortcuts** for zero-copy access to data in external storage or other Fabric workspaces and tenants, when you don't need a separate copy and want to maintain a single version of truth.

Selecting the appropriate store depends on data structure, latency requirements, query complexity, and integration needs. When the data you need already exists in an accessible location, shortcuts can eliminate the need for replication entirely. For more guidance, see Choosing the right store.

Related content

- End-to-end data lifecycle in Microsoft Fabric
- Get data into Microsoft Fabric
- Prepare and transform data
- Analyze and train data in Microsoft Fabric
- Track and visualize data
- External integration and platform connectivity

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 03/31/2026

Prepare and transform data in Microsoft Fabric

Summarize this article for me

After you ingest data into Microsoft Fabric, you typically need to clean, shape, and enrich it before analysis. Whether your goal is to prepare curated tables in a lakehouse or model-ready data in a warehouse, Fabric provides both low-code and code-first transformation options.

This article describes how to use Dataflow Gen2 for visual, low-code data preparation and how to use notebooks and user data functions for advanced, code-driven transformations. Choose the approach that best fits your role, skill set, and workload requirements.

Transform data with Dataflow Gen2

For low-code data preparation, use Dataflow Gen2. Dataflow Gen2 uses the familiar Power Query experience, the same technology used in Excel and Power BI.

With the Power Query interface, you can apply filters, derive columns, aggregate data, merge queries, and perform other transformations through a visual, step-by-step workflow. In Fabric, Dataflow Gen2 can run as a standalone ETL process or as an activity within a pipeline.

For example, after you ingest raw sales data into a Lakehouse, you can use a dataflow to remove duplicates, standardize column names, apply business rules, and write the cleaned results to curated tables in a Gold layer of the Lakehouse or into a Warehouse.

Dataflow Gen2 runs in the cloud by using Fabric capacity, enabling it to scale to large datasets and complex transformations without requiring custom code. Data analysts and BI developers can prepare data independently, while still writing output to Lakehouse or Warehouse tables as part of Fabric's unified storage foundation.

Code-first preparation with notebooks and user data functions

For advanced transformation scenarios with code, use notebooks, Spark jobs, and user data functions in the Data Engineering experience.

A Fabric notebook provides a Jupyter-style environment in the Fabric portal. You can write code in languages such as Python, T-SQL, or Scala to work with data stored in OneLake.

Notebooks are well suited for complex transformations, custom algorithms, data science workflows, and integration with external libraries. For example, you can load raw JSON or Parquet files from a lakehouse into a Spark DataFrame, join them with other datasets, apply windowed aggregations, enrich the data, and save the results back as Delta tables in OneLake.

Notebooks integrate directly with lakehouses and warehouses in the same workspace. You can read and write data without additional credential configuration because operations run within the Fabric security context. You can also orchestrate and schedule notebooks by using the notebook activity in Data Factory pipelines.

Fabric user data functions enable you to encapsulate reusable Python logic in Fabric. You can use them to implement advanced business rules, call external services, or build modular transformation components. User data functions support PyPI libraries, can connect to Fabric data sources, and can expose REST endpoints for external integration. These capabilities make them suitable for enterprise scenarios that require reusable, governed transformation logic.

You can invoke User Data Functions from Notebooks, Pipelines, Activator rules, and as part of Translytical task flows in Power BI reports.

Related content

- End-to-end data lifecycle in Microsoft Fabric
- Get data into Microsoft Fabric
- Store data in Microsoft Fabric
- Analyze and train data in Microsoft Fabric
- Track and visualize data
- External integration and platform connectivity

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 03/13/2026

Analyze and train data in Microsoft Fabric

Summarize this article for me

Microsoft Fabric provides tools for advanced analytics, machine learning (ML), and AI model operationalization, all within a single unified platform. The Data Science workload is designed for data scientists and analysts to explore, prepare, and analyze data, build and track ML models, and operationalize AI workflows. Fabric IQ Data Agents, Operations Agents, and Copilot in Power BI enhance interaction with data through natural language, automation, and insight-driven actions.

In this article, you'll learn about:

- AI agents for conversational analytics and operational automation
- Data science workflows for model training, tracking, and deployment
- Developer and user access options with GraphQL and Copilot

AI agents

AI agents in Microsoft Fabric help teams move from passive reporting to active decision support. Data Agents make governed data easier to explore through natural-language questions, while Operations Agents monitor business conditions and trigger actions when rules are met. Together, they connect insights and automation so teams can respond faster, reduce manual effort, and make decisions with more context.

Data agent

Fabric Data Agents allow conversational Q&A over enterprise data using generative AI. Users can ask plain-English questions and get structured, secure, read-only answers without needing SQL, DAX, or KQL. Data Agents use Azure OpenAI Assistant APIs to identify relevant OneLake data sources, including Lakehouses, Warehouses, Power BI semantic models, KQL databases, and ontologies. You can configure agents with custom instructions, examples, and domain-specific guidance to improve response relevance.

Data Agents integrate with Microsoft Foundry, Copilot Studio, and Microsoft 365 Copilot to extend capabilities from conversational analytics to AI workflows:

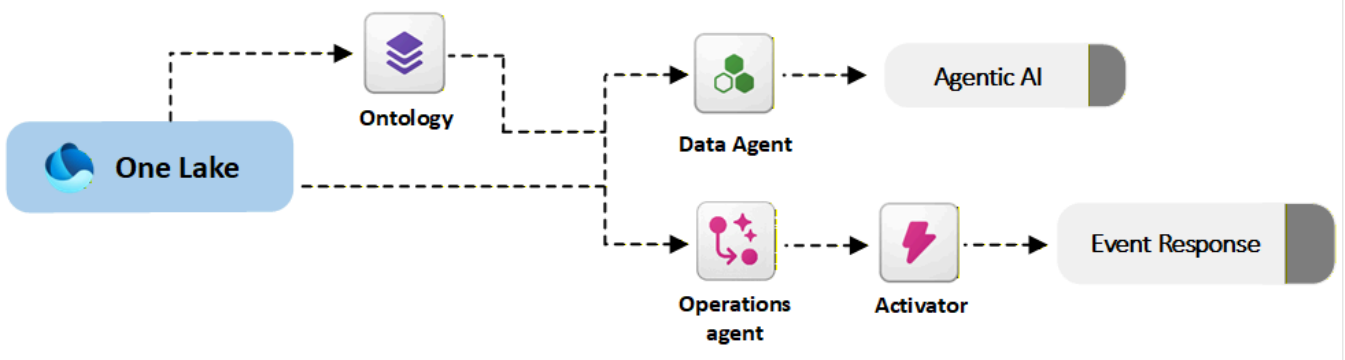
- Foundry IQ provides a shared context layer where Data Agents contribute structured business insights alongside other agents, enabling multistep reasoning and orchestration across enterprise systems.

- Copilot Studio lets you embed these agents as custom skills in Teams, web apps, or line-of-business applications, injecting live business context into Copilot prompts and combining Q&A with workflow automation.
- The integration with Microsoft 365 Copilot lets these agents surface governed, ontology-driven insights directly within productivity tools like Outlook, Excel, and Teams, combining conversational analytics with workflow automation.

Operations agent

Operations Agents are autonomous, ontology-driven AI components that monitor real-time data streams, interpret events, and execute or recommend actions. They use the ontology to apply rules and objectives, enabling proactive decision-making rather than reactive responses. They integrate with Activator and Power Automate to trigger workflows in ERP, CRM, and other systems, while Teams provides alerts and human approvals. Unlike Data Agents, which focus on answering questions, Operations Agents continuously act on live conditions, learning from results to improve future decisions and transform operations into adaptive, context-aware automation.

The following diagram shows how Data Agents and Operations Agents in Fabric IQ use governed enterprise data and automation services to deliver insights and trigger actions.



Choose between data agents and operations agents

Data Agents and Operations Agents in Microsoft Fabric IQ serve distinct roles. Data Agents provide conversational analytics by answering user questions in natural language, using ontology for semantic grounding and querying multiple sources like Lakehouses, Warehouses, and Power BI models. They integrate externally through Teams, Copilot Studio, and custom apps for insight delivery.

In contrast, Operations Agents focus on autonomous decision-making. They monitor real-time data streams against ontology-based rules to trigger or recommend actions. They integrate with Power Automate (through Activator), Teams for alerts and approvals, and external

operational systems like ERP or CRM. Data Agents democratize data access for insights, while Operations Agents drive proactive, governed automation for operational optimization.

Data science workflows

Fabric Data Science covers the full ML lifecycle: data exploration, preparation, model experimentation, tracking, deployment, and consumption. Tools you need include notebooks, Apache Spark, MLflow, and AutoML, all within a unified platform. Data scientists can develop and operationalize ML models alongside data engineers and analysts in one place.

Track experiments with MLflow

Experiments in Microsoft Fabric organize and track model training runs. An experiment in Fabric works like a MLflow experiment, it contains a collection of runs, where each run is one execution of model training code. Because Fabric integrates with MLflow, every run can automatically log relevant information such as hyperparameters, metrics, tags, code version, and output items without requiring custom logging code. MLflow tracking is natively built into Fabric's notebooks and Spark jobs, so data scientists can use MLflow APIs or Fabric's UI to create experiments and record runs.

Register and deploy ML models

ML models in Fabric are registered machine learning models. Fabric's model management uses MLflow-powered registries to store, version, and track models. After selecting the best experiment run, register the model in Fabric to store metadata like hyperparameters, metrics, and environment details. Models are saved in a standardized MLflow format, which enables interoperability across Spark and Python environments.

Models can be deployed for batch scoring in Spark or through real-time endpoints for low-latency predictions.

The following diagram shows the end-to-end Data Science workflow in Fabric, from data preparation and experimentation to model registration and deployment.

Developer data access with GraphQL

The API for GraphQL provides a single, flexible endpoint to query multiple Fabric data sources, including Warehouses, SQL Databases, Lakehouses, and mirrored databases. It supports schema discovery, generated queries, relationship modeling, and interactive query testing. It makes it easier to expose specific tables, views, and fields while enabling fast, client-driven data access across Fabric environments.

Copilot in Power BI

Copilot in Power BI enables natural-language data interaction. Users can explore data, generate insights, create visuals, and generate DAX expressions.

The standalone Copilot experience supports cross-item conversational analysis, automatically selecting a relevant data source such as a report, semantic model, or Fabric data agent that users can access. It asks clarifying questions when needed and can immediately deliver insights once it selects the right report or model. Preparing data for AI and approving semantic models improves accuracy and ensures high-quality responses.

Related content

- End-to-end data lifecycle in Microsoft Fabric
- Get data into Microsoft Fabric
- Store data in Microsoft Fabric
- Prepare and transform data
- Track and visualize data
- External integration and platform connectivity

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 03/13/2026

Track and visualize data

Turning raw data into actionable insights is a key step in the data lifecycle. Microsoft Fabric combines Power BI, Real-Time Intelligence, and Fabric IQ to help business analysts explore, visualize, and respond to data in near real time. Using semantic models, streaming data, and AI-enhanced analytics, you can create governed reports, dashboards, and operational alerts that drive informed decision-making.

In this article, you'll learn about:

- Creating Power BI reports and translytical taskflows for interactive analysis
- Using Real-Time Intelligence to monitor and act on streaming data
- Applying Fabric IQ business semantics, ontologies, and graph models to unify enterprise data

Explore and share insights with Power BI

Power BI reports use semantic models as the analytical backbone. These models centralize business logic, calculations, and governance, ensuring consistency across reports and teams. Business analysts can build interactive dashboards, drill into metrics, and share insights securely across the organization. Use Power BI when you need structured, reusable, and governed analytics built on semantic models stored in OneLake.

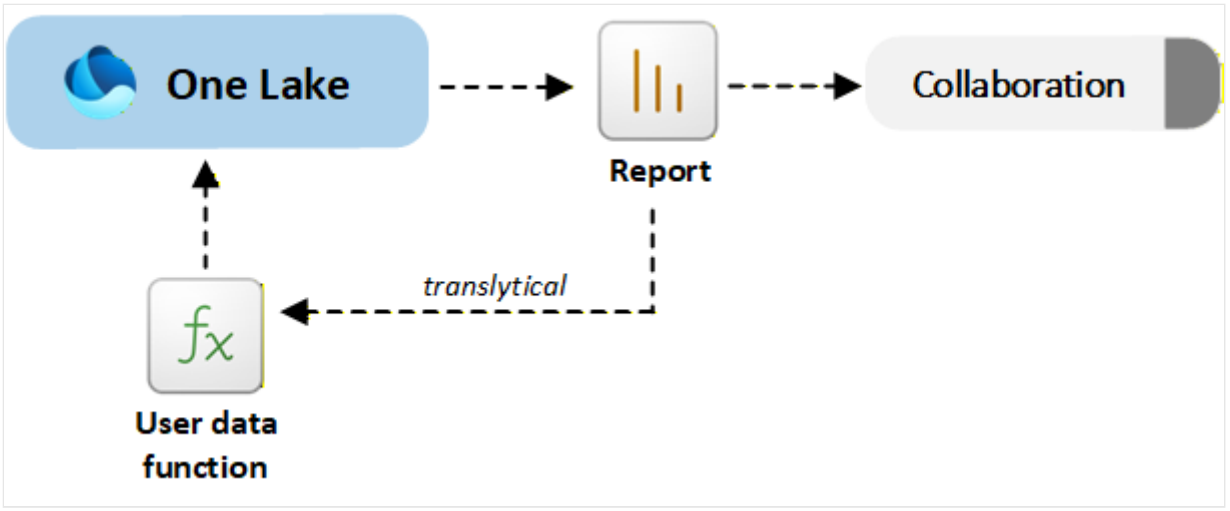
You can distribute and embed Power BI reports across Microsoft 365 applications:

- Pin reports in Microsoft Teams channels or chats for collaborative analysis.
- Embed reports in SharePoint pages using the Power BI web part.
- Insert live reports into PowerPoint presentations.
- Connect to Power BI datasets from Excel to create PivotTables and charts on top of semantic models.

Take action from reports with translytical taskflows

Translytical taskflows allow you to move from insight to action directly inside a Power BI report. You can invoke user data functions to update records, trigger workflows, or write back to OneLake without leaving the report interface. Use translytical taskflows when insights and operational updates must happen together, such as adjusting inventory, updating sales forecasts, or resolving support cases.

The following diagram shows how Power BI reports and translytical taskflows combine analytics and action in a single, governed experience:



Monitor and analyze real-time data

Real-Time Intelligence with Eventhouse provides a unified analytics surface that spans KQL, T-SQL through the SQL analytics endpoint, and notebooks, enabling end-to-end real-time-to-historical analysis on the same data without copies or data movement.

The Real-Time Intelligence workload in Microsoft Fabric enables you to ingest, process, analyze, and act on streaming data with minimal latency. It combines capabilities from the following services:

Ingest streaming data with Eventstream

Eventstream ingests streaming data from sources such as Azure Event Hubs, Kafka, IoT devices, or REST APIs. It applies real-time transformations and routes data into services like Eventhouse or Lakehouse.

Eventstream supports routing and schema alignment, timestamp normalization, partitioning, and integration with Activator. As Eventstreams feed high-velocity data into Fabric, Activator continuously monitors these streaming events against defined rules in near real time.

Analyze events with Eventhouse

Eventhouse is optimized for high-volume event and time-series analytics. It automatically indexes and partitions data and uses Kusto Query Language for fast filtering, aggregation, joins, and anomaly investigation. Use Eventhouse when you need scalable analysis of telemetry, operational metrics, or log data.

Eventhouse also exposes a managed SQL analytics endpoint aligned with the Eventhouse data model and Fabric governance. The endpoint provides familiar T-SQL access (for example, T-SQL queries and connectivity from BI tools) on Eventhouse data without data movement.

In addition, Eventhouse integrates with Fabric notebooks, which support KQL, T-SQL, Python, and Spark. You can use notebooks to explore, transform, and build machine learning workflows on the same Eventhouse data, keeping work within Fabric and avoiding copies.

Integration with Activator happens through KQL querysets, which let you define reusable queries that continuously evaluate conditions on Eventhouse data. Activator can subscribe to these query outputs and apply rules based on the query results. When a KQL query detects a condition (for example, CPU usage exceeds 90% or a trend indicates failure risk), Activator triggers automated actions like sending alerts, launching Power Automate flows, or executing Fabric pipelines. This combination of KQL's analytical power with Activator's orchestration enables real-time, data-driven automation across business scenarios.

Visualize live metrics with real-time dashboards

Real-time dashboards provide live, interactive visualizations of streaming data, enabling you to monitor key metrics and operational signals as they happen. Built on top of Eventhouse (KQL databases), these dashboards let you query and display time-series data with minimal latency, offering insights into system performance, customer behavior, or sensor activity in real time. They support dynamic filtering, auto-refresh, and alerting capabilities, making them ideal for use cases like IT monitoring, manufacturing telemetry, financial transaction tracking, and customer support analytics. Real-time dashboards also integrate with Activator, so you can not only observe but also respond to critical events directly from the dashboard interface, turning insights into action without delay.

Real-time dashboards differ from regular Power BI reports primarily in their ability to visualize and respond to live streaming data with minimal latency. Power BI reports are typically built on imported datasets refreshed on a schedule and are best for historical analysis and interactive exploration. Real-time dashboards, on the other hand, are designed for operational monitoring and immediate insight.

Anomaly detection

The anomaly detector automatically identifies unusual patterns or outliers in streaming or time-series data. Detection runs natively against live Eventhouse tables and incoming streaming data without copying or exporting data, supporting both real-time detection and evaluations over

historical windows. It works by analyzing data ingested into Eventhouse (KQL databases) and applying statistical models or machine learning techniques to detect deviations from expected behavior. These anomalies could represent system failures, fraud, performance degradation, or other critical events that require attention. Unlike static threshold-based alerts, the anomaly detector adapts to the data's historical trends and seasonality, making it more effective at identifying subtle or context-sensitive issues.

When an anomaly is detected, it can trigger downstream actions through Activator, such as sending alerts, updating dashboards, or launching automated workflows. This capability supports operational scenarios where early detection of anomalies can prevent downtime, financial loss, or customer dissatisfaction. It enables you to move from reactive monitoring to proactive intervention across business processes.

Use anomaly detection when static thresholds are not sufficient and patterns must be evaluated dynamically.

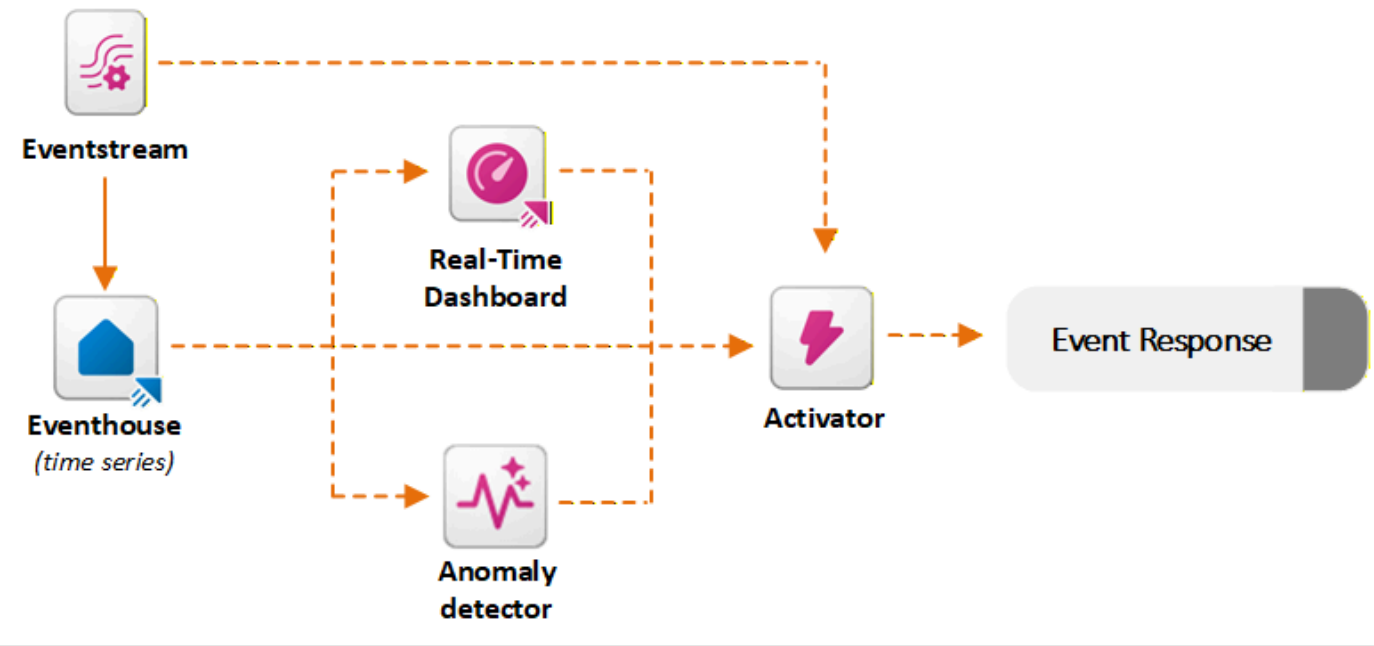
Automate responses with Activator

Activator continuously monitors streaming or event data and evaluates rules you define. It supports simple threshold rules and stateful pattern detection. For example, when a metric becomes critical or decreases over time. This capability enables you to automate decisions and operational tasks in near real time, bridging live data insights to instant actions. Use Activator to bridge real-time insights to automated operational responses.

When conditions are met, Activator can:

- Send alerts to Microsoft Teams or email
- Trigger Power Automate flows
- Launch Fabric pipelines or other Fabric items

The following diagram shows how Real-Time Intelligence in Fabric ingests streaming data, analyzes events, and triggers actions with low latency:



Apply business semantics with Fabric IQ

Fabric IQ provides a shared business context layer across your data estate. It maps data in Lakehouses, Warehouses, Eventhouses, and semantic models to a unified ontology.

Fabric Ontology is a shared, machine-understandable vocabulary of your business that defines the key entities (for example, Customer, Product, or Plane), their relationships, properties, business rules, and possible actions while maintaining all terms in the language of the business. It brings together a live, connected representation of how your business operates, mapped directly to the underlying data in OneLake. This model lets users and agents see not just tables, but also relationships like "Customers place Orders for Products," "Flights have Segments and Crews," and "Delayed shipments impact Revenue." This data-centric context is critical for any AI that's expected to make decisions or analyses about the business.

When you define an ontology element (such as an entity "Flight" with properties like Status or Delay), you map it to the table and fields in an Eventhouse, Lakehouse, or Warehouse that contain that information, without copying or moving the data. After the data lands in OneLake, it becomes part of the live ontology.

Beyond business entities and relationships, ontologies can also define actionable rules, such as "If inventory < threshold, trigger replenishment." Operations Agents use these rules to trigger workflows in Activator. Operations Agents can reason over both live streaming and historical data in Eventhouse mapped through the ontology, and they can invoke Activator with context derived from Eventhouse-backed entities. When an Operations Agent invokes Activator to run a Power Automate flow, it passes parameters derived from ontology properties, such as CustomerID and

OrderStatus. This approach ensures that automation flows operate with full business context, not just raw IDs.

This semantic layer enables consistent analytics, AI reasoning, and automation.

Explore connected data with graph models

Graph models provide a connected network of ontology-defined entities and relationships. They enable multi-hop reasoning, impact analysis, and advanced algorithms such as shortest path and community detection. This integration lets AI agents and analytics tools query complex relationships efficiently. It offers real-time insights into dependencies and cascading effects that traditional relational models struggle to handle. You can interrogate graph models by using GraphQL queries through Fabric APIs.

Related content

- End-to-end data lifecycle in Microsoft Fabric
- Get data into Microsoft Fabric
- Store data in Microsoft Fabric
- Prepare and transform data
- Analyze and train data in Microsoft Fabric
- External integration and platform connectivity

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/20/2026

Integrate Microsoft Fabric with external systems and platform connectivity

Microsoft Fabric integrates with external services to automate workflows, enable collaboration, support developer extensibility, and interoperate with other data platforms. You can trigger actions from real-time data, connect Fabric to Microsoft 365 apps, integrate with developer tools and APIs, and securely share data across cloud ecosystems by using OneLake's zero-copy access patterns.

This article describes how to:

- Trigger automated workflows from Fabric data
- Collaborate by using Microsoft 365 applications
- Integrate with AI agents and orchestration tools
- Access Fabric programmatically through developer tools and APIs
- Interoperate with open data formats and external platforms
- Connect Fabric to core Azure services for security, networking, and governance

Automate actions with Data Activator

Activator provides real-time automation by monitoring data streams and triggering actions when defined conditions occur. Activator integrates with:

- **Power Automate** to execute workflows such as sending notifications, updating records, or initiating business processes. You can create triggers in Fabric and link them to Power Automate flows to alert users when a KPI threshold is crossed.
- Microsoft Teams to post adaptive cards or messages to channels when real-time conditions are met.
- Outlook to send automated emails for alerts or approvals.
- **Fabric items** to initiate actions such as refreshing a semantic model or starting a pipeline run.

Collaborate across Microsoft 365

Microsoft Fabric works within the Microsoft 365 ecosystem, enabling collaborative analytics and data-driven workflows across familiar productivity tools. OneLake's zero-copy model means that

data shared through shortcuts or cross-tenant data sharing stays in place while remaining accessible across collaboration surfaces:

- Microsoft Teams: Share reports and dashboards in channels or chats. You can embed Power BI visuals in Teams tabs, collaborate on insights, and receive data-driven alerts.
- Excel: Use Analyze in Excel to connect PivotTables and advanced analysis tools to Fabric semantic models while maintaining live connectivity.
- PowerPoint: Embed or export Power BI reports into presentations so stakeholders can view live or snapshot analytics.
- SharePoint and OneDrive: Publish reports to SharePoint Online and store supporting files in OneDrive with Microsoft 365 security and compliance controls.

These integrations enable analytics to flow naturally into collaboration, review, and decision-making processes.

Integrate with agentic AI solutions

Fabric supports agent-based AI scenarios through a shared semantic foundation. This foundation integrates with three key components to deliver end-to-end agentic capabilities:

- **Microsoft Foundry** hosts fine-tuned models and knowledge retrieval systems. It supports integration through open standards like the Model Context Protocol (MCP). It can use Fabric Data Agents as part of an agentic AI workflow.
- **Copilot Studio** enables you to design conversational agents and automate processes that invoke Fabric Data Agents for governed insights.
- **Microsoft 365 Copilot** surfaces these capabilities in tools such as Teams and Outlook, routing queries to appropriate agents while respecting enterprise security controls. Microsoft 365 Copilot can consume Fabric Data Agents as part of an agentic AI workflow.

Together, these services extend Fabric from analytics into contextual AI automation.

Use developer tools and APIs

Fabric supports code-first development and DevOps practices.

- **Visual Studio Code**: Use Fabric extensions and CLI tools to create and manage items such as lakehouses, notebooks, and user data functions.

- **Connect to Fabric data sources**

- **SQL analytics endpoints**: Lakehouses and warehouses expose SQL connection strings so you can query data from standard SQL tools.
- **GraphQL APIs**: Programmatically access data and metadata to build custom applications and integrations.

These capabilities enable automation, CI/CD workflows, and custom solutions that extend Fabric beyond the web interface. You can also use Fabric REST APIs to programmatically create and manage OneLake shortcuts and consume shared datasets across workspaces and tenants.

Unified data estate

The unified data estate gives you a consistent foundation for cross-platform analytics. With open formats, API parity, and zero-copy access patterns, you can connect external engines while keeping governance and data ownership centralized in OneLake.

OneLake interoperability

OneLake provides an open and interoperable data foundation by supporting widely adopted standards. OneLake achieves interoperability through two key strategies:

- **Support for Delta Lake and Iceberg file formats**
  - **Delta Lake support**: Enables ACID transactions, schema enforcement, and time travel across large datasets.
  - **Apache Iceberg support**: Supports advanced table management features such as partitioning and schema evolution.
- **Universal data access through APIs**
  - **Table APIs**: Provide programmatic access to tables regardless of file format.
  - **ADLS Gen2 compatibility**: Supports the same REST APIs and hierarchical namespace features as Azure Data Lake Storage Gen2.

By using these open file formats and APIs, OneLake ensures that data stored in OneLake can be read and written by multiple compute engines without conversion, making it a true interoperable data foundation.

OneLake shortcuts

OneLake shortcuts are logical pointers that reference data stored in external systems (such as Azure Data Lake Storage, Amazon S3, or Google Cloud Storage) or in other Fabric workspaces. Shortcuts enable zero-copy access across Fabric workloads without duplication or ETL.

Referenced data appears as part of the local OneLake namespace, so all Fabric compute engines can query shortcut targets alongside native data.

Cross-tenant data sharing

External data sharing lets you share live, governed datasets across Microsoft Entra tenant boundaries without copying data. Data owners control access by granting OneLake permissions to external identities, and governance policies (such as sensitivity labels and access controls) remain enforced at the source. You can combine cross-tenant data sharing with shortcuts so that external recipients reference shared data as part of their own OneLake namespace.

Azure Databricks integration

The OneLake and Azure Databricks integration provides a foundation for unified analytics and AI. This integration combines OneLake's governed, open data estate with Databricks' data engineering and machine learning capabilities. The bidirectional integration between OneLake and Azure Databricks takes the following forms:

- **Catalog mirroring**: Mirrors Unity Catalog tables into OneLake as shortcuts, enabling Fabric workloads to discover and access Databricks-managed data. Use catalog mirroring when you want to bring Databricks datasets into Fabric for analysis with Fabric compute engines.
- **OneLake catalog federation**: Connects Unity Catalog to a OneLake catalog, automatically synchronizing schemas and tables so that Fabric lakehouse items appear in Unity Catalog alongside Databricks assets. Queries run on Azure Databricks compute while reading data from OneLake without copying (zero-copy). Governance remains centralized in OneLake. Use catalog federation when you want Azure Databricks to discover and query OneLake data with governed, table-level access and schema synchronization.
- **Direct access to OneLake**: Databricks can read and write OneLake data directly by using Delta Lake compatibility and ADLS Gen2 APIs. While direct access provides file-level read/write operations, catalog federation provides governed, table-level discovery and schema synchronization within Unity Catalog. Use direct access for operational read/write scenarios where file-level control is needed.

Snowflake integration

The OneLake and Snowflake integration delivers a zero-copy architecture that lets you unify analytics and AI without duplicating data. This approach is built on open standards and shortcut technology, ensuring interoperability while maintaining governance and performance.

- **Shortcuts to Snowflake Iceberg tables**: Fabric uses shortcuts that reference Snowflake tables directly in OneLake. Data remains in Snowflake but appears in the OneLake logical data estate. Fabric workloads (Power BI, Dataflows, AI agents) can query Snowflake data alongside native OneLake data without ETL or replication. Governance and security policies remain intact across both platforms.
- **Snowflake external tables over OneLake**: Snowflake can query OneLake data in Delta or Parquet formats, which enables high-performance queries on OneLake data from Snowflake compute.

Platform architecture

Azure

Microsoft Fabric is built as a software as a service (SaaS) platform on Azure's infrastructure, meaning it relies on core Azure services for identity, networking, security, monitoring, and more. In practice, a Fabric tenant and its capacities are first-class Azure resources under your Azure subscription. This allows Fabric to integrate with Azure's billing system, use Azure's monitoring and logging capabilities, enforce security through Azure identity and networking features, and interoperate with Azure data services.

Azure billing

Microsoft Fabric capacities represent the provisioned compute tiers for Fabric and are managed as Azure resources within your subscription. When you enable Fabric, you create an Azure Fabric capacity resource that represents the Fabric tenant's analytics capacity. In the Azure portal, you can pause and resume your capacity and scale it up and down as needed. Your Azure subscription's billing system handles billing for Fabric. All Fabric usage charges for compute and storage accrue under your subscription and appear in Microsoft Cost Management alongside other Azure services.

Networking

Microsoft Fabric supports Azure Private Endpoints for private, internal-only access to Fabric endpoints. When you enable Private Link for Fabric at the tenant level or workspace level, Fabric exposes private endpoint URLs for services (OneLake, Warehouse SQL analytics endpoints, and others) that are accessible only through your Azure Virtual Network. Traffic routes through Azure's private backbone network instead of the public internet. With **Block Public Internet Access** enabled in Fabric, attempts to use Fabric from outside your VNet (or without the private

endpoint) are rejected. This integration ensures that sensitive data transfer between Fabric and your network stays off the public internet, which satisfies stringent compliance requirements.

Microsoft Fabric also supports Managed Virtual Networks (Managed VNets) and Managed Private Endpoints to enhance network isolation and secure connectivity. When Private Link is enabled for a Fabric tenant or workspace, Fabric automatically provisions a Managed VNet for compute workloads such as Spark jobs, so that they run in an isolated environment rather than shared public pools. Managed Private Endpoints also let Fabric services (like Lakehouse, Warehouse, and OneLake) securely connect to Azure resources or on-premises data sources without exposing traffic to the public internet. These features use Azure's networking backbone to enforce compliance, reduce attack surface, and provide granular control over data ingress and egress within enterprise environments.

Microsoft Entra ID

Fabric authenticates and manages users through Microsoft Entra ID. Every Fabric tenant is linked to an Entra ID tenant. User sign-in, single sign-on (SSO), and access control for Fabric use the same identities and groups as Azure. You can use your corporate Entra ID accounts to sign in to Fabric and use existing security features like conditional access policies and multi-factor authentication across Fabric. Entra ID integration also enables service principals and managed identities for Fabric inbound and outbound authentication scenarios:

- **Inbound authentication**: Use service principals to access Fabric's admin APIs or to enable access to Fabric item endpoints such as SQL, OneLake, and Eventhouse.
- **Outbound authentication**: Use the Workspace identity or other service principals when setting up connections to Azure data sources.

GitHub and Azure DevOps

Microsoft Fabric integrates natively with Azure DevOps and GitHub to enable version control, CI/CD, and collaborative development for analytics items such as pipelines, notebooks, and dataflows. Workspaces in Fabric can be linked to a Git repository, which lets you synchronize items between Fabric and your source control system. This integration supports branching strategies, pull requests, and code reviews, so that changes to Fabric items follow the same governance and DevOps practices as application code. With Azure DevOps, you can automate deployment pipelines for Fabric content across environments (for example, dev, test, and prod) by using YAML-based workflows. GitHub integration uses GitHub Actions for automation, enabling continuous integration and deployment of Fabric assets. These capabilities help teams

maintain consistency, enforce version history, and streamline collaboration, making Fabric a first-class citizen in modern DevOps workflows.

Microsoft Purview

Microsoft Purview works with Microsoft Fabric to classify and protect sensitive data, help prevent data loss, audit user activity, and identify data risks. Supported capabilities include sensitivity labels, protection policies, Data Loss Prevention, Audit, Insider Risk Management, and security and compliance controls for Copilots and agents. Use OneLake Catalog to discover, govern, secure, and manage Fabric data.

Related content

- End-to-end data lifecycle in Microsoft Fabric
- Get data into Microsoft Fabric
- Store data in Microsoft Fabric
- Prepare and transform data
- Analyze and train data in Microsoft Fabric
- Track and visualize data

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

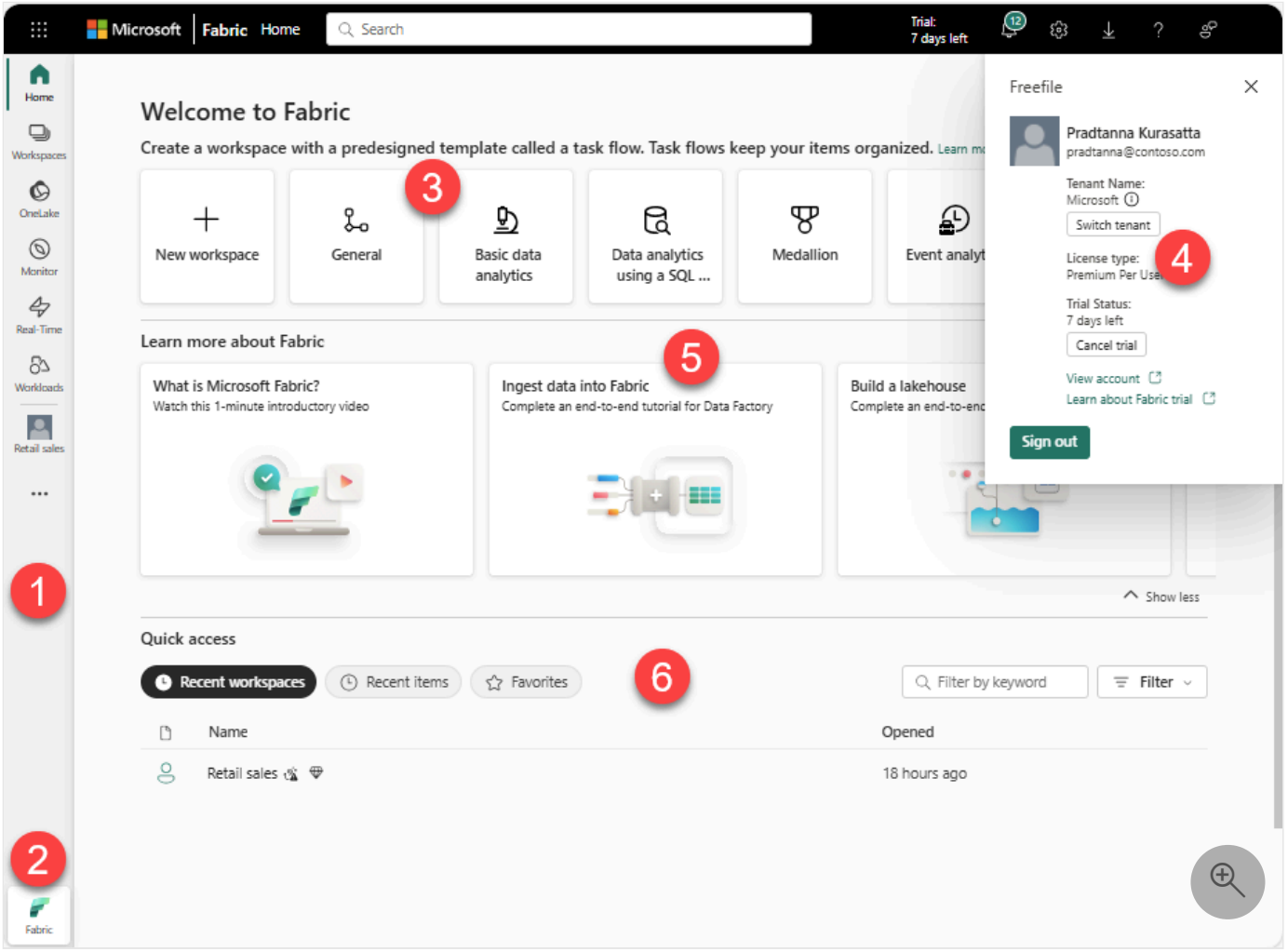
Last updated on 09/29/2026

Use Fabric Home to find, personalize, and manage content

Home is your personalized Fabric start page. Use it to find items and workspaces, multitask with tabs, personalize settings, and get contextual help to work faster.

Home at a glance

Home is your personalized start page. It shows supported item types (apps, reports, warehouses, lakehouses, notebooks, task flows, and more) you can access. Some preview or restricted item types might not appear.



Key areas:

1. Navigation pane (nav pane): Switch views (Home, Browse, Workloads, OneLake, and more) and open workspaces
2. Fabric and Power BI switcher

3. Create: Start a new item or task flow
4. Top bar: Search, Help (?), Feedback, Notifications, Settings, and Account manager
5. Learning and getting started resources
6. Your content: Recent workspaces, recent items, and favorites

> **Important**
>
> Home lists only supported content you can access. If a license or subscription change removes access, Fabric prompts you to upgrade or start a trial.

Work with workspaces

A workspace is a collaborative container for related items.

Open a workspace:

1. Select **Workspaces** in the nav pane, then select a workspace.
2. Enter part of the workspace name in global search and select the workspace.
3. Use the workspace selector at the bottom of the nav pane.
4. Open an item in a workspace to activate that workspace.

Workspace behavior:

- If no workspace is open, **My workspace** opens by default.
- When you create a new item, it's added to the active workspace unless you choose a different one. Most creation dialogs default to the active workspace.
- If task flow templates are enabled in your tenant, they appear in the first row on Home to help you start structured solutions. See Task flows in Microsoft Fabric.

For governance details, see Workspaces.

Create items and explore workloads

Workloads are Fabric capability areas, like Data Factory, Data Engineering, and Real-Time Intelligence.

To explore:

1. Select **Workloads** in the nav pane to open the Workload hub.

2. Review available workloads and their landing pages (overview, supported item types, samples, learning links).
3. Open a workload to create items for that capability.

> **Note**
>
> Your organization or Microsoft can add more workloads over time. Tabs in the Workload hub refresh as new workloads are provisioned.

More about workloads: Workloads in Fabric.

Multitask with tabbed navigation to access resources

The tabbed navigation allows you to work with multiple items, monitoring pages, and workspaces at the same time. It enhances navigation and reduces the need to switch context.

Tabs for open items

When you create or open an item, it appears in a tab horizontally across the top of the Fabric portal and shows the item name, icon for the item type (for example, notebook, pipeline, semantic model, and so on). You can hover over the tab to view the workspace it's created in. You can drag a tab to the required position. Tabs make it easier to identify and switch between open items without losing focus.

> **Note**
>
> The tabbed navigation is only available in the Fabric view, not in Power BI view. For instance, when you switch to the Power BI view from the left nav switcher, the tabbed experience isn't available.

You can right-click any tab in the horizontal tab bar to access the following options, similar to Microsoft Edge:

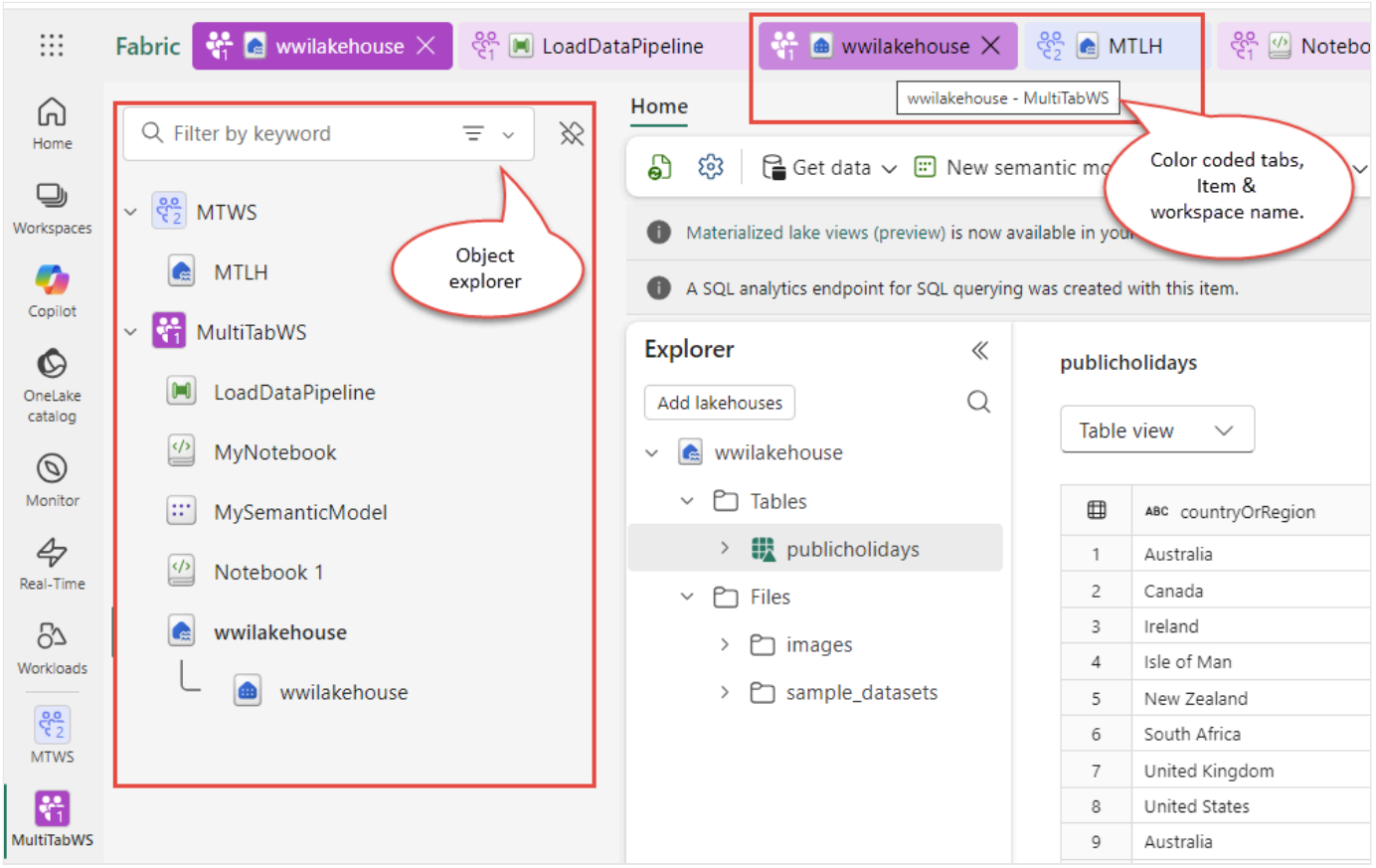
- **Open in new browser tab**: Opens the selected Fabric tab in a new browser tab with the same item and context.
- **Pin tab**: Keeps a tab fixed on the left side for quick access to important items. Pinned tabs stay visible even when you open more items. To unpin, right-click the tab and select **Unpin** or select the pinned icon.

Multiple open workspaces

You can open and work across multiple workspaces side by side. Items are color-coded and numbered to indicate which workspace they belong to. This feature helps reduce confusion and improves context when working in multiple environments.

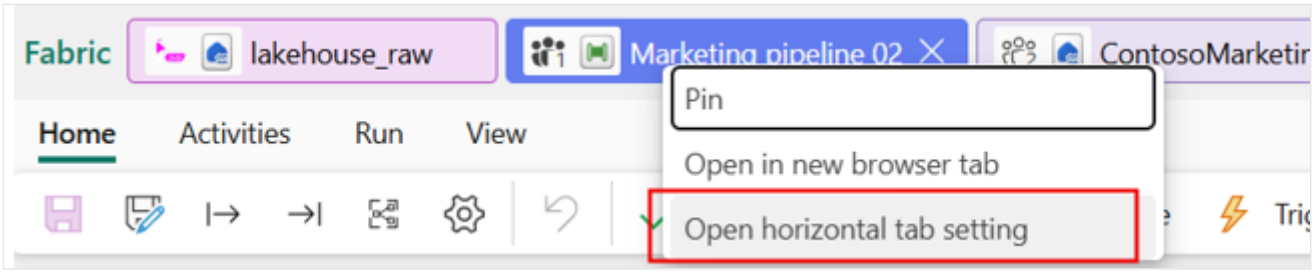
Object explorer

The object explorer provides a structured view of items across all your currently open workspaces. Use it to quickly locate and open resources without having to switch between pages. You can pin the object explorer for easy access. Use the filter option to view items of a specific type or search for an item by a keyword. The items are organized by the hierarchy they belong to.



Customize horizontal tab display

You can also configure how horizontal tabs are displayed. Right-click any tab and select **Open horizontal tab setting** menu and choose the display style that fits your needs.



You can choose from the following two tab display patterns:

1. **Full tab names**

    - Displays the complete name of each tab.
    - Ideal when clarity and readability of item names are most important.
    - Tabs expand to fit their full text for easy identification.

2. **Adaptive truncated names**

    - Automatically shortens tab names when horizontal space becomes limited.
    - Keeps more tabs visible at the same time. It's useful for multitasking or complex development workflows.
    - The layout intelligently adapts to your window size and the number of open tabs.

After you make a selection, changes apply instantly, it's not required to refresh the browser or tabs. When the number of open tabs exceeds the limit that can be displayed on the screen, an overflow menu appears. This menu provides a complete list of currently open tabs, allowing you to quickly navigate to any item without losing track of your work.

Keyboard shortcuts for object explorer and horizontal tabs

Use keyboard shortcuts in horizontal tabs and object explorer to switch between items, expand or collapse folders, and jump to specific tabs. This functionality is similar to an IDE environment.

Prerequisites:

- Switch to the Fabric view using the lower-left experience switcher.
- Enable the multitasking (horizontal tabs and object explorer) feature.

Keyboard shortcuts in object explorer

Object explorer helps you browse workspaces, folders, and items. Use the following shortcuts to quickly navigate:

Expand table

Action	Shortcut
Expand workspace or subfolder	→ Right arrow
Collapse workspace or subfolder	← Left arrow
Navigate to the previous object	↑ Up arrow (If the object is a collapsed workspace/subfolder, Left arrow functions the same as Up arrow)
Navigate to the next object	↓ Down arrow (If the object is an expanded workspace/subfolder, Right arrow functions the same as Down arrow)

Use keyboard shortcuts in horizontal tabs

Horizontal tabs let you switch between multiple open items. Use the following shortcuts to move efficiently across tabs:

Expand table

Action	Shortcut
Navigate to the Nth tab	* Windows: Alt + Number (1–9) * macOS: Control + Number (1–9)

More open items

The previous limit of 10 open items has been increased to 20. You can now keep up to 20 resources active at once. This is especially useful for complex workflows that require multiple pipelines, notebooks, or reports.

Multitasking across browser tabs

Fabric now helps you multitask by carrying your current work into additional browser tabs. When you open Fabric in another tab using the same browser profile, your open items, tab order, pinned items, and tab groups are available there. Any items inherited into the new browser tab count toward the 20 concurrent open item limit. If you use multiple browser tabs for separate tasks, we recommend closing items that you no longer need to avoid hitting this limit. Your

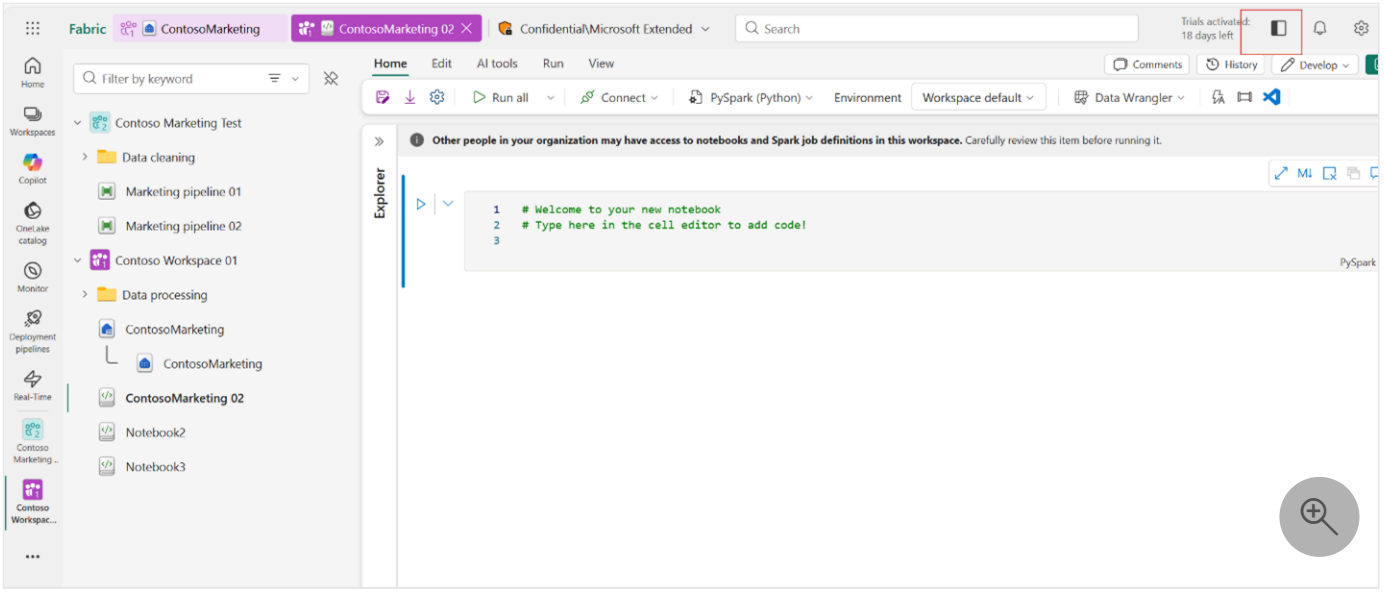
multitasking context is retained in the same browser profile. It isn't carried over when you use a different browser, browser profile, or device.

> **Note**
>
> Context carryover is currently applied automatically. Based on customer feedback we've received since launching this feature, we're working on rolling out a setting that will let you choose whether opening Fabric in another browser tab carries over your existing context or starts with a clean context.

Maximize editing space with focus mode

Focus mode helps developers create a distraction-free workspace by hiding the navigation sidebar and object explorer. It maximizes the editor pane by expanding it to use the full monitor area, removing side panels and navigation layers to reduce distractions, and enabling writers to focus and work more productively without UI clutter.

To enable focus mode, open the item you want to edit and select **Focus Mode** from the global header. The navigation sidebar and object explorer are collapsed, maximizing the editor pane.



To exit, select **Exit Focus Mode**, the same toggle control turns it off.

Find content fast: Search, filter, sort

Use the following table to compare Fabric's search and filtering tools. Use global search for broad, cross-workspace discovery (names, creators, tags), the local keyword filter to narrow the

current view, sorting to quickly order columns, and the Filters panel for precise refinements by type, time, or owner.

Expand table

Feature	How to use	Notes / examples
Global search	Use the search box in the top bar to find items by name, title, creator, tag, or workspace. Results show only content you can access.	Examples: part of a report name; a colleague's name to see items they shared; a tag (if tagging is enabled)—see Tags overview. Tag results appear only if your organization enables tagging.
Local keyword filtering	Use the **Filter by keyword** field on canvases (for example, Browse) to narrow the current list without leaving the page.	Applies only to the current view.
Sorting	Select a column header (for example, Name or Refreshed) to sort; select again to toggle direction.	Not all columns are sortable—hover to confirm.
Filters panel	Select **Filter** (upper-right of a content list) to refine results by type, time, and owner.	Examples: Type (report, notebook), Time (recently accessed or modified), Owner (creator).

Considerations

> **Note**
>
> Global search uses Azure AI Search. It isn't available in sovereign clouds or regions where Azure AI Search isn't supported. Tag and community content visibility depend on feature enablement. See Azure AI Search regions.

Use the help pane effectively

Select the **?** icon to open the contextual Help pane:

- Feature-aware view: Shows topics and community discussions relevant to the current screen.
- Forum topics: Community posts tied to on-screen features (shown when the tenant enables forum integration).
- Other resources: Support and feedback links.

- Search box: Enter keywords to search documentation and forums; use the dropdown to refine.

Keep it docked as you work or close it to reclaim space. Use the back arrow to return to the default view.

Personalize via settings

Select the gear icon in the top bar to open settings. Links shown depend on your role (for example, admin vs contributor) and enabled preview features.

Expand table

Section	What you can do
Preferences	Set display language, personalize UI behavior, manage notifications, configure item settings (per item type), enable developer mode (where applicable).
Resources and extensions	Manage personal and group storage, Power BI item settings, connections and gateways, embed codes, Azure Analysis Services migrations.
Governance and insights	Open OneLake Catalog and select **Govern** for governance and administration. If OneLake Catalog and Govern aren't yet available in your region, use the **Admin portal**.

Learn more: Govern in OneLake Catalog • Fabric administration You can also select region-specific English language variants, including English (United States), English (United Kingdom), and English (Canada), from the language settings. Each variant applies its associated date and time format throughout the Fabric experience, making it easier to display dates and times in the format commonly used in your region.

Learn more: Admin portal • Microsoft Purview hub (preview)

Manage your account, notifications, and feedback

- Account manager (profile photo): View Workspace type, trial status, organizational context
- Notifications: Alerts, system messages, subscriptions
- Feedback: Send product feedback to Microsoft
- Settings: Open the settings pane
- Help (?): Open contextual help and search

For licensing guidance, see Licenses. For trial info, see Start a Fabric trial.

Get support

If self-help doesn't solve your problem:

- Open the **Help** pane and scroll to **Support links**.
- Ask the community in the forums.
- If entitled, open a support ticket. See Support options.

Tips to stay efficient

- Pin only the nav pane buttons you use often. Use right-click or the ellipsis (...) menu > **Unpin** / **Pin** (UI may vary).
- Use tabs and **Object explorer** (preview) to reduce context switching.
- Use global search for infrequently used items; refine with filters.
- Keep **Help** open while you learn a new workload.
- Review settings regularly as your workflow evolves.

Related content

- Workspaces
- Task flows in Microsoft Fabric
- Get in-product help
- Workloads in Fabric
- Start a Fabric trial
- Licenses

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

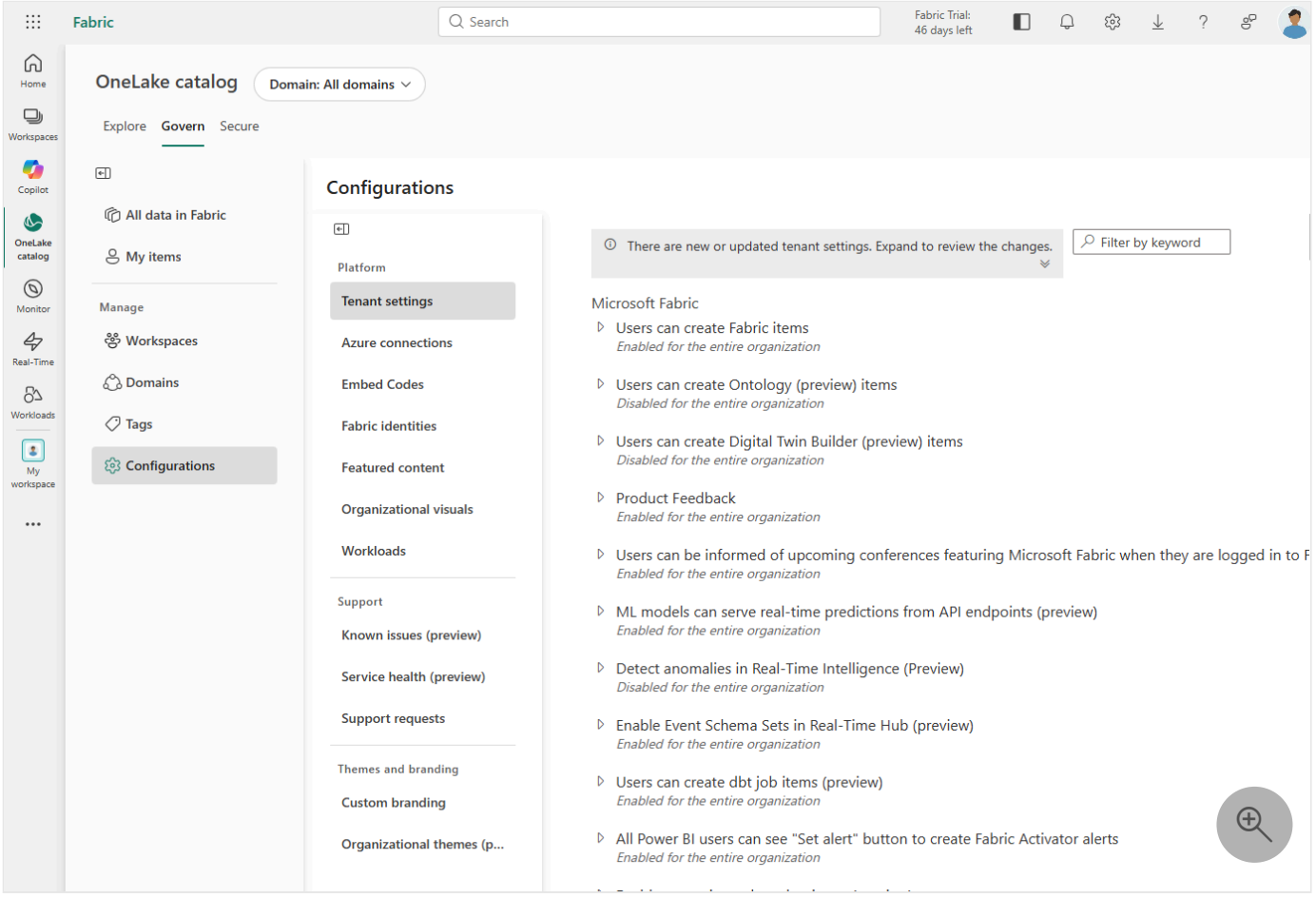
Last updated on 09/15/2026

OneLake catalog overview

OneLake catalog is the central experience in Microsoft Fabric for discovering data, understanding governance status, and managing security across your Fabric environment. It's a centralized place that helps you find, explore, and use the Fabric items you need, and govern the data you own.

You access the catalog from the Fabric navigation pane. The catalog is also embedded in Microsoft Teams, Microsoft Excel, and Microsoft Copilot Studio so you can discover and act on items directly from these applications. You can also programmatically discover catalog metadata across workspaces by using the Microsoft Fabric Catalog Search REST API.

OneLake catalog is organized into three experiences—Explore, Govern, and Secure.



The Govern tab in the OneLake catalog is where you understand your governance status, act on recommendations, and manage administrative settings for the data you own. Explore and Secure are the two other tabs in the OneLake catalog.

Manage and govern your Fabric estate in Govern

Govern is the centralized experience for managing and governing organizational content in Microsoft Fabric.

In addition to governance capabilities such as domains, endorsements, metadata scanning, policies, and governance insights, Govern also provides access to administrative experiences including workspace management, capacity management, tenant settings, and other organization-wide controls.

As administrative capabilities continue to move from the Admin portal into Fabric, administrators can use Govern as their primary destination for managing and governing their organization's data estate.

Find and understand data

The **Explore tab** is where you find and understand Fabric items. It has an items list with an in-context item details view that makes it possible to browse through and explore items without losing your list context. It also provides selectors and filters to narrow down and focus the list, making it easier to find what you need. By default, OneLake catalog opens on the Explore tab. You can also open and work across multiple workspaces side by side using the object explorer.

For more information, see Discover and explore Fabric items in the OneLake catalog and the Microsoft Fabric Catalog Search REST API.

Understand Governance Health

The **Govern tab** helps you understand and improve the **Governance Health** of the data you own in Fabric.

Govern provides a unified experience for governing and managing your organization's Fabric estate.

In the Govern tab, **Governance Insights** explain the current governance status of your data and highlight the areas that need attention, so you always know where you stand.

Administrative experiences are now surfaced within Govern. Fabric administrators can access governance insights, tenant configuration, workspace and capacity management, policies, and other administrative controls from a single location.

Take action

After you understand your Governance Health, the Govern tab presents **Recommended Actions** you can take to improve the governance status of your data, along with guidance resources that help you act.

Govern serves both governance and administrative personas, helping organizations manage, secure, monitor, and govern their Fabric data estate from a centralized experience. A single person often holds more than one of these responsibilities—a Capacity Admin might also be a Fabric Admin, Domain Admin, Workspace Admin, or data owner—so Govern focuses on the outcome you want rather than on a single role.

For more information, see Govern your data in Fabric.

Secure data access

The **Secure tab** centralizes security management in Microsoft Fabric by providing a unified view of workspace roles and OneLake security roles across items. From a single location, you can review access, understand permissions, and manage security roles. Admins can audit permissions, view user access, and create, edit, or delete security roles to keep data access consistent.

For more information, see Secure your data.

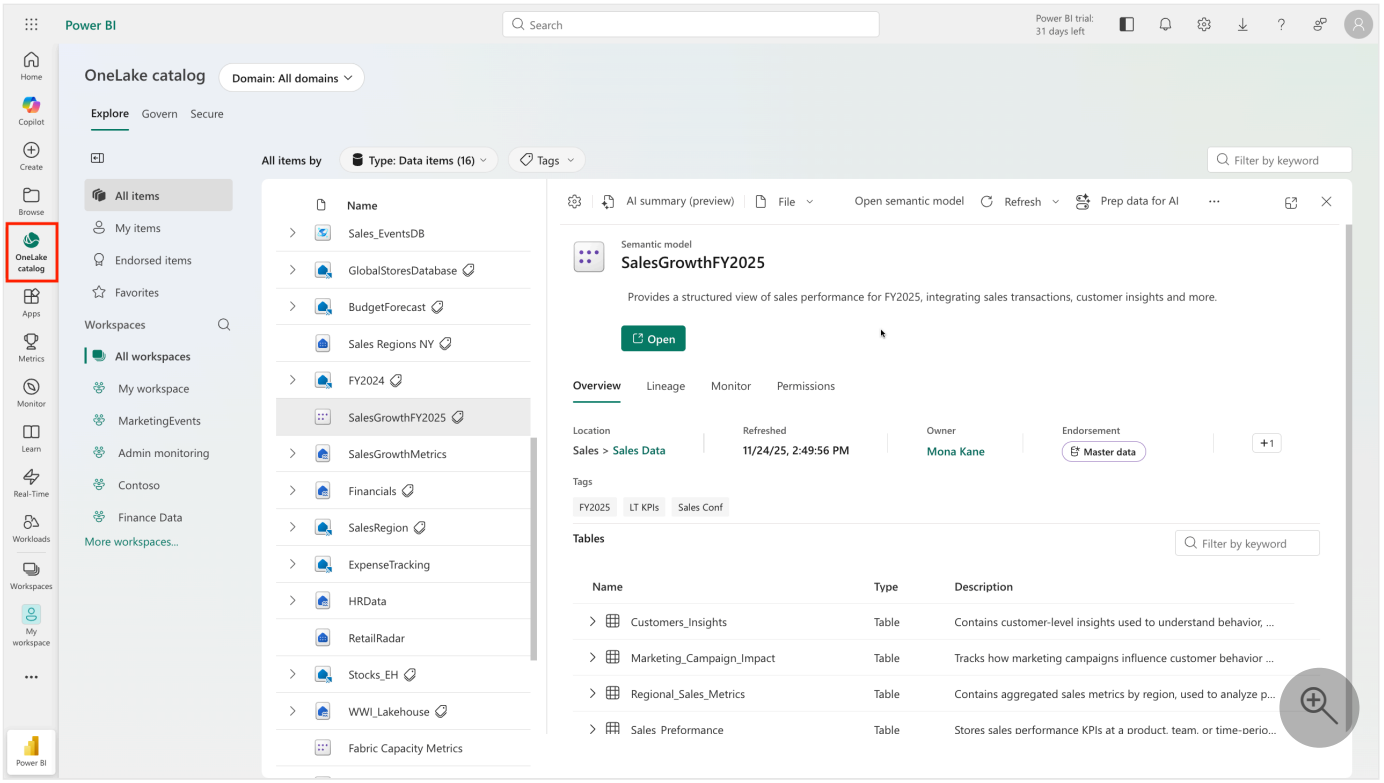
Choose the right experience

Expand table

I want to...	Use	Where
Find, browse, and inspect Fabric items	Explore tab	OneLake catalog → Explore
Understand my Governance Health and act on Recommended Actions	Govern tab	OneLake catalog → Govern
Manage workspace and OneLake security roles, audit permissions, review user access	Secure tab	OneLake catalog → Secure
Search and explore catalog items and metadata programmatically	REST API	OneLake catalog REST API overview

Open OneLake catalog

To open OneLake catalog, select the OneLake icon in the Fabric navigation pane. Select the tab you're interested in if it isn't displayed by default.



Related content

Discover

- Discover and explore Fabric items in the OneLake catalog
- Microsoft Fabric Catalog Search REST API

Govern

- Govern your data in Fabric
- Fabric domains

Secure

- Secure your data

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/29/2026

End-to-end tutorials in Microsoft Fabric

In this article, you find a comprehensive list of end-to-end tutorials available in Microsoft Fabric. These tutorials guide you through a scenario that covers the entire process, from data acquisition to data consumption. They're designed to help you develop a foundational understanding of the Fabric UI, the various experiences supported by Fabric and their integration points, and the professional and citizen developer experiences that are available.

Multi-experience tutorials

The following table lists tutorials that span multiple Fabric experiences.

Expand table

Tutorial name	Scenario
Lakehouse	In this tutorial, you ingest, transform, and load the data of a fictional retail company, Wide World Importers, into the lakehouse and analyze sales data across various dimensions.
Data Science	In this tutorial, you explore, clean, and transform a dataset of 10,000 bank customers, and build a machine learning model to predict which customers are likely to churn.
Real-Time Intelligence	In this tutorial, you use the streaming and query capabilities of Real-Time Intelligence to analyze London bike share data. You learn how to stream and transform the data, run KQL queries, build a Real-Time Dashboard and a Power BI report to gain insights and respond to this real-time data.
Digital twin builder (preview) in Real-Time Intelligence	In this tutorial, you set up a digital twin builder (preview) item and use it to contextualize sample data streamed from Real-Time Intelligence. Then you project your data to Eventhouse using a Fabric notebook, and extract further insights by running KQL queries and visualizing the digital twin builder data in a Real-Time Dashboard.
Data warehouse	In this tutorial, you build an end-to-end data warehouse for the fictional Wide World Importers company. You ingest data into data warehouse, transform it using T-SQL and pipelines, run queries, and build reports.
Fabric SQL database	The tutorial provides a comprehensive guide to utilizing the SQL database in Fabric. This tutorial is tailored to help you navigate through the process of database creation, setting up database objects, exploring autonomous features, and combining and visualizing data. Additionally, you learn how to create a GraphQL endpoint, which serves as a modern approach to connecting and querying your data efficiently.

Tutorial name	Scenario
Fabric Activator	The tutorial is designed for customers who are new to Fabric Activator. Using a sample eventstream, you learn your way around Activator. Once you're familiar with the terminology and interface, you create your own object, rule, and activator.

Experience-specific tutorials

The following tutorials walk you through scenarios within specific Fabric experiences.

Expand table

Tutorial name	Scenario
Power BI	In this tutorial, you build a dataflow and pipeline to bring data into a lakehouse, create a dimensional model, and generate a compelling report.
Data Factory	In this tutorial, you ingest data with pipelines and transform data with dataflows, then use the automation and notification to create a complete data integration scenario.
Data Science end-to-end AI samples	In this set of tutorials, learn about the different Data Science experience capabilities and examples of how ML models can address your common business problems.
Data Science - Price prediction with R	In this tutorial, you build a machine learning model to analyze and visualize the avocado prices in the US and predict future prices.
Application lifecycle management	In this tutorial, you learn how to use deployment pipelines together with git integration to collaborate with others in the development, testing, and publication of your data and reports.
Digital twin builder (preview)	In this tutorial, you model and contextualize data from multiple sources into a digital twin builder (preview) ontology. You explore the ontology with queries, then create a Power BI dashboard to visualize the data.
Graph in Microsoft Fabric (preview)	In this tutorial, you load sample data into a lakehouse and build a graph model with nodes and edges. You then query the graph using both the visual query builder and GQL (Graph Query Language) to explore relationships and patterns in the data.

Related content

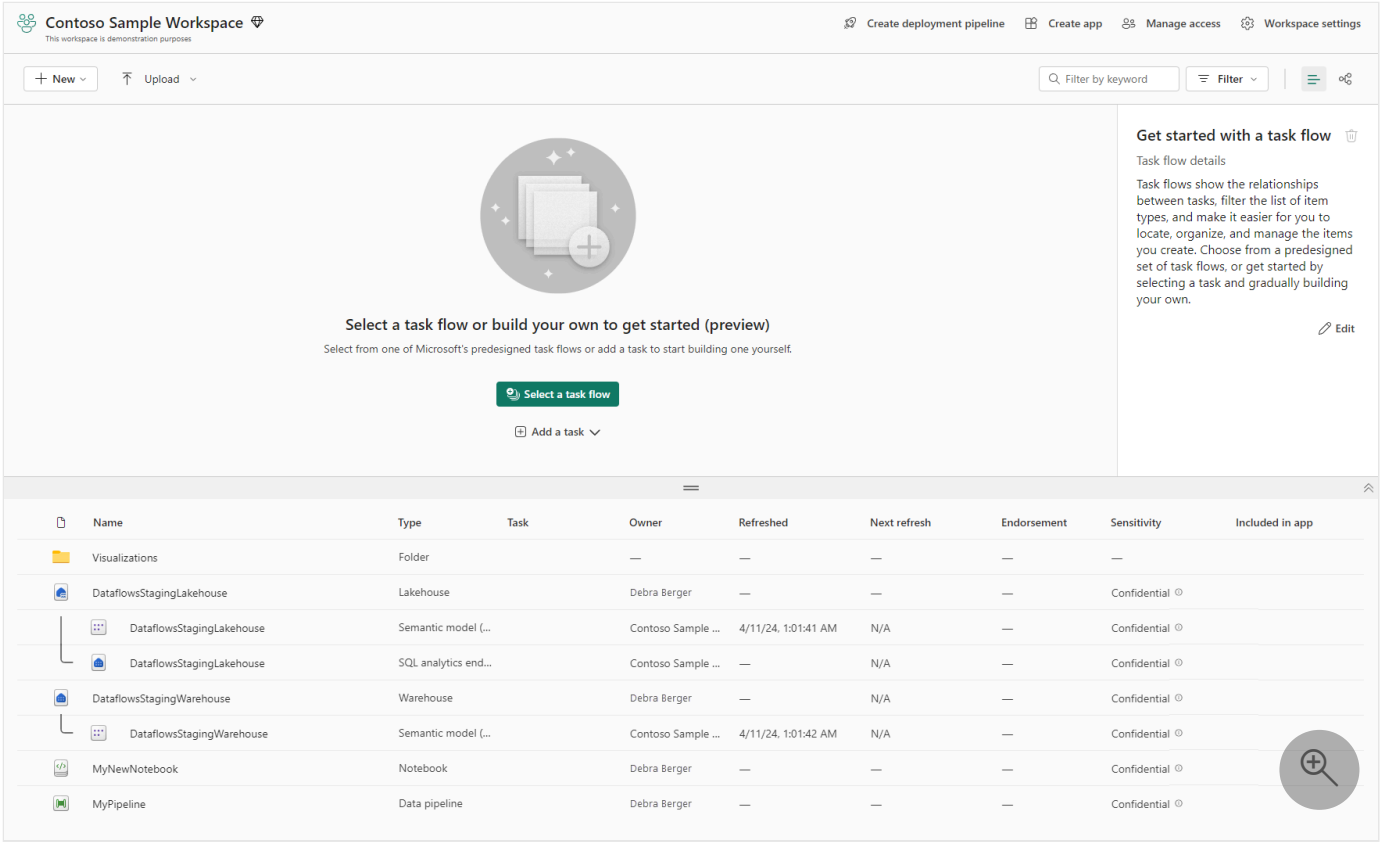
- Create a workspace
- Discover data items in the OneLake catalog

Last updated on 07/30/2026

Workspaces in Microsoft Fabric and Power BI

Workspaces are places to collaborate with colleagues to create collections of items such as lakehouses, warehouses, and reports, and to create task flows. This article describes workspaces, how to manage access to them, and what settings are available.

If you're just interested in creating a workspace, see Create a workspace.



Work with workspaces

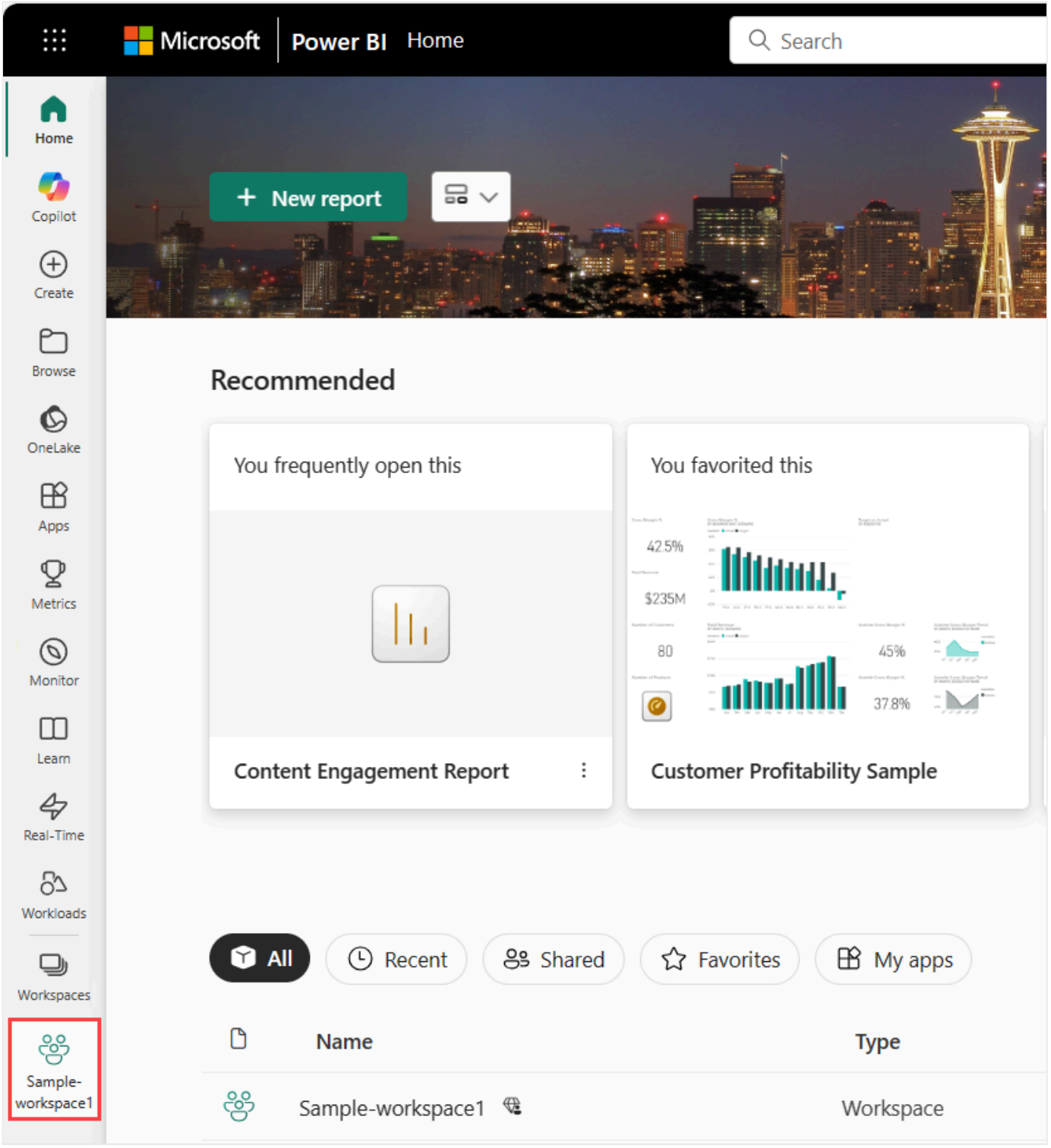
Here are some useful tips about working with workspaces.

- **Set up a task flow for the workspace** to organize your data project and to help others understand and work on your project. Read more about task flows.
- **Pin workspaces** to the top of the workspace flyout list to quickly access your favorite workspaces. Read more about pin workspaces.
- **Use granular workspace roles** for flexible permissions management in the workspaces: Admin, Member, Contributor, and Viewer. Read more about workspace roles.

- **Create folders in the workspace**: Organize and manage artifacts in the workspace. Read more about creating folders in workspaces.
- **Navigate to current workspace** from anywhere by selecting the icon on left nav pane. Read more about current workspace in this article.
- **Workspace settings**: As workspace admin, you can update and manage your workspace configurations in workspace settings.
- **Manage a workspace in Git**: Git integration in Microsoft Fabric enables Pro developers to integrate their development processes, tools, and best practices straight into the Fabric platform. Learn how to manage a workspace with Git.
- **Contact list**: Specify who receives notification about workspace activity. Read more about workspace contact lists in this article.

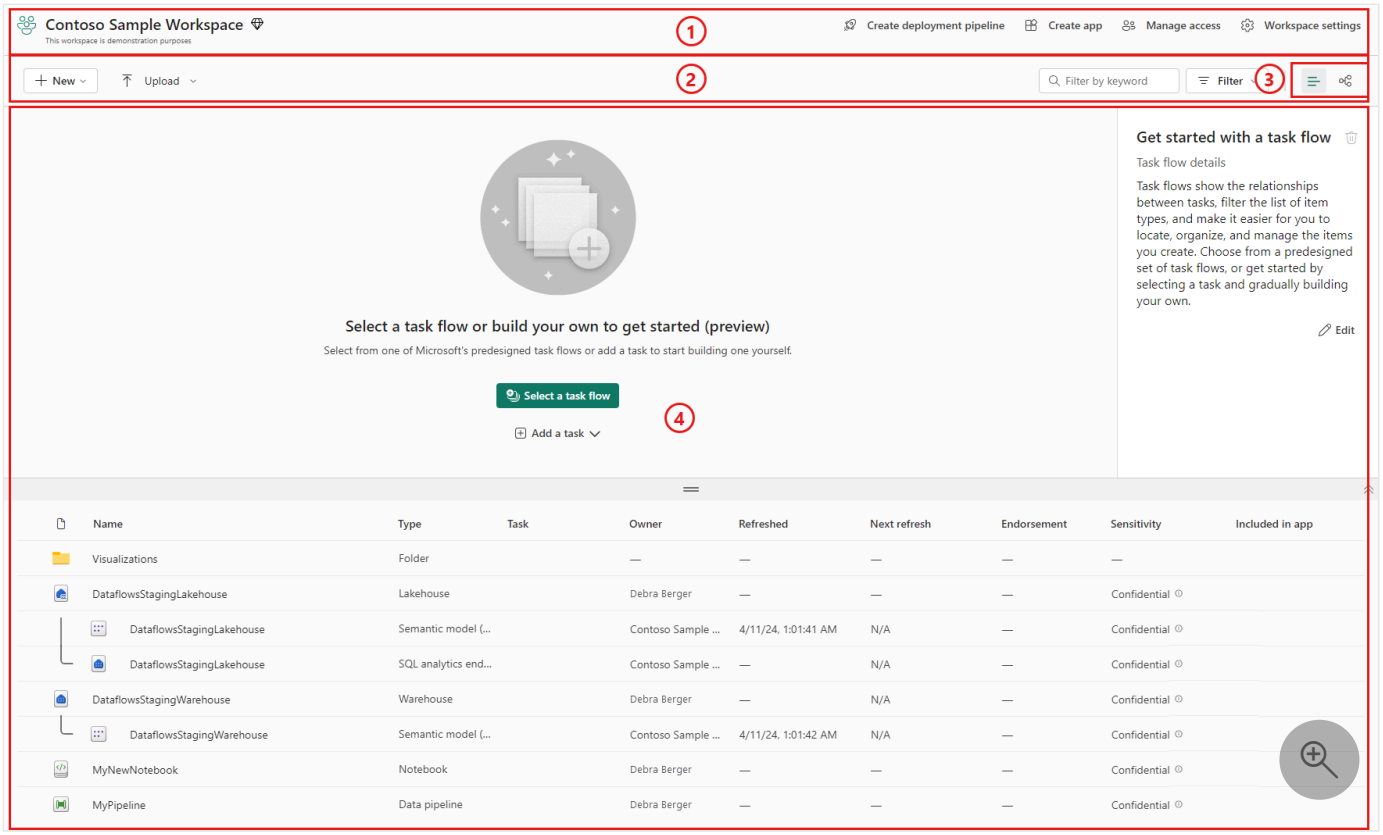
Current workspace

After you select and open a workspace, this workspace becomes your current workspace. You can quickly navigate to it from anywhere by selecting the workspace icon from left nav pane.



Workspace layout

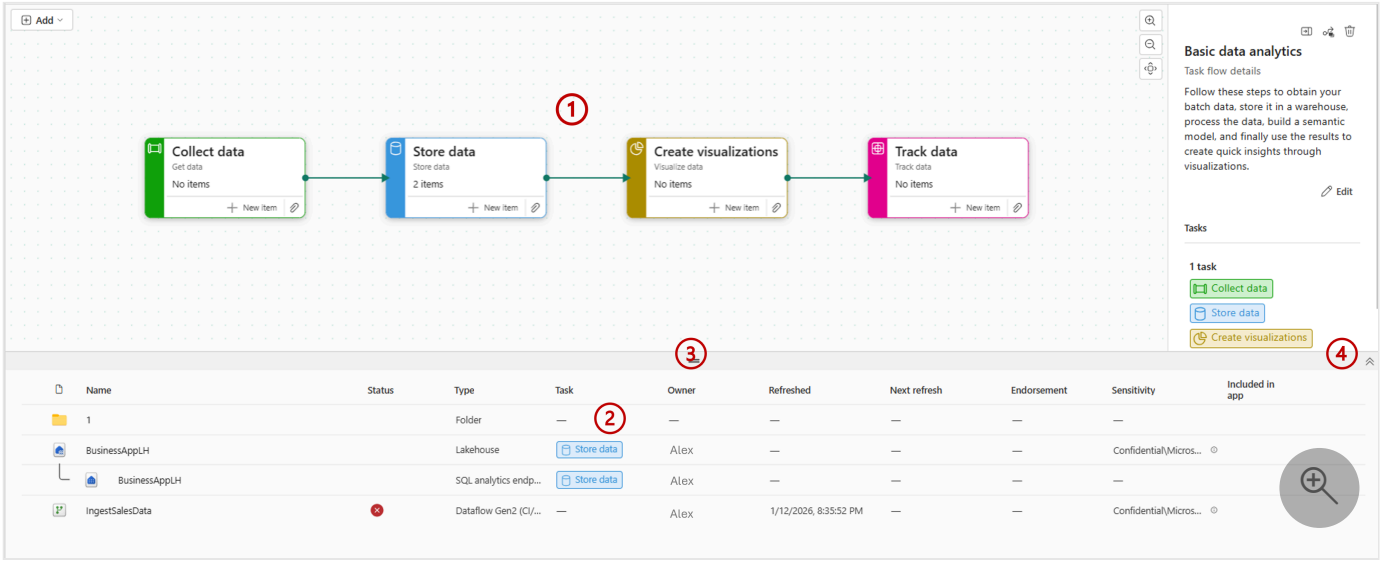
A workspace consists of a header, a toolbar, and a view area. There are two views that can appear in the view area: list view and lineage view. You select the view you want to see with controls on the toolbar. The following image shows these main workspace components, with list view selected.



1. **Header**: The header contains the name and brief description of the workspace, and also links to other functionality.
2. **Toolbar**: The toolbar contains controls for adding items to the workspace and uploading files. It also contains a search box, filter, and the list view and lineage view selectors.
3. **List view and lineage view selectors**: The list view and lineage view selectors enable you to choose which view you want to see in the view area.
4. **View area**: The view area displays either list view or lineage view.

List view

List view is divided into the task flow and the items list.



1. **Task flow**: The task flow is where you can create or view a graphical representation of your data project. The task flow shows the logical flow of the project - it doesn't show the flow of data. Read more about task flows.
2. **Items list**: The items list is where you see the items and folders in the workspace. If you have tasks in the task flow, you can filter the items list by selecting the tasks. In the list view, there are several columns which show the key information of items. For example, the **Status** column shows long-term, non-permanent state of items by using different kinds of indicators, such as trial expiring, dataflow refresh status etc.
3. **Resize bar**: You can resize the task flow and items list by dragging the resize bar up or down.
4. **Show/Hide task flow**: If you don't want to see the task flow, you can hide it using the hide/show arrows at the side of the separator bar.

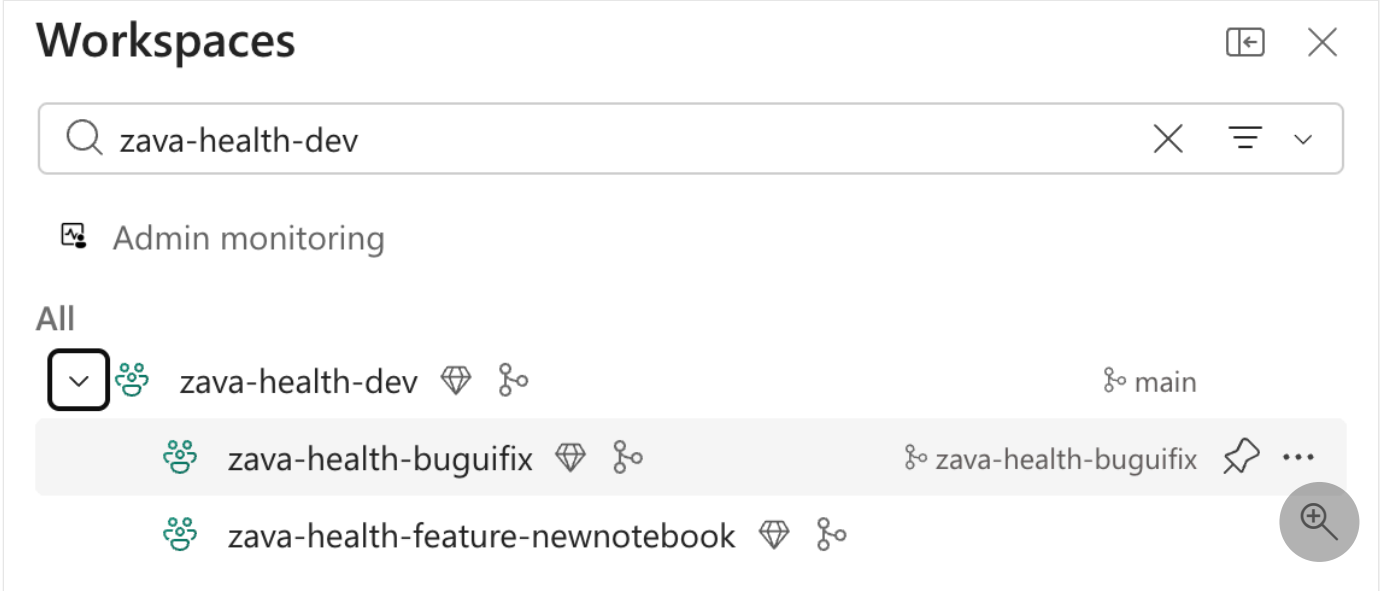
Lineage view

Lineage view shows the flow of data between the items in the workspace. Read more about lineage view.

Branched workspace

A **Branched workspace** is a workspace that is linked to a source workspace. It lets developers work on changes in an isolated environment, understand how their work relates to other workspaces, and promote changes back to the main workspace with confidence.

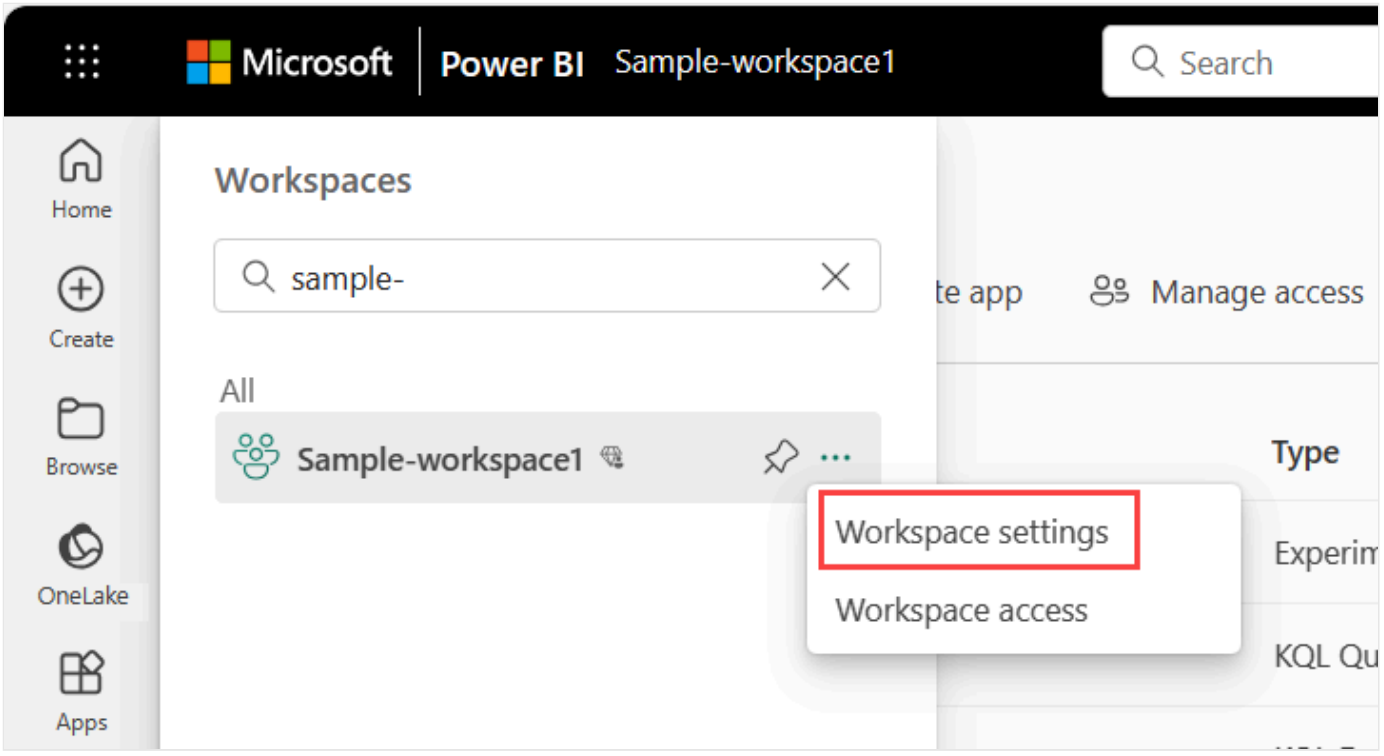
The relationship (link) between a *Branched workspace* and its source workspace is established when the user performs Git Integration -branch-out operation. For more information, see development process using branched workspaces.



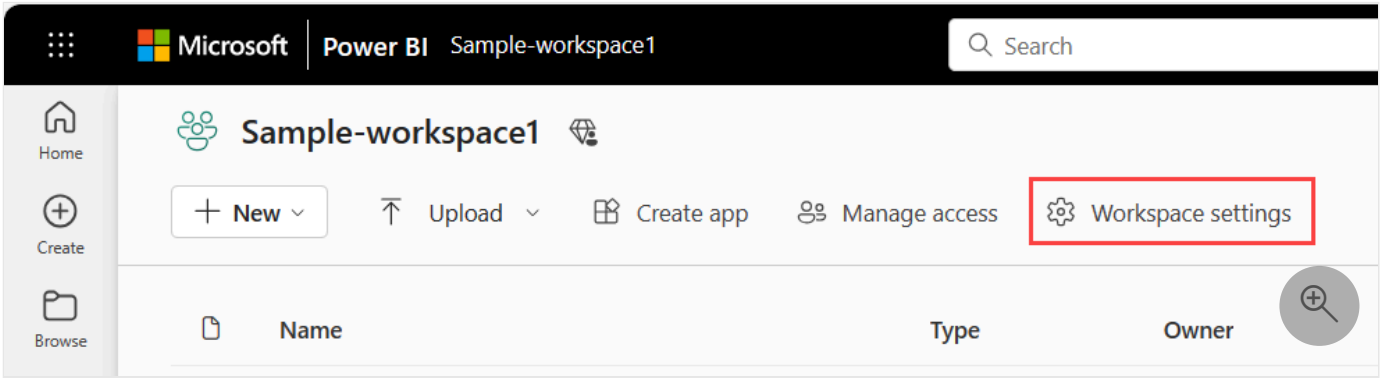
Workspace settings

Workspace admins can use workspace settings to manage and update the workspace. The settings include general settings of the workspace, like the basic information of the workspace, contact list, SharePoint, license, Azure connections, storage, and other experiences' specific settings.

To open the workspace settings, you can select the workspace in the nav pane, then select **More options (...)** > **Workspace settings** next to the workspace name.



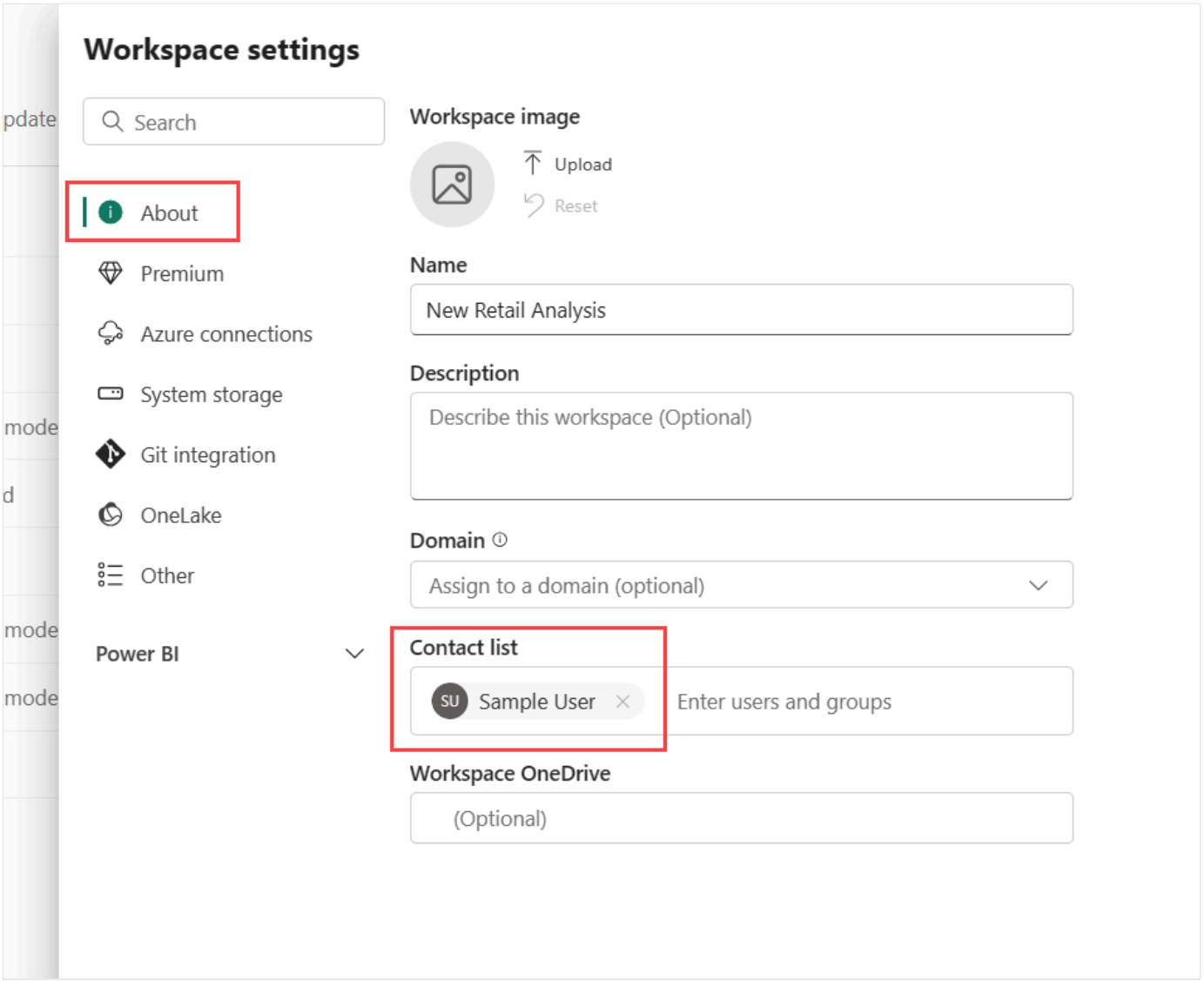
You can also open it from the workspace page.



Workspace contact list

The Contact list feature allows you to specify which users receive notification about issues occurring in the workspace. By default, the one who created the workspace is in the contact list. You can add others to that list while creating workspace or in workspace settings after creation.

Users or groups in the contact list are also listed in the user interface (UI) of the workspace settings, so workspace users know whom to contact.



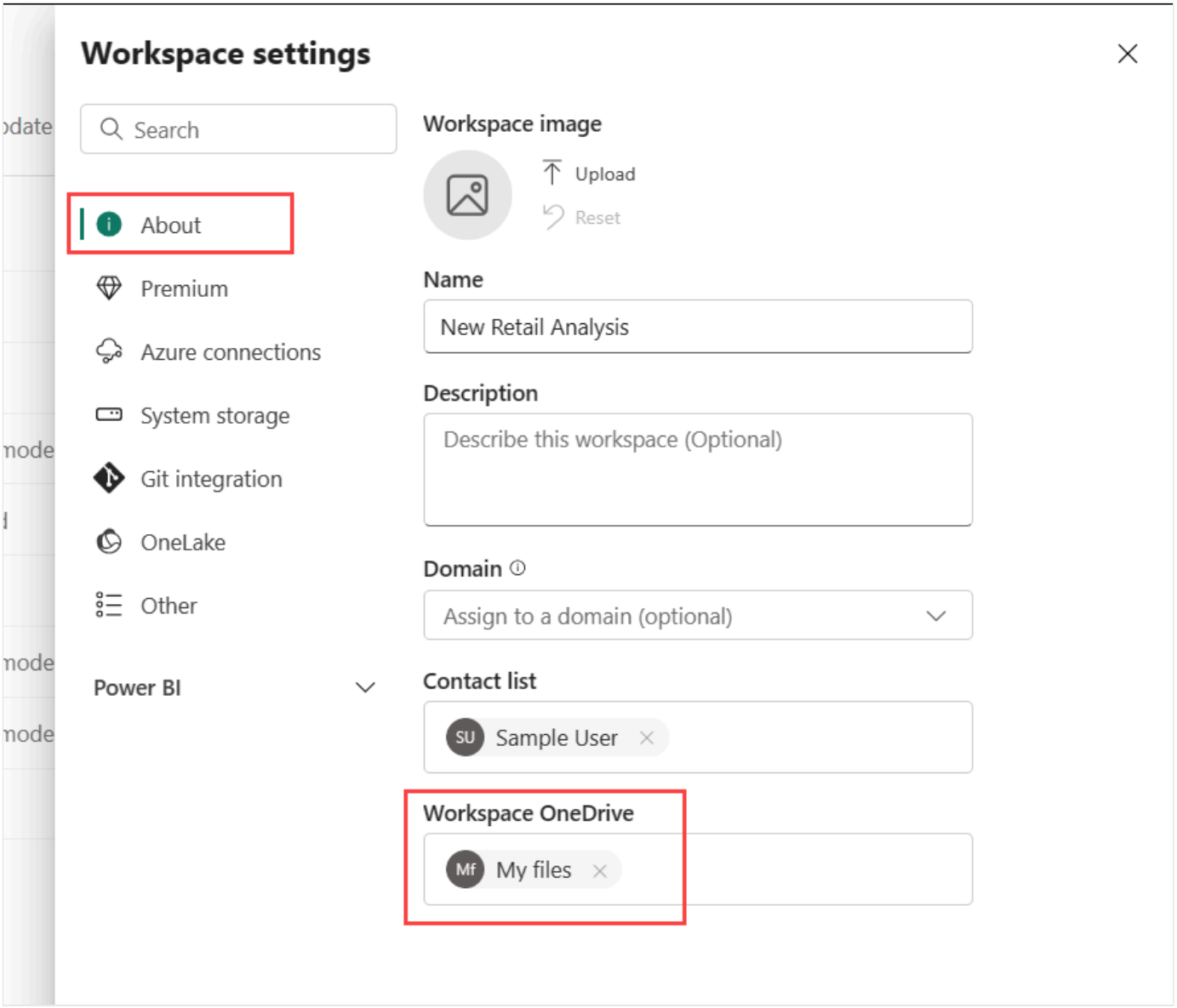
Microsoft 365 and SharePoint

The Workspace SharePoint feature allows you to configure a Microsoft 365 Group whose SharePoint document library is available to workspace users. You create the Group outside of Microsoft Fabric first, with one available method being from SharePoint. Read about creating a SharePoint shared library.

> **Note**
>
> Creating Microsoft 365 Groups may be restricted in your environment, or the ability to create them from your SharePoint site may be disabled. If this is the case, speak with your IT department.

Microsoft Fabric doesn't synchronize permissions between users or groups with workspace access, and users or groups with Microsoft 365 Group membership. A best practice is to give access to the workspace to the same Microsoft 365 Group whose file storage you configured. Then manage workspace access by managing membership of the Microsoft 365 Group.

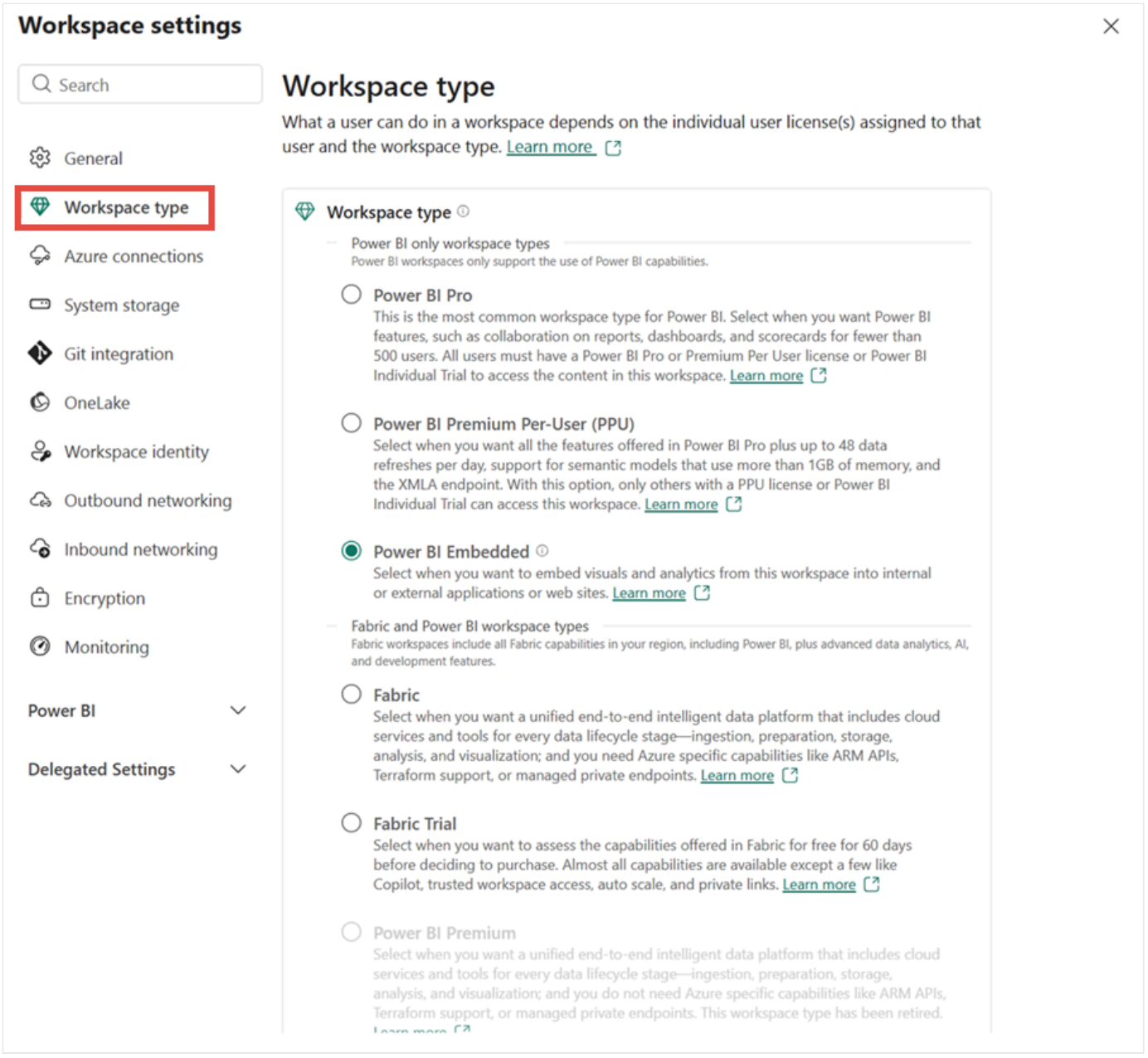
You can configure SharePoint in workspace settings by typing in the name of the Microsoft 365 group that you created earlier. Type just the name, not the URL. Microsoft Fabric automatically picks up the SharePoint for the group.



Workspace type

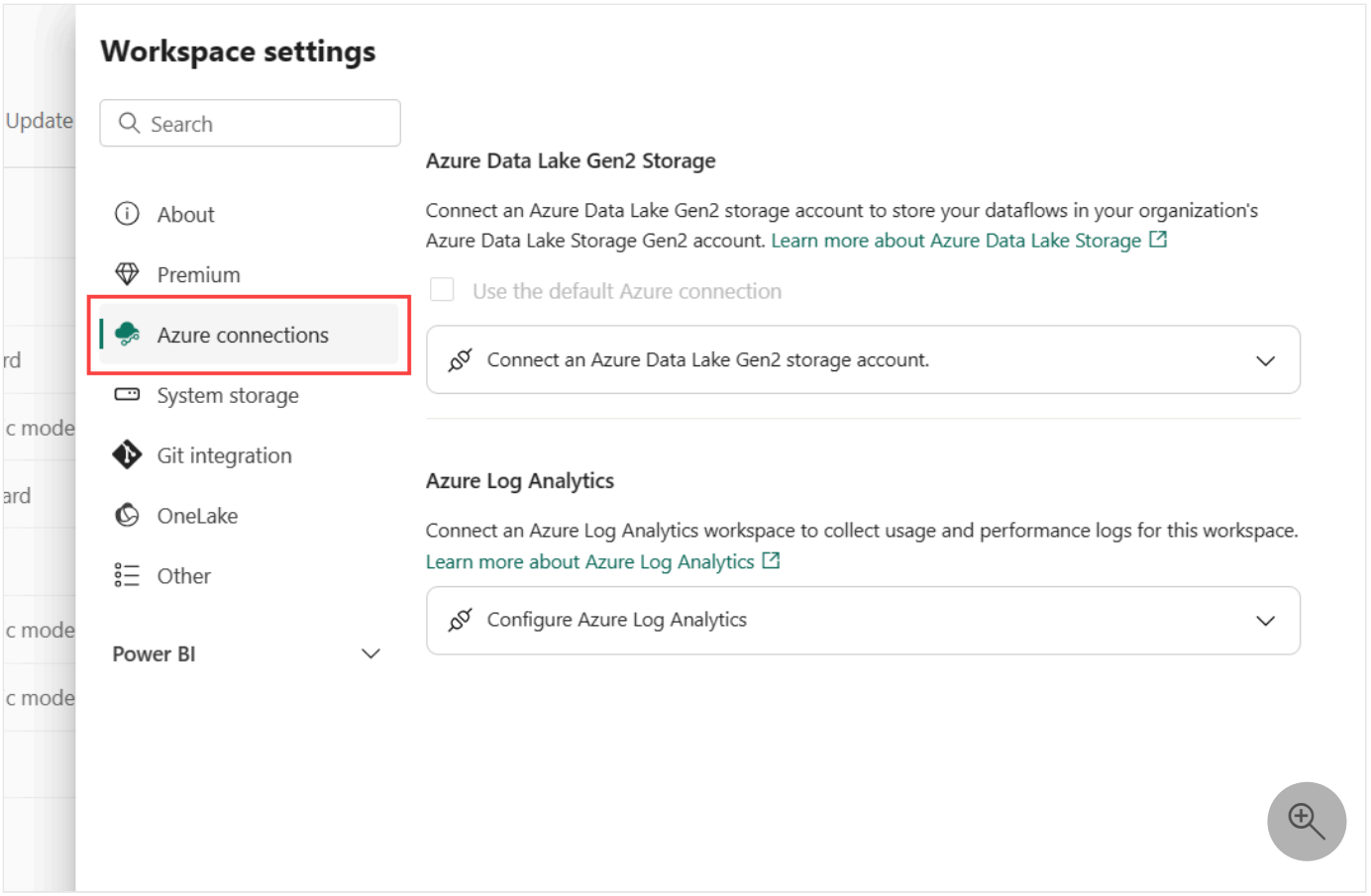
By default, workspaces are created in your organization's shared capacity. When your organization has other capacities, workspaces including *My Workspaces* can be assigned to any capacity in your organization. You can configure it while creating a workspace or modify it after creation in the workspace settings.

Workspace types are organized into two categories, **Power BI workspace types** and **Fabric workspace types**. To learn more, see licenses article.



Azure connections configuration

Workspace admins can configure dataflow storage to use Azure Data Lake Gen 2 storage and Azure Log Analytics (LA) connection to collect usage and performance logs for the workspace in workspace settings.



With the integration of Azure Data Lake Gen 2 storage, you can bring your own storage to dataflows, and establish a connection at the workspace level. Read Configure dataflow storage to use Azure Data Lake Gen 2 for more detail.

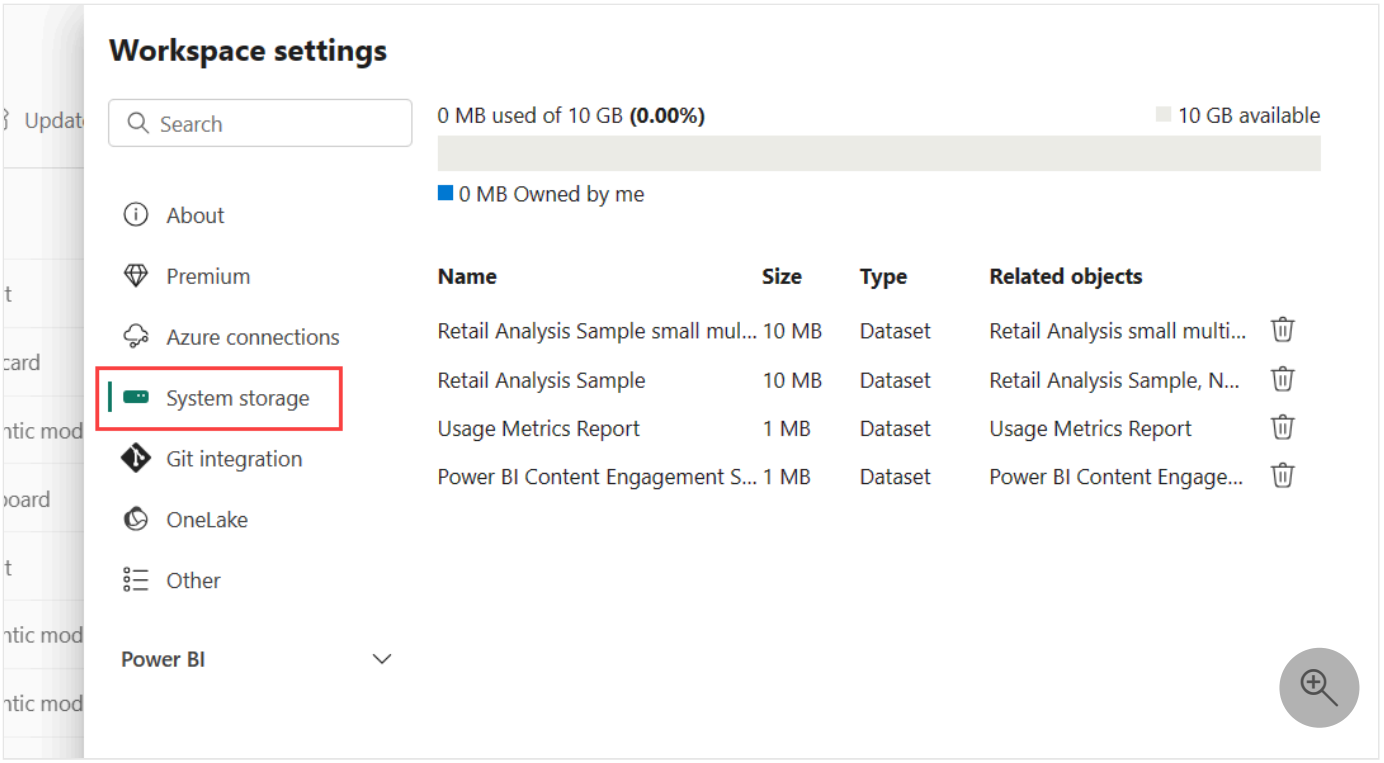
After the connection with Azure Log Analytics (LA), activity log data is sent continuously and is available in Log Analytics in approximately 5 minutes. Read Using Azure Log Analytics for more detail.

System storage

System storage is the place to manage your semantic model storage in your individual or workspace account so you can keep publishing reports and semantic models. Your own semantic models, Excel reports, and those items that someone has shared with you, are included in your system storage.

In the system storage, you can view how much storage you have used and free up the storage by deleting the items in it.

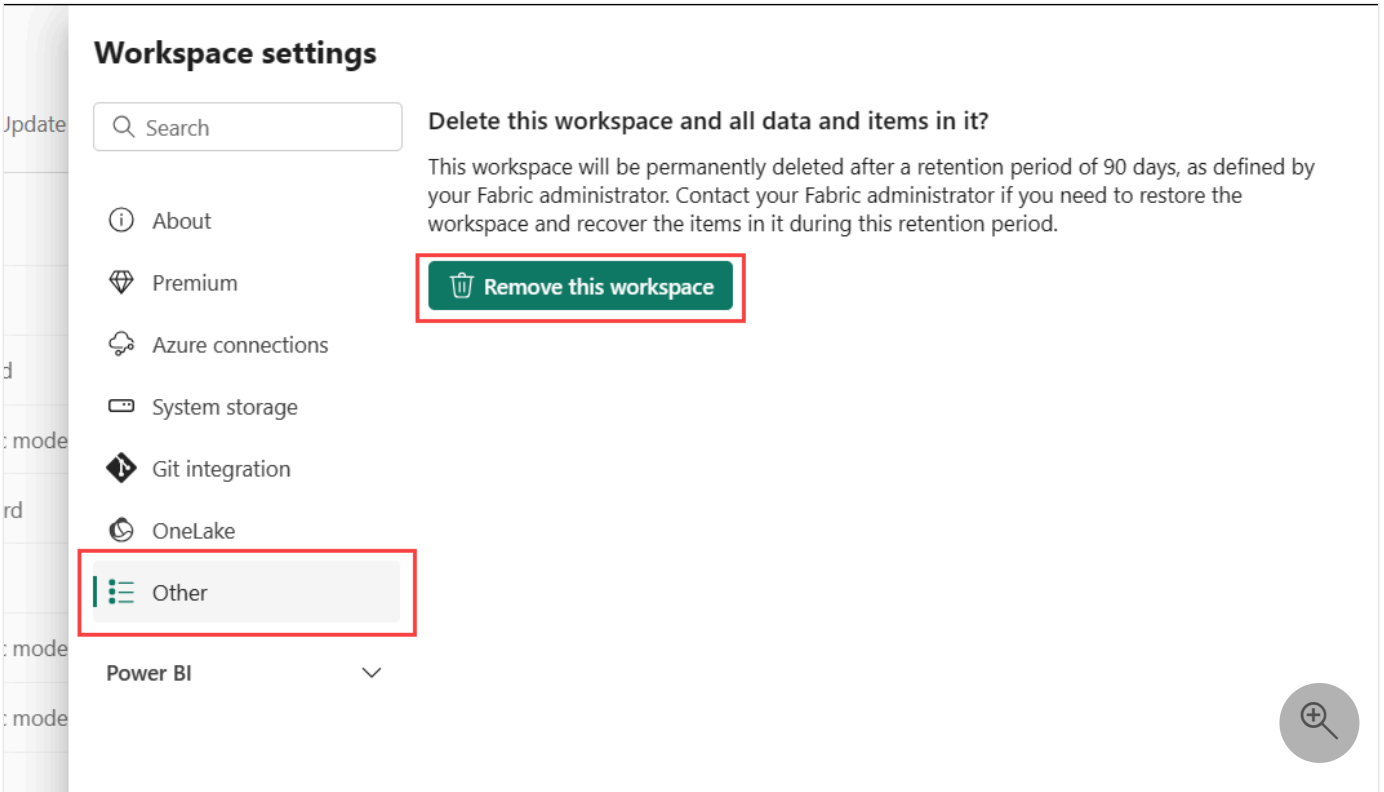
Keep in mind that you or someone else may have reports and dashboards based on a semantic model. If you delete the semantic model, those reports and dashboards don't work anymore.



Remove the workspace

As an admin for a workspace, you can delete it. When you delete the workspace, everything contained within the workspace is deleted for all group members, and the associated app is also removed from AppSource.

In the Workspace settings pane, select **Other** > **Remove this workspace**.



> ⚠ **Warning**
>
> If the workspace you're deleting has a workspace identity, that workspace identity will be irretrievably lost. In some scenarios this could cause Fabric items relying on the workspace identity for trusted workspace access or authentication to break. For more information, see Delete a workspace identity.

Remove a workspace in a deployment pipeline

If you attempt to delete a workspace via the workspace settings, and that workspace is currently assigned to a Deployment Pipeline, the deletion will fail.

To remove the workspace, you can return to the workspace page and select the **View Deployment Pipeline** button. This will direct you to the relevant pipeline, where you can unassign the workspace and then remove it.

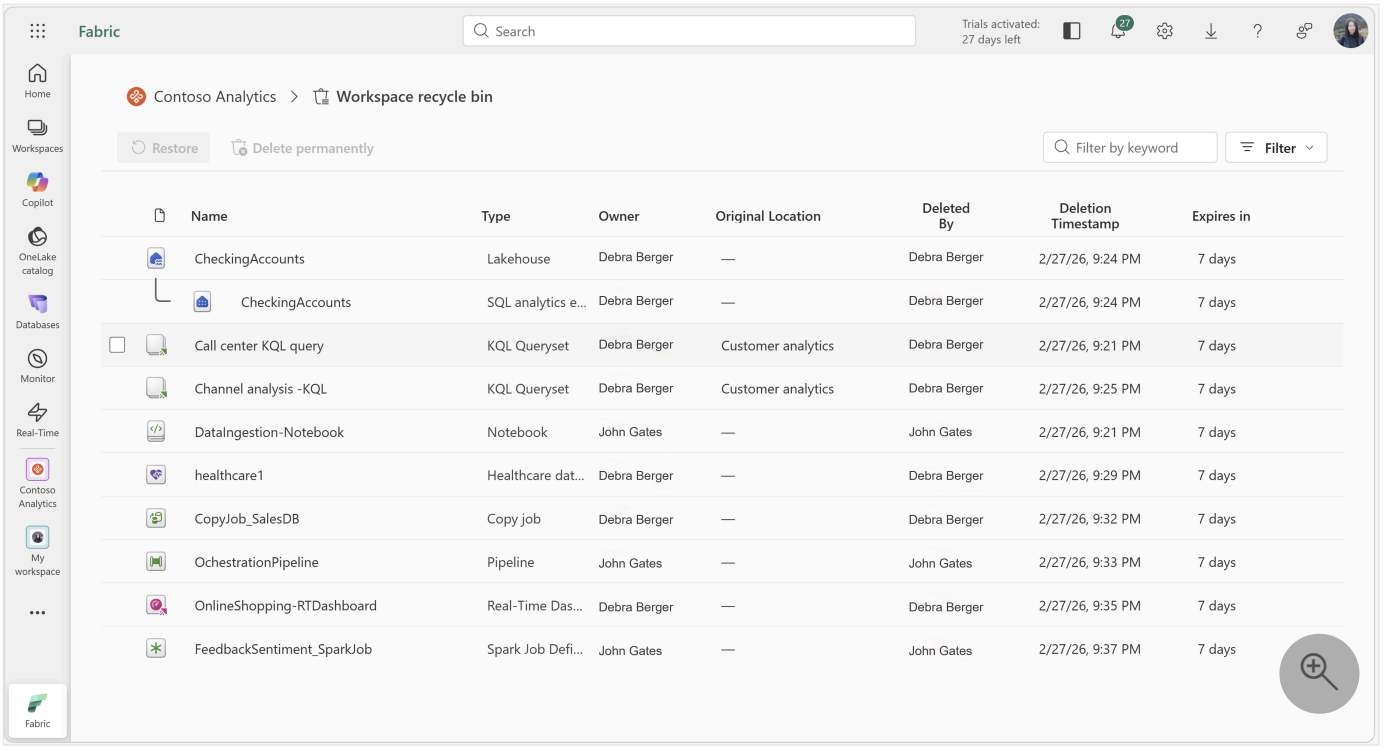
For more information, see Remove a workspace from a deployment pipeline.

Workspace recycle bin (preview)

Each workspace has a recycle bin that shows items in a retention period. When you delete a supported item type and turn on the **Item Recovery** tenant setting, the item moves to the recycle bin instead of being permanently deleted. The **Item Recovery** tenant setting is turned on by default.

To open the recycle bin, select **Recycle bin** from the workspace navigation. From the recycle bin, you can:

- **Restore** a deleted item to its original location in the workspace. Workspace contributors, members, and admins can restore items.
- **Delete permanently** to remove an item before the retention period ends. Only workspace admins can permanently delete items.



Items remain in the recycle bin for the duration of the retention period configured by the tenant admin (3 to 90 days). At the end of the retention period, the system automatically permanently deletes items.

For more information, see Recover or permanently delete items.

Administering and auditing workspaces

Administration for workspaces is in the Microsoft Fabric admin portal. Microsoft Fabric admins decide who in an organization can create workspaces and distribute apps. Read about managing users' ability to create workspaces in the "Workspace settings" article.

Admins can also see the state of all the workspaces in their organization. They can manage, recover, and even delete workspaces. Read about managing the workspaces themselves in the "Admin portal" article.

Auditing

Microsoft Fabric audits the following activities for workspaces.

Expand table

Friendly name	Operation name
Created Microsoft Fabric folder	CreateFolder

Friendly name	Operation name
Deleted Microsoft Fabric folder	DeleteFolder
Updated Microsoft Fabric folder	UpdateFolder
Updated Microsoft Fabric folder access	UpdateFolderAccess

Read more about Microsoft Fabric auditing.

Sign-in problems

See the following table if you have trouble signing in.

Expand table

Message	Meaning	How to fix
`browser_storage_unsupported`	Your browser is blocking access to storage, which is required for signing in.	- Check your browser's privacy settings. - Make sure cookies and local storage (like `sessionStorage` or `localStorage`) are allowed. - Try disabling any extensions that might block storage access.
`cluster_resolution_failure_401`	Your network or proxy is blocking authentication headers needed to connect to the service.	- Ask your IT admin to add **app.powerbi.com** to the allowlist of your proxy or firewall. This helps your network keep the necessary sign-in information in place when trying to connect to the service.
`popup_window_blocked`	Error opening popup window. The browser blocked popups or the environment doesn't support popups.	Allow popups for the site, then retry sign in.

Considerations and limitations

Limitations to be aware of:

- Workspaces can contain a maximum of 1,000 Fabric and Power BI items.
- Certain special characters aren't supported in workspace names when using an XMLA endpoint. As a workaround, use URL encoding of special characters, for example, for a forward slash **/**, use **%2F**.

- A user or a service principal can be a member of up to 1,000 workspaces.

Related content

- Create workspaces
- Give users access to workspaces
- Recover or permanently delete items

Last updated on 07/28/2026

Create a workspace

Summarize this article for me

This article explains how to create *workspaces* in Microsoft Fabric. In workspaces, you create collections of items such as lakehouses, warehouses, and reports. For more background, see the Workspaces article.

To create a workspace:

1. In the nav pane, select **Workspaces**.
2. At the bottom of the Workspace pane that opens, select **New workspace**.
3. The **Create a workspace** pane opens.
    - Give the workspace a unique name (mandatory).
    - Provide a description of the workspace (optional).
    - Assign the workspace to a domain (optional).

      If you are a domain contributor for the workspace, you can associate the workspace to a domain, or you can change an existing association. For information about domains, see Domains in Fabric.
4. When done, either continue to the advanced settings, or select **Apply**.

You can open and work across multiple workspaces side by side. Items are color-coded and numbered to indicate which workspace they belong to. To learn more see, object explorer and tabbed navigation in Fabric portal.

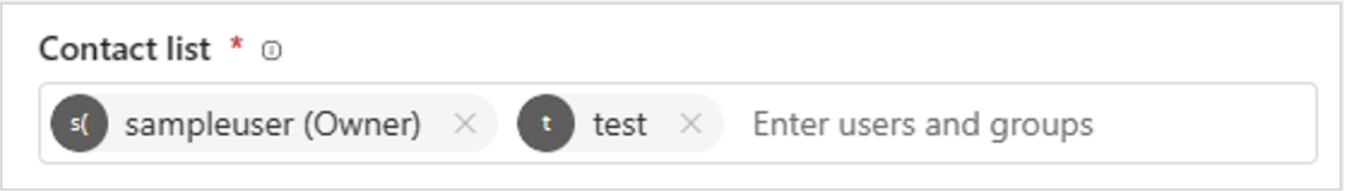
Advanced settings

Expand **Advanced** and you see advanced setting options:

Contact list

Contact list is a place where you can put the names of people as contacts for information about the workspace. Accordingly, people in this contact list receive system email notifications for workspace level changes.

By default, the first workspace admin who created the workspace is the contact. You can add other users or groups according to your needs. Enter the name in the input box directly, it helps you to automatically search and match users or groups in your org.



Workspace type

Different workspace types provide different sets of features for your workspace. You can change the workspace type after creating a workspace. You can do so from the workspace settings, but some migration effort is needed.

> **Note**
>
> Currently, if you want to downgrade the workspace type from a Power BI Premium to a Power BI Pro (Shared capacity), you must first remove any non-Power BI Fabric items that the workspace contains. Only after you remove such items will you be allowed to downgrade the capacity. For more information, see Moving data around.

Default storage format

Power BI semantic models can store data in a highly compressed in-memory cache for optimized query performance, enabling fast user interactivity. With Premium capacities, large semantic models beyond the default limit can be enabled with the Large semantic model storage format setting. When enabled, semantic model size is limited by the Premium capacity size or the maximum size set by the administrator. Learn more about large semantic model storage format.

Template apps

Power BI template apps are developed for sharing outside your organization. If you check this option, a special type of workspace (template app workspace) is created. It's not possible to revert it back to a normal workspace after creation.

Dataflow storage (preview)

Data used with Power BI is stored in internal storage provided by Power BI by default. With the integration of dataflows and Azure Data Lake Storage Gen 2 (ADLS Gen2), you can store your dataflows in your organization's Azure Data Lake Storage Gen2 account. Learn more about dataflows in Azure Data Lake Storage Gen2 accounts.

Reassign a workspace to a different capacity

When you create a workspace, it's assigned to a capacity. The capacity that new workspaces are assigned to by default is determined by the capacity type and/or by the configuration of Fabric or capacity administrators. After a workspace is created, you can reassign it to another available capacity if you have the workspace admin role, the capacity admin role, or the capacity contributor role.

You can reassign a workspace to a different capacity via workspace types in the workspace settings.

1. Open the workspace settings and choose **Workspace type**. Information about the current workspace type is displayed.
2. Select **Edit**. The list of available workspace types appears.
3. Select the desired workspace type and specify the capacity the workspace will be hosted on.

> **Note**
>
> - You can choose specific capacities only when you select one of the Fabric workspace types. Fabric automatically reserves shared capacity for Power BI workspace types.
> - The types of items contained in the workspace can affect the ability to change workspace types and/or move the workspace to a capacity in a different region. See Moving data around for detail.

Give users access to your workspace

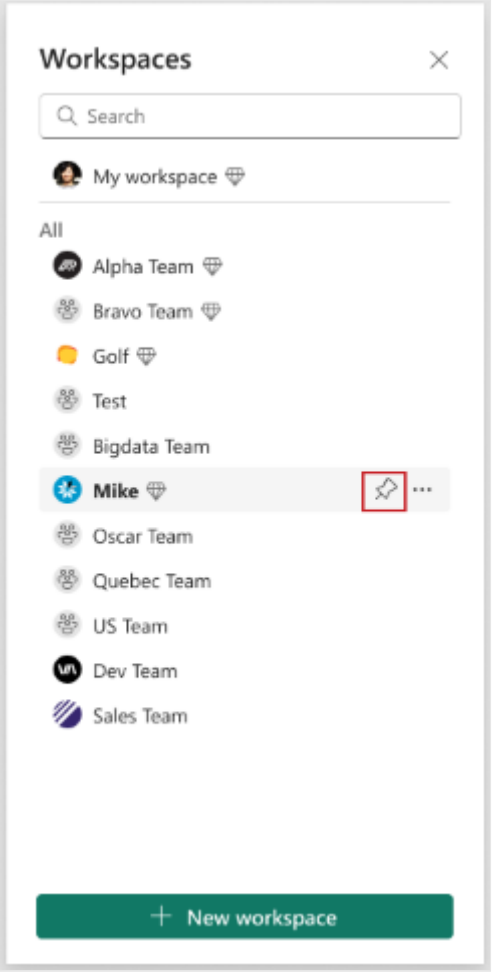
Now that you've created the workspace, you'll want to add other users to *roles* in the workspace, so you can collaborate with them. See these articles for more information:

- Give users access to a workspace
- Roles in workspaces

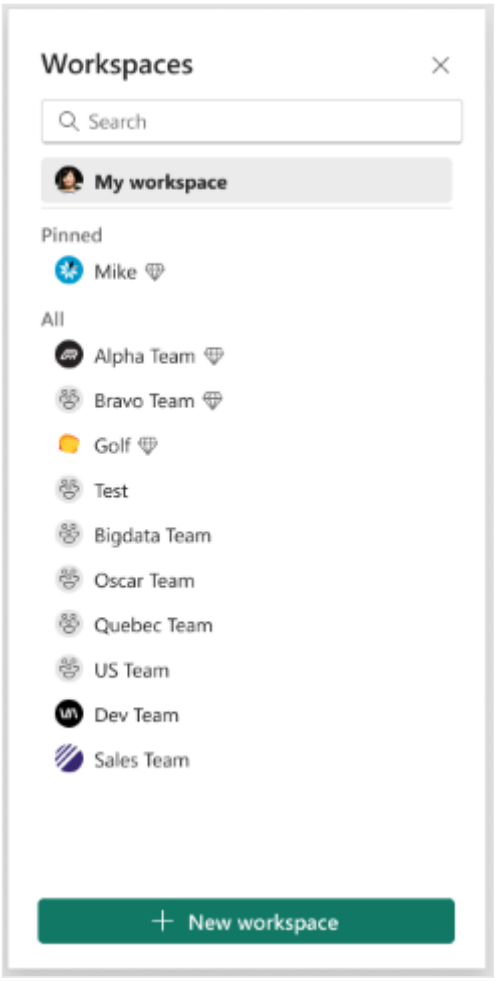
Pin workspaces

Quickly access your favorite workspaces by pinning them to the top of the workspace flyout list.

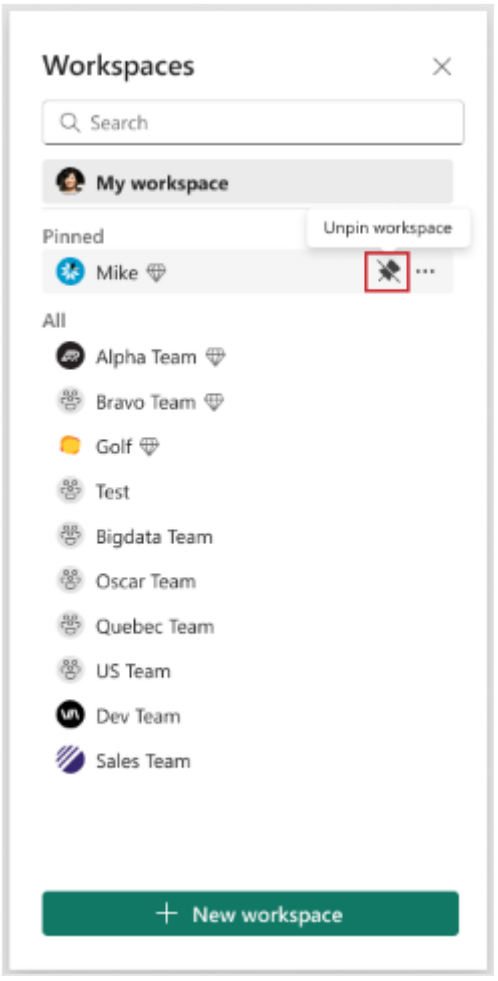
1. Open the workspace flyout from the nav pane and hover over the workspace you want to pin. Select the **Pin to top** icon.



2. The workspace is added in the **Pinned** list.



3. To unpin a workspace, select the unpin button. The workspace is unpinned.



Related content

- Read about workspaces

Last updated on 02/12/2026

Roles in workspaces in Microsoft Fabric

Workspace roles let you manage who can do what in a Microsoft Fabric workspace. Microsoft Fabric workspaces sit on top of OneLake and divide the data lake into separate containers that can be secured independently. Workspace roles in Microsoft Fabric extend the Power BI workspace roles by associating new Microsoft Fabric capabilities such as data integration and data exploration with existing workspace roles. For more information on Power BI roles, see Roles in workspaces in Power BI.

You can either assign roles to individuals or to security groups, Microsoft 365 groups, and distribution lists. To grant access to a workspace, assign those user groups or individuals to one of the workspace roles: Admin, Member, Contributor, or Viewer. Here's how to give users access to workspaces. Microsoft Entra ID service principals (app registrations) can also be assigned to workspace roles and inherit the same permissions as users for API-based operations in Fabric, including the Items REST API and the Job Scheduler API.

To create a new workspace, see Create a workspace.

Everyone in a user group gets the role that you assign. If someone is in several user groups, they get the highest level of permission that's provided by the roles that they're assigned. If you nest user groups and assign a role to a group, all the contained users have permissions.

Users in workspace roles have the following Microsoft Fabric capabilities, in addition to the existing Power BI capabilities associated with these roles.

Microsoft Fabric workspace roles

Expand table

Capability	Admin	Member	Contributor	Viewer
Update and delete the workspace.	✅			
Add or remove people, including other admins.	✅			
Add members or others with lower permissions.	✅	✅		
Allow others to reshare items.[1]	✅	✅		
Create or modify database items.	✅	✅	✅	

Capability	Admin	Member	Contributor	Viewer
Create or modify database mirroring items.	✅	✅	✅	
Create or modify warehouse items.	✅	✅	✅	
Create workspace identity	✅			
Connect workspace to a Git repository	✅			
View and read content of pipelines, notebooks, Spark job definitions, ML models and experiments, and eventstreams.	✅	✅	✅	✅
View and read content of KQL databases, KQL query-sets, digital twin builder items, and real-time dashboards.	✅	✅	✅	✅
View and read content of event schema sets.[5]	✅	✅	✅	✅
Connect to SQL analytics endpoint of Lakehouse or the Warehouse	✅	✅	✅	✅
Read Lakehouse and Data warehouse data and shortcuts[2] with T-SQL through TDS endpoint (ReadData).	✅	✅	✅	✅
Read Lakehouse and Data warehouse data and shortcuts[2] through OneLake APIs and Spark (ReadAll).	✅	✅	✅	
Read Lakehouse data through Lakehouse explorer (ReadAll).	✅	✅	✅	
Subscribe to OneLake events.	✅	✅	✅	
Write or delete pipelines, notebooks, Spark job definitions, ML models, and experiments, and eventstreams.	✅	✅	✅	
Write or delete Eventhouses[3], KQL Querysets, Real-Time Dashboards, digital twin builder data, and schema and data of KQL Databases, Lakehouses, data warehouses, and shortcuts.	✅	✅	✅	
Create, write, or delete event schema sets, and the schemas and event types they contain.[5]	✅	✅	✅	
Execute or cancel execution of notebooks, Spark job definitions, ML models, and experiments.	✅	✅	✅	
Execute or cancel execution of pipelines.	✅	✅	✅	
View execution output of pipelines, notebooks, ML models and experiments.	✅	✅	✅	✅
Schedule data refreshes via the on-premises gateway.[4]	✅	✅	✅	
Modify gateway connection settings.[4]	✅	✅	✅	

[1] Contributors and Viewers can also share items in a workspace, if they have Reshare permissions.

[2] Other permissions are needed to read data from shortcut destination. Learn more about shortcut security model.

[3] Other permissions are needed to perform certain operations on data in an Eventhouse. Learn more about the hybrid role-based access control model.

[4] Keep in mind that you also need permissions on the gateway. Those permissions are managed elsewhere, independent of workspace roles and permissions.

[5] Publishing and consuming business event data is governed separately by data access roles, not by workspace roles. Learn more about managing data access for business events.

> **Note**
>
> The Create/Modify and Write/Delete permissions for notebooks apply equally when using the Items REST API. Admin, Member, and Contributor can perform notebook CRUD operations via the API, while Viewer cannot. For Job Scheduler API execution, Admin, Member, and Contributor can start and cancel runs; all roles including Viewer can monitor run status and view execution output, including run metadata such as status and exit values returned by notebook runs.

Related content

- Roles in workspaces in Power BI
- Create workspaces
- Give users access to workspaces
- Fabric and OneLake security
- OneLake shortcuts
- Data warehouse security
- Data engineering security
- Data science roles and permissions
- Role-based access control in Eventhouse

Note: The author created this article with assistance from AI. Learn more

Last updated on 08/18/2026

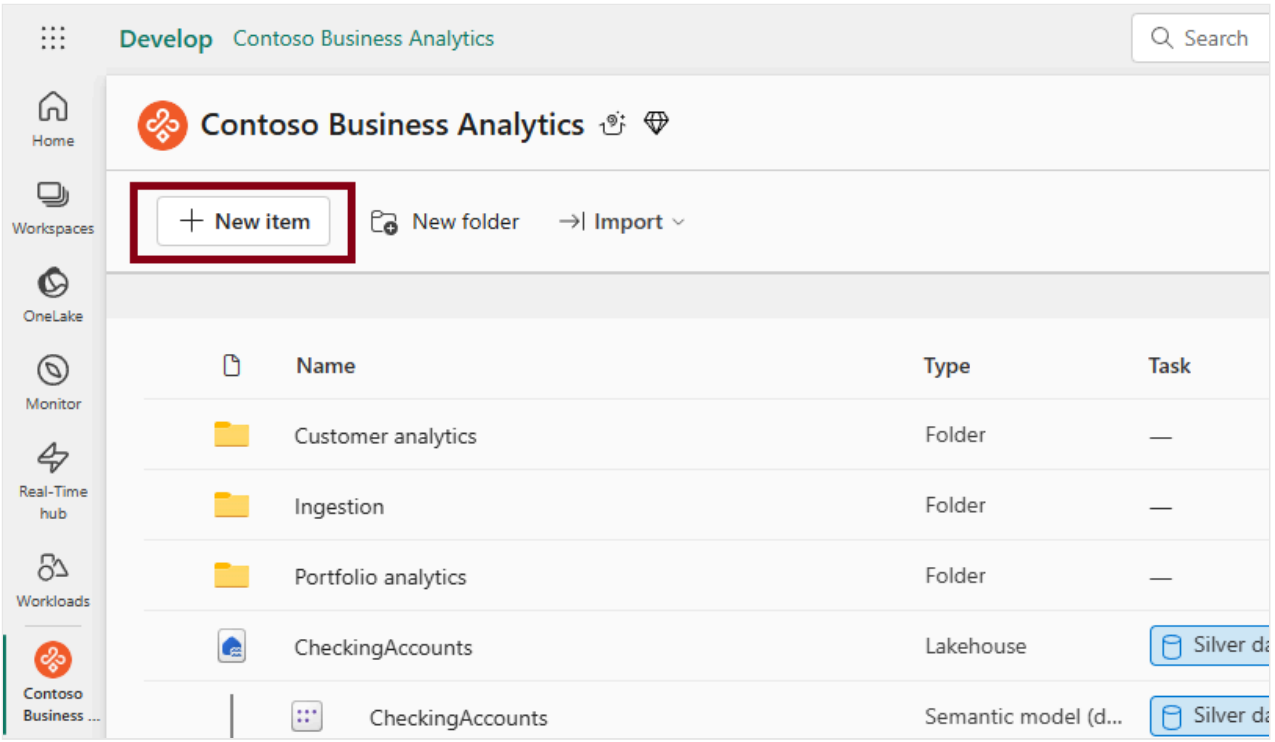
Create items in workspaces

09/16/2025

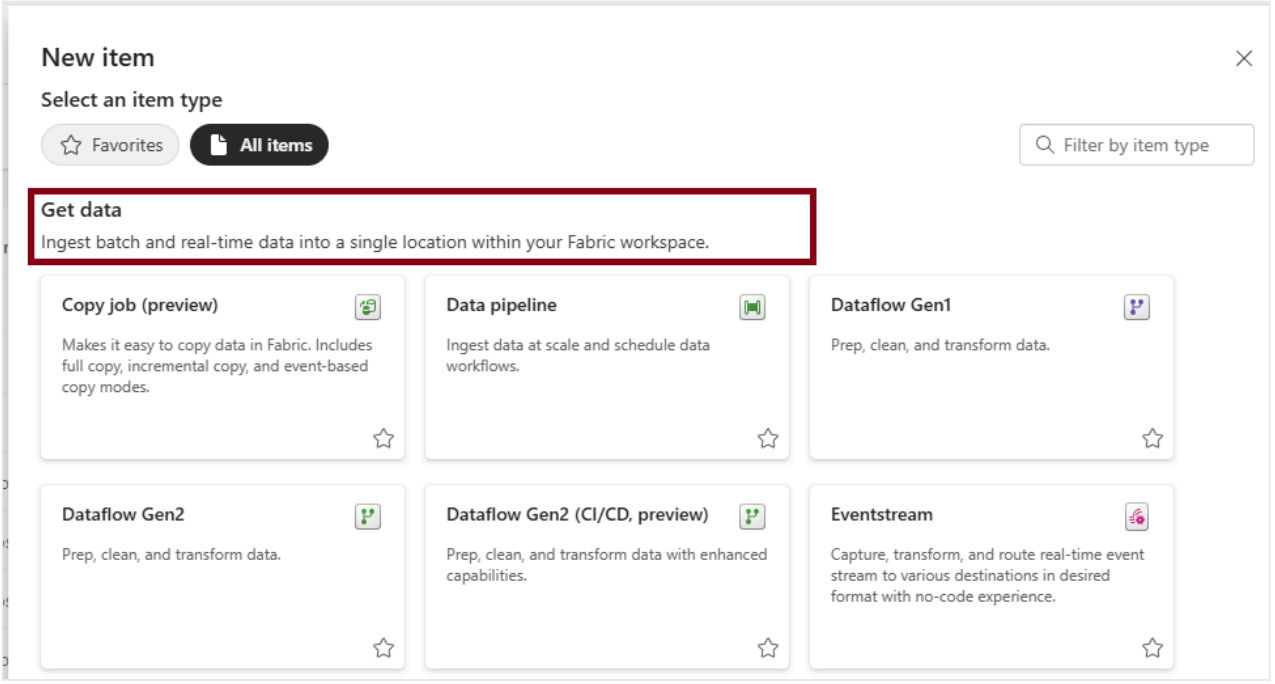
This article explains how to create items in workspaces in Microsoft Fabric. For more information about items and workspaces, see the Microsoft Fabric terminology and Workspaces article.

Create an item in a workspace

1. In a workspace, select **New item**



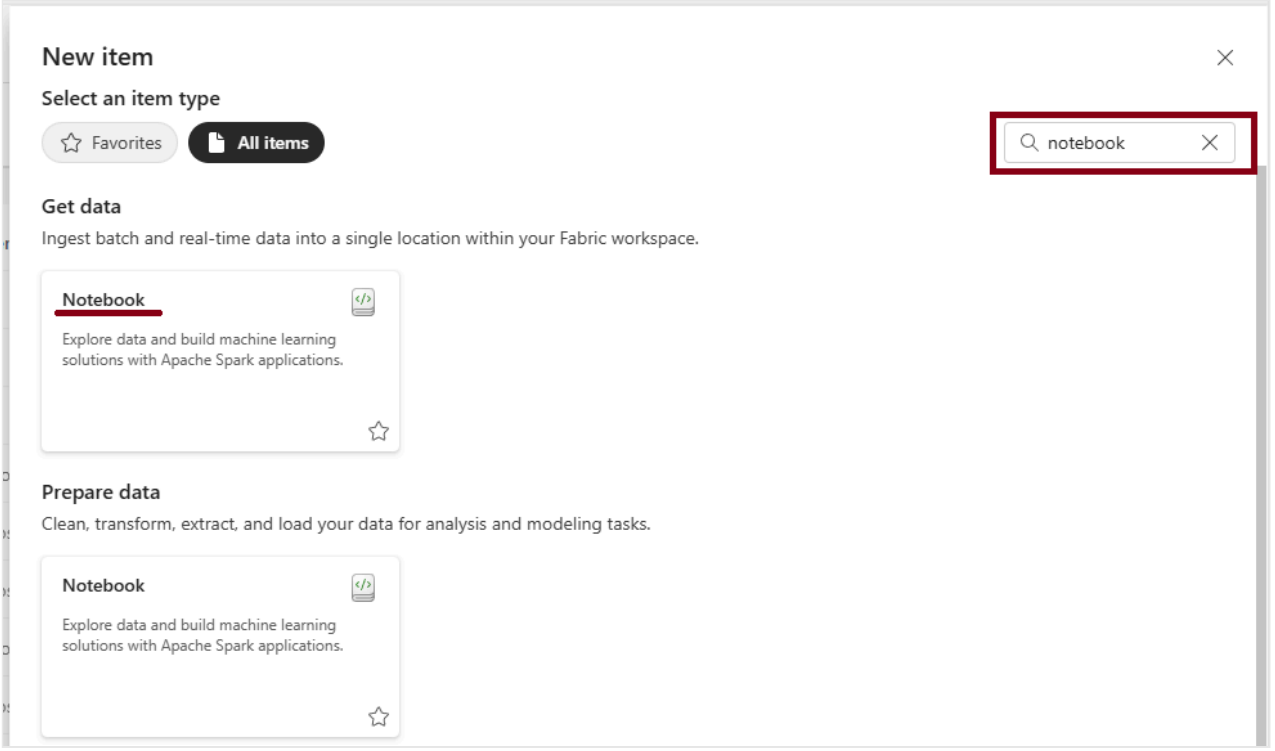
2. You can see all items are categorized by tasks. Each task represents daily job-to-be-done when you build a data solution: get data, store data, prepare data, analyze and train data, track data, visualize data, and develop data. Inside each category, item types are sorted alphabetically. You can scroll down and up to browse all item types which are available for you to create.



3. Select the card of item type you need to create, you can start the creation process of an item.

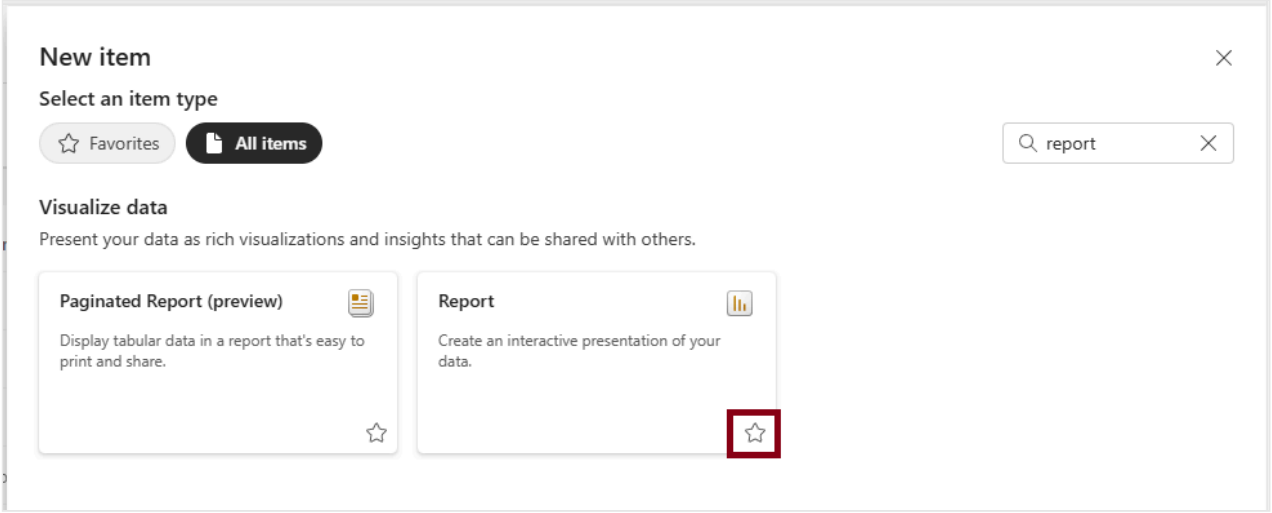
Search by item type

1. To find out the item type you need, enter the keyword of an item type, you can search in this panel.

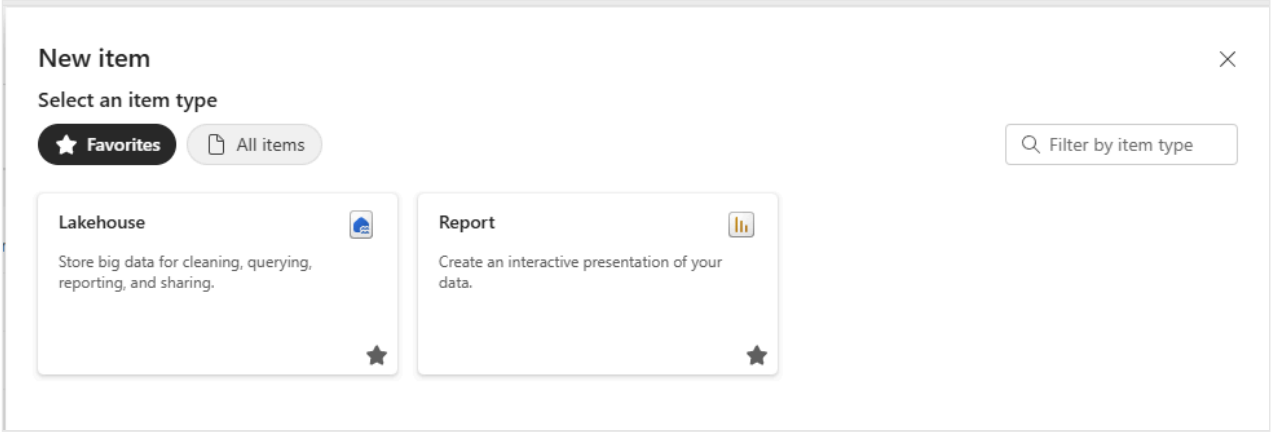


Add items to Favorites

1. Select the star button on a card of an item type, you can add this item type to your 'Favorites'

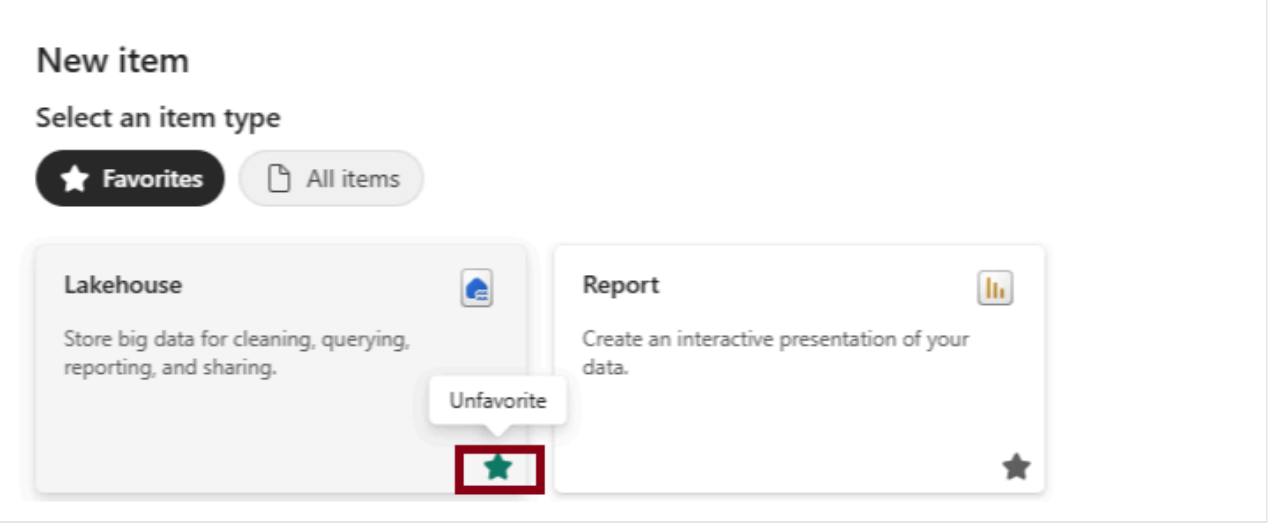


2. Select 'Favorite' and you can see all item types you added to 'Favorites'



3. Next time, when you select 'New item' button, 'Favorites' is shown by default so that you can quickly access the items you need to create most frequently

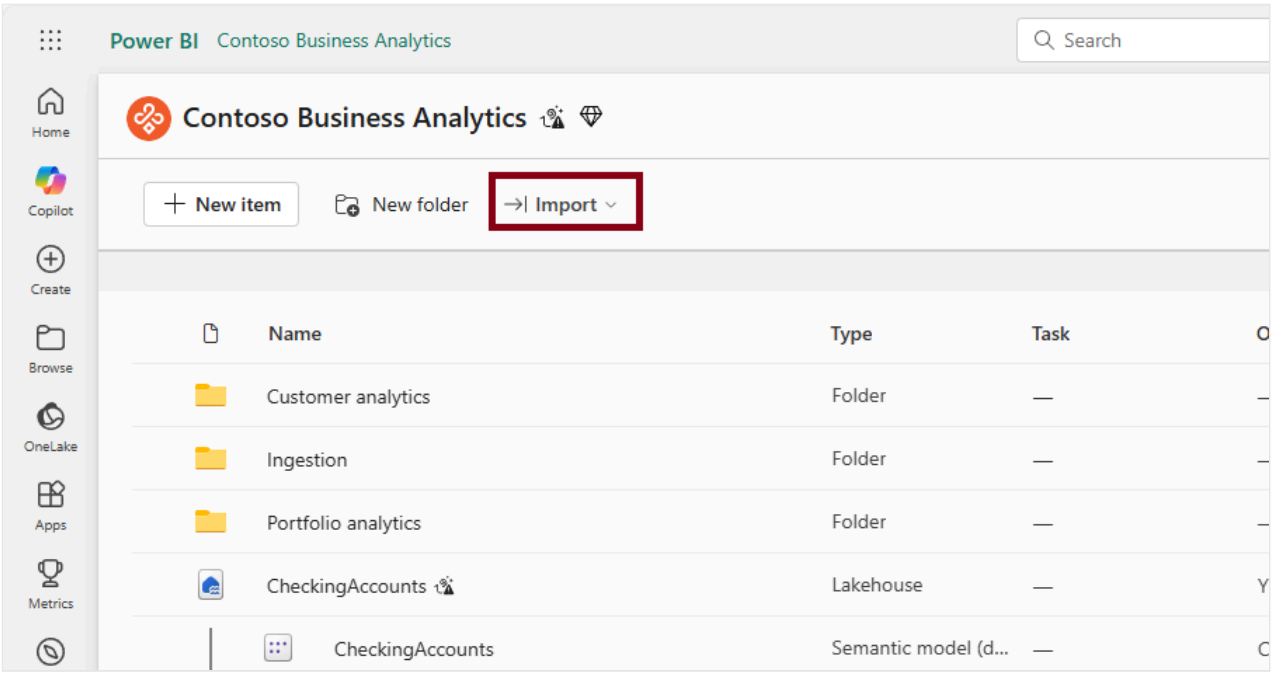
4. By clicking on the star button again, you can unfavorite the item types.



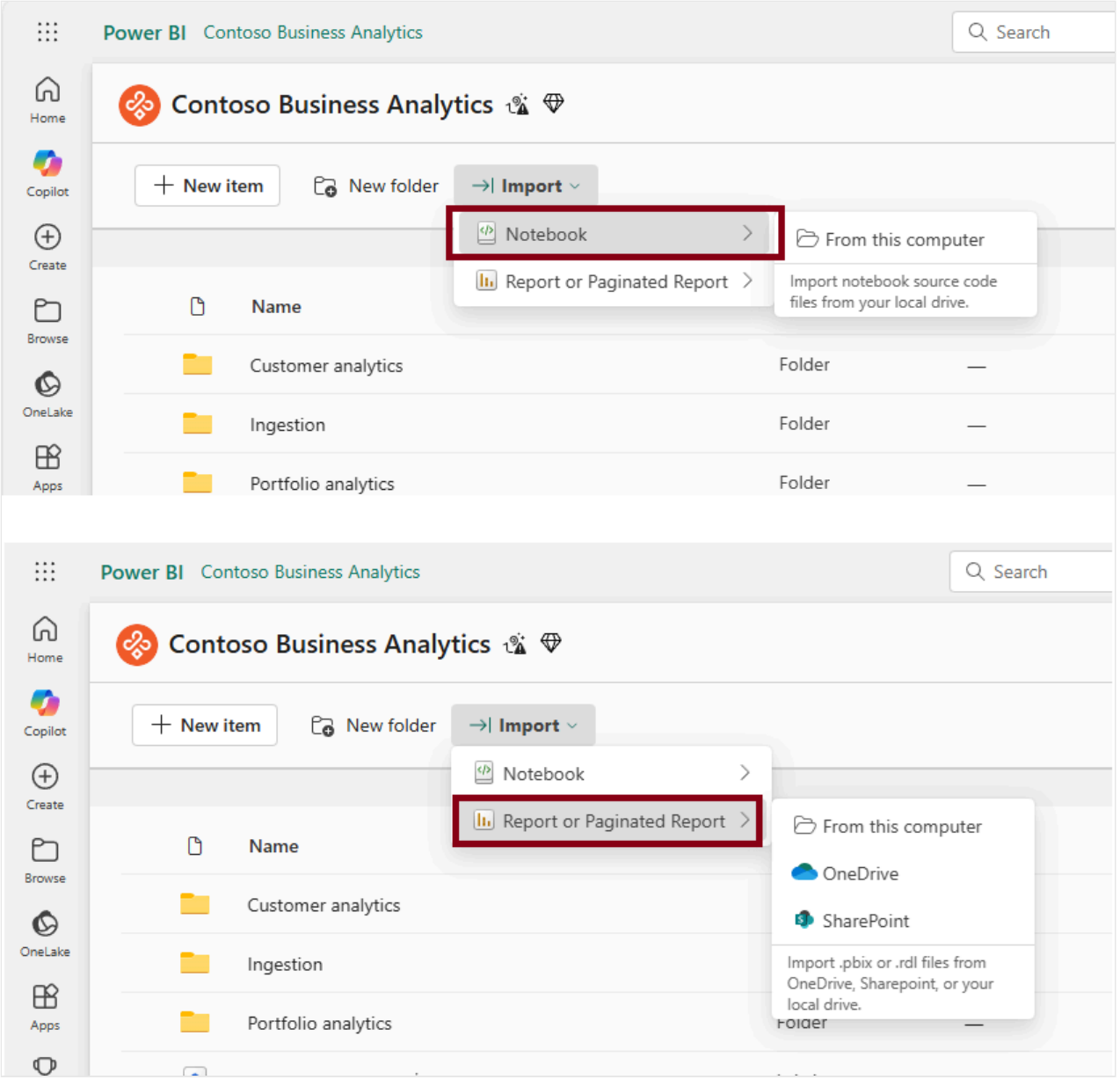
Import items

You can also import files from outside Fabric to create Fabric items in a workspace.

1. Select 'Import' in a workspace, you can see all item types you can create by importing the files from somewhere else.

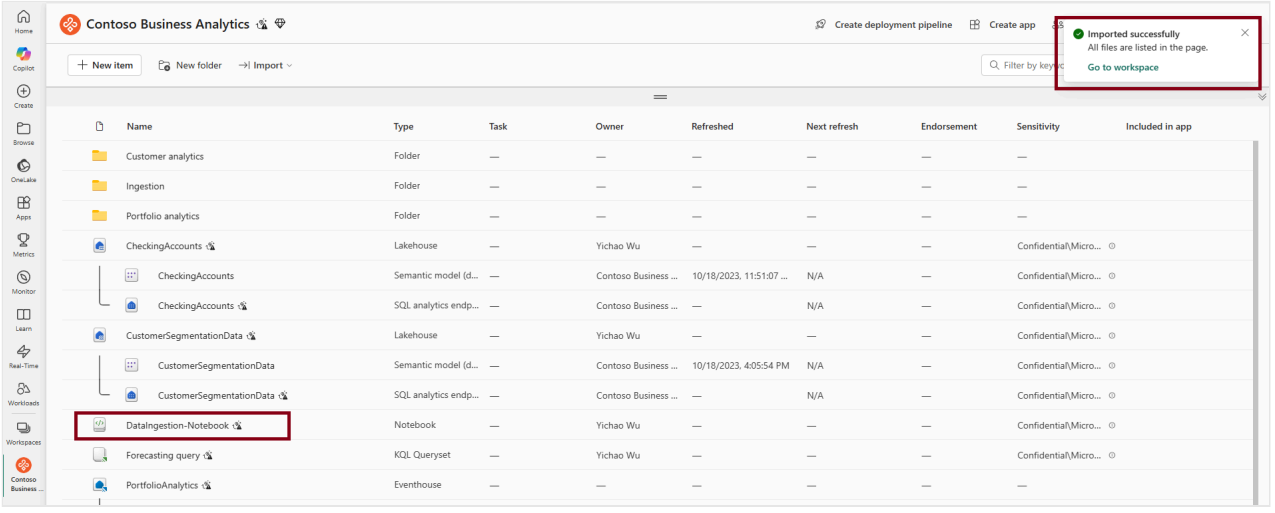


2. Select the item type you want to import, and select the location where your files locate.



3. Select the file you want to import and confirm.

4. Check if new items are created in workspace and the import process is completed successfully.



Rename items

1. In the workspace list, go to the item and click on 'More options'.
2. Select 'Settings' and go to the 'About' tab.
3. Change the name of the item. When renaming items, you can reuse the name of a previously deleted item of the same type within 30 seconds of deletion. You cannot reuse the name of an existing item of the same type in the workspace.

Related content

- Create workspaces

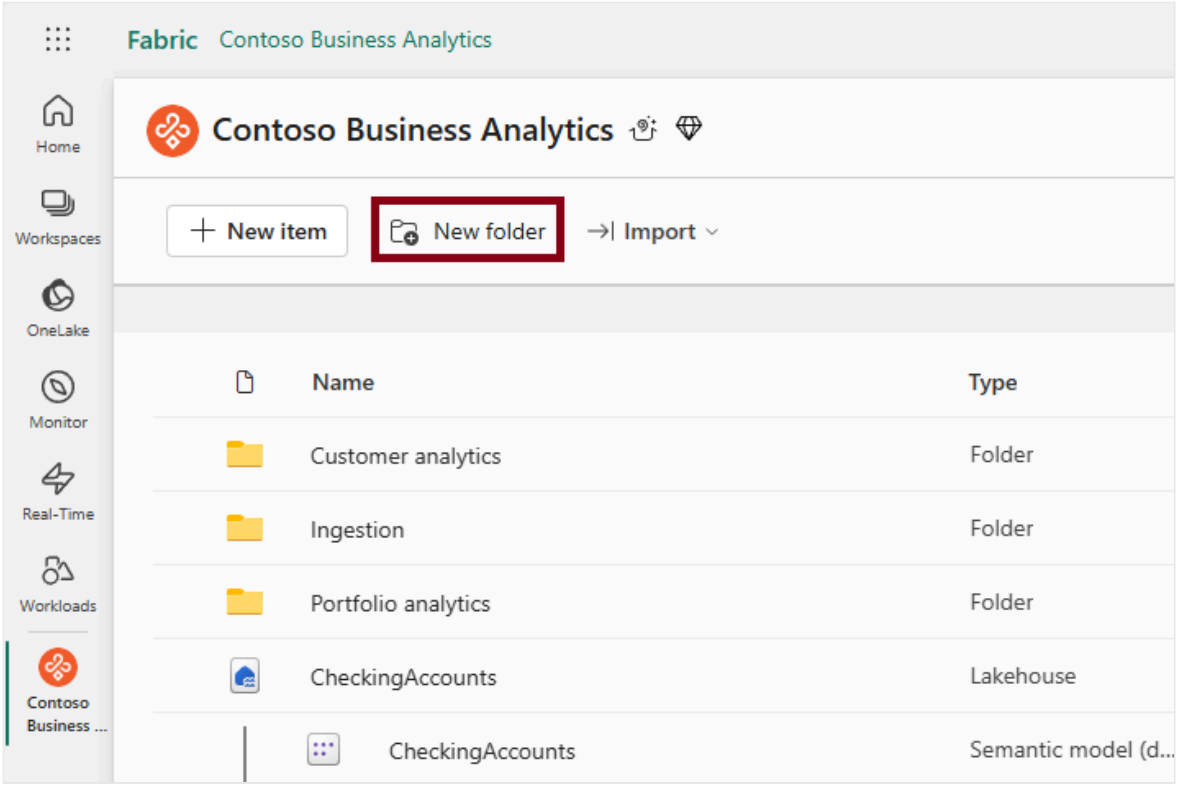
Create folders in workspaces

Article • 01/26/2025

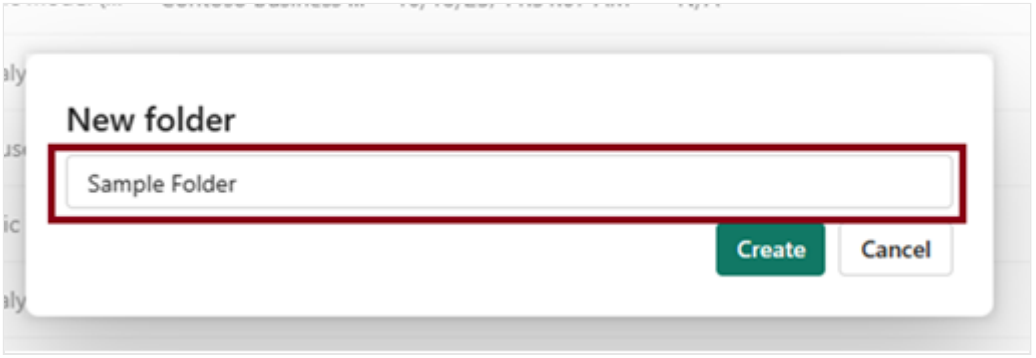
This article explains what folders in workspaces are and how to use them in workspaces in Microsoft Fabric. Folders are organizational units inside a workspace that enable users to efficiently organize and manage artifacts in the workspace. For more information about workspaces, see the Workspaces article.

Create a folder in a workspace

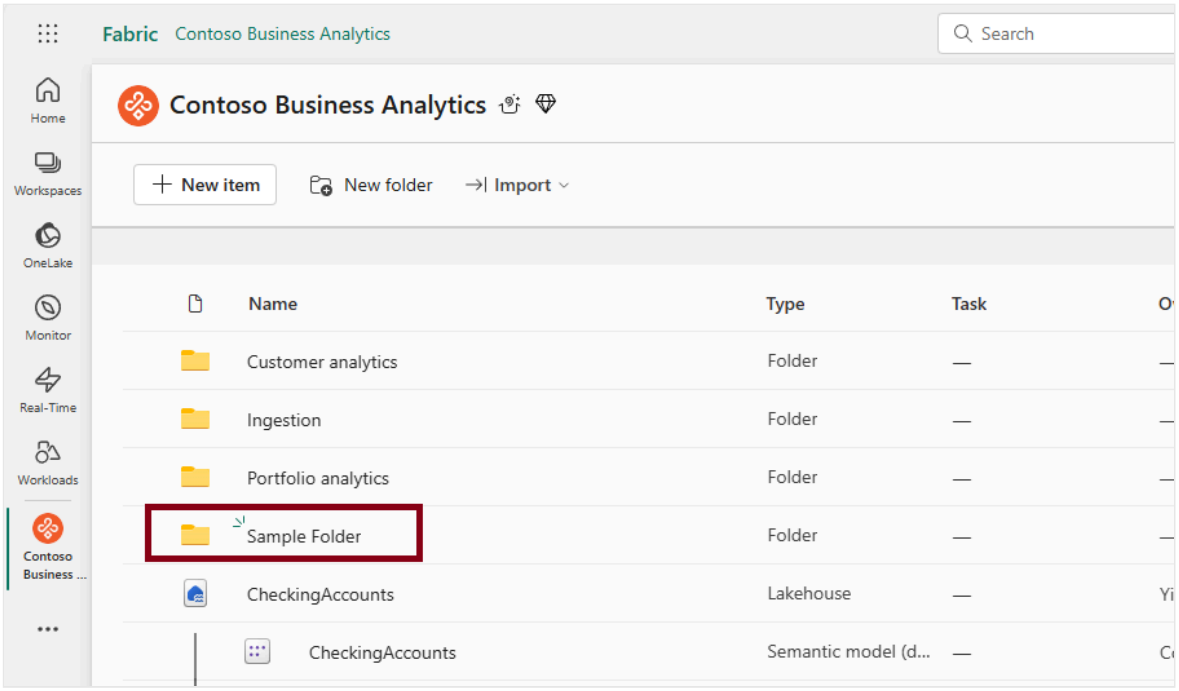
1. In a workspace, select **New folder**.



2. Enter a name for the folder in the **New folder** dialog box. See Folder name requirements for naming restrictions.



3. The folder is created successfully.



4. You can create nested subfolders in a folder in the same way. A maximum of 10 levels of nested subfolders can be created.

> **Note**
>
> You can nest up to 10 folders in the root folder.

Folder name requirements

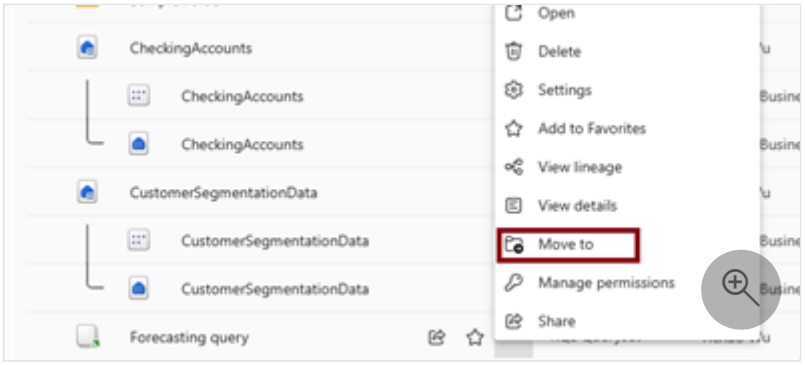
Folder names must follow certain naming conventions:

- The name can't include C0 and C1 control codes.
- The name can't contain leading or trailing spaces.
- The name can't contain these characters: ~"#.&*:<>?/{|}.
- The name can't contain system-reserved names, including: $recycle.bin, recycled, recycler.
- The name length can't exceed 255 characters.
- You can't have more than one folder with the same name in a folder or at the root level of the workspace.

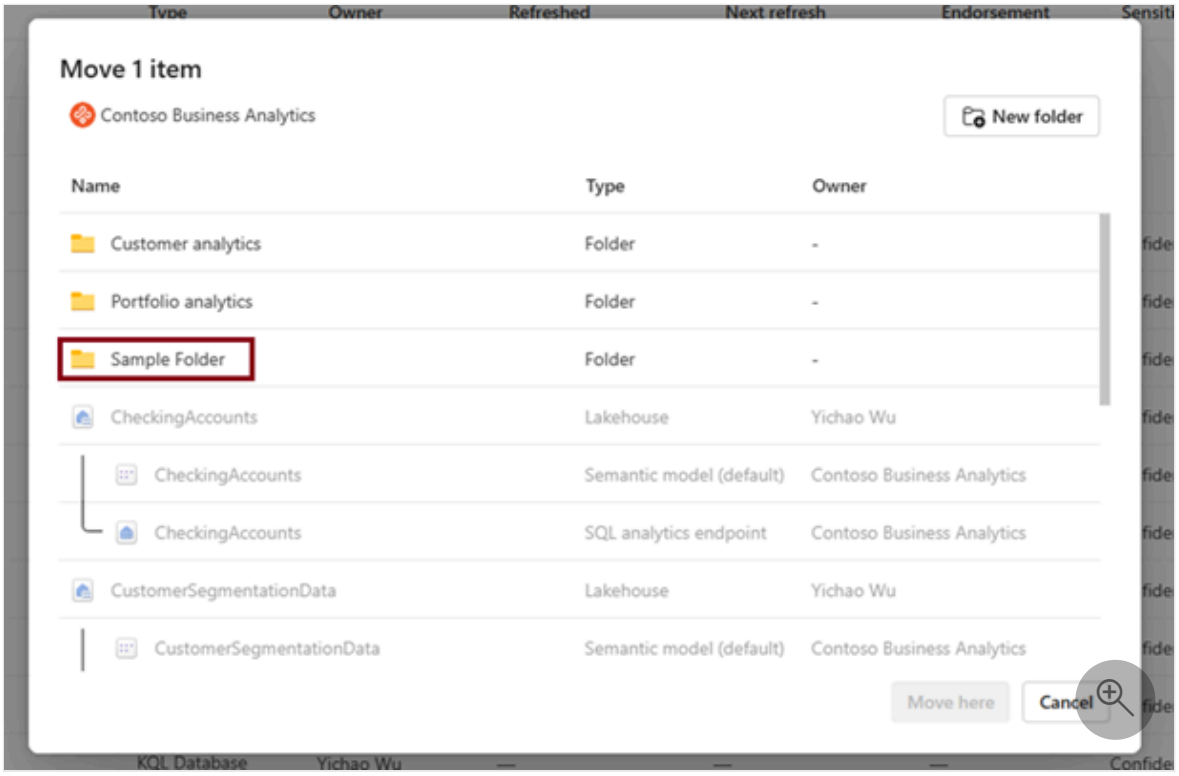
Move items into a folder

Move a single item

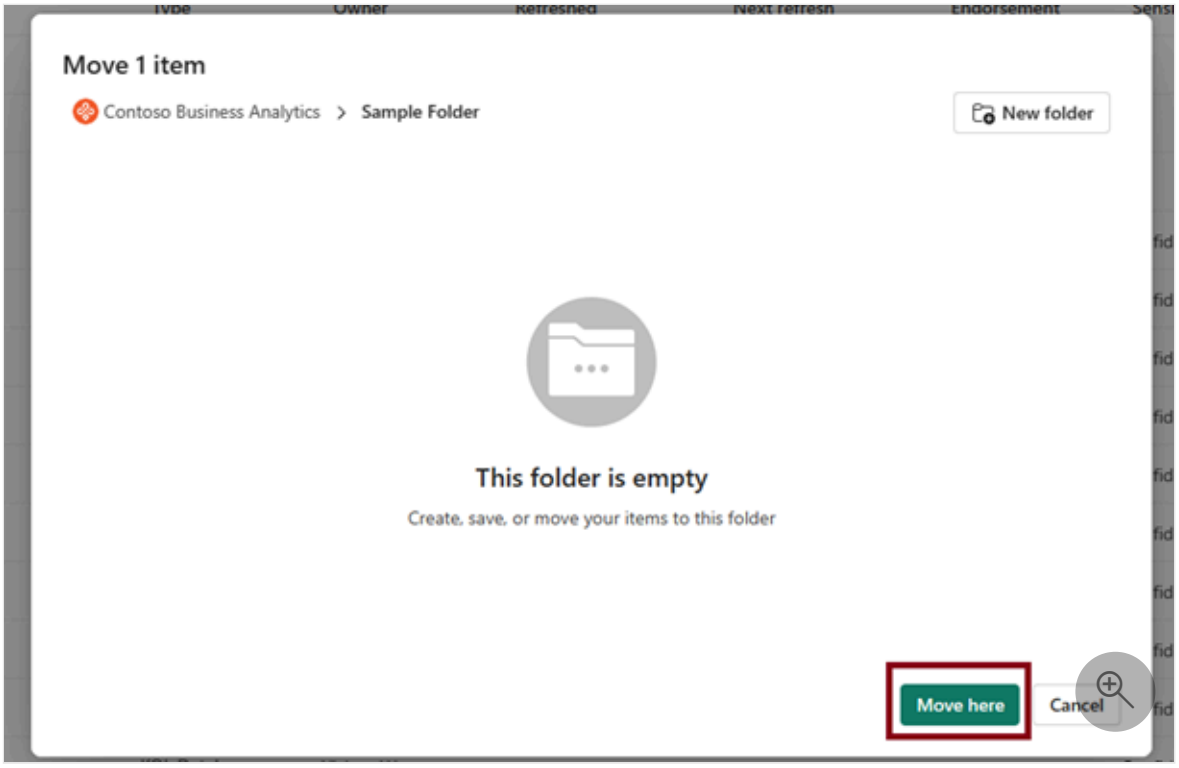
1. Select the context menu (...) of the item you want to move, then select **Move to**.



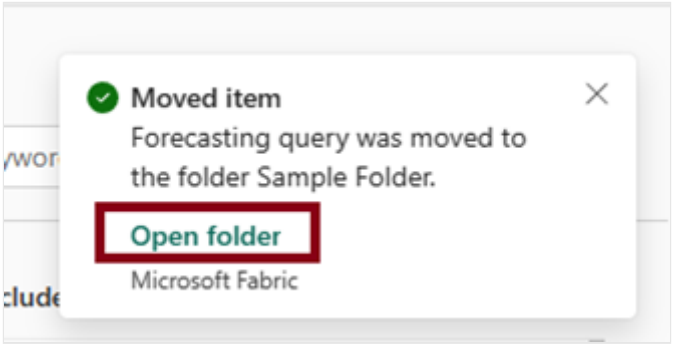
2. Select the destination folder where you want to move this item.



3. Select **Move here**.

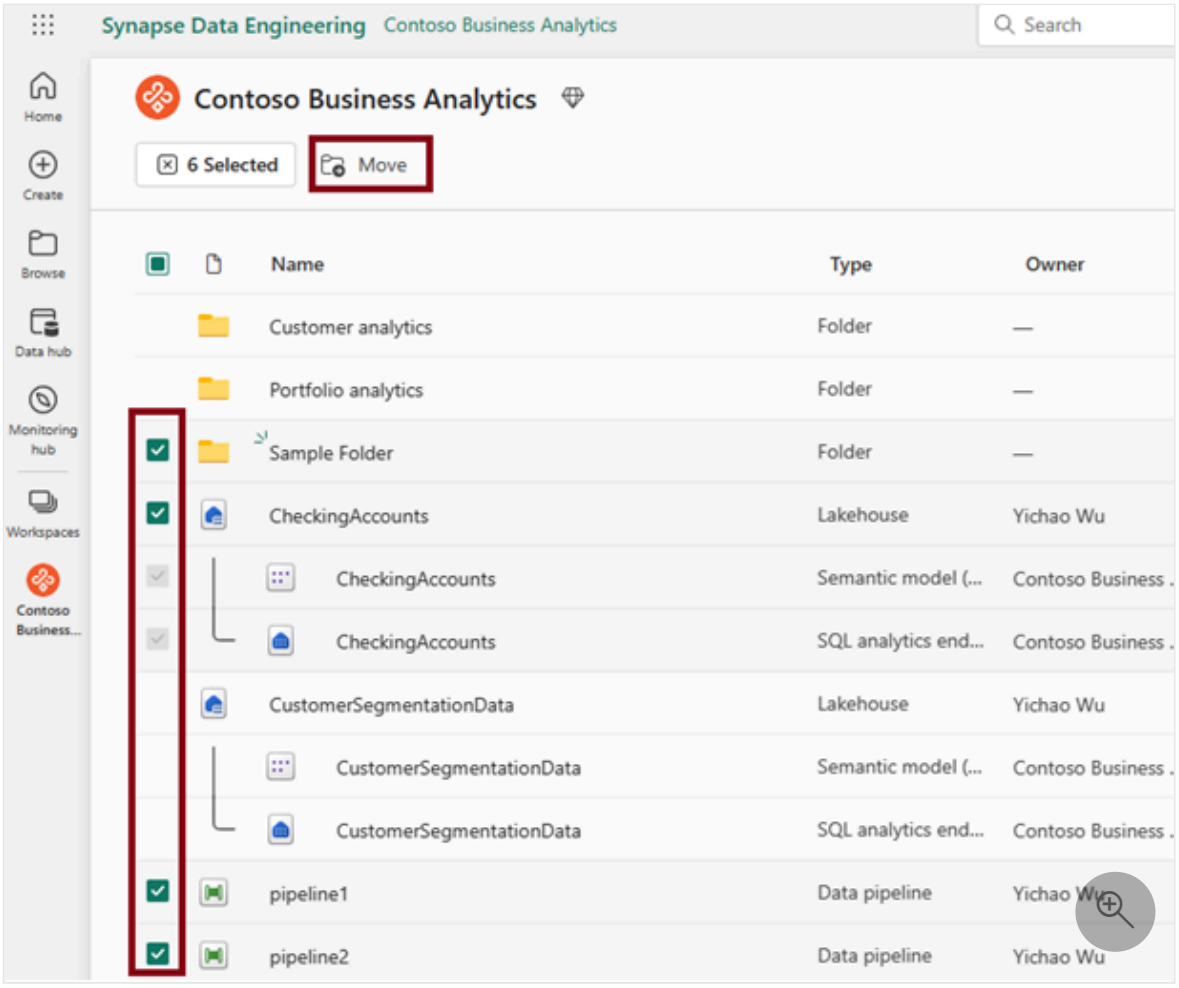


4. By selecting **Open folder** in the notification or navigating to the folder directly, you can go to the destination folder to check if the item moved successfully.

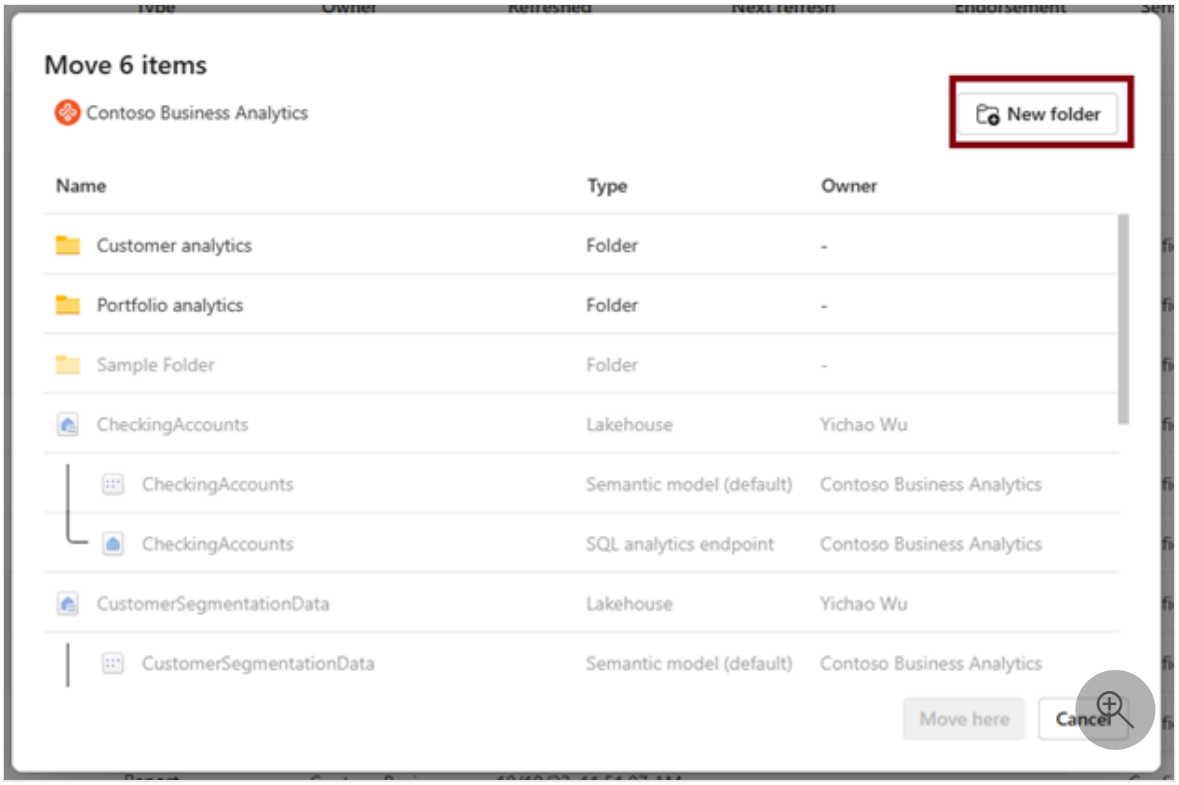


Move multiple items

1. Select multiple items, then select **Move** from the command bar.

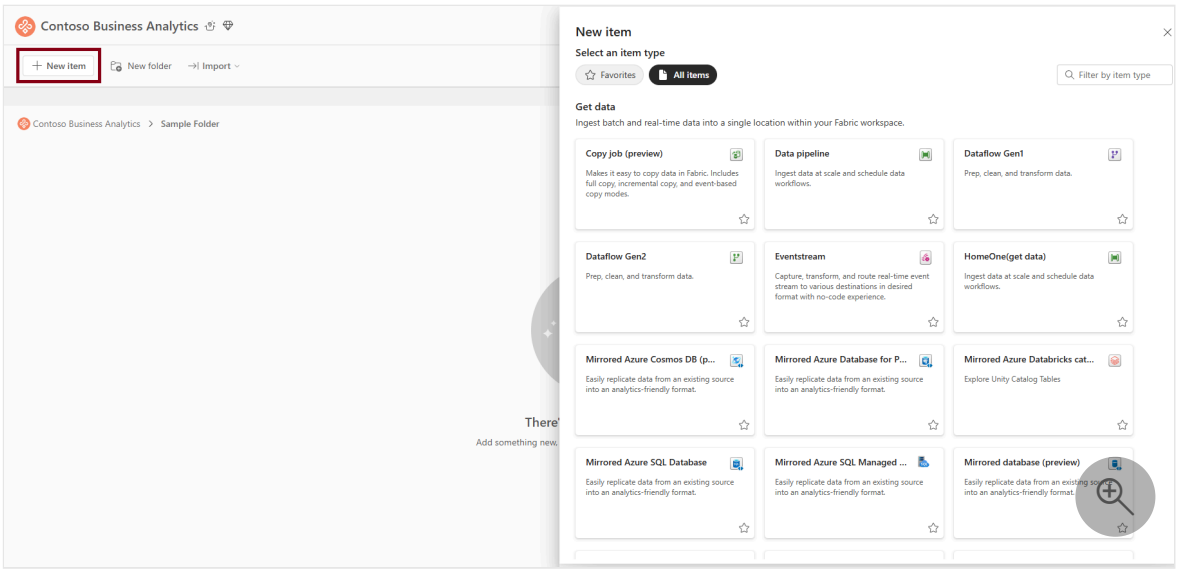


2. Select a destination where you want to move these items. You can also create a new folder if you need it.



Create an item in a folder

1. Go to a folder, select **New**, then select the item you want to create. The item is created in this folder.

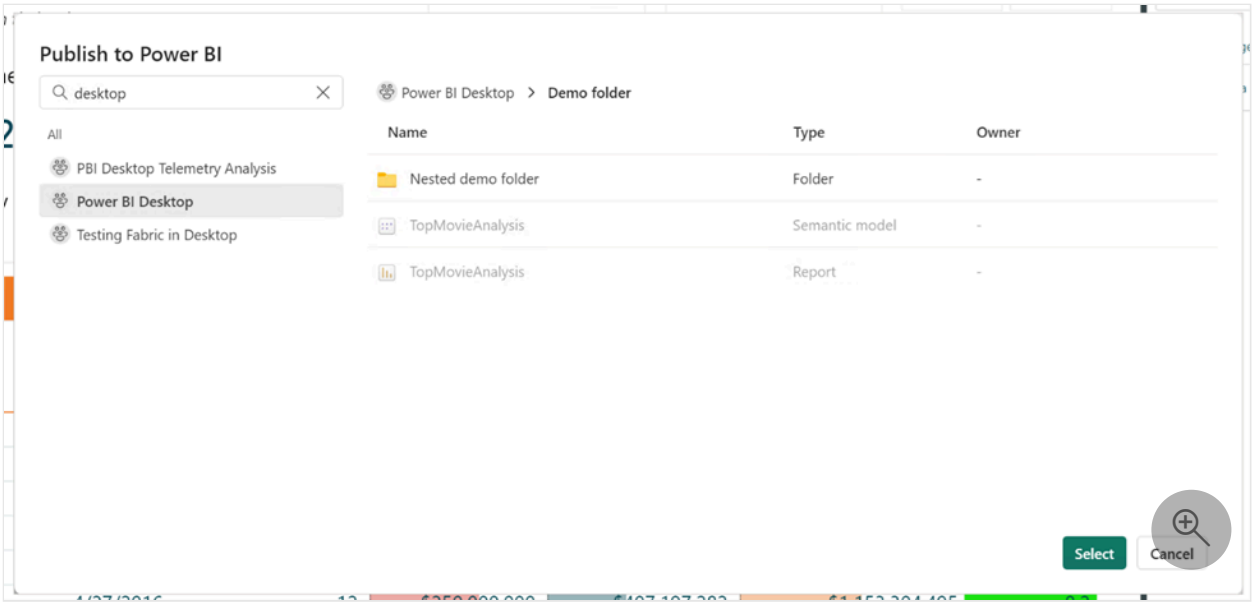


> **Note**
>
> Currently, you can't create certain items in a folder:
>
> - Dataflows gen2
> - Streaming semantic models
> - Streaming dataflows
>
> If you create items from the home page or the **Create** hub, items are created at the root level of the workspace.

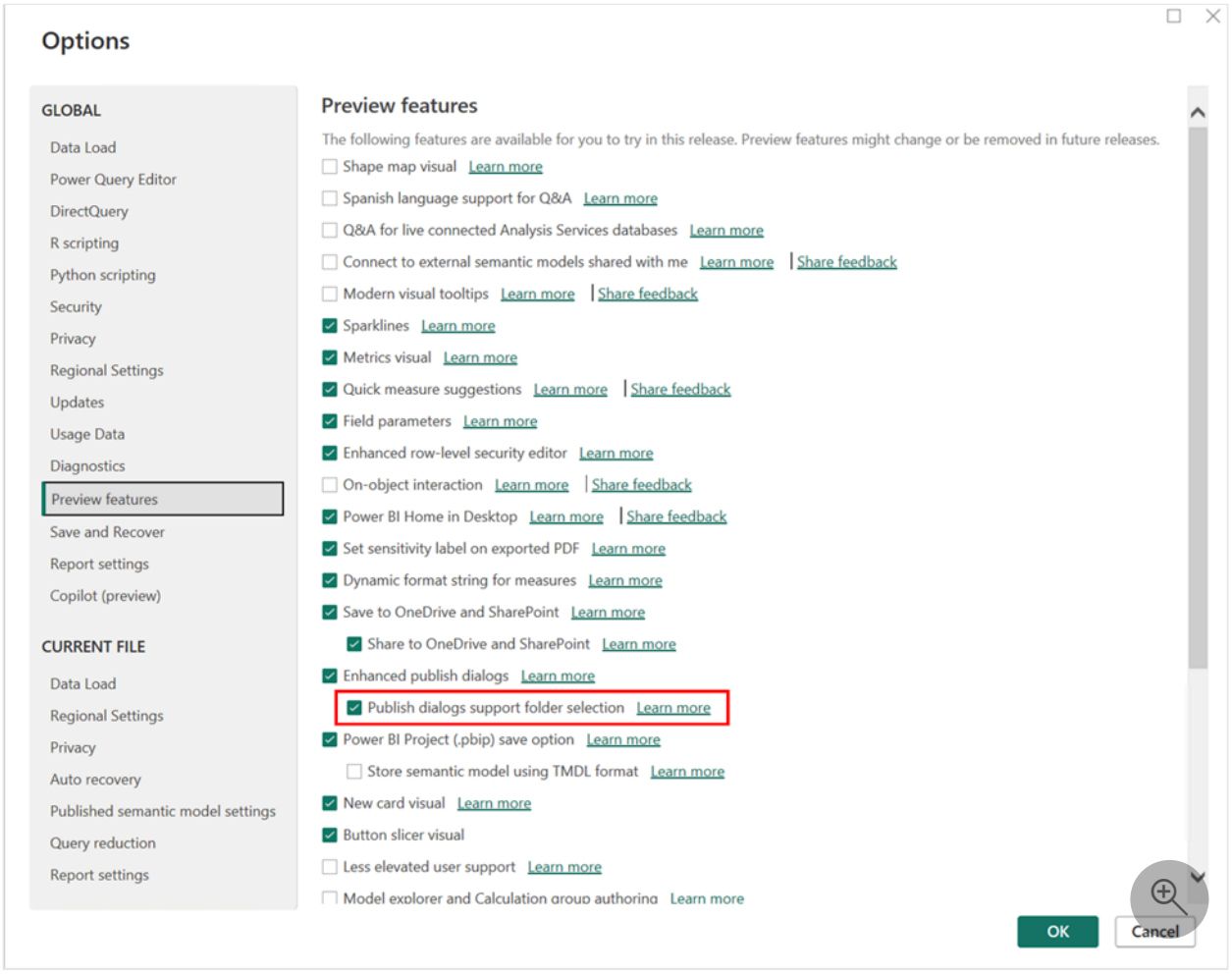
Publish to folder (preview)

You can now publish your Power BI reports to specific folders in your workspace.

When you publish a report, you can choose the specific workspace and folder for your report, as illustrated in the following image.

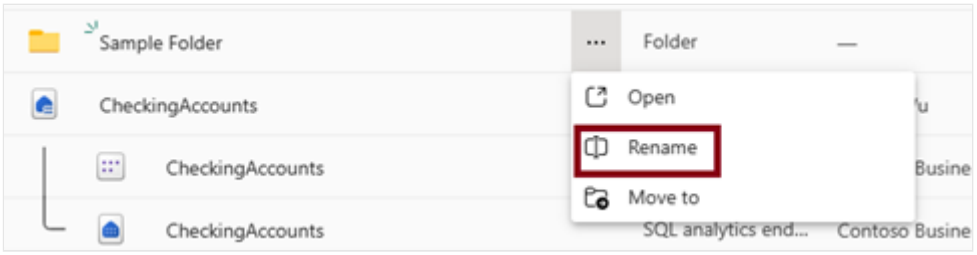


To publish reports to specific folders in the service, make sure that in Power BI Desktop, the **Publish dialogs support folder selection** setting is enabled in the **Preview features** tab in the options menu.

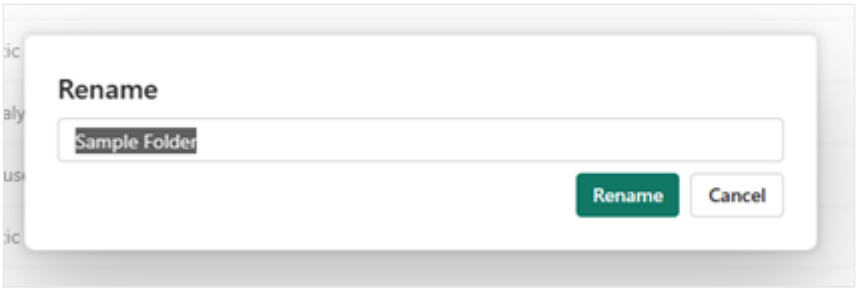


Rename a folder

1. Select the context (...) menu, then select **Rename**.



2. Give the folder a new name and select the **Rename** button.

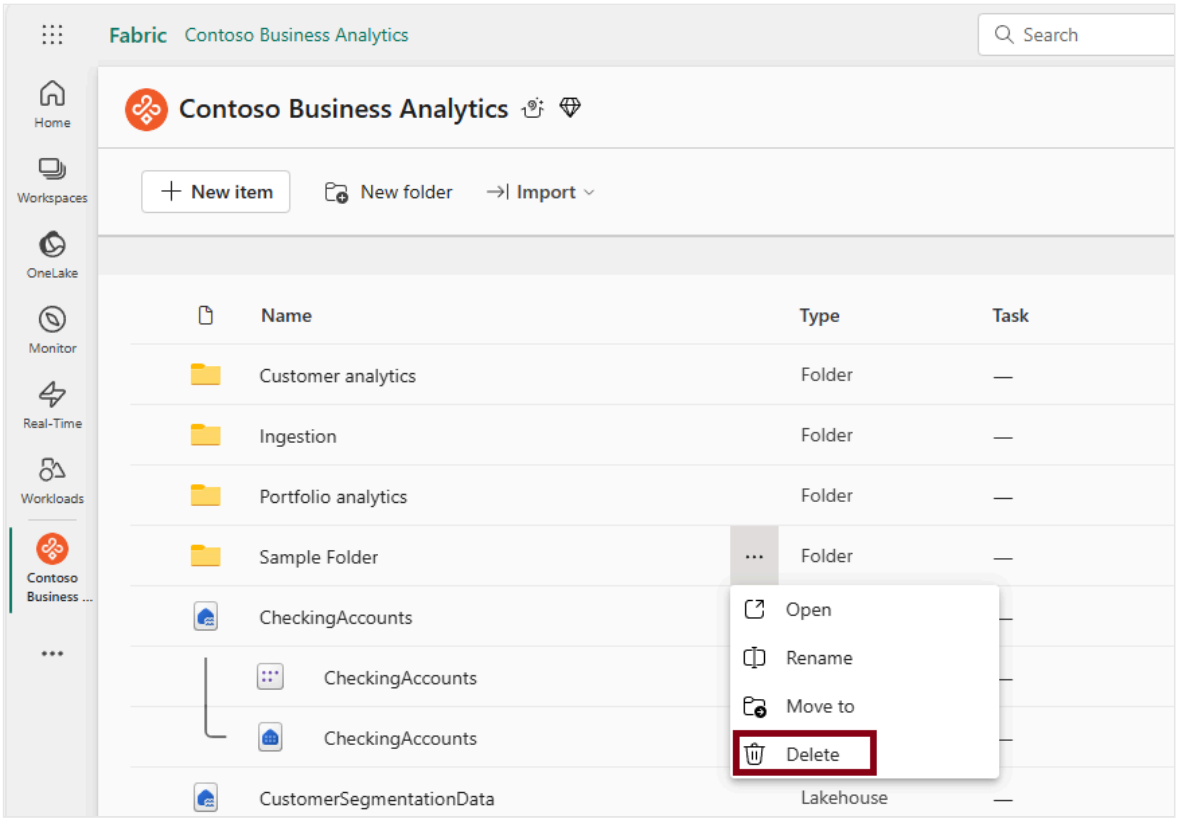


> ⓘ **Note**
>
> When renaming a folder, follow the same naming convention as when you're creating a folder. See Folder name requirements for naming restrictions.

Delete a folder

1. Make sure the folder is empty.

2. Select the context menu (...) and select **Delete**.



> **Note**
>
> Currently you can only delete empty folders.

Permission model

Workspace admins, members, and contributors can create, modify, and delete folders in the workspace. Viewers can only view folder hierarchy and navigate in the workspace.

Currently, folders inherit the permissions of the workspace where they're located.

Expand table

Capability	Admin	Member	Contributor	Viewer
Create folder	✅	✅	✅	❌
Delete folder	✅	✅	✅	❌
Rename folder	✅	✅	✅	❌
Move folder and items	✅	✅	✅	❌
View folder in workspace list	✅	✅	✅	✅

Considerations and limitations

- Currently dataflows gen2, streaming semantic models, and streaming dataflows can't be created in folders.
- If you trigger item creation from the home page, create hub, and industry solution, items are created at the root level of workspaces.
- Git doesn't currently support workspace folders.
- If folders **is enabled** in the Power BI service but **not enabled** in Power BI Desktop, republishing a report that is in a nested folder will replace the report in the nested folder.
- If Power BI Desktop folders **is enabled** in Power BI Desktop, but **not enabled** in the service and you publish to a nested folder, the report will be published to the general workspace.
- When publishing reports to folders, report names must be unique throughout an entire workspace, regardless of their location. Therefore, when publishing a report to a workspace that has another report with the same name in a different folder, the report will publish to the location of the already existing report. If you want to move the report to a new folder location in the workspace, you need to make this change in the Power BI service.
- Folders are not supported in Template App workspaces.

Related content

- Folders in deployment pipelines
- Create workspaces
- Give users access to workspaces

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Give users access to workspaces

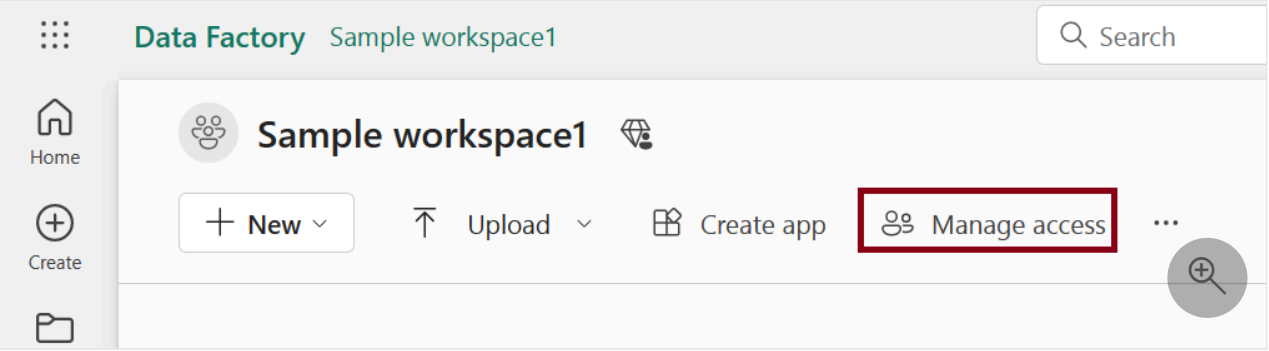
08/12/2025

After you create a workspace in Microsoft Fabric, or if you have an Admin or Member role in a workspace, you can give others access to it by adding them to the different roles. Workspace creators are automatically admins. For an explanation of the different roles, see Roles in workspaces.

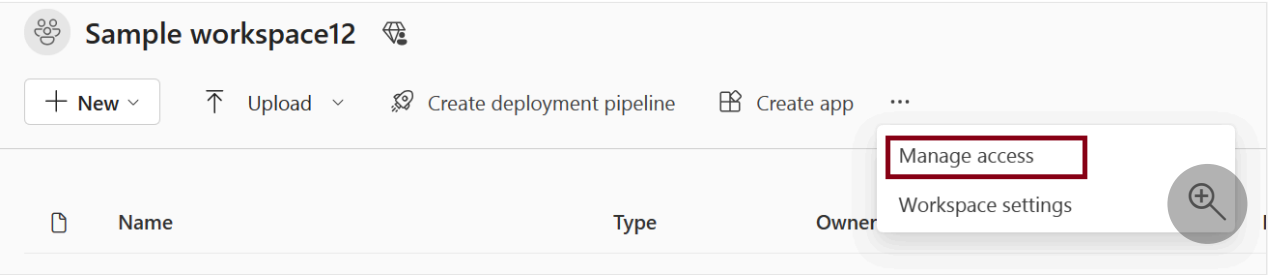
> **Note**
>
> To enforce row-level security (RLS) on Power BI items for Microsoft Fabric Pro users who browse content in a workspace, assign them the Viewer role.
>
> After you add or remove workspace access for a user or a group, the permission change only takes effect the next time the user logs into Microsoft Fabric.

Give access to your workspace

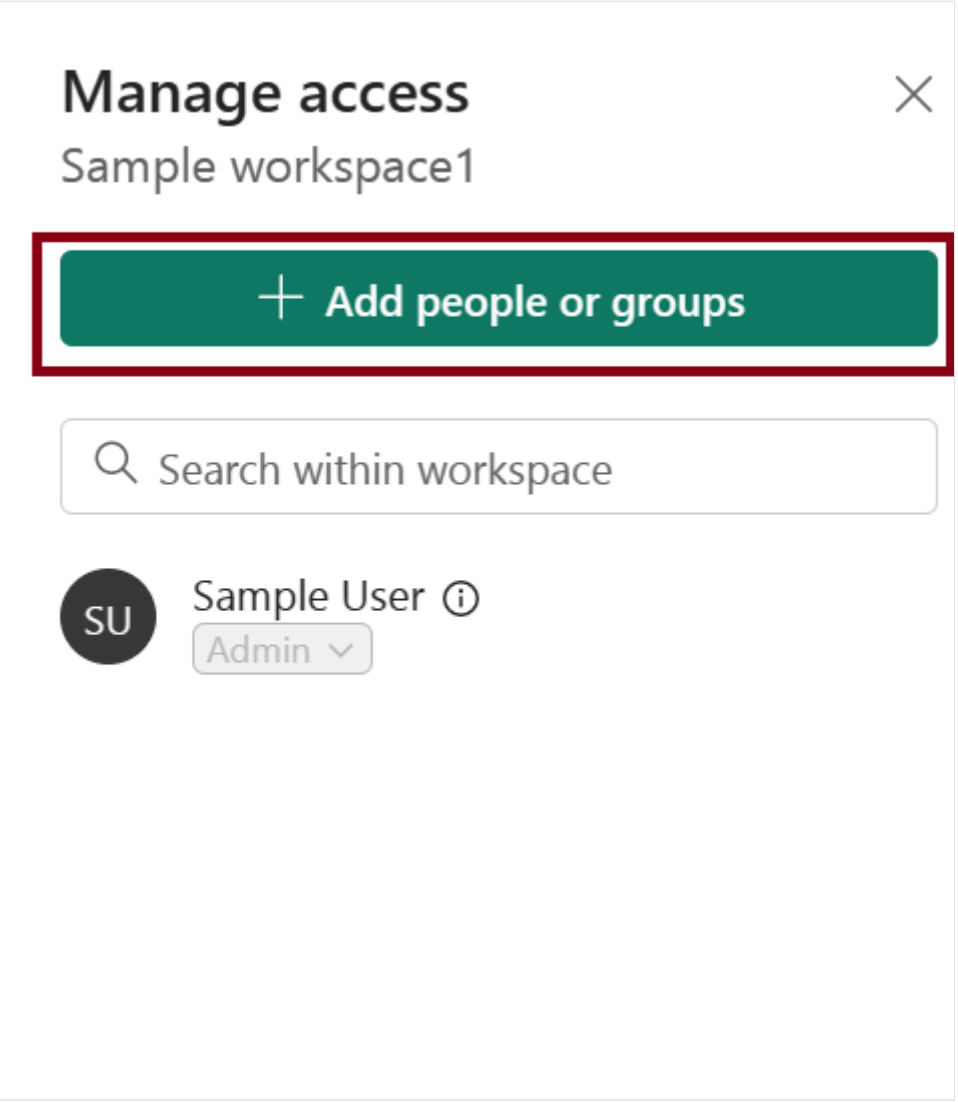
1. Because you have the Admin or Member role in the workspace, on the command bar of the workspace page, you see **Manage access**. Sometimes this entry is in the **More options** (...) menu.



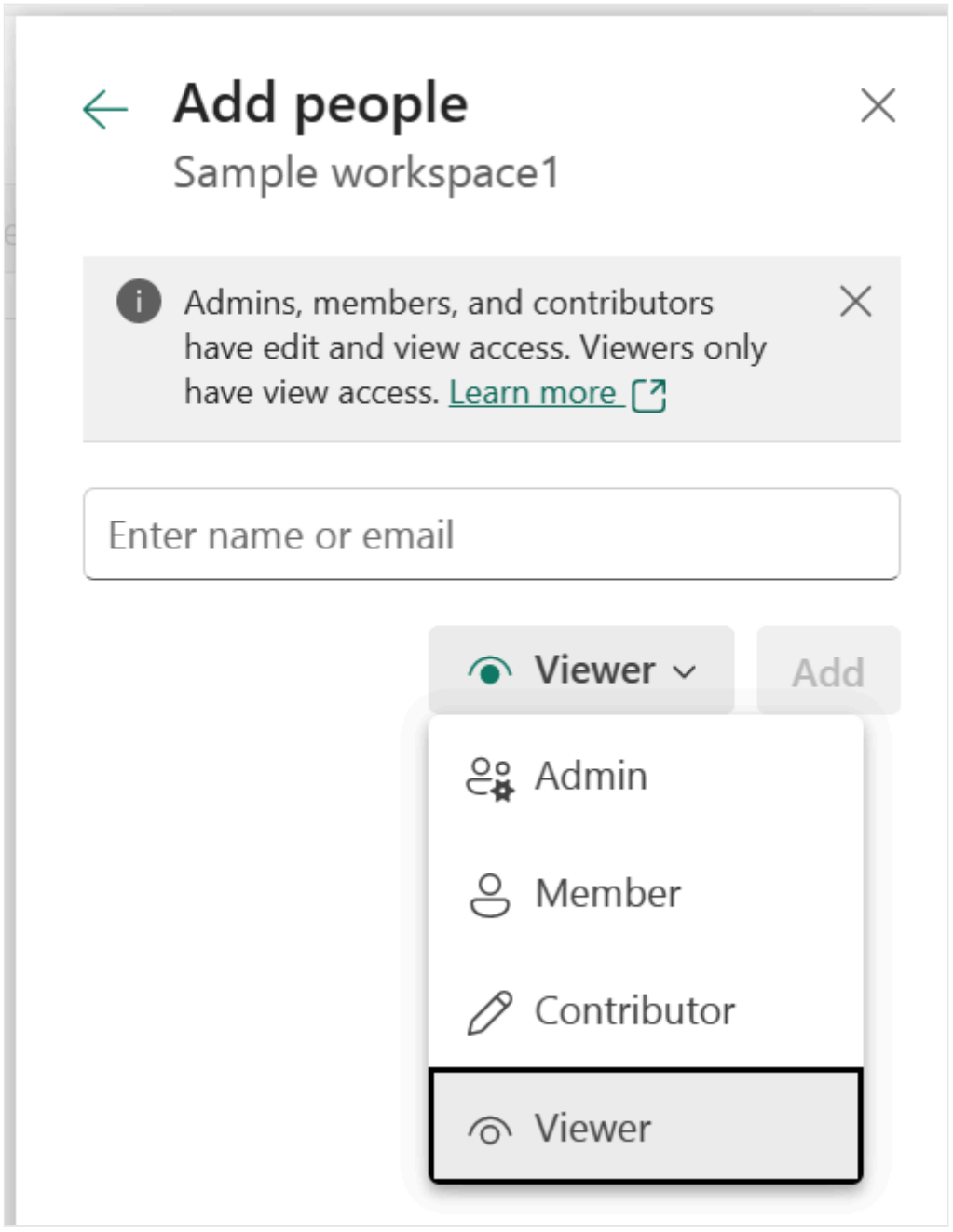
Manage access in the **More options** menu:



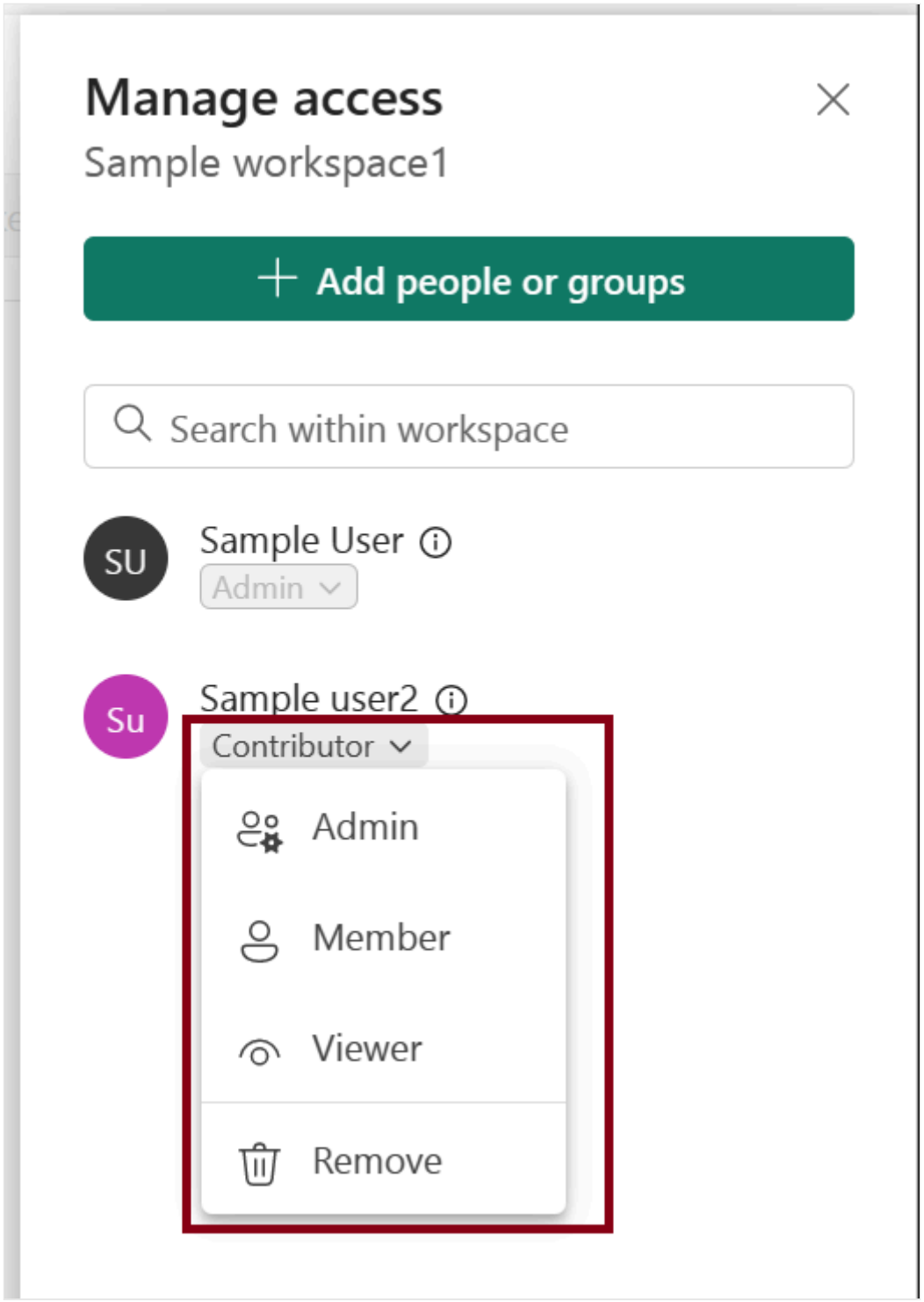
2. Select **Add people or groups**.



3. Enter name or email, select a role, and select **Add**. You can add security groups, mail-enabled security groups, distribution lists, Microsoft 365 groups, or individuals to these workspaces as admins, members, contributors, or viewers. If you have the member role, you can only add others to the member, contributor, or viewer roles.



4. You can view and modify access later if needed. Use the **Search** box to search for people or groups who already have access to this workspace. To modify access, select the dropdown arrow and select a role.



Workspace access limits

Each workspace will be limited to a maximum of 1,000 users or groups in workspace roles (Admin, Member, Contributor, Viewer). This change does not affect the number of users within a group itself. For example, if there is a workspace with one group, that group can have over 1,000 users. The limitation also covers external guests. Workspace admins or members attempting to add users or groups after this limit is reached, will get an error in the workspace access flow.

Related content

- Read about the workspace experience
- Create a workspace
- Roles in workspaces

Get started with Git integration

This article walks you through the following basic tasks in Fabric's Git integration tool:

- Connect to a Git repo
- Commit changes
- Update from Git
- Disconnect from Git

We recommend reading the overview of Git integration before you begin.

Prerequisites

To integrate Git with your Microsoft Fabric workspace, you need to set up the following prerequisites for both Fabric and Git.

Fabric prerequisites

To access the Git integration feature, you need a Fabric capacity. A Fabric capacity is required to use all supported Fabric items. If you don't have one yet, sign up for a free trial. Customers that already have a Power BI Premium capacity, can use that capacity, but keep in mind that certain Power BI SKUs only support Power BI items.

In addition, the following tenant switches must be enabled in **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**:

- Users can create Fabric items - require in case of Fabric artifacts use
- Users can synchronize workspace items with their Git repositories
- Create workspaces (only if you want to branch out to a new workspace.)
- Users can synchronize workspace items with GitHub repositories: For GitHub users only

These switches can be enabled by the tenant admin, capacity admin, or workspace admin, depending on your organization's settings.

Git prerequisites

Git integration is currently supported for Azure DevOps and GitHub. To use Git integration with your Fabric workspace, you need the following in either Azure DevOps or GitHub:

Azure DevOps

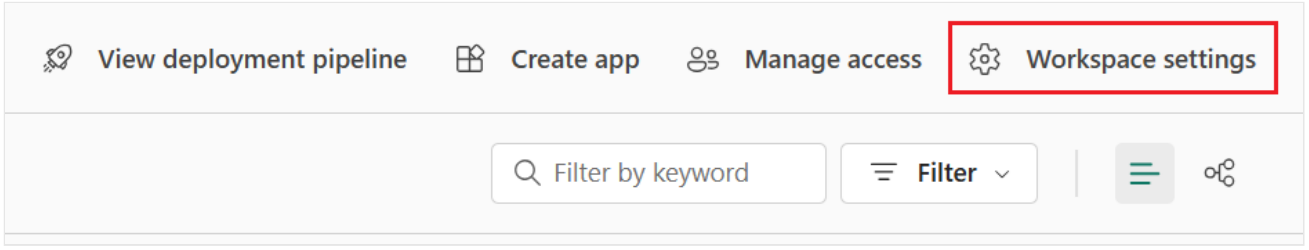
- An Active **Azure DevOps account** registered to same Fabric user (supported even if Azure DevOps organization reside in a different tenant than Fabric tenant). Create a free account.
- Access to an existing repository.

Connect a workspace to a Git repo

Connect to a Git repo

Only a workspace admin can connect a workspace to a repository, but once connected, anyone with permission can work in the workspace. If you're not an admin, ask your admin for help with connecting. To connect a workspace to an Azure or GitHub Repo, follow these steps:

1. Sign into Fabric and navigate to the workspace you want to connect with.
2. Go to **Workspace settings**



3. Select **Git integration**.
4. Select your Git provider. Currently, Azure DevOps and GitHub are supported.

Azure DevOps Connect

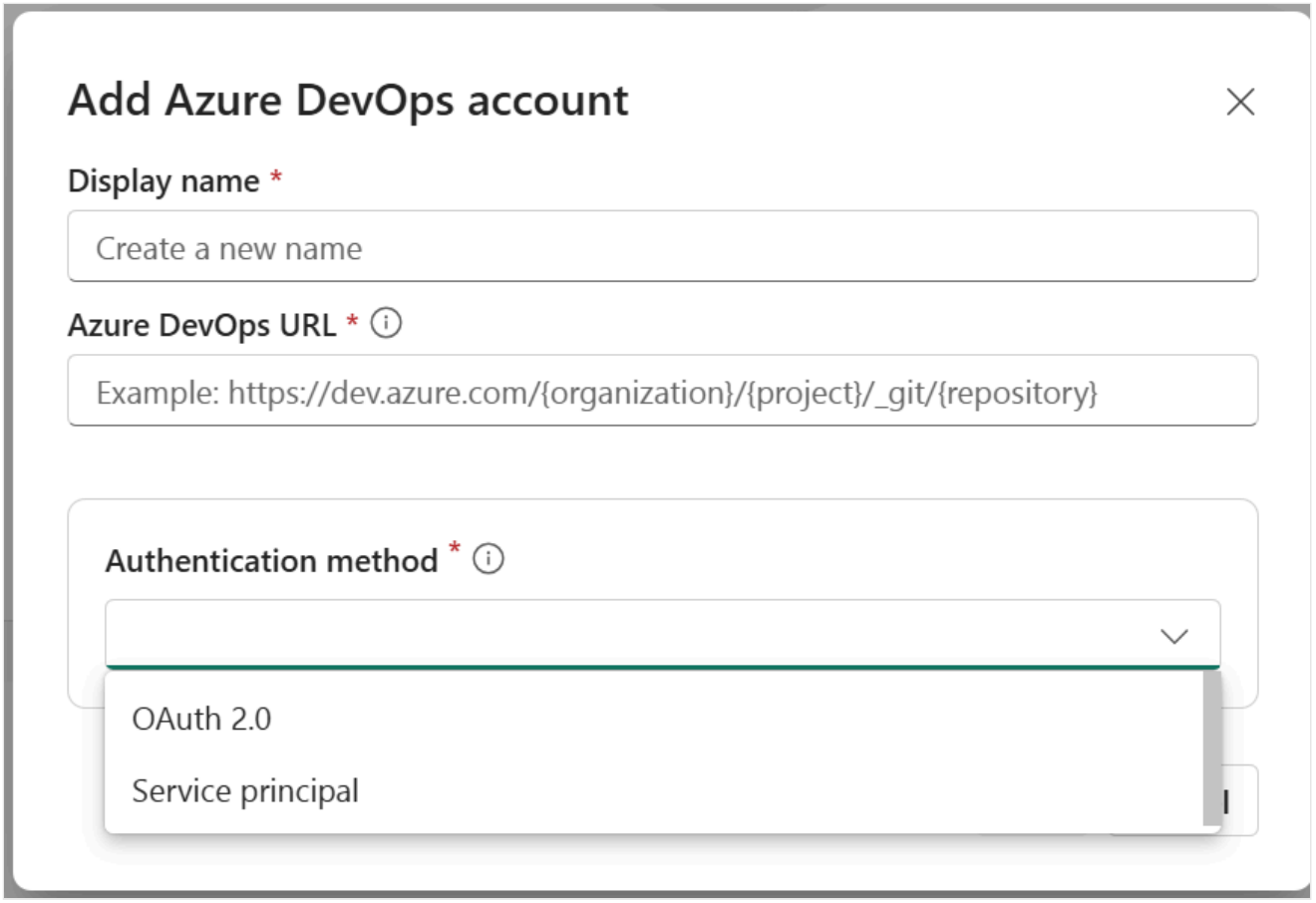
If you select Azure DevOps, select **Connect** to automatically sign into the Azure Repos account registered to the Microsoft Entra user signed into Fabric.

If you have already signed in to Azure from Fabric using a different account, select your account from the list and select **Connect**.

If it's your first time signing in from Fabric, or you want to add a new account, select **Add account**.

If it's the first time connecting, you need to Authorize your user. Provide the following information:

- *Display name* - must be unique for each user
- *Azure DevOps URL* - URL of the Azure DevOps repository. URL must be in the format `https://dev.azure.com/{organization}/{project}/_git/{repository}` or `https://{organization}.visualstudio.com/{project}/_git/{repo}`.
- *Authentication* - You can authenticate either with *OAuth2* or a *Service Principal*. For more information see Azure DevOps - Git Integration with service principal



After you sign in, select **Connect** to allow Fabric to access your account

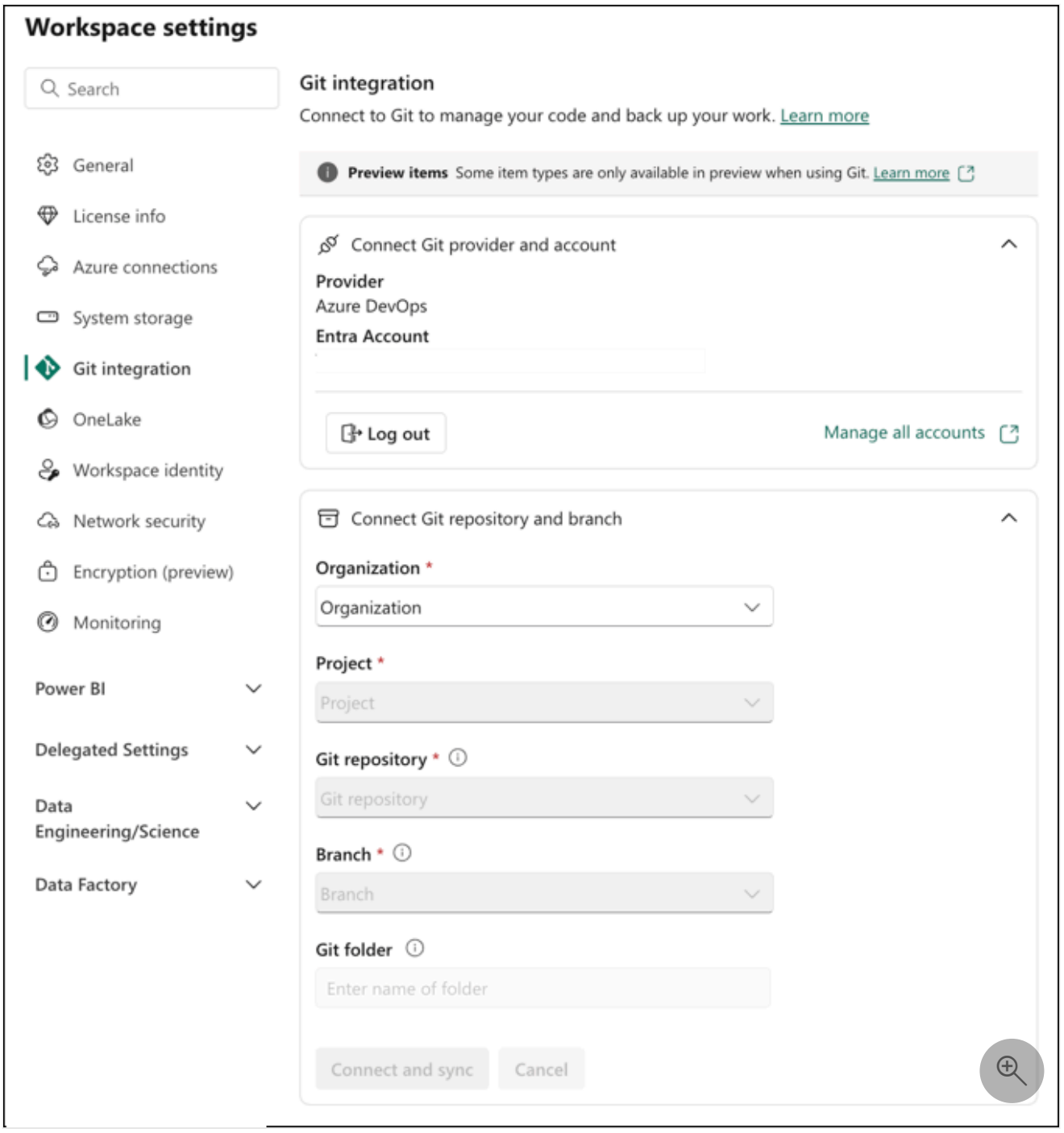
Connect to a workspace

If the workspace is already connected to Azure DevOps/GitHub, follow the instructions for Connecting to a shared workspace.

Azure DevOps branch connect

1. From the dropdown menu, specify the following details about the branch you want to connect to:

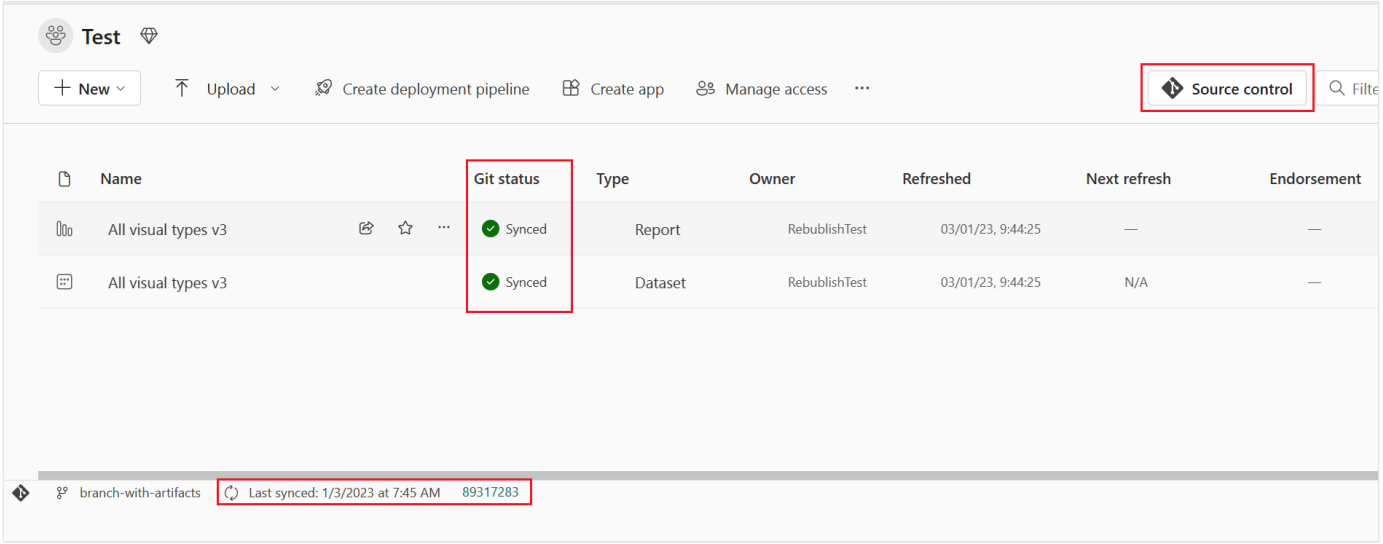
- Organization
- Project
- Git repository.
- Branch (Select an existing branch using the drop-down menu, or select **+ New Branch** to create a new branch. You can only connect to one branch at a time.)
- Folder (Type in the name of an existing folder or enter a name to create a new folder. If you leave the folder name blank, content is created in the root folder. You can only connect to one folder at a time.)



Select **Connect and sync**.

During the initial sync, if either the workspace or Git branch is empty, content is copied from the nonempty location to the empty one. If both the workspace and Git branch have content, you're asked which direction the sync should go. For more information on this initial sync, see Connect and sync.

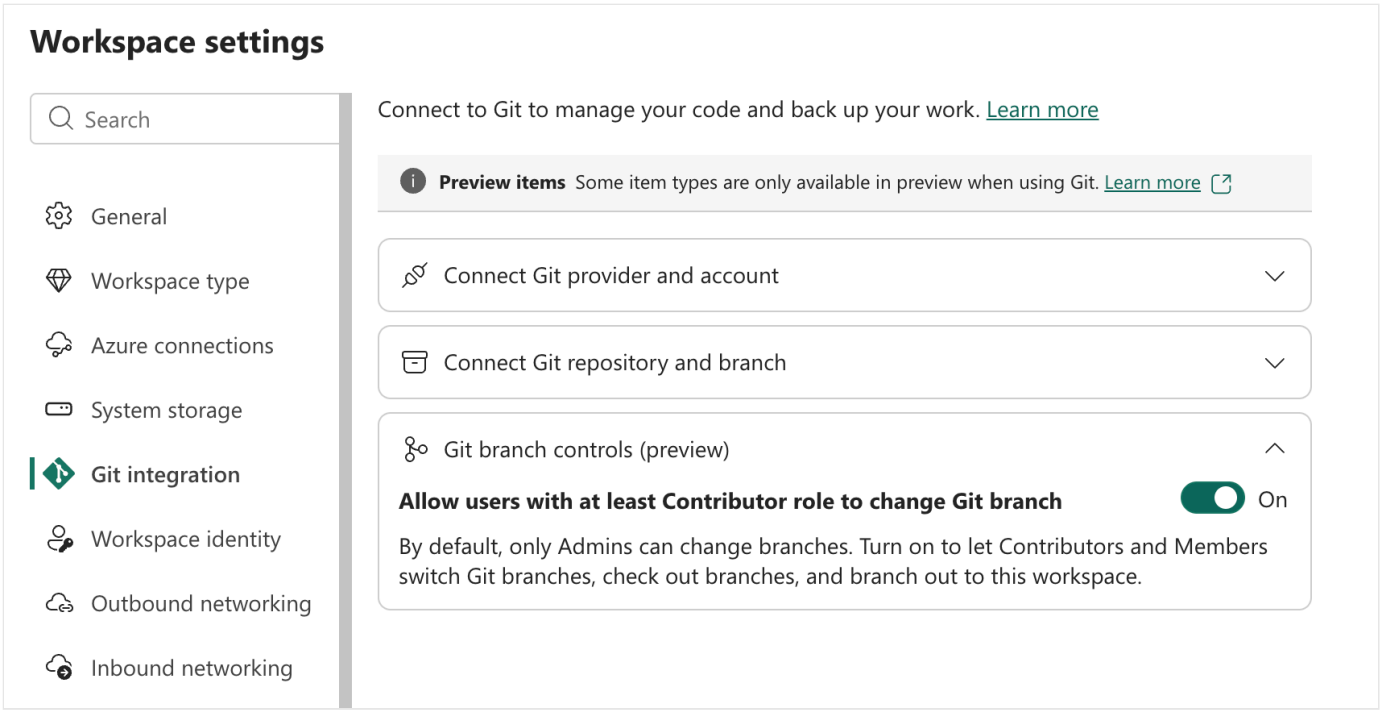
After you connect, the workspace displays information about source control that you can use to view the connected branch, the status of each item in the branch, and the time of the last sync.



To keep your workspace synced with the Git branch, commit any changes you make in the workspace to the Git branch, and update your workspace whenever anyone creates new commits to the Git branch.

Allow Contributors and Members to switch branches

By default, branch operations such as **Switch branch** and **Checkout to new branch** require the workspace Admin role. However, a workspace Admin can enable the workspace-level setting **Allow users with at least Contributor role to change Git branch**. When this setting is enabled, Contributors or Members who have write access to all artifacts in the workspace can also switch branches and create new branches under their own identity.



This setting:

- Is configured per-workspace (there's no tenant-level switch to disable it).
- Requires an active Git connection on the workspace.
- Doesn't change permissions for connecting or disconnecting a workspace—those actions remain Admin-only.

For the detailed permissions table including Git role requirements, see Permissions.

Commit changes to git

Once you successfully connect to a Git folder, edit your workspace as usual. Any changes you save are saved in the workspace only. When you're ready, you can commit your changes to the Git branch, or you can undo the changes and revert to the previous status.

Read more about commits.

Commit to Git

To commit your changes to the Git branch, follow these steps:

1. Go to the workspace.

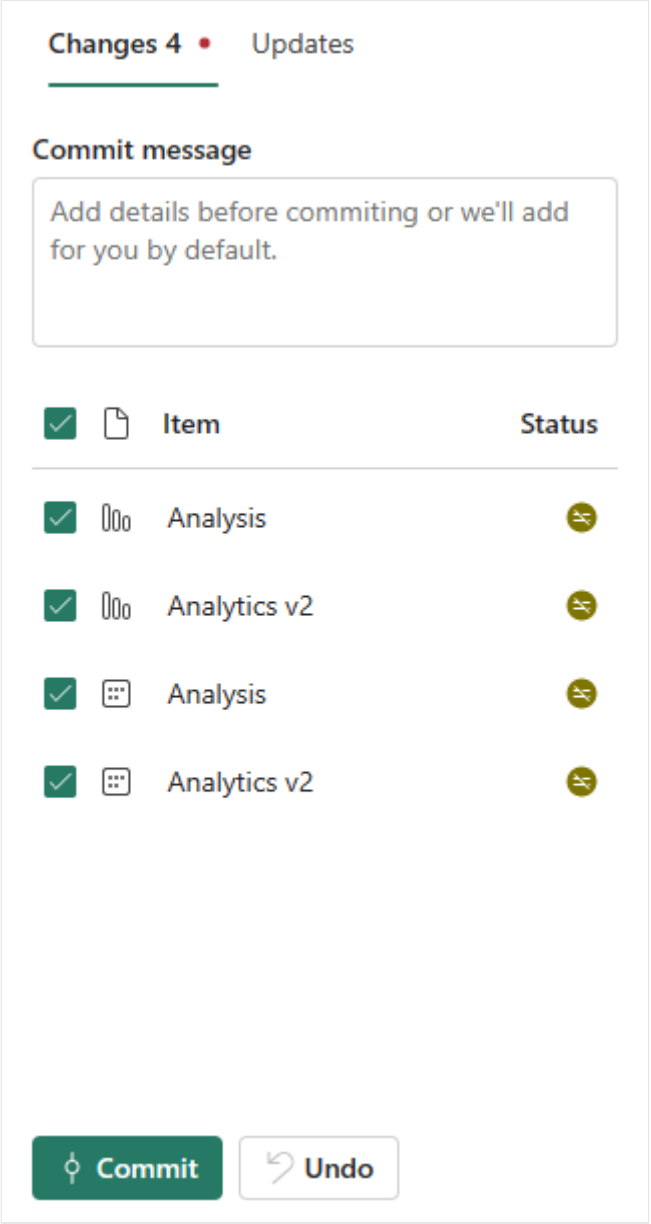
2. Select the **Source control** icon. This icon shows the number of uncommitted changes.

3. Select the **Changes** from the **Source control** panel. A list appears with all the items you changed, and an icon indicating if the item is *new* , *modified* , *conflict* , *same change* , or *deleted* .

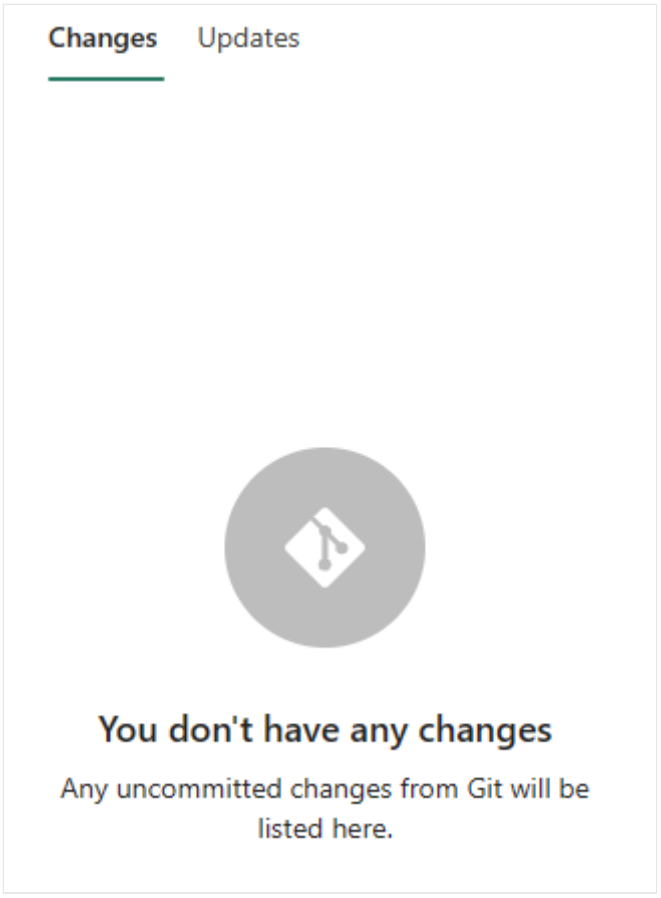
4. Select the items you want to commit. To select all items, check the top box.

5. Add a comment in the box. If you don't add a comment, a default message is added automatically.

6. Select **Commit**.



After the changes are committed, the items that were committed are removed from the list, and the workspace points to the new commit that it synced to.



After the commit is completed successfully, the status of the selected items changes from **Uncommitted** to **Synced**.

> ⓘ **Note**
>
> You can also commit changes to Git from the compare experience. For supported items, the preview file-level commit capability lets you commit specific files within an item. For more information, see Compare and commit changes.

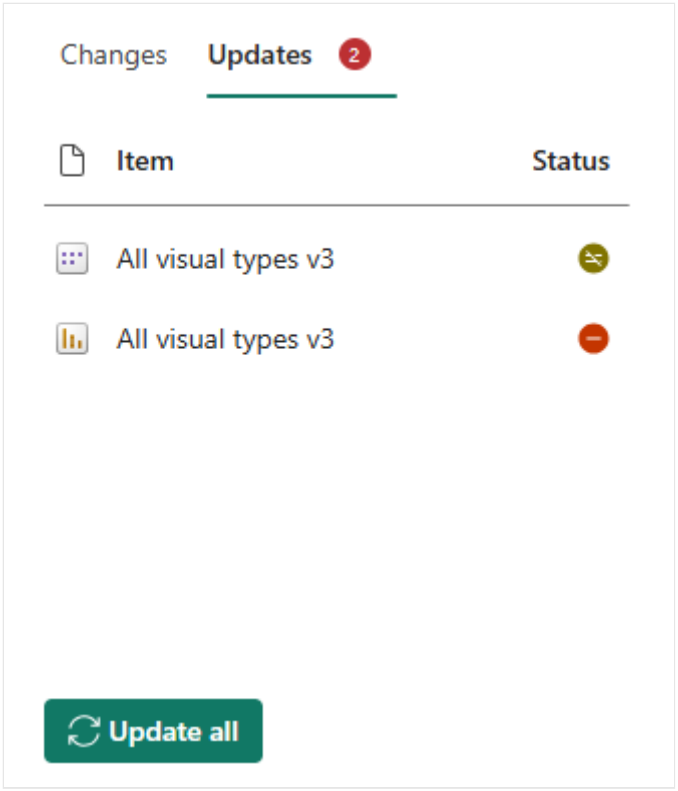
Update workspace from Git

Whenever anyone commits a new change to the connected Git branch, a notification appears in the relevant workspace. Use the **Source control** panel to pull the latest changes, merges, or reverts into the workspace and update live items. Changes to folders are also updated. Read more about updating.

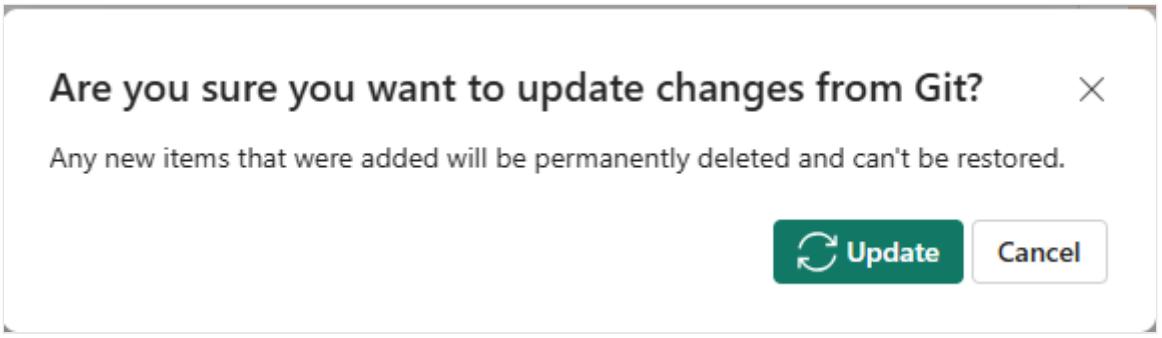
To update a workspace, follow these steps:

1. Go to the workspace.
2. Select the **Source control** icon.

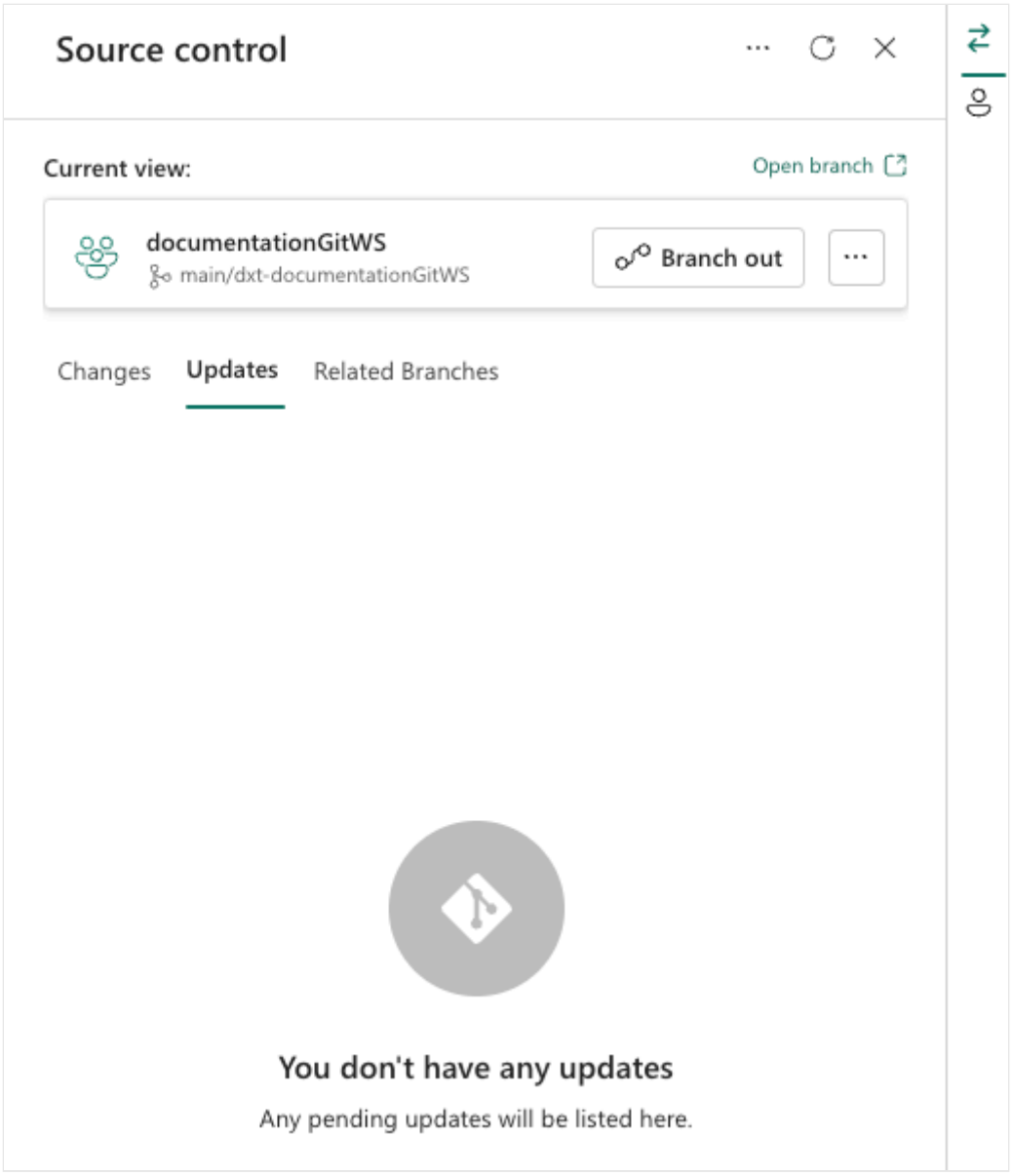
3. Select **Updates** from the Source control panel. A list appears with all the items that were changed in the branch since the last update.
4. Select **Update all**.



5. On the confirmation dialog, select **Update**.



After it updates successfully, the list of items is removed, and the workspace is fully synced with the connected Git branch.



After the update is completed successfully, the status of the items changes to **Synced**.

Disconnect a workspace from Git

Only a workspace admin can disconnect a workspace from a Git Repo. If you're not an admin, ask your admin for help with disconnecting. If you're an admin and want to disconnect your repo, follow these steps:

1. Go to **Workspace settings**
2. Select **Git integration**
3. Select **Disconnect workspace**
4. Select **Disconnect** again to confirm.

Permissions

The actions you can take on a workspace depend on the permissions you have in both the workspace and the Git repo. For a more detailed discussion of permissions, see Permissions.

Related content

- Understand the Git integration process
- Manage Git branches

Last updated on 09/01/2026

Take ownership of Fabric items

Article • 04/07/2025

When a user leaves the organization, or if they don't sign in for more than 90 days, it's possible that any Fabric items they own will stop working correctly. In such cases, anyone with read and write permissions on such an item (such as workspace admins, members, and contributors) can take ownership of the item, using the procedure described in this article.

When a user takes over ownership of an item using this procedure, they also become the owner of any child items the item might have. You can't take over ownership of child items directly - only through the parent item.

> **Note**
>
> Items such as semantic models, reports, datamarts, dataflows gen1 and dataflows gen2 have existing functionality for changing item ownership that remains the same. This article describes the procedure for taking ownership of other Fabric items.

Prerequisites

To take over ownership of a Fabric item, you must have read and write permissions on the item.

Take ownership of a Fabric item

To take ownership of a Fabric item:

1. Navigate to the item's settings. Remember, the item can't be a child item.
2. In the **About** tab, select **Take over**.
3. A message bar indicates whether the take over was successful.

   If the take over fails for any reason, select **Take over** again.

Expand table

Operation status	Error message	Next step
Success	Successfully took over the item.	None.
Partial Failure	Can't take over child items. Try again.	Retry take over of parent item.
Complete Failure	Can't take over <item_name>. Try again.	Retry take over of parent item.

> **Note**
>
> Data Pipeline items require the additional step of ensuring that the **Last Modified By** user is also updated after taking item ownership. You can do this by making a small edit to the item and saving it. For example, you could make a small change to the activity name.

> **Important**
>
> The take over feature doesn't cover ownership change of related items. For instance, if a data pipeline has notebook activity, changing ownership of the data pipeline doesn't change the ownership of the notebook. Ownership of related items needs to be changed separately.

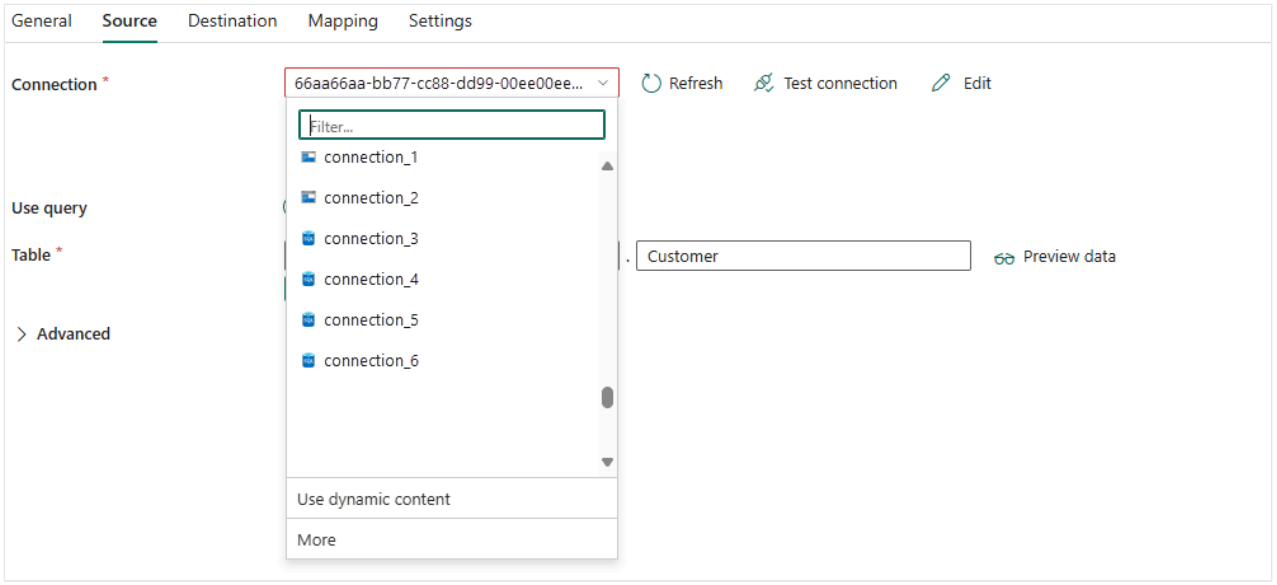
Repair connections after Fabric item ownership change

Some connections that use the previous item owner's credentials might stop working if the new item owner doesn't have access to the connection. In such cases, you might see a warning message.

In this scenario, the new item owner can fix connections by going into the item and replacing the connection with a new or existing connection. The following sections describe the steps for doing this procedure for several common item types. For other item types that have connections, refer to the item's connection management documentation.

Pipelines

1. Open the pipeline.
2. Select the activity created.
3. Replace the connection in the source and/or destination with the appropriate connection.



KQL Queryset

1. Open the KQL queryset.
2. In the **Explorer** pane, add another connection or select an existing one.

Real-Time Dashboard

1. Open the real-time dashboard in edit mode.
2. Choose **New data source** on the tool bar.
3. Select **Add+** to add new data sources.
4. In the new or existing tile, select the appropriate data source.

User data functions

1. Open the item and go to **Manage Connections**.
2. Select **Add data connection** to add a new connection and use that in the data function.

Dataflow Gen2 (CI/CD)

This is a separate item from Dataflows Gen1 and Gen2. Refer to the creation experience here.

1. Open the item and go to **Manage Connections**.
2. Navigate to the relevant connection and select **Edit Connection** to make updates.

Considerations and limitations

- The following Fabric items don't support ownership change.
  - Mirrored Cosmos DB
  - Mirrored SQL DB
  - Mirrored SQL Managed Instance
  - Mirrored Snowflake
  - Mirrored database

  If a mirrored database stops working because the item owner has left the organization or their credentials are disabled, create a new mirrored database.
- The option to take over an item isn't available if the item is a system-generated item not visible or accessible to users in a workspace. For instance, a parent item might have system-generated child items - this can happen when items such as Eventstream items and Data Activator items are created through the Real-Time hub. In such cases, the take over option is not available for the parent item.
- Currently, there's no API support for changing ownership of Fabric items. This doesn't impact existing functionality for changing ownership of items such as semantic models, reports, dataflows gen1 and gen2, and datamarts, which continues to be available. For information about taking ownership of warehouses, see Change ownership of Fabric Warehouse.
- This Fabric-item takeover feature doesn't cover ownership takeover as a service principal.

Workspace identity

A Fabric workspace identity is an automatically managed service principal that can be associated with a Fabric workspace. Fabric workspaces with a workspace identity can securely read or write to firewall-enabled Azure Data Lake Storage Gen2 accounts through trusted workspace access for OneLake shortcuts. Fabric items can use the identity when connecting to resources that support Microsoft Entra authentication. Fabric uses workspace identities to obtain Microsoft Entra tokens without the customer having to manage any credentials.

Workspace identities can be created in the workspace settings of any workspace except My workspaces.

When you create a workspace identity, Fabric creates a service principal in Microsoft Entra ID to represent the identity. An accompanying app registration is also created. Fabric automatically manages the credentials associated with workspace identities, thereby preventing credential leaks and downtime due to improper credential handling.

> **Note**
>
> Fabric workspace identity is **generally available**. You can create a workspace identity in any workspace except **My workspace**.

While Fabric workspace identities share some similarities with Azure managed identities, their lifecycle, administration, and governance are different. Fabric manages the independent lifecycle of a workspace identity. You can optionally associate a Fabric workspace with an identity. When you permanently delete the workspace, Fabric deletes the identity. The name of the workspace identity is always the same as the name of the workspace it's associated with.

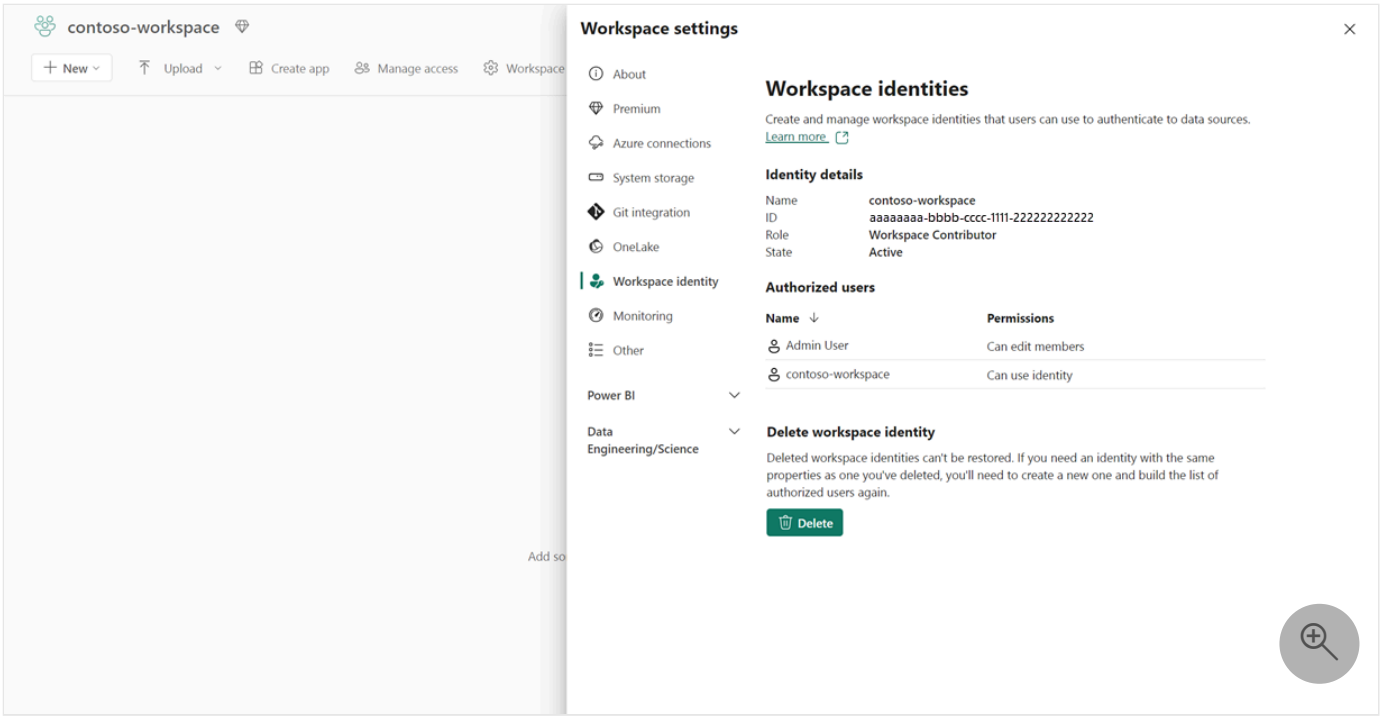
Create and manage a workspace identity

You must be a workspace admin to be able to create and manage a workspace identity. The workspace you're creating the identity for can't be a **My Workspace**.

1. Sign in to the Microsoft Fabric portal.
2. Select **Workspaces**, and then select the workspace you want to create a workspace identity for.
3. In the workspace, select the **Workspace settings** (gear) icon.

4. Select the **Workspace identity** tab.
5. Select the **+ Workspace identity** button.

When the workspace identity has been created, the tab displays the workspace identity details and the list of authorized users.



The sections of the workspace identity configuration are described in the following sections.

Identity details

Expand table

Detail	Description
Name	Workspace identity name. The workspace identity name is the same as the workspace name.
ID	The workspace identity GUID. This is a unique identifier for the identity.
Role	The workspace role assigned to the identity.
State	The state of the workspace. Possible values: *Active*, *Inactive*, *Deleting*, *Unusable*, *Failed*, *DeleteFailed*

Authorized users

For information, see Access control.

Delete a workspace identity

When an identity is deleted, Fabric items relying on the workspace identity for trusted workspace access or authentication will break. **Deleted workspace identities cannot be restored.**

> ⓘ **Note**
>
> When you delete a workspace, Fabric also deletes its workspace identity, but not immediately. The workspace stays in a *Deleted* state for the duration of the workspace retention period, and Fabric doesn't delete the workspace identity until the workspace is permanently deleted. For details, see Deleting the identity.
>
> If you restore the workspace after deletion, the workspace identity isn't restored. If you want the restored workspace to have a workspace identity, you must create a new one.

How to use workspace identity

Workspace identity currently can be used in two ways:

- For authentication: To learn how this identity surfaces as an authentication option when connecting OneLake shortcuts, pipelines, semantic models, and Dataflows Gen2 (CI/CD) to data sources, see Authenticate with workspace identity.
- For trusted workspace access: Shortcuts in a workspace that has a workspace identity can be used for trusted service access. For more information, see trusted workspace access.

Security, administration, and governance of the workspace identity

The following sections describe who can use the workspace identity, and how you can monitor it in Microsoft Purview and Azure.

Access control

Workspace identity can be created and deleted by workspace admins. By default, the workspace identity is not granted any workspace role.

> ⚠ **Warning**
>
> Workspace identity is an automatically managed service principal created by Fabric users. Access to this identity should be carefully managed and monitored, as any individual given

access to the identity is allowed to assume it.

Workspace identity is supported for authentication to target resources in connections. Only users with an admin, member, or contributor role in the workspace can configure the workspace identity for authentication in connections.

Application Administrators or users with higher roles can view, modify, and delete the service principal and app registration associated with the workspace identity in Azure.

> ⚠ **Warning**
>
> Modifying or deleting the service principal or app registration in Azure is not recommended, as it will cause Fabric items relying on workspace identity to stop working. Such changes may be reverted. Adding API permissions to allow access to target resources is supported for workspace identities. Additionally, adhere to the principle of least privilege when managing Application Administrator roles. Ensure that only appropriate users are assigned this role. For more details, refer to **Application Administrators**

Administer the workspace identity in Fabric

Fabric administrators can manage the workspace identities that they create in their tenant. Use **Fabric identities** under **Configurations** in the **Govern** section of the OneLake catalog.

1. In the **OneLake catalog**, select **Govern** > **Configurations** > **Fabric identities**.
2. Select a workspace identity, and then select **Details**.
3. In the Details tab, you can view additional information related to the workspace identity.
4. You can also delete a workspace identity.

> ⓘ **Note**
>
> Workspace identities cannot be restored after deletion. Be sure to review the consequences of deleting a workspace identity described in Delete a workspace identity.

Review workspace identity events in Microsoft Purview Audit

You can view the audit events generated when creating and deleting workspace identities in Microsoft Purview Audit. To access the log:

1. Go to Microsoft Purview Audit.
2. In the audit search form that appears, use the **Activities - friendly names** field to search for *fabric identity* to find the activities related to workspace identities. Currently, the following activities related to workspace identities are:

    - Created Fabric Identity for Workspace
    - Retrieved Fabric Identity for Workspace
    - Deleted Fabric Identity for Workspace
    - Retrieved Fabric Identity Token for Workspace

Administer the workspace identity in Azure

The application associated with the workspace identity can be viewed under both **Enterprise applications and App registrations** in the Azure portal.

Enterprise applications

The application associated with the workspace identity can be seen in **Enterprise Applications** in the Azure portal. Fabric Identity Management app is its configuration owner.

> ⚠ **Warning**
>
> Modifications to the application made here will cause the workspace identity to stop working, and such changes may be reverted. Adding API permissions to allow access to target resources is supported for workspace identities. Additionally, adhere to the principle of least privilege when managing Application Administrator roles. Ensure that only appropriate users are assigned this role. For more details, refer to Application Administrators

To view the audit logs and sign-in logs for this identity:

1. Sign in to the Azure portal.
2. Navigate to **Microsoft Entra ID** > **Enterprise Applications**.
3. Select either **Audit logs** or **Sign in logs**, as desired.

App registrations

The application associated with the workspace identity can be seen under **App registrations** in the Azure portal. No modifications should be made there, as this will cause the workspace identity to stop working.

The app registration stays listed as long as the workspace identity exists in Fabric, including while a deleted workspace is in its retention period. For more information, see Deleting the identity.

Advanced scenarios

The following sections describe scenarios involving workspace identities that might occur.

Deleting the identity

The workspace identity can be deleted in the workspace settings. When an identity is deleted, Fabric items relying on the workspace identity for trusted workspace access or authentication will break. Deleted workspace identities can't be restored.

Fabric also deletes its workspace identity but keeps the workspace in a *Deleted* state for the length of the workspace retention period so that an admin can restore it. During the retention period, Fabric doesn't deprovision the workspace identity, and its service principal and app registration remain visible in the Azure portal.

Fabric deprovisions the workspace identity when you permanently delete the workspace. This permanent deletion happens in one of the following ways:

- The retention period expires and a background job permanently deletes the workspace.
- A Fabric admin permanently deletes the workspace during the retention period. For more information, see Permanently delete a deleted collaborative workspace during the retention period.

After permanent deletion starts, the workspace identity moves to the *Deleting* state, and it might take some time before the associated service principal and app registration no longer appear in Microsoft Entra ID.

If you regularly delete workspaces and recreate them with the same name, you might see more than one app registration with that name in your tenant while the deleted workspaces are still in their retention period. To remove a workspace identity without waiting for the retention period to expire, use one of the following options:

- Before you delete the workspace, delete the workspace identity on the **Workspace identity** tab of the workspace settings.
- Ask a Fabric admin to permanently delete the workspace during the retention period.

> ⚠ **Warning**

Don't delete the service principal or app registration of a workspace identity directly in the Azure portal. Fabric can't find the application when it deprovisions the identity, which causes errors and can delay the cleanup of the workspace identity.

If you restore the workspace after deletion, the workspace identity **is not** restored. If you want the restored workspace to have a workspace identity, you must create a new one.

Renaming the workspace

When a workspace gets renamed, the workspace identity is also renamed to match the workspace name. However its Microsoft Entra application and service principal remain the same. Note that there can be multiple application and app registration objects with same name in a tenant.

Considerations and limitations

- You can create a workspace identity in any workspace except **My workspace**.
- You can create and use a workspace identity regardless of the capacity SKU assigned to the workspace.
- Using workspace identity as a credential in cloud connections is supported in any capacity.
- Trusted workspace access to firewall-enabled Storage accounts requires the workspace to be assigned to a purchased Microsoft Fabric capacity (F SKU).
- If you migrate a workspace with a workspace identity to a non-Fabric capacity, to a non-F SKU Fabric capacity, or to a trial capacity, the identity isn't disabled or deleted, but Fabric items that rely on trusted workspace access stop working.
- A default of 10,000 workspace identities can be created in a tenant. You can also specify your own maximum in the tenant settings, which becomes the upper limit for Fabric identity creation across your tenant. For more information, see Define maximum number of Fabric identities in a tenant.
- Fabric doesn't remove a workspace identity when you delete its workspace. Fabric removes it only after the workspace is permanently deleted. As a result, your tenant can contain more than one app registration with the same name if you delete workspaces and recreate them with the same name. For more information, see Deleting the identity.

- Azure Data Lake Storage Gen2 shortcuts in a workspace that has a workspace identity will be capable of trusted service access.
- Workspace identity isn't supported in B2B or cross-tenant scenarios.

Troubleshooting issues with creating a workspace identity

- If you can't create a workspace identity because the creation button is disabled, make sure you have the workspace administrator role.
- If you run into issues the first time you create a workspace identity in your tenant, try the following steps:

    1. If the workspace identity state is *failed*, wait for an hour and then delete the identity.
    2. After the identity has been deleted, wait 5 minutes and then create the identity again.

Related content

- Authenticate with workspace identity
- Trusted workspace access
- Fabric identities

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/17/2026

What is workspace monitoring (preview)?

Workspace monitoring is the primary way to enable optional observability experiences for your Fabric deployment. With workspace monitoring, you can collect and organize logs and metrics from supported Microsoft Fabric items in a workspace, create alerts through activators, use the Operations Agent to automatically investigate issues in the system, and more.

How workspace monitoring works

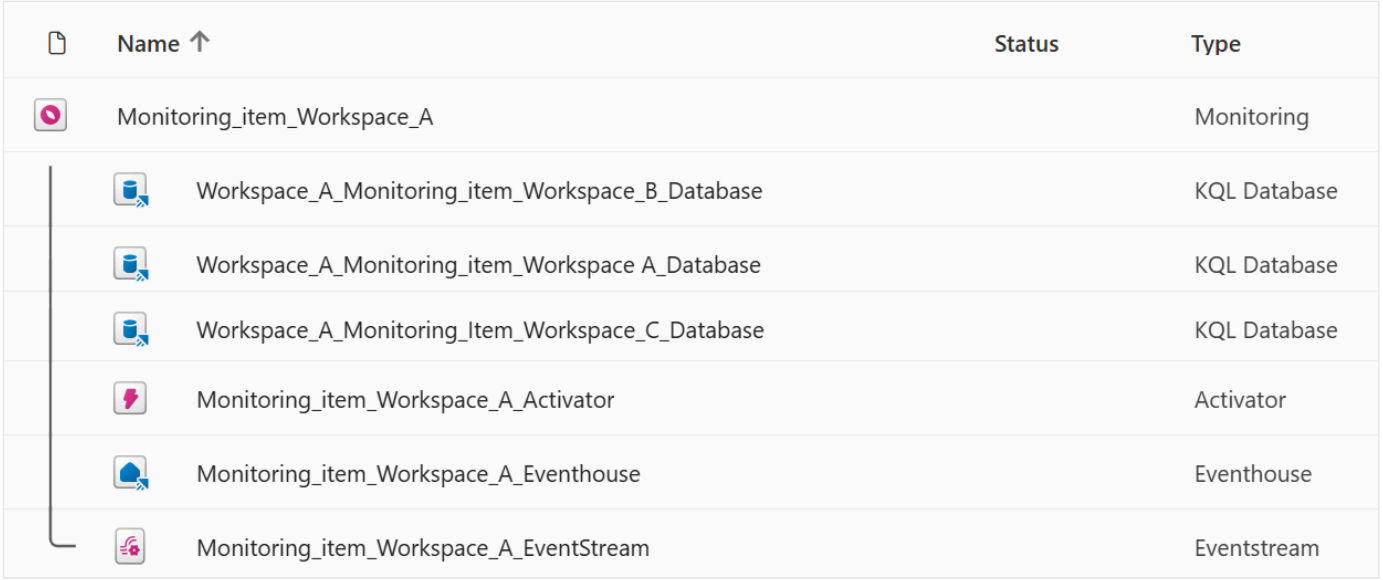
You manage workspace monitoring through a **monitoring item**. You can send data to the Eventhouse in this monitoring item, or send data to the Eventhouse in another monitoring item. Sending data to another monitoring item lets you centralize telemetry from multiple workspaces. The monitoring item also contains an activator, Operations Agent, and items related to monitoring for other experiences you opt into.

Data collection is off when you create a monitoring item. After you turn on collection, supported Fabric items send logs and metrics to tables in the monitoring database. Workspace monitoring doesn't backfill earlier activity.

Workspace users with at least the Contributor role can query the monitoring database. Use Kusto Query Language (KQL) to correlate activity across supported items, or connect the database to other Fabric experiences.

You can use the collected data in these ways:

- Query tables with KQL for interactive investigation and historical analysis.
- Save reusable queries in a KQL Queryset.
- Build a real-time dashboard or Power BI report.
- Create alerts based on query results.



Supported telemetry

After you enable workspace monitoring, you can query the following publicly documented telemetry. The tables that receive data depend on the supported items and activity in your workspace.

Expand table

Workload	Fabric artifact name	Supported events and logs
Data Warehouse	Warehouse, SQL analytics endpoint, and warehouse snapshot	• Warehouse query execution logs
Real-Time hub	Job Events	Job event logs
Real-Time hub	Event schema set	Event schema set operation logs
Data Engineering	GraphQL API	• Graph QL metrics • Graph QL operation logs
Data Factory	Copy job	Copy job activity run details logs
Data Factory	Pipeline activity logs	Pipeline Activity Run Logs
Real-Time Intelligence	Activator	Activator rule notifications
Real-Time Intelligence	Eventhouse	• Metric operation logs • Command logs • Data operation logs • Query logs • Ingestion results logs

Workload	Fabric artifact name	Supported events and logs
		• Capacity throttling logs • Sub-optimal size logs • Scale-out event logs
Real-Time Intelligence	Eventstream	• Eventstream Node Status • Eventstream Metrics • Eventstream Error Metrics
Mirroring	Mirrored database	Mirrored database execution logs
Power BI	Semantic models	Semantic model operation logs
Mirroring	Mirrored database	Mirrored database execution logs
Power BI	Semantic model	Semantic model operation logs

Sample queries

Workspace monitoring sample queries are available in the Fabric samples GitHub repository.

Visualize and alert on monitoring data

Use Real-Time Dashboard and Power BI templates to visualize workspace monitoring data. You can also start with the workspace monitoring dashboard templates and customize them for your environment.

To create proactive notifications, save a KQL query that identifies the condition you want to detect, and then create an alert from the query results. Workload-specific articles in the supported telemetry table provide example queries and alert scenarios.

Manage retention and caching

You can change the data policies for the Eventhouse under the Monitoring Item:

- The **retention policy** controls how long data remains available before it's automatically removed.
- The **caching policy** controls how long data remains in local SSD storage. Cached data provides faster query performance.

To configure these policies, open the monitoring KQL Database in the Eventhouse, and select **Manage** > **Data policies**. The caching period must be less than or equal to the retention period.

For full details, see Change data policies.

Considerations and limitations

- Only one Monitoring Item can collect telemetry for a workspace at a time.
- When you configure workspace monitoring, choose whether to send data to the Eventhouse in this Monitoring Item or to the Eventhouse in another Monitoring Item. You can't change this selection later.
- To send data to the Eventhouse in another Monitoring Item, both workspaces must be in the same Azure region.
- Monitoring Item names can contain up to 128 characters. Supporting items are named automatically. If the generated KQL Database name would exceed its 256-character limit, Fabric truncates parts of the name and adds a hash to keep the name unique.
- A workspace can contain a maximum of 1,000 Fabric and Power BI items, including parent and child items. Each workspace that sends data to a destination Eventhouse gets its own KQL Database in the destination workspace. Each database counts toward the destination workspace's item limit.
- The monitoring database is read-only.
- Monitoring data is retained for 30 days by default.
- A supported table might not appear until the corresponding item generates telemetry.
- Workspace monitoring uses Fabric capacity. For details, see Eventhouse and KQL Database consumption and Eventstream capacity consumption.
- Monitoring Eventstream ingestion, monitoring Eventhouse operations, queries on the monitoring Eventhouse, and Real-Time Dashboards that use the monitoring database continue to operate when the capacity is throttled. Power BI reports and Activator alerts that use the monitoring database respect the capacity state and can be throttled.
- Workspace Monitoring supports Private Links. If you encounter issues, disable Private Links on your workspace, then enable Workspace Monitoring, and finally re-enable Private Links.

Legacy workspace monitoring

Legacy Workspace Monitoring creates an Eventhouse database in your workspace that collects and organizes logs and metrics from the Fabric items in the workspace. Workspace contributors can query the database to learn more about the performance of their Fabric items. Use the updated version as described in the rest of the document going forward.

Legacy considerations and limitations

- The legacy Workspace Monitoring Eventhouse is a read-only item.
  - To delete the database, use the workspace settings. Before recreating a deleted database, wait about 15 minutes.
  - To share the database, grant users a workspace *member* or *admin* role.
- You can't configure ingestion to filter for specific log type or category such as *error* or *workload type*.
- User data operation logs aren't available even though the table is available in the monitoring database.
- If a supported table is missing from the monitoring Eventhouse, it might be because the Eventhouse was created before the table became available. To resolve this issue, go to the **Monitoring** tab in the workspace settings pane, turn off the **Log workspace activity** setting, and then turn it on again.
- Private links aren't supported for legacy workspace monitoring.

Related content

- Enable monitoring in your workspace
- Visualize workspace monitoring
- Manage and monitor an Eventhouse

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/29/2026

Configure workspace monitoring

This article explains how to configure a workspace Monitoring Item from workspace settings. Configuring workspace monitoring provisions the monitoring environment. Data collection begins only after you turn it on and doesn't include earlier workspace activity.

Prerequisites

To enable workspace monitoring, meet all the following conditions:

- The workspace is assigned to a Power BI Premium or a Fabric capacity.
- The Workspace admins can turn on monitoring for their workspaces tenant setting is enabled. A Fabric administrator manages this setting.
- The Users can create Fabric items tenant setting is enabled for you. If this setting is disabled at the tenant level, workspace admins can't enable workspace monitoring. The **Monitoring** option appears unavailable in **Workspace settings**. You won't see any error, but as the prerequisites aren't met, the monitoring eventhouse database isn't created, and monitoring remains inactive.
- You're an admin for the workspace.

Choose where to send monitoring data

When you configure workspace monitoring, choose one of the following destinations:

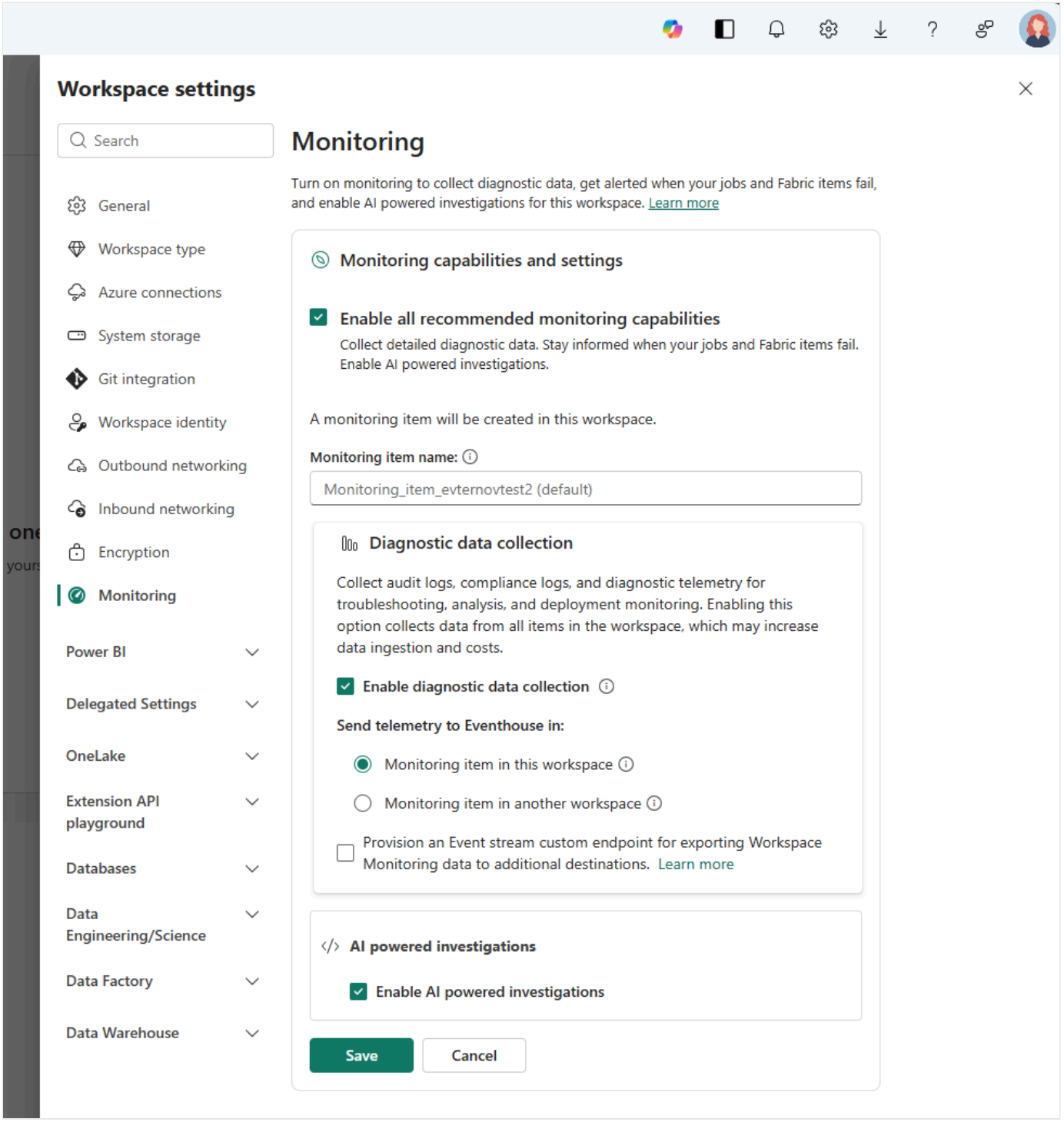
- **Send data to Eventhouse in this Monitoring Item** provisions an Eventhouse with a read-only KQL Database, an Eventstream, and an Activator in the current workspace.
- **Send data to Eventhouse in another Monitoring Item** sends workspace telemetry to the monitoring environment owned by an existing Monitoring Item. Use this option to centralize monitoring for multiple workspaces. Both workspaces must be in the same Azure region.

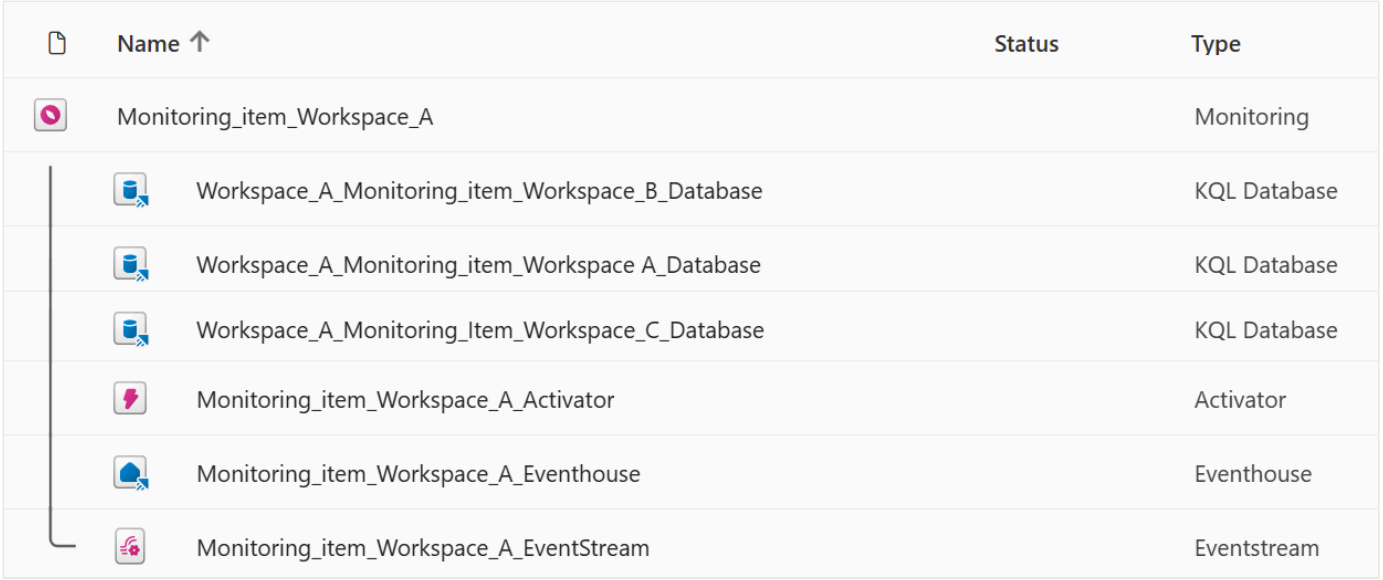
You can't change the destination after you configure workspace monitoring.

Configure monitoring and start collection

1. Open the workspace, and select **Workspace settings**.

2. Select **Monitoring**.
3. Choose whether to send data to the Eventhouse in this Monitoring Item or to the Eventhouse in another Monitoring Item.
4. If you send data to another Monitoring Item, select the destination Monitoring Item.
5. Optionally, enable a destination Custom Endpoint in the Eventstream.
6. Optionally, enable agentic investigation capabilities to add an Operations Agent.
7. Complete the configuration to create the Monitoring Item.
8. Turn on data collection.





Supported Fabric items begin sending telemetry after you turn on collection. The time before the first records appear depends on item activity and ingestion latency.

Query monitoring data

Open the Monitoring Item, open its monitoring KQL Database, select a table, and use **Query data** to create a KQL query. For example, the following query returns the most recent item job events:

Kusto

```
ItemJobEventLogs
| top 100 by Timestamp desc
```

Only tables for supported telemetry are populated. For table descriptions and workload-specific examples, see Supported telemetry.

Manage retention and caching

The monitoring KQL database supports Eventhouse retention and caching policies. Monitoring data is retained for 30 days by default. To change how long data is retained or cached, open the monitoring KQL database and select **Manage** > **Data policies**. The caching period must be less than or equal to the retention period.

- The **retention policy** controls how long data remains available before it's automatically removed.
- The **caching policy** controls how long data remains in local SSD storage. Cached data provides faster query performance.

For policy behavior, requirements, and configuration steps, see Change data policies.

Share the Monitoring Item

You can share the Monitoring Item with people who don't have a role in the workspace. In the list of workspace items or in the open Monitoring Item, select **Share** on the appropriate KQL Database, choose the audience and permissions, and then send or copy the sharing link. For more information, see Share items in Microsoft Fabric.

Send monitoring data to a custom endpoint

You can configure the Monitoring Item's Eventstream with a custom endpoint destination. The destination lets an external application consume monitoring events by using the Event Hubs, Kafka, or AMQP protocol.

Before configuring the endpoint, you must enable it from the Workspace Settings. Through the preview period, this setting can only be enabled at creation time.

If enabled, use the normal Eventstream configuration process. Open the Eventstream from the Monitoring Item, select **Edit**, add a **Custom endpoint** destination, connect it to the stream, and publish the eventstream. Then, use the destination's **Details** pane to get authentication and connection information for your application. For prerequisites and detailed steps, see Add a custom endpoint or custom app destination to an eventstream.

Stop or resume collection

Use the data collection control in the Monitoring Item to stop or resume collection. Stopping collection prevents new telemetry from arriving but preserves the Monitoring Item, supporting items, and previously collected data.

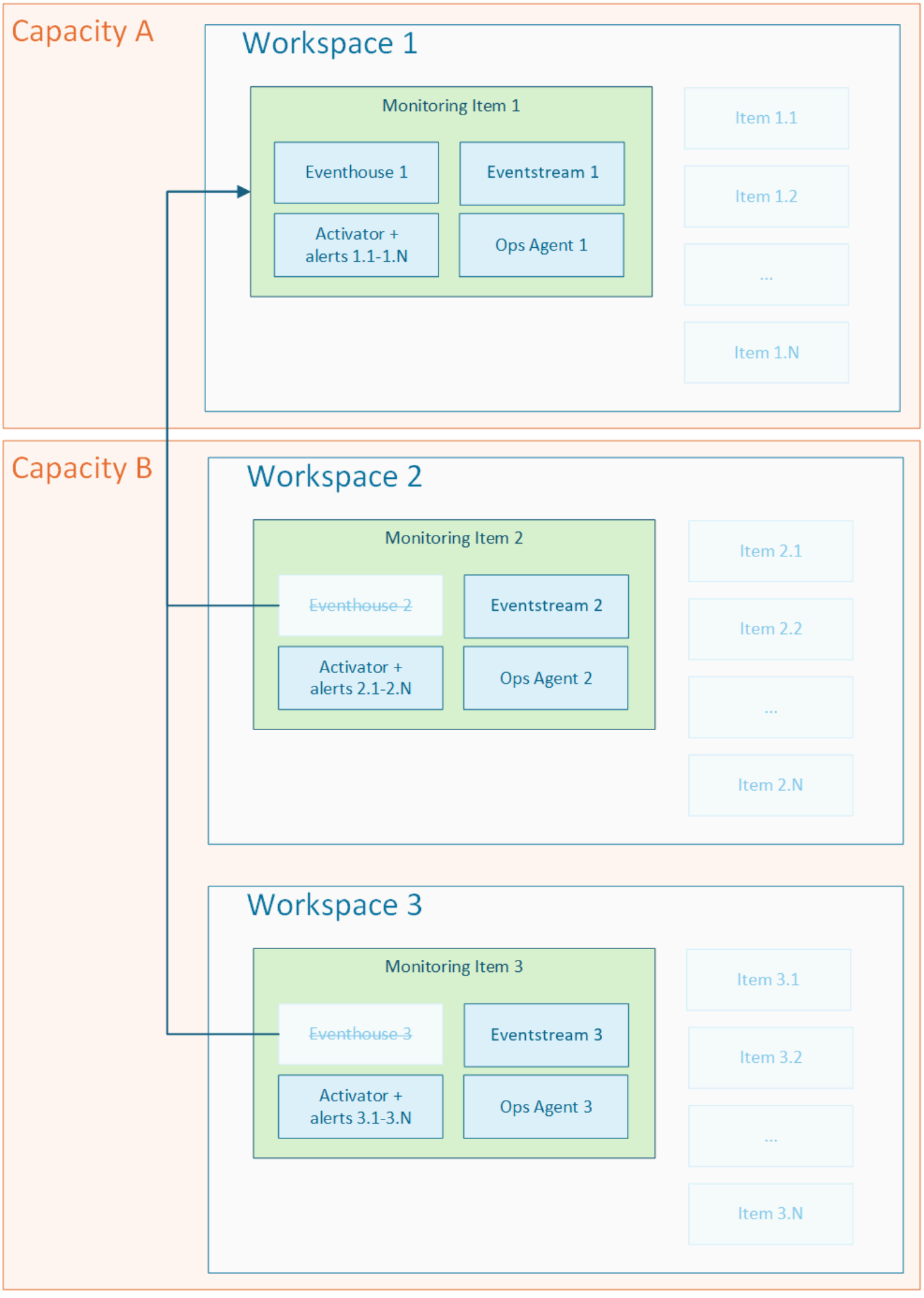
Deleting a Monitoring Item that sends data to another Monitoring Item doesn't delete the destination monitoring environment or the data already sent to it.

> **Important**
>
> If you enable the Operations Agent, you can't disable it later.

Recommended architecture

Consolidate to as few Eventhouses as possible. In particular, isolate a workspace on its own Capacity to host a central monitoring Eventhouse. While this architecture isn't always possible - see the limitations section for valid reasons such as monitoring across more than 1,000 workspaces or across different regions - centralize to as few as possible. This approach enables simpler analysis across workspaces and better cost management through resource reuse. Isolating monitoring onto its own capacity shields monitoring from capacity-related conditions such as throttling. It also shields production workloads from excessive capacity usage by monitoring workloads.

A sample implementation of the preceding architecture is as follows. Capacity A contains the central monitoring Eventhouse. The Monitoring Items in workspaces 2 and 3, both hosted on a different Capacity (Capacity B), send their telemetry to this central Eventhouse.


Capacity A
Workspace 1
Monitoring Item 1
Eventhouse 1
Eventstream 1
Activator + alerts 1.1-1.N
Ops Agent 1
Item 1.1
Item 1.2
...
Item 1.N
Capacity B
Workspace 2
Monitoring Item 2
Eventhouse 2
Eventstream 2
Activator + alerts 2.1-2.N
Ops Agent 2
Item 2.1
Item 2.2
...
Item 2.N
Workspace 3
Monitoring Item 3
Eventhouse 3
Eventstream 3
Activator + alerts 3.1-3.N
Ops Agent 3
Item 3.1
Item 3.2
...
Item 3.N

Item naming

When you turn on Workspace Monitoring, Fabric creates a Monitoring item and several other entities under it. One of these entities is a KQL database to store your monitoring data. Fabric automatically names this KQL database by using the pattern: `workspace name + monitoring item name + _Database`. For example, a workspace called Sales with a Monitoring item called Telemetry gets a database called Sales_Telemetry_Database.

Why the name is sometimes adjusted

Workspace names allow more characters than KQL database names do. If your workspace or item name contains a character that a KQL database doesn't accept, Fabric replaces it with an underscore.

- Fabric keeps exactly as they are: letters in any language (including accented and non-Latin characters), numbers, spaces, hyphens, periods, and underscores.
- Fabric replaces with an underscore: brackets and parentheses, quotes, and other symbols such as `& % $ # @ * ? / \ | < > : ;`.

Limitations

- Only one Monitoring Item can collect telemetry for a workspace at a time.
- You can't change the Eventhouse destination after configuration.
- To send data to the Eventhouse in another Monitoring Item, both workspaces must be in the same Azure region.
- Monitoring Item names can contain up to 128 characters. If an automatically generated KQL Database name would exceed 256 characters, Fabric truncates parts of the name and adds a hash to keep the name unique.
- A workspace can contain a maximum of 1,000 Fabric and Power BI items, including child items. Each workspace that sends data to a destination Eventhouse gets its own KQL Database in the destination workspace, and each database counts toward that workspace's item limit.

Troubleshooting

- If **Monitoring** isn't available in **Workspace settings**, verify the tenant setting, workspace role, and capacity prerequisites.
- If provisioning doesn't finish, refresh the page and reopen the Monitoring Item to check its status.
- If a supported table is empty, generate the corresponding item activity and allow time for ingestion.

- If you send data to the Eventhouse in another Monitoring Item, verify that the destination Monitoring Item still exists and both workspaces are in the same Azure region.

Legacy workspace monitoring

Migrate from legacy workspace monitoring

Migration isn't automatic. To move a workspace from legacy Workspace Monitoring to the updated version:

1. Open the workspace, and select **Workspace settings** > **Monitoring**.
2. Turn off **Log workspace activity** to pause legacy data collection. You can't migrate while legacy data collection is running.
3. Select the option in the migration banner to move to the updated monitoring experience.
4. Configure the Monitoring Item and turn on data collection.

Previously collected data remains in the legacy Eventhouse and isn't transferred to the updated monitoring database. After you turn on updated data collection, telemetry can take up to one hour to begin flowing to the new database. This delay occurs while cached settings for the legacy destination expire; no action is required.

Enable legacy workspace monitoring

You can only enable the legacy Workspace Monitoring feature if you didn't already enroll in the updated version as described in the rest of the document. If you need to revert from the updated version to the legacy one, you must delete the Monitoring Item from the workspace. The option to onboard through the legacy path becomes available.

Follow these steps to enable legacy Workspace Monitoring in your workspace:

1. Go to the workspace you want to enable monitoring for, and select **Workspace settings** (⚙).

2. In **Workspace settings**, select **Monitoring**.

3. Select **+Eventhouse** and wait for Fabric to create the database.

Related content

- Workspace monitoring overview
- Monitor Fabric items with item job event logs
- Visualize workspace monitoring

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

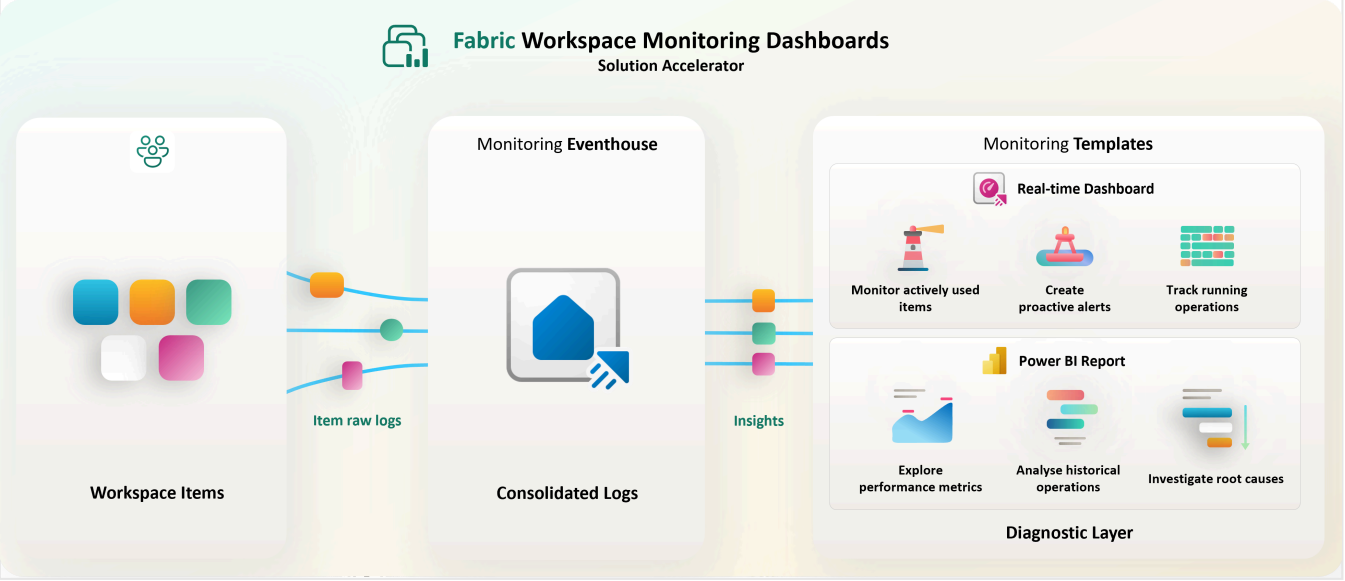
Last updated on 09/29/2026

Visualize your workspace monitoring in a Real-Time dashboard or Power BI report

Fabric workspace monitoring provides rich telemetry across your workspace assets, including Eventhouses, Power BI Semantic Models, Data Engineering (GraphQL), and Mirrored Databases. The workspace monitoring data is stored in an Eventhouse, which is part of Fabric Real-Time Intelligence. This telemetry includes diagnostic logs, ingestion results, query activity, and system metrics. The system collects and organizes all this data in a secure, read-only Eventhouse database.

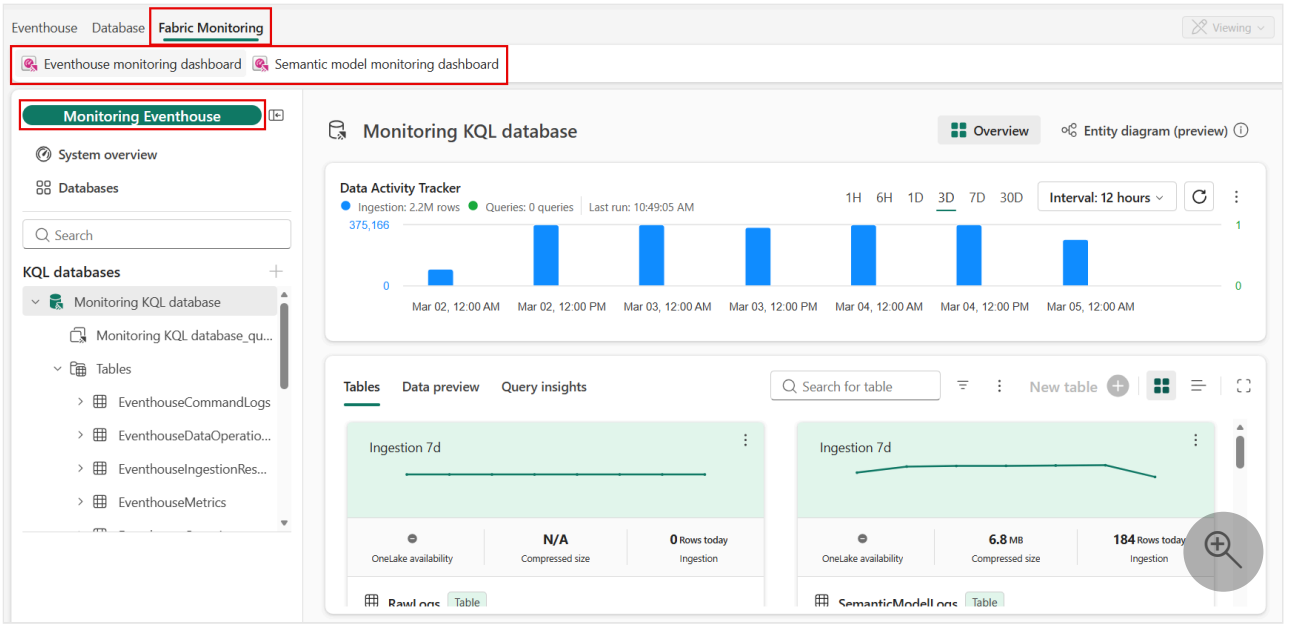
To help you turn this data into actionable insights, this article guides you through two approaches to connect your monitoring data to a Real-Time dashboard or a Power BI report:

- **Built-In Templates Solution** – Use prebuilt templates with visualizations to quickly create dashboards for monitoring Eventhouses and Semantic Models.
- **Step-by-Step Solution** – Set up a custom monitoring solution using the end-to-end sample, giving you more flexibility to customize your dashboards and reports.



Built-In Templates Solution

The Real-Time dashboard Templates with built-in visualizations enable you to easily transform your data into actionable insights. Currently, two templates are available: one for Eventhouse items and another for Semantic Model items.



Prerequisites:

- A Power BI Premium capacity or a workspace with Microsoft Fabric-enabled capacity.
- Workspace monitoring must be enabled in your workspace.

Getting started:

You can create these dashboards directly from any Workspace Monitoring Eventhouse. To get started, go to your Workspace Monitoring Eventhouse, open the upper ribbon, and select Fabric Monitoring.

Create a template-based dashboard:

Choose the template that best suits your needs:

- Eventhouse Monitoring Dashboard – use this dashboard to monitor Eventhouse items in your workspace. You can track:
  - Ingestion results and logs.
  - Commands and queries monitoring.
  - Metrics related to Eventhouse performance.

- Semantic Model Monitoring Dashboard – use this dashboard to monitor Semantic Models in your workspace. You can use semantic model logs to:

  - Identify periods of high or unusual Analysis Services engine activity by capacity, workspace, report, or user.

  - Analyze query performance and trends, including external DirectQuery operations.

  - Track semantic model refresh durations, overlaps, and processing steps.

  - Monitor custom operations sent using the Premium XMLA endpoint.

Once you create the dashboards, they're ready to use immediately, providing instant visibility into your workspace. You can also edit them to meet your specific needs by modifying the queries to add custom filters or joining the data with other monitoring data.

Related content

- Sample Gallery
- End-to-End Sample
- Workspace Monitoring Overview

Last updated on 03/18/2026

Monitor Fabric items with item job event logs

Item job event logs are part of workspace monitoring logs and are stored in the Eventhouse KQL database. Use these logs to monitor job results for Fabric items, such as:

- Analyze job performance and trends.
- identify failed or long-running jobs by capacity, workspace, item type, or user.

Supported item and job types

The following table lists the supported Fabric item types and their corresponding job types that generate event logs. Use it as a reference to understand which operations are tracked.

Expand table

Item type	Job type
Data Pipeline	Data Pipeline
Notebook	RunNotebook, RunNotebookInteractive, PipelineRunNotebook, SparkSecurityControl
Lakehouse	TableMaintenance, TableLoad, LakehouseOperation, LivyBatch, LivySession, MaterializedLakeViews, SparkSecurityControl
Warehouse	SqlAnalyticsEndpoint
CopyJob	CopyJob
DataflowFabric	Refresh, Publish
DBTItem	DBTItem
DigitalOperationsOperationalInsight	UpdateLibrariesJob
MLExperiment	MLExperimentRun
GraphIndex	Algorithm, Load, Refresh
DigitalTwinBuilder	UpdateLibrariesJob
DigitalTwinBuilderFlow	ExecuteOperations
Databricks	SyncJob

Item type	Job type
DigitalOperationsAction	ExecuteOperations
SustainabilityDataManager	IngestAzureEmissions, IngestM365Emissions

Item job event logs schema

The following table defines the schema of the event log data, including column names, data types, and descriptions. It helps you interpret log entries and build queries.

Expand table

Column name	Type	Description
Timestamp	datetime	UTC timestamp when the log entry was generated.
ItemId	string	Unique ID of the item logging the data.
ItemKind	string	Type of item logging the operation.
ItemName	string	Name of the Fabric item logging this operation.
WorkspaceId	string	Unique ID of the Fabric workspace containing the item.
WorkspaceName	string	Name of the workspace containing the item.
CapacityId	string	Unique ID of the capacity hosting the item.
DurationMs	long	Time taken by the job in milliseconds.
ExecutingPrincipalId	string	User ID or service principal ID executing the job.
ExecutingPrincipalType	string	Indicates user or service principal.
WorkspaceMonitoringTableName	string	Name of the table where records belong.
JobInstanceId	string	Unique ID of the job instance.
JobInvokeType	string	Job trigger type: on-demand or scheduled.
JobType	string	Job type (see table above).
JobStatus	string	Status: Not started, In progress, Completed, Failed.
JobDefinitionObjectId	string	ID of the scheduler that triggered the job.
JobScheduleTime	datetime	Scheduled job start time.
JobStartTime	datetime	Actual job start time.

Column name	Type	Description
JobEndTime	datetime	Actual job end time.

Considerations

If the *ItemJobEventLogs* table is missing from the Eventhouse, it may be because the Eventhouse was created before this feature became available. Use the following steps to fix this issue:

1. Go to the **Monitoring** tab in the workspace settings pane.
2. Turn off the **Log workspace activity** setting, then turn it on again.

Related content

- Enable workspace monitoring
- What is workspace monitoring?

Last updated on 04/16/2026

Endorse Fabric and Power BI items

Article • 01/26/2025

Fabric provides three ways to endorse valuable, high-quality items to increase their visibility: **promotion** and **certification** and designating them as **master data**.

- **Promotion**: Promotion is a way to highlight items you think are valuable and worthwhile for others to use. It encourages the collaborative use and spread of content within an organization.

  Any item owner, as well as anyone with write permissions on the item, can promote the item when they think it's good enough for sharing.

- **Certification**: Certification means that the item meets the organization's quality standards and can be regarded as reliable, authoritative, and ready for use across the organization.

  Only authorized reviewers (defined by the Fabric administrator) can certify items. Item owners who wish to see their item certified and aren't authorized to certify it themselves need to follow their organization's guidelines about getting items certified.

- **Master data**: Being labeled as master data means that the data item is regarded by the organization as being core, single-source-of-truth data, such as customer lists or product codes.

  Only authorized reviewers (defined by the Fabric administrator) can label data items as master data. Item owners who wish to see their item endorsed as master data and aren't authorized to apply the **Master data** badge themselves need to follow their organization's guidelines about getting items labeled as master data.

Currently it's possible to promote or certify all Fabric and Power BI items except Power BI dashboards.

Master data badges can only be applied to items that contain data, such as lakehouses and semantic models.

This article describes:

- How to promote items.
- How to certify items if you're an authorized reviewer, or request certification if you're not.

- How to apply the **Master data** badge to a data item if you are authorized to do so, or request master data designation if you're not.

See the endorsement overview to learn more about endorsement.

Promote items

To promote an item, you must have write permissions on the item you want to promote.

1. Go to the settings of the item you want to promote.
2. Expand the endorsement section and select **Promoted**.

   If you're promoting a Power BI semantic model and see a **Make discoverable** checkbox, it means you can make it possible for users who don't have access to the semantic model to find it. See semantic model discovery for more detail.
3. Select **Apply**.

Certify items

Item certification is a significant responsibility, and you should only certify an item if you feel qualified to do so and have reviewed the item.

To certify an item:

- You must be authorized by the Fabric administrator.

  > **Note**
  >
  > If you aren't authorized to certify an item yourself, you can request item certification.

- You must have write permissions on the item you want to apply the **Certified** badge to.

1. Carefully review the item and determine whether it meets your organization's certification standards.
2. If you decide to certify the item, go to the workspace where it resides, and open the settings of the item you want to certify.
3. Expand the endorsement section and select **Certified**.

If you're certifying a Power BI semantic model and see a **Make discoverable** checkbox, it means you can make it possible for users who don't have access to the semantic model to find it. See semantic model discovery for more detail.

4. Select **Apply**.

Label data items as master data

Labeling data items as master data is a significant responsibility, and you should perform this task only if you feel you are qualified to do so.

To label a data item as master data:

- You must be authorized by the Fabric administrator.

  > **Note**
  >
  > If you aren't authorized to designate a data item as master data yourself, you can request the master data designation.

- You must have write permissions on the item you want to apply the **Master data** badge to.

1. Carefully review the data item and determine whether it is truly core, single-source-of-truth data that your organization wants users to find and use for the kind of data it contains.

2. If you decide to label the item as master data, go to the workspace where it located, and open the settings of the item's settings..

3. Expand the endorsement section and select **Master data**.

4. Select **Apply**.

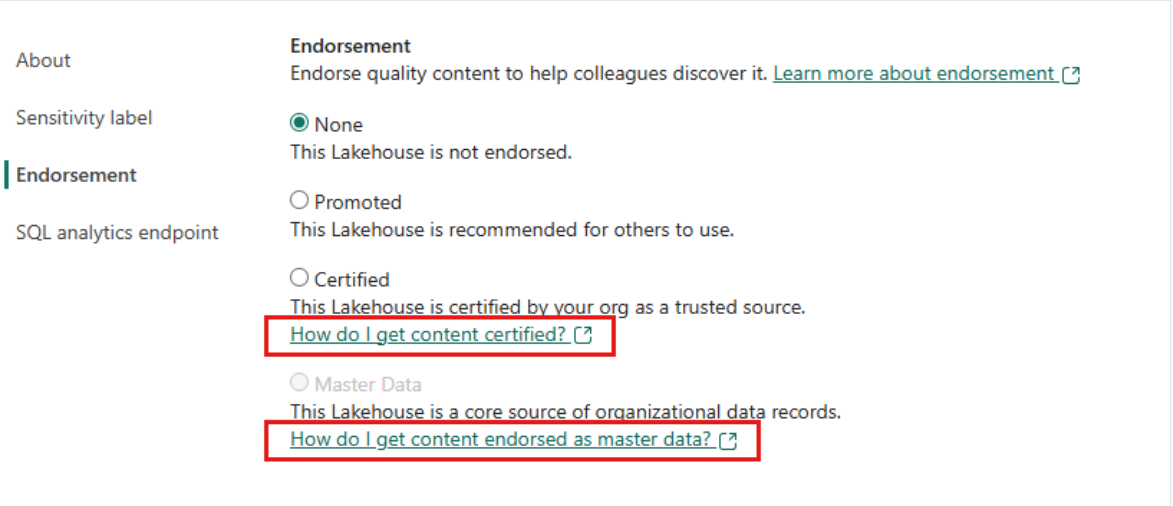
Request certification or master data designation

If you would like to certify your item or get it labeled as master data but aren't authorized to do so, follow the steps below.

1. Go to the workspace where the item you want endorsed as certified or master data is located, and then open the settings of that item.

2. Expand the endorsement section. The **Certified** or **Master data** button will be greyed if you're not authorized to endorse items as certified or as master data.

3. Select relevant link, **How do I get content certified** or **How do I get content endorsed as Master data** to find out how to get your item endorsed the way you want it to be:



> **Note**
>
> If you clicked one of the links but got redirected back to this note, it means that your Fabric admin has not made any information available. In this case, contact the Fabric admin directly.

Related content

- Read more about endorsement
- Enable item certification (Fabric admins)
- Enable master data endorsement (Fabric admins)
- Read more about semantic model discoverability

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Share items in Microsoft Fabric

Article • 04/06/2025

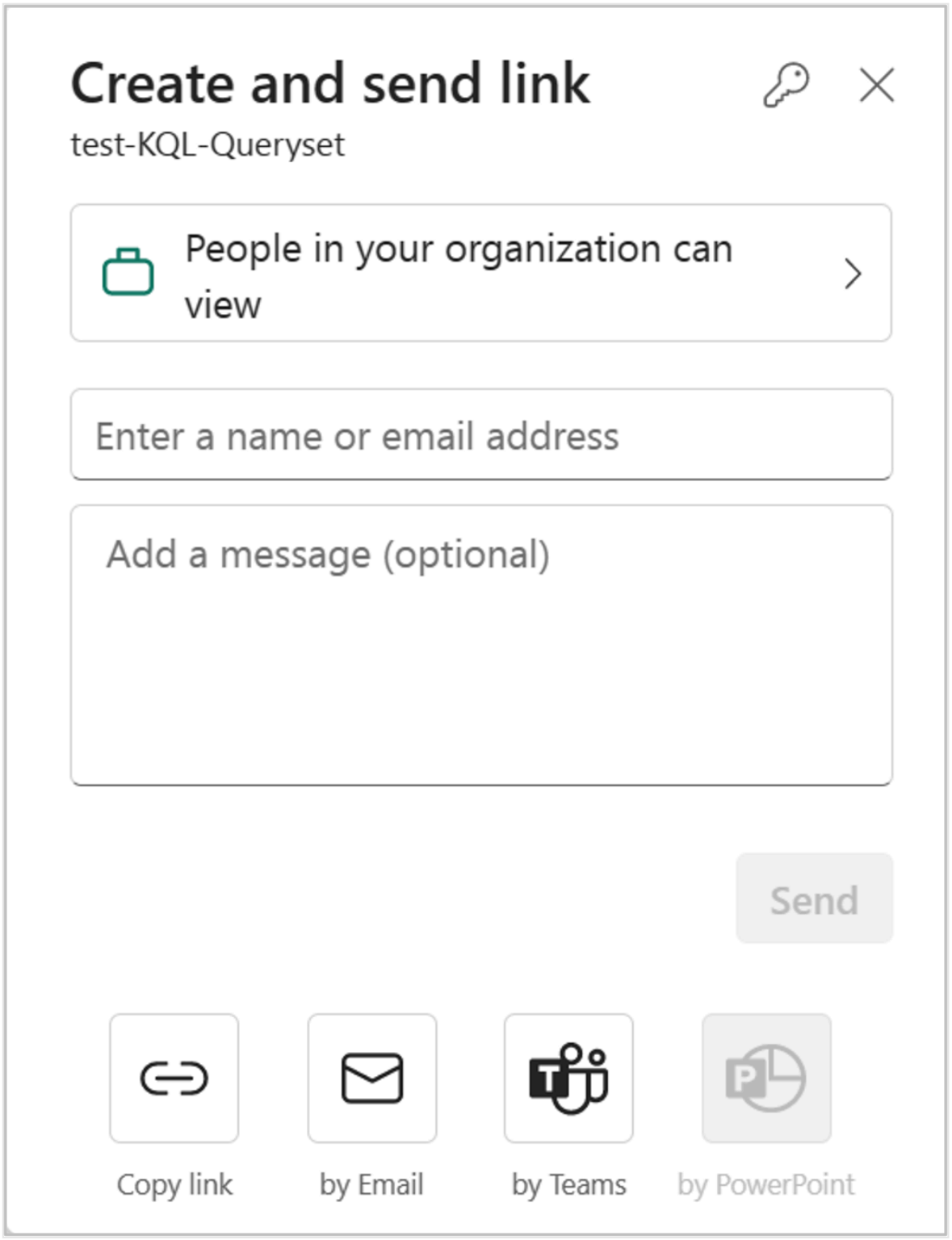
Workspaces are the central places where you collaborate with your colleagues in Microsoft Fabric. Besides assigning workspace roles, you can also use item sharing to grant and manage item-level permissions in scenarios where:

- You want to collaborate with colleagues who don't have a role in the workspace.
- You want to grant additional item level-permissions for colleagues who already have a role in the workspace.

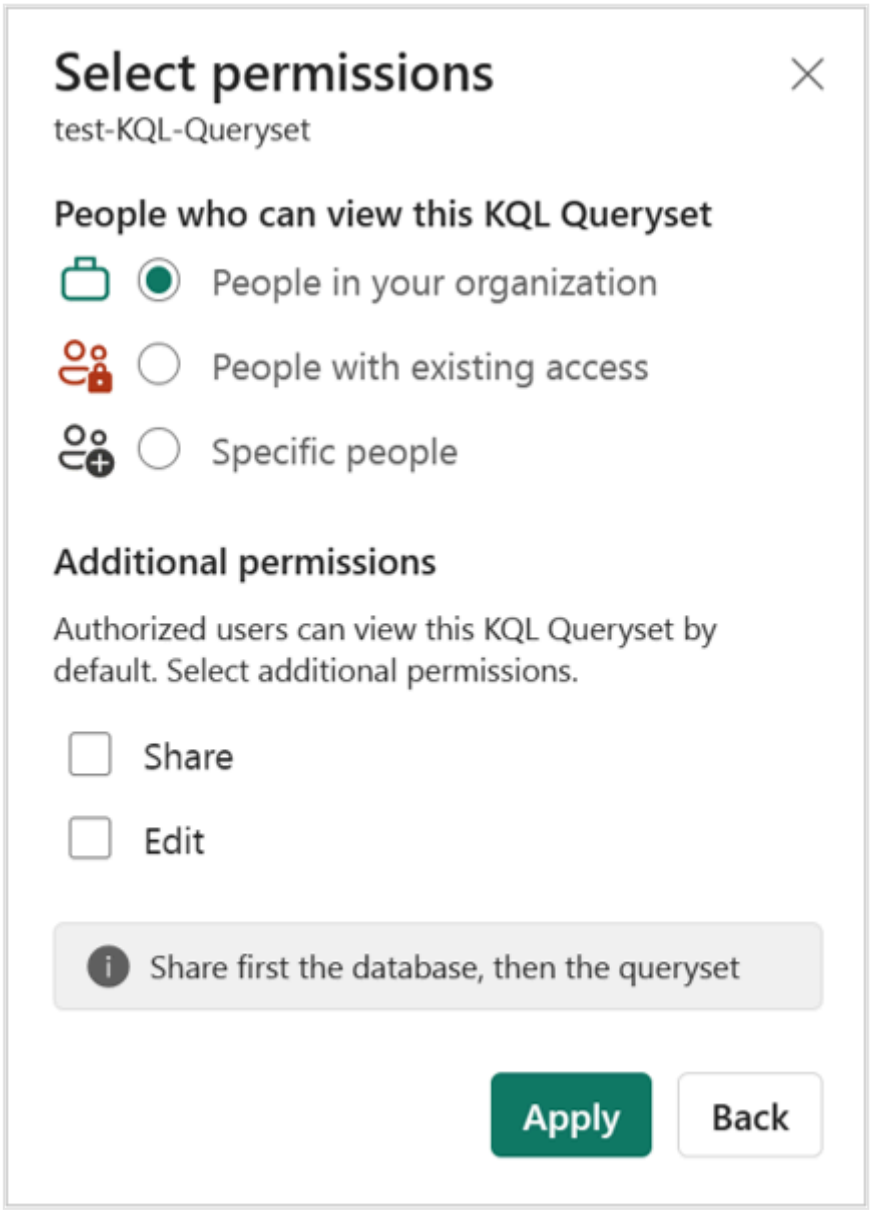
This document describes how to share an item and manage its permissions.

Share an item via link

1. In the list of items, or in an open item, select the **Share** button .
2. The **Create and send link** dialog opens. Select **People in your organization can view**.



3. The **Select permissions** dialog opens. Choose the audience for the link you're going to share.



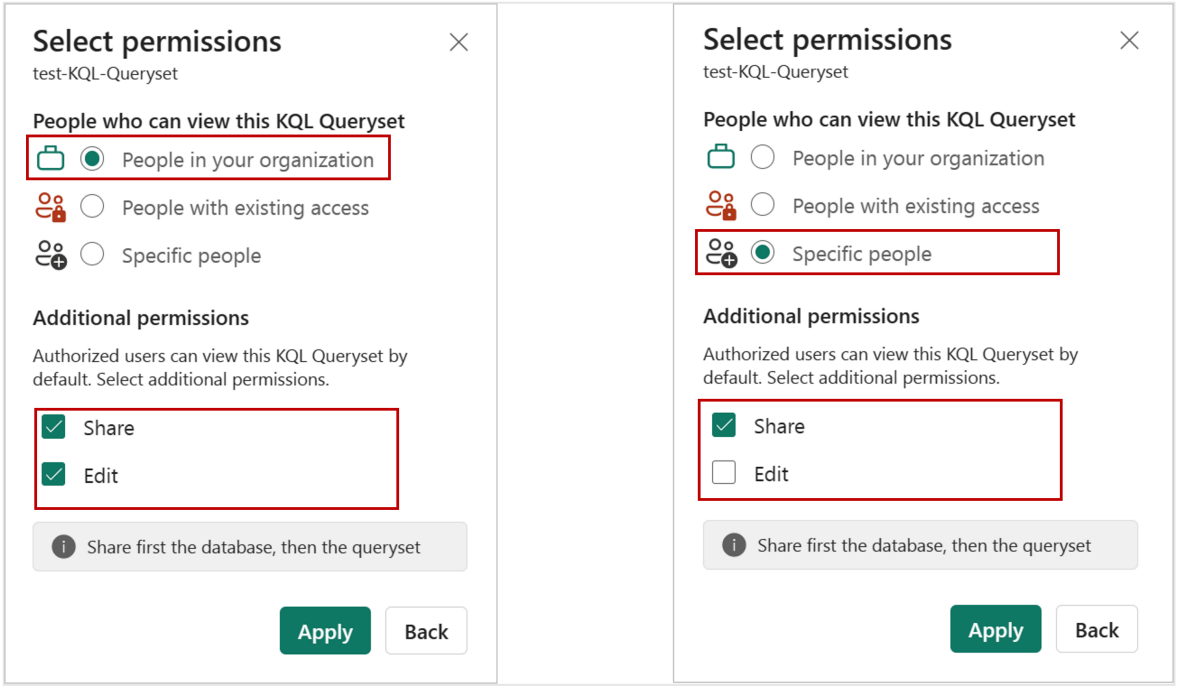
You have the following options:

- **People in your organization** This type of link allows people in your organization to access this item. It doesn't work for external users or guest users. Use this link type when:
  - You want to share with someone in your organization.
  - You're comfortable with the link being shared with other people in your organization.
  - You want to ensure that the link doesn't work for external or guest users.
- **People with existing access** This type of link generates a URL to the item, but it doesn't grant any access to the item. Use this link type if you just want to send a link to somebody who already has access.
- **Specific people** This type of link allows specific people or groups to access the report. If you select this option, enter the names or email addresses of the people you wish to share with. This link type also lets you share to guest

users in your organization's Microsoft Entra ID. You can't share to external users who aren't guests in your organization.

> **Note**
>
> If your admin has disabled shareable links to **People in your organization**, you can only copy and share links using the **People with existing access** and **Specific people** options.

4. Choose the permissions you want to grant via the link.

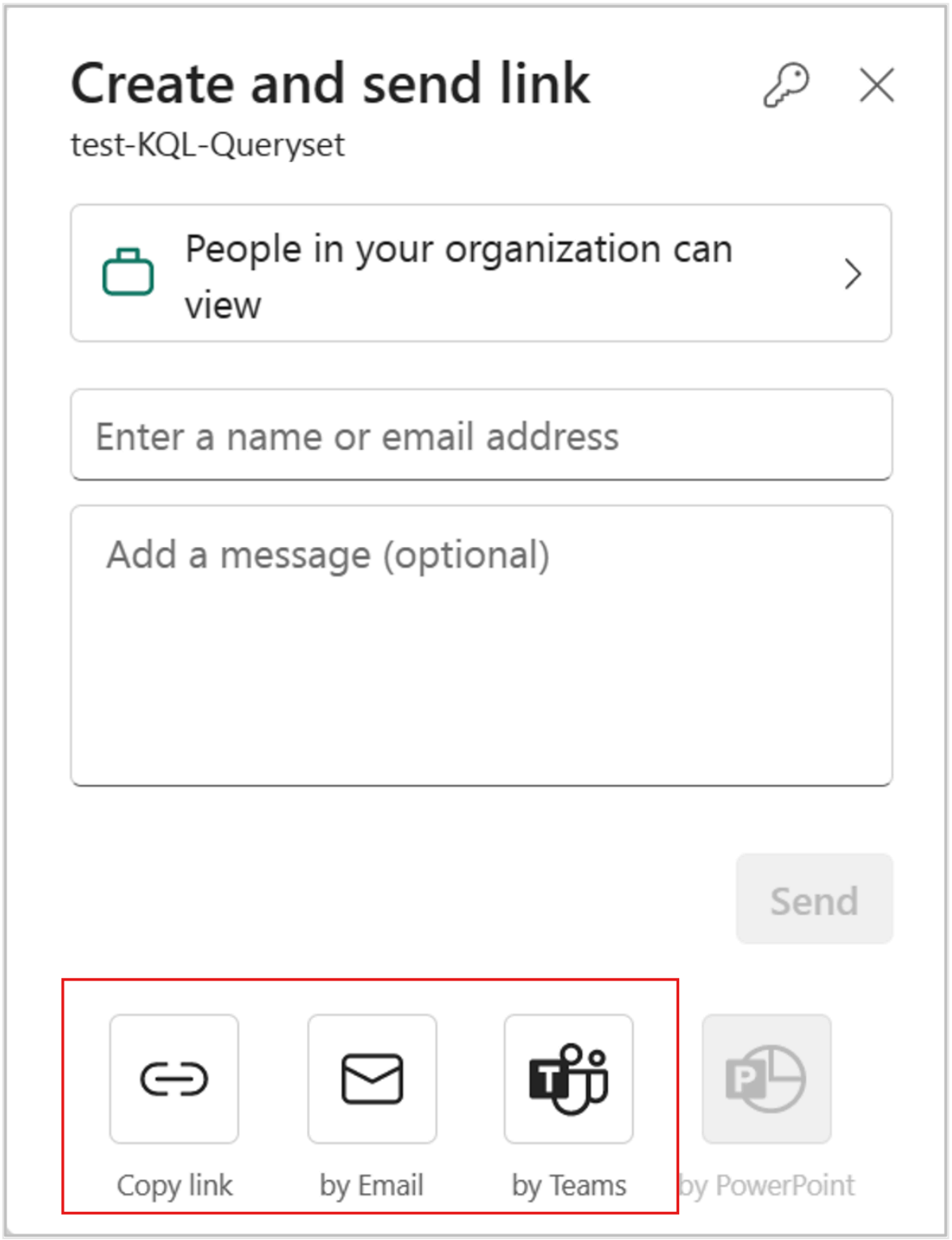


Links that give access to **People in your organization** or **Specific people** always include at least read access. However, you can also specify if you want the link to include additional permissions as well.

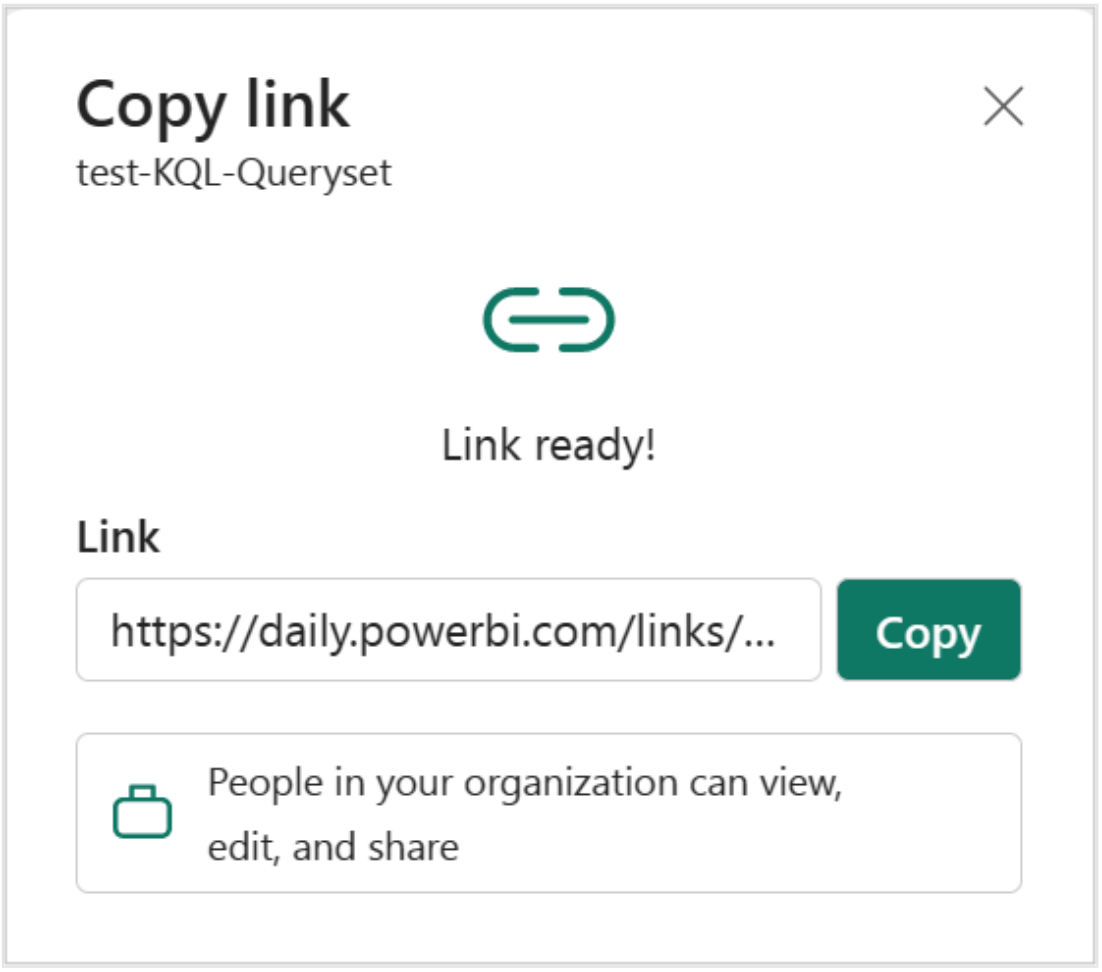
> **Note**
>
> The **Additional permissions** settings vary for different items. Learn more about the item permission model.
>
> Links for **People with existing access** don't have additional permission settings because these links don't give access to the item.

Select **Apply**.

5. In the **Create and send link** dialog, you have the option to copy the sharing link, generate an email with the link, or share it via Teams.

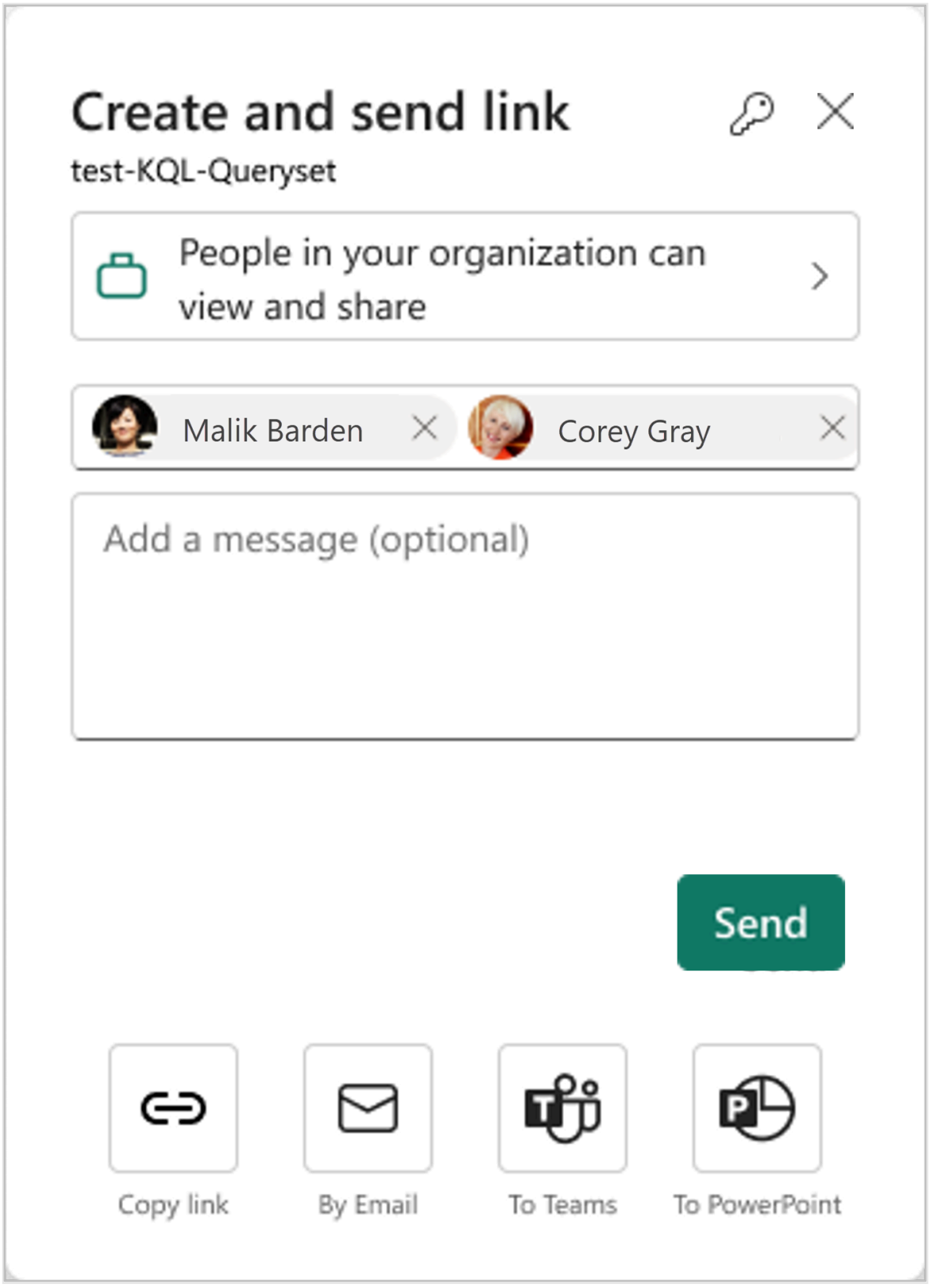


- **Copy link**: This option automatically generates a shareable link. Select **Copy** in the **Copy link** dialog that appears to copy the link to your clipboard.



- **by Email**: This option opens the default email client app on your computer and creates an email draft with the link in it.
- **by Teams**: This option opens Teams and creates a new Teams draft message with the link in it.

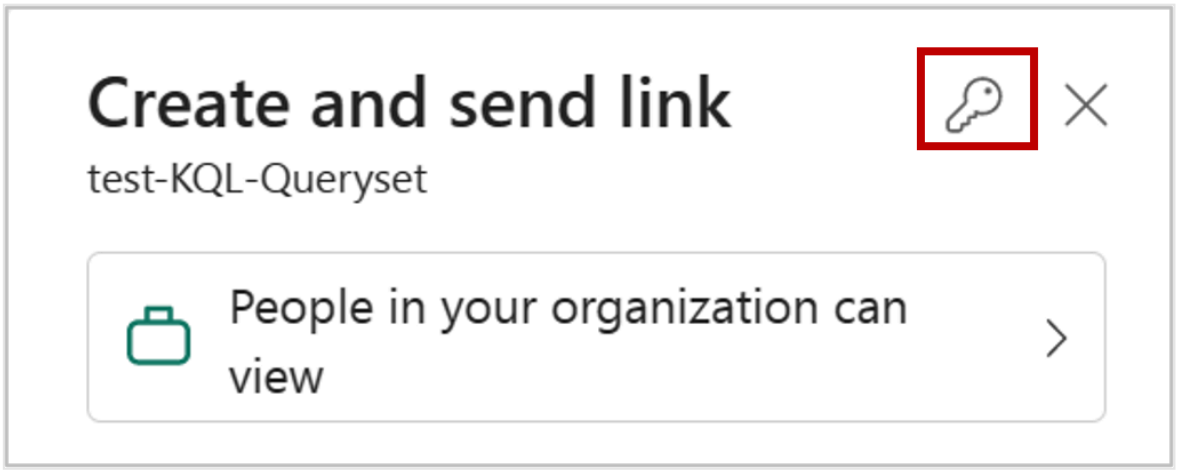
6. You can also choose to send the link directly to **Specific people** or groups (distribution groups or security groups). Enter their name or email address, optionally type a message, and select **Send**. An email with the link is sent to your specified recipients.



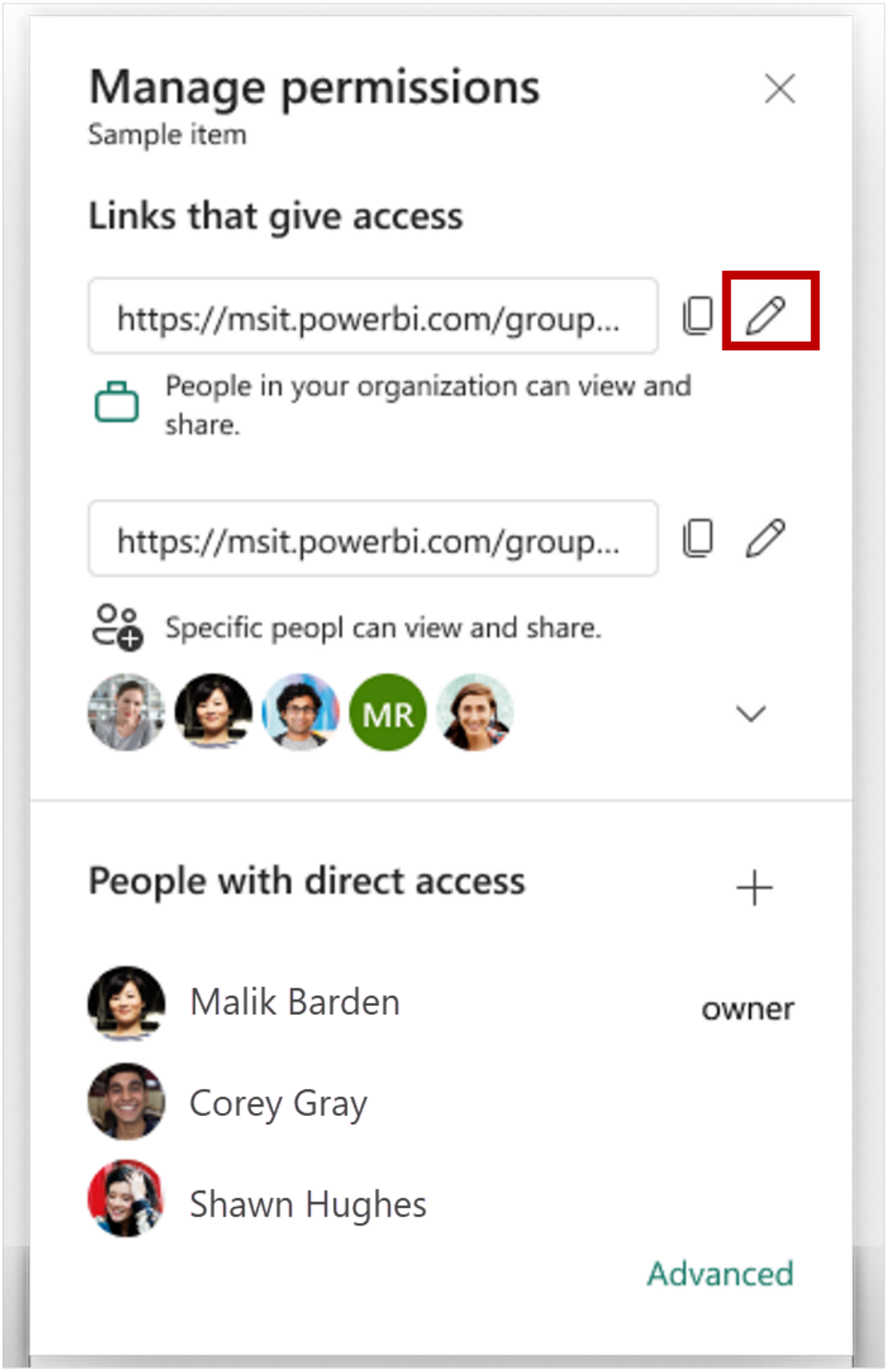
When your recipients receive the email, they can access the report through the shareable link.

Manage item links

1. To manage links that give access to the item, in the upper right of the sharing dialog, select the **Manage permissions** icon:

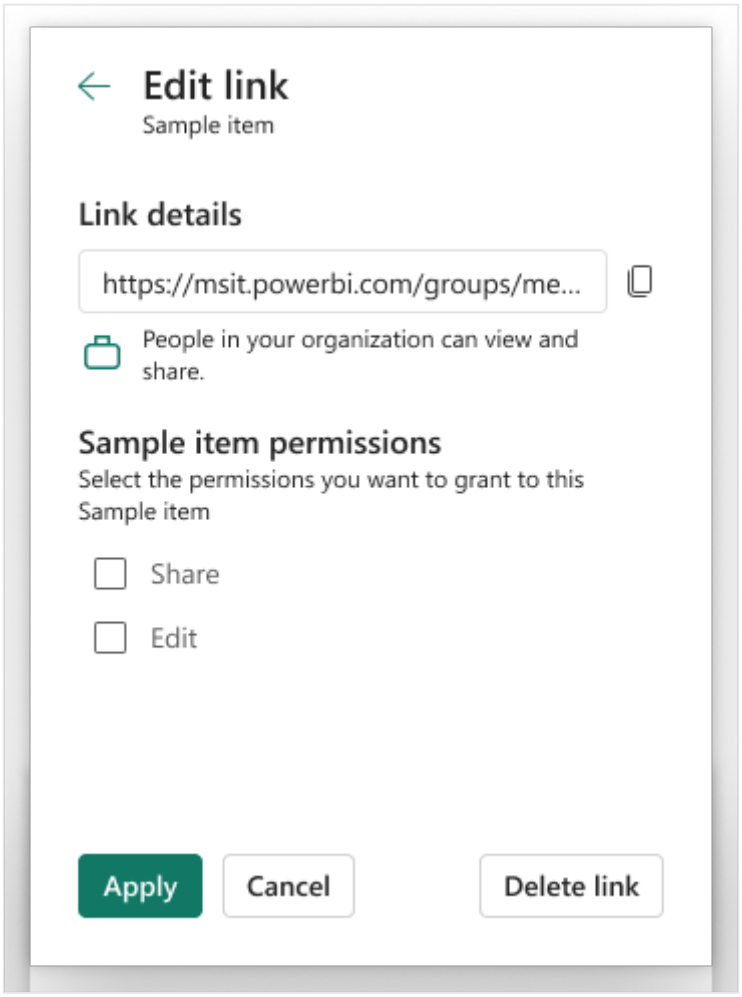


2. The **Manage permissions** pane opens, where you can copy or modify existing links or grant users direct access. To modify a given link, select **Edit**.

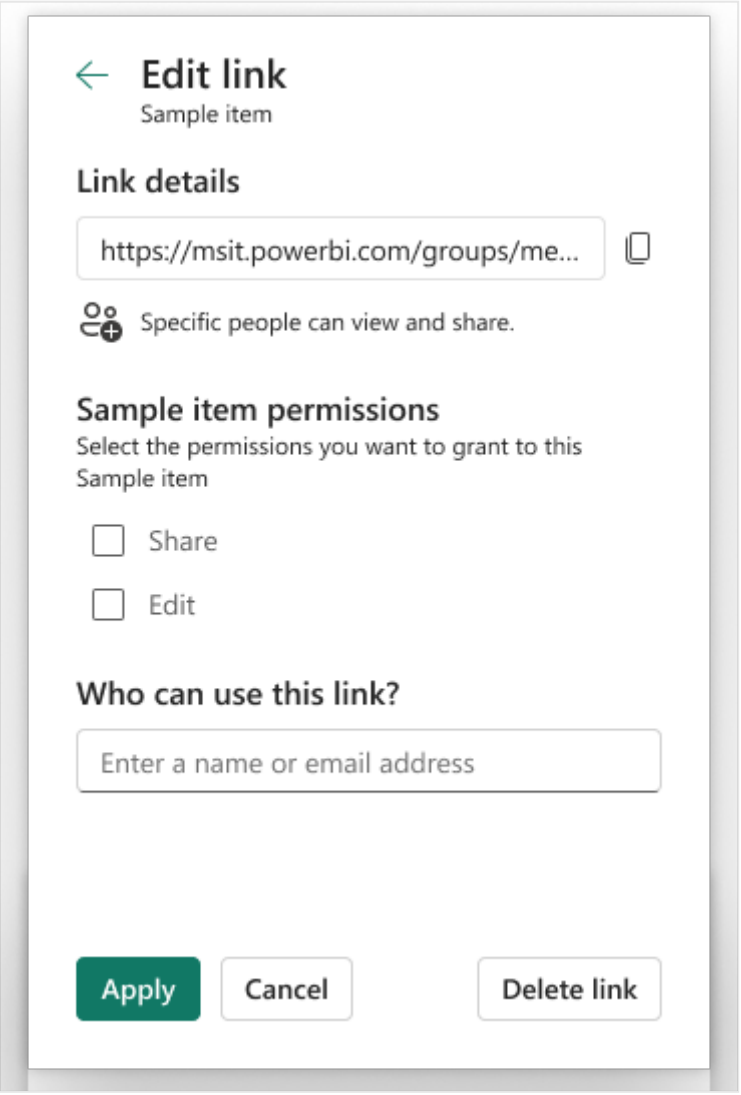


3. In the **Edit link** pane, you can modify the permissions included in the link, people who can use this link, or delete the link. Select **Apply** after your modification.

   This image shows the **Edit link** pane when the selected audience is **People in your organization can view and share**.

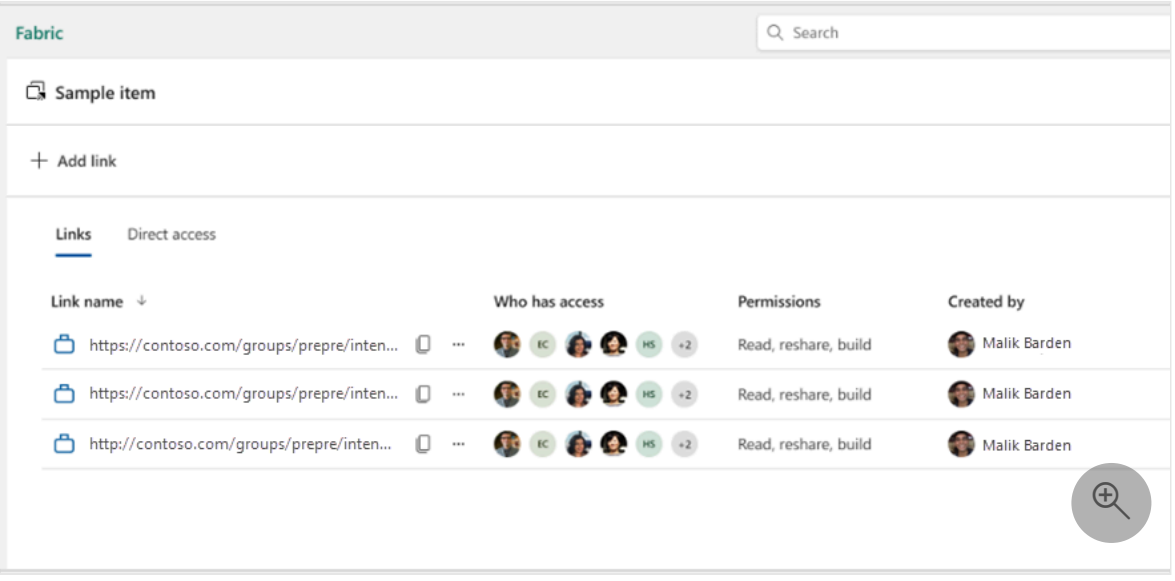


This image shows the **Edit link** pane when the selected audience is **Specific people can view and share**. Note that the pane enables you to modify who can use the link.



4. For more access management capabilities, select the **Advanced** option in the footer of the Manage permissions pane. On the management page that opens, you can:

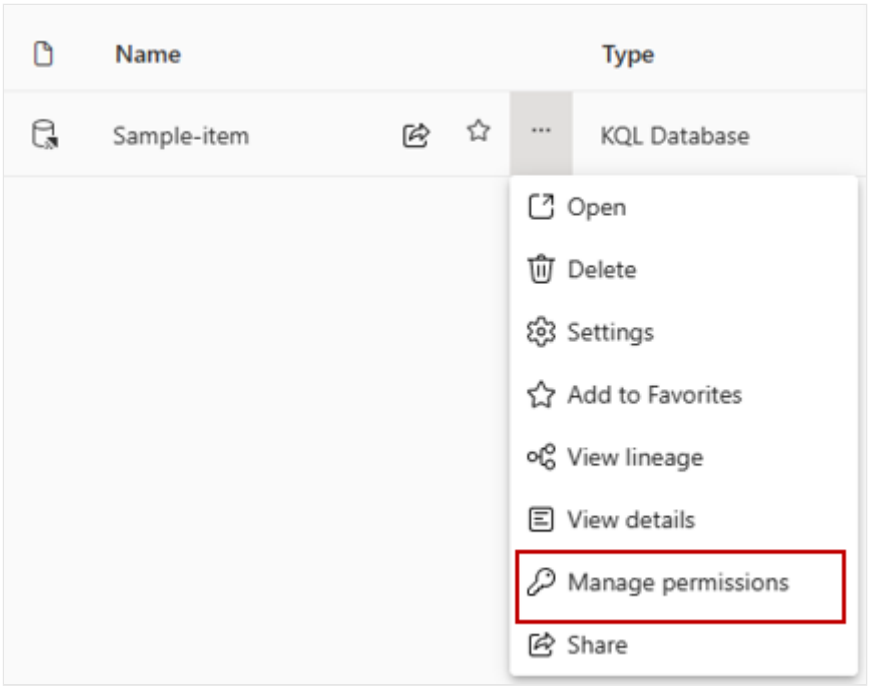
   - View, manage, and create links.
   - View and manage who has direct access and grant people direct access.
   - Apply filters or search for specific links or people.



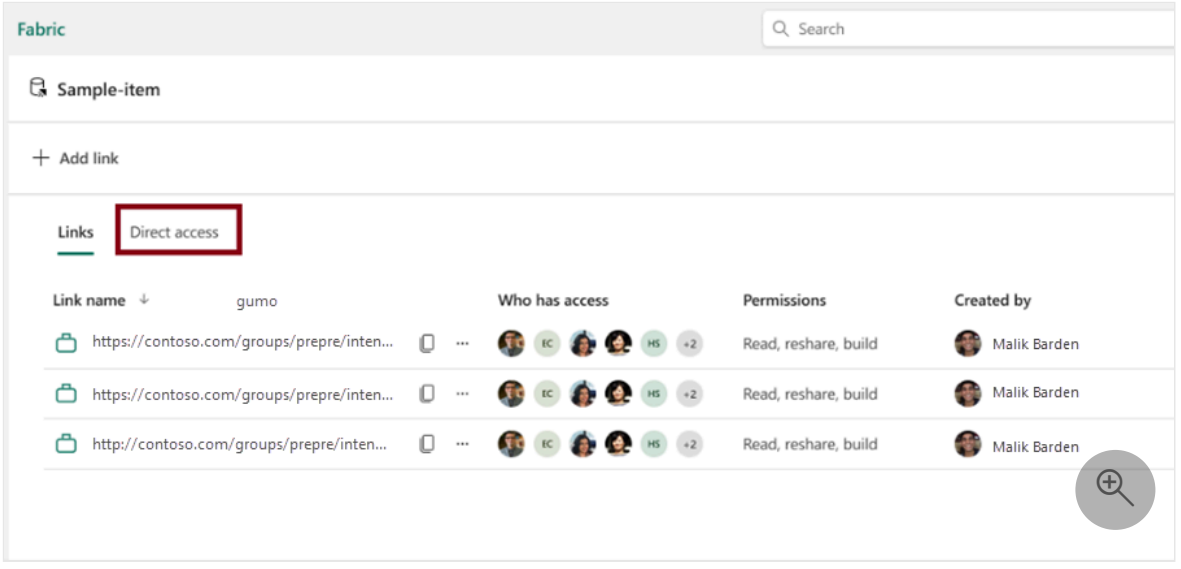
Grant and manage access directly

In some cases, you need to grant permission directly instead of sharing link, such as granting permission to service account, for example.

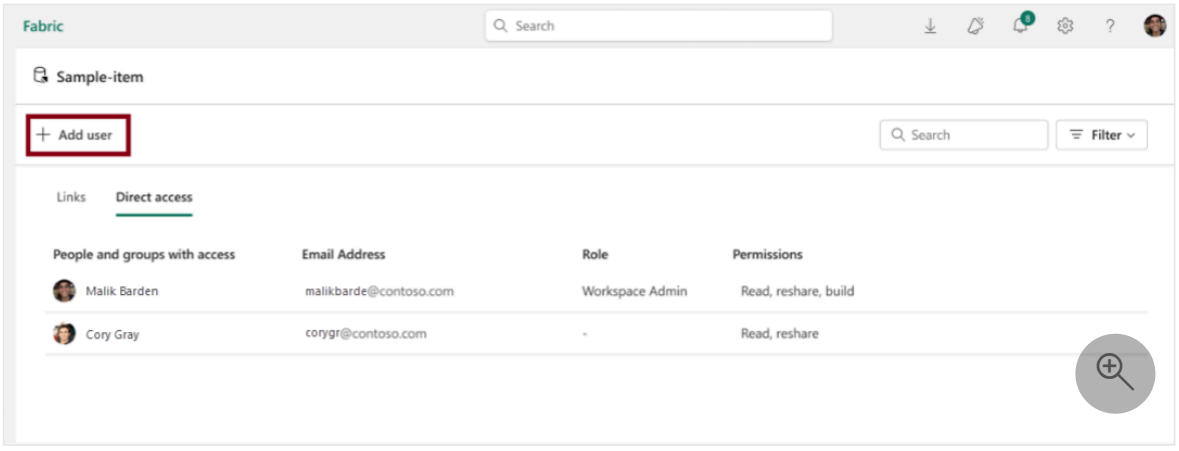
1. Select **Manage permission** from the context menu.



2. Select **Direct access**.

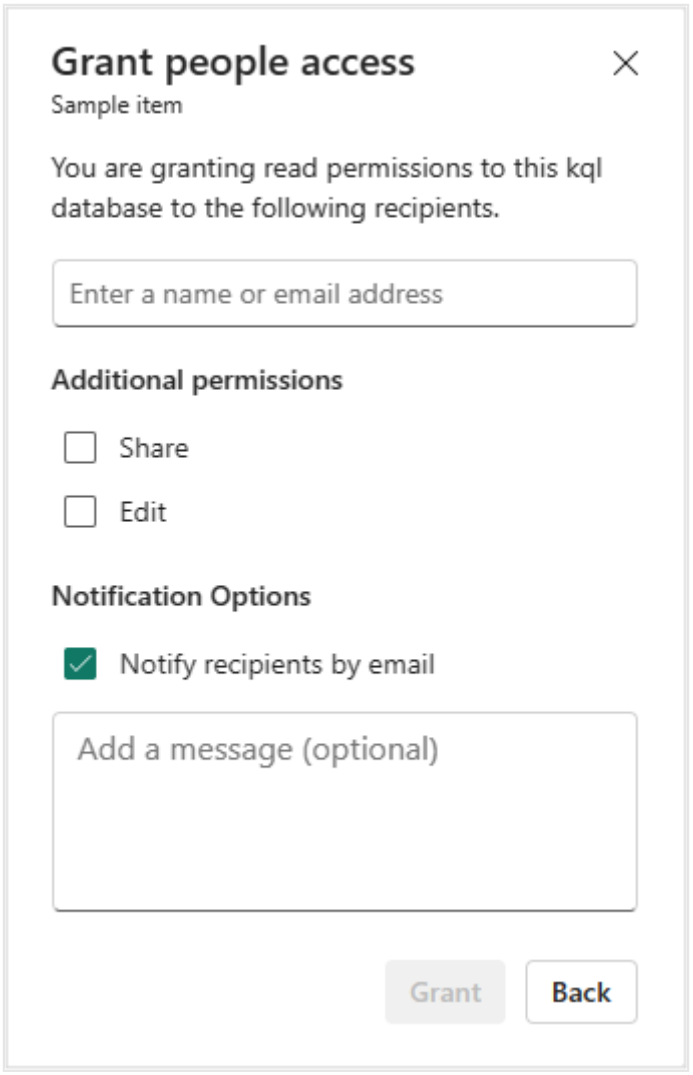


3. Select **Add user**.

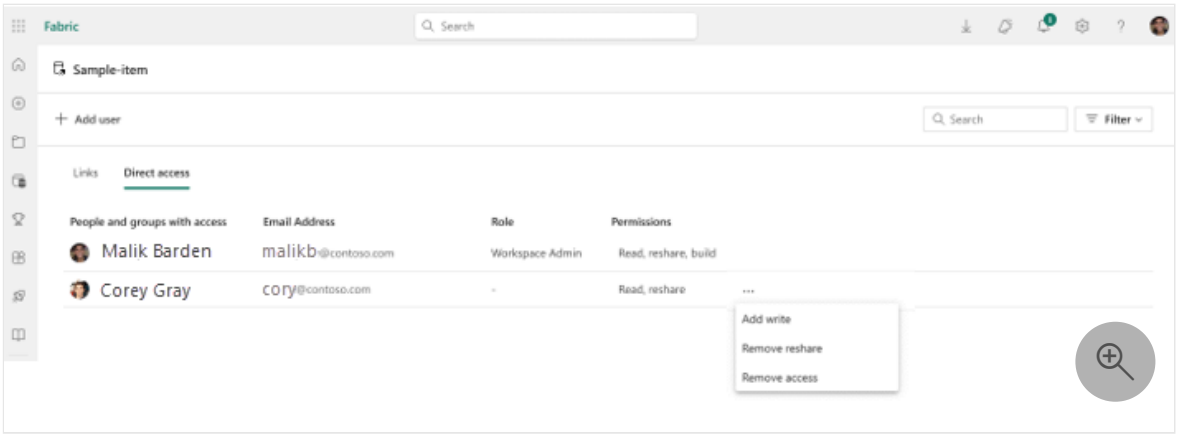


4. Enter the names of people or accounts that you need to grant access to directly. Select the permissions that you want to grant. You can also optionally notify recipients by email.

5. Select **Grant**.



6. You can see all the people, groups, and accounts with access in the list on the permission management page. You can also see their workspace roles, permissions, and so on. By selecting the context menu, you can modify or remove the permissions.



> **Note**
>
> You can't modify or remove permissions that are inherited from a workspace role in the permission management page. Learn more about workspace roles and the item permission model.

Item permission model

Depending on the item being shared, you may find a different set of permissions that you can grant to recipients when you share. Read permission is always granted during sharing, so the recipient can discover the shared item in the OneSource data hub and open it.

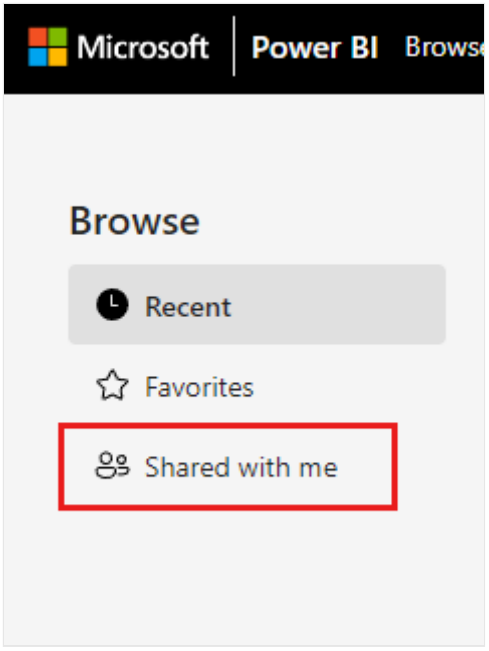
Expand table

Permission granted while sharing	Effect
Read	Recipient can discover the item in the data hub and open it. Connect to the Warehouse or SQL analytics endpoint of the Lakehouse.
Edit	Recipient can edit the item or its content.
Share	Recipient can share the item and grant permissions up to the permissions that they have. For example, if the original recipient has *Share*, *Edit*, and *Read* permissions, they can at most grant *Share*, *Edit*, and *Read* permissions to the next recipient.
Read All with SQL analytics endpoint	Read data from the SQL analytics endpoint of the Lakehouse or Warehouse data through TDS endpoints.
Read all with Apache Spark	Read Lakehouse or Data warehouse data through OneLake APIs and Spark. Read Lakehouse data through Lakehouse explorer.

Permission granted while sharing	Effect
Subscribe to OneLake events	Subscribe to OneLake events for lakehouses, data warehouses, mirrored databases, SQL databases, and KQL databases.
Build	Build new content on the semantic model.
Execute	Execute or cancel execution of the item.

Considerations and limitations

- When a user's permission on an item is revoked through the manage permissions experience, it can take up to two hours for the change to take effect if the user is signed-in. If the user is not signed in, their permissions will be evaluated the next time they sign in, and any changes will only take effect at that time.
- The **Shared with me** option in the **Browse** pane currently only displays Power BI items that have been shared with you. It doesn't show you non-Power BI Fabric items that have been shared with you.



- The sharing dialog for lakehouses, data warehouses, mirrored databases, and SQL databases shows an option to subscribe to OneLake events. Permission to subscribe to OneLake events is granted along with the *Read All* Apache Spark permission. This is a temporary limitation.

Related content

- Workspace roles

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Apply sensitivity labels to Fabric items

Article • 01/26/2025

Sensitivity labels from Microsoft Purview Information Protection on items can guard your sensitive content against unauthorized data access and leakage. They're a key component in helping your organization meet its governance and compliance requirements. Labeling your data correctly with sensitivity labels ensures that only authorized people can access your data. This article shows you how to apply sensitivity labels to your Microsoft Fabric items.

> ⓘ **Note**
>
> For information about applying sensitivity labels in Power BI Desktop, see Apply sensitivity labels in Power BI Desktop.

Prerequisites

Requirements needed to apply sensitivity labels to Fabric items:

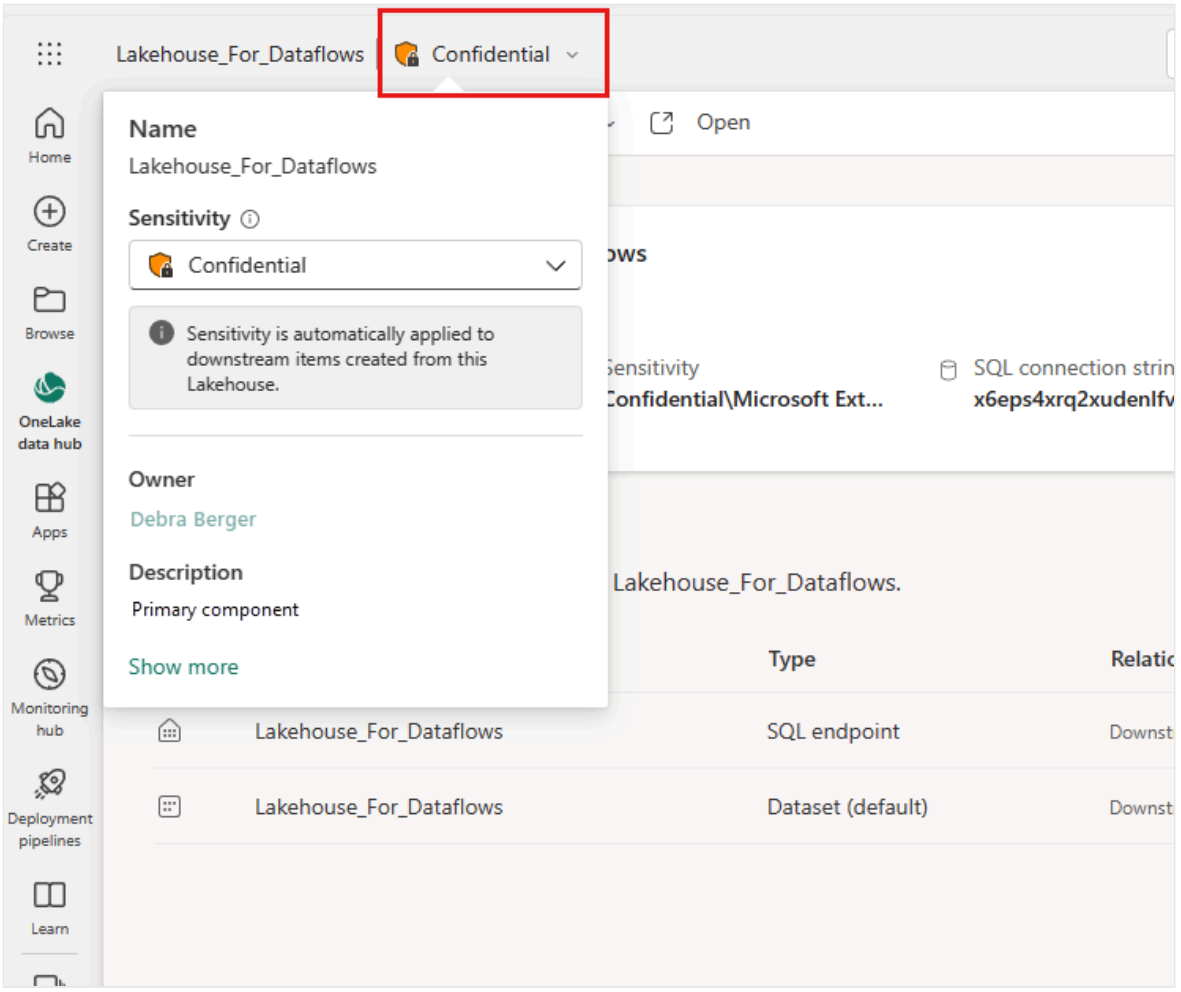
- Power BI Pro or Premium Per User (PPU) license
- Edit permissions on the item you wish to label.

> ⓘ **Note**
>
> If you can't apply a sensitivity label, or if the sensitivity label is greyed out in the sensitivity label menu, you may not have permissions to use the label. Contact your organization's tech support.

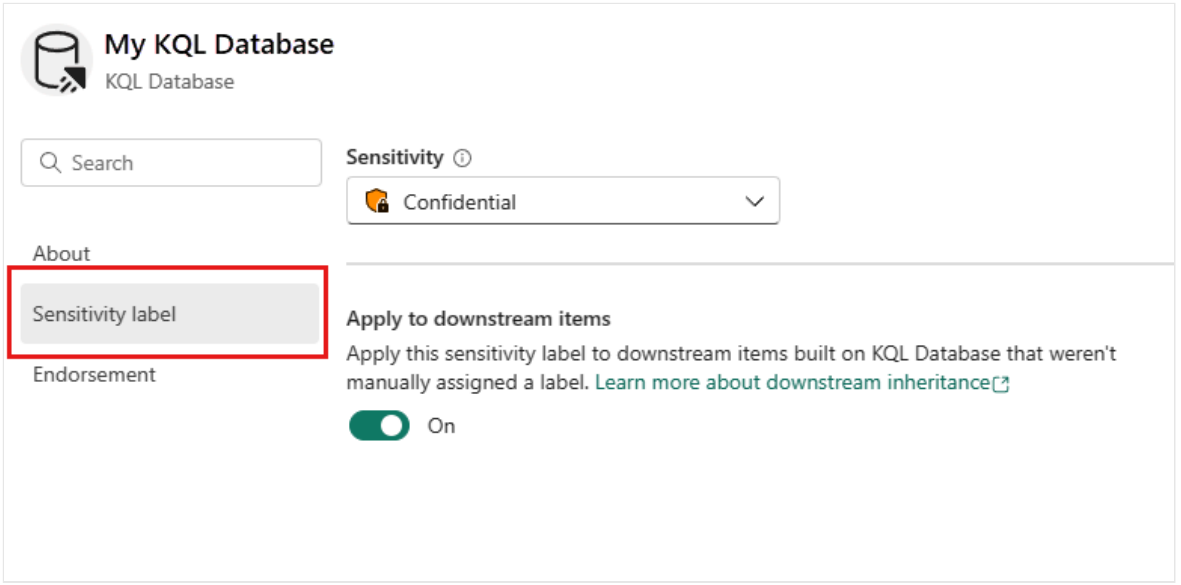
Apply a label

There are two common ways of applying a sensitivity label to an item: from the flyout menu in the item header, and in the item settings.

- From the flyout menu - select the sensitivity indication in the header to display the flyout menu:



- In items settings - open the item's settings, find the sensitivity section, and then choose the desired label:



Related content

- Sensitivity label overview

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Job scheduler in Microsoft Fabric

Automate and streamline your data analytics workflows in Microsoft Fabric with the job scheduler. This article explains how to use the job scheduler to set up recurring jobs, manage multiple schedules, and integrate scheduling with your CI/CD processes. Learn how to keep your items up to date and ensure reliable task automation in your Fabric workspace.

What is the job scheduler?

The job scheduler is a built-in tool that lets you schedule jobs to run automatically at specified times. Use it to trigger data refreshes, pipeline runs, or other supported tasks for items in your Fabric workspace.

Access the job scheduler

To access the job scheduler:

1. Open your Fabric workspace and locate the item you want to schedule (for example, Notebook).
2. Select the item's contextual menu.
3. Select **Schedule** from the menu to open the job scheduler interface.

You can also access the scheduler from within an item's settings by selecting the **Schedule** label.

> ⓘ **Note**
>
> To set up or modify a schedule, or to configure failure notifications, you must have at least the **Contributor** role in the workspace, or **Write** permission on the item. Users with only **Viewer** access can see existing schedules but can't make changes. For more information, see Roles in workspaces.

Set up your schedule

After you open the scheduler for an item:

1. Toggle the scheduler **On** to enable scheduling.
2. Use the configuration panel to set when and how often the job runs.

3. Save your schedule to activate it.

Choose how often you want the job to run, and specify the start and end times. The scheduler supports various recurrence types, including minute-based, hourly, daily, weekly, monthly, and specific dates or weekdays of a month.

Expand table

Recurrence type	Details and options
Minute-based	Every 1 to 720 minutes
Hourly	Every 1 to 72 hours
Daily	Up to 10 times per day
Weekly	On selected weekdays, up to 10 times per week
Monthly	Every 1 to 12 months
Specific dates/weekdays of a month	Select specific dates or weekdays in a month

If your configuration doesn't match a valid time, the scheduler skips that run and waits for the next valid time.

To ensure your schedule is valid:

- Set a valid start and end time for your schedule.
- Make sure the end time is after the start time.
- The scheduler automatically handles special cases like daylight saving time and invalid dates (for example, February 30).

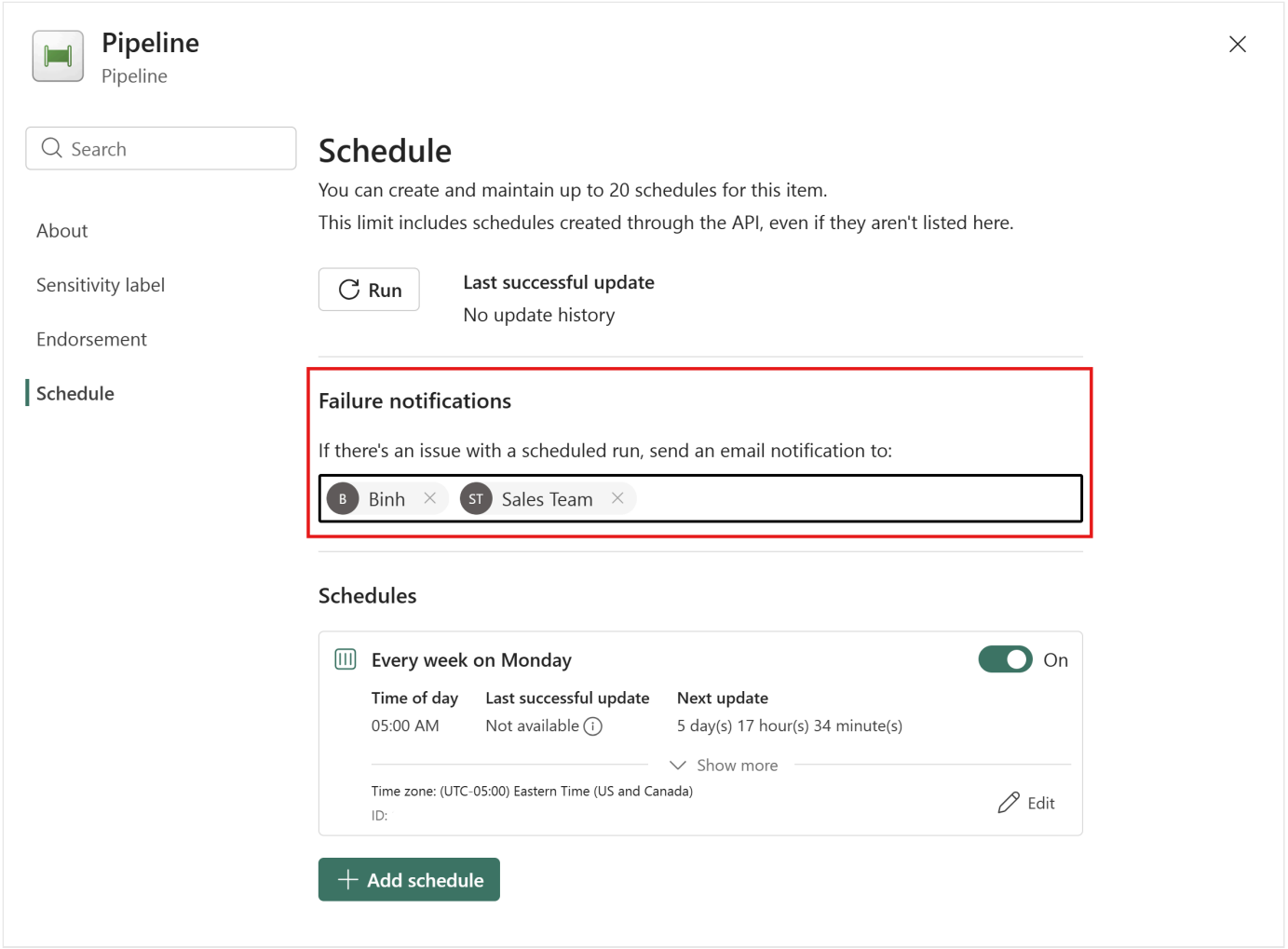
Job throttling limits

- The Scheduler limits each user to 100 job submissions and 100 Get Job requests per minute to maintain system stability. Requests beyond these limits are automatically rejected.
- Fabric limits each item to 100 concurrent jobs. All jobs submitted beyond this number are automatically failed. A job counts toward this limit as long as it's in any active state (starting, preparing, queueing, or running). The slot is released only after the job completes.
- Jobs can run for a maximum of 24 days. Any jobs exceeding this duration are automatically terminated.

> **Important**
>
> Schedules become expired if a user doesn't log in to Fabric for 90 consecutive days. For more information, see Refresh tokens in the Microsoft identity platform.

Receive notifications for failed scheduled jobs

You can receive email notifications when a job triggered by a **schedule** fails to complete.



Configure failure notifications

While configuring a schedule, add users or groups under **Failure notifications** to receive emails when a scheduled run fails.

Important considerations

- **Available for Fabric items that support scheduling**: Notifications are available across all Fabric items that support scheduling (for example, Pipelines, Notebooks, and Dataflows

Gen2).

- **Applies to all schedules**: Notifications are configured at the item level and the settings apply across all schedules for the item.
- **Scheduled runs only**: Notifications are sent only for failures from scheduled runs. Notifications aren't sent for manually triggered runs.
- **Recipients**: Notifications can be sent to users or groups in your Microsoft Entra tenant, including internal users and B2B guest users. Direct external email addresses aren't supported.
- **Language**: Notifications are sent in the display language of the recipient's Fabric account. English is used as a fallback.

Notification content

When a scheduled run fails, the email includes:

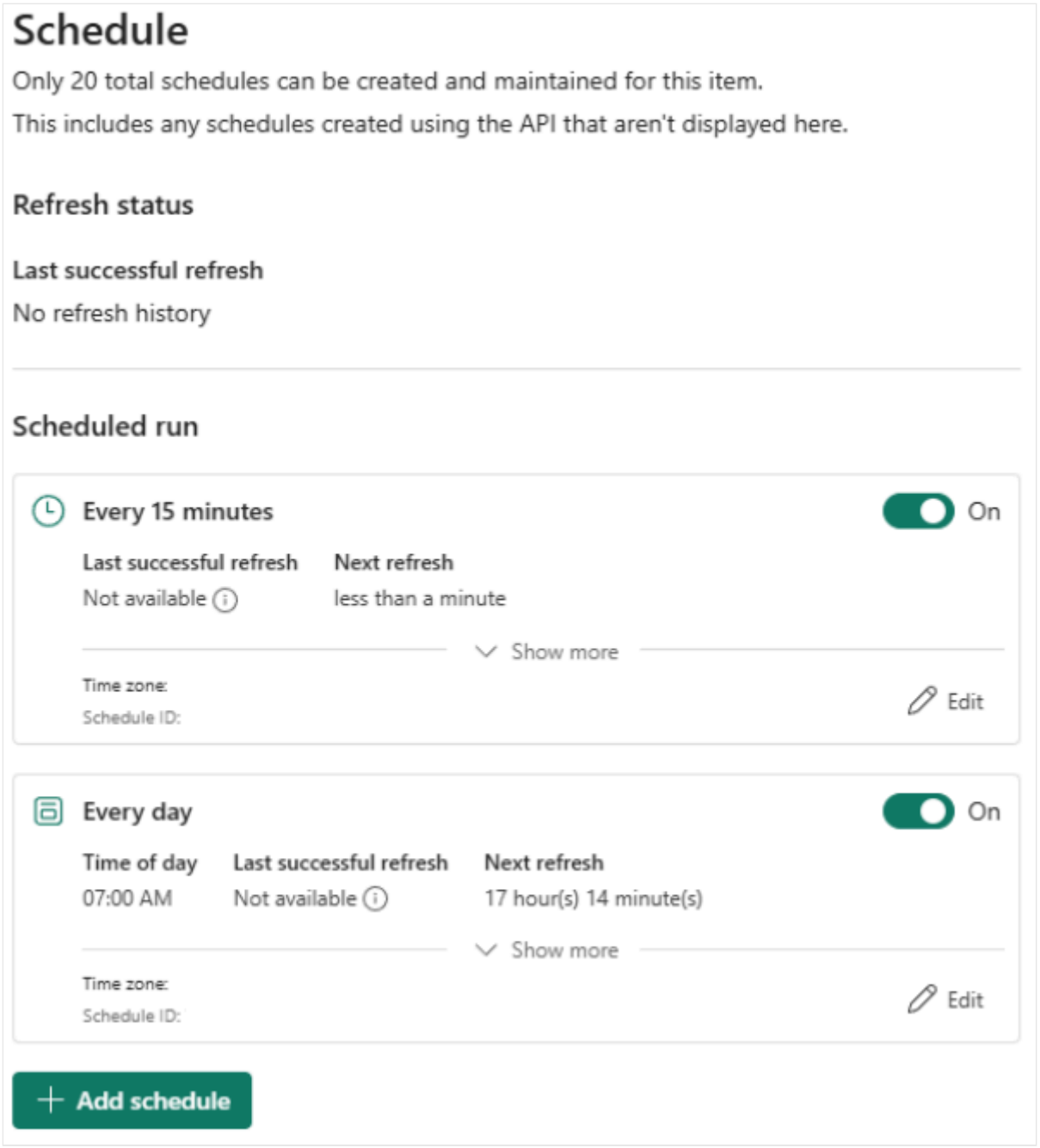
- Item name and type
- Submitter
- Error details
- Run time (UTC)
- Link to view details in the monitoring hub
- Technical details for troubleshooting, including Activity ID, Request ID, and timestamps

> **Tip**
>
> To view and manage job failure notifications across all your scheduled items in one place, see Schedule failures in the monitoring hub.

Manage multiple schedules

Create and manage multiple schedules for a single item. Use different schedules to run jobs at different times or with different settings.

1. On the **Schedule** screen, view, add, and edit multiple schedule configurations for each item.
2. Select the schedule you want to manage, or create a new one for different job types or timing needs.



Scheduler auto-disable

If a scheduler repeatedly triggers jobs and encounters consecutive failures, the Fabric platform shuts down the scheduler and places it into an *auto-disabled* state to prevent further predictable job failures. To resume normal job scheduling, you must manually restart the scheduler through the UI, API, or CI/CD process.

> **Note**
>
> The failure threshold that triggers the auto-disabled state varies by item and is typically set to 10 consecutive failed runs.

Automate schedules with CI/CD

The job scheduler supports CI/CD integration, so you can deploy and manage schedules as part of your development workflow.

Expand table

Workflow	Description
Deployment pipelines	Schedules are included when you deploy an item
Git integration	Schedules are stored in a `.schedules` file in your item definition
Public API	Manage schedules by using code

When you deploy an item, its schedules are automatically included, so you don't need to recreate them manually. For step-by-step instructions, see CI/CD workflow options in Fabric.

If you use variable library in the parameters, data sync might experience latency after CI/CD operations. This behavior is expected and will be improved in future releases.

> **Important**
>
> All items that had a scheduler configured prior to CI/CD being enabled appear as "uncommitted" when running `git status`. Carefully review and confirm the changes that need to be committed to avoid unintended actions. Items without prior scheduler configuration aren't affected. We apologize for any inconvenience this may cause.

Summary

The job scheduler in Microsoft Fabric lets you automate recurring jobs, manage multiple schedules for each item, and integrate scheduling into your CI/CD workflows. Use it to streamline your data analytics process, and make sure your jobs run reliably and efficiently.

Related content

- CI/CD workflow options in Fabric
- Monitor job activity and status in the monitoring hub

Last updated on 05/13/2026

Delta Lake overview

Microsoft Fabric uses Delta Lake as its universal storage format across all workloads. Whether you're building data warehouses, creating lakehouses, orchestrating pipelines, or analyzing data with Power BI, your data is stored in Delta Lake format in OneLake.

For most scenarios, Delta Lake works transparently in the background—you don't need to understand it to use Fabric effectively. However, when you're optimizing performance, integrating with external systems, or troubleshooting cross-workload scenarios, understanding Delta Lake helps you work more efficiently.

What is Delta Lake?

Delta Lake is an open-source storage layer that brings reliability and performance to data lakes. At its core, Delta Lake combines two elements:

- **Parquet files** — An efficient columnar storage format for analytics.
- **Transaction log** — A record of all changes to the data that enables ACID guarantees.

This combination provides capabilities that traditional file-based storage can't offer:

- **ACID transactions**: Atomic, consistent, isolated, and durable operations ensure data integrity even when multiple processes read and write simultaneously
- **Time travel**: Query data as it existed at any point in time, roll back mistakes, or audit historical changes
- **Schema evolution**: Add, remove, or modify columns without rewriting existing data
- **Unified batch and streaming**: Process real-time and batch data using the same table format

Delta Lake is maintained as a Linux Foundation project with an open specification, which means it works across platforms—Databricks, Azure Synapse Analytics, AWS, and open-source Apache Spark environments all support Delta Lake tables.

Why Fabric uses Delta Lake

Microsoft Fabric chose Delta Lake as its universal format to solve a fundamental challenge: enabling all workloads to share data without duplication or complex ETL processes.

OneLake + Delta Lake = single source of truth

OneLake stores all data as Delta Parquet by default. This architectural decision means:

- **No data silos**: A lakehouse, warehouse, and KQL database can all read from the same Delta table
- **No ETL between workloads**: Data written by one engine is immediately readable by others
- **One copy of data**: Your organization maintains a single version of the truth, reducing storage costs and eliminating data synchronization issues

Performance optimization built on Delta

Fabric extends Delta Lake with performance features designed for the platform:

- **V-Order**: Fabric-specific write optimization that reorganizes data for faster query performance across all engines
- **Automatic table maintenance**: Fabric can automatically compact small files and clean-up old data
- **Integration with Fabric services**: Delta tables work seamlessly with Power BI Direct Lake, SQL analytics endpoints, pipelines, and notebooks

This combination of open standards and Fabric optimizations gives you both portability and performance.

Understanding your Delta Lake journey

How much you need to know about Delta Lake depends on what you're doing in Fabric:

Delta Lake is transparent for most users

If you're working with these scenarios, Fabric handles Delta Lake automatically:

- **Querying data with SQL**: SQL analytics endpoints and data warehouses abstract Delta behind a SQL interface—you query tables using T-SQL without thinking about file formats
- **Building reports in Power BI**: Power BI Direct Lake reads Delta tables seamlessly to deliver real-time analytics
- **Loading data with pipelines**: Data Factory copy activity writes to Delta format by default when targeting lakehouse tables

For these experiences, Delta Lake is an implementation detail you don't need to manage.

You benefit from understanding Delta when

Certain scenarios require knowledge of Delta Lake features:

- **Optimizing query performance**: Understanding V-Order, table compaction, and partitioning strategies helps you tune tables for faster queries
- **Integrating with external Delta tables**: Connecting to Delta tables in Databricks, Snowflake, Amazon S3, or other platforms requires knowledge of feature compatibility
- **Using advanced Spark features**: Deletion vectors, column mapping, type widening, and liquid clustering provide powerful capabilities but need explicit configuration
- **Troubleshooting cross-workload scenarios**: When data written by one engine doesn't behave as expected in another, understanding Delta feature support helps you diagnose issues

Choosing your learning path

Your entry point to Delta Lake documentation depends on your role:

Data engineers and Spark developers should start with Lakehouse and Delta Lake tables for comprehensive coverage of Delta in Spark contexts, including optimization patterns, table maintenance, and runtime configurations.

SQL users and data warehouse developers should explore Data Warehouse architecture to understand how Delta powers warehouse capabilities like time travel, clones, and ACID compliance.

Pipeline and data integration developers should review Lakehouse connector in Data Factory to learn how pipelines write Delta tables and handle column mapping.

Anyone working across workloads should reference Delta Lake table format interoperability for the authoritative feature support matrix showing which Delta capabilities work in each Fabric experience.

Table optimization and maintenance

While Fabric handles many optimization tasks automatically, you can improve performance for specific scenarios:

- **V-Order optimization**: Write data with V-Order enabled to improve read performance across all query engines
- **Table compaction**: Combine small files into larger ones using OPTIMIZE commands to reduce query overhead

- **Z-ordering**: Organize data by frequently filtered columns to speed up selective queries
- **Vacuum operations**: Remove old file versions to reduce storage costs

For detailed guidance on when and how to use these techniques across different workloads, see Table maintenance and optimization.

Delta Lake and external systems

Delta Lake's open specification enables integration with systems outside Fabric:

- **OneLake shortcuts**: Reference Delta tables stored in Amazon S3, Azure Data Lake Storage, or Databricks without copying data
- **Delta sharing**: Share Delta tables across organizations using the open Delta Sharing protocol
- **Cross-platform compatibility**: Tables created in Databricks or Azure Synapse can be read in Fabric, subject to feature support constraints

When integrating external Delta tables, consult the feature interoperability matrix to understand which Delta Lake features are supported by each Fabric experience.

Related content

- Delta Lake table format interoperability — Feature support matrix for Delta capabilities across Fabric workloads
- Lakehouse and Delta Lake tables — Comprehensive guide for Spark developers
- Table maintenance and optimization — Cross-workload optimization strategies
- OneLake, the OneDrive for data — Learn about Fabric's unified data lake
- Delta Lake official documentation — Open-source project specifications and features

ⓘ **Note:** This article contains content created with AI. Learn more

---

Last updated on 06/05/2026

Delta Lake table format interoperability

In Microsoft Fabric, the Delta Lake table format is the standard for analytics. Delta Lake is an open-source storage layer that brings ACID (Atomicity, Consistency, Isolation, Durability) transactions to big data and analytics workloads.

All Fabric experiences natively generate and consume Delta Lake tables, providing a unified product experience. Delta Lake tables produced by one compute engine, such as Fabric Data Warehouse or Synapse Spark, can be consumed by any other engine, such as Power BI. When you ingest data into Fabric, it stores the data as Delta tables by default. You can easily integrate external data containing Delta Lake tables by using OneLake shortcuts.

Delta Lake features and Fabric experiences

To achieve interoperability, all the Fabric experiences align on the Delta Lake features and Fabric capabilities. Some experiences can only write to Delta Lake tables, while others can read from it.

- **Writers**: Data warehouses, eventstreams, and exported Power BI semantic models into OneLake
- **Readers**: SQL analytics endpoint and Power BI direct lake semantic models
- **Writers and readers**: Fabric Spark runtime, dataflows, pipelines, and Kusto Query Language (KQL) databases

The following matrix shows key Delta Lake features and their availability on each Fabric experience.

Expand table

Fabric capability	Column mappings	Deletion vectors	V-order writing	Table optimization and maintenance	Partitions
Data warehouse Delta Lake export	Name: Yes ID: No	Yes	Yes	Yes	Read: N/A (not applicable) Write: No
SQL analytics endpoint	Name: Yes ID: No	Yes	N/A (not applicable)	N/A (not applicable)	Read: Yes Write: N/A (not applicable)

Fabric capability	Column mappings	Deletion vectors	V-order writing	Table optimization and maintenance	Partitions
Lakehouse explorer and preview	Name: Yes ID: No	Yes	N/A (not applicable)	Yes	Read: Yes Write: N/A (not applicable)
Fabric Spark Runtime 2.0	Name: Yes ID: Yes	Yes	Yes	Yes	Read: Yes Write: Yes
Fabric Spark Runtime 1.3	Name: Yes ID: Yes	Yes	Yes	Yes	Read: Yes Write: Yes
Fabric Spark Runtime 1.2	Name: Yes ID: Yes	Yes	Yes	Yes	Read: Yes Write: Yes
Fabric Spark Runtime 1.1	Name: Yes ID: Yes	No	Yes	Yes	Read: Yes Write: Yes
Python notebooks	Name: No ID: No	No	No	Yes	Read: Yes Write: Yes
Dataflows Gen2	Name: Yes ID: No	Yes	Yes	No	Read: Yes Write: Yes
Pipelines	Name: No ID: No	No	Yes	No	Read: Yes Write: Yes, overwrite only
Power BI direct lake semantic models	Name: Yes ID: Yes	Yes	N/A (not applicable)	N/A (not applicable)	Read: Yes Write: N/A (not applicable)
Export Power BI semantic models into OneLake	Name: Yes ID: No	N/A (not applicable)	Yes	No	Read: N/A (not applicable) Write: No
KQL databases	Name: Yes ID: No	Yes	No	No [1]	Read: Yes Write: Yes
Eventstreams	Name: No ID: No	No	No	No	Read: N/A (not applicable) Write: Yes

[1] KQL databases provide certain table maintenance capabilities such as retention. Data is removed at the end of the retention period from OneLake. For more information, see One Logical copy.

Expand table

Fabric capability	Liquid Clustering	TIMESTAMP_NTZ	Delta reader/writer version and default table features	Type Widening
Data warehouse Delta Lake export	No	No	Reader: 3 Writer: 7 Deletion Vectors, Column Mappings (name)	Yes
SQL analytics endpoint	Yes	No	N/A (not applicable)	Yes
Lakehouse explorer and preview	Yes	Yes	N/A (not applicable)	Yes
Fabric Spark Runtime 2.0	Yes	Yes	Reader: 3 Writer: 7 Deletion Vectors	Yes
Fabric Spark Runtime 1.3	Yes	Yes	Reader: 1 Writer: 2	No
Fabric Spark Runtime 1.2	Yes, read only	Yes	Reader: 1 Writer: 2	No
Fabric Spark Runtime 1.1	Yes, read only	No	Reader: 1 Writer: 2	No
Python notebooks	No	Yes	Reader: 1 Writer: 2	No
Dataflows Gen2	Yes, read only	No	Reader: 1 Writer: 2	Yes
Pipelines	Yes, read only	No	Reader: 1 Writer: 2	Yes: DI Data Pipelines (Copy, Lookup activity) No: DI CopyJob pipelines
Power BI direct lake semantic models	Yes	No	N/A (not applicable)	Yes
Export Power BI semantic models into OneLake	No	No	Reader: 2 Writer: 5 Column Mappings (name)	No

Fabric capability	Liquid Clustering	TIMESTAMP_NTZ	Delta reader/writer version and default table features	Type Widening
KQL databases	No	No	Reader: 1 Writer: 1	No
Eventstreams	No	No	Reader: 1 Writer: 2	No

> **Note**
>
> - Fabric doesn't write column mappings by default, except where noted. The default Fabric experience generates tables that are compatible across the service. Delta Lake tables produced by third-party services might have incompatible table features.
> - Some Fabric experiences don't offer table optimization and maintenance capabilities, such as bin-compaction, V-order, deletion vector merge (PURGE), and clean up of old unreferenced files (VACUUM). To keep Delta Lake tables optimal for analytics, follow the techniques in Run Delta table maintenance in Lakehouse for tables ingested using those experiences.
> - For comprehensive cross-workload guidance on table maintenance strategies for different consumption scenarios, see Cross-workload table maintenance and optimization in Microsoft Fabric.

Current limitations

Currently, Fabric doesn't support these Delta Lake features:

- Delta Lake 3.x Uniform. This feature is supported only in the Data Engineering Spark-compute (Notebooks, Spark Jobs).
- Identity columns writing (Azure Databricks feature).
- Lakeflow Spark Declarative Pipelines (Azure Databricks feature).
- Delta Lake 4.x features outside of Lakehouse and Spark Notebooks and Jobs: collations, variant type, coordinated commits, and more. Type widening is supported where noted on the previous features matrix in this article.
- **Python notebooks Delta Lake feature support**: Python notebooks use delta-rs (`deltalake`), DuckDB, and Polars instead of the Spark Delta Lake reader and writer to read and write Delta Lake tables. The `deltalake` versions currently pinned by Fabric don't support some Delta Lake table features, including deletion vectors, column mapping, liquid

clustering writes and optimization, and type widening. Polars `read_delta` and `write_delta` use `deltalake` as the backend, so they share the same limitations.

- **Python notebooks Delta Lake workarounds and fallback**: The DuckDB `delta_scan` function can provide a read-only workaround for some tables that `deltalake` can't read, such as tables with deletion vectors. If a Delta Lake capability isn't available or doesn't work in a Python notebook, use a PySpark notebook instead. PySpark notebooks provide the most complete Delta Lake support in Fabric and cover the Delta Lake scenarios listed for Fabric Spark runtimes in these tables. For more information, see Data interaction in Python notebooks and the Choosing between Python and PySpark Notebooks in Microsoft Fabric.

Special characters on table names

Fabric supports special characters as part of the table names. This feature allows the usage of Unicode characters to compose table names in Fabric experiences.

The following special characters are either reserved or not compatible with at least one of Fabric technologies and must not be used as part of a table name: `"` (double quotes), `'` (single quote), `#`, `%`, `+`, `:`, `?`, or ` (backtick).

Related content

- Delta Lake in Microsoft Fabric overview
- Cross-workload table maintenance and optimization in Microsoft Fabric
- Lakehouse and Delta Lake tables
- Direct Lake overview
- Querying tables from the Warehouse through its published Delta Lake Logs

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 10/07/2026

Cross-workload table maintenance and optimization in Microsoft Fabric

Delta tables in Microsoft Fabric can serve Spark, SQL analytics endpoint, Power BI Direct Lake, Warehouse, and other Fabric experiences from data stored in OneLake. Optimal cross-workload performance depends on two factors:

- The workload that creates and maintains the table.
- The engines that consume the table.

Lakehouse tables are commonly managed by Spark, Fabric pipeline Copy activity, or Dataflow Gen2. Spark is the most common writer and provides the broadest layout and maintenance controls. Warehouse and database mirroring manage their physical layouts automatically. Mirrored catalogs retain the layout managed in the source system. Consumer requirements are generally compatible, but Power BI Direct Lake has additional storage requirements for optimal performance.

Use one shared table whenever its requirements are compatible. For the exceptions that justify another table, see When to create another table.

Understand layout ownership

Start by identifying which workload owns the physical table layout. The controls in the following table are the key controls relevant to cross-workload table layout and maintenance, not an exhaustive list of each engine's capabilities.

Expand table

Data store	Writer or ingestion method	Layout and maintenance ownership	Key controls
Lakehouse	Spark	User-managed	**File sizing**: adaptive target file size and file-level compaction targets. **Write and maintenance**: deletion vectors, auto compaction, optimize write, OPTIMIZE, and VACUUM. **Data organization**: liquid clustering, partitioning, Z-Order, and V-Order.

Data store	Writer or ingestion method	Layout and maintenance ownership	Key controls
Lakehouse	Fabric pipeline Copy activity or Dataflow Gen2	The service writes the data; the lakehouse owner maintains the table	Destination-specific write settings. Run compatible maintenance separately by using Spark, Lakehouse maintenance, or a pipeline maintenance activity.
Warehouse	Fabric Data Warehouse, Fabric pipeline Copy activity, or Dataflow Gen2	Warehouse-managed	Data clustering and the warehouse-level V-Order setting.
Mirrored item	Mirroring service	Depends on the mirroring type	Database mirroring uses a system-managed V-Ordered Delta layout with no direct layout controls. Mirrored catalogs retain the source file layout, which you can optimize in the source system when supported.

Cross-workload guidance

The following table summarizes the recommended approach by producer and consumer.

Expand table

Producer	Consumer	Recommended approach
Lakehouse: Spark writer	Spark	Use Fabric Spark runtime 2.0 or later defaults and enable auto compaction. Consider liquid clustering when measured predicates benefit from improved file skipping.
Lakehouse: Spark writer	SQL analytics endpoint	Use the same layout recommended for Spark. Don't set a static target file size, arbitrary row limit, or V-Order solely for SQL analytics endpoint performance.
Lakehouse: Spark writer	Power BI Direct Lake	Use the same layout recommended for Spark and additionally enable V-Order, or use the readHeavyForPBI resource profile.
Lakehouse: Fabric pipeline or Dataflow Gen2 writer	Spark, SQL analytics endpoint, or Power BI Direct Lake	Monitor the resulting file layout and schedule compatible lakehouse maintenance separately. Some destination modes, such as Dataflow Gen2 incremental refresh, impose maintenance restrictions.
Warehouse	Fabric Data Warehouse or Spark	Use the system-managed layout. Fabric Data Warehouse automatically manages compaction and other maintenance.

Producer	Consumer	Recommended approach
		Use data clustering to improve file skipping for workloads with recurring selective predicates.
Warehouse	Power BI Direct Lake	Keep the default Warehouse V-Order setting. Use data clustering when it benefits shared query patterns.
Mirroring	Spark, SQL analytics endpoint, or Power BI Direct Lake	For database mirroring, use the system-managed V-Ordered Delta layout. For mirrored catalogs, optimize the underlying files in the source system when supported. See What is Mirroring in Fabric?.

Optimize Lakehouse tables

Lakehouse Delta tables require an explicit maintenance strategy regardless of whether Spark, Pipeline Copy activity, or Dataflow Gen2 writes them. Spark is the primary example in this section because it provides the broadest layout and maintenance controls in Fabric.

> **Important**
>
> Table maintenance is critical for optimal write and read performance across engines. Even append-only workloads that initially perform well without maintenance can accumulate excessive small files, which affect Spark, SQL analytics endpoint, Direct Lake, and external data readers. See **Compacting Delta tables** for automatic and manual compaction methods.

Use the Spark runtime defaults

When Spark writes the table, use Fabric Spark runtime 2.0 or later defaults:

- Keep adaptive target file size enabled. It automatically selects a target for each table from 128 MB to 1 GB.
- Keep file-level compaction targets enabled to avoid rewriting files that met an earlier adaptive target.
- Keep deletion vectors enabled.
- Don't impose an arbitrary maximum row count per file. Row width varies, so a row limit can create excessive small files for narrow tables.

In Fabric Spark runtime 1.3, adaptive target file size, file-level compaction targets, and deletion vectors are available as opt-in settings.

When Pipeline Copy activity or Dataflow Gen2 writes the table, inspect the resulting file layout and schedule maintenance separately. Don't assume that these writers apply Spark runtime defaults.

- Fabric pipelines can orchestrate a Lakehouse maintenance activity after writes.

> **Important**
>
> Dataflow Gen2 lakehouse destinations that use incremental refresh don't support OPTIMIZE or REORG TABLE. Follow the Dataflow Gen2 incremental refresh limitations.

Prevent and compact small files

For Spark-written tables, prefer auto compaction. This feature evaluates table fragmentation after writes and runs compaction only when needed. It eliminates the need for a separate table-health check before maintenance runs.

Use the following guidance for exceptions and complementary features:

Expand table

Scenario	Recommended approach
Spark-written table	Enable auto compaction as the default maintenance strategy.
Streaming or microbatch writes	Enable auto compaction and optimize write to reduce small-file accumulation.
Workloads with strict write-latency requirements	Schedule OPTIMIZE separately instead of running synchronous auto compaction.
Existing table with accumulated small files	Run a one-time OPTIMIZE, then enable auto compaction for ongoing maintenance.
Tables with frequent updates, deletes, or merges	Keep deletion vectors and auto compaction enabled.

OPTIMIZE compacts files and automatically purges a file's deletion vectors when more than 5% of its records are referenced by deletion vectors. Use REORG TABLE ... APPLY (PURGE) only when you must physically purge records below that threshold or meet a specific compliance requirement.

> **Note**

Auto compaction purges deletion vectors only when the partition also meets its small-file trigger. If a workload performs updates or deletes without generating small files, periodically run OPTIMIZE to purge qualifying deletion vectors. Use REORG TABLE ... APPLY (PURGE) when you must force a physical purge.

Run VACUUM on a separate schedule to remove unreferenced files after the retention period. `VACUUM` reclaims storage but doesn't improve the active file layout.

> ⚠ **Warning**
>
> Don't shorten the VACUUM retention period without evaluating time-travel requirements and concurrent readers or writers. Removing files too early can make required table versions unavailable.

Organize data for file skipping

Use liquid clustering when recurring filter or processing patterns benefit from improved file skipping. Liquid clustered tables require OPTIMIZEor auto compaction to organize newly written data.

Avoid partitioning by default. Use it when a specific requirement justifies the operational tradeoffs, such as isolating concurrent writers that update, delete, or merge data across disjoint partitions. For more information, see When to use partitioning.

For existing partitioned tables, consider Z-Order when selective predicates commonly filter on the same columns within a partition.

Optimize Warehouse-managed tables

Fabric Data Warehouse manages the physical Delta table layout regardless of the ingestion method.

Use the strategic controls that Warehouse exposes to tune the data layout:

- Apply data clustering to large tables when queries repeatedly use selective predicates on the same columns.
- Keep V-Order enabled for read-oriented and mixed workloads. V-Order is enabled by default.
- Consider disabling V-Order for write-intensive warehouse workloads.

> ⚠ **Warning**
>
> Disabling V-Order is a warehouse-level, irreversible operation. Test the complete read and write workload before disabling it.

For complete Warehouse guidance, see Performance guidelines in Fabric Data Warehouse.

Optimize mirrored data

Your ability to improve the physical layout depends on whether Fabric replicates the data or references source files:

- **Database mirroring**: Fabric replicates source data into Delta tables in OneLake and manages the V-Ordered file layout and maintenance. You can't directly configure target file size, deletion-vector cleanup, liquid clustering, partitioning, or V-Order on the mirrored destination.
- **Mirrored catalogs**: Fabric synchronizes metadata and uses OneLake shortcuts to reference source data in place. Fabric doesn't rewrite or maintain these files. Improve the physical layout and cleanup in the source system when its supported features allow it. Those changes are visible through the shortcuts without creating another copy in Fabric.

For database-mirrored data:

- Use selective predicates and avoid unnecessary columns in Spark and SQL queries.
- Design Power BI semantic models and DAX measures for efficient Direct Lake consumption.

For mirrored catalogs:

- Use the source platform's supported table-maintenance and layout features.
- Evaluate the source file and row-group distribution for the Fabric consumers that query the shortcuts.
- For Direct Lake, evaluate creating an additional dimensionally modeled, V-Ordered serving layer when the source layout can't meet performance requirements.

For mirroring concepts, types, and supported sources, see What is Mirroring in Fabric? and How metadata mirroring works.

Apply consumer-specific optimization

Spark and SQL analytics endpoint perform well on the same adaptive lakehouse layout. Use adaptive target file size, prevent excessive small files, and apply liquid clustering when measured predicates benefit from improved file skipping. Don't enable V-Order solely for Spark or SQL analytics endpoint performance. For engine-specific details, see SQL analytics endpoint performance considerations.

Power BI Direct Lake

Direct Lake uses the same underlying Delta tables but adds recommendations related to transcoding and incremental framing:

- **File and row-group layout**: Avoid small row groups and uneven row group distribution, this creates more VertiPaq column segments and increases transcoding overhead.
- **V-Order**: Follow the producer-specific recommendation in the cross-workload guidance. For Spark-written tables primarily consumed through Direct Lake, enable V-Order or use the readHeavyForPBI resource profile.
- **Update patterns**: Prefer append-friendly update patterns where possible to preserve existing Parquet files and support incremental framing.

> **Note**
>
> Direct Lake generally performs best with row groups between 1 million and 16 million rows. Evaluate row-group distribution and Direct Lake performance before changing a supported producer setting.

For Spark-written tables, `spark.sql.parquet.native.writer.maxRowGroupRowCount` sets the maximum rows per row group when the native execution engine writes the Parquet files. The default value is `0`, which doesn't impose a maximum. If analysis shows that row-group sizing is affecting Direct Lake performance, set a tested limit before writing or rewriting the table. For example:

Python

```
spark.conf.set("spark.sql.parquet.native.writer.maxRowGroupRowCount", 8_000_000)
```

Don't set the limit solely to reach a specific row count. Row width, compression, file distribution, and capacity parallelism also affect performance. Use Delta Analyzer to evaluate the resulting layout.

For detailed guidance on framing, transcoding, row groups, update patterns, and Delta Analyzer, see Understand Direct Lake query performance.

Apply the guidance to medallion layers

Bronze, Silver, and Gold describe data purpose and refinement. They don't determine whether layout is user-managed or system-managed, and they don't require separate copies for each consumer.

Expand table

Layer	Primary goal	Cross-workload guidance
Bronze (landing)	Preserve source fidelity and ingestion throughput	Prioritize write throughput while maintaining Spark-written tables with auto compaction. Avoid Power BI Direct Lake semantic models on raw Bronze tables unless the model and data shape are intentionally designed for that use.
Silver (curated)	Provide validated, conformed data for reuse	Reuse the table across compatible Fabric consumers. For Spark-written lakehouse tables, enable V-Order only when Direct Lake is a primary consumer.
Gold (serving)	Serve business-ready dimensions, facts, aggregates, and analytics models	Prefer this layer for Direct Lake semantic models. Reuse the table across compatible consumers and apply the producer-specific controls described in this article.

Resolve layout and maintenance issues

Use producer-aware remediation. Apply Spark maintenance commands to lakehouse tables when the destination mode supports those operations. Treat the signals as indicators rather than universal thresholds, and validate them against the table's write pattern and consumer performance.

Expand table

Condition	Signal	Lakehouse table	Warehouse table
Excessive small files	File count rises faster than active table size, and files remain below the adaptive target.	With Spark, run a one-time OPTIMIZE for the existing backlog, then enable auto compaction. For Pipeline Copy activity or Dataflow Gen2 writes, schedule supported lakehouse maintenance separately.	No action. Warehouse compaction is automatic.

Condition	Signal	Lakehouse table	Warehouse table
Legacy oversized files	Files remain much higher than the current adaptive target, and too few files limit scan parallelism.	Rewrite the table by using an overwrite or `CREATE OR REPLACE TABLE AS SELECT` with adaptive target file size enabled.	No action. Warehouse manages file size automatically.
Deletion-vector accumulation	DESCRIBE HISTORY metrics show deletion vectors being added or updated faster than compaction removes them, potentially increasing read overhead.	Keep auto compaction enabled. If deletion vectors accumulate without triggering small-file compaction, schedule OPTIMIZE. Use REORG TABLE ... APPLY (PURGE) only for explicit purge requirements.	No action. Cleanup is system-managed.
Poor file skipping	Selective predicates scan a large share of the table, or clustering-quality evaluation shows poor organization.	With Spark, configure liquid clustering or use Z-Order for an existing partitioned table.	Configure Warehouse data clustering.
Direct Lake transcoding overhead	Delta Analyzer shows excessive files, small row groups, or broad retranscoding after updates.	Compact small files, review row groups, and apply V-Order to Spark-written tables. Optionally, configure liquid clustering to improve compression quality within Parquet files.	Keep V-Order enabled and evaluate data clustering.
Unreferenced file storage growth	OneLake storage grows faster than active table size after data-changing operations.	Run VACUUM according to retention requirements.	No action. Cleanup is system-managed.

For mirrored data, follow the producer-specific remediation in Optimize mirrored data. Database mirroring is system-managed; for mirrored catalogs, apply supported maintenance in the source platform.

For lakehouse tables, Spark-supported inspection options include:

- Run DESCRIBE DETAIL to inspect file count, total size, and the evaluated `delta.targetFileSize.adaptive` property.
- Run DESCRIBE HISTORY to review write patterns and maintenance history.
- Use Delta Analyzer when you need detailed Direct Lake row group and update-pattern analysis.

Inspect average file size

Use DESCRIBE DETAIL to calculate the average file size as an initial indicator of the table layout:

Python

```
details = spark.sql("DESCRIBE DETAIL schema_name.table_name").first()

table_size_gb = details["sizeInBytes"] / (1024**3)
num_files = details["numFiles"]
avg_file_size_mb = (
    details["sizeInBytes"] / num_files / (1024**2)
    if num_files
    else 0
)

print(f"Table size: {table_size_gb:.2f} GB")
print(f"Number of files: {num_files}")
print(f"Average file size: {avg_file_size_mb:.2f} MB")
```

An average can hide skew between partitions or recent and previously compacted files. If the average indicates a possible layout issue, inspect the individual Parquet files or use Delta Analyzer to evaluate the distribution before changing maintenance settings.

When to create another table

Don't create another physical table solely because multiple Fabric engines consume the data.

Create another table when it has an independent purpose, such as:

- A transformation or aggregation that changes the data's grain or business meaning.
- Different security, retention, or data-quality requirements.
- A latency or refresh requirement that the shared table can't meet.
- A consumer-specific layout whose measured benefit outweighs its storage, processing, lineage, and governance costs.

Related content

- Tune the size of Delta table data files
- Compacting Delta tables
- Deletion vectors for Delta tables
- Apply liquid clustering on Delta tables
- Partitioning for Delta tables
- Optimize Delta Lake tables with V-Order
- SQL analytics endpoint performance considerations
- Understand Direct Lake query performance
- Performance guidelines in Fabric Data Warehouse

- Data clustering in Fabric Data Warehouse
- What is Mirroring in Fabric?

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 08/26/2026

Fabric CI/CD concepts and best practices

In Microsoft Fabric (Fabric), continuous integration and continuous deployment (CI/CD) refers to the development and release practices you use to version Fabric workspace items, move changes through environments, and keep deployed Fabric solutions consistent across workspaces. Because a Fabric solution can include many item types, data sources, and environment-specific settings, a CI/CD plan helps you and your team collaborate without overwriting each other's work or relying on manual reconfiguration during deployment.

This article explains the Fabric CI/CD concepts and best practices that help you plan Git integration, branched workspaces, variable libraries, deployment pipelines, automation tools, and the `fabric-cicd` library. Not every recommendation applies to every project; use the guidance to choose practices that match your solution's scale, governance requirements, and release process.

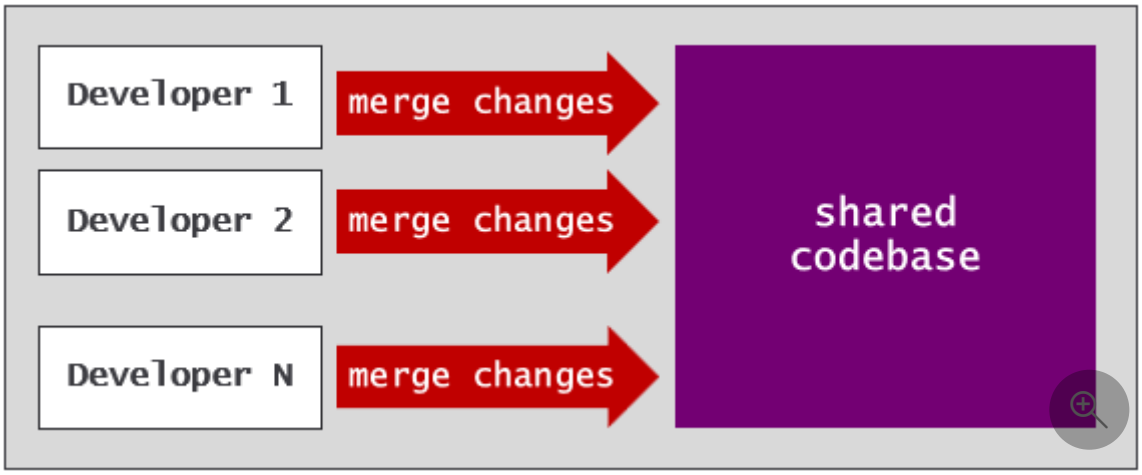
What CI/CD concepts should you know for Fabric?

Before learning about Fabric's built-in CI/CD capabilities, you should understand several background topics. This section reviews the core concepts and terminology of traditional CI/CD, and explains the difference between a Fabric *solution* and a Fabric *CI/CD project*.

Traditional CI/CD fundamentals

CI/CD represents a set of core principles and best practices that the software industry widely adopts. Its goal is to coordinate and automate the software development lifecycle. CI/CD focuses on two aspects of managing the application lifecycle: continuous integration (CI) and continuous deployment (CD).

Continuous integration (CI) is the practice of merging code changes into a shared repository regularly. Each merge can trigger automated workflows that validate the complete application by running tests. CI detects bugs early, which keeps the codebase in a working state and ready to deploy to production. It also lets you and your team work on the same application simultaneously by structuring how each person merges work into the shared codebase.



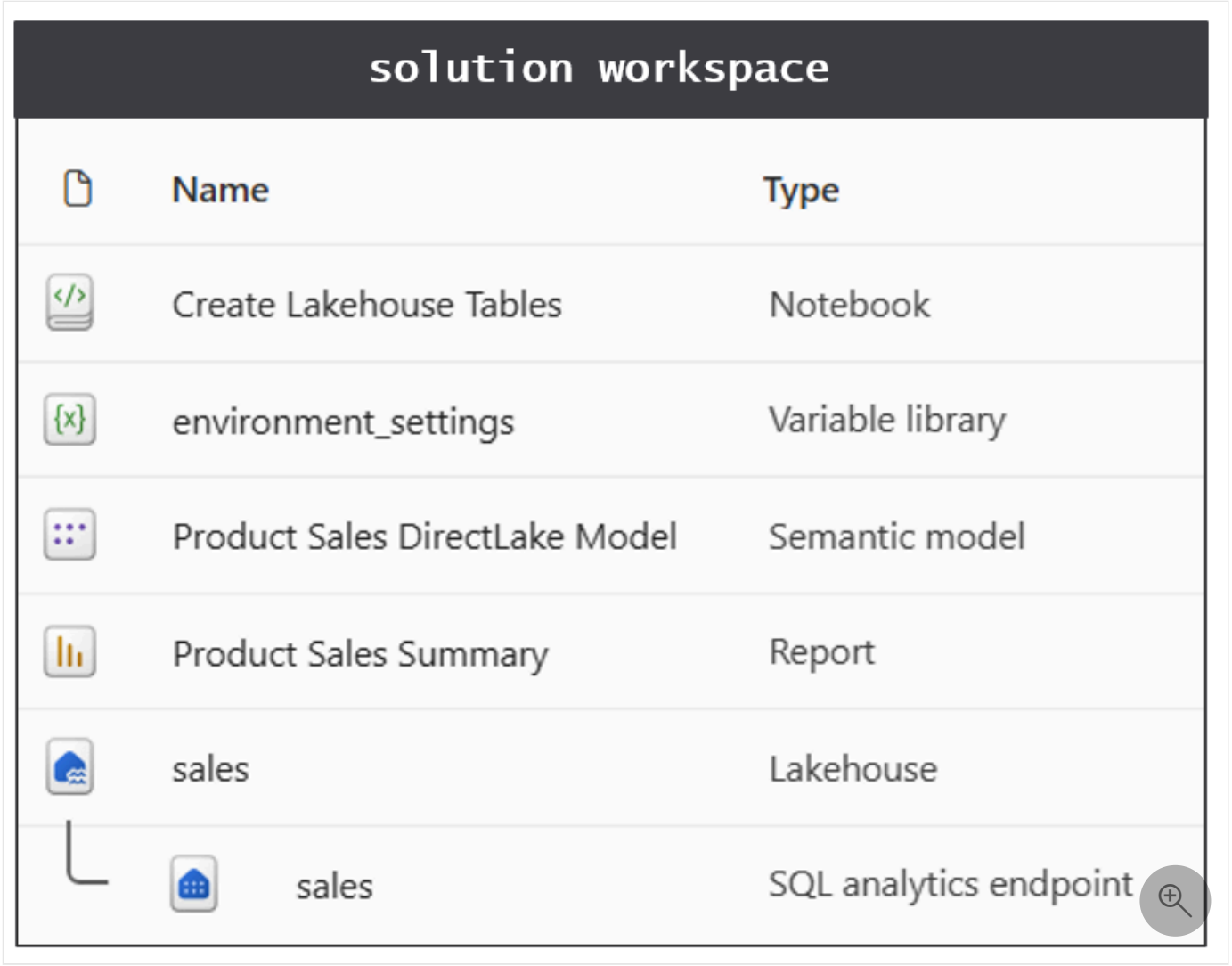
Continuous deployment (CD) is the practice of automating the deployment of code. A manual-approval process often precedes deployment, and one or more approvers must approve a release. Once approvers approve a release, a continuous-deployment process automatically deploys those changes to a target environment.

With continuous deployment, you can build automated processes that route the lifecycle of application code through a sequence of environments. Promoting code changes through environments enables enhanced testing and manual-approval gates so only high-quality code reaches production. The release process in a CI/CD lifecycle often promotes changes through a standard set of environments, such as **dev**, **test**, and **prod**.

Fabric solutions

You deliver value to users on the Fabric platform by building *Fabric solutions*. A Fabric solution is a composition of workspace items, such as variable libraries, lakehouses, notebooks, pipelines, semantic models, and reports. The Fabric platform currently offers more than 35 distinct workspace item types, and that number continues to grow.

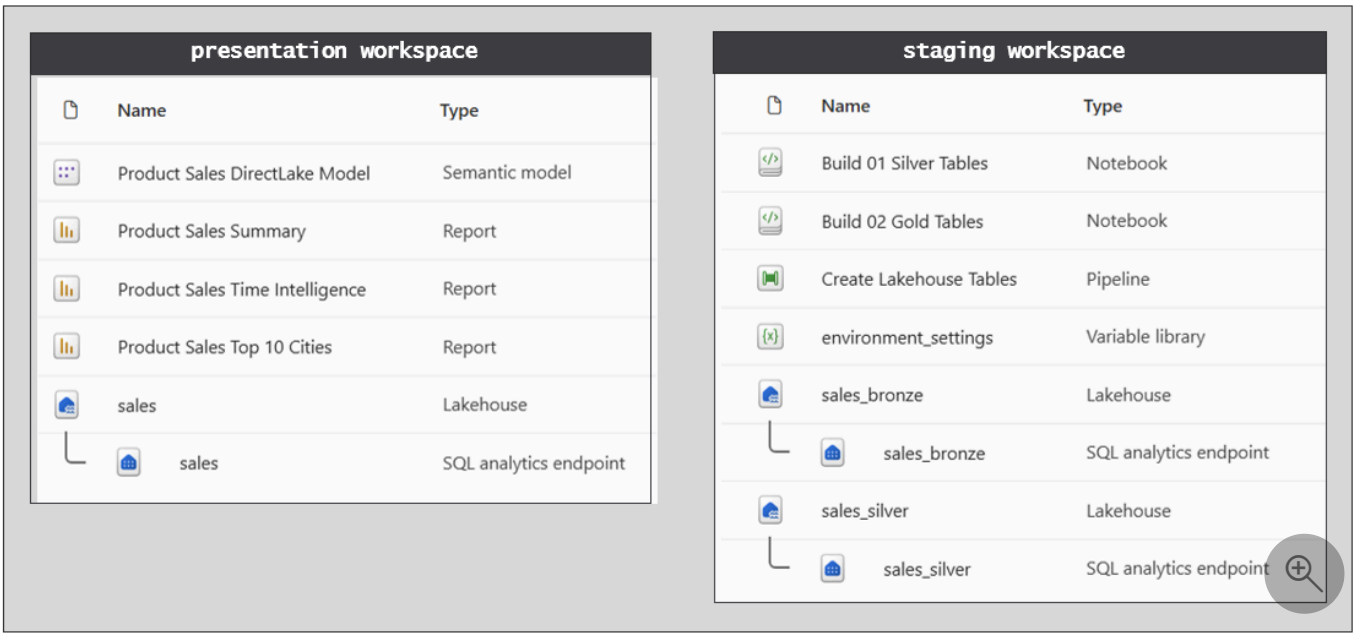
One important decision you must make when designing a Fabric solution is how many workspaces it requires to run. In many scenarios, you can design a Fabric solution to run inside a single workspace. The following screenshot shows a simple example of a Fabric solution with five workspace items that run within a single workspace.



When building a Fabric solution with many items, you can partition items by using *workspace folders*. Workspace folders help organize items by role or by type. They also help declutter the top-level folder, which is the default view for business users.

There are scenarios where it's either impractical or impossible to deploy all the items for a Fabric solution to a single workspace. For example, Fabric imposes a limit of 1,000 items per workspace. If you build a solution with more than 1,000 items, you must design it to span two or more workspaces. There can also be design factors related to security, least privilege, and item ownership that lead you to a multi-workspace design.

Consider a Fabric solution that uses a medallion architecture with a security requirement preventing business users from accessing staging data in the bronze and silver lakehouses. You can't use workspace folders inside a single workspace to configure security. When you add a workspace role assignment, that assignment grants access to everything inside the workspace. The best way to enforce the security requirement is to split the solution into a **staging workspace** and a **presentation workspace**.

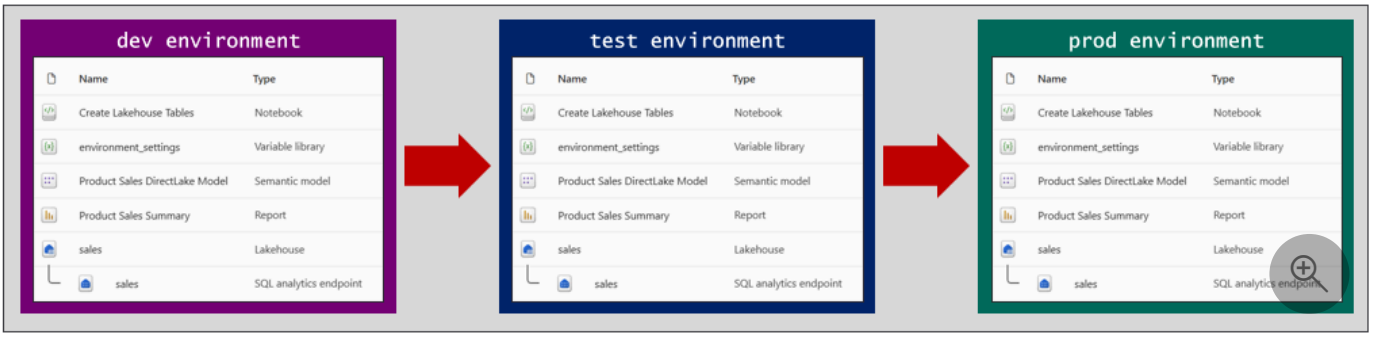


The staging workspace contains the items that run ETL jobs and store staging data. The presentation workspace contains items for business users. After deploying this solution, you configure access to each workspace independently. Business users who need access to reports, the semantic model, and the gold lakehouse get a role assignment on the presentation workspace, with no access to the staging workspace.

Fabric CI/CD projects

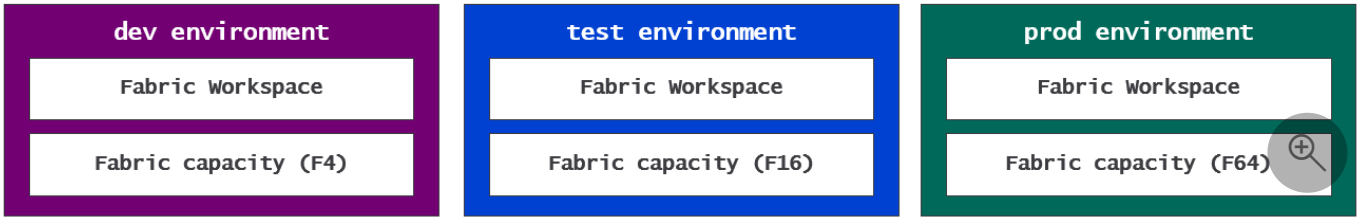
Imagine you've prototyped a Fabric solution by using the platform's analytics and AI capabilities, shared it with several peers for review, and received positive feedback. You then present it to the leadership team, who decide to prepare the solution for production so the team can release it to a large audience. That moment is when a Fabric CI/CD project begins.

A *Fabric CI/CD project* applies CI/CD concepts to a Fabric solution. While the industry widely uses the term *application lifecycle management (ALM)*, it can be helpful to consider Fabric CI/CD as *solution lifecycle management (SLM)*. You create multiple environments and run an instance of the Fabric solution in each one. You also build the CI/CD processes that propagate solution updates from one environment to the next.



One essential aspect of planning a Fabric CI/CD project is defining the requirements for each environment. An *environment* is a set of resources that gives you a place to deploy and run the Fabric solution. At a minimum, an environment for a Fabric solution requires a workspace that you assign to a Fabric capacity. If the solution requires two workspaces, then each environment needs two workspaces.

Another important consideration is whether each environment requires its own separate capacity. While you can assign workspaces from all environments to a single shared capacity, the recommended practice is to isolate environments by creating a separate Fabric capacity for each one. The Fabric CI/CD project examples in this guidance use three environments named **dev**, **test**, and **prod**. Your project can use other common names, such as **qa**, **uat**, or **ppe**.



When planning environments, you might decide to include other types of Azure resources. For example, a Fabric solution might access data files from an Azure Storage account. In that case, plan to create a separate Storage account for each environment. A primary motivation for environments is the ability to run a Fabric solution against different data sources for development, testing, and production. During environment planning, map out all URLs and paths that items use to connect to external data sources so you can build a parameterization strategy that lets items in each environment connect to their environment-specific data sources.

What Fabric capabilities support CI/CD?

The Fabric platform provides the following capabilities to support CI/CD processes:

Expand table

Capability	What it supports in Fabric CI/CD
Git integration	Versions Fabric workspace item definitions in Azure DevOps, GitHub, or GitHub Enterprise.
Branched workspaces	Creates and manages feature branches and feature workspaces from the Fabric UI.
Variable libraries	Stores environment-specific configuration values outside item definitions.

Capability	What it supports in Fabric CI/CD
Workspace item types	Determines which Fabric items support Git, deployment pipelines, REST APIs, variables, and auto-binding.
Item definitions	Stores the source files and metadata that represent Fabric workspace items in Git.
Deployment pipelines	Promotes item changes through staged workspaces for testing and production release.

The rest of this section examines each of these capabilities, along with several closely related topics. The goal is to build your understanding of how these components work together before you move on to Fabric automation and workflow development.

Git synchronization

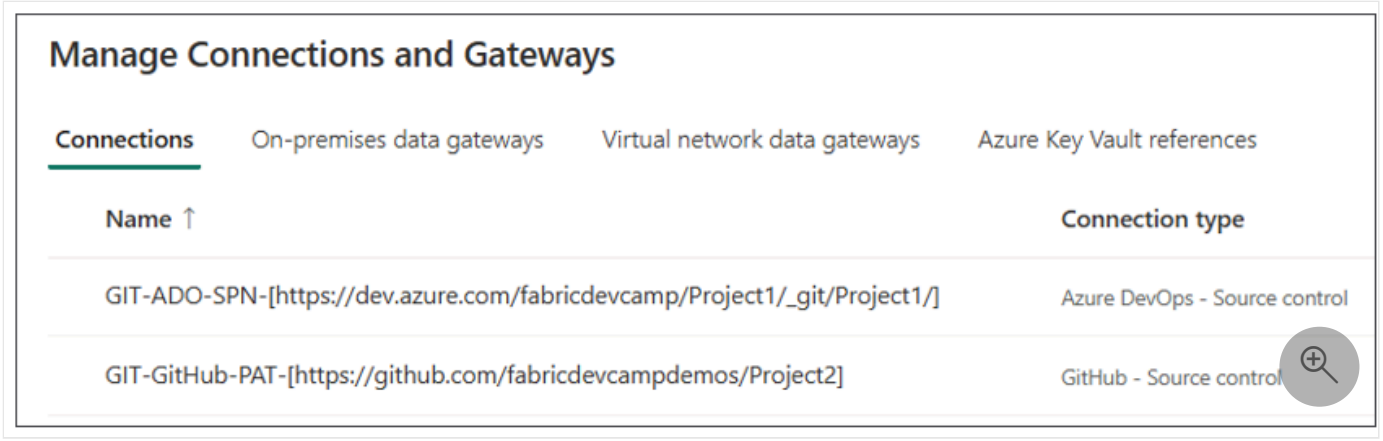
The Fabric CI/CD infrastructure starts with its support for Git integration. Imagine you're on a team building a data analytics solution in a Fabric workspace that includes a lakehouse, notebook, semantic model, and report. Fabric lets you maintain the source code for all four workspace items as a single source of truth in a Git repository. Fabric can version, track, and compare workspace item updates.

Fabric's supported Git providers include Azure DevOps, GitHub, and GitHub Enterprise. For the most current information about Git provider support and limitations, see Introduction to Git integration. For the typical first-time setup, see Get started with Git integration.

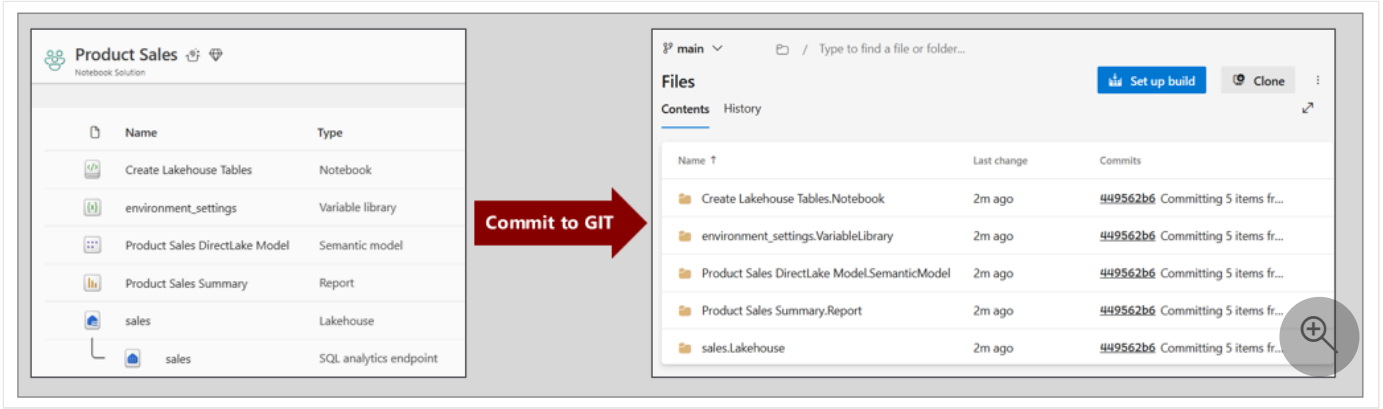
You enable Fabric's Git integration at workspace scope by connecting a workspace to a branch in a Git repository:

1. Create a Fabric connection to the target Git repository.
2. Configure the workspace-level Git setting that connects the Fabric workspace to a specific branch.

Fabric provides the **Azure DevOps - Source control** connector for connecting to Azure DevOps repositories. When you create a connection by using this connector, you configure credentials by using a Microsoft Entra identity, which can be either a user principal or a service principal. For GitHub and GitHub Enterprise, Fabric provides the **GitHub - Source control** connector, which uses a personal access token (PAT) for credentials.

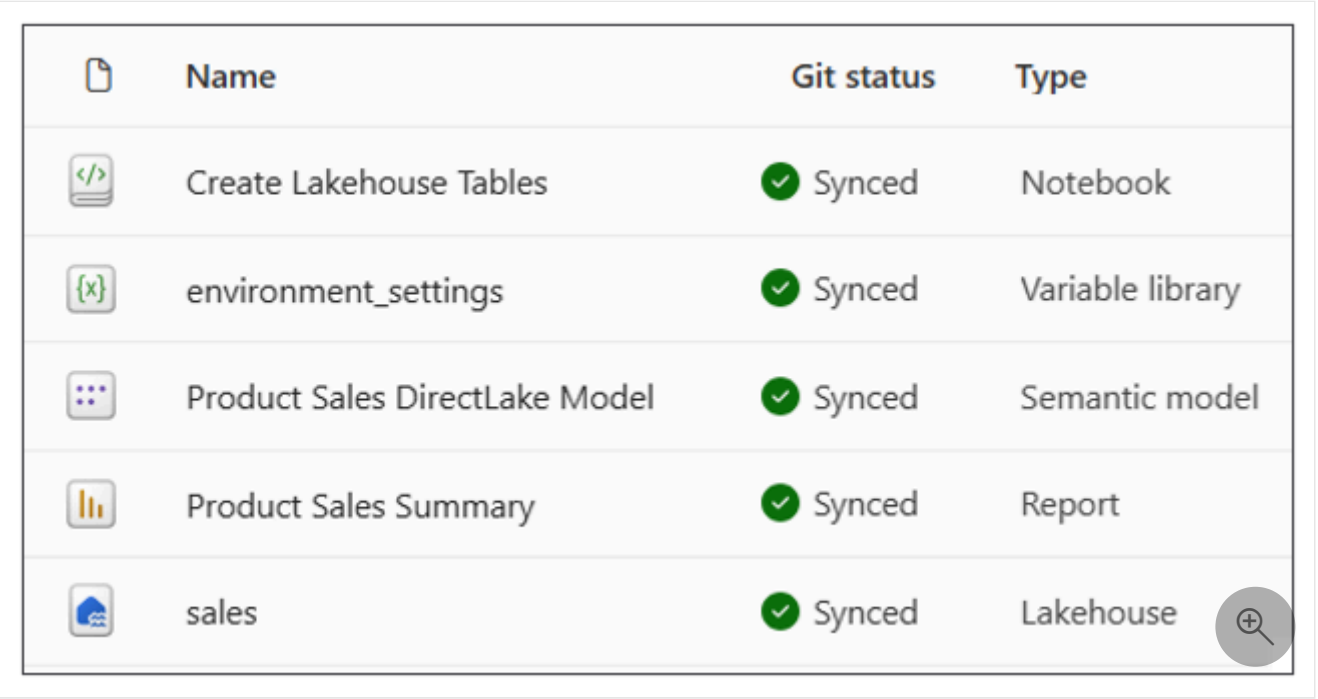


An essential aspect of Fabric Git integration is its ability to dynamically generate item definitions from workspace items. When you connect a workspace to a branch, Fabric runs a **Commit to Git** operation that generates an item definition for each item and persists it to the target branch as a folder containing item definition files.



Fabric uses a naming convention for item definition folders that combines the item display name, a period, and the item type in the format of `[Item Display Name].[Item Type]`. For example, you'll see folder names, such as `sales.Lakehouse` and `Product Sales Summary.Report`.

After you connect and synchronize a workspace with a Git branch, the workspace **List** view displays a **Git status** column that shows **Synced** to indicate all items are in sync with the underlying item definitions in Git.



Fabric Git integration also supports synchronization in the other direction. Consider a scenario where Git synchronization has synchronized all items in a workspace with a Git branch. A peer's pull request later merges new changes into that branch from another branch. You can run an **Update from Git** operation to synchronize those changes from the item definitions in Git back to the items in the workspace.

When you run **Update from Git**, Fabric checks whether a target workspace item already exists for each item definition by matching display name and item type. If Fabric finds a matching item, it updates the existing item to keep it in sync. If Fabric can't find a matching item, it creates a new item from the definition.

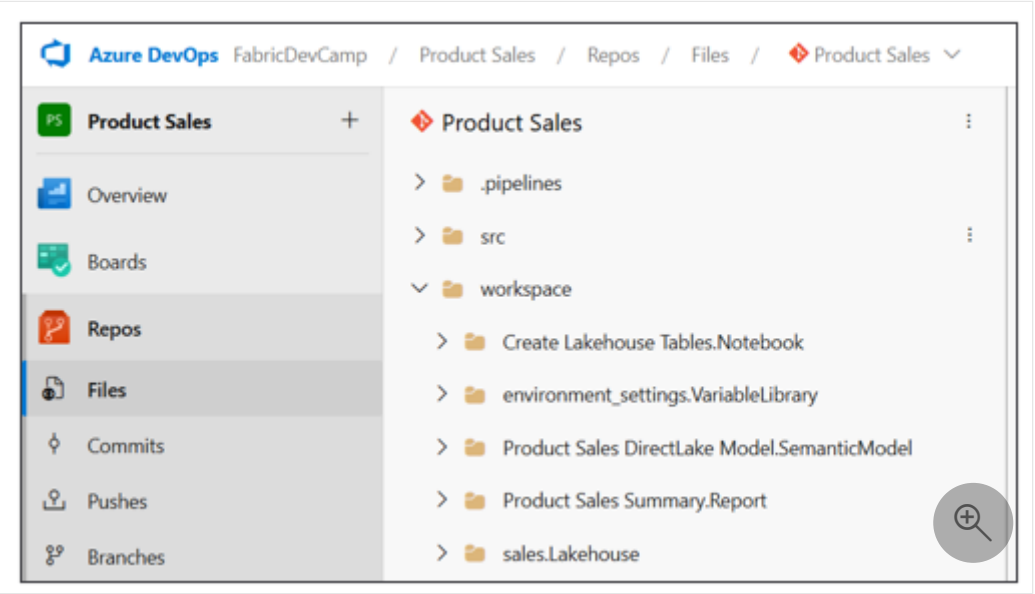
Consider what happens when you connect a Git branch containing item definitions to an empty workspace and run **Update from Git**. Fabric creates a new item from each definition in Git. This leads to an important observation: if you have a Git branch that contains the item definitions for a Fabric solution, you can use that branch to deploy the solution to a target workspace. This is the foundation of Git-based deployment patterns this article discusses later.

Git folder settings

When you use the Fabric UI to configure a connection between a workspace and a Git branch, you can specify an optional **Git folder** setting. If you leave **Git folder** at its blank default, Git synchronization creates the item definition folders in the root folder of the target branch. By specifying a value for the **Git folder** setting, you direct Git synchronization to write its output to a child folder in the Git branch instead.

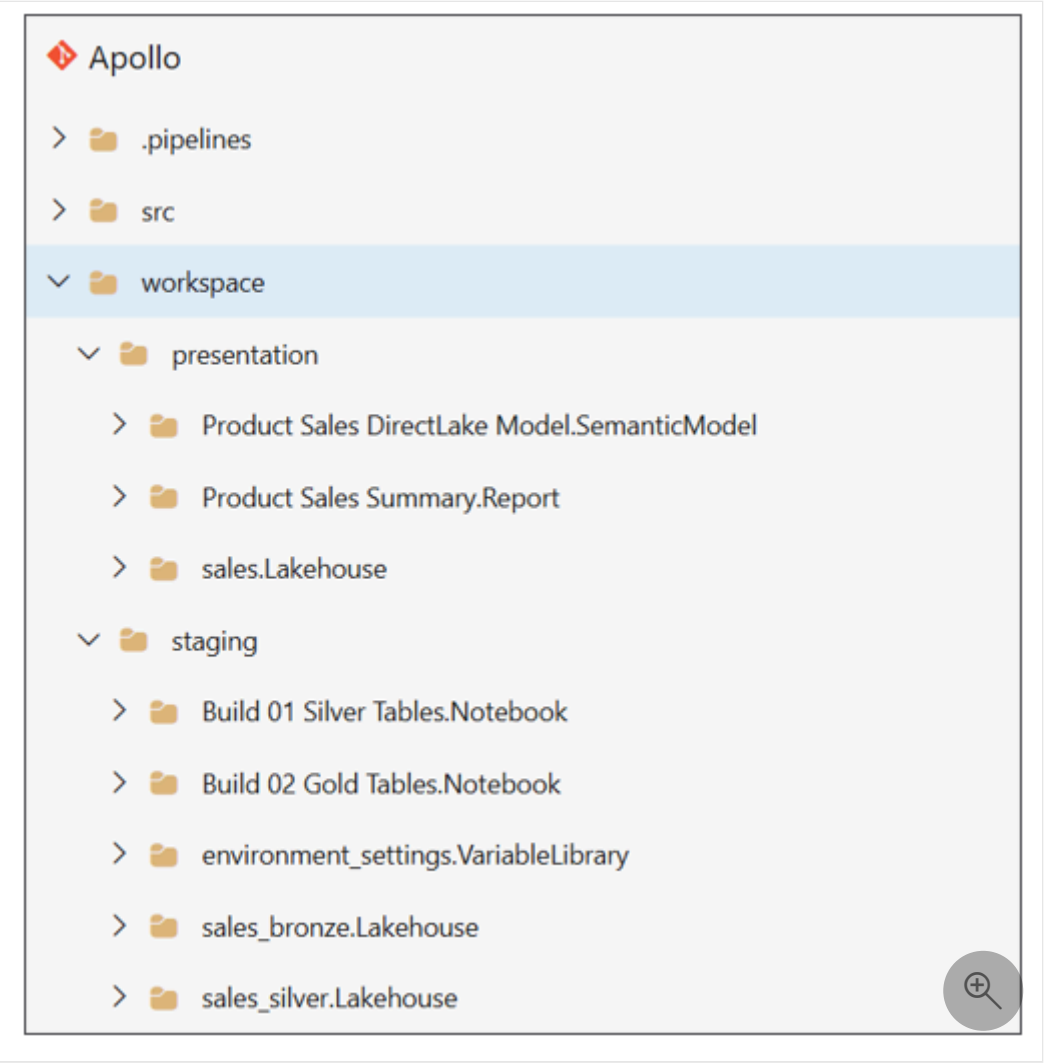
There are two important scenarios for configuring a **Git folder** setting.

Scenario 1: a Fabric CI/CD project that mixes workspace items with other project files. If your Git repository also needs to hold workflow files, such as YAML or Python scripts, configuring a Git folder setting (for example, `/workspace`) keeps the item definitions for workspace items separate from those other project files and avoids confusion. The Git - Connect REST API provides the `directoryName` parameter for the same purpose.



Scenario 2: a Fabric solution distributed across multiple workspaces. Revisit the medallion-architecture solution from earlier—a staging workspace with notebooks and lakehouses for bronze and silver data, and a presentation workspace with the gold lakehouse, a semantic model, and a report. While you *could* connect each workspace to its own Git repository, avoid that pattern: you need a single Git source of truth for the complete solution, even when the solution spans multiple workspaces.

The recommended practice is to synchronize both workspaces to the same Git branch in the same repository, with a different **Git folder** setting for each workspace. For example, connect the staging workspace with a **Git folder** setting of `/workspace/staging` and the presentation workspace with a **Git folder** setting of `/workspace/presentation`. This design is essential for multi-workspace solutions because it effectively creates a single Git source of truth for the entire solution.



Feature workspaces

Development teams widely adopt *feature branches* in modern software development. A development process based on feature branches lets you work and commit changes in isolation. You can configure feature branches with workflows that automatically run linters, validation checks, and security scans whenever you commit changes, improving the quality of code that reaches production.

After you commit changes to a feature branch, you typically create a pull request to merge those changes into the shared branch. Git providers, such as Azure DevOps and GitHub, let you configure pull requests with manual-approval processes that are as simple or as complex as your scenario requires. When the approval process completes, the Git provider automatically merges the changes into the *integration branch—the shared branch where you and your team merge changes together.* The integration branch is the single source of truth that's always ready for deployment to test and production environments. Remember that the *integration branch* is a concept, not a name. It typically has a name like **main** or **dev**.

If you're migrating to Fabric with experience in traditional CI/CD, some aspects of Fabric CI/CD will be familiar while other aspects won't. For example, you may already be familiar with creating feature branches and pull requests. However, Fabric CI/CD differs because you must match and connect each feature branch to a feature workspace.

At a high level, a development process for a Fabric CI/CD project that uses feature workspaces starts with a single source of truth in Git. This requires a Git repository and a decision about which branch will serve as the integration branch. You can then create the single source of truth by creating a set of item definitions in the integration branch.

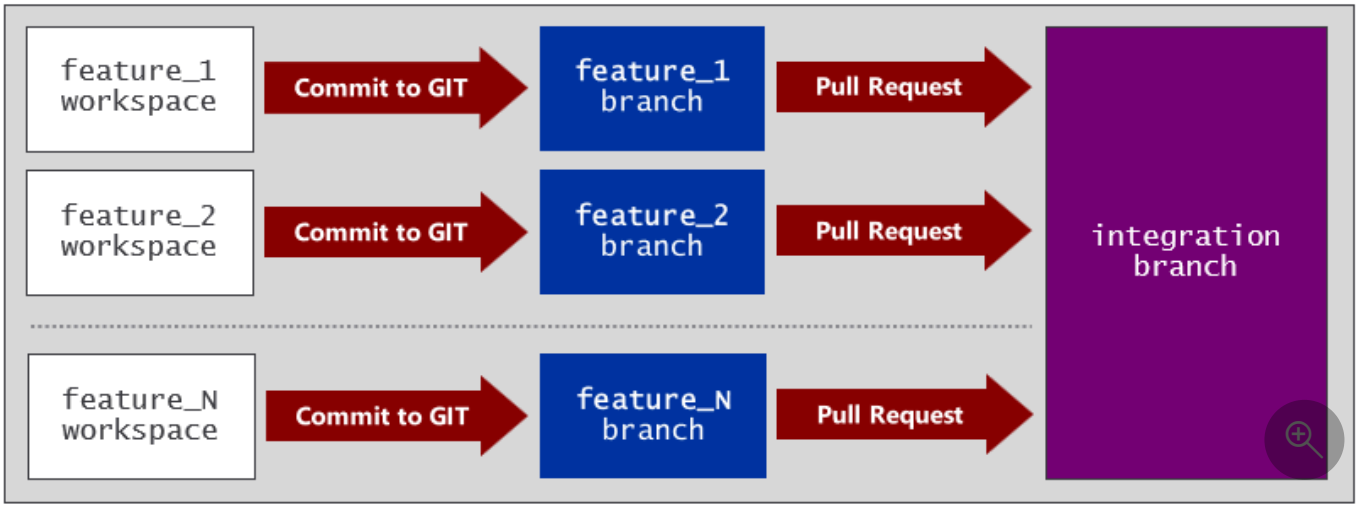
While there are several approaches you can use to create the initial set of item definitions in the integration branch, a common approach involves creating an operational version of the Fabric solution in a **dev** workspace. This makes it possible to connect the **dev** workspace to the integration branch and run a **Commit to Git** operation. The **Commit to Git** operation creates an item definition in the integration branch for each item in the **dev** workspace.

Treat the **Commit to Git** operation that creates item definitions in the integration branch as a one-time operation. After you create the item definitions that represent the single source of truth, don't directly update items or run **Commit to Git** operations in the **dev** workspace. Instead, merge future changes from feature branches into the integration branch. You can enforce this discipline by configuring the integration branch in the hosting Git repository with a branch policy that allows updates only through pull requests.

Once you configure the integration branch as a single source of truth, you can begin to create feature workspaces. This process typically involves creating a new feature branch based on the integration branch, creating a new workspace, connecting it to the feature branch, and running an **Update from Git** operation. This flow automatically populates the feature workspace with a matching set of workspace items.

When you initialize a feature workspace with an **Update from Git** operation, you often need to complete additional configuration before the workspace is ready for development. The How can you automate Fabric CI/CD? section discusses that configuration.

Once you standardize on a process for creating feature workspaces, you can begin to build what's required for continuous integration. The following diagram shows a development process that uses feature branches and feature workspaces. This approach can scale to accommodate a larger number of developers or development teams.



Remember that feature branches should be short-lived. A feature branch that remains active too long increases the risk of merge conflicts with the integration branch. It's a common practice to delete a feature branch as soon as a pull request completes and merges its changes into the integration branch.

When you manage the lifecycle of feature workspaces, choose one of these patterns:

Expand table

Feature workspace lifecycle pattern	Use when	Tradeoff
Create and delete with each feature branch	You want an isolated workspace for every pull request.	Highest isolation, but more workspace provisioning and cleanup.

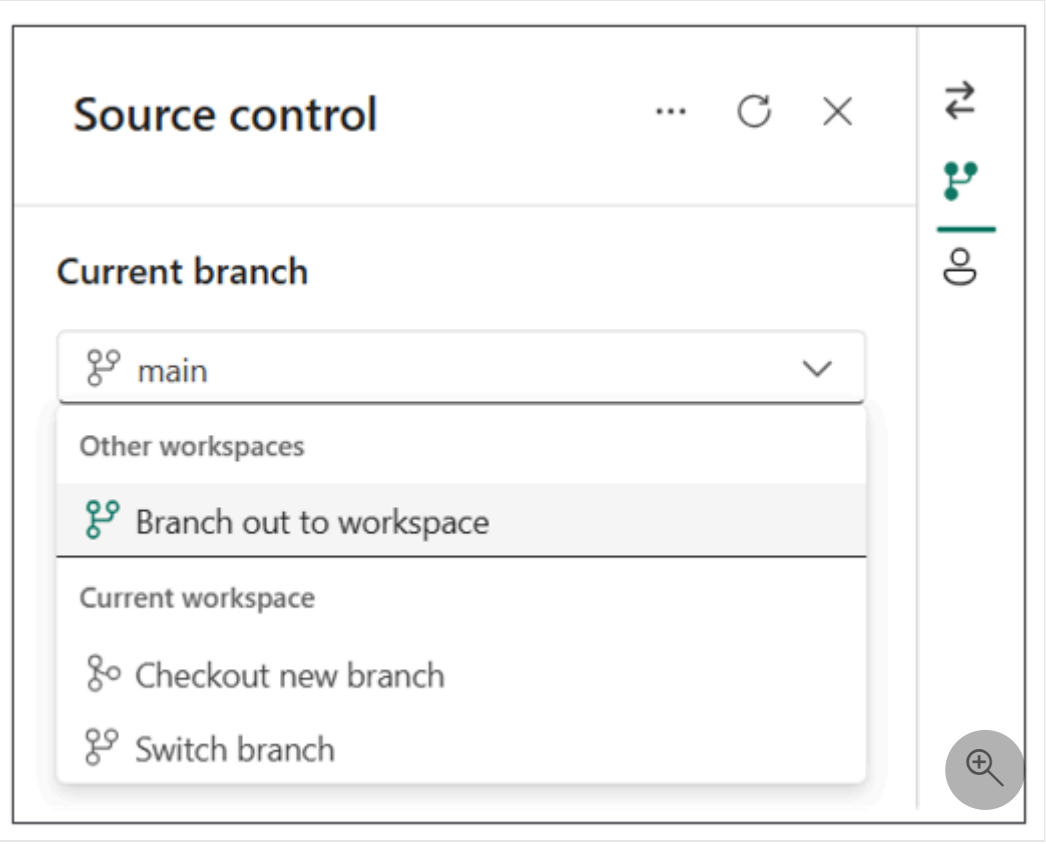
Feature workspace lifecycle pattern	Use when	Tradeoff
Recycle feature workspaces across branches	You have dedicated developers or teams that work repeatedly in the same area, such as Spark notebooks, semantic models, or reports.	Lower provisioning overhead, but you must keep reused workspaces synchronized and clean.

For day-to-day branch operations from inside Fabric, see Manage branches.

Branched workspaces

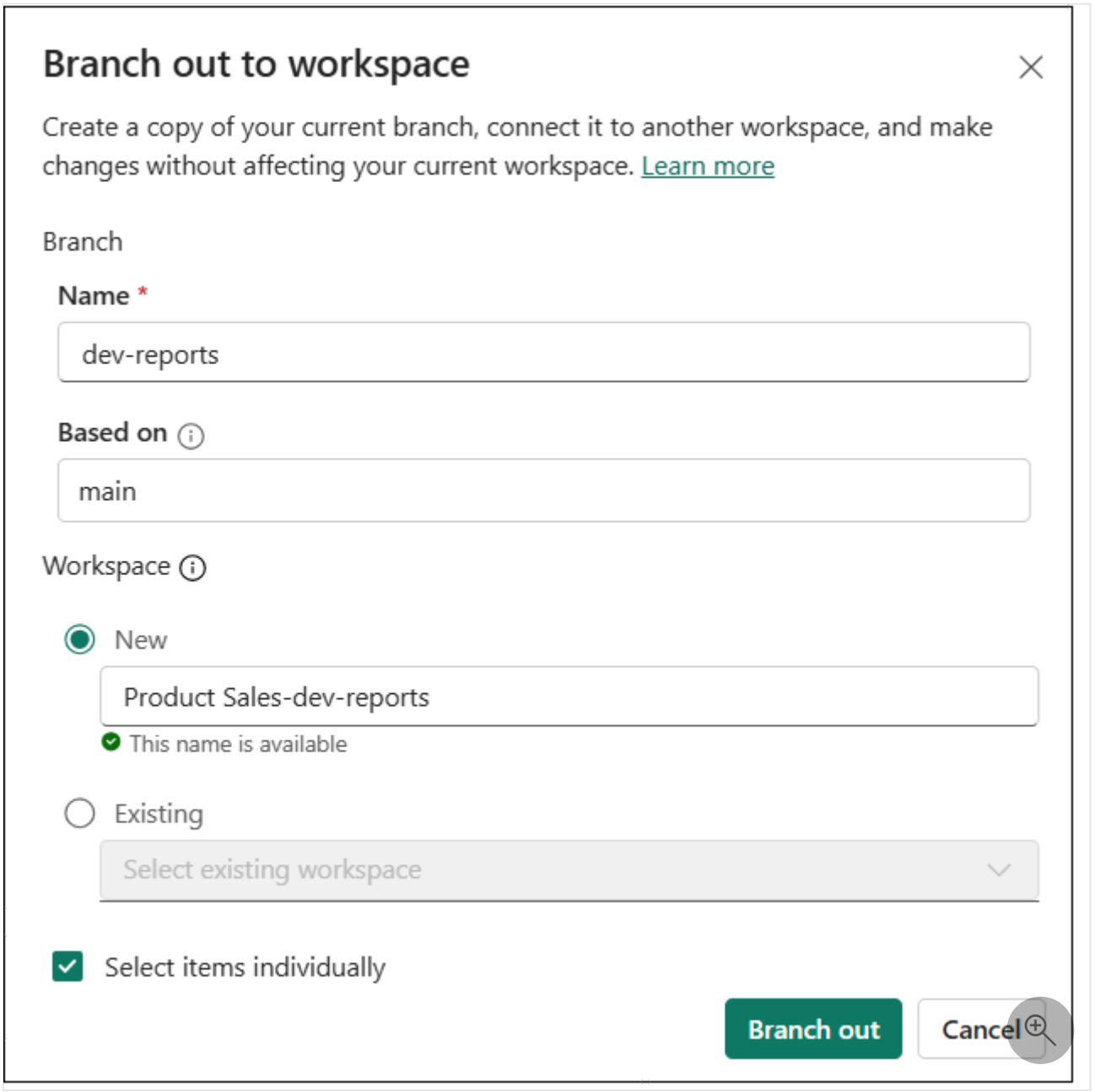
Fabric offers a capability known as *branched workspaces* to help create and manage feature workspaces. Branched workspaces simplify creating feature workspaces and switching between feature branches because Fabric automatically creates new Git branches and manages Git connections.

Consider a scenario in which you connect the **dev** workspace to an integration branch named **main** in a Git repository. If you navigate in the **dev** workspace to the **Branches** panel of the **Source control** pane, you can find a set of branching commands that include **Branch out to workspace**.



When you run the **Branch out to workspace** command, Fabric prompts you with a dialog to enter a new branch name. In the **Branch out to workspace** dialog, choose whether to create a new

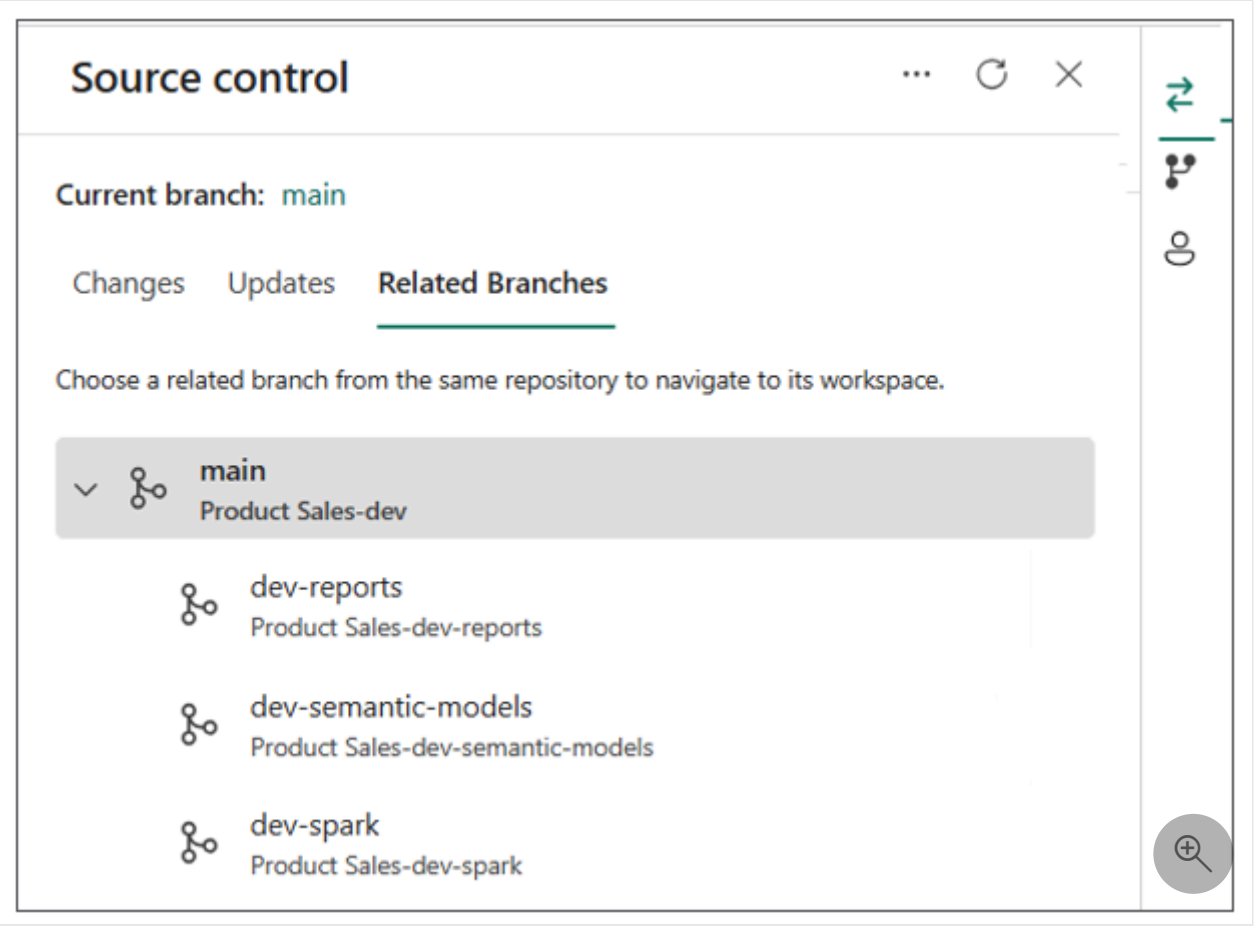
feature workspace or connect to an existing feature workspace.



The **Branch out to workspace** dialog provides an option to **Select items individually** to enable a capability known as selective branching. If you select this option and select the **Branch out** button, you're prompted with another dialog where you select which items to include in the branch-out process.

Selective branching is particularly useful for workspaces that contain many items. A full branch-out process can require significant time when the **dev** workspace contains hundreds of items. Selective branching lets you complete the branch-out process faster when you only need to work on a small subset of items. After the initial branch out, you can also incrementally add other items to the branched workspace as needed.

When you branch out to a feature workspace, Fabric creates a relationship between the branched workspace and the shared **dev** workspace that connects to the integration branch. This makes it easier to visualize the high-level development process for a Fabric CI/CD project. If you navigate the **Related Branches** view in the **Source control** pane, you can see that the top navigation node in the hierarchy displays the integration branch named **main** and the name of the shared **dev** workspace. Each child node displays the name of each feature workspace along with its underlying Git branch name.



Within each branched workspace, Fabric also provides a workspace breadcrumb menu that shows the relationship between a feature workspace and the shared **dev** workspace connected to the integration branch.

An important consideration when working with the **Branch out to workspace** command involves the permissions required in the Git repository and in Fabric. To branch out to a new workspace, you must have permissions to create new branches in the Git repository. You also need permissions in Fabric to create new workspaces and to assign workspaces to a Fabric capacity. For more information about permissions, see Required Git permissions for popular actions.

Microsoft is enhancing branched workspace capabilities to address permissions issues. Fabric plans to introduce a delegated model to enable self-service branch-out operations for users who don't have permissions to create workspaces or assign a workspace to a Fabric capacity.

In addition to the **Branch out to workspace** command, the **Source control** pane includes other commands for managing branched workspaces and their connections to feature branches. The **Switch branch** command lets you switch the connection for the current workspace from one branch to another in the same Git repository. Before switching to another branch, you must either commit or discard any uncommitted changes in the workspace. When switching to a different branch, Fabric runs an **Update from Git** operation that can add, update, or delete workspace items in the current workspace.

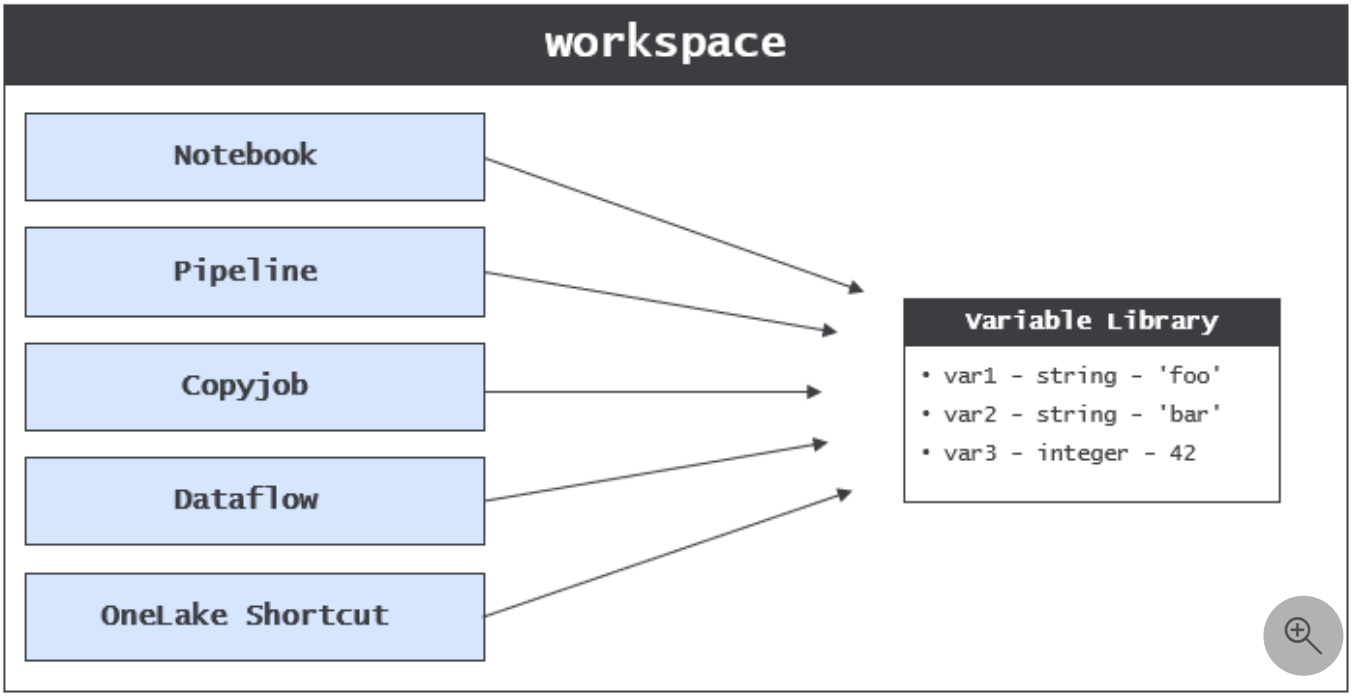
The **Check out new branch** command runs a set of operations that can help resolve merge conflicts between a feature branch and the integration branch. This command lets you switch from the current branch to a new branch without discarding changes in the current branch.

For more information about permissions for the branch-out experience, see Development process using branched workspace and Git integration process. To review changes before committing or updating, see Compare changes with granular compare.

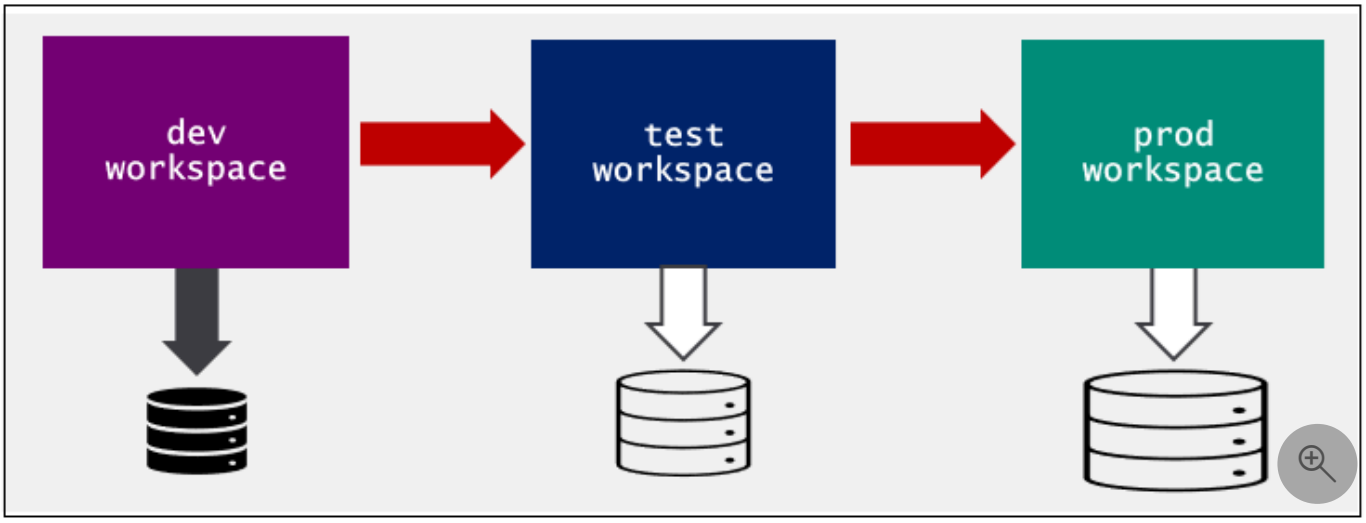
Variable libraries

An essential aspect of the CI/CD lifecycle involves promoting changes through a sequence of environments. The canonical example is a Fabric solution lifecycle that moves changes through environments, such as **dev**, **test**, and **prod**. When you need to test and validate code for a project across multiple environments, it's essential to define an effective parameterization strategy for environment-specific settings.

The Fabric platform provides the *variable library* to assist with parameterization. The variable library is a creatable type of workspace item in which you can define a set of variables. You can define other workspace items, such as notebooks, pipelines, copy jobs, and dataflows, to read variable values from a variable library. This makes it possible to avoid hardcoding configuration settings that change across environments.



Consider the classic scenario in which you need to build a release process that moves workspace item updates through a sequence of workspaces representing environments, such as **dev**, **test**, and **prod**. The problem is that you must configure items in different workspaces with different settings to connect to a different database or to some other type of data source. To solve this problem, items in each workspace require access to their own connection settings.

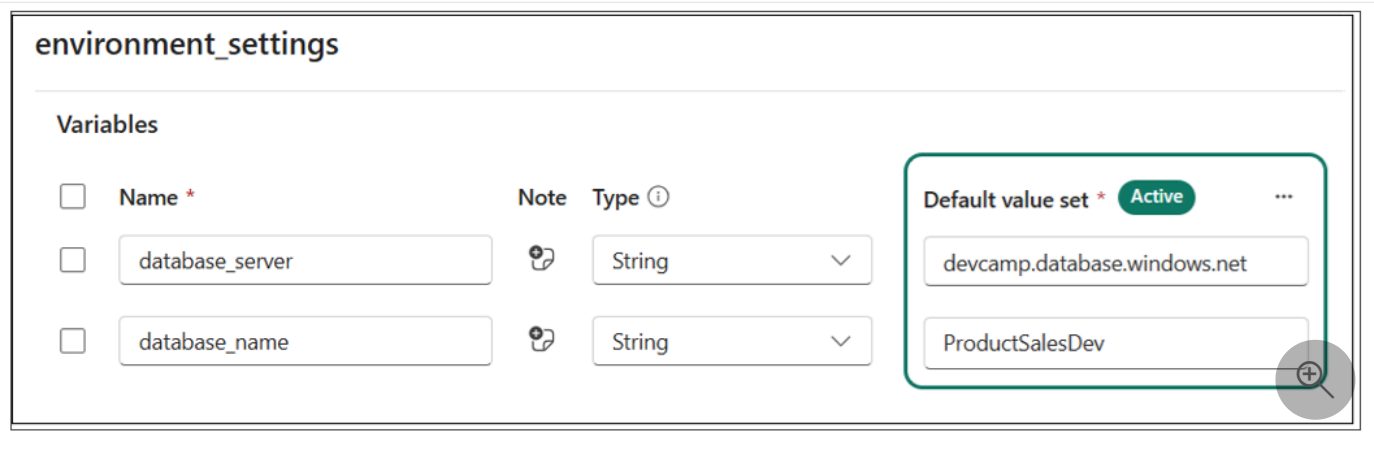


The variable library solves the problem of parameterizing these types of settings across workspaces. Use a variable library to avoid hardcoding the configuration settings required to connect to a database. Consider a scenario in which you must write Python code in a Fabric notebook to connect to a SQL database. Avoid hardcoding these configuration settings into your code as literal strings.

Python

```
database_server = '<server-name>.database.windows.net'
database_name = 'ProductSalesDev'
```

These hardcoded settings are specific to a single environment. As an alternative, you can use a variable library to support parameterization. In the variable library, define two string variables named **database_server** and **database_name**.



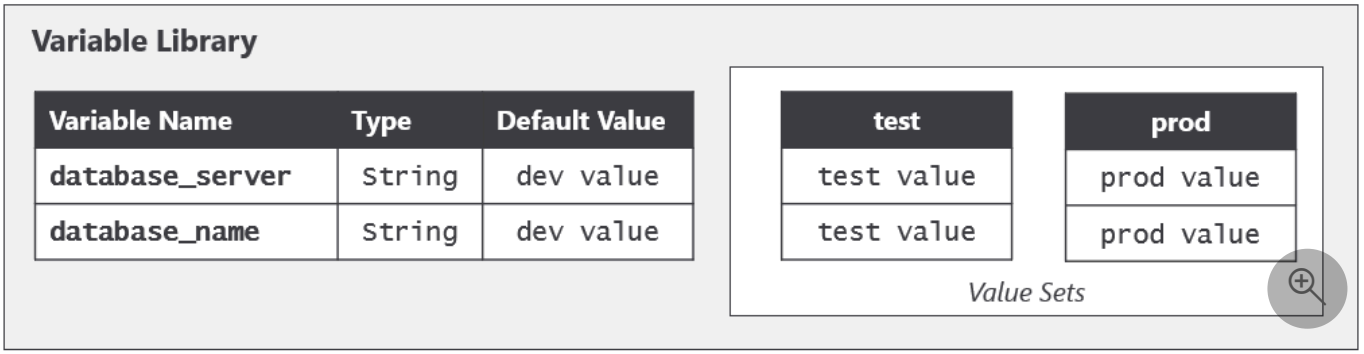
Once you have created the variable library with the two variables, you can remove the hardcoded configuration values from the notebook and replace them with code to read the variable values at run time.

Python

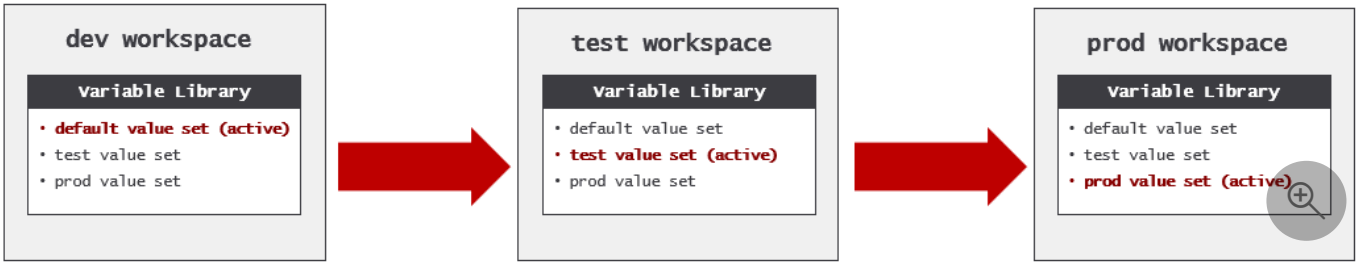
```
database_server =
notebookutils.variableLibrary.get("$(/**/environment_settings/database_server)")
database_name =
notebookutils.variableLibrary.get("$(/**/environment_settings/database_name)")
```

Variable Name	Type	Default Value
database_server	String	devcamp.database.windows.net
database_name	String	product-sales-dev

A *value set* provides alternate values for each variable in the variable library. Consider an example of a variable library in which you configured the default values for the **dev** environment. You can extend the variable library by adding a value set for the **test** workspace and a second value set for the **prod** workspace.



The item definition for a variable library includes variables and value sets. However, the item definition doesn't contain anything to indicate which value set is active. That's because Fabric maintains a separate workspace-level setting that tracks which value set is active in the context of that workspace. That means three different workspaces could each contain a variable library with an identical item definition. You can configure each workspace to use a different value set.



You can create variables in a variable library based on standard types, including **String**, **Number**, **Integer**, **DateTime**, and **GUID**. Variable libraries additionally support two important reference variable types: **Connection reference** and **Item reference**. Using these reference variable types makes it easier to edit variable values through the Fabric UI. When you edit the value of a reference-type variable, the Fabric UI displays an item picker showing a list of candidate items from which to choose.

Use *connection reference variables* to parameterize connections for ETL items, such as pipelines and shortcuts in a lakehouse. Design items that connect to external data sources by using a variable library that includes a connection reference variable and other variables that parameterize data-source paths. A connection reference variable removes environment-specific values from the item definition.

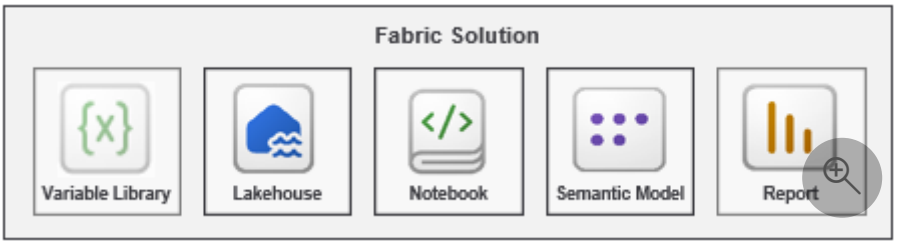
Item reference variables are useful for managing dependencies between items in the same solution. This is especially true when you need to manage an item dependency that spans across workspace boundaries. Revisit the scenario with the multi-workspace solution introduced earlier. A notebook in the staging workspace needs to write its output to a lakehouse in the presentation workspace with gold layer tables. You can use an item reference variable to parameterize the

target lakehouse. This removes environment-specific code from the notebook because the notebook can use the item reference to determine where to write its output.

For more on variable libraries in CI/CD, see What is a variable library?, CI/CD and variable libraries, Variable types, Value sets, and Variable library permissions. For details on the two reference variable types, see Connection reference variable type and Item reference variable type.

Workspace item types

Every Fabric solution is a composition of workspace items, and every item uses a *workspace item type* that defines its capabilities and the user experience it provides in the Fabric service.



The workspace item type abstraction is what allows artifacts from different Fabric workloads to behave in a similar and consistent manner. Each item type provides Git synchronization so Fabric can generate item definitions and store them in Git. Each item type responds to an **Update from Git** command by either creating or updating items from the definitions in Git.

Fabric is an evolving platform. Microsoft regularly updates workspace item types and continually adds new item types. As a result, some item types are more mature than others with respect to their CI/CD capabilities.

There are four primary CI/CD capabilities that some, but not all, workspace item types support:

Expand table

Workspace item type capability	Why it matters for Fabric CI/CD
Read variables from a variable library	Lets items use environment-specific values without hardcoded settings.
Create and update items through Fabric REST APIs using item definitions	Enables automated deployment through scripts, tools, and libraries.
Call Fabric REST APIs as a service principal	Supports unattended CI/CD jobs in Azure Pipelines, GitHub Actions, and other automation hosts.
Auto-bind workspace item dependencies	Reconnects related items after Git synchronization or deployment without manual rebinding.

Whenever possible, use variable libraries to manage parameterization across environments. When you encounter an item type that doesn't support variable libraries, you may need to fall back to direct edits of the item definition—either in Git or through a Fabric REST API call—to achieve the same goal.

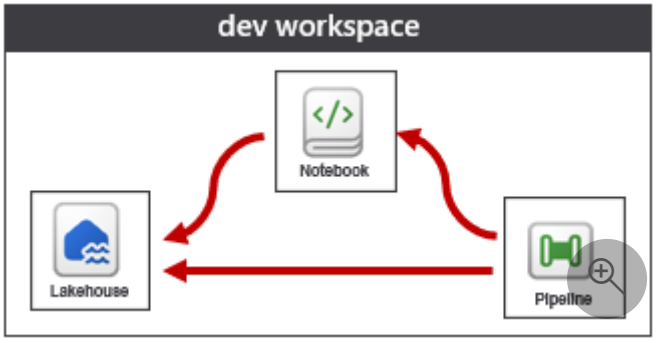
You'll also find differences across item types in their support for the Fabric REST APIs. Some item types support creating and updating items by using item definitions; others don't. Some item types don't yet support calls to the Fabric REST APIs under the identity of a service principal. This is especially common while an item type is in public preview.

For the current per-item-type CI/CD support matrix, see the supported items lists for Git integration supported items and deployment-pipeline supported items.

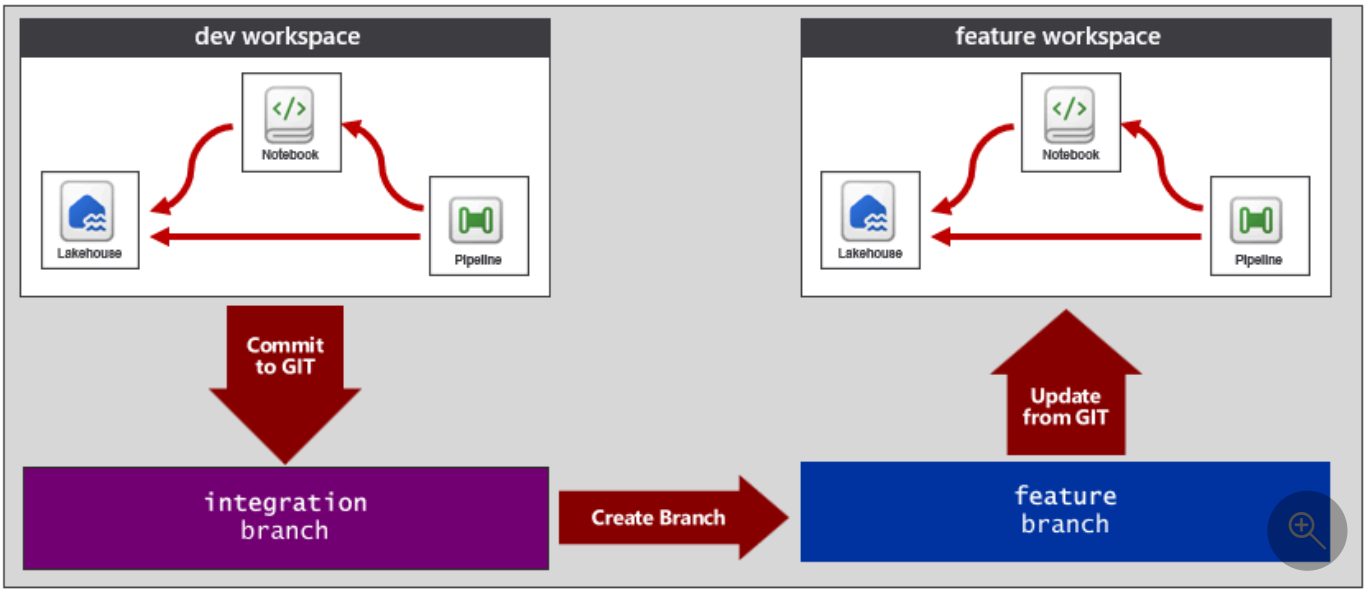
Auto-binding support

Most Fabric workspace item types can self-manage their relationships to other items in the same workspace. *Auto-binding* is the term for this behavior: as part of the Git synchronization process, item types that support auto-binding rebind their dependencies automatically. Item types that don't support auto-binding require your attention. You might need to run a script after an **Update from Git** operation to correctly rebind item dependencies.

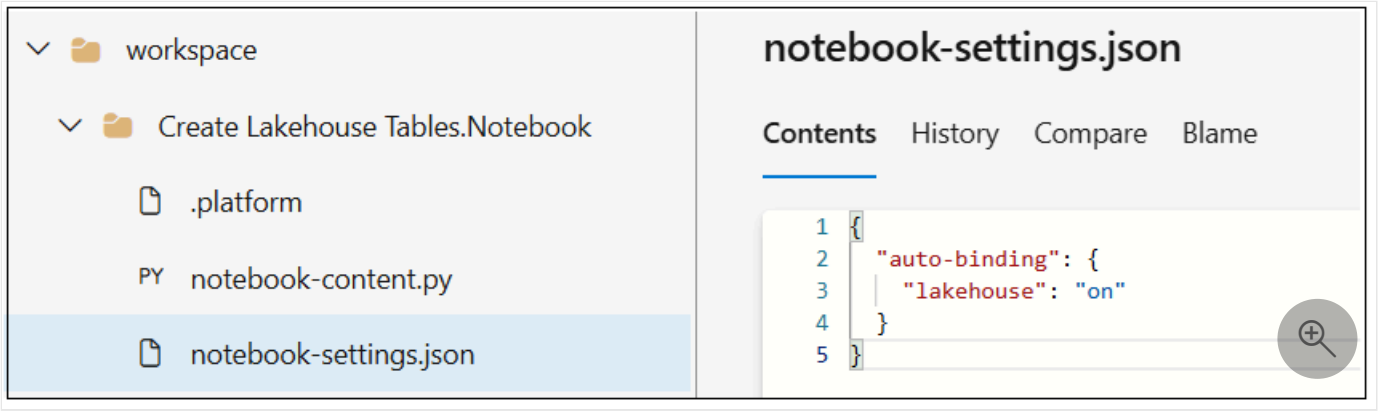
Consider a Fabric solution composed of a lakehouse, a notebook, and a pipeline, where the notebook depends on the lakehouse and the pipeline depends on both the lakehouse and the notebook. After you build the solution in the **dev** workspace, three dependencies exist between these items.



Now consider what happens when you use Git synchronization to replicate these three items from the **dev** workspace to a feature workspace. The outcome you want to avoid is workspace items in the feature workspace that retain dependencies pointing back to items in the **dev** workspace. Instead, you want each item to resolve its dependencies to the related items in the *same* workspace. With auto-binding, the notebook and pipeline in the feature workspace properly rebind to siblings in their new workspace.



Auto-binding maturity varies. Fabric notebooks didn't support auto-binding until Microsoft added auto-binding support to the Notebook item type in March 2026. Notebooks also don't support auto-binding by default; you must extend a notebook's item definition with a `notebook-settings.json` file.



If you build a Fabric solution with an item type that lacks auto-binding support, plan to write a post-sync script that reestablishes the item's relations. When auto-binding fails or item dependencies don't resolve as expected, see Dependency errors for diagnostics and remediation.

Fabric only supports auto-binding behavior between items that exist in the same workspace. When designing multi-workspace solutions, use item reference variables in a variable library to manage item dependencies that span workspace boundaries.

Item definitions

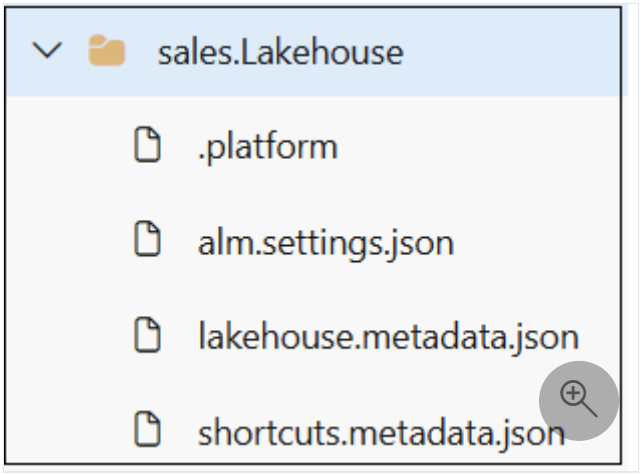
Item definitions are an essential component of Fabric's CI/CD support. An item definition is a portable, named folder containing a set of files with the settings and metadata that represent an item of a specific workspace item type. The files in an item definition vary significantly from one

item type to another. The idea that Fabric can serialize every item into a folder in Git is the foundation of the Fabric CI/CD infrastructure.

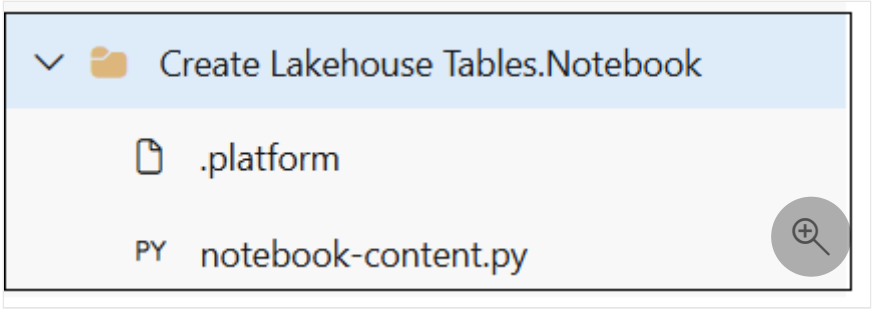
Every item definition requires a file named `.platform`—the *platform file*. The platform file contains metadata, such as the item type and display name. It also contains a `logicalId` value that Fabric uses internally to track logical instances of workspace items as they propagate across Git branches and workspaces.

```
{
  "$schema": "https://developer.microsoft.com/json-schemas/fabric/gitIntegration/platformProperties/2.0.0/schema.json",
  "metadata": {
    "type": "Lakehouse",
    "displayName": "sales"
  },
  "config": {
    "version": "2.0",
    "logicalId": "bfefcc84-2d46-aaa7-4d92-11a7e0385922"
  }
}
```

In addition to the platform file, each workspace item type defines its own schema for the files it requires and allows in an item definition. For example, the item definition for a lakehouse contains files, such as `alm.settings.json`, `lakehouse.metadata.json`, and `shortcuts.metadata.json`.

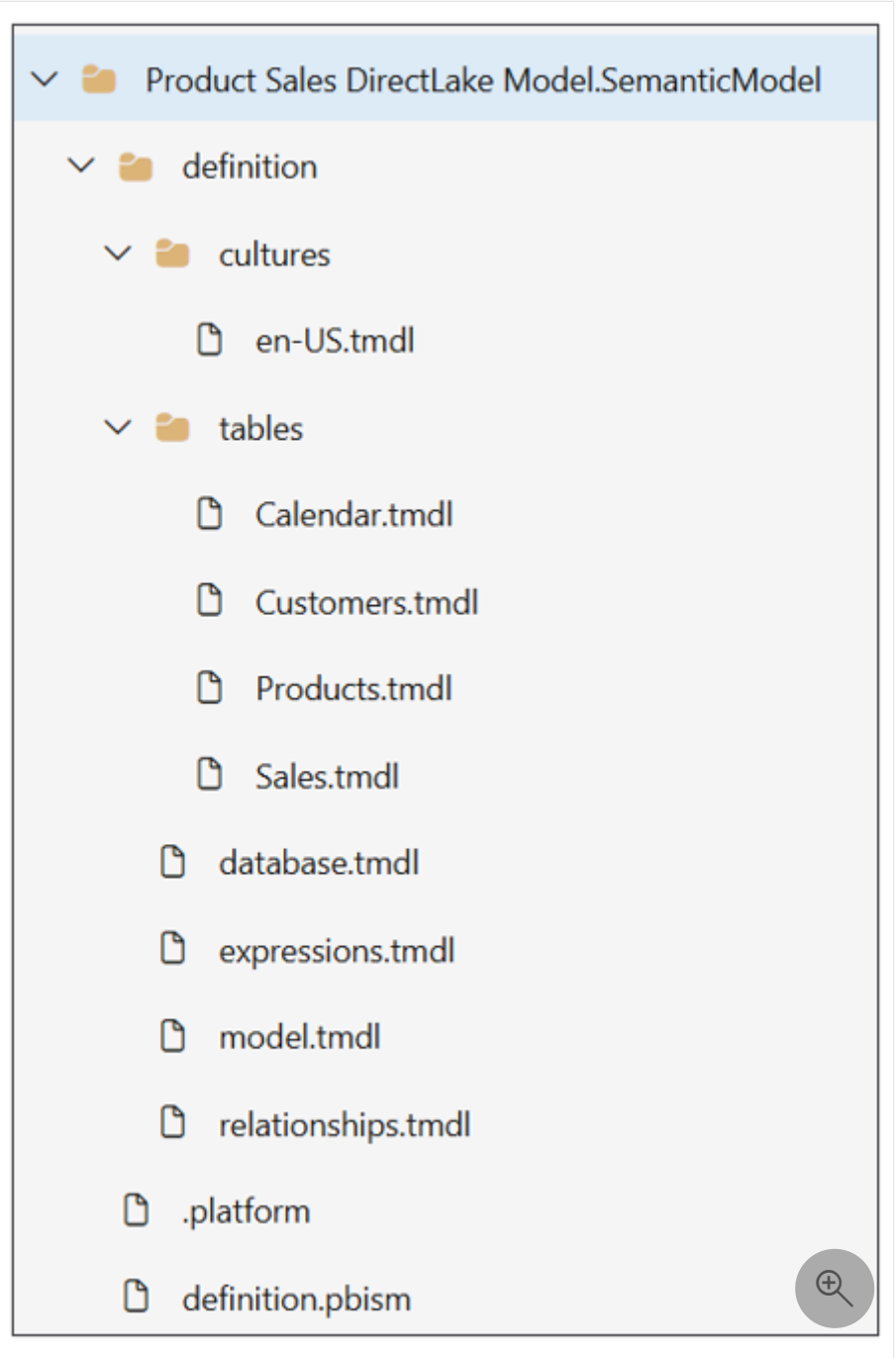


The minimal item definition for a notebook requires only a `notebook-content.py` file in addition to the platform file.



Fabric allows creating the `notebook-content` file with a `.ipynb` extension to support the Jupyter Notebook format. Fabric also supports creating the `notebook-content` file with extensions, such as `.sql`, `.scala`, and `.r`, to support other programming languages. As this article discussed earlier, the item definition for a notebook requires a `notebook-settings.json` file to add support for auto-binding.

Some item definitions contain a small number of files; others don't. The item definition for a semantic model can include dozens or even hundreds of TMDL files structured across several folders. While a large file count adds complexity, it enables much more granular collaboration: splitting each table into its own file lets one developer work on measures in the **Sales** table while another developer changes the **Calendar** table—essential when multiple developers are working on a large semantic model concurrently.



In this section, you've seen examples of item definitions for lakehouses, notebooks, and semantic models. You'll work with other workspace item types as well. For each, build a basic understanding of the item definition files. A good starting point is to review the item definition files in Git that Git synchronization created for an item type you're working with.

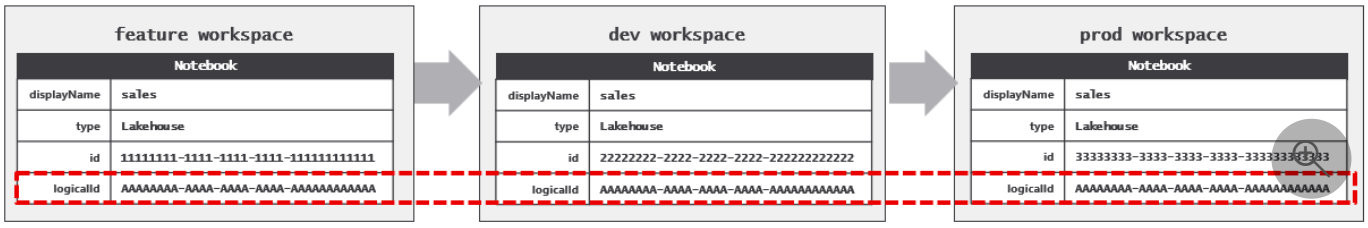
For the full item-definition file format and the source-code conventions Fabric uses, see Item source code format.

The role of the logicalId

Using Fabric Git integration, you can replicate a set of workspace items from one workspace to another. Imagine you created a lakehouse named **sales** and used Git synchronization to replicate

the lakehouse across three workspaces. All three lakehouse instances share the same type and display name, but Fabric assigns each a unique ID.

To support auto-binding, Fabric needs a way to track that these three lakehouse instances are all associated with a single logical item. That's where the `logicalId` comes in. While each lakehouse instance has its own unique ID, all three lakehouses share the same `logicalId`.



The `logicalId` matters because it allows Fabric to avoid persisting workspace-specific item IDs into Git. Using `logicalId` values instead of workspace-specific item IDs is what makes auto-binding possible. Examine the following snippet from the item definition for a pipeline that has a dependency on a lakehouse in the same workspace:

JSON

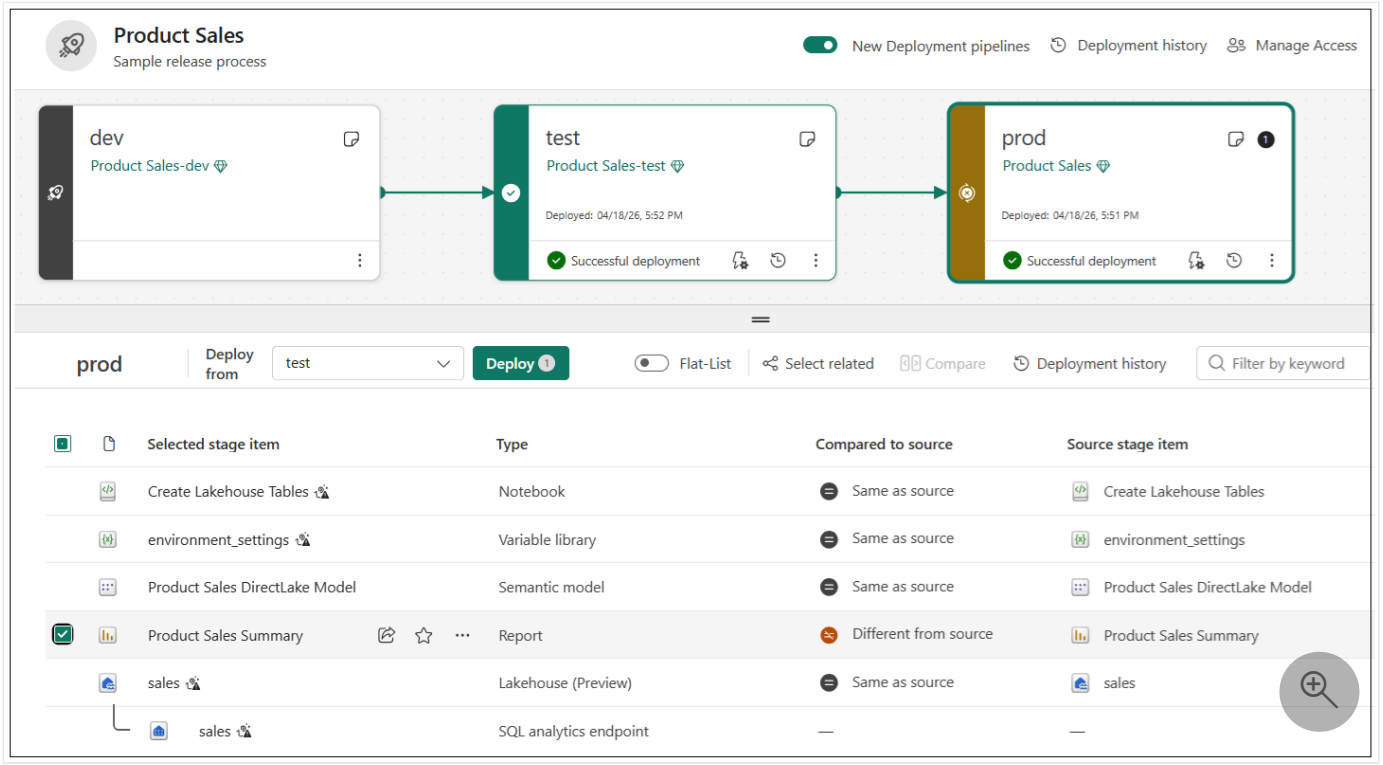
```
{
  "typeProperties": {
    "linkedService": {
      "name": "sales_lakehouse",
      "properties": {
        "typeProperties": {
          "workspaceId": "00000000-0000-0000-0000-000000000000",
          "artifactId": "<lakehouse-logicalId>",
          "rootFolder": "Files"
        }
      }
    }
  }
}
```

The `workspaceId` is the empty GUID `00000000-0000-0000-0000-000000000000`, which signals that `artifactId` is the `logicalId` of the lakehouse, not a workspace-specific item ID. When you run **Update from Git** on a new workspace, Fabric uses the `logicalId` to look up the workspace-specific lakehouse ID and rebind the pipeline to the lakehouse to reestablish the relation.

It's unlikely you'll ever need to work directly with the `logicalId`—treat it as an internal property Fabric uses. Never modify a `logicalId` value: Fabric doesn't support doing so, and changing it can lead to unpredictable behavior. For more on logical IDs and the conflicts they can resolve, see Logical ID conflicts and Resolve conflicts.

Deployment pipelines

The Fabric platform provides *deployment pipelines* as a continuous-deployment mechanism to help manage the lifecycle of workspace items. The key concept behind deployment pipelines is that they can deploy changes to workspace items through a set of stages where you assign each stage to a Fabric workspace associated with an environment. The following screenshot shows the deployment-pipeline user interface that deploys item-level changes across workspaces.



Deployment pipelines reduce manual effort by hiding low-level details associated with propagating changes through a sequence of workspaces. Deployment pipelines are similar to Git synchronization in that they automatically rebind item relationships when deploying a Fabric solution with item dependencies between lakehouses, notebooks, and pipelines.

Microsoft originally introduced deployment pipelines in the Power BI service several years before it first released Fabric. In their initial release, deployment pipelines offered *deployment rules* as a mechanism for supporting parameterization. For example, you can configure a semantic model with a deployment rule to update its data-source location so the semantic model in each of the three workspaces connects to its own environment-specific database.

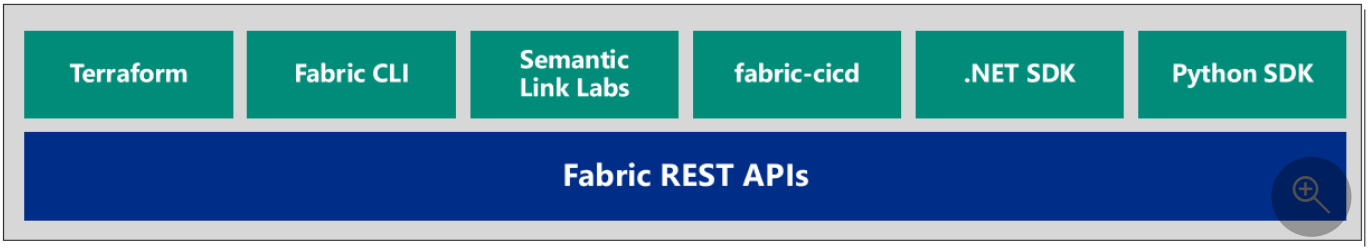
Deployment pipelines in Fabric continue to support deployment rules. However, variable libraries should usually be your first choice for parameterization because they're the strategic Fabric capability for environment-specific configuration.

Deployment pipelines assist with continuous deployment, which represents only the second half of the CI/CD lifecycle. Combine deployment pipelines with Fabric Git integration to implement an end-to-end solution.

How can you automate Fabric CI/CD?

Implementing a Fabric project with end-to-end CI/CD support requires a significant number of steps. Your ability to automate the setup process is essential. The alternative—known as *ClickOps*—involves completing setup tasks manually by using a browser. Relying on ClickOps is unreliable because it's prone to human error, causing unnecessary delays and the need to troubleshoot configuration errors. It's especially harmful when you're trying to create a repeatable plan for disaster recovery.

The Fabric platform provides public APIs that make it possible to automate setup and CI/CD processes for a Fabric project. You typically use the Fabric REST APIs to create workspaces, connections, and workspace items. While you can write code that calls the Fabric REST APIs directly, you can also reduce implementation effort by using developer tools and libraries that call the Fabric REST APIs on your behalf. The following diagram shows the developer tools and libraries that this article discusses.



This section starts by examining common scenarios where you need to write automation logic. Then it describes three developer tools and libraries: **Terraform** for infrastructure-as-code provisioning, the **Fabric CLI** for general-purpose automation logic, and **Semantic Link Labs** for in-Fabric scripting. The section then explains why the `fabric-cicd` library is a best practice for building a release process and concludes with a primer on Fabric REST API programming when you need the greatest level of control with Fabric automation.

Automation targets

The first part of implementing a Fabric CI/CD project is configuring the project's infrastructure. The infrastructure for a Fabric CI/CD project includes tenant-scoped items in Fabric, such as workspaces, capacities, connections, gateways, and deployment pipelines. You also need to

create a Git repository and configure Git integration support between Fabric workspaces and Git branches.

After a Fabric CI/CD project is operational, there's often an ongoing need to create and configure feature workspaces. The flow for creating a feature workspace involves creating a new feature branch from the integration branch and then running **Update from Git** against a new workspace.

The problem is that **Update from Git** doesn't complete all required configuration. It creates a lakehouse, but it doesn't automatically populate the lakehouse with tables. It creates a semantic model, but it doesn't automatically create and bind a connection to the semantic model's data source.

You often need additional automation after **Update from Git** initializes or updates a workspace. As you begin developing these types of scripts, consider two different types of jobs:

- **Post-deploy jobs** run only once after **Update from Git** first initializes an empty workspace.
- **Post-sync jobs** run each time after **Update from Git** completes on a workspace.

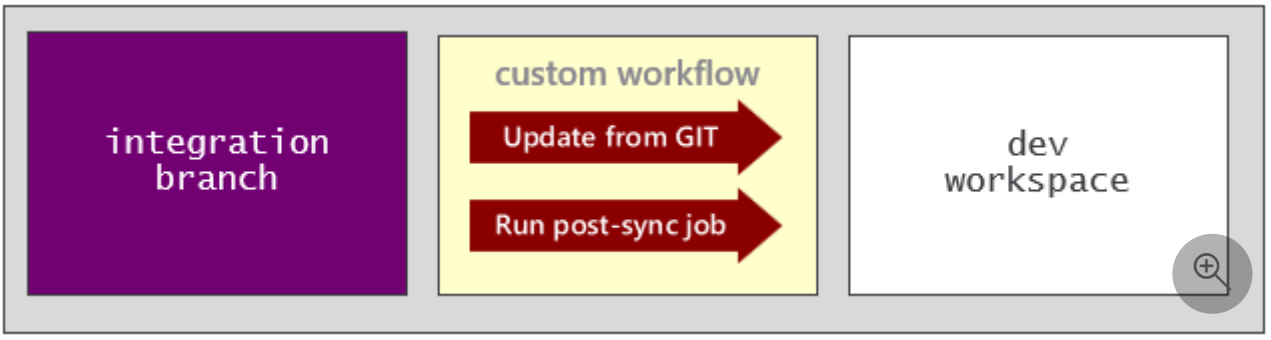
A *post-deploy job* is a set of one-time tasks that are part of the workspace initialization process. A post-deploy job commonly automates tasks such as setting the active value set for a variable library, running a notebook to populate a lakehouse with data, and creating and binding a connection for each semantic model. If your Fabric solution includes a semantic model that uses import mode, you should also automate the process of running a refresh operation to ingest and store the imported data.

Avoid putting ETL logic directly into the scripts you write for post-deploy jobs. Instead, factor out ETL logic into ETL-focused workspace items, such as notebooks, pipelines, copy jobs, user-defined functions, and dataflows. If a post-deploy script discovers a notebook by name and runs it, you can update the ETL logic in the notebook without requiring any updates to the script.

A *post-sync job* contains logic that must run every time after an **Update from Git** operation completes. Consider the common scenario in which you need to automate keeping the **dev** workspace in sync with the integration branch. The high-level flow of the development process begins with a developer making a series of changes to items in a feature workspace and

committing those changes to a feature branch. The developer then creates a pull request, and reviewers approve it to merge those changes into the integration branch.

When a pull request merges changes into the integration branch, you must find a way to synchronize those changes to items in the **dev** workspace. Automate this synchronization process by creating a workflow that runs whenever a pull request merges changes from a feature branch into the integration branch.



The workflow should include logic that begins by running an **Update from Git** operation to sync changes from the integration branch to the **dev** workspace. However, some workspace items might require additional modification after the **Update from Git** operation completes. This can happen when workspace items don't support auto-bind behavior. It can also happen when items don't yet support variable libraries but require environment-specific settings.

After a post-sync job finishes, verify the target workspace is in the expected state: Git status is **Synced**, the required variable library value set is active, and item dependencies point to related items in the same workspace.

> **Important**
>
> Regardless of which tool you use or whether you call Fabric APIs directly, you must consider the identity that runs your automation jobs. Every call to a Fabric REST API executes under the identity of a specific Microsoft Entra security principal. Fabric REST APIs support two types of identity: user principals and service principals.
>
> Execute API calls as a service principal whenever possible. This is especially important when CI/CD workflow automation logic executes in a cloud-based platform, such as Azure

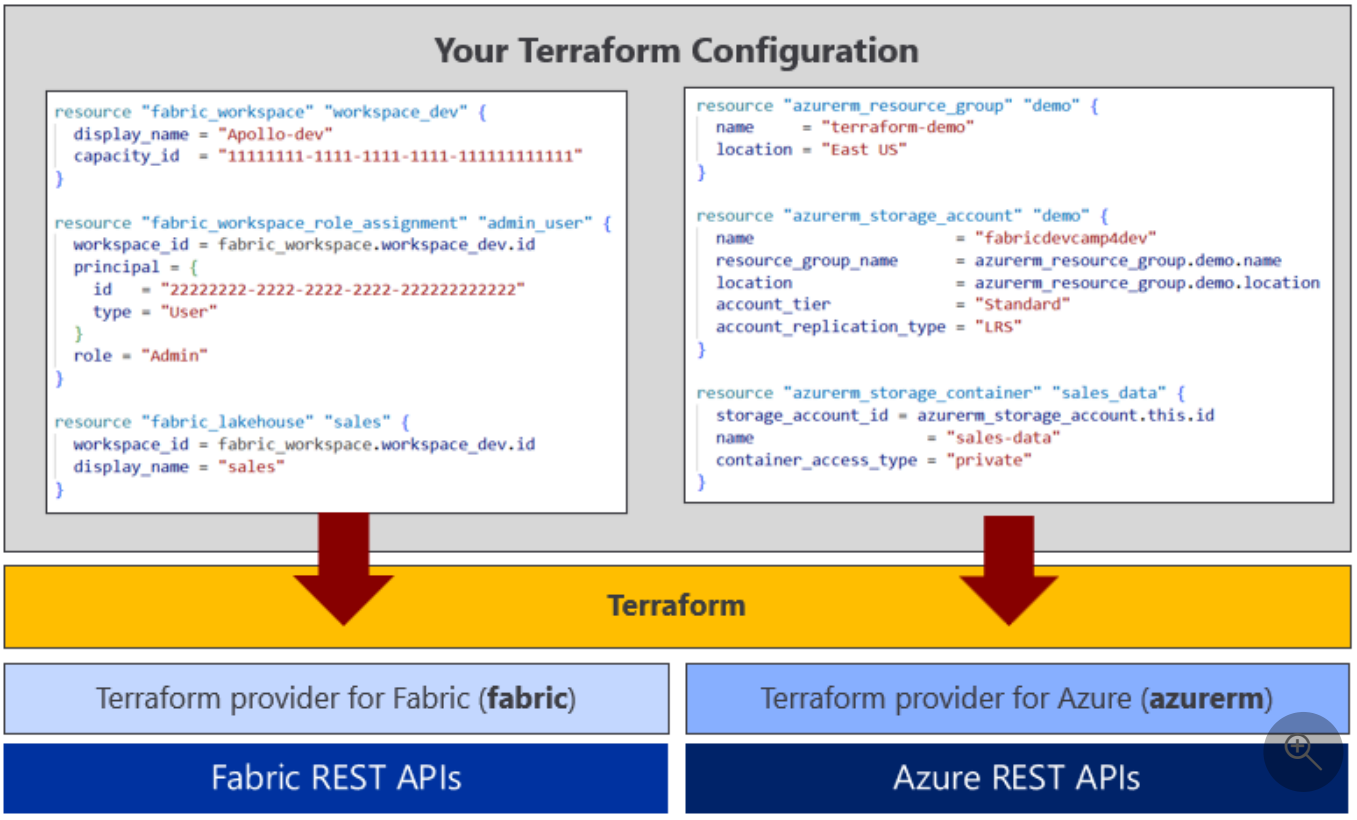
Pipelines or GitHub Actions.

Terraform for project infrastructure

Terraform is an open-source tool built on the principles of *infrastructure as code (IaC)*. IaC defines the infrastructure for a software project by using configuration files instead of manual processes or procedural programming logic. For organizations managing cloud deployments, the use of IaC is a well-established best practice in DevOps and CI/CD.

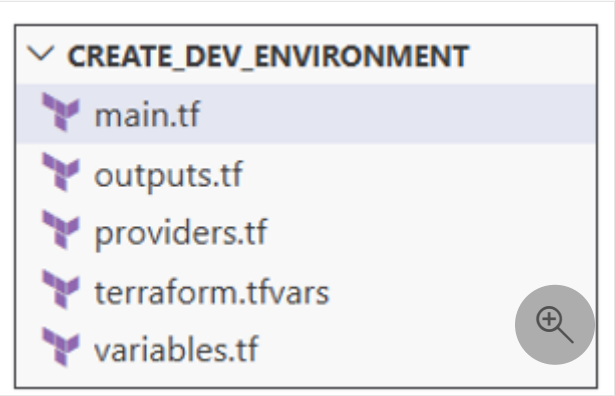
Terraform offers the advantage of a deployment process that is both versionable and repeatable. You can version a Terraform configuration by adding its files to a Git repository, as you would version any source file. A Terraform deployment process is repeatable: you can run it repeatedly and it produces the same outcome. A repeatable process provides consistency and the fastest path to disaster recovery.

Architecturally, Terraform plays the role of an orchestrator that completes its work indirectly by calling APIs through providers. Terraform doesn't call those APIs directly; instead, it delegates to *Terraform providers*, plug-in components designed to interact with a specific set of cloud-based APIs. The Terraform registry contains thousands of providers covering platforms, such as Azure, Amazon Web Services, and Google Cloud. Microsoft released the *Fabric provider for Terraform* in 2025, making it possible to set up the infrastructure for a Fabric CI/CD project by using an IaC-based provisioning process.



For more on Terraform, see the Terraform documentation on the HashiCorp Developer site.

You define a Terraform configuration by using a set of files that contain *HashiCorp Configuration Language (HCL)*. A typical project layout uses files, such as `providers.tf`, `variables.tf`, `main.tf`, `output.tf`, and `terraform.tfvars`. The `variables.tf` file defines a set of typed variables. The `terraform.tfvars` file assigns variable values when Terraform is running locally in a client tool, such as Visual Studio Code. Because `terraform.tfvars` often contains sensitive information, such as a client secret that authenticates as a service principal, add a `.gitignore` entry so that you never upload `terraform.tfvars` to a Git repository.

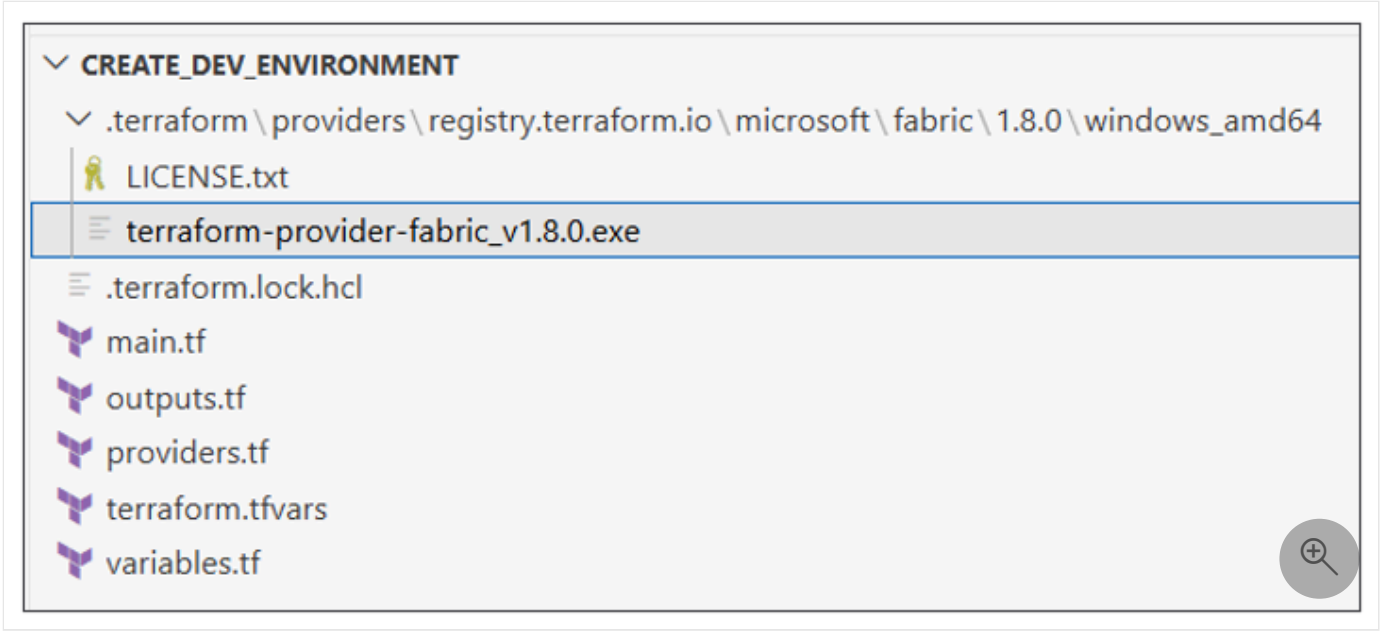


The `providers.tf` file contains a `terraform` block with `required_version` and `required_providers` arguments. The `required_providers` block specifies each provider with its source and version. The same file contains a `provider` block that authenticates the Fabric provider as a service principal:

Terraform

```
terraform {
  required_version = ">= 1.8, < 2.0"
  required_providers {
    fabric = {
      source  = "microsoft/fabric"
      version = "1.8.0"
    }
  }
}

provider "fabric" {
  tenant_id     = var.tenant_id
  client_id     = var.client_id
  client_secret = var.client_secret
}
```

Once the configuration contains what's required to initialize the provider, run terraform init to download the executable code for the provider along with security configuration that verifies no one has tampered with the downloaded provider code.



After initializing the Fabric provider, define resources by adding `resource` blocks to `main.tf`. Each `resource` block requires a provider-specific resource type and a local resource name. The Fabric provider offers the `fabric_workspace` resource type for creating workspaces and the `fabric_workspace_role_assignment` resource type for configuring access through workspace role assignments:

Terraform

```
resource "fabric_workspace" "workspace_main" {
  display_name = var.workspace_name
  capacity_id  = var.capacity_id
}

resource "fabric_workspace_role_assignment" "admin_group" {
  workspace_id = fabric_workspace.workspace_main.id
  principal = {
    id   = var.admin_group
    type = "Group"
  }
  role = "Admin"
}

resource "fabric_workspace_role_assignment" "dev_group" {
  workspace_id = fabric_workspace.workspace_main.id
  principal = {
    id   = var.dev_group
    type = "Group"
  }
  role = "Member"
}
```

When you create a `fabric_workspace` resource, specify argument values for `display_name` and `capacity_id`. The `fabric_workspace` resource is also assigned a local name (`workspace_main` in the

previous listing). The local name matters because it lets you create references by using syntax, such as `fabric_workspace.workspace_main.id`.

Once you have a configuration with a set of resources, terraform plan reviews the steps required to deploy the configuration without making changes. terraform apply executes the job that creates the resources needed to match your configuration.

The first time you run `terraform apply`, Terraform creates a state file named `terraform.tfstate` that tracks the property values for each resource as Terraform creates or updates it. By default, Terraform creates `terraform.tfstate` locally in the root folder of the current configuration. Using a local state file is convenient when you're first learning to use Terraform and when you're developing and testing a configuration in Visual Studio Code.

> **Important**
>
> In production, it's critical to initialize the configuration to manage the Terraform state file in cloud storage, such as an Azure Storage account. The state file often contains sensitive data, such as authentication credentials, so limit access to the state file and encrypt its contents.

A configuration commonly contains an `output.tf` file with `output` blocks that capture property values for resources that the configuration creates. The following listing shows two output blocks that provide values for a workspace ID and DFS endpoint:

Terraform

```
output "workspace_id" {
  description = "ID of Fabric workspace"
```

```
  value       = fabric_workspace.workspace_main.id
}

output "workspace_dfs_endpoint" {
  description = "DFS endpoint for Fabric workspace"
  value       = fabric_workspace.workspace_main.onelake_endpoints.dfs_endpoint
}
```

By tracking the state of resources in the current deployment, Terraform compares what's in the configuration against the current deployment. This lets Terraform determine what resources it needs to create, update, or destroy each time you run `terraform apply`. Running `terraform apply` a second time without any configuration changes does nothing—Terraform determines the state file matches the current deployment.

Terraform also helps manage resources over the lifetime of a project. To update an existing resource, change the configuration and run `terraform apply` again. For example, to change the workspace role assignment with the local name `dev_group` from **Member** to **Contributor**, update the resource argument, save your changes, and run `terraform apply`. Terraform handles the sequence of API calls required to make the change.

Fabric capacities are an Azure resource, and you must create them with the *Terraform provider for Azure (azurerm)*. You can authenticate the Azure provider as a service principal in a similar way to the Fabric provider. You also need to include the `subscription_id` for the subscription where you provision the capacity. The service principal must have the appropriate permissions in that subscription to create a Fabric capacity:

Terraform

```
terraform {
  required_version = ">= 1.8, < 2.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "4.63.0"
    }
  }
}

provider "azurerm" {
  features        {}
  tenant_id       = var.tenant_id
  client_id       = var.client_id
  client_secret   = var.client_secret
  subscription_id = var.subscription_id
}
```

The following listing demonstrates how to create a Fabric capacity. First create an Azure resource group by using an `azurerm_resource_group` resource, then create a capacity in that resource group by using an `azurerm_fabric_capacity` resource:

Terraform

```
data "azurerm_client_config" "current" {}

locals {
  spn_object_id = data.azurerm_client_config.current.object_id
}

resource "azurerm_resource_group" "main" {
  name     = var.resource_group_name
  location = var.capacity_location
}

resource "azurerm_fabric_capacity" "main" {
  name                = var.capacity_name
  resource_group_name = azurerm_resource_group.main.name
  location            = azurerm_resource_group.main.location
  administration_members = [
    local.spn_object_id,
    var.admin_user_upn
  ]
  sku {
    name = var.capacity_sku_size
    tier = "Fabric"
  }
}
```

In addition to `resource` blocks, Terraform supports `data` blocks for scenarios where you need to query a data source. The previous listing uses a `data` block based on the `azurerm_client_config` data source to dynamically retrieve the Microsoft Entra object ID for the service principal that runs the Terraform deployment. The `locals` block creates a local variable named `spn_object_id` holding the service principal object ID, which the `azurerm_fabric_capacity` resource then uses to add the service principal to the capacity's administrators group.

When you add an `azurerm_fabric_capacity` resource, you must configure a `sku` block with a `name` argument that controls the capacity SKU size. For example, you might provision the capacities for your project environments with an **F4** for dev, an **F16** for test, and an **F64** for prod. If the test environment capacity is undersized and you need to increase it to **F32**, update the test configuration with the new SKU name and run `terraform apply`.

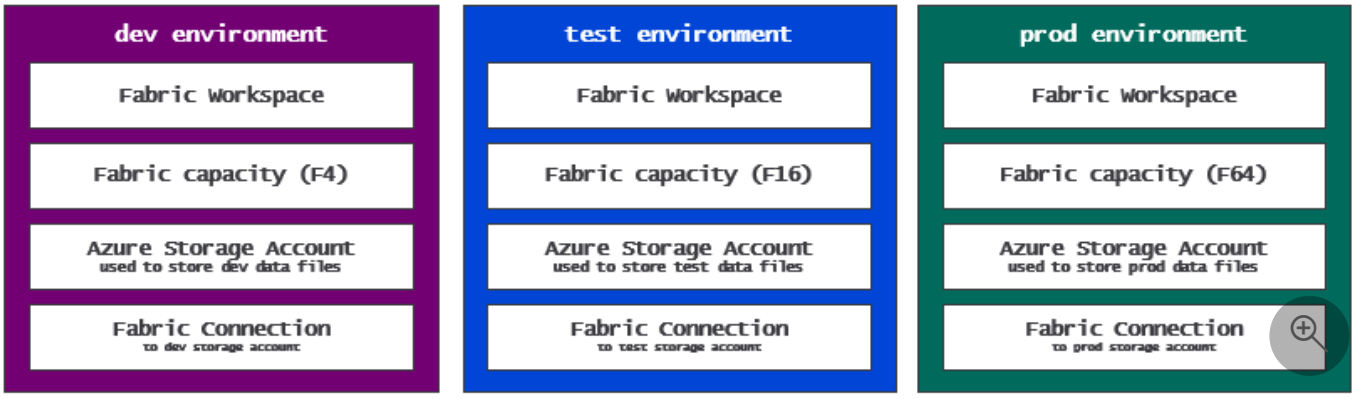
Now consider creating a Fabric capacity and a workspace in the same configuration. You need to load both the Azure provider and the Fabric provider. One detail requires careful handling: the

Azure provider and the Fabric provider use different formats for the capacity ID. Fabric uses a GUID-based ID for capacities while Azure uses a long string identifier containing the subscription ID and resource group name. The best way to handle this is to add a `data` block based on a `fabric_capacity` data source to query the capacity for its GUID-based ID:

Terraform

```
data "fabric_capacity" "main" {
  display_name = var.capacity_name
  depends_on   = [azurerm_fabric_capacity.main]
}

resource "fabric_workspace" "main" {
  display_name = var.workspace_name
  capacity_id  = data.fabric_capacity.main.id
  depends_on   = [data.fabric_capacity.main]
}
```

A Terraform configuration for building Fabric CI/CD project environments typically includes workspaces, capacities, and connections. You can also provision other Azure resources, such as Azure Storage accounts, Azure SQL databases, or Azure Key Vault instances. The following diagram shows an example of a Fabric CI/CD project plan that includes a standard set of resources that three environments require. Each environment contains the same set of resource types. Terraform supports parameterization that lets you provision resources for each environment with its own unique settings.



When using Terraform, it's recommended that you create a separate configuration for each environment. After all, you don't want a problem in the configuration for the **dev** environment or the **test** environment to affect the **prod** environment. You can create configurations for these environments by using this common provisioning flow:

Expand table

Sequence	Terraform resource	Resource purpose
1	`azurerm_fabric_capacity`	Creates the Fabric capacity for the environment.
2	`fabric_workspace`	Creates a workspace assigned to the capacity.
3	`fabric_workspace_role_assignment`	Configures workspace role assignments for access control.
4	`azurerm_storage_account`	Creates an Azure Storage account for environment-specific data files.
5	`azurerm_storage_container`	Creates a container in the Storage account.
6	`azurerm_storage_account_blob_container_sas`	Creates a SAS token credential to access the Storage container.
7	`fabric_connection`	Creates a Fabric connection used to access data files in the Storage account.
8	`fabric_connection_role_assignment`	Configures connection permissions.

For more on Terraform providers, see the Microsoft Fabric Terraform provider in the Terraform Registry and the Azure provider documentation.

This example demonstrates how Terraform helps enforce security and governance best practices. A Terraform configuration can manage access control to project resources by adding role assignments. If anything changes in the future with respect to permissions or access control, you can update the configuration and run the `terraform apply` command.

The example also demonstrates a security best practice for credential management. The Terraform configuration creates a credential in the form of a SAS token and then uses that credential to create a Fabric connection. This approach eliminates the need to share connection credentials with other humans or external processes. The Terraform configuration manages credentials in a secured, encrypted state file in cloud storage. Terraform creates the connection, configures its permissions, and passes the connection ID back to you as output.

Fabric CLI

Fabric CLI is a developer tool for writing automation logic for Fabric CI/CD processes. The Fabric CLI abstracts the underlying Microsoft APIs to provide a single, focused experience for Fabric automation. It handles Microsoft Entra authentication, access token acquisition, and HTTP request dispatch. It also simplifies running automation logic under the identity of a service principal.

The Fabric CLI supports both an *interactive mode* and a *scripting mode*. Interactive mode lets you type and execute a sequence of commands. For example, you can use the Fabric CLI in interactive mode to create a new workspace and then create a lakehouse inside that workspace. The Fabric CLI also provides commands to load data into lakehouse tables and to automate running notebooks and pipelines that contain ETL logic.

Scripting mode makes it possible to write and version automation logic for a Fabric CI/CD project. Use a shell script or a programming language, such as PowerShell or Python, to execute a sequence of Fabric CLI commands. If you decide not to use Terraform, you can write scripts that call Fabric CLI commands to create and manage tenant-level items, such as workspaces, capacities, connections, domains, and gateways.

The Fabric CLI provides workspace-item commands for creating and managing Fabric items. Commands, such as `create`, `get`, `set`, and `rm`, perform CRUD operations on workspace items. There are also two Fabric CLI commands for working with item definitions: `import` creates or updates a workspace item by using an item definition. The complementary `export` command exports a workspace item to a local folder as a set of definition files.

For more information, see the Fabric CLI documentation.

Another important capability of the Fabric CLI is its integration with cloud-based Git providers, such as Azure DevOps and GitHub. If you're developing GitHub Actions workflows, you can install and load the Fabric CLI executable so your workflows can call Fabric CLI commands. If you're developing Azure Pipelines workflows, the **Fabric automation tools** extension for Azure DevOps adds the `FabricCLITask@0` task that provisions the Fabric CLI into the pipeline agent without manual installation. For more information, see Fabric automation tools for Azure DevOps.

Semantic Link Labs

Semantic Link is a capability in Microsoft Fabric that connects data science and data analytics by integrating Power BI semantic models with Python and Spark environments. Semantic Link lets data scientists load data from a semantic model into a `FabricDataFrame` object that they can use directly with data science libraries, such as *pandas* and *scikit-learn*.

Semantic Link Labs is a Python library designed for use in Fabric notebooks. It provides access to Semantic Link when you work in Fabric and adds functionality for working with workspace items, such as semantic models, reports, lakehouses, notebooks, and variable libraries. The goal of Semantic Link Labs is to simplify technical processes so you can focus on higher-level activities.

Semantic Link Labs completes its work by calling public Microsoft APIs, such as the Fabric REST APIs, Power BI REST APIs, and the Tabular Object Model. When you use Semantic Link Labs, you benefit from the simplicity of a single API surface without managing the details of acquiring access tokens or dispatching calls to multiple APIs.

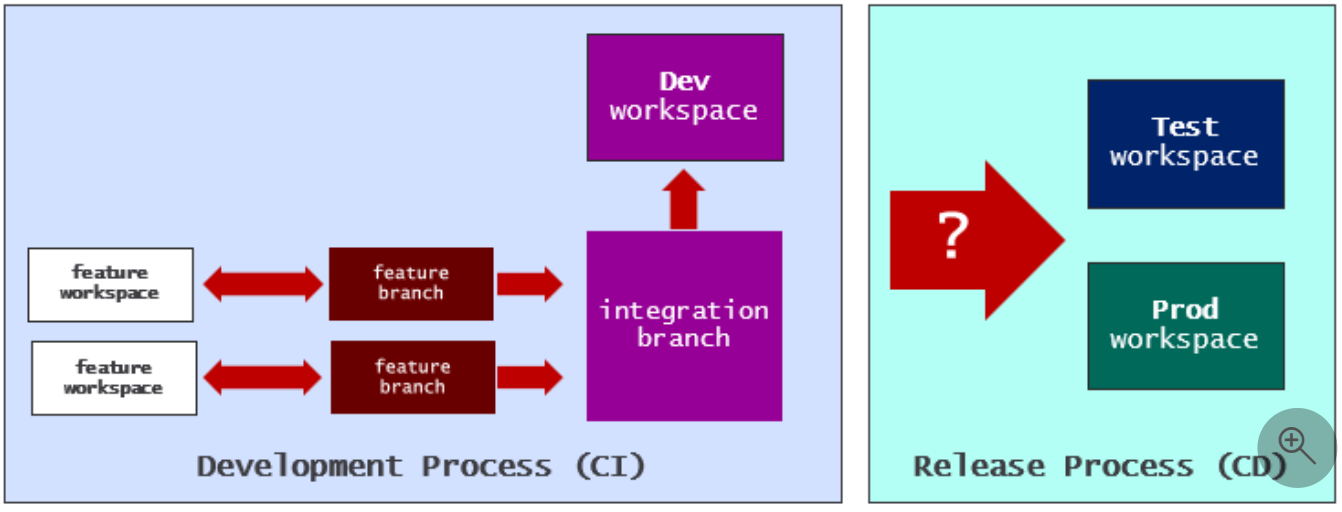
Semantic Link Labs lets you work with workspace items from the Data Engineering workload, such as lakehouses, notebooks, and environments. It simplifies creating lakehouses and notebooks, configuring a notebook to use a specific lakehouse as its default lakehouse, running jobs for notebooks and pipelines, and monitoring job execution to determine whether a job completed successfully.

For more on Semantic Link Labs, see the Semantic Link Labs project on GitHub.

Semantic Link Labs works only inside the Fabric environment. You can't load the Semantic Link Labs library in a non-Fabric environment, such as Azure Pipelines or GitHub Actions workflows. To use Semantic Link Labs functionality from an external automation host, maintain the code that calls Semantic Link Labs in a Fabric notebook, and automate running that notebook from your external host.

Release process options

Earlier sections described the fundamentals of building a development process by using feature branches and feature workspaces. The goal of the development process is to create a single source of truth in the integration branch that is always ready for deployment. To complete the full CI/CD lifecycle, you must also build a release process that deploys a Fabric solution to workspaces in environments for testing and production.



This article examines three options for building a release process:

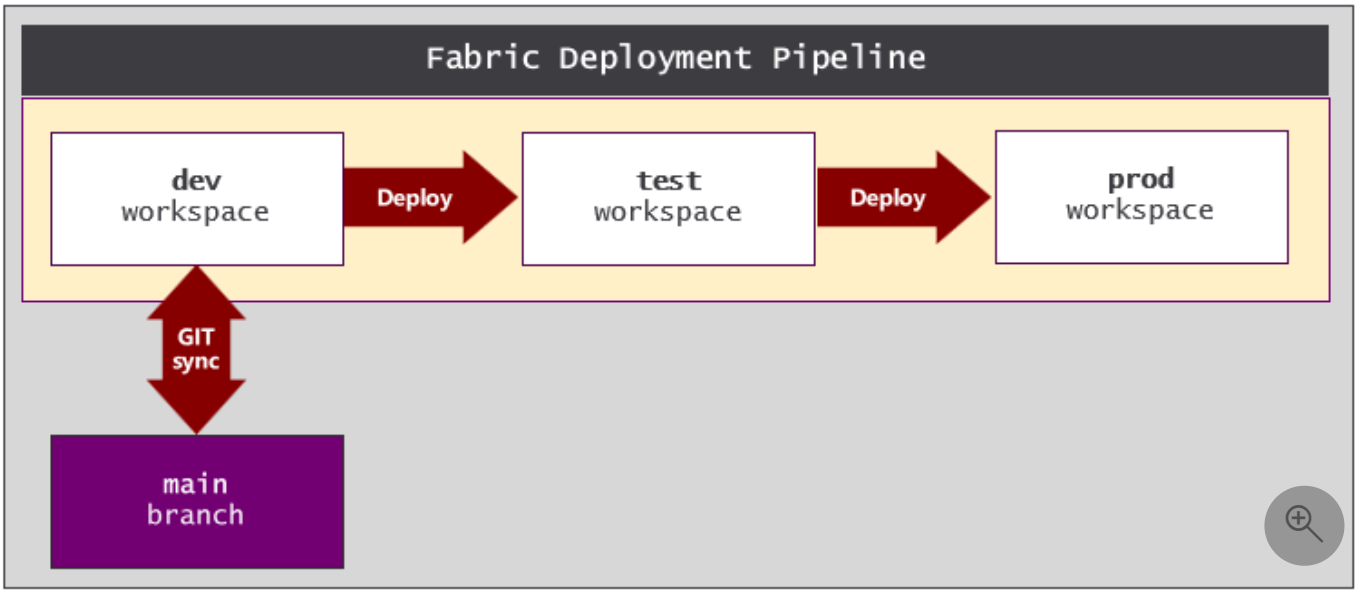
Expand table

Release process option	Best fit	Main tradeoff
Deployment pipeline	Low-code promotion across staged workspaces.	Lower setup effort, but less automation flexibility and more limitations for large or cross-tenant projects.
Git synchronization	Branch-based releases where target workspaces can stay connected to Git.	Uses familiar Git flows, but requires post-sync jobs for environment-specific settings.
API-driven approach	Scalable releases that need custom branching, parameterization, or many target workspaces.	Most flexible, but requires tooling, such as `fabric-cicd` or custom API code.

The first and lowest-effort option for building a release process is using a deployment pipeline. Deployment pipelines offer a low-code approach that can fit small-to-medium-sized projects, especially those exclusively focused on semantic models and Power BI reports.

As mentioned earlier, deployment pipelines assist with building a release process and not a development process. When you choose to use a deployment pipeline in a Fabric CI/CD project, you still need to plan how to build a development process. At a minimum, connect the first workspace in a deployment pipeline to a Git branch to gain basic CI/CD capabilities, such as versioning items and reverting to an earlier version of an item if something goes wrong with an update.

The following diagram shows a deployment pipeline with the first workspace connected to the **main** branch in a Git repository. In the simplest development scenario, you can make direct updates to items in the **dev** workspace and then use **Commit to Git** operations to commit changes to Git where you can use them to revert to earlier item versions when necessary.



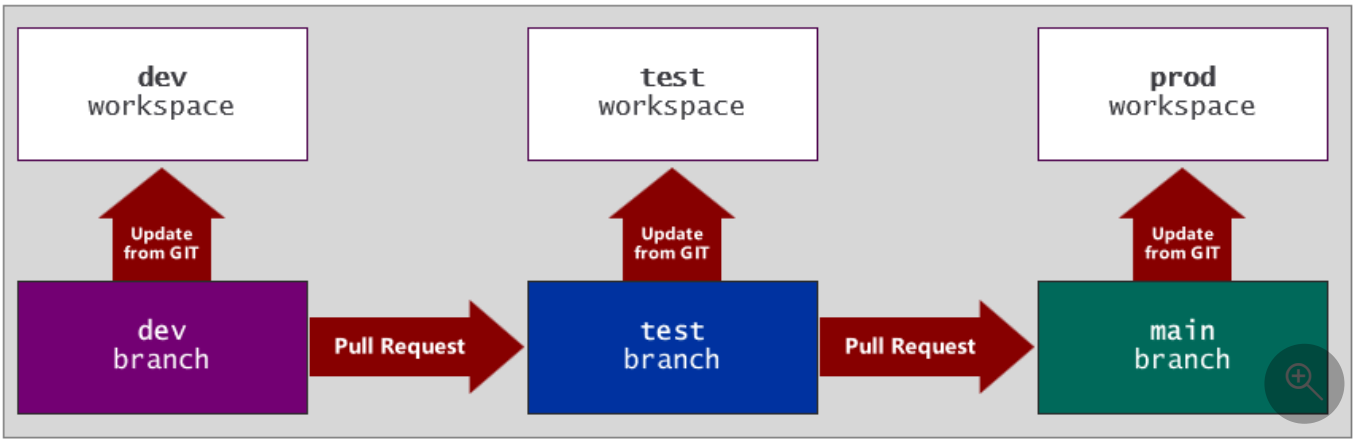
If you decide to build the release process by using a deployment pipeline, you still have the ability to design a scalable development process by using feature branches and feature workspaces. As an example, you can use the **main** branch as the integration branch for your development process. This makes it possible to create new feature workspaces by using the **Branch out to workspace** command to branch out from the **dev** workspace. After you commit changes and a pull request merges them into the **main** branch, use an **Update from Git** operation to synchronize the changes to items in the **dev** workspace. Once item-level changes propagate to the **dev** workspace, use the **Deploy** command to deploy them across the other workspaces in the deployment pipeline.

While deployment pipelines provide the lowest-effort path to build a release process, they also have noteworthy limitations. For example, a deployment pipeline and all of its associated workspaces must exist inside the same Microsoft Entra ID tenant. If you have a requirement to create the **dev** workspace in one Microsoft Entra ID tenant and the **prod** workspace in another, you can't use deployment pipelines to build your release process.

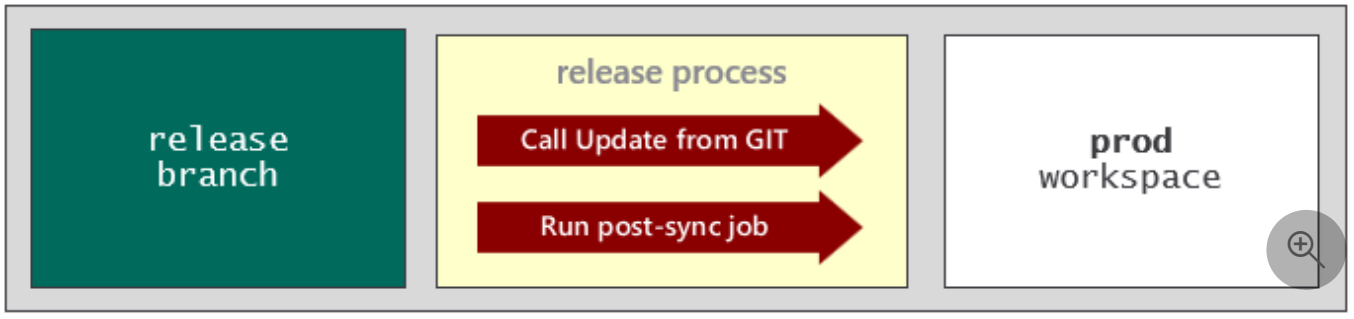
There are some other factors to consider as well when deciding whether to use deployment pipelines. Setting up a deployment pipeline always requires some degree of manual configuration because public APIs don't support certain setup tasks. Deployment pipelines are also limited when it comes to setting up manual-approval processes. This is especially true when compared to what's possible with pull requests and approval gates in a Git repository. Moreover, deployment pipelines aren't well suited to handle larger projects where there are workspaces containing hundreds of items.

A second option is to build a release process by using Git synchronization. For example, you can use a GitFlow branching strategy in which you configure a Git repository with three long-lived branches named **dev**, **test**, and **main**. The **dev** branch serves as the integration branch with code

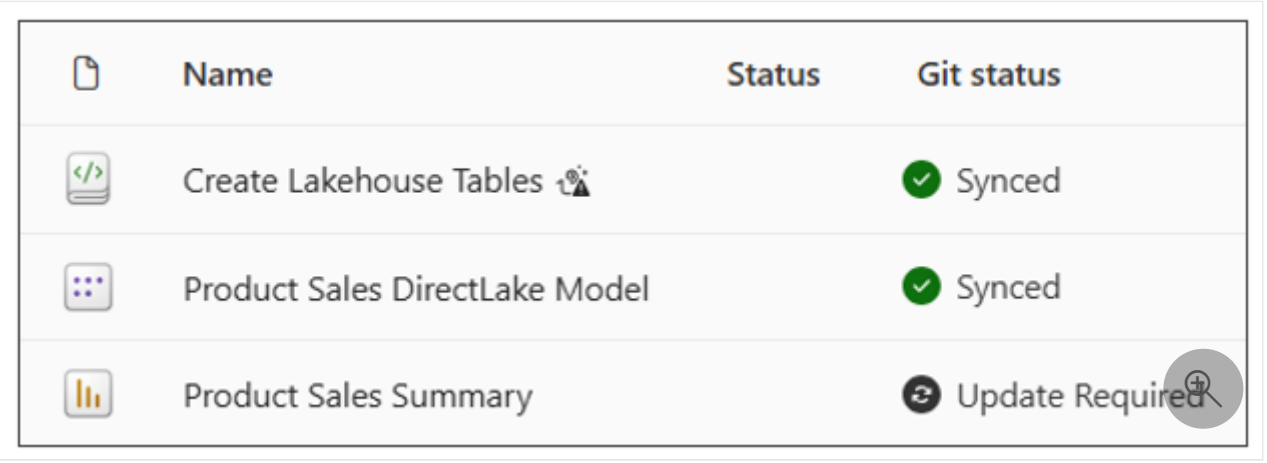
ready for deployment. Pull requests push changes to the **test** branch and to the **main** branch, which act as dedicated release branches. You can then use an **Update from Git** operation to deploy changes from a release branch to its target workspace.



There are two important issues to consider when deciding whether to build a release process by using Git synchronization. Environment-specific settings that you commit to Git cause one issue. For example, Git synchronization commits the data-source location for semantic models in the **dev** workspace to Git and then propagates it to semantic models in the **test** workspace and the **prod** workspace. That means you need to run a post-sync job after an **Update from Git** operation completes to update each semantic model with the data-source path that is appropriate for its target environment.



A second issue with Git synchronization is that it requires a connection between the target workspace and a Git branch. When you connect a workspace to a Git branch, the Fabric user interface displays indicators and messages to control and monitor Git synchronization. For example, the workspace summary page displays the **Status** column with values, such as **Synced** or **Update required**, which you might prefer to hide from users in a production workspace.

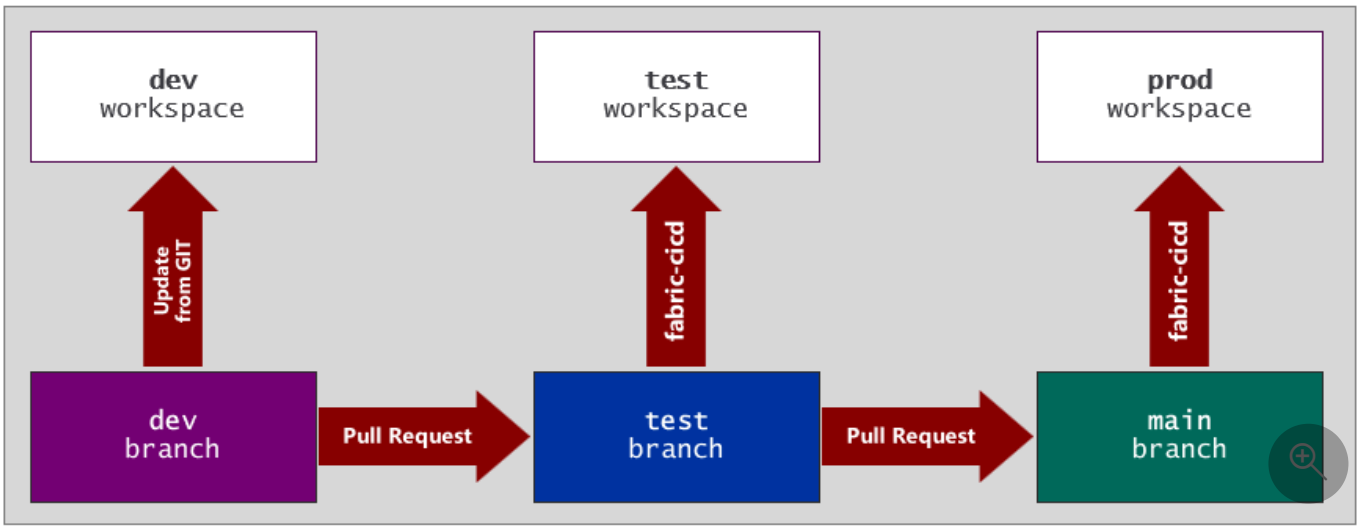


The third option for building a release process is using an *API-driven approach*. An API-driven approach can scale better than the other two choices for larger projects in which workspaces contain hundreds of items.

Using an API-driven release process also provides the flexibility to use whatever Git branching strategy your organization prefers. For example, you might prefer a GitFlow branching strategy or standardize on trunk-based development. Once you've chosen a branching strategy, you can then adapt an API-driven release process to accommodate it.

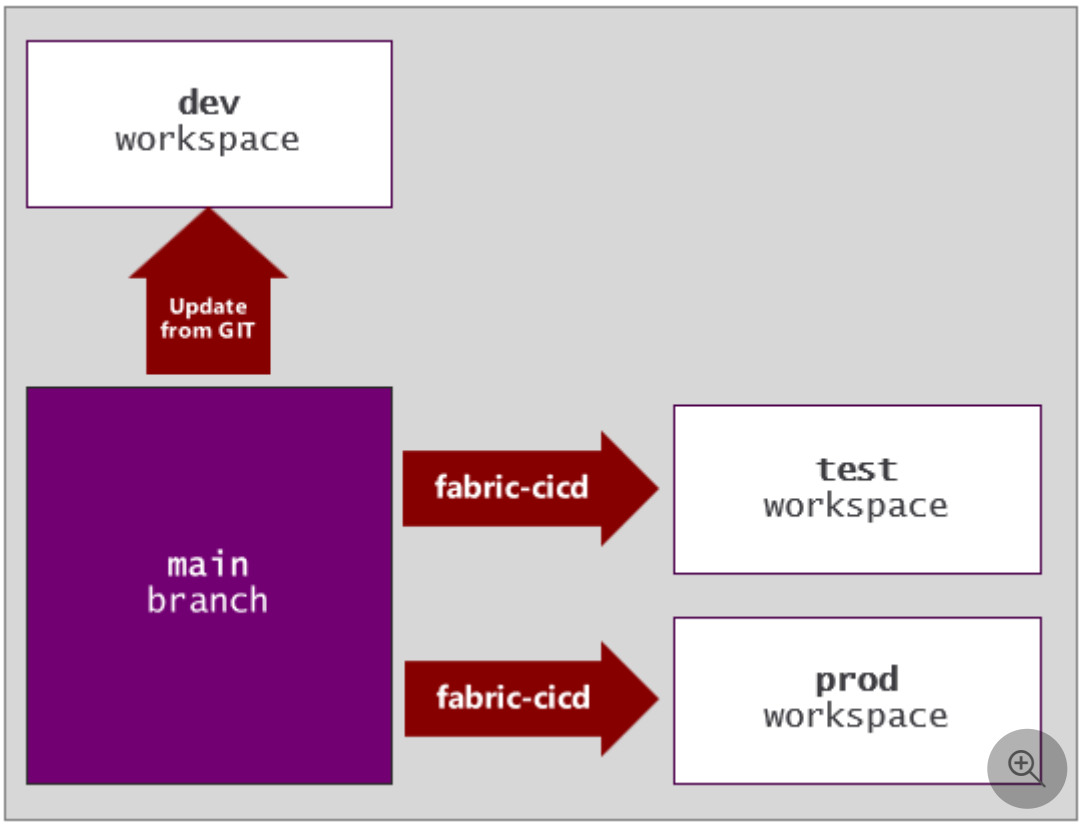
You could implement a release process by using the Fabric REST APIs, but that approach requires non-trivial coding effort. The `fabric-cicd` library reduces that effort by providing an API-driven release process that can adapt to common branching strategies.

Consider the release process in the following screenshot, which uses a GitFlow branching strategy. The **dev** branch serves as the integration branch. This release process uses pull requests to push changes to **test** and **main**, which serve as release branches. The use of pull requests makes it possible to configure manual-approval processes and to trigger workflows that run when a pull request completes. When a pull request completes and merges its changes into one of the release branches, a workflow runs automatically and uses `fabric-cicd` to deploy changes to the target workspace.

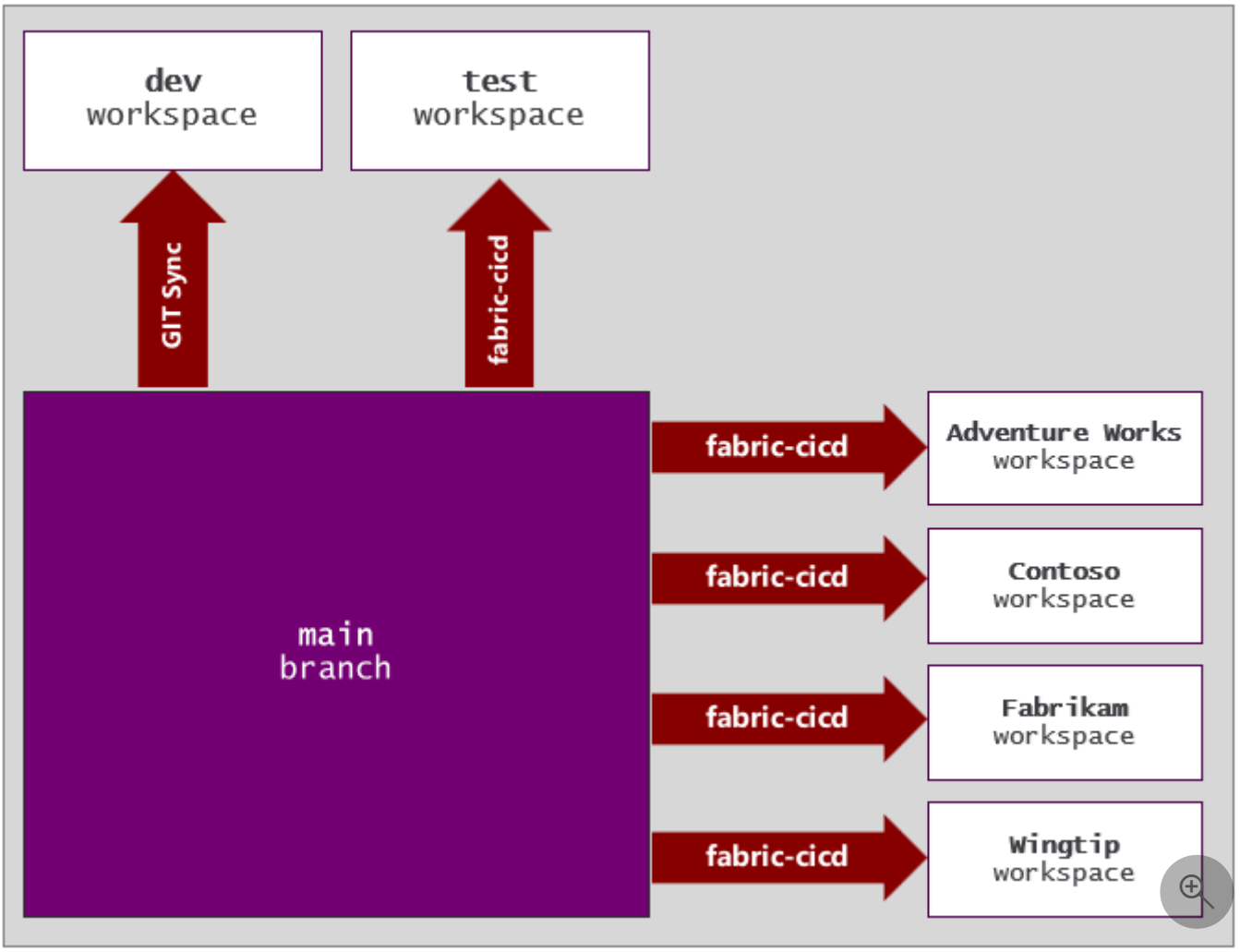


This example is similar to the Git synchronization example shown earlier because both use the GitFlow branching strategy. However, `fabric-cicd` supports parameterization, which makes it possible to dynamically update environment-specific settings as part of the deployment process. Compare this to Git synchronization, which pushes environment-specific settings to items in the target workspace and then requires additional automation after an **Update from Git** operation to apply a fix.

Another release process can use a different branching strategy. Consider a scenario in which your organization has standardized on trunk-based development by using a branching strategy based on a single long-lived branch. The following diagram shows a release process in which the **main** branch acts as both the integration branch and the release branch. The `fabric-cicd` library provides the flexibility of building a release process with a one-to-many mapping between a release branch and multiple target workspaces.



The ability to configure a one-to-many mapping between a release branch and target workspaces is suitable for independent software vendors (ISVs) that build *multi-tenant solutions*. The following diagram shows a release process that uses `fabric-cicd` to deploy a Fabric solution to multiple customer-specific workspaces, also called *tenant workspaces*. As pull requests merge new changes into **main**, workflows can deploy those changes to the **test** workspace for validation before deploying them to tenant workspaces.



In the previous diagram, Git synchronization pushes **main** branch changes to the **dev** workspace instead of `fabric-cicd`. You need Git synchronization to build a development process by using branched workspaces. By connecting the **dev** workspace to **main**, you make it possible to run the **Branch out to workspace** command to create and manage feature branches and feature workspaces.

When building a release process, you often need to configure a manual-approval process in which one or more approvers approve a release before the workflow deploys it to a target workspace. When using a GitFlow branching strategy, you can configure pull requests with the required manual-approval process. When using trunk-based development, the release process doesn't involve pull requests.

Azure DevOps and GitHub provide another way to configure a manual-approval process without using pull requests. You can extend a Git repository by creating *environments*. An environment in a Git repository acts as a deployment target that you can configure with a manual-approval process. When you run a workflow that targets an environment that has a manual-approval process, the workflow pauses until the approval process completes. Once the approval is

complete, the workflow resumes and can use the `fabric-cicd` library to deploy the new release to the target workspace.

The fabric-cicd library

`fabric-cicd` is an open-source Python library that Microsoft designed to enable code-first deployment. Code-first deployment uses the idea that you can create the single source of truth for a Fabric solution by assembling a set of item definitions. Once you've assembled the item definitions for a Fabric solution, you can then use these item definitions as templates to deploy a matching set of items to a target workspace.

While `fabric-cicd` is an open-source project, the Fabric product team officially supports and recommends it as a best practice.

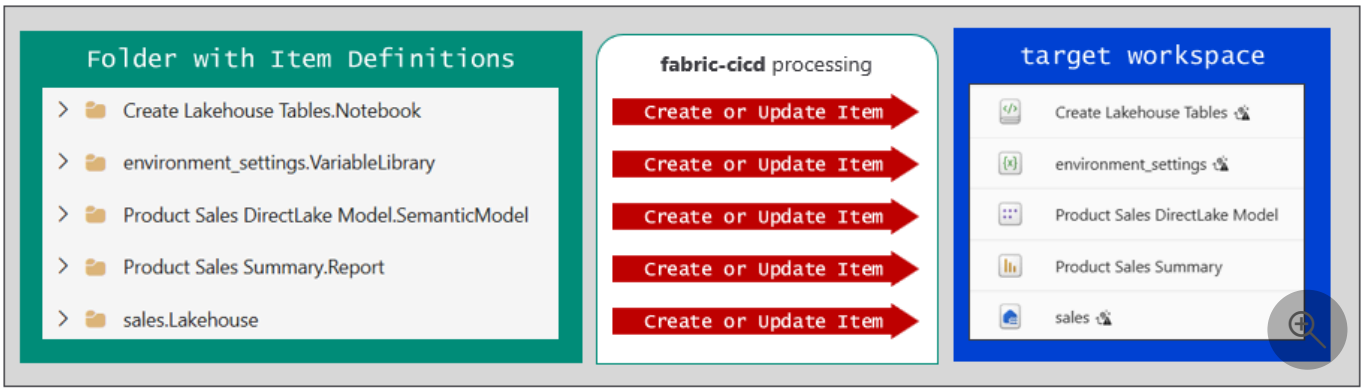
A stated goal of the `fabric-cicd` project is to assist you if you prefer not to interact directly with the Fabric REST APIs. Running a deployment job with `fabric-cicd` only requires a small number of lines of code, while the library handles discovery, create-or-update checks, dependency sequencing, and item relationship binding.

To run a deployment job by using `fabric-cicd`, you must supply two important inputs:

Expand table

`fabric-cicd` deployment input	Purpose
Root folder with item definitions	Identifies the source item definitions to deploy, including definitions in child folders.
Target environment	Identifies where `fabric-cicd` creates or updates matching workspace items.

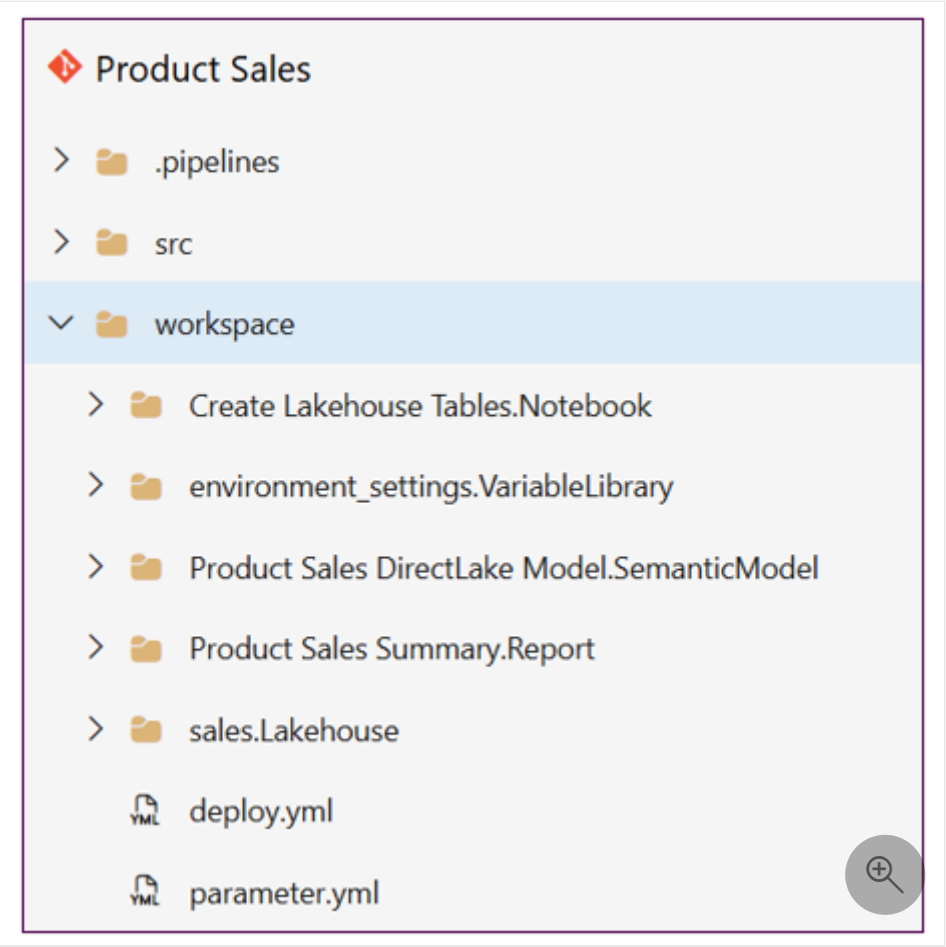
When you start a deployment job, `fabric-cicd` enumerates the root folder to discover every item definition. For each item definition that `fabric-cicd` discovers, it runs a check on the target workspace to determine whether a matching item already exists. `fabric-cicd` requires this check so it can decide whether to create a new item or update an existing item.



`fabric-cicd` is able to automatically reestablish relationships for items that support auto-binding. The `fabric-cicd` library implements auto-binding behavior in a manner similar to Git synchronization by using the `logicalId` found in the platform file of an item definition. When a `fabric-cicd` deployment job runs, it uses the `logicalId` to map out and rebind item dependencies between items in the target workspace.

When `fabric-cicd` runs a deployment job, it processes items in a specific sequence to handle item dependencies. `fabric-cicd` can't create an item with a dependency until after it has created the items on which it depends. As an example, notebooks can have dependencies on lakehouses. Pipelines can have dependencies on lakehouses and on notebooks. `fabric-cicd` handles this dependency tree by deploying lakehouses first, then notebooks, and then pipelines. The library also includes logic to detect whether any pipelines have dependencies on other pipelines. The sequencing ensures that pipelines with dependencies are always deployed after other pipelines on which they depend.

The following example shows how to run a deployment job. You can start by adding a pair of YAML files to the same root folder that holds the item definitions for a Fabric solution. Use the first file, named `deploy.yml`, to configure settings for `fabric-cicd` deployment jobs. Use the second file, named `parameter.yml`, to configure parameterization to manage environment-specific settings committed to Git.



The following example shows configuration-based deployment by using `fabric-cicd`. The YAML content in `deploy.yml` contains a top-level key named `core` with the following child keys:

Expand table

`deploy.yml` key	What it configures
`workspace_id`	Maps environment names, such as `test` and `prod`, to target workspace IDs.
`repository_directory`	Points to the root folder that contains Fabric item definitions.
`item_types_in_scope`	Limits deployment to the listed Fabric workspace item types.
`parameter`	Points to the parameterization file used for environment-specific replacements.

YAML

```yaml
core:
  workspace_id:
    test: "a0a0a0a0-bbbb-cccc-dddd-e1e1e1e1e1e1"
    prod: "b1b1b1b1-cccc-dddd-eeee-f2f2f2f2f2f2"

  repository_directory: "/workspace"

  item_types_in_scope:
```

```
    - VariableLibrary
    - Lakehouse
    - Notebook
    - DataPipeline
    - SemanticModel
    - Report

  parameter: "parameter.yml"
```

Target environments are an important concept when using `fabric-cicd`. The configuration in this example defines two target environments named `test` and `prod`. The configuration maps each environment to a target workspace by using a unique `workspace_id` value. Whenever you start a deployment job, you must pass a parameter indicating which environment you want to target. You'll experience an error if you try to start a deployment job by using an environment name that `deploy.yml` doesn't define.

You can replace `workspace_id` with `workspace` to configure target workspaces by using their display name instead of their ID.

`fabric-cicd` handles variable libraries specially. If your solution contains a variable library, `fabric-cicd` always deploys it first before any other items that depend on it. After creating a variable library, `fabric-cicd` can also activate a specific value set. Value set activation requires matching names between the environment passed to `fabric-cicd` and the value set to activate. If you run a `fabric-cicd` deployment job with a target environment name, such as `test` or `prod`, ensure the variable library also contains value sets named `test` or `prod`.

The following Python code starts a `fabric-cicd` deployment job. You can install the `fabric-cicd` library from the Python Package Index (PyPI) with the pip install command.

Bash

```
pip install fabric-cicd
```

You can use the `fabric-cicd` library to build release processes in workflows that run in Azure DevOps or GitHub. The ability to install on demand by using `pip install` helps integrate `fabric-cicd` with Azure Pipelines or GitHub Actions.

The following Python code demonstrates a minimal deployment job with `fabric-cicd`. The example creates a credential to authenticate with Microsoft Entra ID as a service principal by using credentials stored in environment variables. You start a deployment job by calling the `deploy_with_config` function and passing parameter values for `token_credential`, `config_file_path`, and `environment`.

Python

```python
import os
from azure.identity import ClientSecretCredential
from fabric_cicd import deploy_with_config

# Create a credential to authenticate as a service principal.
credential = ClientSecretCredential(
    tenant_id=os.environ["AZURE_TENANT_ID"],
    client_id=os.environ["AZURE_CLIENT_ID"],
    client_secret=os.environ["AZURE_CLIENT_SECRET"],
)

# Deploy to the test workspace.
deploy_with_config(
    token_credential=credential,
    config_file_path="../cicd/workspace/deploy.yml",
    environment="test",
)

# Deploy to the prod workspace.
deploy_with_config(
    token_credential=credential,
    config_file_path="../cicd/workspace/deploy.yml",
    environment="prod",
)
```

When calling `deploy_with_config`, you must pass an `environment` value that maps to a target environment defined in `deploy.yml`. With multiple deployment targets in configuration, you can build a release process with a one-to-many mapping between a single release branch and multiple target workspaces. The previous code listing switches between deploying to the **test** workspace and the **prod** workspace by passing a different environment name when calling `deploy_with_config`.

Next, examine the `fabric-cicd` support for parameterization, which makes it possible to dynamically update settings in item definition files during a deployment job. This parameterization support provides a reliable way to manage environment-specific settings that you committed to Git.

Consider an example of a semantic model that connects to an Azure Storage account but requires different data-source paths for the three environments **dev**, **test**, and **prod**. The problem is that Git commits the data-source path for the semantic model in the **dev** workspace to the integration branch. You need parameterization to update the semantic models in the **test** workspace and the **prod** workspace with their environment-specific data-source paths.

The following YAML listing demonstrates configuring `parameter.yml` with a find-and-replace operation. There's a top-level key named `find_replace` with nested keys named `find_value` and `replace_value`. You assign the `find_value` key a string value from the **dev** workspace that you want to replace. The `replace_value` key contains a nested key-value pair for each target environment. In this example, the configuration defines two target environments named `test` and `prod`.

YAML

```yaml
find_replace:
  - find_value: "https://<dev-storage-account>.dfs.core.windows.net/productsales/"
    replace_value:
      test: "https://<test-storage-account>.dfs.core.windows.net/productsales/"
      prod: "https://<prod-storage-account>.dfs.core.windows.net/productsales/"
```

By default, `fabric-cicd` processes a `find_replace` operation by inspecting every file in every item definition whose type the `item_type_in_scope` property setting includes. This can lead to longer processing times in scenarios in which you have many items or item definitions, such as a semantic model with many files.

To optimize performance, use `fabric-cicd` parameterization filters to control which items and files `fabric-cicd` examines during a replacement operation. For example, you can extend the `find_value` key by adding filtering keys, such as `item_type`, `item_name`, and `file_path`. The following configuration demonstrates how to filter a `find_replace` operation to restrict processing to a single file named `expressions.tmdl` in the item definition for a single semantic model by using its display name.

YAML

```yaml
find_replace:
  - find_value: "https://<dev-storage-account>.dfs.core.windows.net/productsales/"
    replace_value:
      test: "https://<test-storage-account>.dfs.core.windows.net/productsales/"
      prod: "https://<prod-storage-account>.dfs.core.windows.net/productsales/"
    item_type: "SemanticModel"
    item_name: "Product Sales Imported Model"
    file_path: "/definition/expressions.tmdl"
```

The parameterization support in `fabric-cicd` supports more than simple *find-and-replace* operations. `fabric-cicd` also supports parameterization by using regular expressions and dynamic variables. To demonstrate these advanced parameterization capabilities, examine a common scenario in which a semantic model connects to the SQL endpoint of a lakehouse in the same workspace. This scenario requires parameterization because each semantic model requires

a unique data-source path to reference the SQL endpoint for the lakehouse in the same workspace.

The item definition for a semantic model includes an `expressions.tmdl` file that typically contains data-source paths and Power Query code that connects to its data source. The `expressions.tmdl` file for a semantic model that uses the DirectLake on SQL model includes a call to the `SQL.Database` function that accepts two parameters. The first parameter is for the SQL endpoint server path, and the second parameter references the target lakehouse.

text

```
Sql.Database(<SQL Endpoint Connect String>, "c2c2c2c2-dddd-eeee-ffff-a3a3a3a3a3a3")
```

The second parameter passed to `SQL.Database` identifies the target lakehouse. For this lakehouse parameter, you can pass either the lakehouse ID or the lakehouse display name. Using the lakehouse ID is problematic because it's always different across workspaces, requiring additional parameterization. Instead, pass the lakehouse name. The lakehouse name stays the same across all environments, eliminating the need for parameterization.

text

```
Sql.Database(<SQL Endpoint Connect String>, "sales")
```

The first parameter passed to `Sql.Database` is the SQL endpoint connection string, which ends with `.datawarehouse.fabric.microsoft.com`. The first part of the SQL endpoint connection string is always unique to a specific workspace. This means the SQL endpoint connection string requires parameterization.

text

```
Sql.Database("<workspace-unique-path>.datawarehouse.fabric.microsoft.com",
```

`fabric-cicd` supports parameterization by using regular expressions to identify *capture zones* used in find and replace operations. The following regular expression demonstrates defining a capture zone for the SQL endpoint connection string.

text

```
Sql\.Database\(\s*"([^"]*datawarehouse\.fabric\.microsoft\.com[^"]*)"\s*,
```

Once you have created the regular expression with a capture zone to replace the SQL endpoint connection string, you can use it in a `find_replace` operation as long as you add an `is_regex` key with a value set to `true`.

YAML

```yaml
find_replace:
  - find_value: 'Sql\.Database\(\s*"
([^"]*datawarehouse\.fabric\.microsoft\.com[^"]*)"\s*,'
    replace_value:
      test: $items.Lakehouse.sales.$sqlendpoint
      prod: $items.Lakehouse.sales.$sqlendpoint
    is_regex: "true"
    item_type: ["SemanticModel"]
    file_path: "**/expressions.tmdl"
```

In addition to using a regular expression with a capture zone, the previous example demonstrates how `fabric-cicd` supports dynamic variables. You can create a dynamic variable to reference a lakehouse named **sales** by using the syntax `$items.Lakehouse.sales`. In this example, the `$sqlendpoint` property of the dynamic lakehouse variable provides the replacement value in the `replace_value` key.

You can simplify the example. In the previous listing, both environments use the same dynamic variable expression in the `replace_value` key. In a parameterization scenario where all environments can use the same expressions, use the `_ALL_` key to eliminate redundant environment keys.

YAML

```yaml
find_replace:
  - find_value: 'Sql\.Database\(\s*"
([^"]*datawarehouse\.fabric\.microsoft\.com[^"]*)"\s*,'
    replace_value:
      _ALL_: $items.Lakehouse.sales.$sqlendpoint
    is_regex: "true"
    item_type: ["SemanticModel"]
    file_path: "**/expressions.tmdl"
```

This example demonstrates parameterization by using regular expressions, capture zones, and dynamic variables. You can use dynamic variables to obtain the `$id` property for any type of workspace item. You can also use a dynamic `$workspace` variable to obtain the `$id` property and the `$name` property when you need the ID or display name of the current workspace.

To learn more about `fabric-cicd`, see the fabric-cicd documentation home page, fabric-cicd code samples, fabric-cicd code reference, and fabric-cicd parameterization guidance.

Data orchestration

Fabric provides several workspace item types for building data-centric solutions. This support builds on one set of workspace item types that act as data containers and a complementary set of workspace item types that focus on running ETL processes and data integration. When designing a Fabric solution with a data container item, such as a lakehouse, you must plan how to build and run ETL logic to populate the lakehouse. You can choose between several types of ETL-focused item types, such as notebooks, pipelines, copy jobs, user-defined functions, dataflows, and Spark job definitions.

Author and maintain the ETL logic for a Fabric solution by using workspace items that are part of the same solution. When you build ETL logic by using workspace items, such as notebooks or pipelines, Git can track and version the ETL logic over time along with all other items in the Fabric solution. Avoid designing Fabric solutions that depend on external ETL processes or scripts that aren't part of the current solution.

You have several options for deploying a Fabric solution with a lakehouse to a target workspace.

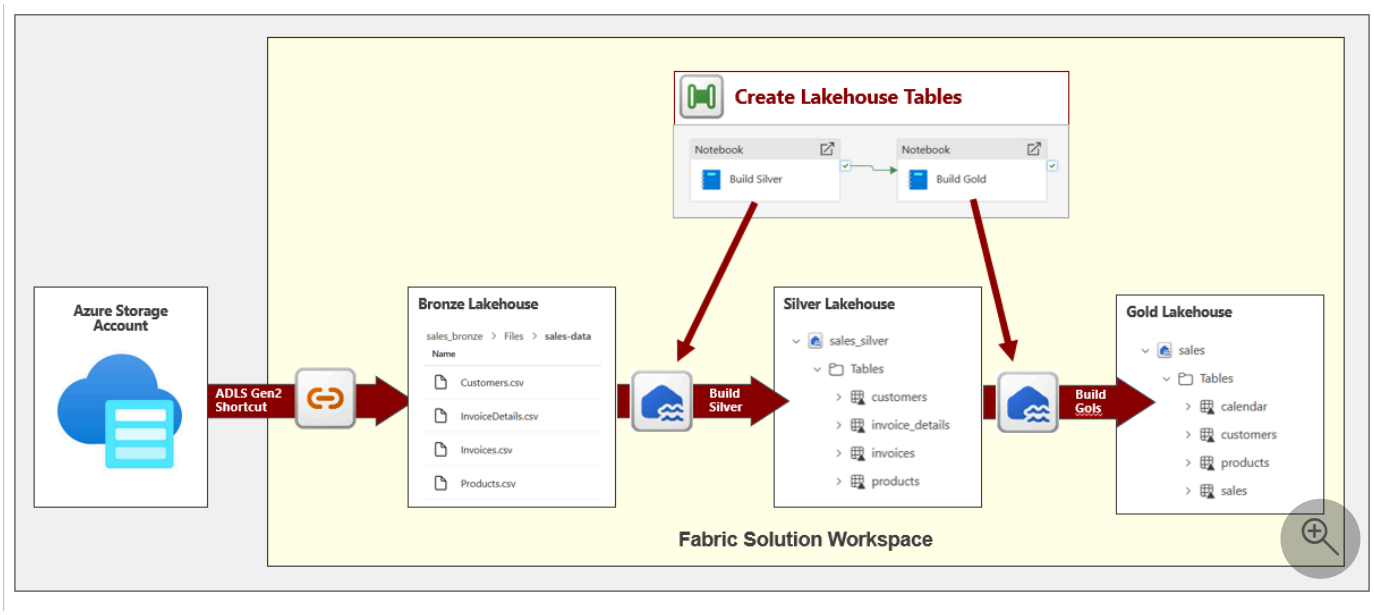
Expand table

Lakehouse deployment option	What happens after deployment
Deployment pipeline	The lakehouse item is promoted to the target workspace as an empty data container.
Git synchronization	**Update from Git** creates or updates the lakehouse item from its item definition.
`fabric-cicd`	The deployment job creates or updates the lakehouse item from the source item definition.

Whichever of these three deployment options you choose, the deployment result is the same. Each option creates the lakehouse as an empty data container that requires additional work. More specifically, you need to design a Fabric solution with a *data orchestration strategy* that addresses requirements for data ingestion, data transformation, and data storage.

The following diagram shows an example of data orchestration in a Fabric solution. This solution uses a medallion architecture in which data flows across three lakehouses that serve as storage

for the bronze, silver, and gold layers. The bronze lakehouse uses an Azure Data Lake Storage Gen2 (ADLS Gen2) shortcut to enable access to data files in an Azure Storage account container. One notebook builds the silver layer by loading the data files from the bronze lakehouse and persisting them as Delta tables in the silver lakehouse. A second notebook builds the gold layer by loading tables from the silver lakehouse and transforming them into a star schema before saving them as Delta tables in the gold lakehouse.



An important aspect of this data orchestration strategy is that it exposes a top-level pipeline named **Create Lakehouse Tables**. When you run this pipeline, it runs the two notebooks in sequence to execute all the necessary ETL logic to populate all lakehouses with data. After you deploy this Fabric solution, which creates lakehouses in an empty state, you can then automate running this pipeline to execute the data orchestration that moves the workspace into a state where it's ready for use.

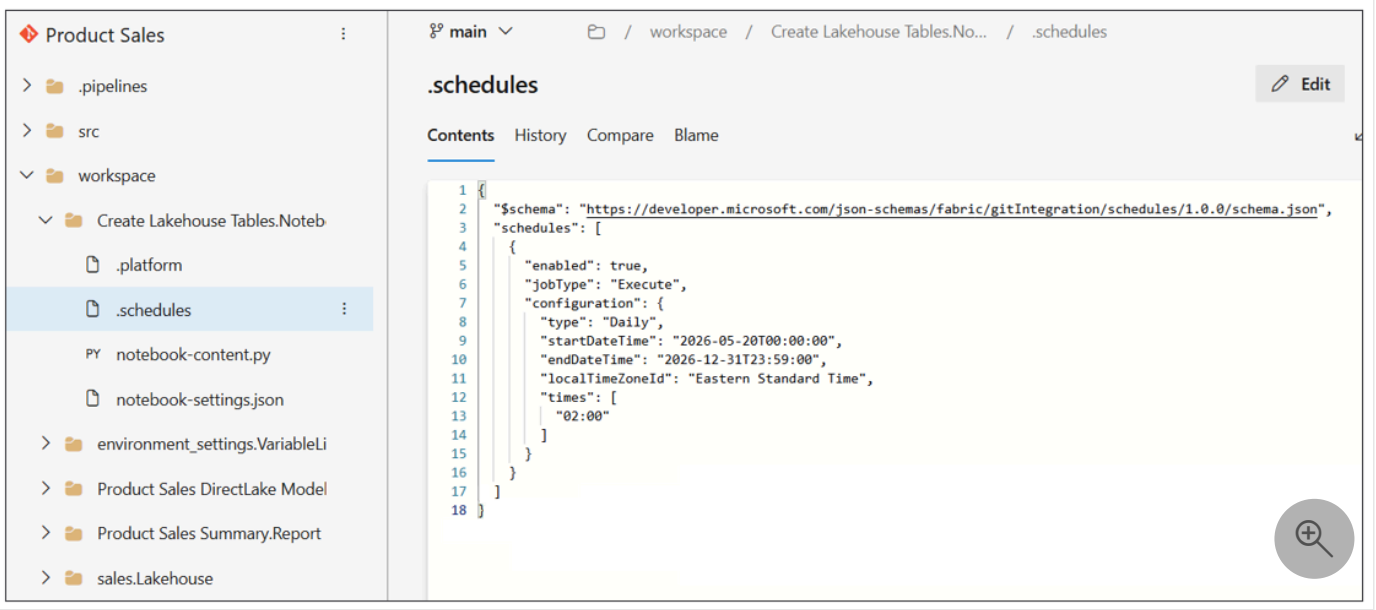
A Fabric solution should expose a single item with top-level ETL logic that you can run to populate data container items with data. Whether you implement a data orchestration strategy by using notebooks, pipelines, or another item type, the important point is that when you first deploy a Fabric solution, you can follow up deployment by running a single job with the data orchestration that prepares the workspace for users.

When designing a data orchestration strategy, you must also consider data refresh requirements. Consider what happens after the initial deployment populates data container items with data for the first time. How do you plan to keep the data in a lakehouse up to date? Automate the processes for refreshing data by using *scheduled jobs*.

For example, you could schedule a pipeline that processes a full data refresh to run once a day while designing another pipeline with incremental refresh logic to run as a scheduled job every

15 minutes. The key point is that adding scheduled jobs should be part of the data orchestration strategy because it enables a Fabric solution to self-manage all its data refresh requirements.

Fabric makes it possible to extend the item definition of notebooks and pipelines by using a `.schedules` file. The `.schedules` file contains JSON content with a `schedules` element containing a list of one or more scheduled jobs. By extending the item definitions for ETL items in a Fabric solution with `.schedules` files, you can automate a data refresh strategy.



One final issue to consider with data orchestration is how to address changes to schemas over time. Consider a scenario where you need to update the table schema for a lakehouse that deployment already created and populated with data. Deploying table schema changes to a lakehouse is challenging because Fabric doesn't track any metadata for the table schema in the underlying item definition for a lakehouse. The most common way to update the table schema for a lakehouse is to run a notebook with procedural programming logic to add and remove tables and table columns.

In a feature workspace, you might write and test code in a notebook to update the lakehouse table schema. After testing the code against a lakehouse in the feature workspace, you commit your changes to Git and use a pull request to push the notebook with the schema update logic to the integration branch.

At this point, you can use an **Update from Git** operation to deploy the notebook to the **dev** workspace. Likewise, you can use `fabric-cicd` to deploy the notebook to the **test** workspace or the **prod** workspace. However, deploying the notebook to the workspace with the lakehouse isn't enough. You still need to run the notebook in the target workspace to update the lakehouse schema.

Managing schema and pushing schema changes with a Fabric warehouse differs significantly from doing so with a lakehouse. The best practice for implementing CI/CD processes with a warehouse schema involves using the **SqlPackage** and **Data-tier Application Framework** tooling Microsoft provides. For more information, see the SqlPackage documentation.

Fabric REST API programming

Fabric offers several developer tools and libraries that reduce implementation effort. You have seen examples of Terraform, Fabric CLI, Semantic Link Labs, and `fabric-cicd`, which hide low-level details associated with authentication, acquiring access tokens, and executing HTTP requests. However, you might encounter scenarios in which you need a greater degree of control. You can use *Fabric REST API programming* to execute API calls by using a familiar programming language, such as C# or Python.

This guidance article recommends using Terraform as your first choice to provision the infrastructure for Fabric CI/CD projects. But there could be factors that make you decide against using Terraform in a particular Fabric CI/CD project. As an alternative, you can write automation scripts that call the Fabric REST APIs to create tenant-level Fabric items, such as workspaces, connections, gateways, and deployment pipelines. Using Fabric REST APIs also makes it possible to fully automate the process of setting up Git integration for a Fabric CI/CD project. You create a Git source control connection and then use that connection to bind workspaces to Git branches.

Fabric REST API programming provides the most control for managing the lifecycle of workspace items. This is due to the ability to work directly with item definitions when calling CRUD APIs to create and update workspace items. This makes it possible to deploy a Fabric solution to a target workspace by using a repeatable process that creates workspace items with the correct relationships to other items in the same workspace. For example, you can create a lakehouse and a notebook in such a way that you configure the notebook to use the new lakehouse as its default lakehouse.

You can use the Fabric REST APIs to control Git synchronization. The `Update from Git` API can initialize or update a feature workspace from an underlying Git branch. It's also common to call `Update from Git` on the **dev** workspace after pull requests merge feature branch changes into the integration branch. The complementary `Commit to Git` API supports scenarios in which you need to automate the commit operation to push workspace item changes to its underlying Git branch.

For API fundamentals, see Fabric REST APIs.

If you program directly against the Fabric REST APIs, you need to understand three essential concepts: *paginated results*, *long-running operations (LROs)*, and *throttling*. These concepts help

you write correct and reliable code:

Expand table

Fabric REST API concept	Why automation code must handle it	Reference
Paginated results	List operations can return partial result sets that require continuation requests.	Fabric REST API pagination
Long-running operations (LROs)	Create, update, and deployment calls can return an operation to poll before results are available.	Fabric REST API long-running operations
Throttling	High-volume automation can receive throttling responses and must retry according to service guidance.	Fabric REST API throttling

Developer tools, such as Terraform and Fabric CLI, automatically handle paginated results, long-running operations, and throttling.

Microsoft offers two Software Development Kits (SDKs) for the Fabric REST APIs. Microsoft provides a *Fabric .NET SDK* for C# developers and a *Fabric Python SDK* for Python developers. These SDKs reduce repetitive code by abstracting low-level details for executing HTTP requests, transmitting access tokens, and converting between JSON and strongly typed objects. The two SDKs also automatically handle paginated results, long-running operations, and throttling errors.

The following listing shows basic example code for using the Fabric Python SDK to create a new workspace. This code begins by creating a `ClientSecretCredential` object that authenticates as a service principal. Then the code creates a `FabricClient` object named `fabric_client`, which exposes methods to execute calls to Fabric REST API endpoints. The code in the listing creates a workspace by calling `create_workspace`. It then calls `list_workspaces` and uses the result to enumerate the existing set of workspaces.

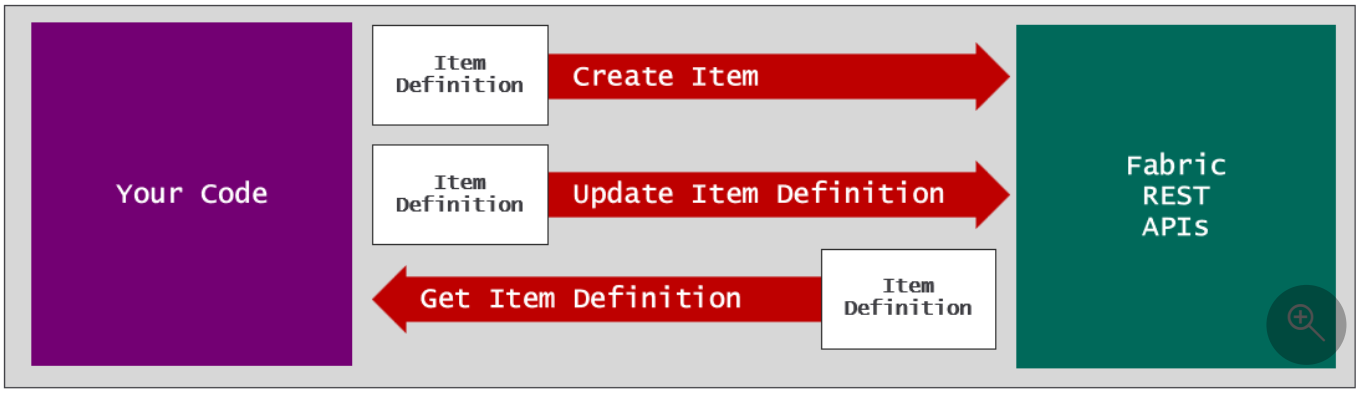
Python

```python
"""Basic example for Fabric REST API Python SDK"""

import os
from azure.identity import ClientSecretCredential
from microsoft_fabric_api import FabricClient

credential = ClientSecretCredential(
    tenant_id=os.getenv("AZURE_TENANT_ID"),
    client_id=os.getenv("AZURE_CLIENT_ID"),
    client_secret=os.getenv("AZURE_CLIENT_SECRET")
)
```

```
fabric_client = FabricClient(credential)

# Create a request body for creating a new workspace.
create_workspace_request = {
    "displayName": "Hello Python SDK",
    "description": "Sample workspace",
    "capacityId": os.getenv("FABRIC_CAPACITY_ID")
}

# Call the Create Workspace API.
workspace = fabric_client.core.workspaces.create_workspace(create_workspace_request)

# Enumerate workspaces by calling the List Workspaces API.
workspaces = fabric_client.core.workspaces.list_workspaces()
for workspace in workspaces:
    print(f'{workspace.display_name} - [{workspace.id}]')
```

For more SDK details, see the Microsoft Fabric Python SDK on PyPI and the Microsoft Fabric .NET SDK on NuGet.

The Fabric REST APIs offer three primary CRUD APIs that involve programming with item definitions. The **Create Item** API accepts an item definition when creating a new item. The **Update Item Definition** API accepts an item definition when updating an existing item. The **Get Item Definition** API retrieves the item definition for an existing item.



For more details, see Create Item API, Update Item Definition API, and Get Item Definition API.

Programming with item definitions and passing them to Fabric REST API endpoints requires a more complicated, low-level style of programming. The following example shows what's involved. Imagine that you need to program the Fabric REST APIs to deploy a Fabric solution by using the following sequence of API calls.

- Call the **Create Workspace** API to create a workspace.
- Call the **Create Item** API to create a lakehouse inside the workspace.
- Call the **Create Item** API to create a notebook that uses the lakehouse as its default.

The call to **Create Workspace** creates a workspace and returns the workspace ID. The first call to **Create Item** creates a lakehouse and returns the lakehouse ID. The second call to **Create Item** must create a notebook that uses the lakehouse as its default.

Suppose you want to create the notebook by using an item definition that includes a file named `notebook-content.py` with Python code. As this article discussed earlier, `notebook-content.py` contains metadata at the top that tracks the configuration of its default lakehouse. The Fabric service hides the metadata from users when they open the notebook in the web-based notebook editor that the Fabric service provides. However, you can view this metadata by examining the raw file contents of `notebook-content.py`, as the following listing shows.

Python

```
# Fabric notebook source

# METADATA ********************

# META {
# META   "dependencies": {
# META     "lakehouse": {
# META       "default_lakehouse": "{LAKEHOUSE_ID}",
# META       "default_lakehouse_name": "sales",
# META       "default_lakehouse_workspace_id": "{WORKSPACE_ID}",
# META       "known_lakehouses": []
# META     }
# META   }
# META }
```

Before calling **Create Item**, you need to update the contents of `notebook-content.py` with the correct workspace ID and lakehouse ID. This means your code must track the workspace ID and lakehouse ID to perform a find-and-replace operation on the contents of `notebook-content.py`.

> ⊗ **Caution**
>
> Directly manipulating item definition files assumes you know the item definition schema. If you update an item definition file with invalid syntax, you will experience errors when calling **Create Item** and **Update Item Definition**.

When programming with an item definition, you need to enumerate each of its files to load its contents and relative file paths into memory. Once your code loads the contents of `notebook-content.py` into memory, you can perform the search-and-replace operation to update it with the correct metadata for its default lakehouse.

The next step is to prepare the item definition so you can pass it across the network. This involves assembling a JSON payload for the body of an HTTP request. You need to pass the contents of multiple files across the network in a single HTTP request. To accomplish this, convert the contents of each file into a base64-encoded format. By formatting file contents by using base64 encoding, you can add the content of any file to a JSON element as a standard string value. This makes it possible to transmit all the files an item definition requires across the network in a single call to the **Create Item** API.

The following listing shows an example of JSON in the request body in a call to **Create Item**. The `definition` element contains a `parts` collection that contains a part element for each file in the item definition. Each part element includes a relative `path` value and a `payload` value for the file contents in a base64-encoded format.

JSON

```
{
  "displayName": "Create Lakehouse Tables",
  "type": "Notebook",
  "definition": {
    "parts": [
      {
        "path": ".platform",
        "payload": "<base64-encoded-file-content>",
        "payloadType": "InlineBase64"
      },
      {
        "path": "notebook-content.py",
        "payload": "<base64-encoded-file-content>",
        "payloadType": "InlineBase64"
      },
      {
        "path": "notebook-settings.json",
        "payload": "<base64-encoded-file-content>",
        "payloadType": "InlineBase64"
      }
    ]
  }
}
```

Another common scenario is programming the Fabric REST APIs to update an existing workspace item. For example, you might need to update the metadata for an existing notebook to switch its default lakehouse from one lakehouse to another. You can accomplish this by using the following sequence.

- Call **Get Item Definition** to retrieve a JSON response with the current item definition.
- Extract the `notebook-content.py` file content and convert it from base64 to plain text.

- Perform a find-and-replace operation on `notebook-content.py` to update the workspace ID and lakehouse ID.
- Convert the updated contents of `notebook-content.py` back to the base64-encoded format.
- Update the JSON element for the item definition with the base64-encoded content for `notebook-content.py`.
- Call **Update Item Definition**, passing the JSON with the updated item definition.

Fabric REST API programming provides more control than the Fabric developer tools and libraries this article discussed earlier, but it also adds complexity. When you use Terraform, Fabric CLI, Semantic Link Labs, or `fabric-cicd`, these tools and libraries handle the work of converting file contents between base64 encoding and clear text automatically. These tools and libraries provide less direct control, but they make implementation easier.

Terraform and `fabric-cicd` also check whether target items already exist. They use these checks to determine whether to create a new item or update an existing item. To implement the same logic with the Fabric REST APIs, the implementation first calls the **Get Items** API to determine whether a target item exists. It must follow that with conditional logic that determines whether to call **Create Item** or **Update Item Definition**.

The Fabric REST APIs offer both *item-generic API endpoints* and *item-specific API endpoints*. You've already seen examples of using item-generic APIs, such as **Create Item** and **Update Item Definition**. However, the Fabric REST APIs also offer item-specific APIs, such as **Create Lakehouse** and **Create Notebook**. This raises the question of whether you should use item-generic APIs or item-specific APIs.

For creating, updating, and deleting workspace items, it doesn't matter whether you use the item-generic APIs or item-specific APIs because the result is the same. For this reason, you might prefer using item-generic APIs to create and update workspace items because they allow you to write generic code that you can reuse with any type of workspace item.

The only time where it's essential to use item-specific APIs is when you need to query a workspace item to retrieve its properties. As an example, compare the results of calling **Get Item** versus **Get Lakehouse**. You call **Get Item** by executing an HTTP GET request to a URL that targets the **items** endpoint for a specific workspace.

HTTP

```
https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/items/{itemId}
```

The HTTP response from **Get Item** returns a JSON element that contains properties common to all types of workspace items.

JSON

```
{
  "displayName": "sales",
  "description": "Container for sales data",
  "type": "Lakehouse",
  "workspaceId": "{WORKSPACE_ID}",
  "id": "{LAKEHOUSE_ID}"
}
```

Now compare this to a call to **Get Lakehouse**, which uses a URL targeting the **lakehouses** endpoint for a workspace.

HTTP

```
https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/lakehouses/{lakehouseId}
```

A call to **Get Lakehouse** returns a JSON response with a `properties` element containing additional properties that are specific to lakehouses. The following JSON listing shows the custom lakehouse properties that include data-source paths used to access lakehouse data through OneLake or its SQL endpoint.

JSON

```
{
  "displayName": "sales",
  "description": "Container for sales data",
  "type": "Lakehouse",
  "workspaceId": "{WORKSPACE_ID}",
  "id": "{LAKEHOUSE_ID}",
  "properties": {
    "oneLakeTablesPath": "https://onelake.dfs.fabric.microsoft.com/{workspaceId}-
{ItemId}/Tables",
    "oneLakeFilesPath": "https://onelake.dfs.fabric.microsoft.com/{workspaceId}-
{ItemId}/Files",
    "sqlEndpointProperties": {
      "connectionString": "
{WORKSPACE_UNIQUE_PATH}.datawarehouse.fabric.microsoft.com",
      "id": "{LAKEHOUSE_ID}",
      "provisioningStatus": "Success"
    }
  }
}
```

Fabric bulk import and export APIs overview

The Microsoft Fabric Bulk Import Item Definitions API and Bulk Export Item Definitions API move multiple Fabric item definitions in one request. These Fabric APIs capture, copy, and restore Fabric workspace items as JSON-based item definitions instead of processing one item at a time.

Use Bulk Import Item Definitions and Bulk Export Item Definitions when you need to back up a Fabric workspace, clone items between workspaces, migrate items across tenants, or build custom lifecycle workflows for Git providers that Fabric doesn't support natively.

In this article, you learn when to use Fabric bulk import and export APIs, how their request and response payloads work, and when to prefer `fabric-cicd` for release automation.

How do Fabric bulk import and export APIs support backup, restore, clone, and migration?

Fabric REST APIs, such as Create Item API, Update Item Definition API, and Get Item Definition API, support CRUD operations that use item definitions. These single-item APIs process one item definition per call. The Bulk Import Item Definitions API and Bulk Export Item Definitions API, which entered public preview in March 2026, extend that model by sending multiple item definitions in a single API call for batch processing.

Use the following scenario map to choose the correct bulk import and export workflow:

Expand table

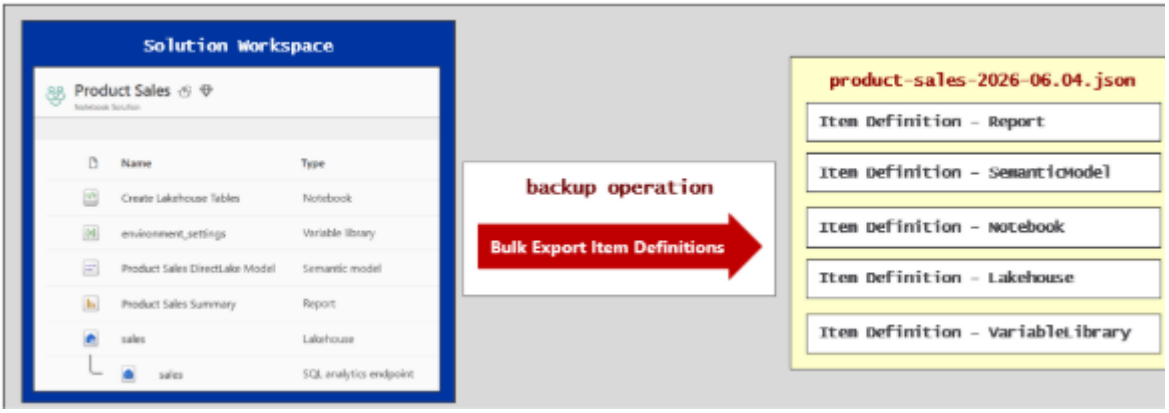
Scenario	Use Bulk Export Item Definitions to	Use Bulk Import Item Definitions to	Persist the JSON output?
Backup and restore	Export item definitions from a source Fabric workspace.	Restore those definitions into the same workspace or a replacement workspace.	Yes. Save the JSON in Git or cloud storage for disaster recovery.
Clone or refactor workspaces	Export selected item definitions from a source Fabric workspace.	Create matching items in one or more target workspaces.	Optional. Persist only if you need an audit trail or repeatable clone package.
Cross-tenant migration	Export item definitions from a workspace in the source Microsoft Entra ID tenant.	Import those definitions into a workspace in the target Microsoft Entra ID tenant.	Yes. Persist the JSON between tenant-specific authentication contexts.

Scenario	Use Bulk Export Item Definitions to	Use Bulk Import Item Definitions to	Persist the JSON output?
Custom Git synchronization	Convert exported JSON item definitions into Git folder files.	Convert Git folder files back into JSON item definitions for import.	Yes. The Git branch becomes the persisted authoritative version.

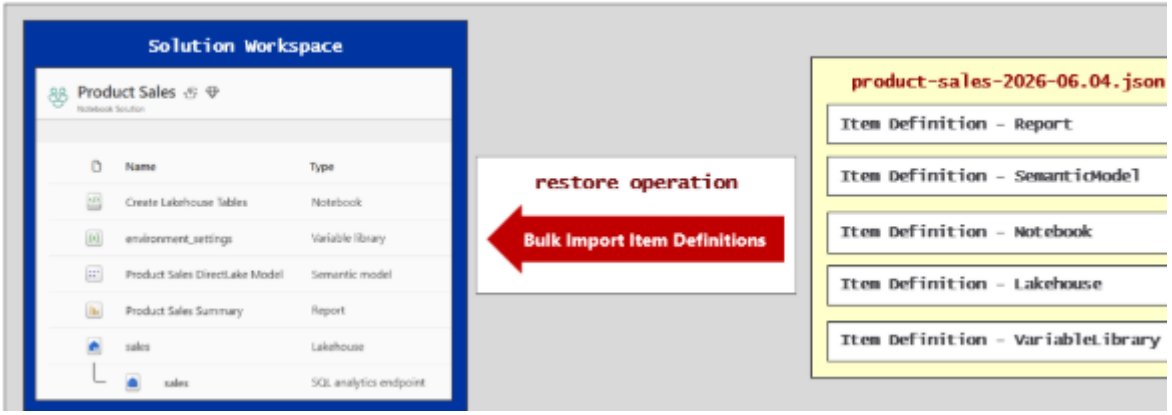
Back up and restore Fabric workspace items

Backup and restore support version control and disaster recovery. For backup and restore, Bulk Export Item Definitions generates JSON item definitions that you can persist to storage. Bulk Import Item Definitions can later use those definitions to create or update items in a target Fabric workspace.

In a simple backup-and-restore workflow for a single-workspace solution, Bulk Export Item Definitions returns a JSON result that contains an item definition for each item in the source workspace. You can save the result as a JSON file in a Git repository branch or in another type of cloud storage, such as an Azure Storage account container.

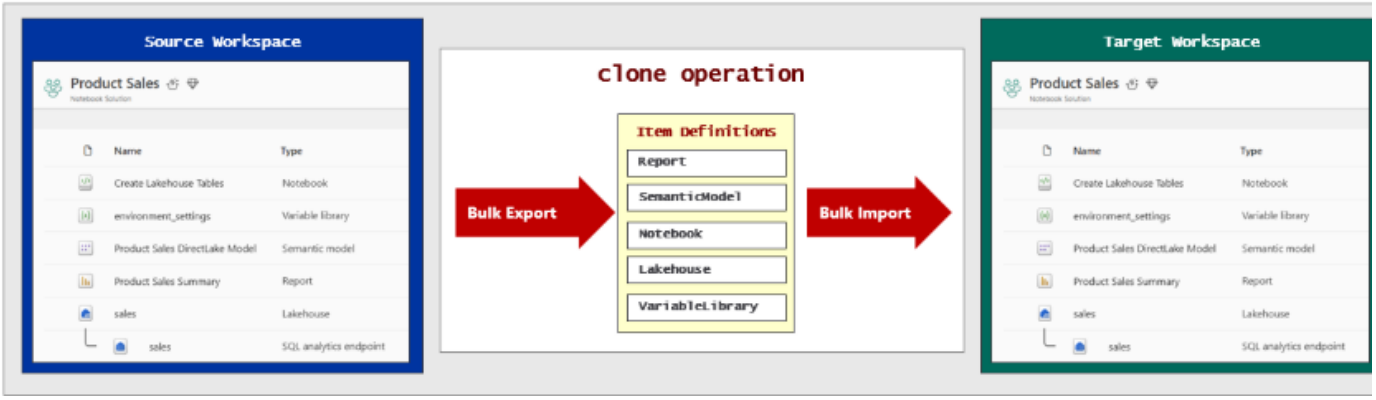


A restore operation uses Bulk Import Item Definitions with JSON that contains the list of item definitions. You can prefilter the item-definition list to limit the restore operation to a single item or to a subset of items.

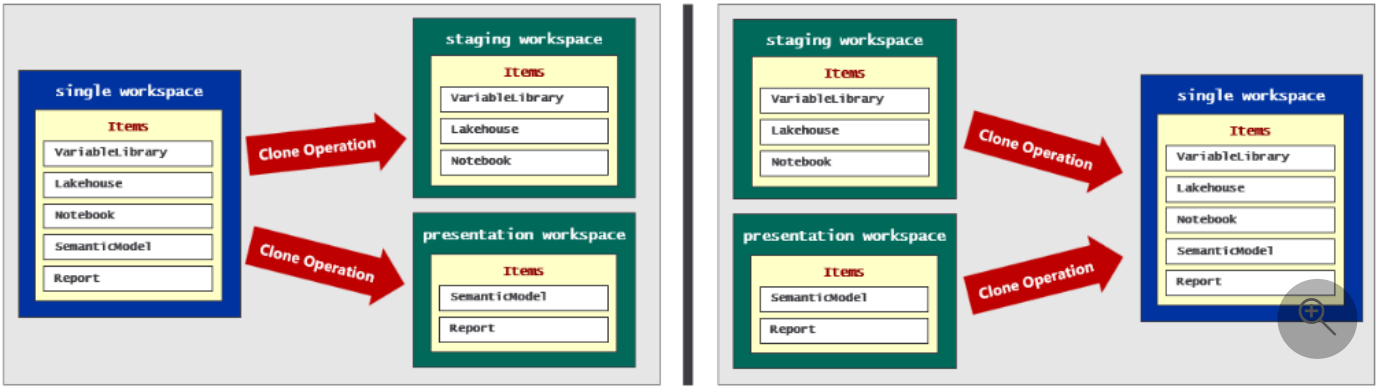


Clone and refactor Fabric workspace items

Cloning copies Fabric items from one workspace to another without deleting the original source items. Bulk Export Item Definitions retrieves item definitions for the items in a source Fabric workspace. Bulk Import Item Definitions then uses those definitions to clone the items in a target Fabric workspace. For clone-only workflows, you don't need to persist the JSON result that contains the item definitions as a file.



Selective cloning controls which items move from the source workspace to the target workspace. For example, you can refactor a Fabric solution from a single workspace to a multi-workspace layout for increased scalability or more granular access control. Alternatively, item cloning can reduce complexity by aggregating items from multiple workspaces into a single-workspace design.



A *clone operation* is different from a *move operation* because it doesn't delete the original source item. To delete the source item after a clone operation, use the Delete Item API as a separate step.

Rebind dependencies and run post-deploy tasks

For item types that support auto-binding, the bulk import and export APIs reestablish dependencies automatically. The Bulk Export Item Definitions API facilitates auto-binding by returning item definitions that track dependencies by using `logicalId` values instead of source-

workspace item IDs. Bulk Import Item Definitions discovers these `logicalId` values and uses them to rebind item dependencies in the target workspace.

The bulk import and export APIs don't eliminate the need for post-deploy jobs. A call to Bulk Import Item Definitions doesn't populate a lakehouse with data. After cloning a Fabric solution into a new workspace, post-deploy orchestration might still need to run a notebook or pipeline to populate data containers. Other post-deploy tasks can include activating a value set for a variable library or updating imported items that don't support variable libraries or auto-binding.

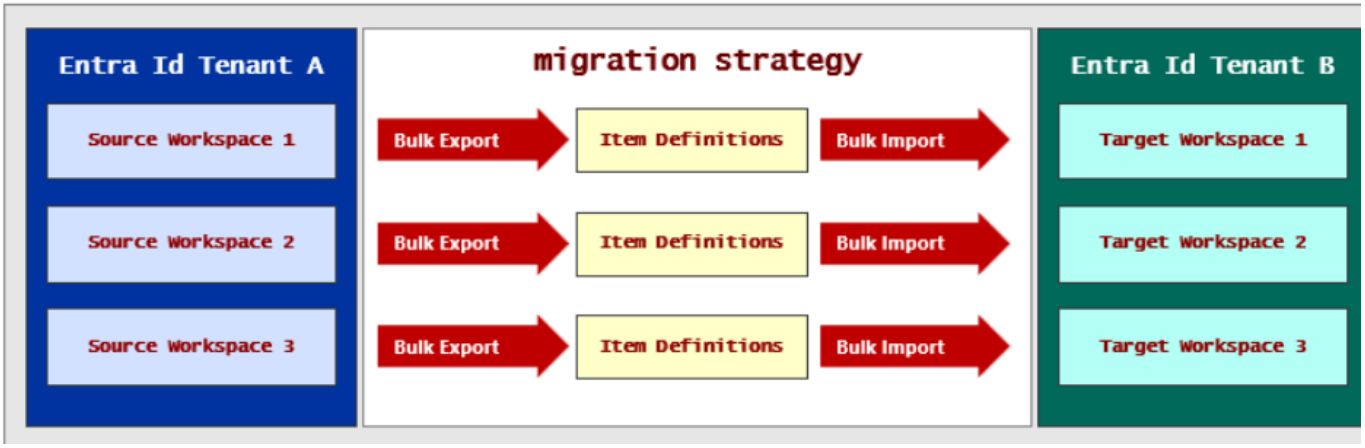
What don't bulk import and export APIs handle automatically?

Bulk import and export APIs don't automatically handle these tasks:

- Export or import unsupported workspace item types. When export mode is `All`, Fabric skips unsupported item types instead of returning them in the JSON package.
- Populate data containers, run notebooks or pipelines, activate value sets for variable libraries, or create shortcuts. These tasks require separate post-deploy orchestration or Fabric REST API calls.
- Resolve release-time parameterization dependencies, such as retrieving a new lakehouse SQL endpoint before a dependent semantic model import. These dependencies require sequencing imports or retrieving values before importing a dependent item.

Migrate Fabric items across Microsoft Entra ID tenants

Cross-workspace cloning also supports migration scenarios. The source workspace and target workspace don't need to exist in the same Microsoft Entra ID tenant. For example, an acquisition might require moving all items from workspaces in one Microsoft Entra ID tenant to new workspaces in a different Microsoft Entra ID tenant.



A cross-tenant migration requires separate authentication for each Microsoft Entra ID tenant. For example, a service principal in Microsoft Entra ID tenant A can call Bulk Export Item Definitions, while a different service principal in Microsoft Entra ID tenant B can call Bulk Import Item Definitions.

How does Bulk Export Item Definitions export Fabric workspace items?

The Bulk Export Item Definitions API uses an HTTP POST request to a source-workspace endpoint, where `{workspaceId}` is the Fabric workspace ID that contains the items to export. The request body controls whether Fabric exports all supported items in the workspace or only selected items.

HTTP

```
https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/items/bulkExportDefinitions?beta=true
```

> **Note**
>
> While the Bulk Import Item Definitions API and Bulk Export Item Definitions API are in preview, you must add a query string of `?beta=true`. You can remove this query string once the APIs reach general availability (GA).

The export request body uses these fields:

Expand table

Field	Required	Value	Purpose
`mode`	Yes	`All` or `Selective`	Controls whether Fabric exports all supported items or only the item IDs listed in `items`.
`items[].id`	Required when `mode` is `Selective`	Fabric item ID	Identifies each workspace item to export.

To export all supported items in the source workspace, set `mode` to `All`.

JSON

```
{ "mode": "All" }
```

The bulk import and export APIs only support workspace item types that offer CRUD APIs that use item definitions. When `mode` is `All`, Fabric skips unsupported workspace item types without returning an error.

To export selected workspace items, set `mode` to `Selective` and add an `items` list that contains the `id` values for the items to export.

JSON

```
{
    "mode": "Selective",
    "items": [
        { "id": "a0a0a0a0-bbbb-cccc-dddd-e1e1e1e1e1e1" },
        { "id": "b1b1b1b1-cccc-dddd-eeee-f2f2f2f2f2f2" },
        { "id": "c2c2c2c2-dddd-eeee-ffff-a3a3a3a3a3a3" }
    ]
}
```

For large requests, calls to the Bulk Export Item Definitions API and Bulk Import Item Definitions API can run asynchronously as *long-running operations (LROs)*. If you're not using one of the Fabric REST API SDKs, your implementation must monitor LRO progress and retrieve the return value upon completion.

A Bulk Export Item Definitions API response contains these top-level lists:

Expand table

Response list	Purpose
`itemDefinitionsIndex`	Maps each exported item to its relative `rootPath` and source-workspace item `id`.
`definitionParts`	Contains the exported item-definition files, including each file `path`, base64-encoded `payload`, and `payloadType`.

JSON

```
{
    "itemDefinitionsIndex": [],
    "definitionParts": []
}
```

The `itemDefinitionsIndex` list contains metadata for each exported item. The Bulk Export Item Definitions API generates the relative `rootPath` from the item display name and item type in the format `/[Item Display Name].[Item Type]`. The `id` value maps each exported item definition back to its item of origin in the source workspace.

JSON

```
{
    "itemDefinitionsIndex": [
        {
            "id": "a0a0a0a0-bbbb-cccc-dddd-e1e1e1e1e1e1",
            "rootPath": "/Product Sales Summary.Report"
        },
        {
            "id": "b1b1b1b1-cccc-dddd-eeee-f2f2f2f2f2f2",
            "rootPath": "/Product Sales DirectLake Model.SemanticModel"
        }
    ]
}
```

The `definitionParts` list contains an element for each item-definition file. Each element represents a file with a relative `path` value, a `payload` value that holds the file contents in base64-encoded format, and a `payloadType` value, such as `InlineBase64`.

```
{
    "itemDefinitionsIndex": [ ... ],
    "definitionParts": [
        {
            "path": "/Product Sales Summary.Report/.platform",
            "payload":"[base64 encoded content]"
            "payloadType": "InlineBase64"
        },
        {
            "path": "/Product Sales Summary.Report/definition.pbir",
            "payload":"[base64 encoded content]"
            "payloadType": "InlineBase64"
        },
        {
            "path": "/Product Sales Summary.Report/definition/version.json",
            "payload":"[base64 encoded content]"
            "payloadType": "InlineBase64"
        },
        {
            "path": "/Product Sales Summary.Report/StaticResources/SharedResources/BaseThemes/CY24SU10.json",
            "payload":"[base64 encoded content]"
```

How does Bulk Import Item Definitions import Fabric item definitions?

The Bulk Import Item Definitions API uses an HTTP POST request to a target-workspace endpoint, where `{workspaceId}` is the Fabric workspace ID that receives the imported items. The request body contains item-definition files that Fabric assembles into item definitions for batch processing.

HTTP

```
https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/items/bulkImportDefinitions?beta=true
```

The import request body uses these fields:

Expand table

Field	Required	Value	Purpose
`definitionParts`	Yes	List of item-definition files	Provides the files that Fabric assembles into one or more item definitions.
`definitionParts[].path`	Yes	Relative item-definition file path	Identifies the item folder and file name, such as `/Product Sales Summary.Report/.platform`.
`definitionParts[].payload`	Yes	Base64-encoded file content	Provides the contents of the item-definition file.
`definitionParts[].payloadType`	Yes	`InlineBase64`	Tells Fabric how to interpret the `payload` value.
`options.allowPairingByName`	No	`true` or `false`	Allows Fabric to update an existing target item by matching display name and type when `logicalId` values differ.

At minimum, the request body must include a `definitionParts` list of item-definition files.

JSON

```
{
    "definitionParts": [
        {
            "path": "/Product Sales Summary.Report/.platform",
            "payload": "<base64-encoded-content>",
            "payloadType": "InlineBase64"
        },
        {
            "path": "/Product Sales Summary.Report/definition.pbir",
            "payload": "<base64-encoded-content>",
            "payloadType": "InlineBase64"
        },
        {
            "path": "/Product Sales Summary.Report/definition/report.json",
            "payload": "<base64-encoded-content>",
            "payloadType": "InlineBase64"
        }
    ]
}
```

Fabric dynamically assembles the item-definition files into distinct item definitions and processes each definition independently. For each definition, Fabric checks whether another item in the target workspace has the same display name and type. If no match exists, Fabric creates a new item. If a match exists, Fabric evaluates the `logicalId` before deciding whether to update the item or return an error.

Fabric tracks an internal `logicalId` property for each item. When the display name and type match an existing item, Fabric checks whether the item definition and target item have the same `logicalId`. If the values match, Fabric updates the target item. Backup-and-restore workflows commonly use this behavior because the exported item definition retains the source item's `logicalId`.

If the display name and type match but the `logicalId` values differ, the Bulk Import Item Definitions API returns a `DuplicateDisplayNameAndType` exception by default. To update the existing item by name despite the mismatch, add an `options` element with `allowPairingByName` set to `true`.

JSON

```
{
    "definitionParts": [
        {
            "path": "/Product Sales Summary.Report/.platform",
            "payload": "<base64-encoded-content>",
            "payloadType": "InlineBase64"
        }
    ],
    "options": {
        "allowPairingByName": true
    }
}
```

A successful call to the Bulk Import Item Definitions API returns JSON with an `importItemDefinitionsDetails` list. The response fields describe each import operation:

Expand table

Response field	Purpose
`itemId`	Identifies the created or updated item in the target workspace.
`itemDisplayName`	Shows the display name of the imported item.
`itemType`	Shows the Fabric item type, such as `Report` or `SemanticModel`.

Response field	Purpose
`itemLogicalId`	Identifies the logical item identity used for dependency rebinding.
`operationType`	Indicates whether Fabric created or updated the item.
`operationStatus`	Indicates whether the import operation succeeded or failed.

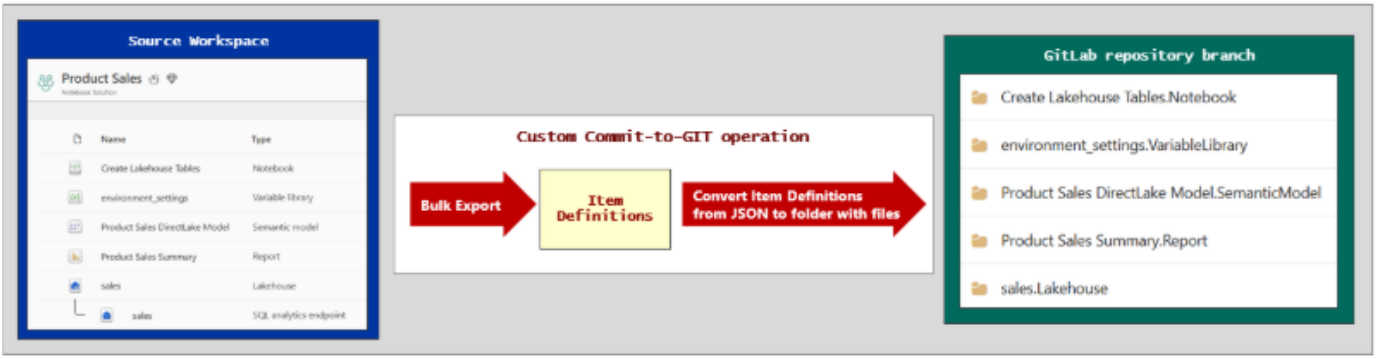
JSON


```
{
    "importItemDefinitionsDetails": [
        {
            "itemId": "a0a0a0a0-bbbb-cccc-dddd-e1e1e1e1e1e1",
            "itemDisplayName": "Product Sales Summary",
            "itemType": "Report",
            "itemLogicalId": "aaaaaaaa-0000-1111-2222-bbbbbbbbbbbb",
            "operationType": "Create",
            "operationStatus": "Succeeded"
        },
        {
            "itemId": "b1b1b1b1-cccc-dddd-eeee-f2f2f2f2f2f2",
            "itemDisplayName": "Product Sales DirectLake Model",
            "itemType": "SemanticModel",
            "itemLogicalId": "bbbbbbbb-1111-2222-3333-cccccccccccc",
            "operationType": "Update",
            "operationStatus": "Succeeded"
        }
    ]
}
```

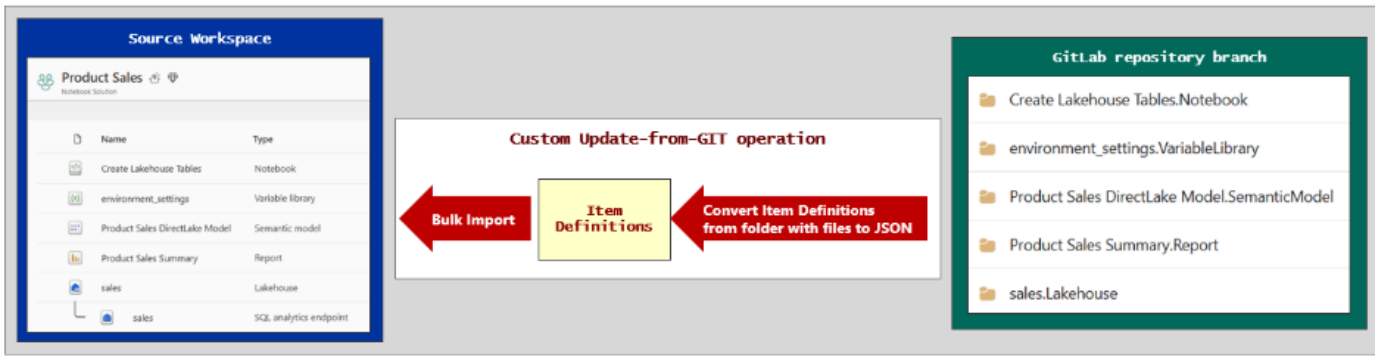

How can custom Git synchronization use Fabric bulk import and export APIs?

The bulk import and export APIs can support Git providers that Fabric doesn't natively support. You can use supported providers, such as Azure DevOps, GitHub, or GitHub Enterprise, or build custom synchronization logic if you require GitLab or another provider. The logic uses Bulk Export Item Definitions and Bulk Import Item Definitions to implement equivalents of the **Commit to Git** and **Update from Git** operations.

In a custom **Commit to Git** operation, Bulk Export Item Definitions retrieves item definitions from a source workspace in JSON format. Custom code enumerates the item-definition files, uses each `path` value to create a structured item-definition folder in a Git branch, and converts each base64-encoded `payload` value to clear text before saving it.



A custom **Update from Git** operation reverses the flow. Custom code enumerates files in the Git folder, builds a JSON request body with the correct `path` values, encodes file contents as base64 in each `payload`, and calls Bulk Import Item Definitions to import the item definitions into a target workspace.



When should you use the fabric-cicd library instead of bulk import and export APIs?

You can use the bulk import and export APIs in release processes for continuous integration and continuous delivery (CI/CD). However, you should usually prefer the fabric-cicd Python library. The `fabric-cicd` library provides deployment capabilities and conveniences that aren't available when programming directly against the bulk import and export APIs.

Use this decision table when choosing between `fabric-cicd` and the bulk import and export APIs for Fabric release automation:

Expand table

Requirement	Prefer	Why
Standard Fabric CI/CD deployment	`fabric-cicd`	The library provides built-in deployment orchestration, parameterization, post-deploy actions, and orphan control.

Requirement	Prefer	Why
Unsupported Git provider integration	Bulk Import Item Definitions and Bulk Export Item Definitions	The APIs let you build custom Commit to Git and Update from Git equivalents for providers, such as GitLab.
Backup, clone, or cross-tenant migration workflow	Bulk Import Item Definitions and Bulk Export Item Definitions	The APIs provide JSON item-definition packages that can be persisted, moved, and imported across workspaces or tenants.

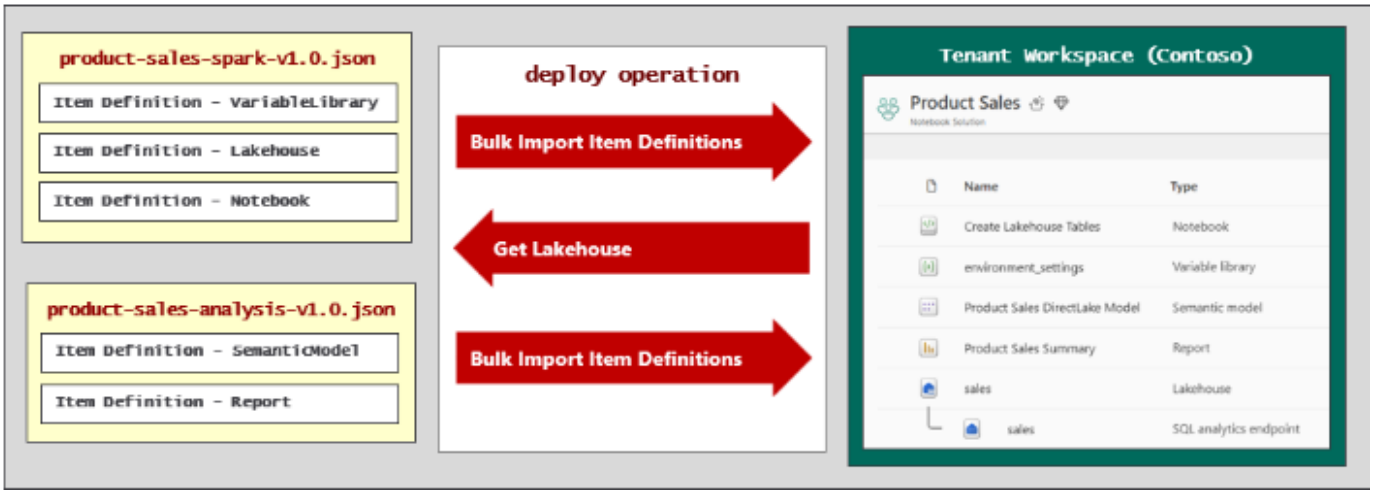
`fabric-cicd` provides configurable parameterization support, while the bulk import and export APIs require custom parameterization code. That code must convert file content between base64 encoding and clear text to run search-and-replace operations.

You can configure `fabric-cicd` with post-deploy actions that call Fabric REST APIs. For example, post-deploy actions can activate a value set after creating a variable library or create shortcuts after creating a lakehouse. `fabric-cicd` also supports orphan control to delete items when Git no longer contains their source item definition. With the bulk import and export APIs, these post-deploy actions require custom calls to Fabric REST APIs.

The `fabric-cicd` library also supports dynamic variables. For example, use the parameterization support in `fabric-cicd` to update the datasource path for a semantic model by using a dynamic lakehouse variable that exposes a property named `$sqlendpoint`. The `fabric-cicd` library enables dynamic variables for a lakehouse by calling the Get Lakehouse API as a post-deploy action to cache the SQL endpoint server path.

With the bulk import and export APIs, dynamic lakehouse variable deployment requires handling timing dependencies directly. A semantic model definition might need the lakehouse SQL endpoint path before import, but the endpoint isn't available until after the lakehouse exists. In that case, a single Bulk Import Item Definitions call can't import the lakehouse and semantic model. The release process must import them in separate batches.

The following diagram shows an example of deploying a Fabric solution with a lakehouse and a dependent semantic model. The first call to Bulk Import Item Definitions imports the first batch of items, which includes the lakehouse. After the first Bulk Import Item Definitions call creates the lakehouse, the release process calls the Get Lakehouse API to retrieve the server path for its SQL endpoint. The release process then uses the SQL endpoint server path to update the semantic model definition for the call to Bulk Import Item Definitions, which imports the second batch of items.



The main conclusion is to prefer `fabric-cicd` over the bulk import and export APIs for most Fabric CI/CD release processes. The `fabric-cicd` library provides a configurable, lower-code approach that handles many low-level Fabric REST API programming tasks. The bulk import and export APIs remain important for these lifecycle scenarios:

- Implement backup-and-restore capabilities without Git.
- Clone items across workspaces.
- Refactor item distribution across workspaces in a Fabric solution.
- Migrate items in Fabric solutions across Microsoft Entra ID tenant boundaries.
- Integrate a Git provider that Fabric doesn't natively support.

Automation summary

- Always execute automation as service principals, not user principals.
- Use **Terraform** at project startup to provision resources each environment requires.
- Use **Fabric CLI** or **Semantic Link Labs** to script post-sync jobs and post-deploy jobs.
- Use **Fabric REST APIs** to script post-sync jobs and post-deploy jobs with greater control and flexibility.
- Build release processes by using an API-driven approach that can adapt to any Git branching strategy.
- Implement an API-driven release process by using `fabric-cicd` with configuration-based deployment.
- Use `fabric-cicd` parameterization to dynamically update environment-specific settings during deployment.

How do you develop Fabric CI/CD workflows?

After you choose an approach for automation logic, decide where that code should live. Store the code that runs Fabric CI/CD processes in the same Git repository that holds the item definitions. This design keeps workflow logic, deployment configuration, and solution source files versioned together.

Fabric supports two primary Git providers: Azure DevOps and GitHub. Both Git providers offer the capability to run workflow jobs on demand, on a schedule, or in response to Git events, such as commits and pull requests. Both Git providers also provide a way to store variables, protect secrets, model deployment environments, and configure manual approvals that reviewers must complete before a workflow can deploy its changes to a target workspace.

Regardless of whether you use Azure DevOps or GitHub, a Fabric CI/CD repository commonly contains three categories of files.

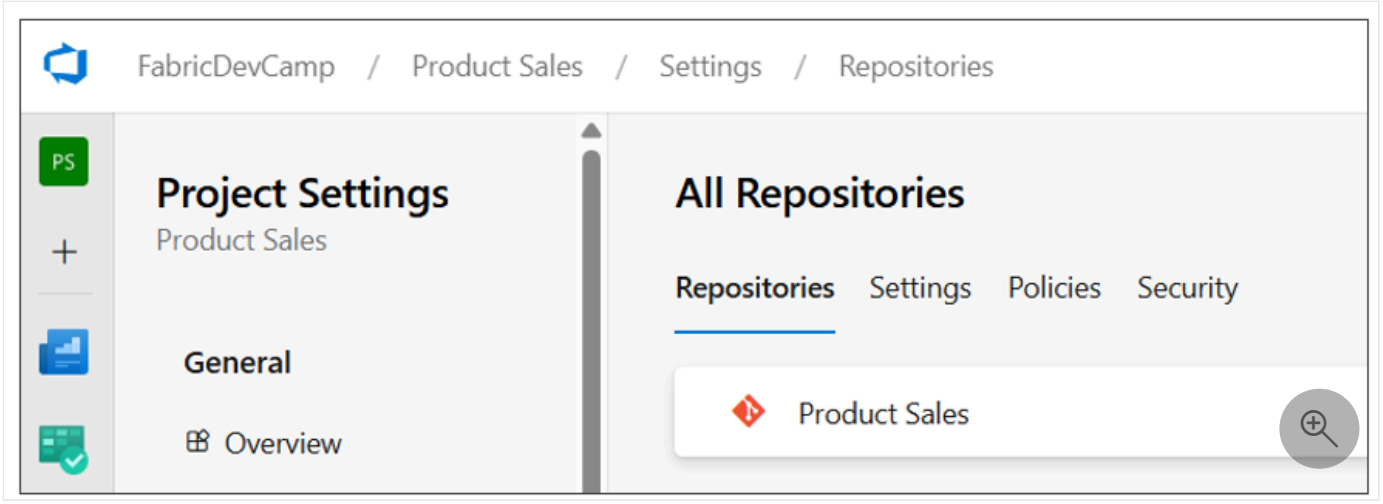
Expand table

Repository file category	Typical location	Purpose
Item definitions	`workspace` or another Git folder configured for Fabric synchronization.	Stores source files for Fabric workspace items.
Deployment scripts	`src` or another scripts folder.	Runs setup, synchronization, deployment, or post-deploy logic.
Workflow YAML files	`.pipelines` for Azure Pipelines or `.github/workflows` for GitHub Actions.	Defines the workflow jobs that invoke deployment scripts.

The YAML file content for defining workflows and folder names differs between Azure DevOps and GitHub. However, the goal is the same: source files for item definitions and files with automation logic move through review and release together. The following two sections examine the unique details of each Git provider.

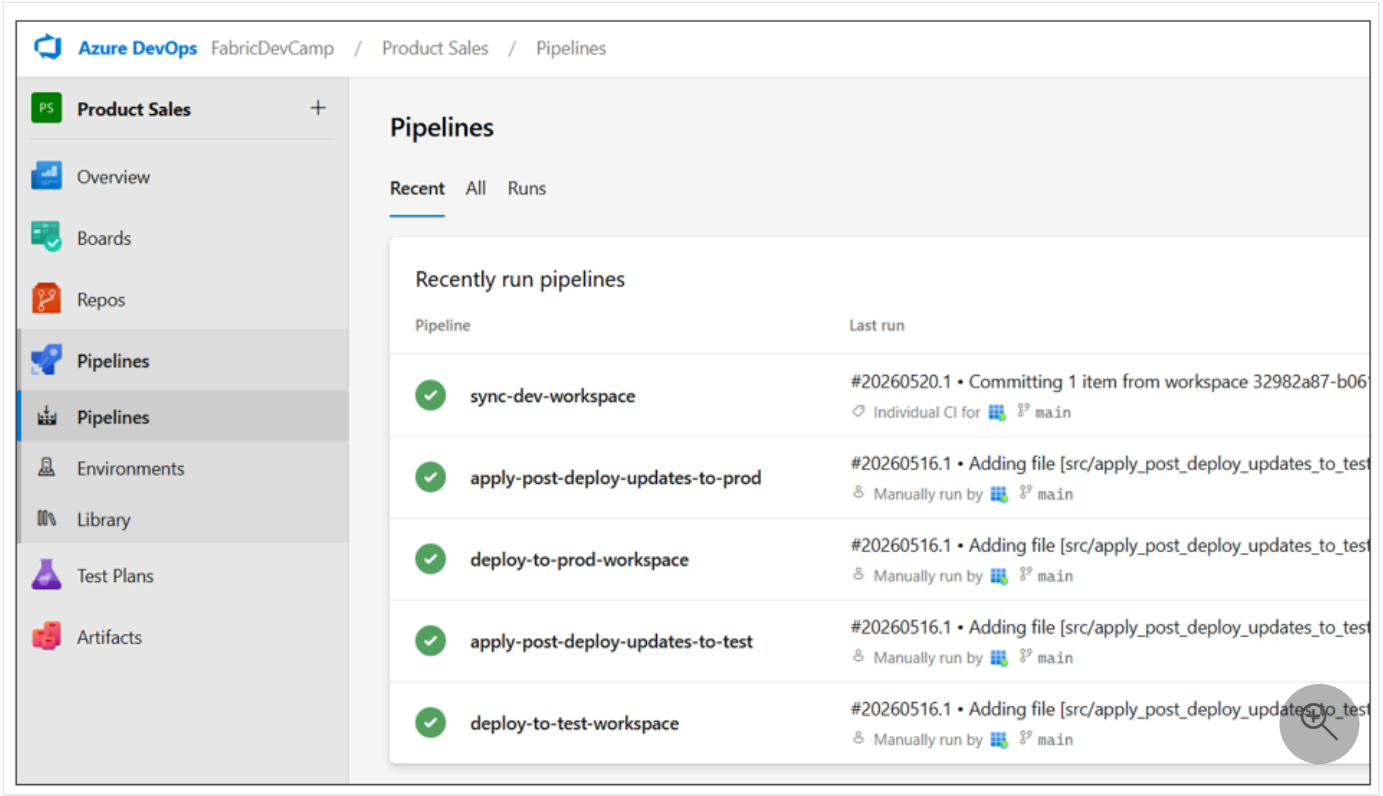
Azure Pipelines workflows

Azure DevOps is a Microsoft cloud platform that provides a suite of development tools for software teams to plan, build, test, and deploy applications. Microsoft designed it with CI/CD features to support the entire software development lifecycle. When you create a new project in Azure DevOps, Azure DevOps automatically creates the project with a Git repository of the same name. For example, creating a project named **Product Sales** will automatically create a repository inside the project named **Product Sales**.



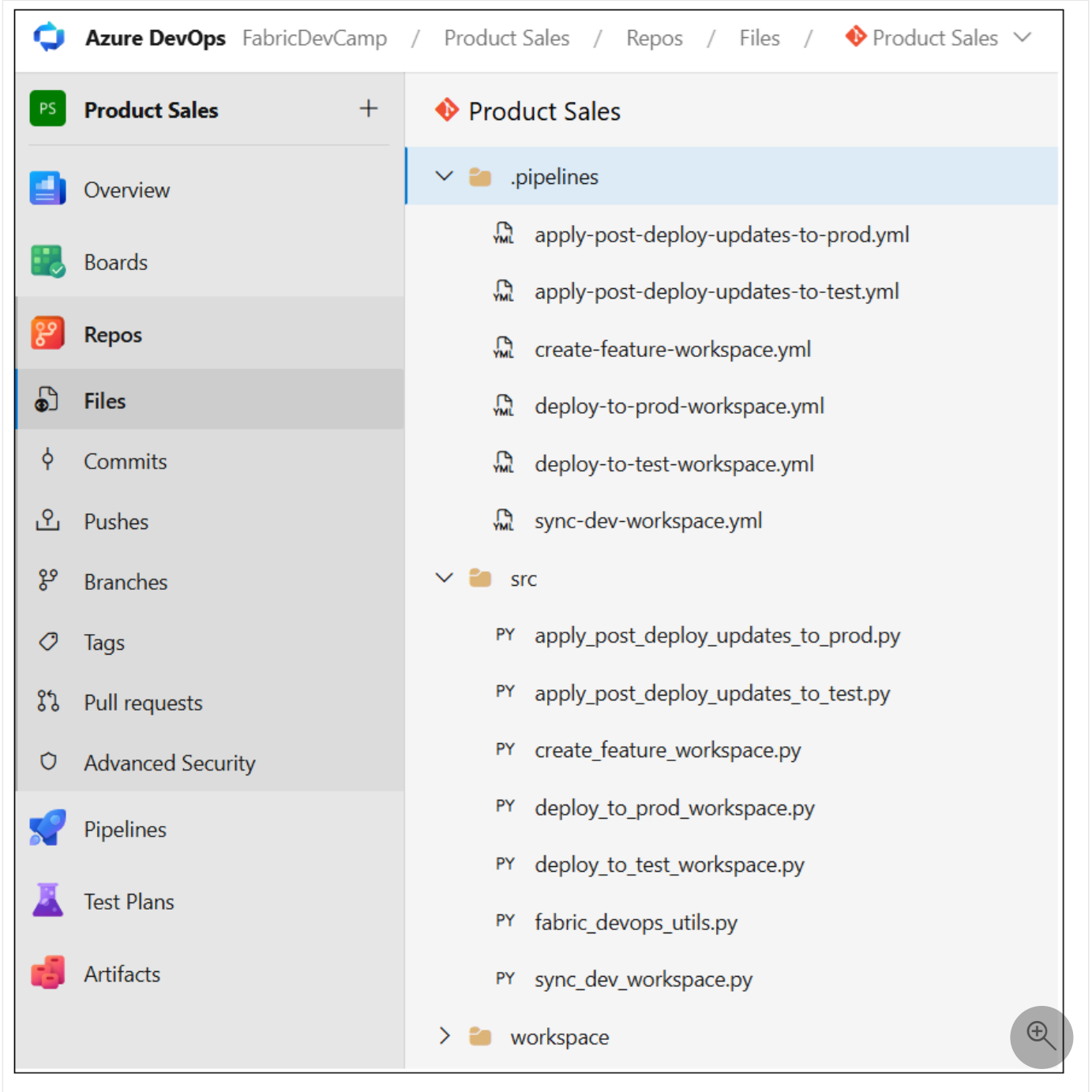
Azure DevOps includes *Azure Pipelines* as its workflow-development platform. Azure Pipelines provide the capability to integrate, test, and deploy code by using the principles of CI/CD. When you create an Azure pipeline, you can configure it to run on demand or on a schedule, such as every night at midnight. Alternatively, you can define an Azure pipeline with a trigger to run automatically in response to a code commit or the completion of a pull request.

To create an Azure pipeline, you must first add a YAML file with a pipeline definition into your repository. Once you have added the YAML file, you must then explicitly register it with the Azure DevOps project so Azure DevOps recognizes it as an Azure pipeline. You can complete this registration through the Azure DevOps user interface. You can automate it by using the Azure DevOps CLI or the Azure REST APIs. Once you register a pipeline, you should be able to see it in the **Pipelines** page in an Azure DevOps project.



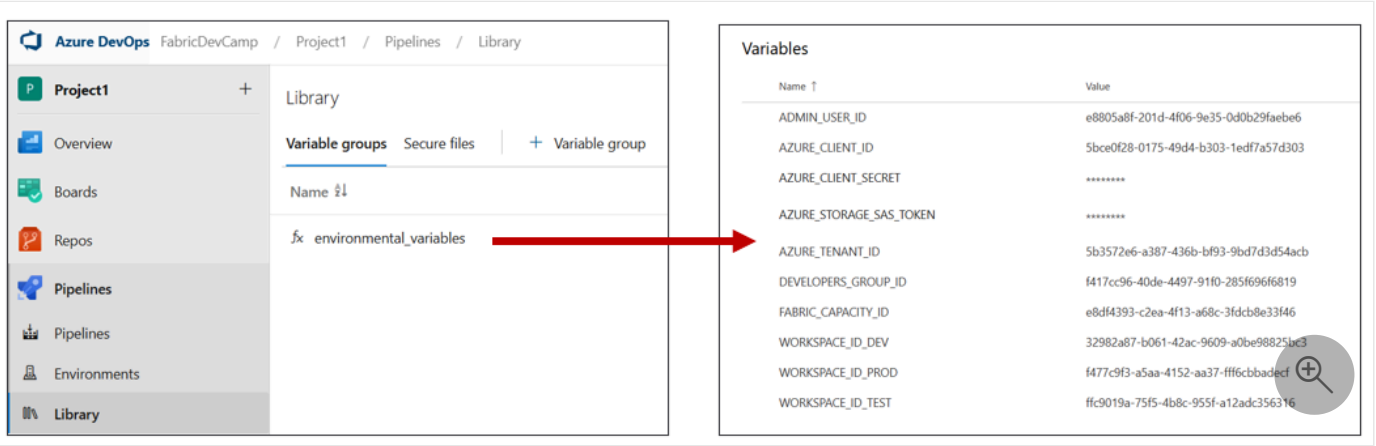
When you create an Azure pipeline, you can use different triggers for different parts of the CI/CD lifecycle. A validation pipeline can run any time a developer creates a new pull request to check files, run tests, or validate item definitions before reviewers approve the merge. You can configure a release pipeline to run after changes merge into a release branch or after a release manager starts it manually. Keeping validation and release responsibilities separate makes the workflow easier to govern.

You can write the code for an Azure pipeline in a programming language, such as Python. You do this by referencing a script with Python code from the YAML file. The following screenshot shows an example of YAML files for Azure pipelines in the **.pipelines** folder and their associated Python scripts in the **src** folder. Below these two folders you can also see the **workspace** folder that contains the item definitions for a specific Fabric solution.



As you begin to develop Azure pipelines for a Fabric CI/CD project, you'll find your code requires access to environment-specific settings, such as the IDs for workspaces, capacities, users, and groups. You might also need credentials for a Microsoft Entra ID application to authenticate as a service principal. Use the Azure Pipelines service to create variables and secrets that make these types of environment settings available to the code in your Azure pipelines.

To create variables and secrets that are accessible to Azure pipelines, create a *variable group*. Once you have created a variable group in the current Azure DevOps project, you can add a set of named variables. The following screenshot shows a typical set of variables and secrets for Azure pipelines in a Fabric CI/CD project.

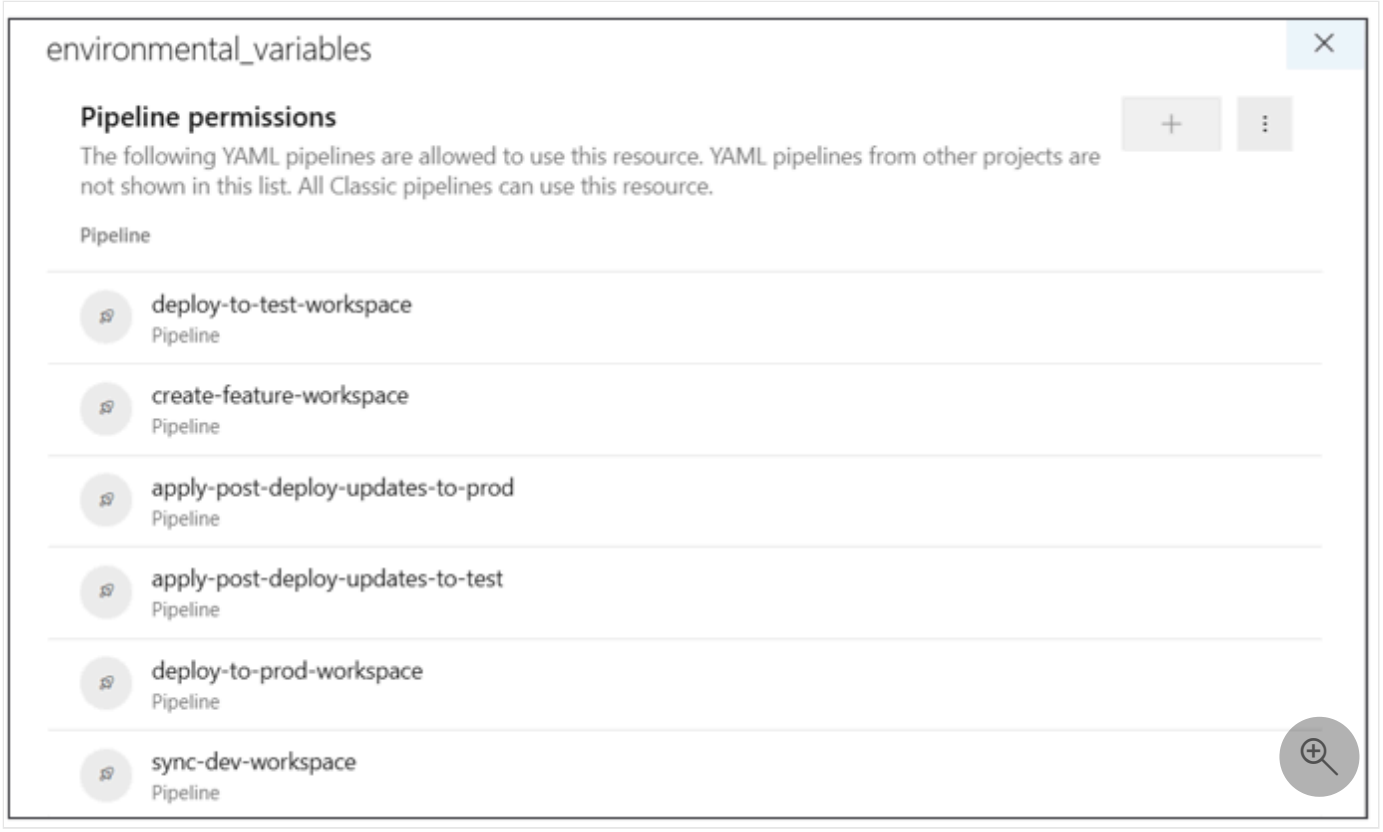


When you create a variable that contains sensitive data, you can mark that variable as a secret. For example, your pipeline might need access to a client secret for authenticating as a service principal. When you mark a variable as a secret, the Azure DevOps service protects its value in three ways: it persists the variable in the cloud by using an encrypted format rather than plain text, it prevents users from viewing the value in the Azure DevOps user interface after saving, and it masks the secret value in logs. If the secret value appears in pipeline output, Azure Pipelines replaces it with `***` so users won't see it in build logs.

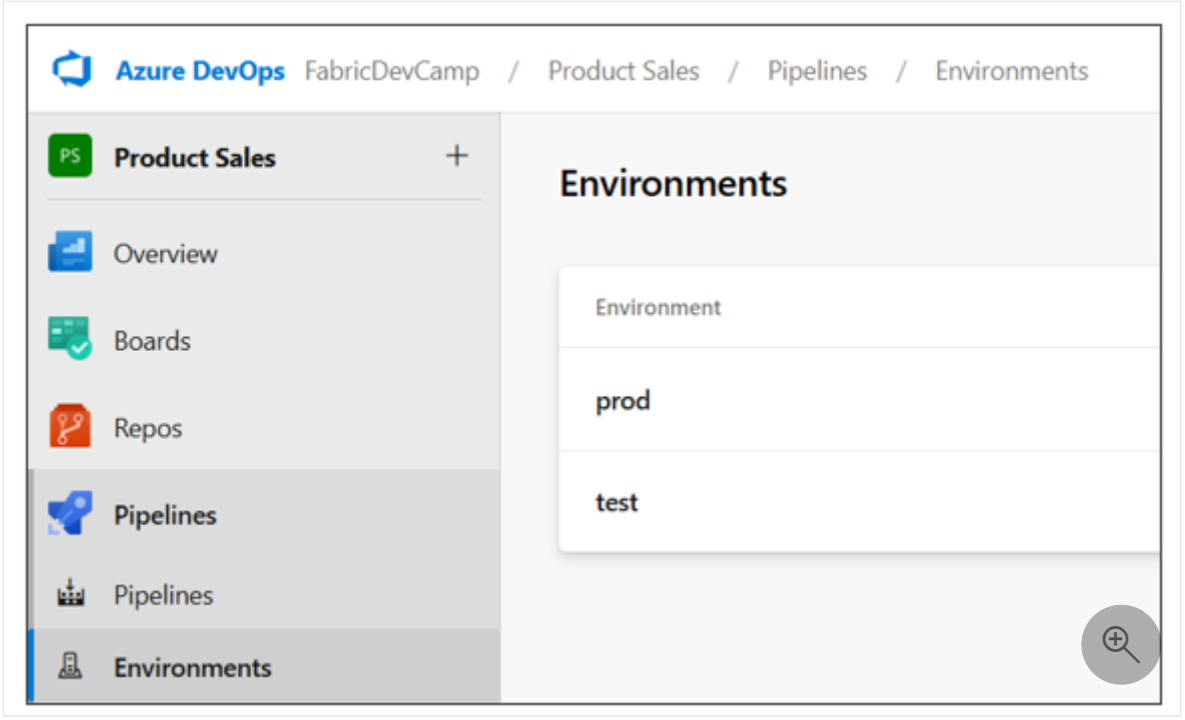
> **Important**
>
> You can integrate variable groups in Azure DevOps with Azure Key Vault. This integration makes it possible to access Azure Key Vault secrets from code running in an Azure pipeline. Microsoft recommends integrating a variable group with Azure Key Vault because you can take advantage of Azure Key Vault features, such as conditional access policies, auditing, and secret rotation.

By default, an Azure pipeline doesn't have access to the variables in a variable group. Instead, you must configure permissions for a variable group to ensure the code in your Azure pipelines has

access to its variables and secrets. You can configure these permissions by using the **Pipeline permissions** dialog in the Azure DevOps user interface or by using the Azure DevOps CLI or the Azure REST APIs.

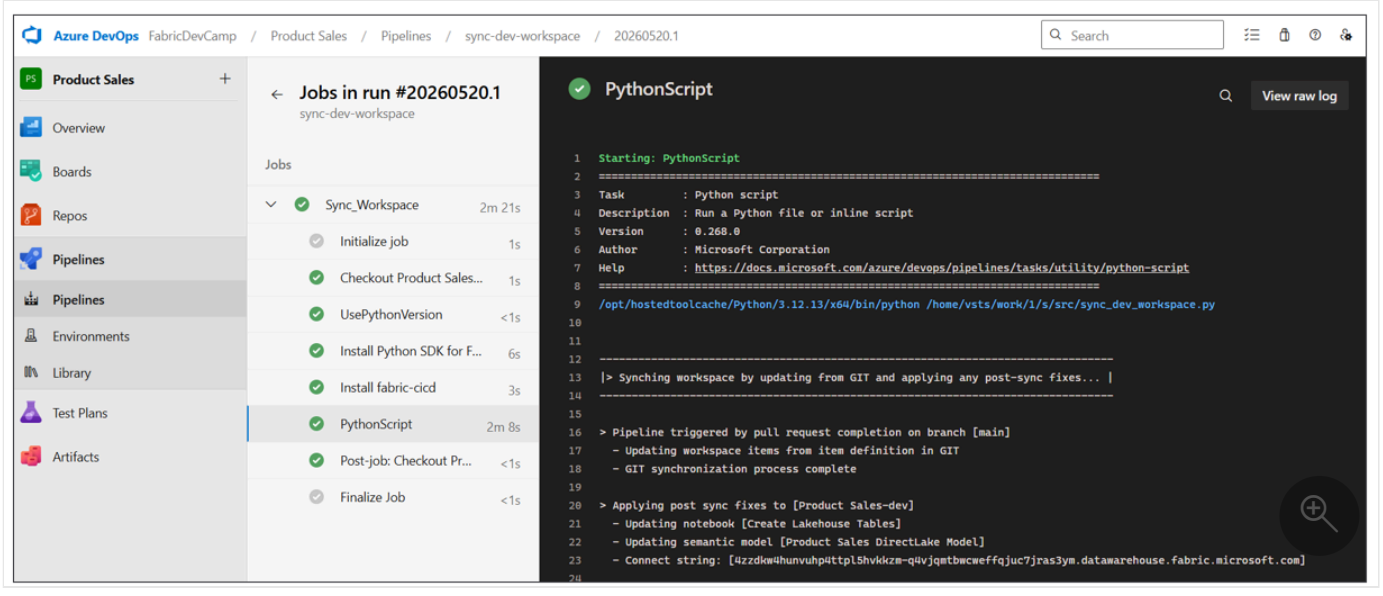


When building a release process by using an Azure pipeline, you often need to configure a manual-approval process. This can be complex with trunk-based development because the release process doesn't involve pull requests. However, you can solve this problem by creating **environments** in your Azure DevOps project with names, such as **test** and **prod**. Once you have created these two environments, you can configure each of them with a manual-approval process.



Once you create the two environments and configure them with a manual-approval process, you can create Azure pipelines that target them. When you run an Azure pipeline that targets one of these environments, the pipeline job pauses until the interactive approval process is complete. Once the approval process completes, the pipeline resumes its execution and can use the `fabric-cicd` library to deploy the new release to a target workspace.

This guidance helps you begin developing Azure pipelines for Fabric CI/CD projects. Become familiar with monitoring pipeline runs so you can test and debug your code. Add logging to your pipeline logic to report successful operations and provide diagnostic information about any errors that occur. The following screenshot shows how an Azure pipeline can log its progress in a Fabric CI/CD workflow.

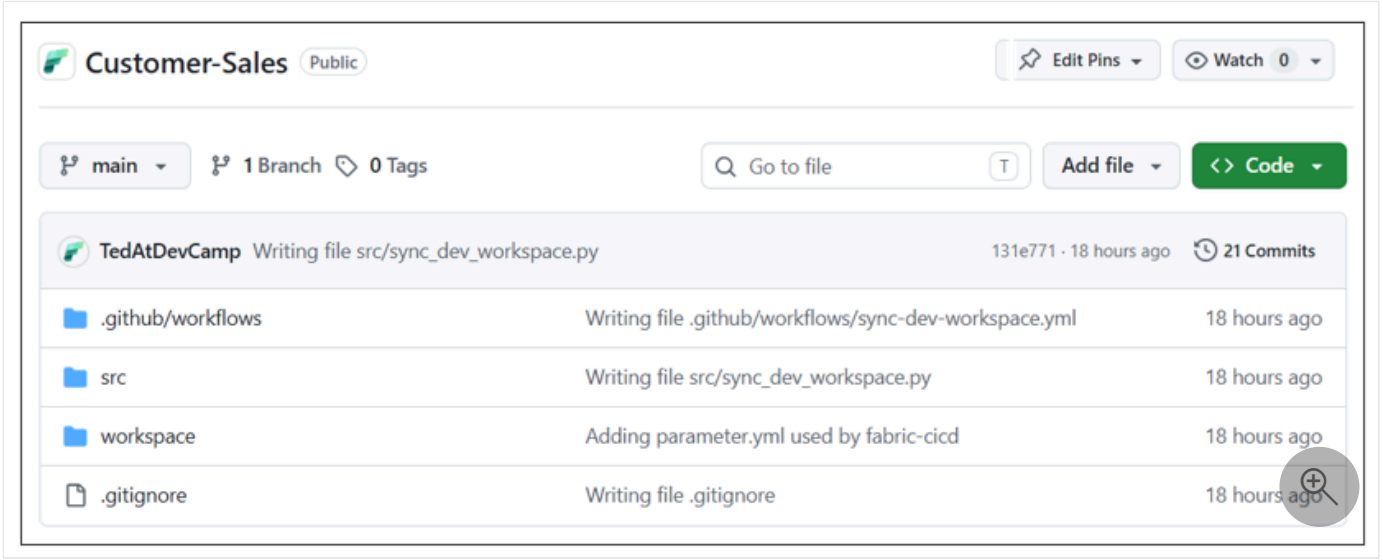


For more details on Azure DevOps and Azure pipelines, see Azure Pipelines documentation, Azure Pipelines key concepts, YAML pipeline editor, Manage variable groups, and Use Azure Key Vault secrets in Azure Pipelines.

For a complete tutorial, see Tutorial: CI/CD using Azure DevOps and the fabric-cicd library.

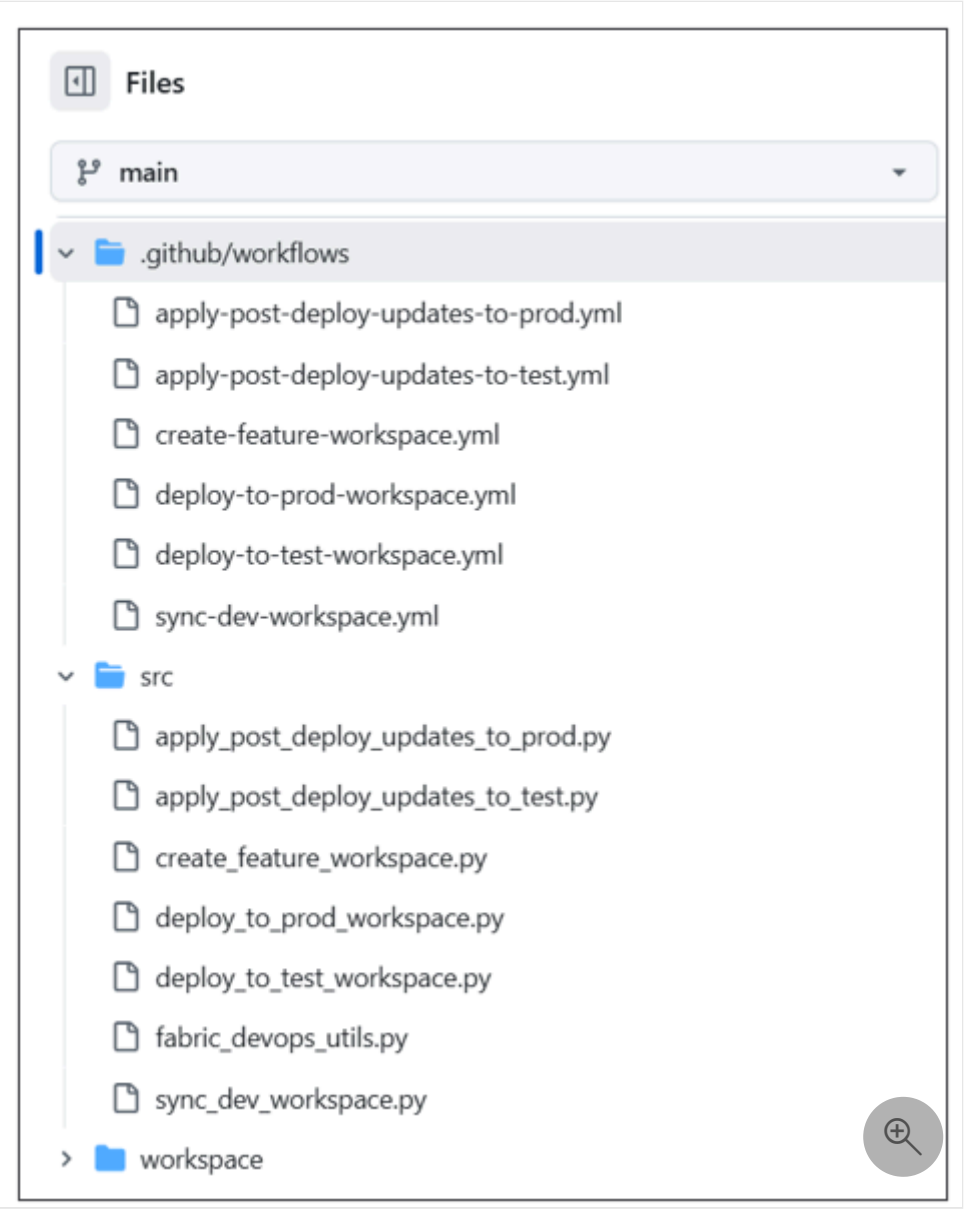
GitHub Actions workflows

GitHub is different from Azure DevOps with respect to how you work with projects and repositories. With GitHub, you can create repositories without first creating a project. While GitHub supports creating projects to assist with organizing repositories, the use of projects in GitHub is optional. When using GitHub for a Fabric CI/CD project, you only need to create a repository to begin.



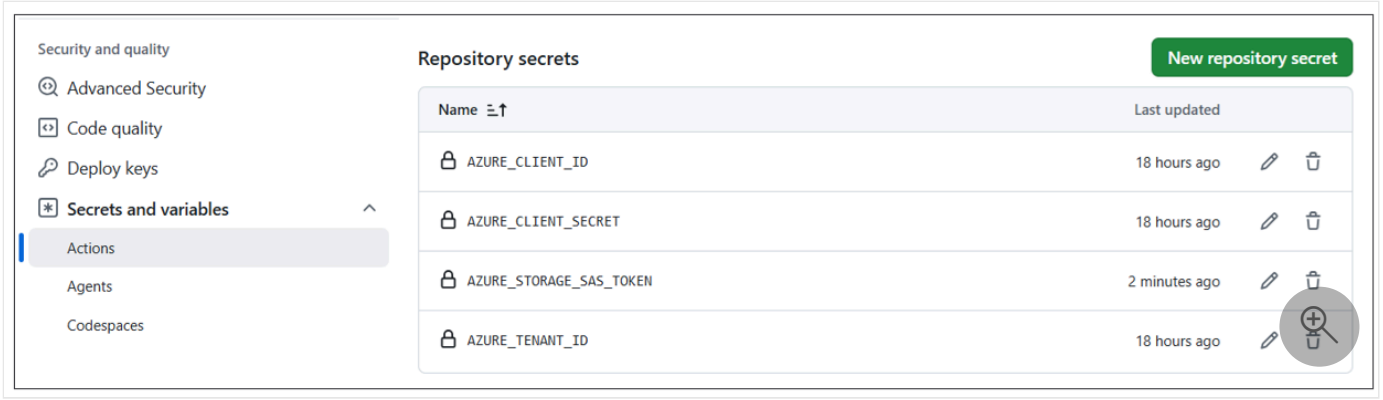
GitHub Actions is the platform in GitHub for developing workflows for CI/CD processes. You can create workflows by using GitHub Actions that can be run on demand, run on a schedule, or run in response to Git events associated with a commit or a pull request.

With GitHub Actions, you create a new workflow by copying a YAML file with a workflow definition into a special repository folder named `.github/workflows`. Unlike Azure pipelines, there's no need to register the YAML file with GitHub. Copying the YAML file into the `.github/workflows` folder is all you need to do for GitHub to recognize it as a workflow action. Inside the YAML file, it's possible to reference a script written in a language, such as Python. The following screenshot shows a typical example of the file structure of workflow files for a Fabric CI/CD project in a GitHub repository.

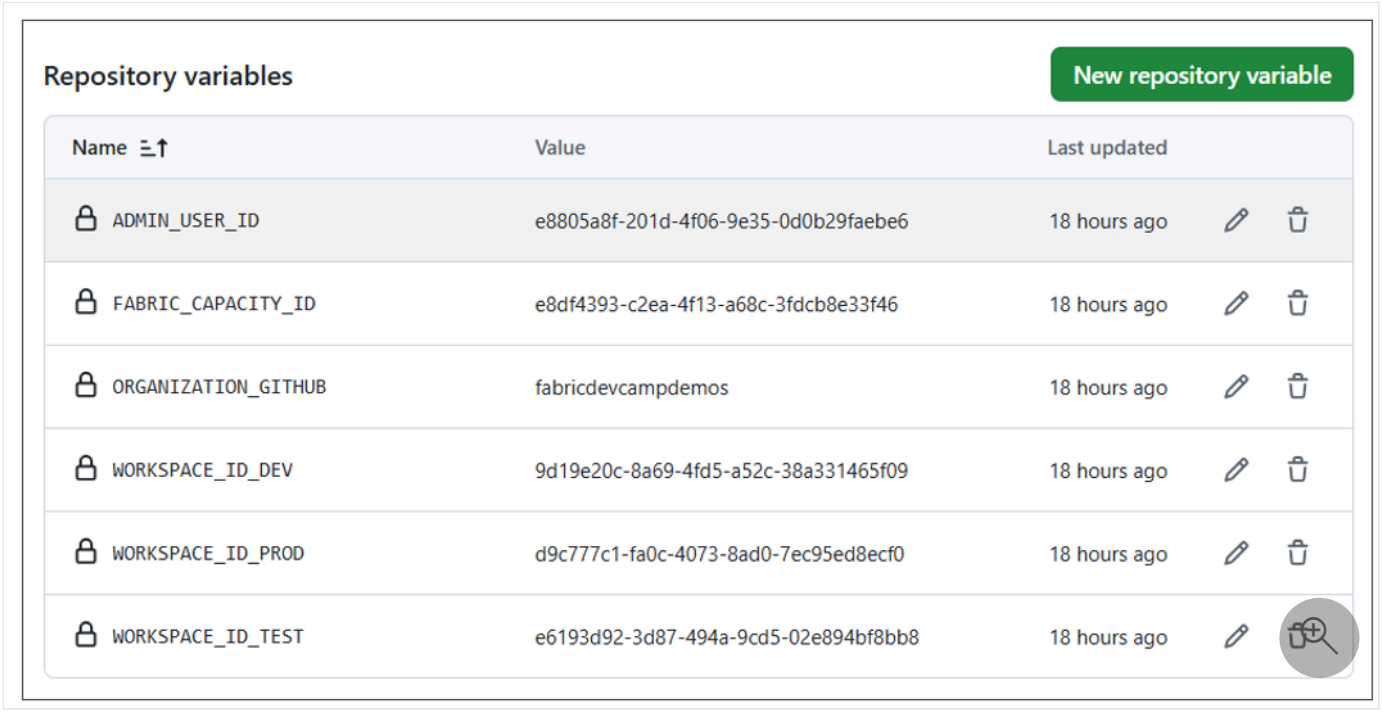


As you begin to develop GitHub Actions for a Fabric CI/CD project, you need a way to track environment settings, such as the IDs for workspaces, capacities, users, and groups. Your code might also need authentication credentials to authenticate as a service principal. In GitHub, you can create secrets and variables to track these types of environment settings at two different levels. You can create secrets and variables at the scope of the repository or at the scope of a specific environment.

To create a secret or a variable, navigate to the **Settings** page and select **Secrets and variables** > **Actions**. When you create a secret, GitHub protects it by encrypting its value and redacting it from logs. When a GitHub Action runs, GitHub decrypts a secret at runtime, injects its value into your workflow and then immediately purges the secret from memory when the workflow completes.

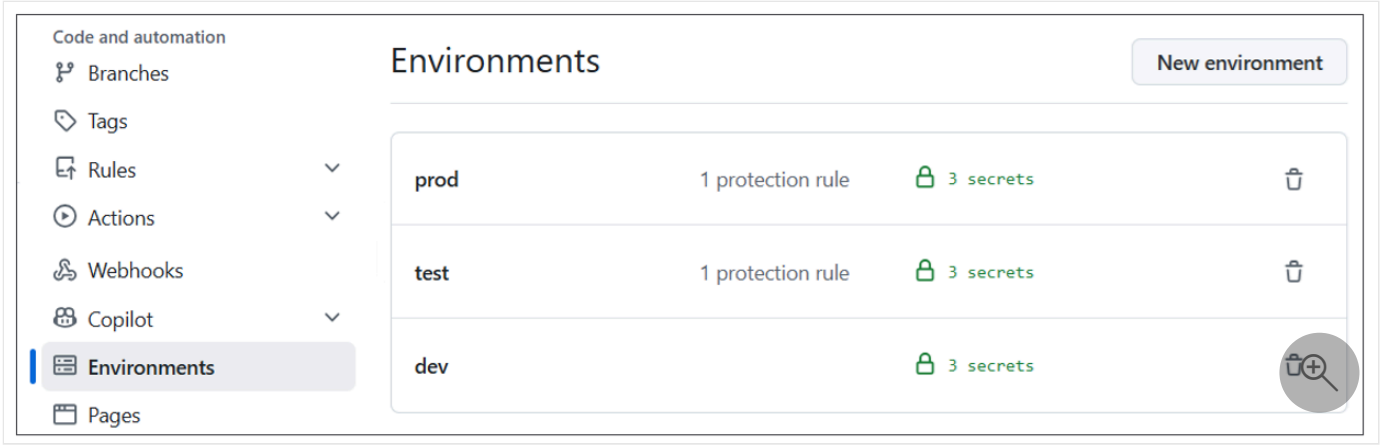


You can also add variables to a GitHub repository for environment settings that aren't sensitive. The following screenshot shows an example of variables created for GitHub Actions in a Fabric CI/CD project.



While creating secrets and variables at the repository level is adequate for some projects, you might find you need more granularity in other projects. GitHub supports extending a repository by creating *environments* and configuring them with their own secrets and variables.

Consider a Fabric CI/CD project in which all workspaces don't exist within the same Microsoft Entra ID tenant. For example, imagine the **dev** workspace, the **test** workspace, and the **prod** workspace all exist in different Microsoft Entra ID tenants. In this scenario, you need different authentication credentials for each Microsoft Entra ID tenant. You can start by creating three different environments in your GitHub repository named **dev**, **test**, and **prod**.

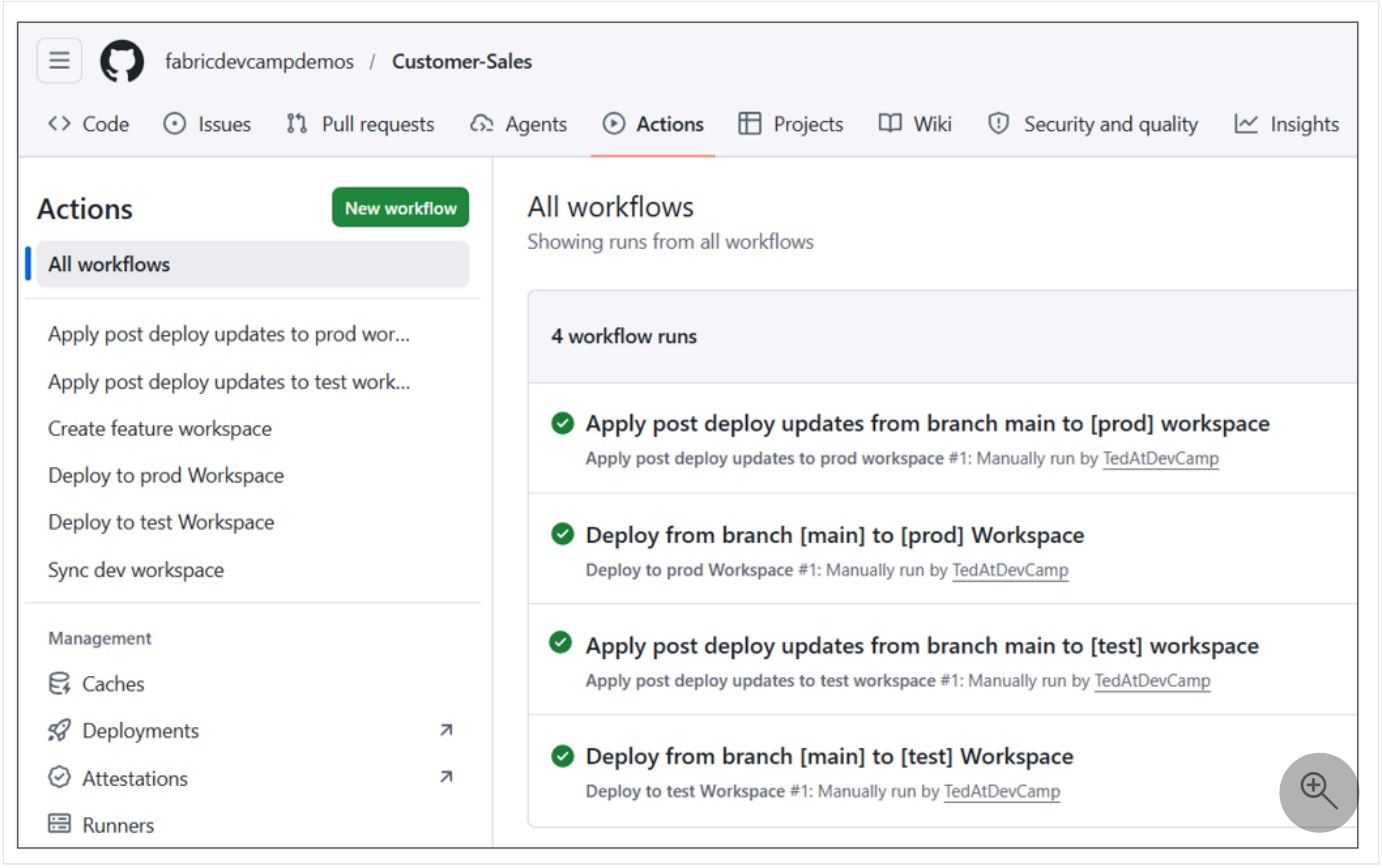


Then you can configure each environment with the authentication credentials specific to its Microsoft Entra ID tenant.

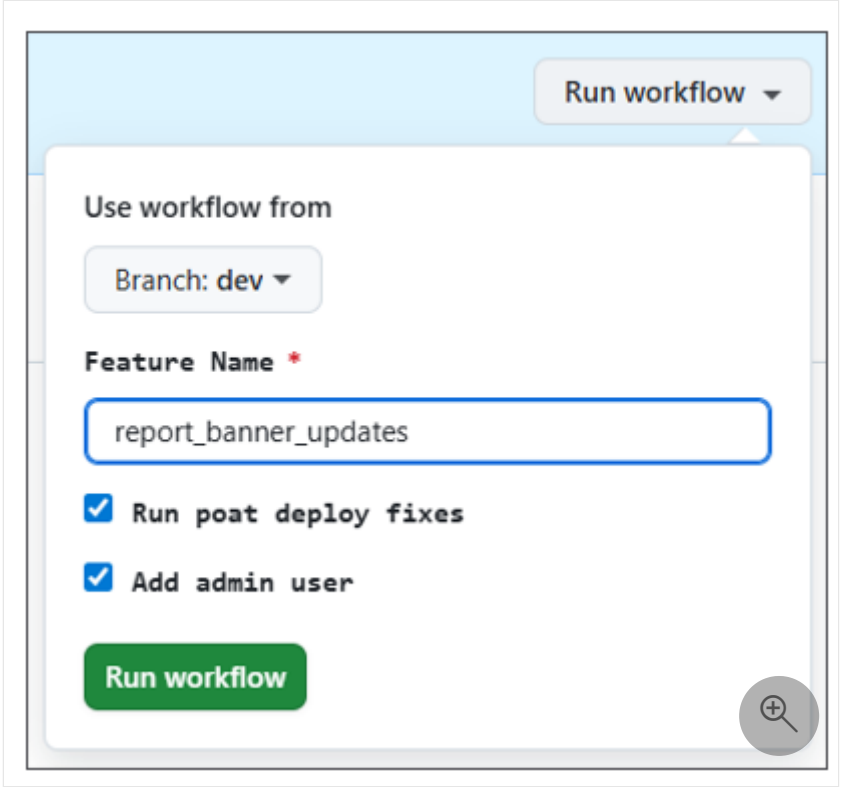
In addition to tracking environment-specific secrets and variables, you can configure an environment with protection rules. For example, you can create a protection rule for the **test** environment and the **prod** environment to require a manual-approval process with one or more approvers.

Once you have configured an environment with a manual-approval process, you can create GitHub Actions that target it. When you run a GitHub Action targeting an environment with a manual-approval process, the GitHub Actions job pauses until the manual-approval process completes. Once the approval is complete, the GitHub Actions job resumes its execution and can use the `fabric-cicd` library to deploy the new release to a target workspace.

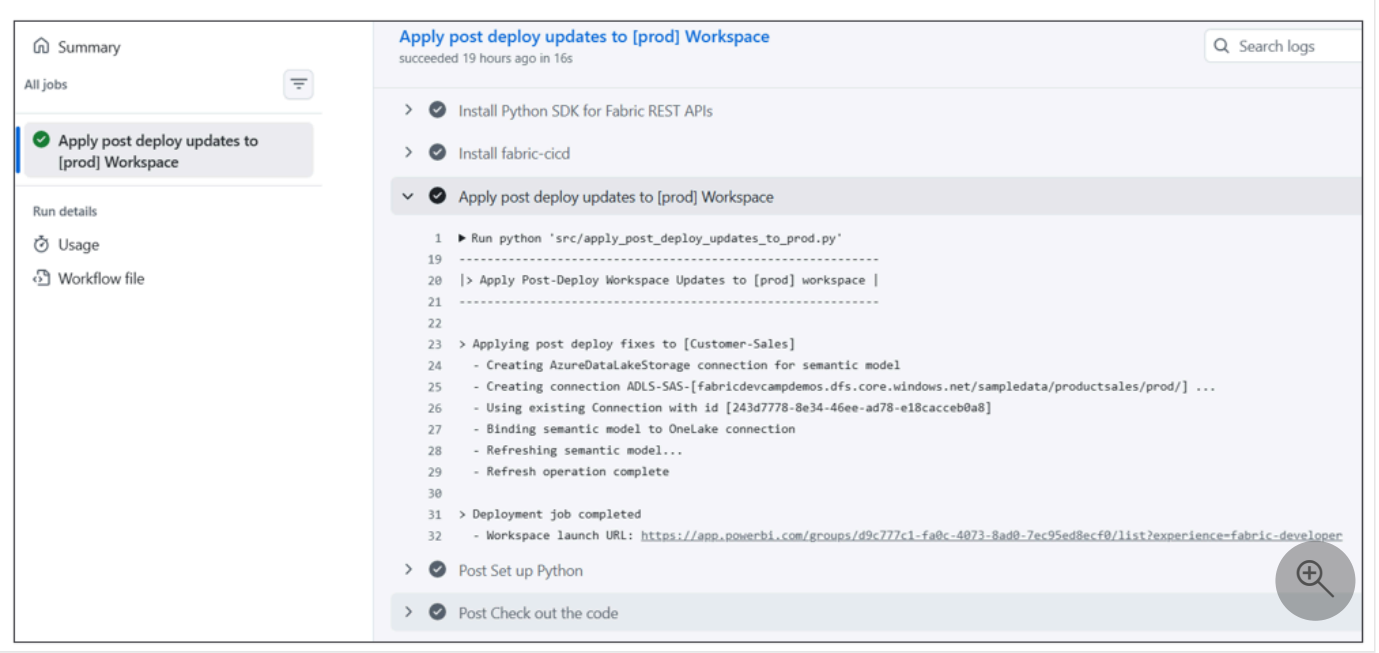
Once you have created workflow actions in a GitHub repository by copying YAML files into the `.github/workflows` folder, you can view them in the left navigation of the **Actions** page.



By selecting a workflow action in the left navigation, you can view its runs to see whether each run succeeded or failed. You can open a specific run's details if you'd like to view its logs. You can also run an action on demand by opening the **Run workflow** menu and selecting the **Run workflow** button. If you need to collect input before running the workflow, you can define input fields in the workflow's YAML file. The following screenshot shows an example of a workflow action that defines input parameters that collect values by using a text box and two checkboxes.



As you begin to develop workflows by using GitHub Actions, become familiar with monitoring workflow runs so you can test and debug your code. Add logging logic to your workflow actions to report successful operations and display diagnostic information about any errors that occur. The following screenshot shows an example of a GitHub Action that logs its progress.



What are Fabric CI/CD best practices?

This article presented the DevOps and CI/CD capabilities that the Fabric platform provides and explained how to apply them when setting up and managing Fabric CI/CD projects. This final section provides a checklist of best practices that you can reference as you build, deploy, and manage real-world Fabric solutions.

Fabric solution design

- Design the Fabric solution as a composition of workspace item types.
- Design a medallion architecture by using lakehouse items as storage containers for bronze, silver, and gold layers.
- Design ETL logic by using items, such as notebooks, pipelines, copy jobs, dataflows, and user-defined functions.
- Design semantic models and reports to analyze and visualize data in lakehouse tables.
- Determine whether the solution can run in a single workspace or if it requires multiple workspaces.
- Prototype the Fabric solution and validate its value to business users through peer review.

Fabric CI/CD project planning

- Plan the project environments (for example, **dev**, **test**, and **prod**).
- Plan what tenant-level Fabric items each environment requires (for example, **workspaces**, **connections**, and **gateways**).
- Plan what Azure resources each environment requires (for example, **Fabric capacity** and **Azure Storage account**).
- Select a Git provider by choosing between **Azure DevOps**, **GitHub**, and **GitHub Enterprise**.
- Select a Git branching strategy for the project (for example, **GitFlow** or **trunk-based development**).

Security best practices

- Use service principal authentication for all DevOps and CI/CD automation.
- Never commit files with sensitive credentials to Git.

Fabric CI/CD project setup

- Use Terraform to provision Fabric CI/CD project environment resources by using the principles of *infrastructure as code*.
- Use Terraform to create the workspace or workspaces and a separate capacity that each environment requires.
- Use Terraform to create other Azure resources that each environment requires (for example, **Azure Storage account** and **Azure Key Vault**).
- Use Terraform to configure permissions and access control for resources for Microsoft Entra ID users, groups, and service principals.
- Use Terraform to create all connections required for each environment as part of the project startup process.
- Let Terraform manage connection credentials (for example, SAS tokens) so secrets always remain encrypted.
- For production scenarios, configure Terraform to store the state file in protected cloud storage in an encrypted format.
- If you're not using Terraform, write repeatable setup scripts by using **Fabric CLI**, **Semantic Link Labs**, or **Fabric REST APIs**.

Parameterization for environment-specific settings

- Determine which items contain settings with environment-specific values (for example, **data-source paths** and **connection IDs**).
- Create a *variable library* and add **variables** to parameterize environment-specific settings.

- Update item settings to read variables from the variable library to remove environment-specific settings.
- Use **connection reference variables** to parameterize connections to external data sources.
- Use **item reference variables** in multi-workspace solutions to manage item dependencies across workspaces.
- Extend the variable library by creating **value sets** for testing and production environments.

Data orchestration strategy

- Implement a medallion architecture by using lakehouse items as storage containers for bronze, silver, and gold layers.
- Implement ETL logic by using items, such as notebooks, pipelines, copy jobs, dataflows, and user-defined functions.
- Expose a single top-level pipeline or notebook to run end-to-end ETL processing to populate all lakehouses with data.
- Write a **post-deploy script** to automate running a single top-level pipeline or notebook after the deployment operation.
- Configure ETL items with scheduled jobs for ongoing data refresh through `.schedules` files in item definitions.
- Manage schema changes to lakehouse tables by writing code in a notebook with table management logic.
- Manage schema changes to a warehouse by using *SqlPackage* and *Data-tier Application Framework* tooling instead.

Continuous integration development process

- Create a Git repository and configure branches for the selected branching strategy (for example, **GitFlow** or **trunk-based development**).
- Create a single source of truth in the **integration branch** by adding the item definitions for a Fabric solution.
- Create a Git connection between the **dev** workspace and the **integration branch**.
- Create Git connections with the **Git folder** setting to separate item definition folders from other workflow-related files.
- For multi-workspace solutions, use unique **Git folder** settings to connect all workspaces to a single Git branch.
- Configure an *integration branch policy* to prohibit direct commits and require pull requests to merge all changes.

- Use the *branched workspaces* capabilities of Fabric to create and manage feature branches and feature workspaces.
- Delete feature branches after pull requests to keep them short-lived.
- Recycle feature workspaces across multiple feature branches for dedicated teams.
- Review item definition structures to understand what files each item type requires or supports.
- For notebooks, enable auto-binding by adding `notebook-settings.json` to their item definition.
- For items that don't support auto-binding, write **post-sync scripts** to reestablish relationships.

Continuous deployment release process

- Configure the Git repository with variables (for example, **workspace IDs** and **Microsoft Entra IDs for users and groups**).
- Configure the Git repository with secrets (for example, **authentication credentials for a service principal**).
- Use `fabric-cicd` and its support for configuration-based deployment to build a code-first release process.
- Configure `fabric-cicd` environment names to match value set names in the variable library to enable auto-activation.
- Use `fabric-cicd` parameterization support to update environment-specific settings during deployment.
- When possible, use pull requests to configure manual-approval processes for deployment jobs.
- When using trunk-based development, create environments in the hosting repository to configure manual-approval processes.

Next steps for Fabric CI/CD

- Get started with Git integration
- Development process using branched workspace
- Fabric CLI documentation.
- Tutorial: CI/CD using Azure DevOps and the fabric-cicd library

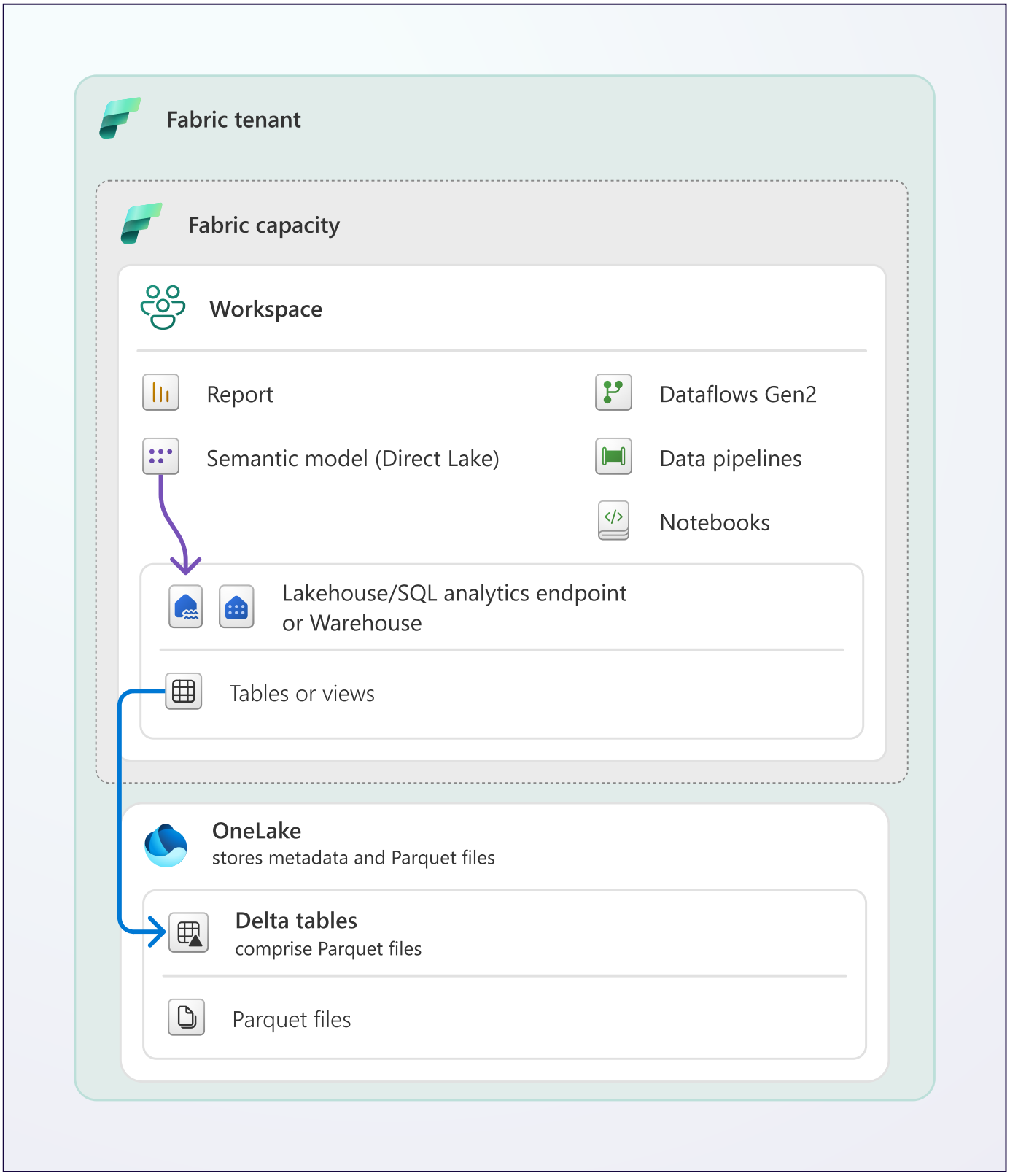
ⓘ**Note:** This article contains content created with AI. Learn more

---

Last updated on 06/29/2026

Direct Lake overview

Direct Lake is a Power BI semantic model table storage mode option available in Microsoft Fabric. It's optimized for large volumes of data to be quickly loaded into memory from Delta tables available in the OneLake—the single store for all analytics data. Once loaded into memory, the semantic model enables high-performance interactive analysis.



Direct Lake is ideal for semantic models connecting to large Fabric lakehouses, warehouses, and other Fabric data sources with Delta tables. Direct Lake is especially useful when replicating the entire data volume into an Import table is impractical or impossible. Direct Lake and Import queries are processed by the VertiPaq query engine, whereas DirectQuery federates queries to the underlying data source. Direct Lake and Import queries normally outperform DirectQuery queries when loading and interacting with visuals in reports.

However, Direct Lake differs from Import mode in an important way: a refresh operation for a Direct Lake semantic model is conceptually different from a refresh operation for an Import semantic model. Import mode replicates the data and creates an entire cached copy of the data for the semantic model, whereas a Direct Lake refresh copies only metadata (known as framing, described later in this article), which can take a few seconds to complete. The Direct Lake refresh is a low-cost operation that analyzes the metadata of the latest version of the Delta tables and updates references to the latest files in OneLake. In contrast, an Import refresh produces a copy of the data, which can take considerable time and consume significant data source and capacity resources (memory and CPU). Direct Lake moves data preparation to OneLake and, in doing so, uses the full breadth of Fabric technologies for data prep, including Spark jobs, T-SQL DML statements, dataflows, pipelines, and more.

Direct Lake storage mode offers the following key benefits:

- Similar to Import mode, Direct Lake queries are processed by the VertiPaq engine, and thus deliver query performance comparable to Import mode without the management overhead of data refresh cycles to load the entire data volume.
- Uses existing Fabric investments by seamlessly integrating with large lakehouses, warehouses, and other Fabric sources with Delta tables. For example, Direct Lake is an ideal choice for the *gold* analytics layer in the medallion lakehouse architecture.
- Maximizes return on investment (ROI) because analyzed data volumes can exceed the capacity's max memory limits, since only the data that's needed to answer a query is loaded into memory.
- Minimizes data latencies by quickly and automatically synchronizing a semantic model with its sources, making new data available to business users without refresh schedules.

> **Tip**
>
> Direct Lake performance depends on well-tuned Delta tables. For comprehensive guidance on optimizing tables for Direct Lake consumption, including V-Order and row group recommendations, see **Cross-workload table maintenance and optimization**.

When should you use Direct Lake storage mode?

The primary use case for Direct Lake storage mode is typically for IT-driven analytics projects that use lake-centric architectures. In such scenarios, you have, or expect to accumulate, large volumes of data in OneLake. The fast loading of that data into memory, frequent and fast refresh operations, efficient use of capacity resources, and fast query performance are all important for this use case.

> **Note**
>
> Import and DirectQuery tables in semantic models are still relevant in Fabric, and they're the right choice of semantic model for some scenarios. For example, Import storage mode often works well for a self-service analyst who needs the freedom and agility to act quickly, and without dependency on IT to add new data elements.
>
> A semantic model with Import tables and Direct Lake tables offers the scale and flexibility needed for many BI scenarios.
>
> Also, OneLake integration automatically writes data for tables in Import storage mode to Delta tables in OneLake without involving any migration effort, which lets you realize many of the benefits of Fabric that are made available to Import semantic model users, such as integration with lakehouses through shortcuts, SQL queries, notebooks, and more. We recommend this option as a quick way to reap the benefits of Fabric without necessarily or immediately redesigning your existing data warehouse and/or analytics system.

Direct Lake depends on data preparation being done in the data lake. Data preparation can be done by using various tools, such as Spark jobs for Fabric lakehouses, T-SQL DML statements for Fabric warehouses, dataflows, pipelines, and others, which helps ensure that data preparation logic is performed upstream in the architecture to maximize reusability. However, if the semantic model author doesn't have the ability to modify the source item, for example, if a self-service analyst doesn't have write permissions on a lakehouse that's managed by IT, then augmenting the model with Import storage mode tables might be a good choice, because Import mode supports data preparation by using Power Query, which is defined as part of the semantic model.

Be sure to consider your current Fabric capacity license and the Fabric capacity guardrails when you consider Direct Lake storage mode. Also, factor in the considerations and limitations, which are described later in this article.

> **Tip**
>
> We recommend that you produce a **prototype**—or proof of concept (POC)—to determine whether a Direct Lake semantic model is the right solution, and to mitigate risk.

Key concepts and terminology

This article assumes familiarity with the following concepts:

- Users load and interact with visuals in Power BI reports generating DAX queries to the semantic model.
- **Storage mode**: The semantic model processes the DAX queries differently depending on the table storage mode used. For example:
  - Import and Direct Lake storage modes use the VertiPaq engine to process DAX queries and return results to the Power BI report and user.
  - DirectQuery translates DAX queries to the query syntax of the data source, such as a SQL query, and runs them on the underlying source database. These source databases aren't typically optimized for heavy query load coming from reports and aggregated queries needed by the visuals, and may result in slower performance when compared to Import and Direct Lake modes.

Storage mode is a property of a table in the semantic model. When a semantic model includes tables with different storage modes, it's referred to as a composite model. For more information about storage modes, see Semantic model modes in the Power BI service.

Direct Lake table storage mode has two options:

- **Direct Lake on OneLake** can use data from one or more Fabric data sources with Delta tables. Direct Lake on OneLake doesn't fall back to DirectQuery mode via the SQL analytics endpoint of the data source. Semantic models with Direct Lake on OneLake tables can also have Import tables added from other data sources.
- **Direct Lake on SQL** can use the data from a single Fabric data source with Delta tables. The SQL analytics endpoint is used for Delta table and SQL view discovery and permission checks. Direct Lake on SQL analytics endpoints fall back to DirectQuery table storage mode when they can't load the data directly from a Delta table, such as when the data source is a SQL view or when the warehouse uses SQL-based granular access control. The semantic model property, **Direct Lake behavior**, controls the fallback behavior.

Comparison of storage modes

Use the following comparisons to understand the data sources and capabilities supported by each storage mode.

Supported Fabric data sources

Engine support: For Direct Lake on OneLake, the Analysis Services engine supports any Fabric item whose OneLake storage resolves to valid Delta tables, either natively or through shortcuts. You can create models for other compatible items programmatically by using an XMLA-compliant tool or a Fabric notebook. For more information, see Create the model.

Creation experience support: The following table shows end-to-end support through these experiences for each Direct Lake option.

Expand table

Fabric data source	Direct Lake on OneLake	Direct Lake on SQL
Lakehouse	Supported	Supported
Warehouse	Supported	Supported
SQL database in Fabric	Supported	Supported
Mirrored database	Supported	Supported
Mirrored Azure Databricks catalog	Supported	Supported
Snowflake database	Not supported	Supported
Cosmos DB in Fabric	Supported	Not supported
KQL database [1]	Supported	Not supported
Warehouse snapshot	Not supported	Not supported
API for GraphQL	Not supported	Not supported

[1] For a KQL database, turn on OneLake availability to make its data available in Delta Lake format.

Warehouse snapshots and APIs for GraphQL don't expose Delta tables in OneLake as source tables for Direct Lake.

Compare storage mode capabilities

The following table compares Direct Lake storage mode to Import and DirectQuery storage modes.

Expand table

Capability	Direct Lake on OneLake	Direct Lake on SQL analytics endpoints	Import	DirectQuery
Licensing	Fabric capacity subscription (SKUs) only	Fabric capacity subscription (SKUs) only	Any Fabric or Power BI license (including Microsoft Fabric Free licenses)	Any Fabric or Power BI license (including Microsoft Fabric Free licenses)
Data source	One or more Fabric items whose OneLake storage resolves to valid Delta tables, either natively or through shortcuts	A single supported Fabric data source with a SQL analytics endpoint, as listed in Supported Fabric data sources	Any connector	Any connector that supports DirectQuery mode
Connect to SQL analytics endpoint views	No	Yes – but will automatically fall back to DirectQuery mode	Yes	Yes
Composite models	Yes - can combine with Import storage mode tables in Power BI web modeling and DirectQuery tables with XMLA tools.	No [1]	Yes – can combine with DirectQuery, Dual, and Direct Lake storage mode tables	Yes – can combine with import, Dual, and Direct Lake storage mode tables
Single sign-on (SSO)	Yes	Yes	Not applicable	Yes
Calculated tables	Yes (preview) – calculated tables are supported for Direct Lake on OneLake; see considerations and limitations.	No – except calculation groups, what-if parameters, and field parameters, which implicitly create calculated tables	Yes	No – calculated tables use Import storage mode even when they refer to other tables in DirectQuery mode
Calculated columns	Yes - User Context only (Preview)	No	Yes	Yes
Hybrid tables	No	No	Yes	Yes

Capability	Direct Lake on OneLake	Direct Lake on SQL analytics endpoints	Import	DirectQuery
Model table partitions	No – however partitioning can be done at the Delta table level	No – however partitioning can be done at the Delta table level	Yes – either automatically created by incremental refresh, or manually created by using the XMLA endpoint	No
User-defined aggregations	No	No	Yes – Import aggregation tables on DirectQuery tables are supported	Yes
SQL analytics endpoint object-level security or column-level security	No	Yes – but might produce errors when permission is denied	Yes – but must duplicate permissions with semantic model object-level security	Yes – but queries might produce errors when permission is denied
SQL analytics endpoint row-level security (RLS)	No	Yes – but queries will fall back to DirectQuery mode	Yes – but must duplicate permissions with semantic model RLS	Yes
Semantic model row-level security (RLS)	Yes – but it's strongly recommended to use a fixed identity cloud connection	Yes – but it's strongly recommended to use a fixed identity cloud connection	Yes	Yes
Semantic model object-level security (OLS)	Yes	Yes	Yes	Yes
Large data volumes without refresh requirement	Yes	Yes	No	Yes
Reduce data latency	Yes – when automatic updates is enabled, or programmatic reframing	Yes – when automatic updates is enabled, or programmatic reframing	No	Yes

Capability	Direct Lake on OneLake	Direct Lake on SQL analytics endpoints	Import	DirectQuery
Power BI Embedded	Yes [2]	Yes [2]	Yes	Yes

[1] When using Direct Lake on SQL analytics endpoints, you can't combine Direct Lake storage mode tables with DirectQuery or Dual storage mode tables *in the same semantic model*. However, you can use Power BI Desktop to create a composite model on a Direct Lake semantic model and then extend it with new tables (by using import, DirectQuery, or Dual storage mode) or calculations. For more information, see Build a composite model on a semantic model.

[2] Requires a V2 embed token. If you're using a service principal, you must use a fixed identity cloud connection.

For details on how Direct Lake works, including column loading (transcoding), framing, automatic updates, and DirectQuery fallback, see How Direct Lake works.

For details on permissions, authentication, OLS/RLS, and data-access rule options, see Integrate Direct Lake security.

Fabric capacity requirements

Direct Lake semantic models require a Fabric capacity license. Also, there are capacity guardrails and limitations that apply to your Fabric capacity subscription (SKU), as presented in the following table.

> **Important**
>
> The first column in the following table also includes Power BI Premium capacity subscriptions (P SKUs). Power BI Premium per-capacity SKUs are being retired. New deployments should use Fabric capacity subscriptions (F SKUs).
>
> For more information, see the **Power BI Premium to Microsoft Fabric migration overview**.

Expand table

Fabric SKU	Parquet files per table	Row groups per table	Rows per table (millions)	Max model size on disk/OneLake (GB)	Max memory (GB) [1]
F2	1,000	1,000	300	10	3
F4	1,000	1,000	300	10	3
F8	1,000	1,000	300	10	3
F16	1,000	1,000	300	20	5
F32	1,000	1,000	300	40	10
F64/FT1/P1	5,000	5,000	1,500	Unlimited	25
F128/P2	5,000	5,000	3,000	Unlimited	50
F256/P3	5,000	5,000	6,000	Unlimited	100
F512/P4	10,000	10,000	12,000	Unlimited	200
F1024/P5	10,000	10,000	24,000	Unlimited	400
F2048	10,000	10,000	24,000	Unlimited	400
F4096	10,000	10,000	24,000	Unlimited	400
F8192	10,000	10,000	24,000	Unlimited	400

[1] For Direct Lake semantic models, *Max Memory* represents the upper memory resource limit for how much data can be paged in. For this reason, it's not a guardrail; however, it can have a performance impact if the amount of data is large enough to cause excessive paging in and out of the model data from the OneLake data.

If guardrails are exceeded, the behavior depends on the table storage mode:

- **Direct Lake on OneLake**: Similar to Import mode, refresh fails and the model cannot be queried until the Delta tables are optimized to fall within guardrail limits.
- **Direct Lake on SQL**: Falls back to DirectQuery mode if fallback is enabled. Refresh succeeds with a warning, and queries still return results, but with slower performance.

The *Max model size on disk/OneLake* guardrail is evaluated at the model level and affects all queries. All other guardrails presented in the table are evaluated per query. It's therefore important that you optimize your Delta tables and Direct Lake semantic model to avoid having to unnecessarily scale up to a higher Fabric SKU.

Additionally, *Capacity unit* and *Max memory per query limits* apply to Direct Lake semantic models. For more information, see Capacities and SKUs.

Considerations and limitations

Direct Lake semantic models have some considerations and limitations.

> **Note**
>
> The capabilities and features of Direct Lake semantic models are evolving rapidly. Be sure to check back periodically to review the latest list of considerations and limitations.

Expand table

Consideration / limitation	Direct Lake on OneLake	Direct Lake on SQL (analytics endpoint)
When the SQL analytics endpoint enforces row-level security, DAX queries are processed differently depending on the type of Direct Lake mode employed. When Direct Lake on OneLake is employed, queries will succeed, and SQL based RLS is not applied. Direct Lake on OneLake requires the user has access to the files in OneLake, which doesn't observe SQL based RLS.	Queries will succeed.	Yes, unless fallback is disabled in which case queries will fail.
If a table in the semantic model is based on a (non-materialized) SQL view, DAX queries are processed differently depending on the type of Direct Lake mode employed. Direct Lake on SQL analytics endpoints will fall back to DirectQuery in this case. It isn't supported to create a Direct Lake on OneLake table based on a non-materialized SQL view. You can instead use a lakehouse materialized view because Delta tables are created. Alternatively, use a different storage mode such as Import or Direct Lake for	Not applicable	Yes, unless fallback is disabled in which case queries will fail.

Consideration / limitation	Direct Lake on OneLake	Direct Lake on SQL (analytics endpoint)
tables based on non-materialized SQL views.		
Composite modeling, which means Direct Lake semantic model tables can be mixed with tables in other storage modes, such as Import, DirectQuery, or Dual (except for special cases, including calculation groups, what-if parameters, and field parameters).	Supported	Not supported
Calculated columns and calculated tables that reference columns or tables in Direct Lake storage mode. Calculation groups, what-if parameters, and field parameters, which implicitly create calculated tables, and calculated tables that don't reference Direct Lake columns or tables are supported in all scenarios.	Yes - Calculated Columns are User Context only	Not supported
Direct Lake storage mode tables don't support complex Delta table column types. Binary and GUID semantic types are also unsupported. You must convert these data types into strings or other supported data types.	Not supported	Not supported
Table relationships require the data types of related columns to match.	Yes	Yes
One-side columns of relationships must contain unique values. Queries fail if duplicate values are detected in a one-side column.	Yes	Yes
Auto date/time intelligence in Power BI Desktop to create relationships using only the date part of a datetime column. Note: Marking your own date table as a date table and creating relationships using date columns is supported.	Supported	Not supported
The length of string column values is limited to 32,764 Unicode characters.	Yes	Yes
Non-numeric floating point values, such as *NaN* (not a number), aren't supported.	Yes	Yes

Consideration / limitation	Direct Lake on OneLake	Direct Lake on SQL (analytics endpoint)
Publish to web from Power BI using a service principal is only supported when using a fixed identity for the Direct Lake semantic model.	Yes	Yes
In the web modeling experience, validation is limited for Direct Lake semantic models. User selections are assumed to be correct, and no queries are issued to validate cardinality or cross filter selections for relationships, or for the selected date column in a marked date table.	Yes	Yes
In the Fabric portal, the *Direct Lake* tab in the refresh history lists Direct Lake-related refresh failures. Successful refresh (framing) operations aren't typically listed unless the refresh status changes, such as from no previous run or refresh failure to refresh success or refresh success with warning.	Yes	Yes
Your Fabric SKU determines the maximum available memory per Direct Lake semantic model for the capacity. When the limit is exceeded, queries to the semantic model might be slower due to excessive paging in and out of the model data.	Yes	Yes
Creating a Direct Lake semantic model in a workspace that is in a different region of the data source workspace isn't supported. For example, if the Lakehouse is in West Central US, then you can only create semantic models from this Lakehouse in the same region. To find what region you are in, see find your Fabric home region.	Yes	Yes A workaround is to create a lakehouse in the other region's workspace and shortcut to the tables before creating the semantic model.
Embedding reports requires a V2 embed token.	Yes	Yes
Service principal profiles for authentication.	Not supported	Not supported

Consideration / limitation	Direct Lake on OneLake	Direct Lake on SQL (analytics endpoint)
Power BI Direct Lake semantic models can be created and queried by Service Principals and Viewer role membership with service principals is supported, but the default Direct Lake semantic models on lakehouse/warehouse don't support this scenario.	Yes	Yes
Shortcuts in a lakehouse can be used as data sources for semantic model tables.	Supported	Supported
Create Direct Lake models in personal workspaces (My Workspace).	Not supported	Not supported
Deployment pipeline rules to rebind data source.	Not supported	Supported
Adding multiple tables from the same data source table.	Not supported in Power BI Desktop or web modeling. It is possible to add multiple tables from the same data source table using XMLA-based external tools. Using **Edit tables** in Power BI tooling and **refresh** result in an error with multiple tables from the same data source table in the semantic model.	Not supported in Power BI Desktop or web modeling. It is possible to add multiple tables from the same data source table using XMLA-based external tools. Using **Edit tables** in Power BI tooling and **refresh** result in an error with multiple tables from the same data source table in the semantic model.
Direct Lake tables created by using XMLA applications are initially in an unprocessed state until the application sends a refresh command. Be sure to refresh the model to process its tables when you create a new semantic model.	Queries involving unprocessed tables return an error.	Queries involving unprocessed tables fall back to DirectQuery mode, unless fallback is disabled in which case queries fail.
XMLA-compliant tools must support `compatibilityLevel` 1604 or higher to work with Direct Lake semantic models and expose Direct Lake-specific metadata.	Yes	Yes
Using any gateway (on-premises or VNET) to refresh Direct Lake semantic models	Not supported. Direct Lake on OneLake supports only cloud connections and cannot operate through any gateway.	Not supported. Direct Lake on SQL (analytics endpoint) supports only cloud connections and cannot operate through any gateway.

- Analyze in Excel pivot tables (and other MDX clients) have the same limitations as DirectQuery with Direct Lake tables in the semantic model. Session-scoped MDX statements, such as named sets, calculated members, default members, etc. are not supported. Query-scoped MDX statements, such as the 'WITH' clause, are supported. Direct Lake table user-defined hierarchies are not supported. Import table user-defined hierarchies are supported even with Direct Lake tables in the semantic model.

- Power BI Desktop can live edit a semantic model with Direct Lake tables and Import tables. Calculation groups, what-if parameters, and field parameters, which implicitly create calculated tables, and calculated tables that don't reference Direct Lake columns or tables can also be included.

- Power BI web modeling can open any semantic model, including Direct Lake tables with other storage mode tables.

- DAX query view when live editing or live connected, and writing DAX queries in the web, are supported for Direct Lake on SQL, Direct Lake on OneLake, and true composite (Direct Lake on OneLake + Import from any data source) semantic models.

- TMDL view is supported when live editing in Power BI Desktop.

- Creating reports with a live connection is supported for all semantic models, when the report author has at least build access.

- Direct Lake on SQL connection expression in the semantic model must refer to the SQL analytics endpoint by GUID, not by friendly name, to use **Edit tables** and **refresh** operations in Power BI Desktop and Power BI web modeling. The connection expression can be updated in TMDL view or XMLA based external tools. The GUID is available in the URL when viewing the SQL analytics endpoint in the browser.

Related content

- Cross-workload table maintenance and optimization
- Develop Direct Lake semantic models
- Manage Direct Lake semantic models
- Understand Direct Lake query performance
- Analyze query processing for Direct Lake semantic models

ⓘ**Note:** The author created this article with assistance from AI. Learn more

Last updated on 09/18/2026

Develop Direct Lake semantic models

This article describes design topics relevant to developing Direct Lake semantic models.

Choose the right storage mode

Direct Lake storage mode has two forms. Use the following guidance to choose the Direct Lake type that best fits your scenario. For more information, see full comparison of storage modes.

Use **Direct Lake on OneLake** when...

- You generally expect faster query performance.
- You use one or more Fabric items whose OneLake storage resolves to valid Delta tables, either natively or through shortcuts.
- You need composite modeling with Import or DirectQuery tables.
- You want to use OneLake security.
- You need modeling features like calculated columns or calculated tables that reference Direct Lake tables (preview).
- You want consistent Direct Lake behavior without DirectQuery fallback (`DirectLakeOnly` mode).

Use **Direct Lake on SQL** when...

- You depend on security rules defined in the SQL analytics endpoint with delegated identity mode: RLS, CLS, OLS.
- Your model is based on SQL analytics endpoint tables or views from a single supported Fabric data source.
- You need unsupported Direct Lake scenarios to fall back to DirectQuery if fallback is enabled (for example, queries against SQL views).

Create the model

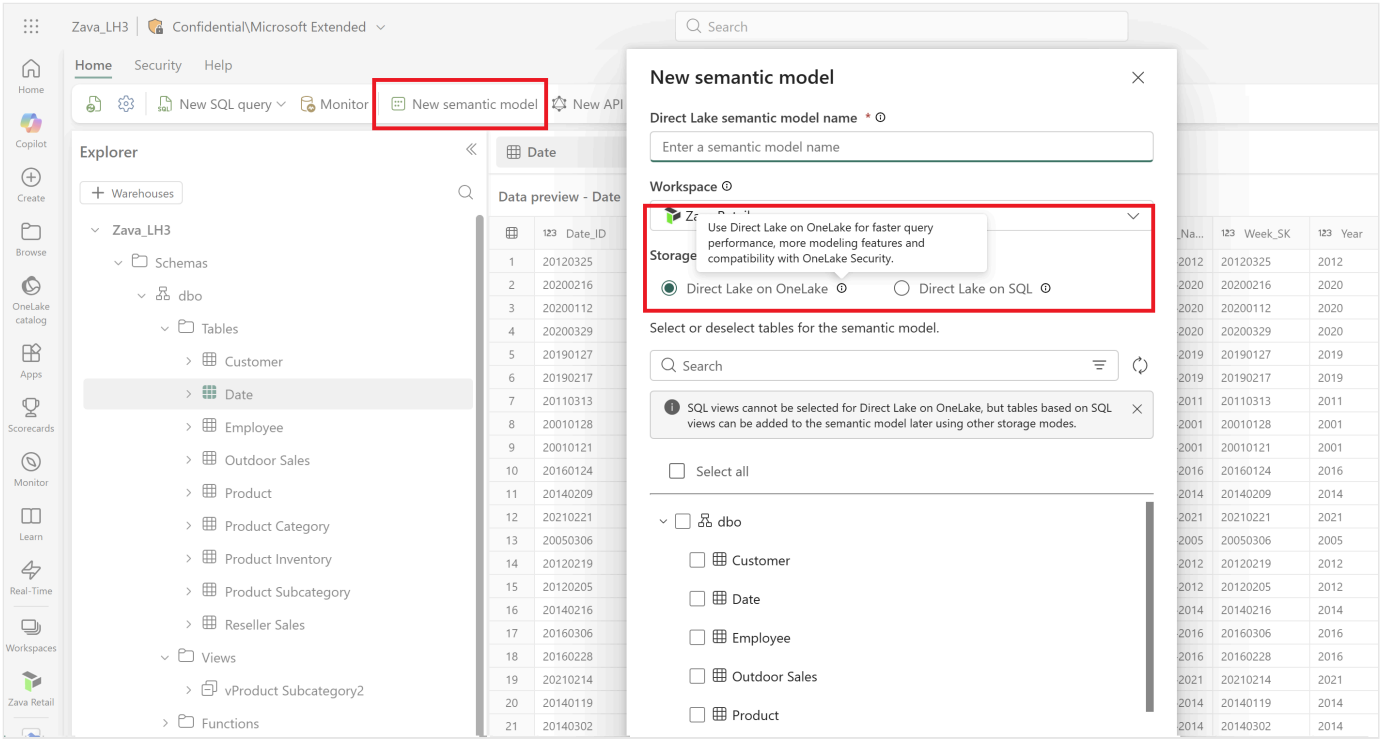
You can create a Direct Lake semantic model in Power BI Desktop or from many Fabric items in the browser. For example, from an open Lakehouse you can choose **New semantic model** to create a new semantic model in Direct Lake storage mode.

The following table shows the most common creation methods for each type of Direct Lake semantic model.

Expand table

	Supports Direct Lake on OneLake	Supports Direct Lake on SQL
Creation from the **Power BI service**. For example, select **Create** from the left navigation bar, then select **OneLake catalog**.	Yes	No
Creation from **Power BI Desktop**.	Yes	No
Creation from the **SQL analytics endpoint** page. For example, open the SQL analytics endpoint for a lakehouse, then select **New semantic model**.	Yes	Yes

When you create a Direct Lake semantic model from the SQL analytics endpoint, you can choose between the two Direct Lake types, as shown in the following image:



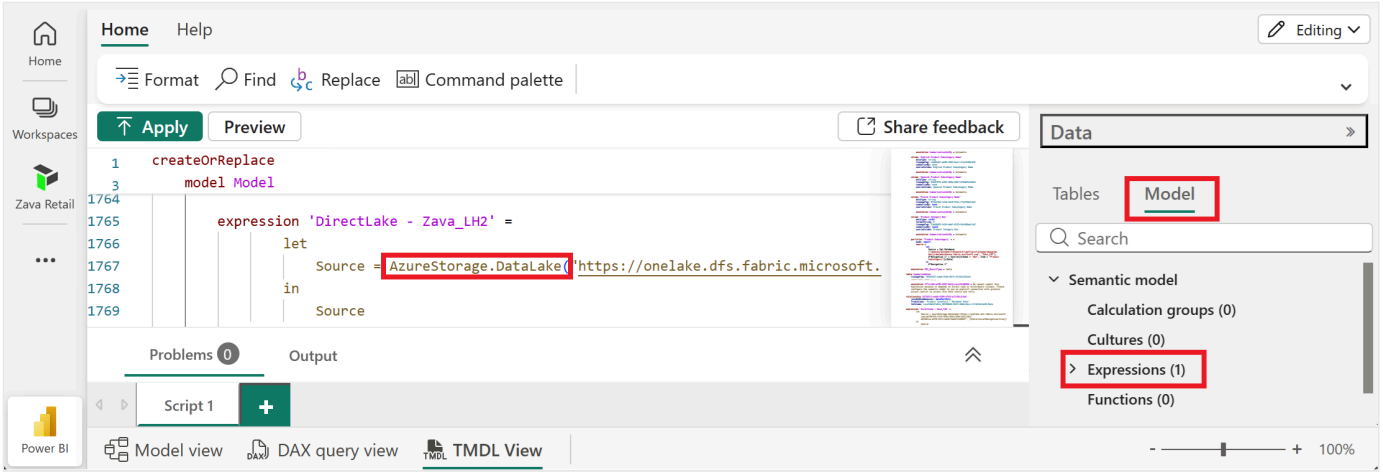
This dialog defaults to Direct Lake on OneLake if the SQL analytics endpoint is in user identity mode, or Direct Lake on SQL if the SQL analytics endpoint is in delegated mode. For more information, see Access modes in SQL analytics endpoint.

Alternatively, as with any Power BI semantic model, you can continue the development of your model by using an XMLA-compliant tool, like SQL Server Management Studio (SSMS) (version 19.1 or later) or open-source, community tools. Fabric notebooks can also programmatically create and edit semantic models with semantic link and semantic link labs.

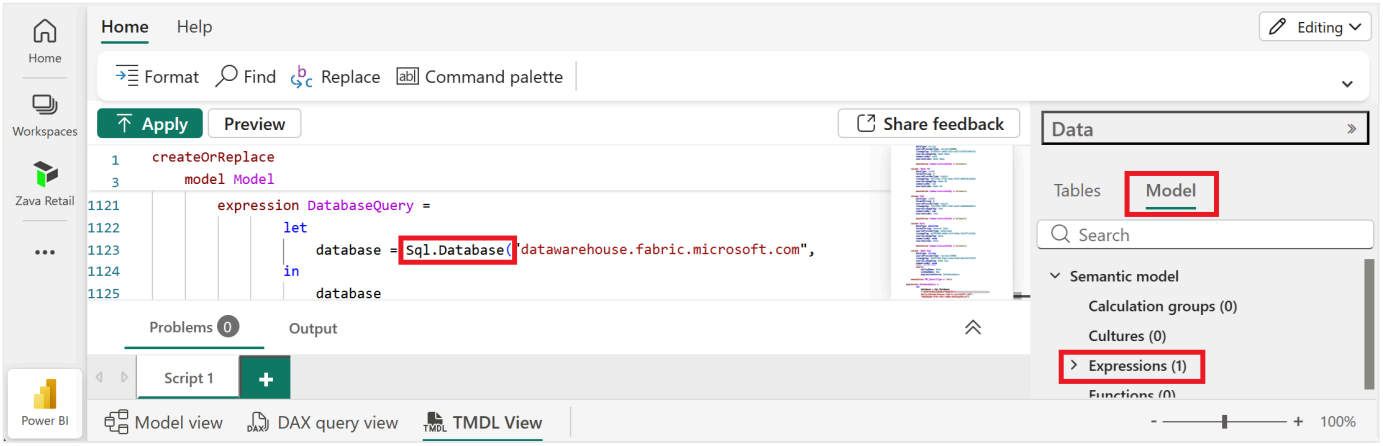
Power Query connectors

An easy way to tell if an existing Direct Lake semantic model is on OneLake or SQL is to check the connector in TMDL view. From the **Model** tab, drag the **Expressions** node to see the M expression definition.

Direct Lake on OneLake uses the Azure Data Lake Storage connector.



Direct Lake on SQL uses the SQL Server connector, or the `OneLake.SqlAnalytics()` connector.



Direct Lake composite models

Direct Lake on OneLake enables creation of multi-source, composite models that combine tables in Direct Lake mode with other tables in Import/DirectQuery mode, providing valuable flexibility for dimension tables while leveraging Direct Lake for very large fact tables to avoid costly refreshes and management overhead. For more information, see the Composite semantic models with Direct Lake and Import storage mode tables section.

Model tables

Model tables are normally based on the table in the source Fabric data item. Direct Lake on SQL also allows selection of a SQL view. Queries to a model table based on a SQL view fall back to DirectQuery mode, which might result in slower query performance.

> **Note**
>
> With Direct Lake on OneLake, tables based on SQL views can be added using other storage modes like Import and/or DirectQuery because Direct Lake on OneLake supports composite models.

Tables should include columns for filtering, grouping, sorting, and summarizing, in addition to columns that support model relationships.

For more information about columns to include in your semantic model tables, see Understand Direct Lake query performance.

For information about enforcing data-access rules with object-level security (OLS) and row-level security (RLS), see Integrate Direct Lake security.

Direct Lake model metadata

When you connect to a Direct Lake semantic model with the XMLA endpoint, the metadata looks like that of any other model. However, Direct Lake models show the following differences:

- The `compatibilityLevel` property of the database object is 1604 (or higher).
- The mode property of Direct Lake partitions is set to `directLake`.
- Direct Lake partitions use shared expressions to define data sources.
  - **Direct Lake on SQL** — The expression points to the SQL analytics endpoint of the lakehouse or warehouse. Direct Lake uses the SQL analytics endpoint to discover schema and security information, but it loads the data directly from OneLake (unless it falls back to DirectQuery mode for any reason).
  - **Direct Lake on OneLake** — The expression points directly to the OneLake storage location of the Fabric data source. Direct Lake uses the OneLake APIs for schema discovery, security checks, and data loading. Direct Lake on OneLake doesn't fall back to DirectQuery mode.

Post-publication tasks

After you publish a Direct Lake semantic model, you should complete some setup tasks. For more information, see Manage Direct Lake semantic models.

Related content

- Direct Lake overview
- Manage Direct Lake semantic models
- Understand Direct Lake query performance
- OneLake integration for semantic models

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/02/2026

How Direct Lake works

Typically, queries sent to a Direct Lake semantic model are handled from an in-memory cache of the columns sourced from Delta tables. The underlying storage for a Delta table is one or more Parquet files in OneLake. Parquet files organize data by column rather than by row. Semantic models load entire columns from Delta tables into memory as queries require them.

Direct Lake on OneLake isn't coupled with the SQL endpoint. This architecture offers tighter integration with OneLake features such as OneLake security and more efficient DAX query plans because, for example, checking for SQL based security isn't required. DirectQuery fallback isn't supported by Direct Lake on OneLake.

With Direct Lake on SQL endpoints, a DAX query might use *DirectQuery fallback*, which involves seamlessly switching to DirectQuery mode. DirectQuery fallback retrieves data directly from the SQL analytics endpoint of the lakehouse or the warehouse. For example, fallback occurs when SQL based security is detected in the SQL endpoint. In this case, a DirectQuery operation sends a query to the SQL analytics endpoint. Fallback operations might result in slower query performance.

The following sections describe Direct Lake concepts and features, including column loading, framing, automatic updates, and DirectQuery fallback.

Column loading (transcoding)

Direct Lake semantic models load data from OneLake only when a query requests a column for the first time. This on-demand data loading process is known as *transcoding*.

When the semantic model receives a DAX (or Multidimensional Expressions—MDX) query, it first determines what columns are needed to produce a query result. Any column directly used by the query is needed, and also columns required by relationships and measures. Typically, the number of columns needed to produce a query result is significantly smaller than the number of columns defined in the semantic model.

Once it understands which columns are needed, the semantic model determines which columns are already in memory. If any columns needed for the query aren't in memory, the semantic model loads all data for those columns from OneLake. Loading column data is typically a fast operation, however it can depend on factors such as the cardinality of data stored in the columns.

Columns loaded into memory are then *resident* in memory. Future queries that involve only resident columns don't need to load any more columns into memory.

A column remains resident until there's reason for it to be removed (evicted) from memory. Reasons that columns might get removed include:

- The model or table was refreshed after a Delta table update at the source (see Framing in the next section).
- No query used the column for some time.
- Other memory management reasons, including memory pressure in the capacity due to other, concurrent operations.

Your choice of Fabric SKU determines the maximum available memory for each Direct Lake semantic model on the capacity. For more information about resource guardrails and maximum memory limits, see Fabric capacity requirements.

Framing

Framing gives model owners point-in-time control over what data loads into the semantic model. Framing is a Direct Lake operation that a refresh of a semantic model triggers. In most cases, it takes only a few seconds to complete. That's because it's a low-cost operation where the semantic model analyzes the metadata of the latest version of the Delta Lake tables and updates to reference the latest Parquet files in OneLake.

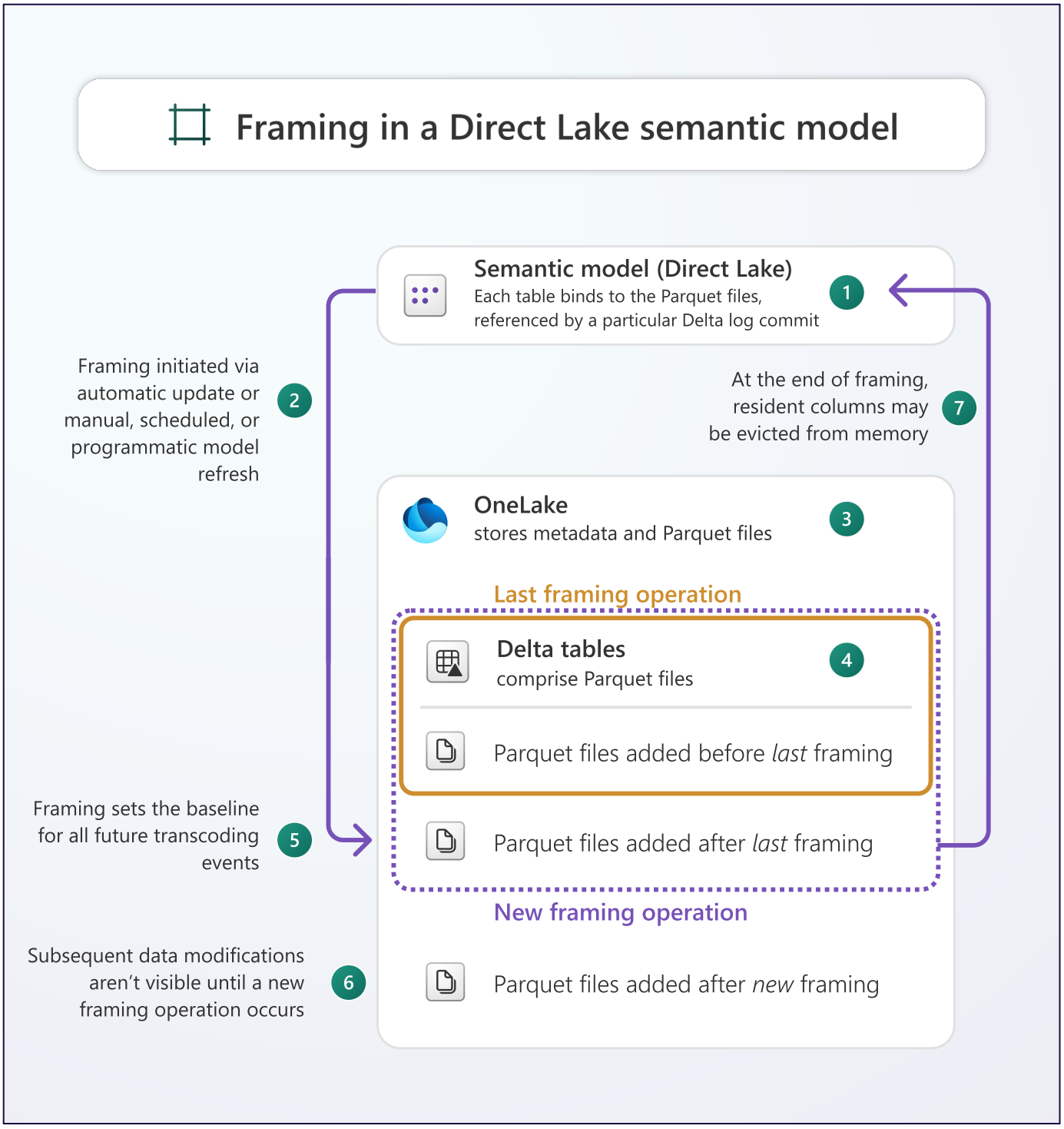
When framing occurs, the process might evict resident table column segments and dictionaries from memory if the underlying data changed. The point in time of the refresh becomes the new baseline for all future transcoding events. From this point, Direct Lake queries only consider data in the Delta tables as of the time of the most recent framing operation. For that reason, Direct Lake tables are queried to return data based on the state of the Delta table *at the point of the most recent successful framing operation*. That time isn't necessarily the latest state of the Delta tables.

The semantic model analyzes the Delta log of each Delta table during framing to drop only the affected column segments and to reload newly added data during transcoding. An important optimization is that dictionaries usually aren't dropped when incremental framing takes effect, and new values are added to the existing dictionaries. This incremental framing approach helps reduce the reload burden and benefits query performance. In the ideal case, when a Delta table receives no updates, no reload is necessary for columns already resident in memory. Queries

show far less performance impact after framing because incremental framing essentially enables the semantic model to update substantial portions of the existing in-memory data in place.

> **Note**
>
> Framing can fail if a Delta table exceeds the Fabric capacity guardrails, such as when a Delta table has more than 10,000 parquet files. For more information about resource guardrails, see Fabric capacity requirements.

The following diagram shows how Direct Lake framing operations work.



The diagram depicts the following processes and features.

Expand table

Item	Description
1	A semantic model exists in a Fabric workspace.
2	Framing operations take place periodically, and they set the baseline for all future transcoding events. Framing operations can happen automatically, manually, on schedule, or programmatically.
3	OneLake stores metadata and Parquet files, which are represented as Delta tables.
4	The last framing operation includes Parquet files related to the Delta tables, and specifically the Parquet files that were added before the *last* framing operation.
5	A later framing operation includes Parquet files added after the *last* framing operation.
6	Resident columns in the Direct Lake semantic model might be evicted from memory, and the point in time of the refresh becomes the new baseline for all future transcoding events.
7	Subsequent data modifications, represented by new Parquet files, aren't visible until the next framing operation occurs.

It's not always desirable to have data representing the latest state of any Delta table when a transcoding operation takes place. Consider that framing can help you provide consistent query results in environments where data in Delta tables is transient. Data can be transient for several reasons, such as when long-running extract, transform, and load (ETL) processes occur.

You can refresh a Direct Lake semantic model manually, automatically, or programmatically.

Automatic updates

Direct Lake semantic models have a model-level setting named **Keep your Direct Lake data up to date** that performs automatic updates of Direct Lake tables. It's enabled by default. It ensures that data changes in OneLake are automatically reflected in the Direct Lake semantic model. You can find the setting in the Fabric portal, in the **Refresh** section of the semantic model settings.

When the setting is enabled, the semantic model performs a framing operation whenever data modifications in the underlying Delta tables are detected. The framing operation is scoped to only the tables where data modifications are detected.

We recommend that you leave the setting on, especially when you have a small or medium-sized semantic model. It's especially useful when you have low-latency reporting requirements and

Delta tables are modified regularly.

In some situations, you might want to disable automatic updates. For example, you might need to allow completion of data preparation jobs or an extract, transform, and load (ETL) process before exposing any new data to consumers of the semantic model. When disabled, you can trigger a refresh manually in the Fabric portal, on a refresh schedule, or programmatically by using the Power BI REST API or the Tabular Object Model (TOM).

> **Important**
>
> Power BI suspends automatic updates when a *non-recoverable error* is encountered during a refresh. A non-recoverable error can occur, for example, when a refresh fails after several attempts. If your automatic updates stop, verify that the semantic model can be refreshed successfully. Power BI automatically resumes automatic updates after a subsequent on-demand refresh completes without errors.

> **Tip**
>
> You can set up **automatic page refresh** in your Power BI reports. This feature automatically refreshes a specific report page, provided that the report connects to a Direct Lake semantic model (or other types of semantic model).

DirectQuery fallback

When you use Direct Lake on SQL endpoints, a query sent to a Direct Lake semantic model can fall back to DirectQuery mode. In this mode, the table no longer operates in Direct Lake mode. It retrieves data directly from the SQL analytics endpoint of the lakehouse or warehouse. Such queries always return the latest data because they're not constrained to the point in time of the last framing operation. However, fallback operations might result in slower query performance.

> **Important**
>
> If possible, always design your solution or size your capacity to avoid DirectQuery fallback. That's because it might result in slower query performance.

Direct Lake on OneLake runs exclusively in DirectLakeOnly mode and doesn't support DirectQuery fallback. For new semantic models, it's the recommended Direct Lake option.

When Direct Lake mode is used

Queries stay in Direct Lake mode (avoiding DirectQuery fallback) only when *all* of the following conditions are true:

- The semantic model doesn't reference any tables that have SQL row-level security (RLS) defined at the SQL analytics endpoint.
- The semantic model doesn't reference any tables that have SQL dynamic data masking (DDM) defined at the SQL analytics endpoint.
- The semantic model doesn't reference any tables that have SQL object-level security (OLS) defined at the SQL analytics endpoint.
- The semantic model doesn't reference any tables based on unmaterialized SQL views.
- No single table in the semantic model exceeds guardrail limits as defined in Fabric capacity requirements:
  - Number of Parquet files is within limit.
  - Number of row groups is within limit.
  - Number of rows is within limit.
- You refreshed (framed) the semantic model after you created or modified the underlying Delta tables.

A single table that exceeds any guardrail limit prevents Direct Lake mode for the entire model.

Control fallback with DirectLakeBehavior

When Direct Lake conditions aren't met, the behavior of your semantic models depends on the **DirectLakeBehavior** setting. This setting only applies to Direct Lake on SQL endpoints.

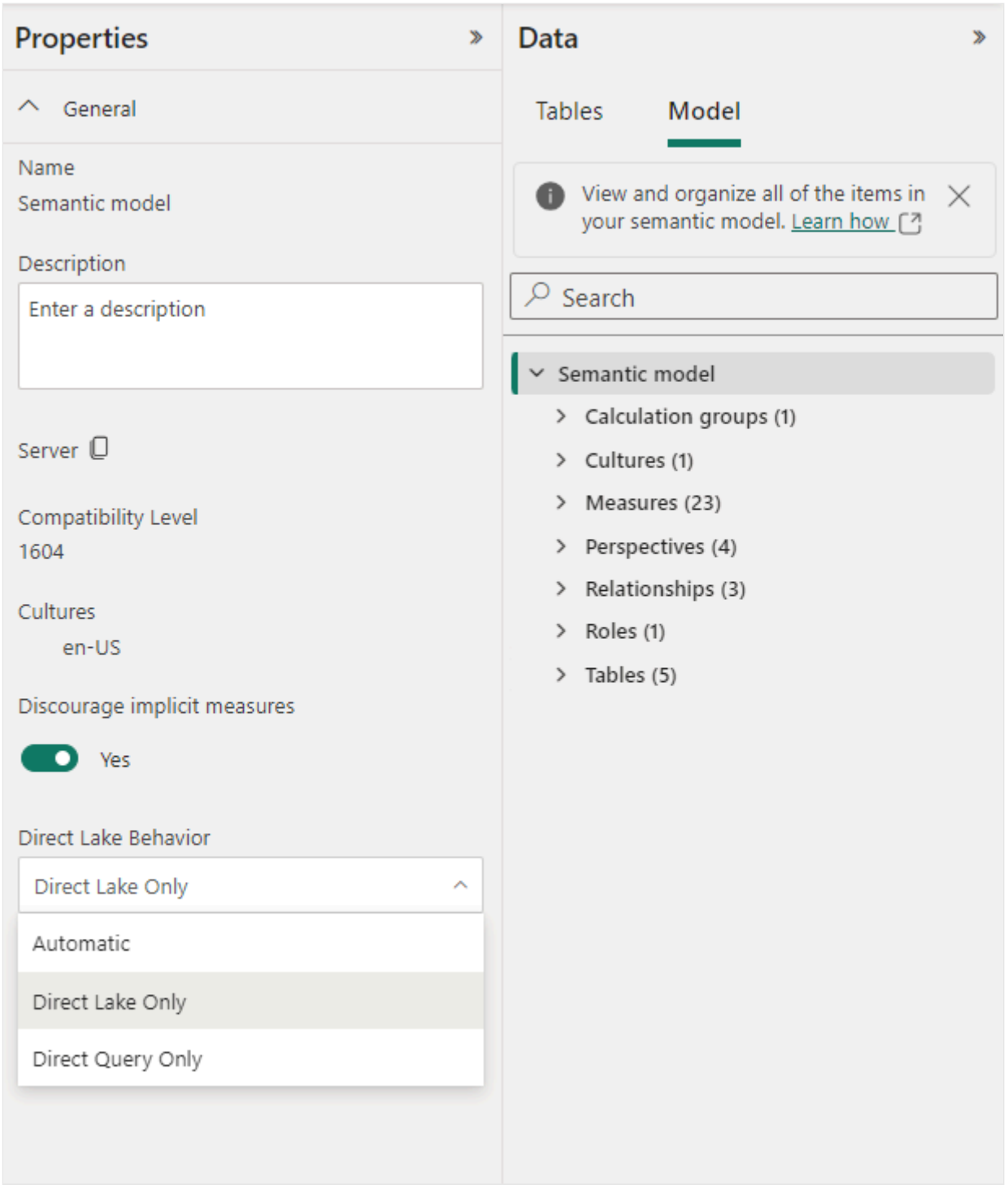
Set the **DirectLakeBehavior** property to one of the following three values:

Expand table

Value	Description
Automatic	(Default) If conditions aren't met, the query silently falls back to DirectQuery mode. Reports continue to work, but performance might be slower. Use this mode in production to preserve functionality for your users.
DirectLakeOnly	If conditions aren't met, the query fails with an error. Use this mode during development to identify and resolve errors.
DirectQueryOnly	The query always uses DirectQuery mode. Use to this mode during development to measure fallback performance.

Set DirectLakeBehavior

In the **Model** view, open the **Properties** pane for the semantic model and change the **Direct Lake behavior** setting.



Set DirectLakeBehavior programmatically

You can also configure the **DirectLakeBehavior** property by using Tabular Object Model (TOM) or Tabular Model Scripting Language (TMSL).

The following example specifies all queries use Direct Lake mode only:

C#

```
// Disable fallback to DirectQuery mode.
database.Model.DirectLakeBehavior = DirectLakeBehavior.DirectLakeOnly;
database.Model.SaveChanges();
```

For more information, see Model.DirectLakeBehavior Property.

Diagnose fallback

To identify which tables are falling back and why, run the following DAX query:

DAX

```
EVALUATE TABLETRAITS()
```

The `[DirectLakeFallbackInfo]` column shows the fallback reason for each table. A value of `None` means the table is using Direct Lake mode.

Fix common fallback causes

Use this table to identify the fix for each fallback scenario:

Expand table

Fallback cause	How to fix
Table isn't framed	Refresh the semantic model to frame the tables. After adding tables programmatically via TOM or TMSL, always refresh before querying.
Table is based on a SQL view	Materialize the view as a delta table, or accept DirectQuery performance for that table.
Table doesn't exist	Verify the delta table exists in the lakehouse or warehouse. Check for schema drift or deleted tables.
Transient error	Retry the query. If persistent, check capacity health and refresh the semantic model.
OLS defined at SQL endpoint	Move object-level security to the semantic model, or accept DirectQuery fallback.
RLS or DDM defined at SQL endpoint	Move row-level security to the semantic model, or accept DirectQuery fallback.
Delta table exceeds guardrails	Run `OPTIMIZE` and `VACUUM` on the delta table to reduce parquet files and row groups. If the table still exceeds limits, upgrade to a higher Fabric SKU.

Fallback cause	How to fix
Capacity under memory pressure	Reduce concurrent workloads, optimize other models, or upgrade the capacity SKU.

For detailed query-level analysis using Performance Analyzer or SQL Server Profiler, see Analyze query processing for Direct Lake semantic models.

Related content

- Direct Lake overview
- Develop Direct Lake semantic models
- Manage Direct Lake semantic models
- Understand Direct Lake query performance

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 08/14/2026

Understand Direct Lake query performance

06/27/2025

Apart from semantic model design and query complexity, Direct Lake performance specifically depends on well-tuned Delta tables for efficient and fast column loading (transcoding) and optimal query execution. Make sure you apply V-Order optimization. Also, keep the number of Parquet files small, use large row groups, and strive to minimize the effect of data updates on the Delta log. These are common best practices that can help to ensure fast query execution in cold, semiwarm, warm, and hot Direct Lake mode states.

This article explains how Direct Lake performance depends on Delta table health and efficient data updates. Understanding these dependencies is crucial. You learn that the data layout in your Parquet files is as important for query performance as are a good semantic model design and well-tuned data analysis expression (DAX) measures.

What you need to know

This article assumes that you're already familiar with the following concepts:

- **Delta tables in OneLake**: Detailed information about Delta tables in OneLake is available in the Microsoft Fabric fundamentals documentation.
- **Parquet files, row groups, and Delta log**: The Parquet file format, including how data is organized into row groups and column chunks, and how metadata is handled, is explained in the Parquet file format documentation. See also the Delta transaction log protocol documentation.
- **Delta table optimization and V-Order**: See the Fabric Lakehouse documentation about Delta table maintenance, such as Delta Lake table optimization and V-Order.
- **Framing and transcoding**: Refreshing a Direct Lake semantic model (framing) and on-demand column data loading (transcoding) are important concepts, covered at an introductory level in the Direct Lake overview article.
- **Formula and Storage Engine**: When a DAX query is executed, the Formula Engine generates the query plan and retrieves the necessary data and initial aggregations from the Storage Engine. The optimizations discussed in this article focus on the Storage Engine. To learn more about the Formula Engine and the Storage Engine, explore the Analysis Services developer documentation.
- **VertiPaq and VertiScan**: In import mode and in Direct Lake mode, the Storage Engine uses its VertiPaq engine to maintain a columnar in-memory store. VertiScan enables the Formula Engine to interact with VertiPaq. For more information, see the Analysis Services developer documentation.

- **Dictionary encoding**: Both Parquet files and VertiPaq use dictionary encoding, which is a data compression technique applied to individual columns of various data types, such as int, long, date, and char. It works by storing each unique column value in memory as an integer using Run Length Encoding (RLE)/Bit-Packing Hybrid encoding. VertiPaq always uses dictionary encoding, but Parquet might switch to plain encoding or delta encoding under some circumstances, as explained in Encodings in the Parquet File-Format documentation, which would require Direct Lake to re-encode the data with corresponding effect on transcoding performance.
- **Column chunks and column segments**: Refer to the way Parquet and VertiPaq store column data for efficient data retrieval. Each column in a table is divided into smaller chunks that can be processed and compressed independently. VertiPaq calls these chunks segments. You can use the DISCOVER_STORAGE_TABLE_COLUMN_SEGMENTS schema rowset to retrieve information about the column segments in a Direct Lake semantic model.
- **Python and Jupyter Notebooks**: Jupyter Notebooks provide an interactive environment to write and run Python code. Basic knowledge of Python is helpful if you want to follow the code snippets later in this chapter. For more information, see the Python language reference. For information on how to use notebooks in Microsoft Fabric, see How to use notebooks - Microsoft Fabric.

What affects Direct Lake query performance

This section summarizes the main factors affecting Direct Lake performance. Subsequent sections offer more detailed explanations:

- **V-Order compression**: The effectiveness of compression can affect query performance, as better compression leads to faster data loading and more efficient query processing. Data loading is fast because streaming compressed data boosts transcoding efficiency. Query performance is also optimal because V-Order compression enables VertiScan to compute results directly on top of compressed data, skipping the decompression step.
- **Data types**: Using appropriate data types for columns can improve compression and performance. For example, use integer data types instead of strings where possible and avoid storing integers as strings.
- **Segment size and count**: VertiPaq stores column data in segments. A large number of smaller segments can negatively affect query performance. For large tables, Direct Lake prefers large segment sizes, such as between 1 million to 16 million rows.
- **Column cardinality**: High cardinality columns (columns with many unique values) can slow down performance. Reducing cardinality where possible can help.
- **Indexes and aggregations**: Columns with lower cardinality benefit from dictionary encoding, which can speed up queries by reducing the amount of data that needs to be scanned.

- **DirectQuery fallback**: Fallback operations might result in slower query performance as the data must now be fetched from the SQL analytics endpoint of the Fabric data source. Moreover, fallback relies on hybrid query plans to support both DirectQuery and VertiScan with some tradeoffs on performance even when Direct Lake doesn't need to fallback. If possible, disable DirectQuery fallback to avoid hybrid query plans.
- **Degree of memory residency**: Direct Lake semantic models can be in cold, semiwarm, warm, or hot state with increasingly better performance from cold to hot. Transitioning quickly from cold to warm is a key to good Direct Lake performance.
  - *Cold*: VertiPaq store is empty. All data required to answer a DAX query must be loaded from Delta tables.
  - *Semiwarm*: Direct Lake only drops those column segments during framing that belong to removed row groups. This means that only updated or newly added data must be loaded. A Direct Lake semantic model can also go into semiwarm state under memory pressure when it must unload segments and join indexes due to memory pressure.
  - *Warm*: The column data required to answer a DAX query is already fully loaded into memory.
  - *Hot*: The column data is already fully loaded into memory, VertiScan caches are populated, and the DAX queries hit the caches.
- **Memory pressure**: Direct Lake must load all column data required to answer a DAX query into memory, which can exhaust available memory resources. With insufficient memory, Direct Lake must unload previously loaded column data, which Direct Lake then might have to reload again for subsequent DAX queries. Adequately sizing Direct Lake semantic models can help to avoid frequent reloading.

Memory residency and query performance

Direct Lake performs best in the warm or hot state, while cold states result in slower performance. Direct Lake avoids falling back to cold as much as possible by using incremental framing.

Bootstrapping

After the initial semantic model load, no column data is resident in memory yet. Direct Lake is cold. When a client submits a DAX query to a Direct Lake semantic model in the cold state, Direct Lake must perform the following main tasks so that the DAX query can be processed and answered:

- VertiPaq dictionary transcoding. Direct Lake must merge the local Parquet dictionaries for each column chunk to create a global VertiPaq dictionary for the column. This **merge** operation affects query response time.

- Loading Parquet column chunks into column segments. In most cases, this is a direct remapping of Parquet data IDs to VertiPaq IDs when both sides can use RLE/Bit-Packing Hybrid encoding. If the Parquet dictionaries use plain encoding, VertiPaq must convert the values to RLE/Bit-Packing Hybrid encoding, which takes longer.
  - Direct Lake performance is optimal on V-Ordered Parquet files because V-Ordering increases the quality of RLE compression. Direct Lake can load tightly packed V-Ordered data faster than less compressed data.

- Generating join indexes. If the DAX query accesses columns from multiple tables, Direct Lake must build join indexes according to the table relationships so that VertiScan can join the tables correctly. To build the join indexes, Direct Lake must load the dictionaries of the key columns participating in the relationship and the column segments of the primary key column (the column on the One side of the table relationship).

- Applying Delta deletion vectors. If a source Delta table uses deletion vectors, Direct Lake must load these deletion vectors to ensure deleted data is excluded from query processing.

  > ⓘ **Note**
  >
  > The cold state can also be induced by sending a `processClear` followed by a `processFull` XMLA command to the model. The `ProcessClear` command removes all data and the association with the framed Delta table version. The `ProcessFull` XMLA command performs framing to bind the model to the latest available Delta commit version.

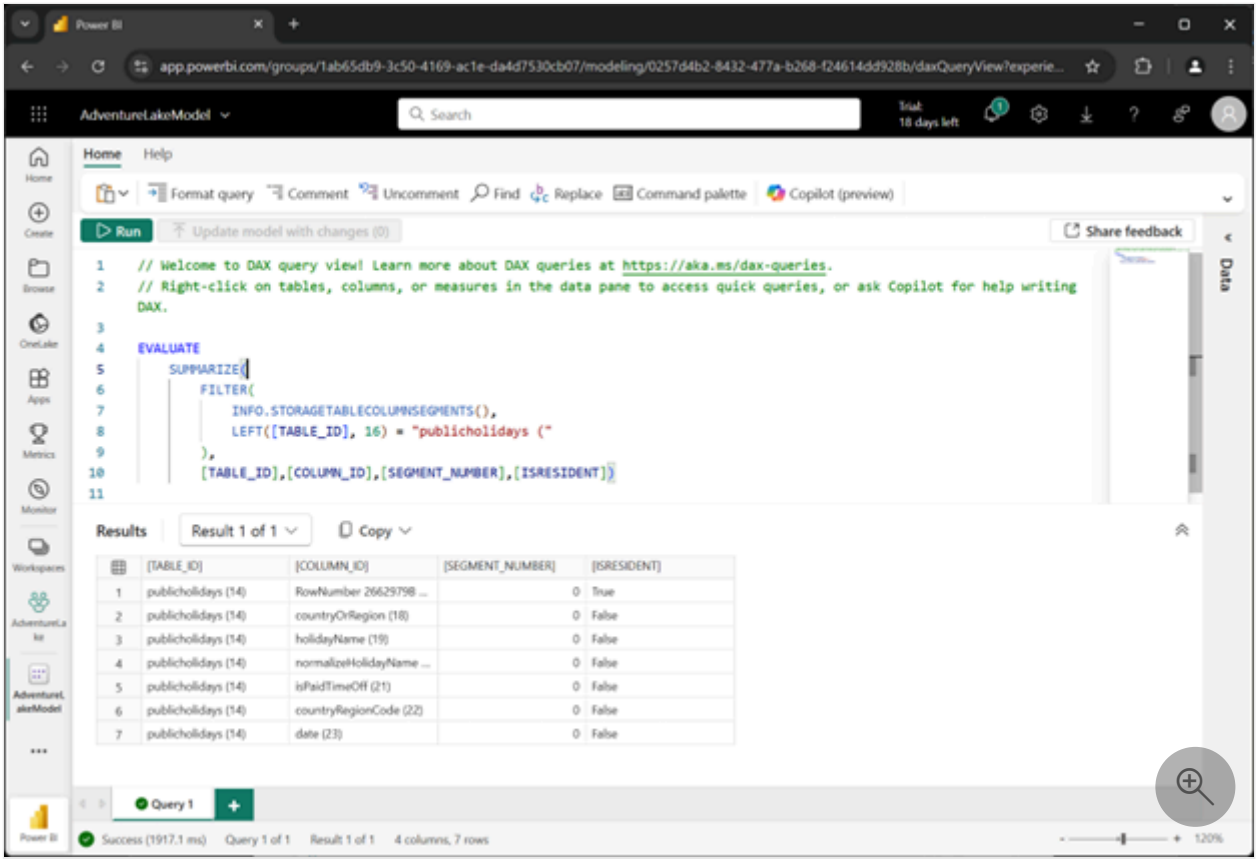
Incremental framing

During framing, Direct Lake analyzes the Delta log of each Delta table and drops loaded column segments and join indexes only when the underlying data has changed. Dictionaries are retained to avoid unnecessary transcoding and new values are simply added to the existing dictionaries. This incremental framing approach reduces the reload burden and benefits cold query performance.

You can analyze incremental framing effectiveness by using the `INFO.STORAGETABLECOLUMNSEGMENTS()` DAX function, which wraps the `DISCOVER_STORAGE_TABLE_COLUMN_SEGMENTS` schema rowset. Follow these steps to ensure meaningful results:

1. Query your Direct Lake semantic model to ensure it is in warm or hot state.

2. Update the Delta table you want to investigate and refresh the Direct Lake semantic model to perform framing.
3. Run a DAX query to retrieve column segment information by using the `INFO.STORAGETABLECOLUMNSEGMENTS()` function, as in the following screenshot. The screenshot uses a small sample table for illustration purposes. Each column only has a single segment. The segments aren't resident in memory. This indicates a true cold state. If the model was warm before framing, this means that the Delta table was updated using a destructive data loading pattern, making it impossible to use incremental framing. Delta table update patterns are covered later in this article.



> **Note**
>
> When a Delta table receives no updates, no reload is necessary for columns already resident in memory. When using nondestructive update patterns, queries show far less performance effect after framing because incremental framing essentially enables Direct Lake to update substantial portions of the existing in-memory data in place.

Full memory residency

With dictionaries, column segments, and join indexes loaded, Direct Lake reaches the warm state with query performance on par with import mode. In both modes, the number and size of column segments play a crucial role in optimizing query performance.

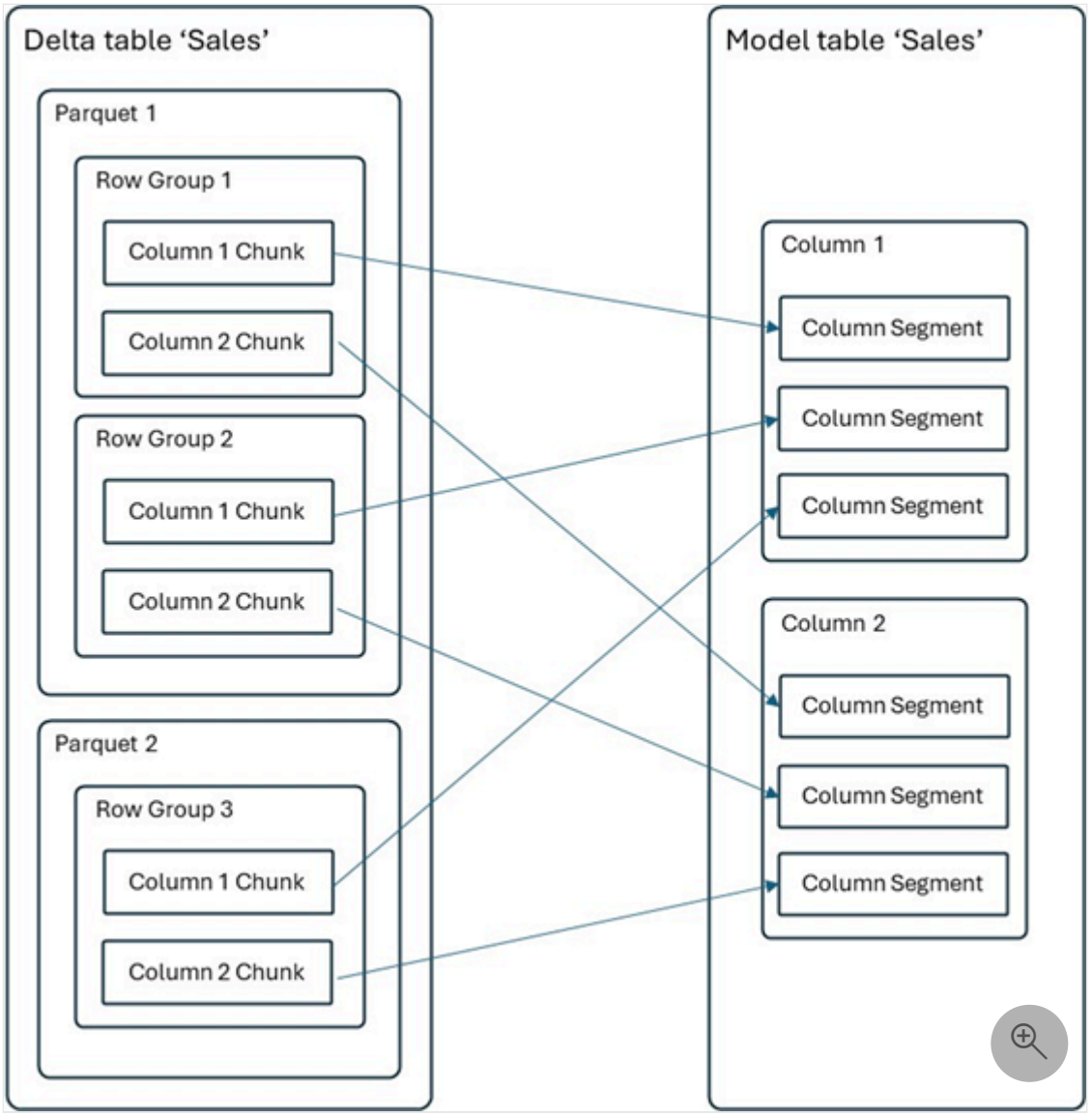
Delta table differences

Parquet files organize data by columns rather than rows. Direct Lake also organizes data by columns. The alignment facilitates seamless integration, yet there are important differences, specifically concerning row groups and dictionaries.

Row groups versus column segments

A row group in a Parquet file consists of column chunks, and each chunk contains data for a specific column. A column segment in a semantic model, on the other hand, also contains a chunk of column data.

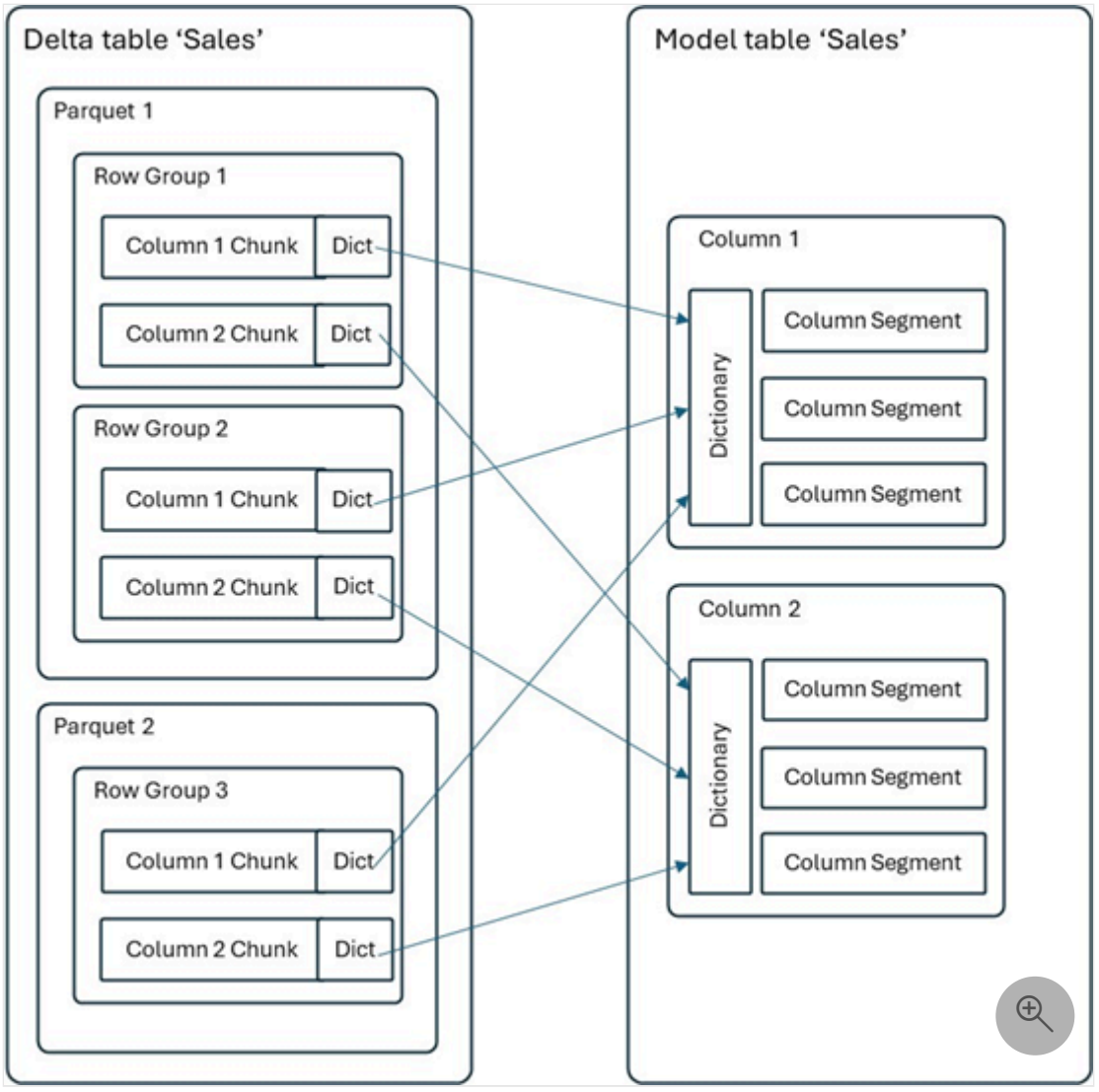
There's a direct relationship between the total row group count of a Delta table and the segment count for each column of the corresponding semantic model table. For example, if a Delta table across all its current Parquet files has three row groups in total, then the corresponding semantic model table has three segments per column, as illustrated in the following diagram. In other words, if a Delta table has a large number of tiny row groups, a corresponding semantic model table would also have a large number of tiny column segments. This would negatively affect query performance.



> **Note**
>
> Because Direct Lake prefers large column segments, the row groups of the source Delta tables should ideally be large.

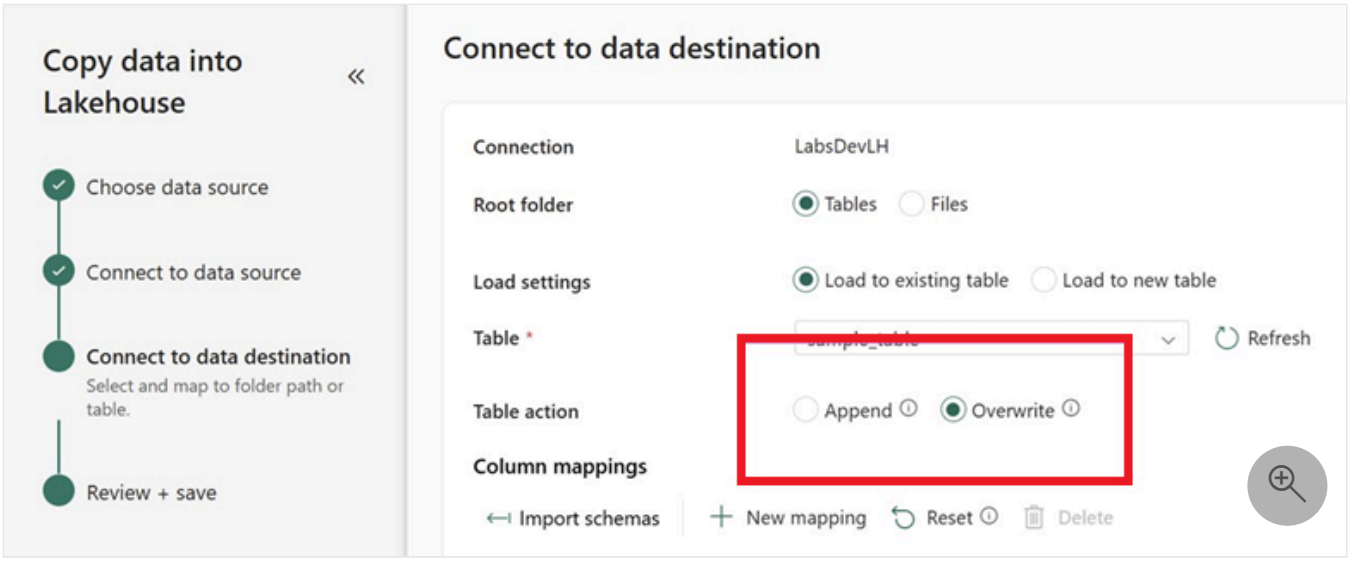
Local dictionaries versus global dictionary

The total row group count of a Delta table also has direct effect on dictionary transcoding performance because Parquet files use local dictionaries while Direct Lake semantic models use a global dictionary for each column, as depicted in the following diagram. The higher the number of row groups, the higher the number of local dictionaries that Direct Lake must merge to create a global dictionary, and the longer it takes for transcoding to complete.



Delta table update patterns

The method used to ingest data into a Delta table can greatly influence incremental framing efficiency. For instance, using the **Overwrite** option when loading data into an existing table erases the Delta log with each load. This means Direct Lake can't use incremental framing and must reload all the data, dictionaries, and join indexes. Such destructive update patterns negatively affect query performance.



This section covers Delta table update patterns that enable Direct Lake to use incremental framing, preserving VertiPaq column store elements like dictionaries, column segments, and join indexes, to maximize transcoding efficiency and boost cold query performance.

Batch processing without partitioning

This update pattern collects and processes data in large batches at scheduled intervals, such as on a weekly or monthly basis. As new data arrives, old data is often removed in a rolling or sliding window fashion to keep the table size under control. However, removing old data can be a challenge if the data is spread across most of the Parquet files. For example, removing one day out of 30 days might affect 95% of the Parquet files instead of 5%. In this case, Direct Lake would have to reload 95% of the data even for a relatively small **delete** operation. The same issue also applies to updates of existing rows because updates are combined deletes and appends. You can analyze the effect of **delete** and **update** operations by using Delta Analyzer, as explained later in this article.

Batch processing with partitioning

Delta table partitioning can help to reduce the effect of **delete** operations as the table is divided into smaller Parquet files stored in folders based on the distinct values in the partition column. Commonly used partition columns include date, region, or other dimensional categories. In the previous example of removing one day out of 30 days, a Delta table partitioned by date would constrain the deletes to only the Parquet files of the partition for that day. However, it's important to note that extensive partitioning could result in a substantially increased number of Parquet files and row groups, thereby causing an excessive increase in column segments within the Direct Lake semantic model, negatively affecting query performance. Choosing a low-cardinality partition column is crucial for query performance. As a best practice, the column should have fewer than 100-200 distinct values.

Incremental loading

With incremental loading, the update process only inserts new data into a Delta table without affecting existing Parquet files and row groups. There are no deletes. Direct Lake can load the new data incrementally without having to discard and reload existing VertiPaq column store elements. This method works well with Direct Lake incremental framing. Delta table partitioning isn't necessary.

Stream processing

Processing data near real-time, as it arrives, can cause a proliferation of small Parquet files and row groups, which can negatively affect Direct Lake performance. As with the incremental loading pattern, it isn't necessary to partition the Delta table. However, frequent table maintenance is essential to ensure that the number of Parquet files and row groups remains within the guardrail limits specified in the Direct Lake overview article. In other words, don't forget to run Spark Optimize regularly, such as daily or even more often. Spark Optimize is covered again in the next section.

> **Note**
>
> Actual real-time analysis is best implemented using Eventstreams, KQL databases, and Eventhouse. Refer to the Real-Time Intelligence documentation in Microsoft Fabric for guidance.

Delta table maintenance

Key maintenance tasks include vacuuming and optimizing Delta tables. To automate maintenance operations, you can use the Lakehouse APIs as explained in the Manage the Lakehouse with Microsoft Fabric REST API documentation.

Vacuuming

Vacuuming removes Parquet files no longer included in the current Delta commit version and older than a set retention threshold. Removing these Parquet files doesn't affect Direct Lake performance because Direct Lake only loads the Parquet files that are in the current commit version. If you run VACUUM daily with the default values, the Delta commit versions of the last seven days are retained for time travel.

> **Important**

Because a framed Direct Lake semantic model references a particular Delta commit version, you must ensure that the Delta table keeps this version until you refresh (frame) the model again to move it to the current version. Otherwise, users encounter query errors when the Direct Lake semantic model tries to access Parquet files that no longer exist.

Spark Optimize

Delta table optimization merges multiple small Parquet files into fewer large files. Because this can affect cold Direct Lake performance, it's good practice to optimize infrequently, such as over weekends or at the end of the month. Optimize more often if small Parquet files accumulate quickly (high-frequency small updates) to ensure the Delta table stays within guardrail limits.

Partitioning can help to minimize optimization effect on incremental framing because partitioning effectively collocates the data. For example, partitioning a large Delta table based on a low-cardinality date_key column would constrain weekly maintenance to a maximum of seven partitions. The Delta table would retain most of the existing Parquet files. Direct Lake would only have to reload seven days of data.

Analyzing Delta table updates

Use Delta Analyzer or similar tools to study how Delta table updates affect Parquet files and row groups. Delta Analyzer lets you track the evolution of Parquet files, row groups, column chunks, and columns in response to **append**, **update**, and **delete** operations. Delta Analyzer is available as a standalone Jupyter Notebook. It's also available in the semantic-link-labs library. The following sections use semantic-link-labs. This library is easy to install in a notebook using the `%pip install semantic-link-labs` command.

Row group size

The ideal row-group size for Direct Lake semantic models is between 1 million and 16 million rows, yet Fabric might use larger row group sizes for large tables if the data is compressible. Generally, we don't recommend that you change the default row group size. It's best to let Fabric manage the Delta table layout. But it's also a good idea to double check.

The following Python code can serve as a starting point to analyze the row group sizes and other details of a Delta table in a Fabric notebook-connected lakehouse. The following table shows the output for a sample table with 1 billion rows.

```
import sempy_labs as labs
from IPython.display import HTML
from IPython.display import clear_output

table_name = "<Provide your table name>"

# Load the Delta table and run Delta Analyzer
df = spark.read.format("delta").load(f"Tables/{table_name}")
da_results = labs.delta_analyzer(table_name)

# Display the table summary in an HTML table.
clear_output()

df1 = da_results['Summary'].iloc[0]

html_table = "<table border='1'><tr><th>Column Name</th><th>{table_name}</th>
</tr>"
for column in da_results['Summary'].columns:
    html_table += f"<tr><td>{column}</td><td>{df1[column]}</td></tr>"
html_table += "</table>"

display(HTML(html_table))
```

Output:

Expand table

Parameter	Value
Table name	sales_1
Row count	1000000000
Row groups	24
Parquet files	8
Max rows per row group	51210000
Min rows per row group	22580000
Avg rows per row group	41666666.666666664
VOrder enabled	True
Total size	7700808430
Timestamp	2025-03-24 03:01:02.794979

The Delta Analyzer summary shows an average row group size of approximately 40 million rows. This is larger than the recommended maximum row group size of 16 million rows.

Fortunately, the larger row group size doesn't cause significant issues for Direct Lake. Larger row groups facilitate continuous segment jobs with minimal overhead in the Storage Engine. Conversely, small row groups, those significantly under 1 million rows, can cause performance issues.

More important in the previous example is that Fabric distributed the row groups across eight Parquet files. This aligns with the number of cores on the Fabric capacity to support efficient parallel **read** operations. Also important is that the individual row group sizes don't deviate too far from the average. Large variations can cause nonuniform VertiScan load, resulting in less optimal query performance.

Rolling window updates

For illustration purposes, the following Python code sample simulates a rolling window update. The code removes the rows with the oldest DateID from a sample Delta table. It then updates the DateID of these rows and inserts them back again into the sample table as the most recent rows.

```
from pyspark.sql.functions import lit

table_name = "<Provide your table name>"
table_df = spark.read.format("delta").load(f"Tables/{table_name}")

# Get the rows of the oldest DateID.
rows_df = table_df[table_df["DateID"] == 20200101]
rows_df = spark.createDataFrame(rows_df.rdd, schema=rows_df.schema)

# Delete these rows from the table
table_df.createOrReplaceTempView(f"{table_name}_view")
spark.sql(f"DELETE From {table_name}_view WHERE DateID = '20200101'")

# Update the DateID and append the rows as new data
rows_df = rows_df.withColumn("DateID", lit(20250101))
rows_df.write.format("delta").mode("append").save(f"Tables/{table_name}")
```

The `get_delta_table_history` function in the semantic-link-labs library can help to analyze the effect of this rolling window update. See the following Python code sample. See also the table with the output after the code snippet.

```
import sempy_labs as labs
from IPython.display import HTML
from IPython.display import clear_output
```

```
table_name = "<Provide your table name>"
da_results = labs.get_delta_table_history(table_name)

# Create a single HTML table for specified columns
html_table = "<table border='1'>"
# Add data rows for specified columns
for index, row in da_results.iterrows():
    for column in ['Version', 'Operation', 'Operation Parameters', 'Operation
Metrics']:
        if column == 'Version':
            html_table += f"<tr><td><b>Version</b></td><td><b>{row[column]}</b>
</td></tr>"
        else:
            html_table += f"<tr><td>{column}</td><td>{row[column]}</td></tr>"
html_table += "</table>"

# Display the HTML table
display(HTML(html_table))
```

Output:

Expand table

Version	Description	Value
2	**Operation**	WRITE
	Operation parameters	{'mode': 'Append', 'partitionBy': '[]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '548665', 'numOutputBytes': '4103076'}
1	**Operation**	DELETE
	Operation parameters	{'predicate': '["(DateID#3910 = 20200101)"]'}
	Operation metrics	{'numRemovedFiles': '8', 'numRemovedBytes': '7700855198', 'numCopiedRows': '999451335', 'numDeletionVectorsAdded': '0', 'numDeletionVectorsRemoved': '0', 'numAddedChangeFiles': '0', 'executionTimeMs': '123446', 'numDeletionVectorsUpdated': '0', 'numDeletedRows': '548665', 'scanTimeMs': '4820', 'numAddedFiles': '18', 'numAddedBytes': '7696900084', 'rewriteTimeMs': '198625'}
0	**Operation**	WRITE
	Operation parameters	{'mode': 'Overwrite', 'partitionBy': '[]'}

Version	Description	Value
	Operation metrics	{'numFiles': '8', 'numOutputRows': '1000000000', 'numOutputBytes': '7700892169'}

The Delta Analyzer history above shows that this Delta table now has the following three versions:

- **Version 0**: This is the original version with eight Parquet files and 24 row groups as discussed in the previous section.
- **Version 1**: This version reflects the **delete** operation. Although only a single day's worth of data (DateID = '20200101') was removed from the sample table with five years of sales transactions, all eight Parquet files were affected. In the Operation Metrics, `numRemovedFiles` is eight [Parquet files] and `numAddedFiles` is 18 [Parquet files]. The means that the **delete** operation replaced the original eight Parquet files with 18 new Parquet files.
- **Version 3**: The Operation Metrics reveal that one more Parquet file with 548,665 rows was added to the Delta table.

After the rolling window update, the most current Delta commit version includes 19 Parquet files and 21 row groups, with sizes ranging from 500 thousand to 50 million rows. The rolling window update of 548,665 rows affected the entire Delta table of 1 billion rows. It replaced all Parquet files and row groups. Incremental framing can't be expected to improve cold performance in this case, and the increased variation in row group sizes is unlikely to benefit warm performance.

Delta table updates

The following Python code updates a Delta table in essentially the same way as described in the previous section. On the surface, the update function only changes the DateID value of the existing rows that match a given DateID. However, Parquet files are immutable and can't be modified. Below the surface, the **update** operation removes existing Parquet files and adds new Parquet files. The outcome and effect are the same as for the rolling window update.

```
from pyspark.sql.functions import col, when
from delta.tables import DeltaTable

# Load the Delta table
table_name = "<Provide your table name>"
delta_table = DeltaTable.forPath(spark, f"Tables/{table_name}")

# Define the condition and the column to update
```

```
condition = col("DateID") == 20200101
column_name = "DateID"
new_value = 20250101

# Update the DateID column based on the condition
delta_table.update(
    condition,
    {column_name: when(condition, new_value).otherwise(col(column_name))}
)
```

Partitioned rolling window updates

Partitioning can help to reduce the effect of table updates. It might be tempting to use the date keys, but a quick cardinality check can reveal that this isn't the best choice. For example, the sample table discussed so far contains sales transactions for the last five years, equivalent to about 1800 distinct date values. This cardinality is too high. The partition column should have fewer than 200 distinct values.

```

column_name = 'DateID'
table_name = "<Provide your table name>"
table_df = spark.read.format("delta").load(f"Tables/{table_name}")

distinct_count = table_df.select(column_name).distinct().count()
print(f"The '{column_name}' column has {distinct_count} distinct values.")

if distinct_count <= 200:
    print(f"The '{column_name}' column is a good partitioning candidate.")

table_df.write.format("delta").partitionBy(column_name).save(f"Tables/{table_name}
_by_date_id")
    print(f"Table '{table_name}_by_date_id' partitioned and saved successfully.")
else:
    print(f"The cardinality of the '{column_name}' column is possibly too high.")
```

Output:

```

The 'DateID' column has 1825 distinct values.
The cardinality of the 'DateID' column is possibly too high.
```

If there's no suitable partition column, it can be created artificially by reducing the cardinality of an existing column. The following Python code adds a Month column by removing the last two digits of the DateID. This produces 60 distinct values. The sample code then saves the Delta table partitioned by the Month column.

```
from pyspark.sql.functions import col, expr

column_name = 'DateID'
table_name = "sales_1"
table_df = spark.read.format("delta").load(f"Tables/{table_name}")

partition_column = 'Month'
partitioned_table = f"{table_name}_by_month"
table_df = table_df.withColumn(partition_column, expr(f"int({column_name} /
100)"))

distinct_count = table_df.select(partition_column).distinct().count()
print(f"The '{partition_column}' column has {distinct_count} distinct values.")

if distinct_count <= 200:
    print(f"The '{partition_column}' column is a good partitioning candidate.")

table_df.write.format("delta").partitionBy(partition_column).save(f"Tables/{partit
ioned_table}")
    print(f"Table '{partitioned_table}' partitioned and saved successfully.")
else:
    print(f"The cardinality of the '{partition_column}' column is possibly too
high.")
```

Output:

```
The 'Month' column has 60 distinct values.
The 'Month' column is a good partitioning candidate.
Table 'sales_1_by_month' partitioned and saved successfully.
```

The Delta Analyzer summary now shows that the Delta table layout is well aligned with Direct Lake. The average row group size is about 16 million rows, and the mean absolute deviation of the row group sizes and therefore segment sizes is fewer than 1 million rows.

Expand table

Parameter	Value
Table name	sales_1_by_month
Row count	1000000000
Row groups	60
Parquet files	60

Parameter	Value
Max rows per row group	16997436
Min rows per row group	15339311
Avg rows per row group	16666666.666666666
VOrder enabled	True
Total size	7447946016
Timestamp	2025-03-24 03:01:02.794979

After a rolling window update against a partitioned sample table, the Delta Analyzer history shows that only one Parquet file was affected. See the following output table. Version 2 has exactly 16,445,655 rows copied over from the old Parquet file into a replacement Parquet file, and Version 3 adds a new Parquet file with 548,665 rows. In total, Direct Lake only needs to reload about 17 million rows, a sizeable improvement over a 1-billion-rows reload without partitioning.

Expand table

Version	Description	Value
2	**Operation**	WRITE
	Operation parameters	{'mode': 'Append', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '548665', 'numOutputBytes': '4103076'}
1	**Operation**	DELETE
	Operation parameters	{'predicate': '["(DateID#3910 = 20200101)"]'}
	Operation metrics	{'numRemovedFiles': '1', 'numRemovedBytes': '126464179', 'numCopiedRows': '16445655', 'numDeletionVectorsAdded': '0', 'numDeletionVectorsRemoved': '0', 'numAddedChangeFiles': '0', 'executionTimeMs': '19065', 'numDeletionVectorsUpdated': '0', 'numDeletedRows': '548665', 'scanTimeMs': '1926', 'numAddedFiles': '1', 'numAddedBytes': '121275513', 'rewriteTimeMs': '17138'}
0	**Operation**	WRITE
	Operation parameters	{'mode': 'Overwrite', 'partitionBy': '["Month"]'}

Version	Description	Value
	Operation metrics	{'numFiles': '60', 'numOutputRows': '1000000000', 'numOutputBytes': '7447681467'}

Append-only pattern followed by Spark Optimize

Append-only patterns don't affect existing Parquet files. They work well with Direct Lake incremental framing. However, don't forget to optimize your Delta tables to consolidate Parquet files and row groups, as discussed earlier in this article. Small and frequent appends can accumulate files quickly and can distort the uniformity of the row group sizes.

The following output shows the Delta Analyzer history for a nonpartitioned table compared to a partitioned table. The history includes seven appends and one subsequent **optimize** operation.

Expand table

Version	Description	Value in *default* layout	Value in *partitioned* layout
8	**Operation**	OPTIMIZE	OPTIMIZE
	Operation parameters	{'predicate': '[]', 'auto': 'false', 'clusterBy': '[]', 'vorder': 'true', 'zOrderBy': '[]'}	{'predicate': '["('Month >= 202501)"]', 'auto': 'false', 'clusterBy': '[]', 'vorder': 'true', 'zOrderBy': '[]'}
	Operation metrics	{'numRemovedFiles': '8', 'numRemovedBytes': '991234561', 'p25FileSize': '990694179', 'numDeletionVectorsRemoved': '0', 'minFileSize': '990694179', 'numAddedFiles': '1', 'maxFileSize': '990694179', 'p75FileSize': '990694179', 'p50FileSize': '990694179', 'numAddedBytes': '990694179'}	{'numRemovedFiles': '7', 'numRemovedBytes': '28658548', 'p25FileSize': '28308495', 'numDeletionVectorsRemoved': '0', 'minFileSize': '28308495', 'numAddedFiles': '1', 'maxFileSize': '28308495', 'p75FileSize': '28308495', 'p50FileSize': '28308495', 'numAddedBytes': '28308495'}
7	**Operation**	WRITE	WRITE
	Operation parameters	{'mode': 'Append', 'partitionBy': '[]'}	{'mode': 'Append', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '547453', 'numOutputBytes': '4091802'}	{'numFiles': '1', 'numOutputRows': '547453', 'numOutputBytes': '4091802'}
6	**Operation**	WRITE	WRITE

Version	Description	Value in *default* layout	Value in *partitioned* layout
	Operation parameters	{'mode': 'Append', 'partitionBy': '[]'}	{'mode': 'Append', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '548176', 'numOutputBytes': '4095497'}	{'numFiles': '1', 'numOutputRows': '548176', 'numOutputBytes': '4095497'}
5	**Operation**	WRITE	WRITE
	Operation parameters	{'mode': 'Append', 'partitionBy': '[]'}	{'mode': 'Append', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '547952', 'numOutputBytes': '4090107'}	{'numFiles': '1', 'numOutputRows': '547952', 'numOutputBytes': '4093015'}
4	**Operation**	WRITE	WRITE
	Operation parameters	{'mode': 'Append', 'partitionBy': '[]'}	{'mode': 'Append', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '548631', 'numOutputBytes': '4093134'}	{'numFiles': '1', 'numOutputRows': '548631', 'numOutputBytes': '4094376'}
3	**Operation**	WRITE	WRITE
	Operation parameters	{'mode': 'Append', 'partitionBy': '[]'}	{'mode': 'Append', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '548671', 'numOutputBytes': '4101221'}	{'numFiles': '1', 'numOutputRows': '548671', 'numOutputBytes': '4101221'}
2	**Operation**	WRITE	WRITE
	Operation parameters	{'mode': 'Append', 'partitionBy': '[]'}	{'mode': 'Append', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '546530', 'numOutputBytes': '4081589'}	{'numFiles': '1', 'numOutputRows': '546530', 'numOutputBytes': '4081589'}
1	**Operation**	WRITE	WRITE
	Operation parameters	{'mode': 'Append', 'partitionBy': '[]'}	{'mode': 'Append', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '1', 'numOutputRows': '548665', 'numOutputBytes': '4101048'}	{'numFiles': '1', 'numOutputRows': '548665', 'numOutputBytes':

Version	Description	Value in *default* layout	Value in *partitioned* layout
			'4101048'}
0	**Operation**	WRITE	WRITE
	Operation parameters	{'mode': 'Overwrite', 'partitionBy': '[]'}	{'mode': 'Overwrite', 'partitionBy': '["Month"]'}
	Operation metrics	{'numFiles': '8', 'numOutputRows': '1000000000', 'numOutputBytes': '7700855198'}	{'numFiles': '60', 'numOutputRows': '1000000000', 'numOutputBytes': '7447681467'}

Looking at the Operation Metrics of Version 8, it's worth pointing out that the **optimize** operation for the nonpartitioned tabled merged eight Parquet files affecting roughly 1 GB of data while the **optimize** operation of the partitioned table merged seven Parquet files affecting only about 25 MB of data. It follows that Direct Lake would perform better with the partitioned table.

Considerations and limitations

Considerations and limitations for optimizing Direct Lake performance are as follows:

- Avoid destructive update patterns on large Delta tables to preserve incremental framing in Direct Lake.
- Small Delta tables don't need to be optimized for incremental framing.
- Aim for a row group size of between 1 million to 16 million rows to create column segments in Direct Lake with 1 million to 16 million rows. Direct Lake prefers large column segments.
- Avoid high cardinality partition columns because Direct Lake transcoding is less efficient with many small Parquet files and row groups than with fewer large Parquet files and row groups.
- Due to unforeseen demand for compute and memory resources, a semantic model might be reloaded onto another Fabric cluster node in cold state.
- Direct Lake doesn't use delta\Parquet statistics for row group\file skipping to optimize data loading.

Integrate Direct Lake security

Direct Lake security ensures that only authorized users can query Delta tables in OneLake. You can manage data access permissions through workspace roles. Workspace contributors, members, and admins can read data in OneLake. You can also grant access to the data in OneLake through item-level and compute permissions. The third option is to leverage OneLake security to enforce granular role-based security across all Fabric compute engines. This article explains how to align permission models, choose single sign-on (SSO) or fixed identities, and leverage object-level security (OLS) and row-level security (RLS). Learn more in OneLake security overview.

Key concepts and terminology

This article assumes you're familiar with these concepts:

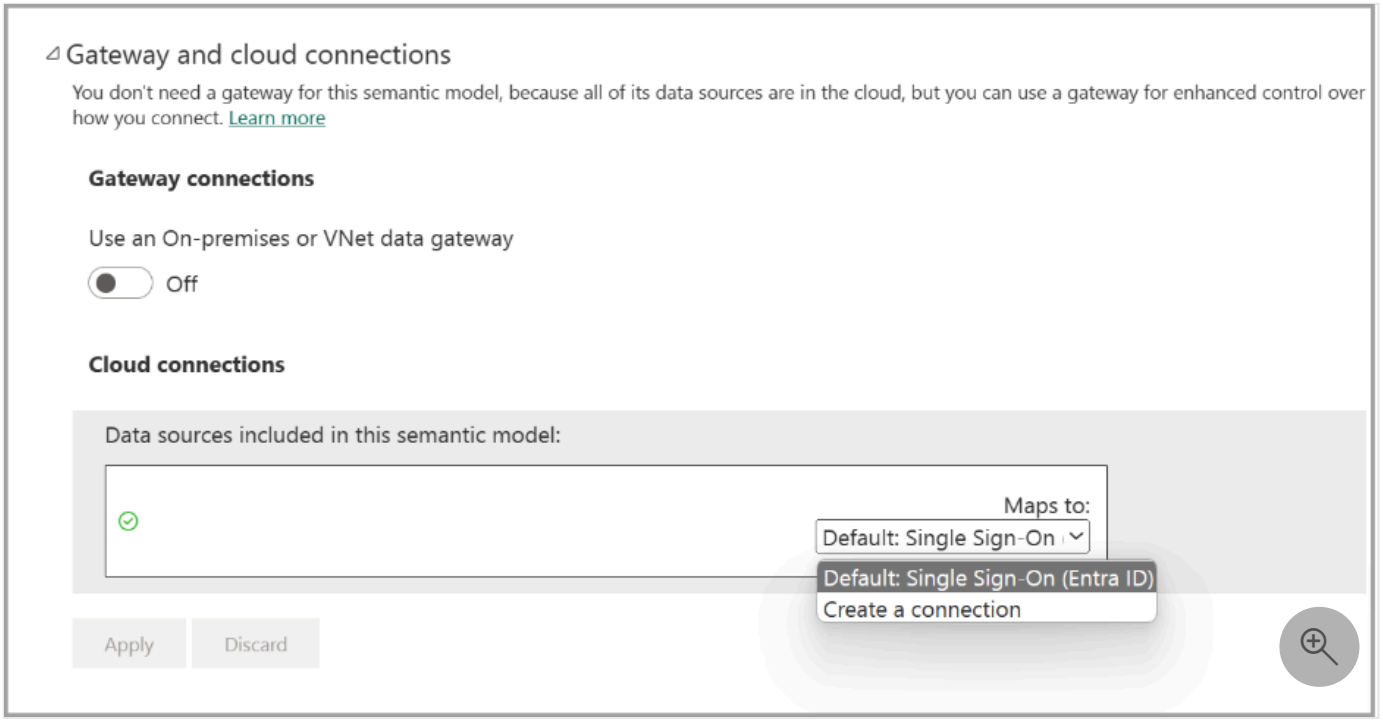
- Direct Lake uses shared M expressions in the semantic model metadata to reference data sources through Power Query data access functions: *AzureStorage.DataLake* for Direct Lake on OneLake and *Sql.Database* for Direct Lake on SQL endpoints. However, Direct Lake doesn't use these functions to read the source Delta tables. It reads the Delta tables directly through OneLake APIs.
- To ensure only authorized users query the data, Direct Lake checks the data access permissions of the effective identity. The effective identity depends on the data connection configuration. By default, Direct Lake uses SSO (Microsoft Entra ID) and uses the identity of the current user querying the semantic model. You can also bind a Direct Lake model to an explicit cloud connection to provide a fixed identity.
- If you grant data access permissions through workspace roles, only members of the Contributors role (or higher) can read data in OneLake. Workspace Viewers, however, don't have *read* permission in OneLake. Viewers and users who aren't members of a workspace role can get *read* access through a combination of item permissions, compute permissions, or OneLake security roles.
- OneLake security lets members of the Workspace Admin and Workspace Member roles define granular role-based security for users in the Viewer role. Specify the tables a Viewer or user with explicit *read* permission can access and exclude specific rows or columns. To learn more about OneLake security roles, see Table security in OneLake, Column-level security in OneLake, and RLS in OneLake.

Connection configuration

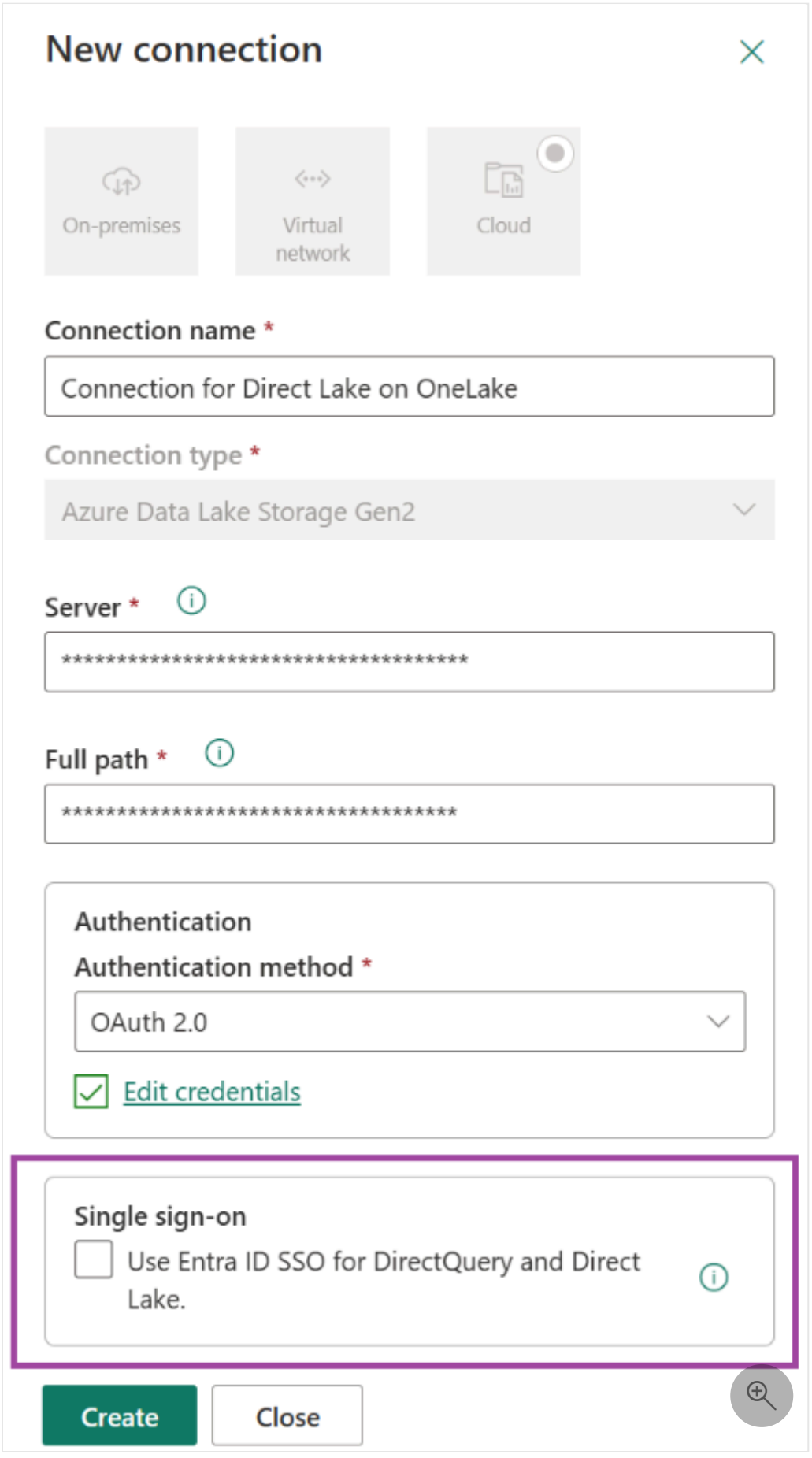
Configure data connections for a Direct Lake model the same way as other semantic model types. See Connect to cloud data sources in the Power BI service for details.

Because Direct Lake connects only to Fabric data sources, the default SSO (Microsoft Entra ID) configuration usually works, so you don't need to bind semantic models to explicit data connections. This approach reduces configuration complexity and lowers management overhead.

With SSO (Microsoft Entra ID), Direct Lake checks that the current user querying the semantic model has *read* access to the data. Only users with *read* access can query the data. The following screenshot shows a Direct Lake model using the default SSO configuration.



When you use an explicit data connection with a fixed identity instead of SSO, Direct Lake doesn't require every user to have *read* permission on the underlying data. If Microsoft Entra SSO remains disabled in the data connection, the fixed identity's permissions determine what data Direct Lake can access.


New connection
On-premises
Virtual network
Cloud
Connection name *
Connection for Direct Lake on OneLake
Connection type *
Azure Data Lake Storage Gen2
Server *
Full path *
Authentication
Authentication method *
OAuth 2.0
Edit credentials
Single sign-on
Use Entra ID SSO for DirectQuery and Direct Lake.
Create
Close

Note

You can configure a data connection to use both SSO and a fixed identity. Direct Lake checks the current user's permissions at query time and uses the fixed identity for framing and transcoding at refresh time. To use a fixed identity for both queries and refreshes, make sure SSO is disabled in the data connection configuration.

Tip

Use SSO for interactive scenarios where per-user authorization is required. Use a fixed-identity cloud connection for embedded or read-only consumer scenarios where source-level access is scoped to a single service account. Apply least-privilege principles at both the source and workspace levels, and test and validate behavior for both authentication modes before production deployment.

Authentication requirements

Direct Lake models use Microsoft Entra ID authentication. In the data connection configuration, choose **OAuth 2.0**, **Service Principal**, or **Workspace Identity** as the authentication method. Other methods, like key or SAS authentication, might appear in the configuration UI but aren't supported for Direct Lake models.

Permission requirements

The permission requirements differ between Direct Lake on SQL endpoints and Direct Lake on OneLake. This difference exists because Direct Lake on SQL endpoints relies on the SQL Analytics Endpoint of the target data source, whereas Direct Lake on OneLake uses the OneLake APIs for permission checks.

Direct Lake on SQL endpoints

Direct Lake on SQL endpoints checks permissions through the SQL analytics endpoint to see if the effective identity trying to access the data has the right permissions. The effective identity doesn't need permission to read Delta tables directly in OneLake. It only needs *read* access to the Fabric item, like a lakehouse, and SELECT permission on a table through its SQL analytics endpoint. Fabric grants the semantic model the permissions it needs to read the Delta tables and related Parquet files to **load column data** into memory. The semantic model can read the SQL analytics endpoint regularly to check what data the querying user (or fixed identity) can access.

Direct Lake on OneLake

Direct Lake on OneLake doesn't use a SQL analytics endpoint to check permissions. It uses OneLake security. When OneLake security is on, Direct Lake on OneLake uses the current user (or fixed identity) to figure out OneLake security roles and enforce OLS and RLS on the target Fabric item. If OneLake security isn't on, Direct Lake on OneLake needs the effective identity to have Read and ReadAll permissions on the target Fabric item to access its Delta tables in OneLake. For more information about Read and ReadAll permissions, see Share items and set item-level permissions.

> **Note**
>
> Contributors (or higher) have Read and ReadAll permissions in OneLake. Viewers and users who aren't members of a workspace role must get Read and ReadAll permissions or be added to a OneLake security group. For more information about managing OneLake security groups, see OneLake data access control model.

Direct Lake users

The following scenarios list minimum permission requirements.

Expand table

Scenario	Direct Lake on SQL endpoints	Direct Lake on OneLake	Comments
Users can view reports	- Grant *Read* permission for the reports and *Read* permission for the semantic model. - If Direct Lake uses SSO, grant users at least *Read* permission for the target Fabric item and SELECT permissions for the tables.	- Grant *Read* permission for the reports and *Read* permission for the semantic model. - If Direct Lake uses SSO, grant users at least *Read* permission for the target Fabric item and add them to a OneLake security role or grant them *ReadAll* permission.	Reports don't need to belong to the same workspace as the semantic model. For more information, see Strategy for read-only consumers.
Users can create reports	- Grant *Build* permission for the semantic model. - If Direct Lake uses SSO, grant users at least *Read* permission for the target Fabric item and SELECT permissions for the tables.	- Grant *Build* permission for the semantic model. - If Direct Lake uses SSO, grant users at least *Read* permission for the target Fabric item and	Users can only build reports on the tables and columns they have access to. This condition might be a subset

Scenario	Direct Lake on SQL endpoints	Direct Lake on OneLake	Comments
		add them to a OneLake security role or grant them *ReadAll* permission.	of the full set of tables and columns in the model. For more information, see Strategy for content creators.
Users can query the semantic model but are denied querying the lakehouse or SQL analytics endpoint	- Bind the Direct Lake model to a cloud connection with a fixed identity and leave SSO disabled. - Grant the fixed identity at least *Read* permission for the target Fabric item and SELECT permissions for the tables. - Don't grant users any permission for the target Fabric item.	- Bind the Direct Lake model to a cloud connection with a fixed identity and leave SSO disabled. - Grant the fixed identity at least *Read* permission for the target Fabric item and add it to a OneLake security role or grant it *ReadAll* permission. - Don't grant users any permission for the target Fabric item.	Only suitable when the cloud connection uses a fixed identity.
Users can query the semantic model and the SQL analytics endpoint but are denied querying the lakehouse	- Grant *Read* and *ReadData* permissions for the target Fabric item.	Not applicable.	**Important**: Queries sent to the SQL analytics endpoint bypass data access permissions that the semantic model enforces.
Manage the semantic model, including refresh settings	- Requires semantic model ownership.	- Requires semantic model ownership.	For more information, see Semantic model ownership.

> **Important**
>
> Always test permissions before releasing your semantic model and reports to production.

For more information, see Semantic model permissions.

Direct Lake owners

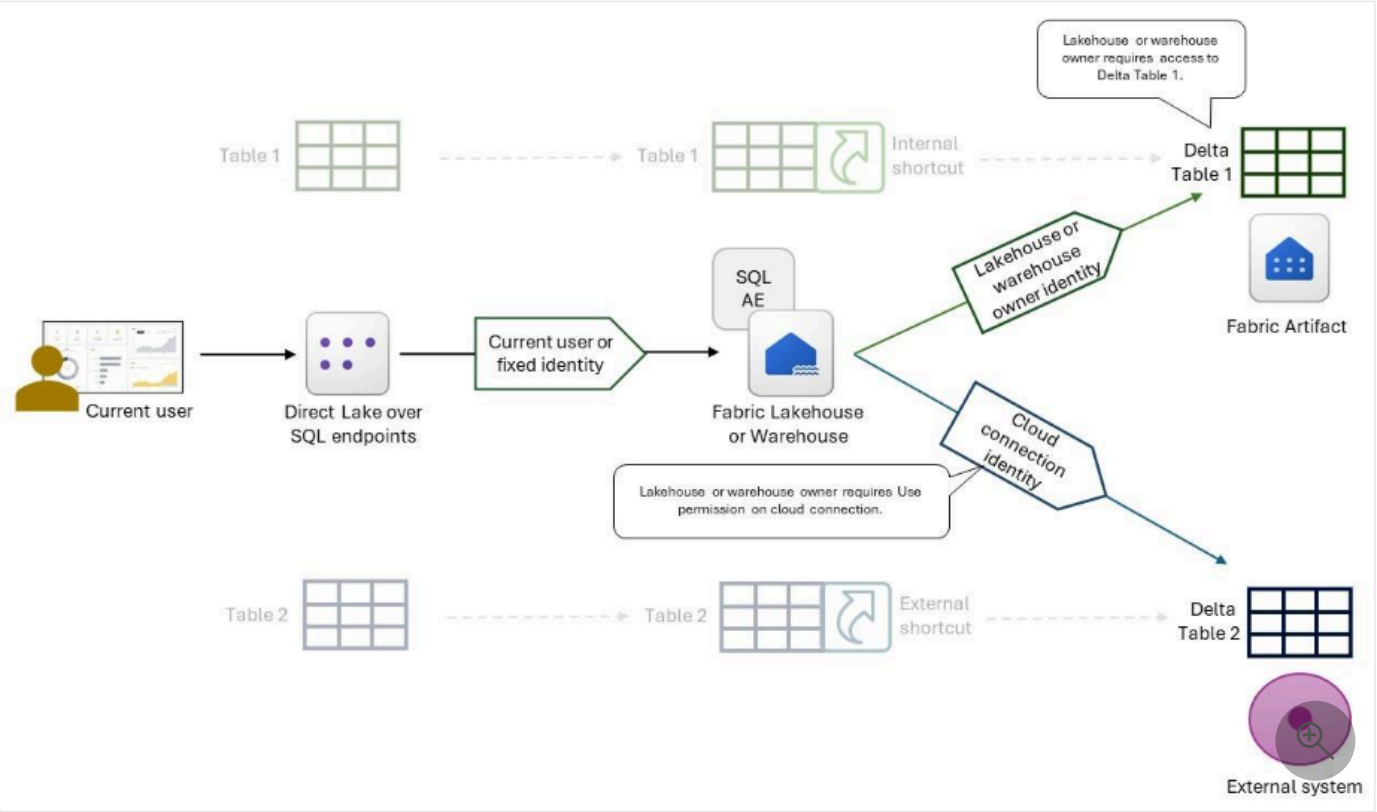
In addition to the effective identity (current user or fixed identity), Direct Lake requires the semantic model owner to have *read* access to the source tables so that Direct Lake can frame the semantic model as part of data refresh. No matter who refreshes a Direct Lake model, Direct Lake checks the owner's permission to ensure the model is allowed to access the data. The owner's data access permission requirements are the same as for users querying the model.

If the semantic model owner doesn't have the required data access permissions, Direct Lake raises the following error during framing: `We cannot refresh this semantic model because one or multiple source tables either do not exist or access was denied. Please contact a data source admin to verify that the tables exist and ensure that the owner of this semantic model does have read access to these tables. Some restricted tables including fully restricted and partially restricted (indicating column constraints): '\<list of tables\>'.`

Shortcuts to source tables

Shortcuts are OneLake objects that you add to a Fabric lakehouse or other Fabric item to point to internal or external storage locations. In a Direct Lake model, Delta tables added through shortcuts appear as native in the connected Fabric item because shortcuts are transparent when you access data through the OneLake API.

When you access shortcuts through Direct Lake over SQL endpoints, Direct Lake first validates that the effective identity (current user or fixed identity) can access the table in the semantic model's data source. For internal shortcuts, after that check passes, Direct Lake uses the data source owner's identity to read the Delta table through the shortcut at the table's Fabric item. The data source owner must have access permission in the target OneLake location. For external shortcuts, the data source owner also needs Use permission on the cloud connection to the external system that hosts the Delta table. For more information, see OneLake shortcuts.



Direct Lake over OneLake has different permission requirements because the SQL Analytics Endpoint isn't involved. When a user accesses data through an internal shortcut to another OneLake location, the effective identity (current user or fixed identity) must have permission in the target location. The effective identity must be a Contributor (or higher), have Read and ReadAll permissions, or be in a OneLake security role that grants *read* access.

Object-level security (OLS) and row-level security (RLS)

Both OneLake security and Direct Lake models support OLS and RLS. OLS enables item owners and admins to secure specific tables or columns. RLS can be used to restrict data access at the row level based on filters. You can define OLS and RLS in OneLake security, in a Direct Lake model, or in both locations.

> **ⓘ Important**
>
> Direct Lake doesn't support SQL analytics endpoint OLS/RLS in memory. Direct Lake over SQL endpoints handles these restrictions differently depending on the type. If a query touches a table or column that's restricted by SQL analytics endpoint OLS or column-level security (CLS), the query returns an error. If a query references a table that enforces RLS or a view at the SQL analytics endpoint, the query falls back to DirectQuery mode. If DirectQuery fallback is disabled, queries that depend on RLS or views over SQL endpoints fail. Direct Lake

over OneLake avoids these limitations. For details, see [How queries are evaluated in Direct Lake on SQL](#).

Direct Lake on OneLake OLS/RLS with OneLake security OLS/RLS

Direct Lake on OneLake evaluates access to OLS/RLS secured objects by resolving the effective identity's OneLake security roles and applying the defined OLS/RLS rules. The OneLake security roles are handled the same as Direct Lake roles. If the effective identity belongs to multiple roles in OneLake security and Direct Lake, Direct Lake first unions the OneLake security roles, then intersects the result with the Direct Lake roles.

This table lists common troubleshooting situations caused by conflicting OneLake security and Direct Lake rules.

Expand table

Scenario	Comments
No rows returned due to RLS filtering	If the effective identity lacks row-level access permissions, queries can return empty results. This behavior is expected when RLS filters exclude all rows for the current user.
Can't find table Column can't be found Failed to resolve name Not a valid table, variable, or function name	These errors usually occur when object permissions are missing after applying OneLake security roles.

OLS/RLS scope differences

Enforcing OLS and RLS in OneLake security applies the rules across all compute engines and ensures unified access control for users. This means that, regardless of the compute engine—lakehouse, warehouse, semantic model, or other item—OneLake security rules control the user's data access. In contrast, OLS/RLS defined within a Direct Lake semantic model only apply within the scope of that model. Other compute engines don't apply these Direct Lake security rules, which can produce different results when users access the data through other paths.

Important

When you use both OneLake security OLS/RLS and Direct Lake OLS/RLS, users who have OneLake access can still retrieve and work with the data—even if Direct Lake model rules further restrict data—because model-level rules don't extend beyond the model. Use OneLake security for comprehensive access control across all compute engines.

OneLake OLS and semantic model metadata

Semantic model metadata includes definitions of tables, columns, relationships, and other schema elements. Users with *build* or higher permissions can view the model metadata via XML for Analysis (XMLA) and REST APIs. For more information, see Semantic model permissions.

To protect sensitive table and column names in OneLake with OneLake OLS, remember that OneLake security applies only to members of the workspace Viewer role. OneLake OLS doesn't prevent members of the Contributor (or higher) workspace role from discovering secured tables or columns because they already have Write permission to all workspace items. Members of the Viewer role with *build* or higher permissions on a Direct Lake model can discover sensitive schema information through the semantic model metadata. These higher privileged viewers still don't have data access, but they can see that the secured tables and columns exist.

A Direct Lake model might exist in the same workspace as the source item or in a separate workspace. Grant a viewer in the same workspace *build* (or higher) access to a Direct Lake model through item permissions. In a separate workspace, a user might be a Contributor (or higher) or have *build* (or higher) item permissions to access the model metadata.

OneLake OLS and Git integration

Git integration enables developers to integrate their application lifecycle management (ALM) processes into the Fabric platform. The Git repository preserves the workspace structure, including all supported items. Developers have full visibility to the metadata of all their items in the Git repository. Direct Lake model metadata lets them see that secured tables or columns exist even if they don't have access to the target data source in another workspace. For more information, see What is Microsoft Fabric Git integration?

How queries are evaluated in Direct Lake on SQL

The reason to develop Direct Lake semantic models is to achieve high performance queries over large volumes of data in OneLake. Therefore, you should strive to design a solution that maximizes the chances of in-memory querying.

The following steps approximate how Direct Lake on SQL queries are evaluated (and whether they fail). The benefits of Direct Lake storage mode are only possible when the fifth step is achieved.

1. If the query contains any table or column that's restricted by semantic model OLS, an error result is returned (report visuals fail to render).
2. If the query contains any column that's restricted by SQL analytics endpoint CLS (or the table is denied), an error result is returned (report visuals fail to render).
   a. If the cloud connection uses SSO (default), CLS is determined by the access level of the report consumer.
   b. If the cloud connection uses a fixed identity, CLS is determined by the access level of the fixed identity.
3. If the semantic model uses Direct Lake on SQL endpoints and the query contains any table in the SQL analytics endpoint that enforces RLS or a view is used, the query falls back to DirectQuery mode.
   a. If the cloud connection uses SSO (default), RLS is determined by the access level of the report consumer.
   b. If the cloud connection uses a fixed identity, RLS is determined by the access level of the fixed identity.
4. If the query exceeds the guardrails of the capacity, it falls back to DirectQuery mode.
5. Otherwise, the query is satisfied from the in-memory cache. Column data is loaded into memory when required.

> **Important**
>
> Direct Lake on OneLake doesn't support fallback to DirectQuery mode. If any table in the SQL analytics endpoint enforces RLS or the query exceeds the guardrails of the capacity, an error result is returned (report visuals fail to render).

Data-access rule options

You can set up data-access rules in:

- The semantic model.
- The SQL analytics endpoint (Direct Lake on SQL endpoints only).
- OneLake security.

Rules in the semantic model

If you must enforce data-access rules, do so in OneLake security so that the rules apply across all compute engines and ensure unified access control for users. Use semantic model RLS or OLS when report consumers aren't granted permission to query the lakehouse or warehouse and the cloud connection uses a fixed identity instead of SSO. SSO implies that end users can access the data source directly and might therefore bypass security rules in the semantic model.

> **Important**
>
> Semantic model item permissions can be set explicitly via Power BI apps, or acquired implicitly via workspace roles.
>
> Notably, semantic model data-access rules aren't enforced for users who have *Write* permission on the semantic model. Conversely, data-access rules do apply to users who are assigned to the *Viewer* workspace role. However, users assigned to the *Admin*, *Member*, or *Contributor* workspace role implicitly have *Write* permission on the semantic model and so data-access rules aren't enforced. For more information, see Roles in workspaces.

Rules at multiple layers

You can enforce data-access rules at all layers. However, this approach involves extra complexity and management overhead. In this case, use a fixed identity for the cloud connection instead of SSO.

Compare data-access rule options

The following table compares data-access setup options for Direct Lake on SQL endpoints and Direct Lake on OneLake.

Expand table

Apply data-access rules to	Direct Lake on SQL	Direct Lake on OneLake	Comment
Semantic model only	Supported	Supported	Use this option when users aren't granted item permissions to query the lakehouse or warehouse. Set up the cloud connection to use a fixed identity. Achieve high query performance from the in-memory cache.

Apply data-access rules to	Direct Lake on SQL	Direct Lake on OneLake	Comment
SQL analytics endpoint only	Supported (falls back to DirectQuery)	Not applicable	Depends on the Fabric data item (like Lakehouse or Warehouse) using delegated identity mode. Use this option when users need to access data from either the warehouse or the semantic model, and with consistent data-access rules. Ensure SSO is enabled for the cloud connection. Query performance might be slow due to DirectQuery fallback.
OneLake security only	Not applicable	Supported	Use this option for unified access control across all Fabric compute engines. OneLake security enforces OLS and RLS consistently for all users accessing the data through any path. Achieve high query performance from the in-memory cache.
Multiple layers (semantic model and SQL endpoint)	Supported	Not applicable	This option involves extra management overhead. Set up the cloud connection to use a fixed identity.
Multiple layers (semantic model and OneLake security)	Not applicable	Supported	OneLake security rules are applied first, then semantic model rules. Consider consolidating rules at one layer to reduce complexity.

Considerations and limitations

Consider these Direct Lake security limitations.

> **Note**
>
> The capabilities and features of Direct Lake semantic models and OneLake security evolve rapidly. Check back periodically for updates.

- Assign workspace viewers OneLake security roles that grant *read* access to the source Fabric items. If a source item has shortcuts to another Fabric item, the user also needs *read* access to each shortcut's target Fabric item.
- Use a fixed identity to isolate users from a source Fabric item. Bind the Direct Lake model to a cloud connection. Keep SSO disabled on the cloud connection to use the fixed identity for refreshes and queries.

- Direct Lake semantic models that rely on Fabric OneLake security on the source item don't support backup operations.
- Bidirectional relationships aren't supported in a Direct Lake model if the source Fabric item relies on OneLake security RLS.
- OneLake security doesn't support dynamic definitions or complex role configurations, such as combining multiple OLS and RLS roles across related tables.
- Consolidate OneLake security RLS and OLS permissions into one role per user instead of assigning multiple roles.
- If the OneLake security configuration changes, such as due to shortcut changes in the target item, refresh Direct Lake on OneLake models that access that item. You must refresh the models manually or by using refresh APIs.
- If a Lakehouse has OneLake security:
  - The SQL analytics endpoint is, by default, fixed identity to the owner of the Lakehouse, so the SQL analytics endpoint OneLake security is the same as the owner (no limitations). Direct Lake on SQL stays using Direct Lake, unless extra SQL granular access roles are added.
  - The SQL analytics endpoint can be changed to SSO. When this happens, OneLake security roles are added as SQL granular access control rules and the user is blocked from editing them directly on the SQL analytics endpoint. At this point, Direct Lake on SQL falls back to DirectQuery 100% of the time.

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 08/31/2026

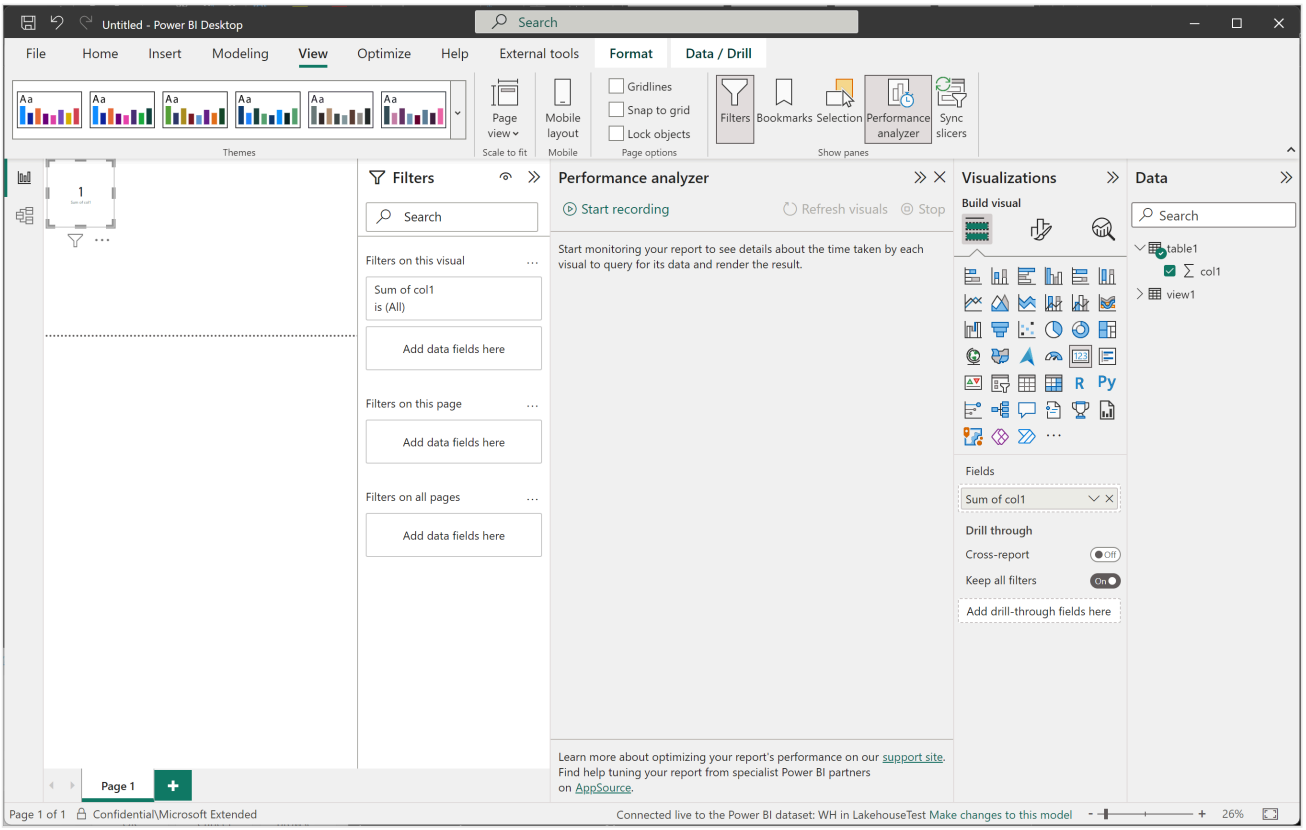
Analyze query processing for Direct Lake semantic models

Power BI semantic models in *Direct Lake* mode read Delta tables directly from OneLake — unless they have to fall back to *DirectQuery* mode. Typical fallback reasons include memory pressures that can prevent loading of columns required to process a DAX query, and certain features at the data source might not support Direct Lake mode, like SQL views in a Warehouse and Lakehouse. In general, Direct Lake mode provides the best DAX query performance unless a fallback to DirectQuery mode is necessary. Because fallback to DirectQuery mode can impact DAX query performance, it's important to analyze query processing for a Direct Lake semantic model to identify if and how often fallbacks occur.

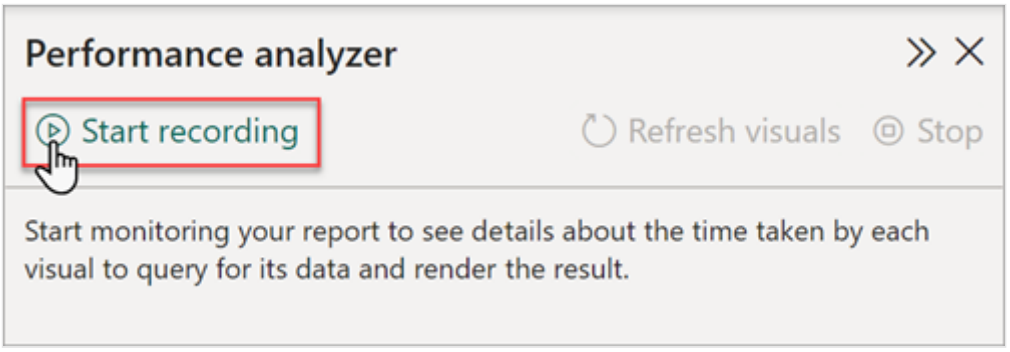
Analyze by using Performance analyzer

Performance analyzer can provide a quick and easy look into how a visual queries a data source, and how much time it takes to render a result.

1. Start Power BI Desktop. On the startup screen, select **New** > **Report**.
2. Select **Get Data** from the ribbon, then select **Power BI semantic models**.
3. In the **OneLake catalog** page, select the Direct Lake semantic model you want to connect to, and then select **Connect**.
4. Place a card visual on the report canvas, select a data column to create a basic report, and then on the **View** menu, select **Performance analyzer**.

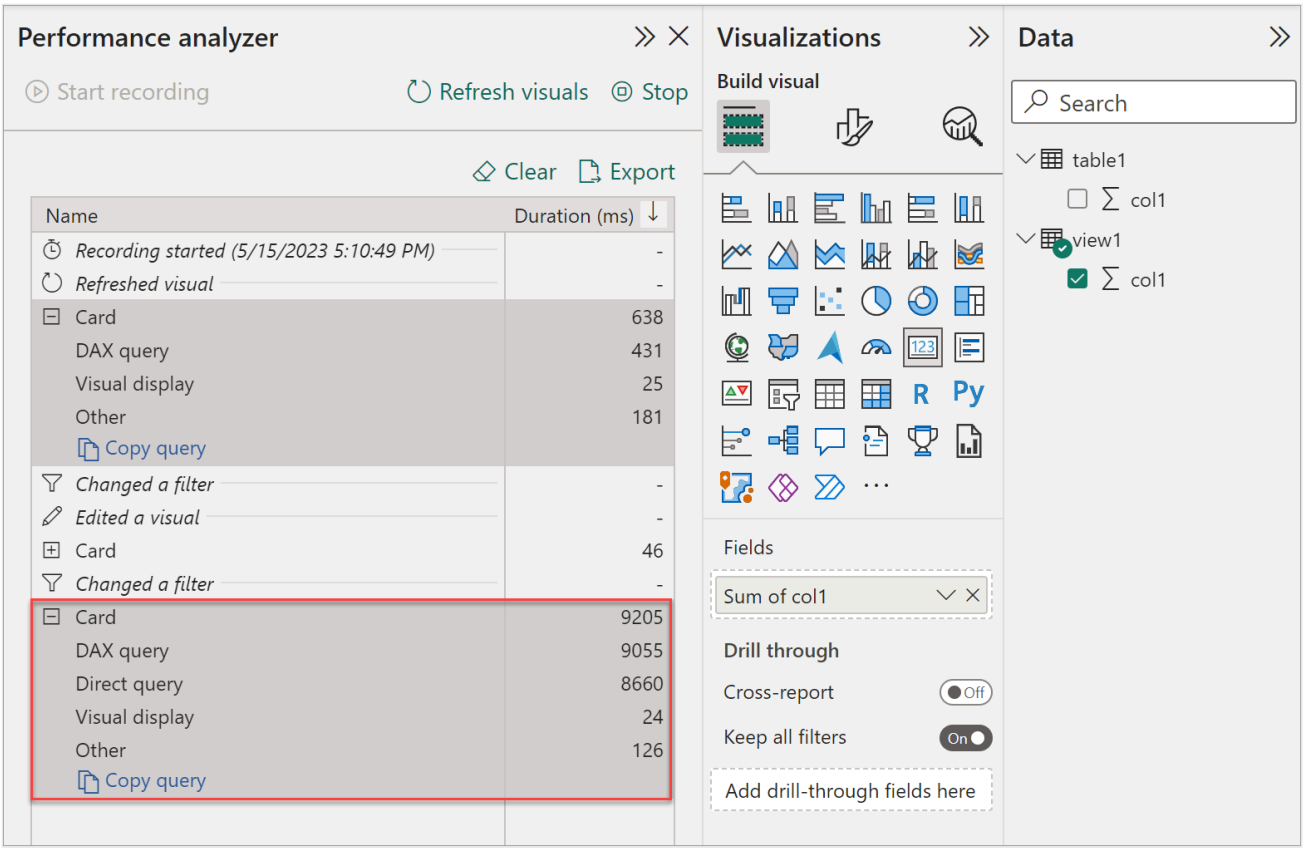


5. In the **Performance analyzer** pane, select **Start recording**.



6. In the **Performance analyzer** pane, select **Refresh visuals**, and then expand the Card visual. The card visual doesn't cause any DirectQuery processing, which indicates the semantic model was able to process the visual's DAX queries in Direct Lake mode.

   If the semantic model falls back to DirectQuery mode to process the visual's DAX query, you see a **Direct query** performance metric, as shown in the following image:



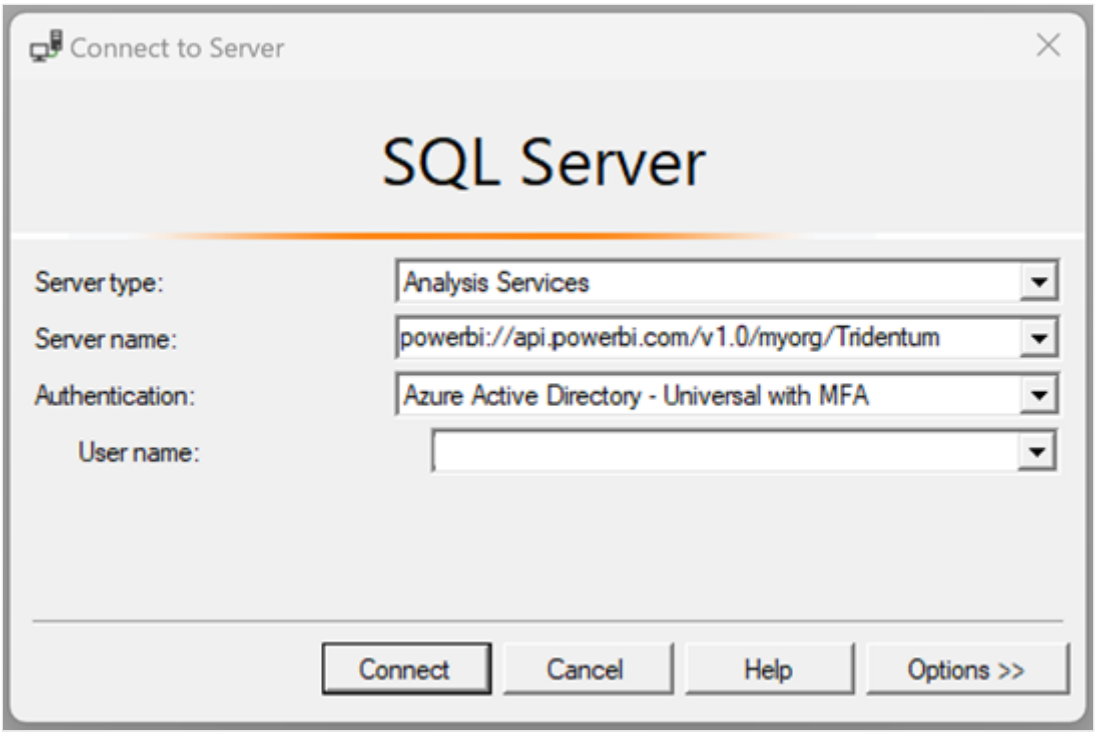
Analyze by using SQL Server Profiler

SQL Server Profiler can provide more details about query performance by tracing query events. It's installed with SQL Server Management Studio (SSMS). Before starting, make sure you have the latest version of SSMS installed.

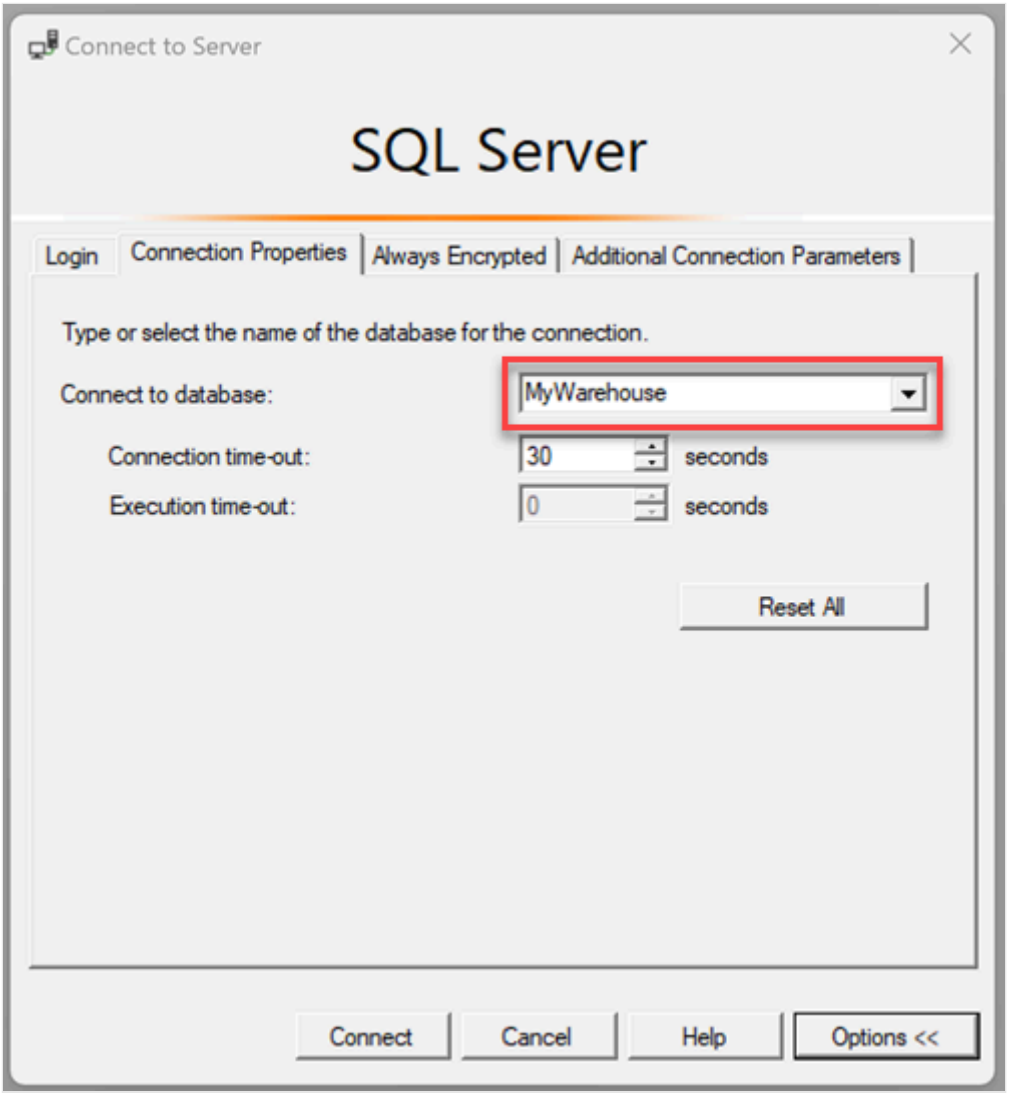
1. Start SQL Server Profiler from the Windows menu.

2. In SQL Server Profiler, select **File** > **New Trace**.

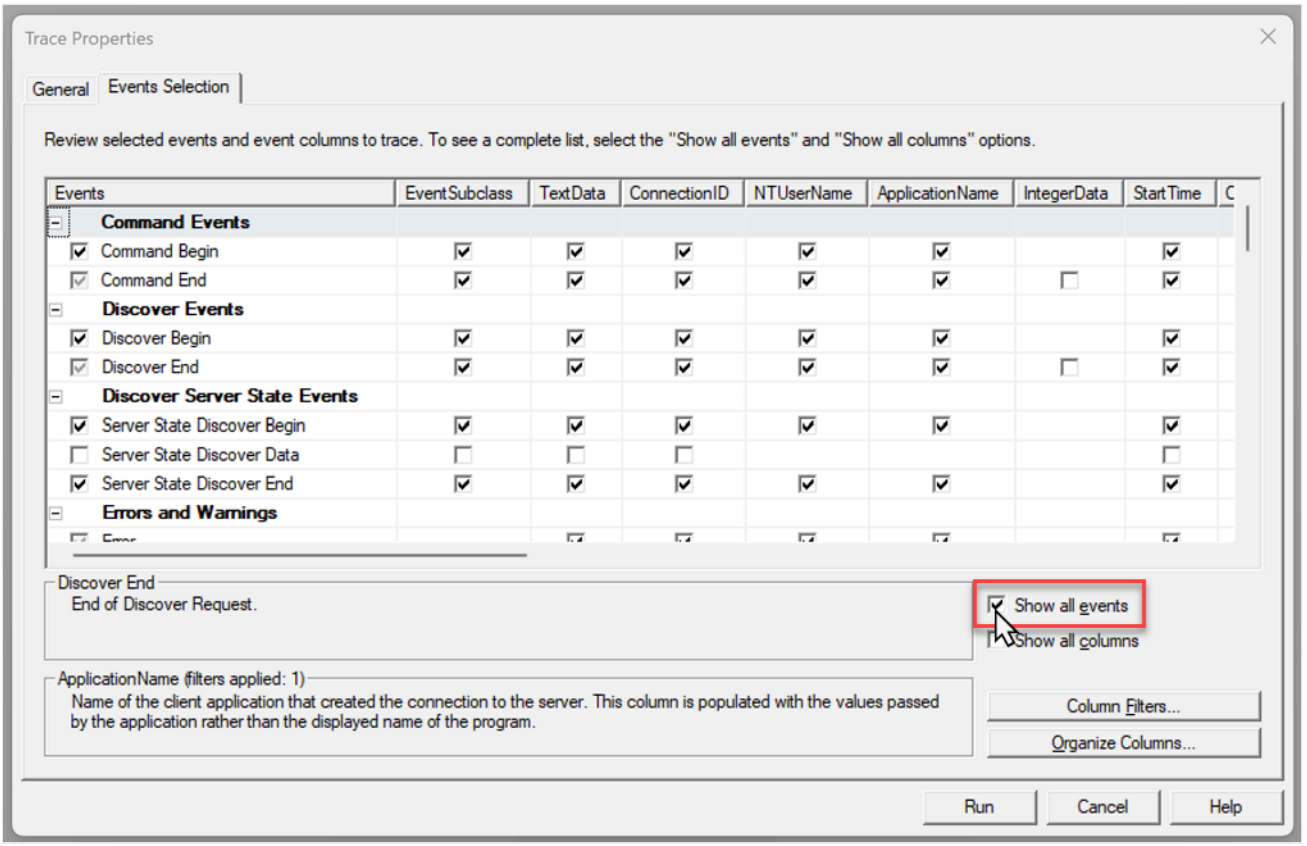
3. In **Connect to Server** > **Server type**, select **Analysis Services**, then in **Server name**, enter the URL to your workspace, then select an authentication method, and then enter a username to sign in to the workspace.



4. Select **Options**. In **Connect to database**, enter the name of your semantic model and then select **Connect**. Sign in to Microsoft Entra ID.



5. In **Trace Properties** > **Events Selection**, select the **Show all events** checkbox.

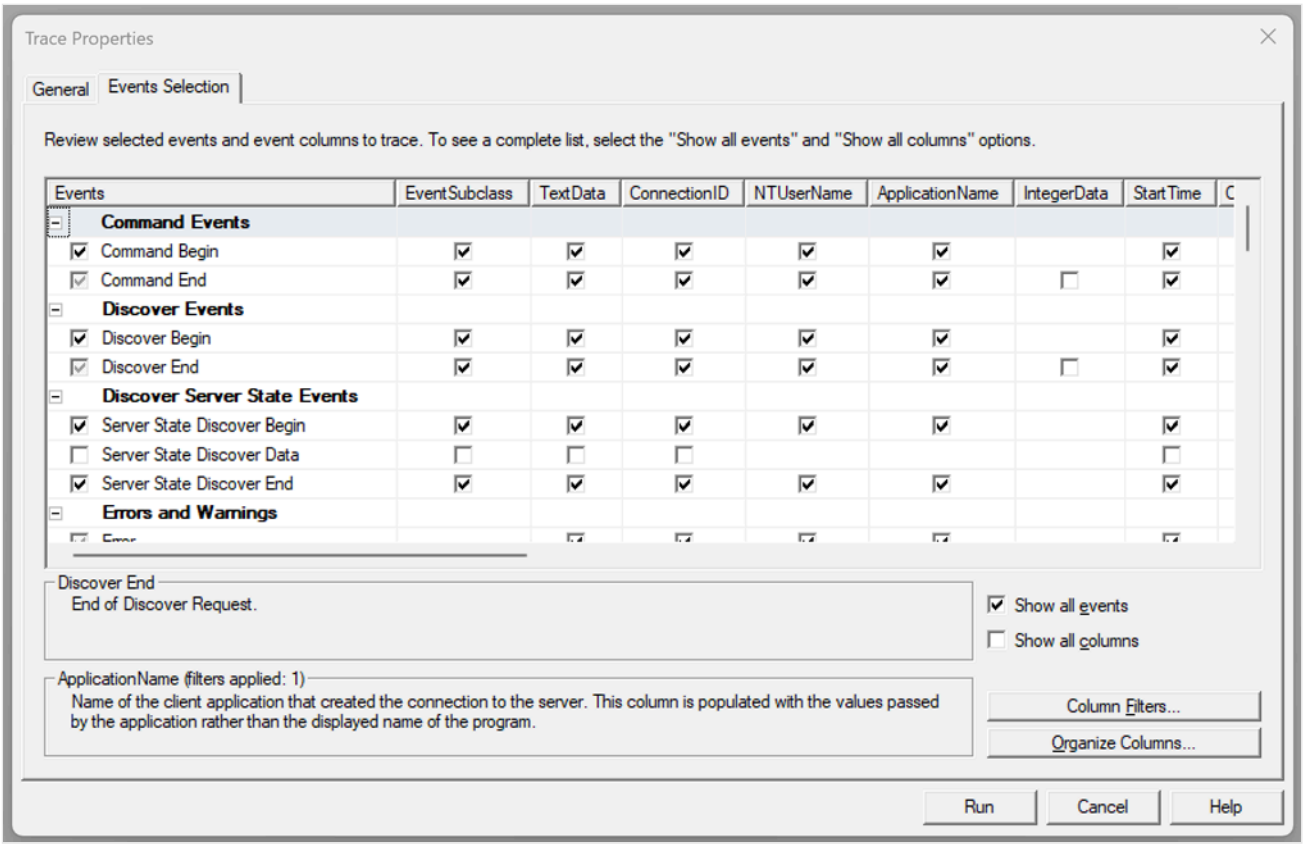


6. Scroll to **Query Processing**, and then select checkboxes for the following events:

Expand table

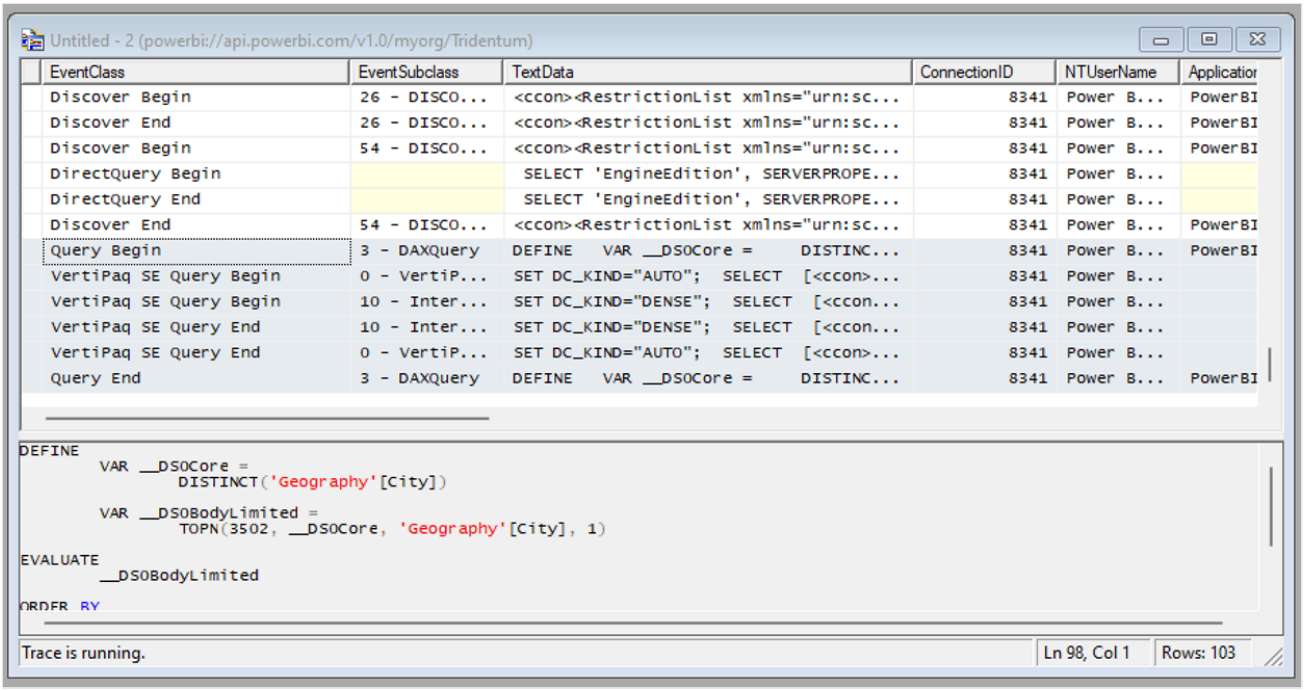
Event	Description
DirectQuery_Begin **DirectQuery_End**	If DirectQuery Begin/End events appear in the trace, the semantic model might have fallen back to DirectQuery mode. However, note that the presence of EngineEdition queries and possibly queries to check Object-Level Security (OLS) do not represent a fallback because the engine always uses DirectQuery mode for these non-query processing related checks.
VertiPaq_SE_Query_Begin **VertiPaq_SE_Query_Cache_Match** **VertiPaq_SE_Query_Cache_Miss** **VertiPaq_SE_Query_End**	VertiPaq storage engine (SE) events in Direct Lake mode are the same as for import mode.

It should look like this:



7. Select **Run**. In Power BI Desktop, create a new report or interact with an existing report to generate query events. Review the SQL Server Profiler trace report for query processing events.

   The following image shows an example of query processing events for a DAX query. In this trace, the VertiPaq storage engine (SE) events indicate that the query was processed in Direct Lake mode.



Related content

- Delta Lake in Microsoft Fabric overview
- Direct Lake overview

Last updated on 06/05/2026

Direct Lake in web modeling

Open data model, or web modeling, works with semantic models with Direct Lake tables. Direct Lake tables can be **Direct Lake on SQL** or **Direct Lake on OneLake**, which have different considerations when creating and editing in the web. For the source types available with each option, see Supported Fabric data sources.

Expand table

Scenario	Direct Lake on OneLake	Direct Lake on SQL
Creating in the web	• Select **Create** in the left navigation, then **OneLake catalog**. • Select **New semantic model** from a supported Fabric data source. • Select **OneLake catalog** from web modeling. • Select **New item** from a workspace, select **Semantic model**, then select **OneLake catalog**.	Select **New semantic model** from the SQL analytics endpoint of a supported Fabric data source.
Editing in the web	Select **Open data model** from the semantic model details page or context menu.	Select **Open data model** from the semantic model details page or context menu.

Edit in Desktop is available when web modeling to continue live editing any Direct Lake semantic model in Power BI Desktop.

Create a semantic model with Direct Lake tables

To create a semantic model with **Direct Lake on OneLake tables**, take the following steps.

1. Select **Create** from the left navigation bar, select **OneLake catalog**, and choose a supported Fabric data source. Alternatively, open a supported source item and select **New semantic model**.
2. Give your semantic model a name, pick a Fabric workspace for it, and select the tables to include. Then press **OK**.

The semantic model is created and now you're live editing the modeling in the browser.

To create a semantic model with **Direct Lake on SQL tables**, take the following steps.

1. Open the SQL analytics endpoint of a supported Fabric data source, go to **Reporting**, and then select **New semantic model**.
2. Give your semantic model a name, pick a Fabric workspace for it, and select the tables to include. Then press **OK**.

The semantic model is created and now you're live editing the modeling in the browser.

> **Note**
>
> Check your pop-up blocker if web modeling doesn't appear after clicking OK.

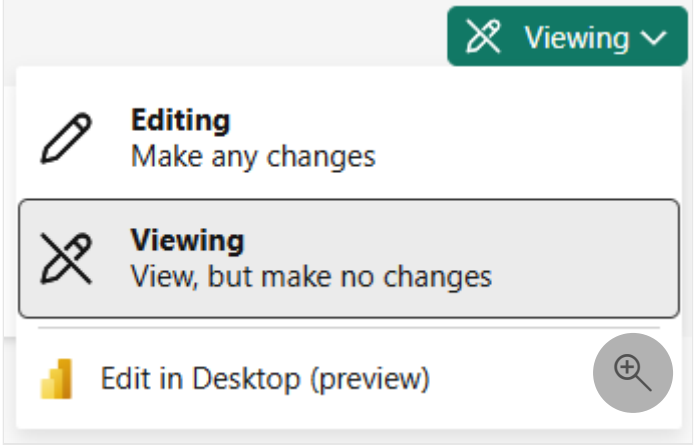
The differences between Direct Lake on OneLake and Direct Lake on SQL are explained in the Overview.

Edit a semantic model in Direct Lake mode

To edit a semantic model with Direct Lake tables later, take the following steps.

1. Navigate to the semantic model in the Fabric Portal. **Home**, **OneLake catalog**, and **search** at the top of the page are available to help you find it.
2. Select the semantic model to open the details page, or use the context-menu, then select **Open data model**.

Now you're live editing the semantic model in the web. The model opens by default in **viewing mode** to avoid accidental edits. Change to **editing mode** in the top right-hand corner of the window. **Edit in Desktop** is also available to change to live edit in Power BI Desktop.



Composite semantic models with Direct Lake and import storage mode tables

A composite semantic model has tables in different storage modes. Direct Lake on OneLake table storage mode already could mix tables from other Fabric data sources, such as lakehouses, warehouses, SQL databases in Fabric, and mirrored databases. And with this update, now that flexibility is extended much further with the ability to add in import tables from any data source, from 100s of connectors in Power Query online.

Unlike traditional DirectQuery and import composite models that only support limited relationships, Direct Lake and import composite models support regular relationships, which keeps report performance as expected.

Import tables can be added to an existing semantic model with Direct Lake on OneLake tables. Edit the semantic model in Power BI web modeling and choose **Get data** or **Transform data** from the ribbon.

> ⓘ **Note**
>
> If you haven't set up a credential for this data source before you may be prompted to do that in the Power Query online experience. After that, if the load data fails on saving the transformations, go to the schedule refresh page of the semantic model and set the credentials there as well before returning to web modeling and refreshing.

Direct Lake on OneLake tables can be added to an existing semantic model with import storage mode tables too. Edit the semantic model in Power BI web modeling and choose **OneLake catalog** from the ribbon.

Use the **Refresh** button in the web modeling ribbon to update import table data, reframe Direct Lake tables, and perform a schema sync on all tables to get the latest column information from the respective data sources.

Convert a Direct Lake table to Import storage mode

You can convert an individual table in a **Direct Lake on OneLake** semantic model from Direct Lake to Import storage mode. Converting a table creates a composite model that combines Direct Lake and Import tables. You can perform this conversion only in Power BI web modeling. Power BI Desktop doesn't offer per-table conversion from Direct Lake to Import storage mode.

> ⓘ **Important**

Converting a table to Import storage mode is an irreversible operation. After you convert the table, you can't set the mode back to Direct Lake. The table's Power Query M connector is converted to a SQL connector, and the table uses the Fabric SQL analytics endpoint for the data source.

To convert one or multiple tables:

1. Open the semantic model in Power BI web modeling and switch to **editing** mode.
2. Select one or multiple tables you want to convert.
3. Open the **Properties** pane and change the **Storage mode** value from **Direct Lake** to **Import**, and then select **Continue** in the confirmation dialog.
4. In the Power Query online experience, select **Configure connection** and provide the connection details for the data source.
5. Select **Save** to load the data.
6. Return to web modeling and select **Refresh**.

The tables are now in Import storage mode alongside your Direct Lake tables.

Related content

- Edit data models in the Power BI service
- Composite semantic models in Power BI

Last updated on 09/18/2026

Direct Lake in Power BI Desktop

Power BI Desktop can create and edit Power BI semantic models with Direct Lake tables. Semantic models with Direct Lake tables are created in, and edited from, the Fabric workspace, not on your local machine, so when using Power BI Desktop, you **live edit** the semantic model where it is. There's no publish action as changes made in Power BI Desktop happen to the semantic model in the Fabric workspace. This experience is the same as when you're editing the semantic model in the web, or in the Fabric workspace, by choosing the **Open data model** action.

Version history is available and automatically creates a version each time you start a live editing session, so you can undo an accidental change. **Git integration** is also available for semantic models, giving you full control over changes. And **deployment pipelines** can also be used to only live edit a semantic model in a development workspace before pushing to a production workspace.

In a semantic model with import tables, the data is downloaded and locally available on your computer. In a semantic model with Direct Lake tables, the data remains in the OneLake. When visuals use data, the semantic model provides the data from where it is stored. Learn more about Direct Lake query performance.

Metadata for the semantic model that is the information about the tables columns, measures, relationships, and all other semantic modeling features, can be downloaded, with the data, as a **PBIX file** for semantic models not using Direct Lake tables. Metadata for the semantic model when you include Direct Lake tables can also be downloaded, without the data, using the **Power BI Project (PBIP)** format. Learn more about Direct Lake with PBIP.

Power BI reports can be created from all semantic models from Power BI Desktop with a live connection by choosing a **Power BI semantic model** from the **OneLake catalog** and selecting **Connect**. Reports can also be created in the Fabric workspace from many places, including the right-click context menu and choosing **create a report**. Learn more about building reports.

This article discusses more details about live editing in Power BI Desktop, and how to create and add Direct Lake tables to a semantic model in Power BI Desktop.

Create a semantic model with Direct Lake tables

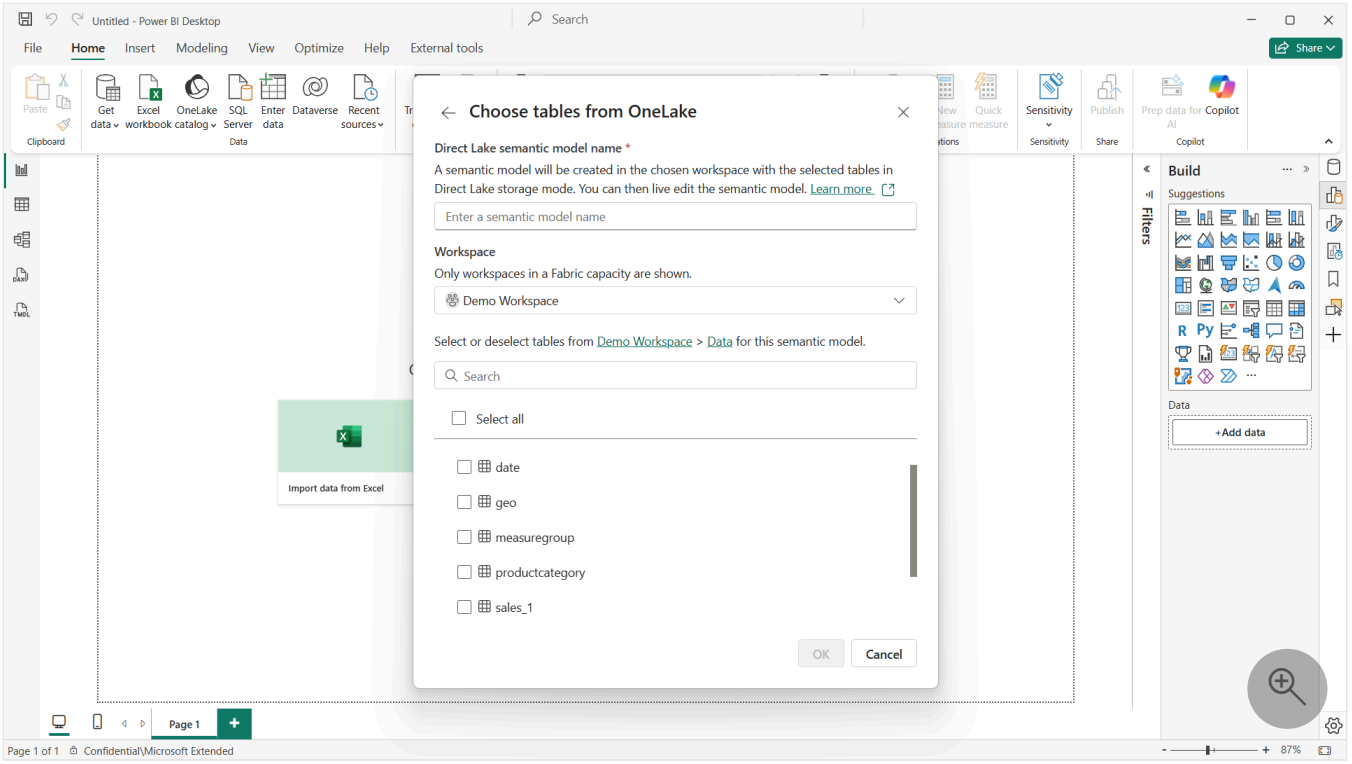
To create a semantic model with Direct Lake tables, take the following steps.

1. Open **Power BI Desktop** and select **OneLake catalog**.
2. Select a Fabric item, such as a **Lakehouse** or **Warehouse**, and press **Connect**.

3. Give your semantic model a name, pick a Fabric workspace for it, and select the tables to include. Then press **OK**.

The semantic model is created in the Fabric workspace and now you're live editing the semantic model in Power BI Desktop.

Semantic models with Direct Lake tables created in Power BI Desktop use **Direct Lake on OneLake** storage mode. The differences between Direct Lake on OneLake and Direct Lake on SQL are explained in the Overview.

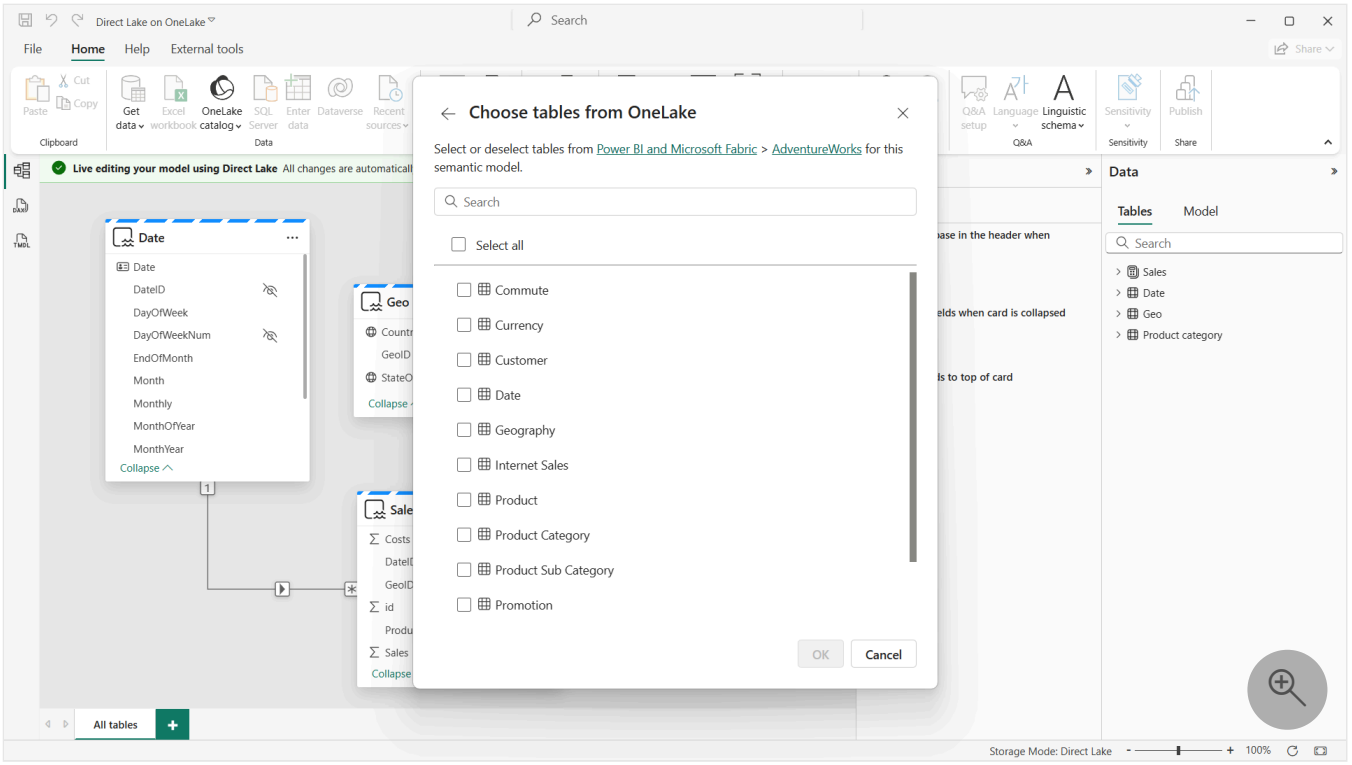


Add Direct Lake tables from other Fabric items

To add Direct Lake tables from other Fabric items, take the following steps.

1. While live editing a Direct Lake on OneLake semantic model in Power BI Desktop, open the **OneLake catalog** and select another Fabric item, such as a **Lakehouse** or **Warehouse**.
2. In the dialog, select the tables you want to include then press **OK**.

The tables are added to your semantic model and you can continue live editing.

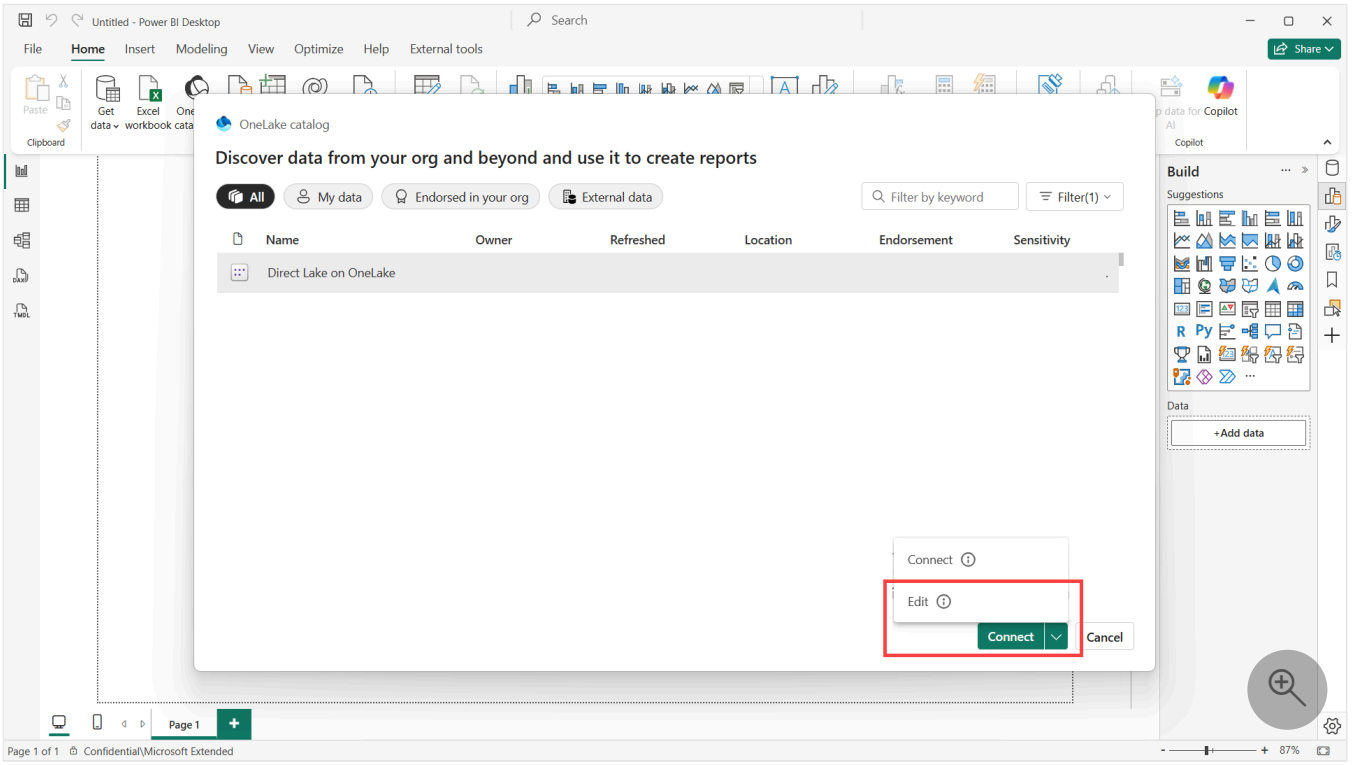


Live edit a semantic model with Direct Lake or import tables

To edit a semantic model with Direct Lake tables later, take the following steps.

1. In a **new instance of Power BI Desktop**, open the **OneLake catalog** and select the **Power BI semantic model**.
2. Select the **Connect** dropdown list and choose **Edit**.

Now you're live editing the semantic model.



> **Note**
>
> Semantic models with Direct Lake and import table storage modes are supported. **Edit tables**, **OneLake catalog**, and **Transform data** are only available in web modeling. Use Direct Lake in Power BI web modeling.

Alternatively, if you have exported the semantic model to a Power BI Project (PBIP), take the following steps.

1. Double-click the PBIP file on in the Power BI Project (PBIP) folder.
2. Or, in Power BI Desktop choose **File** then **Open** and navigate to the **PBIP file in the Power BI Project (PBIP)** folder.

Live editing in Power BI Desktop differences

Live editing in Power BI Desktop is different than editing a local model with import and DirectQuery tables, and different than editing a report with a live connection.

Report view

The report view is removed when live editing, unless you're live editing with Power BI Project (PBIP).

To create a report, follow these steps in Power BI Desktop.

1. Select **File** then **Blank report** to create a new report.
2. Open the **OneLake catalog** and choose the **Power BI semantic model** you're live editing (it should show at the top of the list).
3. Select **Connect**.
4. Now you can create the report. Save the file and publish to the Fabric workspace when ready.

Learn more about building reports.

Table view

The table view is also removed when live editing, unless you have a calculation group or calculated table in the semantic model. These derived tables use import storage mode. Calculated tables without direct references to Direct Lake table columns are allowed. A common example is using INFO.VIEW DAX functions to self-document the semantic model.

> **Note**
>
> Import tables from any data source can be added to the semantic model with Direct Lake on OneLake tables using web modeling. Use Direct Lake in Power BI web modeling.

Saving

As you make changes to your semantic model, your changes are automatically saved and the **Save** button is disabled when in Live edit mode. Changes made in Power BI Desktop automatically happen to the semantic model in the Fabric workspace.

Version history creates a version at the beginning of each live editing session if you need to revert a change. There's no undo action available as you make changes. Git integration or using deployment pipelines to first live edit in a development workspace then pushing to a production environment are also available to live edit without affecting downstream users.

There's no local file created but if you would like a local copy of the metadata, you can export to a Power BI Project (PBIP) and continue live editing with a **Save** button for the local metadata. You can utilize local Git techniques to undo changes. To export to Power BI Project (PBIP), go to **File** then **Export**, and choose **Power BI Project (PBIP)**.

If two or more users are live editing the same semantic model and a conflict occurs, Power BI Desktop alerts one of the users, and syncs the model to the latest version. Any changes you were trying to make will need to be performed again after the model sync. This behavior is the same behavior as editing data models in the Power BI service, also called web modeling.

Refresh

Selecting the Refresh button when live editing a semantic model with Direct Lake tables performs a schema refresh and reframe the Direct Lake tables.

The schema refresh checks the tables definitions in the model and compares it to the same named table in the data source for any changes to columns. Changes detected from the data source, in this case a Fabric artifact, are made to the semantic model. For example, a column was added to a table. Changing the table or column name in the semantic model in Power BI Desktop persist after a refresh.

Changing a table or column name at the data source removes the table or column on the next schema refresh. You can use TMDL view to see the SourceLineageTag property and update it to the new name to avoid the semantic model removing it on schema refresh.

Another way to perform a schema refresh is to return to Edit tables and select **OK**. Go to **Transform data** dropdown list then **Data source settings** and select **Edit tables**.

Scheduled refresh in the Fabric workspace only reframe the Direct Lake tables without a schema refresh. Learn more about refresh in Power BI.

Power BI Project (PBIP)

When working on a Power BI Project (PBIP) with a semantic model with Direct Lake tables, Power BI Desktop needs to connect to a semantic model in a Fabric workspace, also termed as a remote semantic model. Remote modeling is live editing, as all changes you make are immediately applied to the semantic model in the workspace. In addition, you can save your semantic model and report definitions, or metadata, to your local PBIP files. The PBIP files can later be deployed to a Fabric workspace using a deployment mechanism such as Fabric Git Integration. Learn more about remote modeling with Power BI Project (PBIP)

Name in header links

Selecting the name of the semantic model in the top left corner of Power BI Desktop expands to show the location of the semantic model in the Fabric workspace. Selecting the workspace name or semantic model name navigates you to them in the web. Version history is also available.

TMDL view

TMDL (Tabular Model Definition Language) view can be used with Direct Lake semantic models. The TMDL scripts aren't saved unless you're live editing with a Power BI Project (PBIP). Learn

more about TMDL view.

DAX query view

DAX (Data Analysis Expressions) query view can be used with Direct Lake semantic models. The DAX queries aren't saved unless you're live editing with a Power BI Project (PBIP). Learn more about DAX query view.

Migrate Direct Lake on SQL semantic models to Direct Lake on OneLake

If you already have an existing **Direct Lake on SQL** semantic model and want to migrate to **Direct Lake on OneLake**, you can by using **TMDL view**. Direct Lake on OneLake offers the advantage of having tables from multiple sources and no fallback to DirectQuery.

These migration steps aren't recommended if you're using SQL analytics endpoint views in the Direct Lake on SQL semantic model.

To change to Direct Lake on OneLake, follow these steps.

1. **Live edit** the semantic model you want to migrate in Power BI Desktop.
2. In the header, open the dropdown list on the name and choose **Version history** to make a version to return to, if you want to have that option.
3. Go to **TMDL view**.
4. Drag the **Semantic model** node into the editor to script the entire model.
5. Find the **Expression** toward the bottom of the script.
6. Change `Sql.Database("SQL endpoint connection string", "ID of the SQL analytics endpoint")` to `AzureStorage.DataLake("https://onelake.dfs.fabric.microsoft.com/ID of the workspace/ID of the lakehouse or warehouse")`.
7. If the source is a **Lakehouse without schemas**, remove all `schemaName` property references. Select **Find** in the ribbon to find one. Select it and use `CTRL+SHIFT+L` to select them all, then `CTRL+SHIFT+K` to remove all the lines at once.
8. Then select **Apply**.
9. On success, go to **Model view** to **Refresh** the model. You can adjust credentials in the **Settings** page of the model in the web.

Now the semantic model is using Direct Lake on OneLake. If there are issues, you can restore to the version you created to return to Direct Lake on SQL storage mode.

Requirements and permissions

- XMLA Endpoint must be enabled on the tenant. Learn more in the XMLA endpoint article.
- XMLA Endpoint with *Read Write* access must be enabled at the capacity. Learn more in the tools article.
- User must have *Write* permission on the semantic model. Learn more in the permissions article.
- User must have *Viewer* permission on the lakehouse. Learn more in the lakehouse article.
- This feature is unavailable for users with a free license.

Considerations and limitations

- You can't have multiple data sources when using Direct Lake on SQL. Add data to the Fabric data source used by semantic model. Multiple data sources are supported for Direct Lake on OneLake storage mode.
- You can't publish the Power BI Project (PBIP) from Power BI Desktop. You can use Fabric Deployment mechanisms such as Fabric Git Integration or Fabric Item APIs to publish your local PBIP files to a Fabric workspace.
- You can't validate RLS roles from Power BI Desktop. You can validate the role in the service.
- You can't sign out during live editing without unexpected errors.
- You can open external tools, but the external tool must manage authentication to the remote semantic model.
- You can change the data category to *barcode*, but reports linked to the semantic model can't filter by barcodes.
- You can't live edit externally shared semantic models.
- You can use live edit on import tables only if they're part of a composite model, including Direct Lake on OneLake tables
- Review the current known issues and limitations of Direct Lake.

Related content

- Direct Lake overview
- Power BI Project files

Last updated on 12/18/2025

Direct Lake in Power BI Project (preview)

Article • 05/12/2025

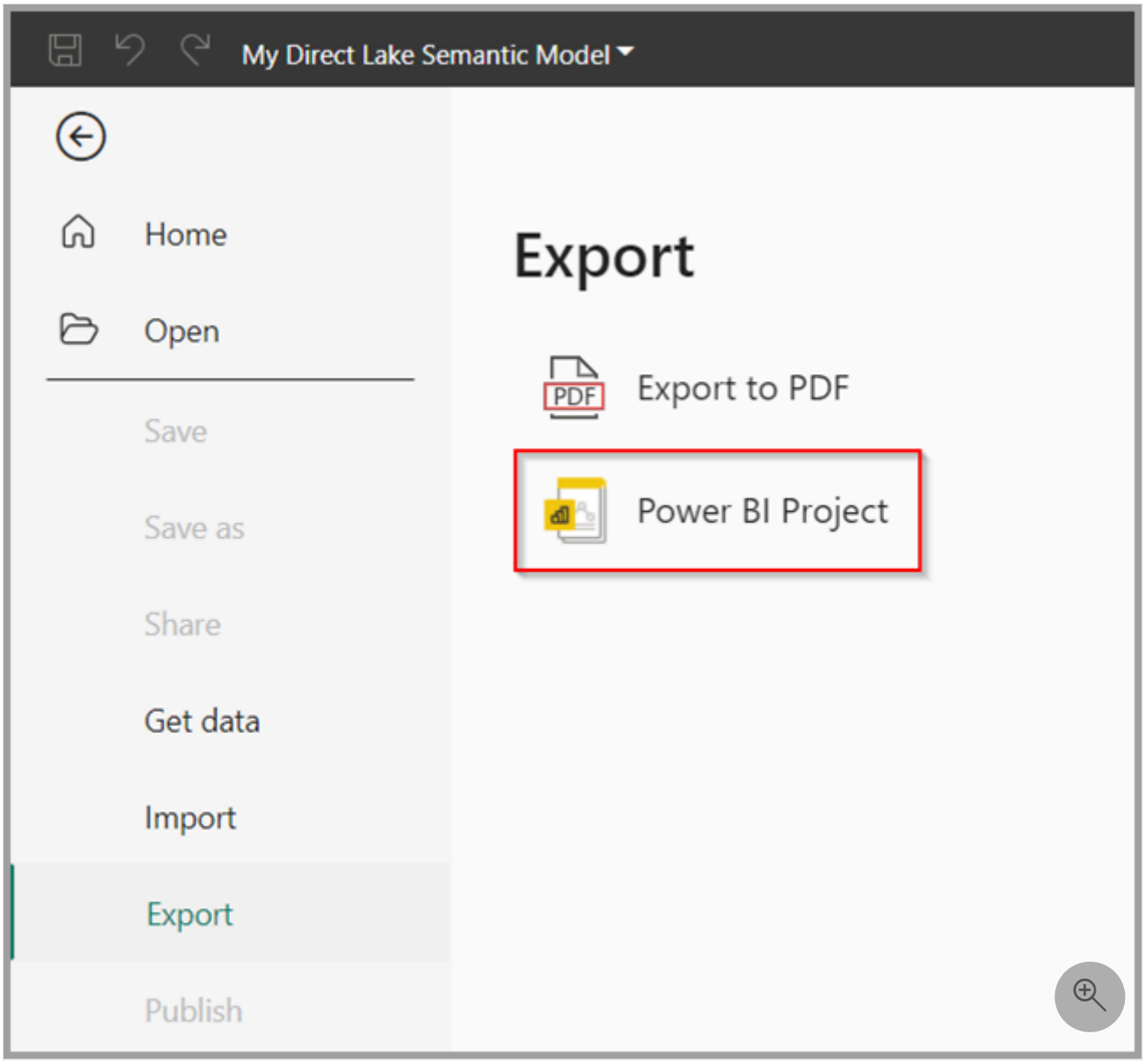
When working on a Power BI Project (PBIP) with a semantic model with Direct Lake tables, Power BI Desktop needs to connect to a semantic model in a Fabric workspace, also termed as a remote semantic model. Remote modeling is live editing, as all changes you make are immediately applied to the semantic model in the workspace. However, unlike live edit, you can save your semantic model and report definitions, or metadata, to your local PBIP files that can later be deployed to a Fabric workspace using a deployment mechanism such as Fabric Git Integration. Learn more about remote modeling with Power BI Project (PBIP).

To support professional enterprise development workflows of semantic models in Direct Lake mode, you can export the definition of your semantic model after opening it for editing, which provides a local copy of the semantic model and report metadata that you can use with Fabric deployment mechanisms such as Fabric Git Integration. The Power BI Desktop **report view** becomes enabled letting you view and edit the local report. **Publish** directly from Power BI Desktop isn't available but you can publish using Git integration. The **Save** button is also enabled to save the local model metadata and report in the Power BI Project folder.

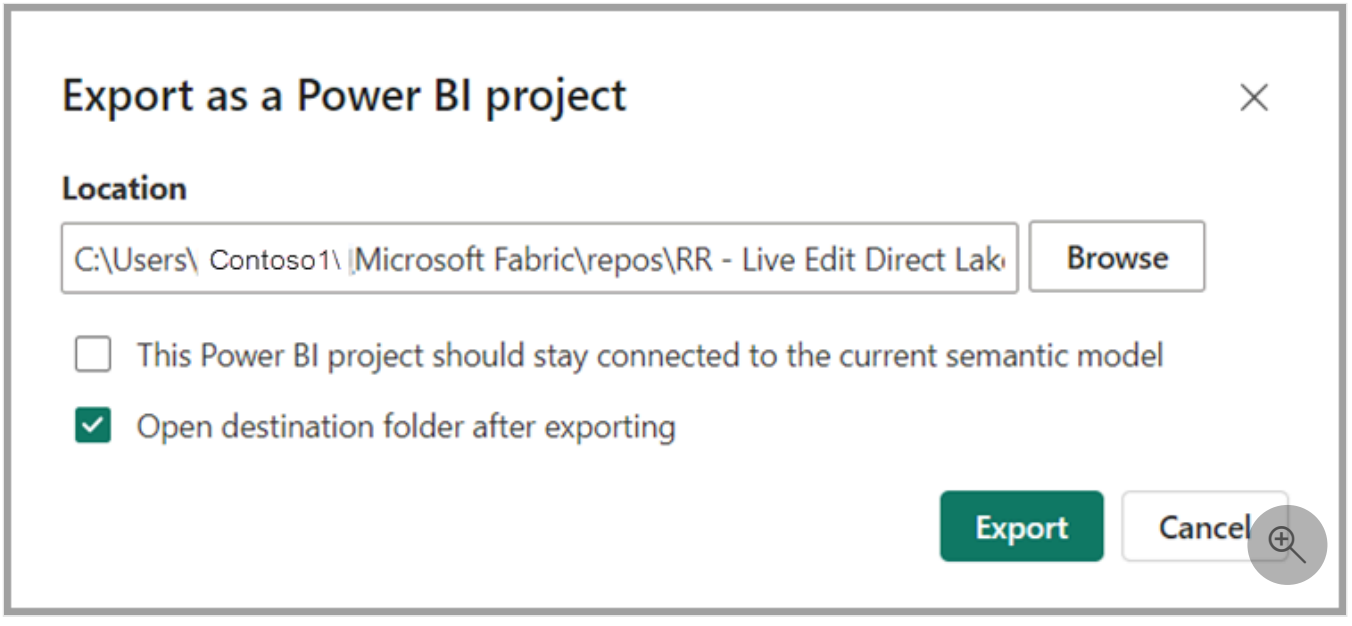
Export to a Power BI Project

To support professional enterprise development workflows of semantic models in Direct Lake mode, you can export the definition of your semantic model after opening it for editing, which provides a local copy of the semantic model and report metadata that you can use with Fabric deployment mechanisms such as Fabric Git Integration. The Power BI Desktop report view becomes enabled letting you view and edit the local report, publish directly from Power BI Desktop isn't available but you can publish using Git integration. The **Save** button is also enabled to save the local model metadata and report in the Power BI Project folder.

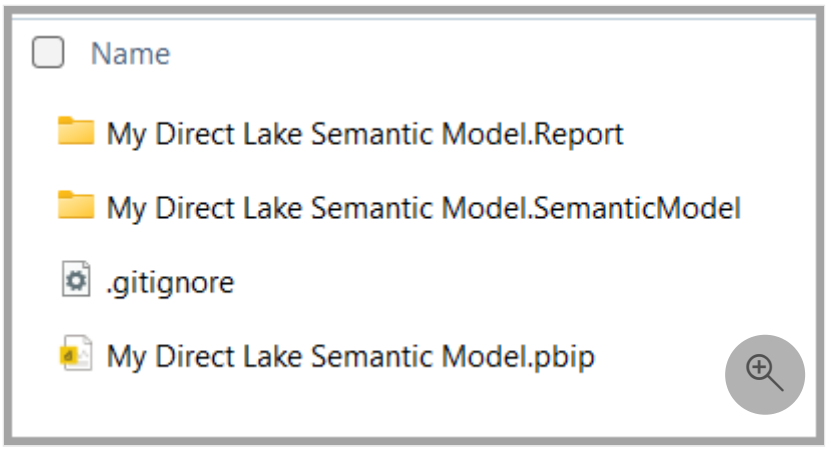
Navigate to **File** > **Export** > **Power BI Project** and export it as a Power BI Project file (PBIP).



By default, the PBIP file is exported to the `%USERPROFILE%\Microsoft Fabric\repos\[Workspace Name]` folder. However, you can choose a different location during the export process.



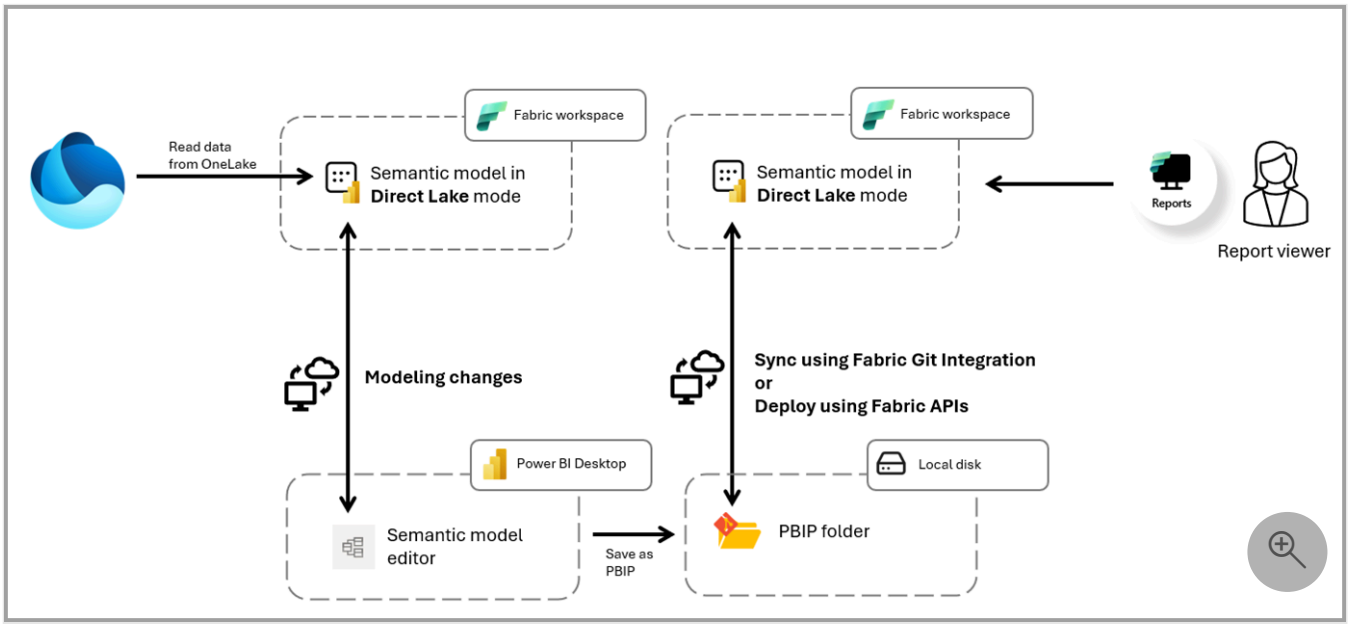
Selecting **Export** opens the folder containing the PBIP files of the exported semantic model along with an empty report.



After exporting, you should open a new instance of Power BI Desktop and open the exported PBIP file to continue editing with a Power BI Project. When you open the PBIP file, Power BI Desktop prompts you to either create a new semantic model in a Fabric workspace, or select an existing semantic model for **remote modeling**.

Remote modeling with a Power BI Project

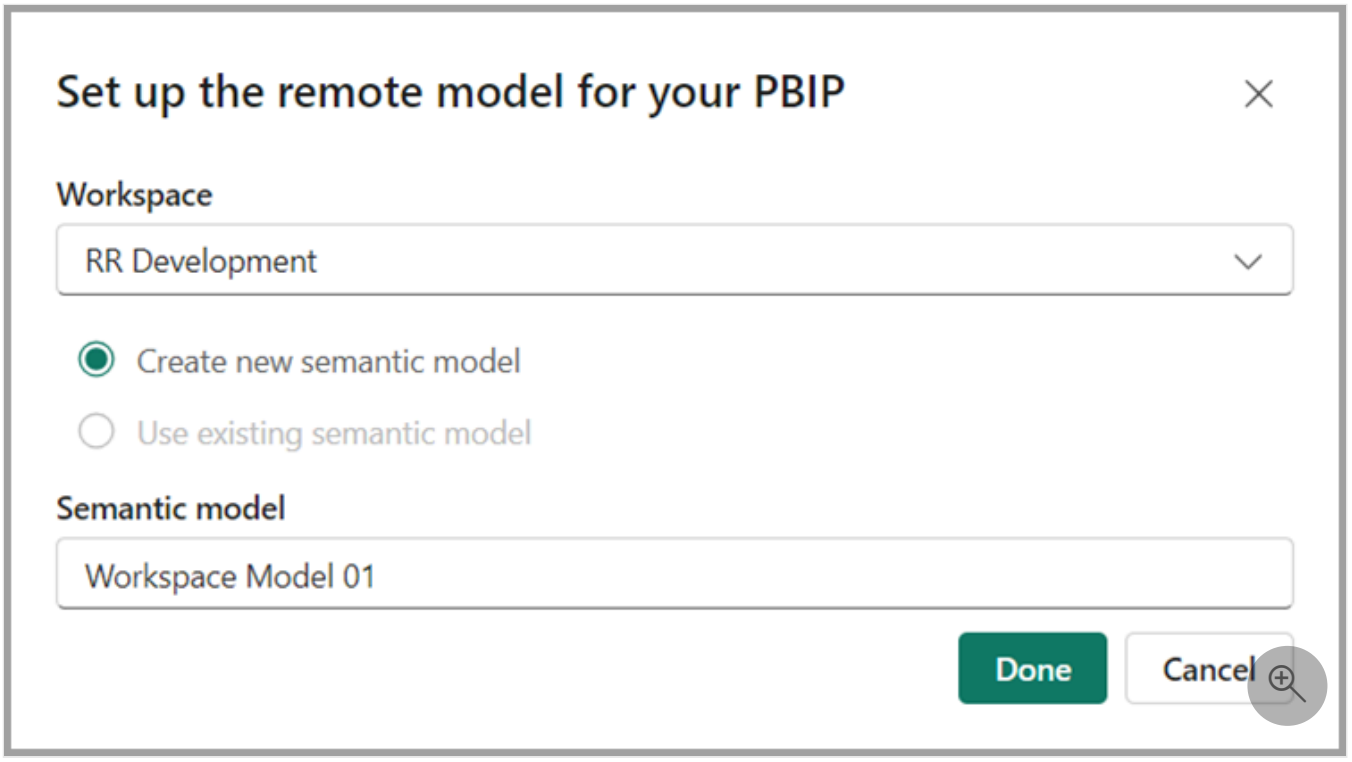
When working on a Power BI Project (PBIP) with a semantic model that can't run on the local Power BI Analysis Services engine, such as Direct Lake mode, Power BI Desktop requires to be connected to a semantic model in a Fabric workspace, a remote semantic model. Like *live edit*, all changes you make are immediately applied to the semantic model in the workspace. However, unlike live edit, you can save your semantic model and report definitions to local PBIP files that can later be deployed to a Fabric workspace using a deployment mechanism such as Fabric Git Integration.



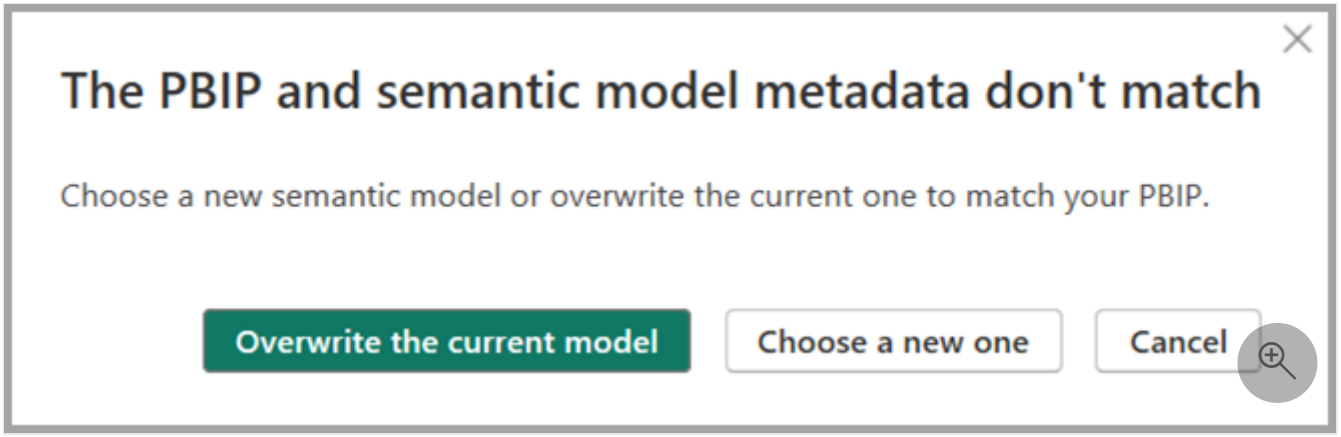
> **Note**
>
> Semantic models in Direct Lake mode, when exported to a Git repository using Fabric Git Integration, can be edited using Power BI Desktop. To do so, make sure at least one report is connected to the semantic model, then open the report's exported definition.pbir file to edit both the report and the semantic model.

Open your Power BI Project

When opening a Power BI Project (PBIP) that require a remote semantic model, Power BI Desktop prompts you to either create a new semantic model or select an existing semantic model in a Fabric workspace.

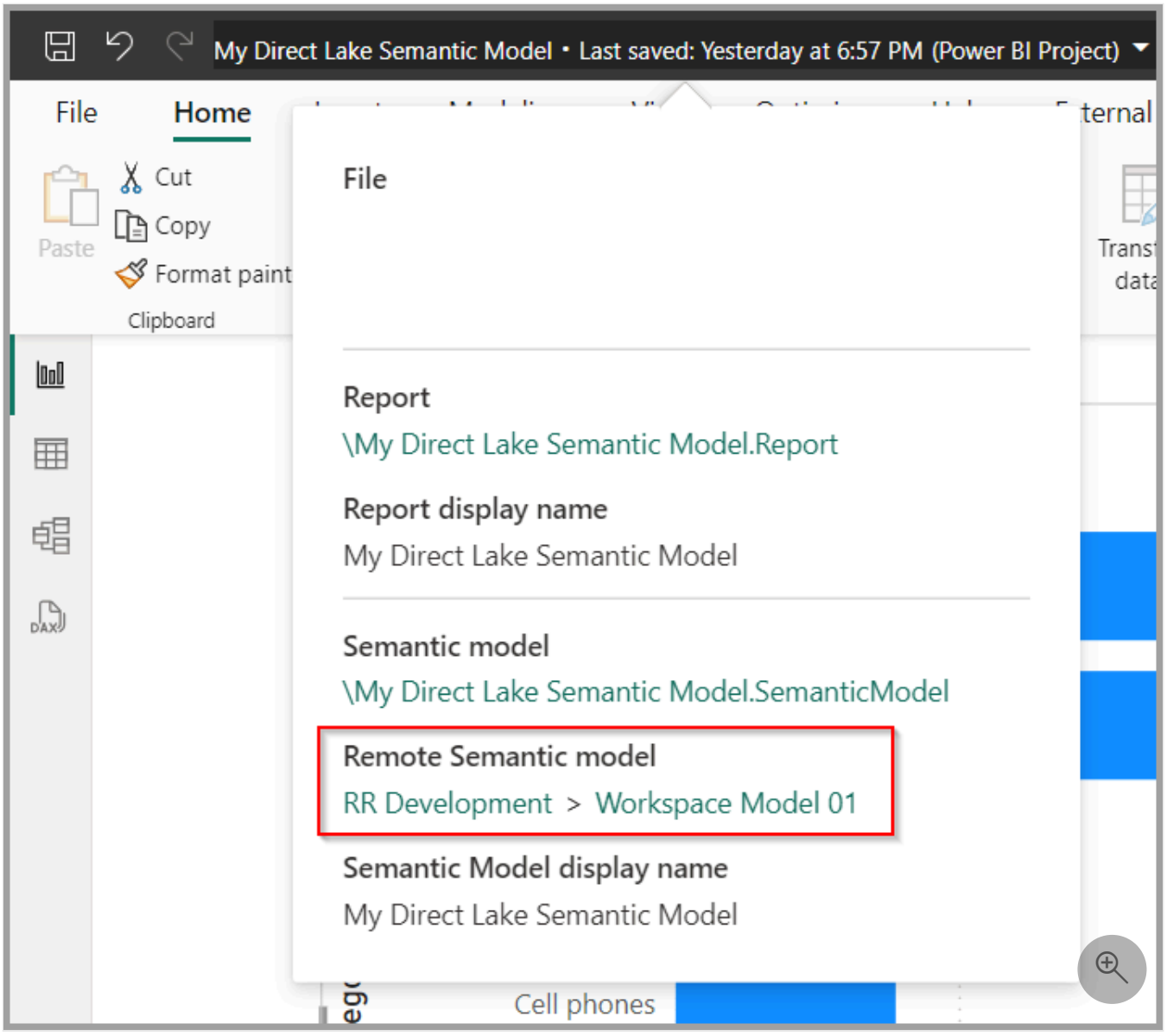


If you select an existent semantic model and the definition differs, Power BI Desktop warns you before overwriting, as shown in the following image.



> **Note**
>
> You can select the same semantic model you exported the PBIP from. However, the best practice when working with a PBIP that requires a remote semantic model is for each developer to work on their own private remote semantic model to avoid conflicts with changes from other developers.

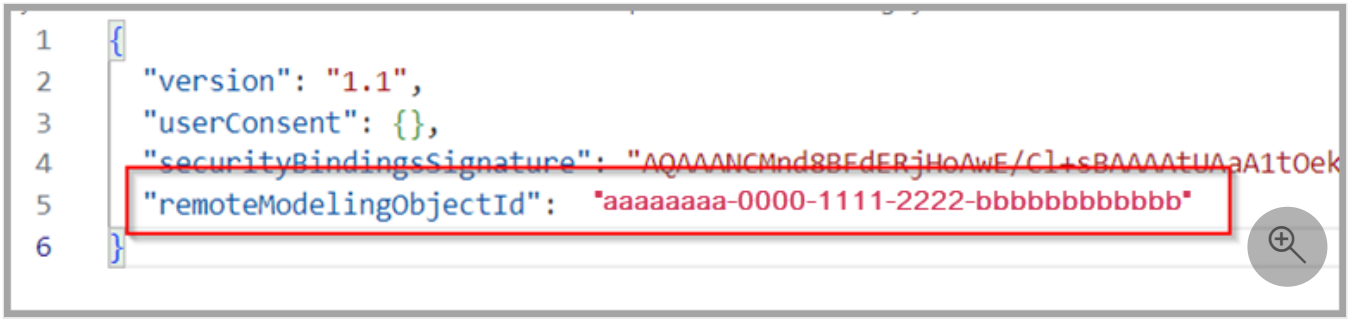
Selecting the title bar displays both the PBIP file location and the remote semantic model living in a Fabric workspace, shown in the following image.



A local setting will be saved in the Power BI Project files with the configured semantic model, next time you open the PBIP, you won't see the prompt, and Fabric semantic model will be overwritten with the metadata from the semantic model in the Power BI Project files.

Change remote semantic model

During the **preview**, if you wish to switch the remote semantic model in the PBIP you must navigate to the `\*.SemanticModel\.pbi\localSettings.json` file. There, you can either modify the *remoteModelingObjectId* property to the ID of the semantic model you want to connect to, or remove the property altogether. Upon reopening the PBIP, Power BI Desktop connects to the new semantic model or prompts you to create or select an existing semantic model.



> ⓘ **Note**
>
> The configuration described in this section is intended solely for local development and should not be used for deployment across different environments.

Related

- Power BI Project (PBIP)

Build Power BI Reports with Direct Lake Tables

Semantic models with Direct Lake tables can be used like any other Power BI semantic model. You can create Power BI reports, Power BI explorations, and run Data Analysis eXpression (DAX) queries.

When a Power BI report shows data in visuals, it requests it from the semantic model. With Direct Lake mode, the semantic model accesses the OneLake delta table to consume data and return it to the Power BI report. For efficiency, the semantic model can keep some recently accessed data in the cache.

With Direct Lake on SQL, when the semantic model can't use Direct Lake it can fall back to DirectQuery and access the data via the SQL analytics endpoint. This behavior is controlled by the **Direct Lake behavior** property.

Creating a report

Creating a report from a Power BI semantic model is easy. The report live connects to the semantic model. In a live connection, the report can be created and edited without editing the semantic model itself. You need at least Build permission on the semantic model to live connect.

Power BI Desktop

Power BI Desktop can live connect to any semantic model in the Power BI service to create a report.

> ⓘ **Note**
>
> Live connect is different from live editing a semantic model in Power BI Desktop. Live connect is also different than having a local semantic model with import or DirectQuery tables and report together.

1. Open **Power BI Desktop**
2. Select **OneLake catalog** or **Get data from other sources** ribbon button
3. Filter by **Semantic model**
4. Select the semantic model with the Direct Lake tables and then **Connect**

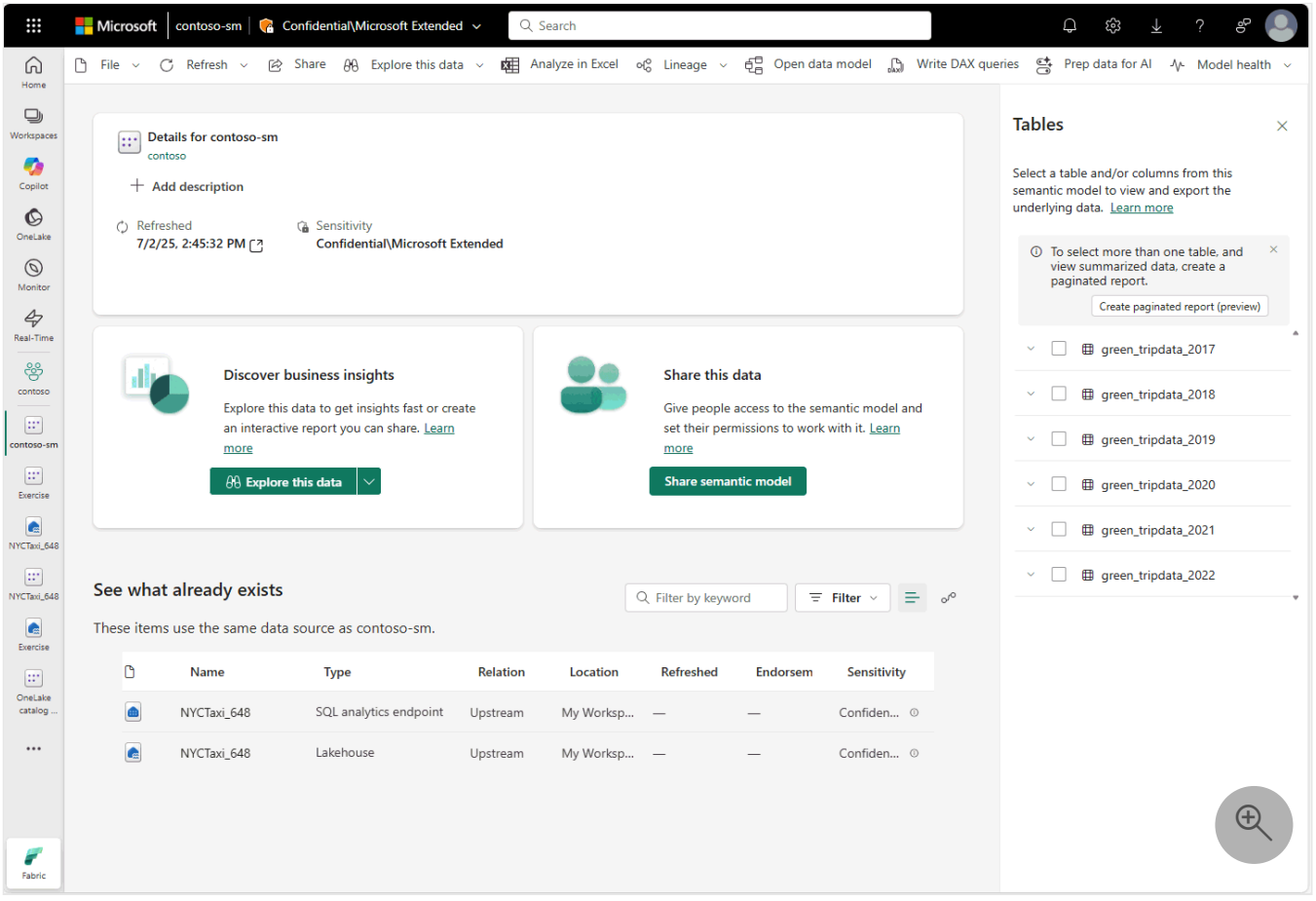
You're now live connected to the semantic model and can start creating the report. Learn more about Power BI reports at the Power BI reporting documentation. Save the file locally and publish to any workspace when ready to see it online and share with others.

In Power BI Desktop, report measures can be created in a live connected report to add a calculation without adding measures to the semantic model itself.

Power BI service or Fabric portal

The Power BI service or Fabric portal has many paths to create a report with a live connection to a semantic model. Here are a few of the paths to create a report.

- Use the context menu (...) of a semantic model in a workspace, then choose **Create report**
- From **Home** choose **New report**, then **Pick a published semantic model** and select the semantic model with Direct lake tables
- From **OneLake catalog** using the drop-down **All items by** filter to **Data** and **Semantic model**, select the semantic model with Direct lake tables then select **Explore** and **Create a blank report** from the top bar
- In web modeling, after choosing **Open data model**, go to **File**, then **Create new report**



Any of these actions create a Power BI report in the web browser.

Other reporting options

There are many other ways to use Power BI semantic models. Here are a few of the other options.

- Explore are created from the context menu or details page of a semantic model in the Power BI service
- Paginated reports are created from the context menu or details page of a semantic model in the Power BI service
- DAX queries can be run from the context menu or details page of a semantic model in the Power BI service or in Power BI Desktop using DAX query view
- Excel with Power BI add-on pane can be used to create refreshable pivot tables or flat tables of data from a semantic model

Related content

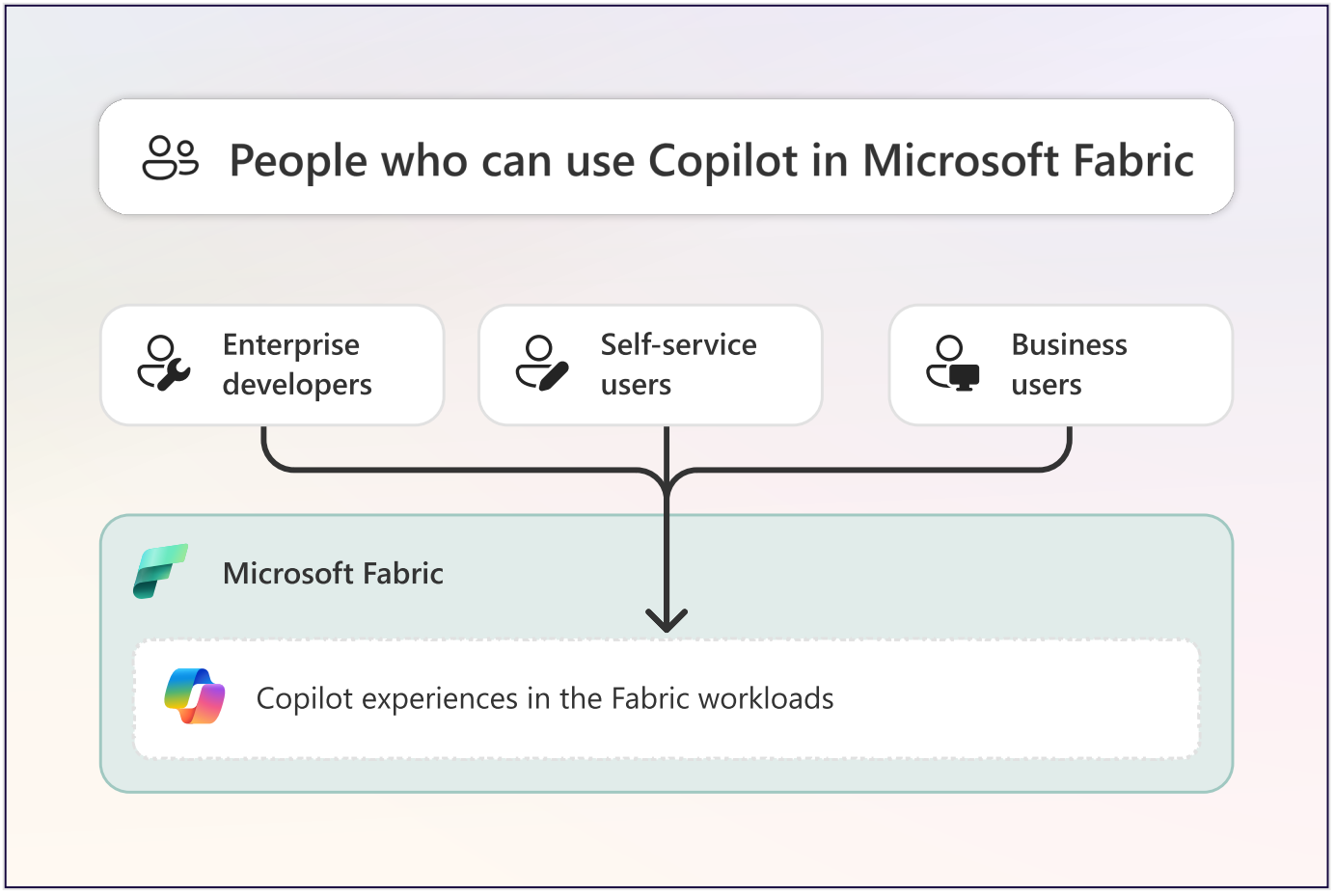
- Power BI reporting

Last updated on 03/31/2026

What is Copilot in Fabric?

Copilot and other generative AI features in preview bring new ways to transform and analyze data, generate insights, and create visualizations and reports in Microsoft Fabric and Power BI. This article introduces you to Copilot in Fabric and explains how it works in the different workloads. It also discusses data security and privacy, responsible use, and regional availability.

Copilot in Microsoft Fabric is a generative AI assistive technology that aims to enhance the data analytics experience in the Fabric platform for users. Copilot uses large-language models (LLMs) that attempt to facilitate user interaction with their data and items in Fabric. People who use Copilot can include people who create, manage, and consume Fabric items, including enterprise developers, self-service users, and business users.



This article helps you understand how Copilot in Fabric works, including its architecture and cost. The information in this article is intended to guide you and your organization to use and manage Copilot effectively. This article is primarily targeted at:

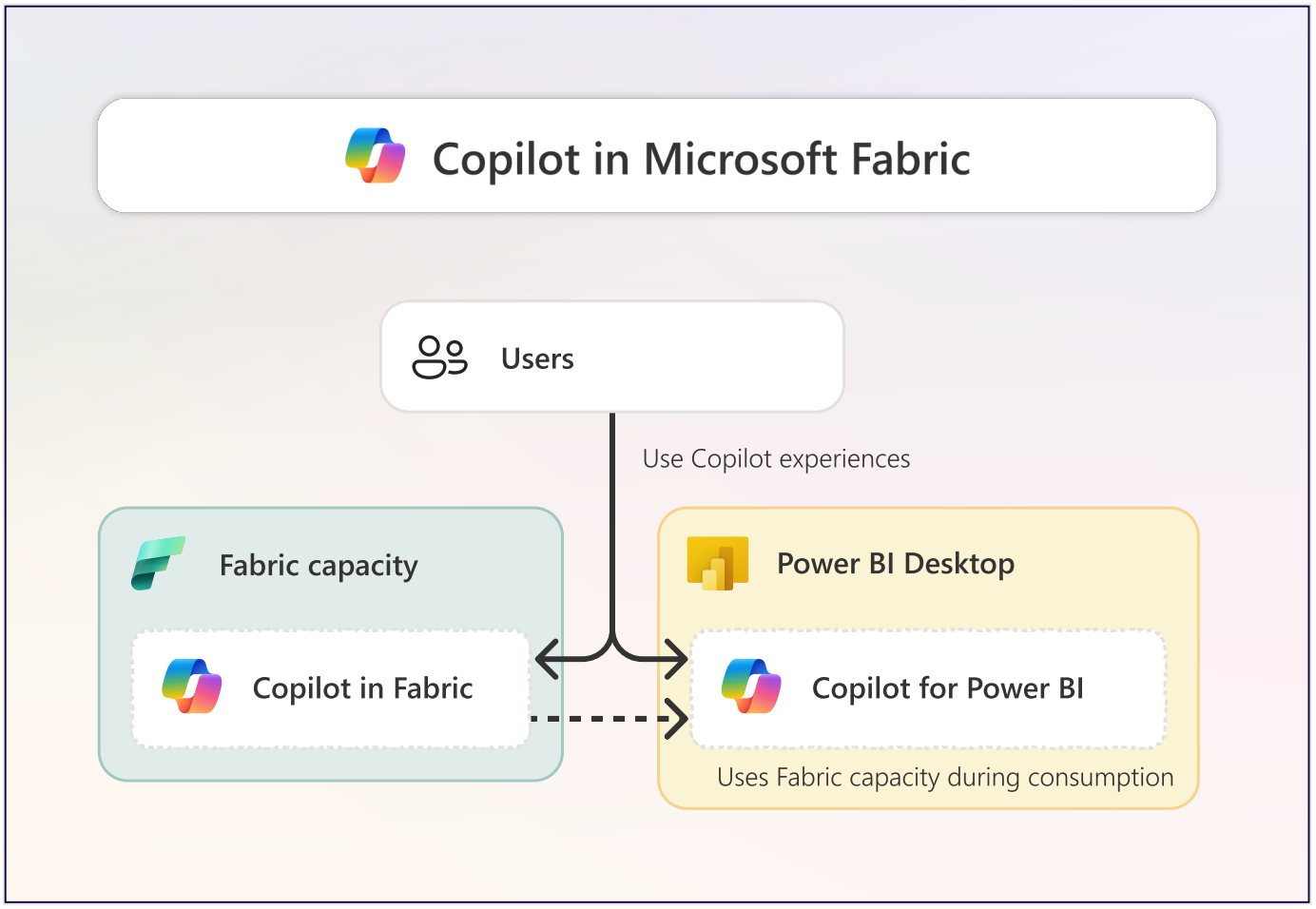
- **BI and analytics directors or managers:** Decision makers who are responsible for overseeing the BI program and strategy, and who decide whether to enable and use Copilot in Fabric or

other AI tools.

- **Fabric administrators:** The people in the organization who oversee Microsoft Fabric and its various workloads. Fabric administrators oversee who can use Copilot in Fabric for each of these workloads and monitor how Copilot usage affects available Fabric capacity.
- **Data architects:** The people responsible for designing, building, and managing the platforms and architecture that support data and analytics in the organization. Data architects consider the usage of Copilot in architecture design.
- **Center of Excellence (COE), IT, and BI teams:** The teams that are responsible for facilitating successful adoption and use of data platforms like Fabric in the organization. These teams and individuals might use AI tools like Copilot themselves, but also support and mentor self-service users in the organization to benefit from them, as well.

> **Important**
>
> Copilot in Fabric aims to *augment* the abilities and intelligence of human users. Copilot can't, and doesn't, aim to *replace* the people who today create and manage reports or other Fabric items. To get the most out of Copilot in Fabric, consider how you enable these individuals and teams to make their work more efficient or improve their outcomes.

There are different Copilots in each of the Fabric workloads, like Data Factory, Data Science, and Power BI. In Power BI Desktop, Copilot experiences are available to users with access to a workspace that uses Fabric capacity or those assigned to a Fabric Copilot capacity.



Enabling and effectively using Copilot in your organization requires deliberate thought and planning. Generative AI is an evolving technology with specific nuances and considerations to keep in mind, so it's important to apply generative AI tools like Copilot to the appropriate problems and scenarios. Furthermore, Copilot in Power BI consumes your available Fabric capacity, so you should manage its usage to avoid overconsumption that can lead to throttling and disruption of your other Fabric operations.

Enable Copilot

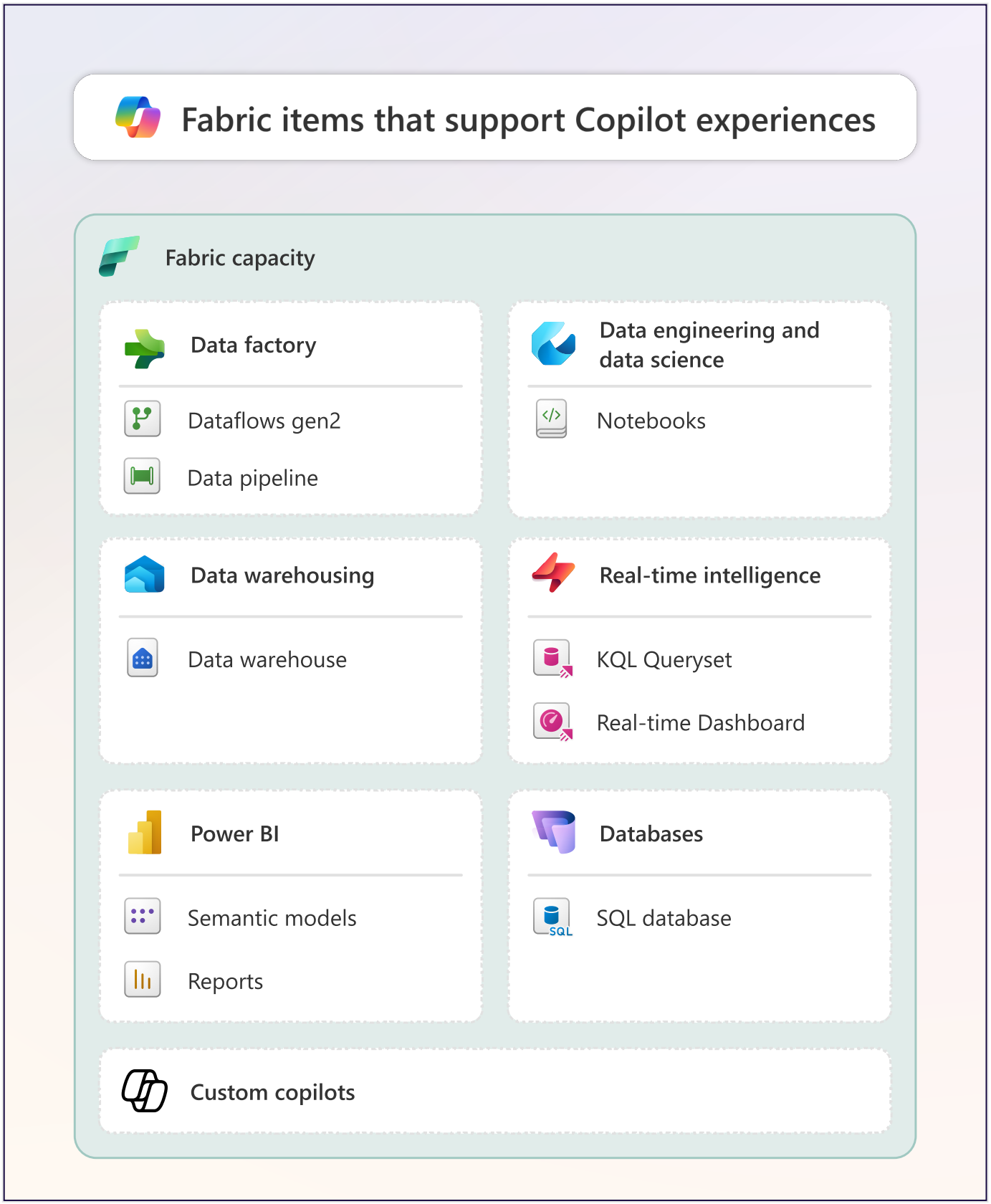
Copilot is enabled by default for tenants with paid Fabric capacities (F2 or higher). As an administrator, review and configure Copilot for your organization by verifying tenant settings, managing capacity-level options, and controlling which users and workspaces have access. You can make Copilot available to all users or specific groups of users, depending on your organization's needs and policies.

> **Note**
>
> Copilot isn't yet supported for sovereign clouds due to GPU availability.

Copilot experiences

You can enable and use Copilot across the various Fabric workloads. Within those workloads, you can use different Copilot experiences. While each of these Copilot experiences uses a similar common architecture, they work in distinct ways.

The following diagram depicts an overview of the different items that support Copilot experiences for each Fabric workload.



Copilot in Fabric in the Data Science and Data Engineering workloads

Copilot for Data Engineering and Data Science is an AI-enhanced toolset tailored to support data professionals in their workflow. It provides intelligent code completion, automates routine tasks, and supplies industry-standard code templates to facilitate building robust pipelines and crafting

complex analytical models. When you use advanced machine learning algorithms, Copilot offers contextual code suggestions that adapt to the specific task at hand, helping you code more effectively and with greater ease. From data preparation to insight generation, Microsoft Fabric Copilot acts as an interactive aide, lightening the load on engineers and scientists and expediting the journey from raw data to meaningful conclusions.

Copilot in notebooks is immediately context-aware without requiring you to start a session. It understands the current workspace, attached Lakehouse schemas, tables, and files, the notebook's structure, and the runtime state. This grounding allows Copilot to provide relevant suggestions and responses from the moment you open the chat pane.

Beyond individual cells, Copilot supports notebook-wide, multi-step assistance. It can generate, refactor, summarize, and validate code across the entire notebook workflow, helping you streamline end-to-end tasks and refactor code into reusable functions.

Copilot also surfaces performance insights based on data size, join patterns, and runtime behavior. For example, it can recommend efficient join strategies, identify potential data shuffles to avoid, and flag data quality issues.

When a cell or Spark job fails, the **Fix with Copilot** feature (preview) provides an error summary, root-cause analysis, and recommended fixes. Copilot can auto-apply code changes with an approval diff so you can review before committing. You can also invoke targeted diagnostics by using the `/fix` command in Copilot chat for a specific cell or the entire notebook.

Copilot in Fabric in the Data Factory workload

Copilot for Data Factory is an AI-enhanced toolset that supports both citizen and professional data wranglers in streamlining their workflow. It provides intelligent code generation to transform data with ease and generates code explanations to help you better understand complex tasks. For more information, see Copilot for Data Factory.

Copilot in Fabric in the Data Warehouse workload

Microsoft Copilot for Fabric Data Warehouse is an AI assistant designed to streamline your data warehousing tasks. Key features of Copilot for Warehouse include natural language to SQL, code completion, quick actions, and intelligent insights. For more information, see Copilot for Data Warehouse.

Copilot in the SQL database workload

Copilot for SQL database in Microsoft Fabric is an AI assistant designed to streamline your OLTP database tasks. Key features of Copilot for SQL database include Natural Language to SQL, code completion, quick actions, and document-based Q&A. For more information, see Copilot for SQL database.

The Fabric portal Query Editor includes a built-in Copilot experience for SQL database that uses the same underlying tooling as the SQL Server Management Studio (SSMS) and Visual Studio Code integrations to provide consistent chat and inline assistance.

Tooling surfaces for SQL database in Fabric

Copilot is also available when you connect to a SQL database in Fabric from external tools:

Copilot is available when connecting to a Fabric SQL database from external tools such as SQL Server Management Studio (SSMS) 22 and the MSSQL extension for Visual Studio Code, offering inline T-SQL assistance and chat-based help grounded to the connected database. These integrations allow you to use Copilot's capabilities while working with your SQL database in the tools you prefer, providing flexibility and convenience in how you interact with your data.

- **SQL Server Management Studio (SSMS)**: When you connect to a SQL database in Fabric from SSMS, you can use Copilot for SQL database features such as code generation, code completion, quick actions, and document-based Q&A. Additionally, you can get inline T-SQL completions and chat-based assistance grounded to the connected database, and analyze execution plans with Copilot's help.
- **MSSQL extension for Visual Studio Code**: The MSSQL extension for Visual Studio Code integrates GitHub Copilot with chat and inline assistance grounded to the connected database, and supports Agent Mode for multi-step, tool-driven workflows.

Copilot is available when connecting to a Fabric SQL database from external tools such as SQL Server Management Studio (SSMS) and the MSSQL extension for Visual Studio Code, offering inline T-SQL assistance and chat-based help grounded to the connected database.

To learn more, see Copilot with SQL Server Management Studio (SSMS) and MSSQL extension for Visual Studio Code in the Copilot for SQL database documentation.

Copilot in Fabric in the Power BI workload

Power BI introduces generative AI that you can use to create reports automatically by selecting the article for a report or by prompting Copilot for Power BI on a particular article. Use Copilot for Power BI to generate a summary for the report page that you created, and generate synonyms

for better Q&A capabilities. For more information on the features and how to use Copilot for Power BI, see Overview of Copilot for Power BI.

Copilot is also available for Power BI apps, where it's scoped to the curated content within an app. App-scoped Copilot helps users search reports, ask questions, and get summaries based on the app's content. For more information, see Overview of Copilot for Power BI apps (preview).

Copilot in Fabric in the Real-Time Intelligence workload

Copilot for Real-Time Intelligence is an advanced AI tool designed to help you explore your data and extract valuable insights. You can input questions about your data, which are then automatically translated into Kusto Query Language (KQL) queries. Copilot streamlines the process of analyzing data for both experienced KQL users and citizen data scientists. Copilot lets you create and refine KQL queries directly in Real-Time Dashboard tiles using natural language.

Copilot is available both in the KQL queryset editor and inside dashboard tile editing, allowing you to write natural language prompts to generate, replace, or refine KQL queries. This approach gives you flexibility to author queries in the context that best fits your workflow - whether building queries in a queryset or tuning visuals directly within dashboard tiles.

For more information, see Copilot for Writing KQL Queries.

Where to find the AI and Copilot experiences in Fabric

For an up-to-date list of the AI and Copilot experiences available across Fabric workloads, the supported items they apply to, and the release status of each capability, see Release status of AI and Copilot experiences in Fabric.

Create your own AI solution accelerators

Build your own copilots

By using the client advisor AI accelerator tool, you can build a custom copilot with your enterprise data. The client advisor AI accelerator uses Azure OpenAI Service, Azure AI Search, and Microsoft Fabric to create custom Copilot solutions. This all-in-one custom copilot empowers client advisors to use generative AI across structured and unstructured data, optimizing daily tasks, and fostering better interactions with clients. For more information, see the GitHub repo.

Agentic applications for unified data foundation

Accelerate intelligent decision-making at scale with secure, agentic AI built on a unified data foundation in Microsoft Fabric. This solution integrates Microsoft Foundry agents and Semantic Kernel to power intelligent workflows that support natural language querying, governed data access, and AI-driven automation—enhancing agility and innovation. For more information, explore the solution on GitHub.

Unified data foundation with Fabric

Build a unified data foundation in Microsoft Fabric using the Modern Analytics, AI, and Governance (MAAG) framework to deliver scalable insights. This accelerator connects Fabric, OneLake, Purview, and Azure Databricks through a medallion lakehouse architecture, with sample domain models and prebuilt Power BI dashboards. With flexible deployment options and built-in governance, it helps you unify and analyze data across domains efficiently. For more information, explore the solution on GitHub.

Real-Time Intelligence for operations

Use Microsoft Fabric's real-time intelligence on a unified data foundation to analyze live and historical telemetry, detect anomalies, and monitor operations through interactive dashboards. The solution includes automated email alerts and an AI-powered data agent for natural-language insights. For more information, explore the solution on GitHub.

How do I use Copilot responsibly?

To learn how to use Copilot responsibly, see the guidance at Privacy, security, and responsible use for Copilot (preview).

Available regions

Available regions for Azure OpenAI Service

To access the prebuilt Azure OpenAI Service, including the Copilot in Fabric, you must have an F2 or higher SKU or a P SKU in the following Fabric regions. The Azure OpenAI Service isn't available on trial SKUs.

The Azure OpenAI Service that powers Fabric Copilot is currently deployed in US datacenters (East US, East US2, South Central US, and West US) and in the EU data boundary. This deployment differs from the standard Azure OpenAI Service available in the Azure portal, which is accessible in many more regions. For details on standard Azure OpenAI region availability, see

Azure OpenAI Service region availability. If your capacity's geographic region is outside the US or the EU data boundary, Copilot is disabled by default unless your Fabric admin enables the **Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance** tenant setting. To learn how to get to the tenant settings, see About tenant settings.

Data processing across geographic areas

The prebuilt Azure OpenAI Service and Copilot in Fabric might process your prompts and results (input and output when using Copilot) outside your capacity's geographic region, depending on where the Azure OpenAI service is hosted. The following table shows the mapping of where data is processed across geographic areas for Copilot in Fabric and Azure OpenAI features.

> **Note**
>
> The data processed for Copilot interactions can include user prompts, meta prompts, structure of data (schema), and conversation history. You don't send any data, such as content in tables, to Azure OpenAI for processing unless you direct it.

Expand table

Geographic area where your Fabric Capacity is located	Geographic area where Azure OpenAI Service is hosted	Data processing outside your capacity's geographic region?	Actions required to use Fabric Copilot
US	US	No	Turn on Copilot.
EU Data Boundary	EU Data Boundary	No	Turn on Copilot.
UK	EU Data Boundary	Yes	Turn on Copilot. Enable cross-geo data processing.
Australia Brazil Canada India Asia Japan Korea South Africa Southeast Asia United Arab Emirates	US	Yes	Turn on Copilot. Enable cross-geo data processing.

Data storage of conversation history cross geographic regions

> **Note**
>
> This section only applies to customers who want to use **Copilot in Notebooks** and Fabric **data agents** (formerly known as Data agent) powered by Azure OpenAI, and whose capacity's geographic region is outside of the EU data boundary and the US.

Conversational agentic experiences such as Copilot in Notebooks and Fabric data agents store conversation history across user sessions so the agent keeps context between sessions. This history is stored inside the Azure security boundary, in the same region and Azure OpenAI resources that process your other Fabric AI requests. Users can delete their conversation history at any time by clearing the chat; if it isn't removed manually, it's stored for 28 days.

For full details and the tenant setting that controls this behavior, see Copilot and Agent tenant settings.

Related content

- What is Microsoft Fabric?
- Copilot for Microsoft Fabric and Power BI: FAQ
- Foundry Tools in Fabric (preview)
- Copilot tenant settings

Note: The author created this article with assistance from AI. Learn more

---

Last updated on 09/04/2026

How Copilot in Microsoft Fabric works

Article • 04/17/2025

Copilot in Microsoft Fabric is a generative AI assistive technology that aims to enhance the data analytics experience in the Fabric platform. This article helps you understand how Copilot in Fabric works and provides some high-level guidance and considerations about how you might best use it.

> **Note**
>
> Copilot's capabilities are evolving over time. If you plan to use Copilot, ensure that you keep up to date with the monthly updates to Fabric and any changes or announcements to the Copilot experiences.

This article helps you to understand how Copilot in Fabric works, including its architecture and cost. The information in this article is intended to guide you and your organization to use and manage Copilot effectively. This article is primarily targeted at:

- **BI and analytics directors or managers:** Decision makers who are responsible for overseeing the BI program and strategy, and who decide whether to enable and use Copilot in Fabric or other AI tools.
- **Fabric administrators:** The people in the organization who oversee Microsoft Fabric and its various workloads. Fabric administrators oversee who can use Copilot in Fabric for each of these workloads and monitor how Copilot usage affects available Fabric capacity.
- **Data architects:** The people responsible for designing, building, and managing the platforms and architecture that support data and analytics in the organization. Data architects consider the usage of Copilot in architecture design.
- **Center of Excellence (COE), IT, and BI teams:** The teams that are responsible for facilitating successful adoption and use of data platforms like Fabric in the organization. These teams and individuals might use AI tools like Copilot themselves, but also support and mentor self-service users in the organization to benefit from them, as well.

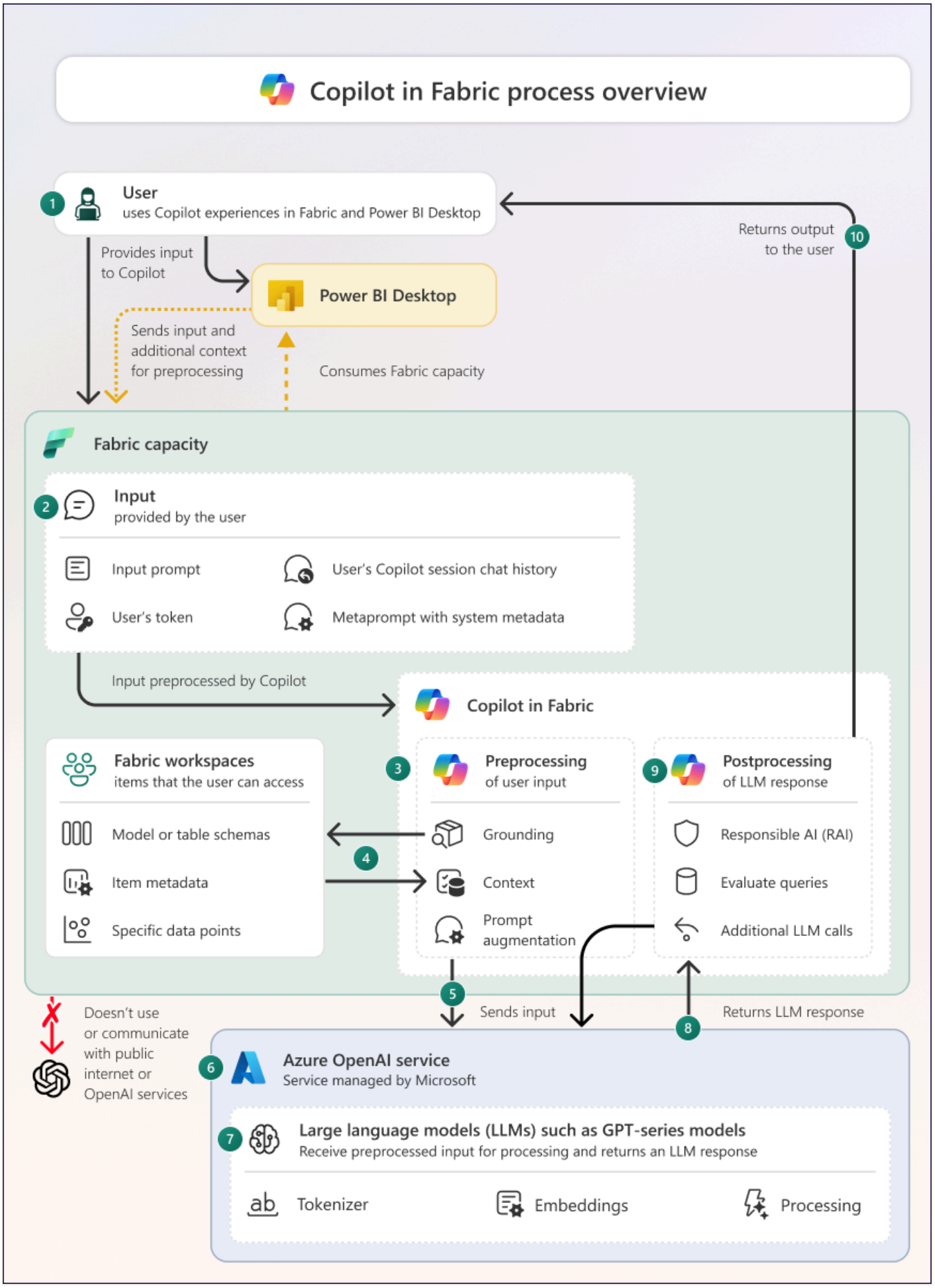
Overview of how Copilot in Fabric works

Copilot in Fabric works similar to the other Microsoft Copilots, such as Microsoft 365 Copilot, Microsoft Security Copilot, and Copilots and generative AI in Power Platform. However, there are several aspects that are specific to how Copilot in Fabric works.

Process overview diagram

The following diagram depicts an overview of how Copilot in Fabric works.

> **Note**
>
> The following diagram depicts the general architecture of Copilot in Fabric. However, depending on the specific workload and experience, there might be additions or differences.



The diagram consists of the following parts and processes:

Expand table

Item	Description
1	The user provides an input to Copilot in Fabric, Power BI Desktop, or the Power BI mobile app. The input can be a written prompt or another interaction that generates a prompt. All interactions with Copilot are user-specific.
2	The input contains information that includes the prompt, the user's token, and context like the user's Copilot session chat history and a meta-prompt with system metadata, including where the user is and what they're doing in Fabric or Power BI Desktop.
3	Copilot handles preprocessing and postprocessing of user inputs and large language model (LLM) responses, respectively. Certain specific steps done during preprocessing and postprocessing depend on which Copilot experience an individual is using. Copilot must be enabled by a Fabric administrator in the tenant settings to use it.
4	During preprocessing, Copilot performs grounding to retrieve additional contextual information to improve the specificity and usefulness of the eventual LLM response. Grounding data might include metadata (such as the schema from a lakehouse or semantic model) or data points from items in a workspace, or the chat history from the current Copilot session. Copilot only retrieves grounding data that a user has access to.
5	Preprocessing results in the final inputs: a final prompt and grounding data. Which data is sent depends on the specific Copilot experience and what the user is asking for.
6	Copilot sends the input to the Azure OpenAI Service. This service is managed by Microsoft and isn't configurable by the user. Azure OpenAI doesn't train models with your data. If Azure OpenAI isn't available in your geographical area and you've enabled the tenant setting *Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance*, then Copilot might send your data outside of these geographical areas.
7	Azure OpenAI hosts LLMs like the GPT series of models. Azure OpenAI doesn't use the public services or APIs of OpenAI, and OpenAI doesn't have access to your data. These LLMs tokenize the input and use embeddings from their training data to process the inputs into a response. LLMs are limited in the scope and scale of their training data. Azure OpenAI contains configuration that determines how the LLM processes the input and which response it returns. It's not possible for customers to view or change this configuration. The call to the OpenAI Service is done via Azure, and not over the public internet.
8	The LLM response is sent from Azure OpenAI to Copilot in Fabric. This response comprises text, which might be natural language, code, or metadata. The response might include inaccurate or low-quality information. It's also non-deterministic, meaning that a different response might be returned for the same input.
9	Copilot postprocesses the LLM response. Postprocessing includes filtering for responsible AI, but also involves handling the LLM response and producing the final Copilot output. The specific steps taken during postprocessing depend on the Copilot experience an individual use.
10	Copilot returns the final output to the user. The user checks the output before use, as the output contains no indication of reliability, accuracy, or trustworthiness.

The next sections describe the five steps in the Copilot process depicted in the previous diagram. These steps explain in detail how Copilot goes from user input to user output.

Step 1: A user provides input to Copilot

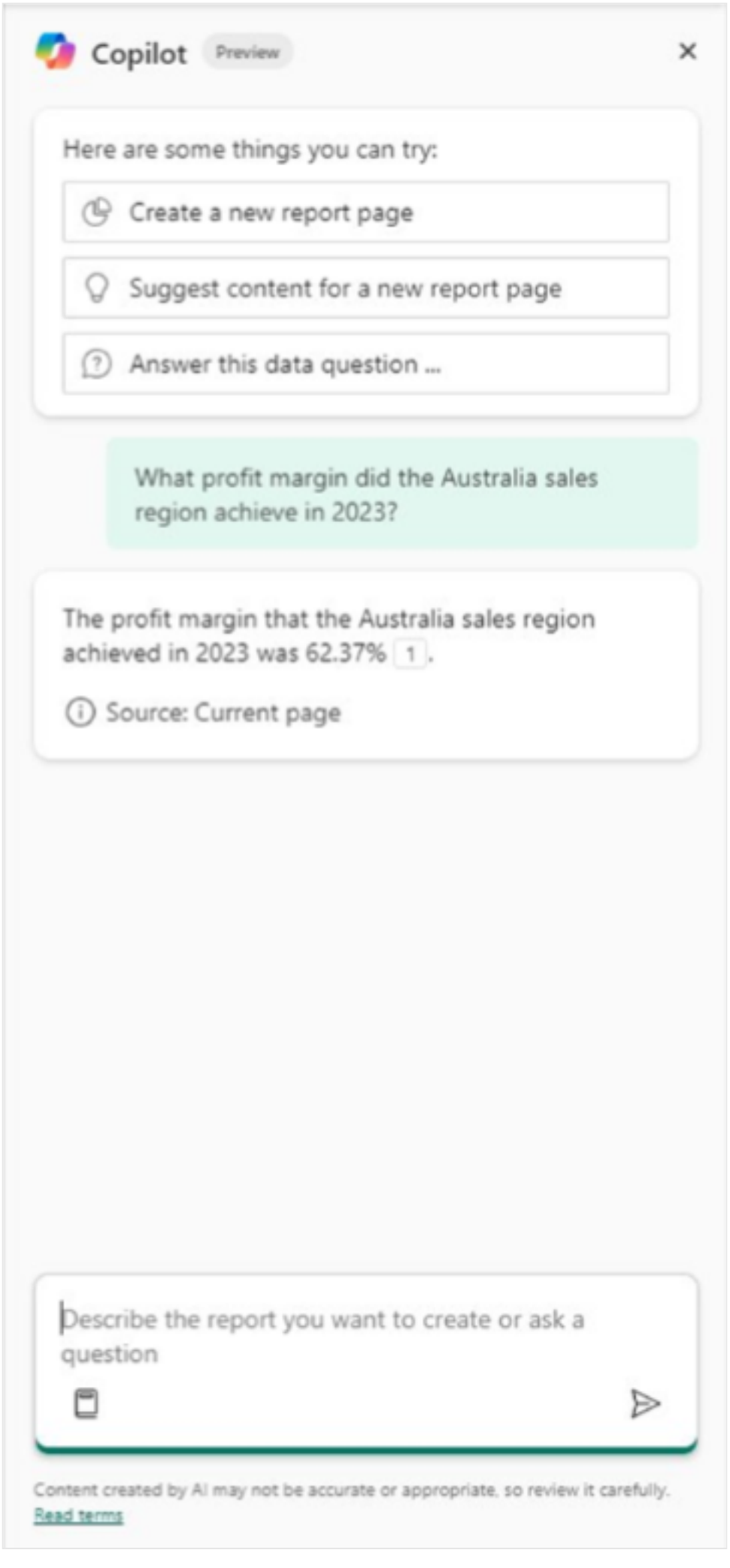
To use Copilot, a user must first submit an input. This input can be a written prompt that the user submits themselves, or it can be a prompt generated by Copilot when the user selects an interactive element in the UI. Depending on the specific Fabric workload, item, and Copilot experience that someone uses, they have different ways to provide an input to Copilot.

The following sections describe several examples of how a user can provide inputs to Copilot.

Input via the Copilot chat panel

With many Copilot experiences in Fabric, you can extend a Copilot chat panel to interact with Copilot using natural language like you would with a chatbot or messaging service. In the Copilot chat panel, you can write a natural language prompt describing the action that you want Copilot to take. Alternatively, the Copilot chat panel might contain buttons with suggested prompts that you can select. Interacting with these buttons causes Copilot to generate a corresponding prompt.

The following image shows an example of using the Copilot chat panel to ask a data question about a Power BI report.

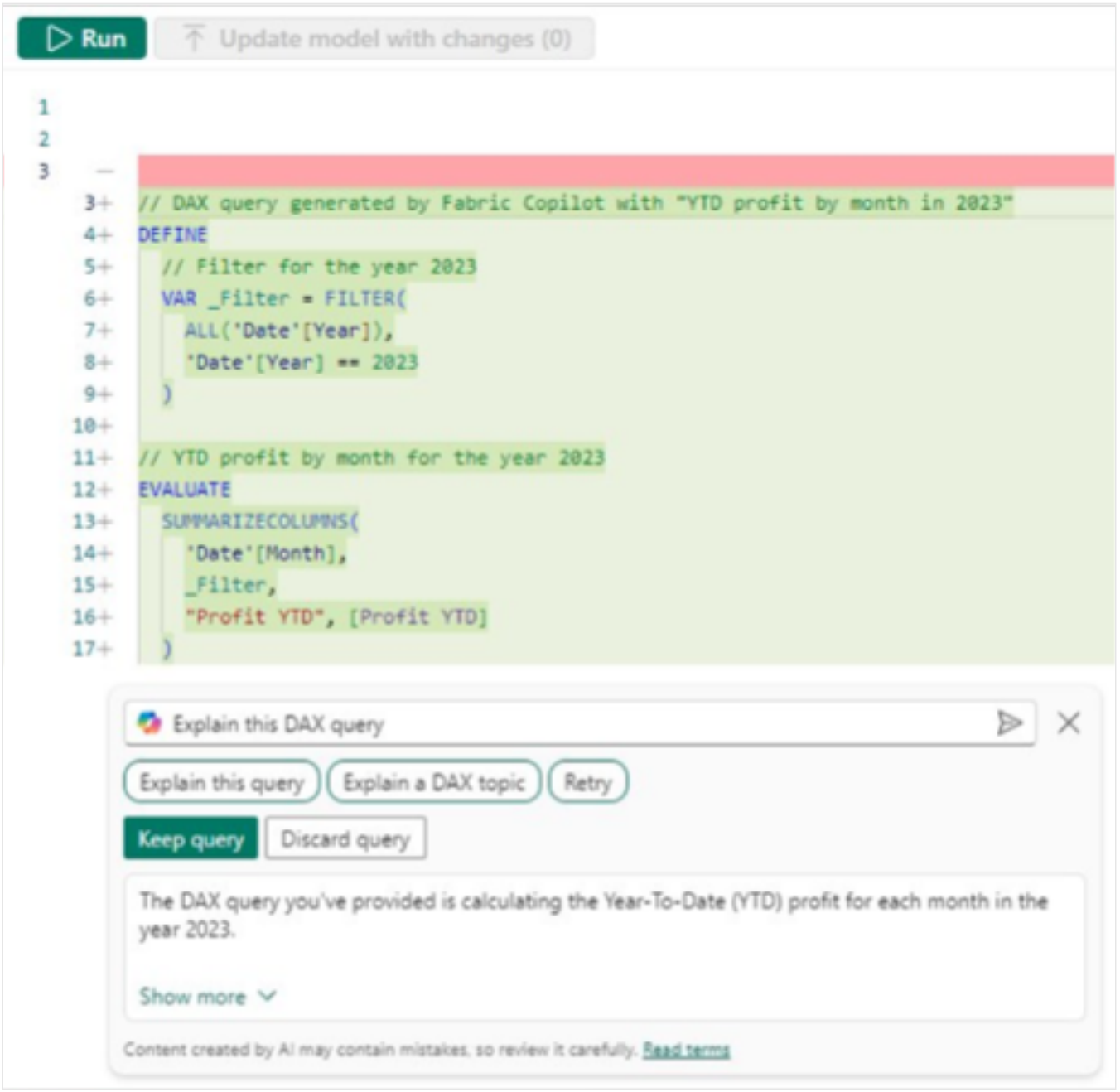

Copilot
Preview
Here are some things you can try:
Create a new report page
Suggest content for a new report page
Answer this data question ...
What profit margin did the Australia sales region achieve in 2023?
The profit margin that the Australia sales region achieved in 2023 was 62.37% 1 .
Source: Current page
Describe the report you want to create or ask a question
Content created by AI may not be accurate or appropriate, so review it carefully.
Read terms

> **Note**
>
> If you use the Microsoft Edge browser, you might also have access to Copilot there. Copilot in Edge can also open a Copilot chat panel (or sidebar) in your browser. The Copilot in Edge can't interact with or use any of the Copilot experiences in Fabric. While both Copilots have a similar user experience, Copilot in Edge is completely separate from Copilot in Fabric.

Input via context-dependent pop-up windows

In certain experiences, you can select the Copilot icon to trigger a pop-up window to interact with Copilot. Examples include when you use Copilot in the DAX query view or in the TMDL scripting view of Power BI Desktop. This pop-up window contains an area for you to enter a natural language prompt (similar to the Copilot chat panel) as well as context-specific buttons that can generate a prompt for you. This window might also contain output information, such as explanations about DAX queries or concepts when using Copilot in the DAX query view.

The following image shows an example of someone using the Copilot experience in the DAX query view to explain a query that they generated by using Copilot in Power BI.



Types of user inputs

Copilot inputs can be from a written prompt or a button in the UI:

- **Written prompt:** A user can write a prompt in the Copilot chat panel, or in other Copilot experiences, like the DAX query view in Power BI Desktop. Written prompts require that the user adequately explains the instruction or question for Copilot. For instance, a user can ask a question about a semantic model or report when using Copilot in Power BI.

- **Button:** A user can select a button in the Copilot chat panel or other Copilot experiences to provide an input. Copilot then produces the prompt based on the user selection. These buttons can be the initial input to Copilot, such as the suggestions in the Copilot chat panel. However, these buttons might also appear during a session when Copilot makes suggestions or requests clarifications. The prompt Copilot generates depends on the

context, such as the chat history of the current session. An example of a button input is when you ask Copilot to suggest synonyms for model fields, or descriptions for model measures.

Additionally, you can provide inputs in different services or applications:

- **Fabric:** You can use Copilot in Fabric from your web browser. This is the only way to use Copilot for any items that you exclusively create, manage, and consume in Fabric.
- **Power BI Desktop:** You can use Copilot in Power BI Desktop with semantic models and reports. These include both the development and consumption Copilot experiences for the Power BI workload in Fabric.
- **Power BI mobile app:** You can use Copilot in the Power BI mobile app if the report is in a supported workspace (or an app connected to that workspace) with Copilot enabled.

> **Note**
>
> To use Copilot with Power BI Desktop, you must configure Power BI Desktop to use Copilot consumption from a supported workspace backed by Fabric capacity. Then you can use Copilot with semantic models published to any workspace, including Pro and PPU workspaces.

While you can't alter the prompts that Copilot generates when you select a button with written prompts, you can ask questions and provide instructions using natural language. One of the most important ways to improve the results you get with Copilot is to write clear and descriptive prompts that accurately convey want you want to do.

Improve written prompts for Copilot

The clarity and quality of the prompt a user submits to Copilot can affect the usefulness of the output the user receives. What constitutes a good written prompt depends on the specific Copilot experience that you're using; however, there are some techniques that you can apply to all experiences to improve your prompts in general.

Here are several ways to improve prompts that you submit to Copilot:

- **Use English language prompts:** Today, Copilot features work best in the English language. That's because the corpus of training data for these LLMs is mostly English. Other languages might not perform as well. You can try to write prompts in other languages, but for the best results, we recommend that you write and submit English language prompts.

- **Be specific:** Avoid ambiguity or vagueness in questions and instructions. Include sufficient details to describe the task you want Copilot to perform, and what is the output that you expect.

- **Provide context:** Where necessary, provide relevant context for your prompt, including what you intend to do or what question you want to answer with an output. For instance, the key components for a good prompt could include:
  - *Goal:* What output you want Copilot to achieve.
  - *Context:* What you intend to do with that particular output and why.
  - *Expectations:* What you expect the output will look like.
  - *Source:* What data or fields Copilot should use.

- **Use verbs:** Refer explicitly to specific actions that you want Copilot to take, such as "*create* a report page" or "*filter* to customer key accounts".

- **Use correct and relevant terminology:** Refer explicitly to the appropriate terms in your prompt, like function, field, or table names, visual types, or technical terminology. Avoid misspellings, acronyms, or abbreviations, as well as superfluous grammar, or atypical characters like Unicode characters or emojis.

- **Iterate and troubleshoot:** When you don't get the expected result, try to adjust your prompt and re-submit it to see if this improves the output. Some Copilot experiences also provide a *Retry* button to re-submit the same prompt and check for a different result.

> **ⓘ Important**
>
> Consider training users to write good prompts before you enable Copilot for them. Ensure that users understand the difference between a clear prompt that can produce useful results, and a vague prompt that doesn't.
>
> Also, Copilot and many other LLM tools are *non-deterministic*. This means that two users submitting the same prompt that uses the same grounding data can obtain different results. This non-determinism is inherent to the underlying technology of generative AI, and is an important consideration when you expect or need deterministic results, such as an answer to a data question, like "*What are the sales in August 2021?*"

Other input information that Copilot uses in preprocessing

Aside from input that a user provides, Copilot also retrieves additional information that it uses in preprocessing during the next step. This information includes:

- **The user's token.** Copilot doesn't operate under a system account or authority. All information sent to and used by Copilot is specific to the user; Copilot can't allow a user to view or access items or data that they don't already have permission to view.
- **The Copilot session chat history for the current session.** For chat experiences or the Copilot chat panel, Copilot always provides the Chat history for use in preprocessing as part of the grounding data context. Copilot doesn't remember or use chat history from previous sessions.
- **Meta-prompt with system metadata.** A meta-prompt provides additional context about where the user is and what they're doing in Fabric or Power BI Desktop. This meta-prompt information is used during preprocessing to determine which skill or tool Copilot should use to answer the user's question.

Once a user submits their input, Copilot proceeds to the next step.

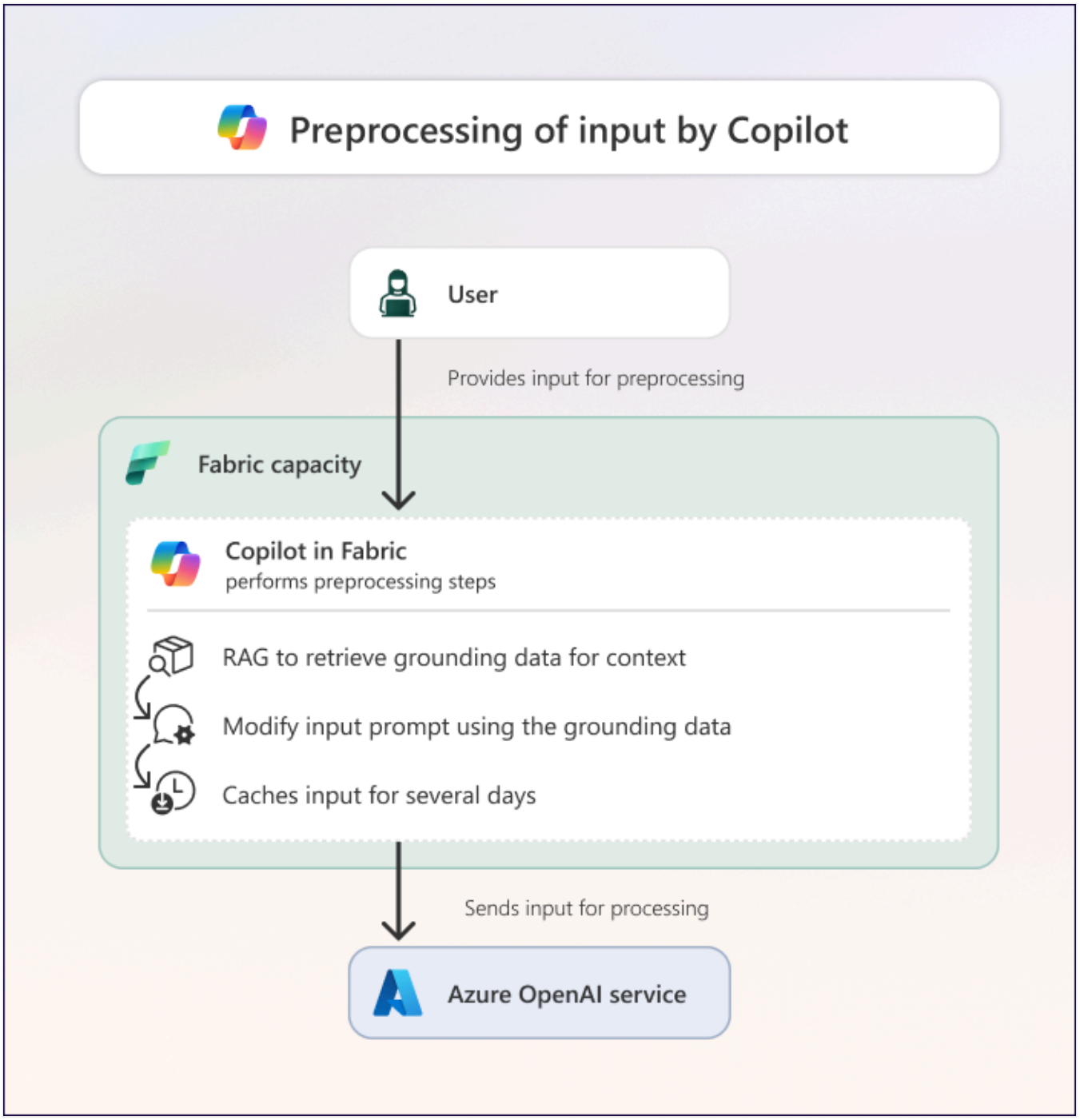
Step 2: Copilot preprocesses the input

Before submitting a prompt to the Azure OpenAI Service, Copilot *preprocesses* it. Preprocessing constitutes all actions that are handled by Copilot between when it receives the input and when this input is processed in the Azure OpenAI Service. Preprocessing is necessary to ensure that Copilot's output is specific and appropriate to your instructions or questions.

You can't affect what preprocessing is done by Copilot. However, it's important to understand preprocessing so that you know what data Copilot uses and how it gets it. This is pertinent to understanding the cost of Copilot in Fabric, and when you troubleshoot why it produces an incorrect or unexpected result.

> **Tip**
>
> In certain experiences, you can also make changes to items so that their grounding data is better structured for Copilot to use. An example is performing linguistic modeling in a semantic model, or adding synonyms and descriptions to semantic model measures and columns.

The following diagram depicts what happens during preprocessing by Copilot in Fabric.



After receiving user input, Copilot performs preprocessing, which involves the following steps:

- **Grounding:** Copilot performs retrieval augmented generation (RAG) to collect *grounding data*. Grounding data comprises relevant information from the current context in which you're using Copilot in Fabric. Grounding data might include context such as:
  - The chat history from the current session with Copilot.
  - Metadata about the Fabric item that you're using with Copilot (like the schema of your semantic model or lakehouse, or metadata from a report visual).
  - Specific data points, such as those displayed in a report visual. Report metadata in the visual configuration also contains data points.
  - Meta-prompts, which are supplemental instructions provided for each experience to help ensure a more specific and consistent output.

- **Prompt augmentation:** Depending on the scenario, Copilot rewrites (or *augments*) the prompt based on the input and grounding data. The augmented prompt should be better and more context-aware than your original input prompt.

- **Caching:** In certain scenarios, Copilot caches your prompt and the grounding data for 48 hours. Caching the prompt ensures that repeated prompts return the same results while cached, that they return these results faster, and that you aren't consuming Fabric capacity just for repeating a prompt in the same context. Caching occurs in two different places:
  - The browser cache of the user.
  - The first back-end cache in the home region of the tenant, where it's stored for auditing purposes. No data is cached in the Azure OpenAI Service or the location of the GPUs. For more information about caching in Fabric, refer to the Microsoft Fabric security whitepaper.

- **Sending input to Azure OpenAI:** Copilot sends the augmented prompt and the relevant grounding data to the Azure OpenAI Service.

When Copilot performs grounding, it only collects information from data or items that a user can access normally. Copilot respects workspace roles, item permissions, and data security. Copilot also can't access data from other users; interactions with Copilot are specific to each individual user.

The data that Copilot collects during the grounding process and what Azure OpenAI processes depends on the specific Copilot experience that you use. For more information, see What data does Copilot use and how is it processed?.

After the preprocessing is finished and Copilot has sent the input to Azure OpenAI, the Azure OpenAI Service can process that input to produce a response and output that's sent back to Copilot.

Step 3: Azure OpenAI processes the prompt and generates an output

All Copilot experiences are powered by the Azure OpenAI Service.

Understand the Azure OpenAI Service

Copilot uses Azure OpenAI—not OpenAI's publicly available services—to process all data and return a response. As mentioned earlier, this response is produced by an *LLM*. LLMs are a specific approach to "narrow" AI that focus on using deep learning to find and reproduce

patterns in unstructured data; specifically, text. Text in this context includes natural language, metadata, code, and any other semantically meaningful arrangement of characters.

Copilot currently uses a combination of GPT models, including the Generative Pre-trained Transformer (GPT) series of models from OpenAI.

> **Note**
>
> You can't choose or change the models Copilot uses, including using other foundation models or your own models. Copilot in Fabric uses various models. It's also not possible for you to alter or configure the Azure OpenAI Service to behave differently with Copilot in Fabric; this service is managed by Microsoft.
>
> The models used by Copilot in Fabric presently don't use any fine-tuning. The models instead rely on grounding data and meta-prompts to create more specific and useful outputs.

The models used by Copilot in Fabric presently don't use any fine-tuning. The models instead rely on grounding data and meta-prompts to create more specific and useful outputs.

Microsoft hosts the OpenAI models in Microsoft's Azure environment and the service doesn't interact with any public services by OpenAI (for example, ChatGPT or the public OpenAI APIs). Your data isn't used to train models and isn't available to other customers. For more information, see Azure OpenAI Service.

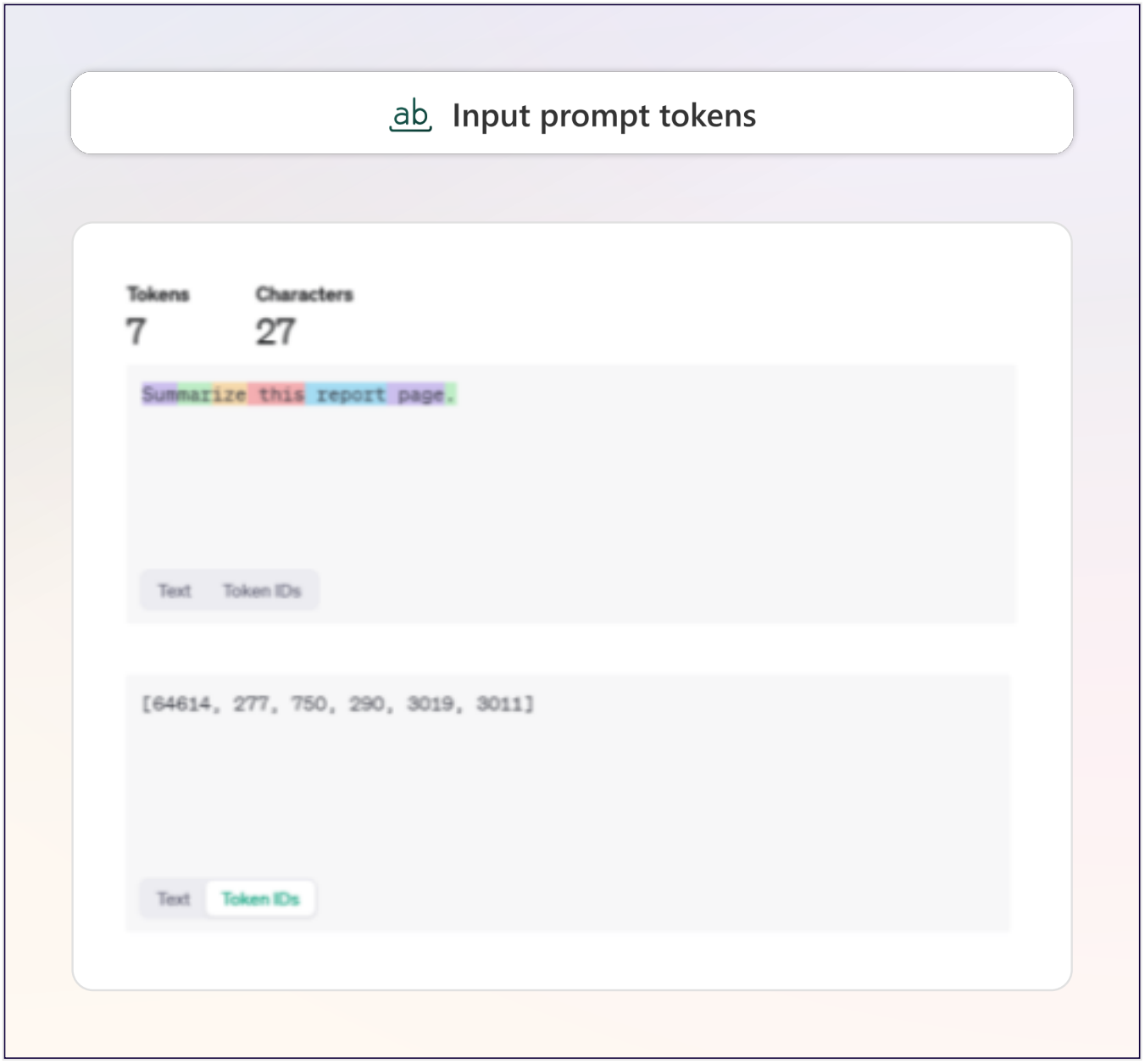
Understand tokenization

It's essential that you understand tokenization since the cost of Copilot in Fabric (which is how much Fabric capacity Copilot consumes) is determined by the number of tokens produced by your Copilot inputs and outputs.

To process the text input from Copilot, Azure OpenAI must first convert that input into a numerical representation. A key step in this process is *tokenization*, which is the partitioning of input text into different, smaller parts, called *tokens*. A token is a set of co-occurring characters, and it's the smallest unit of information that an LLM uses to produce its output. Each token has a corresponding numerical ID, which becomes the vocabulary of the LLM to encode and use text as numbers. There are different ways to tokenize text, and different LLMs tokenize input text in different ways. Azure OpenAI uses *Byte-Pair Encoding (BPE)*, which is a method of sub-word tokenization.

To better understand what a token is and how a prompt becomes tokens, consider the following example. This example shows an input prompt and its tokens, estimated using the

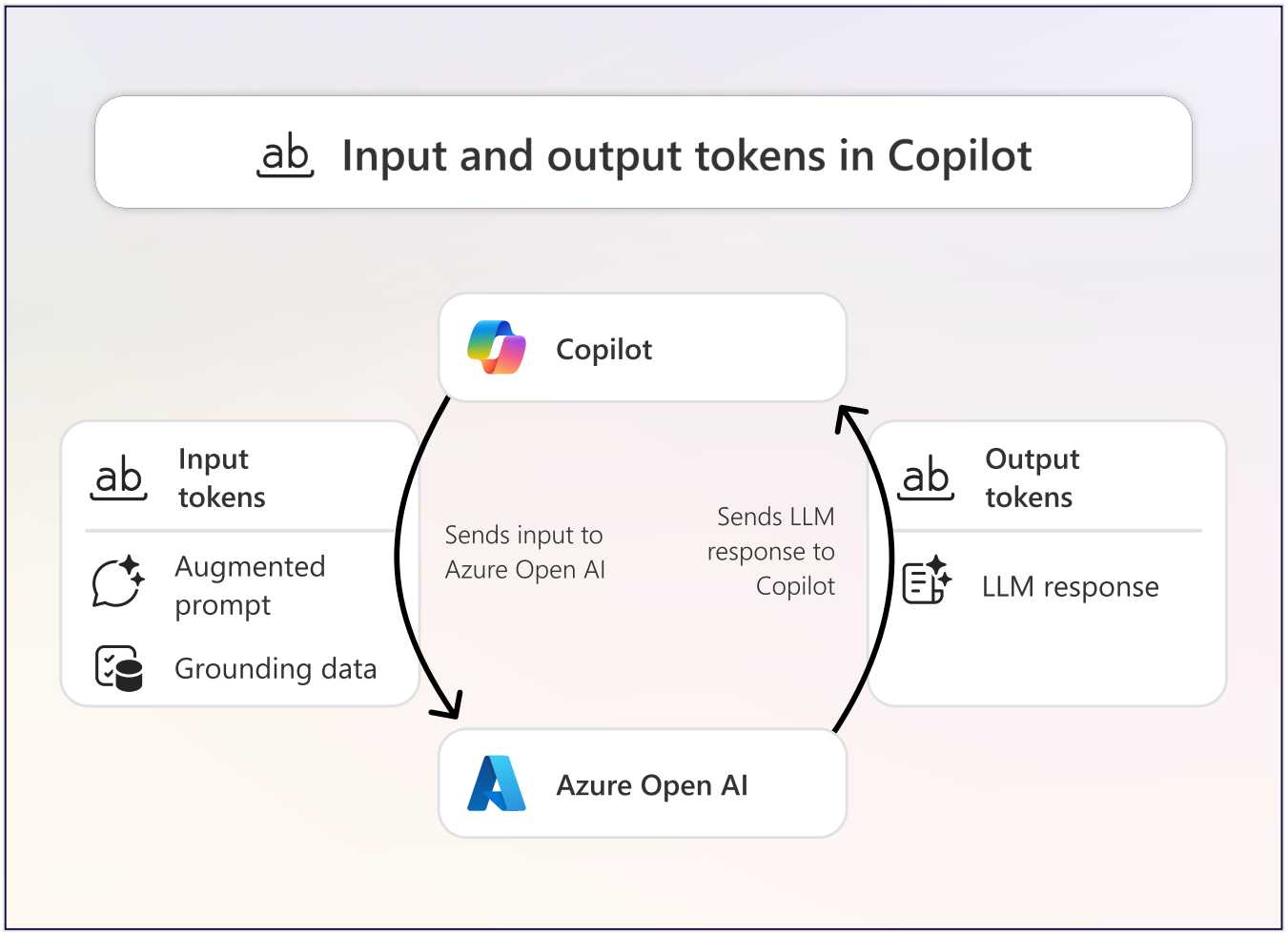
OpenAI Platform tokenizer (for GPT4). Beneath the highlighted tokens in the prompt text is an array (or list) of the numerical token IDs.



In the example, each differently colored highlight indicates a single token. As mentioned previously, Azure OpenAI uses *subword* tokenization, so a token isn't a word, but it also isn't a character, or a fixed number of characters. For instance "*report*" is a single token, but "." is, too.

To reiterate, you should understand what a token is because the cost of Copilot (or its Fabric capacity consumption rate) is determined by tokens. Therefore, understanding what a token is and how input and output tokens are created helps you understand and anticipate how Copilot usage results in consumption of Fabric CUs. For more information about the cost of Copilot in Fabric, see the appropriate section later in this article.

Copilot in Fabric uses both input and output tokens, as depicted in the following diagram.



Copilot creates two different kinds of tokens:

- *Input tokens* result from tokenizing both the final prompt and any grounding data.
- *Output tokens* result from tokenizing the LLM response.

Some Copilot experiences result in multiple LLM calls. For instance, when asking a data question about models and reports, the first LLM response might be a query evaluated against a semantic model. Copilot then sends the result of that evaluated query to Azure OpenAI again and requests a summary, which Azure OpenAI returns with another response. These additional LLM calls might be handled and the LLM responses combined during the postprocessing step.

> **Note**
>
> With Copilot in Fabric, except for changes to a written input prompt, you can only optimize input and output tokens by adjusting the configuration of the relevant items, like hiding columns in a semantic model or reducing the number of visuals or pages in a report. You can't intercept or modify the grounding data before it's sent to Azure OpenAI by Copilot.

Understand processing

It's important that you understand how an LLM in Azure OpenAI processes your data and produces an output, so that you can better understand why you get certain outputs from Copilot and why you should critically appraise them before further use or decision-making.

> **Note**
>
> This article provides a simple, high-level overview of how the LLMs that Copilot uses (like GPTs) work. For technical details and a deeper understanding of how GPT models process input to produce a response, or about their architecture, read the research papers ***Attention Is All You Need*** (2017) by Ashish Vaswani and others, and ***Language Models are Few-Shot Learners*** (2020) by Tom Brown and others.

The purpose of Copilot (and LLMs in general) is to provide a context-appropriate, useful output, based on the input that a user provides and other relevant grounding data. An LLM does this by interpreting the meaning of tokens in a similar context, as seen in their training data. To get a meaningful semantic understanding of tokens, LLMs have been trained on massive datasets thought to comprise both copyrighted and public domain information. However, this training data is limited in terms of content freshness, quality, and scope, which creates limitations for LLMs and the tools that use them, such as Copilot. For more information about these limitations, see Understand the limitations of Copilot and LLMs later in this article.

The semantic meaning of a token is captured in a mathematical construct referred to as an *embedding*, which turns tokens into dense vectors of real numbers. In simpler terms, embeddings provide LLMs with the semantic meaning of a given token, based on the other tokens around it. This meaning depends on the LLM training data. Think of tokens like unique building blocks, while embeddings help an LLM know what block to use when.

Using tokens and embeddings, the LLM in Azure OpenAI processes your input and generates a response. This processing is a computationally intensive task that requires significant resources, which is where the cost comes from. An LLM produces its response one token at a time, where it selects each token using a computed probability based on the input context. Each generated token is also added to that existing context before producing the next token. The final response of the LLM must therefore always be text, which Copilot might later postprocess to make a more useful output for the user.

It's important to understand several key aspects about this generated response:

- It's non-deterministic; the same input can produce a different response.
- It can be interpreted as low-quality or incorrect by the user in their context.
- It's based on the LLM training data, which is finite and limited in its scope.

Understand the limitations of Copilot and LLMs

It's important to understand and acknowledge the limitations of Copilot and the underlying technology that it uses. Understanding these limitations helps you to get value from Copilot while also mitigating risks inherent to using it. To use Copilot in Fabric effectively, you should understand the use-cases and scenarios that best fit this technology.

It's important to keep the following considerations in mind when you use Copilot in Fabric:

- **Copilot in Fabric is non-deterministic.** Except for when a prompt and its output are cached, the same input can produce different outputs. When you accept a range of possible outputs—like a report page, a code pattern, or a summary—this is less of a problem, because you can tolerate and might even expect variety in the response. However, for scenarios when you expect only one correct answer, you might want to consider an alternative approach to Copilot.

- **Copilot in Fabric can produce low quality or inaccurate outputs:** Like all LLM tools, it's possible for Copilot to produce outputs that might not be correct, expected, or suitable for your scenario. This means that you should avoid using Copilot in Fabric with sensitive data or in high-risk areas. For example, you shouldn't use Copilot outputs to answer data questions about business-critical processes, or to create data solutions that might affect the personal or collective well-being of individuals. Users should check and validate Copilot outputs before they use them.

- **Copilot has no understanding of "accuracy" or "truthfulness":** The outputs that Copilot provides don't provide an indication of trustworthiness, reliability, or similar sentiments. The underlying technology involves pattern recognition and is unable to evaluate the quality or usefulness of its outputs. Users should critically evaluate outputs before they use these outputs in other work or decision-making.

- **Copilot can't reason, understand your intent, or know context beyond its input:** While the grounding process of Copilot ensures that outputs are more specific, grounding alone can't give Copilot all the information that it needs to answer your questions. For instance, if you use Copilot to generate code, Copilot still doesn't know what you'll do with that code. This means that the code might work in one context, but not another, and users must either modify the output or their prompt to address this.

- **Copilot outputs are limited by the training data of the LLMs it uses:** In certain Copilot experiences, such as those where you generate code, you might want Copilot to generate code with a newly released function or pattern. However, Copilot won't be able to do this effectively if there are no examples of that in the training data of the GPT models it uses, which has a cutoff in the past. This also happens when you try to apply Copilot to contexts that are sparse in its training data, like when using Copilot with the TMDL editor

in Power BI Desktop. In these scenarios, you should be particularly vigilant and critical of low-quality or inaccurate outputs.

> **⚠ Warning**
>
> To mitigate the risks of these limitations and considerations, and the fact that Copilot, LLMs, and generative AI are nascent technology, you *should not* use Copilot in Fabric for autonomous, high-risk, or business-critical processes and decision-making.

For more information, see Security guidance for LLMs.

Once the Azure OpenAI Service processes the input and produces a response, it returns this response as an output to Copilot.

Step 4: Copilot performs postprocessing on the output

Upon receiving the response from Azure OpenAI, Copilot performs additional postprocessing to ensure that the response is appropriate. The purpose of postprocessing is to filter out inappropriate content.

To perform postprocessing, Copilot might perform the following tasks:

- **Responsible AI checks:** Ensuring Copilot complies with the responsible AI standards at Microsoft. For more information, see What should I know to use Copilot responsibly?
- **Filtering with Azure content moderation:** Filtering responses to ensure that Copilot only returns responses appropriate to the scenario and experience. Here are some examples of how Copilot performs filtering with Azure content moderation:
  - *Unintended or improper use:* Content moderation ensures that you can't use Copilot in unintended or improper ways, such as asking questions about other topics outside the scope of the workload, item, or experience you're using.
  - *Inappropriate or offensive outputs:* Copilot prevents outputs that could contain unacceptable language, terms, or phrases.
  - *Attempts of prompt injection:* Copilot prevents prompt injection, where users attempt to hide disruptive instructions in grounding data, like in object names, descriptions, or code comments in a semantic model.
- **Scenario-specific constraints:** Depending on which Copilot experience you use, there might be additional checks and handling of the LLM response before you receive the output. Here are some examples of how Copilot enforces scenario-specific constraints:
  - *Code parsers:* Generated code might be put through a parser to filter out low-quality responses and errors to ensure that the code runs. This happens when you generate

DAX queries by using Copilot in the DAX query view of Power BI Desktop.
    - *Validation of visuals and reports*: Copilot checks that visuals and reports can render before returning them in an output. Copilot doesn't validate whether the results are accurate or useful, or whether the resulting query will time out (and produce an error).

- **Handling and using the response:** Taking the response and adding additional information or using it in other processes to provide the output to the user. Here are some examples of how Copilot might handle and use a response during postprocessing:
    - *Power BI report page creation:* Copilot combines the LLM response (report visual metadata) with other report metadata, which results in creating a new report page. Copilot might also apply a *Copilot theme* if you haven't created any visuals in the report yet. The theme isn't part of the LLM response, and it includes a background image, as well as colors and visual styles. If you've created visuals, then Copilot doesn't apply the Copilot theme and uses the theme you already have applied. When changing a report page, Copilot will also delete the existing page and replace it with a new one with the adjustments applied.
    - *Power BI data questions:* Copilot evaluates a query against a semantic model.
    - *Data factory dataflow gen2 transformation step suggestion:* Copilot modifies the item metadata to insert the new step, adjusting the query.

- **Additional LLM calls:** In certain scenarios, Copilot might perform additional LLM calls to enrich the output. For instance, Copilot might submit the result of an evaluated query to the LLM as a new input and request an explanation. This natural language explanation is then packaged together with the query result in the output that a user sees in the Copilot chat panel.

If content is filtered out in the output, then Copilot will either re-submit a new, modified prompt, or return a standard response.

- **Resubmit a new prompt:** When a response doesn't meet scenario-specific constraints, Copilot will produce another modified prompt to try again. In some circumstances, Copilot might suggest several new prompts for the user to select before submitting the prompt to generate a new output.

- **Standard response:** The standard response in this case would indicate a generic error. Depending on the scenario, Copilot might provide additional information to guide the user to produce another input.

> ⓘ **Note**
>
> It isn't possible to view the original, filtered responses from Azure OpenAI, or to alter the standard responses from or behavior of Copilot. This is managed by Microsoft.

After postprocessing is complete, Copilot will then return an output to the user.

Step 5: Copilot returns the output to the user

The output for a user can take the form of natural language, code, or metadata. This metadata will typically be rendered in the UI of Fabric or Power BI Desktop, such as when Copilot returns a Power BI visual or suggests a report page. For some Copilot in Power BI experiences, the user can provide both inputs and outputs to Copilot via the Power BI mobile app.

In general, outputs can either allow user intervention or be fully autonomous and not allow the user to alter the result.

- **User intervention:** These outputs allow a user to modify the result before it's evaluated or displayed. Some examples of outputs that allow user intervention include:
  - Generation of code like DAX or SQL queries, which a user can choose to keep or run.
  - Generation of measure descriptions in a semantic model, which a user can choose to keep, modify, or delete.

- **Autonomous:** These outputs can't be altered by the user. The code might be evaluated directly against a Fabric item, or the text isn't editable in the pane. Some examples of autonomous outputs include:
  - Answers to data questions about a semantic model or report in the Copilot chat panel, which automatically evaluate queries against the model and show the result.
  - Summaries or explanations of code, items, or data, which automatically choose what to summarize and explain, and show the result.
  - Creation of a report page, which automatically creates the page and visuals in the report.

Sometimes, as part of the output, Copilot might also suggest an additional, follow-up prompt, such as requesting clarification, or another suggestion. This is typically useful when the user wants to improve the result or continue working on a specific output, like explaining a concept to understand generated code.

Outputs from Copilot might contain low-quality or inaccurate content

Copilot has no way to evaluate or indicate the usefulness or accuracy of its outputs. As such, it's important that users evaluate this themselves whenever they use Copilot.

To mitigate risks or challenges from LLM hallucinations in Copilot, consider the following advice:

- Train users to use Copilot and other similar tools that leverage LLMs. Consider training them on the following topics:

    - What Copilot can and can't do.
    - When to use Copilot and when not to use it.
    - How to write better prompts.
    - How to troubleshoot unexpected results.
    - How to validate outputs by using trusted online sources, techniques, or resources.

- Test items with Copilot before you allow these items to be used with it. Certain items require certain preparatory tasks to ensure that they work well with Copilot.

- Avoid using Copilot in autonomous, high-risk, or business-critical decision-making processes.

> **Important**
>
> Additionally, review the **supplemental preview terms for Fabric**, which include terms of use for Microsoft Generative AI Service Previews. While you can try and experiment with these preview features, we recommend that you don't use Copilot features in preview in production solutions.

Privacy, security, and responsible AI

Microsoft is committed to ensuring that our AI systems are guided by our AI principles and Responsible AI Standard. See Privacy, security, and responsible use of Copilot in Fabric for a detailed overview. See also Data, privacy, and security for Azure OpenAI Service for detailed information specific for Azure OpenAI.

For an overview specifically for each Fabric workload, see the following articles:

- Responsible use in Data Factory
- Responsible use in Data Science and Data Engineering
- Responsible use in Data Warehousing
- Responsible use in Power BI
- Responsible use in Real-Time Intelligence

Cost of Copilot in Fabric

Unlike other Microsoft Copilots, Copilot in Fabric doesn't require additional per-user or per-capacity licenses. Rather, Copilot in Fabric consumes from your available Fabric capacity units (CUs). The consumption rate of Copilot is determined by the number of tokens in your inputs and outputs when you use it across the various experiences in Fabric.

If you have a Fabric capacity, you're using either a pay-as-you-go or reserved instance. In both cases, Copilot consumption works the same. In a pay-as-you-go scenario, you're billed per second that your capacity is active until you pause your capacity. Billing rates have no relationship to the usage of your Fabric CUs; you pay the same amount if your capacity is fully utilized or completely unused. As such, Copilot doesn't have a direct cost or impact on your Azure billing. Rather, Copilot consumes from the available CUs which other Fabric workloads and items also use, and if you use too much, users will experience reduced performance and throttling. It's also possible to enter a state of CU debt called *carryforward*. For more information about throttling and carryforward, see Throttle triggers and throttle stages.

The following sections explain more about how you should understand and manage Copilot consumption in Fabric.

> **Note**
>
> For more information, see **Copilot in Fabric consumption**.

Copilot consumption in Fabric is determined by tokens

Copilot consumes your available Fabric CUs, also commonly referred to as *capacity*, *compute*, or *resources*. The consumption is determined by the input and output tokens when you use it. To review, you can understand input and output tokens as a result of tokenizing the following:

- *Input tokens:* Tokenization of your written prompt and grounding data.
- *Output tokens:* Tokenization of the Azure OpenAI response, based on the input. Output tokens are three times more expensive than input tokens.

You can limit the number of input tokens by using shorter prompts, but you can't control what grounding data Copilot uses for preprocessing, or the number of output tokens that the LLM in Azure OpenAI returns. For instance, you can expect that the report creation experience for Copilot in Power BI will have a high consumption rate, since it might use grounding data (like your model schema) and might produce a verbose output (report metadata).

Inputs, outputs, and grounding data are converted to tokens

To reiterate from an earlier section in this article, it's important to understand the tokenization process so that you know what kinds of inputs and outputs produce the highest consumption.

Optimizing prompt tokens isn't likely to have a significant effect on your Copilot costs. For instance, the number of tokens in a written user prompt is typically much smaller than the tokens of grounding data and outputs. Copilot handles the grounding data and outputs

autonomously; you can't optimize or influence these tokens. For instance, when using Copilot in Power BI, Copilot might use the schema from of your semantic model or metadata from your report as grounding data during preprocessing. This metadata likely comprises many more tokens than your initial prompt.

Copilot performs various system optimizations to reduce input and output tokens. These optimizations depend on the Copilot experience that you're using. Examples of system optimizations include:

- **Schema reduction:** Copilot doesn't send the entire schema of a semantic model or lakehouse table. Instead, it uses embeddings to determine which columns to send.
- **Prompt augmentation:** When rewriting the prompt during preprocessing, Copilot tries to produce a final prompt that will return a more specific result.

Additionally, there are various user optimizations that you can implement to limit what grounding data Copilot can see and use. These user optimizations depend on the item and experience that you're using. Some examples of user optimizations include:

- **Hiding fields or marking tables as *private* in a semantic model:** Any hidden or private objects won't be considered by Copilot.
- **Hiding report pages or visuals:** Similarly, any hidden report pages or visuals hidden behind a report bookmark are also not considered by Copilot.

> **Tip**
>
> User optimizations are mainly effective for improving the usefulness of Copilot outputs, rather than optimizing Copilot cost. For more information, see articles specific to the various workloads and Copilot experiences.

You have no visibility on the tokenization process, and you can only minimally impact the input and output tokens. As such, the most effective way for you to manage Copilot consumption and avoid throttling is by managing Copilot usage.

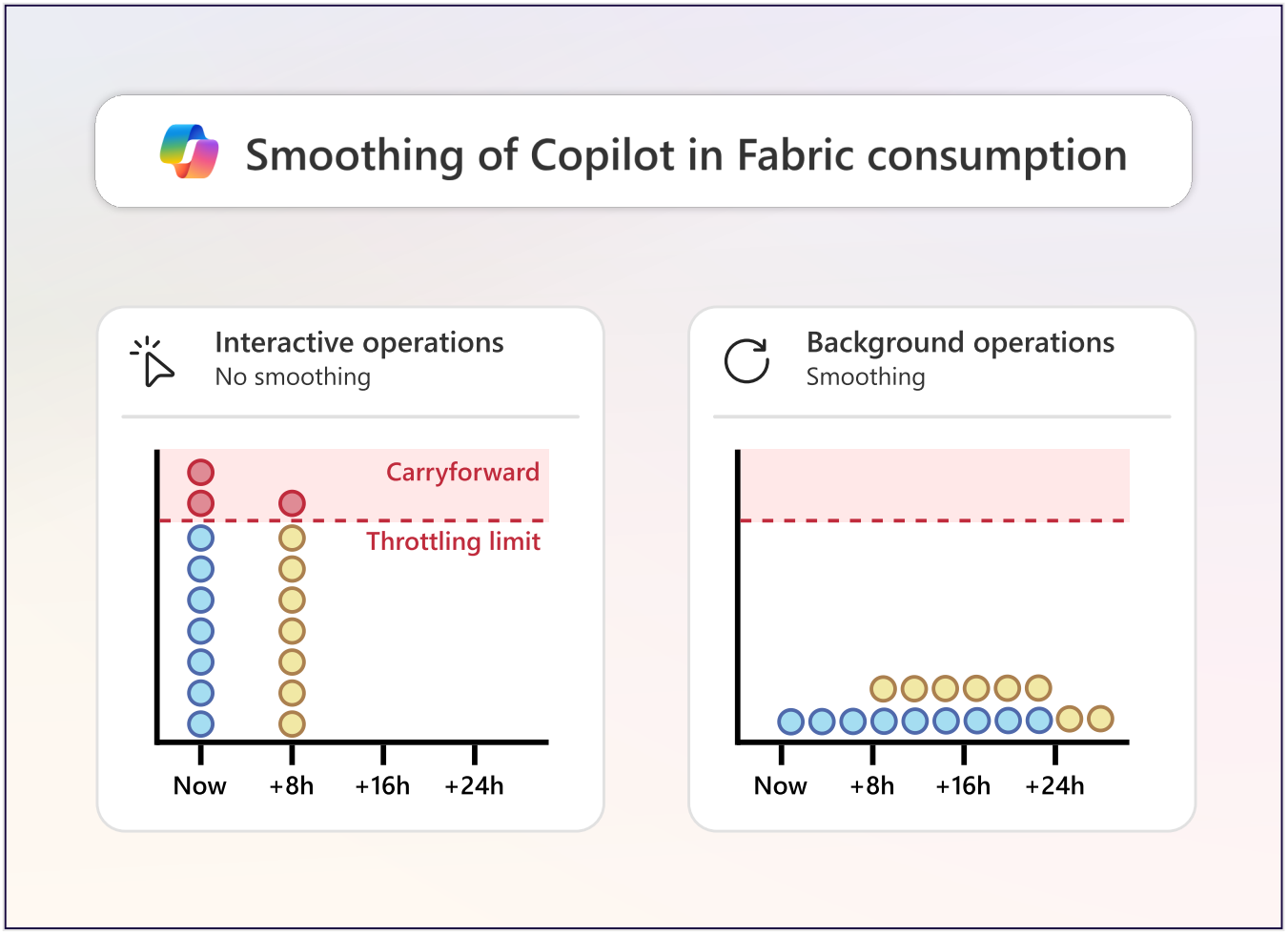
Copilot is a background operation that's smoothed

Concurrent usage of Copilot in Fabric—when many individuals are using it at the same time—is handled by a process called *smoothing*. In Fabric, any operation classified as a *background operation* has its CU consumption divided over a 24-hour window, starting from the time of the operation to exactly 24 hours later. This is in contrast to *interactive operations*, like semantic model queries from individuals using a Power BI report, which aren't smoothed.

> **Note**
>
> To simplify your understanding, background and interactive operations classify different things that happen in Fabric for billing purposes. They don't necessarily relate to whether an item or feature is *interactive for a user* or *happening in the background*, as their names might suggest.

For example, if you use 48 CUs with a background operation now, it results in 2 CUs of consumption now, and also 2 CUs each hour from now until 24 hours from now. If you use 48 CUs with an interactive operation, it results in an observed 48 CUs used *now,* and has no effect on future consumption. However, smoothing also means that you can potentially accumulate CU consumption *in that window* if your usage of Copilot or other Fabric workloads is sufficiently high.

To better understand smoothing and its effects on your Fabric CU consumption, consider the following diagram:



The diagram depicts an example of a scenario with high concurrent usage of an interactive operation (which isn't smoothed). An interactive operation crosses the throttling limit (the available Fabric capacity) and enters carryforward. This is the scenario without smoothing. In contrast, background operations like Copilot have the consumption spread over 24 hours.

Subsequent operations within that 24-hour window will "stack" and contribute to total cumulative consumption in that period. In the smoothed scenario of this example, the background operations like Copilot would contribute to future CU consumption, but don't trigger throttling or cross any limits.

Monitor Copilot consumption in Fabric

Fabric administrators can monitor how much Copilot consumption is occurring in your Fabric capacity by using the Microsoft Fabric Capacity Metrics app. In the app, a Fabric administrator can view a breakdown by activity and user, helping them to identify the individuals and areas where they might need to focus during periods of high consumption.

> **Tip**
>
> Rather than considering abstract calculations like tokens to CUs, we recommend that you focus on the *percentage of your Fabric capacity* that you've utilized. This metric is the simplest to understand and act upon, because once you reach 100% utilization, you can experience **throttling**.
>
> You can find this information on the **timepoint page of the app**.

> **Note**
>
> When you pause a capacity, the smoothed usage is compacted into the timepoint in which the capacity becomes paused. This compaction of the smoothed consumption results in a peak of observed consumption, which doesn't reflect your actual usage. This peak will often produce notifications and warnings that you've exhausted your available Fabric capacity, but these are false positives.

Alleviate high utilization and throttling

Copilot consumes Fabric CUs, and even with smoothing, you might encounter situations of high utilization, which leads to high consumption, and throttling of your other Fabric workloads. The following sections discuss some strategies you can use to alleviate the impact on your Fabric capacity in this scenario.

User training and allowlisting

An important way to ensure effective adoption of any tool is to equip users with sufficient mentoring and training, and to gradually roll out access as people complete such training.

Effective training is a preventative measure to avoid high utilization and throttling preemptively, by educating users about how to use Copilot effectively and on what not to do.

You can best control who can use Copilot in Fabric by creating an *allowlist* of users with access to the feature from the Fabric tenant settings. This means that you enable Copilot in Fabric only for users who belong to specific security groups. If necessary, you might create separate security groups for each of the Fabric workloads where you can enable Copilot to obtain finer grain control over who can use which Copilot experiences. For more information about creating security groups, see Create, edit, or delete a security group.

Once you add specific security groups to the Copilot tenant settings, you can put together onboarding training for users. A Copilot training course should cover basic topics, such as the following.

> **Tip**
>
> Consider creating an overview training for basic concepts about LLMs and generative AI, but then create workload-specific training for users. Not every individual needs to learn about every Fabric workload if it isn't necessarily relevant to them.

- **LLMs:** Explain the basics of what an LLM is and how it works. You shouldn't go into technical details, but you should explain concepts like prompts, grounding, and tokens. You can also explain how LLMs can get meaning from input and produce context-appropriate responses because of their training data. Teaching this to users helps them understand how the technology works and what it can and can't do.

- **What Copilot and other generative AI tools are used for:** You should explain that Copilot isn't an autonomous agent and isn't intended to replace humans in their tasks, but meant to augment individuals to potentially perform their current tasks better and faster. You should also emphasize cases where Copilot isn't suitable, using specific examples, and explain what other tools and information individuals might use to address their problems in those scenarios.

- **How to critically appraise Copilot outputs:** It's important that you guide users about how they can validate Copilot outputs. This validation depends on the Copilot experience they're using, but in general, you should emphasize the following points:
  - Check each output before you use it.
  - Evaluate and ask yourself if the output is correct or not.
  - Add comments to generated code to understand how it works. Alternatively, ask Copilot for explanations for that code, if necessary, and cross-reference that explanation with trusted sources.

    - If the output produces an unexpected result, troubleshoot with different prompts or by performing manual validation.

- **Risks and limitations of LLMs and generative AI:** You should explain key risks and limitations of Copilot, LLMs, and generative AI, such as those mentioned in this article:
    - They're non-deterministic.
    - They provide no indication or guarantees of accuracy, reliability, or truthfulness.
    - They can hallucinate and produce inaccurate or low-quality outputs.
    - They can't generate information that spans outside the scope of their training data.

- **Where to find Copilot in Fabric:** Provide A a high-level overview of the different workloads, items, and Copilot experiences that someone might use.

Scale your capacity

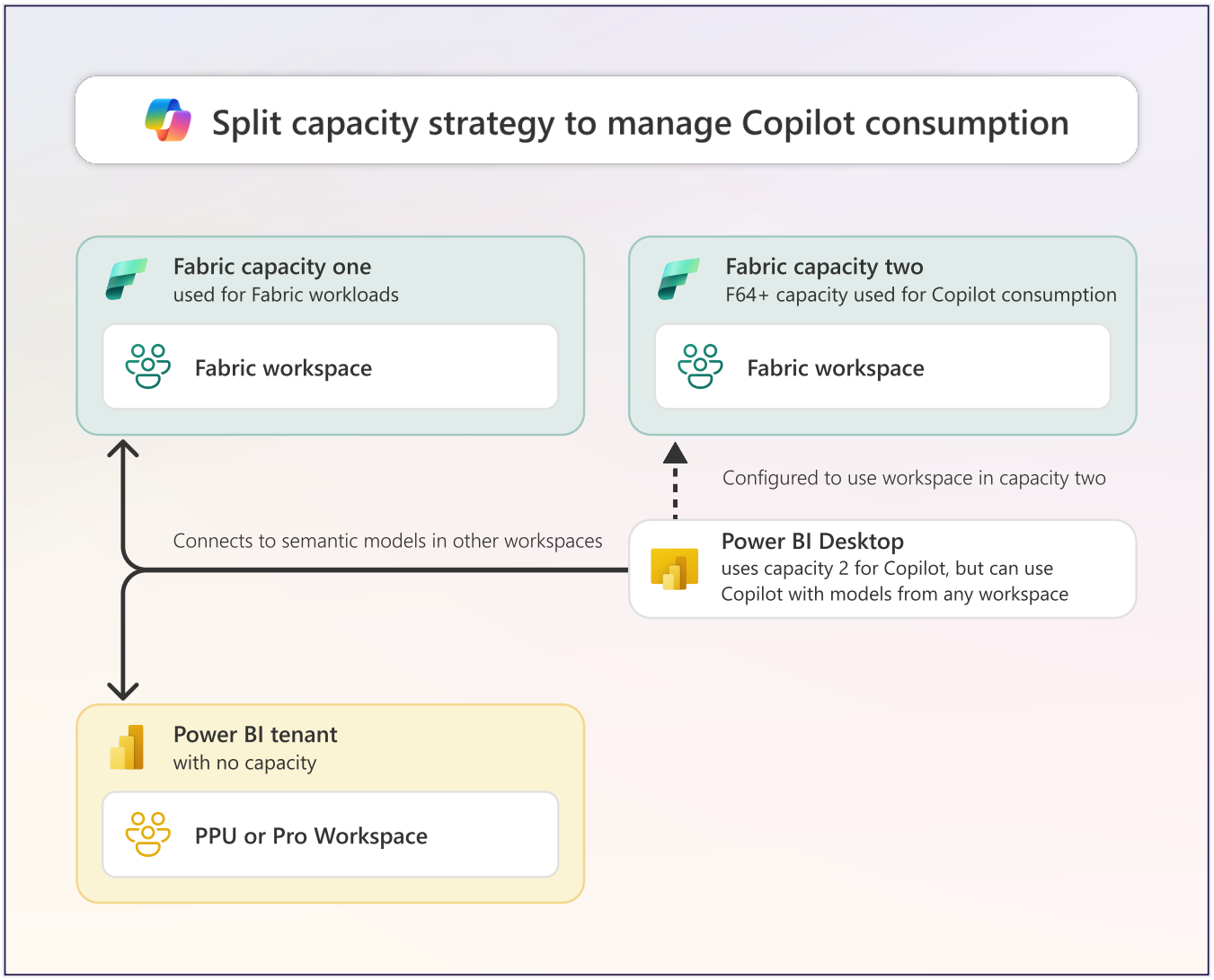
When you experience throttling in Fabric due to Copilot consumption or other operations, you can temporarily scale (or resize) your capacity to a higher SKU. This is a reactive measure that temporarily elevates your cost to alleviate short-term issues due to throttling or carryforward. This is particularly helpful when you experience throttling primarily due to background operations, since the consumption (and thus the impact) might be spread over a 24-hour window.

Split-capacity strategies

In scenarios where you expect a high usage of Copilot in Fabric (such as in large organizations), you might consider isolating Copilot consumption from your other Fabric workloads. In this split-capacity scenario, you prevent Copilot consumption from negatively impacting other Fabric workloads by enabling Copilot only on a separate F64 or higher SKU, which you only use for dedicated Copilot experiences. This split-capacity strategy produces higher cost, but it might make it easier to manage and govern Copilot usage.

> **Tip**
>
> You can use some Copilot experiences with items in other capacities that don't support or enable Copilot. For example, in Power BI Desktop, you can link to a workspace with an F64 SKU Fabric capacity, but then connect to a semantic model in an F2 or PPU workspace. Then, you can use Copilot experiences in Power BI Desktop, and the Copilot consumption will only affect the F64 SKU.

The following diagram depicts an example of a split-capacity strategy to isolate Copilot consumption with experiences like those in Copilot in Power BI Desktop.



You can also use a split-capacity solution by assigning Copilot consumption to a separate capacity. Assigning Copilot consumption to a separate capacity ensures that high utilization of Copilot doesn't impact your other Fabric workloads and the business-critical processes that depend on them. Of course, using any split-capacity strategy requires that you already have two or more F64 or higher SKUs. As such, this strategy might not be manageable for smaller organizations or organizations with a limited budget to spend on their data platforms.

Irrespective of how you choose to manage Copilot, what's most important is that you monitor Copilot consumption in your Fabric capacity.

Related content

- Overview of Copilot in Fabric
- Copilot for Microsoft Fabric and Power BI: FAQ
- Copilot tenant settings

Copilot in Fabric glossary and terms

Article • 04/17/2025

This article provides an overview of selected key terms and their definitions in the context of Copilot in Fabric. Refer to these terms when you read articles on Copilot in Fabric, or when you use Copilot and need a reminder about what a term or name means.

Expand table

Term	Definition (in context of Copilot in Fabric)
Azure AI Search (*formerly Azure Cognitive Search*)	A search and retrieval system with a comprehensive set of advanced search technologies, built for high-performance applications at any scale. Azure AI Search is the primary recommended retrieval system when building RAG-based applications on Azure, with native LLM integrations between Azure OpenAI Service and Azure Machine Learning. , You use Azure AI Search to create your own copilots.
Azure OpenAI Service (Azure OpenAI)	Provides REST API access to OpenAI's language models. Azure OpenAI Service is a Microsoft-managed service which doesn't use the public OpenAI services or resources. Copilot sends the preprocessed input to Azure OpenAI so that the input can be processed. After processing, Azure OpenAI returns an LLM response to Copilot for postprocessing. You can't view, access, or modify Azure OpenAI Services used for Copilot in Fabric.
Capacity units (CUs)	How you measure capacity usage by using the Microsoft Fabric Capacity Metrics app. The number of CUs you have depends on your Fabric capacity SKU; higher SKUs have a higher number of available CUs. When you use 100% of your available CUs, you can enter a state of throttling, which might result in degraded performance and errors. Copilot in Fabric consumes from your available Fabric CUs.
Capacity usage	The compute impact of all your capacity operations. Copilot in Fabric is a background operation that results in capacity usage.
Consumption (of Fabric CUs)	The use of Fabric CUs. Synonymous with capacity usage. Using Copilot in Fabric results in consumption of your Fabric CUs.
Copilot	A generative AI assistant that aims to enhance the data analytics experience in the Fabric platform. There are different Copilots in each workload, and different Copilot experiences depending on what item and UI you're working with.
Embedding (in the context of LLMs)	The process of turning tokens into dense vectors of real numbers. Embeddings provide LLMs with the semantic meaning of a given token based on other tokens

Term	Definition (in context of Copilot in Fabric)
	around it in a sentence or paragraph.
Experience	A functional module for Copilot, such as the DAX query experience, which generates DAX code, and the report page generation experience, which generates visuals for a Power BI report page. When you use Copilot in Fabric, you use different Copilot experiences. Each workload and item can have multiple experiences.
Foundation model	The base models made available by vendors such OpenAI (for the GPT models) or Anthropic (for Claude models). Each model has its own training data and works differently, such as using different ways to tokenize and process input to produce a response. Copilot in Fabric uses the GPT foundation models from OpenAI, hosted by Microsoft in the Azure OpenAI Service. You can't change or fine-tune these models yourself.
Generative AI	A form of artificial intelligence in which models are trained to generate new original content based on natural language input. Copilot in Fabric is a tool that leverages generative AI technology, which attempts to enhance the data analytics experience in the Fabric platform.
Grounding	A preprocessing technique where additional contextual information is retrieved and used to enhance the specificity and usefulness of the LLM response. Grounding data is always specific to a user and respects the permissions to items and any enforced data security. Copilot in Fabric performs grounding during preprocessing of the input prompt, which might involve retrieving information from the current active Copilot session, item metadata, or specific data points.
Input	The prompt or interaction given to Copilot which starts the Copilot process. Copilot in Fabric uses different inputs; mainly, a written, natural language input from the user, or a generated natural language input from the user interacting with a button or similar UI element.
Large language model (LLM)	Deep learning models trained using large text corpora to generate text. LLMs are based on the idea of auto-regressive models, where they have been trained to predict the next word (or the most probable ones) given the previous ones. LLMs can be used to process large amounts of text and learn the structure and syntax of human language. Copilot in Fabric leverages the GPT series of LLMs from OpenAI, which are hosted in the Azure OpenAI Service.
Meta-prompt	A prompt that isn't provided by the user. Meta-prompts are provided by Copilot and configured by Microsoft. Each Copilot experience uses their own meta-prompts

Term	Definition (in context of Copilot in Fabric)
	to improve the specificity and usefulness of Copilot outputs. For instance, the DAX Query Copilot experience uses a meta-prompt that contains multiple examples of DAX queries and expressions.
Natural language	A naturally occurring or conversational language. Copilot in Fabric can receive user input from a natural language prompt.
Operation (of Fabric)	Activities that occur in Fabric and result in capacity usage. On-demand requests and operations that can be triggered by user interactions with the UI, such as data model queries generated by report visuals, are classified as *interactive* operations. Longer running operations such as semantic model or dataflow refreshes are classified as *background* operations. Copilot in Fabric is a background operation.
Orchestrator	A task during Copilot preprocessing that determines what skill or tool Copilot should use. The orchestrator determines this by using system information from a meta-prompt provided during the input.
Output	What Copilot returns to a user after postprocessing. Outputs can consist of low quality and inaccurate content, so users should critically evaluate each output before further use or decision-making. Copilot in Fabric can return different outputs depending on the experience that an individual is using.
Preprocessing	Activities by Copilot where the user input is taken and enhanced, or additional information is retrieved, in order to try to produce a more specific and useful output.
Postprocessing	Activities by Copilot where the LLM response is taken, filtered, and handled to produce the final output. Postprocessing activities vary greatly depending on the specific Copilot experience that an individual is using.
Prompt	An input for Copilot written in natural language by a user or generated by Copilot in response to user interaction.
Q&A (Power BI feature)	A feature in Power BI that lets you ask questions of your data using natural data and get a response. Q&A uses natural language processing, but not generative AI. Copilot in Fabric can be used to enhance the Q&A experience; for instance, it can generate synonyms for linguistic modeling.
Response (of an LLM)	The result of a processed input. An LLM response is always text, but that text can be either natural language, code, or metadata.
Responsible AI	A set of guiding principles and practices that, in theory, should mitigate risks and

Term	Definition (in context of Copilot in Fabric)
(RAI)	improve ethical use of AI if they're followed.
Retrieval augmented generation (RAG)	An architecture that augments the capabilities of an LLM by adding an information retrieval system that provides grounding data. Copilot in Fabric uses RAG during preprocessing.
Skill	A specific task within a Copilot experience. For instance, filtering a report page is a skill of the Copilot report page summary experience.
Stock-keeping unit (SKU)	The size of your Fabric capacity. A SKU is indicated by the letter *F* followed by a number, such as F2, F8, or F64. Larger numbers correspond to larger SKUs. Copilot in Fabric is only available for SKUs of F64 or higher.
Smoothing	A process in Fabric where capacity usage of a background operation is spread over a 24-hour window, starting from when the operation begins to exactly 24 hours later. Smoothing operations reduce the impact peak concurrent usage on your Fabric capacity. All Copilot in Fabric capacity usage is smoothed, because it's a background operation.
Token	The smallest unit of information that an LLM uses. A token comprises one or more characters of frequently co-occurring text. Each token has a corresponding unique integer for a given LLM, known as a *Token ID*. Tokens are necessary to convert natural language into a numerical representation, which is what an LLM uses to process input and return a response.
Tokenization	The process of converting natural language inputs to tokens. Tokenization is done by an LLM, and different LLMs tokenize inputs in different ways.
Workload	The different functional areas of Fabric, like Data Engineering, Data Science, or Power BI. Different workloads in Fabric use different Copilots. There are different Copilot experiences in each workload.

Related content

- What is Microsoft Fabric?
- Copilot in Fabric: FAQ

Enable and configure Copilot in Microsoft Fabric

Copilot in Microsoft Fabric is an AI assistant that helps you transform data, generate insights, and create visualizations across Fabric and Power BI. For details on Copilot functionality and responsible AI use, see Overview of Copilot in Fabric.

Although Copilot is enabled by default for tenants with paid Fabric capacities (F2 or higher), you need to verify and configure tenant settings, manage capacity-level options, and control which users and workspaces have access.

This article covers the tenant settings, delegated capacity configuration, and workspace assignments required to fully enable Copilot in your organization.

Prerequisites

- A paid Fabric capacity (F2 or higher) or a Power BI Premium capacity (P1 or higher). Pro and PPU workspaces don't directly support Copilot features. To use Copilot in these workspaces, enable a Fabric Copilot capacity and assign the workspace to that capacity.
- A Fabric capacity in a supported region. You can buy a Fabric capacity from the Azure portal.
- One or more security groups to scope which users can use Copilot. Security groups let you limit rollout to specific users, for example, after they complete prerequisite training. You can create and manage security groups from the Microsoft 365 admin center.

> **Note**
>
> You don't need to enable Fabric to use Copilot in Power BI. You can enable Copilot for the Power BI workload without enabling the other Fabric workloads — for instance, if you have a P SKU.

After you meet the prerequisites, complete the following steps to enable Copilot. Select a link in the *Step* column for detailed instructions.

Expand table

Step	Where	Description
1. Enable Copilot tenant settings	OneLake catalog Govern	Enable the Copilot tenant settings and scope them to specific security groups.
2. Configure delegated capacity settings	OneLake catalog Govern	If tenant settings are delegated, enable Copilot in the capacity-level settings.
3. Assign workspaces and provision access	Workspace settings	Assign a workspace to a Copilot-enabled capacity and grant access to users.
4. Enable Copilot in Power BI Desktop (optional)	Power BI Desktop	If you use Power BI Desktop, select a Copilot-enabled workspace as your active workspace.

Enable Copilot tenant settings

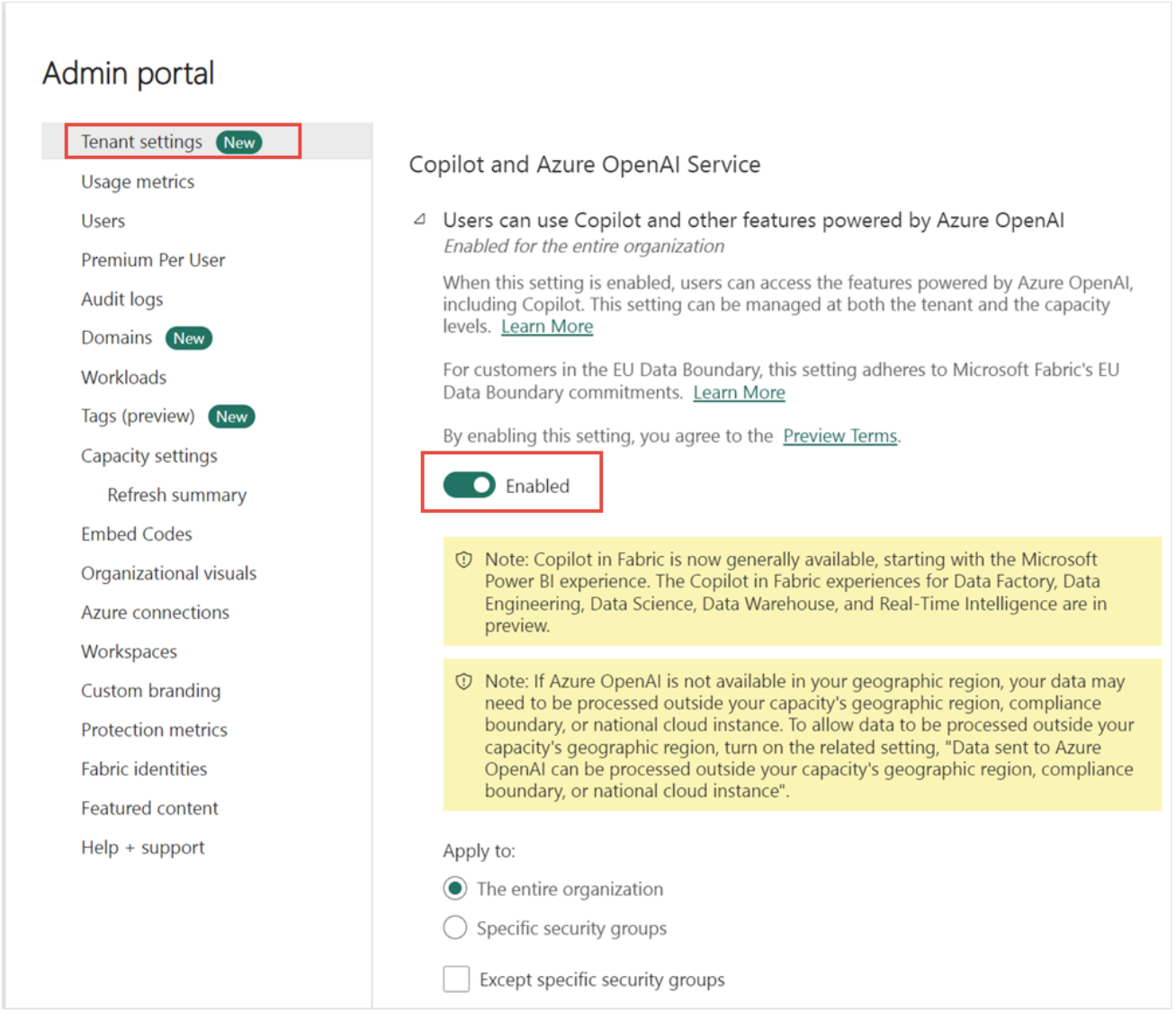
You can choose to enable Copilot for only select security groups. Copilot in Fabric is enabled by default for the entire tenant.

> ⚠ **Warning**
>
> Enabling Copilot for your entire tenant without proper planning can lead to higher capacity utilization and other potential risks. Consider enabling Copilot for specific security groups and workspaces only after you prepare.
>
> You can't enable only specific Copilot experiences, like using Copilot in the DAX query view of Power BI semantic models. You can only control whether Copilot is enabled at the workload level.

1. Sign in to the Fabric admin portal.
2. Select **Tenant settings**.
    - Enable the Users can use Copilot and other features powered by Azure OpenAI setting.
    - Enable the Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance setting.
    - Enable the Users can use Copilot, AI Agents, and other AI experiences powered by OpenAI as a Microsoft subprocessor setting.
    - Enable the Data sent to OpenAI as a Microsoft subprocessor can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance

setting.

3. Optionally, apply each setting to specific security groups rather than the entire organization.



For more information about these settings, see Copilot tenant settings and OpenAI as a subprocessor in Microsoft Fabric.

Configure delegated capacity settings

If the Copilot tenant settings are delegated to capacity administrators, each capacity administrator must also enable Copilot for their specific Fabric capacity.

1. In **OneLake catalog** > **Govern** > **Capacities**, open the Fabric capacity you plan to use with Copilot.

2. In the delegated tenant settings, enable the **Users can use Copilot and other features powered by Azure OpenAI** setting.

3. Enable the **Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance** setting.
4. Enable the **Users can use Copilot, AI Agents, and other AI experiences powered by OpenAI as a Microsoft subprocessor** setting.
5. Enable the **Data sent to OpenAI as a Microsoft subprocessor can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance** setting.
6. Optionally, apply each setting to specific security groups.

Assign workspaces and provision access

Fabric items that use Copilot must be in a workspace assigned to a supported capacity. Assign workspace roles to users who need Copilot.

1. Assign a workspace to a Fabric capacity that has Copilot enabled.
2. Grant workspace access to users who need Copilot. Items using Copilot must be in this workspace, but they might consume or refer to items in other workspaces or capacities.

Enable Copilot in Power BI Desktop

To use Copilot in Power BI Desktop, you must connect to a workspace that is assigned to a Copilot-enabled Fabric capacity.

1. Open Power BI Desktop.
2. Select a workspace that has Copilot enabled as your active workspace.

Copilot usage in Power BI Desktop counts against the Fabric capacity assigned to the selected workspace.

Related content

- What is Microsoft Fabric?
- Copilot in Fabric: FAQ
- OpenAI as a subprocessor in Microsoft Fabric
- Foundry Tools in Fabric (preview)
- Copilot tenant settings

- Copilot in Power BI
- Troubleshoot Copilot unavailability after a trial expires

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/29/2026

OpenAI as a subprocessor in Microsoft Fabric

> **Important**
>
> The information in this article only applies to OpenAI models operated by OpenAI (provided by OpenAI as a subprocessor). The information doesn't apply to OpenAI models operated by Microsoft (Azure OpenAI).

Microsoft Fabric is expanding options for how OpenAI models can be delivered within Copilot, AI Agents and other AI experiences. In addition to OpenAI models that Microsoft operates (Azure OpenAI), Fabric now provides OpenAI-operated models through OpenAI as a subprocessor. This option gives your organization the foundation for more model flexibility, including quicker access to newest AI model innovations, while maintaining enterprise-grade commitments and safeguards.

OpenAI is included in the Microsoft Online Services Subprocessors List and is available for use within Fabric. As a subprocessor, OpenAI operates with Microsoft oversight through contractual safeguards and appropriate technical and organizational measures. The Microsoft Product Terms and Microsoft Data Protection Addendum (DPA) apply to use of OpenAI models through Microsoft Fabric, except as otherwise disclosed in Current limitations.

For more information about subprocessor data access, see Microsoft Data Access. To see a list of Microsoft subprocessors, see the Service Trust Portal.

Considerations for using OpenAI as a subprocessor

- OpenAI models delivered through OpenAI as a subprocessor in Copilot experiences are currently excluded from country/region-specific processing commitments when applicable.
- Access to OpenAI-operated models isn't currently available for use in government clouds (GCC, GCC High, DoD) or sovereign clouds.
- OpenAI-operated models are included in the EU Data Boundary, except as otherwise noted in the EU Data Boundary documentation.
- Enabling this feature will allow data to egress outside of Fabric's compliance boundary to a third-party system or infrastructure that may have lower compliance controls (for example, may not comply with FedRAMP or Supply Chain Risk Management). The tenant

administrator is responsible for ensuring features meet their organization's security requirements and compliance obligations.

- When enabled, Fabric users may configure Fabric data agents to be consumed from other services such as Microsoft Foundry, Microsoft Copilot Studio, Microsoft 365 Copilot, or as an MCP server ("non-Fabric services"). When users connect to these non-Fabric services, responses returned by Fabric data agents may be sent outside of Fabric's compliance boundary or geographic region, and processed or stored according to the applicable terms and data handling policies of the non-Fabric services.

Manage OpenAI as a subprocessor in the Fabric Admin Portal

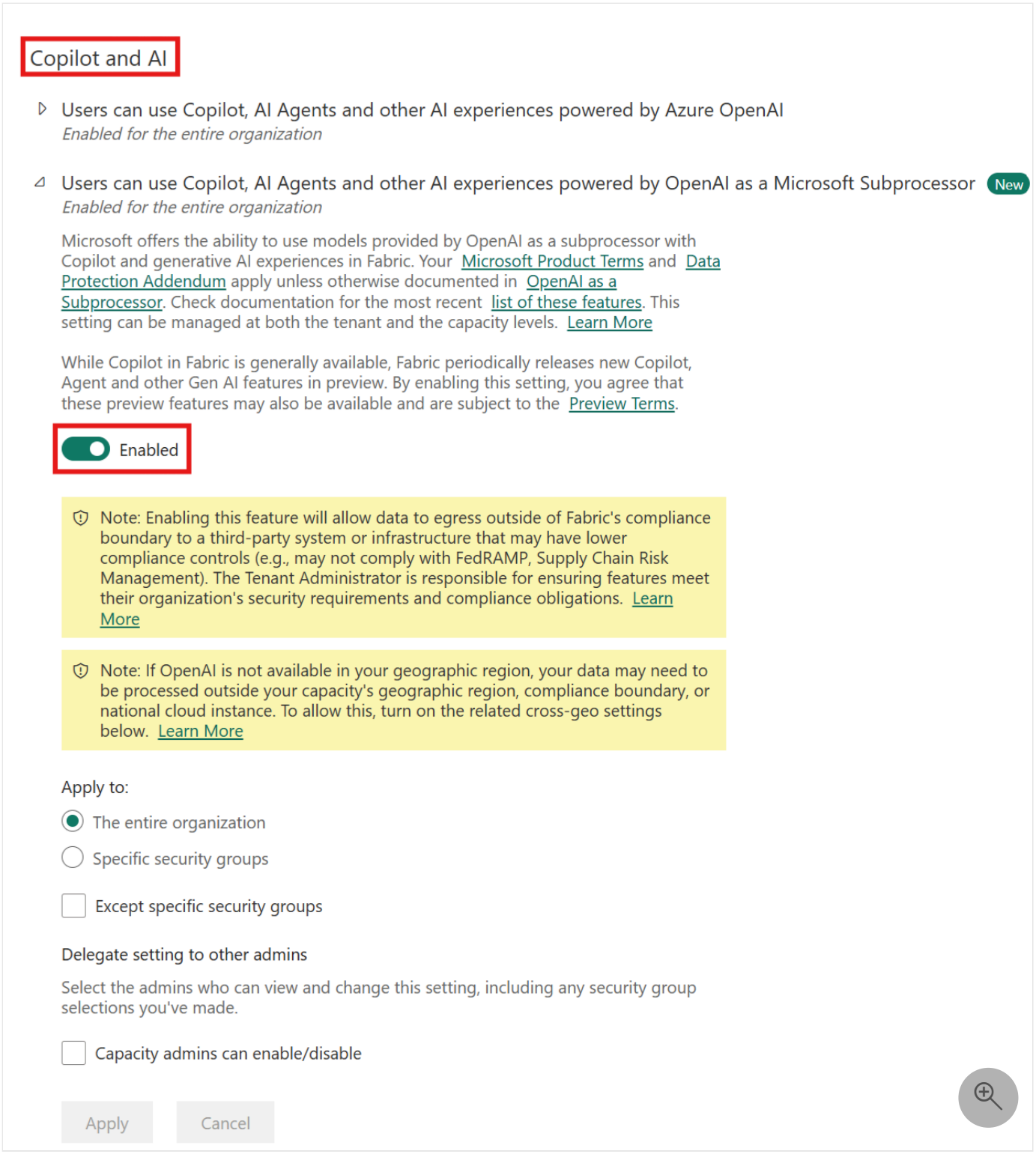
Enable the use of OpenAI-operated models

You can choose to enable OpenAI-operated models so that they're available for your organization. You must be a member of the Fabric Administrator or Global Administrator role to perform this task. For more information, see Understand Microsoft Fabric admin roles.

To enable the use of OpenAI-operated models:

1. Go to **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**.
2. Under the **Copilot and AI** section, locate **Users can use Copilot, AI Agents, and other AI experiences powered by OpenAI as a Microsoft Subprocessor**.
3. Toggle the setting to **Enabled**.
4. Under **Apply to**, select **The entire organization**, or specify security groups to include. Optionally, specify security groups to exclude.
5. Select **Apply**.

The following screenshot shows how to configure this setting:



You can restrict user access to AI provider subprocessors by assigning permissions to specific security groups in tenant settings. These assignments are applied at the tenant level and enforced across all Fabric Copilot and AI Agent experiences. When access is limited by security group membership, only members of the specified groups can use Copilot features that rely on that AI provider. For more information on tenant settings, see About tenant settings.

Disable the use of OpenAI-operated models

Some features or models may only be available when the use of OpenAI is enabled. If you disable OpenAI as a subprocessor, such features or models may no longer be accessible.

You can disable the use of OpenAI-operated models in the Fabric admin portal. You must be a member of the Fabric Administrator or Global Administrator role to perform this task. For more information, see Understand Microsoft Fabric admin roles.

To disable the use of OpenAI-operated models:

1. Go to **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**.
2. Under the **Copilot and AI** section, locate **Users can use Copilot, AI Agents, and other AI experiences powered by OpenAI as a Microsoft Subprocessor**.
3. Toggle the setting to **Disabled**.
4. Select **Apply**.

After you disable OpenAI as a subprocessor, users won't have the option to use OpenAI-operated AI models in Fabric Copilot experiences. You can choose to enable OpenAI-operated models at a later date if desired.

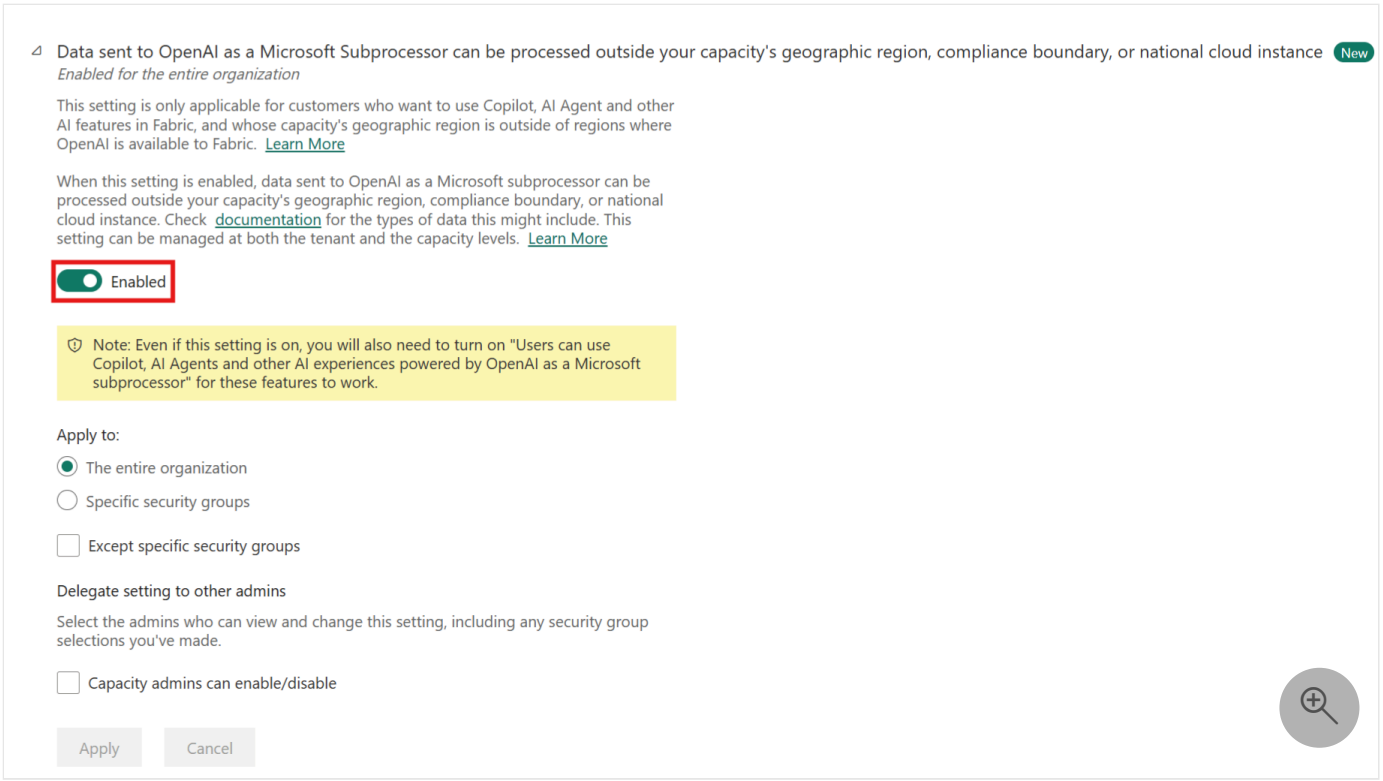
Data residency considerations for OpenAI

Data residency can be controlled by the setting **Data sent to OpenAI as a Microsoft subprocessor can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance**. This setting is only applicable for customers who want to use Copilot and AI features in Fabric powered by OpenAI, and whose capacity's geographic region is outside of the EU data boundary and the US. When this setting is enabled, service background jobs may execute across geographic boundaries at no additional charge to the tenant capacity to support end user experiences.

To configure this setting:

1. Go to **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**.
2. Under the **Copilot and AI** section, locate **Data sent to OpenAI as a Microsoft Subprocessor can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance**.
3. Toggle the setting to **Enabled** or **Disabled** based on your organization's data residency requirements.
4. Under **Apply to**, select **The entire organization** or specify security groups. Select **Apply**.

The following screenshot shows how to configure this setting:



Default: Disabled

Configure delegated capacity settings

In addition to tenant-level settings, Fabric administrators can delegate AI settings management to capacity administrators. When delegation is enabled, capacity admins can control whether Copilot and AI features — including OpenAI as a subprocessor — are available for workspaces assigned to their capacity.

Capacity-level settings work in conjunction with tenant-level settings:

- If OpenAI as a subprocessor is **disabled** at the tenant level, capacity administrators can't enable it for their capacity regardless of delegation settings.
- If OpenAI as a subprocessor is **enabled** at the tenant level and delegation is active, capacity administrators can choose to enable or disable OpenAI for their specific capacity.

To delegate the OpenAI subprocessor setting to capacity administrators:

1. Go to **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**.
2. Under the **Copilot and AI** section, locate **Users can use Copilot, AI Agents, and other AI experiences powered by OpenAI as a Microsoft Subprocessor**.
3. Enable **Allow capacity admins to override this setting**.
4. Select **Apply**.

For more information, see Copilot tenant settings.

Current limitations

The following exclusions apply:

- OpenAI-operated models available through Microsoft aren't FedRAMP High authorized. If your organization requires FedRAMP High prior to use, consult with your authorization official to determine whether use of OpenAI-operated models is permitted within your environment.
- A Payment Card Industry (PCI) Data Security Standard (DSS) Attestation of Compliance (AOC) isn't available for OpenAI-operated models.
- A Health Information Trust Alliance (HITRUST) Common Security Framework (CSF) Certification Letter isn't available for OpenAI-operated models.
- A System and Organization Controls (SOC) 1 Type 2 report isn't available for OpenAI-operated models.

Related content

- Privacy, security, and responsible use of Copilot in Fabric
- Copilot tenant settings
- Enable Copilot in Microsoft Fabric
- Microsoft Fabric data processing

ⓘ**Note:** The author created this article with assistance from AI. Learn more

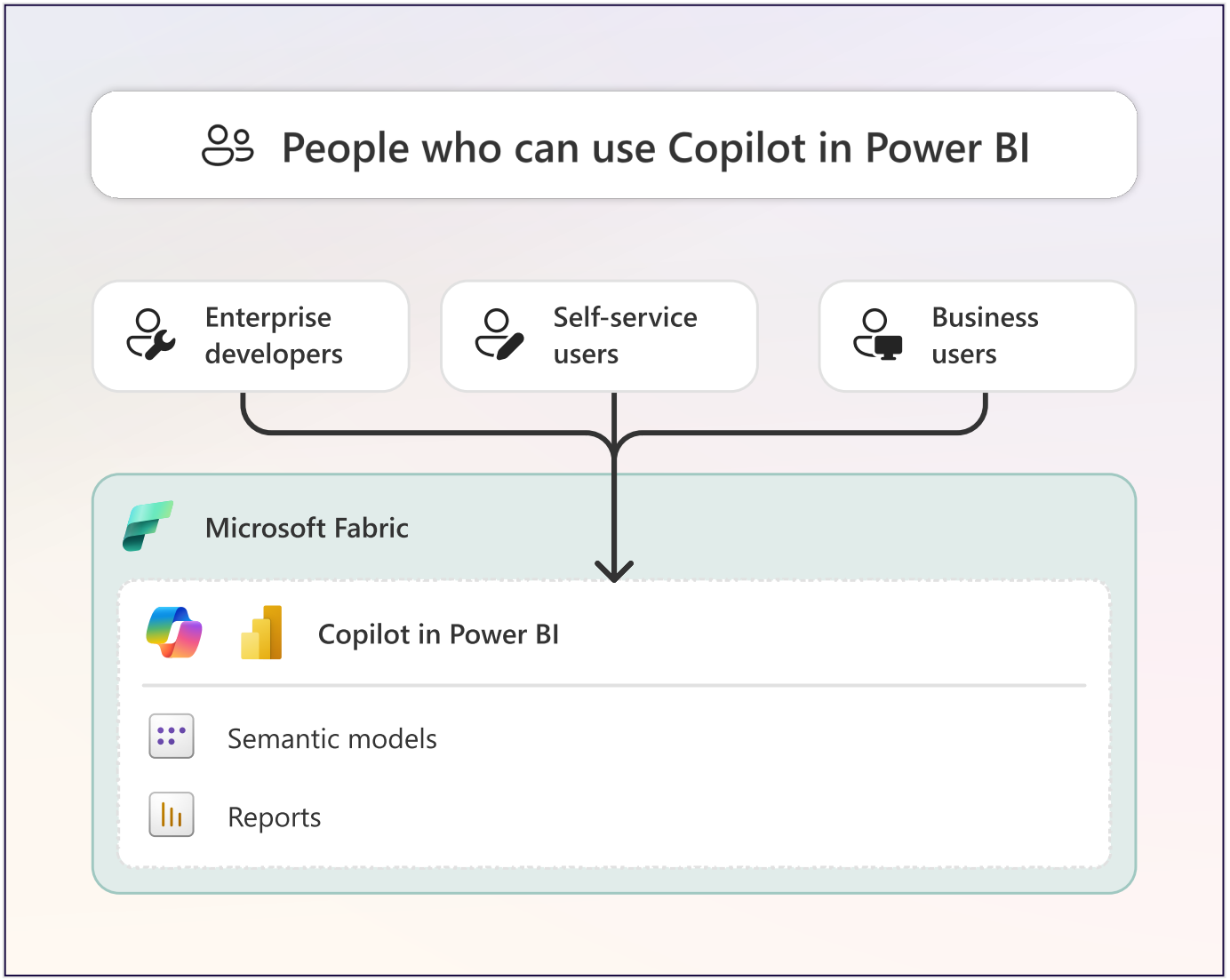
---

Last updated on 09/29/2026

Copilot in Power BI integration overview

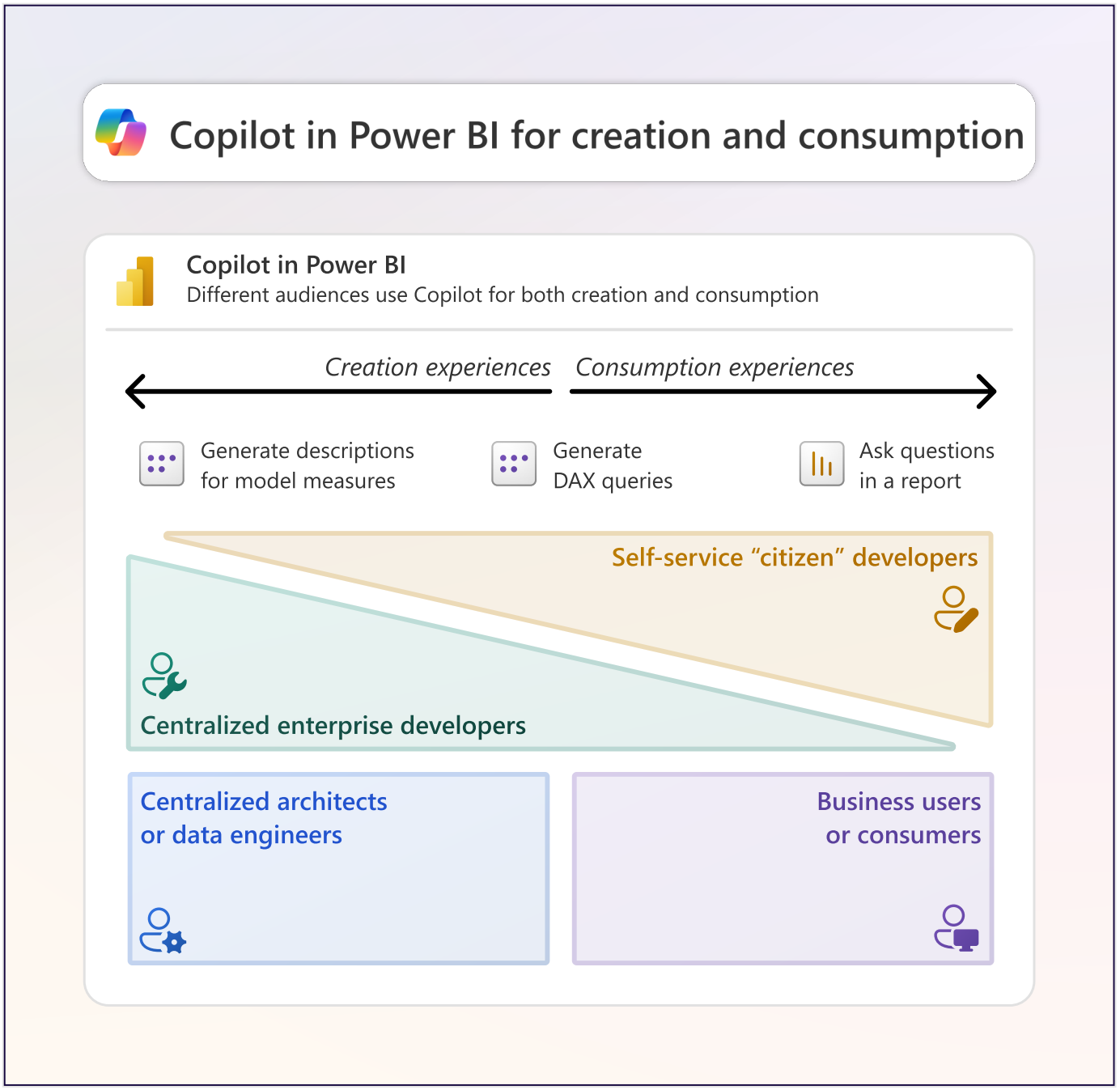
Copilot in Microsoft Fabric is a generative AI assistant that enhances the data analytics experience in the Fabric platform. Each Fabric workload, including Power BI, offers different Copilots. Various Power BI personas, including enterprise developers, self-service users, and business users, can use the different Copilot in Power BI experiences. This article provides an in-depth overview of how Copilot in Power BI enhances the data analytics experience by assisting users in creating and consuming semantic models and reports.

> **Note**
>
> For more information, see the Copilot in Power BI documentation.



Copilot in Power BI helps both Power BI developers and analysts *create* models and reports, while also giving business users new ways to *consume* those models and reports. The following

diagram shows how different personas in Power BI can use the various Copilot experiences to create or consume their data.



In summary, Copilot in Power BI has experiences that support the creation of Power BI items, like generating descriptions for model measures. Centralized enterprise developers, architects, data engineers, or self-service developers might all use these experiences to support their development tasks. Copilot experiences can also support consumption of Power BI items, such as asking questions in the Copilot chat pane of a report. Self-service users, business users, and even enterprise developers can all leverage these experiences to help them answer questions about their data in specific scenarios. Finally, some Copilot experiences support both creation and consumption, like generating DAX in a semantic model to either query and explore the model, or to create model measures that contain calculations and business logic.

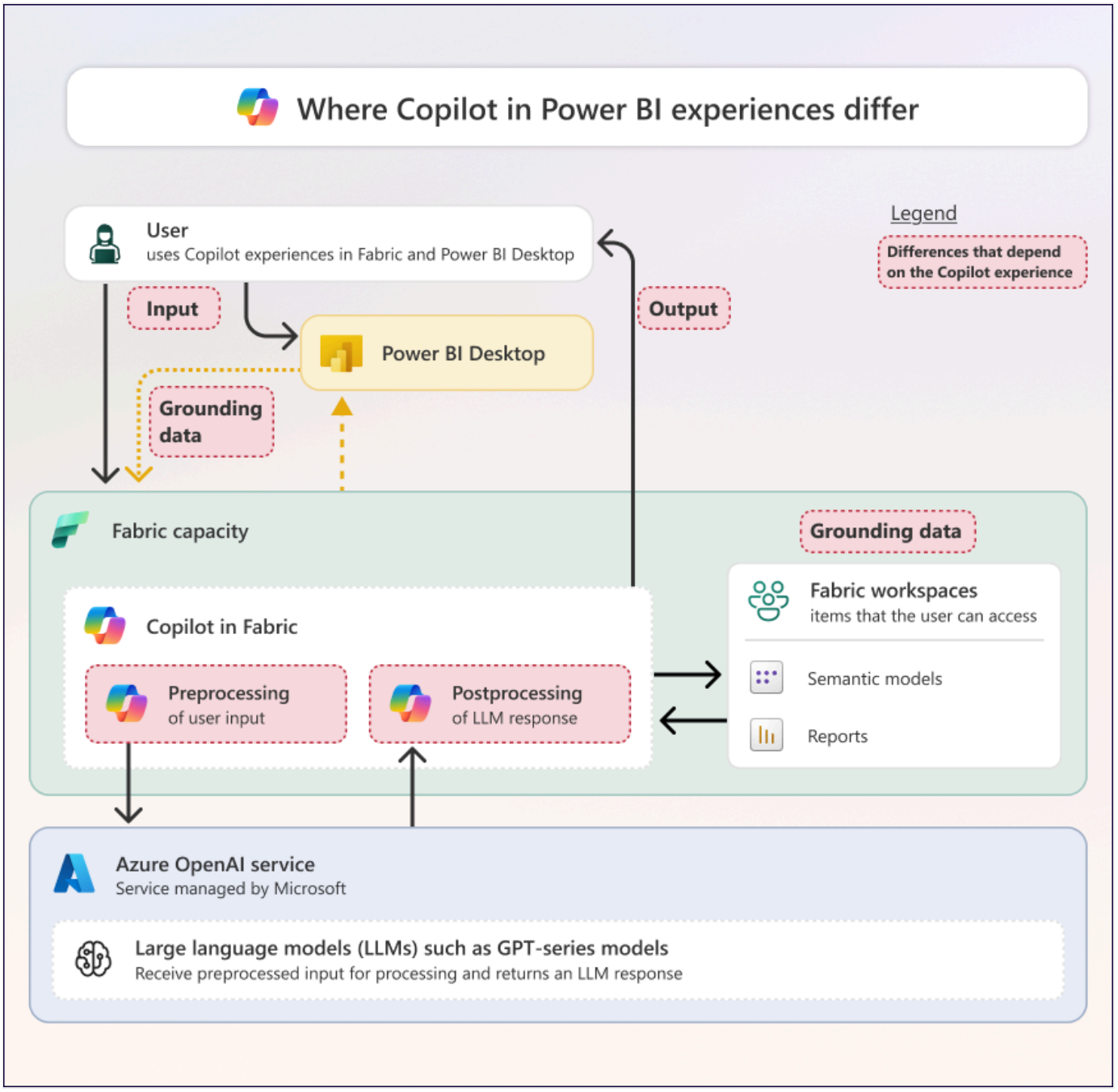
> **Note**
>
> Like Copilot in the other Fabric workloads, Copilot in Power BI can't *replace* the people who create semantic models or reports in your organization. Rather, Copilot aims to *augment* those individuals so that they're more efficient in their tasks and so that they can improve the models and reports that they create.
>
> Likewise, generated reports or answered data questions by Copilot can't replace the models and reports created by Power BI developers or analysts. These people typically have a deeper and broader understanding of the business problem and its surrounding context.

Copilot in Power BI provides new opportunities and approaches to work with your data. However, to get the most value from these new opportunities and approaches, you need a healthy data culture and good adoption of Power BI. This culture includes preparing effective user training and enablement, as well as monitoring and governing Copilot usage. Enabling and using Copilot in Power BI requires careful deliberation. You can't just turn on Copilot and expect to see productivity enhancement across your organization.

Overview of how Copilot in Power BI works

Copilot in Power BI works similarly to the general process of Copilot in Fabric. However, Copilot in Power BI differs in several areas, depending on the specific Copilot experience that an individual uses.

The following diagram depicts the different areas that differ between the Copilot experiences with Copilot in Power BI compared to Copilot in other workloads.



In summary, several areas of the Copilot architecture differ depending on which Copilot experience you use.

- **Input:** Different experiences require users to provide input in different ways.
  - The *ask data questions* experiences for models or reports require users to write a prompt in the Copilot chat pane.
  - The *generate measure descriptions* experience requires users to push a button in the Properties pane in the Model view of Power BI Desktop.

- **Preprocessing and grounding data:** Depending on the experience, Copilot retrieves different grounding data. Examples of preprocessing differences between the Copilot experiences in Power BI include:

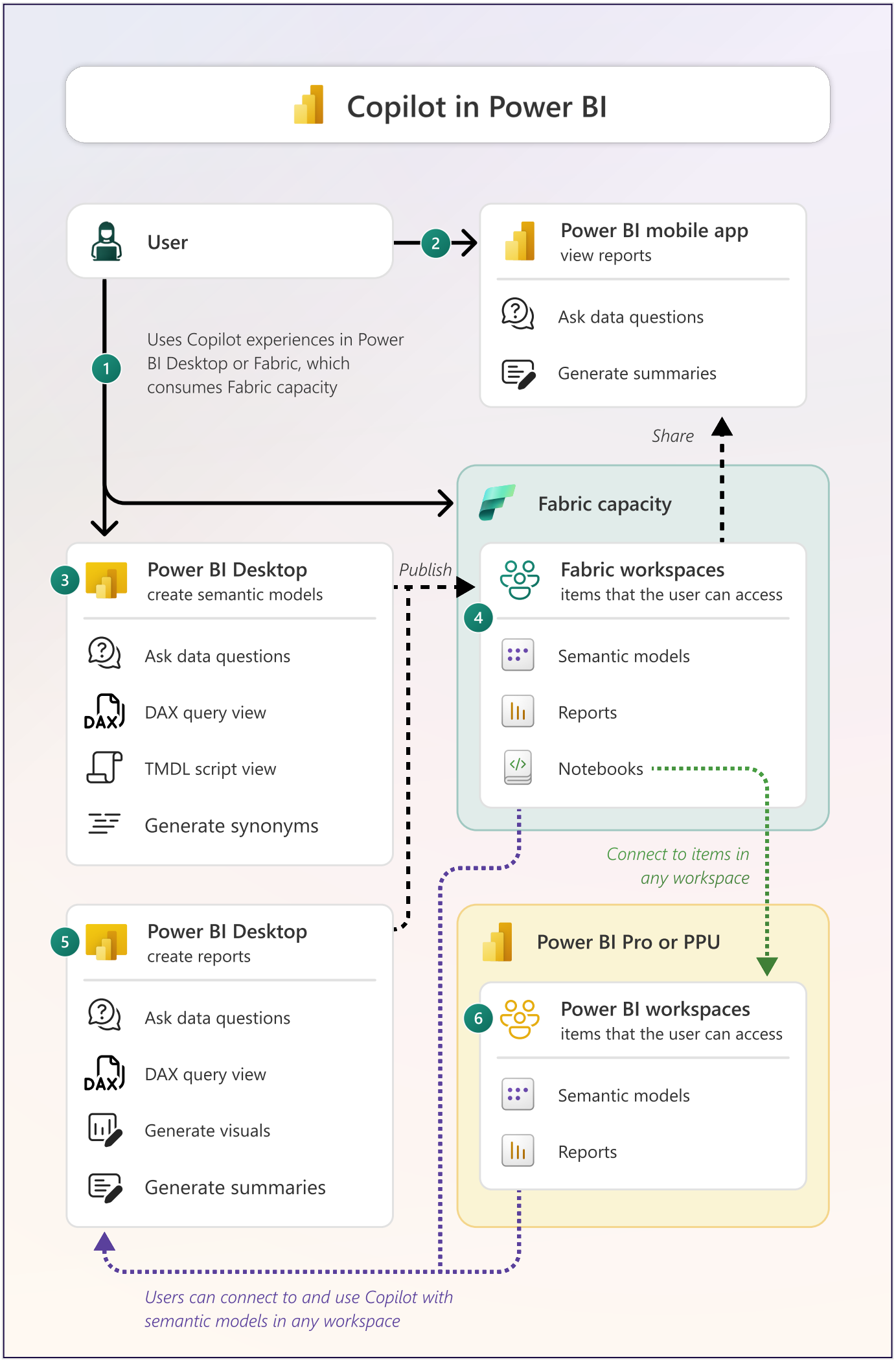
- The *ask data questions of a semantic model* experience might use the semantic model schema or linguistic model to generate an appropriate query. A semantic model developer can control what part of the schema Copilot can see and use by hiding fields. They can also increase the usefulness of their Copilot outputs by optimizing their model to work better with Copilot, such as adding descriptions.
- The *report page summarization* experience might use report metadata or even specific data points from your report visuals to make the summary. A report developer can control what part of the report Copilot can see and use by hiding pages and visuals.

- **Postprocessing:** Depending on the experience, Copilot handles the large language model (LLM) response in different ways. Examples of postprocessing differences between the Copilot experiences in Power BI include:
  - The *DAX query view* experience runs DAX from the LLM response through a DAX parser to ensure the generated DAX query runs. If not - for instance, if the LLM produces hallucinations - Copilot requests a new LLM response.
  - The *report page creation* experience takes the LLM response and combines it with an existing Copilot theme, the report metadata, and other information to create the new page in the Power BI report.

- **Output:** Depending on the experience, Copilot returns different types of outputs to a user after postprocessing. Examples of output differences between the Copilot experiences in Power BI include:
  - The *DAX query view* experience produces DAX queries if the user asks to generate or suggest DAX code, but it produces a natural language explanation of a DAX concept if the user asks how generated code works.
  - The *report page creation* experience produces a new report page if the user asks to generate a report page or to change an existing, previously generated report page.

Copilot in Power BI usage process flow

The following diagram gives an overview of how someone can use the different Copilot experiences with Copilot in Power BI.


Copilot in Power BI
User
2
Power BI mobile app
view reports
Ask data questions
Generate summaries
1
Uses Copilot experiences in Power BI Desktop or Fabric, which consumes Fabric capacity
Share
Fabric capacity
3
Power BI Desktop
create semantic models
Publish
Fabric workspaces
items that the user can access
4
Ask data questions
DAX query view
TMDL script view
Generate synonyms
Semantic models
Reports
Notebooks
Connect to items in any workspace
5
Power BI Desktop
create reports
Power BI Pro or PPU
6
Power BI workspaces
items that the user can access
Ask data questions
DAX query view
Generate visuals
Generate summaries
Semantic models
Reports
Users can connect to and use Copilot with semantic models in any workspace

The diagram consists of the following parts and processes:

Expand table

Item	Description
1	The user provides an input to Copilot, which could be a written prompt or another interaction that generates a prompt. All interactions with Copilot are user specific. The user can interact with Copilot in Power BI either by using the Power BI mobile app, Power BI Desktop, or in a Fabric workspace.
2	In the Power BI mobile app, users can access a standalone Copilot experience from the app home screen on phones and tablets to chat with data without first opening a specific report. Users can generate summaries, key insights, and auto-generated visuals with citations, share results with colleagues, and use voice dictation for prompts on iOS. Users can also view reports in Power BI apps or Fabric Org Apps and ask questions about specific reports by using the in-report Copilot. For more information, see Overview of Copilot in Power BI mobile apps.
3	In Power BI Desktop, users can use Copilot while they develop semantic models. They can ask data questions, generate DAX queries in the DAX query view, or generate synonyms for Q&A linguistic modeling or measure descriptions for the semantic model. To use Copilot in Power BI Desktop, users need to connect to a supported workspace on Fabric capacity.
4	In a workspace that uses a workspace type of a supported Fabric capacity, users can use Copilot with semantic models and reports that they publish from Power BI Desktop, upload via Git integration, or create in Fabric. Users might also use notebooks to help develop and manage semantic models by using semantic link and could use Copilot in those notebooks (even though notebooks aren't an item of the Power BI workload).
5	In Power BI Desktop, users can connect to a semantic model from any workspace and create reports. They might use Copilot to ask data questions, generate DAX, visuals, or summaries to support the report creation process. To use Copilot in Power BI Desktop, users need to connect to a supported workspace on Fabric capacity.
6	Workspaces that have a workspace type of Power BI Pro, Premium Per User (PPU), or an unsupported F SKU don't support Copilot. Users can consume items from these workspaces in Power BI Desktop and use Copilot, as long as they set up Power BI Desktop to consume Fabric capacity from a supported workspace. Users can also create reports in these workspaces, and use Copilot in these reports, as long as that report connects to a semantic model in supported workspace.

Note

While this diagram focuses on Power BI, Power BI developers and analysts don't necessarily use the Power BI workload in Fabric exclusively. These individuals also likely use the other Fabric workloads to create items and leverage the other various Copilot experiences.

Enable Copilot in Power BI

To use Copilot in Power BI, first enable it. The steps to enable Copilot in Power BI are similar to the steps you take to enable Copilot in Fabric. You can enable or disable Copilot specifically for Power BI, like with the other workloads.

Consider these scenarios:

- **P SKUs:** You can also use Copilot in Power BI if you have a P SKU.
- **Cross-workspace consumption:** Some Copilot experiences still work when you consume items located in workspaces that use a workspace type that doesn't support Copilot, like Pro, PPU, or an unsupported SKU. For example, you can use Copilot in Power BI Desktop while consuming a semantic model published to workspaces that don't support Copilot. Similarly, you can also connect to these semantic models from certain Fabric items, like notebooks using the Semantic Link feature.
- **Cross-capacity consumption:** Fabric administrators can delegate Copilot consumption to a Fabric Copilot capacity (FCC). This delegation can prevent Copilot usage from impacting your main Fabric capacity that other Fabric workloads and items need to support business-critical processes or decision making.
- **Mobile apps:** Copilot interactions in the Power BI mobile apps require a supported Fabric capacity. Copilot activity from mobile is billed against Fabric capacity, including when delegated to a Fabric Copilot capacity (FCC).

Responsible use of Copilot in Power BI

Keep the following considerations in mind when you use Copilot in Power BI:

- **Governance:** Set up security groups and grant access to Copilot in Power BI only after users complete training or demonstrate their understanding of the technology, its limitations, and its use cases. If you're enabling Copilot for the first time, use this approach to incrementally roll out Copilot in Power BI in phases when you're ready.
- **Preparation of semantic models:** Put significant effort into preparing your semantic model to work well with Copilot. These preparations include ensuring proper naming conventions, field descriptions, following good modeling practices, setting up linguistic modeling, and

other steps. Neglecting these preparations can lead to Copilot producing more unhelpful and inaccurate results. The standalone Copilot experience in Power BI mobile apps surfaces clear warnings when a semantic model isn't prepared for AI and directs users to preparation guidance.

- **Preparation of business users:** If business users should use Copilot in Power BI, explain to them when to use Copilot and when to simply look at and use their reports. Encouraging business users to use Copilot instead of interacting with reports can lead to increased capacity usage and inefficiency, as they might spend more time writing prompts and reading outputs than reading and interpreting visuals. Instead, they can use Copilot to help them find and interpret information in busy reports, particularly when they can't find what they need.
- **Evaluation of outputs:** Copilot outputs can contain inaccurate and low-quality content. Outputs are also nondeterministic, meaning that it's possible that a user receives a different output from a Copilot experience, despite using the same prompt and grounding data. It's important to set expectations with users so that they know what kinds of outputs to expect from Copilot, and how they can evaluate or validate these outputs, for themselves.

For more information, see Privacy, security, and responsible AI use of Copilot in Fabric and Copilot in Power BI.

> ⚠ **Warning**
>
> Content filtering and responsible AI features automatically reject LLM calls that contain certain words and phrases that are potentially harmful or problematic. However, your model might contain valid uses of these words and phrases for certain analyses or scenarios. If your model schema or metadata contains any of these phrases, then you can't use Copilot in Power BI, as all of your prompts result in errors.
>
> Separately, consider enabling Copilot in Fabric for specific security groups and workspaces only after you take the appropriate steps to prepare to use it responsibly. Enabling Copilot before you make these preparations can lead to unnecessary governance risks and challenges with adoption.

Copilot experiences in Power BI

Power BI offers different Copilot experiences that you can use with either semantic models or reports. The following articles in this series describe these experiences in detail and how you can best use them.

- Use Copilot with semantic models
- Use Copilot with Power BI reports
- Copilot for Power BI apps: Report-scoped and app-scoped AI features that help you summarize reports, find items, and answer data questions within Power BI apps.
- Standalone Copilot experience in Power BI: A full-screen Copilot experience in the Power BI service that finds and answers questions about any report, semantic model, or Fabric data agent you have access to.
- Standalone Copilot in Power BI mobile apps: A consumption experience accessible from the Power BI mobile home screen that lets you chat with your data, get summaries and insights, receive auto-generated visuals with citations, share results, and use voice dictation on iOS.

> **Note**
>
> See **Where to find the Copilot experiences in Fabric** for a full overview of each experience by item and workload.

Related content

- Overview of Copilot for Power BI
- Enable Copilot in Fabric

Last updated on 04/21/2026

Use Copilot with semantic models

This article helps you use the Copilot experiences in Power BI for semantic models.

Copilot in Microsoft Fabric is a generative AI assistant that aims to enhance the data analytics experience in the Fabric platform, including the Power BI workload. You can use Copilot in Power BI to support both the development and the consumption of semantic models by both self-service and enterprise users. When used effectively, Copilot can support both analysts and business users to improve productivity and get more value from their semantic models.

However, to use Copilot with semantic models in Power BI, you must first prepare your data, your semantic model, and your users. If you don't prepare these elements, Copilot mainly produces low-quality and inaccurate outputs that might be incorrect or even misleading. The following sections guide you through how you can use Copilot in Power BI to either develop or consume a semantic model with the available Copilot experiences.

> **Note**
>
> For additional information, see the Copilot in Power BI documentation.
>
> Copilot and its capabilities are evolving over time. The Copilot experiences in the following sections might change, and new experiences might become available with semantic models. As such, you should keep up to date with the monthly Power BI releases and any relevant Copilot announcements.

> **Important**
>
> You can't enable or disable Copilot in specific workloads, or for specific Copilot experiences. If you enable Copilot in Fabric in your capacity for a user or security group, then all workloads and Copilot experiences are accessible for those users.

Develop a semantic model with help from Copilot

When you develop a semantic model in Power BI, use Copilot to help improve the efficiency and convenience of certain tasks. Typically, this process involves leveraging Copilot to streamline redundant or repetitious changes for you (such as generating measure descriptions) or helping

you to tackle or understand challenging model design or DAX (such as using Copilot in the DAX query view to explain DAX concepts or to generate DAX queries).

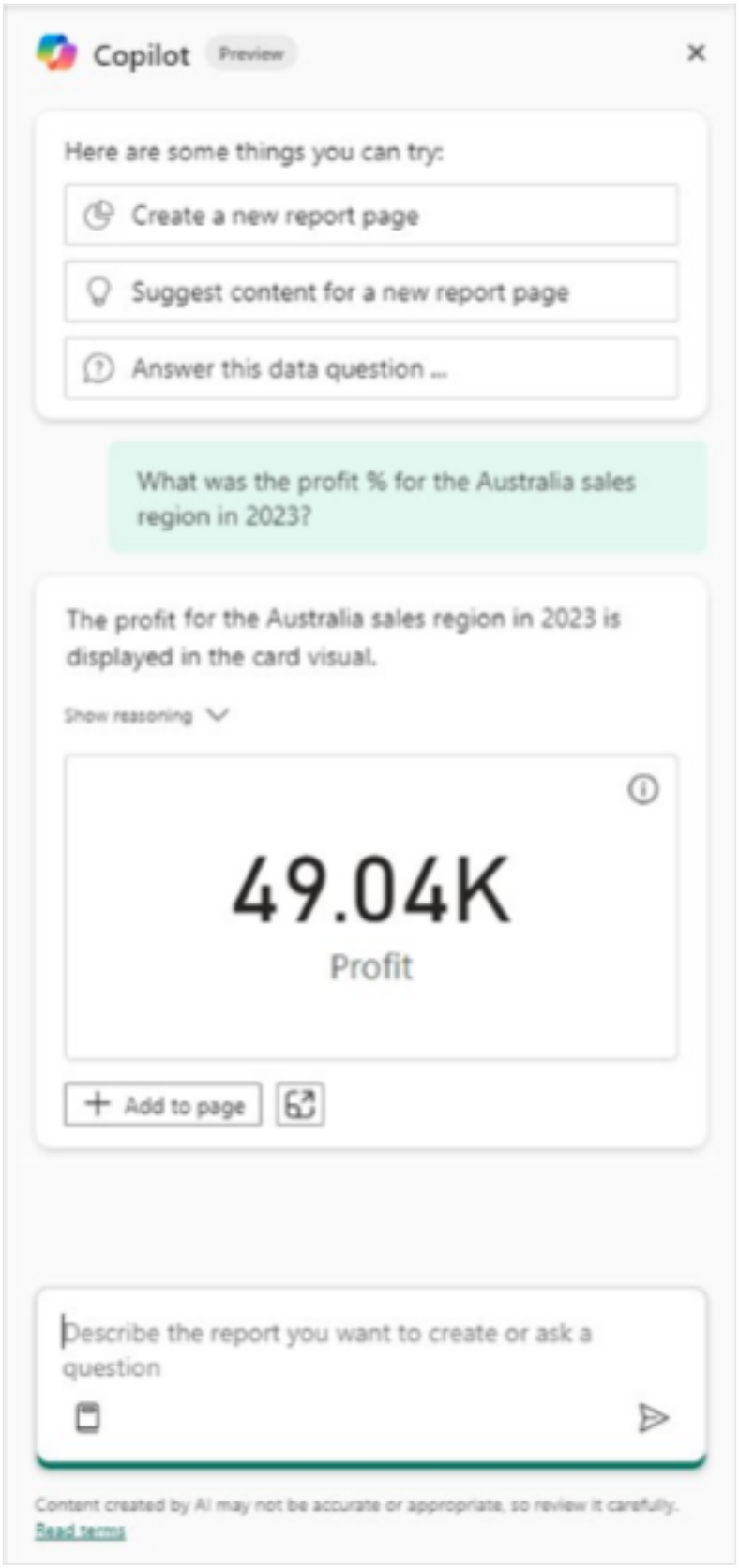
The following sections explain how a Power BI developer or analyst who creates semantic models can leverage the various Copilot experiences in Power BI.

> **Note**
>
> The following sections focus on the experiences that support semantic model development in Power BI Desktop, and not online by using ***Edit data models in the Power BI service***. The guidance and considerations in the following sections might still apply when you edit data models in the Power BI service, but be aware that differences exist between these two experiences.
>
> In general, develop Power BI semantic models by using Power BI Desktop or third-party tools, then publish these models to the Power BI service when you're ready to use them. However, you can also edit data models in the Power BI service if that is your preference, or if you can't use desktop tools to develop your model.

Ask data questions about your semantic model

While developing a semantic model, you often need to explore and query the data. Normally, you use a combination of the Power Query editor, the report canvas, and DAX queries. However, you can also ask data questions about your model and its contents by using the Copilot chat pane. Generally, asking data questions is more of a consumption experience, where business users can ask questions about the data when they can't find the answer in a report. However, developers might use this feature to explore data or validate how Copilot performs on their model.

The following image shows an example of a user asking a data question to Copilot about a semantic model. The image shows the following prompt: *What was the profit % for the Australia sales region in 2023?*



Use cases

Use Copilot to ask data questions for the following use cases:

- **Explore data in your model:** A developer might use the *Ask data questions* experience to explore the data in their model by using Copilot. However, it's likely more efficient to explore the data by adding visuals to the report canvas, or by using DAX queries. That's

because report visuals automatically update whenever you make changes in the model, but answers to data questions in the Copilot chat pane are static, and disappear whenever you close the Copilot chat pane. Additionally, visuals generated by Copilot can contain visual-specific filters that you can't create yourself in Power BI Desktop. These filters can be misleading or troublesome to investigate.

- **Validate Copilot use for consuming the semantic model:** Developers should use the *Ask data questions* experience to understand and test how Copilot consumption experiences perform on their semantic model. This is the main use case for using *Ask data questions* when you develop a semantic model. This use case lets a developer make changes to their semantic model that might better support Copilot to produce more useful outputs.

> **Important**
>
> When you design your semantic model, ensure that you discuss with users and document how they'll consume it, including which items, tools, and experiences they'll use. If users won't use Copilot in Power BI, then a developer doesn't need to use the *Ask data questions* experience to test their model. However, if users plan to use Copilot to consume semantic models now or in the future, then it's essential for developers to account for this during both model design and development.

Benefits

A developer can use *Ask data questions* to understand how the experience will be for business users who consume the semantic model by using Copilot in Power BI. This testing is essential if business users want to use Copilot to consume a semantic model.

Specifics of the experience

The *Ask data questions* experience differs from the standard Copilot in Fabric process in the following areas. This overview applies to use of the *Ask data questions* experience when both *developing* and *consuming* semantic models:

- **Input:** Users provide a written prompt asking a question or requesting specific information from the semantic model.

- **Preprocessing and grounding data:** Copilot retrieves grounding data from the model schema. It performs schema reduction to try to restrict the context to what is most

important. As context, Copilot takes the following information to try to improve the usefulness and specificity of the Copilot output:

- Any report metadata on the current report page. If there's relevant report metadata, then Copilot in Power BI answers data questions from the report instead of the model.
- The conversation with Copilot in the current session. This conversation includes any previous questions and outputs, which includes data points from data questions that Copilot previously answered.
- The semantic model schema, which includes tables, rows, columns, measures, and other objects (like relationships, calculation groups, and so forth).
- The full model linguistic schema.
- Certain semantic model properties, including descriptions, data types, format strings, and data category.

The following information is excluded:

- Any report page that's hidden.
- Any field (measure or column) in the model that's hidden.
- Any table in the model that's marked as *private*.

- **Output:** The output that Copilot provides contains several parts:
  - **Visual:** Copilot answers the data question by rendering a Power BI visual, such as a card, a line chart, or a table. Copilot chooses the visual and its formatting, which the user can't control or request in their prompt. The visual might time out if the underlying model, DAX, or data isn't optimized or is too complex.
  - **Summary:** Copilot summarizes the query result. This summary is generated by sending a semantic query result back to Azure OpenAI (which includes data points) and requesting a natural language explanation.
  - **Errors or clarification request:** If Copilot is unable to produce a response, it might return a generic error message. This error message could include a request for clarification, including suggested variants of the user's original data question.

> **⊗ Caution**
>
> When using Copilot in Power BI with semantic models in Power BI Desktop, Copilot might use report metadata as grounding data. In certain circumstances, report metadata can contain data points, such as column values, which might include sensitive information. This condition is true both for the legacy report metadata format and the new **Power BI enhanced report (PBIR) format**.

Tips to improve Copilot outputs

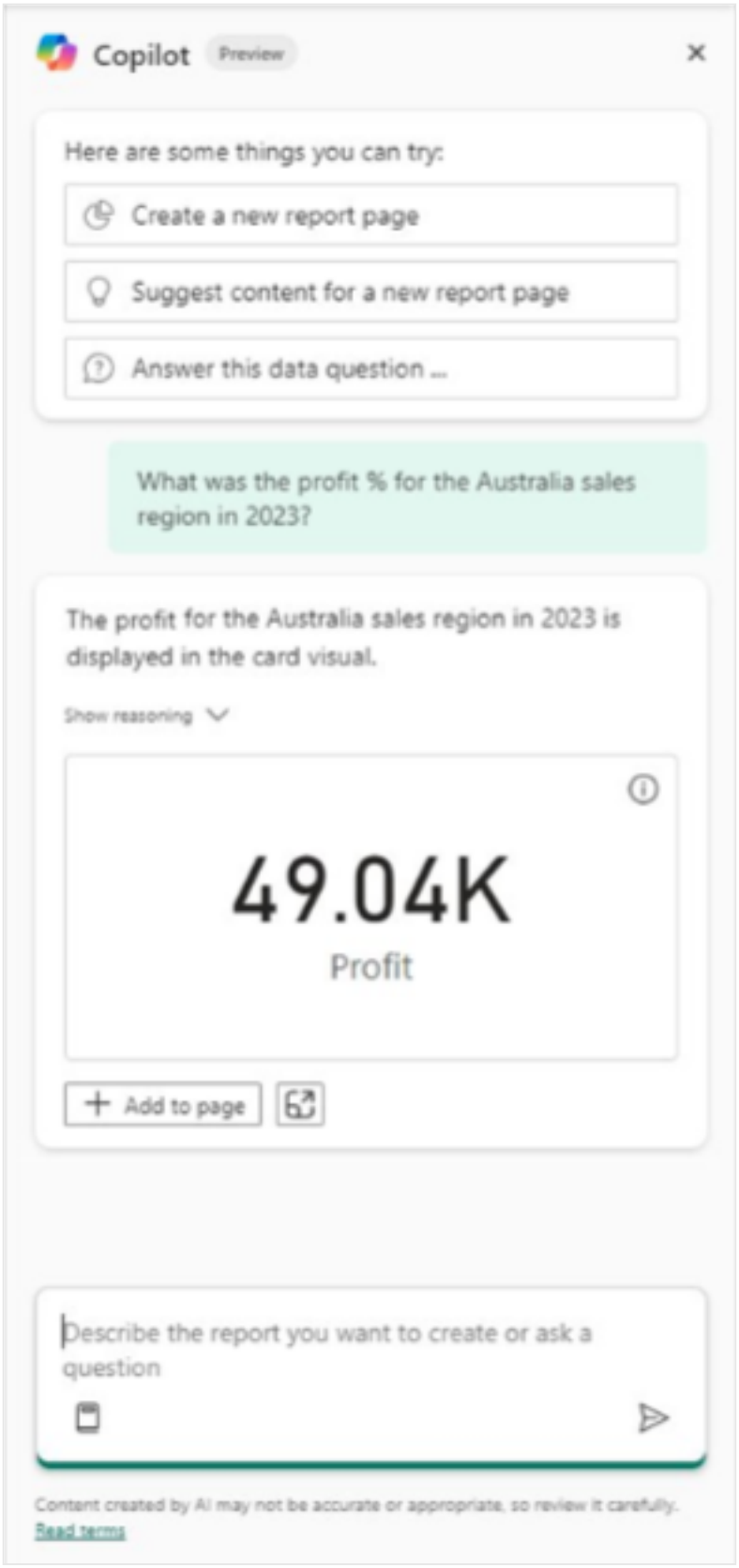
Copilot can produce inaccurate or low-quality outputs, including incorrect answers to data questions. Often, incorrect answers arise from either user mistakes or model issues. To reduce the probability of wrong or unexpected Copilot outputs, address user prompts and data model design.

> **Important**
>
> Inaccurate responses to data questions can lead to incorrect decisions and actions by business users, which produces bad results. To avoid this negative impact, it's important that you mitigate the probability of inaccurate responses by Copilot as much as possible.

A user can only really produce incorrect results by writing a poor prompt. Examples of poor prompts include:

- **Vague or incomplete prompts:** If you inaccurately or incompletely describe the desired output or you use ambiguous language in your prompt, then Copilot is less likely to produce a useful result. When writing prompts, try to be as specific and descriptive as possible in stating your desired result.
- **Incorrect prompts:** If you make spelling mistakes when you refer to a measure, column, or table name, Copilot might not refer to the correct field. When writing prompts, ensure that the fields you mention in your prompt correctly refer to the fields in the semantic model schema. This rule includes avoiding abbreviations, acronyms, or excessive punctuation. You can also use synonyms to refer to fields, but there's no way to validate which synonyms are available for a given field (like how you can see descriptions when hovering on a measure or column in the Data pane).
- **Excessive or inappropriate grounding data:** If you submit a prompt via the Copilot chat pane, Copilot takes the chat history from that session as grounding data during preprocessing. Depending on what that chat history entails, you might get different or unexpected results. When writing prompts, take into consideration that any previous prompts and outputs are used as grounding data. To avoid this problem, select the Copilot button to close and reopen the Copilot chat pane, clearing the chat history before you submit a new prompt.

The following image shows an example of an incorrect output from Copilot due to a poor user prompt:



The image shows the following prompt: *What was the profit % for the Australia sales region in 2023?* In the output, Copilot returns the profit value rather than the profit as a percentage. In this case, the user could obtain a better result from Copilot by specifying more clearly that they want the profit in percentage (rather than using the percentage symbol).

Aside from user prompts, the *Ask data questions* experience can also produce incorrect or unexpected results due to the quality or organization of the underlying semantic model.

Examples of when your semantic model produces poor results include:

- **Poor model design or implementation:** If your semantic model doesn't follow common best practices (such as a star schema design), you're likely to get poor results with any consumption tool or experience. Always strive for quality semantic models to get the most out of Power BI and Fabric. See also Data modeling guidance for Power BI for more tips and guidance to improve your Power BI semantic models.

- **Poor naming conventions:** If your fields have inconsistent or excessive naming conventions (such as acronyms, abbreviations, and punctuation), users are more likely to make mistakes in their prompts and Copilot gets confused when referring to these fields. Copilot works best when tables, columns, and measures have names in human-readable English.

- **Poor model organization:** If you don't organize your model, you're likely to experience more issues with Copilot. Model organization is a large topic that encompasses many tasks, including field descriptions, hiding columns and measures, and avoiding fields with the same names across different tables.

- **Linguistic modeling:** If your semantic model doesn't have linguistic modeling set up, including synonyms for fields or verbs for relationships, Copilot might be more likely to return unexpected results. Power BI relies on the same linguistic modeling as the Q&A feature. Setting up a linguistic model for your semantic model costs additional time and effort on top of your semantic model development tasks. For more information, see Intro to Q&A tooling to train Power BI Q&A.

- **Model complexity:** The more complex your model is, including having more fields, dependencies, and business logic, the more likely you are to experience difficulties when using Copilot. For instance, complex patterns like currency conversion or disconnected tables (like field parameters) might cause unexpected or incorrect results when users reference these fields or tables in their prompts. With more complex models, you might need to consider specific model design decisions to obtain the best results with Copilot. In general, you should test your model with Copilot to determine whether you get consistently correct and reliable results. If not, you might want to consider advising users not to use Copilot to consume your semantic model.

> ⓘ **Note**
>
> For more information, see Prepare a semantic model for Copilot consumption later in this article. See also the separate article update your data model to work well with Copilot for

Power BI for additional considerations.

The following image shows an example of an incorrect output from Copilot due to problems with the semantic model:

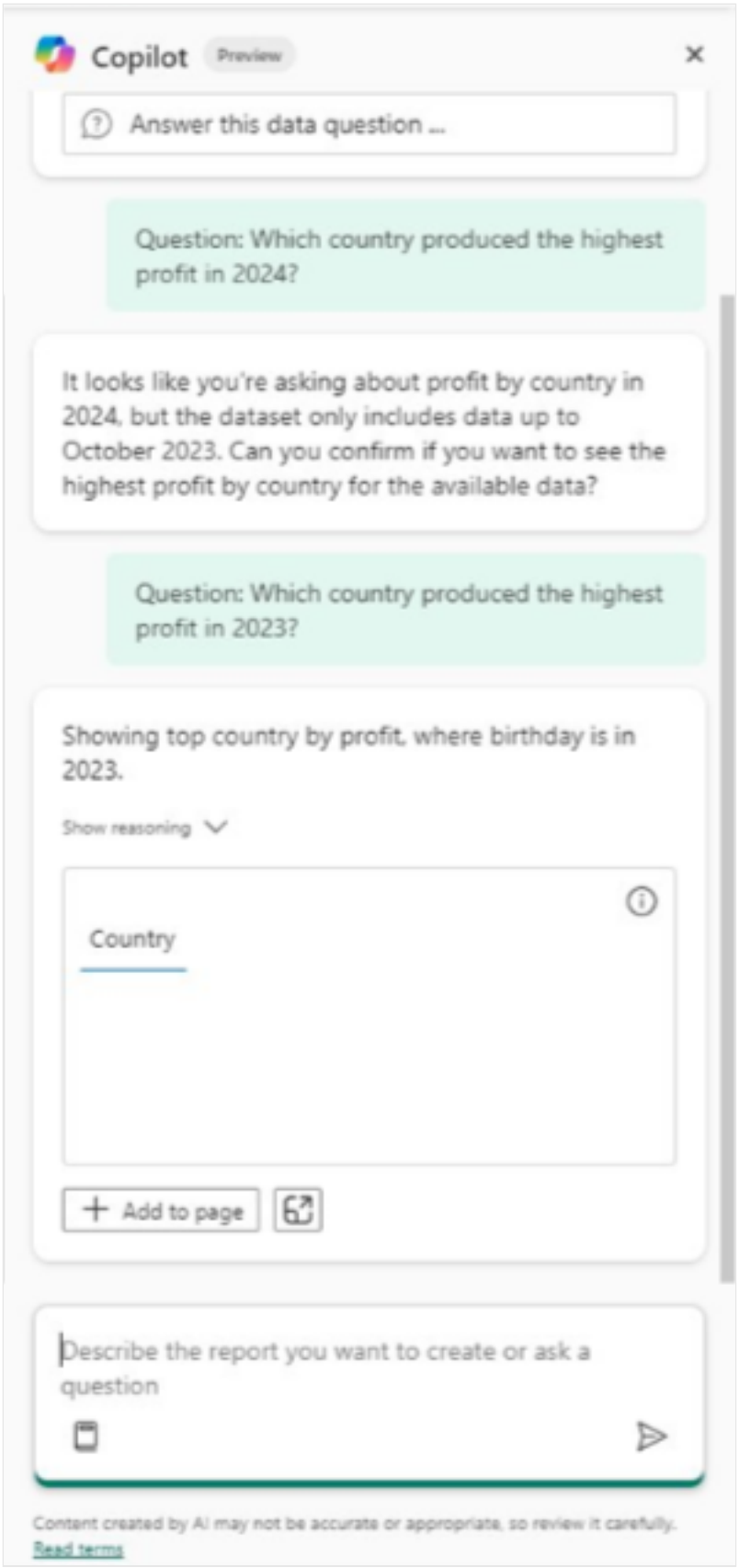
The image depicts the following prompt: *How many units were sold in Australia in 2023?* In the response, Copilot returns the number of units where the *Country* column in the *Customer* table is filtered to "Australia", instead of filtering the *Country* column in the *Sales region* table. In this

case, the user could have improved the prompt to specify sales region country. However, the semantic model developer could also hide or rename the *Country* column in the *Customer* table if it isn't intended to be used by Copilot. Alternatively, they could disable the field from the field synonyms of the linguistic model by using the Q&A setup.

> **Tip**
>
> You can identify which fields and filters Copilot uses to generate a visual or answer a data question by clicking the *Add to page* button. This action adds the visual to the report canvas, where you can select the visual and view any filters Copilot applies or the fields Copilot uses.

Irrespective of prompt or model quality, you can still obtain inaccurate or low-quality outputs from Copilot when you use the *Ask data questions* experience. That's because the underlying model - with its current configuration - is nondeterministic and isn't guaranteed to produce a correct answer, or the same answer with the same prompt, model, and data.

The following image shows an example of Copilot producing inaccurate or inappropriate results:



In this example, the image shows the prompts: *Question: Which country produced the highest profit in 2024?* and *Question: Which country produced the highest profit in 2023?* The outputs show Copilot first asking for clarification after it says that there's no data for 2024 in the model. However, there *is* data in the model in 2024 when filtering to that year using the date table. Copilot then returns a result filtering the *Birthday* column from the *Customer* table, rather than

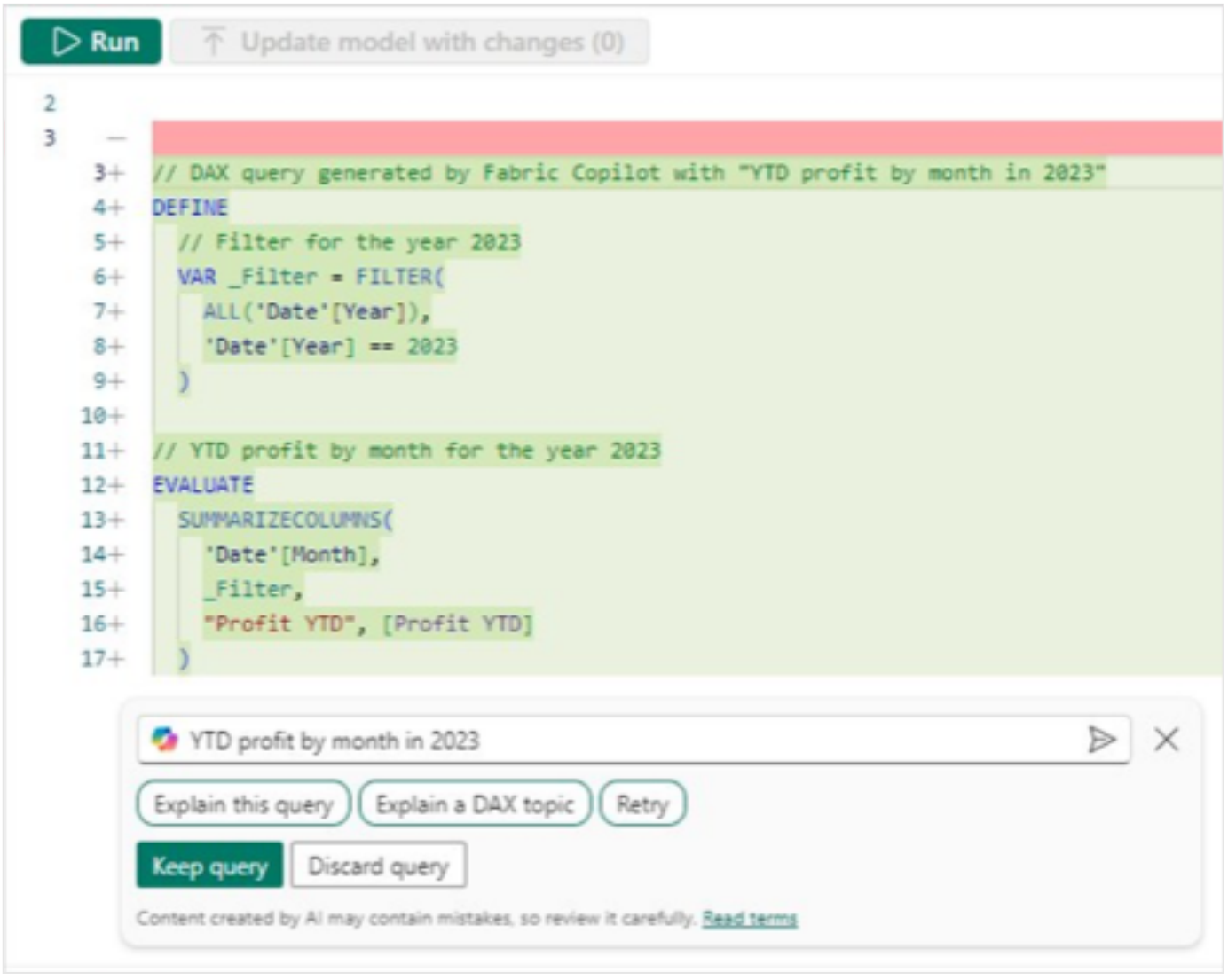
the marked date table in the model. While the user could produce a more specific prompt or the developer could hide the *Birthday* field, in this case, Copilot should produce the expected result.

To mitigate this problem, train users to critically appraise any outputs that they obtain from Copilot in Fabric and Power BI, and what to do to troubleshoot unexpected results or to try a new prompt by closing and reopening the Copilot chat pane or window.

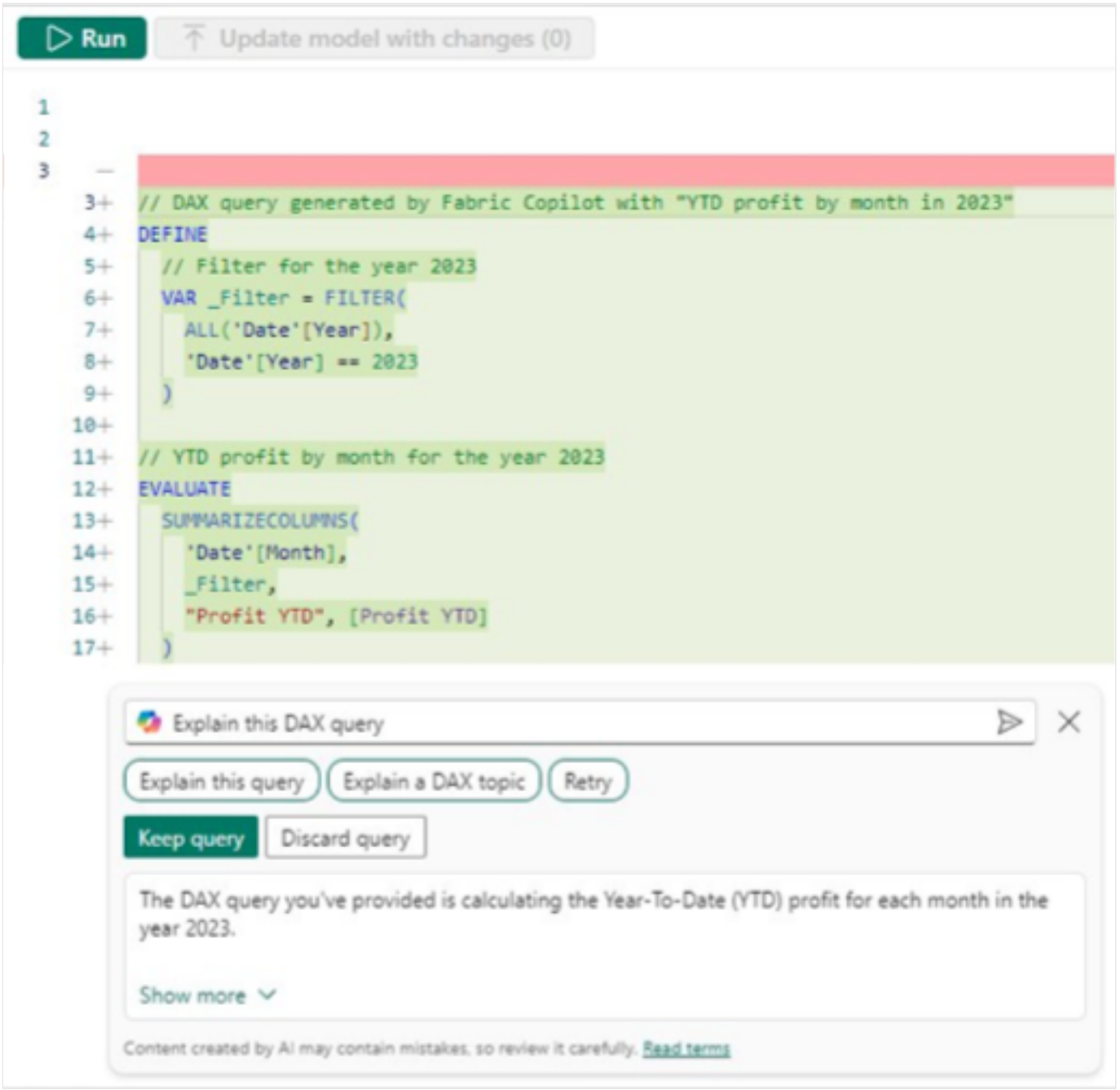
DAX query view

While developing a semantic model, use Copilot to generate DAX queries in the DAX query view. In these queries, you can also define DAX measures, which you can then add to your semantic model. You can also use Copilot to explain DAX concepts either in generated queries or queries that you authored yourself.

The following image shows an example of a user asking Copilot to generate a DAX query.



The image depicts the following prompt: *YTD profit by month in 2023*. The user could then ask Copilot to explain the query, as depicted in the following image.



The image depicts the following prompt: *Explain this DAX query*. The explanation is shown at the bottom of the Copilot window.

Use cases

Use Copilot in the DAX query view for the following use cases:

- **Generate DAX queries:** Use Copilot to suggest DAX queries, which you can use to:
  - Explore or analyze your model or data.
  - Test and validate your model or data.
  - Try new approaches or patterns to calculations in DAX.
- **Refactor existing DAX queries:** Use Copilot to more conveniently or efficiently modify queries that you copied or wrote yourself, such as:

    - Adding comments to explain and document a DAX query or measure.
    - Making the query easier to read and understand, such as a query copied from a Power BI report visual that you're investigating.
    - Making large or repetitious replacements in the code.
    - Asking Copilot to remove or replace variables.

- **Generate DAX measures:** Use Copilot to suggest DAX measures which are defined upstream of a generated query. These measures can be limited to the DAX query you generate or added to your semantic model.

- **Explain DAX concepts:** Use Copilot to explain and better understand different concepts in DAX, such as:
    - DAX query or a defined measure in the query.
    - DAX functions, such as CALCULATE or KEEPFILTERS.

Benefits

The main benefit of using Copilot in the DAX query view is that you get support when you write DAX queries. This support means that less experienced developers and analysts can use Copilot to learn DAX concepts or generate DAX to use in their model. It can also save intermediate and experienced developers time when they need to develop DAX, but they can't remember a particular function or pattern, or when they want to refactor or simplify an existing query.

Other generative AI tools are available for DAX code, but Copilot in Power BI has the following specific benefits:

- Copilot in Power BI is developed in collaboration with the authors of the DAX language.
- Copilot doesn't use any model fine-tuning, but it contains metaprompts with specific DAX examples by Microsoft.
- You don't have to copy or paste code or model information, since Copilot is integrated in Power BI Desktop.
- During postprocessing, Copilot uses a DAX parser to ensure that the query is valid, which reduces the probability of receiving hallucinations in your queries.

> **⚠ Warning**
>
> Inexperienced Power BI developers or analysts should ensure that they validate and understand any DAX that they generate *before* they use it. This validation is particularly important when you use Copilot in the DAX query view to suggest measures that you add to your model. That's because Copilot doesn't know where you'll use that measure, and while

the suggested code might work in the initial DAX query, in a different filter context of your report, it could produce unexpected or incorrect results.

To validate and understand DAX, these inexperienced developers can use Copilot to add comments to the code, explain concepts, and also research the patterns or functions by using verified online sources, such as Microsoft documentation.

Specifics of the experience

The Copilot experiences in the DAX query view have steps during preprocessing and postprocessing that differ from the other Copilot experiences in Fabric.

Copilot in the DAX query view has the following specifics to keep in mind:

- **Input:** Users provide a written prompt in an input box specific to a single DAX query window. They can request a query, request changes to an existing query that Copilot already generated, or explain a DAX concept. In the input box, users can also select buttons to generate queries. The Retry button re-generates the query while neglecting the current context, so that you can try to get a different approach.

- **Preprocessing and grounding data:** Copilot retrieves grounding data from the model schema and anything in the query window. Copilot takes the following information as context to try to improve the usefulness and specificity of the Copilot output:
  - Any text in the current DAX query window, including DAX code you wrote, comments, or previous DAX queries that you generated.
  - The conversation history with Copilot from the currently active session. This history includes previous questions and outputs, but not data points.
  - The semantic model schema, which includes tables, rows, columns, measures, and other objects like relationships and calculation groups. This schema includes all objects irrespective of whether they're hidden or not, except when you have a live connection to a shared semantic model.
  - Synonyms from the model linguistic schema.
  - Certain semantic model properties, including DAX expressions, descriptions (truncated after the first 200 characters), data types, format strings (and format string expressions), and data category.
  - Some statistical aggregations like minimum and maximum values of columns from your model that might be used in a query. These values are data points sent to Copilot as context.

- Copilot might also send the query result back to Azure OpenAI to explain the generated query or its results.

The following information is excluded:

- The conversation history with Copilot in the current session when you select the Retry button.
- Any table in the model that's marked as *private*.
- Comments in DAX expressions.

- **Output:** The output that Copilot provides contains either DAX code and DAX comments in the DAX query window, or explanations of the DAX in the Copilot input box. A user typically must then choose to run and keep the query themselves.

Tips to improve Copilot outputs

To improve the quality of DAX queries generated by Copilot, use the same tips as when you use the *Ask data questions* experience in Copilot. Specifically, ensure that your prompts are clear, accurate, and descriptive, and that your semantic model is well-designed, organized, and doesn't contain too much complexity or exceptions.

> ⓘ **Note**
>
> For more information, see **Prepare a semantic model for Copilot consumption** later in this article. See also the separate article **update your data model to work well with Copilot for Power BI** for additional considerations.

Some tips specific to the DAX query experience are as follows:

- **Differences between local and live connection models:** Copilot works differently depending on whether you're querying a local model open in Power BI Desktop or a shared semantic model in the Power BI service. For instance, when you have a live connection to a shared semantic model and use the DAX query view experience:
  - Copilot can't see the DAX expressions of measures, or any hidden or private objects.
  - Copilot must run the DAX query before returning it to ensure the query is valid.

- **Use the Retry button:** The Retry button empties the Copilot cache, ensuring that you get a new result with the same prompt and grounding data. This feature is useful when you're iteratively working toward a solution that meets your specific requirements.

- **Ask Copilot to add comments to your code:** Comments are a useful way to organize and document your DAX queries, and to help you understand the code that Copilot generates. You can also ask Copilot to explain the DAX concept.
- **Check quality, peer-reviewed sources:** If you still don't understand the generated code with comments and explanations from Copilot, research the functions and patterns online from reputable sources either in the Microsoft documentation or from the Power BI community.
- **Beware of variable usage by Copilot:** Copilot can struggle to use variables appropriately in DAX queries and defined measures. For instance, Copilot might try to filter or group a variable that's already been declared, which isn't possible and produces an unexpected result.
- **Use concise descriptions to distinguish between similarly named fields:** Descriptions help when similar fields exist in the same model, such as *Name* in the *Customer* table and *Name* in the *Store* table.
- **Beware calculation group usage by Copilot:** Copilot can struggle to use calculation groups in your suggested queries. To improve Copilot's use of calculation groups, include calculation item names listed in the calculation group description.
- **Beware newer functions and DAX syntax:** Copilot and other generative AI tools are limited in their training data volume and scope. As such, they're more likely to make mistakes with newer DAX functions or syntax. For these scenarios, you might want to try authoring the query yourself first, then revising it by using Copilot.
- **When generating measures, always ask for a query:** The Copilot experience in the DAX query view is designed to generate DAX queries. You get the best results when you instruct Copilot to perform this task, rather than asking it to generate a measure or another DAX expression.

Generate measure descriptions with Copilot

Use Copilot to generate descriptions for model measures. Measure descriptions help both model consumers and other developers understand the purpose of a measure and how to use it. Also, measure descriptions can improve the usefulness of Copilot outputs for other Copilot experiences, such as when you use Copilot to generate DAX queries in the DAX query view of a Power BI semantic model.

Use Copilot to generate measure descriptions for your model measures. Then, review the results and revise them to ensure they're accurate, concise, and helpful.

Benefits

Creating measure descriptions is normally an arduous task that's often neglected. Generating measure descriptions reduces the time you spend organizing and documenting your semantic model. Unlike a linguistic schema, all semantic models benefit from adding descriptions to tables, columns, and measures. This practice helps improve model documentation and usefulness for others in your organization.

> **Tip**
>
> If you need measure descriptions in another language, you can generate them in English first. Then, you can translate your published model automatically by using other tools, such as semantic link labs in notebooks.

Tips to improve Copilot outputs

This Copilot experience uses the following grounding data for context:

- DAX expressions for measures and calculated objects.
- Field properties, including descriptions, data types, format strings, and data category.
- Field synonyms.

Given this grounding data, you can ensure that suggested synonyms are useful by:

- Using consistent naming conventions.
- Limiting the use of punctuation, acronyms, and abbreviations.
- Naming measures in English.

Copilot in web modeling (Power BI service)

Copilot in web modeling is an AI-powered assistant that's integrated directly into the Power BI service model authoring experience. Unlike the other experiences described in this section, which are primarily available in Power BI Desktop, Copilot in web modeling lets you analyze and improve your semantic models by using natural language without leaving the browser.

When you open a semantic model in the service's **Model view** and switch to Editing mode, Copilot can:

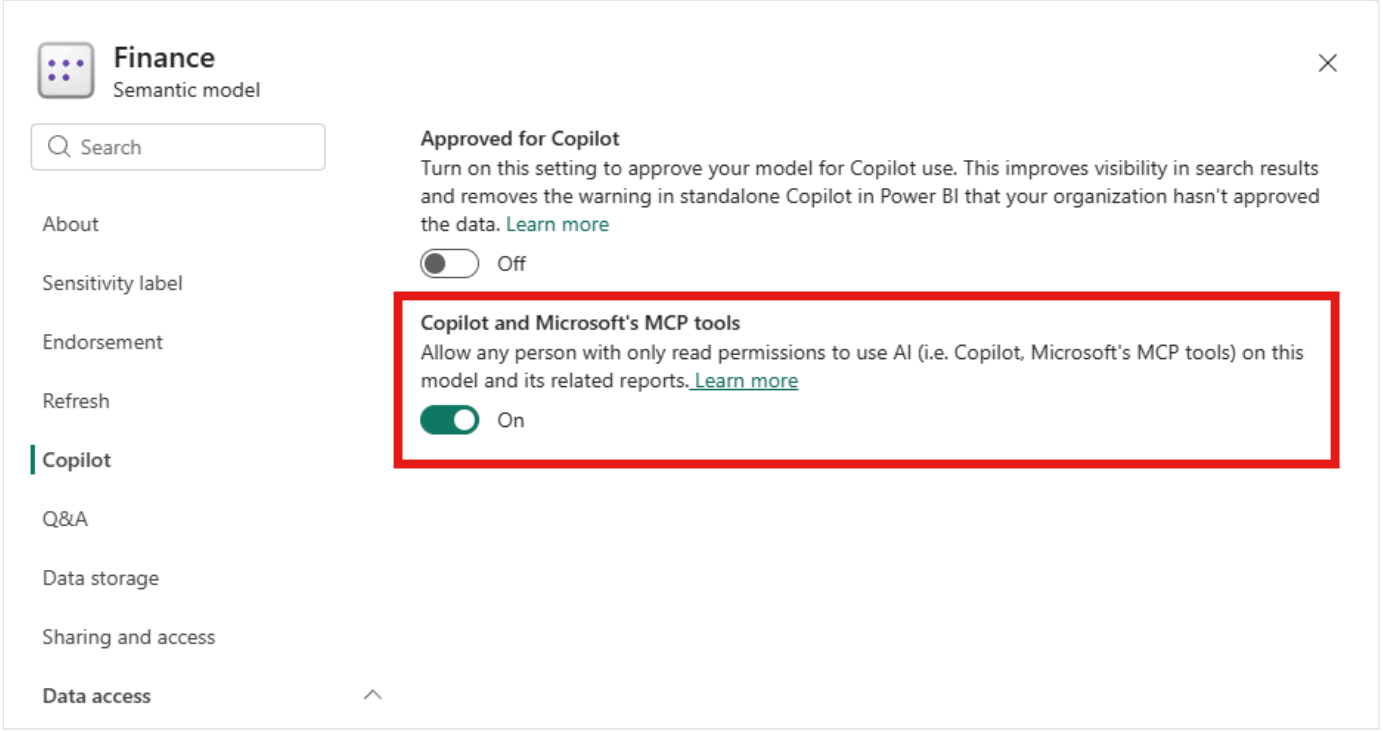
- **Analyze the model for problems**, such as inconsistent naming conventions and unclear structure, and suggest best-practice improvements.
- **Apply schema updates directly**, including renaming tables and columns, creating relationships, and generating DAX measures.

Changes that Copilot proposes or applies in this experience affect the active semantic model just like any other manual edit. Review all suggested updates carefully before accepting them, as you would for any model change. The optimization guidance described in Optimize your semantic model for Copilot in Power BI also applies here: a well-structured, clearly named model produces better Copilot analysis and more accurate suggestions.

Control Copilot access for semantic models

Users with write permissions on a semantic model can control whether users with read-only permissions can access the model through Copilot experiences. This setting can help organizations meet governance and data access requirements while ensuring that Copilot is available only for semantic models intended for Copilot consumption.

This setting is located in the **Copilot** section of the semantic model level settings pane with the description "Allow any person with only read permissions to use AI (for example, Copilot, Microsoft's MCP tools) on this model and its related reports". You can also find this setting on the full semantic model settings page under the **Explore and AI access** section.



This setting applies across the following experiences:

- Power BI Copilot consumption surfaces including report pane, standalone, apps
- M365 Copilot Chat and Cowork
- Data Agent
- Remote Microsoft's MCP tools

By default, this setting is enabled, allowing users with read-only permissions to use supported Copilot and Microsoft MCP experiences to access the semantic model.

When you disable this semantic model-level setting:

- Users with read-only permissions can't access the semantic model through any supported Copilot or remote Microsoft MCP tool experiences. The Copilot entry points are hidden, disabled, or return an access message depending on the experience.
- In experiences that support multiple semantic models, users can continue using Copilot with other eligible semantic models that have the setting enabled.
- In experiences that support multiple semantic models, restricted models and reports are excluded from search results for all users including users with build and write permissions. Users with build and write permissions can still access restricted models and reports through direct access methods, such as navigating to the model or report URL or manually attaching the model or report.

> ⓘ **Note**
>
> Copilot access settings aren't inherited across semantic models. If a semantic model is built on another semantic model, configure Copilot access separately for each model.

Consume a semantic model by using Copilot

You can use Copilot to pose data questions to your semantic models during consumption. This experience is available whenever you use a report, including in Power BI Desktop, a published report in a workspace, app, or OrgApp item, or in the Power BI mobile app.

Consumers might ask data questions of a semantic model in the following scenarios:

- They can't find the information or analysis that they need in their reports.
- They want to see data presented in a different way, and personalize visuals isn't enabled.
- They want to ask a data question using natural language, rather than using tools or code.

> ⓘ **Note**

See **Ask data questions** earlier in this article about developing a semantic model with help from Copilot. The images and guidance there also apply when using the ask data questions experience to consume a semantic model.

For more information about how you can use Copilot in reports, which consume a semantic model, see the article Use Copilot with Power BI reports.

Prepare a semantic model for Copilot consumption

Use Copilot to consume semantic models only after you take the necessary steps to update your data model to work well with Copilot for Power BI.

> **Tip**
>
> Consider using **tags to label semantic models** as ready for Copilot consumption. This method helps data consumers identify models that they can use with Copilot and expect better results. Alternatively, consider the readiness of a semantic model for use with Copilot as a criteria for its **endorsement** to promoted or even **certified** status.
>
> If your models aren't ready for use with Copilot in Power BI, but users still want the flexibility to interrogate the data themselves, consider using **personalize visuals**. Alternatively, you can show users how to use **explorations of the data** or to **connect to the semantic model** from Power BI Desktop or Excel to create their own reports.

Power BI Desktop

In Power BI Desktop, use the Copilot chat pane to ask data questions about your semantic model. This feature works with both local models open in Power BI Desktop and when you're connected to a shared semantic model by using a live connection. For more information, see *Ask data questions* earlier in this article.

You can also use the DAX query view to consume a semantic model by generating DAX queries. Advanced users of Power BI might use this approach if they prefer to explore data by using code.

However, for most data consumers, it's generally more convenient and more efficient to consume and explore a semantic model by adding visuals to the report canvas or by using other items like explorations or Analyze-in-Excel pivot tables. These items typically have a simpler user interface and user experience that's better suited to exploring and understanding data than natural language chat-like experiences, such as Copilot.

> **Tip**
>
> You can use Copilot experiences in Power BI when connected to semantic models that are published to Pro or PPU workspaces. The only requirement is that you configure Power BI Desktop to consume Copilot from a workspace assigned to a paid Fabric capacity (F2 or higher) or Power BI Premium capacity (P1 or higher); this requirement occurs independently of which semantic model that you connect to and consume.

Published report

In a published report, use the Copilot chat pane to ask data questions about the connected semantic model. This experience in Fabric is identical to the *Ask data questions* experience in Power BI Desktop for both model development and consumption.

> **Note**
>
> Slight differences might exist between Copilot experiences in Fabric and in Power BI Desktop. Eventually, over time, these experiences will have parity and work the same way.

Power BI mobile app

In the Power BI mobile app, use the Copilot chat pane in the app to ask data questions for any report. The questions go to the connected semantic model for that report. This feature works similarly to the *Ask data questions* experience explained earlier in this article.

Related content

- Update your data model to work well with Copilot for Power BI
- Overview of Copilot for Power BI
- Copilot in Power BI integration

Note: The author created this article with assistance from AI. Learn more

Last updated on 09/29/2026

Use Copilot with Power BI reports and semantic models

You can use Copilot within Power BI reports and semantic models for various tasks. Both business users and report creators can benefit from Copilot, whether they're asking questions about the data in an open report, kick-starting the creation of new reports, or enhancing their models.

Capabilities for business users

Generate a summary of your report

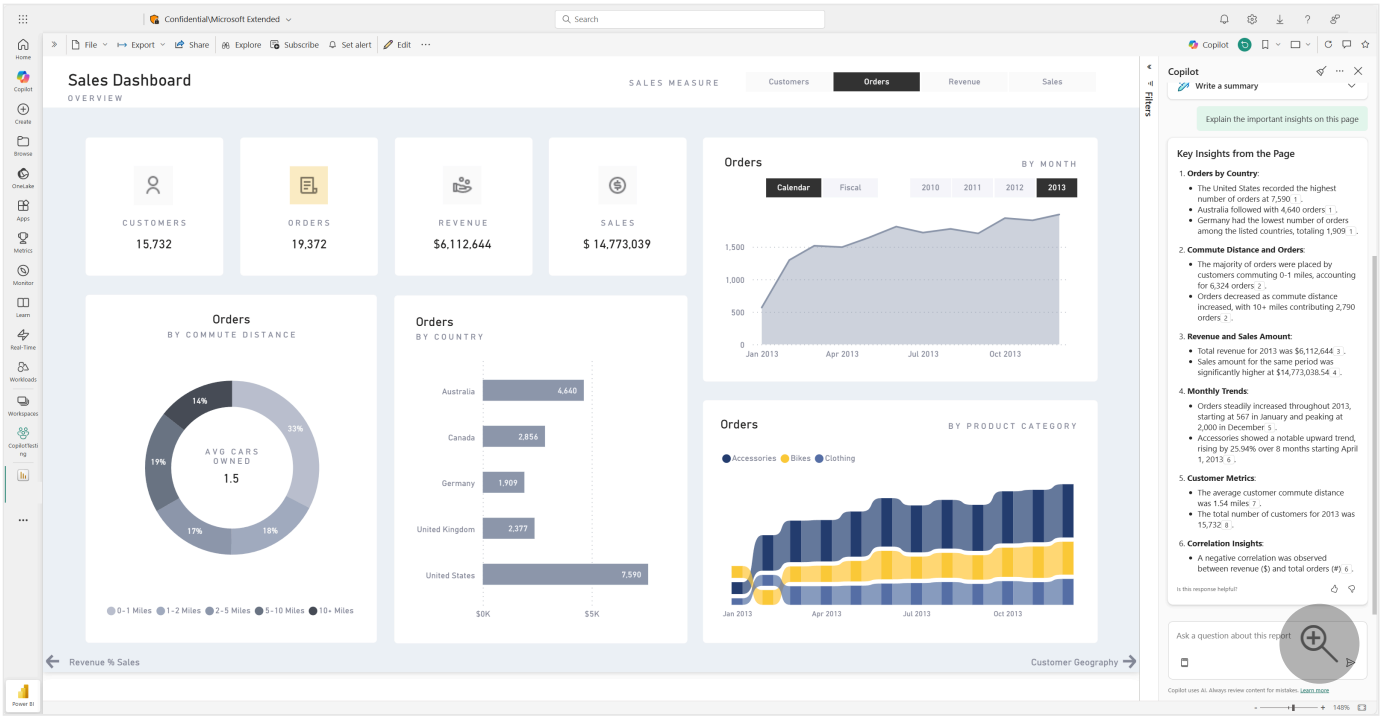
You can summarize report pages by using Copilot in smart narrative visuals on the page, in the Copilot pane, or in report subscriptions. You have the flexibility to refine or guide the summary by customizing prompts, such as "summarize this page using bullet points" or "provide a summary of sales on this page."

You can also ask specific questions about the visualized data on a report page and receive a tailored response. This response includes references to specific visuals, which helps you understand the specific data sources contributing to each part of the answer or summary within the report.

Consumers might use report page summaries in the following scenarios:

- Reports are complex or difficult to understand and consumers want an explanation.
- Consumers have challenges with visual or data literacy and want help.
- Consumers want a natural language summary to help them think through insights.

Learn more about Copilot creating a summary response to prompts about your report.



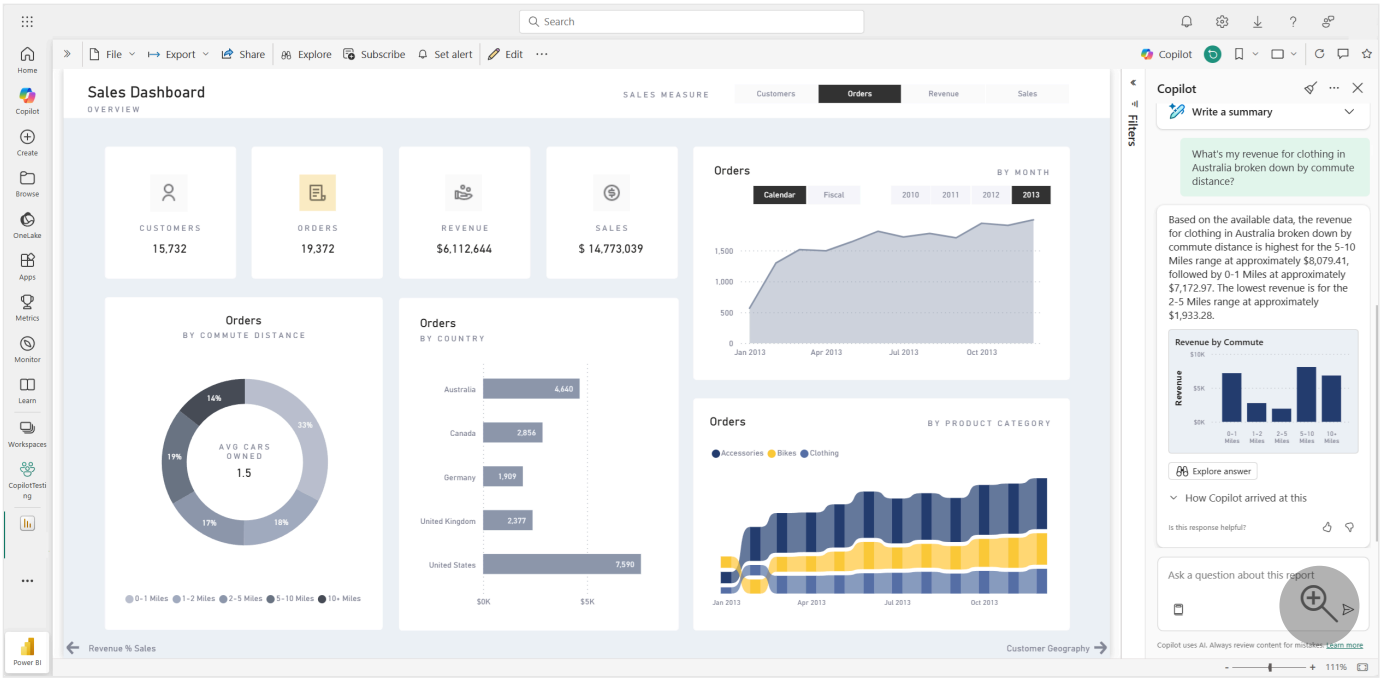
Ask questions about your data

Consumers can ask questions about reports and get answers in the Copilot pane. Copilot can find specific data points for the user and return them as an output, referencing the source visual that it retrieved from.

Copilot can also use measures and columns in your semantic model to help you explore your data in new ways. As you ask questions about your reports and models, Copilot checks if the answer can be found in the report visuals. If existing visuals don't answer the question, Copilot then builds a visual for you using the model.

Here are examples of the type of requests that Copilot can help you with:

- Can I see this measure filtered to a different region or span of time?
- What are the top categories by a certain metric?
- Show me how this measure has changed over time.



Consumers might ask data questions of a semantic model in the following scenarios:

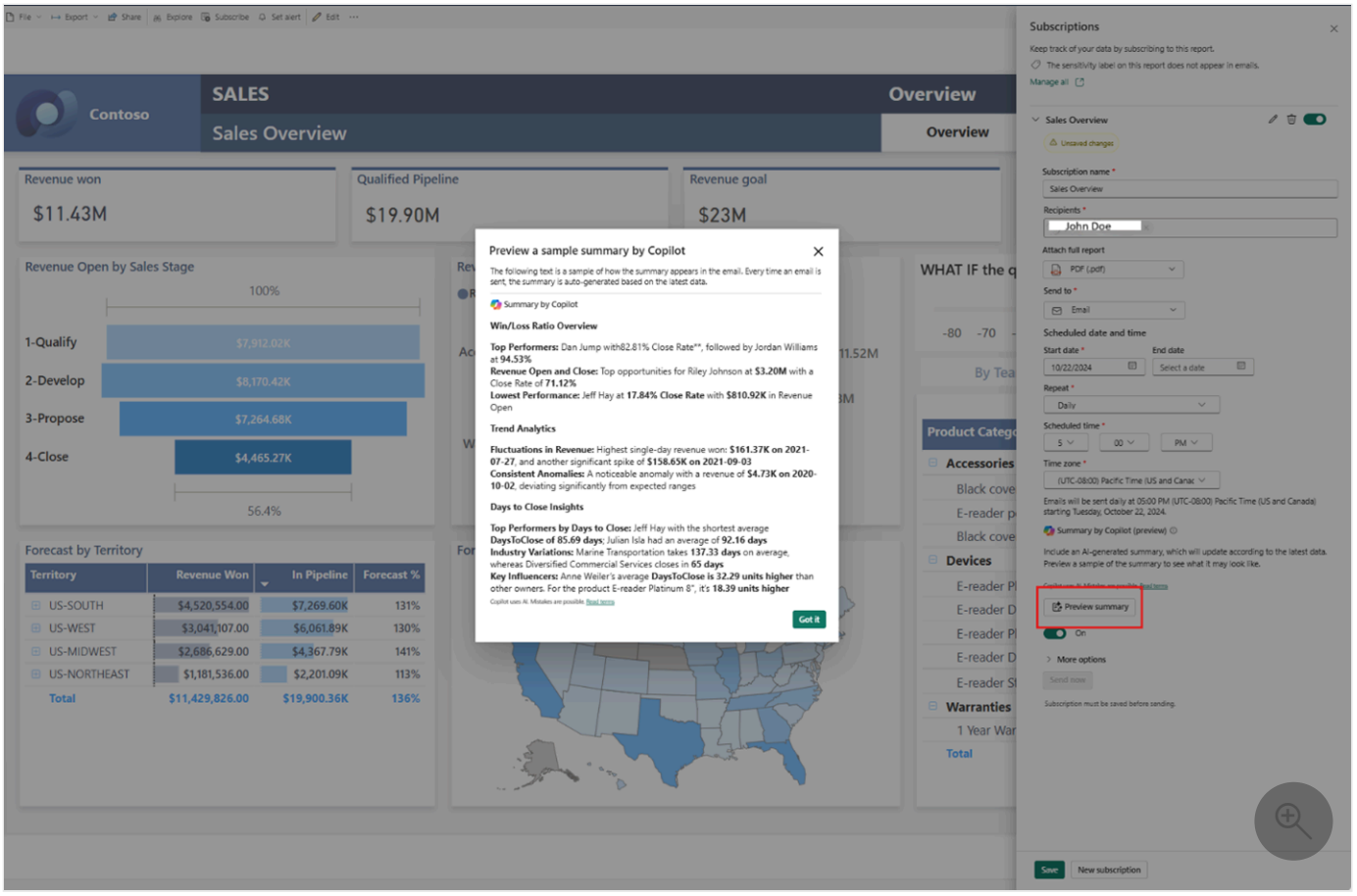
- They can't find the information or analysis that they need in the report.
- They want to see data presented in a different way, and personalize visuals isn't enabled.
- They want to ask a data question using natural language, rather than using tools or code.

See Ask Copilot for data in your semantic model for details.

Create report subscriptions with Copilot summaries

You can include a Copilot-generated summary for your Power BI report page or full report attachment in your email subscription to get insights. All the recipients of the email subscription receive the Copilot summary in the email. This feature is currently supported only for Standard Subscriptions.

See Create report subscriptions with Copilot summaries for details.

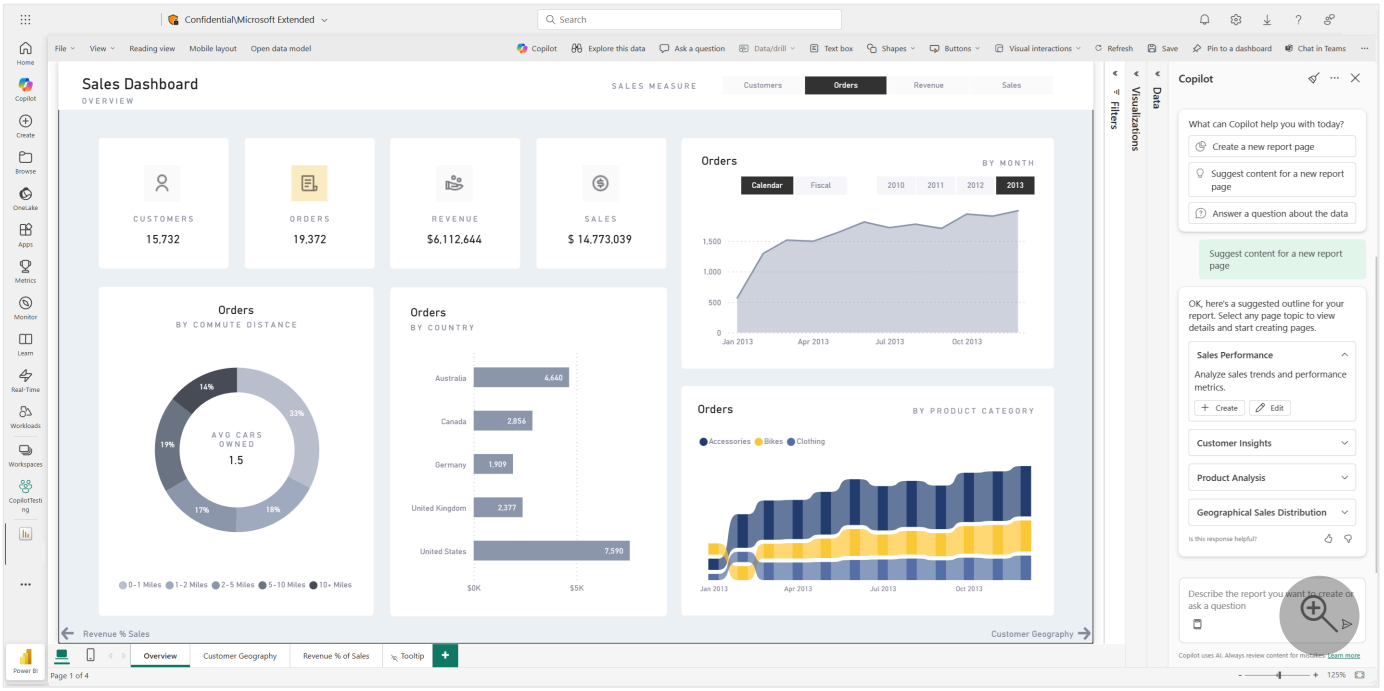


Capabilities for report authors

Suggest content for a report

Copilot can help you get started on a new report by suggesting topics based on your data. When you select this option directly in the chat, Copilot evaluates the data and provides a report outline with suggested pages that you can explore and choose to create for you.

- A report outline of suggested pages for your report: for example, what each page in the report is about, and how many pages it creates.
- The visuals for the individual pages.



Create a report page

Copilot for Power BI can help you create a report page by identifying the tables, fields, measures, and charts for your data. If you give Copilot a high-level prompt that's specific to your data, it can generate a report page that you can then customize and modify, using the existing editing tools. Copilot can help you get started on your report page quickly and save you a lot of time and effort in the process.

Here are some examples of high-level prompts to get you started:

- Create a page to evaluate the performance of different shifts based on good count, reject count, and alarm count over time.
- Create a page to analyze the efficiency of the production line and overall equipment effectiveness.
- Create a page to compare the cost and material of each product and their impact on production.

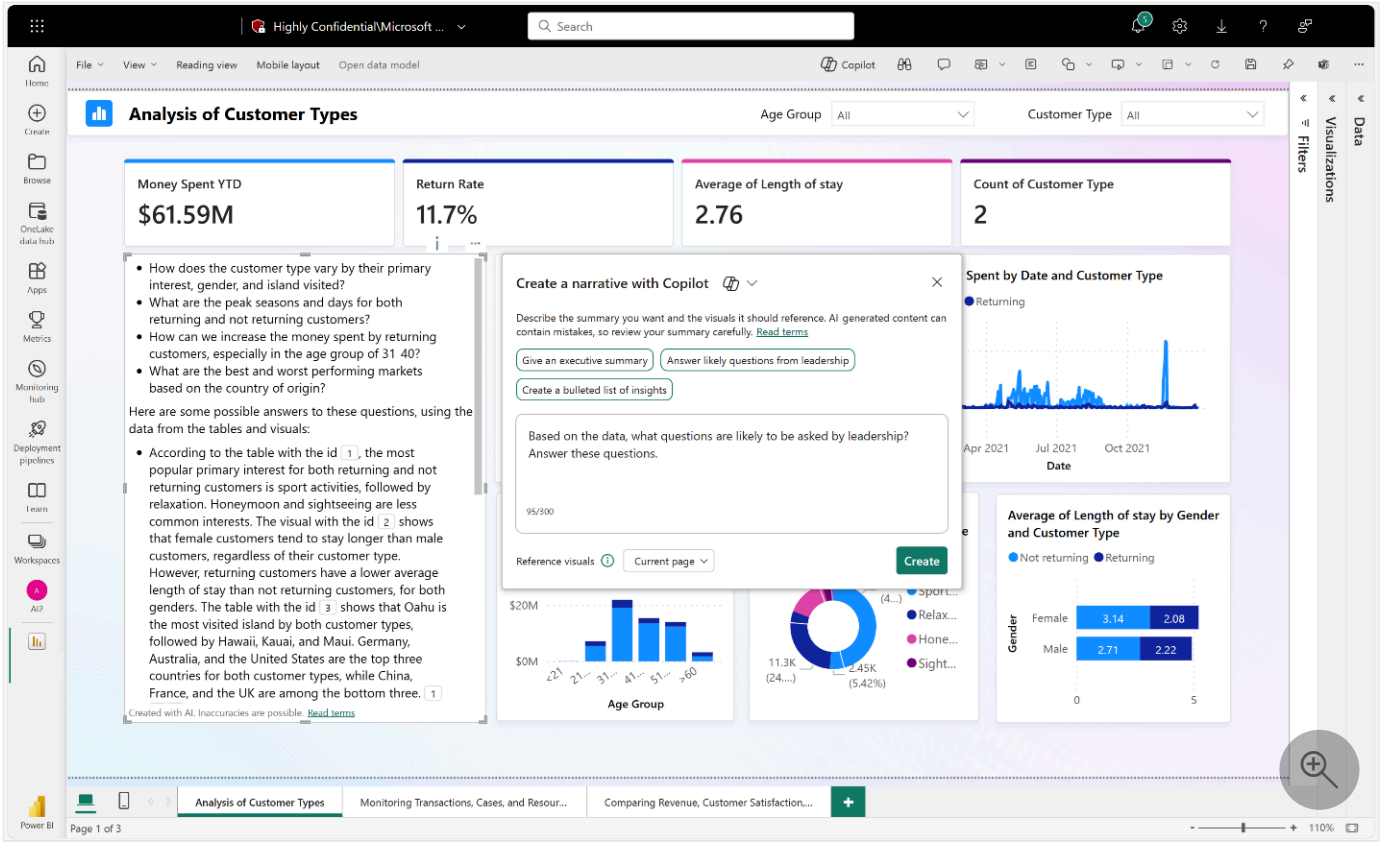
If you need help with writing prompts that get you the report page you want, see Write Copilot prompts that produce results in Power BI for guidance.

Summarize the underlying semantic model

Summarize a Power BI semantic model by asking Copilot to "summarize the model" in the Copilot pane. This summary can help you gain a better understanding of data in your semantic model, identify important insights, and improve your data exploration experience. Ultimately, better understanding the semantic model can help you build more meaningful reports.

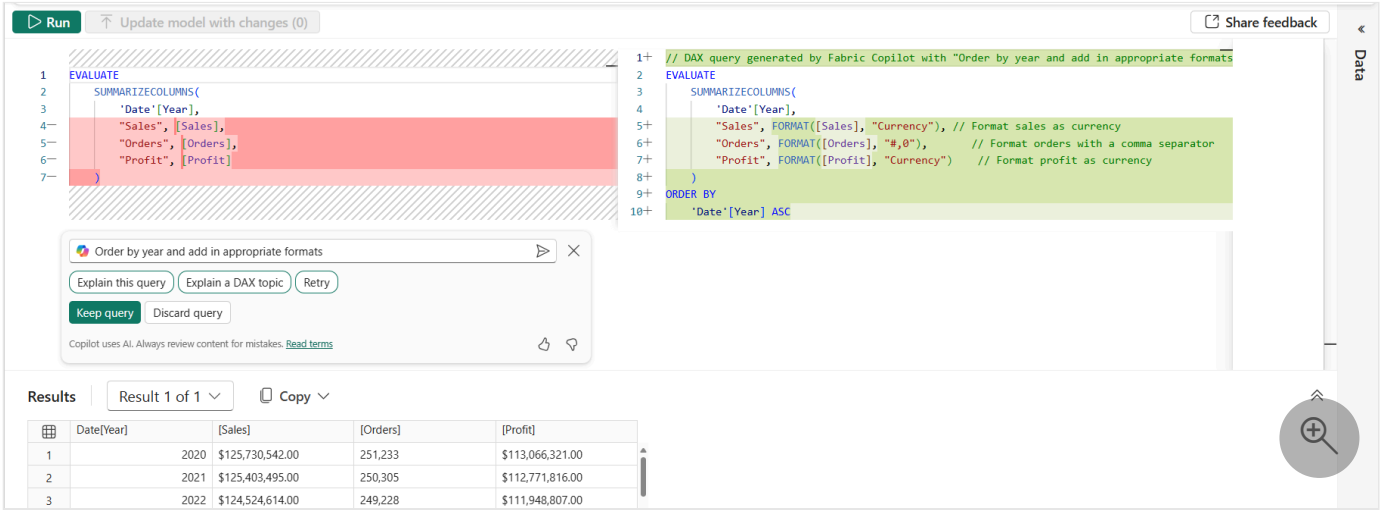
Create a summary visual on the report itself

Use Copilot for Power BI to quickly create a narrative about a report page with just a few clicks. This narrative can summarize the entire report, specific pages, or even specific visuals that you select. For more information, see Create a narrative with Copilot for Power BI.



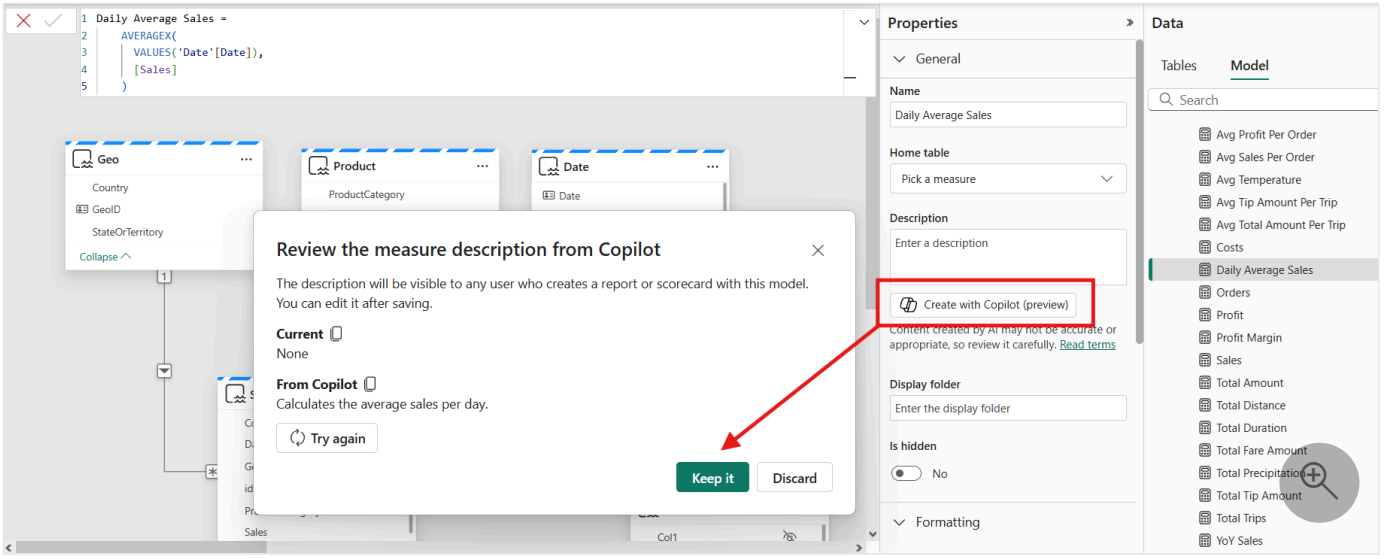
Write DAX queries

Copilot can write a data analysis expression (DAX) query. For example, you can type in a prompt to describe what DAX query you want it to generate, and select **Send** or press **Enter**. To run the returned query, select **Keep it** to add it to the query tab. Then select **Run** or press **F5** to see the results of the DAX query. For more information, see Write DAX queries.



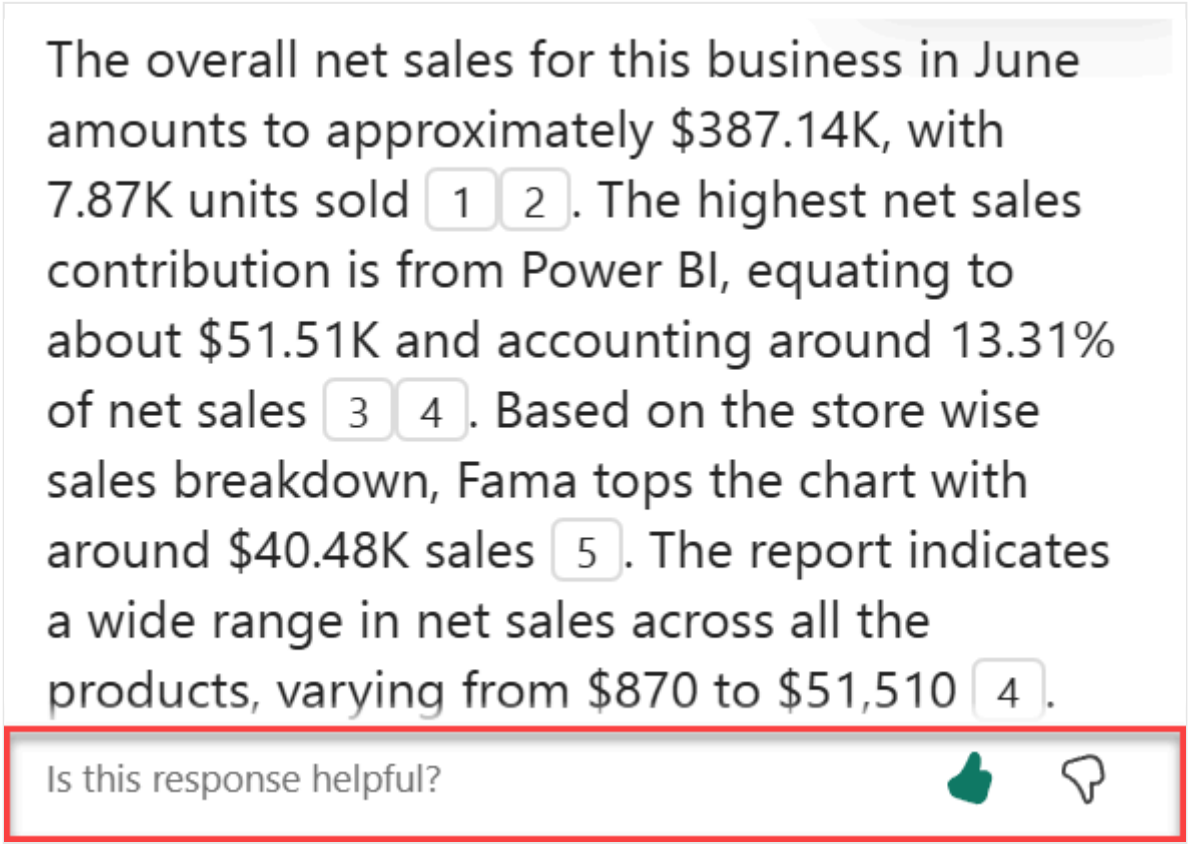
Add descriptions for semantic model measures

Copilot can add descriptions to your semantic model measures. People who build reports from your semantic model can see the name and description of your measures, which makes the description property essential documentation. Use Copilot to create measure descriptions.



Send feedback

To help us improve Copilot, provide your feedback about our products. Give us your feedback directly in the Copilot pane by selecting the feedback button.



Copilot requirements

Read about requirements to use Copilot with reports and semantic models.

ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 05/26/2026

Copilot for Microsoft Fabric and Power BI: FAQ

This article answers frequently asked questions about Copilot for Microsoft Fabric and Power BI.

> **Note**
>
> - Your administrator needs to enable the tenant switch before you start using Copilot. See the article Copilot tenant settings for details.
> - Your F or P capacity needs to be in one of the regions listed in this article, Fabric region availability.
> - If your tenant or capacity is outside the US or France, Copilot is disabled by default unless your Fabric tenant admin enables the Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance tenant setting in **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**.
> - Copilot in Microsoft Fabric isn't supported on trial SKUs. Only paid SKUs (F2 or higher, or P1 or higher) are supported. If you're using an Azure subscription with monthly credits (such as through a Visual Studio/MSDN benefit), you may not have access to the required SKUs unless billing is enabled and the appropriate capacity is provisioned.
> - For more information, see the article Overview of Copilot in Fabric.

Power BI

Can Copilot be enabled for specific workspaces within a tenant?

Copilot is enabled at the tenant level and access can be restricted by security groups. If the workspace is tied to an F2 or P1 capacity, Copilot experience will be enabled.

When you're using Copilot, who has access to what data?

The data that Copilot can access depends on your role-level security and user-based permission on Fabric.

If you don't have permission to access specific data, then prompting Copilot for it won't retrieve the information.

Can Copilot prompts be saved for future reference?

Copilot prompts can't be saved for future reference. The only experience where it's possible to view your prompts is by using the chat-magic function in notebooks.

Does enabling Copilot and agreeing to the setting of "Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance" mean all my data is sent or processed outside my country/region?

Not exactly. While the prompt itself is sent to Azure OpenAI, it doesn't mean your data is sent or processed outside your country/region.

The prompt isn't used to train any models.

I loaded my semantic model, but it doesn't meet all the criteria listed in the data evaluation. What should I do?

The criteria listed in Update your data model to work well with Copilot for Power BI is important because it helps you get a better quality report. As long as you meet seven of the eight points, including Consistency, the quality of the reports generated should be good.

If your data doesn't meet that criteria, we recommend spending the time to bring it into compliance.

I was given a Copilot URL, but I can't see the Copilot button. Why is that?

First, check with your admin to see if they have enabled Copilot.

Next, when you select a Copilot-enabled URL, you have to initially load the semantic model. When you've completed loading the semantic model, then you see the Copilot button.

I selected the Copilot button, and it's stuck on Analyzing your semantic model.

Depending upon the size of the semantic model, Copilot might take a while to analyze it. If you've waited longer than 15 minutes and you haven't received any errors, chances are that there's an internal server error.

Try restarting Copilot by closing the pane and selecting the Copilot button again.

I loaded the semantic model and Copilot generated a summary, but I don't think that it's accurate.

This inaccuracy could be because your semantic model has missing values. Because AI is generating the summary, it can try to fill the holes and fabricate data. If you can remove the rows with missing values, this situation could be avoided.

I generated the report visuals, but the quality of the visuals concern me. I wouldn't choose them myself.

We're continuously looking to improve the quality of the Copilot-generated visuals. For now, we recommend that you make the change by using the Power BI visualization tool.

The accuracy of the narrative visual concerns me.

We're continuously working to improve the accuracy of the narrative visual results. We recommend using the custom prompts as an extra tool to try to tweak the summary to meet your needs.

I want to disable Copilot immediately as I'm concerned with the data storage you mentioned.

Contact your help desk to get support from your IT admin.

Why did Copilot respond with general knowledge data?

When a question is related to data in the semantic model, Copilot will use the semantic model to answer the question, otherwise it may answer from the LLM's general knowledge.

I want to suggest new features. How can I do that?

You can submit and vote on ideas for Microsoft Fabric on the Ideas page of the Fabric Community. Read more about giving feedback in the Learn about Microsoft Fabric feedback article.

Real-Time Intelligence

Does Copilot respond to multiple questions in a conversation?

No, Copilot doesn't answer follow-up questions. You need to ask one question at a time.

How can I improve the quality of the Copilot answer?

Provide any tips or relevant information in your question. For example, if you're asking about a specific column, provide the column name and the type of data it contains. If you want to use specific operators or functions, this will also help. The more information you provide, the better the Copilot answer will be.

What access level do I need on a KQL queryset to use Copilot?

You need read access to the KQL queryset to use Copilot. In order to insert and execute the Copilot-generated query in the KQL queryset, you need to have write access to that KQL queryset.

What database does the Copilot-generated query run against?

The Copilot-generated query runs against the database that the KQL queryset is connected to. If you want to change the database, you can do so in the KQL queryset.

Related content

- What is Microsoft Fabric?
- Privacy, security, and responsible use of Copilot in Fabric

---

Last updated on 09/29/2026

Release status of AI and Copilot experiences in Fabric

The following table provides an overview of the AI and Copilot experiences available in Fabric, the workloads and items they apply to, and the release status of each capability. The links in the *Copilot experience* column take you to documentation that describes how to set up and use each feature.

Expand table

Fabric workload	Supported items	Copilot experience	Release status
Data Science and Data Engineering	Notebook (typically with lakehouses and other data items)	- Copilot chat pane in a notebook, with notebook-wide code generation, refactoring, and validation. - Generate code or markdown across notebook workflows (preview). - Add comments, fix errors, or optimize notebook code (preview). - Analyze and visualize data. - Fix with Copilot (preview): error summaries, root-cause analysis, and approval-based code fixes from the chat pane or in-cell Copilot.	Preview
Data Science	Data agent, AI functions, AI services	- Create a Fabric data agent. - Transform and enrich data seamlessly with AI functions. - Foundry Tools in Fabric.	Preview
Data Factory	Dataflows gen2	- Generate a new query.	GA
	Pipeline	- Generate and run a pipeline. - Summarize a pipeline. - Troubleshoot pipeline errors.	GA
Data Warehouse	SQL Queries in Data Warehouse	- Generate SQL queries (preview). - Suggest SQL code completions (preview). - Fix code in SQL queries (preview). - Explain code in SQL queries (preview).	Preview
SQL database	SQL queries in SQL database (Fabric portal Query Editor, SQL	- Generate SQL queries. - Suggest SQL code completions.	GA

Fabric workload	Supported items	Copilot experience	Release status
	Server Management Studio (SSMS), MSSQL extension for Visual Studio Code)	- Fix code in SQL queries. - Explain code in SQL queries. - Inline T-SQL completions, chat-based code generation, and execution plan analysis when connected via SQL Server Management Studio (SSMS) or MSSQL extension for Visual Studio Code.	
Power BI	Semantic models in Power BI Desktop or Power BI service	- Suggest linguistic model synonyms. - Suggest measure descriptions. - Write and explain DAX queries. - Ask questions about your data. - AI Auto-Summary for semantic models (preview).	GA
	Reports (Power BI Desktop, service, or mobile app)	- Power BI home (announced). - Suggest a report page. - Suggest a visual. - Summarize data in a narrative visual. - Explain a report page or visual.	GA
Real-Time Intelligence	KQL queryset	- Copilot for Writing KQL Queries (preview). - Modify or explore a previously generated KQL query.	GA
	Real-time dashboards	- Generate a real-time dashboard. - Explore data in real-time dashboards with Copilot	GA
	Operations agent	- Get started with operations agent.	GA

Last updated on 07/12/2026

Troubleshoot Copilot unavailability after a trial expires

You might see Copilot become unavailable in Power BI when your workspace no longer has the required capacity assigned. This situation can happen after a Microsoft Fabric trial expires, or when you try to use Copilot in a workspace that isn't backed by a supported paid capacity.

This issue often affects you if you expect per-user licenses alone to enable Copilot. Copilot access depends on your workspace or account having a supported capacity, not on a per-user license.

This article helps you confirm whether a capacity change caused the problem and work through the checks needed to restore Copilot access.

Prerequisites

- Access to the Power BI or Fabric portal.
- A workspace where you want to use Copilot.
- Admin, member, or contributor access to at least one workspace if you're using Copilot in Power BI Desktop.

Symptoms

You experience one or more of the following symptoms:

- Copilot isn't available in your workspace, and you see a message similar to the following:

  > Copilot isn't available because none of your workspaces have the necessary capacity assigned. Contact your administrator or learn how Copilot access works.

- Copilot features that worked previously stop working after your Fabric trial expires.
- You lose editing or sharing capabilities and revert to Fabric (Free) because your trial or paid license expired or was removed.

Cause

Copilot access requires a supported capacity. You can lose access when:

- Your Fabric trial expires. Fabric revokes access to the trial capacity and reassigns affected workspaces to Pro. For details, see When your Fabric trial ends.
- Your workspace isn't assigned to a supported capacity. For the list of supported options, see Step 2 in the troubleshooting checklist.
- Your Power BI Pro or Premium Per User (PPU) license expires or is removed, which reduces your Power BI capabilities.

Troubleshooting checklist

Work through these steps to identify why Copilot is unavailable.

Step 1: Check whether your Fabric trial expired

Notifications appear in the Fabric portal and in **OneLake catalog** > **Govern** > **Capacities** as a trial nears expiration. To check the current status, go to **OneLake catalog** > **Govern** > **Capacities** > **Trial**.

Step 2: Verify that your workspace has supported capacity

Confirm that your workspace is assigned to one of these supported capacity options:

- A paid Fabric capacity (F2 or higher).
- A supported Power BI Premium capacity (P1 or higher) with Copilot enabled.
- A Fabric Copilot capacity.

For guidance on assigning a workspace to a Copilot-enabled capacity, see Enable and configure Copilot in Microsoft Fabric.

Step 3: Check whether a Fabric Copilot capacity is assigned to you

If you aren't using a specific workspace capacity, verify with your administrator whether your organization assigned you to a Fabric Copilot capacity. When you're assigned to a Fabric Copilot capacity, you don't need to take other steps to use Copilot. For more information, see Fabric Copilot capacity.

Step 4: Review your current license state

If you lose editing or sharing capabilities, check whether your Pro or PPU license expired or was removed. If it did, you might have reverted to Fabric (Free). To restore access, see Solution 3.

Solution

Pick the solution that matches what you found in the checklist.

Solution 1: Reassign the workspace to a supported capacity

1. Check whether the workspace was previously using a Fabric trial capacity.
2. If the trial expired, reassign the workspace to a supported capacity. For steps, see Assign workspaces and provision access.
3. Use one of the supported capacity options listed in Step 2.

Solution 2: Start a new trial or purchase a paid capacity

1. If your tenant already has active trial capacities, assign the workspace to one of them through **Capacity settings**.
2. If trial options aren't available, switch to a paid option such as an Azure pay-as-you-go Fabric capacity, an F SKU, or a Power BI Premium P SKU. For details, see Buy Fabric capacity in Azure.
3. If applicable in your tenant, ensure the **Users can create Fabric items** setting is enabled so you can create a trial.

Solution 3: Restore your license or request Copilot capacity assignment

1. If your Pro or PPU license expired or was removed, restore it or start an active Fabric capacity trial.
2. If you're using Copilot in Power BI Desktop, make sure you have admin, member, or contributor access to at least one supported workspace.
3. If your organization uses Fabric Copilot capacity, ask your administrator to assign you to that capacity.

Related content

- Enable and configure Copilot in Microsoft Fabric
- Fabric Copilot capacity
- Try Microsoft Fabric for free
- Copilot for Microsoft Fabric and Power BI: FAQ
- Enable Copilot for Power BI
- Copilot in Power BI tutorial: Get started

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/29/2026

Privacy, security, and responsible AI use of Copilot in Fabric

Before your business starts using Copilot in Fabric, you might have questions about how it works, how it keeps your business data secure and adheres to privacy requirements, and how to use generative AI responsibly.

This article provides answers to common questions related to business data security and privacy to help your organization get started with Copilot in Fabric. The article Privacy, security, and responsible use for Copilot in Power BI (preview) provides an overview of Copilot in Power BI. Read on for details about Copilot for Fabric.

> **Note**
>
> - Your administrator needs to enable the tenant switch before you start using Copilot. See the article **Copilot tenant settings** for details.
> - Your F2 or P1 capacity needs to be in one of the regions listed in this article, **Fabric region availability**.
> - If your tenant or capacity is outside the US or EU, Copilot is disabled by default. Your Fabric tenant admin will also need to enable the **Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance** tenant setting in **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings** to use Copilot.
> - Copilot in Microsoft Fabric isn't supported on trial SKUs. Only paid SKUs (F2 or higher, or P1 or higher) are supported.
> - See the article **Overview of Copilot in Fabric and Power BI** for more information.

Business data is secure

- By default, Copilot features use Azure OpenAI Service, which is fully controlled by Microsoft. If your admin opts in to the OpenAI as a Microsoft Subprocessor tenant setting, designated capacities can use OpenAI as a Microsoft subprocessor instead. Your data isn't used to train models and isn't available to other customers.
- You retain control over where your data is processed. Data processed by Copilot in Fabric stays within your capacity's geographic region, unless you explicitly allow data to be

processed outside your region—for example, to let your users use Copilot when the underlying AI service isn't available in your region or availability is limited due to high demand. Learn more about admin settings for Copilot.

- Copilot doesn't store your data for abuse monitoring. To enhance privacy and trust, we've updated our approach to abuse monitoring: previously, we retained data from Copilot in Fabric, containing prompt inputs and outputs, for up to 30 days to check for abuse or misuse. Following customer feedback, we've eliminated this 30-day retention. Now, we no longer store prompt related data, demonstrating our unwavering commitment to your privacy and security.

Check Copilot outputs

- Copilot responses can include inaccurate or low-quality content, so make sure to review outputs before you use them in your work.
- People who can meaningfully evaluate the content's accuracy and appropriateness should review the outputs.
- Today, Copilot features work best in the English language. Other languages might not perform as well.

> **Important**
>
> Review the supplemental preview terms for Fabric, which includes terms of use for Microsoft Generative AI Service Previews.

How Copilot works

In this article, *Copilot* refers to a range of generative AI features and capabilities in Fabric that are powered by Azure OpenAI Service (or, if your admin opts in, OpenAI as a Microsoft subprocessor).

In general, these features are designed to generate natural language, code, or other content based on:

(a) inputs you provide, and,

(b) grounding data that the feature has access to.

For example, Power BI, Data Factory, and data science offer Copilot chats where you can ask questions and get responses that are contextualized on your data. Copilot for Power BI can also

create reports and other visualizations. Copilot for Data Factory can transform your data and explain what steps it has applied. Data science offers Copilot features outside of the chat pane, such as custom IPython magic commands in notebooks. Copilot chats might be added to other experiences in Fabric, along with other features that are powered by Azure OpenAI under the hood.

This information is sent to the AI service that powers Copilot (Azure OpenAI Service by default), where it's processed and an output is generated. Therefore, data processed by the service can include:

- The user's prompt or input.
- Grounding data.
- The AI response or output.

Grounding data might include a combination of dataset schema, specific data points, and other information relevant to the user's current task. Review each experience section for details on what data is accessible to Copilot features in that scenario.

Interactions with Copilot are specific to each user. This means that Copilot can only access data that the current user has permission to access, and its outputs are only visible to that user unless that user shares the output with others, such as sharing a generated Power BI report or generated code. Copilot doesn't use data from other users in the same tenant or other tenants.

By default, Copilot uses Azure OpenAI—not the publicly available OpenAI services—to process all data, including user inputs, grounding data, and Copilot outputs. If your Fabric administrator opts in to the OpenAI as a Microsoft Subprocessor tenant setting, designated capacities can instead use OpenAI as a Microsoft subprocessor. Unless your admin opts in to the subprocessor setting, Microsoft hosts the OpenAI models in the Microsoft Azure environment, and the service doesn't interact with any services by OpenAI, such as the OpenAI API. Your data isn't used to train models and isn't available to other customers. Learn more about Azure OpenAI.

Review the Copilot process

These features follow the same general process:

1. **Copilot receives a prompt from a user**. This prompt could be in the form of a question that a user types into a chat pane, or in the form of an action such as selecting a button that says "Create a report."
2. **Copilot preprocesses the prompt through an approach called grounding**. Depending on the scenario, this might include retrieving relevant data such as dataset schema or chat

history from the user's current session with Copilot. Grounding improves the specificity of the prompt, so the user gets responses that are relevant and actionable to their specific task. Data retrieval is scoped to data that is accessible to the authenticated user based on their permissions. See the section What data does Copilot use and how is it processed? in this article for more information.

3. **Copilot takes the response from Azure OpenAI and postprocesses it**. Depending on the scenario, this postprocessing might include responsible AI checks, filtering with Azure content moderation, or additional business-specific constraints.
4. **Copilot returns a response to the user in the form of natural language, code, or other content**. For example, a response might be in the form of a chat message or generated code, or it might be a contextually appropriate form such as a Power BI report or a Synapse notebook cell.
5. **The user reviews the response before using it**. Copilot responses can include inaccurate or low-quality content, so it's important for subject matter experts to check outputs before using or sharing them.

Just as each experience in Fabric is built for certain scenarios and personas—from data engineers to data analysts—each Copilot feature in Fabric has also been built with unique scenarios and users in mind. For capabilities, intended uses, and limitations of each feature, review the section for the experience you're working in.

Definitions

Prompt or input

The text or action submitted to Copilot by a user. This could be in the form of a question that a user types into a chat pane, or in the form of an action such as selecting a button that says "Create a report."

Grounding

A preprocessing technique where Copilot retrieves additional data that's contextual to the user's prompt, and then sends that data along with the user's prompt to Azure OpenAI in order to generate a more relevant and actionable response.

Response or output

The content that Copilot returns to a user. For example, a response might be in the form of a chat message or generated code, or it might be contextually appropriate content such as a Power BI

report or a Synapse notebook cell.

What data does Copilot use and how is it processed?

To generate a response, Copilot uses:

- The user's prompt or input and, when appropriate,
- Additional data that is retrieved through the grounding process.

This information is sent to the AI service that powers Copilot (Azure OpenAI Service by default), where it's processed and an output is generated. Therefore, data processed by the service can include:

- The user's prompt or input.
- Grounding data.
- The AI response or output.

Grounding data might include a combination of dataset schema, specific data points, and other information relevant to the user's current task. Review each experience section for details on what data is accessible to Copilot features in that scenario.

Interactions with Copilot are specific to each user. This means that Copilot can only access data that the current user has permission to access, and its outputs are only visible to that user unless that user shares the output with others, such as sharing a generated Power BI report or generated code. Copilot doesn't use data from other users in the same tenant or other tenants.

By default, Copilot uses Azure OpenAI—not OpenAI's publicly available services—to process all data, including user inputs, grounding data, and Copilot outputs. If your Fabric administrator opts in to the OpenAI as a Microsoft Subprocessor tenant setting, designated capacities can instead use OpenAI as a Microsoft subprocessor. Unless your admin opts in, Microsoft hosts the OpenAI models in Microsoft's Azure environment and the service doesn't interact with any services by OpenAI (such as the OpenAI API). Your data isn't used to train models and isn't available to other customers. Learn more about Azure OpenAI.

Abuse monitoring of prompts

Copilot and AI experiences in Fabric are currently **not** onboarded to any automated abuse monitoring of your prompts. This means that we currently do not process any customer prompts for monitoring of abusive content. This also means that no retention of prompts is in place for this purpose.

Data residency and compliance

You retain control over where your data is processed. Data processed by Copilot in Fabric stays within your capacity's geographic region, unless you explicitly allow data to be processed outside your region—for example, to let your users use Copilot when the underlying AI service isn't available in your region or availability is limited due to high demand. (See where Azure OpenAI is currently available.)

To allow data to be processed elsewhere, your admin can turn on the setting **Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance**. If you opted in to OpenAI as a Microsoft subprocessor, the equivalent setting is **Data sent to OpenAI as a Microsoft Subprocessor can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance**. Learn more about admin settings for Copilot.

What should I know to use Copilot responsibly?

Microsoft is committed to ensuring that our AI systems are guided by our AI principles and Responsible AI Standard. These principles include empowering our customers to use these systems effectively and in line with their intended uses. Our approach to responsible AI is continually evolving to proactively address emerging issues.

Copilot features in Fabric are built to meet the Responsible AI Standard, which means that they're reviewed by multidisciplinary teams for potential harms, and then refined to include mitigations for those harms.

Before you use Copilot, keep in mind the limitations of Copilot:

- Copilot responses can include inaccurate or low-quality content, so make sure to review outputs before using them in your work.
- People who are able to meaningfully evaluate the content's accuracy and appropriateness should review the outputs.
- Currently, Copilot features work best in the English language. Other languages might not perform as well.

Copilot for Fabric workloads

Privacy, security, and responsible use for:

- Copilot for Data Factory (preview)

- Copilot for Data Science (preview)
- Copilot in Fabric in Data Warehouse (preview)
- Copilot in Fabric in SQL database (preview)
- Copilot for Power BI
- Copilot for Real-Time Intelligence (preview)

Related content

- What is Microsoft Fabric?
- Copilot in Fabric and Power BI: FAQ

Last updated on 10/06/2026

Privacy, security, and responsible AI use of Copilot in Fabric in the SQL database workload

Applies to: ✅ SQL database in Microsoft Fabric

In this article, learn how Copilot in Fabric in SQL database works, how it keeps your business data secure and adheres to privacy requirements, and how to use generative AI responsibly. For more information on Copilot in Fabric, see Privacy, security, and responsible use of Copilot in Fabric.

With Copilot in Fabric SQL databases and other generative AI features, Microsoft Fabric brings a new way to transform and analyze data, generate insights, and create visualizations and reports in your database and other workloads.

For limitations, see Limitations of Copilot in Fabric in SQL database.

Data use of Copilot in Fabric in SQL database

In database, Copilot can only access the database schema that is accessible in the user's database.

By default, Copilot has access to the following data types:

- Previous messages sent to and replies from Copilot for that user in that session.
- Contents of SQL query that the user has executed.
- Error messages of a SQL query that the user has executed (if applicable).
- Schemas of the database.

Tips for working with Copilot in Fabric SQL databases

- Copilot is best equipped to handle SQL database articles, so limit your questions to this area.

- Be explicit about the data you want Copilot to examine. If you describe the data asset, with descriptive table and column names, Copilot is more likely to retrieve relevant data and generate useful outputs.

Evaluation of Copilot in Fabric SQL databases

The product team tested Copilot to see how well the system performs within the context of databases, and whether AI responses are insightful and useful.

The team also invested in additional harm mitigation, including technological approaches to focusing Copilot's output on articles related to SQL databases.

Related content

- Privacy, security, and responsible use for Copilot in Microsoft Fabric
- Copilot in Fabric in SQL database in Fabric

Last updated on 11/18/2025

Privacy, security, and responsible use of Copilot in Data Factory

This article describes how Copilot in Data Factory handles your business data, adheres to privacy requirements, and supports responsible use of generative AI. For an overview of these topics for Copilot in Fabric, see Privacy, security, and responsible use for Copilot.

For considerations and limitations, see Limitations of Copilot in Data Factory.

Data that Copilot in Data Factory can access

Copilot in Data Factory can only access data that meets both of the following conditions:

- The data is accessible to your current Dataflow Gen2 session.
- The data is configured and imported into the Power Query data preview grid.

Copilot doesn't access data outside your active Dataflow Gen2 session or data that hasn't been loaded into the preview grid.

How Copilot in Data Factory is evaluated

- The product team tested Copilot in Data Factory to validate the accuracy and usefulness of AI-generated responses within Dataflow Gen2.
- The team applied harms mitigations, including content filters that focus Copilot output on data integration topics and reduce off-topic or harmful responses.

Tips for working with Copilot in Data Factory

- **Limit prompts to data integration topics.** Copilot in Data Factory is optimized for data integration tasks in Dataflow Gen2 and produces the best results in that area.
- **Include specific data details in your prompts.** Provide query names, column names, and data values so Copilot can generate more accurate and relevant outputs.
- **Break complex requests into smaller tasks.** Splitting a complex transformation into individual steps helps Copilot better understand your requirements and produce more accurate results.

Related content

- Copilot in Data Factory overview
- Copilot in Fabric: FAQ

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/11/2026

Privacy, security, and responsible use of Copilot in notebooks and Fabric data agents

Summarize this article for me

In this article, learn how Microsoft Copilot in Notebooks and and Fabric data agents (formerly known as Data agent) works, how it keeps your business data secure and compliant with privacy requirements, and how to responsibly use generative AI. For an overview of these topics for Copilot in Fabric, see Privacy, security, and responsible use for Copilot (preview).

Data use

Data use in Copilot for notebooks in Fabric

- In notebooks, Copilot can only access data that is accessible to the user's current notebook, either in an attached lakehouse or directly loaded or imported into that notebook by the user. In notebooks, Copilot can't access any data that's not accessible to the notebook.

- By default, Copilot has access to the following data types:
  - Conversation history: Previous messages sent to and replies from Copilot for that user. (see below for more details about storing conversation history)
  - Contents of cells that the user executed.
  - Outputs of cells that the user executed.
  - Schemas of data sources in the notebook.
  - Sample data from data sources in the notebook.
  - Schemas from external data sources in an attached lakehouse.

Data use in Fabric data agents

- Fabric data agents rely on the user's conversation history to better respond to the user questions. (see below for more details about storing conversation history)
- Schema information of the data sources added. This includes table and Column names. (The creator of a data agent selects which tables that should be included.)

How we handle conversation history

For Copilot in Notebooks and Fabric data agents, we store conversation history across user sessions.

Why do we store conversations history and where is it stored?

In order to use fully conversational agentic AI experiences, the agent needs to store conversation history across user sessions to maintain context. This ensures that the AI agent keeps context about what a user asked in previous sessions and is typically a desired behavior in many agentic AI experiences. Experiences such as Copilot in Notebooks and Fabric data agents are AI experiences that store conversation history across user's sessions.

This history is stored inside the Azure security boundary, in the same region and in the same Azure OpenAI resources that process all your Fabric AI requests. The difference in this case is that the conversation history is stored for as long as the user allows. For experiences that don't store conversation history across sessions, no data is stored. Prompts are only processed by Azure OpenAI resources that Fabric uses.

Your users can delete their conversation history at any time, simply by clearing the chat. This option exists both for Copilot in Notebooks and data agents. If the conversation history isn't manually removed, it is stored for 28 days.

Copilot in Notebooks: Responsible AI FAQ

With Copilot in notebooks for Data Science and Data engineering in Microsoft Fabric, we offer an AI assistant to help transform, explore, and build solutions in the context of the notebook.

For considerations and limitations, see Limitations.

How did we evaluate Copilot in notebooks for data science and data engineering?

- The product team tested Copilot to see how well the system performs within the context of notebooks, and whether AI responses are insightful and useful.
- The team also invested in other harm mitigations, including technological approaches to focusing Copilot's output on data science-related topics.

How do you best work with Copilot in notebooks for data science and data engineering?

- Copilot is best equipped to handle data science topics, so limit your questions to this area.

- Explicitly describe the data you want Copilot to examine. If you describe the data asset - for example, by naming files, tables, or columns - Copilot can more likely retrieve relevant data and generate useful outputs.
- For more granular responses, load the data into the notebook as DataFrames, or pin the data in your lakehouse. This gives Copilot more context with which to perform analysis. If an asset is too large to load, pinning it's a helpful alternative.

Fabric data agent: Responsible AI FAQ

What is Fabric data agent?

Data agent is a new Microsoft Fabric feature that allows you to build your own conversational Q&A systems with generative AI. A Fabric data agent makes data insights more accessible and actionable for everyone in your organization. With a Fabric data agent, your team can have conversations, with plain English-language questions, about the data stored in Fabric OneLake, and then receive relevant answers. Even people without technical expertise in AI, or without a deep understanding of the data structure, can receive precise and context-rich answers.

What can data agent do?

Fabric data agent enables natural language interactions with structured data, allowing users to ask questions and receive rich, context-aware answers. It can enable users to connect and get insights from data sources like Lakehouse, Warehouse, Power BI dataset, KQL databases without needing to write complex queries. Data agent is designed to help users access and process data easily, enhancing decision-making through conversational interfaces while maintaining control over data security and privacy.

What are the intended uses for data agent?

- The Fabric data agent is intended to simplify the data querying process. It allows users to interact with structured data through natural language. It supports user insights, decision-making, and generation of answers to complex questions without the need for specialized query language knowledge. Data agent is especially useful for business analysts, decision-makers, and other nontechnical users who need quick, actionable insights from data stored in sources like KQL database, Lakehouse, Power BI dataset, and Warehouse resources.

- The Fabric data agent isn't intended for use cases where deterministic and 100% accurate results are required, because of current LLM limitations.

- The Fabric data agent isn't intended for uses cases that require deep analytics or causal analytics. For example, "why did the sales numbers drop last month?" is out of current scope.
- The Fabric data agent isn't intended to provide advise on physical or psychological injury. For example, scenarios that diagnose patients or prescribe medications.
- The Fabric data agent isn't intended to provide advise on life opportunities or legal status. Examples include scenarios where the AI system could affect an individual's legal status, legal rights, or their access to credit, finances, education, employment, healthcare, housing, insurance, social welfare benefits, services, opportunities, or the terms on which they're provided.

How was The Fabric data agent evaluated? What metrics are used to measure performance?

The product team tested the data agent on various public and private benchmarks, to determine the query quality against different data sources. The team also invested in other harm mitigations, including technological approaches to ensure that the data agent's output is constrained to the context of the selected data sources.

What are the limitations of The Fabric data agent? How can users minimize the impact of The Fabric data agent limitations when using the system?

- Ensure that you use descriptive column names. Instead of "C1" or "ActCu" column names (as examples), use "ActiveCustomer" or "IsCustomerActive." This is the most effective way to get more reliable queries out of the AI.
- To improve the accuracy of the Fabric data agent, you can provide more context with data agent instructions and example queries. These inputs help the Azure OpenAI Assistant API - which powers the Fabric data agent - make better decisions about how to interpret user questions and which data source is most appropriate to use.
- You can use Data agent instructions to guide the underlying agent's behavior, helping it identify the best data source to answer specific types of questions.
- You can also provide sample question-query pairs to demonstrate how the Fabric data agent should respond to common queries. These examples serve as patterns for interpreting similar user inputs and generating accurate results. Sample question-query pairs aren't currently supported for Power BI semantic model data sources.

- Refer to this resource for a full list of current limitations of the data agent.

What operational factors and settings allow for effective and responsible use of the Fabric data agent?

- The Fabric data agent can only access the data that you provide. It uses the schema (table name and column name), as well as the Fabric data agent instructions and example queries that you provide, in the User Interface (UI) or through the SDK.
- The Fabric data agent can only access the data that the user can access. If you use the data agent, your credentials are used to access the underlying database. If you don't have access to the underlying data, the data agent can't access that underlying data. This is true when you consume the data agent across different channels - for example, Microsoft Foundry or Microsoft Copilot Studio - where other users can use the data agent.

Related content

- Privacy, security, and responsible use of Copilot for Data Factory (preview)
- Overview of Copilot for Data Science and Data Engineering (preview)
- Copilot for Data Factory overview
- Copilot in Fabric: FAQ

Last updated on 02/26/2026

Privacy, security, and responsible AI use of Copilot in Fabric in the Data Warehouse workload (Preview)

Article • 04/16/2025

Applies to: ✅ Warehouse in Microsoft Fabric

In this article, you learn how Copilot in Fabric in the Data Warehouse workload works, keeps your business data secure and adheres to privacy requirements, and how to use generative AI responsibly. For more information on Copilot in Fabric, see Privacy, security, and responsible use for Copilot in Microsoft Fabric (preview).

With Copilot in Fabric in Data Warehouse and other generative AI features, Microsoft Fabric brings a new way to transform and analyze data, generate insights, and create visualizations and reports in your warehouse and other workloads.

For considerations and limitations, see Limitations.

Data use of Copilot in Fabric in Data Warehouse

In warehouse, Copilot can only access the database schema that is accessible in the user's warehouse.

By default, Copilot has access to the following data types:

- Previous messages sent to and replies from Copilot for that user in that session.
- Contents of SQL query that the user has executed.
- Error messages of a SQL query that the user has executed (if applicable).
- Schemas of the warehouse.
- Schemas from attached warehouses or SQL analytics endpoints when cross-DB querying.

Tips for working with Copilot in Fabric in Data Warehouse

- Copilot is best equipped to handle data warehousing topics, so limit your questions to this area.
- Be explicit about the data you want Copilot to examine. If you describe the data asset, with descriptive table and column names, Copilot is more likely to retrieve relevant data and generate useful outputs.

Evaluation of Copilot in Fabric in Data Warehouse

The product team tested Copilot to see how well the system performs within the context of warehouses, and whether AI responses are insightful and useful.

The team also invested in additional harm mitigation, including technological approaches to focusing Copilot's output on topics related to data warehousing.

Related content

- Privacy, security, and responsible AI use for Copilot in Microsoft Fabric
- What is Copilot in Fabric in Data Warehouse

Privacy, security, and responsible use for Copilot in Power BI

Microsoft Copilot in Power BI uses generative AI to help you analyze data and create reports. Before you start using Copilot, you might have questions about how it handles your business data, what information it accesses in your semantic models, and how to use generative AI responsibly.

This article explains what data is accessed by Copilot in Power BI, how your data stays secure, and how to evaluate Copilot outputs for accuracy and responsible use. For the overall Fabric privacy and security model, see Privacy, security, and responsible use for Copilot in Microsoft Fabric. For considerations and limitations with Copilot in Power BI, see Considerations and limitations.

Data use in Copilot in Power BI

- Copilot uses the data in a semantic model that you provide, combined with the prompts you enter, to create visuals. Learn more about semantic models.
- To answer data questions from the semantic model, Copilot requires that the Q&A natural language feature be enabled in the semantic model's dataset settings. For more information, see Update your data model to work well with Copilot in Power BI.
- To create measure descriptions in a semantic model, Copilot uses the DAX formula and table name of the selected measure. Copilot doesn't use DAX comments or text in double-quotes within the DAX formula. For more information, see Use Copilot to create measure descriptions.
- To create DAX queries, explain DAX queries, or explain DAX topics, Copilot uses the semantic model metadata, such as table and column names and properties, with any DAX query selected in the DAX query editor combined with the request you enter, to respond. For more information, see Use Copilot to create DAX queries.
- When you add a copilot summary to an email subscription, the copilot summary generated is the same as that generated when you add a narrative visual to a report. For more information, see Copilot summaries in email subscriptions.

Tips for working with Copilot in Power BI

- Be specific in your prompts. Instead of "show me sales data," try "create a bar chart of quarterly sales by region for 2025"

- Review Copilot-generated DAX queries before you apply them to your model.
- Provide context by referencing table and column names that exist in your semantic model.

For more tips, see FAQ for Copilot in Power BI.

Data privacy and security for Copilot in Power BI

- Copilot doesn't store your prompts or responses after your session ends.
- Your data stays within your organization's tenant boundary and isn't used to train foundation models.
- Tenant administrators can enable or disable Copilot through the admin portal.

For the full Copilot privacy and security model, see Privacy, security, and responsible use for Copilot in Microsoft Fabric.

Responsible AI evaluation of Copilot in Power BI

Copilot in Power BI is evaluated against Microsoft's responsible AI principles. Harm mitigation measures include Content filtering, Grounded responses, and Human oversight.

Related content

- Microsoft Copilot in Power BI
- Enable Fabric Copilot in Power BI
- Update your data model to work well with Copilot
- Considerations and limitations
- Copilot in Fabric and Power BI: FAQ

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/11/2026

Privacy, security, and responsible use of Copilot for Real-Time Intelligence

Copilot for Real-Time Intelligence uses generative AI to help you query, explore, visualize, and present real-time data. This article explains how Copilot uses data, how Microsoft evaluates its features, which limitations to consider, and how to use Copilot responsibly.

About Copilot for Real-Time Intelligence

Copilot for Real-Time Intelligence is a shared generative AI capability used across Microsoft Fabric Real-Time Intelligence experiences. Depending on the experience and workflow, Copilot can help you create or modify KQL queries, explore data, generate analytical results and visualizations, and create or modify Real-Time Dashboards. For an overview of these topics for Copilot in Fabric, see Privacy, security, and responsible use for Copilot.

The capabilities available to you depend on the feature, experience, permissions, data source, and release stage.

Copilot for Real-Time Intelligence helps you query, analyze, visualize, and explore real-time data by using natural language.

- Depending on the experience, you can use Copilot to:
- Author or modify KQL queries.
- Ask questions and explore data conversationally.
- Generate tables, visualizations, and natural-language insights.
- Create or modify dashboard tiles.
- Generate an initial Real-Time Dashboard from a description of the intended dashboard.
- Save or share generated analytical results.

Copilot reduces the need for specialized KQL knowledge, but you're responsible for reviewing generated content before using, saving, sharing, or applying it.

Intended use of Copilot for Real-Time Intelligence

Copilot for Real-Time Intelligence helps authorized users query, explore, visualize, and present real-time or operational data.

Example uses include:

- Generating and refining KQL queries in a KQL queryset.
- Creating or modifying the query and visualization for a dashboard tile.
- Exploring data associated with a dashboard, visual, table, or other supported data context.
- Generating an initial dashboard based on available data and the user's described intent.
- Refining generated results through follow-up questions or instructions.

Copilot is an assistive, human-directed capability. It isn't designed to autonomously make consequential decisions or take actions in external systems.

What can Copilot for Real-Time Intelligence do?

Copilot uses generative AI models, including GPT-5 in supported experiences, to interpret natural-language instructions together with contextual information supplied by the product experience.

Depending on the feature, this context can include the connected database schema, tables and columns, user-defined functions, data samples, an existing KQL query, dashboard configuration, a selected visual, previous conversation messages, and the user's dashboard description.

Copilot can generate one or more of the following:

- KQL queries and query modifications.
- Tabular query results.
- Visualizations and visual configuration.
- Natural-language insights or summaries.
- Starter and follow-up prompts.
- Dashboard tiles.
- An initial Real-Time Dashboard.

Some experiences execute generated queries to display results, while others present generated KQL for the user to review and run. The product experience indicates which behavior applies.

How Copilot for Real-Time Intelligence uses data

Copilot processes the user's prompt together with contextual information required for the selected experience. Depending on the feature, this information can include database schema,

user-defined functions, data samples, an existing query, dashboard or visual configuration, underlying query results, and previous messages in the current Copilot interaction.

Copilot operates within the user's existing permissions and doesn't grant additional access to data. Users can only use Copilot with data and items that the product experience authorizes them to access.

Customer data, including dashboard data, isn't used to train foundation models. Microsoft Fabric service terms govern prompt processing. Prompts and responses might be retained for abuse monitoring and service protection.

Fabric tenant administrators can control whether Copilot capabilities are enabled. Additional tenant settings might be required when data is processed outside the capacity's geographic region, compliance boundary, or national cloud instance.

How Microsoft evaluates Copilot for Real-Time Intelligence

Before release, Microsoft evaluates Copilot features based on their specific inputs, outputs, user interactions, and potential risks. Evaluations can include query quality, grounding, harmful content, prompt injection and jailbreak attempts, cross-prompt injection, protected material, and other risks relevant to the feature.

Because Copilot experiences perform different tasks, evaluation methods and datasets can differ between features. For example, a feature that generates KQL is evaluated differently from a feature that generates a dashboard or modifies an existing visualization.

Evaluation results reduce risk but don't guarantee that every generated query, result, visualization, insight, or dashboard is correct. Review generated content and validate important results against the underlying data.

Limitations of Copilot for Real-Time Intelligence

- Copilot can misunderstand ambiguous, complex, lengthy, or insufficiently detailed instructions.
- Generated KQL can be syntactically valid but fail to represent the user's intended business logic.
- Generated results depend on the quality and clarity of schema metadata, table and column names, dashboard context, data samples, and underlying data.

- Copilot might not understand organizational definitions, business rules, exceptions, or causal relationships that aren't represented in its available context.
- Generated insights, visualizations, tiles, or dashboards can be incomplete, inaccurate, or unsuitable for the intended purpose.
- Similar instructions can result in different valid queries, visualizations, or dashboard designs.
- Generated content can overwrite or conflict with manual changes in experiences that modify an existing item. Review the updated item before keeping the generated changes.
- Prompt-injection attempts can try to redirect model behavior. Testing and mitigations reduce but don't eliminate this risk.
- Feature availability, supported data sources, authoring operations, and administrative requirements vary by Copilot experience.
- Don't use Copilot as the sole basis for consequential or high-impact decisions.

Tips for working with Copilot for Real-Time Intelligence

- Clearly describe the task and intended result.
- Specify relevant tables, columns, metrics, filters, time periods, operators, or visual types when known.
- Use meaningful table and column descriptions to help Copilot interpret the schema.
- Break complex analytical or authoring tasks into focused requests and use follow-up instructions to refine the result.
- Inspect generated KQL before relying on it.
- Compare generated explanations and visualizations with the query results and underlying data.
- Review generated or modified dashboards before saving or sharing them.
- Don't include passwords, credentials, secrets, or unnecessary personal information in prompts.
- Validate important outputs before using them to support business or operational decisions.
- Use the available feedback controls to report incorrect or inappropriate results.

Copilot experiences in Real-Time Intelligence

Copilot for Real-Time Intelligence supports multiple experiences, including:

- **KQL query authoring**: Generate and refine KQL in Querysets and supported dashboard editors.
- **Data exploration**: Ask questions about supported dashboard or table data, refine the analysis, and inspect query, table, and visual results.

- **Dashboard tile creation and editing**: Create or modify a dashboard tile through natural-language instructions.
- **Dashboard generation**: Generate an initial Real-Time Dashboard based on selected data and a description of the intended dashboard.

Additional Copilot experiences might be introduced over time. The relevant product documentation describes the capabilities, prerequisites, controls, and limitations of each experience.

Related content

- What is Microsoft Fabric?
- Copilot in Fabric: FAQ

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 10/08/2026

Use Microsoft Purview to manage data security & compliance for Copilot in Fabric

> *Microsoft Purview service description*

Use the following sections to identify the Microsoft Purview capabilities that are supported for AI interactions with Copilot in Fabric, and some get started recommendations for you to manage these AI interactions for security and compliance.

The first Copilot in Fabric that's supported by Microsoft Purview is Copilot for Power BI. Current support extends to all prompts and all text-based responses.

> **Note**
>
> Microsoft Purview support for other Copilot experiences in Fabric and Fabric Data Agent is currently in preview.

As a prerequisite for Microsoft Purview to manage these AI interactions, the **Allow Microsoft Purview to secure AI interactions** option in the Microsoft Fabric admin portal must be **Enabled**, which is the default setting. For more information about this option, see Tenant settings index.

Managing these AI interactions with Microsoft Purview requires you to enable pay-as-you-go billing in your organization.

Capabilities supported

Use the following table to see at a glance the Microsoft Purview capabilities that are supported with Copilot in Fabric.

Expand table

Capability or solution in Microsoft Purview	Supported for AI interactions
DSPM and DSPM for AI (classic)	✓
Auditing	✓
Data classification	✓

Capability or solution in Microsoft Purview	Supported for AI interactions
Sensitivity labels	✗
Encryption without sensitivity labels	✗
Data loss prevention	✗
Insider Risk Management	✓
Communication compliance	✓
eDiscovery	✓
Data Lifecycle Management	✓
Compliance Manager	✓

DSPM and DSPM for AI (classic)

Use Data Security Posture Management or Data Security Posture Management for AI (classic) as your front door to discover, secure, and apply compliance controls for AI usage across your enterprise. Both DSPM versions use existing controls from Microsoft Purview information protection and compliance management with easy-to-use graphical tools and reports to quickly gain insights into AI use within your organization. With personalized recommendations, and one-click policies help you protect your data and comply with regulatory requirements.

AI app-specific information:

- Recommendation **Get guided assistance to AI regulations**, which uses control-mapping regulatory templates from Compliance Manager.
- One-click policies available:
  - **DSPM for AI - Capture interactions for Copilot experiences** from the recommendation or remediation action **Secure interactions in Microsoft Copilot experiences**.
  - **DSPM for AI - Detect risky AI usage** from the recommendation or remediation action **Detect risky interactions in AI apps**.
  - **DSPM for AI - Unethical behavior in AI apps** from the recommendation or remediation action **Detect unethical behavior in AI apps**.
  - **DSPM for AI - Detect sensitive info shared with AI via network** from the recommendation or remediation action **Extend insights into sensitive data in AI app interactions**.

Auditing and AI interactions

Microsoft Purview Audit solutions provide comprehensive tools for searching and managing audit records of activities performed across various Microsoft services by users and admins, and help organizations to effectively respond to security events, forensic investigations, internal investigations, and compliance obligations.

Like other activities, prompts and responses are captured in the unified audit log. Events include how and when users interact with the AI app, and can include in which Microsoft 365 service the activity took place, and references to the files stored in Microsoft 365 that were accessed during the interaction. If these files have a sensitivity label applied, that's also captured.

These events flow into activity explorer in DSPM for AI and the **AI activities** tab in activity explorer from the preview version of DSPM, where the data from prompts and responses can be displayed. You can also use the **Audit** solution from the Microsoft Purview portal to search and find these auditing events.

For more information, see Audit logs for Copilot and AI activities.

Data classification and AI interactions

Microsoft Purview data classification provides a comprehensive framework for identifying and tagging sensitive data across various Microsoft services, including Office 365, Dynamics 365, and Azure. Classifying data is often the first step to ensure compliance with data protection regulations and safeguard against unauthorized access, alteration, or destruction. You can use built-in system classifications or create your own.

Sensitive information types and trainable classifiers can be used to find sensitive data in user prompts and responses when they use AI apps. The resulting information then surfaces in Microsoft Purview Reports overview and activity explorer in DSPM for AI and the **AI activities** tab in activity explorer from the preview version of DSPM.

Insider Risk Management and AI interactions

Microsoft Purview Insider Risk Management helps you detect, investigate, and mitigate internal risks such as IP theft, data leakage, and security violations. It leverages machine learning models and various signals from Microsoft 365 and third-party indicators to identify potential malicious or inadvertent insider activities. The solution includes privacy controls like pseudonymization and role-based access, ensuring user-level privacy while enabling risk analysts to take appropriate actions.

Use the Risky AI usage policy template to detect risky usage that includes prompt injection attacks and accessing protected materials. Insights from these signals are integrated into Microsoft Defender XDR to provide a comprehensive view of AI-related risks.

AI app-specific information:

- For prompts and responses, requires a collection policy such as **DSPM for AI - Capture interactions for Copilot experiences** from the DSPM for AI recommendation **Secure interactions in Microsoft Copilot experiences**.

Communication compliance and AI interactions

Microsoft Purview Communication Compliance provides tools to help you detect and manage regulatory compliance and business conduct violations across various communication channels, which include user prompts and responses for AI apps. It's designed with privacy by default, pseudonymizing usernames and incorporating role-based access controls. The solution helps identify and remediate inappropriate communications, such as sharing sensitive information, harassment, threats, and adult content.

To learn more about using communication compliance policies for AI apps, see Configure a communication compliance policy to detect for generative AI interactions.

AI app-specific information:

- For prompts and responses, requires a collection policy such as **DSPM for AI - Capture interactions for Copilot experiences** from the DSPM for AI recommendation **Secure interactions in Microsoft Copilot experiences**.

eDiscovery and AI interactions

Microsoft Purview eDiscovery lets you identify and deliver electronic information that can be used as evidence in legal cases. The eDiscovery tools in Microsoft Purview support searching for content in Exchange Online, OneDrive for Business, SharePoint Online, Microsoft Teams, Microsoft 365 Groups, and Viva Engage teams. You can then prevent the information from deletion and export the information.

Because user prompts and responses for AI apps are stored in a user's mailbox, you can create a case and use search when a user's mailbox is selected as the source for a search query. For example, select and retrieve this data from the source mailbox by selecting from the query

builder **Add condition** > **Type** > **Contains any of** > **Edit** > *Copilot activity*. This query condition includes all Copilot and other AI application activity.

After the search is refined, you can export the results or add to a review set. You can review and export information directly from the review set.

For more information about identifying and deleting user AI interaction data, see Search for and delete Copilot data in eDiscovery.

AI app-specific information:

- For prompts and responses, requires a collection policy such as **DSPM for AI - Capture interactions for Copilot experiences** from the DSPM for AI recommendation **Secure interactions in Microsoft Copilot experiences**.

Data Lifecycle Management and AI interactions

Microsoft Purview Data Lifecycle Management provides tools and capabilities to manage the lifecycle of organizational data by retaining necessary content and deleting unnecessary content. These tools ensure compliance with business, legal, and regulatory requirements.

Use retention policies to automatically retain or delete user prompts and responses for AI apps. For detailed information about this retention works, see Learn about retention for Copilot & AI apps.

As with all retention policies and holds, if more than one policy for the same location applies to a user, the principles of retention resolve any conflicts. For example, the data is retained for the longest duration of all the applied retention policies or eDiscovery holds.

AI app-specific information:

- For retention policies, select the option for **Microsoft Copilot Experiences**.
- For prompts and responses, requires a collection policy such as **DSPM for AI - Capture interactions for Copilot experiences** from the DSPM for AI recommendation **Secure interactions in Microsoft Copilot experiences**.

Compliance Manager and AI interactions

Microsoft Purview Compliance Manager is a solution that helps you automatically assess and manage compliance across your multicloud environment. Compliance Manager can help you throughout your compliance journey, from taking inventory of your data protection risks to

managing the complexities of implementing controls, staying current with regulations and certifications, and reporting to auditors.

To help you keep compliant with AI regulations, Compliance Manager provides regulatory templates to help you assess, implement, and strengthen your compliance requirements for all generative AI apps. For example, monitoring AI interactions and preventing data loss in AI applications. For more information, see Assessments for AI regulations.

Getting started recommended steps

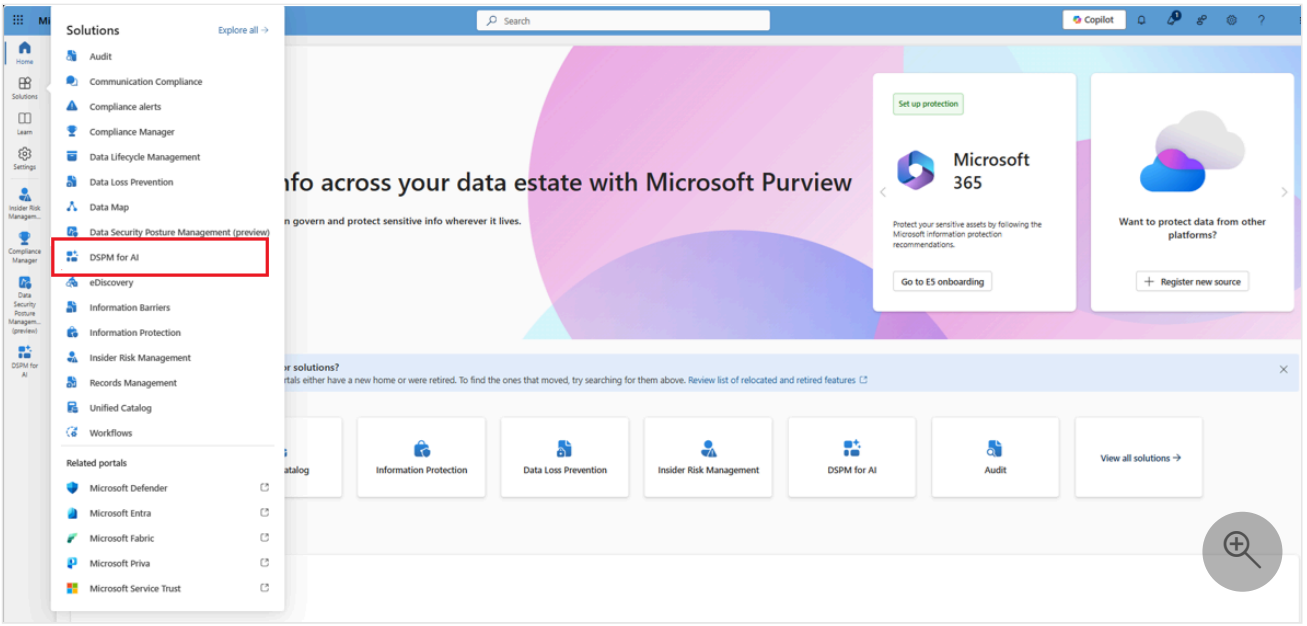
Use the following steps to get started with managing data security & compliance for AI interactions from Copilot in Fabric.

> **Note**
>
> These steps focus on managing a specific AI app or agent. For broader coverage that uses security objectives with guided workflows, use the current version of Data Security Posture Management.

Managing data security & compliance for Copilot in Fabric requires the enterprise version of Microsoft Purview data governance, to support the required APIs.

Because Data Security Posture Management for AI is your front door for securing and managing AI interactions, most of the following instructions use that solution:

- Sign in to the Microsoft Purview portal > **Solutions** > **DSPM for AI (classic)** with an account that has appropriate permissions. For example, an account that's a member of the **Microsoft Entra Compliance Administrator** group role.



Discover potential security risks in Copilot in Fabric interactions

1. From **DSPM for AI (classic)** > **Overview**, in the **Get Started** section, look to see if auditing is on for your tenant. If not, select **Activate Microsoft Purview Audit**.

2. In the **Recommendations** section, select **View all recommendations** to take you to the **Recommendations** page. If they have a status of **Not Started**, take action on the following recommendations that create one-click policies:

    - **Secure interactions in Microsoft Copilot experiences**: This recommendation creates a policy to policy captures prompts and responses for data security posture and regulatory compliance from Copilot in Fabric, and Security Copilot.

    - **Detect risky interactions in AI apps**: This recommendation creates a policy to help calculate user risk by detecting risky prompts and responses in Microsoft 365 Copilot, agents, and other generative AI apps.

3. Wait at least a day for data, and then navigate to the **Reports** page to view the results of your policy. Select **Copilot experiences & agents** and view information such as:

    - Total interactions over time (Microsoft Copilot and agents)
    - Sensitive interactions per AI app
    - Top unethical AI interactions
    - Insider Risk severity
    - Insider risk severity per AI app
    - Potential risky AI usage

4. Select **View details** for each of the report graphs to view detailed activities in the activity explorer.

   From the filters, select the **AI app category** of **Copilot experiences & agents**, and then use the other filters if you need to further refine the displayed data. Then drill down to each activity to view details that include displaying the prompts and response when you're a member of the Microsoft Purview Content Explorer Content Viewer role group. For more information about this requirement, see Permissions for Data Security Posture Management for AI.

Apply compliance controls to Copilot in Fabric

1. From **DSPM for AI (classic)** > **Recommendations** page, under **Not Started**, locate, and select and take actions on the following recommendation:

   - **Detect unethical behavior in AI**: This recommendation creates a policy to detect sensitive information in prompts and responses in Copilot in Fabric and agents, and other generative AI apps.

2. If you need to ensure that Copilot in Fabric interactions are retained for compliance reasons:

   In the Microsoft Purview portal, navigate to **Data Lifecycle Management** > **Policies** > **Retention Policies** and create a retention policy to retain Microsoft Security Copilot interactions by selecting the location **Microsoft Copilot Experiences** and specify the required retention period. For more information, see Create and configure retention policies.

3. If you need to preserve, collect, analyze, review, or export Microsoft Fabric interactions:

   In the Microsoft Purview portal, navigate to **eDiscovery** > **Cases** > **Create case**. In the case, create a search and use the *ItemClass* property and the `IPM.SkypeTeams.Message.Copilot.Fabric.*` value to search for these interactions in your organization.

Routinely review the reports in DSPM for AI to determine if you need to make changes, and use activity explorer and events for deeper analysis of how users are interacting with Copilot in Fabric.

Related resources

- Blog post (July 2025): Using Copilot in Fabric with Confidence: Data Security, Compliance & Governance with DSPM for AI

Last updated on 05/01/2026

What is Copilot in Fabric in the Data Factory workload?

This article provides an overview of Copilot in the Data Factory workload, including its features and benefits. Microsoft Copilot in the Data Factory workload is an AI-enhanced toolset that empowers customers to use natural language to articulate their requirements for creating data integration solutions using Dataflow Gen2. The Copilot in the Data Factory workload operates like a subject-matter expert (SME) collaborating with you to design your data flows, supporting both citizen and professional data wranglers in streamlining their workflows.

Before your business can start using Copilot capabilities in Fabric, your administrator must enable Copilot in Fabric. For more information, see Copilot Fabric Overview.

> **Note**
>
> - Your administrator needs to enable the tenant switch before you start using Copilot. See the article **Copilot tenant settings** for details.
> - Your F2 or P1 capacity needs to be in one of the regions listed in this article, **Fabric region availability**.
> - If your tenant or capacity is outside the US or EU, Copilot is disabled by default. Your Fabric tenant admin will also need to enable the **Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance** tenant setting in the Fabric Admin portal to use Copilot.
> - Copilot in Microsoft Fabric isn't supported on trial SKUs. Only paid SKUs (F2 or higher, or P1 or higher) are supported.
> - See the article **Overview of Copilot in Fabric and Power BI** for more information.

How Copilot works in the Data Factory workload

Copilot in the Data Factory workload enhances productivity, unlocks profound insights, and facilitates the creation of custom AI experiences tailored to your data. As a component of the Copilot in the Fabric experience, it provides intelligent Mashup code generation to transform data using natural language input. It generates code explanations to help you better understand the complex queries and tasks that were previously generated.

Features of Copilot in the Data Factory workload

Copilot in the Data Factory workload offers different capabilities depending on the component you're working with:

With Dataflow Gen2, you can:

- Get data from sources by describing what you need in natural language
- Generate new transformation steps for an existing query
- Provide a summary of the query and the applied steps
- Generate a new query that might include sample data or a reference to an existing query
- Undo the last applied step

With pipelines, you can:

- **Pipeline Generation**: Using natural language, you can describe your desired pipeline, and Copilot understands the intent and generates the necessary pipeline activities
- **Error message assistant**: Troubleshoot pipeline issues with clear error explanation capability and actionable troubleshooting guidance
- **Summarize Pipeline**: Explain your complex pipeline with a summary of content and relations of activities within the Pipeline
- **Build expressions**: Generate and explain pipeline expressions using Copilot's intuitive chat interface.

Best practices for using Copilot in the Data Factory workload

To get the most out of Copilot in the Data Factory workload:

- Be specific and clear in your natural language requests
- Start with simple transformations and build complexity gradually
- Use the "Explain my current query" feature to understand the generated code
- Use the undo functionality to revert changes when needed
- Review the generated steps in the Applied steps list for accuracy
- Use starter prompts to get familiar with Copilot's capabilities

Example prompts

Here are some example prompts you can use with Copilot in the Data Factory workload:

Dataflow Gen2 prompts

Copilot prompt

```
- Only keep European customers
- Count the total number of employees by City
- Only keep orders whose quantities are above the median value
- Create a new query with sample data that lists all the Microsoft OS versions and the
year they were released
- Explain my current query
```

Pipeline prompts

Copilot prompt

```
- Create a pipeline to copy data from SQL Server to Azure Data Lake
- Ingest data from this source to that destination
- Summarize this pipeline
- Explain what this pipeline does
```

> **Note**
>
> AI powers Copilot, so surprises and mistakes are possible.

Responsible AI use of Copilot

Microsoft is committed to ensuring that our AI systems are guided by our AI principles and Responsible AI Standard. These principles include empowering our customers to use these systems effectively and in line with their intended uses. Our approach to responsible AI is continually evolving to address emerging issues proactively.

For specific guidelines on responsible AI use in Data Factory, see Privacy, security, and responsible use of Copilot in Fabric in the Data Factory workload.

Limitations

Here are the current limitations of Copilot in the Data Factory workload:

- Copilot can't perform transformations or explanations across multiple queries in a single input. For instance, you can't ask Copilot to "Capitalize all the column headers for each query in my dataflow."

- Copilot doesn't understand previous inputs and can't undo changes after a user commits a change when authoring, either via the user interface or the chat pane. For example, you can't ask Copilot to "Undo my last five inputs." However, users can still use the existing user interface options to delete unwanted steps or queries.
- Copilot can't make layout changes to queries in your session. For example, if you tell Copilot to create a new group for queries in the editor, it doesn't work.
- Copilot might produce inaccurate results when the intent is to evaluate data that isn't present within the sampled results imported into the session's data.
- Copilot doesn't produce a message for the skills that it doesn't support. For example, if you ask Copilot to "Perform statistical analysis and write a summary over the contents of this query", it doesn't complete the instruction successfully as mentioned previously. Unfortunately, it doesn't give an error message either.

Related content

- Get started with Copilot in Fabric in the Data Factory workload
- Copilot pipeline expressions builder
- Privacy, security, and responsible use of Copilot in Fabric in the Data Factory workload

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 08/31/2026

What is Copilot in the Data Warehouse workload (preview)?

Applies to: ✅ SQL analytics endpoint and Warehouse in Microsoft Fabric

Copilot in Fabric Data Warehouse is an AI assistant designed to streamline your data warehousing tasks. Copilot integrates seamlessly with your data warehouse in Fabric, providing intelligent insights to help you along each step of the way in your T-SQL explorations.

Copilot in Fabric Data Warehouse uses table and view names, column names, primary key, and foreign key metadata to generate T-SQL code. Copilot in Fabric Data Warehouse doesn't use data in tables to generate T-SQL suggestions.

Features of Copilot in Fabric Data Warehouse

Copilot in Fabric Data Warehouse offers the following features:

- **Natural Language to SQL**: Ask Copilot to generate SQL queries using simple natural language questions.
- **Code completion**: Enhance your coding efficiency with AI-powered code completions.
- **Quick actions**: Quickly fix and explain SQL queries with readily available actions.
- **Intelligent Insights**: Receive smart suggestions and insights based on your warehouse schema and metadata.

There are three ways to interact with Copilot in the Fabric Warehouse editor.

- Copilot chat pane: Use the chat pane to ask questions to Copilot through natural language. Copilot responds with a generated SQL query or natural language based on the question asked.
  - **Natural Language to SQL**: Generate T-SQL code and get suggestions of questions to ask to accelerate your workflow.

- Copilot code completion: Start writing T-SQL in the SQL query editor and Copilot automatically generates a code suggestion to help complete your query. The **Tab** key accepts the code suggestion, or keeps typing to ignore the suggestion.

- Copilot quick actions: In the ribbon of the SQL query editor, the **Fix** and **Explain** options are quick actions. Highlight a SQL query of your choice and select one of the quick action

buttons to perform the selected action on your query.

- **Explain:** Copilot can provide natural language explanations of your SQL query and warehouse schema in comments format.
- **Fix:** Copilot can fix errors in your code as error messages arise. Error scenarios can include incorrect or unsupported T-SQL code, wrong spellings, and more. Copilot also provides comments that explain the changes and suggest SQL best practices.

Enable Copilot

- Your administrator needs to enable the tenant switch before you start using Copilot. For more information, see Copilot tenant settings.
- Your **F2 or P1 capacity** needs to be in one of the regions listed in Fabric region availability.
- If your tenant or capacity is outside the US or France, Copilot is disabled by default unless your Fabric tenant admin enables the Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance tenant setting in the Fabric Admin portal.
- Copilot in Microsoft Fabric isn't supported on trial SKUs. Only paid SKUs (F2 or higher, or P1 or higher) are supported at this time.
- For more information, see Overview of Copilot in Fabric and Power BI.

Best practices for using Copilot in Fabric Data Warehouse

Here are some tips for maximizing productivity with Copilot.

- When crafting prompts, start with a clear and concise description of the specific information you're looking for.
- Natural language to SQL depends on expressive table and column names. If your table and columns aren't expressive and descriptive, Copilot might not be able to construct a meaningful query.
- Use natural language that applies to your table and view names, column names, primary keys, and foreign keys of your warehouse. This context helps Copilot generate accurate queries. Specify what columns you want to see, aggregations, and any filtering criteria as explicitly as possible. Copilot should be able to correct typos or understand context given your schema context.
- Create relationships in the model view of the warehouse to increase the accuracy of `JOIN` statements in your generated SQL queries.

- When using code completions, leave a comment at the top of the query with `--` to help guide the Copilot with context about the query you're trying to write.
- Avoid ambiguous or overly complex language in your prompts. Simplify the question while maintaining its clarity. This editing ensures Copilot can effectively translate it into a meaningful T-SQL query that retrieves the desired data from the associated tables and views.
- Currently, natural language to SQL supports English language to T-SQL.

Example prompts

- The following example prompts are clear, specific, and tailored to the properties of your schema and data warehouse, making it easier for Copilot to generate accurate T-SQL queries:

Copilot prompt

```
    - Show me all properties that sold last year
    - Count all the products, group by each category
    - Show all agents who sell properties in California
    - Show agents who have listed more than two properties for sale
    - Show the rank of each agent by property sales and show name, total sales, and
rank
```

> **Note**
>
> AI powers Copilot, so surprises and mistakes are possible.

Responsible AI use of Copilot

To view Microsoft's guidelines for responsible AI in Fabric Data Warehouse, see Privacy, security, and responsible AI use of Copilot.

Microsoft is committed to ensuring that our AI systems are guided by our AI principles and Responsible AI Standard. These principles include empowering our customers to use these systems effectively and in line with their intended uses. Our approach to responsible AI is continually evolving to proactively address emerging issues.

Limitations

Here are the current limitations of Copilot in Fabric in Data Warehouse:

- Copilot doesn't understand previous inputs and can't undo changes after a user commits a change when authoring, either through the user interface or the chat pane. For example, you can't ask Copilot to *Undo my last five inputs*. However, you can still use the existing user interface options to delete unwanted changes or queries.
- Copilot can't make changes to existing SQL queries. For example, if you ask Copilot to edit a specific part of an existing query, it doesn't work.
- Copilot might produce inaccurate results when the intent is to evaluate data. Copilot only has access to the warehouse schema, and none of the data inside.
- Copilot responses can include inaccurate or low-quality content, so ensure you review outputs before using them in your work.
- People who can meaningfully evaluate the content's accuracy and appropriateness should review the outputs.

Related content

- Copilot and Agent tenant settings
- How to use the Copilot chat pane in Fabric Data Warehouse (Preview)
- How to use Copilot quick actions in Fabric Data Warehouse (Preview)
- How to Use Copilot code completion in Fabric Data Warehouse (Preview)
- Privacy, security, and responsible AI use of Copilot in Fabric in the Data Warehouse workload (Preview)

Last updated on 09/02/2026

What is Copilot for writing queries in KQL Queryset?

Copilot helps you transform your natural language questions into Kusto Query Language (KQL) queries effortlessly. Describe your data analysis or extraction needs in plain language, and Copilot generates the corresponding KQL query for you. This functionality empowers users of all skill levels, even those unfamiliar with KQL, to explore and analyze data with ease and efficiency.

For billing information about Copilot, see Announcing Copilot in Fabric pricing.

Prerequisites

- A workspace with a Microsoft Fabric-enabled capacity
- Write access to a KQL queryset

> **Note**
>
> - Your administrator needs to enable the tenant switch before you start using Copilot. See the article **Copilot tenant settings** for details.
> - Your F2 or P1 capacity needs to be in one of the regions listed in this article, **Fabric region availability**.
> - If your tenant or capacity is outside the US or EU, Copilot is disabled by default. Your Fabric tenant admin will also need to enable the **Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance** tenant setting in the Fabric Admin portal to use Copilot.
> - Copilot in Microsoft Fabric isn't supported on trial SKUs. Only paid SKUs (F2 or higher, or P1 or higher) are supported.
> - See the article **Overview of Copilot in Fabric and Power BI** for more information.

How Copilot for writing queries in KQL works

Copilot, powered by OpenAI's advanced language capabilities, simplifies the process of creating Kusto Query Language (KQL) queries by translating natural language questions into KQL. This feature eliminates the complexity of learning KQL, making it accessible to both experienced data

analysts and citizen data scientists. This streamlines data analysis and boosts productivity by making query creation intuitive and straightforward.

Copilot supports **conversational interactions** which allows you to clarify, adapt, and extend your queries dynamically, all while maintaining the context of your previous inputs. You can refine queries and ask follow-up questions without starting over:

- Dynamic query refinement: You can refine the initial KQL generated by Copilot by refining your prompt to remove ambiguity, specify tables or columns, or provide more context.
- Seamless follow-up questions: If the generated KQL is correct but you want to explore the data more deeply, you can ask follow-up questions related to the same task. You can expand the scope of your query, add filters, or explore related data points by building on previous dialogue.

How to access Copilot for writing KQL queries

Access Copilot in Fabric in two ways:

- **Through a KQL queryset** : Go to a new or existing KQL queryset and use the Copilot feature to generate queries from natural language prompts.
- **Through the Edit tile in Real-Time Dashboards**: When editing a tile in a Real-Time dashboard, use Copilot to help create or refine KQL queries directly within the dashboard editing experience.

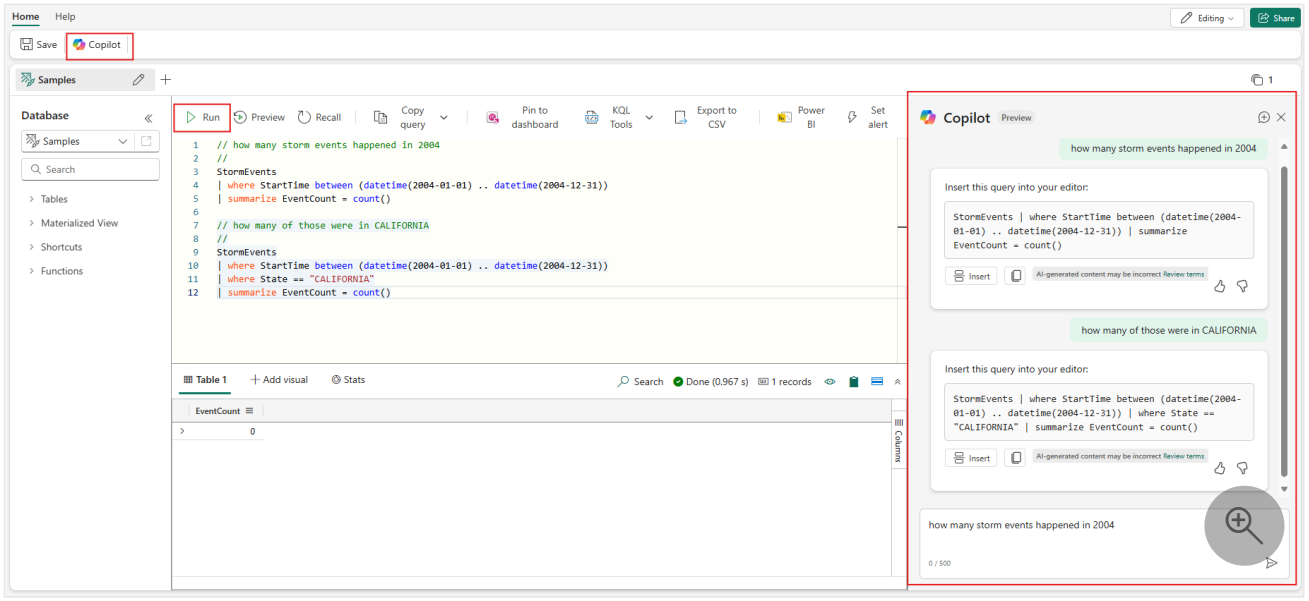
How to use Copilot for writing queries in KQL

Follow the steps to use Copilot in **either** context:

1. In the Copilot pane, enter your question in natural language and press **Enter**.

   After a few seconds, Copilot generates a KQL query based on your input. You can copy the query to the clipboard, **Insert** it to the query editor, or **Replace** the query in context with it. To run the query in the query editor, you must have write access to the KQL queryset.

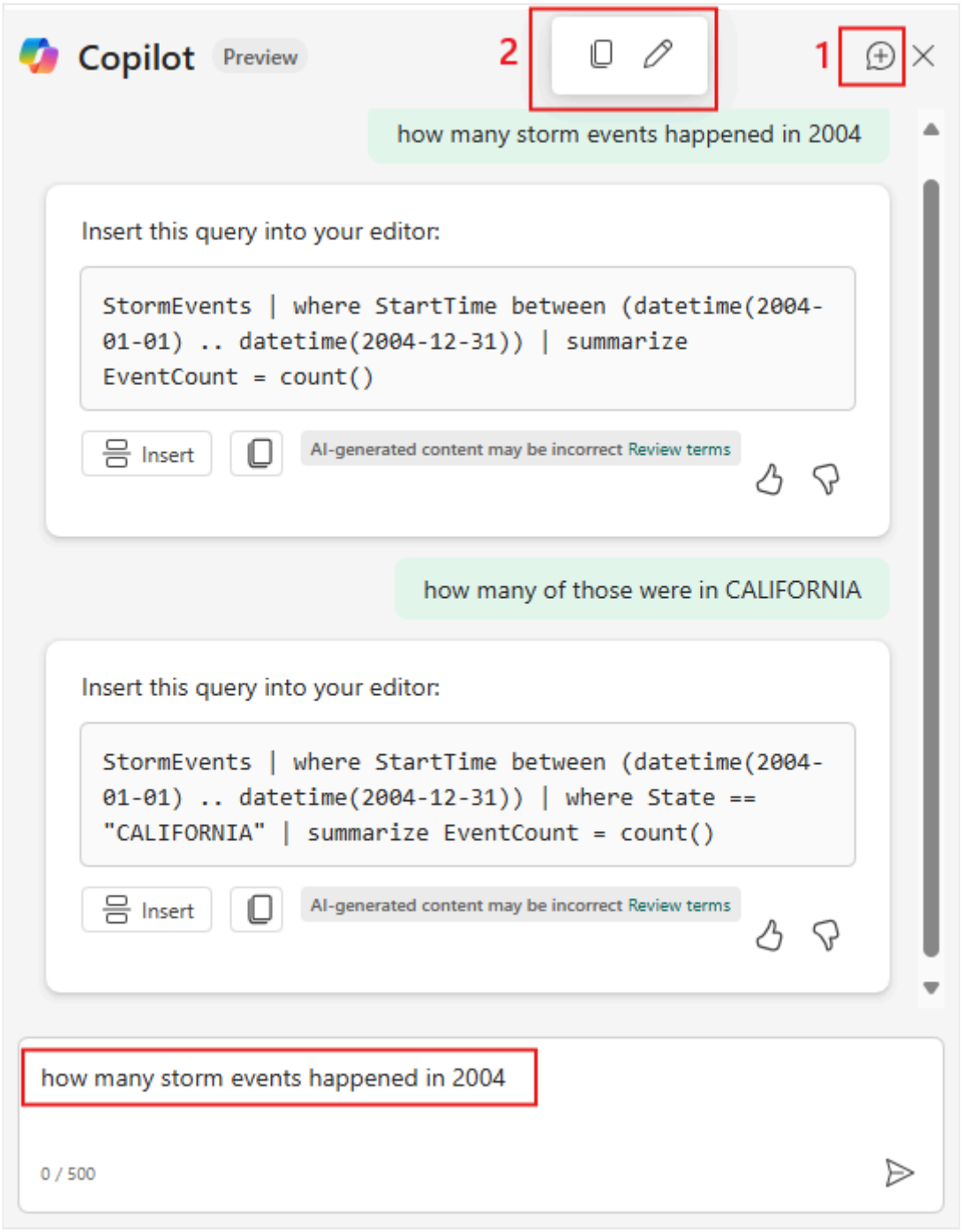
2. Select the **Run** button to execute the query.



> **Note**
>
> - Copilot doesn't generate control commands.
> - Copilot doesn't automatically run the generated KQL query. Run the queries at your own discretion.

You can continue to ask follow-up questions or further refine your query. To start a new chat, select the speech bubble on the top right of the Copilot pane (1).

Hover over a previous question (2) and select the **pencil** icon to copy it to the question box to edit it or copy it to your clipboard.



Related content

- Copilot for Real-Time Intelligence, including best practices and limitations
- Generate Real-Time dashboard with Copilot in Fabric in the Real-time intelligence workload
- Privacy, security, and responsible use of Copilot for Real-Time Intelligence
- Copilot in Fabric
- Copilot for Microsoft Fabric: FAQ

Last updated on 05/04/2026

Generate Real-Time dashboards with Copilot in Fabric in the Real-Time intelligence workload

Within an eventhouse, Copilot can generate a Real-Time Dashboard for a selected database based on the tables it contains. This capability helps you quickly start analyzing trends and anomalies in streaming data.

Describe the insights you want in natural language, and Copilot creates a dashboard tailored to your selected data and goal.

For billing information about Copilot, see Announcing Copilot in Fabric pricing.

Prerequisites

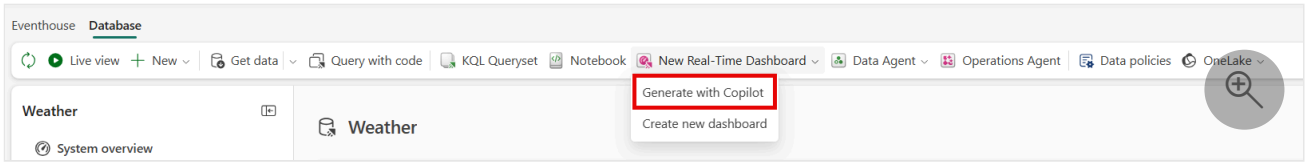
- A workspace with a Microsoft Fabric-enabled capacity

> **Note**
>
> - Your administrator needs to enable the tenant switch before you start using Copilot. See the article Copilot tenant settings for details.
> - Your F2 or P1 capacity needs to be in one of the regions listed in this article, Fabric region availability.
> - If your tenant or capacity is outside the US or EU, Copilot is disabled by default. Your Fabric tenant admin will also need to enable the Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance tenant setting in **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings** to use Copilot.
> - Copilot in Microsoft Fabric isn't supported on trial SKUs. Only paid SKUs (F2 or higher, or P1 or higher) are supported.
> - See the article Overview of Copilot in Fabric and Power BI for more information.

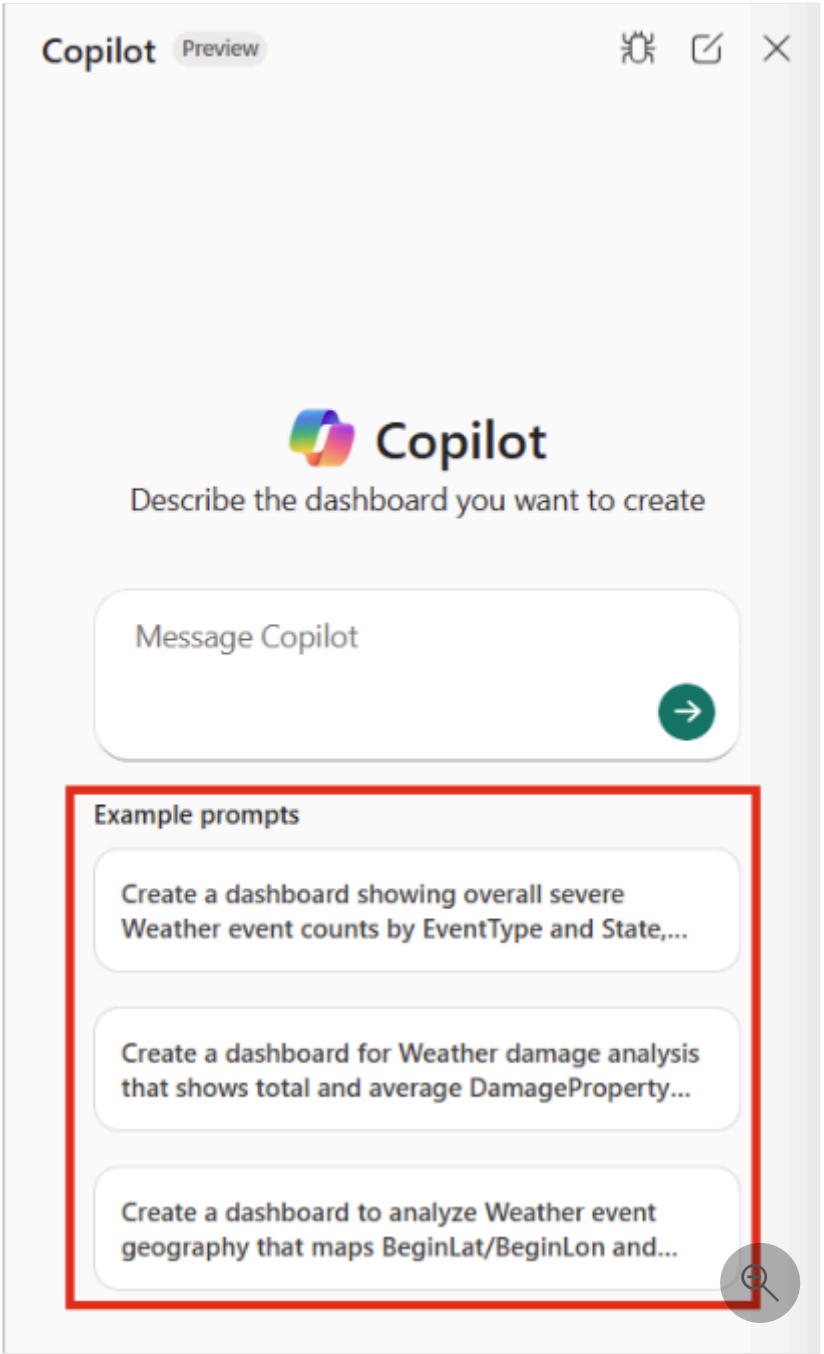
Create a dashboard with Copilot from an eventhouse

1. Go to the home page of the eventhouse that contains the data you want to visualize.

2. In the upper ribbon, under **New Real-Time Dashboard**, select **Generate with Copilot**.

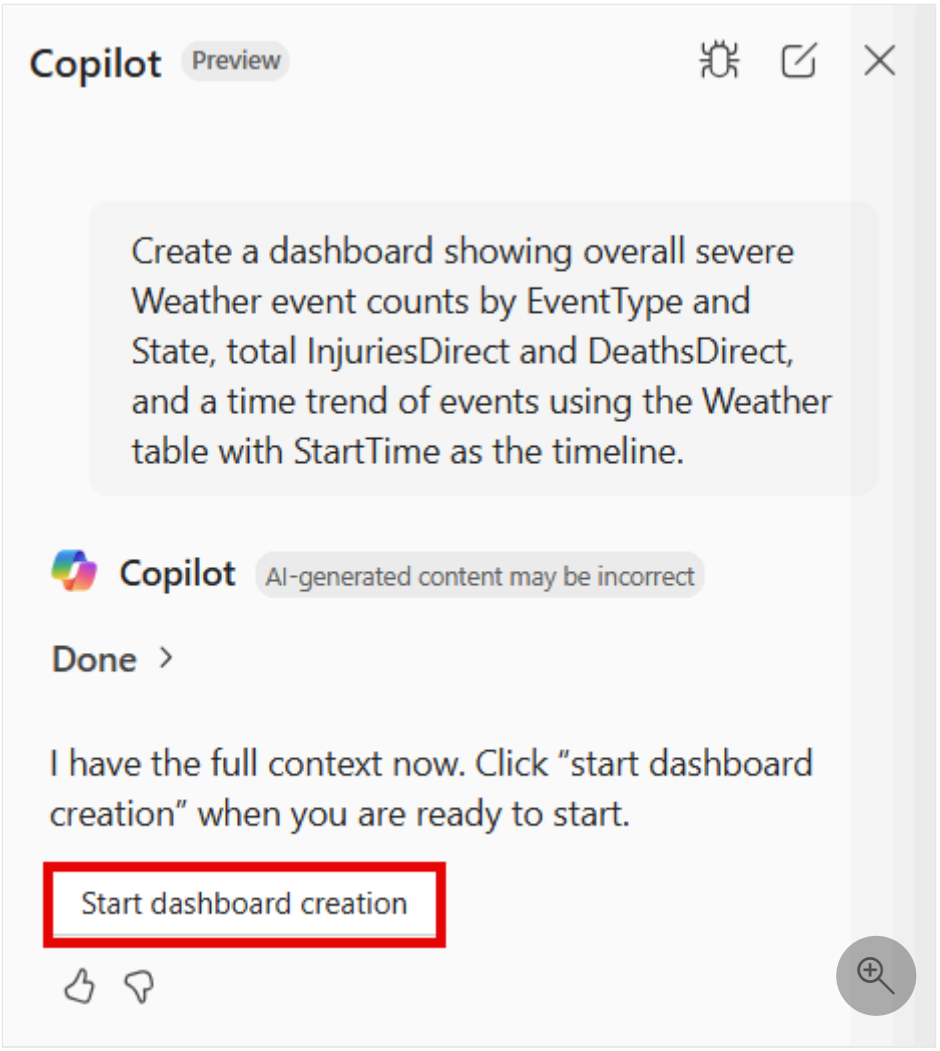


3. In the Copilot pane, describe the dashboard you want. Include your goal, the data to prioritize, and how you want the dashboard organized. Use the data-specific example prompts to help you get started.



4. When analyzing your request, Copilot provides its thought process in the pane. You can follow along to see how Copilot interprets your request.

5. After Copilot generates a response, you can refine your prompt with follow-up requests to adjust the scope or focus of the dashboard, or select **Start dashboard creation** to proceed.



6. Enter a name and location for the dashboard, and then select **Create**.

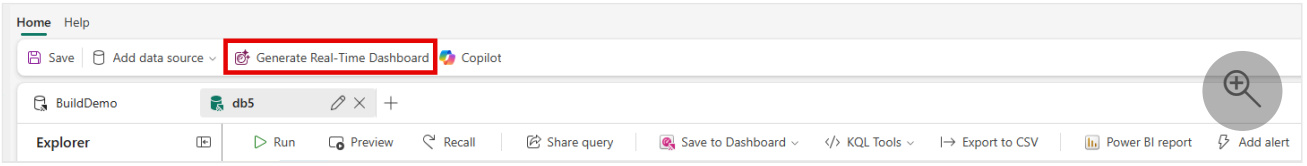
7. Once generated, update the dashboard by adjusting visual types or modifying queries as needed. You can use Copilot to assist with editing visuals or with writing queries.

8. Save your changes when the dashboard reflects the insights you want to monitor.

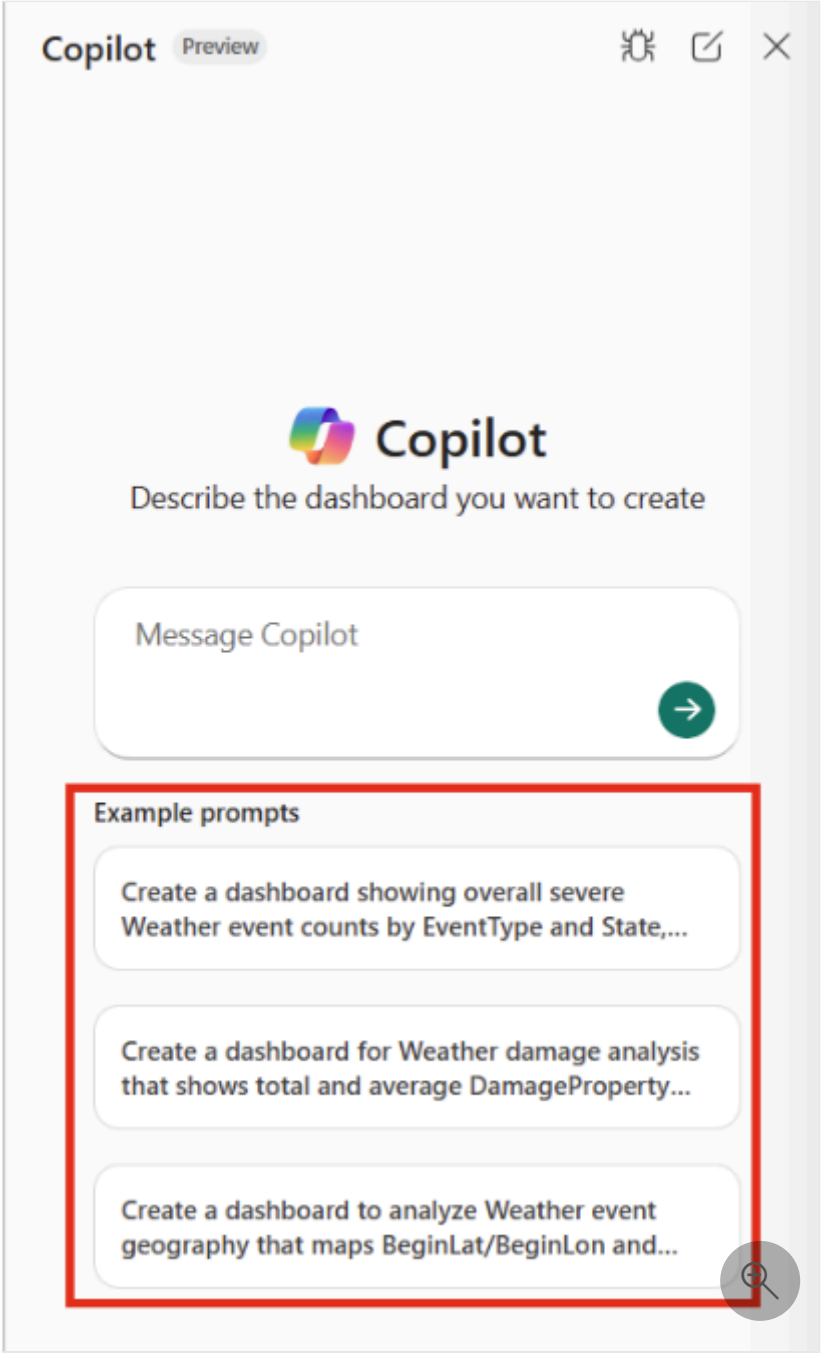
Create a dashboard with Copilot from a KQL Queryset

1. Go to the KQL Queryset that contains the data you want to visualize.

2. In the upper ribbon, select **Generate with Real-Time Dashboard**.

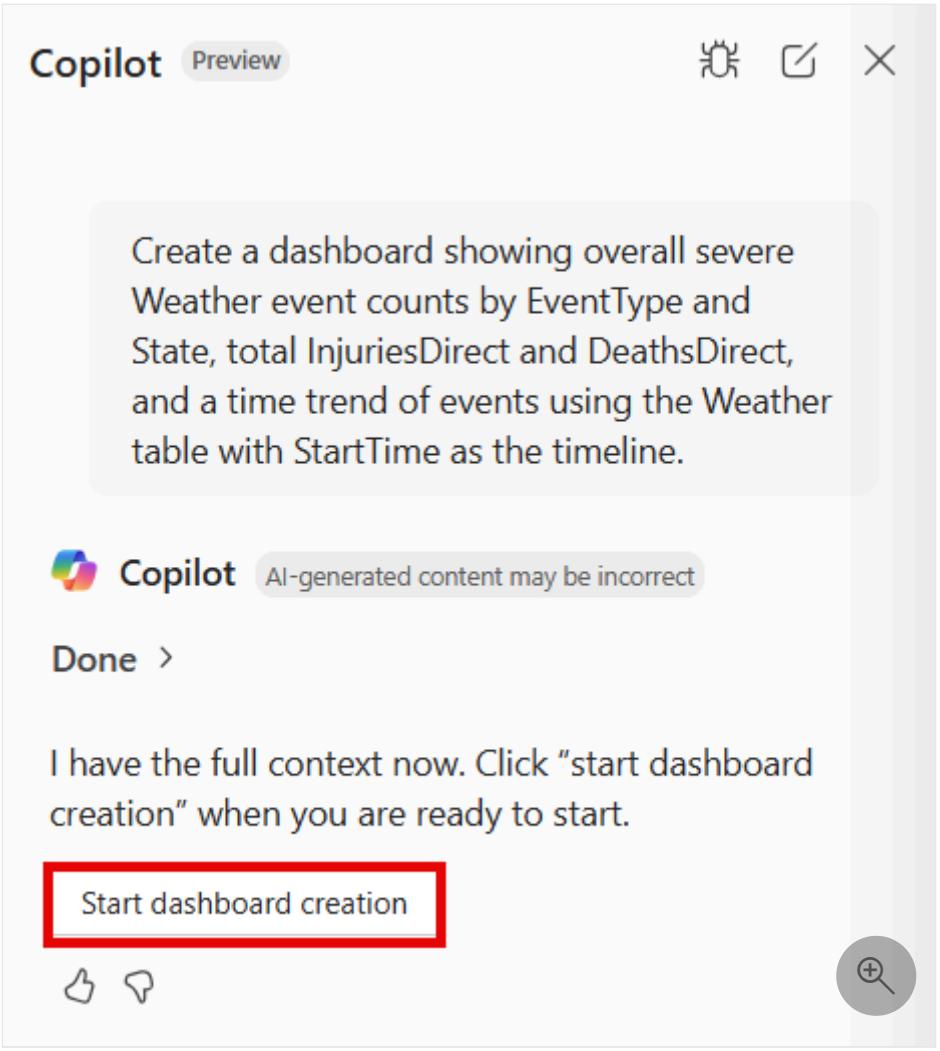


3. In the Copilot pane, describe the dashboard you want. Include your goal, the data to prioritize, and how you want the dashboard organized. Use the data-specific example prompts to help you get started.



4. When analyzing your request, Copilot provides its thought process in the pane. You can follow along to see how Copilot interprets your request.

5. After Copilot generates a response, you can refine your prompt with follow-up requests to adjust the scope or focus of the dashboard, or select **Start dashboard creation** to proceed.



6. Enter a name and location for the dashboard, and then select **Create**.

7. Once generated, update the dashboard by adjusting visual types or modifying queries as needed. You can use Copilot to assist with editing visuals or with writing queries.

8. Save your changes when the dashboard reflects the insights you want to monitor.

Related content

- Create a Real-Time Dashboard
- Explore data with Copilot
- Customize dashboard visuals
- Privacy, security, and responsible use of Copilot for Real-Time Intelligence

---

Last updated on 10/05/2026

Copilot-assisted real-time data exploration (Preview)

Real-Time dashboards help you monitor key metrics, detect anomalies, and make informed decisions. By using Copilot, you can explore the live data behind your dashboard by using natural language, without needing to write KQL queries.

Ask questions about your data, investigate trends, refine results, and generate visualizations. Copilot can explore data across an entire dashboard, a specific data source, or a particular visual. You can then save or share the resulting insights.

> **Important**
>
> This feature is in preview.

Prerequisites

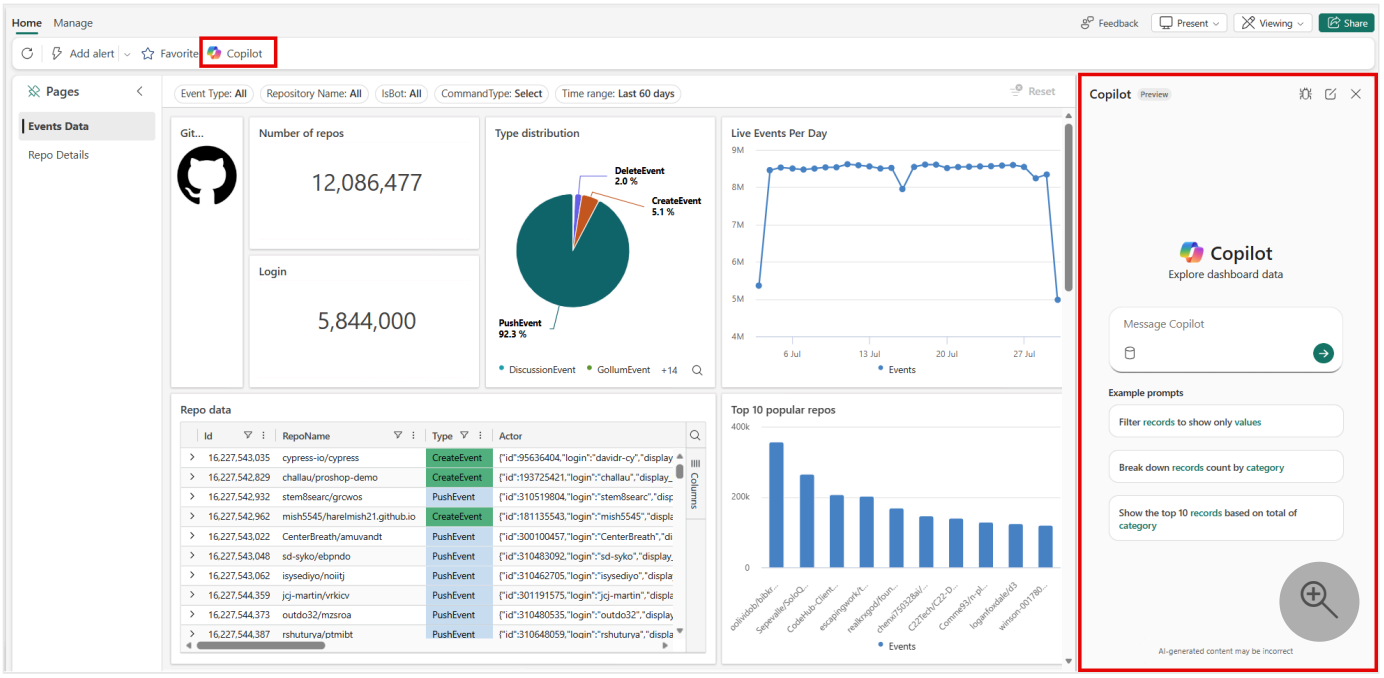
- A workspace with a Microsoft Fabric-enabled capacity.
- A dashboard with visuals. For more information, see Create a Real-Time dashboard.

Explore data with Copilot

Use Copilot to explore live dashboard data by using natural language. For example, you can:

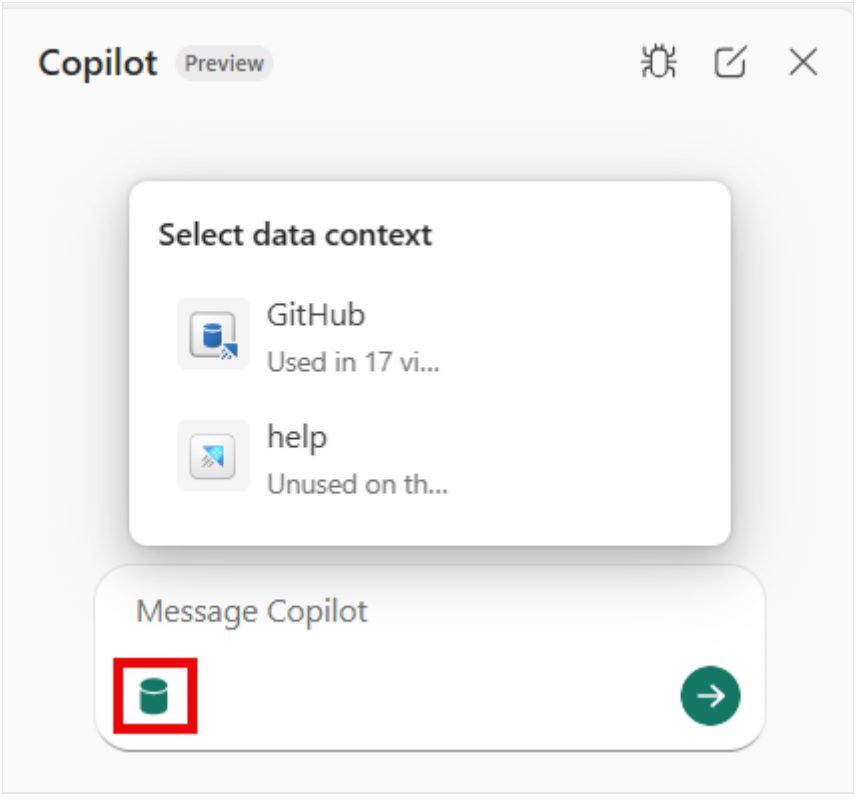
- Change the time range.
- Filter by a column or value.
- Calculate averages, totals, or counts.
- Group results by a column.
- Summarize trends or anomalies across the data.

While Copilot analyzes your request, it provides progress updates so you can follow the exploration process. After the analysis finishes, Copilot summarizes the findings and suggests follow-up questions to help you continue exploring.



In your Fabric workspace, select a Real-Time dashboard, or create a new dashboard, and ensure you're in **Viewing** mode. Use the following steps to explore your data with Copilot:

1. If your dashboard contains multiple data sources, optionally select the data source you want to explore. Copilot uses the selected source as the context for the conversation and provides example prompts tailored to the available data.

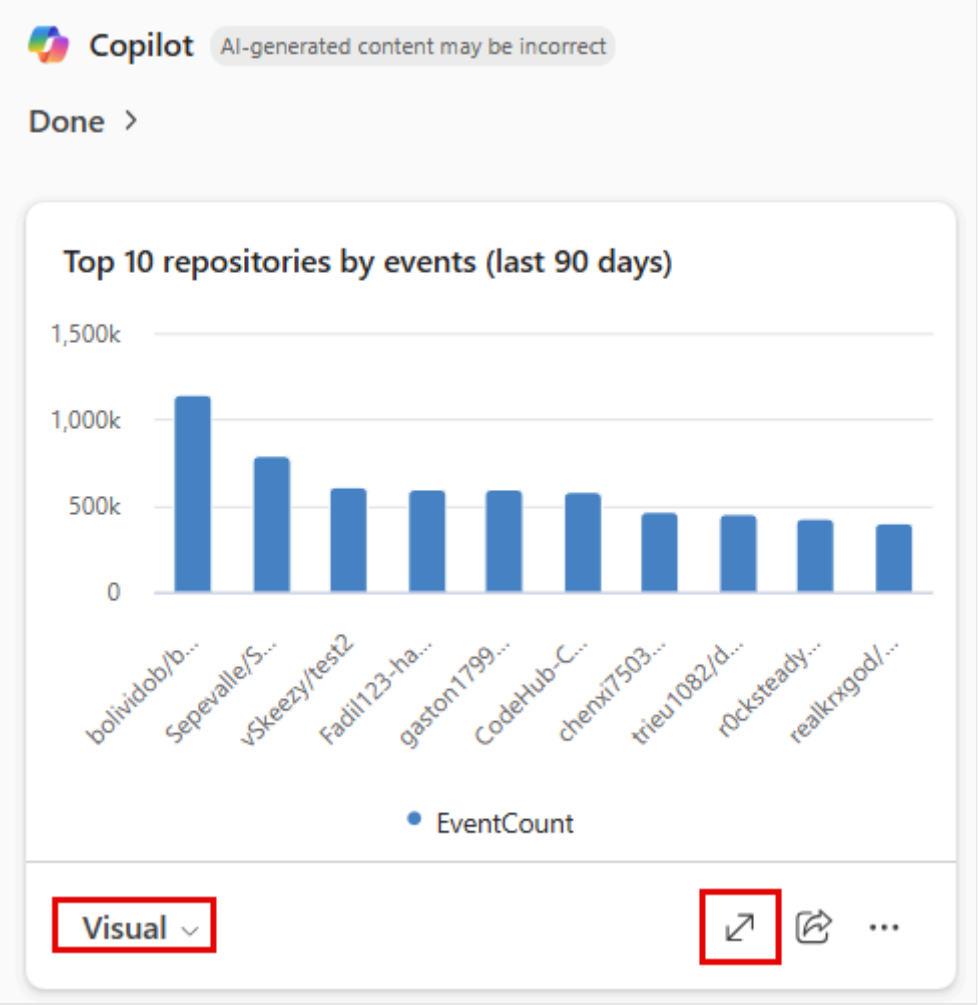


2. Ask a question about the data. For example, "What are the top 10 popular repos?" or "What is the sign-in success rate?".

3. Review the progress updates displayed by Copilot while it analyzes the request.

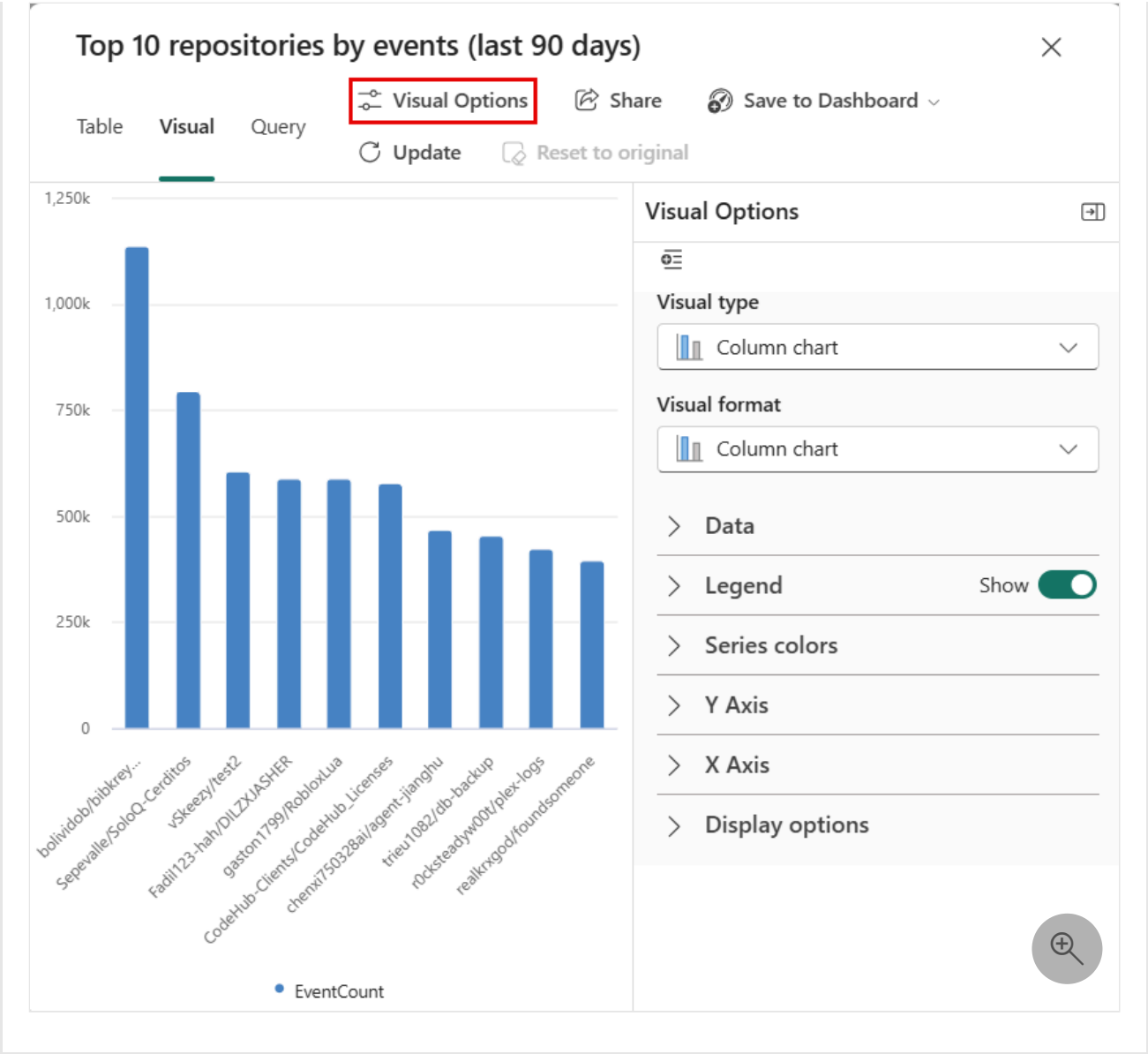
4. When the analysis is complete, review the generated results, summary, and suggested follow-up prompts. Continue exploring by selecting a suggested prompt or entering your own question.

5. Select the maximize icon to see the results reflected in the **Visual**, **Table**, and **Query** tabs, or use the drop-down menu to choose a specific tab.



Visual

The **Visual** tab displays a visualization generated from the query results. Select the **Visual Options** button to customize the visual format and appearance. For more information, see Customize Real-Time Dashboard visuals.



6. Refine the results by asking more questions or Save and share Copilot exploration insights as needed.

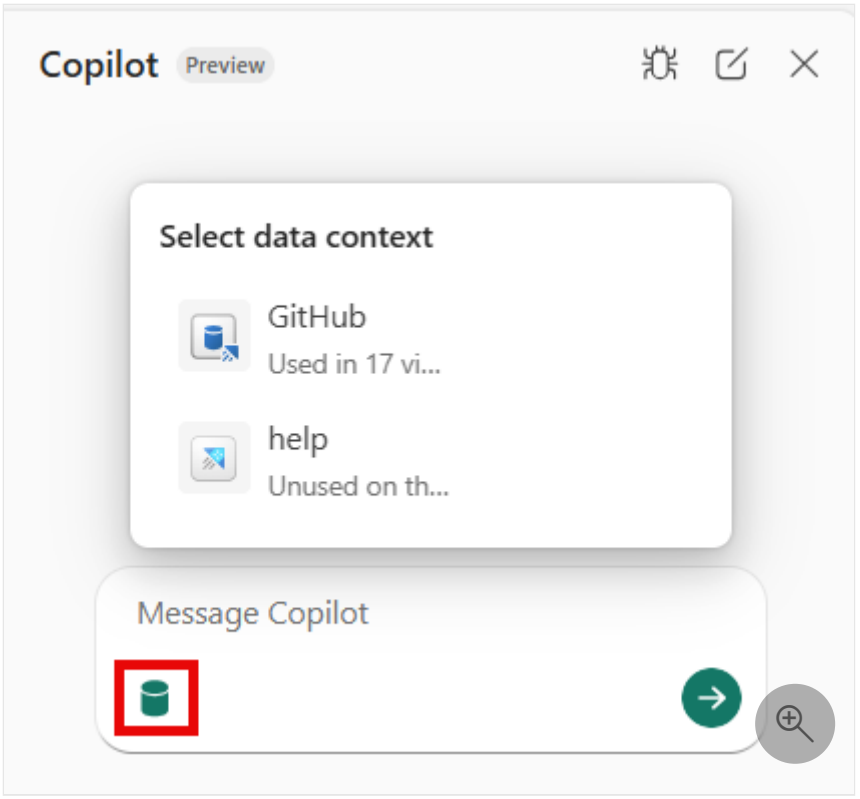
Explore the entire dashboard

By default, Copilot explores data in the context of the entire dashboard. This approach lets you ask questions that span multiple visuals and uncover insights across all dashboard data.
For example, you can ask:

- Which event types are increasing the most across all visuals?
- What unusual patterns appeared in the last 24 hours?
- How did overall performance change compared to the previous week?

Explore a specific data source

In the Copilot side pane, select a specific data source when you want to focus your investigation on a particular dataset rather than the entire dashboard.

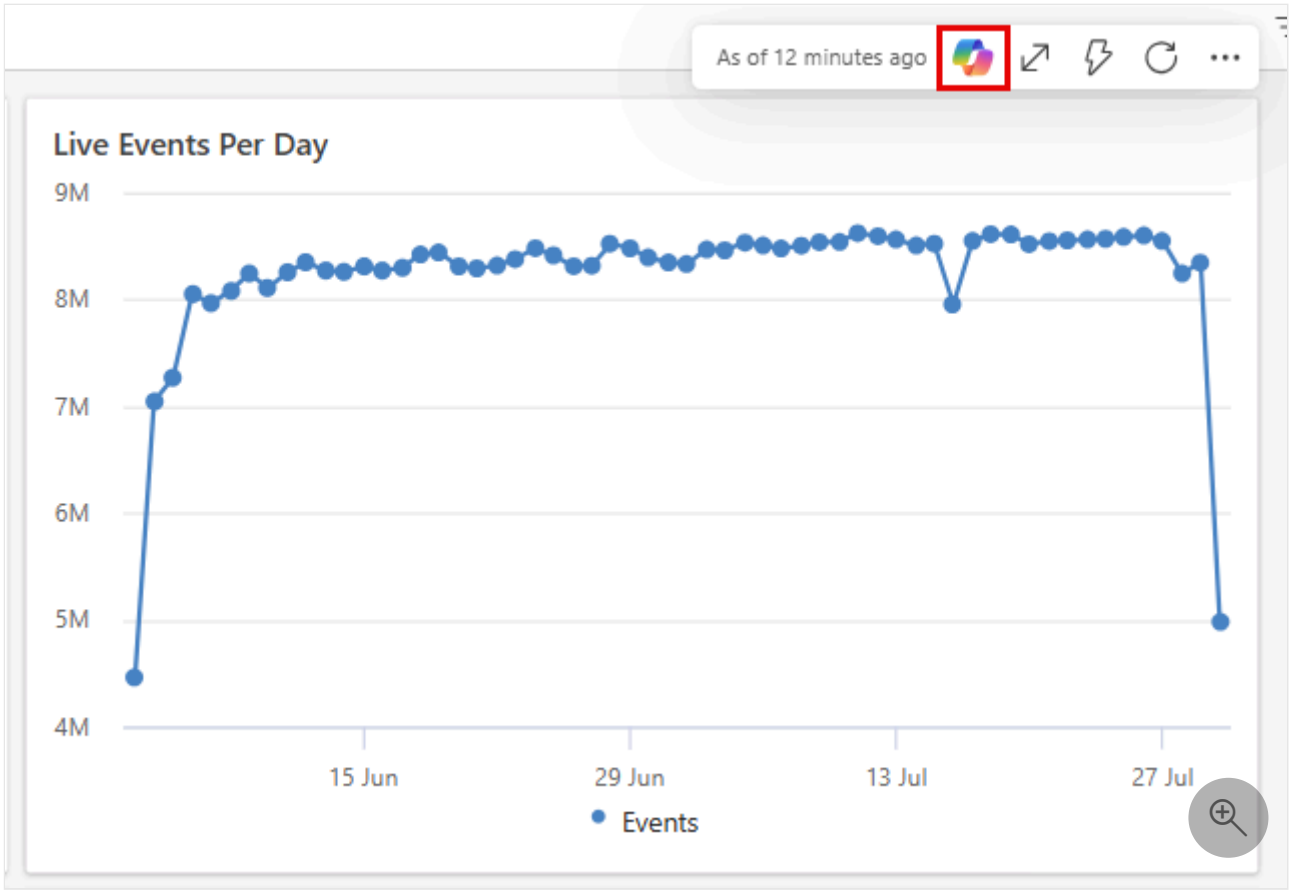


Copilot considers the selected data source as the context for the conversation and generates example prompts tailored to that data source. This approach helps you ask relevant questions and quickly discover insights.

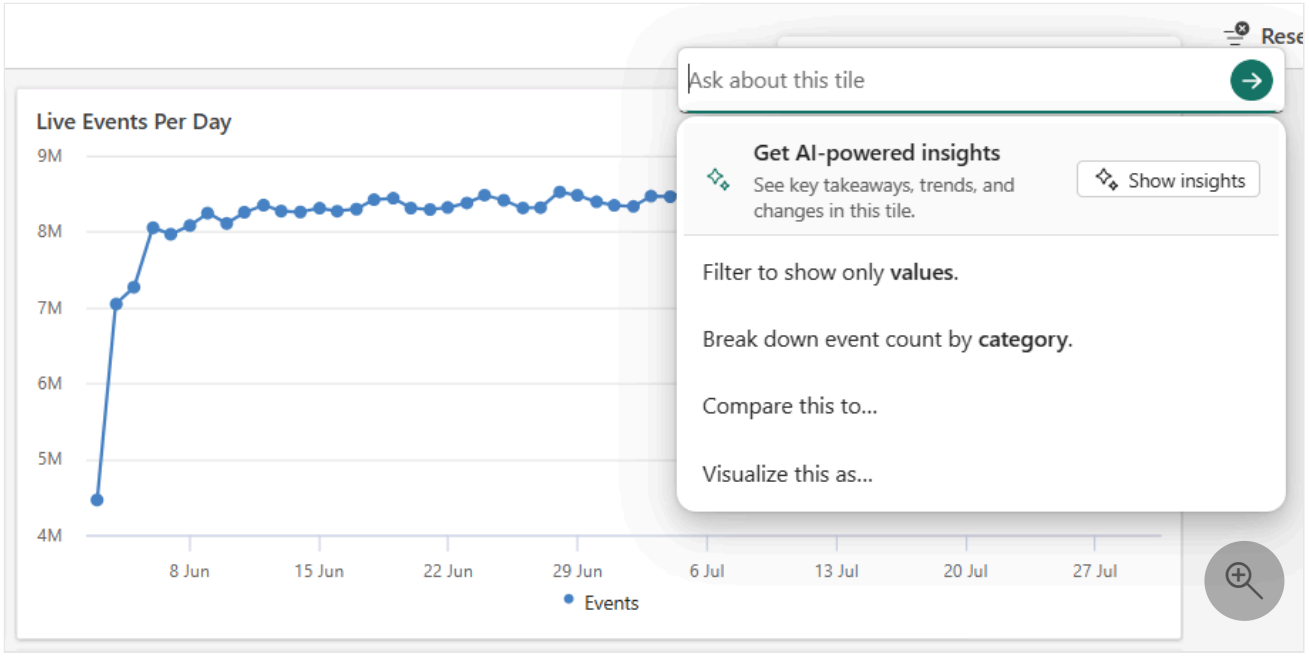
Explore data from a specific visual

You can also begin an exploration from a specific visual. The selected visual provides context for your questions, so you can naturally refer to "this chart" or "these results." Copilot uses that context to understand your intent while analyzing the underlying dataset behind the visual, rather than only the data shown in the visual.

1. Select the Copilot icon on the visual to explore the data.



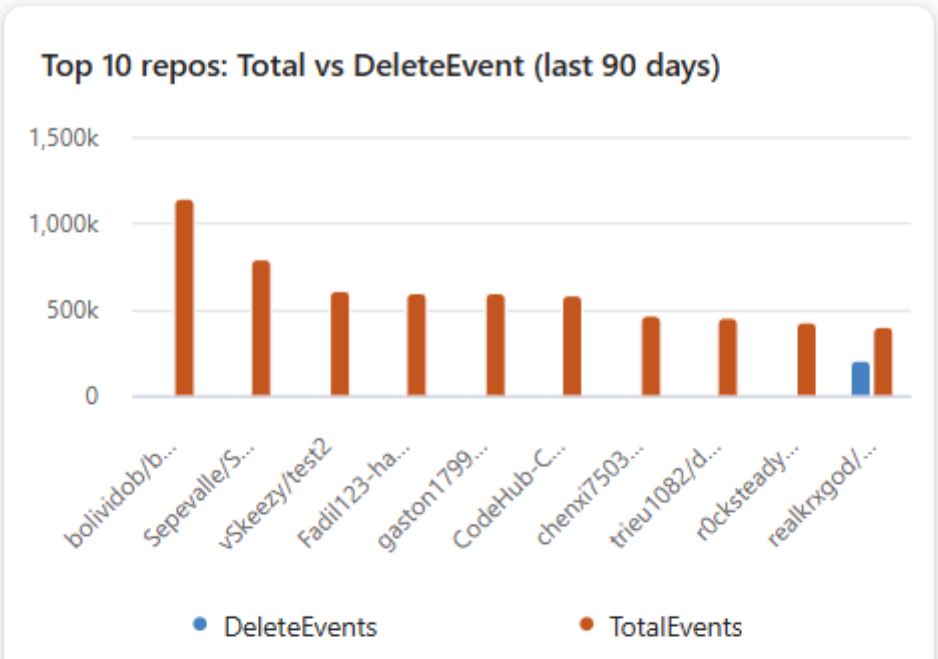
2. A Copilot pop-up dialog opens with suggested prompts to help you get started exploring the visual's data.



3. In the side pane, follow Copilot's thought process as it analyzes the data and generates insights.

4. Review the results and any generated visuals.

Copilot Preview



Visual

This is a column chart comparing **Total events vs DeleteEvent events per repo** over the **last 90 days** (for the same top 10 repos):

- **X-axis**: Repository (`RepoName`)
- **Y-axis**:
  - `TotalEvents` : all event types
  - `DeleteEvents` : only events where `Type == "DeleteEvent"`

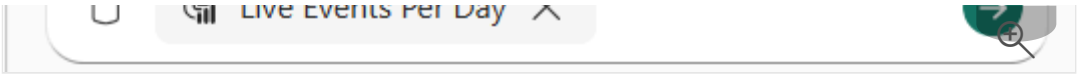
From the data:

- Most top repos have **zero DeleteEvents** despite very high overall activity.
- `realkrxgod/foundsomeone` stands out with a large number of `DeleteEvents` (while still being in the overall top 10).

If you'd like, I can:

- Filter to only repos with **non-zero DeleteEvents** and rank those, or
- Break out other event types (e.g., `PushEvent` , `IssueCommentEvent`) for a multi-type comparison.

Message Copilot



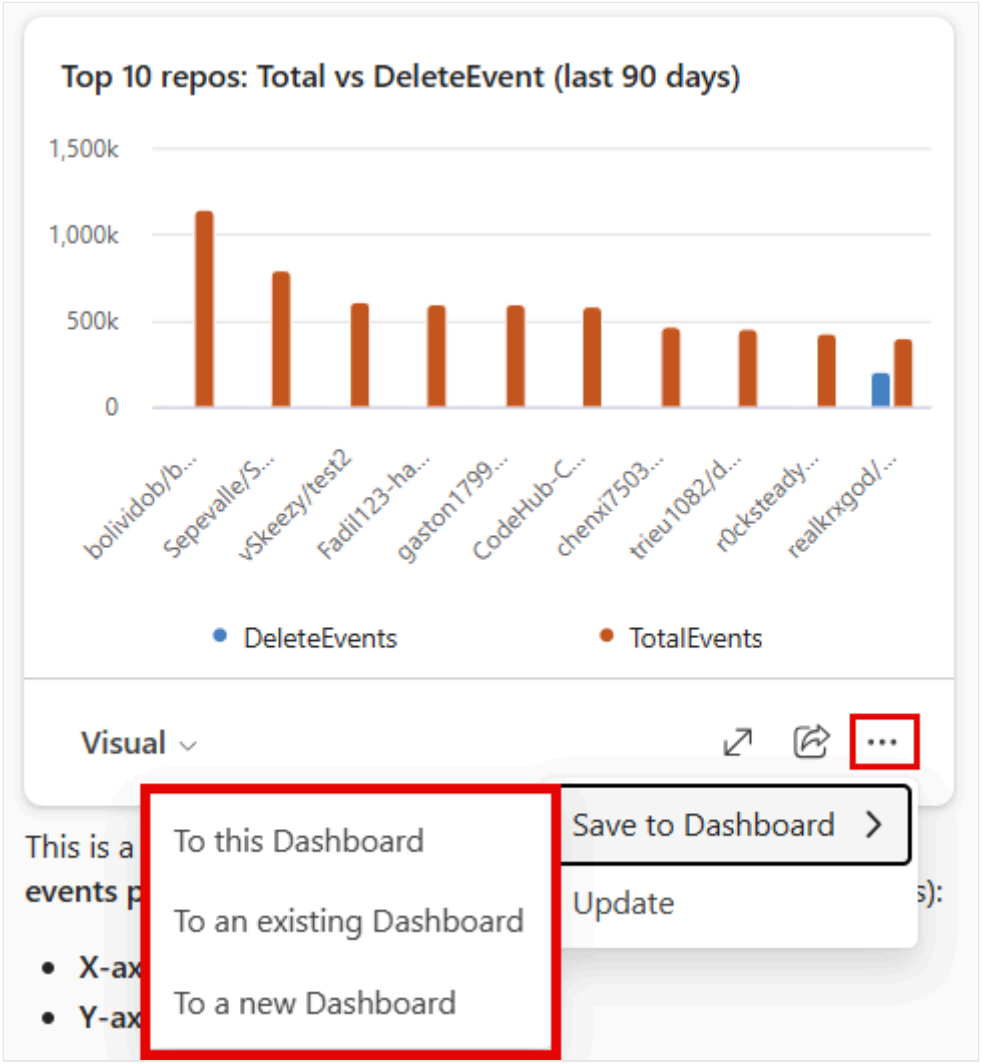
5. Save or share the generated insights.

Save and share Copilot insights

After discovering an insight with Copilot, you can save it to a dashboard or share it with others.

Save insights to a dashboard

Select **Save to dashboard** to save the current visualization and query as a dashboard tile. You can save the tile to the current dashboard, an existing dashboard, or a new dashboard.

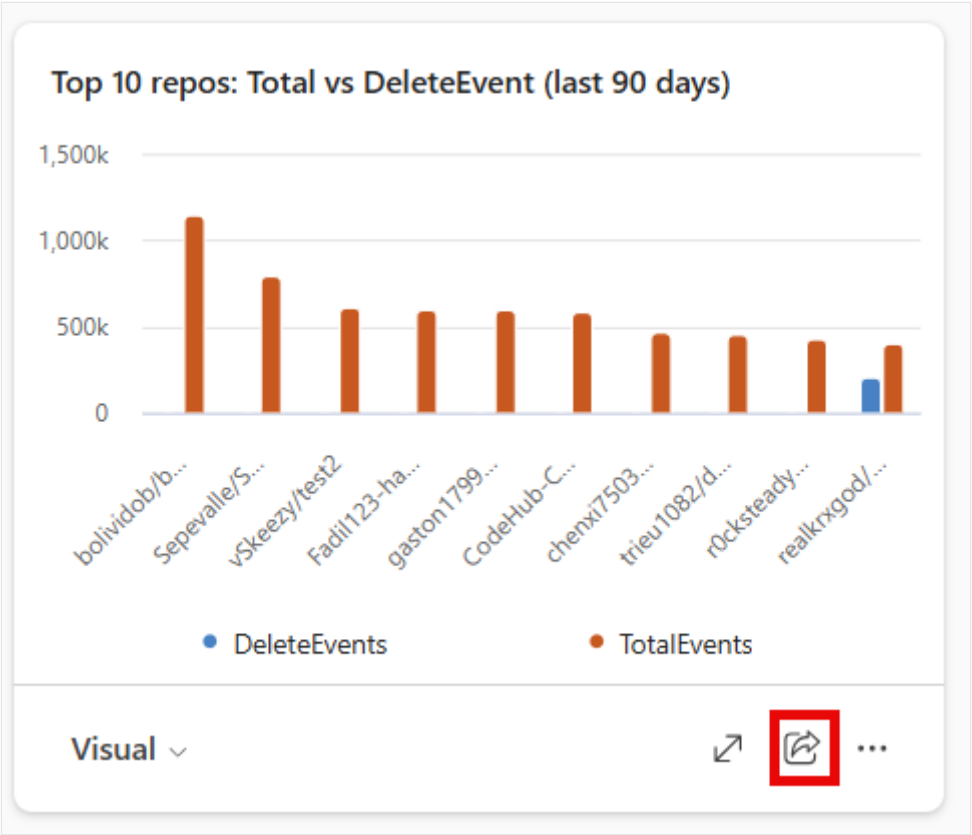


Saved tiles stay connected to the underlying live data and continue updating as new data arrives.

Share insights with others

You can share a link to the Copilot exploration with other users.

1. Select the **share** icon in the Copilot pane or in the expanded view.



2. In the share dialog, choose whether to include the visual in the shared insights, and then select **Copy link**.

3. Share the copied link with others. Recipients can view the results, rerun the query, save insights to a dashboard, and customize the visual when one is included.

Related content

- Create a real-time dashboard
- Customize real-time dashboard visuals
- Use Copilot to edit tiles

ⓘ**Note:** The author created this article with assistance from AI. Learn more

Last updated on 10/04/2026

Use the Copilot for Data Engineering and Data Science chat pane (preview)

> **Important**
>
> This feature is in preview.

This article covers both ways to interact with Copilot in notebooks: the **chat pane**, which opens on the right side of your notebook for multi-step conversations, and **in-cell Copilot**, which lets you generate code or run slash commands directly above a code cell.

For an overview of Copilot capabilities in Data Science and Data Engineering, see Overview of Copilot for Data Engineering and Data Science.

When you open a notebook, Copilot automatically uses notebook context such as your workspace, attached Lakehouse, available schemas, tables, and files, existing notebook code, and runtime.

Copilot supports notebook-wide, multi-step code generation, refactoring, summarization, and validation across entire workflows, not just single cells or isolated prompts. It can coordinate changes across multiple cells in a session, helping you build and optimize end-to-end pipelines without losing context.

Prerequisites

Copilot must be enabled for your tenant and your workspace must be on a supported capacity. If your capacity is outside the US or EU, your Fabric admin might need to enable more tenant settings for cross-geo data processing.

For full requirements, see prerequisites in the Copilot overview.

Get started

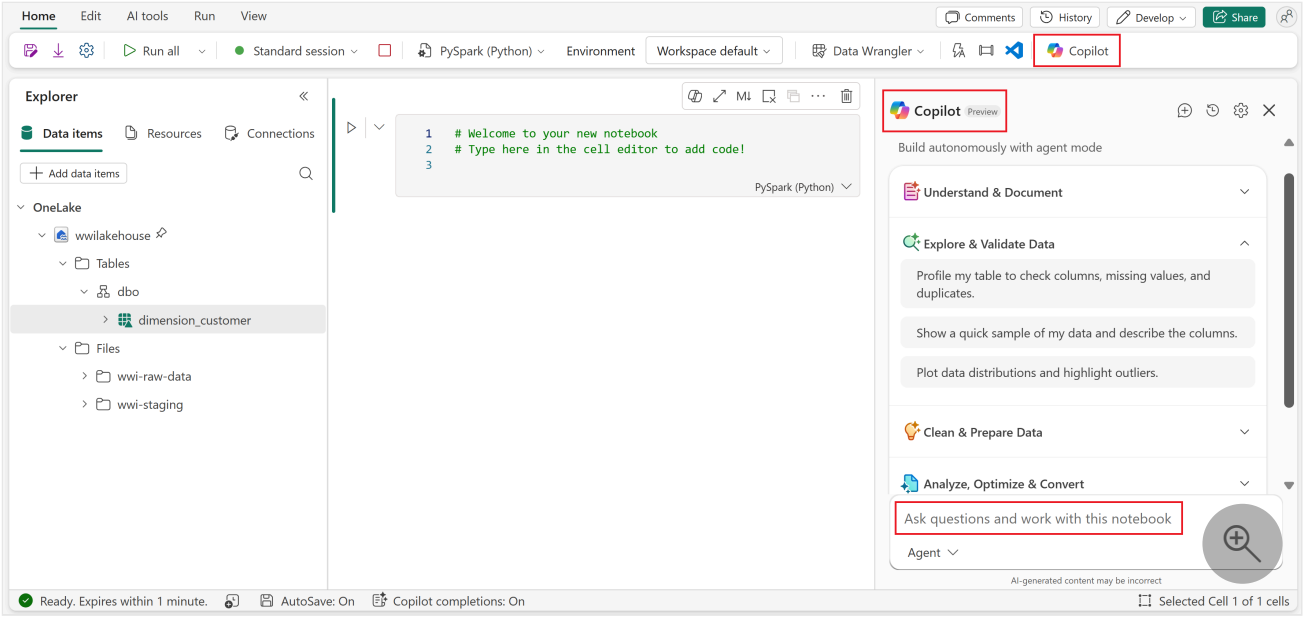
You don't need to install anything or start a session. Copilot is ready to use as soon as you open the pane.

To use Copilot in Fabric notebooks:

1. Create a new notebook or open an existing one.

2. Attach a Lakehouse to provide schema and data context.

3. Select the **Copilot** button on the notebook ribbon.



4. The Copilot chat pane opens on the right side of your notebook.

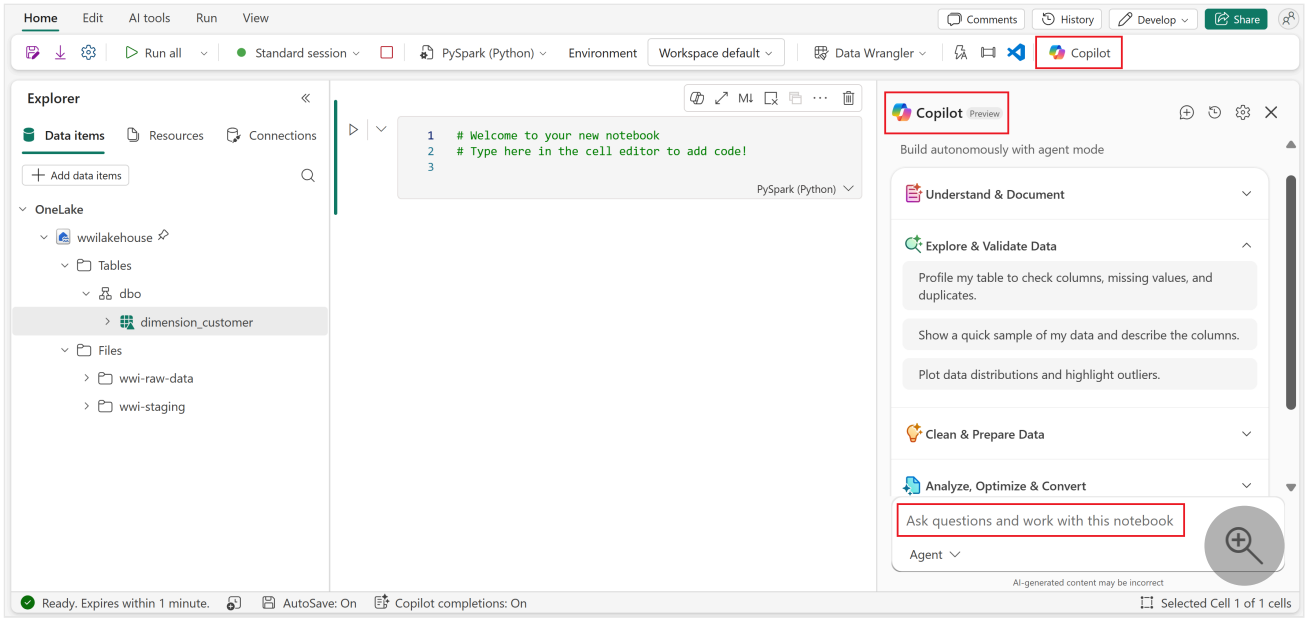
5. Enter a prompt or select a suggested starter prompt.

For detailed instructions, example prompts, and a walkthrough of the chat pane experience, see the example walkthrough section.

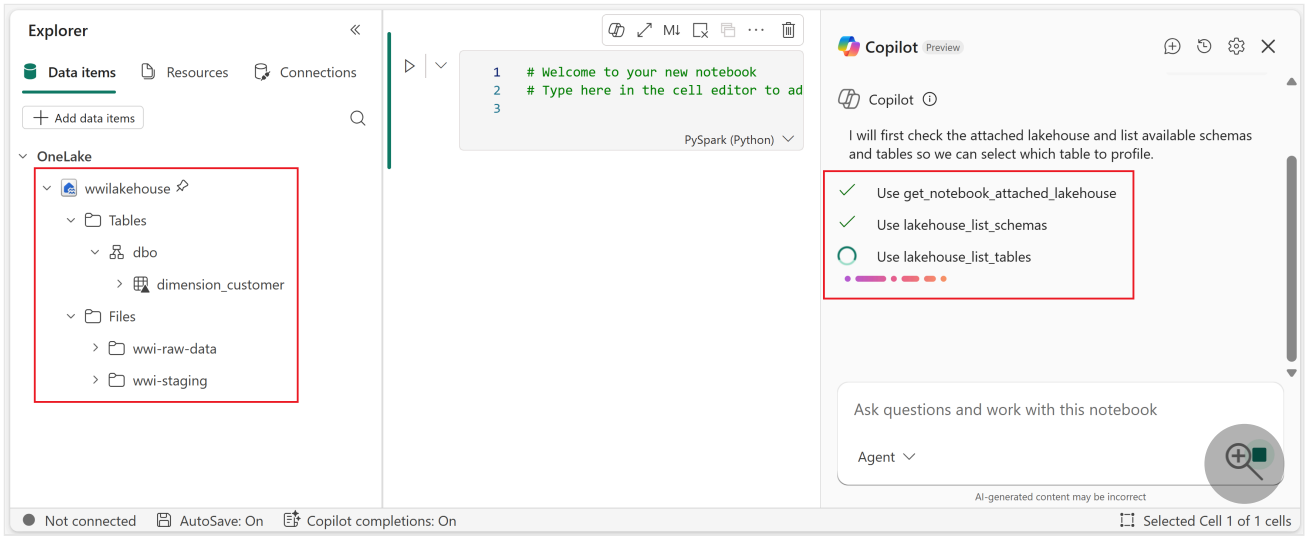
Example walkthrough

The following walkthrough shows one example of an end-to-end Copilot flow, from selecting a starter prompt through reviewing and approving changes. Your experience might vary — Copilot responses depend on your data, notebook context, and how you phrase your prompts.

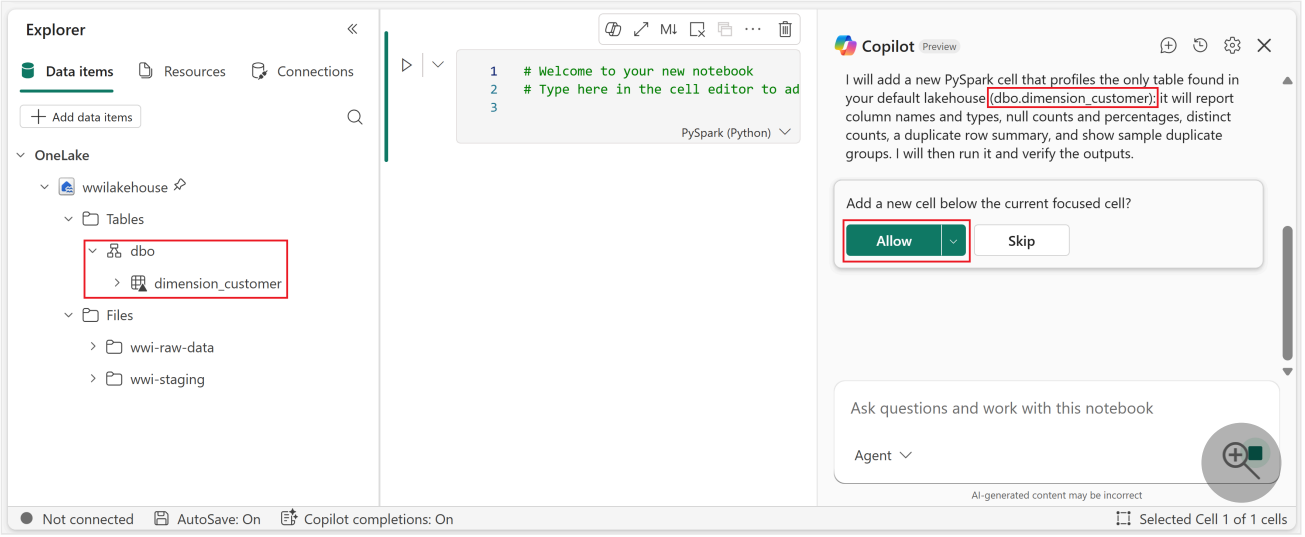
1. Open the notebook and select **Copilot** from the ribbon to open the chat pane. Then choose one of the prebuilt starter prompts or type a custom question in the chat box.



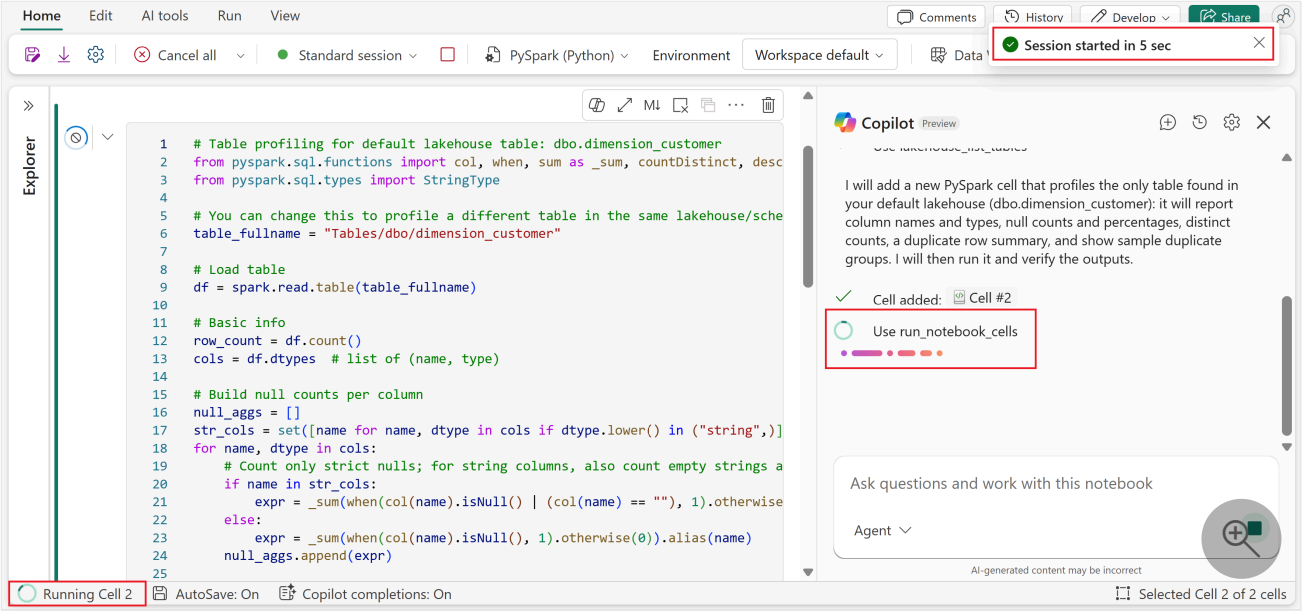
2. In this example, we select the starter prompt **"Profile my table to check columns, missing values, and duplicates"** under **Explore and Validate Data**. Copilot automatically uses notebook context — the attached Lakehouse, available schemas, and tables — to determine which table to profile. You don't need to specify the data source; Copilot discovers it from your workspace.



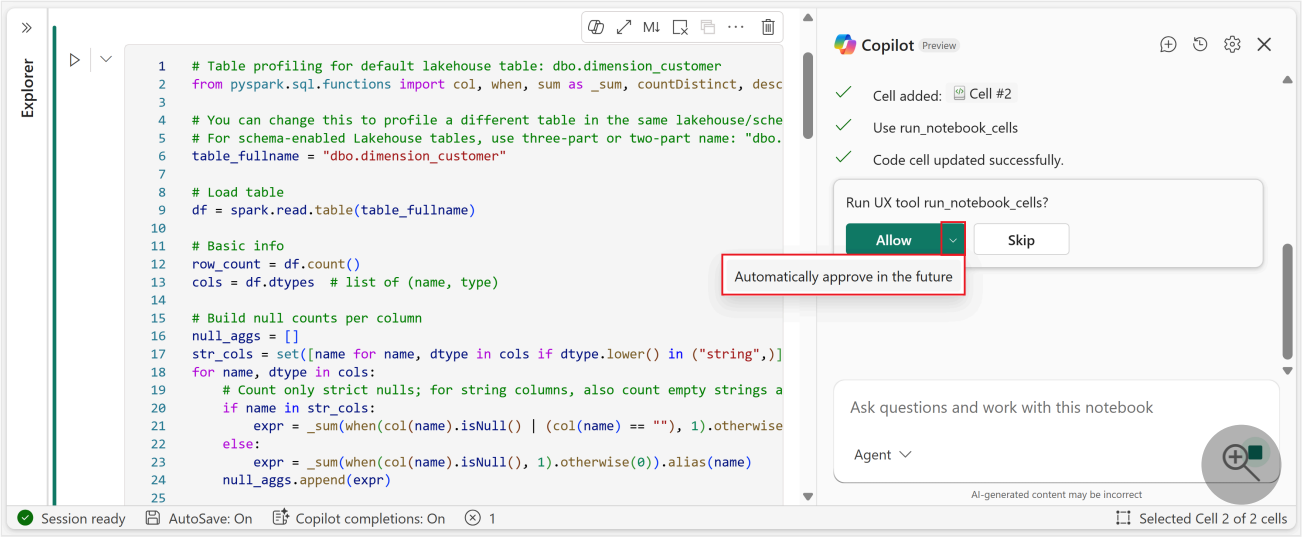
3. Copilot finds the dimension_customer table in the default Lakehouse and requests permission to add a new code cell.



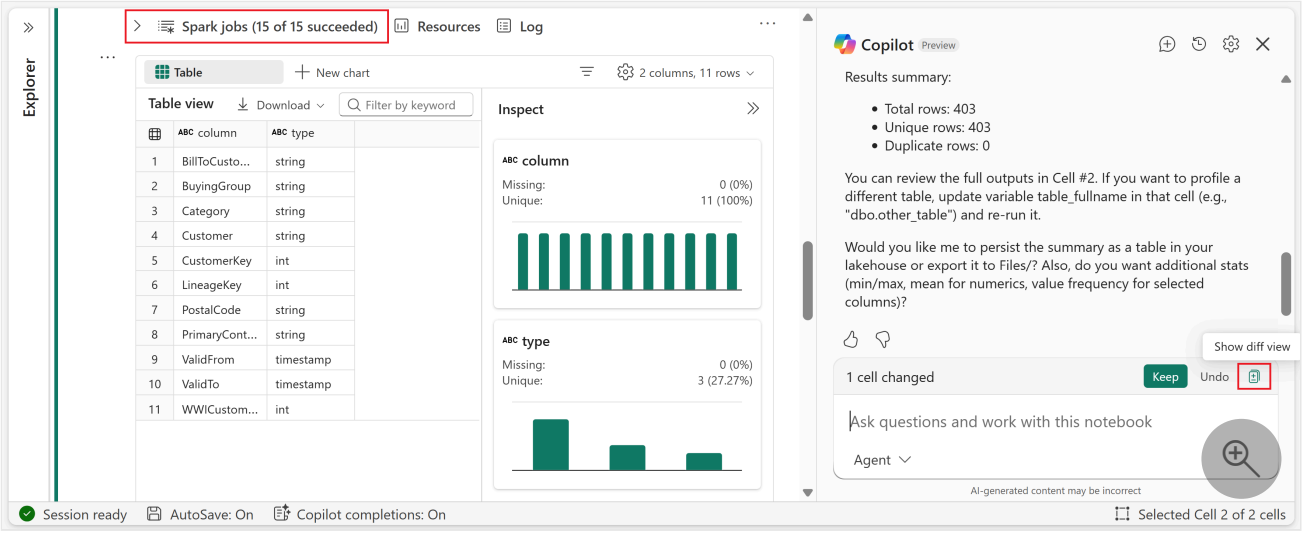
4. After you approve, the Spark session starts and Copilot runs the code cell it created.



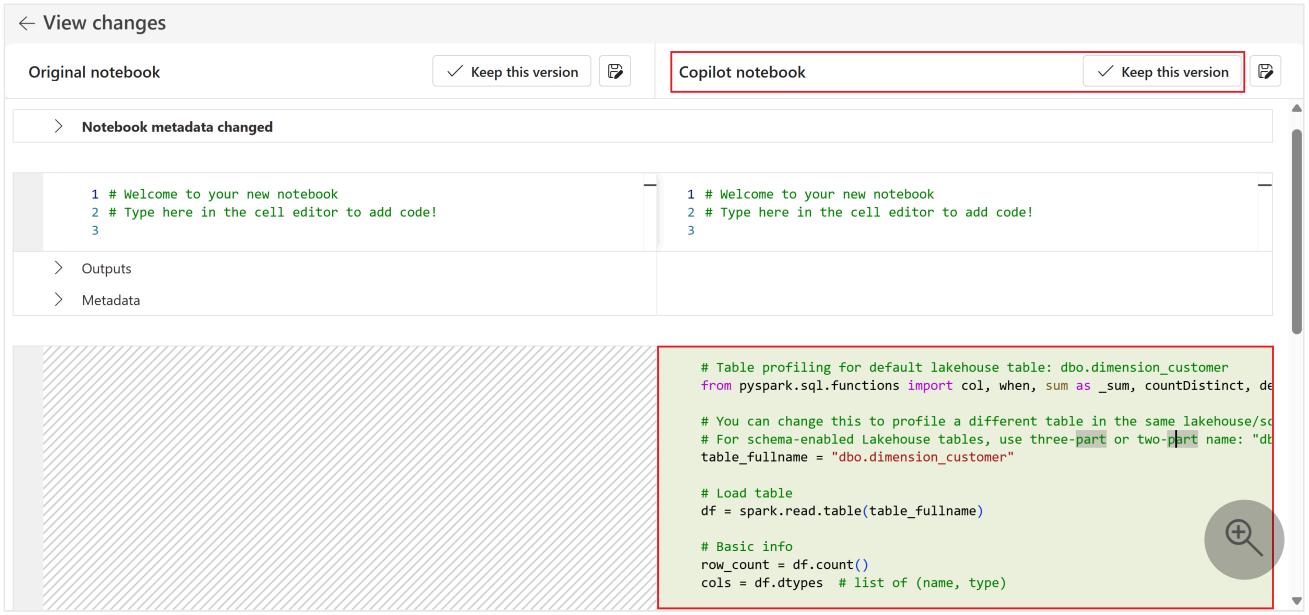
5. Copilot requests permission to edit code or run cells. You can **Allow** the action, **Allow and automatically approve** similar permissions in the future, or **Skip** to prevent Copilot from running the tool. You can change the default approval behavior at any time — see Approval settings for details.



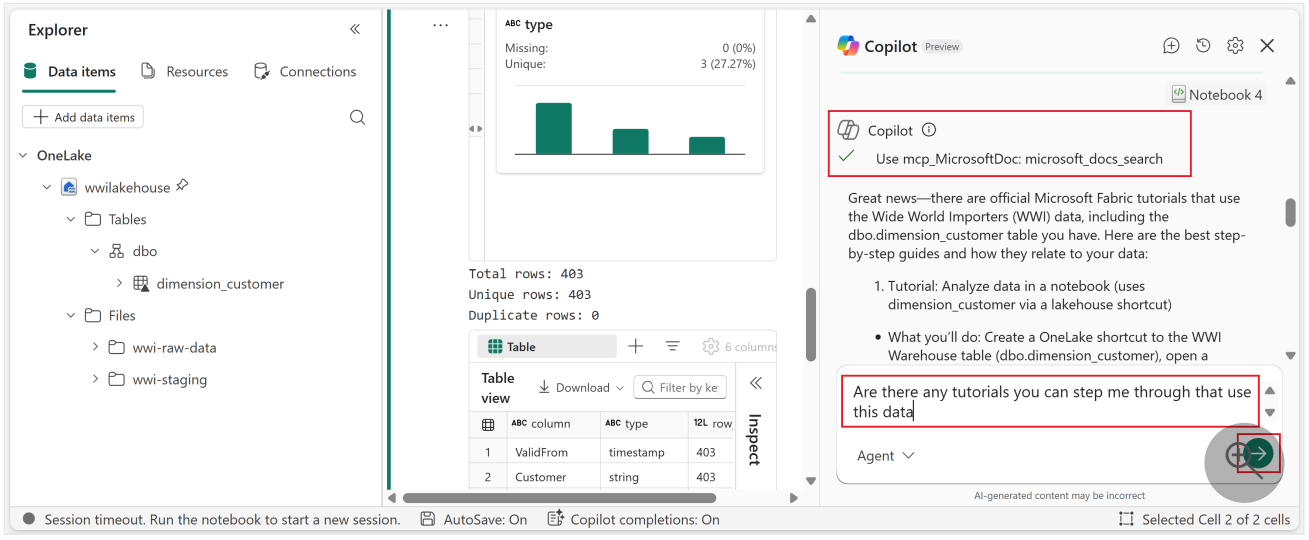
6. After the Spark job completes, you can choose to **Keep** or **Undo** the changes to the notebook. You can also open the **diff view** to see exactly what Copilot changed.



7. The diff view shows the original notebook contents on the left and Copilot's edits on the right. Each side has a **Keep this version** button. Select the version you want to keep — either the original or Copilot's version. You can also go back without choosing either option.



8. You can also type a custom question in the chat box. In this example, Copilot uses one of several tools available to it from MCP servers to process the request. Here, it uses the `microsoft_docs_search` tool to find relevant information.



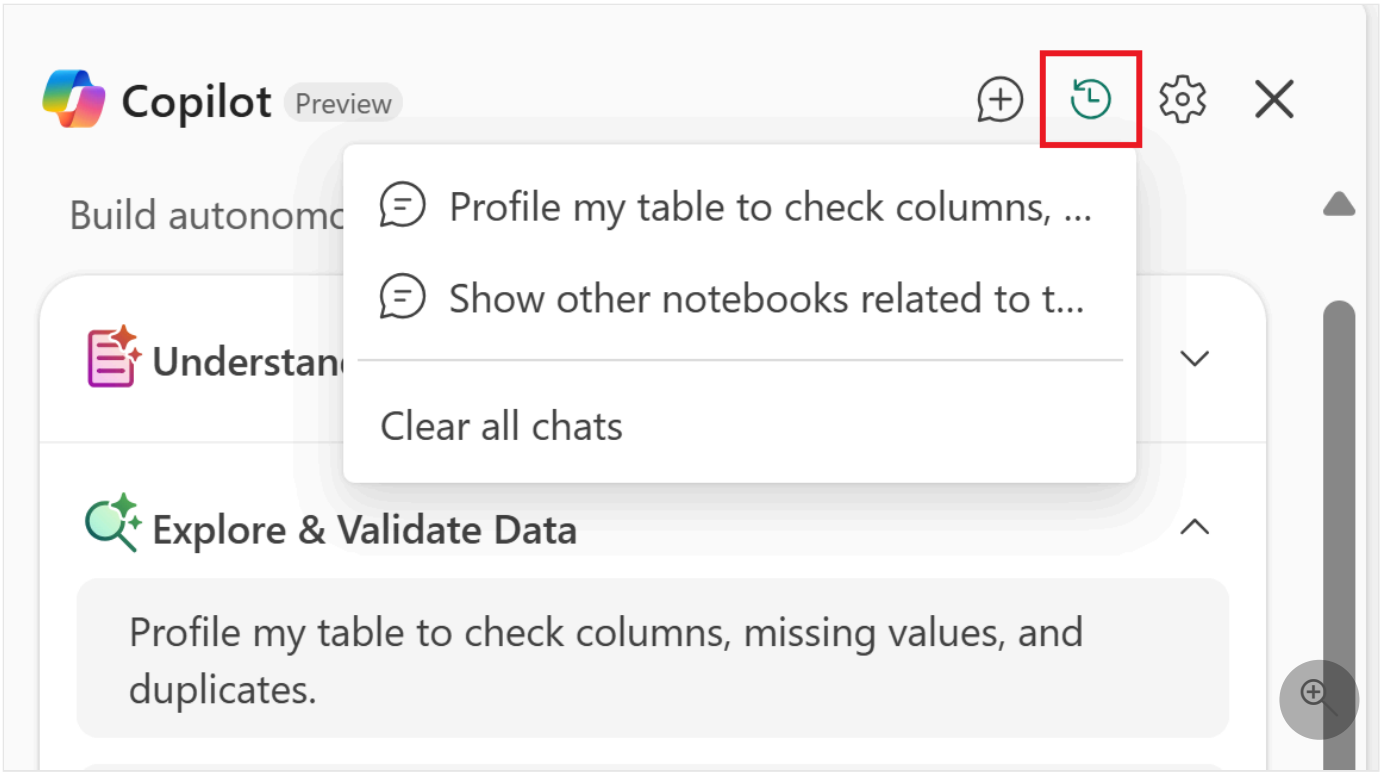
This walkthrough covers the core chat pane flow. The chat pane also supports chat history for reviewing previous conversations, and you can interact with Copilot directly within cells using in-cell Copilot for tasks like fixing, explaining, or optimizing code.

Performance insights and optimization

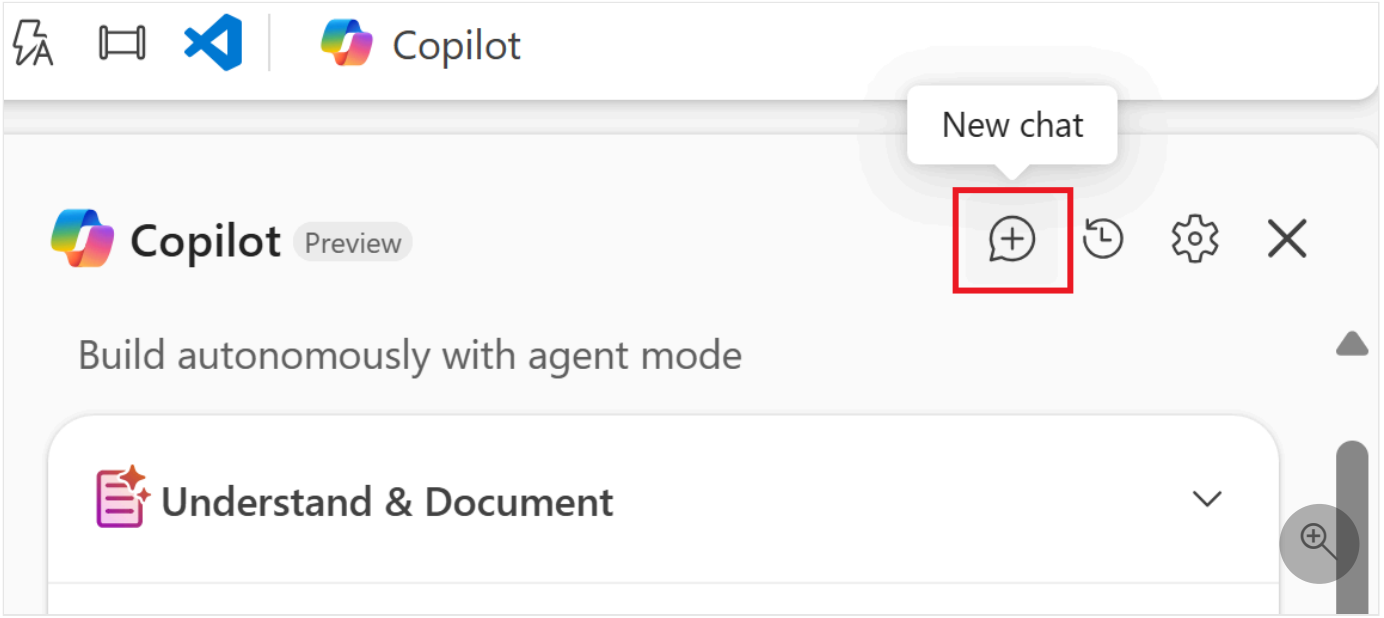
When you ask Copilot for optimization help, it can provide recommendations based on data size, join patterns, and runtime behavior. For example, it can suggest more efficient join strategies, help avoid unnecessary shuffles, identify opportunities to refactor into reusable functions, and highlight data quality issues that affect performance or correctness. You can surface these insights during multi-step conversations or by using the `/optimize` command.

Chat history

Copilot preserves your chat history across sessions. You can view previous conversations by selecting the chat history icon in the chat pane.

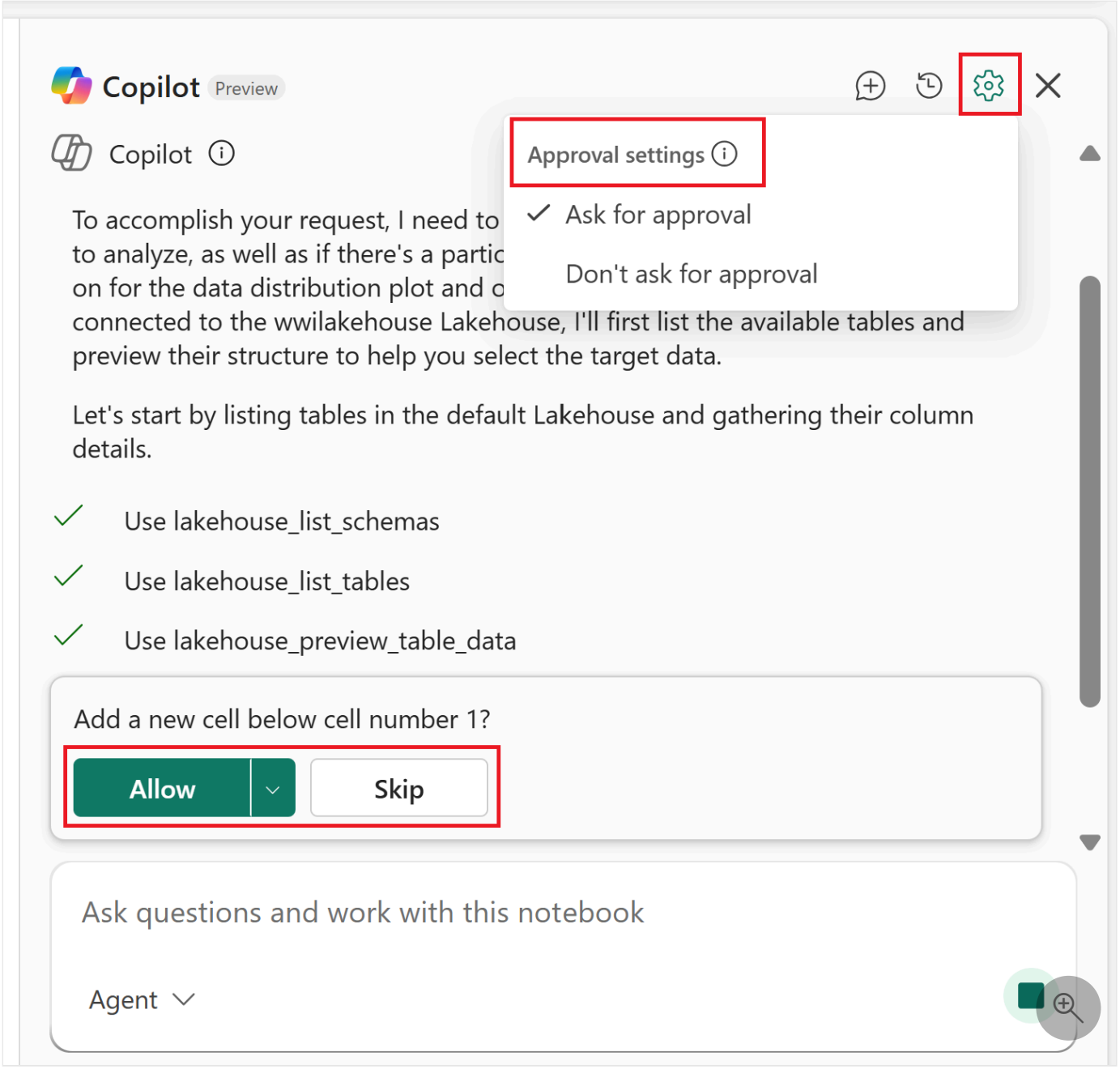


To start a new conversation, select the new chat option.



Approval settings

Copilot includes approval settings that control whether it asks for confirmation before running cells. To change approval settings, select the settings gear icon in the chat pane.



The available options are:

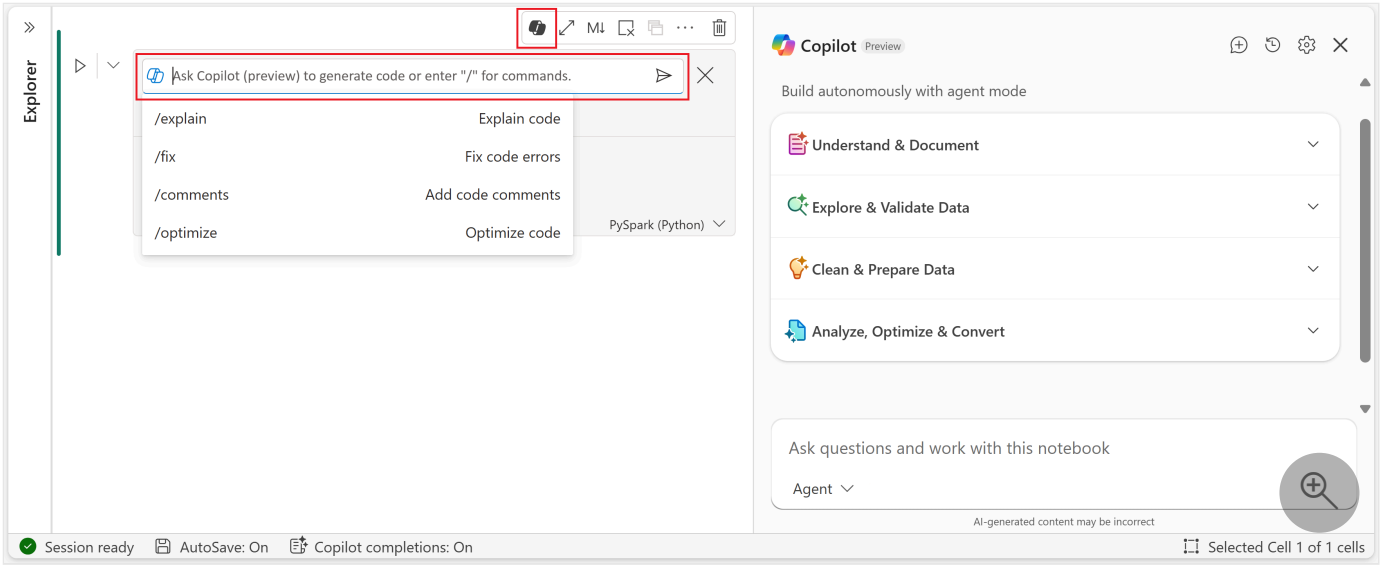
- **Ask for approval** — Copilot asks for confirmation before every cell run.
- **Don't ask for approval** — Copilot runs cells automatically.

High-risk actions, such as running multiple cells at once or installing packages, always require approval regardless of your setting.

When Copilot recommends code changes (for example, through Fix with Copilot or optimization suggestions), it can auto-apply the changes when approved. Copilot always presents an approval diff for review so you can inspect the proposed changes before they're committed. After applying, you can still **Keep** or **Undo** the changes.

In-cell Copilot

In addition to the chat pane, you can interact with Copilot directly within notebook cells. This experience is ideal for quick, targeted actions on a single cell. Select the Copilot button above a code cell to open a text box where you can enter a request or slash command. For example, enter "Generate code for a logistic regression that fits this data" and Copilot writes the code in the cell below.



You can also use the following slash commands for specific actions on existing code:

- `/explain` — Explain code. Provides a plain-language explanation of any code block.
- `/fix` — Fix code errors. Identifies errors and suggests corrections.
- `/comments` — Add code comments. Automatically documents your code with summaries of logic and data changes.
- `/optimize` — Optimize code. Suggests improvements for performance and efficiency, including join strategy selection, shuffle reduction, function refactoring, and detection of potential data quality issues that affect performance or correctness.

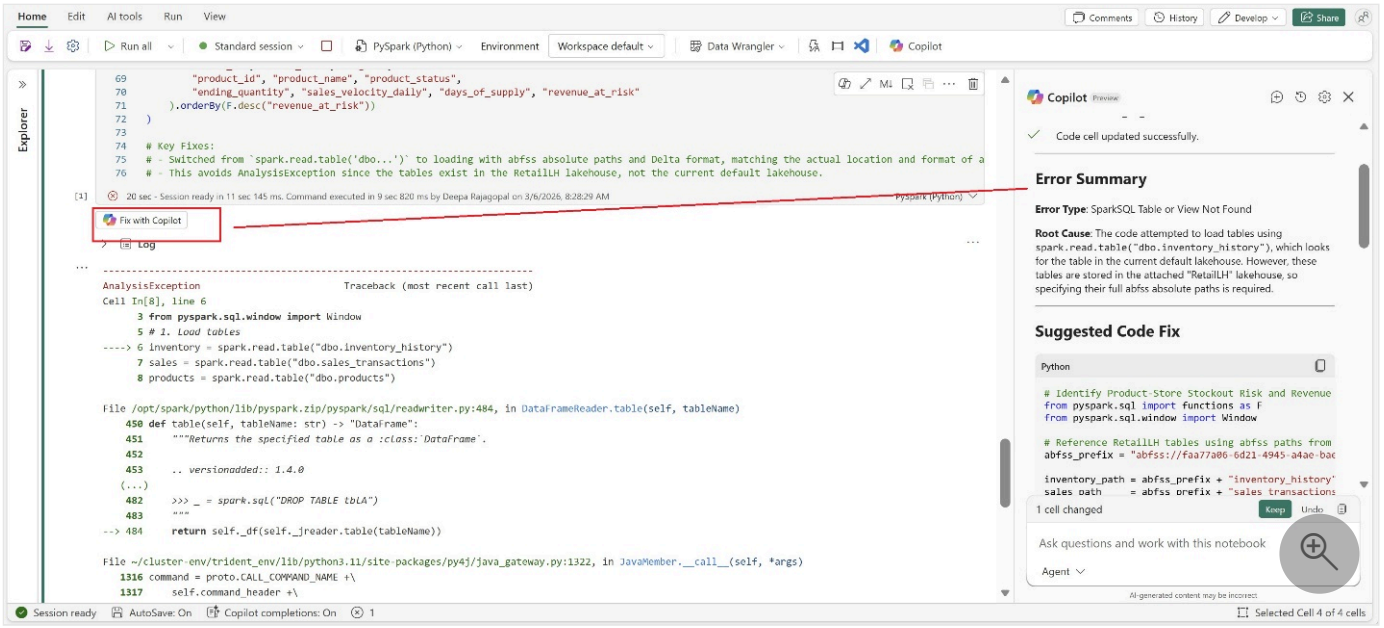
Fabric notebooks also offer inline code completion, which provides AI-powered autocomplete suggestions as you type in code cells.

Diagnose notebook failures

When a notebook cell fails, Copilot can help you diagnose and fix the issue directly in your notebook workflow.

Use Fix with Copilot for cell failures

After a cell execution failure (including Spark job failures surfaced in notebook execution), a **Fix with Copilot** option appears below the failed cell.



When you select **Fix with Copilot**, Copilot uses notebook context such as:

- Code from the failed cell.
- Runtime and execution context.
- Spark execution details and error logs.

Copilot then provides:

- An error summary.
- A likely root cause.
- Recommended next steps.

If a code change is needed, Copilot can suggest an updated version. Review the change in diff view, then choose whether to keep or undo it.

Use `/fix` for targeted or broader troubleshooting

You can also troubleshoot from Copilot chat or in-cell Copilot by using `/fix`.

- Use `/fix` on a specific cell for a focused issue.
- Use `/fix` from chat to run broader diagnostics across the entire notebook, not only a single cell. Copilot can provide a consolidated summary, root-cause analysis across steps, and propose coordinated fixes spanning multiple cells when appropriate.
- Use chat context to continue investigating related failures across multiple cells.

Current behavior

Fix with Copilot is available when a failure occurs in the current notebook session. If you reopen the notebook later, the button isn't shown for previous failures.

FAQ

When does Fix with Copilot appear?

Fix with Copilot appears after a notebook cell execution fails in the current session, including failures surfaced from Spark job execution in the notebook.

What information does Copilot use for diagnostics?

Copilot uses notebook context, including failed cell code, runtime and execution context, and available Spark execution details and error logs.

Does Copilot automatically change my code?

Copilot can suggest code fixes when needed. Review the proposed changes in diff view, then choose whether to keep or undo them.

Can Copilot troubleshoot issues across multiple cells?

Yes. Use `/fix` for a specific cell, and continue in chat to investigate related failures across multiple cells.

Does Copilot replace manual debugging?

No. Copilot accelerates diagnosis and suggests fixes, but you can still inspect logs and perform manual debugging as needed.

Will Fix with Copilot still appear after I reopen a notebook?

No. Today, the button appears only for failures that occur in the current notebook session.

Related content

- Overview of Copilot for Data Engineering and Data Science
- Overview of Copilot in Fabric and Power BI
- Privacy, security, and responsible use of Copilot in notebooks

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Last updated on 05/14/2026

Overview of Copilot for Data Engineering and Data Science (preview)

> **Important**
>
> Copilot for Data Engineering and Data Science is in preview.

Copilot in Fabric is your AI partner for turning data into insights faster, with less friction.

Available across Fabric workloads, each Copilot experience is tailored to the task at hand. This article focuses on the Data Science and Data Engineering notebook experience.

In notebooks, Copilot helps you move from data to working code without breaking your flow. It understands your notebook context, including attached lakehouses, workspace tables and files, and loaded dataframes, to suggest relevant code, explain results, and troubleshoot issues so you can turn questions into trusted insights faster.

For information about Copilot in other workloads, see Overview of Copilot in Fabric and Power BI.

What you can accomplish with Copilot

Copilot supports your full notebook workflow, from exploration to optimization, so you can move from question to validated result without leaving your notebook.

When you open the Copilot chat pane, you can start with a suggested prompt or ask your own question in natural language.

With Copilot, you can:

- Accelerate development with code generation and less repetitive setup.
- Generate and refactor code across multiple cells and entire workflows, not just individual cells.
- Summarize and validate notebook logic end-to-end.
- Explore, validate, and prepare data by profiling tables, sampling data, and cleaning inconsistencies.
- When prompted, surface performance insights such as recommending efficient join strategies, avoiding costly data shuffles, refactoring into reusable functions, and

highlighting data quality issues detected during exploration.
- Resolve issues in context by understanding errors and applying suggested fixes.
- Analyze and optimize notebooks by generating metrics, exploring trends, converting code, and documenting logic for collaboration.

If you want step-by-step usage instructions, prompt examples, and portal walkthroughs, see Use the Copilot chat pane.

For failure diagnostics and recovery workflows, see Diagnose notebook failures with Copilot.

How Copilot works

When you open a notebook, Copilot automatically understands:

- The current workspace
- The attached lakehouse
- Available schemas, tables, and files
- The notebook's structure and existing code
- The execution environment
- The runtime state and recent execution telemetry (for example, data sizes and join behavior)

You don't need to describe your setup. Copilot uses this context to generate relevant, environment-aware responses as your notebook evolves. Responses and recommendations consider current runtime characteristics to propose optimizations aligned to the observed behavior.

Because Copilot is schema-aware, you can ask:

- "How many tables are in the lakehouse?"
- "What are the columns of the table customers?"
- "Create a dataframe from sales.csv."

Copilot generates code and explanations directly in the chat pane, aligned to your environment.

Work the way you prefer: chat pane and in-cell Copilot

Copilot integrates into notebooks in two complementary ways.

- **Chat pane**: Best for multi-step workflows, building pipelines across cells, exploring datasets, and reviewing generated code with diff view. The chat pane supports notebook-wide code generation and refactoring across cells, with the ability to review and apply changes using

an approval diff. It can coordinate changes spanning several cells to build or optimize end-to-end workflows.
- **In-cell Copilot**: Best for focused improvements within a single cell, such as generating code, explaining logic (`/explain`), fixing errors (`/fix`), adding documentation (`/comments`), or optimizing performance (`/optimize`).

Both experiences share the same notebook context, so you can move seamlessly between broader workflow design and targeted refinement.

For end-to-end instructions and examples, see Use the Copilot chat pane. For details on slash commands and in-cell usage, see In-cell Copilot. For inline code completion, see Copilot inline code completion.

Responsible use of AI

Copilot is a productivity tool, not a replacement for human judgment. Always review AI-generated code, explanations, and suggestions before applying them to your notebook. Copilot might produce results that are inaccurate, incomplete, or based on outdated library syntax.

To support responsible use:

- **Review all output** — Validate generated code and results against your data and expectations before running in production.
- **Control what Copilot runs** — By default, Copilot asks for your approval before running cells or editing code. Keep this setting enabled so you can review each action before it executes. For details, see Approval settings.
- **Understand data handling** — Customer data is temporarily stored and processed to detect harmful use of AI. For full details on data privacy, security, and retention, see Privacy, security, and responsible use of Copilot in notebooks.

Prerequisites

Before you can use Copilot in notebooks:

- Confirm that the **Users can use Copilot and other features powered by Azure OpenAI** tenant setting is enabled. This setting is enabled by default, but your Fabric admin might have turned it off.
- Ensure your workspace is on a supported capacity (F2 or higher, or P1 or higher).
- If your capacity is outside the US or EU, ensure your admin enables more tenant settings for cross-geo data processing and storage.

For details on capacity requirements, region availability, required tenant settings, and data processing across geographic areas, see Overview of Copilot in Fabric and Power BI. For the full list of tenant settings, see Copilot tenant settings.

Fix with Copilot

When a cell or Spark job fails, the **Fix with Copilot** action appears below the failed cell. It provides an error summary, root-cause analysis, and recommended fixes. Copilot can auto-apply code changes with an approval diff so you can review before committing. You can also invoke the `/fix` command in Copilot chat for targeted diagnostics on a specific cell or for the entire notebook.

For more information about failure diagnostics, see Diagnose notebook failures.

Known limitations

- Copilot features in the Data Science and Data Engineering experience are currently scoped to notebooks. Copilot can also read Power BI semantic models through semantic link integration.
- Copilot context awareness is optimized for lakehouse scenarios. If you're working with other data sources such as SQL databases, Copilot might not fully recognize the connection context and could default to lakehouse-oriented suggestions. Specify the data source explicitly in your prompts for the best results.
- If your tenant is configured with private link, the Copilot chat pane doesn't work. Inline code suggestions and quick actions might still work, but chat interactions fail to load.
- Code generation with fast-moving or recently released libraries might include inaccuracies or fabrications.
- AI-generated content might be inaccurate. Always review Copilot suggestions before applying them.

Copilot button is disabled in notebooks

In some cases, the Copilot button in Fabric notebooks may appear disabled (grayed out). This indicates that Copilot is not currently available in your environment due to configuration, capacity, or regional requirements not being met.

Copilot relies on several prerequisites across tenant settings, capacity, workspace configuration, and regional availability. If any of these requirements are not satisfied, the Copilot entry point will be disabled.

How to resolve

Use the table below to identify the cause and take appropriate action.

Expand table

#	Reason	User / Admin action
1	Tenant admin has not enabled Copilot. The "Users can use Copilot and other features powered by Azure OpenAI" tenant setting is turned off.	Contact your Fabric/Power BI tenant admin. Go to **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**, and enable **"Copilot and Azure OpenAI Service"**.
2	Capacity SKU does not meet the minimum requirement. Copilot requires a paid Fabric capacity (F2 or higher) or Power BI Premium (P1 or higher). Trial capacities are also supported.	Upgrade your capacity to F2+ or start a Fabric trial at Fabric Trial.
3	Cross-geo data processing not enabled. Your capacity is in a region where Azure OpenAI is not natively available, and the cross-geo setting is off.	Go to **OneLake catalog** > **Govern** > **Configurations** > **Tenant settings**, and enable **"Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance"**.
4	workspace not assigned to an eligible capacity. The workspace containing the notebook is not attached to a capacity that supports Copilot.	Move the workspace to an eligible capacity (F2+ / P1+ / Trial).
5	Copilot not yet available in your region. Azure OpenAI regional availability may limit Copilot in certain geos.	Check Copilot available regions and consider enabling cross-geo processing.

> **Tip**
>
> If you're unsure which setting is causing the issue, start by contacting your tenant administrator, as most Copilot requirements are controlled at the organization level.

Related content

- How to use the Copilot chat pane
- Overview of Copilot in Fabric and Power BI
- Privacy, security, and responsible use of Copilot in notebooks

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/29/2026

Consumption rates and billing for Copilot in Fabric

Copilot consumption is the billing model that measures how Copilot requests use Fabric Capacity Units (CUs). Each Copilot interaction is metered by the number of tokens processed—approximately 1,000 tokens equal 750 words. Input and output tokens are consumed at different rates, and prices are calculated per 1,000 tokens.

Copilot consumption rate per token

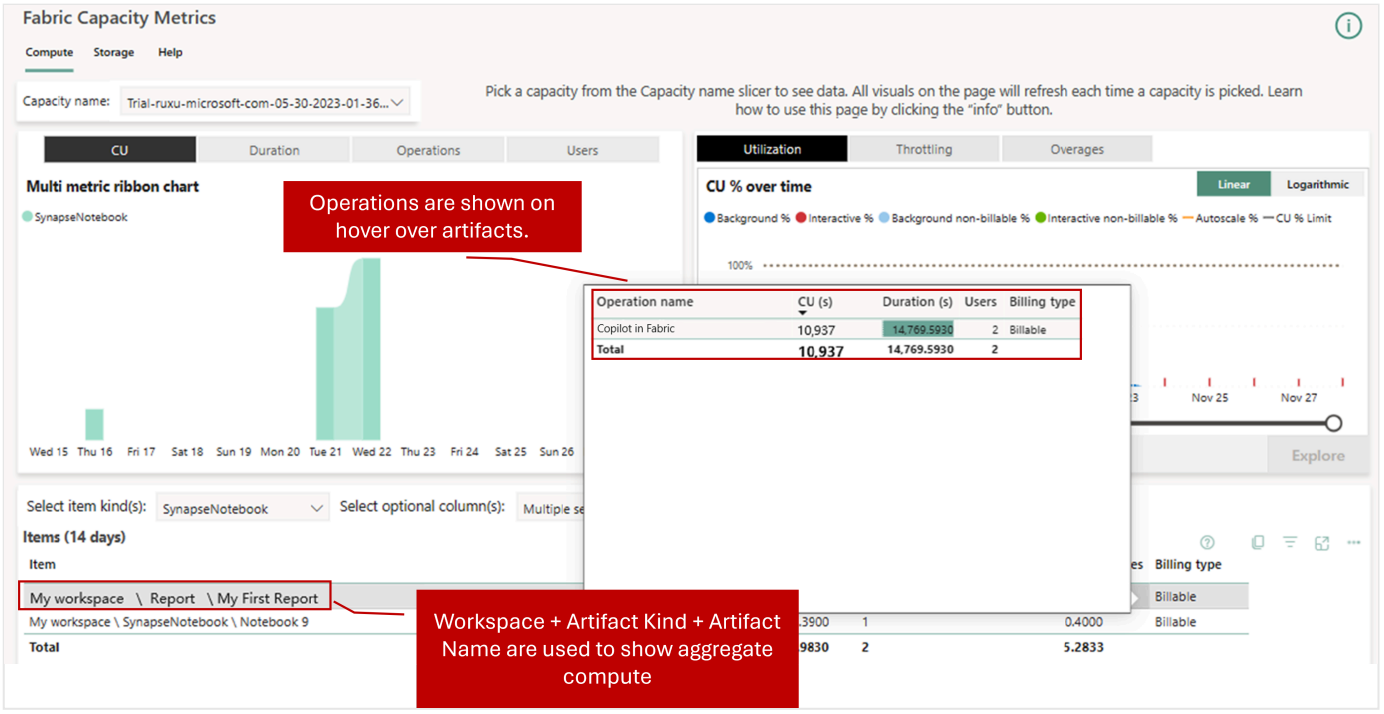
Requests to Copilot in Fabric consume Fabric capacity units (CUs). The following table defines how many CUs are consumed per 1,000 tokens for input prompts and output completions. These rates apply across all Copilot experiences, including Copilot for Power BI, Copilot for Data Factory, and Copilot for Data Engineering and Data Science.

Expand table

Operation in Metrics App	Description	Operation Unit of Measure	Consumption rate
Copilot in Fabric	The input prompt	Per 1,000 Tokens	100 CU seconds
Copilot in Fabric	The output completion	Per 1,000 Tokens	400 CU seconds

Monitor Copilot usage

The Fabric Capacity Metrics app displays the total capacity usage for Copilot operations under the operation name **Copilot in Fabric**. Copilot users can also view a summary of their billing charges under the invoicing item **Copilot in Fabric**.



Consider an incremental approach to Copilot adoption. Before you deploy AI-powered features broadly such as rolling out subscription summaries to hundreds of users or embedding AI narrative visuals on high-traffic dashboards, enable them for a small pilot group first. Use the Fabric Capacity Metrics app to measure consumption for each scenario, then extrapolate before you scale. Deploying broadly without validating consumption patterns can quickly exhaust your capacity.

Background job classification and throttling

Copilot operations in Fabric are classified as **background jobs**, which allows Fabric to handle a higher volume of Copilot requests during peak hours.

Fabric allows operations to access more CU resources than are allocated to a capacity. Fabric smooths CU usage of an interactive job over a minimum of 5 minutes and a background job over a 24-hour period. According to the Fabric throttling policy, the first phase of throttling begins when a capacity has consumed all its available CU resources for the next 10 minutes.

For example, assume each Copilot request has 2,000 input tokens and 500 output tokens. The price for one Copilot request is calculated as follows: (2,000 × 100 + 500 × 400) / 1,000 = 400.00 CU seconds = 6.67 CU minutes.

Since Copilot operations in Fabric are background jobs, each request (~6.67 CU minute job) consumes only one CU minute of each hour of a capacity. For example, a customer on an F64 SKU has 64 × 24 = 1,536 CU hours in a day. Each Copilot request consumes 6.67 / 60 = 0.11 CU

hours, so customers can run over 13,824 Copilot requests per day before they exhaust the capacity. Once the capacity is exhausted, all operations will shut down.

Cross-region data processing for Copilot in Fabric

Copilot in Fabric is powered by Azure OpenAI large language models that are deployed to limited data centers. Customers can enable cross-geo processing in tenant settings to use Copilot to process data in another region where Azure OpenAI Service is available. This region could be outside the user's geographic region, compliance boundary, or national cloud instance. Region mapping prioritizes data residency and attempts to map to a region within the same geographic area whenever feasible.

The cost of Fabric capacity units can vary by region. Regardless of the consumption region where GPU capacity is used, customers are billed based on the capacity unit pricing in their billing region. For example, if a customer's requests are mapped from `region 1` (billing region) to `region 2` (consumption region), the customer is charged based on the pricing in `region 1` (the billing region).

Changes to Copilot in Fabric consumption rate

Consumption rates are subject to change at any time. Microsoft uses reasonable efforts to provide notice via email or through in-product notification. Changes are effective on the date stated in Microsoft's Release Notes or the Microsoft Fabric Blog. If any change to Copilot consumption rates in Fabric materially increases the capacity units (CU) required to use Copilot in Fabric, customers can use the cancellation options available for the chosen payment method.

Related content

- Overview of Copilot in Fabric
- Copilot in Fabric: FAQ
- Foundry Tools in Fabric (preview)

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/11/2026

Data agent in Fabric consumption

This page contains information on how the Fabric Data agent usage is billed and reported. Data agent usage is measured by the number of tokens processed. When you query a Data Agent using natural language, Fabric generates tokens from your request and any supporting context used to process it. Your question typically represents only a portion of the total input tokens and resulting CU consumption. Additional context such as Data Agent Instructions, Data Source Instructions, Example Queries, and conversation history also contribute to token usage and consumption.

Consumption rate

The table below defines consumption in Capacity Units (CUs), when Data agent leverages Azure OpenAI models to process and generate responses.

Expand table

Metrics App Operation Name	Description	Operation Unit of Measure	Consumption rate
AI Query	The input prompt	Per 1,000 Tokens	100 CU seconds
AI Query	The cached input prompt	Per 1,000 Tokens	10 CU seconds
AI Query	The output completion	Per 1,000 Tokens	400 CU seconds

In addition to token consumption, the Data agent might generate and execute queries as part of answering user requests. The execution of these queries is billed separately to the corresponding query engine item. For example, if a query is generated for a Data Warehouse, its execution is billed through the SQL Query operation.

Monitor the usage

The Fabric Capacity Metrics app displays the total capacity usage for Data agent operations under the name *AI Query* Additionally, Data agent users are able to view a summary of their billing charges for the Data agent usage under the *LlmPlugin* item kind.

Capacity utilization type

The AI related activity within the Data agent is classified as *background jobs* to handle a higher volume of Data agent requests during peak hours.

For example, assume each Data agent request has 2,000 input tokens and 500 output tokens. The price for one Data agent request is calculated as follows: (2,000 × 100 + 500 × 400) / 1,000 = 400.00 CU seconds = 6.67 CU minutes. The cost of executing the AI-generated queries is billed through the corresponding query engine running the query.

Since Data agent is a background job, each Data agent request (~6.67 CU minute job) consumes only one CU minute of each hour of a capacity. If you're using an F64 SKU that has 64 * 24 = 1,536 CU hours a day, and each Data agent job consumes 6.67 CU mins / 60 mins = 0.11 CU hours, you can run over 13,824 requests before they exhaust the capacity. However, once the capacity is exhausted, all operations will shut down.

Region mapping

The Data agent is powered by Azure OpenAI large language models, which are deployed to a select set of data centers. Customers can enable cross-geo processing in the tenant settings to use Copilot and the Data agent. This setting allows data to be processed in a region where the Azure OpenAI Service is available. This region could be outside of the user's geographic region, compliance boundary, or national cloud instance. During region mapping, data residency is treated as the primary constraint. Regions are mapped within the same geographic area whenever possible.

The cost of Fabric CUs can vary depending on the region. Regardless of the consumption region where GPU capacity is utilized, customers are billed based on the Fabric pricing in their billing region. For example, if a customer's requests are mapped from `region 1` to `region 2`, with `region 1` being the billing region and `region 2` being the consumption region, the customer is charged based on the pricing in `region 1`.

Changes to Data agent in Fabric consumption rate

Consumption rates are subject to change at any time. Microsoft uses reasonable efforts to provide notice by email or through in-product notification. Changes become effective on the date stated in Microsoft's Release Notes or Microsoft Fabric Blog. If any change to a Data agent in Fabric Consumption Rate materially increases the Capacity Units (CU) required to use Data agent in Fabric, customers can use the cancellation options available for the chosen payment method.

Related content

- Overview of the Data agent in Fabric
- Background jobs in Fabric

Last updated on 07/22/2026

Skills for Microsoft Fabric

Skills for Fabric are reusable instructions that teach AI coding tools how to work with Microsoft Fabric workloads. Each skill gives an AI agent the knowledge it needs to perform a specific Fabric task, such as which REST APIs to call, what query syntax to use, and which operational patterns to follow.

Skills for Fabric are open source under the MIT License and are maintained in the microsoft/skills-for-fabric repository on GitHub.

AI coding tools don't have built-in knowledge of Fabric-specific APIs or query patterns. Skills bridge that gap so that tools like GitHub Copilot CLI, Visual Studio Code, Claude Code, and Cursor can act as Fabric-aware agents that author, query, and govern your Fabric resources.

To get started, see Install Skills for Fabric. For the full list of available skills, see Discover available Skills for Fabric.

What is an AI skill?

A *skill* is a reusable instruction set that teaches an AI coding tool how to perform a specific task. Each skill is a markdown file (`SKILL.md`) that defines:

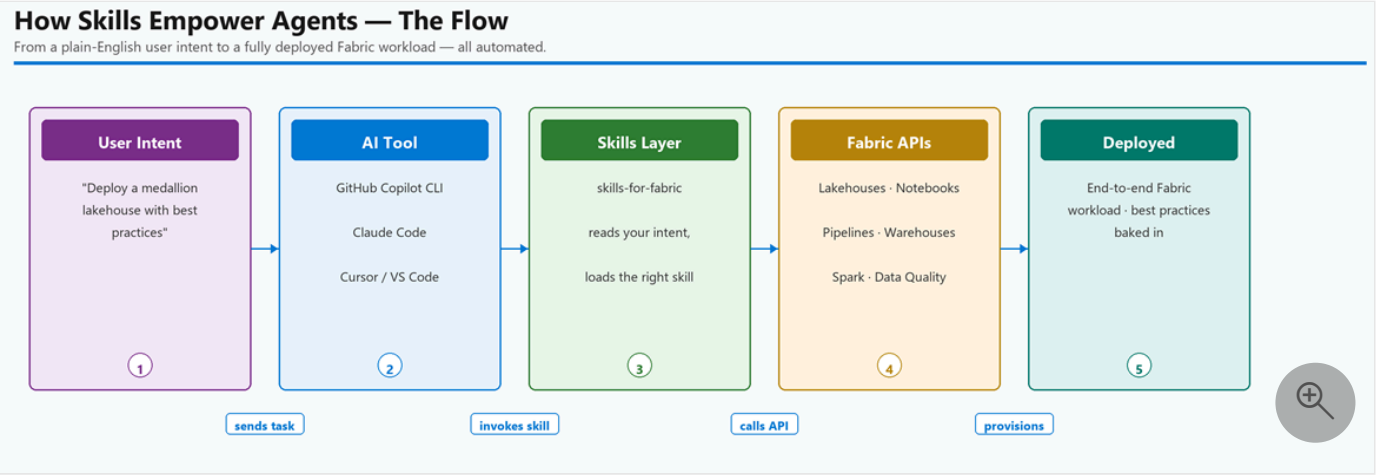
- The intent or trigger phrases the skill responds to.
- The APIs, commands, or query syntax to use.
- The authentication and environment setup the task requires.
- The operational best practices to follow.

AI coding tools load these skill files at session start. When you describe an intent in natural language, the tool matches your prompt against the installed skills and follows the matching skill's instructions to act on your behalf.

Skills are static knowledge. They don't execute code on their own. The AI tool decides which actions to take based on the guidance the skill provides.

How Skills for Fabric work

Skills for Fabric apply this pattern to Microsoft Fabric. Each skill packages the REST API, CLI, T-SQL, KQL, or PySpark knowledge needed for a specific Fabric task. The flow from a natural-language intent to a deployed Fabric workload looks like as follows:



1. **User intent.** You write a natural-language prompt that describes what you want to accomplish.
2. **AI tool.** Your AI coding tool receives the prompt and matches it against the installed skills.
3. **Skills layer.** The matching `SKILL.md` provides workload-specific instructions, including REST API endpoints, query syntax, and authentication patterns.
4. **Fabric APIs.** The AI tool calls Fabric REST APIs, SQL analytics endpoints, KQL, or other Fabric surfaces with the right parameters.
5. **Deployed.** Fabric resources are created or modified end-to-end, with the workload's best practices applied automatically.

Skills differ from Model Context Protocol (MCP) servers. Skills teach the AI assistant *what to do*. MCP servers *do it*. Skills provide the expertise and MCP servers provide the data connection. They both work best together. The following table summarizes the key differences:

Expand table

Aspect	Skills	MCP servers
Purpose	Provide knowledge and patterns	Provide live data access
Content	Markdown documentation	Executable servers
Runtime	Loaded into AI context	Run as separate processes
Example	"How to query a warehouse"	"Execute this SQL query"

> **Note**
>
> Skills produce Fabric item definitions and data-movement code, but they don't currently render finished Power BI reports. For visualizations, use a local Python dashboard, export

results as a PDF, or build Power BI reports manually on top of the semantic models that the skills generate.

Common scenarios

The following table lists realistic intents across three audiences. In each case, you describe the outcome in natural language and the agent applies the matching skills, best practices, and deployment steps.

Expand table

Role	Example intent	What the agent does
Data engineer or developer	"Build a Bronze, Silver, Gold medallion architecture for my NYC taxi data."	Creates lakehouses, notebooks, and pipelines with Spark best practices applied.
Data engineer or developer	"Migrate this SQL Server stored procedure to a Fabric warehouse."	Translates T-SQL, creates the warehouse schema, and loads data with `COPY INTO`.
Data engineer or developer	"Set up Git integration and a deployment pipeline for my workspace."	Configures Fabric Git integration and deployment pipelines.
Data analyst or BI developer	"Author a semantic model over my Gold lakehouse."	Builds a Power BI semantic model definition with dimensions, facts, and relationships.
Data analyst or BI developer	"What tables are in my lakehouse, and how fresh is the data?"	Inspects metadata, row counts, and last refresh times.
Data analyst or BI developer	"Profile this dataset."	Generates summary statistics, null counts, and outlier detection.
Citizen developer or business user	"I have CSV files. Build me an interactive dashboard."	Ingests and models the data, then generates a local Python dashboard you can run.
Citizen developer or business user	"Show me total revenue by product for last quarter."	Writes and runs the warehouse query, and returns the results.
Citizen developer or business user	"Document my entire Fabric workspace."	Scans the workspace and produces structured documentation in Markdown.

For a detailed Data Warehouse example, see how the sqldw-cli operations skill guides read-only investigations of broad warehouse health, failures, query performance, capacity spikes, SQL pool pressure, cluster-key candidates, and lakehouse table health.

Compatible AI tools

Skills for Fabric work with the following AI coding tools:

Expand table

Tool	Installation method
GitHub Copilot CLI	Plugin marketplace (recommended)
Visual Studio Code	Automatically detected from `~/.copilot/` folder (VS Code 1.108+)
Claude Code	Plugin marketplace
Cursor	Clone the repository; `.cursorrules` is auto-detected
Windsurf	Clone the repository; `.windsurfrules` is auto-detected
Codex / Jules / OpenCode	Clone the repository; `AGENTS.md` is auto-detected

Agent specializations

Skills for Fabric also includes experimental *agent specializations* focused teammates that combine prompts and tool selections for specific workflows. Examples include agents focused on data engineering (`FabricDataEngineer`), administration (`FabricAdmin`), app development (`FabricAppDev`), and more.

To use an agent specialization in GitHub Copilot CLI, reference it in your prompt:

Copilot prompt

```
Using FabricAdmin, document my Workspace FabricCLIDemo
```

You can also invoke an agent from GitHub Copilot CLI by prefixing your prompt with the agent name, for example:

Bash

```bash
@FabricDataEngineer help me design a medallion architecture for my lakehouse
```

The available agents evolve frequently. Browse the agents folder on GitHub for the current set, and use the `/agent` command in GitHub Copilot CLI to display installed agents.

MCP server integration with Skills for Fabric

Some installation bundles include MCP server configuration where supported. You can also register additional Fabric MCP servers if your environment provides them.

For more information, see the MCP servers guide, What is the Fabric MCP server?, and What is MCP in Real-Time Intelligence?

Related content

- Install Skills for Fabric
- Discover available Skills for Fabric
- Data Warehouse operations skill
- Skills for Fabric on GitHub
- What is the Fabric MCP server?
- Build AI agents for Real-Time Intelligence
- What is MCP in Real-Time Intelligence?

Note: The author created this article with assistance from AI. Learn more

---

Last updated on 09/11/2026

Install Skills for Fabric

This article explains how to install, configure, and update Skills for Fabric by using GitHub Copilot CLI, Claude Code, or other compatible AI coding tools.

> **Important**
>
> Skills for Fabric ships as two separate plug-ins from the same marketplace: the Fabric collection (`fabric-skills`) and the Power BI plug-in (`powerbi-authoring`). The Fabric collection doesn't include the Power BI plug-in. For full coverage, install both as shown in Install with GitHub Copilot CLI or Claude Code.

Prerequisites

The following prerequisites apply to both the Fabric collection and the Power BI plug-in:

- **An AI coding tool**, such as GitHub Copilot CLI, Claude Code, Visual Studio Code, Cursor, Windsurf, or another `AGENTS.md` compatible tool.
- **Node.js**: Required for the standalone GitHub Copilot CLI, which installs as an npm package (`npm install -g @github/copilot`). For full setup steps, see Set up GitHub Copilot CLI.
- **PowerShell 7**: Required for GitHub Copilot CLI. Verify with `pwsh --version`.
- **Git**: Required to clone skill repositories for tools that don't use the plugin marketplace.
- **Azure CLI**: Most Fabric operations require Azure authentication. After installing, sign in with `az login`.

The Power BI plug-in (`powerbi-authoring`) has these extra prerequisites:

- **A Power BI tenant** with permission to create, edit, and publish semantic models and reports.
- **Power BI Desktop** (recommended) for opening and validating PBIX or PBIP projects locally.

Install with GitHub Copilot CLI or Claude Code

Both GitHub Copilot CLI and Claude Code use the same plugin marketplace workflow. Install both the Fabric collection and the Power BI plug-in for complete coverage.

1. Add the Skills for Fabric marketplace:

```
Bash
/plugin marketplace add microsoft/skills-for-fabric
```

2. Install the Fabric collection. For focused bundles, see Installation bundles.

```
Bash
/plugin install fabric-skills@fabric-collection
```

3. Install the Power BI plug-in. This bundle is separate from `fabric-skills`.

```
Bash
/plugin install powerbi-authoring@fabric-collection
```

4. Quit and restart your tool to load the skills:

```
Bash
/quit
```

5. Verify the installation:

```
Bash
/skills
```

Install with other AI tools

For Cursor, Windsurf, Codex, and similar tools, clone the Skills for Fabric repository. The root-level configuration files (`CLAUDE.md`, `.cursorrules`, `.windsurfrules`, `AGENTS.md`) are automatically detected when you clone the repository into your project folder.

Installation bundles

A bundle is a preconfigured group of skills packaged for the plugin marketplace. Instead of installing skills one at a time, pick a bundle that matches your role or workflow and install the

entire group with a single command. Skills for Fabric offers the following bundles so you can install only the skills you need:

Expand table

Bundle	Description	Install command
`fabric-skills`	Complete bundle: all skills for authoring, consumption, operations, migration, and end-to-end architecture. Doesn't include `powerbi-authoring`.	`/plugin install fabric-skills@fabric-collection`
`fabric-authoring`	Developer skills for REST APIs, CLI automation, notebooks, T-SQL, KQL, Dataflows Gen2, Eventstreams, and semantic models	`/plugin install fabric-authoring@fabric-collection`
`fabric-consumption`	Read-only exploration and query skills across all Fabric workloads	`/plugin install fabric-consumption@fabric-collection`
`fabric-operations`	Performance and health diagnostics	`/plugin install fabric-operations@fabric-collection`
`powerbi-authoring`	Power BI plug-in: semantic model authoring, report planning, design, authoring, management, and PBIP workflows. Install separately from `fabric-skills`.	`/plugin install powerbi-authoring@fabric-collection`

You can filter the full bundle by workload in GitHub Copilot CLI or Claude Code using the following commands:

Bash

```
/plugin install fabric-skills@fabric-collection --filter "sqldw-*"
/plugin install fabric-skills@fabric-collection --filter "spark-*"
/plugin install fabric-skills@fabric-collection --filter "eventhouse-*"
```

Example prompts for common scenarios

After installing a Skills for Fabric bundle, open your AI coding tool and try one of the following prompt examples. Modify the workspace and item names to match your environment.

Analyze data and generate a PDF report

Copilot prompt

```
I have New York City taxi trips in a data warehouse called DemoDW,
in a workspace called FabricCLIDemo. Analyze the data and produce
a PDF report of key indicators for every month (average trip
```

```
duration, average trip fare, number of trips). Include outlier
analysis across months, and a 3-month forecast. Save the results
as a PDF.
```

Document a workspace

Copilot prompt

```
Document my Fabric Workspace: take a look at the workspace called
FabricCLIDemo. Document the data solution, the role of each
item, the lineage, and what happens in each item with my
data. Look at notebooks, pipelines, views, stored procedures, and
semantic models. Write all results to a WorkspaceReport folder in
Markdown format.
```

Build a Medallion Architecture

Copilot prompt

```
Use Microsoft Fabric skills to design a medallion architecture
for NYC taxi data. Download all data dictionaries and trip data for
2025. Create clean tables following the dimensional model with fact
and dimension tables. Create an aggregate reporting view in the SQL
endpoint. Generate a Power BI semantic model definition with
dimensions, facts, and relationships. Before deploying, ask me
which workspace to use.
```

Build an interactive dashboard

Copilot prompt

```
NYC Taxi Trips data is in the DemoDW warehouse of the
FabricCLIDemo workspace. Generate an interactive Python
dashboard that connects to the warehouse using ODBC, takes into
account month, day of week, and time of day, and tells me where
to pick up customers to maximize tips. When done, launch the app.
```

> **Tip**
>
> Consider asking Copilot to plan before execution. Use **Shift+Tab** to move to planning mode, where you enter your prompt. After you're satisfied with the plan, switch back and ask the agent to start executing. Empirically, results are better when the AI plans first.

Update skills

GitHub Copilot CLI and Claude Code check for plug-in updates automatically when you install or reinstall a bundle. To manually refresh a bundle, uninstall and reinstall it:

Bash

```
/plugin uninstall fabric-skills@fabric-collection
/plugin install fabric-skills@fabric-collection
```

Repeat for `powerbi-authoring@fabric-collection` if you have it installed.

Common issues and resolutions

Expand table

Issue	Cause	Resolution
`copilot` command not found	GitHub Copilot CLI isn't installed or isn't on PATH.	Install GitHub Copilot CLI, then close and reopen PowerShell 7. Verify with `copilot --help`.
Plugins don't install correctly	PowerShell version is too old. GitHub Copilot CLI requires PowerShell 7.	Run `pwsh --version` to verify PowerShell 7 is installed. Ensure your terminal profile launches `pwsh`, not `powershell.exe`.
Marketplace add fails or repo is inaccessible	Not authenticated with the correct GitHub account.	Authenticate with your GitHub account and verify you can access the Skills for Fabric repo in a browser. Relogin to Copilot CLI.
No skills appear after marketplace add	Copilot session wasn't restarted, or the plugin wasn't activated.	Run `/quit`, restart Copilot, then run `/plugin marketplace browse fabric-collection` and `/plugin install fabric-skills@fabric-collection`.
Skills installed but not being invoked	Skills are disabled, the prompt is too vague, or skill autoselection didn't trigger.	Run `/skills` to verify skills are enabled. Use explicit prompts that mention your Fabric workspace, warehouse, or lakehouse. Try explicit invocation with `/fabric-skills:<skill-name>`.
Actions occur in the wrong workspace	Workspace wasn't specified in the prompt.	Always specify workspace explicitly. Add "Before making any changes, ask me which workspace to use" to your prompt.
Plugin update not reflected	Cached plugin version is still active.	Restart Copilot CLI. If still stale, uninstall and reinstall the plugin as shown in Update skills.
Workspace creation or capacity permission errors	Insufficient permissions on the Fabric capacity, or the capacity is paused.	Verify you have Contributor or Admin permissions on the target capacity. In the Azure portal, check that the capacity is in an **Active** (Resumed) state.

> **⊗ Caution**
>
> Use caution with the `/yolo` command (which allows the AI to execute actions without confirmation) when provisioning or deploying resources. Start with **Plan mode** to validate the plan summary before switching to **Autopilot mode**.

Related content

- Skills for Fabric overview
- Discover available Skills for Fabric
- Skills for Fabric on GitHub

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 06/30/2026

Discover available skills for Microsoft Fabric

Skills for Fabric span the major Fabric workloads. Because new skills are added to the microsoft/skills-for-fabric repository regularly, this article shows you how to discover what's currently available rather than enumerating every skill.

> **Note**
>
> Skills for Fabric are open source under the MIT License. The catalog evolves quickly. Always check the skills folder on GitHub for the latest list.

Workload areas covered

Skills for Fabric are organized by workload. The repository currently includes skills across these areas:

Expand table

Workload area	What the skills help you do
Data engineering	Author Spark notebooks, manage Spark sessions, diagnose job failures, and build end-to-end Medallion architectures.
Data warehouse	Run authoring T-SQL, query Lakehouse SQL endpoints, and analyze warehouse performance.
Real-time intelligence	Author and query Eventhouses with KQL, build Eventstream topologies, and manage Activator alerts.
Data Factory	Build, refresh, and inspect Dataflows Gen2, and upgrade Gen1 dataflows to Gen2.1.
Power BI	Plan, design, and manage Power BI semantic models and report definitions; query semantic models with DAX; and ask natural-language questions through FabricIQ. Report and semantic model authoring skills ship in the separate `powerbi-authoring` plug-in.
Fabric admin and governance	Document workspaces and inventory items across tenants.
Migration	Port workloads to Microsoft Fabric from Azure Databricks, Azure Synapse Analytics, Azure HDInsight, and Synapse pipelines.

Workload area	What the skills help you do
Discovery and search	Find Fabric items across workspaces.

For the current list of skills in each area, browse the skills folder on GitHub.

List skills installed in your AI tool

After you install Skills for Fabric, use your tool's slash commands to see what's available locally.

GitHub Copilot CLI or Claude Code

1. Start a Copilot CLI or Claude Code session.
2. Run `/env` to list all installed skills, including the Skills for Fabric marketplace.

Visual Studio Code

1. Open the Chat view.
2. Type `/` to view available skills auto-loaded from `~/.copilot/`.

Cursor, Windsurf, and other AGENTS.md-compatible tools

Skills are auto-loaded from the cloned `microsoft/skills-for-fabric` repository at session start. Ask your tool to list available Skills for Fabric or check the `skills/` folder in the cloned repository.

Get details for a specific skill

To see what a skill does, its triggers, and its required parameters:

GitHub Copilot CLI or Claude Code

Run the following command, replacing `<skill-name>` with the skill you want to inspect:

Bash

```
/skills info <skill-name>
```

For example: `/skills info eventhouse-consumption-cli`

Other tools

Open the corresponding `SKILL.md` file in microsoft/skills-for-fabric/skills. Each skill folder contains a `SKILL.md` file with a description, trigger phrases, and usage notes.

Browse the full catalog on GitHub

Each subfolder in microsoft/skills-for-fabric/skills represents one skill. The folder name describes the workload and capability. For example, `sqldw-authoring-cli` is for Data Warehouse authoring via CLI.

To stay current with new releases, watch the Releases page. Your AI coding tool also checks for plug-in updates automatically when you install or reinstall a bundle.

Related content

- Skills for Fabric overview
- Install Skills for Fabric
- Skills for Fabric on GitHub

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

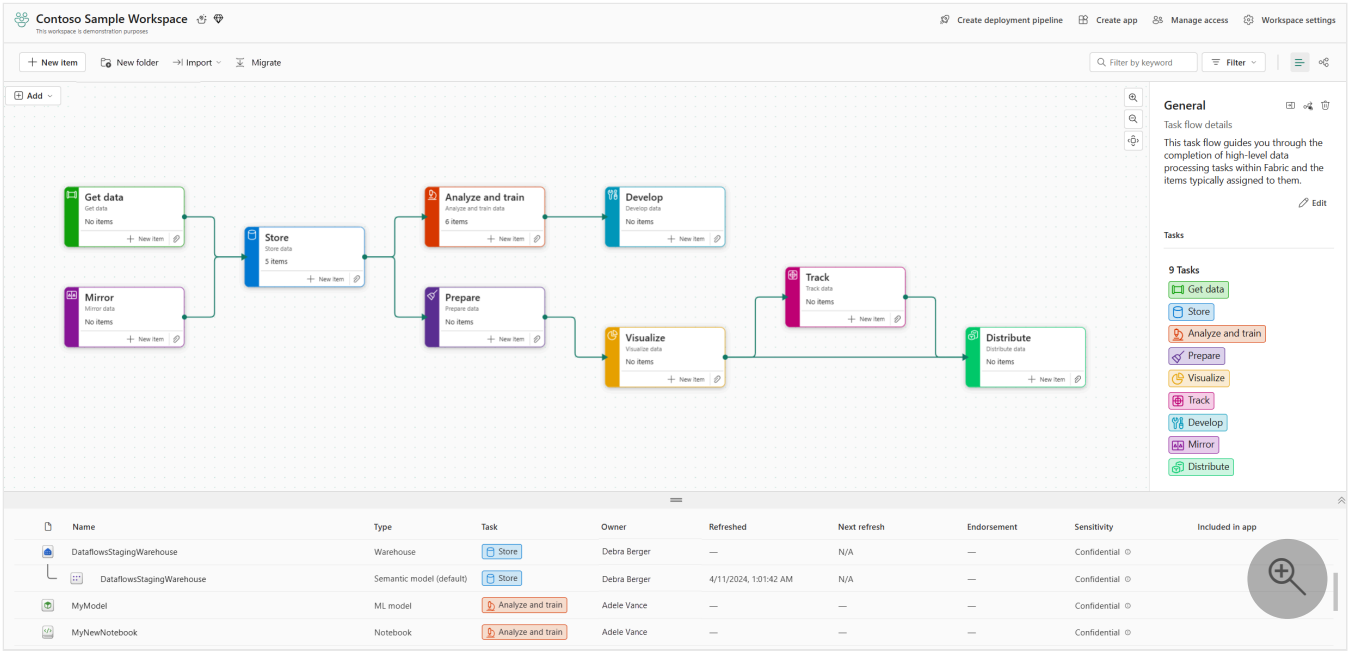
Last updated on 06/30/2026

Task flows in Microsoft Fabric

This article describes the task flows feature in Microsoft Fabric. Its target audience is data analytics solution architects who want to use a task flow to build a visual representation of their project, engineers who are working on the project and want to use the task flow to facilitate their work, and others who want to use the task flow to filter the item list to help navigate and understand the workspace.

Overview

Fabric task flow is a workspace feature that enables you to build a visualization of the flow of work in the workspace. The task flow helps you understand how items are related and work together in your workspace, and makes it easier for you to navigate your workspace, even as it becomes more complex over time. Moreover, the task flow can help you standardize your team's work and keep your design and development work in sync to boost the team's collaboration and efficiency.



Fabric provides a range possibilities for creating a task flow:

- You can start with one of the predefined, end-to-end task flows provided by Fabric. These predefined task flows are based on industry best practices and are intended to make it easier to get started with your project. Once you've got the predefined task flow, you can customize it to meet your needs.

- You can create your own task flow from scratch to suit your specific needs and requirements.
- You can import a task flow that you or someone else has created previously and customize it to meet your needs.
- You can create multiple task flow canvases in a workspace, and maintain separate task flows on each canvas.

The task flow occupies the upper part of workspace list view. It consists of a canvas where you can build the visualization of your data analytics project, and a side pane where you can see and edit details about the task flow, tasks, and connectors.

> ⓘ **Note**
>
> You can resize or hide the task flow using the controls on the horizontal separator bar.

Key concepts

Key concepts to know when working with a task flow are described in the following sections.

Task flow

A task flow is a collection of connected tasks that represent relationships in a process or collection of processes that complete an end-to-end data solution. A workspace can have one or more task flow canvases. In each canvas, you can build a task flow from scratch, use one of Fabric's predefined task flows, or import an existing task flow.

Task flow canvas

A task flow canvas is the workspace area where you create and manage a task flow. A workspace can include multiple canvases so you can organize separate processes or workstreams independently.

Task

A task is a unit of process in the task flow. A task has recommended item types to help you select the appropriate items when building your solution. Tasks also help you navigate the items in the workspace.

Task type

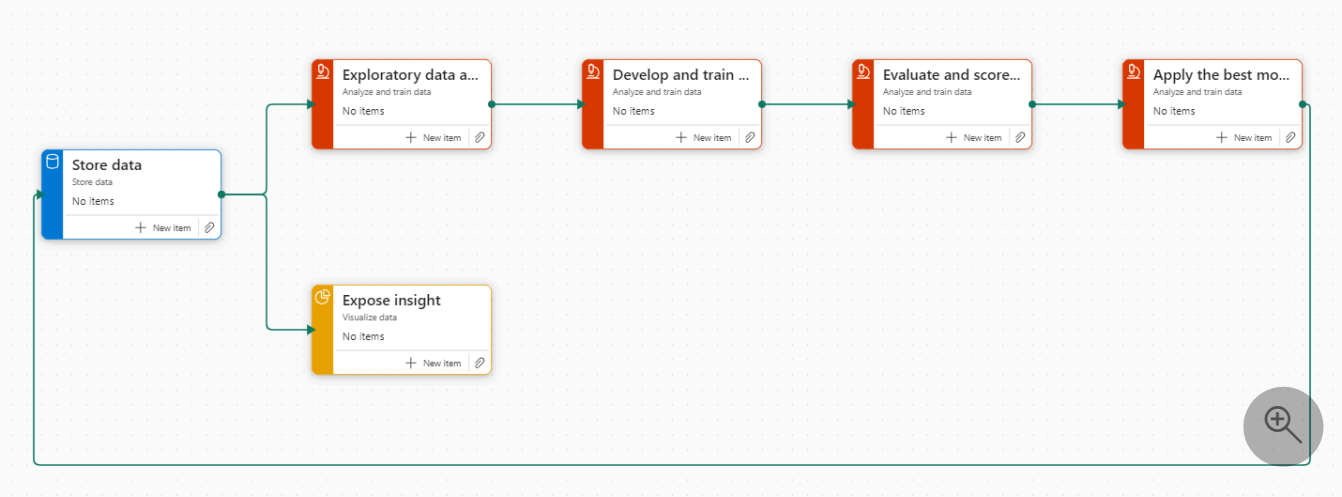
Each task has a task type that classifies the task based on its key capabilities in the data process flow. The predefined task types are:

Expand table

Task type	What you want to do with the task
General	Create a customized task for your project needs that you can assign available item types to.
Get data	Ingest batch and real-time data into a single location within your Fabric workspace.
Mirror data	Replicate your data from any location to OneLake in near real-time.
Store data	Organize, query, and store your ingested data in an easily retrievable format.
Prepare data	Clean, transform, extract, and load your data for analysis and modeling tasks.
Analyze and train data	Propose hypotheses, train models, and explore your data to make decisions and predictions.
Track data	Monitor your streaming or nearly real-time operational data, and make decisions based on gained insights.
Visualize data	Present your data as rich visualizations and insights that can be shared with others.
Distribute data	Package your items for distribution as customized, easy-to-use apps.
Develop data	Create and build your software, applications, and data solutions.

Connector

Connectors are arrows that represent logical connections between the tasks in the task flow. They don't represent the flow of data, nor do they create any actual data connections.



Considerations and limitations

- Creating paginated reports, dataflows Gen1, and semantic models from a task isn't supported.
- Creating reports from a task is supported only if a published semantic model is picked.

Related content

- Set up a task flow
- Work with task flows
- Manage multiple task flow canvases in a workspace

Last updated on 04/10/2026

Set up a task flow

Article • 05/06/2025

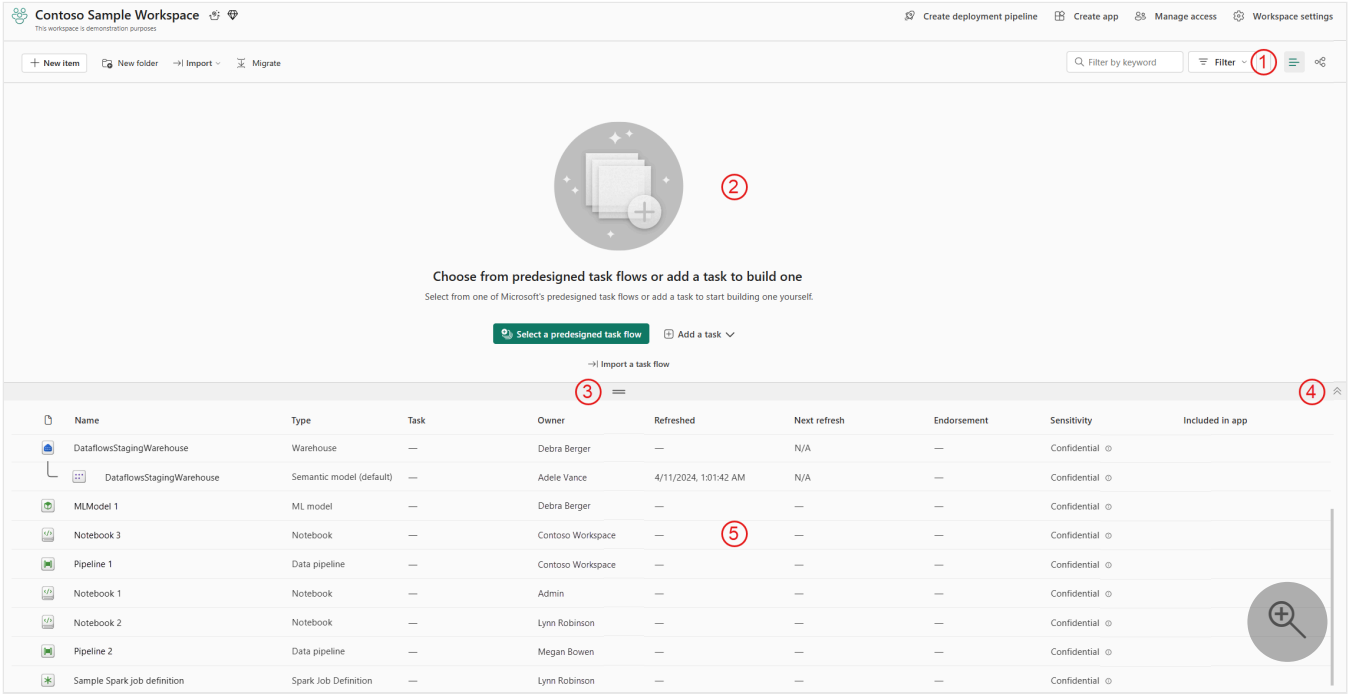
This article describes how to start building a task flow. It targets data analytics solution architects and others who want to create a visualization of a data project.

Prerequisites

To create a task flow in a workspace, you must be a workspace admin, member, or contributor.

Open the workspace

Navigate to the workspace where you want to create your task flow. It should open in list view, but if not, select the **List view** icon. You'll see that the workspace view is split between the task flow, where you'll build your task flow, and the items list, which shows you the items in the workspace. A moveable separator bar allows you to adjust the size of the views. You can also hide the task flow if you want to get it out of the way.



1. List view selector
2. Task flow canvas
3. Resize bar
4. Show/hide task flow
5. Items list

When no task flow has been configured, the task flow area prompts you to choose between starting with a predesigned task flow, adding a task to start building your own custom task

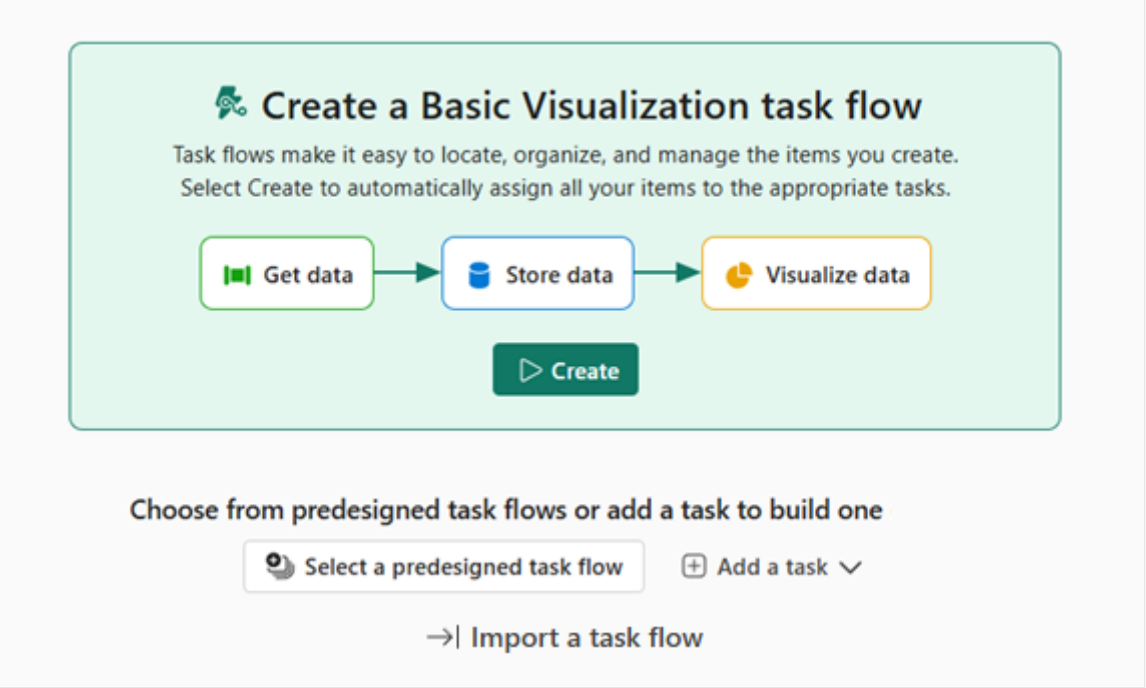
flow, or importing a saved task flow.

To build a task flow, you need to:

- Add tasks to the task flow canvas.
- Arrange the tasks on the task flow canvas in such a way that illustrates the logic of the project.
- Connect the tasks to show the logical structure of the project.
- Assign items to the tasks in the workflow.

To get started, choose either **Select a predesigned task flow**, **Add a task** (to start building one from scratch), or **Import a task flow** (to start with a saved task flow).

If the workspace contains Power BI items only, the task flow canvas will display a basic task flow designed to meet the basic needs of a solution based on Power BI items only.

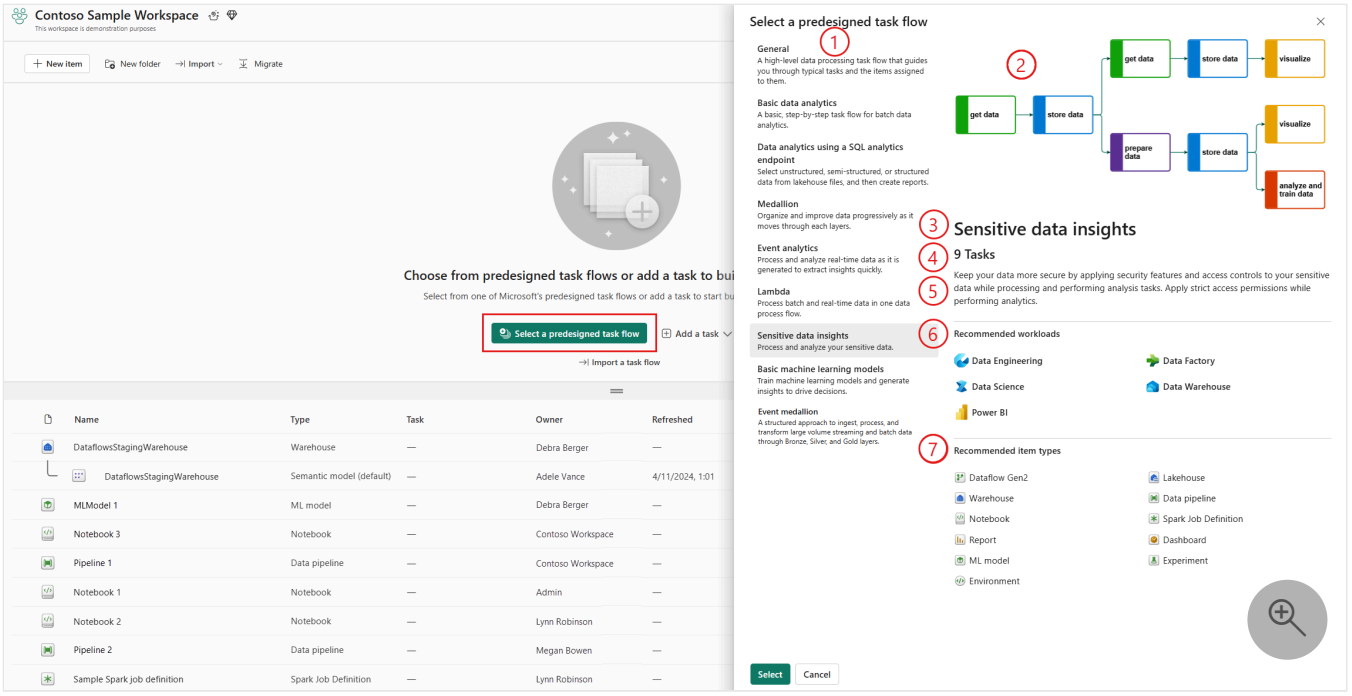


Select **Create** if you want to start with this task flow, or choose either of the previously mentioned options, **Select a predesigned task flow**, **Add a task**, or **Import a task flow**.

Start with a predesigned task flow

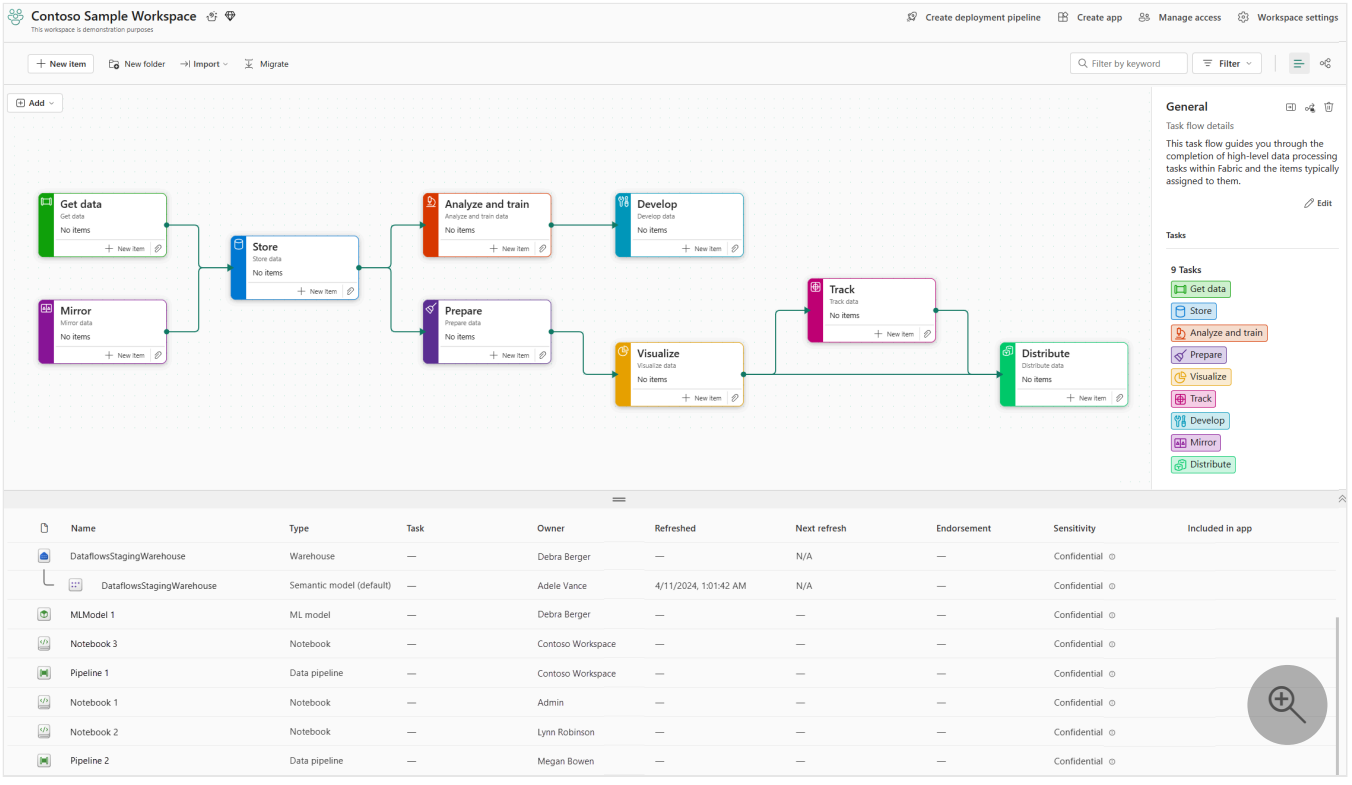
In the empty task flow area, choose **Select a predesigned task flow**.

The side pane lists the predesigned task flows provided by Microsoft. Each predefined task flow has a brief description of its use case. When you select one of the flows, you'll see a more detailed description of the flow and how to use it, and also the workloads and item types that the flow requires.



1. List of predesigned task flows.
2. Layout of selected predesigned task flow.
3. Name of selected predesigned task flow.
4. Number of tasks in the task flow.
5. Detailed description of the task flow and how it's used.
6. The workloads that the task flow typically requires.
7. The item types that are typically used in task flow.

Select the task flow that best fits your project needs and then choose **Select**. The selected task flow will be applied to the task flow canvas.



The task flow canvas provides a graphic view of the tasks and how they're connected logically.

The side pane now shows detailed information about the task flow you selected, including:

- Task flow name.
- Task flow description.
- Total number of tasks in the task flow.
- A list of the tasks in the task flow.

It's recommended that you change the task flow name and description to something meaningful that enables others to better understand what the task flow is all about. To change the name and description, select **Edit** in the task flow side pane. For more information, see Edit task flow details.

The items list shows all the items and folders in the workspace, including those items that are assigned to tasks in the task flow. When you select a task in the task flow, the items list is filtered to show just the items that are assigned to the selected task.

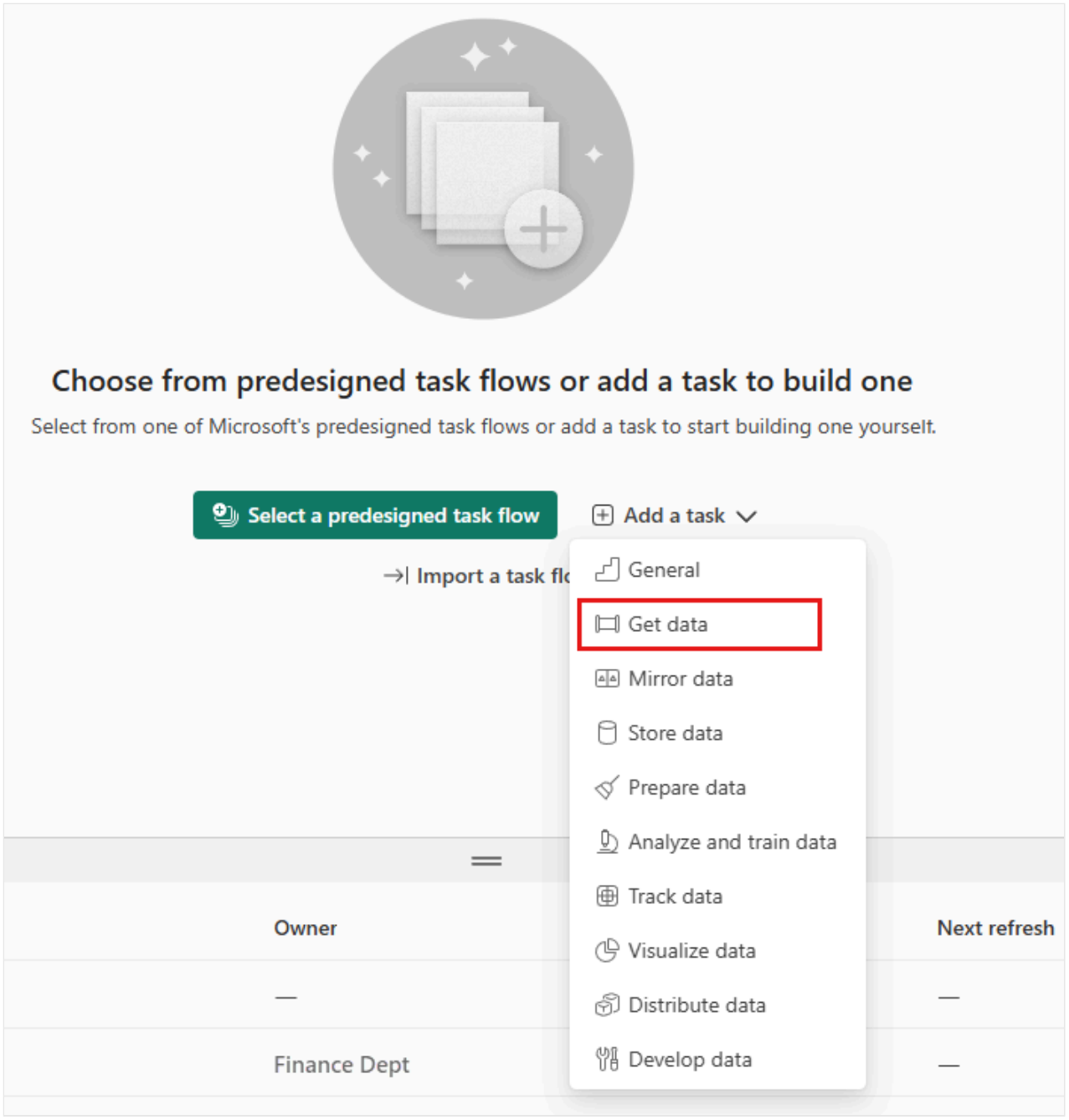
> ⓘ **Note**
>
> Selecting a predefined task flow just places the tasks involved in the task flow on the canvas and indicates the connections between them. It's just a graphical representation - no actual items or data connections are created at this point, and no existing items are assigned to tasks in the flow.

After you've added the predefined task flow to the canvas, you can start modifying it to suit your needs - arranging the tasks on the canvas, updating task names and descriptions, assigning items to tasks, etc. For more information, see Working with task flows.

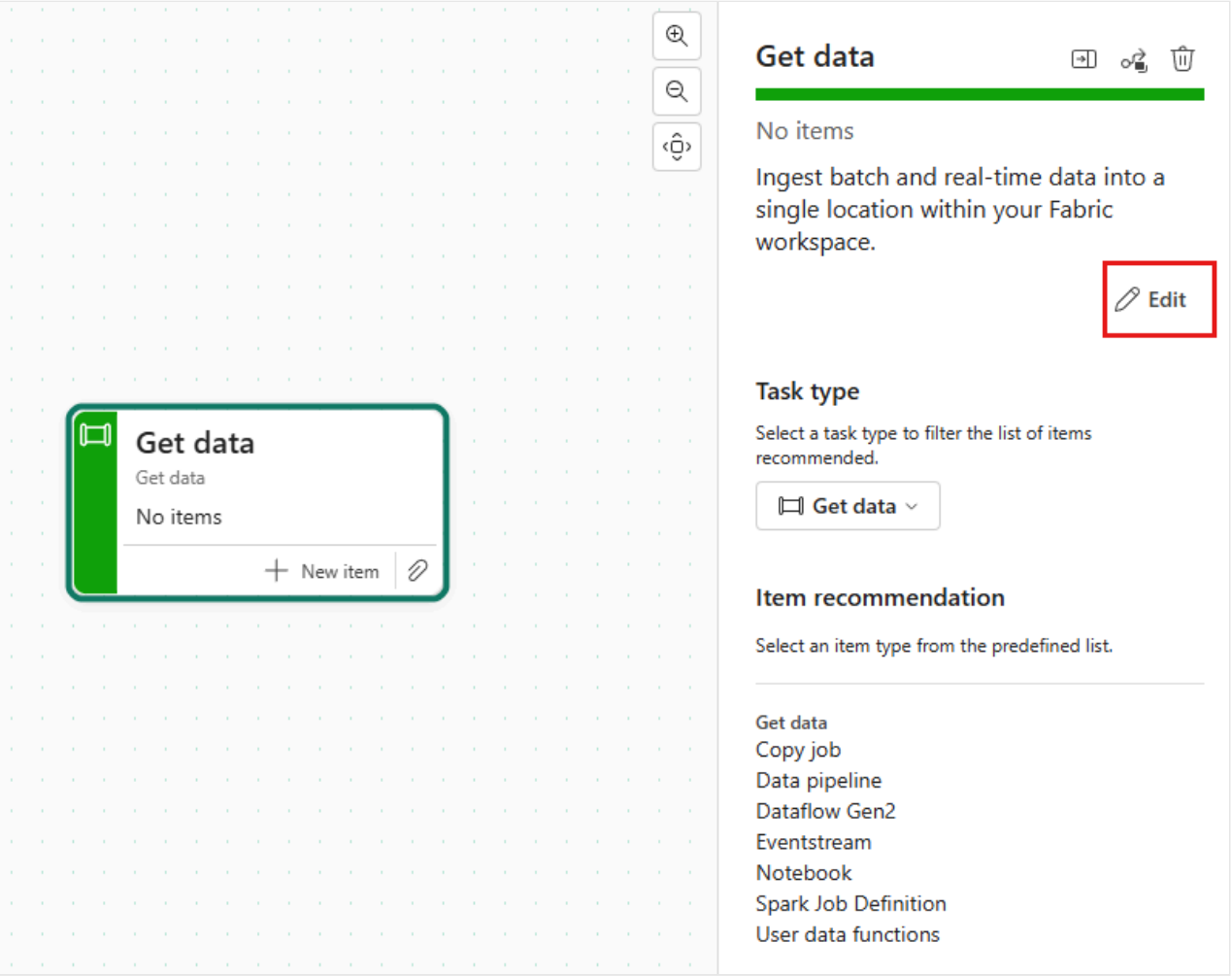
Start with a custom task flow

If you already have a clear idea of what the structure of your task flow needs to be, or if none of the predesigned task flows fit your needs, you can build a custom task flow from scratch task by task.

1. In the empty task flow area, select **Add a task** and choose a task type.

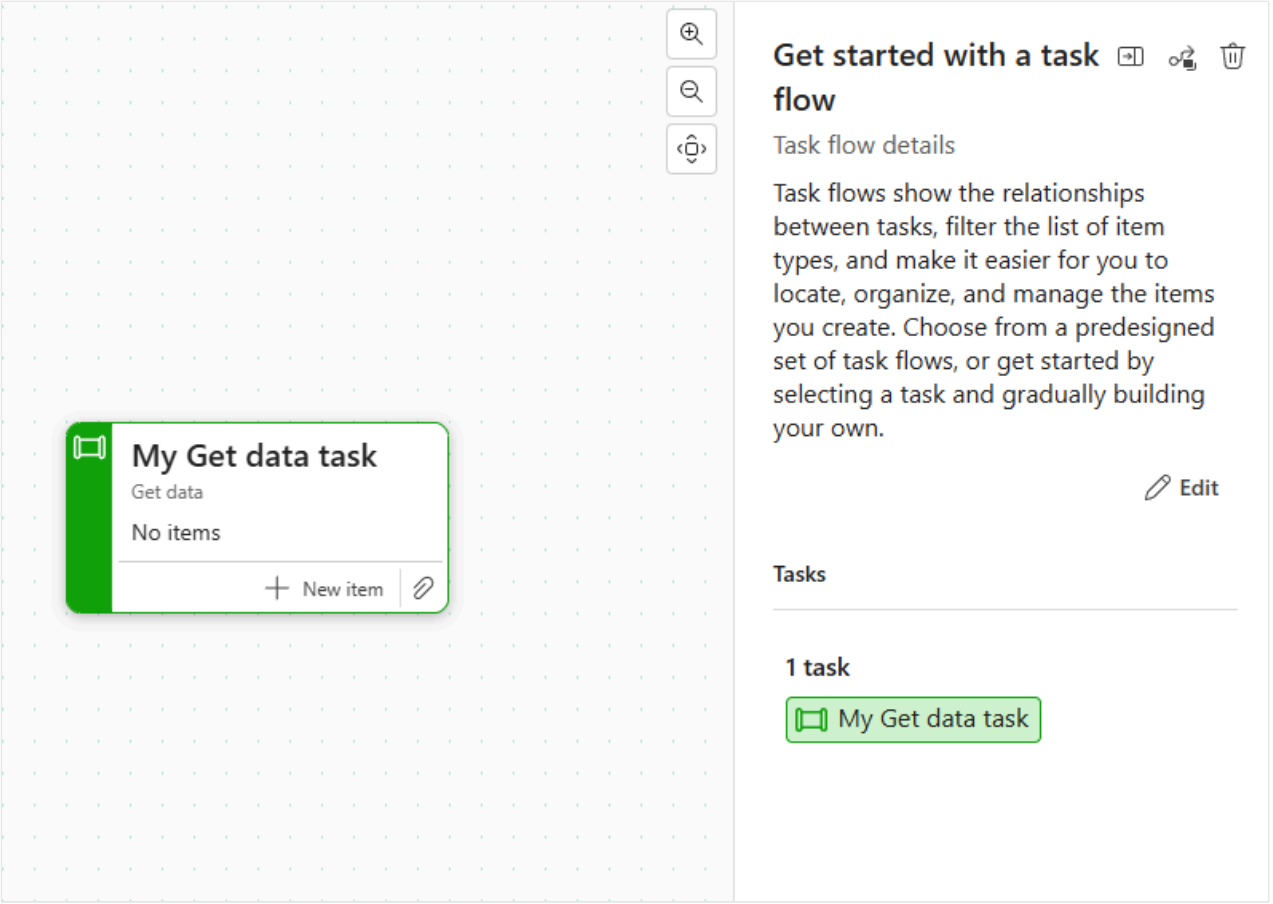


2. A task card appears on the canvas and the task details pane opens to the side.



It's recommended to provide a meaningful name and description of the task to help other members of the workspace understand what the task is for. In the task details side pane, select **Edit**, to provide a meaningful name and description.

3. Deselect the task by clicking on a blank area of the task flow canvas. The side pane will display the task flow details with a default name (*Get started with a task flow*) and description. Note that the task you just created is listed under the **Tasks** section.



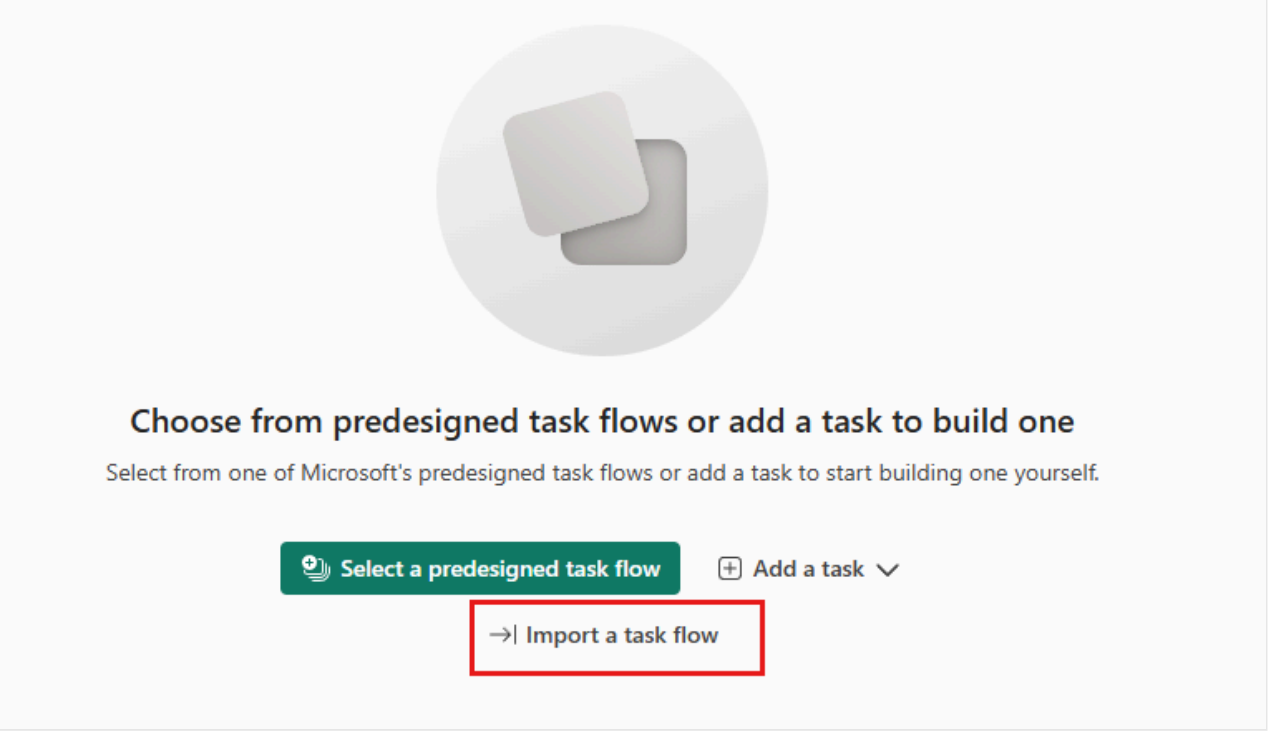
Select **Edit** and provide a meaningful name and description for your new task flow to help other members of the workspace understand your project and the task flow you're creating. For more information, see Edit task flow details.

You can continue to add more tasks to the canvas. You'll also have to perform other actions, such as arranging the tasks on the canvas, connecting the tasks, assigning items to the tasks, etc. For more information, see Working with task flows.

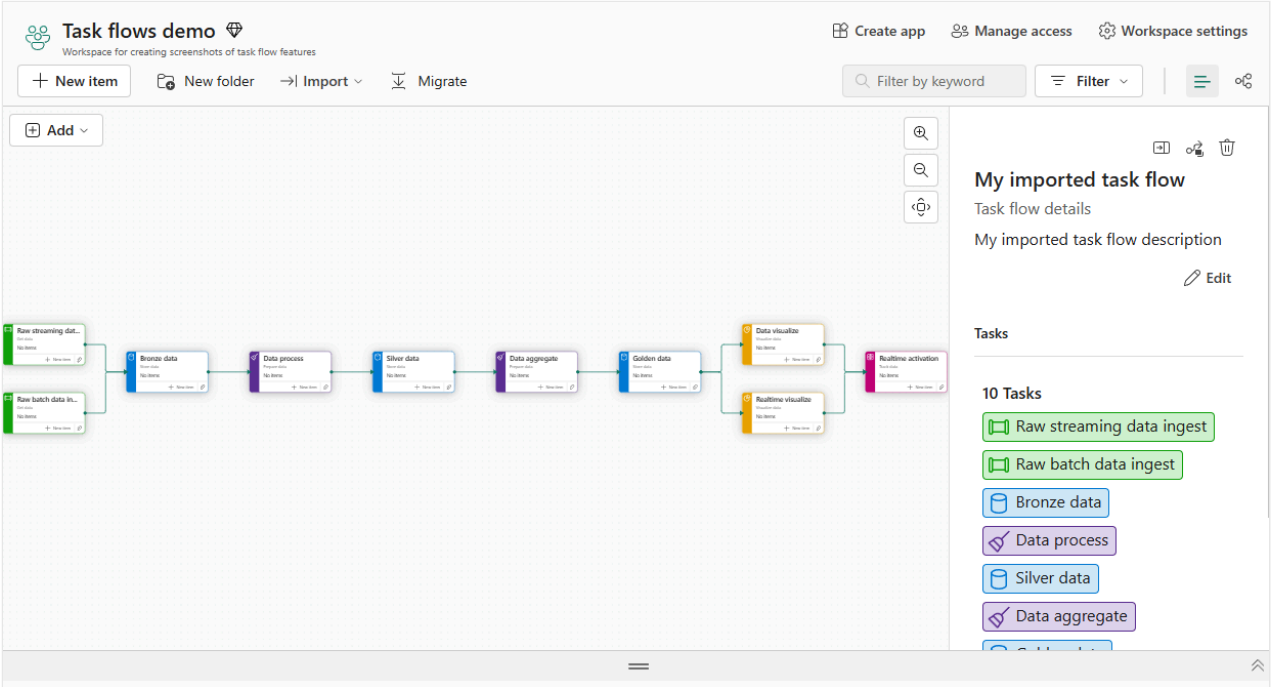
Start by importing an existing task flow

Task flows can be exported as *.json* files from the workspace they were created in. These *.json* files can then be imported into other workspaces for reuse. If there is a *.json* file of a task flow that would help you get started, you can import it into your project's workspace, and then modify it as necessary to suit your project's needs.

1. In the empty task flow area, choose **Import a task flow**.



2. In the **Open** window that appears, navigate to the desired *.json* file and open it. The task flow will appear on the canvas, with the task flow details pane at the side.



   If necessary, change the task flow name and description to something that suits your task flow and enables others to better understand what your task flow is all about. To change the name and description, select **Edit** in the task flow side pane. For more information, see Edit task flow details.

After you've imported the task flow into the canvas, you can start modifying it to suit your needs - arranging the tasks on the canvas, updating task names and descriptions, assigning items to tasks, etc. For more information, see Working with task flows.

For more detail about importing and exporting task flows, see Import or export a task flow.

Related concepts

- Task flow overview
- Work with task flows

Work with task flows

Article • 05/06/2025

This article describes how to work with tasks. The target audience is data analytics solution architects who are designing a data analytics solution, engineers who need to know how to use task flows to facilitate their work, and others who want to use the task flow to filter the item list to help navigate and understand the workspace.

Prerequisites

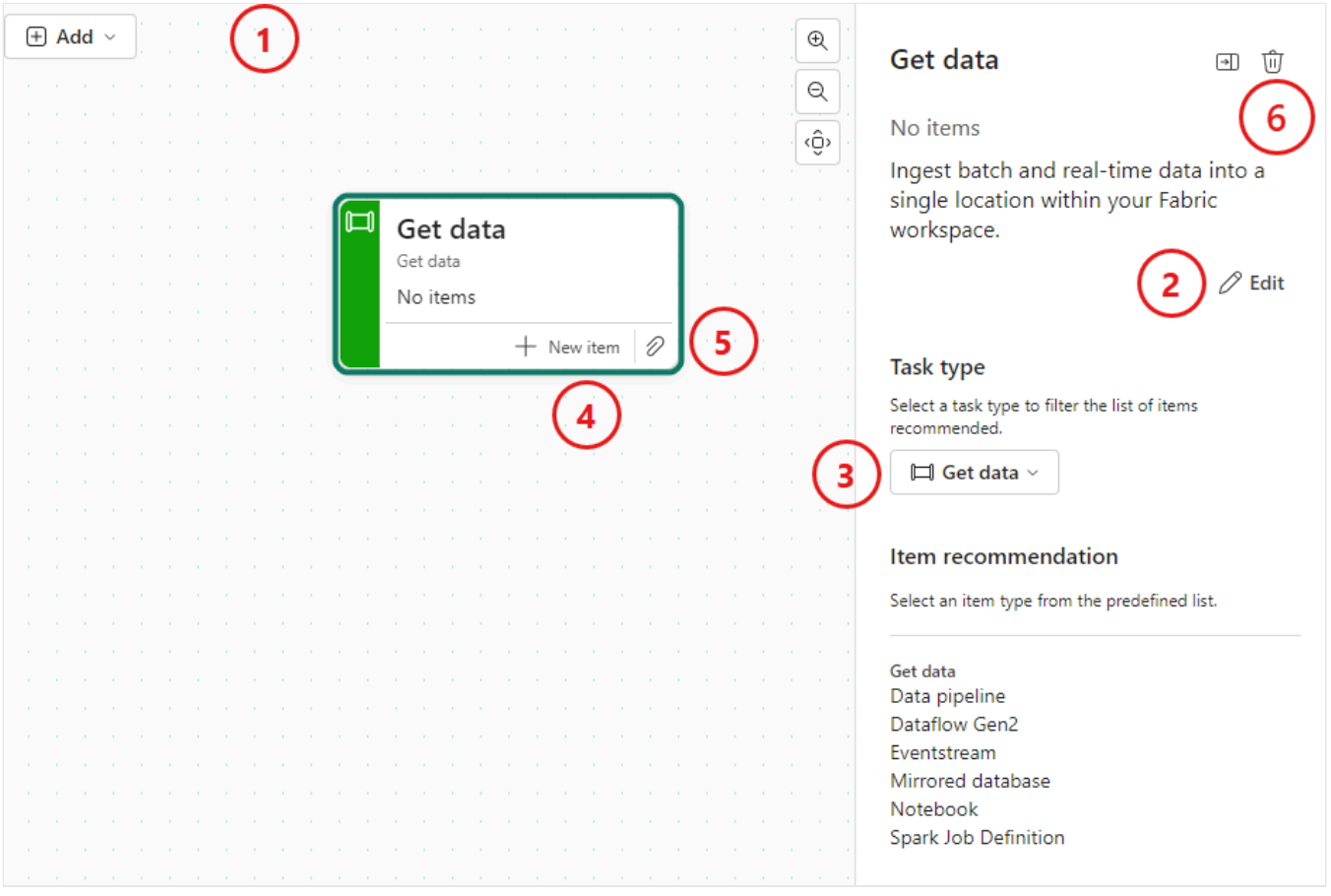
To create or edit the task flow, and to create items in the workspace via the task flow, you need to be an Admin, Member, or Contributor in the workspace.

Admins, Members, Contributors, and Viewers can use the task flow to filter the items list.

Task controls

Much of the work with tasks is accomplished either in the task details pane or via controls on the task card or on the task flow canvas.

Select a task to display the task details pane. The following image shows the main controls for working with tasks.

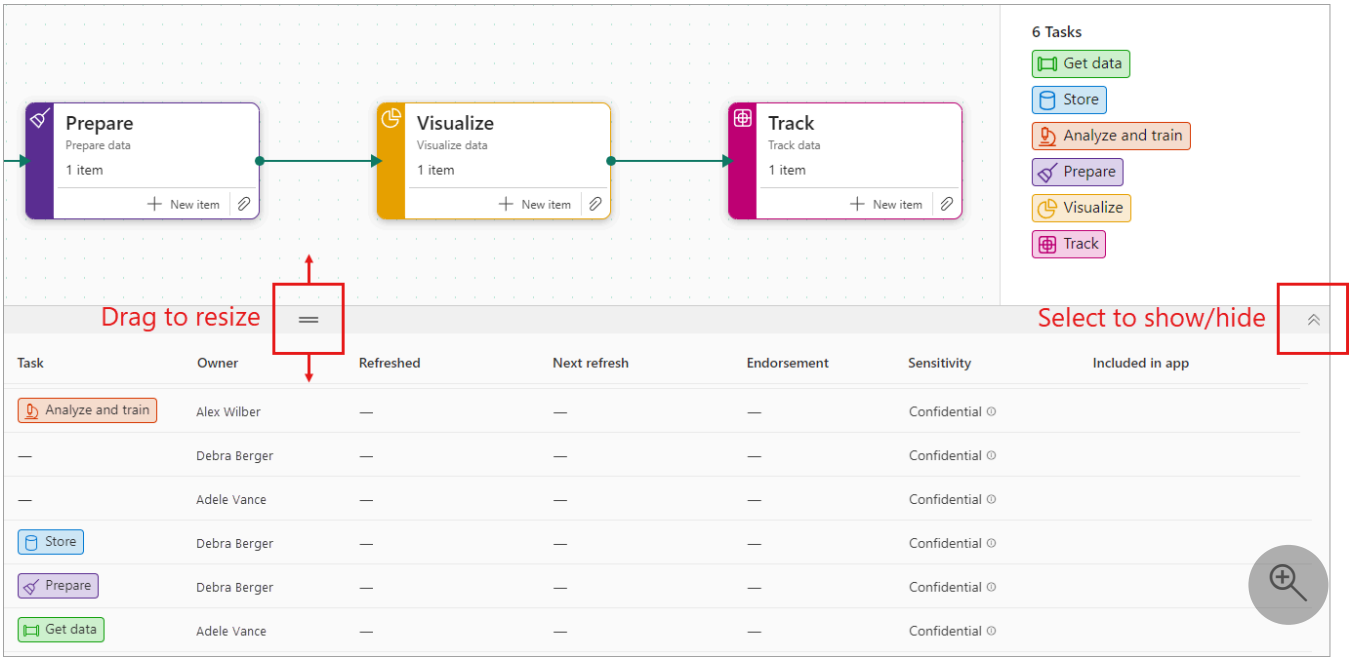


1. Add task or connector
2. Edit task name and description
3. Change task type
4. Create new item for task
5. Assign existing items to task
6. Delete task

Resize or hide the task flow

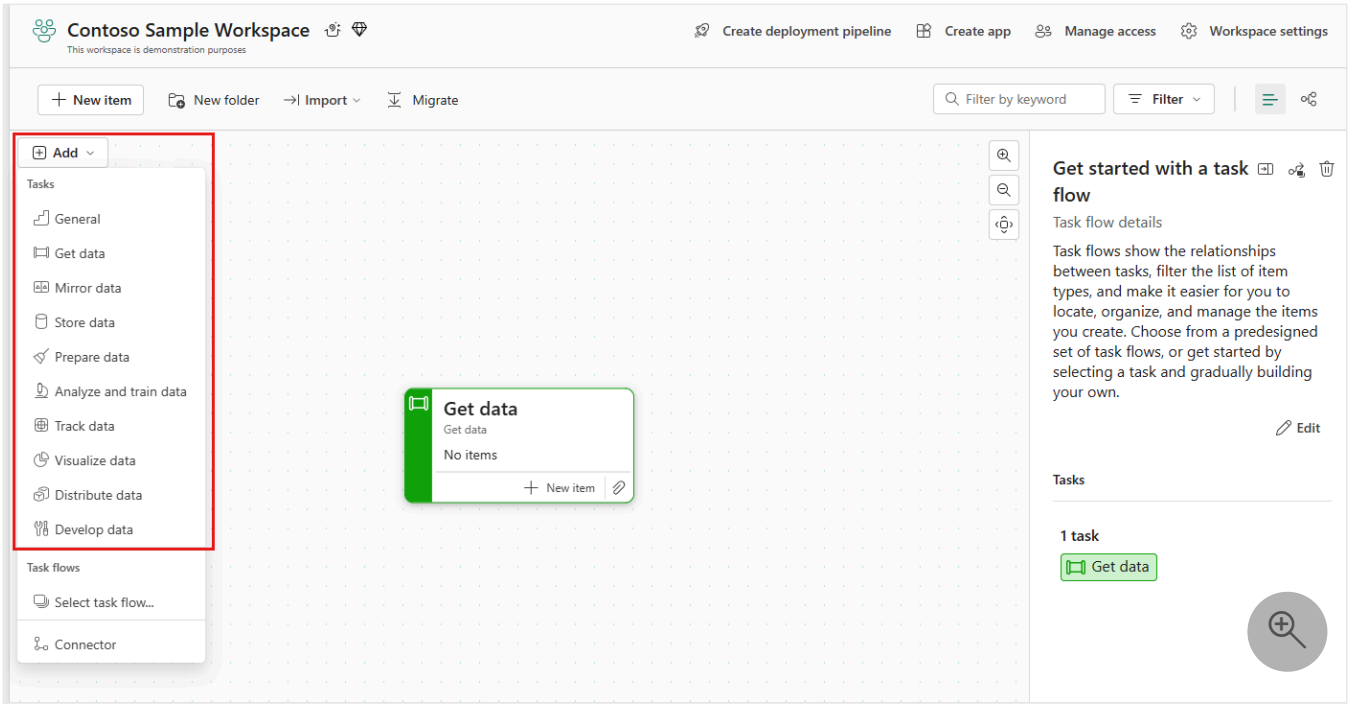
You can resize the task flow, or even hide it, according to your personal needs and preferences. Fabric remembers task flow resize and show/hide choices per user and per workspace, so each time you return to a workspace, the task flow size and show/hide status will be the same as it was the last time you left the workspace.

- To resize the task flow, drag the resize bar on the horizontal separator up or down.
- To show/hide the task flow, select the show/hide control at the right side of the separator.



Add a task

To add a new task to the task flow canvas, open the **Add** dropdown menu and select the desired task type.



The task of the selected task type is added onto the canvas. The name and description of the new task are the default name and description of the task type. Consider changing the name and description of the new task to better describe its purpose in the work flow. A good task name should identify the task and provide a clear indication of its intended use.

Edit task name and description

To edit a task's name or description:

1. Select the task on the canvas to open the task details pane.
2. Select **Edit** and change the name and description fields as desired. When done, select **Save**.

Change task type

To change a task to a different type:

1. Select the task on the canvas to open the task details pane.
2. Open the **Task type** dropdown menu and choose the new desired task type.

> **Note**
>
> Changing the task type doesn't change the task name or description. Consider changing these fields to suit the new task type.

Arrange tasks on the canvas

Part of building a task flow is arranging the tasks in the proper order. To arrange the tasks, select and drag each task to the desired position in the task flow.

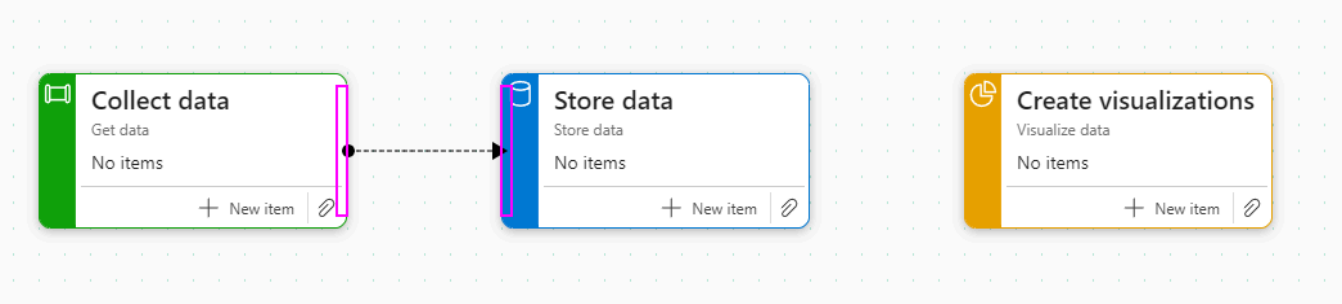
> **Tip**
>
> When you move tasks around on the canvas, they stay in the place where you put them. However, due to a known issue, when you add a new task to the canvas, any unconnected tasks will move back to their default positions. Therefore, to safeguard your arrangement of tasks, it's highly recommended to connect them all with connectors before adding any new tasks to the canvas.

Connect tasks

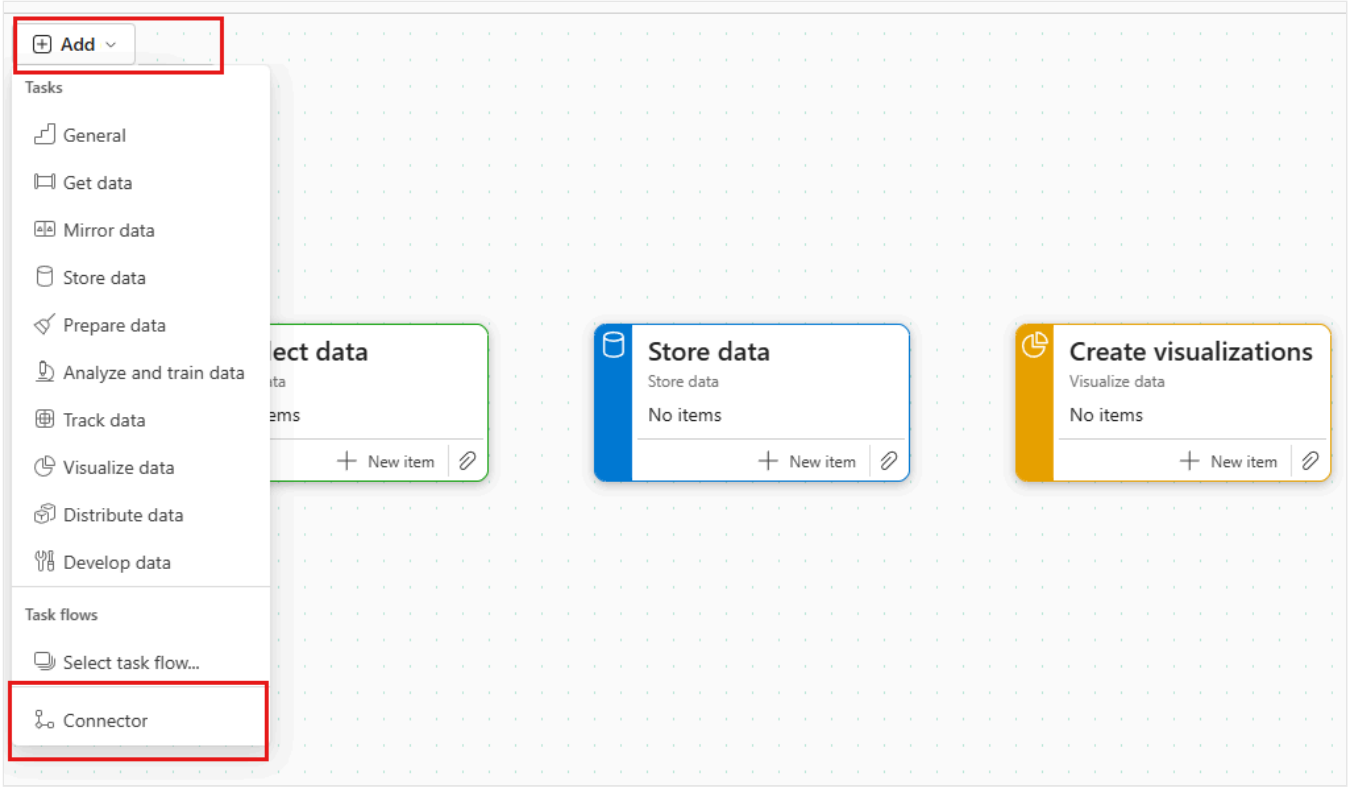
Connectors show the logical flow of work. They don't make or indicate any actual data connections - they are graphic representations of the flow of tasks only.

Add a connector

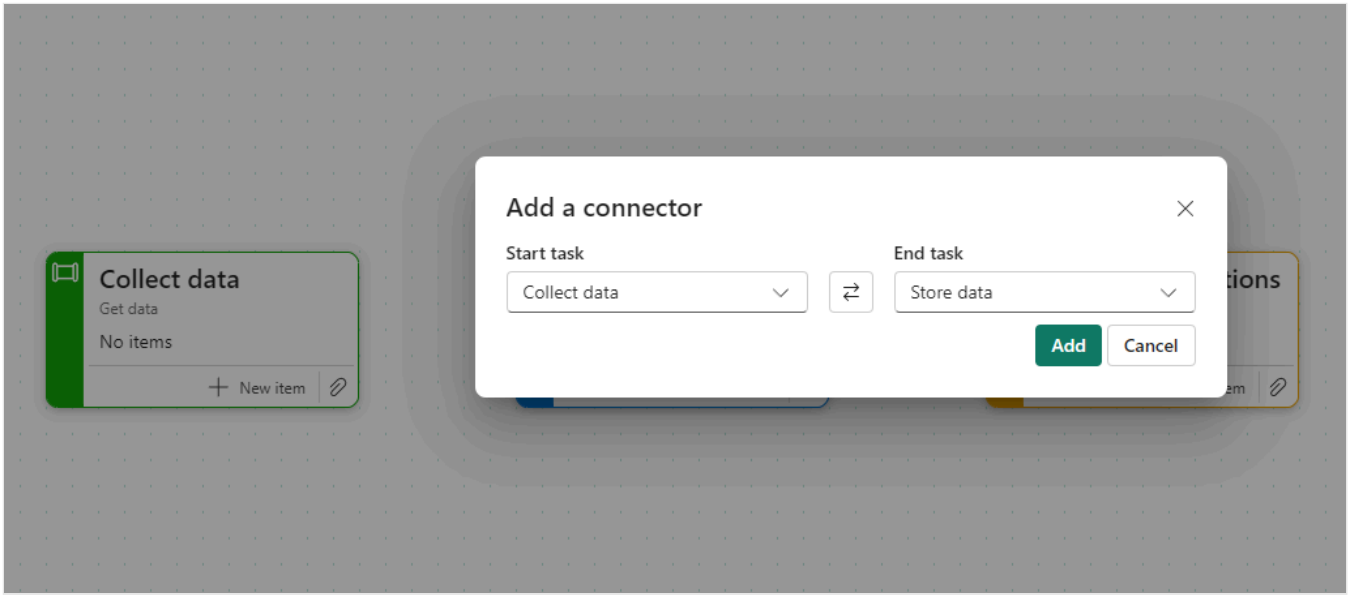
To connect two tasks, select the edge of the starting task and drag to an edge of the next task.



Alternatively, you can select **Add** > **Connector** from the **Add** dropdown on the canvas.



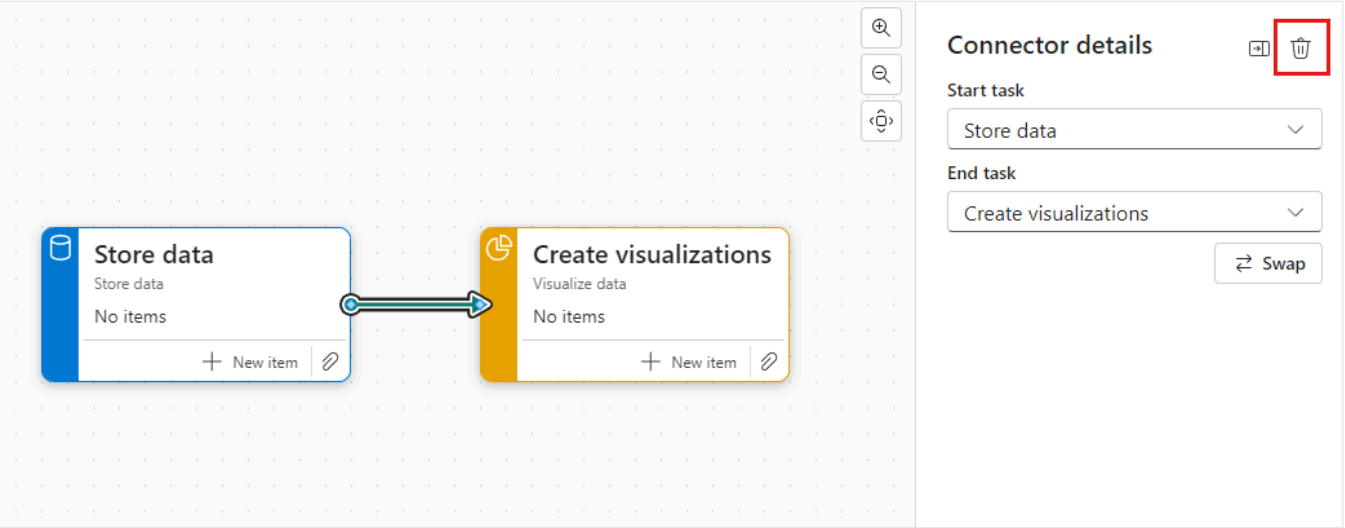
Then, in the **Add connector** dialog, select the start and end tasks, then select **Add**.



Delete a connector

To delete a connector, select it and press **Enter**.

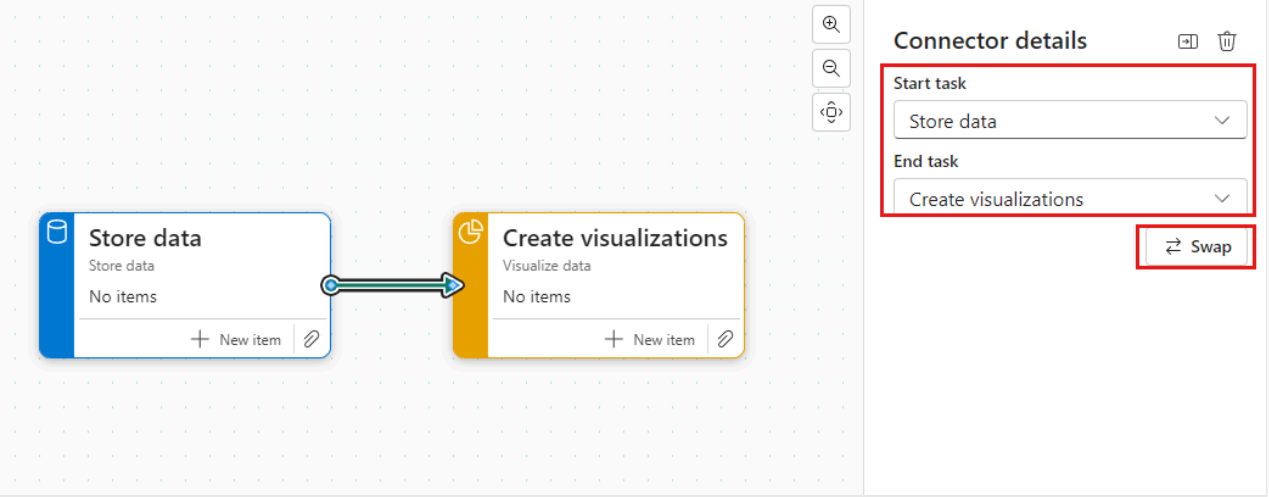
Alternatively, select the connector to open the connector details pane, then select the trash can icon.



Change connector start and end points or direction

To change a connector's start and end values, or switch its direction:

1. Select the connector to open the connector details pane.
2. In the details pane, change the start and end values as desired, or select **Swap** to change connector direction.



Assign items to a task

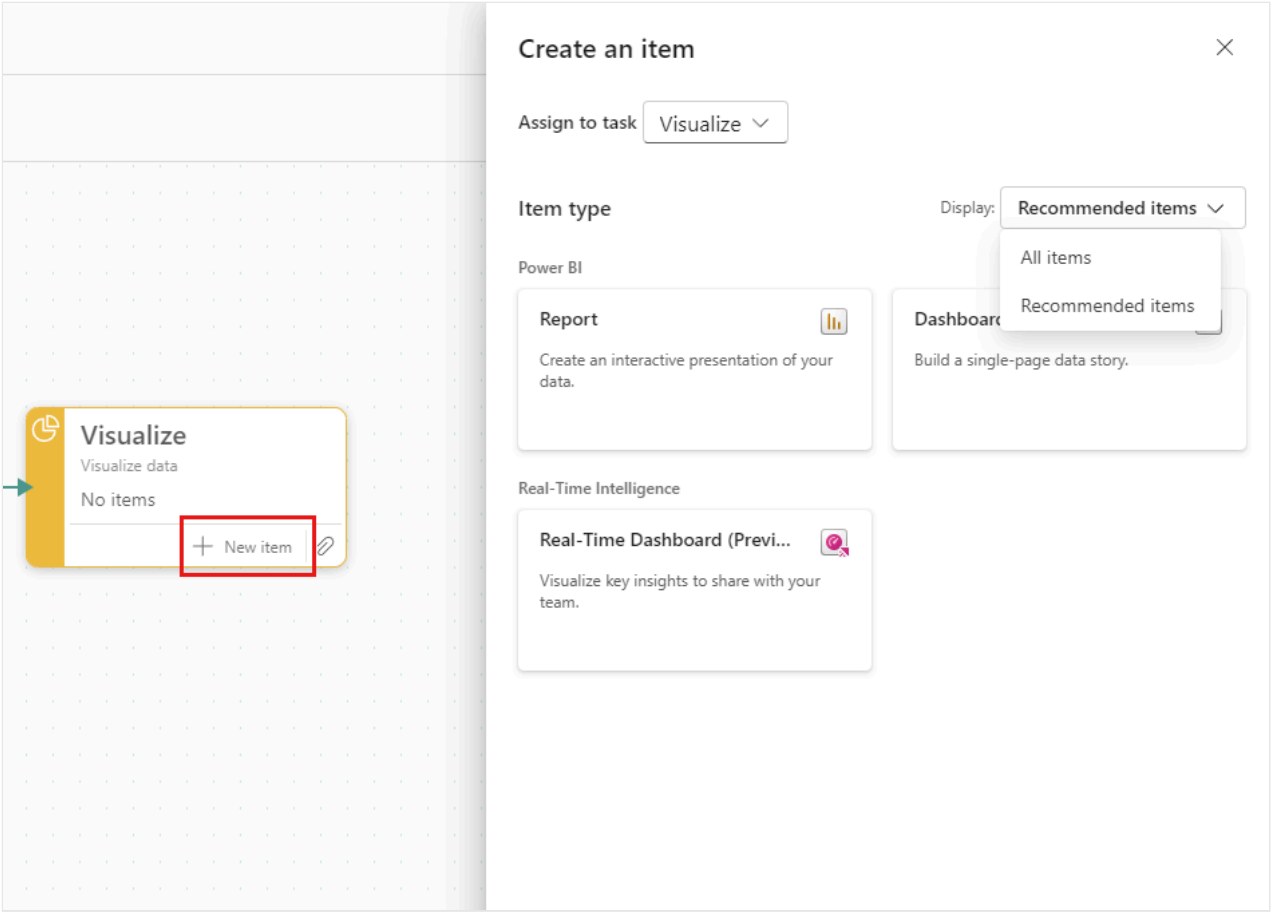
Once a task has been placed on the canvas, you can assign items to it to help structure and organize your work. You can create new items to be assign to the task, or you can assign items that already exist in the workspace.

> **Note**
>
> An item can only be assigned to a single task. It can't be assigned to multiple tasks.

Create a new item for a task

To create a new item for a specific task:

1. Select + **New item** on the task.



2. On the **Create an item** pane that opens, the recommended item types for the task are displayed by default. Choose one of the recommended types.

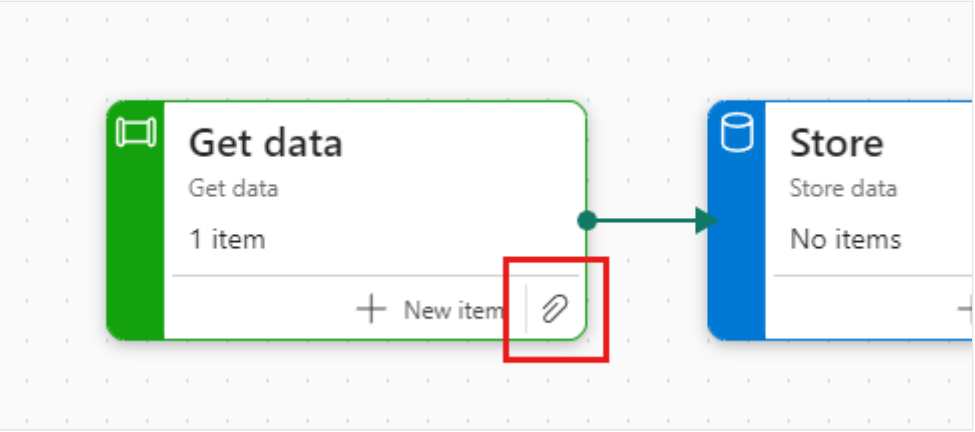
   If you don't see the item type you want, change the **Display** selector from *Recommended items* to *All items*, and then choose the item type you want.

> **Note**
>
> When you first save a new report, you'll be given the opportunity to assign it to any task that exists in the workspace.

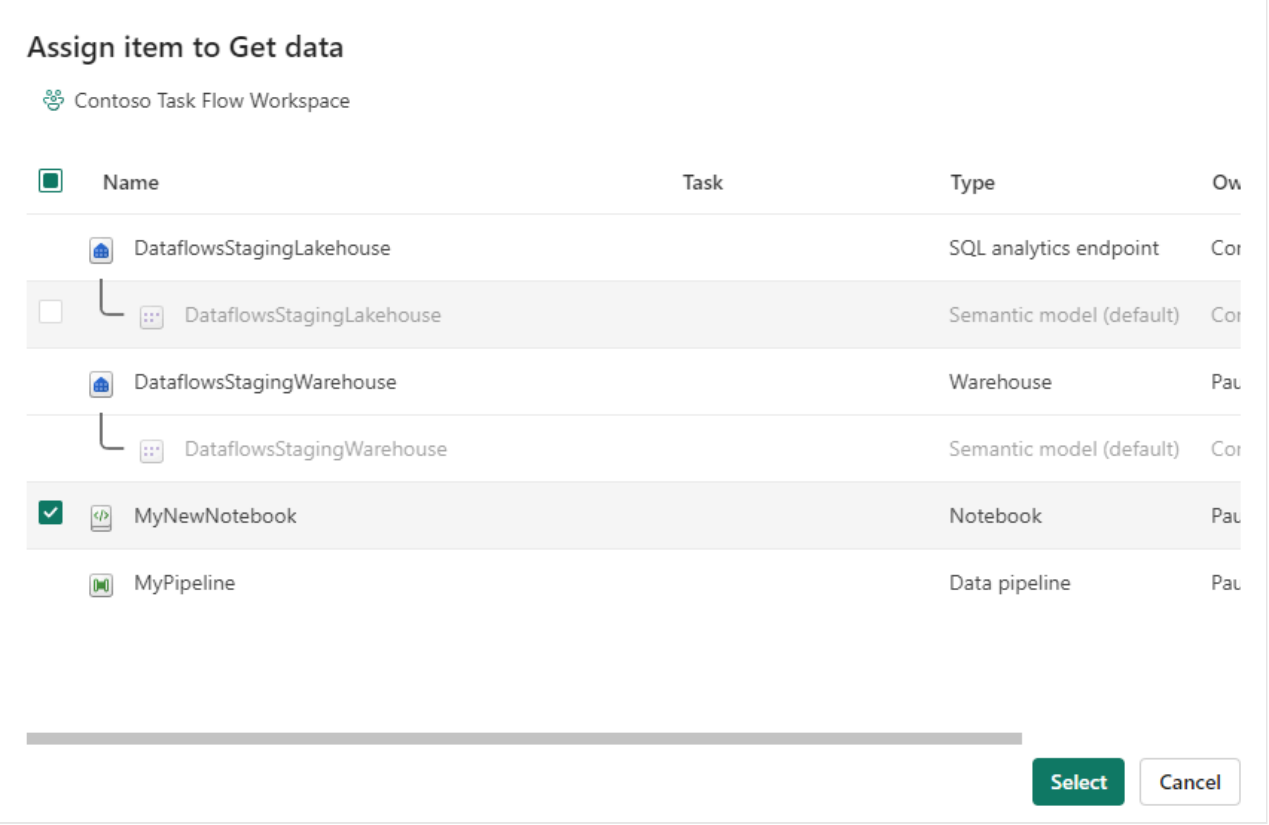
Assign existing items to a task

To assign existing items to a task:

1. Select the clip icon on the task.



2. In the **Assign item** dialog box that opens, hover over item you want to assign to the task and mark the checkbox. You can assign more than one item. When you're done choosing the items you want to assign to the task, choose **Select** to assign the selected items to the task.



The items you selected items are assigned to the task. In the item list, task assignments are shown in the **Task** column.

Unassign items from tasks

You can unassign items from a selected task or from multiple tasks.

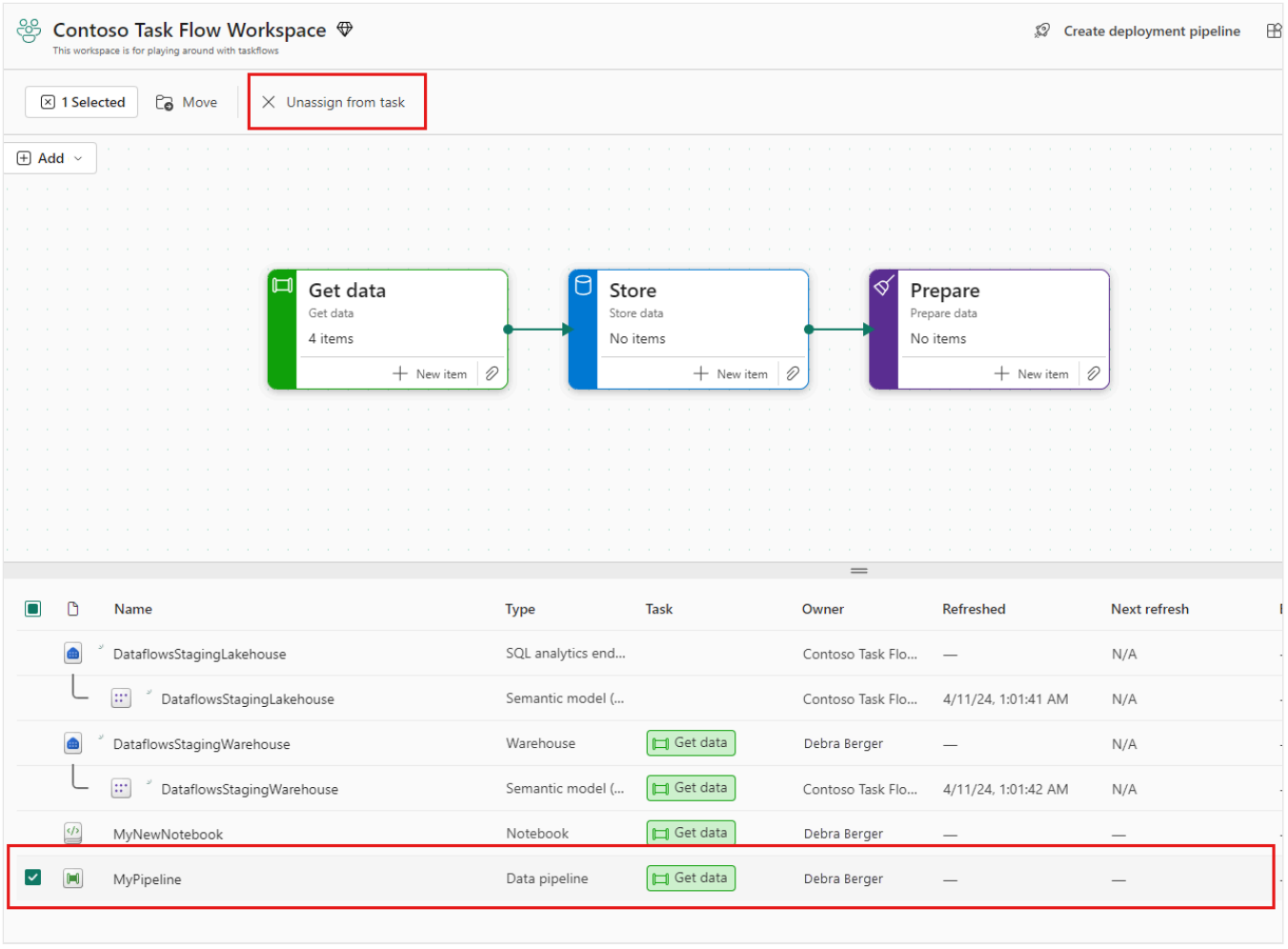
> ⓘ Note

Unassigning items from tasks **does not** remove the items from the workspace.

Unassign items from a task

To unassign items from a task:

1. Select the task you want to unassign the items from. This filters the item list to show just the items that are assigned to the task.
2. In the item list, hover over the items you want to unassign and mark the checkboxes that appear.
3. On the workspace toolbar, choose **Unassign from task** (or **Unassign from all tasks**, if you've selected multiple items).



Unassign items from multiple tasks

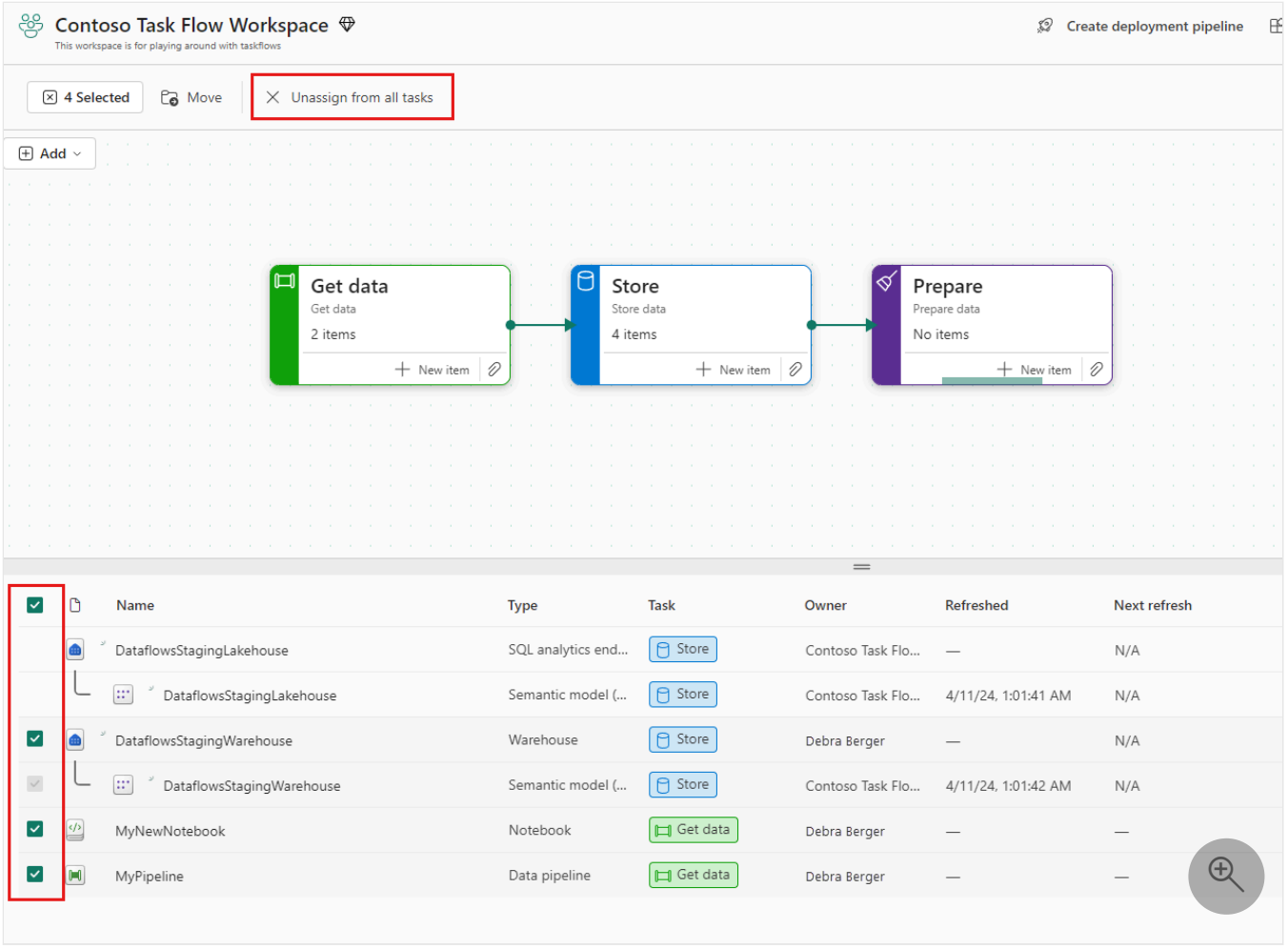
To unassign items from multiple tasks:

1. Select **Clear all** at the top of the items list to clear all filters so that you can see all the items in the workspace. Note that items that are assigned to tasks list the task name in

the **Task** column.

2. Hover over the items you want to unassign and mark the checkboxes.

3. When you've finished making your selections, select **Unassign from all tasks** in the workspace toolbar.



Delete a task

To delete a task:

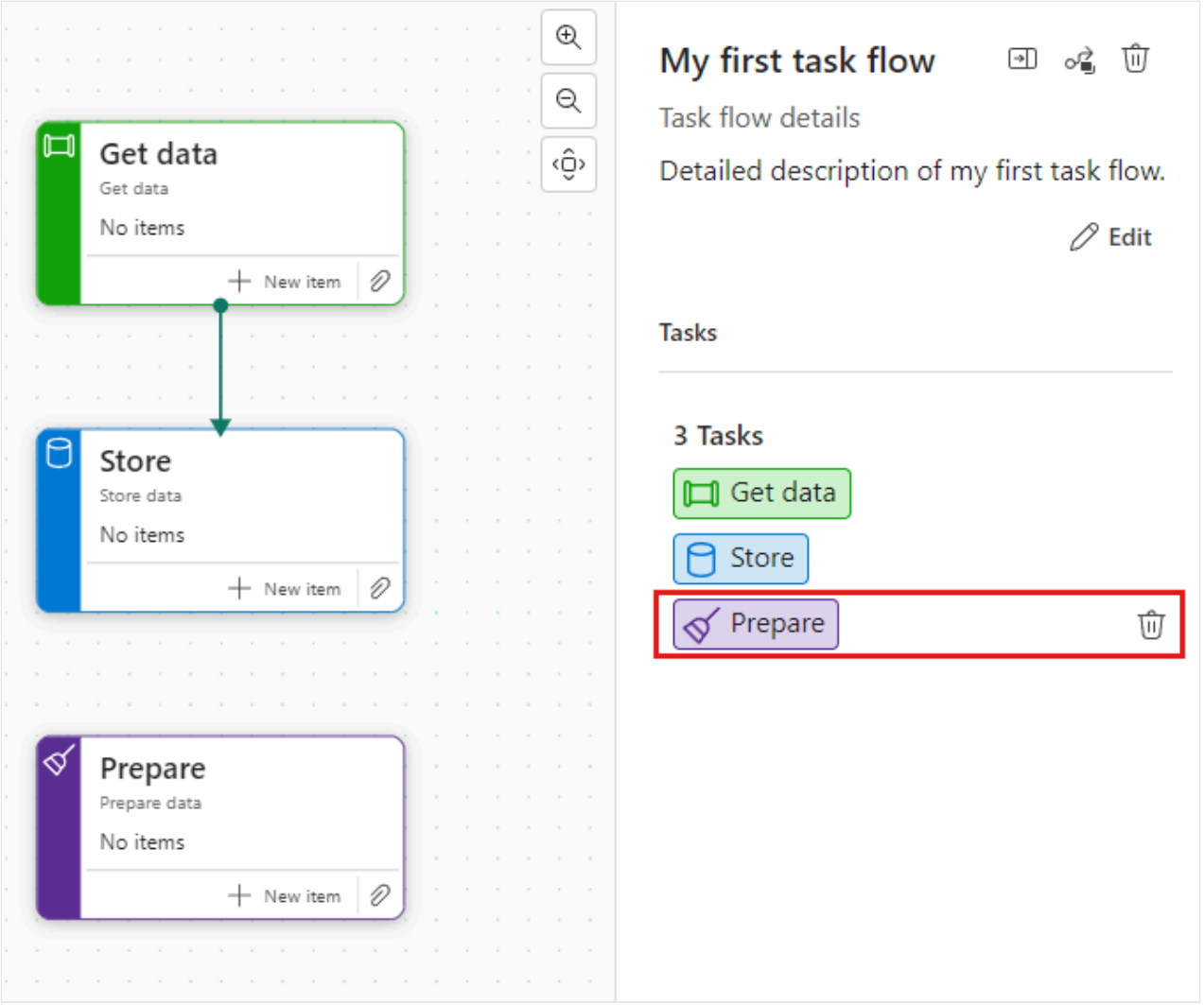
1. Select the task to open the task details pane.

2. Select the trash can icon.

Alternatively,

1. Select the task flow canvas to open the task flow details pane.

2. In the task flow details pane, hover over the task you want to delete in the Tasks list and select the trash can icon.

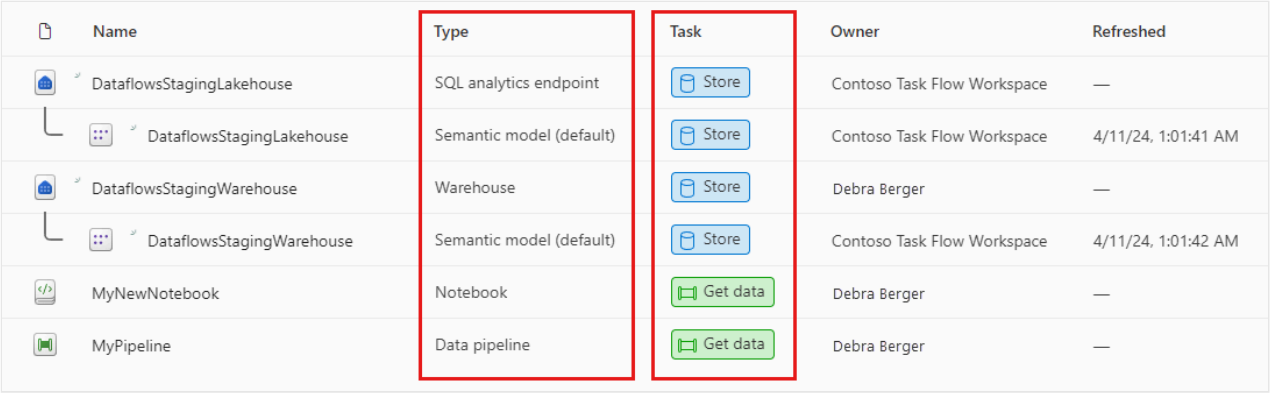


> **Note**
>
> Deleting a task doesn't delete the items assigned to it. They remain in the workspace.

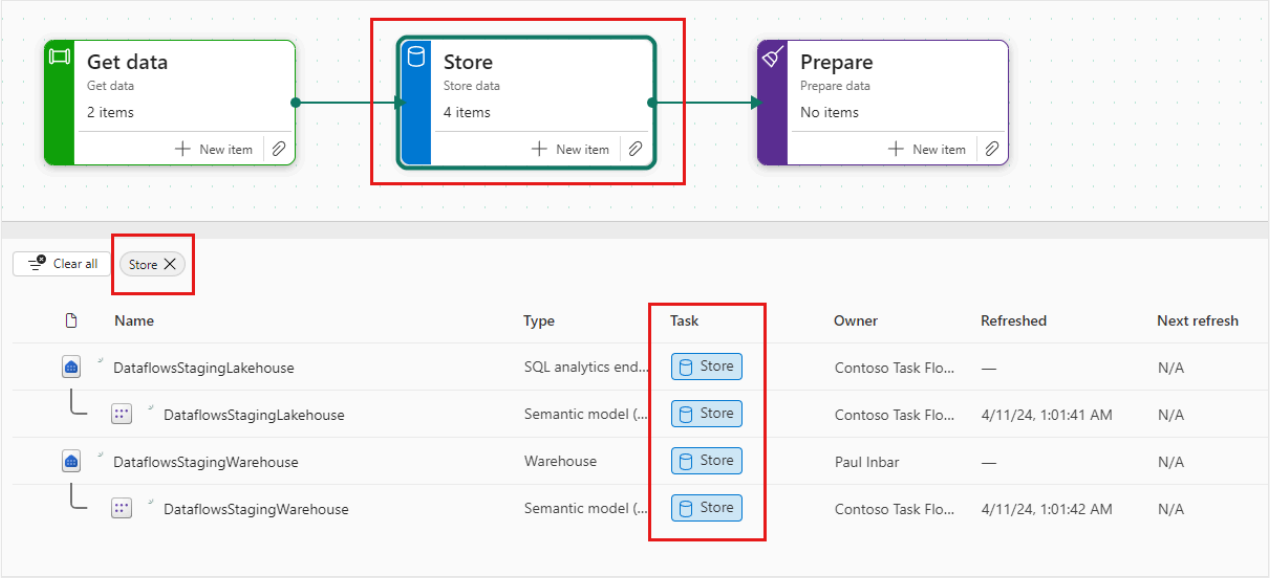
Navigate items with the task flow

With items assigned to tasks in a task flow, you can use the task flow to quickly understand how the items in the workspace work together, and get a clear picture of your work in the workspace.

- For each item that you see in the items list, you can see the item type and what task it's assigned to, if any.



- When you select a task, the items list is filtered to show only the items that are assigned to that task.



Select a new predefined task flow

At any point, you can choose to apply one of the predefined task flows to the canvas.

To select one of the predefined task flows:

1. Open the **Add** dropdown on the canvas and choose **Select task flow**. The predefined task flows pane will open.

2. Choose one of the predefined task flows and the select **Select**. If there already is a task flow on the canvas, you'll be asked whether to overwrite the current task flow or to append the predefined task flow to the current task flow.

Edit task flow details

To edit the task flow name or description:

1. Open the task flow details pane by selecting the task flow canvas.

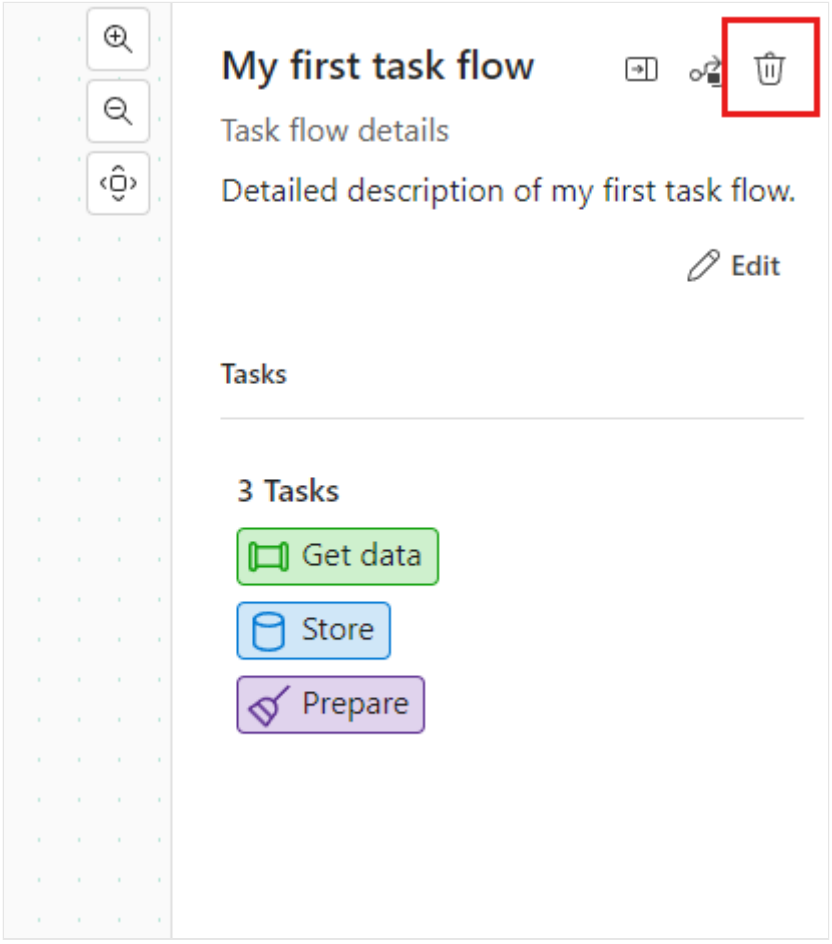
2. Select **Edit** and change the name and description fields as desired. When done, select **Save**.

> ⓘ Note
>
> A good task flow name and description should help others understand the intended purpose and use of the task flow.

Delete a task flow

To delete a task flow:

1. Select a blank area of the canvas to display the task flow details pane.
2. Select the trash icon to delete the task flow.



Deleting a task flow removes all tasks, the task list, and any item assignments, and resets the task flow to its original default empty state.

> ⓘ Note

Items that were assigned to tasks in the deleted task flow remain in the workspace. When you create a new task flow, you need to assign them to the tasks in the new flow.

Import or export a task flow

If you have a task flow that you'd like to reuse, you can export it from the workspace it's in and import it into other workspaces where you'd like to use it.

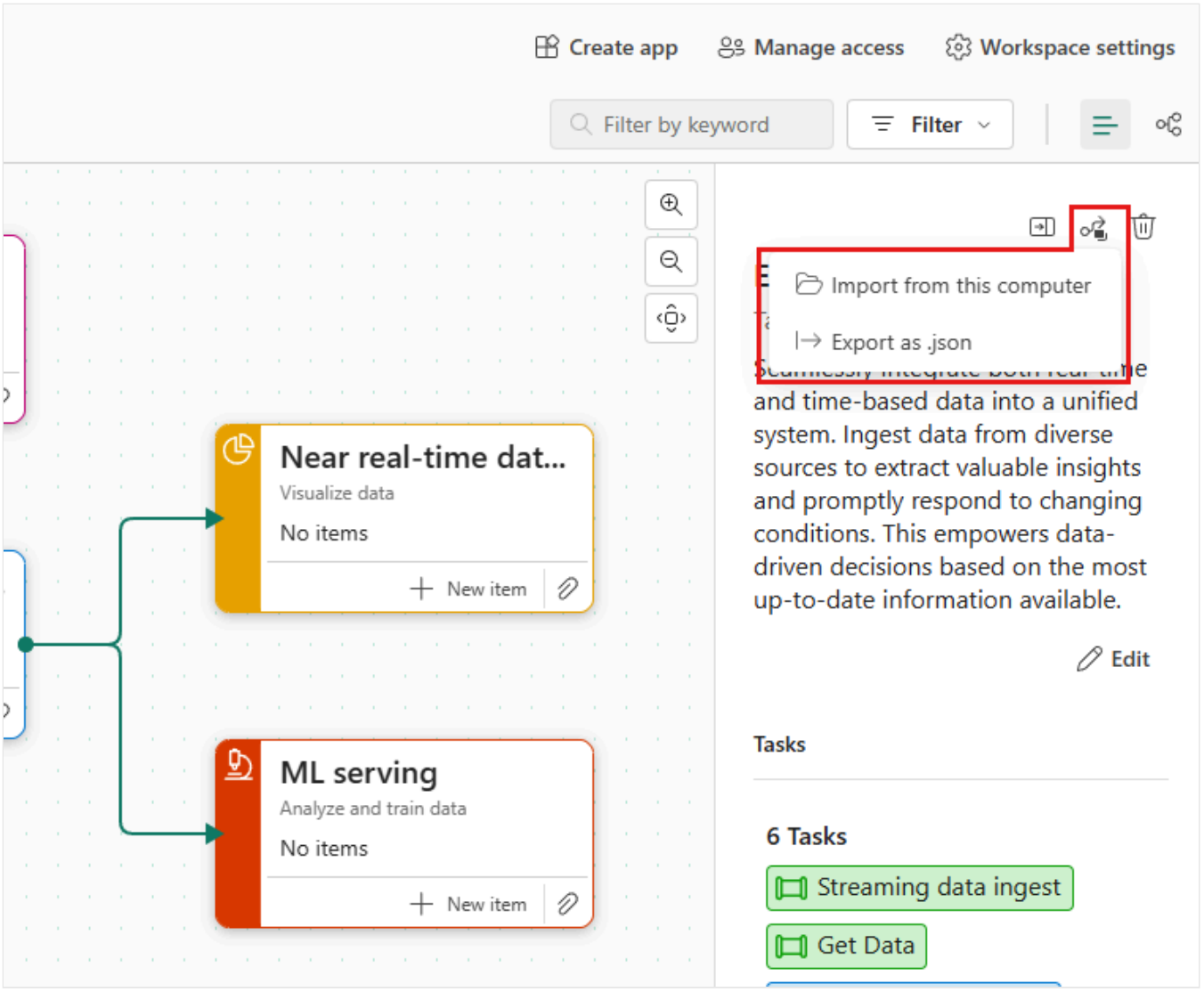
The task flow is exported as a *.json* file that you save to a location of your choice. The *.json* file preserves the structure and flow of the task flow, and includes:

- The task flow name and description.
- The tasks, along with their names and descriptions.
- The connectors between the tasks.

The *.json* file doesn't include item associations. When you import a task flow into a workspace, you have to create the item associations there.

After you've imported the task flow into a new workspace, modify it to suit your needs as described in this article. Make sure the task flow name and description suit the task flow in its new context, and adjust if necessary.

To import or export a task flow select the **Import and export task flow** icon in the task flow details pane and choose the relevant option.



Related concepts

- Task flow overview
- Set up a task flow

Manage multiple task flow canvases in a workspace

This article describes how to work with multiple task flow canvases in a workspace. You can create separate canvases for different data processes, select predesigned task flow, build or import task flows on each canvas, and keep your workspace easier to navigate as projects grow.

Prerequisites

To create, rename, or delete task flow canvases, you must be a workspace Admin, Member, or Contributor.

When to use multiple task flow canvases

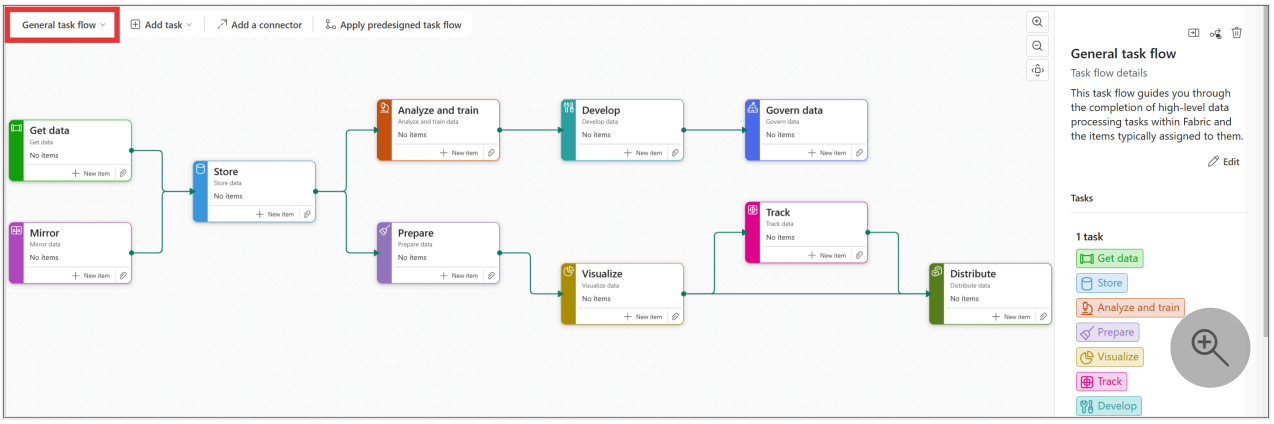
Use multiple canvases when you want to:

- Separate independent solution tracks in the same workspace.
- Keep domain-specific flows isolated from each other.
- Reduce visual complexity in large projects while preserving context.

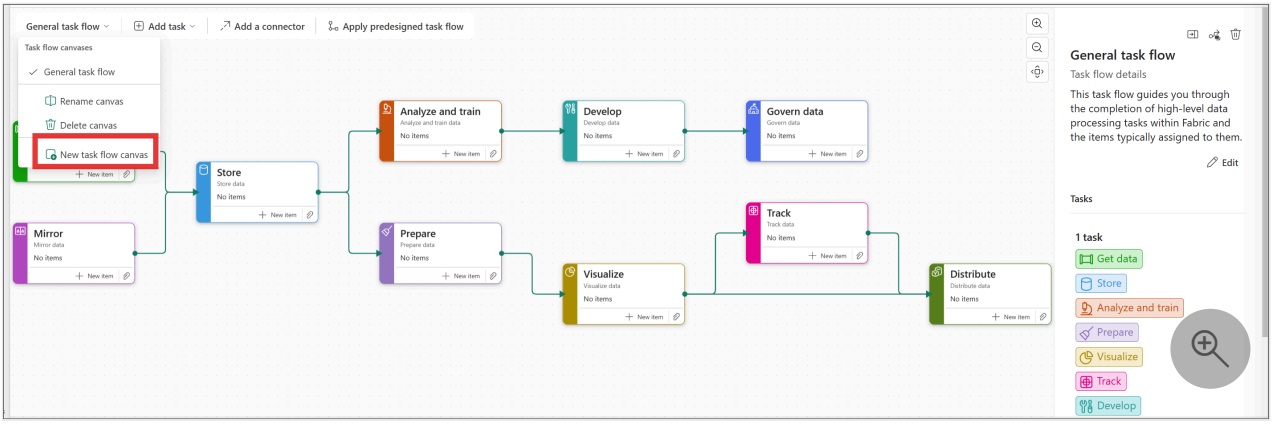
> ⓘ **Note**
>
> One item can be assigned to multiple tasks when those tasks are in different task flow canvases.

Create a task flow canvas

1. In the task flow area, select the task flow canvas selector.



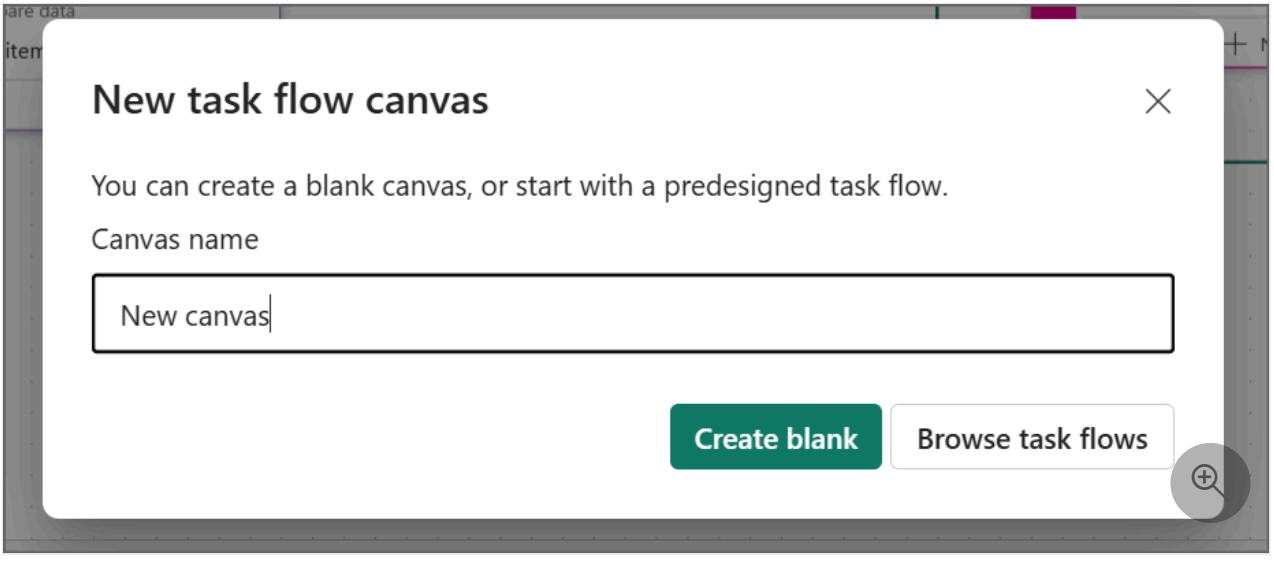
2. In the flyout menu, select **New task flow canvas**.



3. In the **New task flow canvas** dialog, enter a canvas name. The name field is required.

4. Choose one of the following:

   - Select **Create blank** to create an empty canvas.
   - Select **Browse task flows** to start from a predesigned task flow.



The new canvas appears in the canvas selector and opens on the task flow area.

Add a task flow in a canvas

After you create or select a canvas, add a task flow to that canvas.

1. Make sure the target canvas is selected.
2. Choose one of the following options:
    - **Select a predesigned task flow** to start from a Microsoft-provided template.
    - **Add a task** to build a custom task flow from scratch.
    - **Import a task flow** to reuse a previously exported *.json* file.

For detailed steps to build and edit the task flow itself, see Set up a task flow and Work with task flows.

Navigate and switch task flow canvases

Before you rename or delete a canvas, switch to the target canvas first.

1. Select the task flow canvas selector menu.
2. In the canvas list, select and click another task flow canvas to switch to it.

Rename a task flow canvas

1. Open the canvas selector, open canvas options, and then select **Rename canvas**.
2. Enter the new name, and then select **Rename**.

Use clear and descriptive names so you can quickly identify each canvas purpose.

Delete a task flow canvas

1. Open the canvas selector, open canvas options, and then select **Delete canvas**.
2. Confirm the deletion.

> ⚠ **Warning**
>
> Deleting this task flow canvas deletes all tasks and any assignments created between tasks and items. Items in your workspace won't be deleted.

Related content

- Task flows overview
- Set up a task flow
- Work with task flows

Last updated on 04/10/2026

Microsoft Fabric decision guide: copy activity, Copy job, dataflow, Eventstream, or Spark

06/04/2025

Use this reference guide and the example scenarios to help you in deciding whether you need a copy activity, Copy job, a dataflow, an Eventstream, or Spark for your Microsoft Fabric workloads.

Copy activity, Copy job, dataflow, Eventstream, and Spark properties

Expand table

	Pipeline copy activity	Copy job	Dataflow Gen 2	Eventstream	Spark
Use case	Data lake and data warehouse migration, data ingestion, lightweight transformation	Data Ingestion, Incremental copy, Replication, Data Lake and Data Warehouse migration, lightweight transformation	Data ingestion, data transformation, data wrangling, data profiling	event data ingestion, event data transformation	Data ingestion, data transformation, data processing, data profiling
Primary developer persona	Data engineer, data integrator	Business Analyst, Data Integrator, Data Engineer	Data engineer, data integrator, business analyst	Data engineer, data scientist, data developer	Data integrator, data engineer
Primary developer skill set	ETL, SQL, JSON	ETL, SQL, JSON	ETL, M, SQL	SQL, JSON, messaging	Spark (Scala, Python, Spark SQL, R)
Code written	No code, low code	No code, low code	No code, low code	No Code, low code	Code
Data volume	Low to high	Low to high	Low to high	Medium to High	Low to high

	Pipeline copy activity	Copy job	Dataflow Gen 2	Eventstream	Spark
Development interface	Wizard, canvas	Wizard, canvas	Power query	Canvas	Notebook, Spark job definition
Sources	50+ connectors	50+ connectors	150+ connectors	Database supporting CDC (Change Data Capture), Kafka, Messaging Systems that support publish and subscribe pattern, Event streams	Hundreds of Spark libraries
Destinations	40+ connectors	40+ connectors	Lakehouse, Azure SQL database, Azure Data explorer, Azure Synapse analytics	Eventhouse, Lakehouse, Activator Alert, Derived Stream, Custom Endpoint	Hundreds of Spark libraries
Transformation complexity	Low: lightweight - type conversion, column mapping, merge/split files, flatten hierarchy	Low: lightweight - type conversion, column mapping, merge/split files, flatten hierarchy	Low to high: 300+ transformation functions	Low: lightweight	Low to high: support for native Spark and open-source libraries

Scenarios

Review the following scenarios for help with choosing how to work with your data in Fabric.

Scenario 1

Leo, a data engineer, needs to ingest a large volume of data from external systems, both on-premises and cloud. These external systems include databases, file systems, and APIs. Leo

doesn’t want to write and maintain code for each connector or data movement operation. He wants to follow the medallion layers best practices, with bronze, silver, and gold. Leo doesn't have any experience with Spark, so he prefers the drag and drop UI as much as possible, with minimal coding. And he also wants to process the data on a schedule.

The first step is to get the raw data into the bronze layer lakehouse from Azure data resources and various third party sources (like Snowflake Web, REST, AWS S3, GCS, etc.). He wants a consolidated lakehouse, so that all the data from various LOB, on-premises, and cloud sources reside in a single place. Leo reviews the options and selects **pipeline copy activity** as the appropriate choice for his raw binary copy. This pattern applies to both historical and incremental data refresh. With copy activity, Leo can load Gold data to a data warehouse with no code if the need arises and pipelines provide high scale data ingestion that can move petabyte-scale data. Copy activity is the best low-code and no-code choice to move petabytes of data to lakehouses and warehouses from varieties of sources, either ad-hoc or via a schedule.

Scenario 2

Mary is a data engineer with a deep knowledge of the multiple LOB analytic reporting requirements. An upstream team has successfully implemented a solution to migrate multiple LOB's historical and incremental data into a common lakehouse. Mary has been tasked with cleaning the data, applying business logics, and loading it into multiple destinations (such as Azure SQL DB, ADX, and a lakehouse) in preparation for their respective reporting teams.

Mary is an experienced Power Query user, and the data volume is in the low to medium range to achieve desired performance. Dataflows provide no-code or low-code interfaces for ingesting data from hundreds of data sources. With dataflows, you can transform data using 300+ data transformation options, and write the results into multiple destinations with an easy to use, highly visual user interface. Mary reviews the options and decides that it makes sense to use **Dataflow Gen 2** as her preferred transformation option.

Scenario 3

Prashant, a data integrator with deep expertise in business processes and systems. An upstream team has successfully exposed event data from business applications as messages that can be consumed through downstream systems. Prashant has been assigned to integrate event data from business applications into Microsoft Fabric for real-time decision support.

Given the medium to high data volume and the organization's preference for no-code solutions, Prashant seeks a way to seamlessly forward events as they occur without managing extraction schedules. To meet this need, he chooses **Eventstreams** in Microsoft Fabric.

Eventstreams within the Real-Time Intelligence experience enables real-time data ingestion, transformation, and routing to various destinations—all without writing any code.

Scenario 4

Adam is a data engineer working for a large retail company that uses a lakehouse to store and analyze its customer data. As part of his job, Adam is responsible for building and maintaining the data pipelines that extract, transform, and load data into the lakehouse. One of the company's business requirements is to perform customer review analytics to gain insights into their customers' experiences and improve their services.

Adam decides the best option is to use **Spark** to build the extract and transformation logic. Spark provides a distributed computing platform that can process large amounts of data in parallel. He writes a Spark application using Python or Scala, which reads structured, semi-structured, and unstructured data from OneLake for customer reviews and feedback. The application cleanses, transforms, and writes data to Delta tables in the lakehouse. The data is then ready to be used for downstream analytics.

Scenario 5

Rajesh, a data engineer, is tasked with ingesting incremental data from an on-premises SQL Server into an Azure SQL Database. Rajesh's On-premises SQL Server instance already has Change Data Capture (CDC) enabled on key tables.

Rajesh is looking for a simple, low-code, wizard-driven solution that enables him to:

- Select multiple native CDC enabled source tables
- Perform an initial full load
- Automatically switch to incremental data loads based on CDC
- Schedule data refreshes for recurring updates

He wants to avoid writing custom code or managing complex orchestrations. Ideally, he wants a "5x5 wizard" where he can accomplish the setup in just a few clicks.

Rajesh chooses the Copy job feature in Microsoft Fabric. With on-premises gateway support, he securely connects to his SQL Server, selects the desired tables, and configures the flow to land into the target Azure SQL Database.

The Copy job provides a low-friction and scalable data movement experience, meeting Rajesh’s requirements without the need to maintain complex pipelines.

Related content

- How to copy data using copy activity
- Quickstart: Create your first dataflow to get and transform data
- How to create an Apache Spark job definition in Fabric
- How to create an Eventstream in Fabric

Microsoft Fabric decision guide: Choose a data integration strategy

09/26/2025

Microsoft Fabric has a comprehensive suite of tools to handle data and analytics workloads efficiently. With so many options available, including batch, pipeline, and real-time streaming capabilities, it can be challenging to pick the right tool for your specific needs. This decision guide provides a roadmap to help you select the right strategy.

	Data movement				Orchestration		Transformation	
	Mirroring	Copy Job	Copy Activity (Pipeline)	Eventstream	Pipeline	Apache Airflow Job	Notebooks	Dataflow Gen 2
Use Case	Data Replication	Data Ingestion & Replication	Data Ingestion	Stream Data Ingestion & Processing	Low Code Orchestration	Code-first Orchestration	Code-first Data Prep / Transform	Code-free Data Prep / Transform
Flagship Scenarios	Near real-time sync with turn-key set up. Replication.	Incremental Copy / Replication (water-mark + Native CDC), Data Lake / Storage Data Migration, Medallion Ingestion, Out-of-the-box multi-table copy.	Data Lake / Storage Data Migration, Medallion Ingestion, Incremental copy via pipeline expressions & control tables (water-mark only)	Incremental processing, event-driven and real-time AI applications.	Logical grouping of several activities together to perform a task.	Python Code-Centric Authoring	Complex Transformations	Transformation & Profiling
Source	6+ connectors	50+ connectors	50+ connectors	25+ connectors, custom endpoint (AMQP, Kafka, HTTP)	All Fabric compatible sources (depending on selected pipeline activities)	100+ connectors	100+ Spark Libraries	170+ built-in connectors + Custom SDK
Destination	Mirrored database (stored as read-only Delta table in Fabric OneLake)	40+ connectors	40+ connectors	Eventhouse, Activator, Lakehouse, custom endpoint (AMQP, Kafka, HTTP), Spark	All Fabric compatible sources (depending on selected pipeline activities)	100+ connectors	100+ Spark Libraries	7+ connectors
Type of Incoming Data	Near Real-time	Batch / Incremental Copy (water-mark based & change data capture) / Near Real-time	Batch / Bulk / Manual Watermark-based incremental copy	Real-time streaming data, change data capture feeds	All types	All types	All types	All types
Persona	Business Analyst, Database Administrator	Business Analyst, Data Integrator, Data Engineer	Data Integrator, Business Analyst, Data Engineer	Data analyst, data engineer, & integrator	Data Integrator, Business Analyst, Data Engineer	Apache Airflow Users	Data Scientist, Developer	Data Engineer, Data Integrator, Business Analyst
Skillset	None	ETL, SQL	ETL, SQL	SQL, ETL, KQL	ETL, SQL, Spark (Scala, Py, SQL, R)	Python	Spark (Scala, Py, SQL, R)	ETL, M, SQL
Coding Level	No code	No code / Low code	No code / Low code	No code / Low code	No code / Low code	Code-first	Code-first	No code / Low code
Transformation Support	None	Low	Low	Medium (stream analytics)	None	None	High	High (400+ activities)

To choose the right data integration service in Microsoft Fabric, consider these questions:

- **What's your primary goal?** Do you want to ingest data, transform it, replicate it, orchestrate data movement, or stream and act on data in real-time?
- **What's your technical skill level?** Do you prefer no-code or low-code solutions, or are you comfortable working with code?
- **What type of data workload are you working with?** Is it batch, bulk, incremental, continuous streaming, or near real-time?
- **What kind of data transformation do you need?** Are you doing light transformations or complex ones?

For the list of supported connectors across Copy job, Copy activity, and Dataflow Gen 2, see the connector overview. For the list of supported Eventstream sources, see the sources list.

Data movement strategies

Expand table

	Mirroring	Copy Job	Copy Activity (Pipeline)	Eventstreams
Use Case	Data Replication	Data Ingestion & Replication	Data Ingestion	Streaming Data Ingestion & Processing
Flagship Scenarios	Near real-time sync with turn-key setup. Replication	Incremental Copy / Replication (water-mark + Native CDC), Data Lake / Storage Data Migration, Medallion Ingestion, Out-of-the-box multi-table copy.	Data Lake / Storage Data Migration, Medallion Ingestion, Incremental copy via pipeline expressions & control tables (water-mark only)	Incremental processing, event-driven, and real-time AI applications
Source	6+ connectors	50+ connectors	50+ connectors	25+ sources
Destination	Mirrored database (stored as read-only Delta table in Fabric OneLake)	40+ connectors	40+ connectors	4+ destinations
Type of Incoming Data	Near Real-time	Batch / Incremental Copy (water-mark based & change data capture) / Near Real-time	Batch / Bulk / Manual Watermark-based incremental copy	Real-time streaming data, Change Data Capture/Feeds
Persona	Business Analyst, Database Administrator	Business Analyst, Data Integrator, Data Engineer	Data Integrator, Business Analyst, Data Engineer	Data Engineer & Integrator, Data Analyst
Skillset	None	ETL, SQL	ETL, SQL	ETL, SQL, KQL
Coding Level	No code	No code / Low code	No code / Low code	No code / Low code
Transformation Support	None	Low	Low	Medium (stream analytics)

For more details, see the data movement strategy.

Orchestration strategies

Expand table

	Pipeline	Apache Airflow Job
Use Case	Low Code Orchestration	Code-first Orchestration
Flagship Scenarios	Logical grouping of several activities together to perform a task.	Python Code-Centric Authoring
Source	All Fabric compatible sources (depending on selected pipeline activities)	100+ connectors
Destination	All Fabric compatible sources (depending on selected pipeline activities)	100+ connectors
Type of Incoming Data	All types	All types
Persona	Data Integrator, Business Analyst, Data Engineer	Apache Airflow Users
Skillset	ETL, SQL, Spark (Scala, Py, SQL, R)	Python
Coding Level	No code / Low code	Code-first
Transformation Support	None	None

Transformation strategies

Expand table

	Notebooks	Dataflow Gen 2	Eventstreams
Use Case	Code-first Data Prep / Transform	Code-free Data Prep / Transform	Code-free Transformation / SQL-based Stream Analytics
Flagship Scenarios	Complex Transformations	Transformation & Profiling	Stream Processing & Analytics
Source	100+ Spark Libraries	170+ built-in connectors + Custom SDK	25+ sources
Destination	100+ Spark Libraries	7+ connectors	4+ destinations

	Notebooks	Dataflow Gen 2	Eventstreams
Type of Incoming Data	All types	All types	All Types incl. JSON, AVRO, CSV, XML, TXT etc.
Persona	Data Scientist, Developer	Data Engineer, Data Integrator, Business Analyst	Data Engineer & Analyst
Skillset	Spark (Scala, Py, SQL, R)	ETL, M, SQL	SQL, KQL
Coding Level	Code-first	No code / Low code	No code / Low code
Transformation Support	High	High (400+ activities)	Medium

Scenarios

Review these scenarios to help you choose which data integration strategy to use in Microsoft Fabric.

Scenario 1

Hanna is a database administrator for a financial services company. She manages multiple critical SQL Server databases that power the organization's trading applications. The business needs near real-time access to this transactional data for regulatory reporting and risk analysis. However, Hanna needs to avoid impacting the performance of the production systems.

Hanna's challenge is providing analytics teams with up-to-date data without creating extra load on the operational databases. She doesn't want to build complex ETL pipelines or manage data movement processes. The data volumes are substantial, and the business needs the data available for analysis within minutes of transactions occurring in the source systems.

Hanna reviews the options and chooses **Mirroring** as the ideal solution. With Mirroring, she can set up near real-time data replication from her SQL Server databases to Microsoft Fabric with minimal configuration. The mirrored data becomes available in OneLake as Delta tables, enabling downstream analytics without affecting source system performance. Mirroring provides the turnkey setup she needs, automatically managing the complexity of data replication while ensuring business continuity.

Scenario 2

Charlie is a data analyst at a retail company. He's responsible for consolidating sales data from multiple regional databases into a central data warehouse. The company operates across different time zones, and each region's database uses change data capture (CDC) to track inventory and sales transactions. Charlie needs a solution that can handle the initial full load of historical data and then switch to incremental updates based on CDC.

Charlie wants a no-code, wizard-driven approach that lets him select multiple tables from various regional SQL Server instances, perform the initial bulk migration, and then automatically maintain up-to-date data through CDC-based incremental loads. The solution needs to handle both inserts and updates, and should merge changes into the destination without manual intervention.

Charlie evaluates the options and selects **Copy Job** as his preferred approach. Copy Job provides the multi-table selection capability he needs, supports both watermark-based and native CDC incremental copying, and offers an intuitive wizard interface. The out-of-the-box functionality lets him configure the entire data replication process without writing code, and the automatic detection of CDC-enabled tables simplifies the setup process.

Scenario 3

Rukmina is a data engineer at a manufacturing company. She needs to migrate large volumes of historical production data from an on-premises Oracle database to a new Fabric Warehouse. The migration involves copying hundreds of tables with millions of records, and she needs to implement a medallion architecture with bronze, silver, and gold layers. Rukmina has experience with SQL but prefers low-code solutions when possible.

The project requires her to copy raw data to the bronze layer, then apply lightweight transformations like data type conversions and column mapping as the data moves through the medallion layers. Rukmina needs to ensure the solution can handle the high data volumes efficiently and can be scheduled to run incrementally for ongoing operations. The stakeholders want a solution that can scale from gigabytes to petabytes of data as the business grows.

Rukmina reviews the available options and chooses **Copy Activity in Pipelines**. This approach gives her the drag-and-drop interface she prefers while providing the scalability needed for large data volumes. Copy Activity supports the 50+ connectors she needs for various source systems, and the pipeline framework lets her orchestrate the movement between medallion layers. With copy activity, she can implement both historical and incremental data refresh patterns while maintaining the performance required for petabyte-scale operations.

Scenario 4

Julian is a business analyst with strong SQL skills. He needs to orchestrate a complex data processing workflow that involves multiple steps: extracting data from various systems, running data quality checks, performing transformations, loading data into multiple destinations, and sending notifications to stakeholders. The workflow needs to run on a schedule and handle dependencies between different activities.

Julian's organization uses a mix of Azure services and on-premises systems, and the workflow requires both data movement and orchestration logic. He needs to coordinate activities like running stored procedures, calling web APIs, moving files, and executing other pipelines. While Julian is comfortable with SQL and basic scripting, he prefers a visual, low-code approach for building and maintaining these complex workflows.

Julian evaluates the options and selects **Pipelines** as the best fit for his requirements. Pipelines provide the visual canvas and drag-and-drop activities he needs to build complex orchestration workflows. The solution supports logical grouping of activities, dependency management, and scheduling capabilities. With 50+ connectors and various activity types (copy, lookup, stored procedure, web, etc.), Pipelines give him the flexibility to coordinate diverse tasks while maintaining the low-code approach he prefers.

Scenario 5

Darshan is a data scientist with extensive Python experience. He needs to build and maintain complex data processing workflows that integrate machine learning models, custom algorithms, and various external APIs. His organization's data science team prefers code-first approaches and wants to leverage their existing Python expertise, including custom libraries and advanced orchestration patterns.

Darshan needs a solution that supports Python-based directed acyclic graphs (DAGs), can handle complex dependencies between tasks, and integrates with the team's existing DevOps processes. The workflows involve data ingestion from multiple sources, feature engineering, model training, batch scoring, and custom business logic that requires the flexibility of full Python programming. The team values Apache Airflow's ecosystem and wants to maintain compatibility with their existing workflows.

Darshan reviews the options and chooses **Apache Airflow Jobs** as the ideal solution. This code-first approach lets his team use their Python expertise while building sophisticated data processing workflows. Apache Airflow Jobs provides the DAG-based orchestration they're familiar with, supports 100+ connectors through the Airflow ecosystem, and lets them implement custom business logic using Python. The managed service approach eliminates infrastructure concerns while preserving the flexibility and power of Apache Airflow.

Scenario 6

René is a data scientist at a research university. She needs to perform complex data analysis and transformation tasks on large datasets stored across multiple formats and sources. Her work involves statistical analysis, machine learning model development, and custom data processing algorithms that require the full power of distributed computing.

René works with structured and unstructured data including CSV files, JSON documents, Parquet files, and real-time streams. Her analysis requires complex transformations like joins across multiple large datasets, aggregations, statistical computations, and custom algorithms implemented in Python and Scala. She needs the flexibility to work interactively during exploration phases and then operationalize her code for production workloads.

René evaluates her options and chooses **Notebooks** as her primary tool. Notebooks provide the code-first environment she needs with full access to Spark's distributed computing capabilities. She can work with hundreds of Spark libraries, implement complex transformations using multiple languages (Python, Scala, SQL, R), and use the interactive development environment for data exploration. The notebook interface lets her combine code, visualizations, and documentation while providing the high-performance compute needed for her large-scale data processing requirements.

Scenario 7

Ako is a business analyst at a healthcare organization. She needs to integrate data from multiple sources including databases, web services, and file systems to create clean, business-ready datasets. Ako has extensive experience with Power Query from her work in Excel and Power BI, and she prefers visual, no-code interfaces for data preparation tasks.

Ako's responsibilities include cleaning healthcare data, applying business rules, validating data quality, and creating standardized datasets that feed into regulatory reporting systems. The data sources include patient management systems, laboratory information systems, and external API services. She needs to perform complex transformations like data profiling, duplicate removal, standardization of medical codes, and creation of calculated fields based on business logic.

Ako reviews the available options and selects **Dataflow Gen 2** as her preferred solution. Dataflow Gen 2 provides the familiar Power Query experience she knows from other Microsoft tools, while offering enhanced performance and capabilities. With 170+ built-in connectors, she can connect to all her diverse data sources, apply 300+ transformation functions through the visual interface, and take advantage of data profiling tools to ensure data quality. The no-code approach lets her focus on the business logic rather than technical implementation details.

Scenario 8

Ash is a product manager at a telecom company. Her team needs to monitor customer support metrics, like call volumes, wait times, and agent performance, in real time to ensure service-level agreements (SLAs) are met. The data comes from multiple operational systems including CRM, call center logs, and agent assignment databases.

Ash wants to build real-time dashboards and trigger automated workflows when thresholds are breached (for example, when wait times exceed SLA limits). She also wants to avoid building complex ETL pipelines or managing infrastructure.

Ash evaluates the options and selects Fabric Eventstreams. With Eventstreams, she can ingest data from multiple sources using streaming connectors, apply lightweight transformations, and route events to destinations like Eventhouse and Data Activator. She sets up alerts and dashboards that update in seconds, enabling her team to respond quickly to operational issues.

Fabric Eventstreams and Real-Time Intelligence provides the low-latency, low-code experience Ash needs to build event-driven applications without disrupting existing systems.

Get started

Now that you understand which service to use, you can start building your data integration solutions in Microsoft Fabric.

- Get started with Mirroring
- Create a Copy Job
- Create a Copy Activity
- Get started with Pipelines
- Get started with Eventstreams
- Get started with Apache Airflow Jobs
- Create and use Notebooks
- Get started with dataflows

ⓘ **Note:** The author created this article with assistance from AI. Learn more

Microsoft Fabric decision guide: Choose a data movement strategy

Microsoft Fabric offers several ways to bring data into the platform. This guide gives you a clear recommendation for the most common scenarios, then provides detailed feature comparisons when you need to dig deeper.

Quick recommendation: Which data movement option should I use?

Use the medallion architecture as your guide:

- **Gold data (reporting and analytics on processed data)** - Use **Mirroring**. If you already have ETL processing elsewhere and mainly need to bring curated data into Fabric for reporting, Mirroring is the simplest and most cost-effective choice. It's free, requires minimal setup, and continuously replicates your data into OneLake.
- **Bronze data (raw ingestion)** - Start with **Copy job**. When you're ingesting raw data, you'll quickly need transformations, schema mapping, scheduling control, and incremental loading. Copy job gives you these capabilities natively without the complexity of building pipelines.
- **Real-time streaming data** - Use **Eventstreams**. For low-latency, event-driven ingestion and processing, Eventstreams provides real-time pipelines with no-code transformations and routing to multiple destinations.
- **Complex orchestration** - Pipelines will give you the orchestration flexibility you need, and **copy activities in pipelines** offer data object parameterization, and metadata driven data ingestion. Otherwise, copy job activity and copy activity are equitable in a pipeline.

For a full side-by-side breakdown of capabilities and supported features, see the detailed feature comparison.

Capability	Mirroring	Copy job	Copy Activity (Pipeline)	Eventstream
Sources	Databases + 3rd party integration into Open Mirroring	All supported data sources and formats	All supported data sources and formats	25+ sources and all formats
Destinations	Tabular format in Fabric OneLake (read-only)	All supported destinations and formats	All supported destinations and formats	4+ destinations
Flexibility	Turn-key setup with fixed behavior	Simple to use + Advanced knobs	Advanced and fully customized knobs	Simple and customizable options
Custom scheduling	○	●	●	●
Table and Column management	○	●	●	●
Copy behavior: Append, Upsert, Override	○	●	●	●
Advanced observability + auditing	○	●	●	○
Copy modes				
CDC-based continuous replication	●	●	○	●
Batch or bulk copy	○	●	●	●
Native support for Incremental copy (watermark-based)	○	●	○	○
Copy using user defined query	○	●	●	○
Use cases				
Continuous Replication for analytics and reporting	●	●	○	●
Metadata driven ELT/ETL for data warehousing	○	●	●	○
Data consolidation	○	●	●	●
Data migration / Data backup / Data sharing	○	●	●	○
Free of cost	●	○	○	○
Predictable performance	○	●	●	●

Key concepts

- **Mirroring** gives you a **simple and free** way to mirror operational data into Fabric for analytics. It's optimized for ease of use with minimal setup, and it writes to a single, read-only destination in OneLake.
- **Copy activities in Pipelines** is built for users who need **orchestrated, pipeline-based data ingestion workflows**. You can customize it extensively and add transformation logic, but you need to define and manage pipeline components yourself, including tracking the state of the last run for incremental copy.
- **Copy Job** makes data ingestion easier with **native support for multiple delivery styles, including bulk copy, incremental copy, and change data capture (CDC) replication, and you don't need to build pipelines**, while still giving you access to many advanced options. It supports many sources and destinations, and works well when you want more control than Mirroring but less complexity than managing pipelines with Copy activity.
- **Eventstreams**: Designed for real-time ingestion, transformation and processing of streaming data. Supports low-latency pipelines, schema management, and routing to destinations like Eventhouse, Lakehouse, Activator and Custom Endpoints supporting (AMQP, Kafka and HTTP endpoints).

Detailed feature comparison

The following tables compare the full capabilities of each data movement option. Use this section when you need to evaluate specific features for your scenario.

Expand table

	Mirroring	Copy job	Copy Activity (Pipeline)	Eventstreams
Sources	Databases + third-party integration into Open Mirroring	All supported data sources and formats	All supported data sources and formats	25+ sources and all formats
Destinations	Tabular format in Fabric OneLake (read-only)	All supported destinations and formats	All supported destinations and formats	4+ destinations
Flexibility	Simple setup with fixed behavior	Easy to use + Advanced options	Advanced and fully customizable options	Simple and customizable options

Expand table

Capability	Mirroring	Copy job	Copy Activity (Pipeline)	Eventstreams
Custom scheduling		Yes	Yes	Continuous
Table and Column management		Yes	Yes	Yes (schema, event & field management)
Copy behavior: Append, Upsert, Override		Yes	Yes	Append
Advanced observability + auditing		Yes	Yes	
Copy modes				
CDC-based continuous replication	Yes	Yes		Yes
Batch or bulk copy		Yes	Yes	Yes (CDC initial snapshot replication)
Native support for Incremental copy (watermark-based)		Yes		
Copy using user defined query		Yes	Yes	
Use cases				

Capability	Mirroring	Copy job	Copy Activity (Pipeline)	Eventstreams
Continuous Replication for analytics and reporting	Yes	Yes		Yes
Metadata driven ELT/ETL for data warehousing		Yes	Yes	
Data consolidation		Yes	Yes	Yes
Data migration / Data backup / Data sharing		Yes	Yes	Yes
Free of cost	Yes			
Predictable performance		Yes	Yes	Yes

Scenarios

Review these scenarios to help you choose which data movement strategy works best for your needs.

Scenario 1

James is a finance manager at an insurance company. His team uses Azure SQL Database to track policy data, claims, and customer information across multiple business units. The executive team wants to create real-time dashboards for business performance monitoring, but James can't allow analytics queries to slow down the operational systems that process thousands of daily transactions.

James already has ETL processing in place, and his team needs the processed, gold-tier data available in Fabric for executive reporting. He doesn't want to manage scheduling, configure incremental loads, or worry about table selection - he needs everything mirrored automatically. Since this is for reporting only, having the data in a read-only format in OneLake works perfectly. The solution also needs to be cost-effective since it's coming out of his department budget.

James chooses **Mirroring**. Mirroring provides the CDC-based continuous replication he needs, automatically handling all tables without any configuration. The simple setup means he doesn't need technical expertise, and the free cost fits his budget. The read-only tabular format in OneLake gives his team the analytics access they need without affecting operational performance.

Scenario 2

Lisa is a business analyst at a logistics company. She needs to ingest raw shipment data from multiple Snowflake databases into Fabric Lakehouse tables for supply chain analysis. The data includes both historical records for the initial load and new shipments that arrive throughout the day. Lisa wants to run this process on a custom schedule - every 4 hours during business hours.

Since Lisa is bringing in bronze-tier raw data, she knows she'll quickly need transformations, schema mapping, and scheduling control. She needs to select specific tables from each Snowflake instance, map columns to standardized names, and use upsert behavior to handle updates to existing shipment records. She also wants advanced monitoring to track data quality and processing performance.

Lisa selects **Copy job**. Copy job provides the custom scheduling she needs, supports all data sources including Snowflake, and offers the table and column management capabilities for her multi-region setup. The native support for incremental copy with watermark-based detection and upsert behavior lets her handle these requirements without building pipelines.

Scenario 3

David is a senior data engineer at a telecommunications company. He's building a complex data ingestion workflow that needs to extract customer usage data from Oracle using custom SQL queries, apply business transformations, and load it into multiple destinations including both Fabric Warehouse and external systems. The workflow also needs to coordinate with other pipeline activities like data validation and notification steps.

David needs full control over the copy process, including the ability to use user-defined queries to join tables and filter data at the source. He needs advanced and fully customizable configuration options, predictable performance for large data volumes, and the ability to integrate the copy process into broader pipeline orchestration workflows with dependencies and error handling.

David reviews the available options and chooses **Copy Activities in Pipelines**. This approach gives him the advanced and fully customizable configuration he needs, supports user-defined queries for complex data extraction, and provides the pipeline-based orchestration required for his workflow. The advanced monitoring and auditing capabilities help him track the complex process, while the pipeline framework lets him coordinate copy activities with other data processing steps.

Scenario 4

Ash is a product manager at a telecom company. Her team needs to monitor customer support metrics like call volumes, wait times, and agent performance, in real time to ensure SLA compliance and improve customer satisfaction. The data comes from multiple operational systems including CRM platforms, call center logs, and agent assignment databases, and arrives at high frequency throughout the day.

Ash uses **Fabric Eventstreams** to ingest and transform this data in motion. She configures streaming connectors to pull data from various sources, applies transformations using the no-code experience, and routes the processed events to **Eventhouse** for real-time analytics. She integrates **Data Activator** to trigger alerts and automated workflows when SLA thresholds are breached so she can send notifications to supervisors or adjusting staffing levels dynamically.

The result is a real-time dashboard that updates within seconds, giving Ash's team visibility into live performance metrics and enabling fast, data-driven decisions. This streaming architecture eliminates the latency of batch pipelines and empowers the business to respond instantly to customer needs.

Get started

Now that you have an idea of which data movement strategy to use, you can get started with these resources:

- Get started with Mirroring
- Get started with Eventstreams
- Create a Copy Job
- Create a Copy Activity

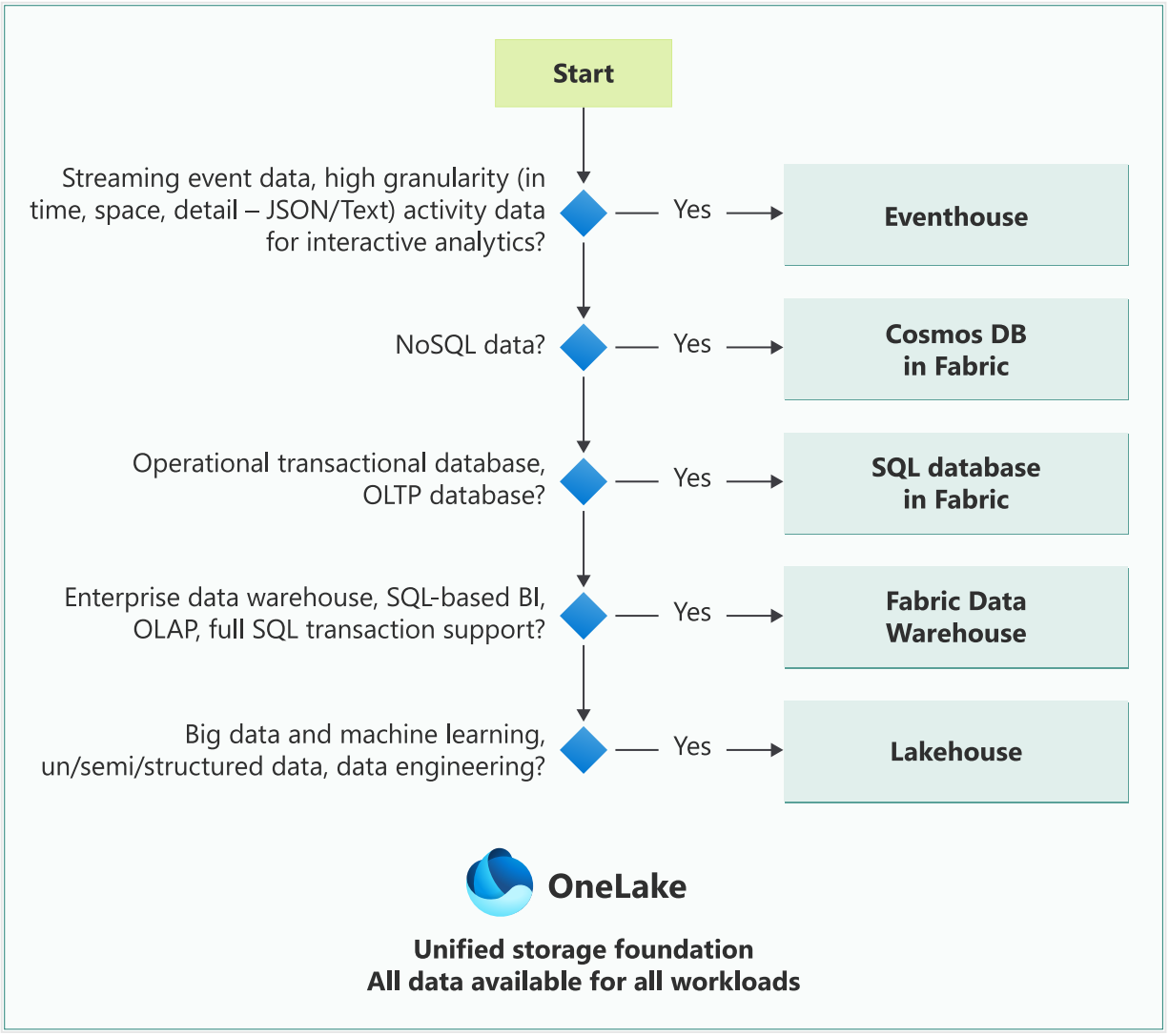
ⓘ **Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 04/22/2026

Microsoft Fabric decision guide: choose a data store

Use this reference guide and the example scenarios to help you choose a data store for your Microsoft Fabric workloads. All data stores are available in unified storage in the OneLake.



Expand table

Ideal use case	Microsoft Fabric workload	Data available in OneLake in open table format by default
Streaming event data, high granularity (in time, space, detail – JSON/Text) activity data for interactive analytics	Eventhouse	Available as opt-in
NoSQL database	Cosmos DB in Fabric	Yes

Ideal use case	Microsoft Fabric workload	Data available in OneLake in open table format by default
Operational transactional, OLTP, or normalized database	SQL database in Fabric	Yes
Develop AI with vector data types	SQL database in Fabric or Cosmos DB in Fabric	Yes
Enterprise data warehouse, SQL-based BI, OLAP, full SQL transaction support, and AI functions (preview)	Data Warehouse	Yes
Big data and machine learning, unstructured, semi-structured, or structured data, data engineering	Lakehouse	Yes

- For streaming event data and high-granularity interactive analytics, use an eventhouse.
- For NoSQL databases, use Cosmos DB in Fabric.
- For operational, transactional, OLTP workloads, or normalized relational databases, use SQL database in Fabric.
- To develop AI with vector data types, use SQL database in Fabric or Cosmos DB in Fabric.
- For enterprise data warehousing, SQL-based BI, OLAP, and full SQL transaction support, use Fabric Data Warehouse.
- For big data and machine learning with unstructured, semi-structured, or structured data, and data engineering, use a lakehouse.
- All Fabric data stores are available in OneLake in open table format by default, with the exception of a KQL database in an eventhouse, where OneLake availability is available as an opt-in feature.

Personas and skillsets

Expand table

Microsoft Fabric workload	Primary developer personas	Primary skillsets, tooling	Primary languages
Eventhouse	App developer, Data scientist, Data engineer	No code, KQL, SQL	KQL (Kusto query language), T-SQL
Cosmos DB in Fabric	AI developer, App developer	NoSQL concepts, REST APIs, similar to Azure Cosmos DB	REST API integration via JavaScript/TypeScript, Python,

Microsoft Fabric workload	Primary developer personas	Primary skillsets, tooling	Primary languages
			C#, Java, and others
SQL database in Fabric	AI developer, App developer, Database developer, Database admin	Database administration and development, similar to Azure SQL Database, SSMS, VS Code, and SQL Server-compatible query tools	T-SQL
Fabric Data Warehouse	Data warehouse developer, Data architect, Data engineer, Database developer	Data warehousing concepts, star schema database design, SSMS, VS Code, and SQL Server-compatible query tools	T-SQL, No code
Lakehouse	Data engineer, Data scientist	PySpark, Delta Lake, notebooks	Spark (Scala, PySpark, Spark SQL, R)

Scenarios

Review these scenarios for help with choosing a data store in Fabric.

Scenario 1

Susan, a professional developer, is new to Microsoft Fabric. They're ready to get started cleaning, modeling, and analyzing data but need to decide whether to build a data warehouse or a lakehouse. After reviewing the details in the previous table, the primary decision points are the available skill set and the need for multi-table transactions.

Susan spent many years building data warehouses on relational database engines, and is familiar with SQL syntax and functionality. Thinking about the larger team, the primary consumers of this data are also skilled with SQL and SQL analytical tools. Susan decides to use a **Fabric warehouse**, which allows the team to interact primarily with T-SQL, while also allowing any Spark users in the organization to access the data.

Susan creates a new data warehouse and interacts with it using T-SQL just like her other SQL server databases. Most of the existing T-SQL code she wrote to build her warehouse on SQL Server works in Fabric Data Warehouse, making the transition easy. If she chooses to, she can even use the same tools that work with her other databases, like SQL Server Management Studio. By using the SQL editor in the Fabric portal, Susan and other team members write analytic

queries that reference other data warehouses and Delta tables in lakehouses simply by using three-part names to perform cross-database queries.

Scenario 2

Rob, a data engineer, needs to store and model several terabytes of data in Fabric. The team has a mix of PySpark and T-SQL skills. Most of the team running T-SQL queries are consumers, and therefore don't need to write `INSERT`, `UPDATE`, or `DELETE` statements. The remaining developers are comfortable working in notebooks, and because the data is stored in Delta, they're able to interact with a similar SQL syntax.

Rob decides to use a **lakehouse**, which allows the data engineering team to use their diverse skills against the data, while allowing the team members who are highly skilled in T-SQL to consume the data.

Scenario 3

Daisy is business analyst experienced with using Power BI to analyze supply chain bottlenecks for a large global retail chain. They need to build a scalable data solution that can handle billions of rows of data and can be used to build dashboards and reports that can be used to make business decisions. The data comes from plants, suppliers, shippers, and other sources in various structured, semi-structured, and unstructured formats.

Daisy decides to use an **Eventhouse** because of its scalability, quick response times, advanced analytics capabilities including time series analysis, geospatial functions, and fast direct query mode in Power BI. She can execute queries by using Power BI and KQL to compare between current and previous periods, quickly identify emerging problems, or provide geo-spatial analytics of land and maritime routes.

Scenario 4

Kirby is an application architect experienced in developing .NET applications for operational data. They need a high concurrency database with full ACID transaction compliance and strongly enforced foreign keys for relational integrity. Kirby wants the benefit of automatic performance tuning to simplify day-to-day database management.

Kirby decides on a **SQL database in Fabric**, with the same SQL Database Engine as Azure SQL Database. SQL databases in Fabric automatically scale to meet demand throughout the business day. They have the full capability of transactional tables and the flexibility of transaction isolation levels from serializable to read committed snapshot. SQL database in Fabric automatically creates

and drops nonclustered indexes based on strong signals from execution plans observed over time.

In Kirby's scenario, data from the operational application must be joined with other data in Fabric: in Spark, in a warehouse, and from real-time events in an Eventhouse. Every Fabric database includes a SQL analytics endpoint, so data to be accessed in real time from Spark or with Power BI queries using DirectLake mode. These reporting solutions spare the primary operational database from the overhead of analytical workloads, and avoid denormalization. Kirby also has existing operational data in other SQL databases, and needs to import that data without transformation. To import existing operational data without any data type conversion, Kirby designs pipelines with Fabric Data Factory to import data into the Fabric SQL database.

Next step

Learn about Fabric migration

Last updated on 09/16/2026

Microsoft Fabric decision guide: Choose between Warehouse and Lakehouse

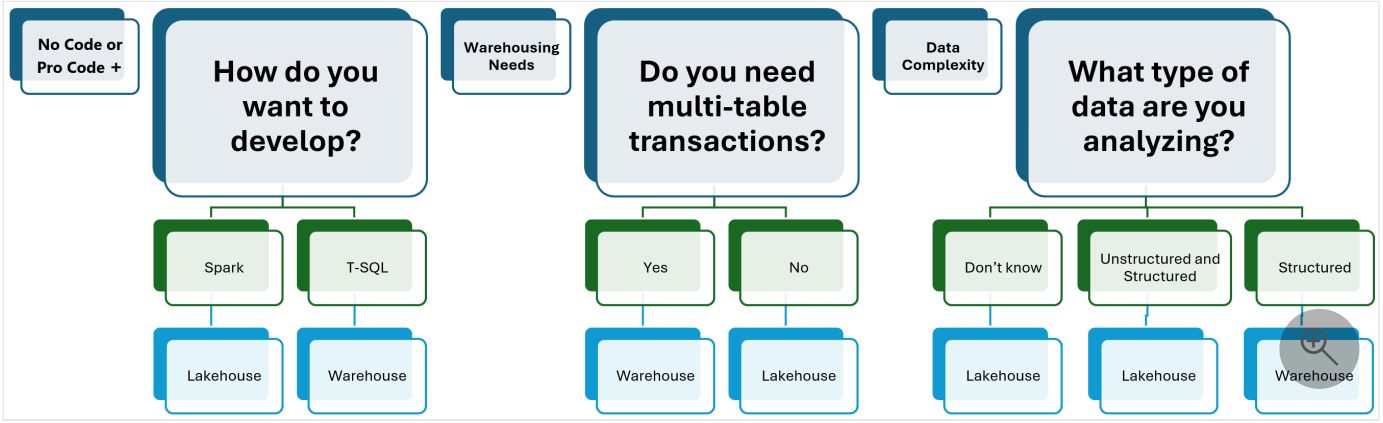
Microsoft Fabric offers two enterprise-scale, open-format data stores for analytics: Warehouse and Lakehouse. Both store data in Delta Lake format in OneLake and share the same SQL engine, but they suit different development styles, data types, and workloads. This article compares them and describes the decision points for each.

This article compares the following options:

- Warehouse
- Lakehouse, including its SQL analytics endpoint

Choose a candidate service

This section helps you select the most likely service for your needs.



Development approach: How do you want to develop?

- Apache Spark (Python, Scala, Spark SQL, or R): Use **Lakehouse**.
- T-SQL: Use **Warehouse**.

Transactions: Do you need multi-table transactions?

- Yes: Use **Warehouse**.
- No: Use **Lakehouse**.

Data type: What type of data are you analyzing?

- Unstructured and structured data, or you're not sure: Use **Lakehouse**.

- Structured data only: Use **Warehouse**.

The result of this section is a starting point. Use the following sections to evaluate each service in detail.

Evaluate each service

Evaluate each service in detail to confirm it meets your needs.

The **Warehouse** item in Fabric Data Warehouse is an enterprise-scale relational warehouse on a data lake foundation, developed primarily with T-SQL.

- Stores data in OneLake in open Delta format, so data engineers and business users can share and collaborate without compromising governance.
- Provides full multi-table ACID transactions, views, functions, and stored procedures through the SQL engine.
- Applies autonomous workload management with no compute or storage configuration to tune, and scales storage and compute independently.
- Runs cross-database queries across warehouses and lakehouses with zero data duplication.
- Ingests, loads, and transforms data at scale through the `COPY INTO` command, pipelines, dataflows, and cross-database queries.
- Integrates with Power BI and all other Fabric workloads, and automatically replicates data to OneLake for external access.

The **Lakehouse** item in Fabric Data Engineering is a data architecture platform for storing, managing, and analyzing structured and unstructured data in a single location.

- Combines the scalability of a data lake with the querying capabilities of a warehouse, so you handle large volumes of data of all types and sizes.
- Uses Delta Lake for ACID transactions, schema enforcement, and time travel.
- Ingests data from many sources and unifies it in open Delta format, and provides live access to external data through shortcuts in OneLake without copying.
- Provides automatic table discovery and registration for a fully managed file-to-table experience for data engineers and data scientists.
- Supports both Apache Spark and SQL access, so data engineers work in notebooks while analysts query with T-SQL.
- Automatically provides a SQL analytics endpoint for read-only T-SQL querying of Delta tables in the lake.

Both Warehouse and Lakehouse are available in Fabric capacities.

Compare different warehousing capabilities

This table compares the Warehouse to the SQL analytics endpoint of the Lakehouse.

Microsoft Fabric offering	Warehouse	SQL analytics endpoint of the Lakehouse
Primary capabilities	ACID-compliant, full data warehousing with multi-table transaction support in T-SQL.	Read-only, system-generated SQL analytics endpoint for T-SQL querying and serving. Supports analytics on the Lakehouse Delta tables and the Delta Lake folders referenced through shortcuts in OneLake.
Developer profile	SQL developers or citizen developers	Data engineers or SQL developers
Data loading	T-SQL (`COPY INTO`, `INSERT`, `CREATE TABLE AS SELECT`), pipelines, dataflows	Apache Spark, pipelines, dataflows, shortcuts
Delta table support	Reads and writes Delta tables	Reads Delta tables

Storage layer

Open Delta format in OneLake

Open Delta format in OneLake

Recommended use case

- Enterprise data warehousing
- Departmental, business unit, or self-service warehousing
- Structured data analysis in T-SQL with tables, views, procedures, and functions, and advanced SQL support for BI
- Exploring and querying Delta tables from the lakehouse
- Staging and archival zone for analysis
- Medallion lakehouse architecture with bronze, silver, and gold zones
- Pairing with a warehouse for enterprise analytics

Development experience

- Warehouse editor with full support for T-SQL data ingestion, modeling, development, and querying
- Read/write support for first-party and third-party tooling
- Lakehouse SQL analytics endpoint with T-SQL support for views, table-valued functions, and queries
- UI experiences for modeling and querying
- Limited T-SQL support for first-party and third-party tooling

---

T-SQL capabilities

Full DQL, DML, and DDL T-SQL support with full transaction support

Full DQL, no DML, and limited DDL such as SQL views and table-valued functions

---

Related content

- Microsoft Fabric decision guide: choose a data store
- What is Fabric Data Warehouse?
- What is a lakehouse in Microsoft Fabric?
- Delta Lake in Microsoft Fabric overview

ⓘ**Note:** The author created this article with assistance from AI. Learn more

---

Last updated on 09/17/2026

Microsoft Fabric adoption roadmap

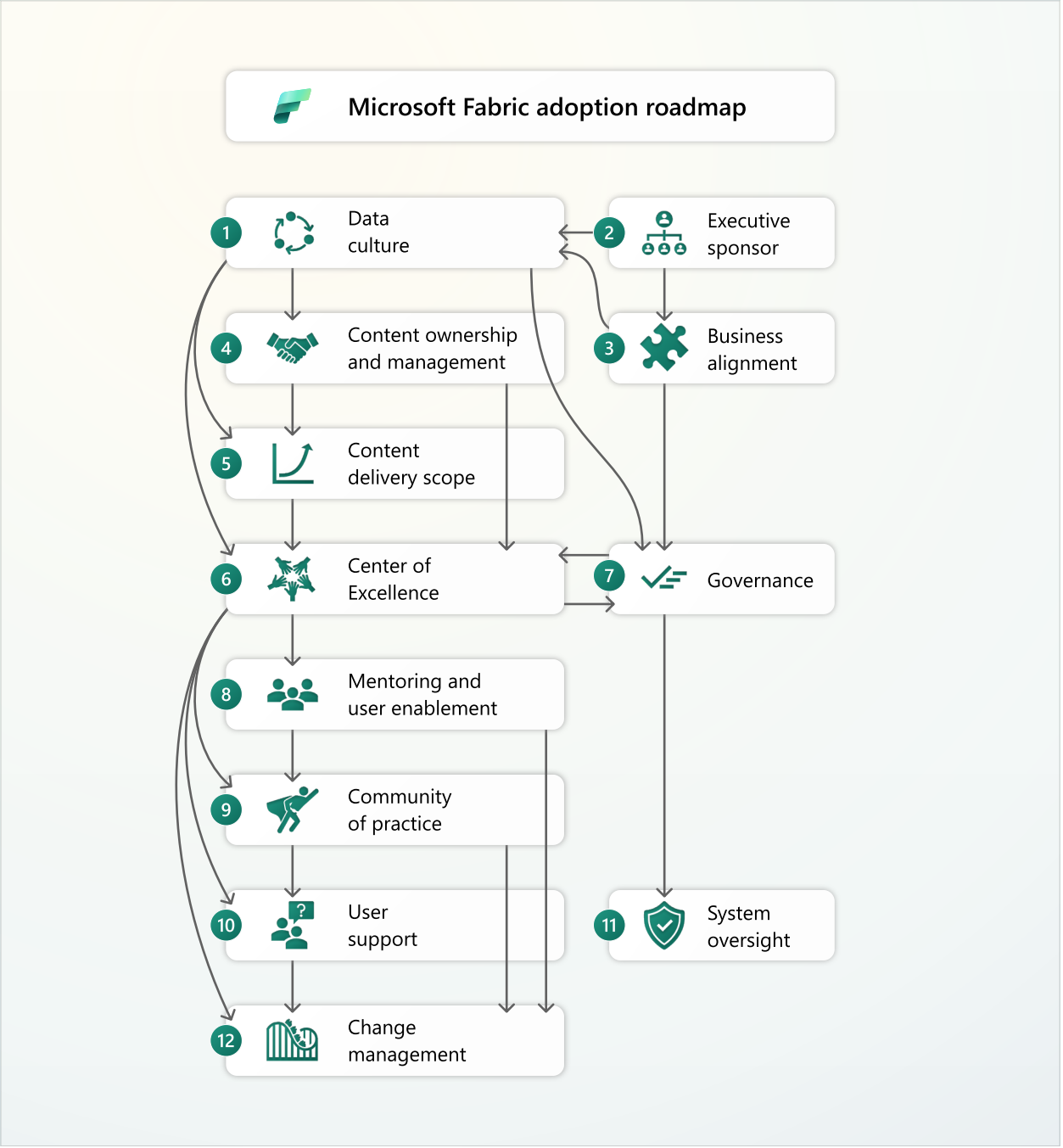
Article • 12/30/2024

The goal of this series of articles is to provide a roadmap. The roadmap presents a series of strategic and tactical considerations and action items that lead to the successful adoption of Microsoft Fabric, and help build a data culture in your organization.

Advancing adoption and cultivating a data culture is about more than implementing technology features. Technology can assist an organization in making the greatest impact, but a healthy data culture involves many considerations across the spectrum of people, processes, and technology.

> **Note**
>
> While reading this series of articles, we recommended that you also take into consideration **Power BI implementation planning** guidance. After you're familiar with the concepts in the Microsoft Fabric adoption roadmap, consider reviewing the **usage scenarios**. Understanding the diverse ways Power BI is used can influence your implementation strategies and decisions for all of Microsoft Fabric.

The diagram depicts the following areas of the Microsoft Fabric adoption roadmap.



The areas in the above diagram include:

Expand table

Area	Description
1	**Data culture**: Data culture refers to a set of behaviors and norms in the organization that encourages a data-driven culture. Building a data culture is closely related to adopting Fabric, and it's often a key aspect of an organization's digital transformation.
2	**Executive sponsor**: An executive sponsor is someone with credibility, influence, and authority throughout the organization. They advocate for building a data culture and adopting Fabric.
3	**Business Alignment**: How well the data culture and data strategy enable business users to achieve business objectives. An effective BI data strategy aligns with the business strategy.

Area	Description
4	**Content ownership and management**: There are three primary strategies for how business intelligence (BI) and analytics content is owned and managed: business-led self-service BI, managed self-service BI, and enterprise BI. These strategies have a significant influence on adoption, governance, and the Center of Excellence (COE) operating model.
5	**Content delivery scope**: There are four primary strategies for content and data delivery: personal, team, departmental, and enterprise. These strategies have a significant influence on adoption, governance, and the COE operating model.
6	**Center of Excellence**: A Fabric COE is an internal team of technical and business experts. These experts actively assist others who are working with data within the organization. The COE forms the nucleus of the broader community to advance adoption goals that are aligned with the data culture vision.
7	**Governance**: Data governance is a set of policies and procedures that define the ways in which an organization wants data to be used. When adopting Fabric, the goal of governance is to empower the internal user community to the greatest extent possible, while adhering to industry, governmental, and contractual requirements and regulations.
8	**Mentoring and user enablement**: A critical objective for adoption efforts is to enable users to accomplish as much as they can within the guardrails established by governance guidelines and policies. The act of mentoring users is one of the most important responsibilities of the COE. It has a direct influence on adoption efforts.
9	**Community of practice**: A community of practice comprises a group of people with a common interest, who interact with and help each other on a voluntary basis. An active community is an indicator of a healthy data culture. It can significantly advance adoption efforts.
10	**User support**: User support includes both informally organized and formally organized methods of resolving issues and answering questions. Both formal and informal support methods are critical for adoption.
11	**System oversight**: System oversight includes the day-to-day administration responsibilities to support the internal processes, tools, and people.
12	**Change management**: Change management involves procedures to address the impact of change for people in an organization. These procedures safeguard against disruption and productivity loss due to changes in solutions or processes. An effective data strategy describes who is responsible for managing this change and the practices and resources needed to realize it.

The relationships in the above diagram can be summarized as follows.

- Your organizational **data culture** vision will strongly influence the strategies that you follow for self-service and enterprise **content ownership and management** and **content delivery scope**.

- These strategies will, in turn, have a big impact on the operating model for your **Center of Excellence** and governance decisions.
- The established **governance** guidelines, policies, and processes affect the implementation methods used for **mentoring and enablement**, the **community of practice**, and **user support**.
- Governance decisions will dictate the day-to-day **system oversight** (administration) activities.
- Adoption and governance decisions are implemented alongside **change management** to mitigate the impact and disruption of change on existing business processes.
- All data culture and adoption-related decisions and actions are accomplished more easily with guidance and leadership from an **executive sponsor**, who facilitates **business alignment** between the business strategy and data strategy. This alignment in turn informs data culture and governance decisions.

Each individual article in this series discusses key topics associated with the items in the diagram. Considerations and potential action items are provided. Each article concludes with a set of maturity levels to help you assess your current state so you can decide what action to take next.

Microsoft Fabric adoption

Successful adoption of analytical tools like Fabric involves making effective processes, support, tools, and data available and integrated into regular ongoing patterns of usage for content creators, consumers, and stakeholders in the organization.

> **ⓘ Important**
>
> This series of adoption articles is focused on *organizational* adoption. See Microsoft Fabric adoption maturity levels for an introduction to the three types of adoption: organizational, user, and solution.

A common misconception is that adoption relates primarily to usage or the number of users. There's no question that usage statistics are an important factor. However, usage isn't the only factor. Adoption isn't just about *using* the technology regularly; it's about using it *effectively*. Effectiveness is much more difficult to define and measure.

Whenever possible, adoption efforts should be aligned across analytics platforms and BI services.

> **Note**
>
> Individuals—and the organization itself—are continually learning, changing, and improving. That means there's no formal end to adoption-related efforts.

The remaining articles in this Power BI adoption series discuss the following aspects of adoption.

- Adoption maturity levels
- Data culture
- Executive sponsorship
- Business alignment
- Content ownership and management
- Content delivery scope
- Center of Excellence
- Governance
- Mentoring and enablement
- Community of practice
- User support
- System oversight
- Change management
- Conclusion and additional resources

> **Important**
>
> You might be wondering how this Fabric adoption roadmap is different from the **Power BI adoption framework**. The adoption framework was created primarily to support Microsoft partners. It's a lightweight set of resources to help partners deploy Power BI solutions for their customers.
>
> This adoption series is more current. It's intended to guide any person or organization that is using—or considering using—Fabric. If you're seeking to improve your existing Power BI of Fabric implementation, or planning a new Power BI or Fabric implementation, this adoption roadmap is a great place to start.

Target audience

The intended audience of this series of articles is interested in one or more of the following outcomes.

- Improving their organization's ability to effectively use analytics.
- Increasing their organization's maturity level related to the delivery of analytics.
- Understanding and overcoming adoption-related challenges faced when scaling and growing.
- Increasing their organization's return on investment (ROI) in data and analytics.

This series of articles will be most helpful to those who work in an organization with one or more of the following characteristics.

- Power BI or other Fabric workloads are deployed with some successes.
- There are pockets of viral adoption, but analytics isn't being purposefully governed across the entire organization.
- Analytics solutions are deployed with some meaningful scale, but there remains a need to determine:
  - What is effective and what should be maintained.
  - What should be improved.
  - How future deployments could be more strategic.
- An expanded implementation of analytics is under consideration or is planned.

This series of articles will also be helpful for:

- Organizations that are in the early stages of an analytics implementation.
- Organizations that have had success with adoption and now want to evaluate their current maturity level.

Assumptions and scope

The primary focus of this series of articles is on the Microsoft Fabric platform.

To fully benefit from the information provided in these articles, you should have an understanding of Power BI foundational concepts and Fabric foundational concepts.

Related content

In the next article in this series, learn about the Fabric adoption maturity levels. The maturity levels are referenced throughout the entire series of articles. Also, see the conclusion article for additional adoption-related resources.

Other helpful resources include:

- Power BI implementation planning
- Questions? Try asking the Fabric Community

- Suggestions? Contribute ideas to improve Fabric

Experienced partners are available to help your organization succeed with adoption initiatives. To engage with a partner, visit the Power BI partner portal.

Acknowledgments

The Microsoft Fabric adoption roadmap articles are written by Melissa Coates, Kurt Buhler, and Peter Myers. Matthew Roche, from the Fabric Customer Advisory Team, provides strategic guidance and feedback to the subject matter experts. Reviewers include Cory Moore, James Ward, Timothy Bindas, Greg Moir, and Chuy Varela.

Feedback

Was this page helpful? Yes No

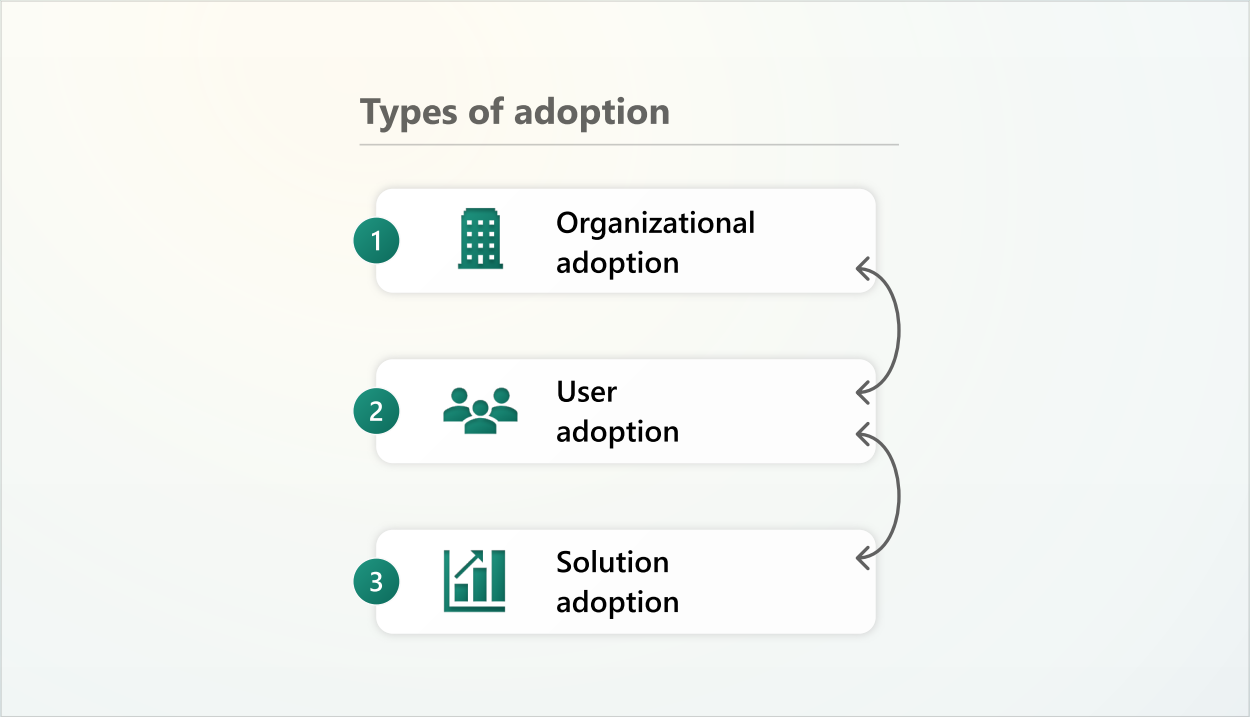
Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap maturity levels

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

There are three inter-related perspectives to consider when adopting an analytics technology like Microsoft Fabric.



The three types of adoption depicted in the above diagram include:

Expand table

Type	Description
1	**Organizational adoption** refers to the effectiveness of your analytics governance processes. It also refers to data management practices that support and enable analytics and business intelligence (BI) efforts.
2	**User adoption** is the extent to which consumers and creators continually increase their knowledge. It's concerned with whether they're actively using analytics tools, and whether they're using them in the most effective way.

Type	Description
3	**Solution adoption** refers to the impact and business value achieved for individual requirements and analytical solutions.

As the four arrows in the previous diagram indicate, the three types of adoption are all strongly inter-related:

- **Solution adoption affects user adoption.** A well-designed and well-managed solution—which could be many things, such as a set of reports, a Power BI app, a semantic model, or a Fabric lakehouse—impacts and guides users on how to use analytics in an optimal way.
- **User adoption impacts organizational adoption.** The patterns and practices used by individual users influence organizational adoption decisions, policies, and practices.
- **Organizational adoption influences user adoption.** Effective organizational practices—including mentoring, training, support, and community—encourage users to do the right thing in their day-to-day workflow.
- **User adoption affects solution adoption.** Stronger user adoption, because of the effective use of analytics by educated and informed users, contributes to stronger and more successful individual solutions.

The remainder of this article introduces the three types of adoption in more detail.

Organizational adoption maturity levels

Organizational adoption measures the state of analytics governance and data management practices. There are several organizational adoption goals:

- Effectively support the community of creators, consumers, and stakeholders
- Enable and empower users
- Right-sized governance of analytics, BI, and data management activities
- Oversee information delivery via enterprise BI and self-service BI with continuous improvement cycles

It's helpful to think about organizational adoption from the perspective of a maturity model. For consistency with the Power CAT adoption maturity model and the maturity model for Microsoft 365, this Microsoft Fabric adoption roadmap aligns with the five levels from the Capability Maturity Model, which were later enhanced by the Data Management Maturity (DMM) model from ISACA (note that the DMM was a paid resource that has since been retired).

Every organization has limited time, funding, and people. So, it requires them to be selective about where they prioritize their efforts. To get the most from your investment in analytics, seek to attain at least maturity level 300 or 400, as discussed below. It's common that different business units in the organization evolve and mature at different rates, so be conscious of the organizational state as well as progress for key business units.

> **Note**
>
> Organizational adoption maturity is a long journey. It takes time, effort, and planning to progress to the higher levels.

Maturity level 100 – Initial

Level 100 is referred to as *initial* or *performed*. It's the starting point for new data-related investments that are new, undocumented, and without any process discipline.

Common characteristics of maturity level 100 include:

- Pockets of success and experimentation with Fabric exist in one or more areas of the organization.
- Achieving quick wins has been a priority, and solutions have been delivered with some success.
- Organic growth has led to the lack of a coordinated strategy or governance approach.
- Practices are undocumented, with significant reliance on tribal knowledge.
- There are few formal processes in place for effective data management.
- Risk exists due to a lack of awareness of how data is used throughout the organization.
- The potential for a strategic investment with analytics is acknowledged. However, there's no clear path forward for purposeful, organization-wide execution.

Maturity level 200 – Repeatable

Level 200 is referred to as *repeatable* or *managed*. At this point on the maturity curve, data management is planned and executed. Defined processes exist, though these processes might not apply uniformly throughout the organization.

Common characteristics of maturity level 200 include:

- Certain analytics content is now critical in importance and/or it's broadly used by the organization.
- There are attempts to document and define repeatable practices. These efforts are siloed, reactive, and deliver varying levels of success.
- There's an over-reliance on individuals having good judgment and adopting healthy habits that they learned on their own.
- Analytics adoption continues to grow organically and produces value. However, it takes place in an uncontrolled way.
- Resources for an internal community are established, such as a Teams channel or Yammer group.
- Initial planning for a consistent analytics governance strategy is underway.
- There's recognition that a Center of Excellence (COE) can deliver value.

Maturity level 300 – Defined

Level 300 is referred to as *defined*. At this point on the maturity curve, a set of standardized data management processes are established and consistently applied across organizational boundaries.

Common characteristics of maturity level 300 include:

- Measurable success is achieved for the effective use of analytics.
- Progress is made on the standardization of repeatable practices. However, less-than-optimal aspects could still exist due to early uncontrolled growth.
- The COE is established. It has clear goals and scope of responsibilities.
- The internal community of practice gains traction with the participation of a growing number of users.
- Champions emerge in the internal user community.
- Initial investments in training, documentation, and resources (such as template files) are made.
- An initial governance model is in place.
- There's an active and engaged executive sponsor.
- Roles and responsibilities for all analytics stakeholders are well understood.

Maturity level 400 – Capable

Level 400 is known as *capable* or *measured*. At this point on the maturity curve, data is well-managed across its entire lifecycle.

Common characteristics of maturity level 400 include:

- Analytics and business intelligence efforts deliver significant value.

- Approved tools are commonly used for delivering critical content throughout the organization.
- There's an established and accepted governance model with cooperation from all key business units.
- Training, documentation, and resources are readily available for, and actively used by, the internal community of users.
- Standardized processes are in place for the oversight and monitoring of analytics usage and practices.
- The COE includes representation from all key business units.
- A champions network supports the internal community. The champions actively work with their colleagues as well as the COE.

Maturity level 500 – Efficient

Level 500 is known as *efficient* or *optimizing* because at this point on the maturity curve, the emphasis is now on automation and continuous improvement.

Common characteristics of maturity level 500 include:

- The value of analytics solutions is prevalent in the organization. Fabric is widely accepted throughout the organization.
- Analytics skillsets are highly valued in the organization, and they're recognized by leadership.
- The internal user community is self-sustaining, with support from the COE. The community isn't over-reliant on key individuals.
- The COE reviews key performance indicators regularly to measure success of implementation and adoption goals.
- Continuous improvement is a continual priority.
- Use of automation adds value, improves productivity, or reduces risk for error.

> **Note**
>
> The characteristics above are generalized. When considering maturity levels and designing a plan, you'll want to consider each topic or goal independently. In reality, it's probably not possible to reach level 500 maturity level for every aspect of Fabric adoption for the entire organization. So, assess maturity levels independently per goal. That way, you can prioritize your efforts where they will deliver the most value. The remainder of the articles in this Fabric adoption series present maturity levels on a per-topic basis.

Individuals—and the organization itself—continually learn, change, and improve. That means there's no formal end to adoption-related efforts. However, it's common that effort is reduced as higher maturity levels are reached.

The remainder of this article introduces the second and third types of adoption: user adoption and solution adoption.

> **Note**
>
> The remaining articles in this series focus primarily on organizational adoption.

User adoption stages

User adoption measures the extent to which content consumers and self-service content creators are actively and effectively using analytics tools such as Fabric. Usage statistics alone don't indicate successful user adoption. User adoption is also concerned with individual user behaviors and practices. The aim is to ensure users engage with solutions, tools, and processes in the correct way and to their fullest extent.

User adoption encompasses how consumers view content, as well as how self-service creators generate content for others to consume.

User adoption occurs on an individual user basis, but it's measured and analyzed in the aggregate. Individual users progress through the four stages of user adoption at their own pace. An individual who adopts a new technology will take some time to achieve proficiency. Some users will be eager; others will be reluctant to learn yet another tool, regardless of the promised productivity improvements. Advancing through the user adoption stages involves time and effort, and it involves behavioral changes to become aligned with organizational adoption objectives. The extent to which the organization supports users advancing through the user adoption stages has a direct correlation to the organizational-level adoption maturity.

User adoption stage 1 – Awareness

Common characteristics of stage 1 user adoption include:

- An individual has heard of, or been initially exposed to, analytics in some way.
- An individual might have access to a tool, such as Fabric, but isn't yet actively using it.

User adoption stage 2 – Understanding

Common characteristics of stage 2 user adoption include:

- An individual develops understanding of the benefits of analytics and how it can support decision-making.
- An individual shows interest and starts to use analytics tools.

User adoption stage 3 – Momentum

Common characteristics of stage 3 user adoption include:

- An individual actively gains analytics skills by attending formal training, self-directed learning, or experimentation.
- An individual gains basic competency by using or creating analytics relevant to their role.

User adoption stage 4 – Proficiency

Common characteristics of stage 4 user adoption include:

- An individual actively uses analytics regularly.
- An individual understands how to use analytic tools in the way in which they were intended, as relevant for their role.
- An individual modifies their behavior and activities to align with organizational governance processes.
- An individual's willingness to support organizational processes and change efforts is growing over time, and they become an advocate for analytics in the organization.
- An individual makes the effort to continually improve their skills and stay current with new product capabilities and features.

It's easy to underestimate the effort it takes to progress from stage 2 (understanding) to stage 4 (proficiency). Typically, it takes the longest time to progress from stage 3 (momentum) to stage 4 (proficiency).

> **Important**
>
> By the time a user reaches the momentum and proficiency stages, the organization needs to be ready to support them in their efforts. You can consider some proactive efforts to encourage users to progress through stages. For more information, see the community of practice and the user support articles.

Solution adoption phases

Solution adoption is concerned with measuring the impact of content that's been deployed. It's also concerned with the level of value solutions provide. The scope for evaluating solution adoption is for one set of requirements, like a set of reports, a lakehouse, or a single Power BI app.

> **Note**
>
> In this series of articles, *content* is synonymous with *solution*.

As a solution progresses to phases 3 or 4, expectations to operationalize the solution are higher.

> **Tip**
>
> The importance of scope on expectations for governance is described in the **content delivery scope** article. That concept is closely related to this topic, but this article approaches it from a different angle. It considers when you already have a solution that is operationalized and distributed to many users. That doesn't immediately equate to phase 4 solution adoption, as the concept of solution adoption focuses on how much value the content delivers.

Solution phase 1 – Exploration

Common characteristics of phase 1 solution adoption include:

- Exploration and experimentation are the main approaches to testing out new ideas. Exploration of new ideas can occur through informal self-service efforts, or through a formal proof of concept (POC), which is purposely narrow in scope. The goal is to confirm requirements, validate assumptions, address unknowns, and mitigate risks.
- A small group of users test the proof of concept solution and provide useful feedback.
- For simplicity, all exploration—and initial feedback—could occur within local user tools (such as Power BI Desktop or Excel) or within a single Fabric workspace.

Solution phase 2 – Functional

Common characteristics of phase 2 solution adoption include:

- The solution is functional and meets the basic set of user requirements. There are likely plans to iterate on improvements and enhancements.
- The solution is deployed to the Fabric portal.
- All necessary supporting components are in place (for example, a gateway to support scheduled data refresh).
- Target users are aware of the solution and show interest in using it. Potentially, it could be a limited preview release, and might not yet be ready to promote to a production workspace.

Solution phase 3 – Valuable

Common characteristics of phase 3 solution adoption include:

- Target users find the solution to be valuable and experience tangible benefits.
- The solution is promoted to a production workspace that's managed, secured, and audited.
- Validations and testing occur to ensure data quality, accurate presentation, accessibility, and acceptable performance.
- Content is endorsed, when appropriate.
- Usage metrics for the solution are actively monitored.
- User feedback loops are in place to facilitate suggestions and improvements that can contribute to future releases.
- Solution documentation is generated to support the needs of information consumers (such as data sources used or how metrics are calculated). The documentation helps future content creators (for example, for documenting any future maintenance or planned enhancements).
- Ownership and subject matter experts for the content are clear.
- Report branding and theming are in place and in line with governance guidelines.

Solution phase 4 – Essential

Common characteristics of phase 4 solution adoption include:

- Target users actively and routinely use the solution, and it's considered essential for decision-making purposes.
- The solution resides in a production workspace well separated from development and test content. Change management and release management are carefully controlled due to the impact of changes.
- A subset of users regularly provides feedback to ensure the solution continues to meet evolving requirements.
- Expectations for the success of the solution are clear and are measured.

- Expectations for support of the solution are clear, especially if there are service level agreements.
- The solution aligns with organizational governance guidelines and practices.
- Most content is certified due to its critical nature.
- Formal user acceptance testing for new changes might occur, particularly for IT-managed content.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about the organizational data culture and its impact on adoption efforts.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Data culture

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

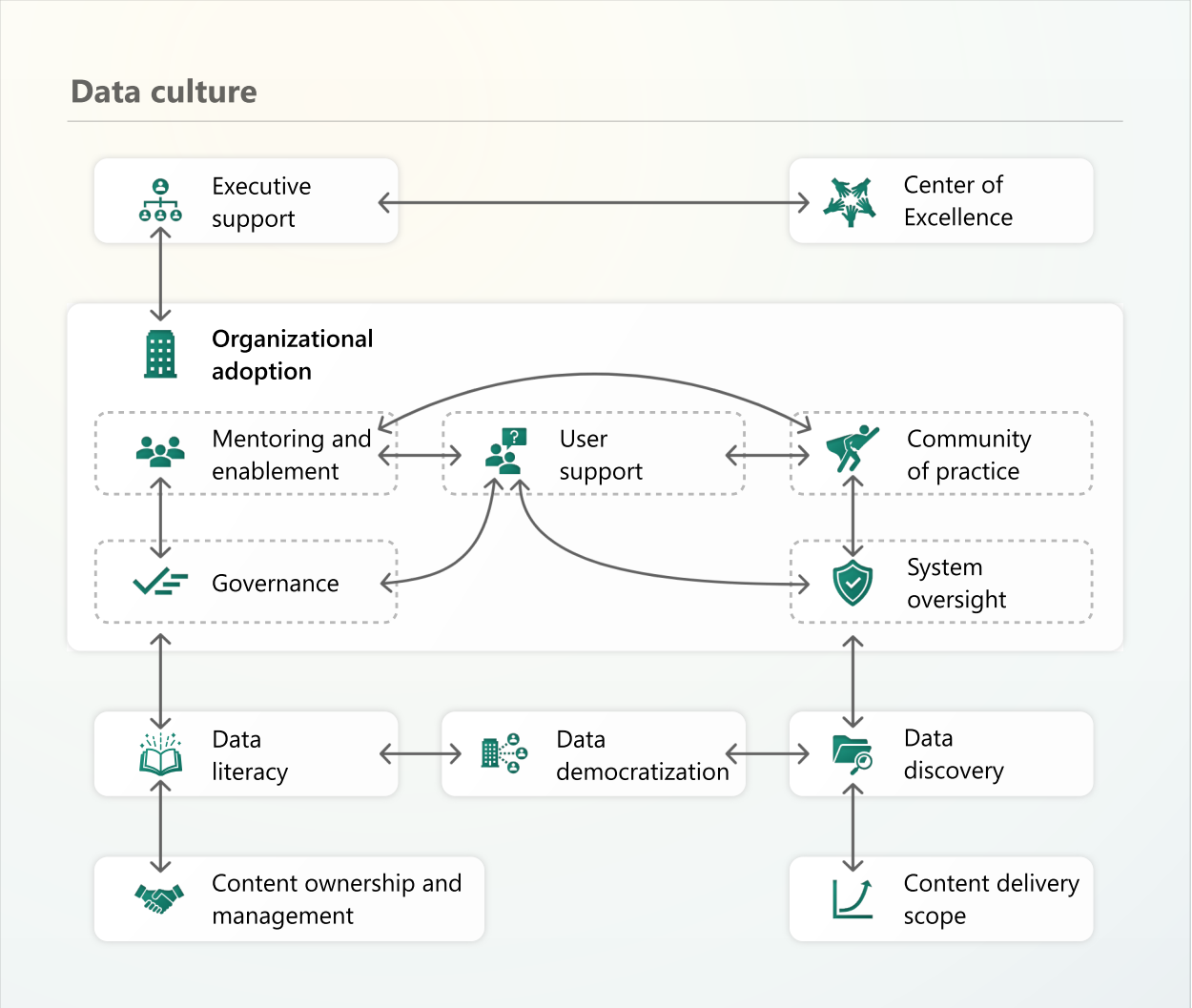
Building a data culture is closely related to adopting analytics, and it's often a key aspect of an organization's digital transformation. The term *data culture* can be defined in different ways by different organizations. In this series of articles, data culture means a set of behaviors and norms in an organization. It encourages a culture that regularly employs informed data decision-making:

- By more stakeholders throughout more areas of the organization.
- Based on analytics, not opinion.
- In an effective, efficient way that's based on best practices approved by the Center of Excellence (COE).
- Based on trusted data.
- That reduces reliance on undocumented tribal knowledge.
- That reduces reliance on hunches and gut decisions.

> **Important**
>
> Think of data culture as what you do, not what you say. Your data culture is not a set of rules (that's governance). So, data culture is a somewhat abstract concept. It's the behaviors and norms that are allowed, rewarded, and encouraged—or those that are disallowed and discouraged. Bear in mind that a healthy data culture motivates employees at all levels of the organization to generate and distribute actionable knowledge.

Within an organization, certain business units or teams are likely to have their own behaviors and norms for getting things done. The specific ways to achieve data culture objectives can vary across organizational boundaries. What's important is that they should all align with the organizational data culture objectives. You can think of this structure as *aligned autonomy*.

The following circular diagram conveys the interrelated aspects that influence your data culture:



The diagram depicts the somewhat ambiguous relationships among the following items:

- Data culture is the outer circle. All topics within it contribute to the state of the data culture.
- Organizational adoption (including the implementation aspects of mentoring and user enablement, user support, community of practice, governance, and system oversight) is the inner circle. All topics are major contributors to the data culture.
- Executive support and the Center of Excellence are drivers for the success of organizational adoption.
- Data literacy, data democratization, and data discovery are data culture aspects that are heavily influenced by organizational adoption.
- Content ownership and management, and content delivery scope, are closely related to data democratization.

The elements of the diagram are discussed throughout this series of articles.

Data culture vision

The concept of data culture can be difficult to define and measure. Even though it's challenging to articulate data culture in a way that's meaningful, actionable, and measurable, you need to have a well-understood definition of what a healthy data culture means to your organization. This vision of a healthy data culture should:

- Originate from the executive level.
- Align with organizational objectives.
- Directly influence your adoption strategy.
- Serve as the high-level guiding principles for enacting governance policies and guidelines.

Data culture outcomes aren't specifically mandated. Rather, the state of the data culture is the result of following the governance rules as they're enforced (or the lack of governance rules). Leaders at all levels need to actively demonstrate through their actions what's important to them, including how they praise, recognize, and reward staff members who take initiative.

> **Tip**
>
> If you can take for granted that your efforts to develop a data solution (such as a semantic model, a lakehouse, or a report) will be valued and appreciated, that's an excellent indicator of a healthy data culture. Sometimes, however, it depends on what your immediate manager values most.

The initial motivation for establishing a data culture often comes from a specific strategic business problem or initiative. It might be:

- A reactive change, such as responding to new agile competition.
- A proactive change, such as starting a new line of business or expanding into new markets to seize a "green field" opportunity. Being data driven from the beginning can be relatively easier when there are fewer constraints and complications, compared with an established organization.
- Driven by external changes, such as pressure to eliminate inefficiencies and redundancies during an economic downturn.

In each of these situations, there's often a specific area where the data culture takes root. The specific area could be a scope of effort that's smaller than the entire organization, even if it's still significant. After necessary changes are made at this smaller scope, they can be incrementally replicated and adapted for the rest of the organization.

Although technology can help advance the goals of a data culture, implementing specific tools or features isn't the objective. This series of articles covers a lot of topics that contribute to adoption of a healthy data culture. The remainder of this article addresses three essential aspects of data culture: data discovery, data democratization, and data literacy.

Data discovery

A successful data culture depends on users working with the right data in their day-to-day activities. To achieve this goal, users need to find and access data sources, reports, and other items.

Data discovery is the ability to effectively locate relevant data assets across the organization. Primarily, data discovery is concerned with improving awareness that data exists, which can be particularly challenging when data is siloed in departmental systems.

Data discovery is a slightly different concept from *search*, because:

- Data discovery allows users to see metadata for an item, like the name of a semantic model, even if they don't currently have access to it. After a user is aware of its existence, that user can go through the standard process to request access to the item.
- Search allows users to locate an existing item when they already have security access to the item.

> **Tip**
>
> It's important to have a clear and simple process so users can request access to data. Knowing that data exists—but being unable to access it within the guidelines and processes that the domain owner has established—can be a source of frustration for users. It can force them to use inefficient workarounds instead of requesting access through the proper channels.

Data discovery contributes to adoption efforts and the implementation of governance practices by:

- Encouraging the use of trusted high-quality data sources.
- Encouraging users to take advantage of existing investments in available data assets.

- Promoting the use and enrichment of existing data items (such as a lakehouse, data warehouse, data pipeline, dataflow, or semantic model) or reporting items (such as reports, dashboards, or metrics).
- Helping people understand who owns and manages data assets.
- Establishing connections between consumers, creators, and owners.

The OneLake catalog and the use of endorsements are key ways to promote data discovery in your organization.

Furthermore, data catalog solutions are extremely valuable tools for data discovery. They can record metadata tags and descriptions to provide deeper context and meaning. For example, Microsoft Purview can scan and catalog items from a Fabric tenant (as well as many other sources).

Questions to ask about data discovery

Use questions like those found below to assess data discovery.

- Is there a data catalog where business users can search for data?
- Is there a metadata catalog that describes definitions and data locations?
- Are high-quality data sources endorsed by certifying or promoting them?
- To what extent do redundant data sources exist because people can't find the data they need? What roles are expected to create data items? What roles are expected to create reports or perform ad hoc analysis?
- Can end users find and use existing reports, or do they insist on data exports to create their own?
- Do end users know which reports to use to address specific business questions or find specific data?
- Are people using the appropriate data sources and tools, or resisting them in favor of legacy ones?
- Do analysts understand how to enrich existing certified semantic models with new data—for example, by using a Power BI composite model?
- How consistent are data items in their quality, completeness, and naming conventions?
- Can data item owners follow data lineage to perform impact analysis of data items?

Maturity levels of data discovery

The following maturity levels can help you assess your current state of data discovery.

Expand table

Level	State of Fabric data discovery
100: Initial	• Data is fragmented and disorganized, with no clear structures or processes to find it. • Users struggle to find and use data they need for their tasks.
200: Repeatable	• Scattered or organic efforts to organize and document data are underway, but only in certain teams or departments. • Content is occasionally endorsed, but these endorsements aren't defined and the process isn't managed. Data remains siloed and fragmented, and it's difficult to access.
300: Defined	• A central repository, like the OneLake catalog, is used to make data easier to find for people who need it. • An explicit process is in place to endorse quality data and content. • Basic documentation includes catalog data, definitions, and calculations, as well as where to find them.
400: Capable	• Structured, consistent processes guide users how to endorse, document, and find data from a central hub. Data silos are the exception instead of the rule. • Quality data assets are consistently endorsed and easily identified. • Comprehensive data dictionaries are maintained and improve data discovery.
500: Efficient	• Data and metadata is systematically organized and documented with a full view of the data lineage. • Quality assets are endorsed and easily identified. • Cataloging tools, like Microsoft Purview, are used to make data discoverable for both use and governance.

Data democratization

Data democratization refers to putting data into the hands of more users who are responsible for solving business problems. It's about enabling more users to make better data-driven decisions.

> **Note**
>
> The concept of data democratization doesn't imply a lack of security or a lack of justification based on job role. As part of a healthy data culture, data democratization helps reduce *shadow IT* by providing semantic models that:
>
> - Are secured, governed, and well managed.
> - Meet business needs in cost-effective and timely ways.

Your organization's position on data democratization will have a wide-reaching impact on adoption and governance-related efforts.

> **Warning**
>
> If access to data or the ability to perform analytics is limited to a select number of individuals in the organization, that's typically a warning sign because the ability to work with data is a key characteristic of a healthy data culture.

Questions to ask about data democratization

Use questions like those found below to assess data democratization.

- Is data and analytics readily accessible, or restricted to limited roles and individuals?
- Is an effective process in place for people to request access to new data and tools?
- Is data readily shared between teams and business units, or is it siloed and closely guarded?
- Who is permitted to have Power BI Desktop installed?

- Who is permitted to have Power BI Pro or Power BI Premium Per User (PPU) licenses?
- Who is permitted to create assets in Fabric workspaces?
- What's the desired level of self-service analytics and business intelligence (BI) user enablement? How does this level vary depending on business unit or job role?
- What's the desired balance between enterprise and self-service analytics, and BI?
- What data sources are strongly preferred for what topics and business domains? What's the allowed use of unsanctioned data sources?
- Who can manage content? Is this decision different for data versus reports? Is the decision different for enterprise BI users versus decentralized users? Who can own and manage self-service BI content?
- Who can consume content? Is this decision different for external partners, customers, or suppliers?

Maturity levels of data democratization

The following maturity levels can help you assess your current state of data democratization.

Expand table

Level	State of data democratization
100: Initial	• Data and analytics are limited to a small number of roles, who gatekeep access to others. • Business users must request access to data or tools to complete tasks. They struggle with delays or bottlenecks. • Self-service initiatives are taking place with some success in various areas of the organization. These activities are occurring in a somewhat chaotic manner, with few formal processes and no strategic plan. There's a lack of oversight and visibility into these self-service activities. The success or failure of each solution isn't well understood. • The enterprise data team can't keep up with the needs of the business. A significant backlog of requests exists for this team.

Level	State of data democratization
200: Repeatable	• There are limited efforts underway to expand access to data and tools. • Multiple teams have had measurable success with self-service solutions. People in the organization are starting to pay attention. • Investments are being made to identify the ideal balance of enterprise and self-service solutions.
300: Defined	• Many people have access to the data and tools they need, although not all users are equally enabled or held accountable for the content they create. • Effective self-service data practices are incrementally and purposely replicated throughout more areas of the organization.
400: Capable	• Healthy partnerships exist among enterprise and self-service solution creators. Clear, realistic user accountability and policies mitigate risk of self-service analytics and BI. • Clear and consistent processes are in place for users to request access to data and tools. • Individuals who take initiative in building valuable solutions are recognized and rewarded.
500: Efficient	• User accountability and effective governance give central teams confidence in what users do with data. • Automated, monitored processes enable people to easily request access to data and tools. Anyone with the need or interest to use data can follow these processes to perform analytics.

Data literacy

Data literacy refers to the ability to interpret, create, and communicate with data and analytics accurately and effectively.

Training efforts, as described in the mentoring and user enablement article, often focus on how to use the technology itself. Technology skills are important to producing high-quality solutions, but it's also important to consider how to purposely advance data literacy throughout the organization. Put another way, successful adoption takes a lot more than merely providing software and licenses to users.

How you go about improving data literacy in your organization depends on many factors, such as current user skillsets, complexity of the data, and the types of analytics

that are required. You might choose to focus on these types of activities related to data literacy:

- Interpreting charts and graphs
- Assessing the validity of data
- Performing root cause analysis
- Discerning correlation from causation
- Understanding how context and outliers affect how results are presented
- Using storytelling to help consumers quickly understand and act

> **Tip**
>
> If you're struggling to get data culture or governance efforts approved, focusing on tangible benefits that you can achieve with data discovery ("find the data"), data democratization ("use the data"), or data literacy ("understand the data") can help. It can also be helpful to focus on specific problems that you can solve or mitigate through data culture advancements.
>
> Getting the right stakeholders to agree on the problem is usually the first step. Then, it's a matter of getting the stakeholders to agree on the strategic approach to a solution, along with the solution details.

Questions to ask about data literacy

Use questions like those found below to assess data literacy.

- Does a common analytical vocabulary exist in the organization to talk about data and BI solutions? Alternatively, are definitions fragmented and different across silos?
- How comfortable are people with making decisions based on data and evidence compared to intuition and subjective experience?
- When people who hold an opinion are confronted with conflicting evidence, how do they react? Do they critically appraise the data, or do they dismiss it? Can they alter their opinion, or do they become entrenched and resistant?
- Do training programs exist to support people in learning about data and analytical tools?

- Is there significant resistance to visual analytics and interactive reporting in favor of static spreadsheets?
- Are people open to new analytical methods and tools to potentially address their business questions more effectively? Alternatively, do they prefer to keep using existing methods and tools to save time and energy?
- Are there methods or programs to assess or improve data literacy in the organization? Does leadership have an accurate understanding of the data literacy levels?
- Are there roles, teams, or departments where data literacy is particularly strong or weak?

Maturity levels of data literacy

The following maturity levels can help you assess your current state of data literacy.

Expand table

Level	State of data literacy
100: Initial	• Decisions are frequently made based on intuition and subjective experience. When confronted with data that challenges existing opinions, data is often dismissed. • Individuals have low confidence to use and understand data in decision-making processes or discussions. • Report consumers have a strong preference for static tables. These consumers dismiss interactive visualizations or sophisticated analytical methods as "fancy" or unnecessary.
200: Repeatable	• Some teams and individuals inconsistently incorporate data into their decision making. There are clear cases where misinterpretation of data has led to flawed decisions or wrong conclusions. • There's some resistance when data challenges pre-existing beliefs. • Some people are skeptical of interactive visualizations and sophisticated analytical methods, though their use is increasing.

Level	State of data literacy
300: Defined	• The majority of teams and individuals understand data relevant to their business area and use it implicitly to inform decisions. • When data challenges pre-existing beliefs, it produces critical discussions and sometimes motivates change. • Visualizations and advanced analytics are more widely accepted, though not always used effectively.
400: Capable	• Data literacy is recognized explicitly as a necessary skill in the organization. Some training programs address data literacy. Specific efforts are taken to help departments, teams, or individuals that have particularly weak data literacy. • Most individuals can effectively use and apply data to make objectively better decisions and take actions. • Visual and analytical best practices are documented and followed in strategically important data solutions.
500: Efficient	• Data literacy, critical thinking, and continuous learning are strategic skills and values in the organization. Effective programs monitor progress to improve data literacy in the organization. • Decision making is driven by data across the organization. Decision intelligence or prescriptive analytics are used to recommend key decisions and actions. • Visual and analytical best practices are seen as essential to generate business value with data.

Considerations and key actions

Checklist - Here are some considerations and key actions that you can take to strengthen your data culture.

- ✓ **Align your data culture goals and strategy**: Give serious consideration to the type of data culture that you want to cultivate. Ideally, it's more from a position of user empowerment than a position of command and control.
- ✓ **Understand your current state**: Talk to stakeholders in different business units to understand which analytics practices are currently working well and which practices

aren't working well for data-driven decision-making. Conduct a series of workshops to understand the current state and to formulate the desired future state.

- ✓ **Speak with stakeholders**: Talk to stakeholders in IT, BI, and the COE to understand which governance constraints need consideration. These conversations can present an opportunity to educate teams on topics like security and infrastructure. You can also use the opportunity to educate stakeholders on the features and capabilities included in Fabric.
- ✓ **Verify executive sponsorship**: Verify the level of executive sponsorship and support that you have in place to advance data culture goals.
- ✓ **Make purposeful decisions about your data strategy**: Decide what the ideal balance of business-led self-service, managed self-service, and enterprise data, analytics and BI use cases should be for the key business units in the organization (covered in the content ownership and management article). Also consider how the data strategy relates to the extent of published content for personal, team, departmental, and enterprise analytics and BI (described in the content delivery scope article). Define your high-level goals and priorities for this strategic planning. Determine how these decisions affect your tactical planning.
- ✓ **Create a tactical plan**: Begin creating a tactical plan for immediate, short-term, and long-term action items. Identify business groups and problems that represent "quick wins" and can make a visible difference.
- ✓ **Create goals and metrics**: Determine how you'll measure effectiveness for your data culture initiatives. Create key performance indicators (KPIs) or objectives and key results (OKRs) to validate the results of your efforts.

Questions to ask about data culture

Use questions like those found below to assess data culture.

- Is data regarded as a strategic asset in the organization?
- Is there a vision of a healthy data culture that originates from executive leadership and aligns with organizational objectives?
- Does the data culture guide creation of governance policies and guidelines?
- Are organizational data sources trusted by content creators and consumers?
- When justifying an opinion, decision, or choice, do people use data as evidence?
- Is knowledge about analytics and data use documented or is there a reliance on undocumented tribal knowledge?

- Are efforts to develop a data solution valued and appreciated by the user community?

Maturity levels of data culture

The following maturity levels will help you assess the current state of your data culture.

Expand table

Level	State of data culture
100: Initial	• Enterprise data teams can't keep up with the needs of the business. A significant backlog of requests exists. • Self-service data and BI initiatives are taking place with some success in various areas of the organization. These activities occur in a somewhat chaotic manner, with few formal processes and no strategic plan. • There's a lack of oversight and visibility into self-service BI activities. The successes or failures of data and BI solutions aren't well understood.
200: Repeatable	• Multiple teams have had measurable successes with self-service solutions. People in the organization are starting to pay attention. • Investments are being made to identify the ideal balance of enterprise and self-service data, analytics, and BI.
300: Defined	• Specific goals are established for advancing the data culture. These goals are implemented incrementally. • Learnings from what works in individual business units is shared. • Effective self-service practices are incrementally and purposely replicated throughout more areas of the organization.
400: Capable	• The data culture goals to employ informed decision-making are aligned with organizational objectives. They're actively supported by the executive sponsor, the COE, and they have a direct impact on adoption strategies. • A healthy and productive partnership exists between the executive sponsor, COE, business units, and IT. The teams are working towards shared goals.

Level	State of data culture
	• Individuals who take initiative in building valuable data solutions are recognized and rewarded.
500: Efficient	• The business value of data, analytics, and BI solutions is regularly evaluated and measured. KPIs or OKRs are used to track data culture goals and the results of these efforts. • Feedback loops are in place, and they encourage ongoing data culture improvements. • Continual improvement of organizational adoption, user adoption, and solution adoption is a top priority.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about the importance of an executive sponsor.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Executive sponsorship

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

When planning to advance the data culture and the state of organizational adoption for data and analytics, it's crucial to have executive support. An executive sponsor is imperative because analytics adoption is far more than just a technology project.

Although some successes can be achieved by a few determined individual contributors, the organization is in a significantly better position when a senior leader is engaged, supportive, informed, and available to assist with the following activities.

- Formulating a strategic vision, goals, and priorities for data, analytics, and business intelligence (BI).
- Providing top-down guidance and reinforcement for the data strategy by regularly promoting, motivating, and investing in strategic and tactical planning.
- Leading by example by actively using data and analytics in a way that's consistent with data culture and adoption goals.
- Allocating staffing and prioritizing resources.
- Approving funding (for example, Fabric licenses).
- Removing barriers to enable action.
- Communicating announcements that are of critical importance, to help them gain traction.
- Decision-making, particularly for strategic-level governance decisions.
- Dispute resolution (for escalated issues that can't be resolved by operational or tactical personnel).
- Supporting organizational change initiatives (for example, creating or expanding the Center of Excellence).

> **Important**
>
> The ideal executive sponsor has sufficient credibility, influence, and authority throughout the organization. They also have an invested stake in data efforts and

the data strategy. When the BI strategy is successful, the ideal executive sponsor also experiences success in their role.

Identifying an executive sponsor

There are multiple ways to identify an executive sponsor.

Top-down pattern

An executive sponsor might be selected by a more senior executive. For example, the Chief Executive Officer (CEO) could hire a Chief Data Officer (CDO) or Chief Analytics Officer (CAO) to explicitly advance the organization's data culture objectives or lead digital transformation efforts. The CDO or CAO then becomes the ideal candidate to serve as the executive sponsor for Fabric (or for data and analytics in general).

Here's another example: The CEO might empower an existing executive, such as the Chief Financial Officer (CFO), because they have a good track record leading data and analytics in their organization. As the new executive sponsor, the CFO could then lead efforts to replicate the finance team's success to other areas of the organization.

> **Note**
>
> Having an executive sponsor at the C-level is an excellent leading indicator. It indicates that the organization recognizes the importance of data as a strategic asset and is advancing its data culture in a positive direction.

Bottom-up pattern

Alternatively, a candidate for the executive sponsor role could emerge due to the success they've experienced with creating data solutions. For example, a business unit within the organization, such as Finance, has organically achieved great success with their use of data and analytics. Essentially, they've successfully formed their own data culture on a smaller scale. A junior-level leader who hasn't reached the executive level (such as a director) might then grow into the executive sponsor role by sharing successes with other business units across the organization.

The bottom-up approach is more likely to occur in smaller organizations. It might be because the return on investment and strategic imperative of a data culture (or digital transformation) isn't yet an urgent priority for C-level executives.

The success for a leader using the bottom-up pattern depends on being recognized by senior leadership.

With a bottom-up approach, the sponsor might be able to make some progress, but they won't have formal authority over other business units. Without clear authority, it's only a matter of time until challenges occur that are beyond their level of authority. For this reason, the top-down approach has a higher probability of success. However, initial successes with a bottom-up approach can convince leadership to increase their level of sponsorship, which might start a healthy competition across other business units in the adoption of data and BI.

Considerations and key actions

Checklist - Here's a list of considerations and key actions you can take to establish or strengthen executive support for analytics.

- ✓ **Identify an executive sponsor with broad authority**: Find someone in a sufficient position of influence and authority (across organizational boundaries) who understands the value and impact of BI. It is important that the individual has a vested interest in the success of analytics in the organization.
- ✓ **Involve your executive sponsor**: Consistently involve your executive sponsor in all strategic-level governance decisions involving data management, BI, and analytics. Also involve your sponsor in all governance data culture initiatives to ensure alignment and consensus on goals and priorities.
- ✓ **Establish responsibilities and expectation**: Formalize the arrangement with documented responsibilities for the executive sponsor role. Ensure that there's no uncertainty about expectations and time commitments.
- ✓ **Identify a backup for the sponsor**: Consider naming a backup executive sponsor. The backup can attend meetings in the sponsor's absence and make time-sensitive decisions when necessary.
- ✓ **Identify business advocates**: Find influential advocates in each business unit. Determine how their cooperation and involvement can help you to accomplish your objectives. Consider involving advocates from various levels in the organization chart.

Questions to ask

Use questions like those found below to assess data literacy.

- Has an executive sponsor of Fabric or other analytical tools been identified?
- If so, who is the executive sponsor?
- If not, is there an informal executive sponsor? Who is the closest to this role? Can you define the business impact of having no executive sponsor?
- To what extent is the strategic importance of Fabric and analytics understood and endorsed by executives?
- Are executives using Fabric and the results of data and BI initiatives? What's the sentiment among executives for the effectiveness of data solutions?
- Is the executive sponsor leading by example in the effective use of data and BI tools?
- Does the executive sponsor provide the appropriate resources for data initiatives?
- Is the executive sponsor involved in dispute resolution and change management?
- Does the executive sponsor engage with the user community?
- Does the executive sponsor have sufficient credibility and healthy relationships across organizational boundaries (particularly the business and IT)?

Maturity levels

The following maturity levels will help you assess your current state of executive support.

Expand table

Level	State of executive support
100: Initial	• There might be awareness from at least one executive about the strategic importance of how analytics can advance the organization's data culture goals. However, neither a sponsor nor an executive-level decision-maker is identified.
200:	• Informal executive support exists for analytics through informal channels and

Level	State of executive support
Repeatable	relationships.
300: Defined	• An executive sponsor is identified. Expectations are clear for the role.
400: Capable	• An executive sponsor is well established with someone with sufficient authority across organizational boundaries. • A healthy and productive partnership exists between the executive sponsor, COE, business units, and IT. The teams are working towards shared data culture goals.
500: Efficient	• The executive sponsor is highly engaged. They're a key driver for advancing the organization's data culture vision. • The executive sponsor is involved with ongoing organizational adoption improvements. KPIs (key performance indicators) or OKRs (objectives and key results) are used to track data culture goals and the results of data, analytics, and BI efforts.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about the importance of business alignment with organizational goals.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Business alignment

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

Business intelligence (BI) activities and solutions have the best potential to deliver value when they're well aligned to organizational business goals. In general, effective business alignment helps to improve adoption. With effective business alignment, the data culture and data strategy enable business users to achieve their business objectives.

You can achieve effective business alignment with data activities and solutions by having:

- An understanding of the strategic importance of data and analytics in achieving measurable progress toward business goals.
- A shared awareness of the business strategy and key business objectives among content owners, creators, consumers, and administrators. A common understanding should be integral to the data culture and decision-making across the organization.
- A clear and unified understanding of the business data needs, and how meeting these needs helps content creators and content consumers achieve their objectives.
- A governance strategy that effectively balances user enablement with risk mitigation.
- An engaged executive sponsor who provides top-down guidance to regularly promote, motivate, and support the data strategy and related activities and solutions.
- Productive and solution-oriented discussions between business teams and technical teams that address business data needs and problems.
- Effective and flexible requirements gathering processes to design and plan solutions.
- Structured and consistent processes to validate, deploy, and support solutions.
- Structured and sustainable processes to regularly update existing solutions so that they remain relevant and valuable, despite changes in technology or business objectives.

Effective business alignment brings significant benefits to an organization. Here are some benefits of effective business alignment.

- Improved adoption, because content consumers are more likely to use solutions that enable them to achieve their objectives.
- Increased business return on investment (ROI) for analytics initiatives and solutions, because these initiatives and solutions will be more likely to directly advance progress toward business goals.
- Less effort and fewer resources spent on change management and changing business requirements, due to an improved understanding of business data needs.

Achieve business alignment

There are multiple ways to achieve business alignment of data activities and initiatives.

Communication alignment

Effective and consistent communication is critical to aligning processes. Consider the following actions and activities when you want to improve communication for successful business alignment.

- Make and follow a plan for central teams and the user community to follow.
- Plan regular alignment meetings between different teams and groups. For example, central teams can plan regular planning and priority alignments with business units. Another example is when central teams schedule regular meetings to mentor and enable self-service users.
- Set up a centralized portal to consolidate communication and documentation for user communities. For strategic solutions and initiatives, consider using a communication hub.
- Limit complex business and technical terminology in cross-functional communications.
- Strive for concise communication and documentation that's formatted and well organized. That way, people can easily find the information that they need.
- Consider maintaining a visible roadmap that shows the planned solutions and activities relevant to the user community in the next quarter.
- Be transparent when communicating policies, decisions, and changes.
- Create a process for people to provide feedback, and review that feedback regularly as part of regular planning activities.

> **Important**

To achieve effective business alignment, you should make it a priority to identify and dismantle any communication barriers between business teams and technical teams.

Strategic alignment

Your business strategy should be well aligned with your data and BI strategy. To incrementally achieve this alignment, we recommend that you commit to following structured, iterative planning processes.

- **Strategic planning**: Define data, analytics, and BI goals and priorities based on the business strategy and current state of adoption and implementation. Typically, strategic planning occurs every 12-18 months to iteratively define high-level desired outcomes. You should synchronize strategic planning with key business planning processes.
- **Tactical planning**: Define objectives, action plans, and a backlog of solutions that help you to achieve your data and BI goals. Typically, tactical planning occurs quarterly to iteratively re-evaluate and align the data strategy and activities to the business strategy. This alignment is informed by business feedback and changes to business objectives or technology. You should synchronize tactical planning with key project planning processes.
- **Solution planning**: Design, develop, test, and deploy solutions that support content creators and consumers in achieving their business objectives. Both centralized content creators and self-service content creators conduct solution planning to ensure that the solutions they create are well aligned with business objectives. You should synchronize solution planning with key adoption and governance planning processes.

> **Important**
>
> Effective business alignment is a key prerequisite for a successful **data strategy**.

Governance and compliance alignment

A key aspect of effective business alignment is balancing user enablement and risk mitigation. This balance is an important aspect of your governance strategy, together with other activities related to compliance, security and privacy, that can include:

- Transparently document and justify compliance criteria, key governance decisions, and policies so that content creators and consumers know what's expected of

them.

- Regularly audit and assess activities to identify risk areas or strong deviations from the desired behaviors.
- Provide mechanisms for content owners, content creators, and content consumers to request clarification or provide feedback about existing policies.

> **⊗ Caution**
>
> A governance strategy that's poorly aligned with business objectives can result in more conflicts and compliance risk, because users will often pursue workarounds to complete their tasks.

Executive alignment

Executive leadership plays a key role in defining the business strategy and business goals. To this end, executive engagement is an important part of achieving top-down business alignment.

To achieve executive alignment, consider the following key considerations and activities.

- Work with your executive sponsor to organize short, quarterly executive feedback sessions about the use of data in the organization. Use this feedback to identify changes in business objectives, re-assess the data strategy, and inform future actions to improve business alignment.
- Schedule regular alignment meetings with the executive sponsor to promptly identify any potential changes in the business strategy or data needs.
- Deliver monthly executive summaries that highlight relevant information, including:
  - Key performance indicators (KPIs) that measure progress toward data, analytics, and BI goals.
  - Fabric adoption and implementation milestones.
  - Technology changes that might impact organizational business goals.

> **ⓘ Important**
>
> Don't underestimate the importance of the role your **executive sponsor** has in achieving and maintaining effective business alignment.

Maintain business alignment

Business alignment is a continual process. To maintain business alignment, consider the following factors.

- **Assign a responsible team**: A working team reviews feedback and organizes re-alignment sessions. This team is responsible for the alignment of planning and priorities between the business and data strategy.
- **Create and support a feedback process**: Your user community requires the means to provide feedback. Examples of feedback can include requests to change existing solutions, or to create new solutions and initiatives. This feedback is essential for bottom-up business user alignment, and it drives iterative and continuous improvement cycles.
- **Measure the success of business alignment**: Consider using surveys, sentiment analysis, and usage metrics to assess the success of business alignment. When combined with other concise feedback mechanisms, this can provide valuable input to help define future actions and activities to improve business alignment and Fabric adoption.
- **Schedule regular re-alignment sessions**: Ensure that data strategic planning and tactical planning occur alongside relevant business strategy planning (when business leadership review business goals and objectives).

> **Note**
>
> Because business objectives continually evolve, you should understand that solutions and initiatives will change over time. Don't assume that requirements for data and BI projects are rigid and can't be altered. If you struggle with changing requirements, it might be an indication that your requirements-gathering process is ineffective or inflexible, or that your development workflows don't sufficiently incorporate regular feedback.

> **Important**
>
> To effectively maintain business alignment, it's essential that user feedback be promptly and directly addressed. Regularly review and analyze feedback, and consider how you can integrate it into iterative strategic planning, tactical planning, and solution planning processes.

Questions to ask

Use questions like those found below to assess business alignment.

- Can people articulate the goals of the organization and the business objectives of their team?
- To what extent do descriptions of organizational goals align across the organization? How do they align between the business user community and leadership community? How do they align between business teams and technical teams?
- Does executive leadership understand the strategic importance of data in achieving business objectives? Does the user community understand the strategic importance of data in helping them succeed in their jobs?
- Are changes in the business strategy reflected promptly in changes to the data strategy?
- Are changes in business user data needs addressed promptly in data and BI solutions?
- To what extent do data policies support or conflict with existing business processes and the way that users work?
- Do solution requirements focus more on technical features than addressing business questions? Is there a structured requirements gathering process? Do content owners and creators interact effectively with stakeholders and content consumers during requirements gathering?
- How are decisions about data or BI investments made? Who makes these decisions?
- How well do people trust existing data and BI solutions? Is there a single version of truth, or are there regular debates about who has the correct version?
- How are data and BI initiatives and strategy communicated across the organization?

Maturity levels

A business alignment assessment evaluates integration between the business strategy and data strategy. Specifically, this assessment focuses on whether or not data and BI initiatives and solutions support business users to achieve business strategic objectives.

The following maturity levels will help you assess your current state of business alignment.

Expand table

Level	ate of data and business alignment**
100: Initial	• Business and data strategies lack formal alignment, which leads to reactive implementation and misalignment between data teams and business users. • Misalignment in priorities and planning hinders productive discussions and effectiveness. • Executive leadership doesn't recognize data as a strategic asset.
200: Repeatable	• There are efforts to align data and BI initiatives with specific data needs without a consistent approach or understanding of their success. • Alignment discussions focus on immediate or urgent needs and focus on features, solutions, tools or data, rather than strategic alignment. • People have a limited understanding of the strategic importance of data in achieving business objectives.
300: Defined	• Data and BI initiatives are prioritized based on their alignment with strategic business objectives. However, alignment is siloed and typically focuses on local needs. • Strategic initiatives and changes have a clear, structured involvement of both the business and data strategic decision makers. Business teams and technical teams can have productive discussions to meet business and governance needs.
400: Capable	• There's a consistent, organization-wide view of how data initiatives and solutions support business objectives. • Regular and iterative strategic alignments occur between the business and technical teams. Changes to the business strategy result in clear actions that are reflected by changes to the data strategy to better support business needs. • Business and technical teams have healthy, productive relationships.
500: Efficient	• The data strategy and the business strategy are fully integrated. Continuous improvement processes drive consistent alignment, and they are themselves data driven.

Level	ate of data and business alignment**
	• Business and technical teams have healthy, productive relationships.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn more about content ownership and management, and its effect on business-led self-service BI, managed self-service BI, and enterprise BI.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Content ownership and management

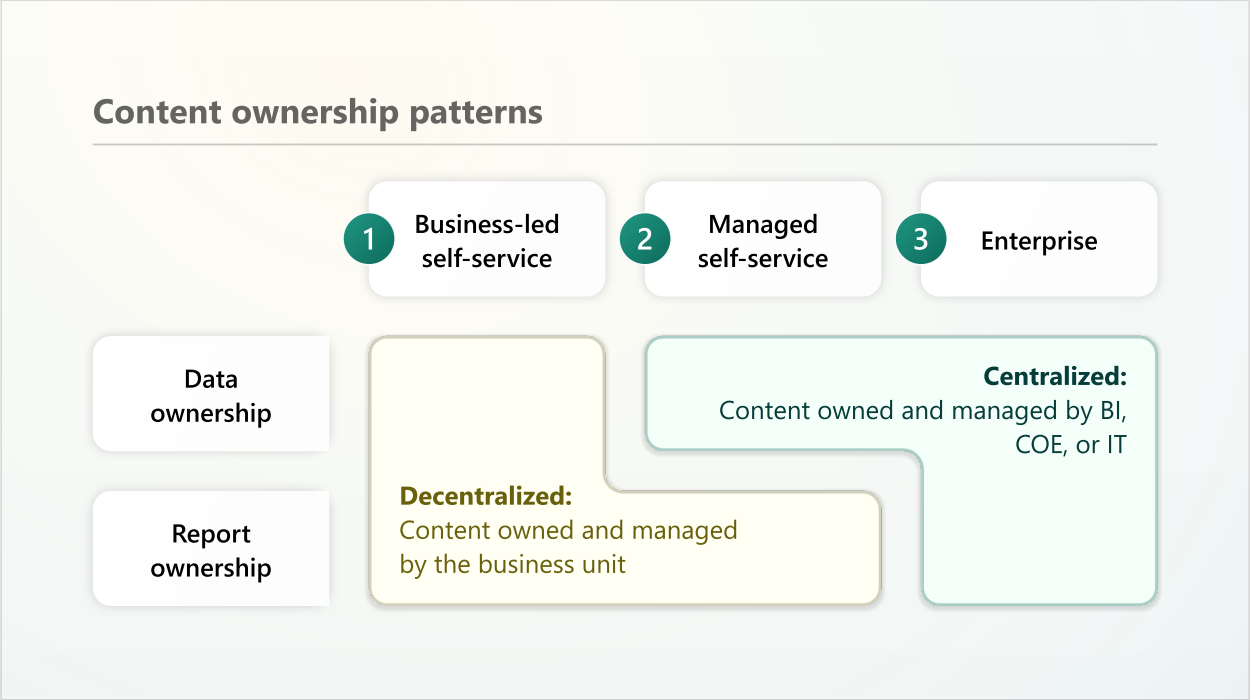
Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

> **Note**
>
> The Power BI implementation planning usage scenarios explore many concepts discussed in this article, focusing on the Power BI workload in Microsoft Fabric. The usage scenario articles include detailed diagrams that you might find helpful to support your planning and decision making.

There are three primary strategies for how data, analytics, and business intelligence (BI) content is owned and managed: business-led self-service, managed self-service, and enterprise. For the purposes of this series of articles, the term *content* refers to any type of data item (like a notebook, semantic model, report, or dashboard).

The organization's data culture is the driver for why, how, and by whom each of these three content ownership strategies is implemented.



The areas in the above diagram include:

Expand table

Area	Description
1	**Business-led self-service**: All content is owned and managed by the creators and subject matter experts within a business unit. This ownership strategy is also known as a *decentralized* or *bottom-up* strategy.
2	**Managed self-service**: The data is owned and managed by a centralized team, whereas business users take responsibility for reports and dashboards. This ownership strategy is also known as *discipline at the core and flexibility at the edge*.
3	**Enterprise**: All content is owned and managed by a centralized team such as IT, enterprise BI, or the Center of Excellence (COE).

It's unlikely that an organization operates exclusively with one content ownership and management strategy. Depending on your data culture, one strategy might be far more dominant than the others. The choice of strategy could differ from solution to solution, or from team to team. In fact, a single team can actively use multiple strategies if it's both a consumer of enterprise content and a producer of its own self-service content. The strategy to pursue depends on factors such as:

- Requirements for a solution (such as a collection of reports, a Power BI app, or a lakehouse).
- User skills.
- Ongoing commitment for training and skills growth.
- Flexibility required.
- Complexity level.
- Priorities and leadership commitment level.

The organization's data culture—particularly its position on data democratization—has considerable influence on the extent of which of the three content ownership strategies are used. While there are common patterns for success, there's no one-size-fits-all approach. Each organization's governance model and approach to content ownership and management should reflect the differences in data sources, applications, and business context.

How content is owned and managed has a significant effect on governance, the extent of mentoring and user enablement, needs for user support, and the COE operating model.

As discussed in the governance article, the level of governance and oversight depends on:

- Who owns and manages the content.
- The scope of content delivery.
- The data subject area and sensitivity level.
- The importance of the data, and whether it's used for critical decision making.

In general:

- Business-led self-service content is subject to the least stringent governance and oversight controls. It often includes personal BI and team BI solutions.
- Managed self-service content is subject to moderately stringent governance and oversight controls. It frequently includes team BI and departmental BI solutions.
- Enterprise solutions are subject to more rigorous governance controls and oversight.

As stated in the adoption maturity levels article, organizational adoption measures the state of data management processes and governance. The choices made for content ownership and management significantly affect how organizational adoption is achieved.

Ownership and stewardship

There are many roles related to data management. Roles can be defined in many ways and can be easily misunderstood. The following table presents possible ways you might conceptually define these roles:

Expand table

Role	Description
Data steward	Responsible for defining and/or managing acceptable data quality levels as well as master data management (MDM).
Subject matter expert (SME)	Responsible for defining what the data means, what it's used for, who might access it, and how the data is presented to others. Collaborates with domain owner as needed and supports colleagues in their use of data.
Technical owner	Responsible for creating, maintaining, publishing, and securing access to data and reporting items.
Domain owner	Higher-level decision-maker who collaborates with governance teams on data management policies, processes, and requirements. Decision-maker for defining appropriate and inappropriate uses of the data. Participates on the data governance board, as described in the governance article.

Assigning ownership for a data domain tends to be more straightforward when managing transactional source systems. In analytics and BI solutions, data is integrated from multiple domain areas, then transformed and enriched. For downstream analytical solutions, the topic of ownership becomes more complex.

> **Note**
>
> Be clear about who is responsible for managing data items. It's crucial to ensure a good experience for content consumers. Specifically, clarity on ownership is helpful for:
>
> - Who to contact with questions.
> - Feedback.
> - Enhancement requests.
> - Support requests.
>
> In the Fabric portal, content owners can set the **contact list property** for many types of items. The contact list is also used in security workflows. For example, when a user is sent a URL to open a **Power BI app** but they don't have permission, they will be presented with an option to make a request for access.

Guidelines for being successful with ownership:

- Define how ownership and stewardship terminology is used in your organization, including expectations for these roles.
- Set contacts for each workspace and for individual items to communicate ownership and/or support responsibilities.
- Specify between two and four workspace administrators and conduct an audit of workspace admins regularly (perhaps twice a year). Workspace admins might be directly responsible for managing workspace content, or it could be that those tasks are assigned to colleagues who do the hands-on work. In all cases, workspace admins should be able to easily contact owners of specific content.
- Include consistent branding on reports to indicate who produced the content and who to contact for help. A small image or text label located in the report footer is valuable, especially when the report is exported from the Fabric portal. A standard template file can encourage and simplify the consistent use of branding.
- Make use of best practices reviews and co-development projects with the COE.

The remainder of this article covers considerations related to the three content ownership and management strategies.

Business-led self-service

With a business-led self-service approach to data and BI, all content is owned and managed by creators and subject matter experts. Because responsibility is retained within a business unit, this strategy is often described as the *bottom-up*, or *decentralized*, approach. Business-led self-service is often a good strategy for personal BI and team BI solutions.

> **Important**
>
> The concept of business-led self-service isn't the same as shadow IT. In both scenarios, data and BI content is created, owned, and managed by business users. However, shadow IT implies that the business unit is circumventing IT and so the solution is not sanctioned. With business-led self-service BI solutions, the business unit has full authority to create and manage content. Resources and support from the COE are available to self-service content creators. It's also expected that the business unit will comply with all established data governance guidelines and policies.

Business-led self-service is most suitable when:

- Decentralized data management aligns with the organization's data culture, and the organization is prepared to support these efforts.
- Data exploration and freedom to innovate is a high priority.
- The business unit wants to have the most involvement and retain the highest level of control.
- The business unit has skilled users capable of—and fully committed to—supporting solutions through the entire lifecycle. It covers all types of items, including the data (such as a lakehouse, data warehouse, data pipeline, dataflow, or semantic model), the visuals (such as reports and dashboards), and Power BI apps.
- The flexibility to respond to changing business conditions and react quickly outweighs the need for stricter governance and oversight.

Here are some guidelines to help become successful with business-led self-service data and BI.

- Teach your creators to use the same techniques that IT would use, like shared semantic models and dataflows. Make use of a well-organized OneLake. Centralize data to reduce maintenance, improve consistency, and reduce risk.

- Focus on providing mentoring, training, resources, and documentation (described in the Mentoring and user enablement article). The importance of these efforts can't be overstated. Be prepared for skill levels of self-service content creators to vary significantly. It's also common for a solution to deliver excellent business value yet be built in such a way that it won't scale or perform well over time (as historic data volumes increase). Having the COE available to help when these situations arise is very valuable.
- Provide guidance on the best way to use endorsements. The promoted endorsement is for content produced by self-service creators. Consider reserving use of the certified endorsement for enterprise BI content and managed self-service BI content (described next).
- Analyze the activity log to discover situations where the COE could proactively contact self-service owners to offer helpful information. It's especially useful when a suboptimal usage pattern is detected. For example, log activity could reveal overuse of individual item sharing when Power BI app audiences or workspace roles might be a better choice. The data from the activity log allows the COE to offer support and advice to the business units. In turn, this information can help increase the quality of solutions, while allowing the business to retain full ownership and control of their content. For more information, see Auditing and monitoring.

Managed self-service

Managed self-service BI is a blended approach to data and BI. The data is owned and managed by a centralized team (such as IT, enterprise BI, or the COE), while responsibility for reports and dashboards belongs to creators and subject matter experts within the business units. Managed self-service BI is frequently a good strategy for team BI and departmental BI solutions.

This approach is often called_discipline at the core and flexibility at the edge_. It's because the data architecture is maintained by a single team with an appropriate level of discipline and rigor. Business units have the flexibility to create reports and dashboards based on centralized data. This approach allows report creators to be far more efficient because they can remain focused on delivering value from their data analysis and visuals.

Managed self-service BI is most suitable when:

- Centralized data management aligns with the organization's data culture.
- The organization has a team of BI experts who manage the data architecture.

- There's value in the reuse of data by many self-service report creators across organizational boundaries.
- Self-service report creators need to produce analytical content at a pace faster than the centralized team can accommodate.
- Different users are responsible for handling data preparation, data modeling, and report creation.

Here are some guidelines to help you become successful with managed self-service BI.

- Teach users to separate model and report development. They can use live connections to create reports based on existing semantic models. When the semantic model is decoupled from the report, it promotes data reuse by many reports and many authors. It also facilitates the separation of duties.
- Use dataflows to centralize data preparation logic and to share commonly used data tables—like date, customer, product, or sales—with many semantic model creators. Refine the dataflow as much as possible, using friendly column names and correct data types to reduce the downstream effort required by semantic model authors, who consume the dataflow as a source. Dataflows are an effective way to reduce the time involved with data preparation and improve data consistency across semantic models. The use of dataflows also reduces the number of data refreshes on source systems and allows fewer users who require direct access to source systems.
- When self-service creators need to augment an existing semantic model with departmental data, educate them to create composite models. This feature allows for an ideal balance of self-service enablement while taking advantage of the investment in data assets that are centrally managed.
- Use the certified endorsement for semantic models and dataflows to help content creators identify trustworthy sources of data.
- Include consistent branding on all reports to indicate who produced the content and who to contact for help. Branding is particularly helpful to distinguish content that is produced by self-service creators. A small image or text label in the report footer is valuable when the report is exported from the Fabric portal.
- Consider implementing separate workspaces for storing data and reports. This approach allows for better clarity on who is responsible for content. It also allows for more restrictive workspace roles assignments. That way, report creators can only publish content to their reporting workspace; and, read and build semantic model permissions allow creators to create new reports with row-level security (RLS) in effect, when applicable. For more information, see Workspace-level planning. For more information about RLS, see Content creator security planning.
- Use the Power BI REST APIs to compile an inventory of Power BI items. Analyze the ratio of semantic models to reports to evaluate the extent of semantic model

reuse.

Enterprise

Enterprise is a centralized approach to delivering data and BI solutions in which all solution content is owned and managed by a centralized team. This team is usually IT, enterprise BI, or the COE.

Enterprise is the most suitable when:

- Centralizing content management with a single team aligns with the organization's data culture.
- The organization has data and BI expertise to manage all items end-to-end.
- The content needs of consumers are well-defined, and there's little need to customize or explore data beyond the reporting solution that's delivered.
- Content ownership and direct access to data needs to be limited to a small number of experts and owners.
- The data is highly sensitive or subject to regulatory requirements.

Here are some guidelines to help you become successful with enterprise data and BI.

- Implement a rigorous process for use of the certified endorsement for content. Not all enterprise content needs to be certified, but much of it probably should be. Certified content should indicate that data quality has been validated. Certified content should also follow change management rules, have formal support, and be fully documented. Because certified content has passed rigorous standards, the expectations for trustworthiness are higher.
- Include consistent branding on enterprise BI reports to indicate who produced the content, and who to contact for help. A small image or text label in the report footer is valuable when the report is exported by a user.
- If you use specific report branding to indicate enterprise BI content, be careful with the *save a copy* functionality that would allow a user to download a copy of a report and personalize it. Although this functionality is an excellent way to bridge enterprise BI with managed self-service BI, it dilutes the value of the branding. A more seamless solution is to provide a separate Power BI Desktop template file for self-service authors. The template defines a starting point for report creation with a live connection to an existing semantic model, and it doesn't include branding. The template file can be shared as a link within a Power BI app, or from the community portal.

Ownership transfers

Occasionally, the ownership of a particular solution might need to be transferred to another team. An ownership transfer from a business unit to a centralized team can happen when:

- A business-led solution is used by a significant number of users, or it now supports critical business decisions. In these cases, the solution should be managed by a team with processes in place to implement higher levels of governance and support.
- A business-led solution is a candidate to be used far more broadly throughout the organization, so it needs to be managed by a team who can set security and deploy content widely throughout the organization.
- A business unit no longer has the expertise, budget, or time available to continue managing the content, but the business need for the content remains.
- The size or complexity of a solution has grown to a point where a different data architecture or redesign is required.
- A proof of concept is ready to be operationalized.

The COE should have well-documented procedures for identifying when a solution is a candidate for ownership transfer. It's very helpful if help desk personnel know what to look for as well. Having a customary pattern for self-service creators to build and grow a solution, and hand it off in certain circumstances, is an indicator of a productive and healthy data culture. A simple ownership transfer could be addressed during COE office hours; a more complex transfer could warrant a small project managed by the COE.

> **Note**
>
> There's potential that the new owner will need to do some refactoring and data validations before they're willing to take full ownership. Refactoring is most likely to occur with the less visible aspects of data preparation, data modeling, and calculations. If there are any manual steps or flat file sources, now is an ideal time to apply those enhancements. The branding of reports and dashboards might also need to change (for example, if there's a footer indicating report contact or a text label indicating that the content is certified).

It's also possible for a centralized team to transfer ownership to a business unit. It could happen when:

- The team with domain knowledge is better equipped to own and manage the content going forward.
- The centralized team has created the solution for a business unit that doesn't have the skills to create it from scratch, but it can maintain and extend the solution

going forward.

> **Tip**
>
> Don't forget to recognize and reward the work of the original creator, particularly if ownership transfers are a common occurrence.

Considerations and key actions

Checklist - Here's a list of considerations and key actions you can take to strengthen your approach to content ownership and management.

- ✓ **Gain a full understanding of what's currently happening**: Ensure you deeply understand how content ownership and management is happening throughout the organization. Recognize that there likely won't be a one-size-fits-all approach to apply uniformly across the entire organization. Review the implementation planning usage scenarios to understand how Power BI and Fabric can be used in diverse ways.
- ✓ **Conduct discussions**: Determine what is currently working well, what isn't working well, and what the desired balance is between the three ownership strategies. If necessary, schedule discussions with specific people on various teams. Develop a plan for moving from the current state to the desired state.
- ✓ **Perform an assessment**: If your enterprise data team currently has challenges related to scheduling and priorities, do an assessment to determine if a managed self-service strategy can be put in place to empower more content creators throughout the organization. Managed self-service data and BI can be extremely effective on a global scale.
- ✓ **Clarify terminology**: Clarify terms used in your organization for owner, data steward, and subject matter expert.
- ✓ **Assign clear roles and responsibilities**: Make sure roles and responsibilities for owners, stewards, and subject matter experts are documented and well understood by everyone involved. Include backup personnel.
- ✓ **Ensure community involvement**: Ensure that all your content owners—from both the business and IT—are part of your community of practice.
- ✓ **Create user guidance for owners and contacts in Fabric**: Determine how you will use the contacts feature in Fabric. Communicate with content creators about how it

should be used, and why it's important.

- ✓ **Create a process for handling ownership transfers**: If ownership transfers occur regularly, create a process for how it will work.
- ✓ **Support your advanced content creators**: Determine your strategy for using external tools for advanced authoring capabilities and increased productivity.

Questions to ask

Use questions like those found below to assess content ownership and management.

- Do central teams that are responsible for Fabric have a clear understanding of who owns what BI content? Is there a distinction between report and data items, or different item types (like Power BI semantic models, data science notebooks, or lakehouses)?
- Which usage scenarios are in place, such as personal BI, team BI, departmental BI, or enterprise BI? How prevalent are they in the organization, and how do they differ between key business units?
- What activities do business analytical teams perform (for example, data integration, data modeling, or reporting)?
- What kinds of roles in the organizations are expected to create and own content? Is it limited to central teams, analysts, or also functional roles, like sales?
- Where does the organization sit on the spectrum of business-led self-service, managed self-service, or enterprise? Does it differ between key business units?
- Do strategic data and BI solutions have ownership roles and stewardship roles that are clearly defined? Which are missing?
- Are content creators and owners also responsible for supporting and updating content once it's released? How effective is the ownership of content support and updates?
- Is a clear process in place to transfer ownership of solutions (where necessary)? An example is when an external consultant creates or updates a solution.
- Do data sources have data stewards or subject matter experts (SMEs) who serve as a special point of contact?
- If your organization is already using Fabric or Power BI, does the current workspace setup comply with the content ownership and delivery strategies that are in place?

Maturity levels

The following maturity levels will help you assess the current state of your content ownership and management.

Expand table

Level	State of content ownership and management
100: Initial	• Self-service content creators own and manage content in an uncontrolled way, without a specific strategy. • A high ratio of semantic models to reports exists. When many semantic models exist only support one report, it indicates opportunities to improve data reusability, improve trustworthiness, reduce maintenance, and reduce the number of duplicate semantic models. • Discrepancies between different reports are common, causing distrust of content produced by others.
200: Repeatable	• A plan is in place for which content ownership and management strategy to use and in which circumstances. • Initial steps are taken to improve the consistency and trustworthiness levels for self-service efforts. • Guidance for the user community is available that includes expectations for self-service versus enterprise content. • Roles and responsibilities are clear and well understood by everyone involved.
300: Defined	• Managed self-service is a priority and an area of investment to further advance the data culture. The priority is to allow report creators the flexibility they need while using well-managed, secure, and trustworthy data sources. • Report branding is consistently used to indicate who produced the content. • A mentoring program exists to educate self-service content creators on how to apply best practices and make good decisions.
400: Capable	• Criteria are defined to align governance requirements for self-service versus enterprise content.

Level	State of content ownership and management
	• There's a plan in place for how to request and handle ownership transfers. • Managed self-service—and techniques for the reuse of data—are commonly used and well-understood.
500: Efficient	• Proactive steps to communicate with users occur when any concerning activities are detected in the activity log. Education and information are provided to make gradual improvements or reduce risk. • Third-party tools are used by highly proficient content creators to improve productivity and efficiency.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about the scope of content delivery.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Content delivery scope

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

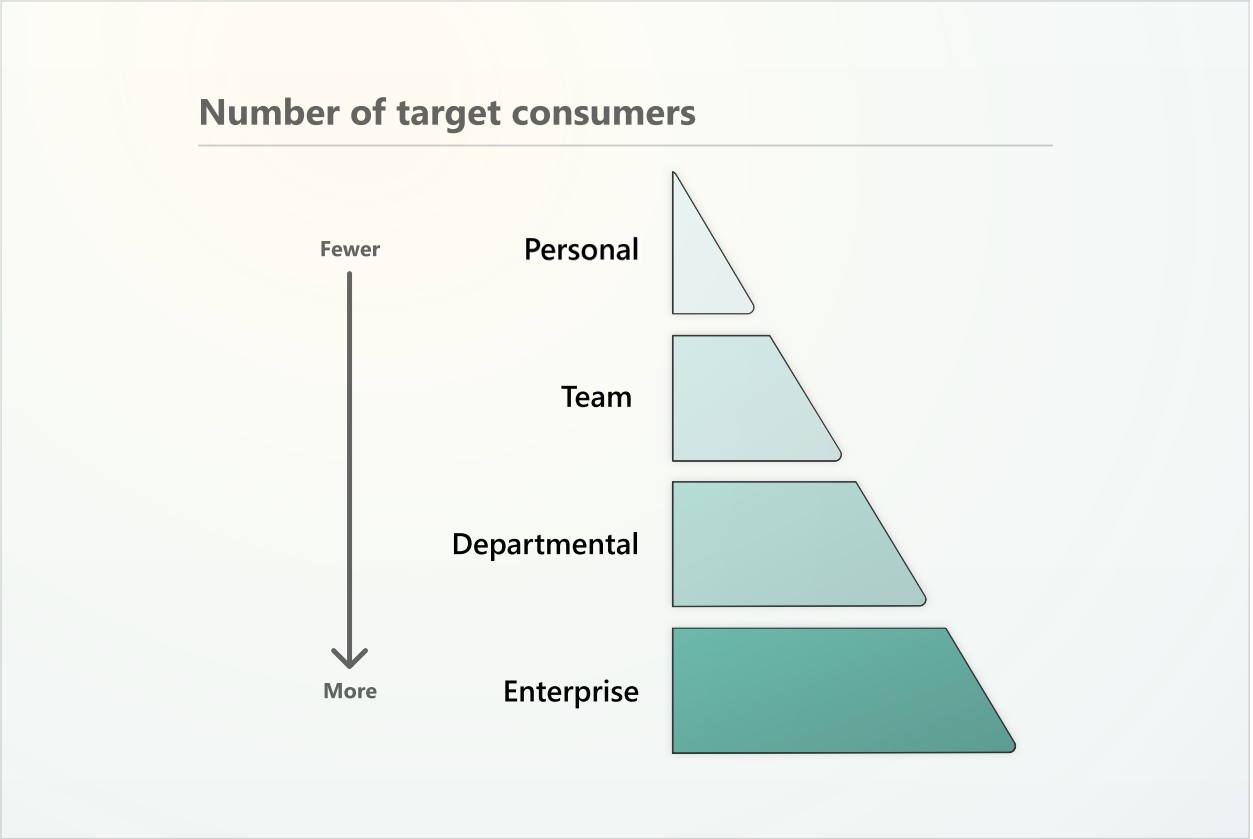
The four delivery scopes described in this article include personal, team, departmental, and enterprise. To be clear, focusing on the scope of a delivered data and business intelligence (BI) solution does refer to the number of people who might view the solution, though the impact is much more than that. The scope strongly influences best practices for not only content distribution, but also content management, security, and information protection. The scope has a direct correlation to the level of governance (such as requirements for change management, support, or documentation), the extent of mentoring and user enablement, and needs for user support. It also influences user licensing decisions.

The related content ownership and management article makes similar points. Whereas the focus of that article was on the content creator, the focus of this article is on the target content usage. Both inter-related aspects need to be considered to arrive at governance decisions and the Center of Excellence (COE) operating model.

> **Important**
>
> Not all data and solutions are equal. Be prepared to apply different levels of data management and governance to different teams and various types of content. Standardized rules are easier to maintain. However, flexibility or customization is often necessary to apply the appropriate level of oversight for particular circumstances. Your **executive sponsor** can prove invaluable by reaching consensus across stakeholder groups when difficult situations arise.

Scope of content delivery

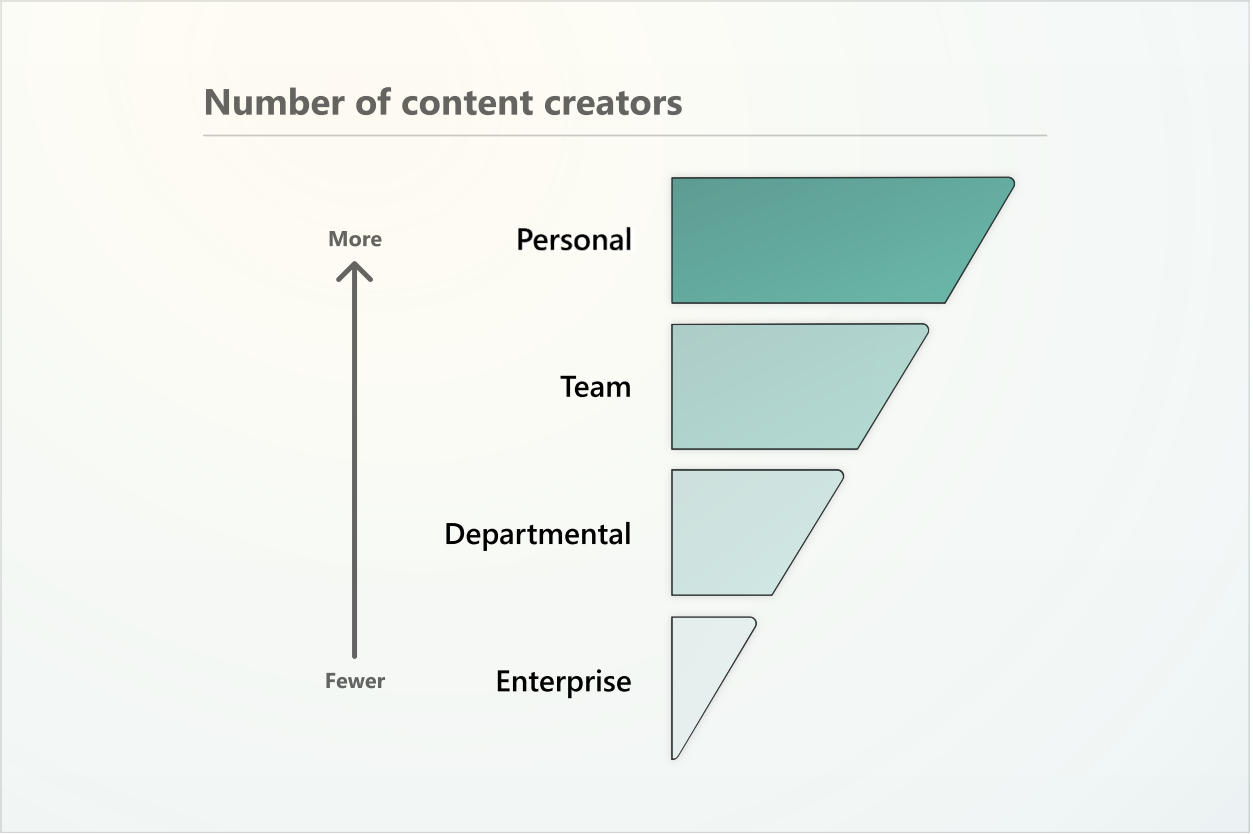
The following diagram focuses on the number of *target consumers* who will consume the content.



The four scopes of content delivery shown in the above diagram include:

- **Personal**: Personal solutions are, as the name implies, intended for use by the creator. Sharing content with others isn't an objective. Therefore, a personal data and BI solution has the fewest number of target consumers.
- **Team**: Collaborates and shares content with a relatively small number of colleagues who work closely together.
- **Departmental**: Delivers content to a large number of consumers, who can belong to a department or business unit.
- **Enterprise**: Delivers content broadly across organizational boundaries to the largest number of target consumers. Enterprise content is most often managed by a centralized team and is subject to additional governance requirements.

Contrast the above four scopes of content delivery with the following diagram, which has an inverse relationship with respect to the number of *content creators*.



The four scopes of content creators shown in the above diagram include:

- **Personal**: Represents the largest number of creators because the data culture encourages any user to work with data using business-led self-service data and BI methods. Although managed self-service BI methods can be used, it's less common with personal data and BI efforts.
- **Team**: Colleagues within a team collaborate and share with each other by using business-led self-service patterns. It has the next largest number of creators in the organization. Managed self-service patterns could also begin to emerge as skill levels advance.
- **Departmental**: Involves a smaller population of creators. They're likely to be considered power users who are using sophisticated tools to create sophisticated solutions. Managed self-service practices are very common and highly encouraged.
- **Enterprise**: Involves the smallest number of content creators because it typically includes only professional data and BI developers who work in the BI team, the COE, or in IT.

The content ownership and management article introduced the concepts of business-led self-service, managed self-service, and enterprise. The most common alignment between ownership and delivery scope is:

- **Business-led self-service ownership**: Commonly deployed as personal and team solutions.
- **Managed self-service ownership**: Can be deployed as personal, team, or departmental solutions.

- **Enterprise ownership**: Typically deployed as enterprise-scoped solutions.

Some organizations also equate self-service content with community-based support. It's the case when self-service content creators and owners are responsible for supporting the content they publish. The user support article describes multiple informal and formal levels for support.

> **Note**
>
> The term *sharing* can be interpreted two ways: It's often used in a general way related to sharing content with colleagues, which could be implemented multiple ways. It can also reference a specific feature in Fabric, which is a specific implementation where a user or group is granted access to a single item. In this article, the term *sharing* is meant in a general way to describe sharing content with colleagues. When the per-item permissions are intended, this article will make a clear reference to that feature. For more information, see Report consumer security planning.

Personal

The Personal delivery scope is about enabling an individual to gain analytical value. It's also about allowing them to more efficiently perform business tasks through the effective personal use of data, information, and analytics. It could apply to any type of information worker in the organization, not just data analysts and developers.

Sharing content with others isn't the objective. Personal content can reside in Power BI Desktop or in a personal workspace in the Fabric portal.

Here are the characteristics of creating content for a personal delivery scope.

- The creator's primary intention is data exploration and analysis, rather than report delivery.
- The content is intended to be analyzed and consumed by one person: the creator.
- The content might be an exploratory proof of concept that may, or may not, evolve into a project.

Here are a few guidelines to help you become successful with content developed for personal use.

- Consider personal data and BI solutions to be like an *analytical sandbox* that has little formal governance and oversight from the governance team or COE. However, it's still appropriate to educate content creators that some general

governance guidelines could still apply to personal content. Valid questions to ask include: Can the creator export the personal report and email it to others? Can the creator store a personal report on a non-organizational laptop or device? What limitations or requirements exist for content that contains sensitive data?

- See the techniques described for business-led self-service, and managed self-service in the content ownership and management article. They're highly relevant techniques that help content creators create efficient and personal data and BI solutions.
- Analyze data from the activity log to discover situations where personal solutions appear to have expanded beyond the original intended usage. It's usually discovered by detecting a significant amount of content sharing from a personal workspace.

> **Tip**
>
> For information about how users progress through the stages of user adoption, see the **Microsoft Fabric adoption roadmap maturity levels**. For more information about using the activity log, see **Tenant-level auditing**.

Team

The Team delivery scope is focused on a team of people who work closely together, and who are tasked with solving closely related problems using the same data. Collaborating and sharing content with each other in a workspace is usually the primary objective.

Content is often shared among the team more informally as compared to departmental or enterprise content. For instance, the workspace is often sufficient for consuming content within a small team. It doesn't require the formality of publishing the workspace to distribute it as an app. There isn't a specific number of users when team-based delivery is considered too informal; each team can find the right number that works for them.

Here are the characteristics of creating content for a team delivery scope.

- Content is created, managed, and viewed among a group of colleagues who work closely together.
- Collaboration and co-management of content is the highest priority.
- Formal delivery of content might occur for report viewers (especially for managers of the team), but it's usually a secondary priority.
- Reports aren't always highly sophisticated or attractive; functionality and accessing the information is what matters most.

Here are some guidelines to help you become successful with content developed for team use.

- Ensure the COE is prepared to support the efforts of self-service creators publishing content for their team.
- Make purposeful decisions about how workspace management will be handled. The workspace is a place to organize related content, a permissions boundary, and the scope for a Power BI app. It's tempting to start with one workspace per team, but that might not be flexible enough to satisfy all needs.
- See the techniques described for business-led self-service and managed self-service in the content ownership and management article. They're highly relevant techniques that help content creators create efficient and effective team data and BI solutions.

> **Tip**
>
> For more information, see **Workspace-level planning**.

Departmental

Content is delivered to members of a department or business unit. Content distribution to a larger number of consumers is a priority for departmental delivery scopes.

Here are the characteristics of departmental content delivery.

- A few content creators typically publish content for colleagues to consume.
- Formal delivery of reports by using Power BI apps is a high priority to ensure consumers have the best experience.
- Additional effort is made to deliver more sophisticated and polished reports. Following best practices for data preparation and higher quality data modeling is also expected.
- Needs for change management and lifecycle management begin to emerge to ensure release stability and a consistent experience for consumers.

Here are a few guidelines to help you become successful with departmental BI delivery.

- Ensure that the COE is prepared to support the efforts of self-service creators. Creators who publish content used throughout their department or business unit might emerge as candidates to become champions. Or, they might become candidates to join the COE as a satellite member.

- Make purposeful decisions about how workspace management will be handled. The workspace is a place to organize related content, a permissions boundary, and the scope for an app. Several workspaces will likely be required to meet all the needs of a large department or business unit.
- Plan how Power BI apps will distribute content to the enterprise. An app can provide a significantly better user experience for consuming content. In many cases, content consumers can be granted permissions to view content via the app only, reserving workspace permissions management for content creators and reviewers only. The use of app audience groups allows you to *mix and match* content and target audience in a flexible way.
- Be clear about what data quality validations have occurred. As the importance and criticality level grows, expectations for trustworthiness grow too.
- Ensure that adequate training, mentoring, and documentation is available to support content creators. Best practices for data preparation, data modeling, and data presentation will result in better quality solutions.
- Provide guidance on the best way to use the promoted endorsement, and when the certified endorsement could be permitted for departmental solutions.
- Ensure that the owner is identified for all departmental content. Clarity on ownership is helpful, including who to contact with questions, feedback, enhancement requests, or support requests. In the Fabric portal, content owners can set the contact list property for many types of items (like reports and dashboards). The contact list is also used in security workflows. For example, when a user is sent a URL to open an app but they don't have permission, they'll be presented with an option to make a request for access.
- Consider using deployment pipelines in conjunction with separate workspaces. Deployment pipelines can support development, test, and production environments, which provide more stability for consumers.
- Consider enforcing the use of sensitivity labels to implement information protection on all content.
- Include consistent branding on reports by:
  - Using departmental colors and styling to indicate who produced the content. For more information, see Content ownership and management.
  - Adding a small image or text label to the report footer, which is valuable when the report is exported from the Fabric portal.
  - Using a standard Power BI Desktop template file. For more information, see Mentoring and user enablement.
- Apply the techniques described for business-led self-service and managed self-service content delivery in the Content ownership and management article. They're highly relevant techniques that can help content creators to create efficient and effective departmental solutions.

Enterprise

Enterprise content is typically managed by a centralized team and is subject to additional governance requirements. Content is delivered broadly across organizational boundaries.

Here are the characteristics of enterprise content delivery.

- A centralized team of experts manages the content end-to-end and publishes it for others to consume.
- Formal delivery of data solutions like reports, lakehouses, and Power BI apps is a high priority to ensure consumers have the best experience.
- The content is highly sensitive, subject to regulatory requirements, or is considered extremely critical.
- Published enterprise-level semantic models and dataflows might be used as a source for self-service creators, thus creating a chain of dependencies to the source data.
- Stability and a consistent experience for consumers are highly important. Application lifecycle management, such as deployment pipelines and DevOps techniques, is commonly used. Change management processes to review and approve changes before they're deployed are commonly used for enterprise content, for example, by a change review board or similar group.
- Processes exist to gather requirements, prioritize efforts, and plan for new projects or enhancements to existing content.
- Integration with other enterprise-level data architecture and management services could exist, possibly with other Azure services and Power Platform products.

Here are some guidelines to help you become successful with enterprise content delivery.

- Governance and oversight techniques described in the governance article are relevant for managing an enterprise solution. Techniques primarily include change management and lifecycle management.
- Plan for how to effectively use Premium Per User or Fabric capacity licensing per workspace. Align your workspace management strategy, like how workspaces will be organized and secured, to the planned licensing strategy.
- Plan how Power BI apps will distribute enterprise content to consumers. An app can provide a significantly better user experience for consuming content. Align the app distribution strategy with your workspace management strategy.
- Consider enforcing the use of sensitivity labels to implement information protection on all content.

- Implement a rigorous process for use of the certified endorsement for enterprise reports and apps. Data assets can be certified, too, when there's the expectation that self-service creators will build solutions based on them. Not all enterprise content needs to be certified, but much of it probably will be.
- Make it a common practice to announce when changes will occur. For more information, see the community of practice article for a description of communication types.
- Include consistent branding on reports, by:
  - Using specific colors and styling, which can also indicate who produced the content. For more information, see Content ownership and management.
  - Adding a small image or text label to the report footer, which can be valuable when the report is exported from the Fabric portal.
  - Using a standard Power BI Desktop template file. For more information, see Mentoring and user enablement.
- Actively use the lineage view to understand dependencies, perform impact analysis, and communicate to downstream content owners when changes will occur.
- See the techniques described for enterprise content delivery in the content ownership and management article. They're highly relevant techniques that help content creators create efficient and effective enterprise solutions.
- See the techniques described in the system oversight article for auditing, governing, and the oversight of enterprise content.

Considerations and key actions

Checklist - Considerations and key actions you can take to strengthen your approach to content delivery.

- ✓ **Align goals for content delivery**: Ensure that guidelines, documentation, and other resources align with the strategic goals defined for Fabric adoption.
- ✓ **Clarify the scopes for content delivery in your organization**: Determine who each scope applies to, and how each scope aligns with governance decisions. Ensure that decisions and guidelines are consistent with how content ownership and management is handled.
- ✓ **Consider exceptions**: Be prepared for how to handle situations when a smaller team wants to publish content for an enterprise-wide audience.

  - Will it require the content be owned and managed by a centralized team? For more information, see the Content ownership and management article, which describes an inter-related concept with content delivery scope.
  - Will there be an approval process? Governance can become more complicated when the content delivery scope is broader than the owner of the content. For example, when an app that's owned by a divisional sales team is distributed to the entire organization.
- ✓ **Create helpful documentation**: Ensure that you have sufficient training documentation and support so that your content creators understand when it's appropriate to use workspaces, apps, or per-item sharing (direct access or link).
- ✓ **Create a licensing strategy**: Ensure that you have a specific strategy in place to handle Fabric licensing considerations. Create a process for how workspaces could be assigned each license type, and the prerequisites required for the type of content that could be assigned to Premium.

> **Important**
>
> At times this article refers to Power BI Premium or its capacity subscriptions (P SKUs). Be aware that Microsoft is currently consolidating purchase options and retiring the Power BI Premium per capacity SKUs. New and existing customers should consider purchasing Fabric capacity subscriptions (F SKUs) instead.
>
> For more information, see **Important update coming to Power BI Premium licensing** and **Power BI Premium FAQ**.

Questions to ask

Use questions like those found below to assess content delivery scope.

- Do central teams that are responsible for Fabric have a clear understanding of who creates and delivers content? Does it differ by business area, or for different content item types?
- Which usage scenarios are in place, such as personal BI, team BI, departmental BI, or enterprise BI? How prevalent are they in the organization? Are there advanced scenarios, like advanced data preparation or advanced data model management, or niche scenarios, like self-service real-time analytics?

- For the identified content delivery scopes in place, to what extent are guidelines being followed?
- Are there trajectories for helpful self-service content to be "promoted" from personal to team content delivery scopes and beyond? What systems and processes enable sustainable, bottom-up scaling and distribution of useful self-service content?
- What are the guidelines for publishing content to, and using, personal workspaces?
- Are personal workspaces assigned to dedicated Fabric capacity? In what circumstances are personal workspaces intended to be used?
- On average, how many reports does someone have access to? How many reports does an executive have access to? How many reports does the CEO have access to?
- If your organization is using Fabric or Power BI today, does the current workspace setup comply with the content ownership and delivery strategies that are in place?
- Is there a clear licensing strategy? How many licenses are used today? How many tenants and capacities exist, who uses them, and why?
- How do central teams decide what gets published to Premium (or Fabric) dedicated capacity, and what uses shared capacity? Do development workloads use separate Premium Per User (PPU) licensing to avoid affecting production workloads?

Maturity levels

The following maturity levels will help you assess the current state of your content delivery.

Expand table

Level	State of content delivery
100: Initial	• Content is published for consumers by self-service creators in an uncontrolled way, without a specific strategy.
200: Repeatable	• Pockets of good practices exist. However, good practices are overly dependent on the knowledge, skills, and habits of the content creator.

Level	State of content delivery
300: Defined	• Clear guidelines are defined and communicated to describe what can and can't occur within each delivery scope. These guidelines are followed by some—but not all—groups across the organization.
400: Capable	• Criteria are defined to align governance requirements for self-service versus enterprise content. • Guidelines for content delivery scope are followed by most, or all, groups across the organization. • Change management requirements are in place to approve critical changes for content that's distributed to a larger-sized audience. • Changes are announced and follow a communication plan. Content creators are aware of the downstream effects on their content. Consumers are aware of when reports and apps are changed.
500: Efficient	• Proactively take steps to communicate with users occur when any concerning activities are detected in the activity log. Education and information are provided to make gradual improvements or reduce risk. • The business value that's achieved for deployed solutions is regularly evaluated.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about the Center of Excellence (COE).

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Center of Excellence

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

A data or analytics Center of Excellence (COE) is an internal team of technical and business experts. The team actively assists others within the organization who are working with data. The COE forms the nucleus of the broader community to advance adoption goals, which align with the data culture vision.

A COE might also be known as *competency center*, *capability center*, or a *center of expertise*. Some organizations use the term *squad*. Many organizations perform the COE responsibilities within their data, analytics, or business intelligence (BI) team.

> **Note**
>
> Having a COE team formally recognized in your organizational chart is recommended, but not required. What's most important is that the COE roles and responsibilities are identified, prioritized, and assigned. It's common for a centralized data or analytics team to take on many of the COE responsibilities; some responsibilities might also reside within IT. For simplicity, in this series of articles, COE means a *specific group of people*, although you might implement it differently. It's also very common to implement the COE with a scope broader than Fabric or Power BI alone: for instance, a Power Platform COE, a data COE, or an analytics COE.

Goals for a COE

Goals for a COE include:

- Evangelizing a data-driven culture.
- Promoting the adoption of analytics.
- Nurturing, mentoring, guiding, and educating internal users to increase their skills and level of self-reliance.

- Coordinating efforts and disseminating knowledge across organizational boundaries.
- Creating consistency and transparency for the user community, which reduces friction and pain points related to finding relevant data and analytics content.
- Maximizing the benefits of self-service BI, while reducing the risks.
- Reducing technical debt by helping users make good decisions that increase consistency and result in fewer inefficiencies.

> **Important**
>
> One of the most powerful aspects of a COE is the cross-departmental insight into how analytics tools like Fabric are used by the organization. This insight can reveal which practices work well and which don't, that can facilitate a bottom-up approach to governance. A primary goal of the COE is to learn which practices work well, share that knowledge more broadly, and replicate best practices across the organization.

Scope of COE responsibilities

The scope of COE responsibilities can vary significantly between organizations. In a way, a COE can be thought of as a consultancy service because its members routinely provide expert advice to the internal community of users. To varying degrees, most COEs handle hands-on work too.

Common COE responsibilities include:

- Mentoring and facilitating knowledge sharing within the internal Fabric community.
- Holding office hours to engage with the internal Fabric community.
- Conducting co-development projects and best practices reviews in order to actively help business units deliver solutions.
- Managing the centralized portal.
- Producing, curating, and promoting training materials.
- Creating documentation and other resources, such as template files, to encourage consistent use of standards and best practices.
- Applying, communicating, and assisting with governance guidelines.
- Handling and assisting with system oversight and Fabric administration.
- Responding to user support issues escalated from the help desk.
- Developing solutions and/or proofs of concept.
- Establishing and maintaining the BI platform and data architecture.

- Communicating regularly with the internal community of users.

Staffing a COE

People who are good candidates as COE members tend to be those who:

- Understand the analytics vision for the organization.
- Have a desire to continually improve analytics practices for the organization.
- Have a deep interest in, and expertise with, analytics tools such as Fabric.
- Are interested in seeing Fabric used effectively and adopted successfully throughout the organization.
- Take the initiative to continually learn, adapt, and grow.
- Readily share their knowledge with others.
- Are interested in repeatable processes, standardization, and governance with a focus on user enablement.
- Are hyper-focused on collaboration with others.
- Are comfortable working in an agile fashion.
- Have an inherent interest in being involved and helping others.
- Can effectively translate business needs into solutions.
- Communicate well with both technical and business colleagues.

> **Tip**
>
> If you have self-service content creators in your organization who constantly push the boundaries of what can be done, they might be a great candidate to become a recognized **champion**, or perhaps even a satellite member of the COE.

When recruiting for the COE, it's important to have a mix of complementary analytical skills, technical skills, and business skills.

Roles and responsibilities

Very generalized roles within a COE are listed below. It's common for multiple people to overlap roles, which is useful from a backup and cross-training perspective. It's also common for the same person to serve multiple roles. For instance, most COE members also serve as a coach or mentor.

Expand table

Role	Description
COE leader	Manages the day-to-day operations of the COE. Interacts with the executive sponsor and other organizational teams, such as the data governance board, as necessary. For an overview of additional roles and responsibilities, see the Governance article.
Coach	Coaches and educates others on data and BI skills via office hours (community engagement), best practices reviews, or co-development projects. Oversees and participates in the discussion channel of the internal community. Interacts with, and supports, the champions network.
Trainer	Develops, curates, and delivers internal training materials, documentation, and resources.
Data analyst	Domain-specific subject matter expert. Acts as a liaison between the COE and the business unit. Content creator for the business unit. Assists with content certification. Works on co-development projects and proofs of concept.
Data modeler	Creates and manages data assets (such as shared semantic model and dataflows) to support other self-service content creators.
Report creator	Creates and publishes reports, dashboards, and metrics.
Data engineer	Plans for deployment and architecture, including integration with other services and data platforms. Publishes data assets which are utilized broadly across the organization (such as a lakehouse, data warehouse, data pipeline, dataflow, or semantic model).
User support	Assists with the resolution of data discrepancies and escalated help desk support issues.

As mentioned previously, the scope of responsibilities for a COE can vary significantly between organizations. Therefore, the roles found for COE members can vary too.

Structuring a COE

The selected COE structure can vary among organizations. It's also possible for multiple structures to exist inside of a single large organization. That's particularly true when there are subsidiaries or when acquisitions have occurred.

> **Note**
>
> The following terms might differ to those defined for your organization, particularly the meaning of *federated*, which tends to have many different IT-related meanings.

Centralized COE

A centralized COE comprises a single shared services team.

Pros:

- There's a single point of accountability for a single team that manages standards, best practices, and delivery end-to-end.
- The COE is one group from an organizational chart perspective.
- It's easy to start with this approach and then evolve to the unified or federated model over time.

Cons:

- A centralized team might have an authoritarian tendency to favor one-size-fits-all decisions that don't always work well for all business units.
- There can be a tendency to prefer IT skills over business skills.
- Due to the centralized nature, it might be more difficult for the COE members to sufficiently understand the needs of all business units.

Unified COE

A unified COE is a single, centralized, shared services team that has been expanded to include embedded team members. The embedded team members are dedicated to supporting a specific functional area or business unit.

Pros:

- There's a single point of accountability for a single team that includes cross-functional involvement from the embedded COE team members. The embedded COE team members are assigned to various areas of the business.
- The COE is one group from an organizational chart perspective.
- The COE understands the needs of business units more deeply due to dedicated members with domain expertise.

Cons:

- The embedded COE team members, who are dedicated to a specific business unit, have a different organizational chart responsibility than the people they serve directly within the business unit. The organizational structure could potentially lead to complications, differences in priorities, or necessitate the involvement of the executive sponsor. Preferably, the executive sponsor has a scope of authority that includes the COE and all involved business units to help resolve conflicts.

Federated COE

A federated COE comprises a shared services team (the core COE members) plus satellite members from each functional area or major business unit. A federated team works in coordination, even though its members reside in different business units. Typically, satellite members are primarily focused on development activities to support their business unit while the shared services personnel support the entire community.

Pros:

- There's cross-functional involvement from satellite COE members who represent their specific functional area and have domain expertise.
- There's a balance of centralized and decentralized representation across the core and satellite COE members.
- When distributed data ownership situations exist—as could be the case when business units take direct responsibility for data management activities—this model is effective.

Cons:

- Since core and satellite members span organizational boundaries, the federated COE approach requires strong leadership, excellent communication, robust project management, and ultra-clear expectations.
- There's a higher risk of encountering competing priorities due to the federated structure.
- This approach typically involves part-time people and/or *dotted line* organizational chart accountability that can introduce competing time pressures.

> **Tip**
>
> Some organizations have success by using a *rotational program*. It involves federated members joining the core COE for a period of time, such as six months. This type of program allows federated members to learn best practices and understand more deeply how and why things are done. Although each federated member remains focused on their specific business unit, they gain a deeper understanding of the organization's challenges. This deeper understanding leads to a more productive partnership over time.

Decentralized COE

Decentralized COEs are independently managed by business units.

Pros:

- A specialized data culture exists that's focused on the business unit, making it easier to learn quickly and adapt.
- Policies and practices are tailored to each business unit.
- Agility, flexibility, and priorities are focused on the individual business unit.

Cons:

- There's a risk that decentralized COEs operate in isolation. As a result, they might not share best practices and lessons learned outside of their business unit.
- Collaboration with a centralized team might be informal and/or inconsistent.
- Inconsistent policies are created and applied across business units.
- It's difficult to scale a decentralized model.
- There's potential rework to bring one or more decentralized COEs in alignment with organizational-wide policies.
- Larger business units with significant funding might have more resources available to them, which might not serve cost optimization goals from an organizational-wide perspective.

> **ⓘ Important**
>
> A highly centralized COE tends to be more *authoritarian*, while highly decentralized COEs tend to be more *siloed*. Each organization will need to weigh the pros and cons that apply to them to determine the best choice. For most organizations, the most effective approach tends to be the unified or federated, which bridges organizational boundaries.

Funding the COE

The COE might obtain its operating budget in multiple ways:

- Cost center.
- Profit center with project budget(s).
- A combination of cost center and profit center.

When the COE operates as a cost center, it absorbs the operating costs. Generally, it involves an approved annual budget. Sometimes this is called a *push* engagement model.

When the COE operates as a profit center (for at least part of its budget), it could accept projects throughout the year based on funding from other business units. Sometimes

this is called a *pull* engagement model.

Funding is important because it impacts the way the COE communicates and engages with the internal community. As the COE experiences more and more successes, they might receive more requests from business units for help. It's especially the case as awareness grows throughout the organization.

> **Tip**
>
> The choice of funding model can determine how the COE actively grows its influence and ability to help. The funding model can also have a big impact on where authority resides and how decision-making works. Further, it impacts the types of services a COE can offer, such as co-development projects and/or best practices reviews. For more information, see the **Mentoring and user enablement** article.

Some organizations cover the COE operating costs with chargebacks to business units based on the usage goals of Fabric. For a shared capacity, this could be based on number of active users. For Premium capacity, chargebacks could be allocated based on which business units are using the capacity. Ideally, chargebacks are directly correlated to the business value gained.

> **Important**
>
> At times this article refers to Power BI Premium or its capacity subscriptions (P SKUs). Be aware that Microsoft is currently consolidating purchase options and retiring the Power BI Premium per capacity SKUs. New and existing customers should consider purchasing Fabric capacity subscriptions (F SKUs) instead.
>
> For more information, see **Important update coming to Power BI Premium licensing** and **Power BI Premium FAQ**.

Considerations and key actions

Checklist - Considerations and key actions you can take to establish or improve your COE.

- ✓ **Define the scope of responsibilities for the COE**: Ensure that you're clear on what activities the COE can support. Once the scope of responsibilities is known, identify the skills and competencies required to fulfill those responsibilities.
- ✓ **Identify gaps in the ability to execute**: Analyze whether the COE has the required systems and infrastructure in place to meet its goals and scope of responsibilities.
- ✓ **Determine the best COE structure**: Identify which COE structure is most appropriate (centralized, unified, federated, or decentralized). Verify that staffing, roles and responsibilities, and appropriate organizational chart relationships (HR reporting) are in place.
- ✓ **Plan for future growth**: If you're starting out with a centralized or decentralized COE, consider how you will scale the COE over time by using the unified or federated approach. Plan for any actions that you can take now that'll facilitate future growth.
- ✓ **Identify customers**: Identify the internal community members, and any external customers, to be served by the COE. Decide how the COE will generally engage with those customers, whether it's a push model, pull model, or both models.
- ✓ **Verify the funding model for the COE**: Decide whether the COE is purely a cost center with an operating budget, whether it will operate partially as a profit center, and/or whether chargebacks to other business units will be required.
- ✓ **Create a communication plan**: Create you communications strategy to educate the internal community of users about the services the COE offers, and how to engage with the COE.
- ✓ **Create goals and metrics**: Determine how you'll measure effectiveness for the COE. Create KPIs (key performance indicators) or OKRs (objectives and key results) to validate that the COE consistently provides value to the user community.

Questions to ask

Use questions like those found below to assess the effectiveness of a COE.

- Is there a COE? If so, who is in the COE and what's the structure?
- If there isn't a COE, is there a central team that performs a similar function? Do data decision makers in the organization understand what a COE does?
- If there isn't a COE, does the organization aspire to create one? Why or why not?
- Are there opportunities for federated or decentralized COE models due to a mix of enterprise and departmental solutions?

- Are there any missing roles and responsibilities from the COE?
- To what extent does the COE engage with the user community? Do they mentor users? Do they curate a centralized portal? Do they maintain centralized resources?
- Is the COE recognized in the organization? Does the user community consider them to be credible and helpful?
- Do business users see central teams as enabling or restricting their work with data?
- What's the COE funding model? Do COE customers financially contribute in some way to the COE?
- How consistent and transparent is the COE with their communication?

Maturity levels

The following maturity levels will help you assess the current state of your COE.

Expand table

Level	State of the Center of Excellence
100: Initial	• One or more COEs exist, or the activities are performed within the data team, BI team, or IT. There's no clarity on the specific goals nor expectations for responsibilities. • Requests for assistance from the COE are handled in an unplanned manner.
200: Repeatable	• The COE is in place with a specific charter to mentor, guide, and educate self-service users. The COE seeks to maximize benefits of self-service approaches to data and BI while reducing the risks. • The goals, scope of responsibilities, staffing, structure, and funding model are established for the COE.
300: Defined	• The COE operates with active involvement from all business units in a unified or federated mode.
400: Capable	• The goals of the COE align with organizational goals, and they are reassessed regularly. • The COE is well-known throughout the organization, and consistently proves its value to the internal user community.

Level	State of the Center of Excellence
500: Efficient	• Regular reviews of KPIs or OKRs evaluate COE effectiveness in a measurable way. • Agility and implementing continual improvements from lessons learned (including scaling out methods that work) are top priorities for the COE.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about implementing governance guidelines, policies, and processes.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Governance

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

Data governance is a broad and complex topic. This article introduces key concepts and considerations. It identifies important actions to take when adopting Microsoft Fabric, but it's not a comprehensive reference for data governance.

As defined by the Data Governance Institute, data governance is "a system of decision rights and accountabilities for information-related processes, executed according to agreed-upon models which describe who can take what actions, with what information, and when, under what circumstances, using what methods."

The term *data governance* is a misnomer. The primary focus for governance isn't on the data itself. The focus is on governing *what users do with the data*. Put another way: the true focus is on governing user's behavior to ensure organizational data is well managed.

When focused on self-service data and business intelligence (BI), the primary goals of governance are to achieve the proper balance of:

- **User empowerment**: Empower the internal user community to be productive and efficient, within requisite guardrails.
- **Regulatory compliance**: Comply with the organization's industry, governmental, and contractual regulations.
- **Internal requirements**: Adhere to the organization's internal requirements.

The optimal balance between control and empowerment will differ between organizations. It's also likely to differ among different business units within an organization. You'll be most successful with a platform like Fabric when you put as much emphasis on user empowerment as on clarifying its practical usage within established guardrails.

> **Tip**

Think of governance as a set of established guidelines and formalized policies. All governance guidelines and policies should align with your organizational **data culture** and adoption objectives. Governance is enacted on a day-to-day basis by your **system oversight** (administration) activities.

Governance strategy

When considering data governance in any organization, the best place to start is by defining a governance strategy. By focusing first on the strategic goals for data governance, all detailed decisions when implementing governance policies and processes can be informed by the strategy. In turn, the governance strategy will be defined by the organization's data culture.

Governance decisions are implemented with documented guidance, policies, and processes. Objectives for governance of a self-service data and BI platform, such as Fabric, include:

- Empowering users throughout the organization to use data and make decisions, within the defined boundaries.
- Improving the user experience by providing clear and transparent guidance (with minimal friction) on what actions are permitted, why, and how.
- Ensuring that the data usage is appropriate for the needs of the business.
- Ensuring that content ownership and stewardship responsibilities are clear. For more information, see the Content ownership and management article.
- Enhancing the consistency and standardization of working with data across organizational boundaries.
- Reducing risk of data leakage and misuse of data. For more information, see the information protection and data loss prevention series of articles article.
- Meeting regulatory, industry, and internal requirements for the proper use of data.

Tip

A well-executed data governance strategy makes it easier for more users to work with data. When governance is approached from the perspective of user empowerment, users are more likely to follow the documented processes. Accordingly, the users become a trusted partner too.

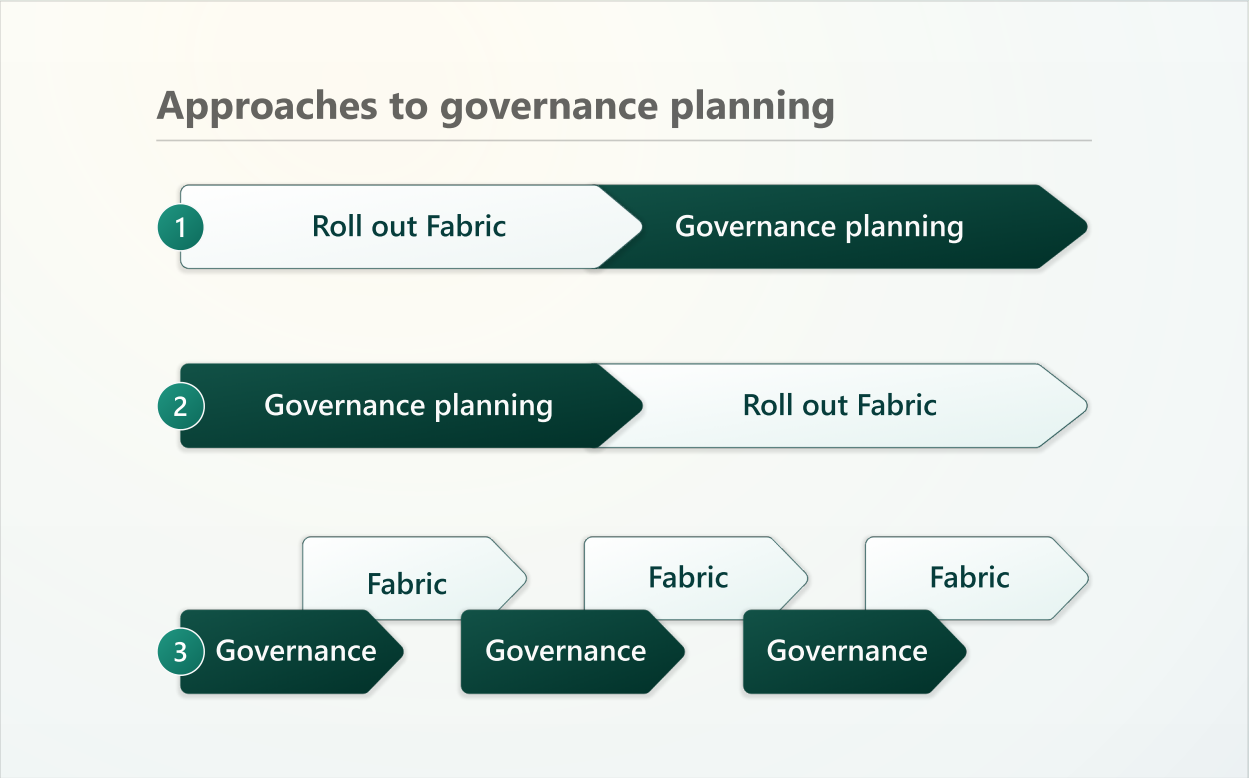
Governance success factors

Governance isn't well-received when it's enacted with top-down mandates that are focused more on control than empowerment. Governing Fabric is most successful when:

- The most lightweight governance model that accomplishes required objectives is used.
- Governance is approached on an iterative basis and doesn't significantly impede productivity.
- A bottom-up approach to formulating governance guidelines is used whenever practical. The Center of Excellence (COE) and/or the data governance team observes successful behaviors that are occurring within a business unit. The COE then takes action to scale out to other areas of the organization.
- Governance decisions are co-defined with input from different business units before they're enacted. Although there are times when a specific directive is necessary (particularly in heavily regulated industries), mandates should be the exception rather than the rule.
- Governance needs are balanced with flexibility and the ability to be productive.
- Governance requirements can be satisfied as part of users' regular workflow, making it easier for users to do the right thing in the right way with little friction.
- The answer to new requests for data isn't "no" by default, but rather "yes and" with clear, simple, transparent rules for what governance requirements are for data access, usage, and sharing.
- Users that need access to data have incentive to do so through normal channels, complying with governance requirements, rather than circumventing them.
- Governance decisions, policies, and requirements for users to follow are in alignment with organizational data culture goals as well as other existing data governance initiatives.
- Decisions that affect what users can—and can't—do aren't made solely by a system administrator.

Introduce governance to your organization

There are three primary timing methods organizations take when introducing Fabric governance to an organization.



The methods in the above diagram include:

Expand table

Method	Strategy followed
1	**Roll out Fabric first, then introduce governance**: Fabric is made widely available to users in the organization as a new self-service data and BI tool. Then, at some time in the future, a governance effort begins. This method prioritizes agility.
2	**Full governance planning first, then roll out Fabric**: Extensive governance planning occurs prior to permitting users to begin using Fabric. This method prioritizes control and stability.
3	**Iterative governance planning with rollouts of Fabric in stages**: Just enough governance planning occurs initially. Then Fabric is iteratively rolled out in stages to individual teams while iterative governance enhancements occur. This method equally prioritizes agility and governance.

Choose method 1 when Fabric is already used for self-service scenarios, and you're ready to start working in a more efficient manner.

Choose method 2 when your organization already has a well-established approach to governance that can be readily expanded to include Fabric.

Choose method 3 when you want to have a balance of control agility. This balanced approach is the best choice for most organizations and most scenarios.

Each method is described in the following sections.

Method 1: Roll out Fabric first

Method 1 prioritizes agility and speed. It allows users to quickly get started creating solutions. This method occurs when Fabric has been made widely available to users in the organization as a new self-service data and BI tool. Quick wins and some successes are achieved. At some point in the future, a governance effort begins, usually to bring order to an unacceptable level of chaos since the self-service user population didn't receive sufficient guidance.

Pros:

- Fastest to get started
- Highly capable users can get things done quickly
- Quick wins are achieved

Cons:

- Higher effort to establish governance once Fabric is used prevalently throughout the organization
- Resistance from self-service users who are asked to change what they've been doing
- Self-service users need to figure out things on their own, which is inefficient and results in inconsistencies
- Self-service users need to use their best judgment, which produces technical debt to be resolved

See other possible cons in the Governance challenges section below.

Method 2: In-depth governance planning first

Method 2 prioritizes control and stability. It lies at the opposite end of the spectrum from method 1. Method 2 involves doing extensive governance planning before rolling out Fabric. This situation is most likely to occur when the implementation of Fabric is led by IT. It's also likely to occur when the organization operates in a highly regulated industry, or when an existing data governance board imposes significant prerequisites and up-front requirements.

Pros:

- More fully prepared to meet regulatory requirements
- More fully prepared to support the user community

Cons:

- Favors enterprise content development more than self-service
- Slower to allow the user population to begin to get value and improve decision-making
- Encourages poor habits and workarounds when there's a significant delay in allowing the use of data for decision-making

Method 3: Iterative governance with rollouts

Method 3 seeks a balance between agility and governance. It's an ideal scenario that does *just enough* governance planning upfront. Frequent and continual governance improvements iteratively occur over time alongside Fabric development projects that deliver value.

Pros:

- Puts equal priority on governance and user productivity
- Emphasizes a *learning as you go* mentality
- Encourages iterative releases to groups of users in stages

Cons:

- Requires a high level of communication to be successful with agile governance practices
- Requires additional discipline to keep documentation and training current
- Introducing new governance guidelines and policies too often causes a certain level of user disruption

For more information about up-front planning, see the Preparing to migrate to Power BI article.

Governance challenges

If your organization has implemented Fabric without a governance approach or strategic direction (as described above by method 1), there could be numerous challenges requiring attention. Depending on the approach that you've taken and your current state, some of the following challenges could be applicable to your organization.

Strategy challenges

- Lack of a cohesive data governance strategy that aligns with the business strategy
- Lack of executive support for governing data as a strategic asset

- Insufficient adoption planning for advancing adoption and the maturity level of BI and analytics

People challenges

- Lack of aligned priorities between centralized teams and business units
- Lack of identified champions with sufficient expertise and enthusiasm throughout the business units to advance organizational adoption objectives
- Lack of awareness of self-service best practices
- Resistance to following newly introduced governance guidelines and policies
- Duplicate effort spent across business units
- Lack of clear accountability, roles, and responsibilities

Process challenges

- Lack of clearly defined processes resulting in chaos and inconsistencies
- Lack of standardization or repeatability
- Insufficient ability to communicate and share lessons learned
- Lack of documentation and over-reliance on tribal knowledge
- Inability to comply with security and privacy requirements

Data quality and data management challenges

- Sprawl of data and reports
- Inaccurate, incomplete, or outdated data
- Lack of trust in the data, especially for content produced by self-service content creators
- Inconsistent reports produced without sufficient data validation
- Valuable data not used or difficult to access
- Fragmented, siloed, and duplicated data
- Lack of data catalog, inventory, glossary, or lineage
- Unclear data ownership and stewardship

Skills and data literacy challenges

- Varying levels of ability to interpret, create, and communicate with data effectively
- Varying levels of technical skillsets and skill gaps
- Lack of ability to confidently manage data diversity and volume
- Underestimating the level of complexity for BI solution development and management throughout its entire lifecycle

- Short tenure with continual staff transfers and turnover
- Coping with the speed of change for cloud services

> **Tip**
>
> Identifying your current challenges—as well as your strengths—is essential to do proper governance planning. There's no single straightforward solution to the challenges listed above. Each organization needs to find the right balance and approach that solves the challenges that are most important to them. The challenges presented above will help you identify how they might affect your organization, so you can start thinking about what the right solution is for your circumstances.

Governance planning

Some organizations have implemented Fabric without a governance approach or clear strategic direction (as described above by method 1). In this case, the effort to begin governance planning can be daunting.

If a formal governance body doesn't currently exist in your organization, then the focus of your governance planning and implementation efforts will be broader. If, however, there's an existing data governance board in the organization, then your focus is primarily to integrate with existing practices and customize them to accommodate the objectives for self-service and enterprise data and BI scenarios.

> **Important**
>
> Governance is a big undertaking, and it's never completely *done*. Relentlessly prioritizing and iterating on improvements will make the scope more manageable. If you track your progress and accomplishments each week and each month, you'll be amazed at the impact over time. The **maturity levels** at the end of each article in this series can help you to assess where you are currently.

Some potential governance planning activities and outputs that you might find valuable are described next.

Strategy

Key activities:

- Conduct a series of workshops to gather information and assess the current state of data culture, adoption, and data and BI practices. For guidance about how to gather information and define the current state of BI adoption, including governance, see BI strategic planning.
- Use the current state assessment and information gathered to define the desired future state, including governance objectives. For guidance about how to use this current state definition to decide on your desired future state, see BI tactical planning.
- Validate the focus and scope of the governance program.
- Identify existing bottom-up initiatives in progress.
- Identify immediate pain points, issues, and risks.
- Educate senior leadership about governance, and ensure executive sponsorship is sufficient to sustain and grow the program.
- Clarify where Power BI fits in to the overall BI and analytics strategy for the organization.
- Assess internal factors such as organizational readiness, maturity levels, and key challenges.
- Assess external factors such as risk, exposure, regulatory, and legal requirements—including regional differences.

Key output:

- Business case with cost/benefit analysis
- Approved governance objectives, focus, and priorities that are in alignment with high-level business objectives
- Plan for short-term goals and priorities (quick wins)
- Plan for long-term and deferred goals and priorities
- Success criteria and measurable key performance indicators (KPIs)
- Known risks documented with a mitigation plan
- Plan for meeting industry, governmental, contractual, and regulatory requirements that impact BI and analytics in the organization
- Funding plan

People

Key activities:

- Establish a governance board and identify key stakeholders.
- Determine focus, scope, and a set of responsibilities for the governance board.
- Establish a COE.
- Determine focus, scope, and a set of responsibilities for COE.
- Define roles and responsibilities.

- Confirm who has decision-making, approval, and veto authority.

Key output:

- Charter for the governance board
- Charter and priorities for the COE
- Staffing plan
- Roles and responsibilities
- Accountability and decision-making matrix
- Communication plan
- Issue management plan

Policies and processes

Key activities:

- Analyze immediate pain points, issues, risks, and areas to improve the user experience.
- Prioritize data policies to be addressed by order of importance.
- Identify existing processes in place that work well and can be formalized.
- Determine how new data policies will be socialized.
- Decide to what extent data policies might differ or be customized for different groups.

Key output:

- Process for how data policies and documentation will be defined, approved, communicated, and maintained
- Plan for requesting valid exceptions and departures from documented policies

Project management

The implementation of the governance program should be planned and managed as a series of projects.

Key activities:

- Establish a timeline with priorities and milestones.
- Identify related initiatives and dependencies.
- Identify and coordinate with existing bottom-up initiatives.
- Create an iterative project plan that's aligned with high-level prioritization.
- Obtain budget approval and funding.
- Establish a tangible way to track progress.

Key output:

- Project plan with iterations, dependencies, and sequencing
- Cadence for retrospectives with a focus on continual improvements

> **ⓘ Important**
>
> The scope of activities listed above that will be useful to take on will vary considerably between organizations. If your organization doesn't have existing processes and workflows for creating these types of outputs, refer to the guidance found in the **adoption roadmap conclusion** for some helpful resources, as well as the **implementation planning BI strategy articles**.

Governance policies

Decision criteria

All governance decisions should be in alignment with the established goals for organizational adoption. Once the strategy is clear, more tactical governance decisions will need to be made which affect the day-to-day activities of the self-service user community. These types of tactical decisions correlate directly to the data policies that get created.

How we go about making governance decisions depends on:

- **Who owns and manages the data and BI content?** The Content ownership and management article introduced three types of strategies: business-led self-service, managed self-service, and enterprise. Who owns and manages the content has a significant impact on governance requirements.
- **What is the scope for delivery of the data and BI content?** The Content delivery scope article introduced four scopes for delivery of content: personal, team, departmental, and enterprise. The scope of delivery has a considerable impact on governance requirements.
- **What is the data subject area?** The data itself, including its sensitivity level, is an important factor. Some data domains inherently require tighter controls. For instance, personally identifiable information (PII), or data subject to regulations, should be subject to stricter governance requirements than less sensitive data.
- **Is the data, and/or the BI solution, considered critical?** If you can't make an informed decision easily without this data, you're dealing with critical data elements. Certain reports and apps could be deemed critical because they meet a

set of predefined criteria. For instance, the content is delivered to executives. Predefined criteria for what's considered *critical* helps everyone have clear expectations. Critical data is usually subject to stricter governance requirements.

> **Tip**
>
> Different combinations of the above four criteria will result in different governance requirements for Fabric content.

Key Fabric governance decisions

As you explore your goals and objectives and pursue more tactical data governance decisions as described above, it will be important to determine what the highest priorities are. Deciding where to focus your efforts can be challenging.

The following list includes items that you might choose to prioritize when introducing governance for Fabric.

- Recommendations and requirements for content ownership and management
- Recommendations and requirements for content delivery scope
- Recommendations and requirements for content distribution and sharing with colleagues, as well as for external users, such as customers, partners, or vendors
- How users are permitted to work with regulated data and highly sensitive data
- Allowed use of unverified data sources that are unknown to IT
- When manually maintained data sources, such as Excel or flat files, are permitted
- Who is permitted to create a workspace
- How to manage workspaces effectively
- How personal workspaces are effectively used
- Which workspaces are assigned to Fabric capacity
- Who is allowed to be a Fabric administrator
- Security, privacy, and data protection requirements, and allowed actions for content assigned to each sensitivity label
- Allowed or encouraged use of personal gateways
- Allowed or encouraged use of self-service purchasing of user licenses
- Requirements for who can certify content, as well as requirements that must be met
- Application lifecycle management for managing content through its entire lifecycle, including development, test, and production stages
- Additional requirements applicable to critical content, such as data quality verifications and documentation

- Requirements to use standardized master data and common data definitions to improve consistency across data assets
- Recommendations and requirements for use of external tools by advanced content creators

If you don't make governance decisions and communicate them well, users will use their own judgment for how things should work—and that often results in inconsistent approaches to common tasks.

Although not every governance decision needs to be made upfront, it's important that you identify the areas of greatest risk in your organization. Then, incrementally implement governance policies and processes that will deliver the most impact.

Data policies

A data policy is a document that defines what users can and can't do. You might call it something different, but the goal remains the same: when decisions—such as those discussed in the previous section—are made, they're documented for use and reference by the community of users.

A data policy should be as short as possible. That way, it's easy for people to understand what is being asked of them.

A data policy should include:

- Policy name, purpose, description, and details
- Specific responsibilities
- Scope of the policy (organization-wide versus departmental-specific)
- Audience for the policy
- Policy owner, approver, and contact
- How to request an exception
- How the policy will be audited and enforced
- Regulatory or legal requirements met by the policy
- Reference to terminology definitions
- Reference to any related guidelines or policies
- Effective date, last revision date, and change log

> **Note**
>
> Locate, or link to, data policies from your centralized portal.

Here are three common data policy examples you might choose to prioritize.

Expand table

Policy	Description
Data ownership policy	Specifies when an owner is required for a data asset, and what the data owner's responsibilities include, such as: supporting colleagues who view the content, maintaining appropriate confidentiality and security, and ensuring compliance.
Data certification (endorsement) policy	Specifies the process that is followed to certify content. Requirements might include activities such as: data accuracy validation, data source and lineage review, technical review of the data model, security review, and documentation review.
Data classification and protection policy	Specifies activities that are allowed and not allowed per classification (sensitivity level). It should specify activities such as: allowed sharing with external users, with or without a non-disclosure agreement (NDA), encryption requirements, and ability to download the data. Sometimes, it's also called a *data handling policy* or a *data usage policy*. For more information, see the Information protection for Power BI article.

⊗ Caution

Having a lot of documentation can lead to a false sense that everything is under control, which can lead to complacency. The level of engagement that the COE has with the user community is one way to improve the chances that governance guidelines and policies are consistently followed. Auditing and monitoring activities are also important.

Scope of policies

Governance decisions will rarely be one-size-fits-all across the entire organization. When practical, it's wise to start with standardized policies, and then implement exceptions as needed. Having a clearly defined strategy for how policies will be handled for centralized and decentralized teams will make it much easier to determine how to handle exceptions.

Pros of organization-wide policies:

- Much easier to manage and maintain
- Greater consistency
- Encompasses more use cases
- Fewer policies overall

Cons of organization-wide policies:

- Inflexible
- Less autonomy and empowerment

Pros of departmental-scope policies:

- Expectations are clearer when tailored to a specific group
- Customizable and flexible

Cons of departmental-scope policies:

- More work to manage
- More policies that are siloed
- Potential for conflicting information
- Difficult to scale more broadly throughout the organization

> **Tip**
>
> Finding the right balance of standardization and customization for supporting self-service data and BI across the organization can be challenging. However, by starting with organizational policies and mindfully watching for exceptions, you can make meaningful progress quickly.

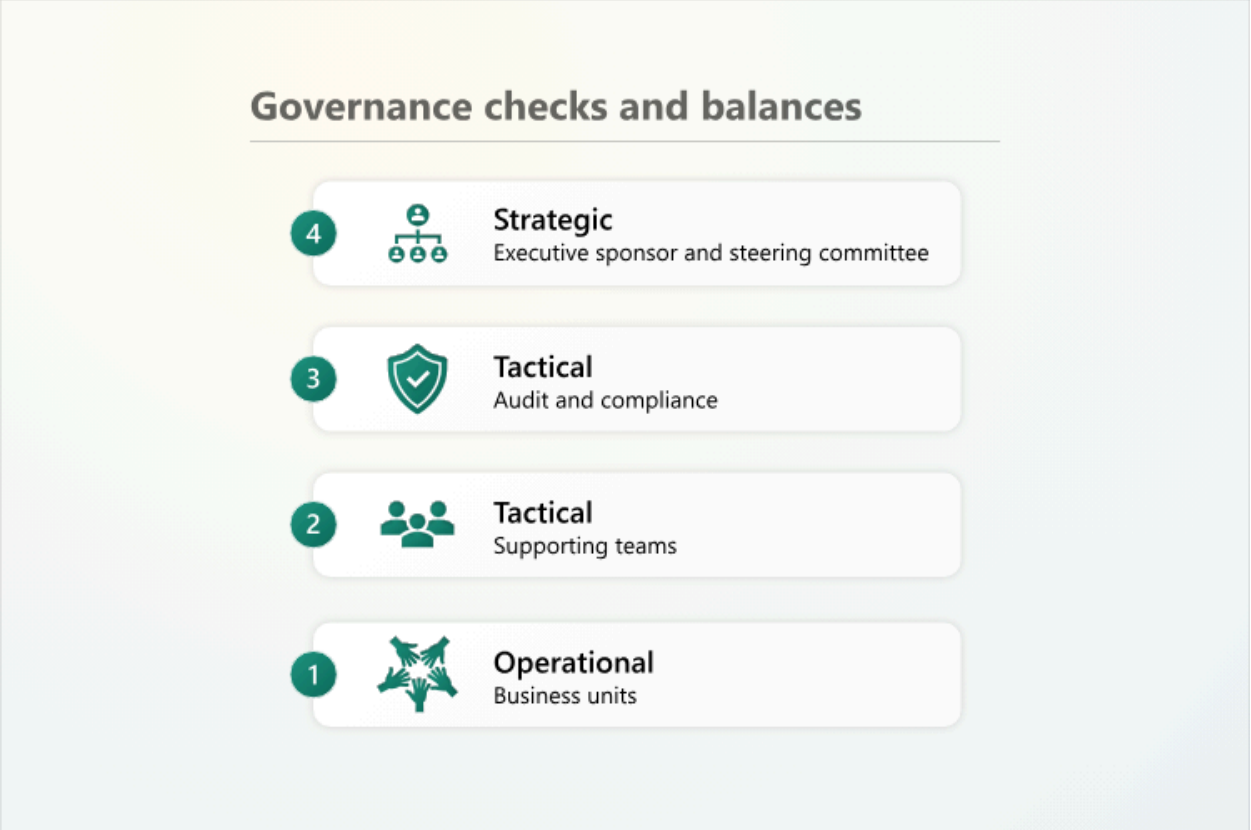
Staffing and accountability

The organizational structure for data governance varies substantially between organizations. In larger organizations there might be a data governance office with dedicated staff. Some organizations have a data governance board, council, or steering committee with assigned members coming from different business units. Depending on the extent of the data governance body within the organization, there could be an executive team separate from a functional team of people.

> **Important**
>
> Regardless of how the governance body is structured, it's important that there's a person or group with sufficient influence over data governance decisions. This person should have authority to enforce those decisions across organizational boundaries.

Checks and balances

Governance accountability is about checks and balances.



Starting with the first level, the levels of checks and balances in the above diagram include:

Expand table

Level	Description
1	**Operational - Business units**: Level 1 is the foundation of a well-governed system, which includes users within the business units performing their work. Self-service data and BI creators have a lot of responsibilities related to authoring, publishing, sharing, security, and data quality. Self-service data and BI consumers also have responsibilities for the proper use of data.
2	**Tactical - Supporting teams**: Level 2 includes several groups that support the efforts of the users in the business units. Supporting teams include the COE, enterprise data and BI, the data governance office, as well as other ancillary teams. Ancillary teams can include IT, security, HR, and legal. A change control board is included here as well.
3	**Tactical - Audit and compliance**: Level 3 includes internal audit, risk management, and compliance teams. These teams provide guidance to levels 1 and 2. They also provide enforcement when necessary.
4	**Strategic - Executive sponsor and steering committee**: The highest level includes the executive-level oversight of strategy and priorities. This level handles any escalated issues that couldn't be solved at lower levels. Therefore, it's important to have a leadership team with sufficient authority to be able to make decisions when necessary.

> **Important**
>
> Everyone has a responsibility to adhere to policies for ensuring that organizational data is secure, protected, and well-managed as an organizational asset. Sometimes this is cited as *everyone is a data steward*. To make this a reality, start with the users in the business units (level 1 described above) as the foundation.

Roles and responsibilities

Once you have a sense for your governance strategy, roles and responsibilities should be defined to establish clear expectations.

Governance team structure, roles (including terminology), and responsibilities vary widely among organizations. Very generalized roles are described in the table below. In some cases, the same person could serve multiple roles. For instance, the Chief Data Officer (CDO) could also be the executive sponsor.

Expand table

Role	Description
Chief Data Officer or Chief Analytics Officer	Defines the strategy for use of data as an enterprise asset. Oversees enterprise-wide governance guidelines and policies.
Data governance board	Steering committee with members from each business unit who, as domain owners, are empowered to make enterprise governance decisions. They make decisions on behalf of the business unit *and* in the best interest of the organization. Provides approvals, decisions, priorities, and direction to the enterprise data governance team and working committees.
Data governance team	Creates governance policies, standards, and processes. Provides enterprise-wide oversight and optimization of data integrity, trustworthiness, privacy, and usability. Collaborates with the COE to provide governance education, support, and mentoring to data owners and content creators.
Data governance working committees	Temporary or permanent teams that focus on individual governance topics, such as security or data quality.
Change management board	Coordinates the requirements, processes, approvals, and scheduling for release management processes with the objective of reducing risk and minimizing the impact of changes to critical applications.
Project	Manages individual governance projects and the ongoing data governance

Role	Description
management office	program.
Fabric executive sponsor	Promotes adoption and the successful use of Fabric. Actively ensures that Fabric decisions are consistently aligned with business objectives, guiding principles, and policies across organizational boundaries. For more information, see the Executive sponsorship article.
Center of Excellence	Mentors the community of creators and consumers to promote the effective use of Fabric for decision-making. Provides cross-departmental coordination of Fabric activities to improve practices, increase consistency, and reduce inefficiencies. For more information, see the Center of Excellence article.
Fabric champions	A subset of content creators found within the business units who help advance the adoption of Fabric. They contribute to data culture growth by advocating the use of best practices and actively assisting colleagues. For more information, see the Community of practice article.
Fabric administrators	Day-to-day-system oversight responsibilities to support the internal processes, tools, and people. Handles monitoring, auditing, and management. For more information, see the System oversight article.
Information technology	Provides occasional assistance to Fabric administrators for services related to Fabric, such as Microsoft Entra ID, Microsoft 365, Teams, SharePoint, or OneDrive.
Risk management	Reviews and assesses data sharing and security risks. Defines ethical data policies and standards. Communicates regulatory and legal requirements.
Internal audit	Auditing of compliance with regulatory and internal requirements.
Data steward	Collaborates with governance committee and/or COE to ensure that organizational data has acceptable data quality levels.
All BI creators and consumers	Adheres to policies for ensuring that data is secure, protected, and well-managed as an organizational asset.

Tip

Name a backup for each person in key roles, for example, members of the data governance board. In their absence, the backup person can attend meetings and make time-sensitive decisions when necessary.

Considerations and key actions

Checklist - Considerations and key actions you can take to establish or strengthen your governance initiatives.

- ✓ **Align goals and guiding principles**: Confirm that the high-level goals and guiding principles of the data culture goals are clearly documented and communicated. Ensure that alignment exists for any new governance guidelines or policies.
- ✓ **Understand what's currently happening**: Ensure that you have a deep understanding of how Fabric is currently used for self-service and enterprise data and BI scenarios. Document opportunities for improvement. Also, document strengths and good practices that would be helpful to scale out more broadly.
- ✓ **Prioritize new governance guidelines and policies**: For prioritizing which new guidelines or policies to create, select an important pain point, high priority need, or known risk for a data domain. It should have significant benefit and can be achieved with a feasible level of effort. When you implement your first governance guidelines, choose something users are likely to support because the change is low impact, or because they are sufficiently motivated to make a change.
- ✓ **Create a schedule to review policies**: Determine the cadence for how often data policies are reevaluated. Reassess and adjust when needs change.
- ✓ **Decide how to handle exceptions**: Determine how conflicts, issues, and requests for exceptions to documented policies will be handled.
- ✓ **Understand existing data assets**: Confirm that you understand what critical data assets exist. Create an inventory of ownership and lineage, if necessary. Keep in mind that you can't govern what you don't know about.
- ✓ **Verify executive sponsorship**: Confirm that you have support and sufficient attention from your executive sponsor, as well as from business unit leaders.
- ✓ **Prepare an action plan**: Include the following key items:
  - **Initial priorities**: Select one data domain or business unit at a time.
  - **Timeline**: Work in iterations long enough to accomplish meaningful progress, yet short enough to periodically adjust.
  - **Quick wins**: Focus on tangible, tactical, and incremental progress.
  - **Success metrics**: Create measurable metrics to evaluate progress.

Questions to ask

Use questions like those found below to assess governance.

- At a high level, what's the current governance strategy? To what extent is the purpose and importance of this governance strategy clear to both end users and the central data and BI teams?
- In general, is the current governance strategy effective?
- What are the key regulatory and compliance criteria that the organization (or specific business units) must adhere to? Where's this criteria documented? Is this information readily available to people who work with data and share data items as a part of their role?
- How well does the current governance strategy align to the user's way of working?
- Is a specific role or team responsible for governance in the organization?
- Who has the authority to create and change governance policies?
- Do governance teams use Microsoft Purview or another tool to support governance activities?
- What are the prioritized governance risks, such as risks to security, information protection, and data loss prevention?
- What's the potential business impact of the identified governance risks?
- How frequently is the governance strategy re-evaluated? What metrics are used to evaluate it, and what mechanisms exist for business users to provide feedback?
- What types of user behaviors create risk when users work with data? How are those risks mitigated?
- What sensitivity labels are in place, if any? Are data and BI decision makers aware of sensitivity labels and the benefits to the business?
- What data loss prevention policies are in place, if any?
- How is "Export to Excel" handled? What steps are taken to prevent data loss prevention? What's the prevalence of "Export to Excel"? What do people do with data once they have it in Excel?
- Are there practices or solutions that are out of regulatory compliance that must be urgently addressed? Are these examples justified with an explanation of the potential business impact, should they not be addressed?

Tip

"Export to Excel" is typically a controversial topic. Often, business users focus on the requirement to have "Export to Excel" possible in BI solutions. Enabling "Export to

Excel" can be counter-productive because a business objective isn't to get data into Excel. Instead, define why end users need the data in Excel. Ask what they do with the data once it's in Excel, which business questions they try to answer, what decisions they make, and what actions they take with the data.

Focusing on business decisions and actions helps steer focus away from tools and features and toward helping people achieve their business objectives.

Maturity levels

The following maturity levels will help you assess the current state of your governance initiatives.

Expand table

Level	State of governance
100: Initial	• Due to a lack of governance planning, the good data management and informal governance practices that are occurring are overly reliant on judgment and experience level of individuals. • There's a significant reliance on undocumented tribal knowledge.
200: Repeatable	• Some areas of the organization have made a purposeful effort to standardize, improve, and document their data management and governance practices. • An initial governance approach exists. Incremental progress is being made.
300: Defined	• A complete governance strategy with focus, objectives, and priorities is enacted and broadly communicated. • Specific governance guidelines and policies are implemented for the top few priorities (pain points or opportunities). They're actively and consistently followed by users. • Roles and responsibilities are clearly defined and documented.
400: Capable	• All Fabric governance priorities align with organizational goals and business objectives. Goals are reassessed regularly. • Processes exist to customize policies for decentralized business units, or to

Level	State of governance
	handle valid exceptions to standard governance policies. • It's clear where Fabric fits into the overall data and BI strategy for the organization. • Fabric activity log and API data is actively analyzed to monitor and audit Fabric activities. Proactive action is taken based on the data.
500: Efficient	• Regular reviews of KPIs or OKRs evaluate measurable governance goals. Iterative, continual progress is a priority. • Agility and implementing continual improvements from lessons learned (including scaling out methods that work) are top priorities for the COE. • Fabric activity log and API data is actively used to inform and improve adoption and governance efforts.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about mentoring and user enablement.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Mentoring and user enablement

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

A critical objective for adoption efforts is to enable users to accomplish as much as they can within the requisite guardrails established by governance guidelines and policies. For this reason, the act of mentoring users is one of the most important responsibilities of the Center of Excellence (COE), and it has a direct influence on how user adoption occurs. For more information about user adoption, see Microsoft Fabric adoption maturity levels.

Skills mentoring

Mentoring and helping users in the Fabric community become more effective can take on various forms, such as:

- Office hours
- Co-development projects
- Best practices reviews
- Extended support

Office hours

Office hours are a form of ongoing community engagements managed by the COE. As the name implies, office hours are times of regularly scheduled availability where members of the community can engage with experts from the COE to receive assistance with minimal process overhead. Office hours are usually group-based, so Fabric champions and other members of the community can also help solve an issue if a topic is in their area of expertise.

Office hours are a very popular and productive activity in many organizations. Some organizations call them *drop-in hours* or even a fun name such as *Power Hour* or *Fabric Fridays*. The primary goal is usually to get questions answered, solve problems, and

remove blockers. Office hours can also be used as a platform for the user community to share ideas, suggestions, and even complaints.

The COE publishes the times for regular office hours when one or more COE members are available. Ideally, office hours are held on a regular and frequent basis. For instance, it could be every Tuesday and Thursday. Consider offering different time slots or rotating times if you have a global workforce.

> **Tip**
>
> One option is to set specific office hours each week. However, users might not show up, so that can end up being inefficient. Alternatively, consider leveraging Microsoft Bookings to schedule office hours. It shows the blocks of time when each COE expert is available, with Outlook integration ensuring availability is up to date.

Office hours are an excellent user enablement approach because:

- Content creators and the COE actively collaborate to answer questions and solve problems together.
- Real work is accomplished while learning and problem solving.
- Others might observe, learn, and participate.
- Individual groups can head to a breakout room to solve a specific problem.

Office hours benefit the COE as well because:

- They're a great way for the COE to identify champions or users with specific skills that the COE didn't previously know about.
- The COE can learn what users throughout the organization are struggling with. It helps inform whether additional resources, documentation, or training might be required.

> **Tip**
>
> It's common for some tough issues to come up during office hours that cannot be solved quickly, such as getting a complex DAX calculation to work, or addressing performance challenges in a complex solution. Set clear expectations for what's in scope for office hours, and if there's any commitment for follow up.

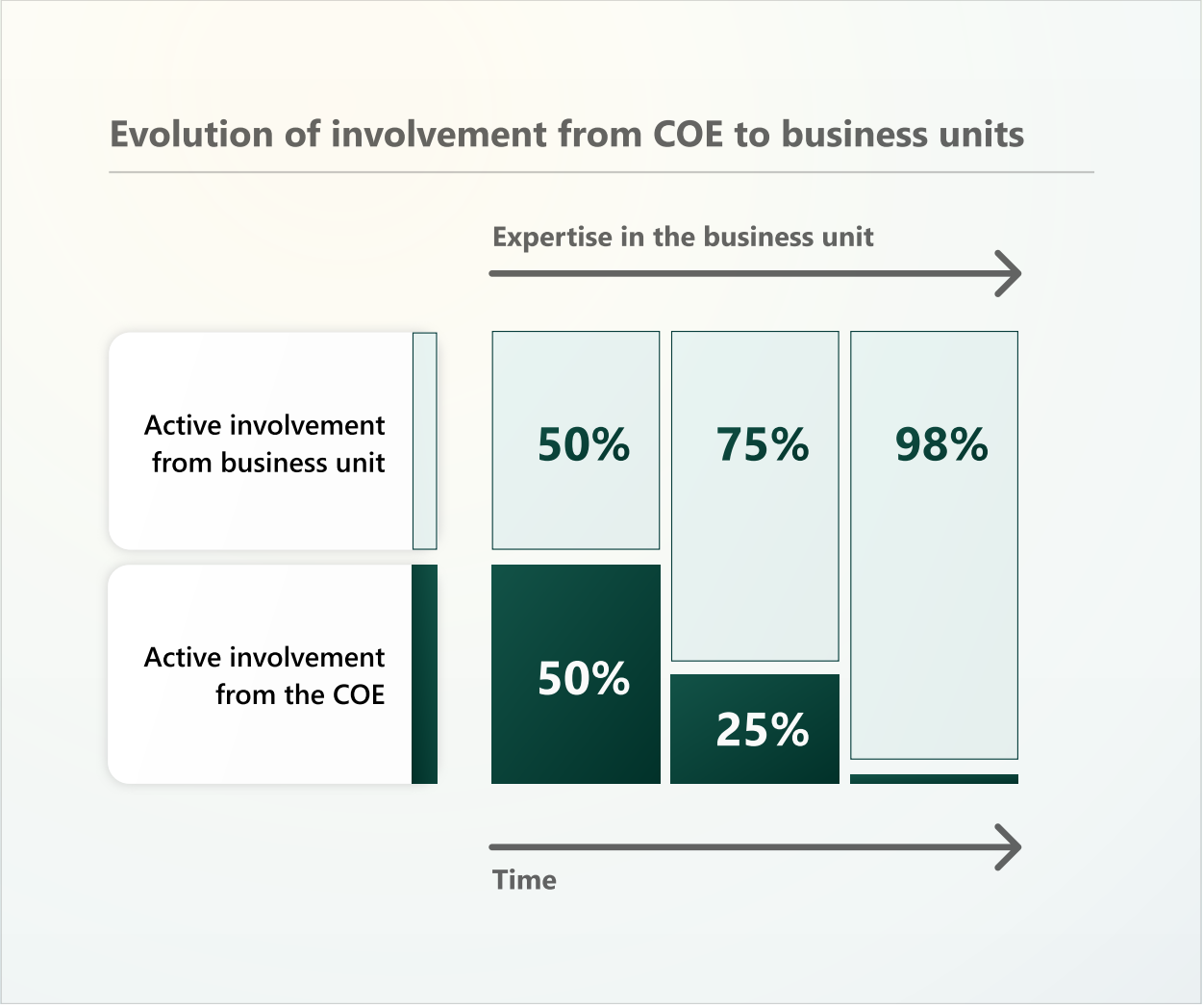
Co-development projects

One way the COE can provide mentoring services is during a *co-development project*. A co-development project is a form of assistance offered by the COE where a user or business unit takes advantage of the technical expertise of the COE to solve business problems with data. Co-development involves stakeholders from the business unit and the COE working in partnership to build a high-quality self-service analytics or business intelligence (BI) solution that the business stakeholders couldn't deliver independently.

The goal of co-development is to help the business unit develop expertise over time while also delivering value. For example, the sales team has a pressing need to develop a new set of commission reports, but the sales team doesn't yet have the knowledge to complete it on their own.

A co-development project forms a partnership between the business unit and the COE. In this arrangement, the business unit is fully invested, deeply involved, and assumes ownership of the project.

Time involvement from the COE reduces over time until the business unit gains expertise and becomes self-reliant.



The active involvement shown in the above diagram changes over time, as follows:

- **Business unit**: 50% initially, up to 75%, finally at 98%-100%.
- **COE**: 50% initially, down to 25%, finally at 0%-2%.

Ideally, the period for the gradual reduction in involvement is identified up-front in the project. This way, both the business unit and the COE can sufficiently plan the timeline and staffing.

Co-development projects can deliver significant short- and long-term benefits. In the short term, the involvement from the COE can often result in a better-designed and better-performing solution that follows best practices and aligns with organizational standards. In the long term, co-development helps increase the knowledge and capabilities of the business stakeholder, making them more self-sufficient, and more confident to deliver quality self-service data and BI solutions in the future.

> **Important**
>
> Essentially, a co-development project helps less experienced users learn the right way to do things. It reduces the risk that refactoring might be needed later, and it increases the ability for a solution to scale and grow over time.

Best practices reviews

The COE could also offer *best practices reviews*. A best practices review can be extremely helpful for content creators who would like to validate their work. They might also be known as *advisory services*, *internal consulting time*, or *technical reviews*. Unlike a co-development project (described previously), a best practices review occurs after the solution has been developed.

During a review, an expert from the COE evaluates self-service Fabric content developed by a member of the community and identifies areas of risk or opportunities for improvement.

Here are some examples of when a best practices review could be beneficial.

- The sales team has a Power BI app that they intend to distribute to thousands of users throughout the organization. Since the app represents high priority content distributed to a large audience, they'd like to have it certified. The standard process to certify content includes a best practices review.
- The finance team would like to assign a workspace to a capacity. A review of the workspace content is required to ensure sound development practices are followed. This type of review is common when the capacity is shared among

multiple business units. (A review might not be required when the capacity is assigned to only one business unit.)
- The operations team is creating a new Fabric solution they expect to be widely used. They would like to request a best practices review before it goes into user acceptance testing (UAT), or before a request is submitted to the change management board.

A best practices review is most often focused on the semantic model design, though the review can encompass all types of data items (such as a lakehouse, data warehouse, data pipeline, dataflow, or semantic model). The review can also encompass reporting items (such as reports, dashboards, or metrics).

Before content is deployed, a best practices review can be used to verify other design decisions, like:

- Code in notebooks follows organizational standards and best practices.
- The appropriate data preparation approach (dataflows, pipelines, notebooks, and others) are used where needed.
- Data sources used are appropriate and query folding is invoked whenever possible where Power Query and dataflows are used.
- Data preparation steps are clean, orderly, and efficient.
- Connectivity mode and storage mode choices (for example, Direct Lake, import, live connection, DirectQuery, and composite model frameworks) are appropriate.
- Location for data sources, like flat files, and original Power BI Desktop files are suitable (preferably stored in a backed-up location with versioning and appropriate security, such as Teams files or a SharePoint shared library).
- Semantic models are well-designed, clean, and understandable, and use a star schema design.
- Model relationships are configured correctly.
- DAX calculations use efficient coding practices (particularly if the data model is large).
- The semantic model size is within a reasonable limit and data reduction techniques are applied.
- Row-level security (RLS) appropriately enforces data permissions.
- Data is accurate and has been validated against the authoritative source(s).
- Approved common definitions and terminology are used.
- Good data visualization practices are followed, including designing for accessibility.

Once the content has been deployed, the best practices review isn't necessarily complete yet. Completing the remainder of the review could also include items such as:

- The target workspace is suitable for the content.
- Workspace security roles are appropriate for the content.
- Other permissions (such as app audience permissions, Build permission, or use of the individual item sharing feature) are correctly and appropriately configured.
- Contacts are identified, and correctly correlate to the owners of the content.
- Sensitivity labels are correctly assigned.
- Fabric item endorsement (certified or promoted) is appropriate.
- Data refresh is configured correctly, failure notifications include the proper users, and uses the appropriate data gateway in standard mode (if applicable).
- All appropriate semantic model best practices rules are followed and, preferably, are automated via a community tool called Best Practices Analyzer for maximum efficiency and productivity.

Extended support

From time to time, the COE might get involved with complex issues escalated from the help desk. For more information, see the User support article.

> **Note**
>
> Offering mentoring services might be a culture shift for your organization. Your reaction might be that users don't usually ask for help with a tool like Excel, so why would they with Power BI? The answer lies in the fact that Power BI and Fabric are extraordinarily powerful tools. They provide data preparation and data modeling capabilities in addition to data visualization. Having the ability to aid and enable users can significantly improve their skills and increase the quality of their solutions—it reduces risks too.

Centralized portal

A single centralized portal, or hub, is where the user community can find:

- Access to the community Q&A forum.
- Announcements of interest to the community, such as new features and release plan updates.
- Schedules and registration links for office hours, lunch and learns, training sessions, and user group meetings.
- Announcements of key changes to content and change log (if appropriate).
- How to request help or support.
- Training materials.

- Documentation, onboarding materials, and frequently asked questions (FAQ).
- Governance guidance and approaches recommended by the COE.
- Report templates.
- Examples of best practices solutions.
- Recordings of knowledge sharing sessions.
- Entry points for accessing managed processes, such as license acquisition, access requests, and gateway configuration.

> **Tip**
>
> In general, only 10%-20% of your community will go out of their way to actively seek out training and educational information. These types of users might naturally evolve to become your **champions**. Everyone else is usually just trying to get the job done as quickly as possible, because their time, focus, and energy are needed elsewhere. Therefore, it's crucial to make information easy for your community users to find.

The goal is to consistently direct users in the community to the centralized portal to find information. The corresponding obligation for the COE is to ensure that the information users need is available in the centralized portal. Keeping the portal updated requires discipline when everyone is busy.

In larger organizations, it can be difficult to implement one single centralized portal. When it's not practical to consolidate into a single portal, a centralized hub can serve as an aggregator, which contains links to the other locations.

> **Important**
>
> Although saving time finding information is important, the goal of a centralized portal is more than that. It's about making information readily available to help your user community do the right thing. They should be able to find information during their normal course of work, with as little friction as possible. Until it's easier to complete a task within the guardrails established by the COE and data governance team, some users will continue to complete their tasks by circumventing policies that are put in place. The recommended path must become the path of least resistance. Having a centralized portal can help achieve this goal.

It takes time for community users to think of the centralized portal as their natural first stop for finding information. It takes consistent redirection to the portal to change habits. Sending someone a link to an original document location in the portal builds

better habits than, for instance, including the answer in an email response. It's the same challenge described in the User support article.

Training

A key factor for successfully enabling self-service users in a Fabric community is training. It's important that the right training resources are readily available and easily discoverable. While some users are so enthusiastic about analytics that they'll find information and figure things out on their own, it isn't true for most of the user community.

Making sure your self-service users (particularly content creators and owners) have access to the training resources they need to be successful doesn't mean that you need to develop your own training content. Developing training content is often counterproductive due to the rapidly evolving nature of the product. Fortunately, an abundance of training resources is available in the worldwide community. A curated set of links goes a long way to help users organize and focus their training efforts, especially for tool training, which focuses on the technology. All external links should be validated by the COE for accuracy and credibility. It's a key opportunity for the COE to add value because COE stakeholders are in an ideal position to understand the learning needs of the community, and to identify and locate trusted sources of quality learning materials.

You'll find the greatest return on investment with creating custom training materials for *organizational-specific* processes, while relying on content produced by others for everything else. It's also useful to have a short training class that focuses primarily on topics like how to find documentation, getting help, and interacting with the community.

> **Tip**
>
> One of the goals of training is to help users learn new skills while helping them avoid bad habits. It can be a balancing act. For instance, you don't want to overwhelm new users by adding in a lot of complexity and friction to a beginner-level class for report creators. However, it's a great investment to make newer content creators aware of things that could otherwise take them a while to figure out. An ideal example is teaching the ability to use a **live connection** to report from an existing semantic model. By teaching this concept at the earliest logical time, you can save a less experienced creator thinking they always need one semantic model for every report (and encourage the good habit of reusing existing semantic models across reports).

Some larger organizations experience continual employee transfers and turnover. Such frequent change results in an increased need for a repeatable set of training resources.

Training resources and approaches

There are many training approaches because people learn in different ways. If you can monitor and measure usage of your training materials, you'll learn over time what works best.

Some training might be delivered more formally, such as classroom training with hands-on labs. Other types of training are less formal, such as:

- Lunch and learn presentations
- Short how-to videos targeted to a specific goal
- Curated set of online resources
- Internal user group presentations
- One-hour, one-week, or one-month challenges
- Hackathon-style events

The advantages of encouraging knowledge sharing among colleagues are described in the Community of practice article.

> **Tip**
>
> Whenever practical, learning should be correlated with building something meaningful and realistic. However, simple demo data does have value during a training course. It allows a learner to focus on how to use the technology rather than the data itself. After completion of introductory session(s), consider offering a *bring your own data* type of session. These types of sessions encourage the learner to apply their new technical skills to an actual business problem. Try to include multiple facilitators from the COE during this type of follow-up session so questions can be answered quickly.

The types of users you might target for training include:

- Content owners, subject matter experts (SMEs), and workspace administrators
- Data creators (for example, users who create semantic models for report creators to use, or who create dataflows, lakehouses, or warehouses for other semantic model creators to use)
- Report creators
- Content consumers and viewers
- Satellite COE members and the champions network

- Fabric administrators

> **Important**
>
> Each type of user represents a different audience that has different training needs. The COE will need to identify how best to meet the needs of each audience. For instance, one audience might find a standard introductory Power BI Desktop class overwhelming, whereas another will want more challenging information with depth and detail for end-to-end solutions that include multiple Fabric workloads. If you have a diverse population of Fabric content creators, consider creating personas and tailoring the experience to an extent that's practical.

The completion of training can be a leading indicator for success with user adoption. Some organizations add an element of fun by granting badges, like *blue belt* or *black belt*, as users progress through the training programs.

Give some consideration to how you want to handle users at various stages of user adoption. Training needs are very different for:

- Onboarding new users (sometimes referred to as *training day zero*).
- Users with minimal experience.
- More experienced users.

How the COE invests its time in creating and curating training materials will change over time as adoption and maturity grows. You might also find over time that some community champions want to run their own tailored set of training classes within their functional business unit.

Sources for trusted Fabric training content

A curated set of online resources is valuable to help community members focus and direct their efforts on what's important. Some publicly available training resources you might find helpful include:

- Microsoft Learn training for Power BI
- Microsoft Learn training for Fabric
- Power BI courses and "in a day" training materials
- LinkedIn Learning for Power BI
- LinkedIn Learning for Fabric

Consider using Microsoft Viva Learning, which is integrated into Microsoft Teams. It includes content from sources such as Microsoft Learn and LinkedIn Learning. Custom

content produced by your organization can be included as well.

In addition to Microsoft content and custom content produced by your organization, you might choose to provide your user community with a curated set of recommended links to trusted online sources. There's a wide array of videos, blogs, and articles produced by the worldwide community. The community comprises Fabric and Power BI experts, Microsoft Most Valued Professions (MVPs), and enthusiasts. Providing a curated learning path that contains specific, reputable, current, and high-quality resources will provide the most value to your user community.

If you do make the investment to create custom in-house training, consider creating short, targeted content that focuses on solving one specific problem. It makes the training easier to find and consume. It's also easier to maintain and update over time.

> **Tip**
>
> The *Help and Support* menu in the Fabric portal is customizable. When your centralized location for training documentation is operational, update the **tenant setting in the Admin portal** with the link. The link can then be accessed from menu when users select the *Get Help* option. Also, be sure to teach users about the *Help* ribbon tab in Power BI Desktop. It includes links to guided learning, training videos, documentation, and more.

Documentation

Concise, well-written documentation can be a significant help for users trying to get things done. Your needs for documentation, and how it's delivered, will depend on how Fabric is managed in your organization. For more information, see the Content ownership and management article.

Certain aspects of Fabric tend to be managed by a centralized team, such as the COE. The following types of documentation are helpful in these situations:

- How to request a Power BI license (and whether there are requirements for manager approval)
- How to request a new capacity
- How to request a new workspace
- How to request a workspace be added to an existing capacity
- How to request access to a gateway data source
- How to request software installation

> **Tip**
>
> For certain activities that are repeated over and over, consider automating them using Power Apps and Power Automate. In this case, your documentation will also include how to access and use the Power Platform functionality.

Different aspects of your documentation can be managed by self-service users, decentralized teams, or by a centralized team. The following types of documentation might differ based on who owns and manages the content:

- How to request a new report
- How to request a report enhancement
- How to request access to data
- How to request new data be prepared and made available for use
- How to request an enhancement to existing data or visualizations

> **Tip**
>
> When planning for a centralized portal, as described earlier in this article, plan how to handle situations when guidance or governance policies need to be customized for one or more business units.

There are also going to be some governance decisions that have been made and should be documented, such as:

- How to request content be certified
- What are the approved file storage locations
- What are the data retention and purge requirements
- What are the requirements for handling sensitive data and personally identifiable information (PII)

Documentation should be located in your centralized portal, which is a searchable location where, preferably, users already work. Either Teams or SharePoint work very well. Creating documentation in either wiki pages or in documents can work equally well, provided that the content is organized well and is easy to find. Shorter documents that focus on one topic are usually easier to consume than long, comprehensive documents.

> **Important**

One of the most helpful pieces of documentation you can publish for the community is a description of the **tenant settings**, and the group memberships required for each tenant setting. Users read about features and functionality online, and sometimes find that it doesn't work for them. When they are able to quickly look up your organization's tenant settings, it can save them from becoming frustrated and attempting workarounds. Effective documentation can reduce the number of help desk tickets that are submitted. It can also reduce the number of people who need to be assigned the Fabric administrator role (who might have this role solely for the purpose of viewing settings).

Over time, you might choose to allow certain types of documentation to be maintained by the community if you have willing volunteers. In this case, you might want to introduce an approval process for changes.

When you see questions repeatedly arise in the Q&A forum (as described in the User support article), during office hours, or during lunch and learns, it's a great indicator that creating new documentation might be appropriate. When the documentation exists, it allows colleagues to reference it when needed. Documentation contributes to user enablement and a self-sustaining community.

Tip

When creating custom documentation or training materials, reference existing Microsoft sites using links whenever possible. Most community bloggers don't keep blog posts or videos up to date.

Power BI template files

A Power BI template is a .pbit file. It can be provided as a starting point for content creators. It's the same as a .pbix file, which can contain queries, a data model, and a report, but with one exception: the template file doesn't contain any data. Therefore, it's a smaller file that can be shared with content creators and owners, and it doesn't present a risk of inappropriately sharing data.

Providing Power BI template files for your community is a great way to:

- Promote consistency.
- Reduce learning curve.
- Show good examples and best practices.
- Increase efficiency.

Power BI template files can improve efficiency and help people learn during the normal course of their work. A few ways that template files are helpful include:

- Reports can use examples of good visualization practices
- Reports can incorporate organizational branding and design standards
- Semantic models can include the structure for commonly used tables, like a date table
- Helpful DAX calculations can be included, like a year-over-year (YoY) calculation
- Common parameters can be included, like a data source connection string
- An example of report and/or semantic model documentation can be included

> ⓘ **Note**
>
> Providing templates not only saves your content creators time, it also helps them move quickly beyond a blank page in an empty solution.

Power BI project files

A Power BI project is a .pbip file. Like a template file (previously described), a project file doesn't contain any data. It's a file format that advanced content creators can use for advanced data model and report management scenarios. For example, you can use project files to save time in development by sharing common model patterns, like date tables, DAX measure expressions, or calculation groups.

You can use Power BI project files with Power BI Desktop developer mode for:

- Advanced editing and authoring (for example, in a code editor such as Visual Studio Code).
- Purposeful separation of semantic model and report items (unlike the .pbix or .pbit files).
- Enabling multiple content creators and developers to work on the same project concurrently.
- Integrating with source control (such as by using Fabric Git integration).
- Using continuous integration and continuous delivery (CI/CD) techniques to automate integration, testing and deployment of changes, or versions of content.

> ⓘ **Note**
>
> Power BI includes capabilities such as .pbit template files and .pbip project files that make it simple to share starter resources with authors. Other Fabric workloads provide different approaches to content development and sharing. Having a set of

starter resources is important regardless of the items being shared. For example, your portal might include a set of SQL scripts or notebooks that present tested approaches to solve common problems.

Considerations and key actions

Checklist - Considerations and key actions you can take to establish, or improve, mentoring and user enablement.

- ✓ **Consider what mentoring services the COE can support**: Decide what types of mentoring services the COE is capable of offering. Types can include office hours, co-development projects, and best practices reviews.
- ✓ **Communicate regularly about mentoring services**: Decide how you will communicate and advertise mentoring services, such as office hours, to the user community.
- ✓ **Establish a regular schedule for office hours**: Ideally, hold office hours at least once per week (depending on demand from users as well as staffing and scheduling constraints).
- ✓ **Decide what the expectations will be for office hours**: Determine what the scope of allowed topics or types of issues users can bring to office hours. Also, determine how the queue of office hours requests will work, whether any information should be submitted ahead of time, and whether any follow up afterwards can be expected.
- ✓ **Create a centralized portal**: Ensure that you have a well-supported centralized hub where users can easily find training materials, documentation, and resources. The centralized portal should also provide links to other community resources such as the Q&A forum and how to find help.
- ✓ **Create documentation and resources**: In the centralized portal, create, compile, and publish useful documentation. Identify and promote the top 3-5 resources that will be most useful to the user community.
- ✓ **Update documentation and resources regularly**: Ensure that content is reviewed and updated on a regular basis. The objective is to ensure that the information available in the portal is current and reliable.
- ✓ **Compile a curated list of reputable training resources**: Identify training resources that target the training needs and interests of your user community. Post the list in the centralized portal and create a schedule to review and validate the list.

- ✓ **Consider whether custom in-house training will be useful**: Identify whether custom training courses, developed in-house, will be useful and worth the time investment. Invest in creating content that's specific to the organization.
- ✓ **Provide templates and projects**: Determine how you'll use templates including Power BI template files and Power BI project files. Include the resources in your centralized portal, and in training materials.
- ✓ **Create goals and metrics**: Determine how you'll measure effectiveness of the mentoring program. Create KPIs (key performance indicators) or OKRs (objectives and key results) to validate that the COE's mentoring efforts strengthen the community and its ability to provide self-service BI.

Questions to ask

Use questions like those found below to assess mentoring and user enablement.

- Is there an effective process in place for users to request training?
- Is there a process in place to evaluate user skill levels (such as beginner, intermediate, or advanced)? Can users study for and achieve Microsoft certifications by using company resources?
- What's the onboarding process to introduce new people in the user community to data and BI solutions, tools, and processes?
- Have all users followed the appropriate Microsoft Learn learning paths for their roles during onboarding?
- What kinds of challenges do users experience due to lack of training or mentorship?
- What impact does lack of enablement have on the business?
- When users exhibit behavior that creates governance risks, are they punished or do they undergo education and mentorship?
- What training materials are in place to educate people about governance processes and policies?
- Where's the central documentation maintained? Who maintains it?
- Do central resources exist, like organizational design guidelines, themes, or template files?

Maturity levels

The following maturity levels will help you assess the current state of your mentoring and user enablement.

Expand table

Level	State of mentoring and user enablement
100: Initial	• Some documentation and resources exist. However, they're siloed and inconsistent. • Few users are aware of, or take advantage of, available resources.
200: Repeatable	• A centralized portal exists with a library of helpful documentation and resources. • A curated list of training links and resources are available in the centralized portal. • Office hours are available so the user community can get assistance from the COE.
300: Defined	• The centralized portal is the primary hub for community members to locate training, documentation, and resources. The resources are commonly referenced by champions and community members when supporting and learning from each other. • The COE's skills mentoring program is in place to assist users in the community in various ways.
400: Capable	• Office hours have regular and active participation from all business units in the organization. • Best practices reviews from the COE are regularly requested by business units. • Co-development projects are repeatedly executed with success by the COE and members of business units.
500: Efficient	• Training, documentation, and resources are continually updated and improved by the COE to ensure the community has current and reliable information. • Measurable and tangible business value is gained from the mentoring program by using KPIs or OKRs.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about the community of practice.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

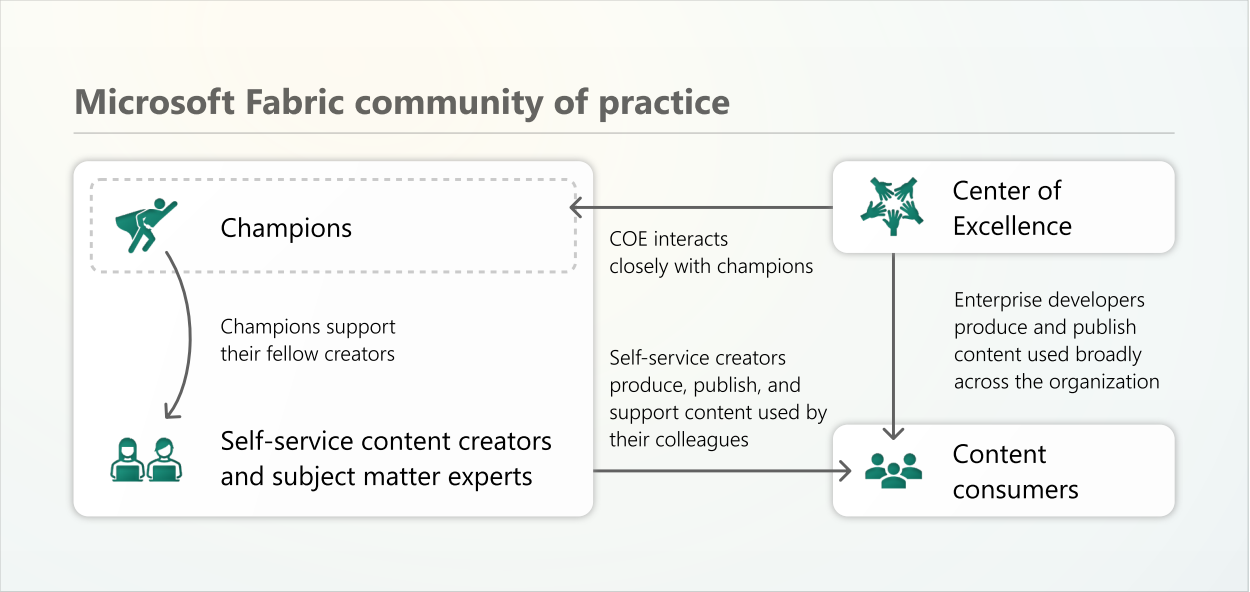
Microsoft Fabric adoption roadmap: Community of practice

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

A *community of practice* is a group of people with a common interest that interacts with, and helps, each other on a voluntary basis. Using a tool such as Microsoft Fabric to produce effective analytics is a common interest that can bring people together across an organization.

The following diagram provides an overview of an internal community.



The above diagram shows the following:

- The **community of practice** includes everyone with an interest in Fabric.
- The **Center of Excellence (COE)** forms the nucleus of the community. The COE oversees the entire community and interacts most closely with its champions.
- **Self-service content creators and subject matter experts (SMEs)** produce, publish, and support content that's used by their colleagues, who are consumers.
- **Content consumers** view content produced by both self-service creators and enterprise business intelligence (BI) developers.
- **Champions** are a subset of the self-service content creators. Champions are in an excellent position to support their fellow content creators to generate effective

analytics solutions.

Champions are the smallest group among creators and SMEs. Self-service content creators and SMEs represent a larger number of people. Content consumers represent the largest number of people in most organizations.

> **Note**
>
> All references to the Fabric community in this adoption series of articles refer to internal users, unless explicitly stated otherwise. There's an active and vibrant worldwide community of bloggers and presenters who produce a wealth of knowledge about Fabric. However, internal users are the focus of this article.

For information about related topics including resources, documentation, and training provided for the Fabric community, see the Mentoring and user enablement article.

Champions network

One important part of a community of practice is its *champions*. A champion is a self-service content creator who works in a business unit that engages with the COE. A champion is recognized by their peers as the *go-to* expert. A champion continually builds and shares their knowledge even if it's not an official part of their job role. Champions influence and help their colleagues in many ways including solution development, learning, skills improvement, troubleshooting, and keeping up to date.

Champions emerge as leaders of the community of practice who:

- Have a deep interest in analytics being used effectively and adopted successfully throughout the organization.
- Possess strong technical skills as well as domain knowledge for their functional business unit.
- Have an inherent interest in getting involved and helping others.
- Are early adopters who are enthusiastic about experimenting and learning.
- Can effectively translate business needs into solutions.
- Communicate well with colleagues.

> **Tip**
>
> To add an element of fun, some organizations refer to their champions network as *ambassadors*, *Jedis*, *ninjas*, or *rangers*. Microsoft has an internal community called *BI Champs*.

Often, people aren't directly asked to become champions. Commonly, champions are identified by the COE and recognized for the activities they're already doing, such as frequently answering questions on an internal discussion channel or participating in lunch and learn sessions.

Different approaches will be more effective for different organizations, and each organization will find what works best for them as their maturity level increases.

> **Important**
>
> Someone very well might be acting in the role of a champion without even knowing it, and without any formal recognition. The COE should always be on the lookout for champions. COE members should actively monitor the discussion channel to see who is particularly helpful. The COE should deliberately encourage and support potential champions, and when appropriate, invite them into a champions network to make the recognition formal.

Knowledge sharing

The overriding objective of a community of practice is to facilitate knowledge sharing among colleagues and across organizational boundaries. There are many ways knowledge sharing occurs. It could be during the normal course of work. Or, it could be during a more structured activity, such as:

Expand table

Activity	Description
Discussion channel	A Q&A forum where anyone in the community can post and view messages. Often used for help and announcements. For more information, see the User support article.
Lunch and learn sessions	Regularly scheduled sessions where someone presents a short session about something they've learned or a solution they've created. The goal is to get a variety of presenters involved, because it's a powerful message to hear firsthand what colleagues have achieved.
Office hours with the COE	Regularly scheduled times when COE experts are available so the community can engage with them. Community users can receive assistance with minimal process overhead. For more information, see the Mentoring and user enablement article.
Internal blog posts or wiki posts	Short blog posts, usually covering technical how-to topics.

Activity	Description
Internal analytics user group	A subset of the community that chooses to meet as a group on a regularly scheduled basis. User group members often take turns presenting to each other to share knowledge and improve their presentation skills.
Book club	A subset of the community select a book to read on a schedule. They discuss what they've learned and share their thoughts with each other.
Internal analytics conferences or events	An annual or semi-annual internal conference that delivers a series of sessions focused on the needs of self-service content creators, subject matter experts, and stakeholders.

> **Tip**
>
> Inviting an external presenter can reduce the effort level and bring a fresh viewpoint for learning and knowledge sharing.

Incentives

A lot of effort goes into forming and sustaining a successful community. It's advantageous to everyone to empower and reward users who work for the benefit of the community.

Rewarding community members

Incentives that the entire community (including champions) find particularly rewarding can include:

- **Contests with a small gift card or time off**: For example, you might hold a performance tuning event with the winner being the person who successfully reduced the size of their data model the most.
- **Ranking based on help points**: The more frequently someone participates in Q&A, they achieve a change in status on a leaderboard. This type of gamification promotes healthy competition and excitement. By getting involved in more conversations, the participant learns and grows personally in addition to helping their colleagues.
- **Leadership communication**: Reach out to a manager when someone goes above and beyond so that their leader, who might not be active in the community, sees the value that their staff member provides.

Rewarding champions

Different types of incentives will appeal to different types of people. Some community members will be highly motivated by praise and feedback. Some will be inspired by gamification and a bit of fun. Others will highly value the opportunity to improve their level of knowledge.

Incentives that champions find particularly rewarding can include:

- **More direct access to the COE**: The ability to have *connections* in the COE is valuable. It's depicted in the diagram shown earlier in this article.
- **Champion of the month**: Publicly thank one of your champions for something outstanding they did recently. It could be a fun tradition at the beginning of a monthly lunch and learn.
- **A private experts discussion area**: A private area for the champions to share ideas and learn from each other is usually highly valued.
- **Specialized or deep dive information and training**: Access to additional information to help champions grow their skillsets (as well as help their colleagues) will be appreciated. It could include attending advanced training classes or conferences.

Communication plan

Communication with the community occurs through various types of communication channels. Common communication channels include:

- Internal discussion channel or forum.
- Announcements channel.
- Organizational newsletter.

The most critical communication objectives include ensuring your community members know that:

- The COE exists.
- How to get help and support.
- Where to find resources and documentation.
- Where to find governance guidelines.
- How to share suggestions and ideas.

> **Tip**

Consider requiring a simple quiz before a user is granted a Power BI or Fabric license. This *quiz* is a misnomer because it doesn't focus on any technical skills. Rather, it's a short series of questions to verify that the user knows where to find help and resources. It sets them up for success. It's also a great opportunity to have users acknowledge any governance policies or data privacy and protection agreements you need them to be aware of. For more information, see the **System oversight** article.

Types of communication

There are generally four types of communication to plan for:

- **New employee communications** can be directed to new employees (and contractors). It's an excellent opportunity to provide onboarding materials for how to get started. It can include articles on topics like how to get Power BI Desktop installed, how to request a license, and where to find introductory training materials. It can also include general data governance guidelines that all users should be aware of.
- **Onboarding communications** can be directed to employees who are just acquiring a license or are getting involved with the community of practice. It presents an excellent opportunity to provide the same materials as given to new employee communications (as mentioned above).
- **Ongoing communications** can include regular announcements and updates directed to all users, or subsets of users, like:
  - Announcements about changes that are planned to key organizational content. For example, changes are to be published for a critical shared semantic model that's used heavily throughout the organization. It can also include the announcement of new features. For more information about planning for change, see the Tenant-level monitoring article.
  - Feature announcements, which are more likely to receive attention from the reader if the message includes meaningful context about why it's important. (Although an RSS feed can be a helpful technique, with the frequent pace of change, it can become noisy and might be ignored.)
- **Situational communications** can be directed to specific users or groups based on a specific occurrence discovered while monitoring the platform. For example, perhaps you notice a significant amount of sharing from the personal workspace a particular user, so you choose to send them some information about the benefits of workspaces and Power BI apps.

Tip

One-way communication to the user community is important. Don't forget to also include bidirectional communication options to ensure the user community has an opportunity to provide feedback.

Community resources

Resources for the internal community, such as documentation, templates, and training, are critical for adoption success. For more information about resources, see the Mentoring and user enablement article.

Considerations and key actions

Checklist - Considerations and key actions you can take for the community of practice follow.

Initiate, grow, and sustain your champions network:

- ✓ **Clarify goals**: Clarify what your specific goals are for cultivating a champions network. Make sure these goals align with your overall data and BI strategy, and that your executive sponsor is on board.
- ✓ **Create a plan for the champions network**: Although some aspects of a champions network will always be informally led, determine to what extent the COE will purposefully cultivate and support champion efforts throughout individual business units. Consider how many champions is ideal for each functional business area. Usually, 1-2 champions per area works well, but it can vary based on the size of the team, the needs of the self-service community, and how the COE is structured.
- ✓ **Decide on commitment level for champions**: Decide what level of commitment and expected time investment will be required of champions. Be aware that the time investment will vary from person to person, and team to team due to different responsibilities. Plan to clearly communicate expectations to people who are interested in getting involved. Obtain manager approval when appropriate.
- ✓ **Decide how to identify champions**: Determine how you will respond to requests to become a champion, and how the COE will seek out champions. Decide if you will openly encourage interested employees to self-identify as a champion and ask to learn more (less common). Or, whether the COE will observe efforts and extend a private invitation (more common).

- ✓ **Determine how members of the champions network will be managed**: One excellent option for managing who the champions are is with a security group. Consider:
  - How you will communicate with the champions network (for example, in a Teams channel, a Yammer group, and/or an email distribution list).
  - How the champions network will communicate and collaborate with each other directly (across organizational boundaries).
  - Whether a private and exclusive discussion forum for champions and COE members is appropriate.
- ✓ **Plan resources for champions**: Ensure members of the champions network have the resources they need, including:
  - Direct access to COE members.
  - Influence on data policies being implemented (for example, requirements for a semantic model certification policy).
  - Influence on the creation of best practices and guidance (for example, recommendations for accessing a specific source system).
- ✓ **Involve champions**: Actively involve certain champions as satellite members of the COE. For more information about ways to structure the COE, see the Center of Excellence article.
- ✓ **Create a feedback loop for champions**: Ensure that members of the champions network can easily provide information or submit suggestions to the COE.
- ✓ **Routinely provide recognition and incentives for champions**: Not only is praise an effective motivator, but the act of sharing examples of successful efforts can motivate and inspire others.

Improve knowledge sharing:

- ✓ **Identify knowledge sharing activities**: Determine what kind of activities for knowledge sharing fit well into the organizational data culture. Ensure that all planned knowledge sharing activities are supportable and sustainable.
- ✓ **Confirm roles and responsibilities**: Verify who will take responsibility for coordinating all knowledge sharing activities.

Introduce incentives:

- ✓ **Identify incentives for champions**: Consider what type of incentives you could offer to members of your champions network.
- ✓ **Identify incentives for community members**: Consider what type of incentives you could offer to your broader internal community.

Improve communications:

- ✓ **Establish communication methods**: Evaluate which methods of communication fit well in your data culture. Set up different ways to communicate, including history retention and search.
- ✓ **Identify responsibility**: Determine who will be responsible for different types of communication, how, and when.

Questions to ask

Use questions like those found below to assess the community of practice.

- Is there a centralized portal for a community of practice to engage in knowledge sharing?
- Do technical questions and requests for support always go through central teams like the COE or support? Alternatively, to what extent is the community of practice engaging in knowledge sharing?
- Do any incentives exist for people to engage in knowledge sharing or improve their skills with data and BI tools?
- Is there a system of recognition to acknowledge significant self-service efforts in teams?
- Are champions recognized among the user community? If so, what explicit recognition do they get for their expertise? How are they identified?
- If no champions are recognized, are there any potential candidates?
- What role do central teams envision that champions play in community of practice?
- How often do central data and BI teams engage with the user community? What medium do these interactions take? Are they bidirectional discussions or unidirectional communications?
- How are changes and announcements communicated within the community of practice?
- Among the user community, who is the most enthusiastic about analytics and BI tools? Who is the least enthusiastic, or the most negative, and why?

Maturity levels

The following maturity levels will help you assess the current state of your community of practice.

Expand table

Level	State of the community of practice
100: Initial	• Some self-service content creators are doing great work throughout the organization. However, their efforts aren't recognized. • Efforts to purposefully share knowledge across the organizational boundaries are rare and unstructured. • Communication is inconsistent, without a purposeful plan.
200: Repeatable	• The first set of champions are identified. • The goals for a champions network are identified. • Knowledge sharing practices are gaining traction.
300: Defined	• Knowledge sharing in multiple forms is a normal occurrence. Information sharing happens frequently and purposefully. • Goals for transparent communication with the user community are defined.
400: Capable	• Champions are identified for all business units. They actively support colleagues in their self-service efforts. • Incentives to recognize and reward knowledge sharing efforts are a common occurrence. • Regular and frequent communication occurs based on a predefined communication plan.
500: Efficient	• Bidirectional feedback loops exist between the champions network and the COE. • Key performance indicators measure community engagement and satisfaction. • Automation is in place when it adds direct value to the user experience (for example, automatic access to a group that provides community resources).

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about user support.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: User support

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

This article addresses user support. It focuses primarily on the resolution of issues.

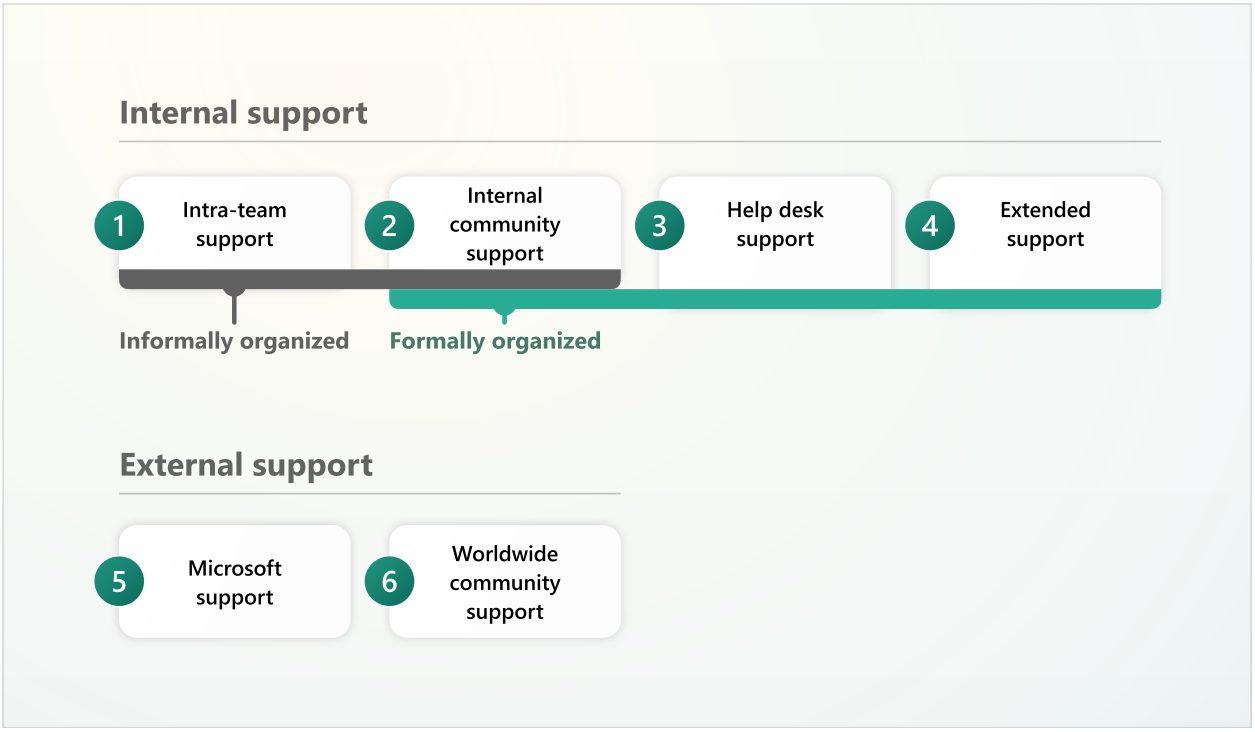
The first sections of this article focus on user support aspects you have control over internally within your organization. The final topics focus on external resources that are available.

For a description of related topics, including skills mentoring, training, documentation, and co-development assistance provided to the internal Fabric user community, see the Mentoring and user enablement article. The effectiveness of those activities can significantly reduce the volume of formal user support requests and increase user experience overall.

Types of user support

If a user has an issue, do they know what their options are to resolve it?

The following diagram shows some common types of user support that organizations employ successfully:



The six types of user support shown in the above diagram include:

Expand table

Type	Description
1	**Intra-team support (internal)** is very informal. Support occurs when team members learn from each other during the natural course of their job.
2	**Internal community support (internal)** can be organized informally, formally, or both. It occurs when colleagues interact with each other via internal community channels.
3	**Help desk support (internal)** handles formal support issues and requests.
4	**Extended support (internal)** involves handling complex issues escalated by the help desk.
5	**Microsoft support (external)** includes support for licensed users and Fabric administrators. It also includes comprehensive documentation.
6	**Community support (external)** includes the worldwide community of experts, Microsoft Most Valued Professionals (MVPs), and enthusiasts who participate in forums and publish content.

In some organizations, intra-team and internal community support are most relevant for self-service data and business intelligence (BI)—content is owned and managed by creators and owners in decentralized business units. Conversely, the help desk and extended support are reserved for technical issues and enterprise data and BI (content is owned and managed by a centralized BI team or Center of Excellence). In some organizations, all four types of support could be relevant for any type of content.

> **Tip**
>
> For more information about business-led self-service, managed self-service, and enterprise data and BI concepts, see the **Content ownership and management** article.

Each of the six types of user support introduced above are described in further detail in this article.

Intra-team support

Intra-team support refers to when team members learn from and help each other during their daily work. Self-service content creators who emerge as your champions tend to take on this type of informal support role voluntarily because they have an intrinsic desire to help. Although it's an informal support mode, it shouldn't be undervalued. Some estimates indicate that a large percentage of learning at work is peer learning, which is particularly helpful for analysts who are creating domain-specific analytics solutions.

> **Note**
>
> Intra-team support does not work well for individuals who are the only data analyst within a department. It's also not effective for those who don't have very many connections yet in their organization. When there aren't any close colleagues to depend on, other types of support, as described in this article, become more important.

Internal community support

Assistance from your fellow community members often takes the form of messages in a discussion channel, or a forum set up specifically for the community of practice. For example, someone posts a message that they're having problems getting a DAX calculation to work or are looking for the right Python module to import. They then receive a response from someone in the organization with suggestions or links.

> **Tip**
>
> The goal of an internal Fabric community is to be self-sustaining, which can lead to reduced formal support demands and costs. It can also facilitate managed self-

service content creation occurring on a broader scale versus a purely centralized approach. However, there will always be a need to monitor, manage, and nurture the internal community. Here are two specific tips:

- Be sure to cultivate multiple experts in the more difficult topics like **T-SQL**, **Python**, **Data Analysis eXpressions (DAX)** and the **Power Query M formula language**. When a community member becomes a recognized expert, they could become overburdened with too many requests for help.
- A greater number of community members might readily answer certain types of questions (for example, report visualizations), whereas a smaller number of members will answer others (for example, complex T-SQL or DAX). It's important for the COE to allow the community a chance to respond yet also be willing to promptly handle unanswered questions. If users repeatedly ask questions and don't receive an answer, it will significantly hinder growth of the community. In this case, a user is likely to leave and never return if they don't receive any responses to their questions.

An internal community discussion channel is commonly set up as a Teams channel or a Yammer group. The technology chosen should reflect where users already work, so that the activities occur within their natural workflow.

One benefit of an internal discussion channel is that responses can come from people that the original requester has never met before. In larger organizations, a community of practice brings people together based on a common interest. It can offer diverse perspectives for getting help and learning in general.

Use of an internal community discussion channel allows the Center of Excellence (COE) to monitor the kind of questions people are asking. It's one way the COE can understand the issues users are experiencing (commonly related to content creation, but it could also be related to consuming content).

Monitoring the discussion channel can also reveal additional analytics experts and potential champions who were previously unknown to the COE.

ⓘ Important

It's a best practice to continually identify emerging champions, and to engage with them to make sure they're equipped to support their colleagues. As described in the **Community of practice** article, the COE should actively monitor the discussion channel to see who is being helpful. The COE should deliberately encourage and

support community members. When appropriate, invite them into the champions network.

Another key benefit of a discussion channel is that it's searchable, which allows other people to discover the information. It is, however, a change of habit for people to ask questions in an open forum rather than private messages or email. Be sensitive to the fact that some individuals aren't comfortable asking questions in such a public way. It openly acknowledges what they don't know, which might be embarrassing. This reluctance might reduce over time by promoting a friendly, encouraging, and helpful discussion channel.

Tip

You might be tempted to create a bot to handle some of the most common, straightforward questions from the community. A bot can work for uncomplicated questions such as "How do I request a license?" or "How do I request a workspace?" Before taking this approach, consider if there are enough routine and predictable questions that would make the user experience better rather than worse. Often, a well-created FAQ (frequently asked questions) works better, and it's faster to develop and easier to maintain.

Help desk support

The help desk is usually operated as a shared service, staffed by the IT department. Users who will likely rely on a more formal support channel include those who are:

- Less experienced users.
- Newer to the organization.
- Reluctant to post a message to the internal discussion community.
- Lacking connections and colleagues within the organization.

There are also certain technical issues that can't be fully resolved without IT involvement, like software installation and upgrade requests when machines are IT-managed.

Busy help desk personnel are usually dedicated to supporting multiple technologies. For this reason, the easiest types of issues to support are those which have a clear resolution and can be documented in a knowledgebase. For instance, software installation prerequisites or requirements to get a license.

Some organizations ask the help desk to handle only very simple break-fix issues. Other organizations have the help desk get involved with anything that is repeatable, like new

workspace requests, managing gateway data sources, or requesting a new capacity.

> **Important**
>
> Your Fabric governance decisions will directly impact the volume of help desk requests. For example, if you choose to limit **workspace creation permissions in the tenant settings**, it will result in users submitting help desk tickets. While it's a legitimate decision to make, you must be prepared to satisfy the request very quickly. Respond to this type of request within 1-4 hours, if possible. If you delay too long, users will use what they already have or find a way to work around your requirements. That might not be the ideal scenario. Promptness is critical for certain help desk requests. Consider that automation by using **Power Apps** and **Power Automate** can help make some processes more efficient. For more information, see **Tenant-level workspace planning**.

Over time, troubleshooting and problem resolution skills become more effective as help desk personnel expand their knowledgebase and experience with supporting Fabric. The best help desk personnel are those who have a good grasp of what users need to accomplish.

> **Tip**
>
> Purely technical issues, for example **data refresh** failure or the need to **add a new user to a gateway data source**, usually involve straightforward responses associated with a service-level agreement (SLA). For instance, there could be an SLA to respond to blocking issues within one hour and resolve them within eight hours. It's generally more difficult to define SLAs for troubleshooting issues, like data discrepancies.

Extended support

Since the COE has deep insight into how Fabric is used throughout the organization, they're a great option for extended support should a complex issue arise. Involving the COE in the support process should be by an escalation path.

Managing requests as purely an escalation path from the help desk gets difficult to enforce since COE members are often well-known to business users. To encourage the habit of going through the proper channels, COE members should redirect users to submit a help desk ticket. It will also improve the data quality for analyzing help desk requests.

Microsoft support

In addition to the internal user support approaches discussed in this article, there are valuable external support options directly available to users and Fabric administrators that shouldn't be overlooked.

Microsoft documentation

Check the Fabric support website for high-priority issues that broadly affect all customers. Global Microsoft 365 administrators have access to additional support issue details within the Microsoft 365 portal.

Refer to the comprehensive Fabric documentation. It's an authoritative resource that can aid troubleshooting and search for information. You can prioritize results from the documentation site. For example, enter a site-targeted search request into your web search engine, like `power bi gateway site:learn.microsoft.com`.

Power BI Pro and Premium Per User end-user support

Licensed users are eligible to log a support ticket with Microsoft.

> **Tip**
>
> Make it clear to your internal user community whether you prefer technical issues to be reported to the internal help desk. If your help desk is equipped to handle the workload, having a centralized internal area collect user issues can provide a superior user experience versus every user trying to resolve issues on their own. Having visibility and analyzing support issues is also helpful for the **COE**.

Administrator support

There are several support options available for Fabric administrators.

For customers who have a Microsoft Unified Support contract, consider granting help desk and COE members access to the Microsoft Services Hub. One advantage of the Microsoft Services Hub is that your help desk and COE members can be set up to submit and view support requests.

Worldwide community support

In addition to the internal user support approaches described in this article, and Microsoft support options described previously, you can leverage the worldwide Fabric community.

The worldwide community is useful when a question can be easily understood by someone without domain knowledge, and when it doesn't involve confidential data or sensitive internal processes.

Publicly available community forums

There are several public community forums where users can post issues and receive responses from any user in the world. Getting answers from anyone, anywhere, can be very powerful and exceedingly helpful. However, as is the case with any public forum, it's important to validate the advice and information posted on the forum. The advice posted on the internet might not be suitable for your situation.

Publicly available discussion areas

It's very common to see people posting Fabric technical questions on social media platforms. You might find discussions, post announcements, and users helping each other.

Community documentation

The Fabric global community is vibrant. Every day, there are a great number of Fabric blog posts, articles, webinars, and videos published. When relying on community information for troubleshooting, watch out for:

- How recent the information is. Try to verify when it was published or last updated.
- Whether the situation and context of the solution found online truly fits your circumstance.
- The credibility of the information being presented. Rely on reputable blogs and sites.

Considerations and key actions

Checklist - Considerations and key actions you can take for user support follow.

Improve your intra-team support:

- ✓ **Provide recognition and encouragement**: Provide incentives to your champions as described in the Community of practice article.
- ✓ **Reward efforts**: Recognize and praise meaningful grassroots efforts when you see them happening.
- ✓ **Create formal roles**: If informal intra-team efforts aren't adequate, consider formalizing the roles you want to enact in this area. Include the expected contributions and responsibilities in the HR job description, when appropriate.

Improve your internal community support:

- ✓ **Continually encourage questions**: Encourage users to ask questions in the designated community discussion channel. As the habit builds over time, it will become normalized to use that as the first option. Over time, it will evolve to become more self-supporting.
- ✓ **Actively monitor the discussion area**: Ensure that the appropriate COE members actively monitor this discussion channel. They can step in if a question remains unanswered, improve upon answers, or make corrections when appropriate. They can also post links to additional information to raise awareness of existing resources. Although the goal of the community is to become self-supporting, it still requires dedicated resources to monitor and nurture it.
- ✓ **Communicate options available**: Make sure your user population knows the internal community support area exists. It could include the prominent display of links. You can include a link in regular communications to your user community. You can also customize the help menu links in the Fabric portal to direct users to your internal resources.
- ✓ **Set up automation**: Ensure that all licensed users automatically have access to the community discussion channel. It's possible to automate license setup by using group-based licensing.

Improve your internal help desk support:

- ✓ **Determine help desk responsibilities**: Decide what the initial scope of Fabric support topics that the help desk will handle.
- ✓ **Assess the readiness level**: Determine whether your help desk is prepared to handle Fabric support. Identify whether there are readiness gaps to be addressed.
- ✓ **Arrange for additional training**: Conduct knowledge transfer sessions or training sessions to prepare the help desk staff.
- ✓ **Update the help desk knowledgebase**: Include known questions and answers in a searchable knowledgebase. Ensure someone is responsible for regular updates to the knowledgebase to reflect new and enhanced features over time.

- ✓ **Set up a ticket tracking system**: Ensure a good system is in place to track requests submitted to the help desk.
- ✓ **Decide whether anyone will be on-call for any issues related to Fabric**: If appropriate, ensure the expectations for 24/7 support are clear.
- ✓ **Determine what SLAs will exist**: When a specific service level agreement (SLA) exists, ensure that expectations for response and resolution are clearly documented and communicated.
- ✓ **Be prepared to act quickly**: Be prepared to address specific common issues extremely quickly. Slow support response will result in users finding workarounds.

Improve your internal COE extended support:

- ✓ **Determine how escalated support will work**: Decide what the escalation path will be for requests the help desk cannot directly handle. Ensure that the COE (or equivalent personnel) is prepared to step in when needed. Clearly define where help desk responsibilities end, and where COE extended support responsibilities begin.
- ✓ **Encourage collaboration between COE and system administrators**: Ensure that COE members and Fabric administrators have a direct escalation path to reach global administrators for Microsoft 365 and Azure. It's critical to have a communication channel when a widespread issue arises that's beyond the scope of Fabric.
- ✓ **Create a feedback loop from the COE back to the help desk**: When the COE learns of new information, the IT knowledgebase should be updated. The goal is for the primary help desk personnel to continually become better equipped at handling more issues in the future.
- ✓ **Create a feedback loop from the help desk to the COE**: When support personnel observe redundancies or inefficiencies, they can communicate that information to the COE, who might choose to improve the knowledgebase or get involved (particularly if it relates to governance or security).

Questions to ask

Use questions like those found below to assess user support.

- Who is responsible for supporting enterprise data and BI solutions? What about self-service solutions?
- How are the business impact and urgency of issues identified to effectively detect and prioritize critical issues?
- Is there a clear process for business users to report issues with data and BI solutions? How does this differ between enterprise and self-service solutions? What are the escalation paths?
- What types of issues do content creators and consumers typically experience? For example, do they experience data quality issues, performance issues, access issues, and others?
- Are any issues closed without them being resolved? Are there "known issues" in data items or reports today?
- Is a process in place for data asset owners to escalate issues with self-service BI solutions to central teams like the COE?
- How frequent are issues in the data and existing solutions? What proportion of these issues are found before they impact business end users?
- How long does it typically take to resolve issues? Is this timing sufficient for business users?
- What are examples of recent issues and the concrete impact on the business?
- Do enterprise teams and content creators know how to report Fabric issues to Microsoft? Can enterprise teams effectively leverage community resources to unblock critical issues?

> **⊗ Caution**
>
> When assessing user support and describing risks or issues, be careful to use neutral language that doesn't place blame on individuals or teams. Ensure everyone's perspective is fairly represented in an assessment. Focus on objective facts to accurately understand and describe the context.

Maturity levels

The following maturity levels will help you assess the current state of your Power BI user support.

Expand table

Level	State of user support
100: Initial	• Individual business units find effective ways of supporting each other. However, the tactics and practices are siloed and not consistently applied. • An internal discussion channel is available. However, it's not monitored closely. Therefore, the user experience is inconsistent.
200: Repeatable	• The COE actively encourages intra-team support and growth of the champions network. • The internal discussion channel gains traction. It's become known as the default place for questions and discussions. • The help desk handles a small number of the most common technical support issues.
300: Defined	• The internal discussion channel is popular and largely self-sustaining. The COE actively monitors and manages the discussion channel to ensure that all questions are answered quickly and correctly. • A help desk tracking system is in place to monitor support frequency, response topics, and priorities. • The COE provides appropriate extended support when required.
400: Capable	• The help desk is fully trained and prepared to handle a broader number of known and expected technical support issues. • SLAs are in place to define help desk support expectations, including extended support. The expectations are documented and communicated so they're clear to everyone involved.
500: Efficient	• Bidirectional feedback loops exist between the help desk and the COE. • Key performance indicators measure satisfaction and support methods. • Automation is in place to allow the help desk to react faster and reduce errors (for example, use of APIs and scripts).

Related content

In the next article in the Microsoft Fabric adoption roadmap series, learn about system oversight and administration activities.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: System oversight

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

System oversight—also known as Fabric administration—is the ongoing, day-to-day, administrative activities. It's specifically concerned with:

- **Governance**: Enact governance guidelines and policies to support self-service and enterprise data and business intelligence (BI) scenarios.
- **User empowerment**: Facilitate and support the internal processes and systems that empower the internal user community to the extent possible, while adhering to the organization's regulations and requirements.
- **Adoption**: Allow for broader organizational adoption of Fabric with effective governance and data management practices.

> **Important**
>
> Your organizational data culture objectives provide direction for your governance decisions, which in turn dictate how Fabric administration activities take place and by whom.

System oversight is a broad and deep topic. The goal of this article is to introduce some of the most important considerations and actions to help you become successful with your organizational adoption objectives.

Fabric administrators

The Fabric administrator role is a defined role in Microsoft 365, which delegates a subset of management activities. Global Microsoft 365 administrators are implicitly Fabric administrators. Power Platform administrators are also implicitly Fabric administrators.

A key governance decision is who to assign as a Fabric administrator. It's a centralized role that affects your entire tenant. Ideally, there are two to four people in the

organization who are capable of managing Fabric. Your administrators should operate in close coordination with the Center of Excellence (COE).

High privilege role

The Fabric administrator role is a high privilege role because:

- **User experience**: Settings that are managed by a Fabric administrator have a significant effect on user capabilities and user experience. For more information, see Govern tenant settings.
- **Full security access**: Fabric administrators can update access permissions for workspaces in the tenant. The result is that an administrator can allow permission to view or download data and reports as they see fit. For more information, see Govern tenant settings.
- **Personal workspace access**: Fabric administrators can access contents and govern the personal workspace of any user.
- **Metadata**: Fabric administrators can view all tenant metadata, including all user activities that occur in the Fabric portal (described in the Auditing and monitoring section below).

> **Important**
>
> Having too many Fabric administrators is a risk. It increases the probability of unapproved, unintended, or inconsistent management of the tenant.

Roles and responsibilities

The types of activities that an administrator will do on a day-to-day basis will differ between organizations. What's important, and given priority in your data culture, will heavily influence what an administrator does to support business-led self-service, managed self-service, and enterprise data and BI scenarios. For more information, see the Content ownership and management article.

> **Tip**
>
> The best type of person to serve as a Fabric administrator is one who has enough knowledge about the tools and workloads to understand what self-service users need to accomplish. With this understanding, the administrator can balance user empowerment and governance.

In addition to the Fabric administrator, there are other roles which use the term *administrator*. The following table describes the roles that are commonly and regularly used.

Expand table

Role	Scope	Description
Fabric administrator	Tenant	Manages tenant settings and other settings in the Fabric portal. All general references to *administrator* in this article refer to this type of administrator.
Capacity administrator	One capacity	Manages workspaces and workloads, and monitors the health of a Fabric capacity.
Data gateway administrator	One gateway	Manages gateway data source configuration, credentials, and users assignments. Might also handle gateway software updates (or collaborate with infrastructure team on updates).
Workspace administrator	One workspace	Manages workspace settings and access.

The Fabric ecosystem of workloads is broad and deep. There are many ways that Fabric integrates with other systems and platforms. From time to time, it'll be necessary to work with other administrators and IT professionals. For more information, see Collaborate with other administrators.

The remainder of this article provides an overview of the most common activities that a Fabric administrator does. It focuses on activities that are important to carry out effectively when taking a strategic approach to organizational adoption.

Service management

Overseeing the tenant is a crucial aspect to ensure that all users have a good experience with Power BI. A few of the key governance responsibilities of a Fabric administrator include:

- **Tenant settings**: Control which Power BI features and capabilities are enabled, and for which users in your organization.
- **Domains**: Group together two or more workspaces that have similar characteristics.
- **Workspaces**: Review and manage workspaces in the tenant.
- **Embed codes**: Govern which reports have been published publicly on the internet.
- **Organizational visuals**: Register and manage organizational visuals.

- **Azure connections**: Integrate with Azure services to provide additional functionality.

For more information, see Tenant administration.

User machines and devices

The adoption of Fabric depends directly on content creators and consumers having the tools and applications they need. Here are some important questions to consider.

- How will users request access to new tools? Will access to licenses, data, and training be available to help users use tools effectively?
- How will content consumers view content that's been published by others?
- How will content creators develop, manage, and publish content? What's your criteria for deciding which tools and applications are appropriate for which use cases?
- How will you install and set up tools? Does that include related prerequisites and data connectivity components?
- How will you manage ongoing updates for tools and applications?

For more information, see User tools and devices.

Architecture

In the context of Fabric, architecture relates to data architecture, capacity management, and data gateway architecture and management.

Data architecture

Data architecture refers to the principles, practices, and methodologies that govern and define what data is collected, and how it's ingested, stored, managed, integrated, modeled, and used.

There are many data architecture decisions to make. Frequently the COE engages in data architecture design and planning. It's common for administrators to get involved as well, especially when they manage databases or Azure infrastructure.

> **Important**
>
> Data architecture decisions have a significant impact on Fabric adoption, user satisfaction, and individual project success rates.

A few data architecture considerations that affect adoption include:

- Where does Fabric fit into the organization's entire data architecture? Are there other existing components such as an enterprise data warehouse (EDW) or a data lake that will be important to factor into plans?
- Is Fabric used end-to-end for data preparation, data modeling, and data presentation or is Fabric used for only some of those capabilities?
- Are managed self-service patterns followed to find the best balance between data reusability and report creator flexibility?
- Where will users consume the content? Generally, the three main ways to deliver content are: the Fabric portal, Power BI Report Server, and embedded in custom applications. Additionally, Microsoft Teams is a convenient alternative for users who spend a lot of time in Teams.
- Who is responsible for managing and maintaining the data architecture? Is it a centralized team, or a decentralized team? How is the COE represented in this team? Are certain skillsets required?
- What data sources are the most important? What types of data will we be acquiring?
- What semantic model connectivity mode and storage mode choices (for example, Direct Lake, import, live connection, DirectQuery, or composite model frameworks) are the best fit for the use cases?
- To what extent is data reusability encouraged using lakehouses, warehouses, and shared semantic models?
- To what extent is the reusability of data preparation logic and advanced data preparation encouraged by using data pipelines, notebooks, and dataflows?

It's important for administrators to become fully aware of Fabric's technical capabilities—as well as the needs and goals of their stakeholders—before they make architectural decisions.

> **Tip**
>
> Get into the good habit of completing a technical proof of concept (POC) to test out assumptions and ideas. Some organizations also call them *micro-projects* when the goal is to deliver a small unit of work. The goal of a POC is to address unknowns and reduce risk as early as possible. A POC doesn't have to be throwaway work, but it should be narrow in scope. Best practices reviews, as described in the **Mentoring and user enablement** article, are another useful way to help content creators with important architectural decisions.

Capacity management

Capacity includes features and capabilities to deliver analytics solutions at scale. There are two types of Fabric organizational licenses: Premium per User (PPU) and capacity. There are several types of capacity licenses. The type of capacity license determines which Fabric workloads are supported.

> **Important**
>
> At times this article refers to Power BI Premium or its capacity subscriptions (P SKUs). Be aware that Microsoft is currently consolidating purchase options and retiring the Power BI Premium per capacity SKUs. New and existing customers should consider purchasing Fabric capacity subscriptions (F SKUs) instead.
>
> For more information, see **Important update coming to Power BI Premium licensing** and **Power BI Premium FAQ**.

The use of capacity can play a significant role in your strategy for creating, managing, publishing, and distributing content. A few of the top reasons to invest in capacity include:

- Unlimited Power BI content distribution to large numbers of read-only users. Content consumption by users with a free Power BI license is available in Premium capacity only, not PPU. Content consumption by free users is also available with an F64 Fabric capacity license or higher.
- Access to Fabric experiences for producing end-to-end analytics.
- Deployment pipelines to manage the publication of content to development, test, and production workspaces. They're highly recommended for critical content to improve release stability.
- XMLA endpoint, which is an industry standard protocol for managing and publishing a semantic model, or querying the semantic model from any XMLA-compliant tool.
- Increased model size limits, including large semantic model support.
- More frequent data refreshes.
- Storage of data in a specific geographic area that's different from the home region.

The above list isn't all-inclusive. For a complete list, see Power BI Premium features.

Manage Fabric capacity

Overseeing the health of Fabric capacity is an essential ongoing activity for administrators. Each capacity SKU includes a set of resources. Capacity units (CUs) are used to measure compute resources for each SKU.

> **⊗ Caution**
>
> Lack of management, and consistently exceeding the limits of your capacity resources can often result in performance challenges and user experience challenges. Both challenges, if not managed correctly, can contribute to negative impact on adoption efforts.

Suggestions for managing Fabric capacity:

- Define who is responsible for managing the capacity. Confirm the roles and responsibilities so that it's clear what action will be taken, why, when, and by whom.
- Create a specific set of criteria for content that will be published to capacity. It's especially relevant when a single capacity is used by multiple business units because the potential exists to disrupt other users if the capacity isn't well-managed. Consider requiring a best practices review (such as reasonable semantic model size and efficient calculations) before publishing new content to a production capacity.
- Regularly use the Fabric capacity metrics app to understand resource utilization and patterns for the capacity. Most importantly, look for consistent patterns of overutilization, which will contribute to user disruptions. An analysis of usage patterns should also make you aware if the capacity is underutilized, indicating more value could be gained from the investment.
- Set the tenant setting so Fabric notifies you if the capacity becomes overloaded, or if an outage or incident occurs.

Autoscale

Autoscale is intended to handle occasional or unexpected bursts in capacity usage levels. Autoscale can respond to these bursts by automatically increasing CPU resources to support the increased workload.

Automated scaling up reduces the risk of performance and user experience challenges in exchange for a financial impact. If the capacity isn't well-managed, autoscale might trigger more often than expected. In this case, the metrics app can help you to determine underlying issues and do capacity planning.

Decentralized capacity management

Capacity administrators are responsible for assigning workspaces to a specific capacity.

Be aware that workspace administrators can also assign a workspace to PPU if the workspace administrator possesses a PPU license. However, it would require that all other workspace users must also have a PPU license to collaborate on, or view, Power BI content in the workspace. Other Fabric workloads can't be included in a workspace assigned to PPU.

It's possible to set up multiple capacities to facilitate decentralized management by different business units. Decentralizing management of certain aspects of Fabric is a great way to balance agility and control.

Here's an example that describes one way you could manage your capacity.

- Purchase a P3 capacity node in Microsoft 365. It includes 32 virtual cores (v-cores).
- Use 16 v-cores to create the first capacity. It will be used by the Sales team.
- Use 8 v-cores to create the second capacity. It will be used by the Operations team.
- Use the remaining 8 v-cores to create the third capacity. It will support general use.

The previous example has several advantages.

- Separate capacity administrators can be set up for each capacity. Therefore, it facilitates decentralized management situations.
- If a capacity isn't well-managed, the effect is confined to that capacity only. The other capacities aren't impacted.
- Billing and chargebacks to other business units are straightforward.
- Different workspaces can be easily assigned to the separate capacities.

However, the previous example has disadvantages, too.

- The limits per capacity are lower. The maximum memory size allowed for semantic models isn't the entire P3 capacity node size that was purchased. Rather, it's the assigned capacity size where the semantic model is hosted.
- It's more likely one of the smaller capacities will need to be scaled up at some point in time.
- There are more capacities to manage in the tenant.

> **Note**
>
> Resources for Power BI Premium per Capacity are referred to as v-cores. However, a Fabric capacity refers to them as capacity units (CUs). The scale for CUs and v-cores

is different for each SKU. For more information, see the Fabric licensing documentation.

Data gateway architecture and management

A data gateway facilitates the secure and efficient transfer of data between organizational data sources and the Fabric service. A data gateway is needed for data connectivity to on-premises or cloud services when a data source is:

- Located within the enterprise data center.
- Configured behind a firewall.
- Within a virtual network.
- Within a virtual machine.

There are three types of gateways.

- **On-premises data gateway (standard mode)** is a gateway service that supports connections to registered data sources for many users to use. The gateway software installations and updates are installed on a machine that's managed by the customer.
- **On-premises data gateway (personal mode)** is a gateway service that supports data refresh only. This gateway mode is typically installed on the PC of a content creator. It supports use by one user only. It doesn't support live connection or DirectQuery connections.
- **Virtual network data gateway** is a Microsoft managed service that supports connectivity for many users. Specifically, it supports connectivity for semantic models and dataflows stored in workspaces assigned to Premium capacity or Premium Per User.

Tip

The decision of **who can install gateway software** is a governance decision. For most organizations, use of the data gateway in standard mode, or a virtual network data gateway, should be strongly encouraged. They're far more scalable, manageable, and auditable than data gateways in personal mode.

Decentralized gateway management

The On-premises data gateway (standard mode) and Virtual network data gateway support specific data source types that can be registered, together with connection

details and how credentials are stored. Users can be granted permission to use the gateway data source so that they can schedule a refresh or run DirectQuery queries.

Certain aspects of gateway management can be done effectively on a decentralized basis to balance agility and control. For example, the Operations group might have a gateway dedicated to its team of self-service content creators and data owners.

Decentralized gateway management works best when it's a joint effort as follows.

Managed by the decentralized data owners:

- Departmental data source connectivity information and privacy levels.
- Departmental data source stored credentials (including responsibility for updating routine password changes).
- Departmental data source users who are permitted to use each data source.

Managed by centralized data owners (includes data sources that are used broadly across the organization; management is centralized to avoid duplicated data sources):

- Centralized data source connectivity information and privacy levels.
- Centralized data source stored credentials (including responsibility for updating routine password changes).
- Centralized data source users who are permitted to use each data source.

Managed by IT:

- Gateway software updates (gateway updates are usually released monthly).
- Installation of drivers and custom connectors (the same ones that are installed on user machines).
- Gateway cluster management (number of machines in the gateway cluster for high availability, disaster recovery, and to eliminate a single point of failure, which can cause significant user disruptions).
- Server management (for example, operating system, RAM, CPU, or networking connectivity).
- Management and backup of gateway encryption keys.
- Monitoring of gateway logs to assess when scale-up or scale-out is necessary.
- Alerting of downtime or persistent low resources on the gateway machine.

> **Tip**
>
> Allowing a decentralized team to manage certain aspects of the gateway means they can move faster. The tradeoff of decentralized gateway management does mean running more gateway servers so that each can be dedicated to a specific area of the organization. If gateway management is handled entirely by IT, it's

imperative to have a good process in place to quickly handle requests to add data sources and apply user updates.

User licenses

Every user needs a commercial license, which is integrated with a Microsoft Entra identity. The user license could be Free, Pro, or Premium Per User (PPU).

A user license is obtained via a subscription, which authorizes a certain number of licenses with a start and end date.

> **Note**
>
> Although each user requires a license, a Pro or PPU license is only required to share Power BI content. Users with a free license can create and share Fabric content other than Power BI items.

There are two approaches to procuring subscriptions.

- **Centralized**: Microsoft 365 billing administrator purchases a subscription for Pro or PPU. It's the most common way to manage subscriptions and assign licenses.
- **Decentralized**: Individual departments purchase a subscription via self-service purchasing.

Self-service purchasing

An important governance decision relates to what extent self-service purchasing will be allowed or encouraged.

Self-service purchasing is useful for:

- Larger organizations with decentralized business units that have purchasing authority and want to handle payment directly with a credit card.
- Organizations that intend to make it as easy as possible to purchase subscriptions on a monthly commitment.

Consider disabling self-service purchasing when:

- Centralized procurement processes are in place to meet regulatory, security, and governance requirements.
- Discounted pricing is obtained through an Enterprise Agreement (EA).

- Existing processes are in place to handle intercompany chargebacks.
- Existing processes are in place to handle group-based licensing assignments.
- Prerequisites are required for obtaining a license, such as approval, justification, training, or a governance policy requirement.
- There's a valid need, such as a regulatory requirement, to control access closely.

User license trials

Another important governance decision is whether user license trials are allowed. By default, trials are enabled. That means when content is shared with a colleague, if the recipient doesn't have a Pro or PPU license, they'll be prompted to start a trial to view the content (if the content doesn't reside within a workspace backed by capacity). The trial experience is intended to be a convenience that allows users to continue with their normal workflow.

Generally, disabling trials isn't recommended. It can encourage users to seek workarounds, perhaps by exporting data or working outside of supported tools and processes.

Consider disabling trials only when:

- There are serious cost concerns that would make it unlikely to grant full licenses at the end of the trial period.
- Prerequisites are required for obtaining a license (such as approval, justification, or a training requirement). It's not sufficient to meet this requirement during the trial period.
- There's a valid need, such as a regulatory requirement, to control access to the Fabric service closely.

> **Tip**
>
> Don't introduce too many barriers to obtaining a Fabric license. Users who need to get work done will find a way, and that way might involve workarounds that aren't ideal. For instance, without a license to use Fabric, people might rely far too much on sharing files on a file system or via email when significantly better approaches are available.

Cost management

Managing and optimizing the cost of cloud services, like Fabric, is an important activity. Here are several activities you can consider.

- Analyze who is using—and, more to the point, not using—their allocated Fabric licenses and make necessary adjustments. Fabric usage is analyzed using the activity log.
- Analyze the cost effectiveness of capacity or Premium Per User. In addition to the additional features, perform a cost/benefit analysis to determine whether capacity licensing is more cost-effective when there are a large number of consumers.
- Carefully monitor and manage Fabric capacity. Understanding usage patterns over time will allow you to predict when to purchase more capacity. For example, you might choose to scale up a single capacity from a P1 to P2, or scale out from one P1 capacity to two P1 capacities.
- If there are occasional spikes in the level of usage, use of autoscale with Fabric is recommended to ensure the user experience isn't interrupted. Autoscale will scale up capacity resources for 24 hours, then scale them back down to normal levels (if sustained activity isn't present). Manage autoscale cost by constraining the maximum number of v-cores, and/or with spending limits set in Azure. Due to the pricing model, autoscale is best suited to handle occasional unplanned increases in usage.
- For Azure data sources, co-locate them in the same region as your Fabric tenant whenever possible. It will avoid incurring Azure egress charges. Data egress charges are minimal, but at scale can add up to be considerable unplanned costs.

Security, information protection, and data loss prevention

Security, information protection, and data loss prevention (DLP) are joint responsibilities among all content creators, consumers, and administrators. That's no small task because there's sensitive information everywhere: personal data, customer data, or customer-authored data, protected health information, intellectual property, proprietary organizational information, just to name a few. Governmental, industry, and contractual regulations could have a significant impact on the governance guidelines and policies that you create related to security.

The Power BI security whitepaper is an excellent resource for understanding the breadth of considerations, including aspects that Microsoft manages. This section will introduce several topics that customers are responsible for managing.

User responsibilities

Some organizations ask Fabric users to accept a self-service user acknowledgment. It's a document that explains the user's responsibilities and expectations for safeguarding

organizational data.

One way to automate its implementation is with a Microsoft Entra terms of use policy. The user is required to view and agree to the policy before they're permitted to visit the Fabric portal for the first time. You can also require it to be acknowledged on a recurring basis, like an annual renewal.

Data security

In a cloud shared responsibility model, securing the data is always the responsibility of the customer. With a self-service data platform, self-service content creators have responsibility for properly securing the content that they shared with colleagues.

The COE should provide documentation and training where relevant to assist content creators with best practices (particularly situations for dealing with ultra-sensitive data).

Administrators can help by following best practices themselves. Administrators can also raise concerns when they see issues that could be discovered when managing workspaces, auditing user activities, or managing gateway credentials and users. There are also several tenant settings that are usually restricted except for a few users (for instance, the ability to publish to web or the ability to publish apps to the entire organization).

External guest users

External users—such as partners, customers, vendors, and consultants—are a common occurrence for some organizations, and rare for others. How you handle external users is a governance decision.

External user access is controlled by tenant settings and certain Microsoft Entra ID settings. For details of external user considerations, review the Distribute Power BI content to external guest users using Microsoft Entra B2B whitepaper.

Information protection and data loss prevention

Fabric supports capabilities for information protection and data loss prevention (DLP) in the following ways.

- **Information protection**: Microsoft Purview Information Protection (formerly known as Microsoft Information Protection) includes capabilities for discovering, classifying, and protecting data. A key principle is that data can be better protected

once it's been classified. The key building block for classifying data is sensitivity labels. For more information, see Information protection for Power BI planning.

- **Data loss prevention for Power BI**: Microsoft Purview Data Loss Prevention (formerly known as Office 365 Data Loss Prevention) supports DLP policies for Power BI. By using sensitivity labels or sensitive information types, DLP policies for Power BI help an organization locate sensitive semantic models. For more information, see Data loss prevention for Power BI planning.
- **Microsoft Defender for Cloud Apps**: Microsoft Defender for Cloud Apps (formerly known as Microsoft Cloud App Security) supports policies that help protect data, including real-time controls when users interact with the Power BI service. For more information, see Defender for Cloud Apps for Power BI planning.

Data residency

For organizations with requirements to store data within a geographic region, Fabric capacity can be set for a specific region that's different from the home region of the Fabric tenant.

Encryption keys

Microsoft handles encryption of *data at rest* in Microsoft data centers with transparent server-side encryption and auto-rotation of certificates. For customers with regulatory requirements to manage the Premium encryption key themselves, Premium capacity can be configured to use Azure Key Vault. Using customer-managed keys—also known as *bring-your-own-key* or *BYOK*—is a precaution to ensure that, in the event of a human error by a service operator, customer data can't be exposed.

Be aware that Premium Per User (PPU) only supports BYOK when it's enabled for the entire Fabric tenant.

Auditing and monitoring

It's critical that you make use of auditing data to analyze adoption efforts, understand usage patterns, educate users, support users, mitigate risk, improve compliance, manage license costs, and monitor performance. For more information about why auditing your data is valuable, see Auditing and monitoring overview.

There are different ways to approach auditing and monitoring depending on your role and your objectives. The following articles describe various considerations and planning activities.

- **Report-level auditing**: Techniques that report creators can use to understand which users are using the reports that they create, publish, and share.
- **Data-level auditing**: Methods that data creators can use to track the performance and usage patterns of data assets that they create, publish, and share.
- **Tenant-level auditing**: Key decisions and actions administrators can take to create an end-to-end auditing solution.
- **Tenant-level monitoring**: Tactical actions administrators can take to monitor the Power BI service, including updates and announcements.

REST APIs

The Power BI REST APIs and the Fabric REST APIs provide a wealth of information about your Fabric tenant. Retrieving data by using the REST APIs should play an important role in managing and governing a Fabric implementation. For more information about planning for the use of REST APIs for auditing, see Tenant-level auditing.

You can retrieve auditing data to build an auditing solution, manage content programmatically, or increase the efficiency of routine actions. The following table presents some actions you can perform with the REST APIs.

Expand table

Action	Documentation resource(s)
Audit user activities	REST API to get activity events
Audit workspaces, items, and permissions	Collection of asynchronous metadata scanning REST APIs to obtain a tenant inventory
Audit content shared to entire organization	REST API to check use of widely shared links
Audit tenant settings	REST API to check tenant settings
Publish content	REST API to deploy items from a deployment pipeline or clone a report to another workspace
Manage content	REST API to refresh a semantic model or take over ownership of a semantic model
Manage gateway data sources	REST API to update credentials for a gateway data source
Export content	REST API to export a report
Create workspaces	REST API to create a new workspace

Action	Documentation resource(s)
Manage workspace permissions	REST API to assign user permissions to a workspace
Update workspace name or description	REST API to update workspace attributes
Restore a workspace	REST API to restore a deleted workspace
Programmatically retrieve a query result from a semantic model	REST API to run a DAX query against a semantic model
Assign workspaces to capacity	REST API to assign workspaces to capacity
Programmatically change a data model	Tabular Object Model (TOM) API
Embed Power BI content in custom applications	Power BI embedded analytics client APIs

> **Tip**
>
> There are many other Power BI REST APIs. For a complete list, see **Using the Power BI REST APIs**.

Planning for change

Every month, Microsoft releases new Fabric features and capabilities. To be effective, it's crucial that everyone involved with system oversight stays current. For more information, see Tenant-level monitoring.

> **Important**
>
> Don't underestimate the importance of staying current. If you get a few months behind on announcements, it can become difficult to properly manage Fabric and support your users.

Considerations and key actions

Checklist - Considerations and key actions you can take for system oversight follow.

Improve system oversight:

- ✓ **Verify who is permitted to be a Fabric administrator**: If possible, reduce the number of people granted the Fabric administrator role if it's more than a few people.
- ✓ **Use PIM for occasional administrators**: If you have people who *occasionally* need Fabric administrator rights, consider implementing Privileged Identity Management (PIM) in Microsoft Entra ID. It's designed to assign just-in-time role permissions that expire after a few hours.
- ✓ **Train administrators**: Check the status of cross-training and documentation in place for handling Fabric administration responsibilities. Ensure that a backup person is trained so that needs can be met timely, in a consistent way.

Improve management of the Fabric service:

- ✓ **Review tenant settings**: Conduct a review of all tenant settings to ensure they're aligned with data culture objectives and governance guidelines and policies. Verify which groups are assigned for each setting.
- ✓ **Document the tenant settings**: Create documentation of your tenant settings for the internal Fabric community and post it in the centralized portal. Include which groups a user would need to request to be able to use a feature. Use the Get Tenant Settings REST API to make the process more efficient, and to create snapshots of the settings on a regular basis.
- ✓ **Customize the *Get Help* links**: When user resources are established, as described in the Mentoring and user enablement article, update the tenant setting to customize the links under the *Get Help* menu option. It will direct users to your documentation, community, and help.

Improve management of user machines and devices:

- ✓ **Create a consistent onboarding process**: Review your process for how onboarding of new content creators is handled. Determine if new requests for software, such as Power BI Desktop, and user licenses (Free, Pro, or PPU) can be handled together. It can simplify onboarding since new content creators won't always know what to ask for.
- ✓ **Handle user machine updates**: Ensure an automated process is in place to install and update software, drivers, and settings to ensure all users have the same version.

Data architecture planning:

- ✓ **Assess what your end-to-end data architecture looks like**: Make sure you're clear on:

    - How Fabric is currently used by the different business units in your organization versus how you want Fabric to be used. Determine if there's a gap.
    - If there are any risks that should be addressed.
    - If there are any high-maintenance situations to be addressed.
    - What data sources are important for Fabric users, and how they're documented and discovered.
- ✓ **Review existing data gateways**: Find out what gateways are being used throughout your organization. Verify that gateway administrators and users are set correctly. Verify who is supporting each gateway, and that there's a reliable process in place to keep the gateway servers up to date.
- ✓ **Verify use of personal gateways**: Check the number of personal gateways that are in use, and by whom. If there's significant usage, take steps to move towards use of the standard mode gateway.

Improve management of user licenses:

- ✓ **Review the process to request a user license**: Clarify what the process is, including any prerequisites, for users to obtain a license. Determine whether there are improvements to be made to the process.
- ✓ **Determine how to handle self-service license purchasing**: Clarify whether self-service licensing purchasing is enabled. Update the settings if they don't match your intentions for how licenses can be purchased.
- ✓ **Confirm how user trials are handled**: Verify user license trials are enabled or disabled. Be aware that all user trials are Premium Per User. They apply to Free licensed users signing up for a trial, and Pro users signing up for a Premium Per User trial.

Improve cost management:

- ✓ **Determine your cost management objectives**: Consider how to balance cost, features, usage patterns, and effective utilization of resources. Schedule a routine process to evaluate costs, at least annually.
- ✓ **Obtain activity log data**: Ensure you have access to the activity log data to assist with cost analysis. It can be used to understand who is—or isn't—using the license assigned to them.

Improve security and data protection:

- ✓ **Clarify exactly what the expectations are for data protection**: Ensure the expectations for data protection, such as how to use sensitivity labels, are documented and communicated to users.
- ✓ **Determine how to handle external users**: Understand and document the organizational policies around sharing Fabric content with external users. Ensure

that settings in Fabric support your policies for external users.
- ✓ **Set up monitoring**: Investigate the use of Microsoft Defender for Cloud Apps to monitor user behavior and activities in Fabric.

Improve auditing and monitoring:

- ✓ **Plan for auditing needs**: Collect and document the key business requirements for an auditing solution. Consider your priorities for auditing and monitoring. Make key decisions related to the type of auditing solution, permissions, technologies to be used, and data needs. Consult with IT to clarify what auditing processes currently exist, and what preferences of requirements exist for building a new solution.
- ✓ **Consider roles and responsibilities**: Identify which teams will be involved in building an auditing solution, as well as the ongoing analysis of the auditing data.
- ✓ **Extract and store user activity data**: If you aren't currently extracting and storing the raw data, begin retrieving user activity data.
- ✓ **Extract and store snapshots of tenant inventory data**: Begin retrieving metadata to build a tenant inventory, which describes all workspaces and items.
- ✓ **Extract and store snapshots of users and groups data**: Begin retrieving metadata about users, groups, and service principals.
- ✓ **Create a curated data model**: Perform data cleansing and transformations of the raw data to create a curated data model that'll support analytical reporting for your auditing solution.
- ✓ **Analyze auditing data and act on the results**: Create analytic reports to analyze the curated auditing data. Clarify what actions are expected to be taken, by whom, and when.
- ✓ **Include additional auditing data**: Over time, determine whether other auditing data would be helpful to complement the activity log data, such as security data.

> **Tip**
>
> For more information, see **Tenant-level auditing**.

Use the REST APIs:

- ✓ **Plan for your use of the REST APIs**: Consider what data would be most useful to retrieve from the Power BI REST APIs and the Fabric REST APIs.
- ✓ **Conduct a proof of concept**: Do a small proof of concept to validate data needs, technology choices, and permissions.

Questions to ask

Use questions like those found below to assess system oversight.

- Are there atypical administration settings enabled or disabled? For example, is the entire organization allowed to publish to web (we strongly advise restricting this feature).
- Do administration settings and policies align with, or inhibit, business the way user work?
- Is there a process in place to critically appraise new settings and decide how to set them? Alternatively, are only the most restrictive settings set as a precaution?
- Are Microsoft Entra security groups used to manage who can do what?
- Do central teams have visibility of effective auditing and monitoring tools?
- Do monitoring solutions depict information about the data assets, user activities, or both?
- Are auditing and monitoring tools actionable? Are there clear thresholds and actions set, or do monitoring reports simply describe what's in the data estate?
- Is Azure Log Analytics used (or planned to be used) for detailed monitoring of Fabric capacities? Are the potential benefits and cost of Azure Log Analytics clear to decision makers?
- Are sensitivity labels and data loss prevention policies used? Are the potential benefits and cost of these clear to decision makers?
- Do administrators know the current number of licenses and licensing cost? What proportion of the total BI spend goes to Fabric capacity, and to Pro and PPU licenses? If the organization is only using Pro licenses for Power BI content, could the number of users and usage patterns warrant a cost-effective switch to Power BI Premium or Fabric capacity?

Maturity levels

The following maturity levels will help you assess the current state of your Power BI system oversight.

Expand table

Level	State of system oversight
100: Initial	• Tenant settings are configured independently by one or more administrators based on their best judgment. • Architecture needs, such as gateways and capacities, are satisfied on an as-needed basis. However, there isn't a strategic plan. • Fabric activity logs are unused, or selectively used for tactical purposes.
200: Repeatable	• The tenant settings purposefully align with established governance guidelines and policies. All tenant settings are reviewed regularly. • A small number of specific administrators are selected. All administrators have a good understanding of what users are trying to accomplish in Fabric, so they're in a good position to support users. • A well-defined process exists for users to request licenses and software. Request forms are easy for users to find. Self-service purchasing settings are specified. • Sensitivity labels are configured in Microsoft 365. However, use of labels remains inconsistent. The advantages of data protection aren't well understood by users.
300: Defined	• The tenant settings are fully documented in the centralized portal for users to reference, including how to request access to the correct groups. • Cross-training and documentation exist for administrators to ensure continuity, stability, and consistency. • Sensitivity labels are assigned to content consistently. The advantages of using sensitivity labels for data protection are understood by users. • An automated process is in place to export Fabric activity log and API data to a secure location for reporting and auditing.
400: Capable	• Administrators work closely with the COE and governance teams to provide oversight of Fabric. A balance of user empowerment and governance is successfully achieved. • Decentralized management of data architecture (such as gateways or capacity management) is effectively handled to balance agility and control. • Automated policies are set up and actively monitored in Microsoft Defender for Cloud Apps for data loss prevention. • Activity log and API data is actively analyzed to monitor and audit Fabric activities. Proactive action is taken based on the data.

Level	State of system oversight
500: Efficient	• The Fabric administrators work closely with the COE actively stay current. Blog posts and release plans from the Fabric product team are reviewed frequently to plan for upcoming changes. • Regular cost management analysis is done to ensure user needs are met in a cost-effective way. • The Fabric REST API is used to retrieve tenant setting values on a regular basis. • Activity log and API data is actively used to inform and improve adoption and governance efforts.

Related content

For more information about system oversight and Fabric administration, see the following resources.

- Administer Microsoft Fabric
- Administer Power BI - Part 1
- Administer Power BI - Part 2
- Administrator in a Day Training – Day 1
- Administrator in a Day Training – Day 2
- Power BI security whitepaper
- External guest users whitepaper
- Power BI implementation planning

In the next article in the Microsoft Fabric adoption roadmap series, learn about effective change management.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap: Change management

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

When working toward improved data and business intelligence (BI) adoption, you should plan for effective *change management*. In the context of data and BI, change management includes procedures that address the impact of change for people in an organization. These procedures safeguard against disruption and productivity loss due to changes in solutions or processes.

> **Note**
>
> Effective change management is particularly important when you migrate to Power BI.

Effective change management improves adoption and productivity because it:

- Helps content creators and consumers use analytics more effectively and sooner.
- Limits redundancy in data, analytical tools, and solutions.
- Reduces the likelihood of risk-creating behaviors that affect shared resources (like Fabric capacity) or organizational compliance (like data security and privacy).
- Mitigates resistance to change that obstructs planning and inhibits user adoption.
- Mitigates the impact of change and improving user wellbeing by reducing the potential for disruption, stress, and conflict.

Effective change management is critical for successful adoption at all levels. To successfully manage change, consider the key actions and activities described in the following sections.

> **Important**
>
> Change management is a fundamental obstacle to success in many organizations. Effective change management requires that you understand that it's about people —not tools or processes.

Successful change management involves empathy and communication. Ensure that change isn't forced or resistance to change is ignored, because it can widen organizational divides and further inhibit effectiveness.

Tip

Whenever possible, we recommend that you describe and promote change as *improvement*—it's much less threatening. For many people, *change* implies a cost in terms of effort, focus, and time. Alternatively, *improvement* means a benefit because it's about making something better.

Types of change to manage

When implementing data and BI solutions, you should manage different types of change. Also, depending on the scale and scope of your implementation, you should address different aspects of change.

Consider the following types of change to manage when you plan for Fabric adoption.

Process-level changes

Process-level changes are changes that affect a broader user community or the entire organization. These changes typically have a larger impact, and so they require more effort to manage. Specifically, this change management effort includes specific plans and activities.

Here are some examples of process-level changes.

- Change from centralized to decentralized approaches to ownership (change in content ownership and management).
- Change from enterprise to departmental, or from team to personal content delivery (change in content delivery scope).
- Change of central team structure (for example, forming a Center of Excellence).
- Changes in governance policies.
- Migration from other analytics products to Fabric, and the changes this migration involves, like:
  - The separation of semantic models and reports, and a model-based approach to analytics.
  - Transitioning from exports or static reports to interactive analytical reports, which can involve filtering and cross-filtering.

    - Moving from distributing reports as PowerPoint files or flat files to accessing reports directly from the Fabric portal.
    - Shifting from information in tables, paginated reports, and spreadsheets to interactive visualizations and charts.
    - Changing from an on-premises or platform as a service (PaaS) platform to a software as a service (SaaS) tool.

> **Note**
>
> Typically, giving up export-based processes or Excel reporting is a significant challenge. That's because these methods are usually deeply engrained in the organization and are tied to the autonomy and data skills of your users.

Solution-level changes

Solution-level changes are changes that affect a single solution or set of solutions. These changes limit their impact to the user community of those solutions and their dependent processes. Although solution-level changes typically have a lower impact, they also tend to occur more frequently.

> **Note**
>
> In the context of this article, a *solution* is built to address specific business needs for users. A solution can take many forms, such as a data pipeline, a lakehouse, a semantic model, or a report. The considerations for change management described in this article are relevant for all types of solutions, and not only reporting projects.

Here are some examples of solution-level changes.

- Changes in calculation logic for KPIs or measures.
- Changes in how master data or hierarchies for business attributes are mapped, grouped, or described.
- Changes in data freshness, detail, format, or complexity.
- Introduction of advanced analytics concepts, like predictive analytics or prescriptive analytics, or general statistics (if the user community isn't familiar with these concepts, already).
- Changes in the presentation of data, like:
    - Styling, colors, and other formatting choices for visuals.
    - The type of visualization.

    - How data is grouped or summarized (such as changing from different measures of central tendency, like average, median, or geometric mean).
- Changes in how content consumers interact with data (like connecting to a shared semantic model instead of exporting information for personal usage scenarios).

How you prepare change management plans and activities will depend on the types of change. To successfully and sustainably manage change, we recommend that you implement incremental changes.

Address change incrementally

Change management can be a significant undertaking. Taking an incremental approach can help you facilitate change in a way that's sustainable. To adopt an incremental approach, you identify the highest priority changes and break them into manageable parts, implementing each part with iterative phases and action plans.

The following steps outline how you can incrementally address change.

1. **Define what's changing**: Describe the change by outlining the before and after states. Clarify the specific parts of the process or situation that you'll change, remove, or introduce. Justify why this change is necessary, and when it should occur.
2. **Describe the impact of the change**: For each of these changes, estimate the business impact. Identify which processes, teams, or individuals the change affects, and how disruptive it will be for them. Also consider any downstream effects the change has on other dependent solutions or processes. Downstream effects might result in other changes. Additionally, consider how long the situation remained the same before it was changed. Changes to longer-standing processes tend to have a higher impact, as preferences and dependencies arise over time.
3. **Identify priorities**: Focus on the changes with the highest potential impact. For each change, outline a more detailed description of the changes and how it will affect people.
4. **Plan how to incrementally implement the change**: Identify whether any high-impact changes can be broken into stages or parts. For each part, describe how it might be incrementally implemented in phases to limit its impact. Determine whether there are any constraints or dependencies (such as when changes can be made, or by whom).
5. **Create an action plan for each phase**: Plan the actions you will take to implement and support each phase of the change. Also, plan for how you can mitigate disruption in high-impact phases. Be sure to include a rollback plan in your action plan, whenever possible.

> **Tip**
>
> Iteratively plan how you'll implement each phase of these incremental changes as part of your quarterly **tactical planning**.

When you plan to mitigate the impact of changes on Power BI adoption, consider the activities described in the following sections.

Effectively communicate change

Ensure that you clearly and concisely describe planned changes for the user community. Important communication should originate from the executive sponsor, or another leader with relevant authority. Be sure to communicate the following details.

- **What's changing**: What the situation is now and what it will be after the change.
- **Why it's changing**: The benefit and value of the change for the audience.
- **When it's changing**: An estimation of when the change will take effect.
- **Further context**: Where people can go for more information.
- **Contact information**: Who people should contact provide feedback, ask questions, or raise concerns.

Consider maintaining a history of communications in your centralized portal. That way, it's easy to find communications, timings, and details of changes after they've occurred.

> **Important**
>
> You should communicate change with sufficient advanced notice so that people are prepared. The higher the potential impact of the change, the earlier you should communicate it. If unexpected circumstances prevent advance notice, be sure to explain why in your communication.

Plan training and support

Changes to tools, processes, and solutions typically require training to use them effectively. Additionally, extra support might be required to address questions or respond to support requests.

Here are some actions you can take to plan for training and support.

- Centralize training and support by using a centralized portal. The portal can help organize discussions, collect feedback, and distribute training materials or

documentation by topic.
- Consider incentives to encourage self-sustaining support within a community.
- Schedule recurring office hours to answer questions and provide mentorship.
- Create and demonstrate end-to-end scenarios for people to practice a new process.
- For high-impact changes, prepare training and support plans that realistically assess the effort and actions needed to prevent the change from causing disruption.

> ⓘ **Note**
>
> These training and support actions will differ depending on the scale and scope of the change. For high-impact, large-scale changes (like transitioning from enterprise to managed self-service approaches to data and BI), you'll likely need to plan iterative, multi-phase plans that span multiple planning periods. In this case, carefully consider the effort and resources needed to deliver success.

Involve executive leadership

Executive support is critical to effective change management. When an executive supports a change, it demonstrates its strategic importance or benefit to the rest of the organization. This top-down endorsement and reinforcement is particularly important for high-impact, large-scale changes, which have a higher potential for disruption. For these scenarios, ensure that you actively engage and involve your executive sponsor to endorse and reinforce the change.

> ⊗ **Caution**
>
> Resistance to change from the executive leadership is often a warning sign that stronger **business alignment** is needed between the business and BI strategies. In this scenario, consider specific alignment sessions and change management actions with executive leadership.

Involve stakeholders

To effectively manage change, you can also take a bottom-up approach by engaging the stakeholders, who are the people the change affects. When you create an action plan to address the changes, identify and engage key stakeholders in focused, limited sessions. In this way you can understand the impact of the change on the people whose work will

be affected by the change. Take note of their concerns and their ideas for how you might lessen the impact of this change. Ensure that you identify any potentially unexpected effects of the change on other people and processes.

Handle resistance to change

It's important to address resistance to change, as it can have substantial negative impacts on adoption and productivity. When you address resistance to change, consider the following actions and activities.

- **Involve your executive sponsor**: The authority, credibility, and influence of the executive sponsor is essential to support change management and resolve disputes.
- **Identify blocking issues**: When change disrupts the way people work, this change can prevent people from effectively completing tasks in their regular activities. For such blocking issues, identify potential workarounds when you take into account the changes.
- **Focus on data and facts instead of opinions**: Resistance to change is sometimes due to opinions and preferences, because people are familiar with the situation prior to the change. Understand why people have these opinions and preferences. Perhaps it's due to convenience, because people don't want to invest time and effort in learning new tools or processes.
- **Focus on business questions and processes instead of requirements**: Changes often introduce new processes to address problems and complete tasks. New processes can lead to a resistance to change because people focus on what they miss instead of fully understanding what's new and why.

Additionally, you can have a significant impact on change resistance by engaging *promoters* and *detractors*.

Identify and engage promoters

Promoters are vocal, credible individuals in a user community who advocate in favor of a tool, solution, or initiative. Promoters can have a positive impact on adoption because they can influence peers to understand and accept change.

To effectively manage change, you should identify and engage promoters early in the process. You should involve them and inform them about the change to better utilize and amplify their advocacy.

> **Tip**

The promoters you identify might also be great candidates for your **champions network**.

Identify and engage detractors

Detractors are the opposite of promoters. They are vocal, credible individuals in a user community who advocate against a tool, solution, or initiative. Detractors can have a significant negative influence on adoption because they can convince peers that the change isn't beneficial. Additionally, detractors can advocate for alternative or solutions marked for retirement, making it more difficult to decommission old tools, solutions, or processes.

To effectively manage change, you should identify and engage detractors early in the process. That way, you can mitigate the potential negative impact they have. Furthermore, if you address their concerns, you might convert these detractors into promoters, helping your adoption efforts.

Tip

A common source of detractors is content owners for solutions that are going to be modified or replaced. The change can sometimes threaten these content owners, who are incentivized to resist the change in the hope that their solution will remain in use. In this case, identify these content owners early and involve them in the change. Giving these individuals a sense of ownership of the implementation will help them embrace, and even advocate in favor, of the change.

Questions to ask

Use questions like those found below to assess change management.

- Is there a role or team responsible for change management in the organization? If so, how are they involved in data and BI initiatives?
- Is change seen as an obstacle to achieving strategic success among people in the organization? Is the importance of change management acknowledged in the organization?

- Are there any significant promoters for data and BI solutions and processes in the user community? Conversely, are there any significant detractors?
- What communication and training efforts are performed to launch new data tools and solutions? How long do they last?
- How is change in the user community handled (for example, with new hires or promoted individuals)? What onboarding activities introduce these new individuals to existing solutions, processes, and policies?
- Do people who create Excel reports feel threatened or frustrated by initiatives to automate reporting with BI tools?
- To what extent do people associate their identities with the tools they use and the solutions they have created and own?
- How are changes to existing solutions planned and managed? Are changes planned, with a visible roadmap, or are they reactive? Do people get sufficient notification about upcoming changes?
- How frequently do changes disrupt existing processes and tools?
- How long does it take to decommission legacy systems or solutions when new ones become available? How long does it take to implement changes to existing solutions?
- To what extent do people agree with the statement *I am overwhelmed with the amount of information I am required to process*? To what extent do people agree with the sentiment *things are changing too much, too quickly*?

Maturity levels

An assessment of change management evaluates how effectively the organization can enact and respond to change.

The following maturity levels will help you assess your current state of change management, as it relates to data and BI initiatives.

Expand table

Level	State of change management
100: Initial	• Change is usually reactive, and it's also poorly communicated. • The purpose or benefits of change aren't well understood, and resistance to

Level	State of change management
	change causes conflict and disruption. • No clear teams or roles are responsible for managing change for data initiatives.
200: Repeatable	• Executive leadership and decision makers recognize the need for change management in data and BI projects and initiatives. • Some efforts are taken to plan or communicate change, but they're inconsistent and often reactive. Resistance to change is still common. Change often disrupts existing processes and tools.
300: Defined	• Formal change management plans or roles are in place. These plans include communication tactics and training, but they're not consistently or reliably followed. Change occasionally disrupts existing processes and tools. • Successful change management is championed by key individuals that bridge organizational boundaries.
400: Capable	• Empathy and effective communication are integral to change management strategies. • Change management efforts are owned by particular roles or teams, and effective communication results in a clear understanding of the purpose and benefits of change. Change rarely interrupts existing processes and tools.
500: Efficient	• Change is an integral part of the organization. People in the organization understand the inevitability of change, and see it as a source for momentum instead of disruption. Change almost never unnecessarily interrupts existing processes or tools. • Systematic processes address change as a challenge of people and not processes.

Related content

In the next article in the Microsoft Fabric adoption roadmap series, in conclusion, learn about adoption-related resources that you might find valuable.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Microsoft Fabric adoption roadmap conclusion

Article • 12/30/2024

> **Note**
>
> This article forms part of the *Microsoft Fabric adoption roadmap* series of articles. For an overview of the series, see Microsoft Fabric adoption roadmap.

This article concludes the series on Microsoft Fabric adoption. The strategic and tactical considerations and action items presented in this series will assist you in your analytics adoption efforts, and with creating a productive data culture in your organization.

This series covered the following aspects of Fabric adoption.

- Adoption introduction
- Adoption maturity levels
- Data culture
- Executive sponsorship
- Business alignment
- Content ownership and management
- Content delivery scope
- Center of Excellence
- Governance
- Mentoring and enablement
- Community of practice
- User support
- System oversight
- Change management

The rest of this article includes suggested next actions to take. It also includes other adoption-related resources that you might find valuable.

Next actions to take

It can be overwhelming to decide where to start. The following series of steps provides a process to help you approach your next actions.

1. **Learn**: First, read this series of articles end-to-end. Become familiar with the strategic and tactical considerations and action items that directly lead to successful analytics adoption. They'll help you to build a data culture in your organization. Discuss the concepts with your colleagues.
2. **Assess current state**: For each area of the adoption roadmap, assess your current state. Document your findings. Your goal is to have full clarity on where you're now so that you can make informed decisions about what to do next.
3. **Clarify your strategic goals**: Ensure that you're clear on what your organization's goals are for adopting Fabric. Confirm that your adoption and data culture goals align with your organization's broader strategic goals for the use of data, analytics, and business intelligence (BI) in general. Focus on what your immediate strategy is for the next 3-12 months. For more information about defining your goals, see the strategic planning article.
4. **Prioritize**: Clarify what's most important to achieve in the next 12-18 months. For instance, you might identify specific user enablement or risk reduction areas that are a higher priority than other areas. Determine which advancements in maturity levels you should prioritize first. For more information about defining your priorities, see the strategic planning article.
5. **Identify future state**: For each area of the roadmap, identify the gaps between what you want to happen (your future state) and what's happening (your current state). Focus on the next 12-18 months for identifying your desired future state.
6. **Customize maturity levels**: Using the information you have on your strategy and future state, customize the maturity levels for each area of the roadmap. Update or delete the description for each maturity level so that they're realistic, based on your goals and strategy. Your current state, priorities, staffing, and funding will influence the time and effort it will take to advance to higher maturity levels.
7. **Define measurable objectives**: Create KPIs (key performance indicators) or OKRs (objectives and key results) to define specific goals for the next quarter. Ensure that the objectives have clear owners, are measurable, time-bound, and achievable. Confirm that each objective aligns with your strategic BI goals and priorities.
8. **Create tactical plans**: Add specific action items to your project plan. Action items will identify who will do what, and when. Include short, medium, and longer-term (backlog) items in your project plan to make it easy to track and reprioritize.
9. **Track action items**: Use your preferred project planning software to track continual, incremental progress of your action items. Summarize progress and status every quarter for your executive sponsor.
10. **Adjust**: As new information becomes available—and as priorities change—reevaluate and adjust your focus. Reexamine your strategic goals, objectives, and action items once a quarter so you're certain that you're focusing on the right actions.

11. **Celebrate**: Pause regularly to appreciate your progress. Celebrate your wins. Reward and recognize people who take the initiative and help achieve your goals. Encourage healthy partnerships between IT and the different areas of the business.
12. **Repeat**: Continue learning, experimenting, and adjusting as you progress with your implementation. Use feedback loops to continually learn from everyone in the organization. Ensure that continual, gradual, improvement is a priority.

A few important key points are implied within the previous suggestions.

- **Focus on the near term**: Although it's important to have an eye on the big picture, we recommend that you focus primarily on the next quarter, next semester, and next year. It's easier to assess, plan, and act when you focus on the near term.
- **Progress will be incremental**: Changes that happen every day, every week, and every month add up over time. It's easy to become discouraged and sense a lack of progress when you're working on a large adoption initiative that takes time. If you keep track of your incremental progress, you'll be surprised at how much you can accomplish over the course of a year.
- **Changes will continually happen**: Be prepared to reconsider decisions that you make, perhaps every quarter. It's easier to cope with continual change when you expect the plan to change.
- **Everything correlates together**: As you progress through each of the steps listed above, it's important that everything's correlated from the high-level strategic organizational objectives, all the way down to more detailed action items. That way, you'll know that you're working on the right things.

Power BI implementation planning

Successfully implementing analytics throughout the organization requires deliberate thought and planning. The Power BI implementation planning series of articles, which is a work in progress, is intended to complement the Microsoft Fabric adoption roadmap. It includes key considerations, actions, decision-making criteria, recommendations, and it describes implementation patterns for important common usage scenarios.

Power BI adoption framework

The Power BI adoption framework describes additional aspects of *how* to adopt Power BI in more detail. The original intent of the framework was to support Microsoft partners with a lightweight set of resources for use when helping their customers deploy and adopt Power BI.

The framework can augment this Microsoft Fabric adoption roadmap series. The roadmap series focuses on the *why* and *what* of adopting Fabric, more so than the *how*.

> **Note**
>
> When completed, the Power BI implementation planning series (described in the previous section) will replace the Power BI adoption framework.

Microsoft's BI transformation

Consider reading about Microsoft's journey and experience with driving a data culture. This article describes the importance of two terms: *discipline at the core* and *flexibility at the edge*. It also shares Microsoft's views and experience about the importance of establishing a COE.

Power Platform adoption

The Power Platform team has an excellent set of adoption-related content. Its primary focus is on Power Apps, Power Automate, and Power Virtual Agents. Many of the ideas presented in this content can be applied to Power BI also.

The Power CAT Adoption Maturity Model, published by the Power CAT team, describes repeatable patterns for successful Power Platform adoption.

The Power Platform Center of Excellence Starter Kit is a collection of components and tools to help you develop a strategy for adopting and supporting Microsoft Power Platform.

The Power Platform adoption best practices includes a helpful set of documentation and best practices to help you align business and technical strategies.

The Power Platform adoption framework is a community-driven project with excellent resources on adoption of Power Platform services at scale.

Microsoft 365 and Azure adoption

You might also find useful adoption-related guidance published by other Microsoft technology teams.

- The Maturity Model for Microsoft 365 provides information and resources to use capabilities more fully and efficiently.

- Microsoft Learn has a learning path for using the Microsoft service adoption framework to drive adoption in your enterprise.
- The Microsoft Cloud Adoption Framework for Azure is a collection of documentation, implementation guidance, best practices, and tools to accelerate your cloud adoption journey.

A wide variety of other adoption guides for individual technologies can be found online. A few examples include:

- Microsoft Teams adoption guide.
- Microsoft Security and Compliance adoption guide.
- SharePoint Adoption Resources.

Industry guidance

The Data Management Book of Knowledge (DMBOK2) is a book available for purchase from DAMA International. It contains a wealth of information about maturing your data management practices.

> **Note**
>
> The additional resources provided in this article aren't required to take advantage of the guidance provided in this Fabric adoption series. They're reputable resources should you wish to continue your journey.

Partner community

Experienced partners are available to help your organization succeed with adoption initiatives. To engage a partner, visit the Power BI partner portal.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

IDEAS journey to a modern data platform with Microsoft Fabric - from petabytes to insights

Article • 03/31/2025

Microsoft Fabric is an AI-powered SaaS platform for end-to-end enterprise analytics. It efficiently supports various data roles across an organization. To optimize data consistency and accessibility, Microsoft used Fabric to enhance its internal analytics infrastructure during rapid AI advancements. Microsoft established IDEAS (Insights, Data, Engineering, Analytics, Systems) organization to build and maintain a comprehensive data analytics platform. IDEAS aims to unify data sources, eliminate silos, and create a single source of truth, boosting productivity and AI adoption across Microsoft. Initially supporting Office products, IDEAS now powers data-driven insights across Microsoft 365, Security, and over 600 internal teams driving AI adoption and productivity. This article details IDEAS's journey with adopting Fabric.

A key function of IDEAS is to serve as the central data and growth engine for the Experiences and Devices(E+D) and Security divisions. IDEAS also acts as the central data plane for all Copilot experiences, driving Copilot's success by aggregating key insights, enabling research, and powering AI experiences across Microsoft. It manages 420 PiB (Pebibyte) of data from 2,700 sources, personalizing experiences across more than 350 product surfaces and billions of customer interactions annually.

Because of its scale and role in powering key Microsoft initiatives like Copilot, IDEAS serves as a real-world testing ground for emerging data technologies. IDEAS is a pilot user and strategic 'customer zero' for Microsoft Fabric. It provides valuable feedback and validates the Fabric's capabilities at a various levels. IDEAS provides key insights that shape Fabric's development, while Fabric enables IDEAS to achieve its vision for the future of its AI-driven data platform. Specifically, Fabric offers key benefits in following four key areas:

- **Activating data for AI innovation:** Fabric's seamless integration with Microsoft tools like Office and Azure AI accelerates the creation of custom AI models and solutions.
- **Streamlining analytics with a unified toolchain:** By providing a unified toolchain for all data roles, Fabric empowers everyone within IDEAS to enhance collaboration, streamline workflows, and maximize the data value.

- **Increasing collaboration and flexibility:** Fabric enables collaboration across different data personas using the same datasets and tools. This flexibility simplifies working with diverse data formats, locations, optimizes engineering processes, and enables teams to work more effectively.
- **Reducing costs and risks:** Fabric's unified data lake minimizes data movement, reducing engineering costs and compliance risks by allowing multiple compute engines to operate on the same copy of data. By maintaining a single copy of data, teams can efficiently use it for multiple purposes, which support effective data governance and compliance.

This partnership aims to deliver substantial business and productivity value by creating a modern data platform to meet today's technological demands. This article explains IDEAS's journey with adopting Fabric.

Building a scalable data foundation with OneLake and Delta lake

A strong, scalable foundation is key to any modern data platform. At the core of Microsoft Fabric is Delta Lake, an open-source storage layer that ensures reliability, performance, and data management for data lakes. Its broad compatibility with data analytics tools supports a unified data ecosystem.

Delta Lake is the foundation of OneLake, Fabric's unified logical data lake. OneLake optimizes data value by eliminating duplication, and ensuring a single source of truth. All Fabric experiences automatically store or mirror data in OneLake using the Delta Lake format. OneLake integrates seamlessly with existing ADLS Gen2 storage enabling a smooth transition for existing datasets. Shortcuts to ADLS Gen2 avoid large-scale data migrations and enhance manageability through centralized access and governance. It also supports various analysis tools, including Spark, SQL, and Power BI.

Power BI's Direct Lake mode enhances this unified experience by enabling fast querying and visualization of data directly from OneLake, eliminating data movement and the need for traditional data marts. This direct access to Delta Lake streamlines analytics workflows. Fabric is also fully integrated with Copilot and AI across all surfaces. These features boost productivity through AI-assisted coding and data analysis, supporting data-driven decision-making.

To apply these capabilities within IDEAS, the first step was to ensure the data integration pipelines could seamlessly generate Delta Lake tables. IDEAS uses the following two ISO-certified data engineering systems:

- **Pharos**: A low-code platform for data preparation and staging. It simplifies data transformation by focusing on well-defined data shapes with consistent metadata, and declarative definitions for output generation.
- **Nitro Hubs**: A comprehensive data engineering system for pipeline authoring and management, with strong data privacy and compliance controls.

IDEAS enhanced these services to generate optimized Delta Lake outputs using the Fabric Spark engine, employing techniques like v-order, partitioning, and appropriate row group sizes. When storing data, IDEAS focuses on organizing it for fast and efficient retrieval, as the workloads are read-intensive. Integrating this capability with core services that manage thousands of pipelines enabled the quick writing of several thousand data assets to ADLS Gen2 storage.

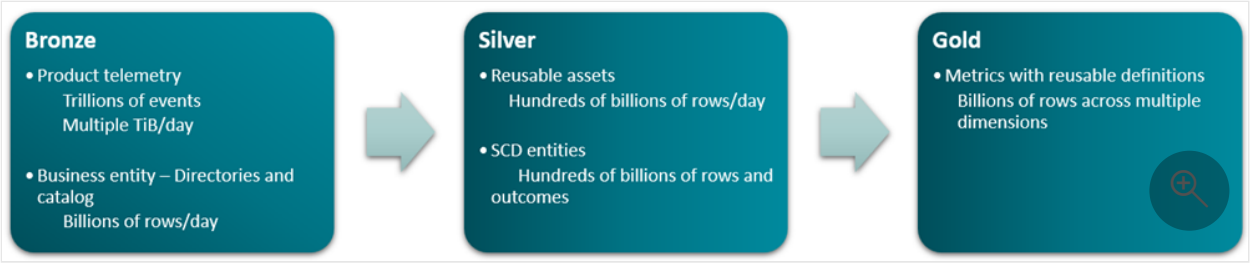
IDEAS analytics require 13 months of historical data, but due to personal identifiers, we must adhere to General Data Protection Regulation(GDPR). To comply, we extended Nitro Hubs GDPR processing capabilities to handle delete requests in Delta Lake tables, using merge commands in Fabric Spark notebooks. We also implemented Time-To-Live (TTL) expiration for date-partitioned Delta tables, ensuring personal data removal within GDPR timeframes. In contrast, our Gold layer data (using the medallion architecture), is aggregated and free of personal identifiers. We currently store over 4 PiB of data in IDEAS OneLake.

Powering Microsoft 365 Copilot Analytics with Fabric

IDEAS manages 420 PiB of data across over 600 teams within Microsoft. The data platform is built entirely on Azure to use scalability for a team of over 600 people. By extending Azure's capabilities, IDEAS has developed a robust and adaptable system. To learn more about the core systems that drive the data lifecycle see Data productivity in ideas

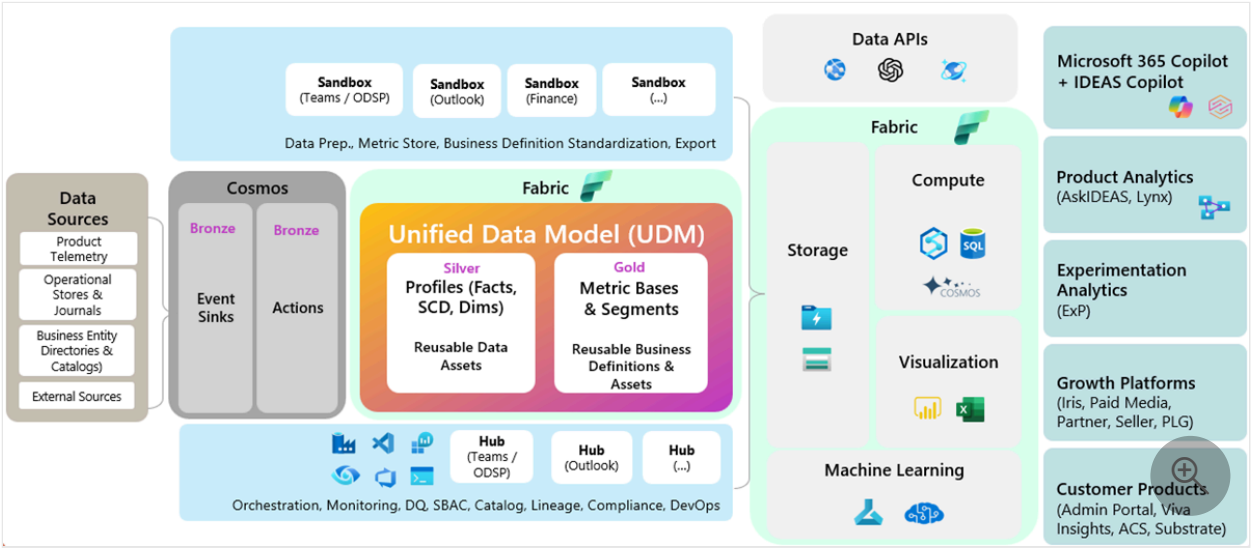
Efficient data access is essential to IDEAS, and Microsoft Fabric has become a key enabler in our strategy. We wanted to shorten the feedback loop for interactive queries and empower faster report and dashboard creation. Our foundation is the Unified Data Model (UDM), a set of durable and extensible data assets designed for company-wide reuse. This reusability is key to maintaining consistency and efficiency.

IDEAS employs the medallion architecture to organize data across three layers: Bronze (raw data), Silver (cleaned and enriched data for analysis), and Gold (curated, aggregated data for business intelligence and reporting with tools like Power BI and Excel).



By making our gold and silver layers available as UDM assets in Fabric through Delta Lake, we enhanced the Microsoft 365 Copilot analytics plane. We provided direct access to preprocessed Silver layer Microsoft 365 Copilot data as Delta Lake tables in OneLake. It dramatically improved query performance and dashboard rendering by eliminating repeated transformations.

Furthermore, exposing our Gold layer Microsoft 365 Copilot metrics as Delta Lake tables simplified data discovery and usability. It enabled the creation of rich dashboards that support business leaders and product teams with Copilot's adoption, performance, and growth. This approach reduced data movement, streamlined the data graph, and cut infrastructure costs. As a result, Microsoft 365 Copilot analytics, now powered by Fabric, plays a vital role in several Microsoft projects.



Scaling governance and automation in Fabric

Our next priorities were organizing workspaces, optimizing lakehouse structure, and automating operations across thousands of assets managed in Fabric. At our scale, governance demands strict adherence to policies that grant access only for legitimate data use scenarios, making manual operations infeasible. To address this, we have partnered closely with the Fabric SDK/API teams to ensure the availability of APIs that enable us to programmatically create Fabric artifacts and apply granular permissions to the appropriate identities. This fully automated approach provides consistency and scalability.

We organized our workspaces into production, development, and exploration environments. Production data is accessible through shortcuts within the production workspace. Only a dedicated workspace identity has privileged access to create and modify them, while all other users have read-only access. Lakehouses reside within the production workspace with broad read access and are referred by internal shortcuts from within exploration workspaces. This approach effectively isolates production data while allowing users to interact with it in a nonproduction setting.

Next sections will delve into our semantic workspaces that are dedicated to host only semantic models and reports. The lakehouses behind the semantic model is in production workspace to control versioning and change management. As Fabric's unified security features evolve, we continue refining our lakehouse access configuration to further streamline our access governance processes. This automated, API-driven approach is essential for data management at scale and for consistent, secure access.

Simplifying reporting with Direct Lake

One of the primary drivers for IDEAS's early adoption of Fabric is the Direct Lake semantic model. This feature enables unified reporting, eliminating the need to manage separate SQL and SSAS infrastructure and allowing users to work in an integrated Fabric interface.

IDEAS utilizes semantic models for a various use cases, which include:

- Single-table reports
- Azure Analysis Services cubes
- Import mode star schemas with multiple dimensions
- Cohort analytics for the Microsoft 365 Copilot analytics plane

Our validation of the Direct Lake approach involved migrating existing reports and models to Fabric, yielding several key findings, which include:

- The importance of effective data modeling. For models containing billions of rows, a robust star schema with numeric keys is crucial to achieve optimal query performance.

- Optimizing data with V-Order during creation using Fabric Spark is critical for maximizing Direct Lake performance.

- Proper Delta table partitioning and row group sizing is vital for optimizing both cold and warm cache query performance.

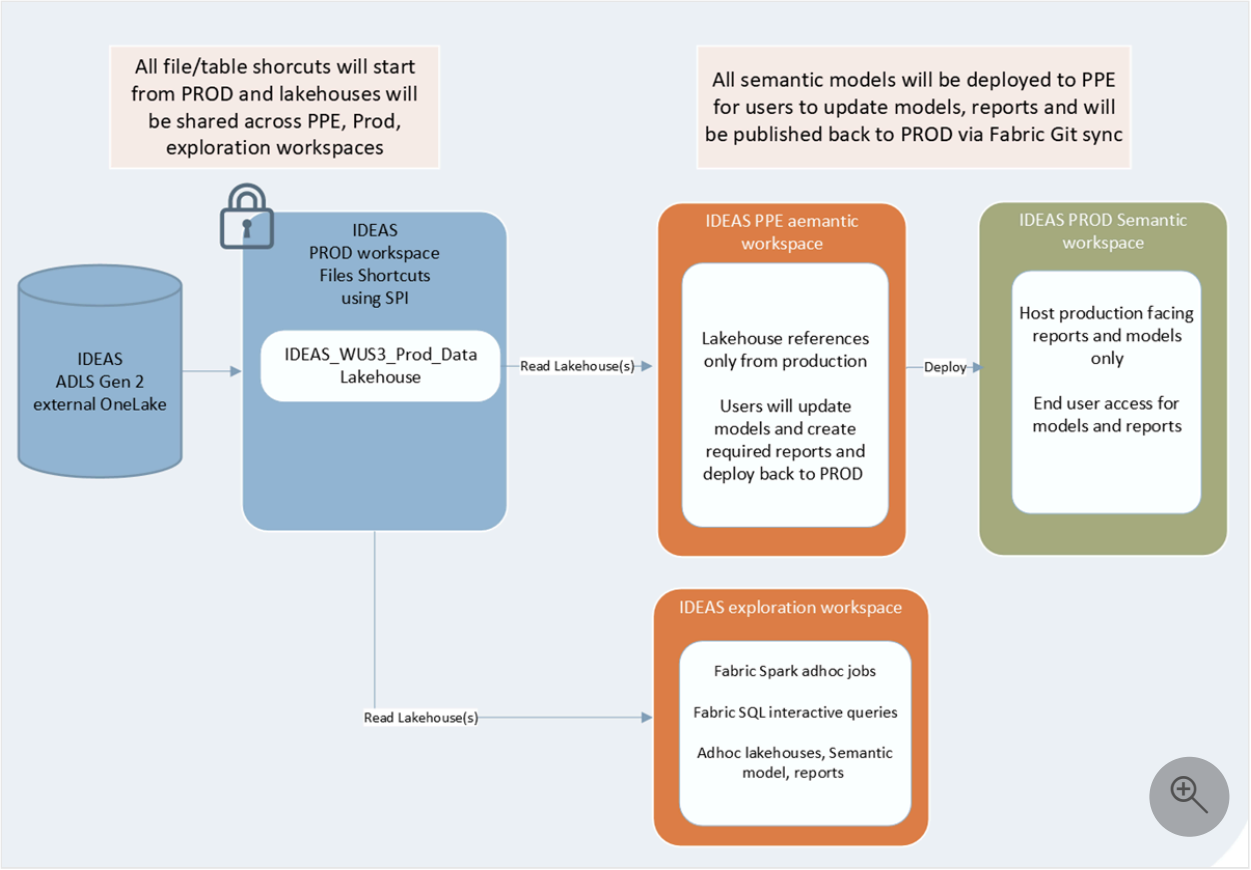
This effort led to the full migration of the Microsoft 365 Copilot analytics plane to Fabric in December 2024. This plane now delivers key business insights for Microsoft 365 Copilot across Microsoft.

Managing the Fabric development lifecycle for compliance and reliability

IDEAS ensures compliance and reliability through strict change management, production isolation, and validation. To meet these requirements within Fabric, we implemented a robust development lifecycle using Git integration and a well-defined workspace organization. This approach ensures that changes are thoroughly tested and validated before reaching production, minimizing disruptions, and preserving data integrity.

We have created dedicated "semantic workspaces" for semantic models and reporting artifacts, ensuring clear separation of concerns. As previously mentioned, lakehouse artifacts reside in a secure, read-only, production workspace, with semantic workspaces referring to these centralized data assets. This architecture supports both compliance and performance.

Our semantic model lifecycle involves individuals making changes within a workspace dedicated to this category of development. Following validation, Fabric's Git integration commits these changes to the appropriate preproduction branch. Through Azure DevOps (ADO) release pipelines, these changes are then promoted to the production Git branch and later synchronized to the production semantic workspaces This ensures that the production semantic workspaces (where end-user-facing models and reports reside) always reflects validated and approved changes. This way it contributes to the stability and reliability of our services.



To further enhance the reliability of our Fabric deployment, we developed a user experience and performance dashboard using workspace telemetry. The Fabric workspace analytics logs provide data on query execution times and errors in semantic models and Power BI reports. Our dashboard, built on Fabric event houses, tracks key query performance metrics and monitors error categories and rates for each query.

In addition to identifying and addressing potential issues, we monitor the impact of issues and the number of affected users. This dual approach allows us to proactively address issues before they spread and to understand and trend the reliability of our reports and semantic models through user feedback. By monitoring the frequency and breadth of user-reported issues, we can directly correlate our reliability targets with real-world usage and reduce user impact over time.

As the Fabric product group improves telemetry and log data, we'll incorporate richer KPIs and metrics into our dashboard. These enhancements improve our ability to proactively detect issues, ensuring optimal performance and reliability. In the next phase, we plan to extend this monitoring process to other Fabric items, including Lakehouse SQL endpoints and Spark notebooks.

Enabling interactive analytics with Fabric

Data users often start with Power BI reports but quickly need deeper exploration beyond the reporting layer. Fabric offers two powerful options for interactive analysis: Fabric

Spark and the SQL analytics endpoint, enabling users to explore data in the Silver and Gold layers of the Unified Data Model (UDM). The OneLake Data Hub, Lakehouse Explorer, and Lineage View provide quick access to data dependencies and upstream sources. However, as data complexity and size increase from Gold to Bronze, querying becomes more challenging.

To build a scalable data platform and prevent fragmentation, IDEAS implemented a federation strategy for key UDM Silver layer assets, serving as authoritative sources of truth. This allowed partner teams to extend these assets with domain-specific attributes. A robust governance process that encompasses review, extension design refinement, data interface definition, and exposure control, ensures data integrity and compliance.

While extensions address data bottlenecks and separate core data from external attributes, they introduced a performance challenge for interactive querying, which typically requires sub-60-second response times. The reliance on repetitive joins between base data and extensions created a bottleneck. To overcome this challenge, we enhanced our data engineering systems to materialize views, prejoining base data with extensions to minimize query-time join operations. Delta Lake's merge and locking capabilities enabled efficient partition updates and merging of extension data per entity. These precomputed Delta tables are now exposed as shortcuts within our lakehouses for both Fabric SQL and Spark access. We're also collaborating with the Fabric product group to explore potential native integration of this functionality. Furthermore, we developed Python modules with multi-parameter input to optimize row filtering and column selection, providing quick access to column descriptions and data freshness within the notebook environment.

Initial testing with Fabric Spark queries on these materialized assets has shown significant performance gains of over 30X.

Securing Our Fabric Environment: A Holistic Approach to Data Governance

Over the past year, we've progressed from an exploratory Fabric workspace to managing multiple F2048 production workspaces with over 4 PiB of data in Delta Lake format. However, data discovery and compliance remain complex challenges, especially as privacy regulations evolve. As IDEAS makes more data available in Fabric, our compliance obligations extend beyond GDPR and Microsoft's commitment to data residency within the EU Data Boundary, ensuring commercial personal data is stored and processed exclusively in Europe.

Microsoft’s global scale and handling of sensitive data drive IDEAS' strong commitment to data privacy and governance. This extends beyond baseline requirements, adhering to various international and industry-specific standards. It translates into robust controls and processes for managing data access.

At IDEAS, we strongly adhere to the principle of least privilege and scenario-based data use for security and compliance. This principle means granting data access only to users or identities with legitimate approval for specific use cases. To prevent unauthorized data transfers, IDEAS actively monitors for data exfiltration within Fabric workspaces. While Fabric provides tenant-level monitoring, Microsoft requires more granular control at the workspace level.

To address this, IDEAS developed Data Exfiltration Monitoring (DEM), a custom capability that collects Fabric telemetry data into a centralized metadata store and applies rules to detect violations. When a violation is detected, DEM triggers corrective actions, such as notifying the user, revoking access, or blocking future access. This allows IDEAS to make sensitive data available within Fabric while maintaining strict compliance. Combined with our investments in automating workspace provisioning, we can manage compliant workspaces across expanding data boundaries. Data exfiltration protection is a key area of focus for the Fabric product group, and we look forward to further enhancements in this space.

Ultimately, we believe that data privacy and governance aren't merely a compliance burden but a fundamental component of building trust and a significant competitive advantage in today's data-driven world. Microsoft prioritizes trust, emphasizing data privacy, user control, and responsible data handling across all services and products. IDEAS fully subscribes to this ethos, recognizing that robust data governance is fundamental to the success and sustainability of our data platform.

Conclusion

Integrating Microsoft Fabric into the IDEAS data platform has enhanced data access and boosted productivity for our data scientists and engineers. By building a unified foundation with OneLake and Delta Lake, enabling interactive analytics, and establishing strong governance, Fabric has provided a robust data analytics environment. As we continue to explore and implement Fabric's capabilities, particularly in areas like real-time analytics and advanced AI integration, we're confident that we’re building a unified and innovative platform that will drive greater insights and impact for Microsoft.

Feedback

Was this page helpful? Yes No

Provide product feedback | Ask the community

Learn about Microsoft Fabric feedback

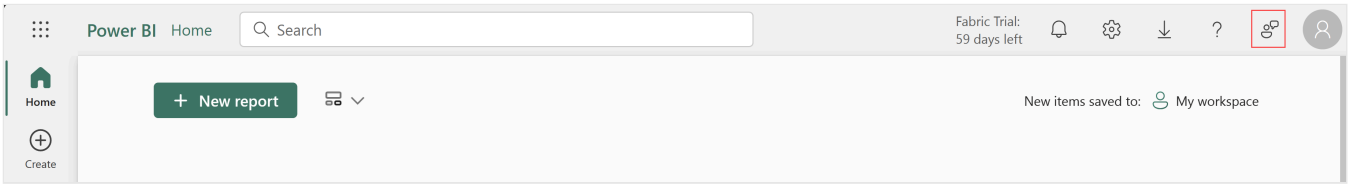
Your feedback is important to us. We want to hear about your experiences with Microsoft Fabric. Your feedback is used to improve the product and shape the way it evolves. This article describes how you can give feedback about Microsoft Fabric, how the feedback is collected, and how we handle this information.

Feedback types

There are three ways to give feedback about Microsoft Fabric, *in-product feedback*, *in-product surveys*, and *community feedback*.

In-product feedback

Give in-product feedback by selecting the **Feedback** button next to your profile picture in the Microsoft Fabric portal.



In-product surveys

From time to time, Microsoft Fabric initiates in-product surveys to collect feedback from users. When you see a prompt, you can choose to give feedback or dismiss the prompt. If you dismiss the prompt, you won't see it again for some time.

Community feedback

There are a few ways you can give feedback while engaging with the Microsoft Fabric community:

- Ideas - Submit and vote on ideas for Microsoft Fabric.
- Issues - Discuss issues and workarounds with the community.
- Community Feedback - Give feedback about Microsoft Fabric and vote for publicly submitted feedback. Top known feedback items remain available in the new portal.

Fabric user panel

Join the Microsoft Fabric user panel to help shape the future of Microsoft Fabric and Power BI. By signing up, you can share real-world experiences and feedback with the Fabric and Power BI product teams through surveys and 1:1 meetings. This is a free opportunity to engage directly with the engineering team behind Fabric and Power BI. Your input plays a key role in improving the tools and experiences we build.

To sign up for our Fabric User Panel, use the following steps:

- Visit the Fabric user panel site to complete the Opt-in form with your details, as well as answer a few Fabric-related questions. These questions help us customize communications and feedback invites based on your interests and usage.
- For each Fabric study or campaign that you'd be a good fit for, you will receive a formal email invite with details on the study and how to participate.
- For any questions on this user panel, contact us.

What kind of feedback is best?

Try to give detailed and actionable feedback. If you have issues, or suggestions for how we can improve, we'd like to hear it.

- **Descriptive title** - Descriptive and specific titles help us understand the issue being reported.
- **One issue** - Providing feedback for one issue ensures the correct logs and data are received with each submission and can be assigned for follow-up. If you want to give feedback for multiple issues, give feedback for each issue separately. Giving feedback for Separate issues helps us identify the volume of feedback we're receiving for a particular issue. If you have more than one issue, submit a new feedback request for each issue.
- **Give details** - Give details about your issue in the description box. Information about your device, operating system, and apps are automatically included in each reported feedback. Add any additional information you think is important. Include detailed steps to reproduce the issue.

How Microsoft uses feedback

Microsoft uses feedback to improve Microsoft products. We get user feedback in the form of questions, problems, compliments, and suggestions. We make sure this feedback makes it back to the appropriate teams, who use feedback to identify, prioritize and make improvements to

Microsoft products. Feedback is essential for our product teams to understand our user's experiences, and directly influences the priority of fixes and improvements.

What do we collect?

Here are the most common items collected or calculated.

- **Comments User** - Submitted comments in the original language.
- **Submission date** - Date and time we got the feedback.
- **Language** - The original language the comment was submitted in.
- **Feedback type** - The type of feedback: Survey feedback or in-product feedback.
- **Survey questions** - Questions that we asked the user during the survey.
- **Survey responses** - User responses to survey questions.
- **App language** - The language of the Microsoft product that was captured on submission.
- **Tenant ID** - When the feedback is submitted from a Microsoft Entra account.
- **User ID** - Microsoft Entra ID or email address of the authenticated user submitting the feedback.

Data handling and privacy

We understand that when you use our cloud services, you're entrusting us with one of your most valuable assets: your data. We make sure the feedback we receive is stored and handled under Microsoft governance rules, and that it can only be accessed for approved uses. We don't use your email, chat, files, or other personal content to target ads to you. When we collect data, we use it to make your experiences better.

To learn more about how we protect the privacy and confidentiality of your data, and how we ensure that it will be used only in a way that is consistent with your expectations, review our privacy principles at the Microsoft Trust Center.

Last updated on 12/12/2025

Fabric known issues

Microsoft Fabric Known issues are no longer published on Microsoft Learn. To view the latest updates, visit the Fabric support page. If you bookmarked the previous location, please update your link to the new page.

Currently active known issues

To see active known issues, follow the below steps:

1. Go to the new Known Issues page
2. On the left menu, select the workload in which you're working
3. On the top right of the screen, select the **Filters** to open the filters
4. Uncheck the **Closed** option so that only **Active** is selected
5. Select **Apply** and view your active known issues

Recently closed known issues

To see closed known issues, follow the below steps:

1. Go to the new Known Issues page
2. On the left menu, select the workload in which you're working
3. On the top right of the screen, select the **Filters** to open the filters
4. Uncheck the **Active** option so that only **Closed** is selected
5. Select **Apply** and view your active known issues

Last updated on 07/21/2025

Microsoft Fabric product, workload, and item icons

07/03/2025

This article provides information about the official collection of icons for Microsoft Fabric that you can use in architectural diagrams, training materials, slide decks or documentation.

Do's

- Use the icons to illustrate how products can work together.
- In diagrams, we recommend including a label that contains the product, experience, or item name somewhere close to the icon.
- Use the icons as they appear within the product.

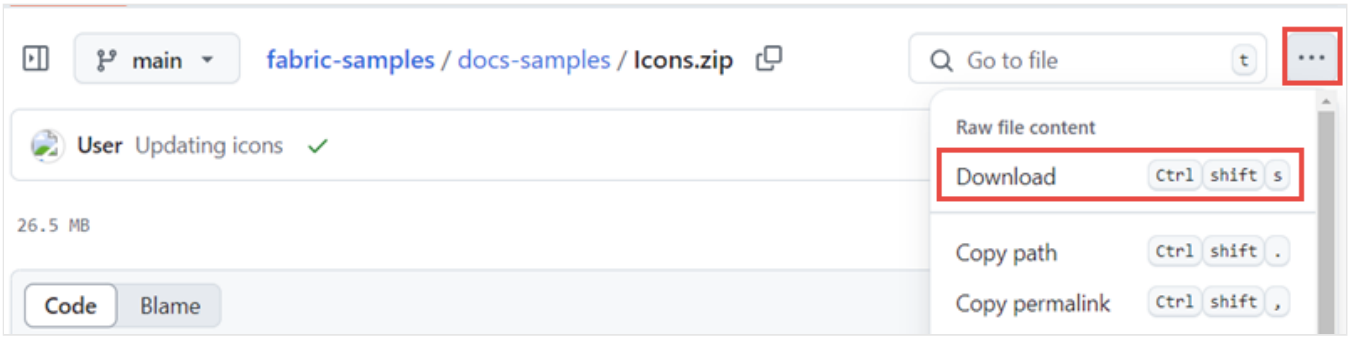
Don'ts

- Don't crop, flip, or rotate icons.
- Don't distort or change icon shape in any way.
- Don't use Microsoft product icons to represent your product or service.

Terms

Microsoft permits the use of these icons in architectural diagrams, training materials, or documentation. You can copy, distribute, and display the icons only for the permitted use unless granted explicit permission by Microsoft. Microsoft reserves all other rights. Fabric icons are also available as a npm package for use in Microsoft Fabric platform extension development. To use these icons, import the package into your project, then use individual SVG files as an image source or as an SVG. You can also directly download the icons from the following GitHub repository. Select the following button to open the repo, select ... from the right hand corner and select **Download**:

Download icons from GitHub



Related content

- Microsoft Power Platform icons
- Azure icons
- Dynamics 365 icons